Neo4j Cypher 25 Manual

Table of Contents

Overview 1
- What is Cypher? 1
- Cypher and SQL: key differences 2
- Cypher and APOC 2

Cypher and Neo4j 3
- Cypher and the different editions of Neo4j 3
- Key Neo4j terminology 3
- Built-in databases in Neo4j 4
- Cypher and Neo4j transactions 4

Cypher and Aura 6
- What is Aura? 6
- Using Cypher on Aura 7
- Aura and the Cypher Cheat Sheet 7

Queries 8
- Core concepts 8
  - Nodes 8
  - Relationships 9
  - Paths 9
- Basic queries 9
  - Creating a data model 9
  - Creating a property graph database 10
  - Finding nodes 10
  - Note on clause composition 11
  - Finding connected nodes 12
  - Finding paths 14
  - Finding recommendations 16
  - Delete a graph 17
- Select Cypher version 17
  - Cypher 25, Cypher 5, and Neo4j explained 17
  - Select the default Cypher version for a database 18
  - Migrating queries from Cypher 5 to Cypher 25 20
  - Select Cypher version for individual queries 20
  - Further reading 21
- Composed queries 21
  - Combined queries (`UNION`) 22
  - Conditional queries (`WHEN`) 26
  - Sequential queries (`NEXT`) 33

Clauses 42

Reading clauses 42
Projecting clauses 42
Reading sub-clauses 42
Writing clauses 43
Reading/Writing clauses 43
Subquery clauses 44
Set operations 44
Multiple graphs 44
Importing data 44
Index and schema clauses 45
Reading hints 45
SHOW commands 45
TERMINATE Commands 46
Administration clauses 46
Clause composition 46
Read-write queries 48
Queries with `UNION` 51
Queries with `CALL {}` subqueries 55
Notes on the implementation 60
CALL 60
CREATE 60
Create nodes 60
Create relationships 61
Comparing `CREATE` and `MERGE` 61
Reusing variables 62
Reusing variables in properties 62
Use parameters with `CREATE` 63
Create using dynamic node labels and relationship types 64
`INSERT` as a synonym of `CREATE` 65
`CREATE` and concurrent transactions 65
Syntax 65
DELETE 66
Example graph 66
Delete single node 67
Delete relationships only 68
Delete a node with all its relationships 68
Delete all nodes and relationships 69
Syntax 70
FILTER 70
Example graph 70
Basic filtering 71

Filter on dynamic labels and types 72
Filter on dynamic properties 72
Differences between `FILTER` and `WHERE` 73
Syntax 75
FINISH 76
FOR 76
Unwinding a list 77
Further behavior 77
Syntax 77
FOREACH 77
Mark all nodes along a path 78
Mark all relationships along a path 78
Create new nodes from a list of name labels 79
Syntax 79
GROUP BY 79
Grouping keys 80
Example graph 80
`GROUP BY` clause template 81
Basic examples 81
GROUP BY and ORDER BY 83
Special cases 84
Errors 85
Syntax 87
LET 87
Example graph 87
Bind values to variables 88
Differences between `LET` and `WITH` 89
Advanced examples 92
Syntax 94
LIMIT 95
Example graph 95
Examples 96
`LIMIT` and side effects 97
Using `LIMIT` as a standalone clause 98
Syntax 98
LOAD CSV 99
Import CSV data into Neo4j 99
Pre-process the data during import 107
Recommendations 110
LOAD CSV and Neo4j functions 111
CSV file format 113

Example 116
Other ways of importing data 117
Syntax 117
MATCH 118
Example graph 118
Find nodes 119
Find relationships 121
MATCH with WHERE predicates 123
MATCH with parameters 124
Find paths 125
Multiple MATCH clauses, the WITH clause, and clause composition 126
MATCH using dynamic node labels and relationship types 127
Syntax 129
MERGE 130
Example graph 130
`MERGE` existing patterns 131
`null` properties 132
`MERGE` new patterns 132
Undirected relationships 135
`ON MATCH` and `ON CREATE` 136
Using map parameters with `MERGE` 137
`MERGE` using dynamic node labels and relationship types 137
Using `MERGE` with constraints 139
Concurrent relationship merges 140
Syntax 141
OPTIONAL MATCH 141
Introduction 141
Example graph 141
OPTIONAL MATCH in more detail 142
Optional relationships 143
Properties on optional elements 144
Optional typed and named relationship 145
Syntax 145
ORDER BY 146
Example graph 146
Basic examples 147
Order by values not in the result 149
Ascending and descending order 150
ORDER BY and pattern matching 151
Null values 152
ORDER BY and the WITH clause 152

Ordering aggregated or DISTINCT results 153
ORDER BY and indexes 153
Using `ORDER BY` as a standalone clause 154
Syntax 154
REMOVE 155
Example graph 155
Remove a property 156
Remove all properties 156
Dynamically remove a property 157
Remove a label from a node 157
Dynamically remove a node label 158
Remove multiple labels from a node 158
Remove multiple labels dynamically from a node 159
Syntax 159
RETURN 159
Introduction 159
Example graph 160
Return nodes 160
Return relationships 160
Return property 161
Return all elements 161
Variable with uncommon characters 162
Column alias 162
Optional properties 162
Other expressions 163
Unique results 163
RETURN ALL 164
Syntax 164
SEARCH 165
SEARCH clause template 165
Vector search 167
Full-text search 179
Limitations of the SEARCH clause 184
Syntax 185
SET 186
Example graph 186
Set a property 187
Update a property 189
Dynamically set or update a property 189
Remove a property 190
Copy properties between nodes and relationships 190

Replace all properties using a map and `=` 191
Remove all properties using an empty map and `=` 191
Mutate specific properties using a map and `+=` 191
Set multiple properties using one `SET` clause 192
Set a property using a parameter 193
Set all properties using a parameter 193
Set a label on a node 194
Dynamically set a node label 194
Set a label using a parameter 195
Set multiple labels on a node 195
Set multiple labels dynamically on a node 195
Set multiple labels using parameters 196
Syntax 196
SHOW 197
Common behavior and rules 198
Composable `SHOW` commands 199
SKIP 199
Example graph 200
Examples 200
Using `SKIP` as a standalone clause 202
`OFFSET` as a synonym to `SKIP` 202
Syntax 203
UNWIND 203
Unwinding a list 204
Creating a distinct list 204
Using `UNWIND` with any expression returning a list 205
Using `UNWIND` with a list of lists 205
Using `UNWIND` with an empty list 206
Using `UNWIND` with an expression that is not a list 206
Creating nodes from a list parameter 206
Syntax 207
USE 207
Syntax 207
USE clause when connected to a standard or system database 208
USE clause when connected to a composite database 208
Examples 209
Syntax 210
WHERE 210
Example graph 210
Basic filtering 211
Filter on dynamic labels and types 212

Filter on dynamic properties 212
Using `WHERE` after `WITH` 213
Filter patterns 213
Syntax 215
WITH 216
Example graph 216
Create new variables 217
Control variables in scope 218
Bind values to variables 219
Aggregations 220
Remove duplicate values 221
Explicitly project values 221
Ordering and pagination 222
Filter results 224
Syntax 225
Subqueries 227
CALL subqueries 227
Example graph 227
Semantics and performance 228
Importing variables 230
Optional subquery calls 234
Conditional `CALL` subqueries 236
Execution order of CALL subqueries 236
Post-union processing 237
Aggregations 238
Note on returning subqueries and unit subqueries 239
Unit subqueries 240
Summary 240
CALL subqueries in transactions 241
Syntax 241
Loading CSV data 241
Deleting a large volume of data 242
Batching 243
Composite databases 244
Error behavior 246
Status report 252
Concurrent transactions 254
Restrictions 267
Further reading 267
COLLECT subqueries 267
Example graph 267

Simple `COLLECT` subquery 268
`COLLECT` subquery with `WHERE` clause 268
Conditional `COLLECT` subquery 269
`COLLECT` subquery with a `UNION` 269
`COLLECT` subquery with `WITH` 270
Using `COLLECT` subqueries inside other clauses 271
Using `COLLECT` vs `collect()` 272
Rules 273
COUNT subqueries 273
Example graph 273
Simple `COUNT` subquery 274
`COUNT` subquery with `WHERE` clause 274
Conditional `COUNT` subquery 275
`COUNT` subquery with a `UNION` 275
`COUNT` subquery with `WITH` 276
Using `COUNT` subqueries inside other clauses 277
Rules 278
EXISTS subqueries 279
Example graph 279
Simple `EXISTS` subquery 280
`EXISTS` subquery with `WHERE` clause 280
Conditional `EXISTS` subqueries 280
Nesting `EXISTS` subqueries 281
`EXISTS` subquery outside of a `WHERE` clause 281
`EXISTS` subquery with a `UNION` 282
`EXISTS` subquery with `WITH` 282
`EXISTS` subquery with `RETURN` 283
Rules 283
Patterns 285
Primer 286
Example graph 286
Matching fixed-length paths 287
Matching variable-length paths 288
Matching shortest paths 291
Fixed-length paths 292
Node patterns 292
Relationship patterns 293
Path patterns 294
Path pattern matching 294
Variable-length paths 296
Quantified path patterns 296

Quantified relationships 300
Group variables 301
Predicates in quantified path patterns 305
Further reading 309
Non-linear paths 309
Equijoins 309
Graph patterns 312
Shortest paths 316
Note on Cypher and GDS shortest paths 317
`SHORTEST k` 317
`ALL SHORTEST` 321
`SHORTEST k GROUPS` 322
`ANY` 323
Partitions 323
Pre-filters and post-filters 324
Pre-filter with a path variable 326
Performance and planning 328
Paths with unique relationships 336
Example graph 337
Cypher's default traversal rules and the Königsberg bridges problem 338
Non-default `MATCH` behavior 340
Paths with repeated nodes and relationships 340
Example graph 340
`REPEATABLE ELEMENTS` and the Königsberg bridges problem 341
Further reading 344
Acyclic paths 344
Example graph 345
Performance improvement with `ACYCLIC` 348
Syntax and semantics 349
Node and relationship patterns 349
Path patterns and graph patterns 355
Variable length paths 359
Shortest paths 371
Match modes and path modes 377
Values and types 386
Property, structural, and constructed values 386
Property types 386
Structural types 387
Constructed types 387
Types and their synonyms 388
Type normalization 389

Property type details 391
Boolean, numeric, and string literals 393
Boolean 393
Numerical 393
String 394
Temporal values 396
Temporal value types 396
Time zones 397
Temporal instants 398
Durations 413
Temporal indexing 422
Formatting temporal values 422
Spatial values 423
The POINT type 424
Coordinate Reference Systems 424
Spatial instants 426
Spatial values and indexes 429
Comparability and orderability 429
Lists 430
Lists in general 430
List range and size 431
Storing lists as properties 433
Maps 434
Literal maps 434
UUIDs 434
The UUID type 434
Construct UUID values 435
Store UUID values as properties 436
UUIDs and client libraries (drivers) 436
Vectors 437
The vector type 437
Construct vector values 438
Store vector values as properties 439
Vectors and client libraries (drivers) 440
Type coercion 440
Supertypes 441
Lists, vector embeddings, and vector indexes 441
Graph references 442
Static graph references 442
Dynamic graph references 442
Rules 442

Working with `null` 443
Logical operations with `null` 443
The `IN` operator and `null` 443
The `[]` operator and `null` 444
Expressions that return `null` 444
Using `IS NULL` and `IS NOT NULL` 445
Casting data values 445
Functions for converting data values 445
Examples 445
Equality, ordering, and comparison of value types 447
Equality and comparison of values 447
Ordering and comparison of values 448
Expressions 453
Predicates 453
Boolean operators 454
Comparison operators 457
List operators 462
String operators 467
Label expression predicates 472
Path pattern expressions 478
Type predicate expressions 481
Node and relationship operators 488
Example graph 488
Static property access 489
Dynamic property access 489
Handling `null` values 490
Mathematical operators 491
Examples 491
Division by zero and `null` operands 493
Order of precedence 494
String concatenation operators 495
Examples 496
String concatenation, `LIST` values, and `null` 498
Temporal operators 499
Adding and subtracting `DURATION` values 500
Multiplying and dividing `DURATION` values 502
List expressions 502
Example graph 502
List element access 503
List slicing 505
List concatenation 507

Add elements to a list 507
List comprehension 508
Pattern comprehension 511
Map expressions 512
Example graph 512
Map operators 513
Map projection 517
Map comprehension 521
Conditional expressions (CASE) 523
Example graph 523
Simple `CASE` 524
Extended Simple `CASE` 525
Generic `CASE` 526
`CASE` with `null` values 527
`CASE` expressions and succeeding clauses 528
Further considerations 529
Functions 530
Aggregating functions 530
Database functions 532
Graph functions 532
List functions 532
LOAD CSV functions 534
Mathematical functions 534
Logarithmic functions 534
Numeric functions 535
Trigonometric functions 536
Predicate functions 537
Scalar functions 538
Security functions 540
String functions 540
Spatial functions 542
Temporal functions 543
Duration functions 543
Instant type functions 543
Format functions 546
User-defined functions 546
UUID functions 547
Vector functions 547
Show functions 548
Syntax 548
Examples 549

Return columns 551
Aggregating functions 552
Example graph 552
avg() 553
collect() 554
collect_list() 555
count() 556
max() 558
min() 560
percentileCont() 561
percentile_cont() 562
percentileDisc() 563
percentile_disc() 564
stDev() 565
stdev_samp() 565
stDevP() 566
stdev_pop() 567
sum() 568
Aggregating expressions and grouping keys 569
Database functions 572
db.nameFromElementId() 572
Graph functions 573
graph.names() 573
graph.propertiesByName() 574
graph.byName() 575
graph.byElementId() 576
List functions 577
Example graph 577
coll.distinct() 578
coll.flatten() 579
coll.indexOf() 580
coll.insert() 582
coll.max() 583
coll.min() 583
coll.remove() 584
coll.sort() 585
keys() 586
labels() 586
nodes() 587
range() 588
reduce() 589

relationships() 590
reverse() 591
tail() 591
toBooleanList() 592
toFloatList() 593
toIntegerList() 594
toStringList() 595
LOAD CSV functions 596
linenumber() 596
file() 596
Mathematical functions 597
Logarithmic functions 597
Numeric functions 601
Trigonometric functions 612
Predicate functions 624
Introduction 624
Example graph 624
all() 625
allReduce() 626
any() 629
exists() 630
isEmpty() 631
none() 632
single() 634
Scalar functions 635
Example graph 635
cardinality() 636
char_length() 638
character_length() 639
coalesce() 639
elementId() 640
endNode() 641
head() 642
id() 643
last() 644
length() 645
path_length() 646
nullIf() 647
properties() 648
randomUUID() 649
size() 650

startNode() 652
timestamp() 652
toBoolean() 653
toBooleanOrNull() 654
toFloat() 655
toFloatOrNull() 655
toInteger() 656
toIntegerOrNull() 657
type() 658
valueType() 659
Security functions 660
abac.native.user_tags() 661
abac.oidc.user_attribute() 661
Spatial functions 662
Example graph 662
point() 663
point.distance() 666
point.withinBBox() 669
String functions 671
btrim() 671
left() 672
lower() 673
ltrim() 674
normalize() 675
replace() 676
reverse() 678
right() 678
rtrim() 679
split() 680
string.indexOf() 681
string.join() 682
string.regexReplace() 683
substring() 683
toLower() 685
toString() 685
toStringOrNull() 686
toUpper() 687
trim() 688
upper() 689
Temporal functions 691
Duration functions 691

Instant type functions 700
Format functions 755
User-defined functions 761
User-defined scalar functions 761
User-defined aggregation functions 762
UUID functions 762
uuid() 762
uuid.leastSignificantBits() 764
uuid.mostSignificantBits() 765
Vector functions 766
vector() 766
vector.similarity.cosine() 768
vector.similarity.euclidean() 768
vector_dimension_count() 770
vector_distance() 771
vector_norm() 772
Indexes 774
Performance indexes 774
Create indexes 774
Show indexes 785
Drop indexes 791
The impact of indexes on query performance 793
Index hints for the Cypher planner 816
Search indexes 830
Full-text indexes 831
Vector indexes 840
Syntax 863
CREATE INDEX 864
SHOW INDEX 867
Query search indexes 867
DROP INDEX 869
Schema 871
Graph types 871
Further reading 872
Set graph types 872
Extend graph types 886
Alter element types 888
Show graph types 891
Drop graph type elements 904
Constraints 907
Create constraints 908

Show constraints 930
Drop constraints 936
Syntax 938
Graph types 938
Constraints 944
Query plans, tuning, and caching 950
Understanding query plans 950
The lifecycle of a Cypher query 950
`EXPLAIN` and `PROFILE` query plans 950
Example query plan 951
Query plan parts 952
Query plan table columns 953
Reading query plan tables 955
Full example 956
Lazy and eager query evaluation 957
Database hits 957
Operators 959
Summary of operators 959
Operators in detail 971
Cypher runtimes 1111
Runtime concepts 1112
Parallel runtime: reference 1122
Query tuning 1126
General recommendations 1126
Query options 1127
Query caches 1131
How the query cache works 1131
Parameter types and the executable query cache 1133
Caching and schema changes 1133
Notifications when a cached plan is not used 1134
Configuring query caches 1134
Unifying query caches 1135
Query size limit 1136
Administration 1137
Syntax 1138
Parsing 1138
Using unicodes in Cypher 1138
Supported whitespace 1139
Supported newline characters 1139
Naming rules and recommendations 1140
Naming rules 1140

Scoping and namespace rules 1141
Recommendations 1141
Length limit of identifiers 1141
Variables 1142
Keywords 1142
List of keywords 1143
Parameters 1152
Introduction 1152
Auto-parameterization 1153
String literal 1153
Regular expression 1153
Case-sensitive `STRING` pattern matching 1154
Create node with properties 1154
Create multiple nodes with properties 1154
Setting all properties on a node 1155
`SKIP` and `LIMIT` 1155
Node id 1155
Multiple node ids 1155
Calling procedures 1156
Comments 1156
Additions, deprecations, removals, and compatibility 1157
Neo4j 2026.10 1157
Deprecations 1157
Updates 1157
New 1158
Neo4j 2026.09 1158
Updates 1158
New 1159
Neo4j 2026.08 1159
New 1159
Neo4j 2026.07 1160
Updates 1160
New 1160
Neo4j 2026.06 1161
Deprecations 1161
Updates 1161
New 1163
Neo4j 2026.05 1163
Updates 1163
New 1164
Neo4j 2026.04 1164

Deprecations 1164
New 1164
Neo4j 2026.03 1164
New 1164
Neo4j 2026.02 1165
Updates 1165
New 1166
Neo4j 2026.01 1167
Updates 1168
New 1168
Neo4j 2025.12 1169
Neo4j 2025.11 1169
Updates 1169
New 1169
Neo4j 2025.10 1169
Updates 1169
New 1170
Neo4j 2025.09 1171
New 1171
Neo4j 2025.08 1171
Updates 1171
New 1172
Neo4j 2025.07 1172
Updates 1172
Neo4j 2025.06 1172
Removals 1172
Deprecations 1174
Updates 1174
New 1176
Appendix 1180
Appendix A: Cypher styleguide 1180
General recommendations 1180
Indentation and line breaks 1180
Casing 1183
Spacing 1184
Patterns 1186
Meta-characters 1188
Full Cypher grammar 1188
GQL conformance 1244
Note on minimum GQL conformance 1244
Supported mandatory GQL features 1245

Currently unsupported mandatory GQL features 1247
Supported optional GQL features 1248
Optional GQL features and analogous Cypher 1253
Additional Cypher features 1255
Tutorials and extended examples 1262
Basic query tuning example 1262
Advanced query tuning example 1274
Cypher Cheat Sheet 1290

Welcome to the Neo4j Cypher Manual.

Cypher is Neo4j's declarative query language, allowing users to unlock the full potential of property graph databases.

> **Note:**
>
> This manual covers Cypher 25. As of Neo4j 2025.06, all new Cypher features are exclusively added to Cypher 25, while Cypher 5 is frozen. As of Neo4j 2026.02, new databases explicitly set Cypher 25 as their query language in their configuration file. For information about how to how to use Cypher 25, see Select Cypher version. For information about new features added to Cypher 25, see Additions, deprecations, removals, and compatibility.

The Cypher Manual aims to be as instructive as possible to readers from a variety of backgrounds and professions, such as developers, administrators, and academic researchers.

If you are new to Cypher and Neo4j, you can visit the Getting Started Guide → Cypher chapter. Additionally, Neo4j GraphAcademy has a variety of free courses tailored for all levels of experience.

For a reference of all available Cypher features, see the Cypher Cheat Sheet.

For a downloadable PDF version of the Cypher Manual, visit the Neo4j documentation archive.

This introduction will cover the following topics:

- Overview
- Cypher and Neo4j
- Cypher and Aura

License: Creative Commons 4.0

Overview

This section provides an overview of Cypher and a brief discussion of how Cypher differs to SQL.

What is Cypher?

Cypher is Neo4j's declarative graph query language. It was created in 2011 by Neo4j engineers as an SQL-equivalent language for graph databases. Similar to SQL, Cypher lets users focus on *what* to retrieve from graph, rather than *how* to retrieve it. As such, Cypher enables users to realize the full potential of their property graph databases by allowing for efficient and expressive queries that reveal previously unknown data connections and clusters.

Cypher provides a visual way of matching patterns and relationships. It relies on the following ascii-art type of syntax: `(nodes)-[:CONNECT_TO]→(otherNodes)`. Rounded brackets are used for circular nodes, and `-[:ARROWS]→` for relationships. Writing a query is effectively like drawing a pattern through the data in the graph. In other words, entities such as nodes and their relationships are visually built into queries. This

makes Cypher a highly intuitive language to both read and write.

Cypher and SQL: key differences

Cypher and SQL are similar in many ways. For example, they share many of the same keywords, such as `WHERE` and `ORDER BY`. However, there are some important differences between the two:

Cypher is schema-flexible

While it is both possible and advised to enforce an open schema with Neo4j using graph types /constraints and indexes, Cypher and Neo4j offers a greater degree of schema-flexibility than SQL and a relational database. More specifically, nodes and relationships in a Neo4j database do not have to have a specific property set to them because other nodes or relationships in the same graph have that property (unless there is an property existence constraint created on that specific property). This means that users are not required to use a fixed schema to represent data and that they can add new attributes and relationships as their graphs evolve.

Query order

SQL queries begin with what a user wants to return, whereas Cypher queries end with the return clause. For example, consider the following two queries (both searching a database for titles of movies with a rating of greater than 7), the first written with SQL and the second with Cypher:

```
SELECT movie.name
FROM movie
WHERE movie.rating > 7
```

```
MATCH (movie:Movie)
WHERE movie.rating > 7
RETURN movie.title
```

Cypher queries are more concise

Due to its intuitive, whiteboard-like method of constructing clauses, Cypher queries are often more concise than their equivalent SQL queries. For example, consider the following two queries (both searching a database for the names of the actors in the movie The Matrix), the first written with SQL and the second with Cypher:

```
SELECT actors.name
FROM actors
    LEFT JOIN acted_in ON acted_in.actor_id = actors.id
    LEFT JOIN movies ON movies.id = acted_in.movie_id
WHERE movies.title = "The Matrix"
```

```
MATCH (actor:Actor)-[:ACTED_IN]->(movie:Movie {title: 'The Matrix'})
RETURN actor.name
```

Cypher and APOC

Neo4j supports the APOC (Awesome Procedures on Cypher) Core library. The APOC Core library provides access to user-defined procedures and functions which extend the use of the Cypher query language into areas such as data integration, graph algorithms, and data conversion.

For more details, visit the APOC Core page.

Cypher and Neo4j

This section discusses aspects of Neo4j that are important to consider when using Cypher.

Cypher and the different editions of Neo4j

Neo4j consists of two editions: a commercial Enterprise Edition, and a Community Edition.

Cypher works almost identically between the two editions, but there are key areas in which they differ:

Feature	Enterprise Edition	Community Edition
Multi-database	A `system` database and any number of user databases.	Only `system` and one user database.
Role-based security	User, role, and privilege management for flexible sub-graph access control.	Multi-user management. All users have full access rights.
Schema	Graph types and the following individual constraints: • node and relationship property existence constraints • node and relationship property type constraints • node and relationship property uniqueness constraints • node and relationship key constraints	Only node and relationship property uniqueness constraints.
Store `VECTOR` values as properties.	`VECTOR` values can be stored as properties if the database runs on block format.	`VECTOR` values cannot be stored as properties using Community Edition.
Vector indexes	Available for embeddings stored as `LIST<INTEGER | FLOAT>` and as `VECTOR` values.	Available for embeddings stored as `LIST<INTEGER | FLOAT>`.
Runtimes	All runtimes: slotted runtime, pipelined runtime (default), parallel runtime.	Only slotted runtime.

The `system` database cannot be deleted. The default user database, `neo4j`, can be deleted. If you want to delete it, first set another database as the default using the procedure `dbms.setDefaultDatabase` (this must be done from the `system` database). For more information, see the Operations Manual → Change the default database.

Key Neo4j terminology

Cypher queries are executed against a Neo4j database, but normally apply to specific graphs. It is important to understand the meaning of these terms and exactly when a graph is not a database.

DBMS

A Neo4j Database Management System is capable of containing and managing multiple graphs contained in databases. Client applications will connect to the DBMS and open sessions against it. A client session provides access to any graph in the DBMS.

Graph

Refers to a data model within a database. Normally there is only one graph within each database, and many administrative commands that refer to a specific graph do so using the database name. Cypher queries executed in a session may declare which graph they apply to, or use a default, given by the session. Composite databases can contain multiple graphs, by means of aliases to other databases. Queries submitted to composite databases may refer to multiple graphs within the same query. For more information, see Operations manual → Composite databases.

Database

A database is a storage and retrieval mechanism for collecting data in a defined space on disk and in memory.

Built-in databases in Neo4j

All Neo4j servers contain a built-in database called `system`, which behaves differently than all other databases. The `system` database stores system data and you can not perform graph queries against it.

A fresh installation of Neo4j includes two databases:

- `system` - the system database described above, containing meta-data on the DBMS and security configuration.
- `neo4j` - the default database, named using the config option `dbms.default_database=neo4j`.

For more information about the *system* database, see the sections on Access control.

Query considerations

Most of the time Cypher queries are reading or updating queries, which are run against a graph. There are also administrative commands that apply to a database, or to the entire DBMS. Administrative commands cannot be run in a session connected to a normal user database, but instead need to be run within a session connected to the `system` database. Administrative commands execute on the `system` database. If an administrative command is submitted to a user database, it is rerouted to the system database.

Cypher and Neo4j transactions

All Cypher queries run within transactions. Modifications done by updating queries are held in memory by the transaction until it is committed, at which point the changes are persisted to disk and become visible to other transactions. If an error occurs - either during query evaluation, such as integer division by zero, or during commit, such as constraint violations - the transaction is automatically rolled back, and no changes are persisted in the graph.

In short, an updating query always either fully succeeds or does not succeed at all.

> **Note:**
>
> A query that makes a large number of updates consequently uses large amounts of memory since the transaction holds changes in memory. For memory configuration in Neo4j, see the Neo4j Operations Manual → Memory configuration.

Explicit and implicit transactions

Transactions in Neo4j can be either explicit or implicit.

Explicit	Implicit
Opened by the user.	Opened automatically.
Can execute multiple Cypher queries in sequence.	Can execute a single Cypher query.
Committed, or rolled back, by the user.	Committed automatically when a transactions finishes successfully.

Queries that start separate transactions themselves, such as queries using `CALL { ... } IN TRANSACTIONS`, are only allowed in *implicit* mode. Explicit transactions cannot be managed directly from queries, they must be managed via APIs or tools.

For examples of the API, or the commands used to start and commit transactions, refer to the API or tool-specific documentation:

- For information on using transactions with a Neo4j driver, see *The session API* in the Neo4j Driver manuals.
- For information on using transactions over the HTTP API, see the HTTP API documentation → Run transactions.
- For information on using transactions within the embedded Core API, see the Java Reference → Cypher queries.
- For information on using transactions within the Neo4j Browser or Cypher-shell, see the Neo4j Browser documentation or the Cypher-shell documentation.

When writing procedures or using Neo4j embedded, remember that all iterators returned from an execution result should be either fully exhausted or closed. This ensures that the resources bound to them are properly released.

DBMS transactions

Beginning a transaction while connected to a DBMS will start a DBMS-level transaction. A DBMS-level transaction is a container for database transactions.

A database transaction is started when the first query to a specific database is issued. Database transactions opened inside a DBMS-level transaction are committed or rolled back when the DBMS-level

transaction is committed or rolled back.

DBMS transactions have the following limitations:

- Only one database can be written to in a DBMS transaction.
- Queries executed in the same transaction must either be of the same statement type, or a combination of schema modifications and read queries. The following statement types exist:
  - Administration commands contain administration clauses, such as `CREATE DATABASE movies`
  - Schema modifications, which contain index and constraint clauses, such as `CREATE CONSTRAINT …`
  - Write queries, which contains writing clauses, such as `CREATE (n) RETURN n`
  - Read queries, which contain none of the above clauses, such as `MATCH (n) RETURN n`

Note:

Calling procedures on the system database does not count as an administration command. The query will be categorized either as a read or write query depending on the defined mode of the procedure. For more information, see the Operations Manual → Procedures.

ACID compliance

Neo4j is fully ACID compliant. This means that:

- Atomicity - If a part of a transaction fails, the database state is left unchanged.
- Consistency — Every transaction leaves the database in a consistent state.
- Isolation — During a transaction, modified data cannot be accessed by other operations.
- Durability — The DBMS can always recover the results of a committed transaction.

Cypher and Aura

This page provides a brief overview of Neo4j Aura and its relationship to Cypher.

What is Aura?

Aura is Neo4j's fully managed cloud service. It consists of AuraDB and AuraDS. AuraDB is a graph database service for developers building intelligent applications, and AuraDS is a Graph Data Science (GDS) service for data scientists building predictive models and analytics workflows.

AuraDB is available on the following tiers:

- AuraDB Free
- AuraDB Professional
- AuraDB Business Critical
- AuraDB Virtual Dedicated Cloud

For more information, see Aura docs - Neo4j AuraDB overview.

AuraDS is available on the following tiers:

- Graph Data Science Community
- Graph Data Science Enterprise
- AuraDS Professional
- AuraDS Enterprise

For more information, see Aura docs - Neo4j AuraDS overview.

Using Cypher on Aura

> **Note:**
>
> As of Neo4j 2026.02, Cypher 25 is the default for newly created organisations. The default query language attached to created databases is determined by the organisation setting "Default Cypher Version" and can be manually changed.

Most Cypher features are available across all tiers of Aura. However, certain features are not supported in Aura instances. For example, it is not possible to create, alter, or drop databases using Aura, nor is it possible to alter or drop servers.

Additionally, some Cypher features are exclusive to AuraDB Business Critical and AuraDB Virtual Dedicated Cloud tiers. These primarily fall under database administration and role-based access control capabilities. For more information, see the Operations Manual → Authentication and authorization.

Aura and the Cypher Cheat Sheet

Each different tier of Aura has a customized version of the Cypher Cheat Sheet which only shows the features of Cypher available for the chosen tier.

The Cypher Cheat Sheet can be accessed here. You can select your desired Aura tier and Cypher version by using the dropdown menus provided.

Queries

This section provides a brief overview of the core concepts of a Cypher query (nodes, relationships, and paths), along with examples of how to query a Neo4j graph database. It also explains how to select the version of Cypher in which queries are run, and how to compose combined queries using `UNION`, conditional queries using `WHEN`, and sequential queries using `NEXT`.

- Core concepts
- Basic queries
- Select Cypher version Introduced in Neo4j 2025.06
- Composed queries
  - Combined queries (`UNION`)
  - Conditional queries (`WHEN`) Cypher 25 only Introduced in Neo4j 2025.06
  - Sequential queries (`NEXT`) Cypher 25 only Introduced in Neo4j 2025.06

Core concepts

Fundamentally, a Neo4j graph database consists of three core entities: nodes, relationships, and paths. Cypher queries are constructed to either match or create these entities in a graph. Having a basic understanding of what nodes, relationships, and paths are in a graph database is therefore crucial in order to construct Cypher queries.

The below examples use the MATCH and RETURN clauses to find and return the desired graph patterns. To learn more about these and many other Cypher clauses, see the section on Clauses.

Nodes

The data entities in a Neo4j graph database are called nodes. Nodes are referred to in Cypher using parentheses `()`.

```
MATCH (n:Person {name:'Anna'})
RETURN n.born AS birthYear
```

In the above example, the node includes the following:

- A `Person` **label**. Labels are like tags, and are used to query the database for specific nodes. A node may have multiple labels, for example `Person` and `Actor`.
- A `name` **property** set to `Anna`. Properties are defined within curly braces, `{}`, and are used to provide nodes with specific information, which can also be queried for and further improve the ability to pinpoint data.
- A **variable**, `n`. Variables allow referencing specified nodes in subsequent clauses.

In this example, the first `MATCH` clause finds all `Person` nodes in the graph with the `name` property set to `Anna`, and binds them to the variable `n`. The variable `n` is then passed along to the subsequent `RETURN`

clause, which returns the value of a different property (`born`) belonging to the same node.

Relationships

Nodes in a graph can be connected with relationships. A relationship must have a start node, an end node, and exactly one type. Relationships are represented in Cypher with arrows (e.g. `-->`) indicating the direction of a relationship.

```
MATCH (:Person {name: 'Anna'})-[r:KNOWS WHERE r.since < 2020]->(friend:Person)
RETURN count(r) As numberOfFriends
```

Unlike nodes, information within a relationship pattern must be enclosed by square brackets. The query example above matches for relationships of type `KNOWS` and with the property `since` set to less than `2020`. The query also requires the relationships to go from a `Person` node named `Anna` to any other `Person` nodes, referred to as `friend`. The count() function is used in the `RETURN` clause to count all the relationships bound by the `r` variable in the preceding `MATCH` clause (i.e. how many friends `Anna` has known since before `2020`).

Note that while nodes can have several labels, relationships can only have one type.

Paths

Paths in a graph consist of connected nodes and relationships. Exploring these paths sits at the very core of Cypher.

```
MATCH (n:Person {name: 'Anna'})-[:KNOWS]-{1,5}(friend:Person WHERE n.born < friend.born)
RETURN DISTINCT friend.name AS youngerConnections
```

This example uses a quantified relationship to find all paths up to `5` hops away, traversing only relationships of type `KNOWS` from the start node `Anna` to other younger `Person` nodes (as defined by the WHERE clause). The `DISTINCT` operator is used to ensure that the `RETURN` clause only returns unique nodes.

Paths can also be assigned variables. For example, the below query binds a whole path pattern, which matches the `SHORTEST` path from `Anna` to another `Person` node in the graph with a `nationality` property set to `Canadian`. In this case, the `RETURN` clause returns the full path between the two nodes.

```
MATCH p = SHORTEST 1 (:Person {name: 'Anna'})-[:KNOWS]-+(:Person {nationality: 'Canadian'})
RETURN p
```

For more information about graph pattern matching, see Patterns.

Basic queries

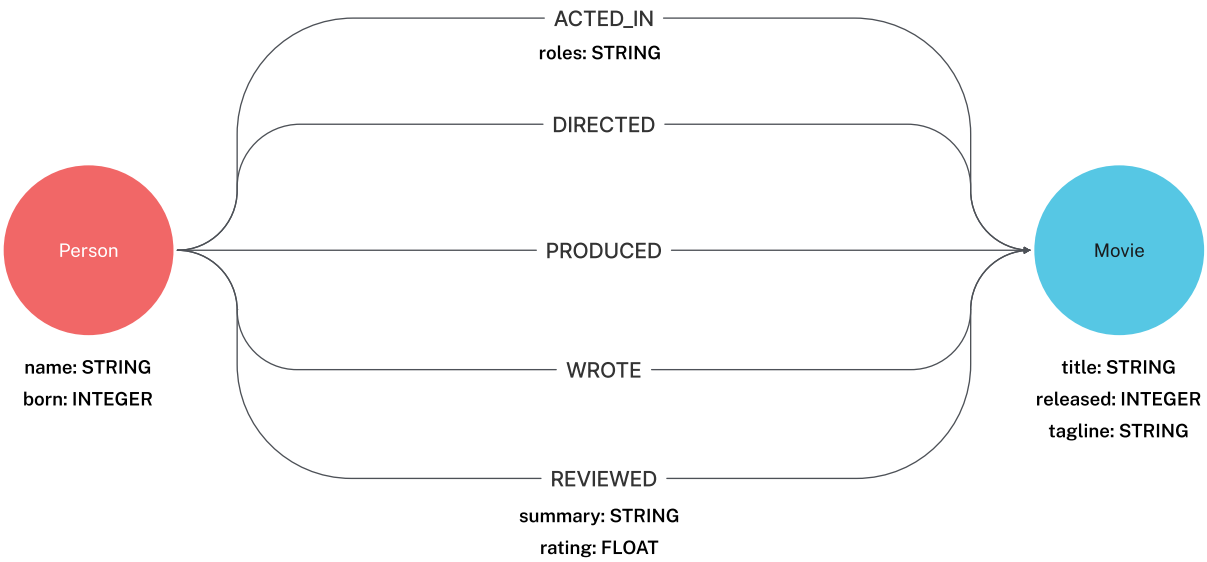
This page contains information about how to create, query, and delete a graph database using Cypher. For more advanced queries, see the section on Subqueries.

The examples below uses the publicly available Neo4j movie database.

Creating a data model

Before creating a property graph database, it is important to develop an appropriate data model. This will provide structure to the data, and allow users of the graph to efficiently retrieve the information they are looking for.

The following data model is used for the Neo4j data model:



It includes two types of node labels:

- `Person` nodes, which have the following properties: `name` and `born`.
- `Movie` nodes, which have the following properties: `title`, `released`, and `tagline`.

The data model also contains five different relationship types between the `Person` and `Movie` nodes: `ACTED_IN`, `DIRECTED`, `PRODUCED`, `WROTE`, and `REVIEWED`. Two of the relationship types have properties:

- The `ACTED_IN` relationship type, which has the `roles` property.
- The `REVIEWED` relationship type, which has a `summary` property and a `rating` property.

To learn more about data modelling for graph databases, enroll in the free Graph Data Modelling Fundamentals course offered by GraphAcademy.

Creating a property graph database

The complete Cypher query to create the Neo4j movie database, can be found here. To create the full graph, run the full query against an empty Neo4j database.

Finding nodes

The `MATCH` clause is used to find a specific pattern in the graph, such as a specific node. The `RETURN` clause specifies what of the found graph pattern to return.

For example, this query will find the nodes with `Person` label and the name `Keanu Reeves`, and return the `name` and `born` properties of the found nodes:

Query

```
MATCH (keanu:Person {name:'Keanu Reeves'})
RETURN keanu.name AS name, keanu.born AS born
```

Table 1. Result

name	born
"Keanu Reeves"	1964
Rows: 1	

It is also possible to query a graph for several nodes. This query matches all nodes with the `Person` label, and limits the results to only include five rows.

Query

```
MATCH (people:Person)
RETURN people
LIMIT 5
```

Table 2. Result

people
{"born":1964,"name":"Keanu Reeves"}
{"born":1967,"name":"Carrie-Anne Moss"}
{"born":1961,"name":"Laurence Fishburne"}
{"born":1960,"name":"Hugo Weaving"}
{"born":1967,"name":"Lilly Wachowski"}
Rows: 5

Note on clause composition

Similar to SQL, Cypher queries are constructed using various clauses which are chained together to feed intermediate results between each other. Each clause has as input the state of the graph and a table of intermediate results consisting of the referenced variables. The first clause takes as input the state of the graph before the query and an empty table of intermediate results. The output of a clause is a new state of the graph and a new table of intermediate results, serving as input to the next clause. The output of the last clause is the result of the query.

Note that if one of the clauses returns an empty table of intermediate results, there is nothing to pass on to subsequent clauses, thus ending the query. (There are ways to circumvent this behaviour. For example, by replacing a `MATCH` clause with OPTIONAL MATCH.)

In the below example, the first `MATCH` clause finds all nodes with the `Person` label. The second clause will then filter those nodes to find all `Person` nodes who were born in the 1980s. The final clause returns the result in a descending chronological order.

Query

```
MATCH (bornInEighties:Person)
WHERE bornInEighties.born >= 1980 AND bornInEighties.born < 1990
RETURN bornInEighties.name as name, bornInEighties.born as born
ORDER BY born DESC
```

Table 3. Result

name	born
"Emile Hirsch"	1985
"Rain"	1982
"Natalie Portman"	1981
"Christina Ricci"	1980
Rows: 4	

For more details, see the section on Clause composition.

Finding connected nodes

To discover how nodes are connected to one another, relationships must be added to queries. Queries can specify relationship types, properties, and direction, as well as the start and end nodes of the pattern.

For example, the following query matches the graph for the director of the movie the Matrix, and returns the `name` property of its directors.

Query

```
MATCH (m:Movie {title: 'The Matrix'})<-[d:DIRECTED]-(p:Person)
RETURN p.name as director
```

Table 4. Result

director
"Lilly Wachowski"
"Lana Wachowski"
Rows: 2

It also possible to look for the type of relationships that connect nodes to one another. The below query searches the graph for outgoing relationships from the `Tom Hanks` node to any `Movie` nodes, and returns the relationships and the titles of the movies connected to him.

Query

```
MATCH (tom:Person {name:'Tom Hanks'})-[r]->(m:Movie)
RETURN type(r) AS type, m.title AS movie
```

The result shows that he has 13 outgoing relationships connected to 12 different `Movie` nodes (12 have

the `ACTED_IN` type and one has the `DIRECTED` type).

Table 5. Result

type	movie
"ACTED_IN"	"Apollo 13"
"ACTED_IN"	"You've Got Mail"
"ACTED_IN"	"A League of Their Own"
"ACTED_IN"	"That Thing You Do"
"ACTED_IN"	"The Da Vinci Code"
"ACTED_IN"	"Cloud Atlas"
"ACTED_IN"	"Joe versus the Volcano"
"ACTED_IN"	"Cast Away"
"ACTED_IN"	"The Green Mile"

type	movie
"ACTED_IN"	"Sleepless in Seattle"
"ACTED_IN"	"The Polar Express"
"ACTED_IN"	"Charlie Wilson's War"
"DIRECTED"	"That Thing You Do"
Rows: 13	

It is possible to further modify Cypher queries by adding label expressions to the clauses. For example, the below query uses a `NOT` label expression (`!`) to return all relationships connected to `Tom Hanks` that are not of type `ACTED_IN`.

Query

```
MATCH (:Person {name:'Tom Hanks'})-[r:!ACTED_IN]->(m:Movie)
Return type(r) AS type, m.title AS movies
```

Table 6. Result

type	movie
"DIRECTED"	"That Thing You Do"
Rows: 1	

For more information about the different label expressions supported by Cypher, see the section on label expressions.

Finding paths

There are several ways in which Cypher can be used to search a graph for paths between nodes.

To search for patterns of a fixed length, specify the distance (hops) between the nodes in the pattern by using a quantifier (`{n}`). For example, the following query matches all `Person` nodes exactly 2 hops away from `Tom Hanks` and returns the first five rows. The `DISTINCT` operator ensures that the result contain no duplicate values.

Query

```
MATCH (tom:Person {name:'Tom Hanks'})--{2}(colleagues:Person)
RETURN DISTINCT colleagues.name AS name, colleagues.born AS bornIn
ORDER BY bornIn
LIMIT 5
```

Table 7. Result

name	bornIn
"Mike Nichols"	1931
"Ian McKellen"	1939

name	bornIn
"James Cromwell"	1940
"Nora Ephron"	1941
"Penny Marshall"	1943
Rows: 5	

It is also possible to match a graph for patterns of a variable length. The below query matches all `Person` nodes between `1` and `4` hops away from `Tom Hanks` and returns the first five rows.

Query

```
MATCH (p:Person {name:'Tom Hanks'})--{1,4}(colleagues:Person)
RETURN DISTINCT colleagues.name AS name, colleagues.born AS bornIn
ORDER BY bornIn, name
LIMIT 5
```

Table 8. Result

name	bornIn
"Max von Sydow"	1929
"Clint Eastwood"	1930
"Gene Hackman"	1930
"Richard Harris"	1930
"Mike Nichols"	1931
Rows: 5	

The quantifier used in the above two examples was introduced with the release of quantified path patterns. Before that, the only way in Cypher to match paths of a variable length was with a variable-length relationship. This syntax is still available in Cypher, but it is not GQL conformant. For more information, see Patterns → Syntax and semantics → Variable-length relationships.

The `SHORTEST` keyword can be used to find a variation of the shortest paths between two nodes. In this example, `ALL SHORTEST` paths between the two nodes `Keanu Reeves` and `Tom Cruise` are found. The `count()` function calculates the number of these shortest paths while the `length()` function calculates the length of each path in terms of traversed relationships.

Query

```
MATCH p = ALL SHORTEST (:Person {name:"Keanu Reeves"})--+(:Person {name:"Tom Cruise"})
RETURN count(p) AS pathCount, length(p) AS pathLength
```

The results show that 2 different paths are tied for the shortest length.

Table 9. Result

pathCount	pathLength
2	4
Rows: 1	

The `SHORTEST` keyword functionally replaces and extendes the `shortestPath()` and `allShortestPaths()` functions. Both functions can still be used, but they are not GQL conformant. For more information, see Patterns → Syntax and semantics → The `shortestPath()` and `allShortestPaths()` functions.

For more information about graph pattern matching, see Patterns.

Finding recommendations

Cypher allows for more complex queries. The following query tries to recommend co-actors for `Keanu Reeves`, who he has yet to work with but who his co-actors have worked with. The query then orders the results by how frequently a matched co-co-actor has collaborated with one of Keanu Reeves' co-actors.

Query

```
MATCH (keanu:Person {name:'Keanu Reeves'})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(coActors:Person),
  (coActors:Person)-[:ACTED_IN]->(m2:Movie)<-[:ACTED_IN]-(cocoActors:Person)
WHERE NOT (keanu)-[:ACTED_IN]->()<-[:ACTED_IN]-(cocoActors) AND keanu <> cocoActors
RETURN cocoActors.name AS recommended, count(cocoActors) AS strength
ORDER BY strength DESC
LIMIT 7
```

Table 10. Result

recommended	strength
"Tom Hanks"	4
"John Hurt"	3
"Jim Broadbent"	3
"Halle Berry"	3
"Stephen Rea"	3
"Natalie Portman"	3
"Ben Miles"	3
Rows: 5	

There are several connections between the `Keanu Reeves` and `Tom Hanks` nodes in the movie database, but the two have never worked together in a film. The following query matches coactors who could introduce the two, by looking for co-actors who have worked with both of them in separate movies:

Query

```
MATCH (:Person {name: 'Keanu Reeves'})-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(coActor:Person),
```

```
  (coActor)-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(:Person {name:'Tom Hanks'})
RETURN DISTINCT coActor.name AS coActor
```

Table 11. Result

coActor
"Charlize Theron"
"Hugo Weaving"
Rows: 2

Delete a graph

To delete all nodes and relationships in a graph, run the following query:

```
MATCH (n)
DETACH DELETE n
```

`DETACH DELETE` is not suitable for deleting large amounts of data, nor does it delete indexes or the schema of a graph. For more information, and alternatives to `DETACH DELETE`, see `DELETE` → Delete all nodes and relationships.

Select Cypher version

Users can specify the version of Cypher in which they want to run their queries, choosing between Cypher 5 and Cypher 25. The Cypher version can be specified either by configuring a default language for the database or by setting it on a per-query basis.

Cypher 25, Cypher 5, and Neo4j explained

Starting in 2025, the Neo4j server transitioned to a calendar-based versioning system. This means Neo4j will no longer use its previous semantic versioning and release pattern (e.g., 5.25, 5.26). Instead, releases from 2025 onwards will follow a YYYY.MM format, beginning with version 2025.01 released in January 2025, followed by 2025.02 released in February 2025, and so on.

Cypher 25 is introduced alongside **Neo4j 2025.06**. It is created as a clone of Cypher 5, but with additional new and updated features, as well as some removed features. For more information about the new, updated, and removed features included in Cypher 25, see the Additions, deprecations, removals, and compatibility page.

Cypher 25 is in an evolving state, and more features will be added to it with future releases of Neo4j. In the releases following Neo4j 2026.06, features will only be added to Cypher 25; no features will be removed until the next Cypher release. **Any new Cypher features introduced in Neo4j 2025.06 or later will be exclusively added to Cypher 25.**

Users running Neo4j version 2025.06 or later can choose to run their queries using the previous version of Cypher: Cypher 5. If so, Neo4j will use Cypher 5 as it existed at the time of the Neo4j 2025.06 release (the

release in which Cypher 5 was frozen). **No new features will be added to Cypher 5** (only performance enhancements and eventual bug fixes may be included in server releases following Neo4j 2025.06). Cypher 5 users must, therefore, migrate to Cypher 25 in order to access new features. For information about Cypher 5, see the Cypher 5 Manual.

> Although Cypher 5 queries are currently supported on Neo4j 2025.06+ databases, they will eventually need to be migrated to Cypher 25, as support for Cypher 5 will be discontinued in a future release (anticipated no earlier than two additional server LTS cycles).

Select the default Cypher version for a database

Neo4j 2026.02 or later

Databases created on Neo4j 2026.02 or later explicitly set Cypher 25 as their query language in the *neo4j.conf* file: `db.query.default_language=CYPHER_25`. Existing databases retain their *neo4j.conf* file. For both, a different query language version can be set with the same setting `db.query.default_language`.

> **Note:**
>
> If the setting is missing, it defaults to `db.query.default_language=CYPHER_5`.

In Aura, Cypher 25 is the default for newly created organisations. The default query language attached to created databases is determined by the organisation setting "Default Cypher Version" and can be manually changed.

Neo4j 2025.06 - 2026.01

Databases created on, or migrated to, Neo4j 2025.06 or later will continue to have Cypher 5 as their default language (unless `db.query.default_language` is set to `CYPHER_25`). This is true for system, standard, and composite Neo4j databases. However, it is possible to set a different default language on both new and existing system, standard, and composite databases.

To select a default Cypher version when creating a database, add `DEFAULT LANGUAGE <language version>` to the `CREATE DATABASE` statement.

Example 1. Select default Cypher version when creating a database

Cypher 25

```
CREATE DATABASE actors SET DEFAULT LANGUAGE CYPHER 25
```

Cypher 5

```
CREATE DATABASE movies DEFAULT LANGUAGE CYPHER 5
```

To alter the default Cypher version on an existing database, add `SET DEFAULT LANGUAGE <language version>` to the `ALTER DATABASE` command.

Example 2. Alter the default Cypher version on an existing database

Cypher 25

```
ALTER DATABASE movies SET DEFAULT LANGUAGE CYPHER 25
```

Cypher 5

```
ALTER DATABASE actors SET DEFAULT LANGUAGE CYPHER 5
```

Selecting `CYPHER 25` ensures that all queries run on that database will be executed using Cypher 25 as it exists in the version of Neo4j that the database is currently running, provided it is on Neo4j 2025.06 or later (unless a query is prepended with `CYPHER 5`, which overrides this default).

Selecting `CYPHER 5` as the default database language ensures that all queries run on that database uses Cypher 5 as it existed at the time of the Neo4j 2025.06 release (unless a query is prepended with `CYPHER 25`, which overrides this default). Any changes introduced after the 2025.06 release will not affect the semantics of the query.

For more information about how to change and view the default language of new and existing databases and remote database aliases, and the new privileges related to changing the default language, see the following links in the Operations Manual:

- Configure the Cypher default version
- Set a default Cypher version for a standard database
- Set a default Cypher version for a composite database
- Set a default Cypher version for remote database aliases
- Alter the default Cypher version of an existing database
- Alter the default Cypher version of a composite database
- Alter the default Cypher version of a remote database alias
- Show the default Cypher version of a database
- Grant privilege to modify the default language of standard databases
- Grant privilege to modify composite databases (required in order to alter the default language of a composite database)

For information about Cypher versions and DBMS upgrades, see the Upgrade and migration guide → Cypher versions.

Migrating queries from Cypher 5 to Cypher 25

It is recommended to modify queries that depend on features deprecated in Cypher 5 and removed in Cypher 25 to align with the changes introduced in Cypher 25.

For example, Cypher 25 disallowed using a `NODE` or `RELATIONSHIP` instead of a `MAP` on the RHS of a `SET` clause, and instead requires the `properties()` function to get the map of properties from nodes or relationships before referencing them in a `SET` clause. The following example demonstrates how a query that works in Cypher 5 can be rewritten to work in Cypher 25.

Original Cypher 5 query

```
MATCH (n:Order)-[r:SHIPPED_TO]->(:Address)
SET n = r
```

Modified Cypher 25 query

```
MATCH (n:Order)-[r:SHIPPED_TO]->(:Address)
SET n = properties(r)
```

Another option is to prepend individual queries with a specific language version.

Select Cypher version for individual queries

To select the Cypher version of a query, prepend it with `CYPHER <language version>`. This selection will override the default language for the database the query is executed against, and allows you to either work with Cypher 25 features in a database that has Cypher 5 as the default language, or to continue running your Cypher 5 queries on a database that has Cypher 25 as the default language.

Queries run with Cypher 5 will eventually need to be updated to Cypher 25 as support for Cypher 5 will be discontinued in the future (anticipated no earlier than two additional server LTS cycles). It is, therefore, recommended to set the default language to Cypher 25 and migrate the necessary queries to its supported syntax rather than prepending individual queries with a Cypher version.

Example 3. Select the Cypher version for a query

Cypher 25 query on a Neo4j 2025.06+ database with Cypher 5 as default language

```
CYPHER 25
MATCH (n:Order)-[r:SHIPPED_TO]->(:Address)
SET n = properties(r)
```

Cypher 5 query on a Neo4j 2025.06+ database with Cypher 25 as default language

```
CYPHER 5
MATCH (n:Order)-[r:SHIPPED_TO]->(:Address)
SET n = r
```

Selecting `CYPHER 25` ensures that the query will be executed using Cypher 25 as it exists in the version of Neo4j that the database is currently running, provided it is on Neo4j 2025.06 or later.

Selecting `CYPHER 5` ensures that the query will be executed using Cypher 5 as it existed at the time of the Neo4j 2025.06 release. Any changes introduced after the 2025.06 release will not affect the query.

Procedures and functions

APOC) are tied to a specific Cypher language version. Therefore, procedures and functions in Neo4j 2025.06+ and APOC 2025.06+ (both of which have Cypher 5 as their default language) may behave differently depending on what version of Cypher is used.

For example, APOC 2025.06 removed Cypher 25 support of the procedure `apoc.create.uuids()`, meaning it is not available to queries running Cypher 25. However, it can still be used on APOC 2025.06 if queries are prepended with `CYPHER 5`, or if the database's default version is set to `CYPHER 5`. In this case, Neo4j will use APOC and Cypher 5 as they existed at the time of the 2025.06 release.

Using a procedure removed in Cypher 25 with APOC 2025.06+

```
CYPHER 5
CALL apoc.create.uuids(10)
```

Combine Cypher version selection with other query options

It is possible to combine Cypher version selection with other query options. The below example selects both the version and the runtime of Cypher for the same query:

Combining Cypher version selection with other query options

```
CYPHER 5 runtime=parallel
MATCH (n:Person)
RETURN n.name
```

Cypher versions and the `OPTIONS` map

The creating a vector index) are tied to a Neo4j version rather than a Cypher version, meaning updates to these settings will be supported in both Cypher 5 and Cypher 25 unless explicitly specified otherwise.

Further reading

- Neo4j Developer Blog: Cypher Versioning

Composed queries

`UNION`, `WHEN` and `NEXT` enable the composition of multiple separate query branches within a single query. `UNION` allows for combining the results of different queries. `WHEN` enables conditional queries, where different query branches can be made to execute depending on a set of criteria. Finally, `NEXT` allows for the linear composition of sequential queries, passing the return values from one query to the next. As such, `UNION`, `WHEN`, and `NEXT` manage in different ways the execution flow and logic of queries and cannot be used

in queries as regular clauses.

For more information, see:

- Combined queries (`UNION`)
- Conditional queries (`WHEN`) Cypher 25 only Introduced in Neo4j 2025.06
- Sequential queries (`NEXT`) Cypher 25 only Introduced in Neo4j 2025.06

Combined queries (`UNION`)

`UNION` combines the results of two or more queries into a single result set that includes all the rows that belong to any queries in the union.

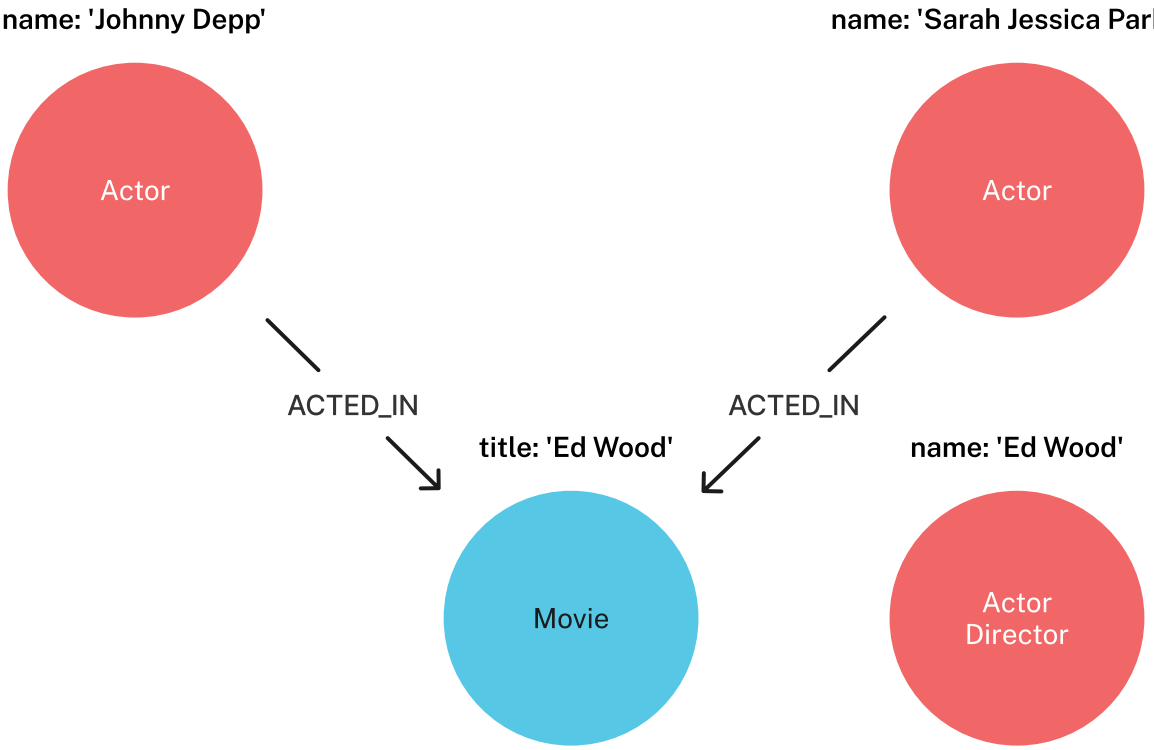
The number and the names of the columns must be identical in all queries combined by using `UNION`.

To keep all the result rows, use `UNION ALL`. Using just `UNION` (or `UNION DISTINCT`) will combine and remove duplicates from the result set.

> **Note:**
>
> If any of the queries in a `UNION` contain updates, the order of queries in the `UNION` is relevant.
>
> Any clause before the `UNION` cannot observe writes made by a clause after the `UNION`. Any clause after `UNION` can observe all writes made by a clause before the `UNION`.
>
> See clause composition in queries with `UNION` for details.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (johnny:Actor {name: 'Johnny Depp'}),
       (sarah:Actor {name: 'Sarah Jessica Parker'}),
       (ed:Actor&Director {name: 'Ed Wood'}),
       (edWoodMovie:Movie {title: 'Ed Wood'}),
       (johnny)-[:ACTED_IN]->(edWoodMovie),
       (sarah)-[:ACTED_IN]->(edWoodMovie)
```

Combine two queries and retain duplicates

Combining the results from two queries is done using `UNION ALL`.

Query

```
MATCH (n:Actor)
RETURN n.name AS name
UNION ALL
MATCH (n:Movie)
RETURN n.title AS name
```

The combined result is returned, including duplicates.

Table 12. Result

name
"Johnny Depp"
"Sarah Jessica Parker"
"Ed Wood"
"Ed Wood"
Rows: 4

Combine two queries and remove duplicates

By not including `ALL` in the `UNION`, duplicates are removed from the combined result set.

Query

```
MATCH (n:Actor)
RETURN n.name AS name
UNION
MATCH (n:Movie)
RETURN n.title AS name
```

The combined result is returned, without duplicates.

Table 13. Result

name
"Johnny Depp"
"Sarah Jessica Parker"
"Ed Wood"
Rows: 3

UNION DISTINCT

Removal of duplicates can also be accomplished by explicitly including `DISTINCT` in the `UNION`. The `UNION DISTINCT` keyword was introduced as part of Cypher's GQL conformance, and using it is functionally the same as using simple `UNION`.

Query

```
MATCH (n:Actor)
RETURN n.name AS name
UNION DISTINCT
MATCH (n:Movie)
RETURN n.title AS name
```

The combined result is returned, without duplicates.

Table 14. Result

name
"Johnny Depp"
"Sarah Jessica Parker"
"Ed Wood"
Rows: 3

Post-union processing

The `UNION` clause can be used within a `CALL` subquery to further process the combined results before a final

output is returned. For example, the below query counts the occurrences of each `name` property returned after the `UNION ALL` within the `CALL` subquery.

Query

```
CALL () {
  MATCH (a:Actor)
  RETURN a.name AS name
UNION ALL
  MATCH (m:Movie)
  RETURN m.title AS name
}
RETURN name, count(*) AS count
ORDER BY count
```

Table 15. Result

name	count
"Ed Wood"	2
"Johnny Depp"	1
"Sarah Jessica Parker"	1
Rows: 3	

For more information, see `CALL` subqueries → Post-union processing.

Combining UNION and UNION ALL

Introduced in Neo4j 2025.06 | Cypher 25 only

To combine `UNION` (or `UNION DISTINCT`) and `UNION ALL` in the same query, enclose one or more `UNION` operations of the same type in curly braces. This allows the enclosed query to act as an argument that can be combined with an outer `UNION` operation of any type.

Combine `UNION` and `UNION ALL`

```
{
   MATCH (n:Actor)
   RETURN n.name AS name
   UNION
   MATCH (n:Director)
   RETURN n.name AS name
}
UNION ALL
MATCH (n:Movie)
RETURN n.title AS name
```

The combined result is returned.

Table 16. Result

name
"Johnny Depp"
"Sarah Jessica Parker"

name
"Ed Wood"
"Ed Wood"
Rows: 4

Conditional queries (`WHEN`)

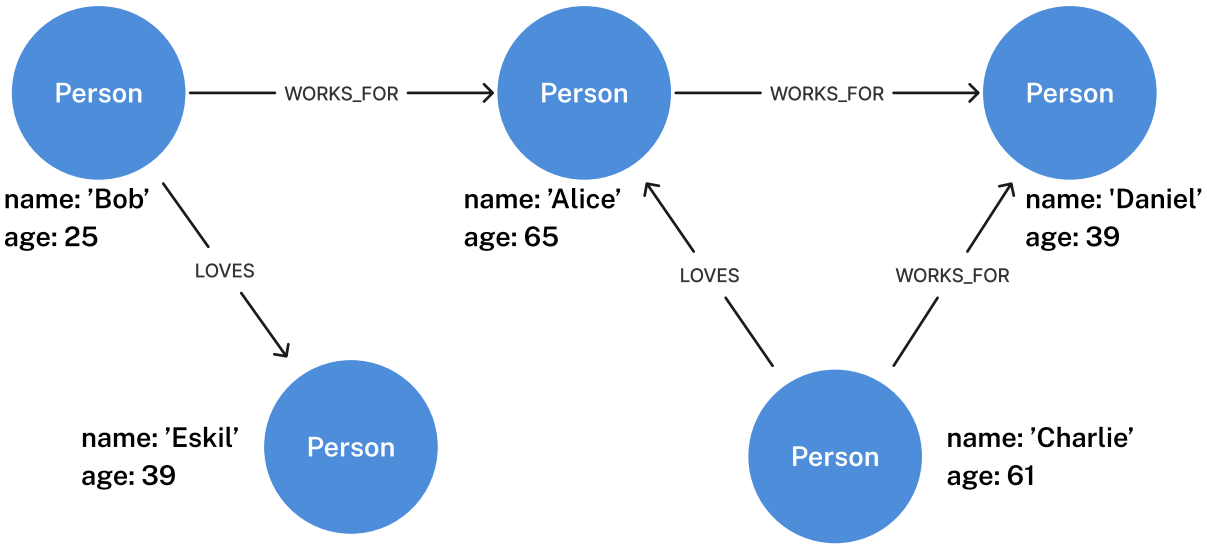
Introduced in Neo4j 2025.06 | Cypher 25 only

`WHEN`, together with `THEN` and `ELSE`, enables different branches of a query to execute based on certain conditions. In this way, it performs similar a control-flow mechanism to the `IF` statement in other programming languages.

> For information about using `WHEN` branches in expressions, see Conditional expressions (CASE)

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65}),
       (bob:Person {name: 'Bob', age: 25}),
       (charlie:Person {name: 'Charlie', age: 61}),
       (daniel:Person {name: 'Daniel', age: 39}),
       (eskil:Person {name: 'Eskil', age: 39}),
       (bob)-[:WORKS_FOR]->(alice),
       (alice)-[:WORKS_FOR]->(daniel),
       (charlie)-[:WORKS_FOR]->(daniel),
       (bob)-[:LOVES]->(eskil),
       (charlie)-[:LOVES]->(alice)
```

Standalone `WHEN` branches

Syntax for standalone `WHEN` branches

```
WHEN predicate THEN [{]
  <conditionalQuery>
  [}]
[WHEN ...]*
[ELSE [{]
  <conditionalQuery>
[}]]
```

The first branch with a predicate that evaluates to `true` will be executed. If no `WHEN` branches are executed and an `ELSE` branch exists, it is executed. If no `WHEN` branches evaluates to `true` and no `ELSE` branch is present, no branches are executed and no rows are produced. The following examples demonstrates this logic:

Conditional logic

```
WHEN false THEN RETURN 1 AS x
WHEN true THEN RETURN 2 AS x
WHEN true THEN RETURN 3 AS x
ELSE RETURN 3 AS x
```

Since the second `WHEN` branch is `true`, it will execute, while the preceding branch (which is `false`) and the succeeding `WHEN` branch (which is `true`) as well as the `ELSE` branch will be skipped.

Table 17. Result

x
2
Rows: 1

Conditionally executing queries in standalone `WHEN` branches

```
WHEN true THEN {
  MATCH (n:Person) WHERE n.name STARTS WITH "A"
  RETURN n.name AS name
}
ELSE {
  MATCH (n:Person)
  RETURN n.name AS name
}
```

The enclosing `{}` in the above example and the below examples are not required (unless explicitly stated), but they clarify the different conditional branches of the query.

Table 18. Result

name
"Alice"
Rows: 1

Rules

Similar to `UNION`, the number and names of the columns must be identical in all parts of a `WHEN`/`ELSE` construct. This means that expressions returned must be aliased using `AS`.

Not allowed: not aliasing returned expressions

```
WHEN true THEN RETURN 2
ELSE RETURN 3
```

Not allowed: using different column names in different branches

```
WHEN true THEN RETURN 2 AS x
ELSE RETURN 3 AS y
```

Not allowed: returning different number of columns in different branches

```
WHEN true THEN RETURN 2 AS x, 3 AS y
ELSE RETURN 3 AS x
```

`WHEN` can also not be positioned as a regular clause in a query. For example, it cannot immediately succeed a `MATCH` clause.

Not allowed: using `WHEN` constructs as regular clauses

```
MATCH (n)-[:WORKS_FOR]->(manager:Person)
WHEN manager IS NULL THEN {
  MERGE (n)-[:WORKS_FOR]->(newManager: Person {name: 'Peter', age: 36})
}
RETURN n.name AS employees,
       m.name AS manager,
       newManager.name AS newManagerNode
```

Instead, if `WHEN` constructs are part of a larger query, they must either be placed within a subquery and/or on different sides of combined `UNION` queries.

Conditional subqueries

`WHEN` can be used inside one or several `CALL` subqueries to execute a set of operations only when a specified condition evaluates to `true`.

Syntax for conditional `CALL` subqueries

```
[<outerQuery>]
<callSubquery> {
  WHEN predicate THEN [{]
    <conditionalQuery>
 [}]
 [WHEN ...]*
 [ELSE [{]
  <conditionalQuery>
 [}]]
}
[<callSubquery> ...]*
[<outerQuery>]
```

Example 4. Single conditional `CALL` subquery

In this example, `WHEN` is used to execute a `CALL` subquery for each row that the condition (`manager IS`

`NULL`) evaluates to `true`.

Conditional `CALL` subquery

```
MATCH (n:Person)
OPTIONAL MATCH (n)-[:WORKS_FOR]->(manager:Person)
CALL (*) {
  WHEN manager IS NULL THEN {
    MERGE (newManager: Person {name: 'Peter', age: 36})
    MERGE (n)-[:WORKS_FOR]->(newManager)
    RETURN newManager, n.name AS employee
  }
}
RETURN newManager.name AS newManager,
       collect(employee) AS employees
```

Because only `Daniel` and `Eskil` had no outgoing `WORKS_FOR` relationships, they have now been connected as employees of the new `Peter` node.

Table 19. Result

newManager	employees
"Peter"	["Daniel", "Eskil"]
Rows: 1	

Example 5. Chaining conditional `CALL` subqueries

Disjointed conditional `CALL` subqueries be chained in succession, each executing depending on a set of defined conditions.

This example uses conditional logic to assign age groups to individuals. The second `CALL` then collects the names and age groupings of their managers (who they `WORK_FOR`), but only for individuals whose manager is older than them.

Chaining several conditional `CALL` subqueries

```
MATCH (n:Person)
OPTIONAL MATCH (n)-[r:WORKS_FOR]->(m:Person)
CALL (*) {
  WHEN n.age > 60 THEN {
    SET n.ageGroup = 'Veteran'
    RETURN n.ageGroup AS ageGroup
  }
  WHEN n.age >= 35 AND n.age <= 59 THEN {
    SET n.ageGroup = 'Senior'
    RETURN n.ageGroup AS ageGroup
  }
  ELSE {
    SET n.ageGroup = 'Junior'
    RETURN n.ageGroup AS ageGroup
  }
}
CALL (*) {
  WHEN m.age > n.age THEN {
    RETURN collect([m.name, m.ageGroup]) AS manager
  }
}
RETURN n.name AS name, ageGroup, manager
```

`Bob` is returned because he is the only person in the graph with an older manager.

Table 20. Result

name	ageGroup	manager
"Bob"	"Junior"	[["Alice", "Veteran"]]
Rows: 1		

`EXISTS`, `COLLECT`, and `COUNT` subquery expressions can also contain `WHEN` branches.

Syntax for conditional `EXISTS`, `COLLECT`, and `COUNT` subqueries

```
[<outerExpression>]
EXISTS|COUNT|COLLECT {
  WHEN predicate THEN [{]
    <conditionalQuery>
 [}]
 [WHEN ...]*
 [ELSE [{]
    <conditionalQuery>
 [}]]
}
[<outerExpression>]
```

Example 6. Conditional `EXISTS` subquery

In this example, `WHEN` is used inside an `EXISTS` subquery to conditionally execute different branches based on the evaluation of the predicate (`n.age > 40`).

Unlike `CALL` subqueries, variables returned in an `EXISTS` subquery are not available to the outer scope (the same is true for `COUNT` and `COLLECT` subqueries).

`WHEN` inside an `EXISTS` subquery

```
MATCH (n:Person)
WHERE EXISTS {
  WHEN n.age > 40 THEN {
    RETURN n.name AS x
  }
  ELSE {
    MATCH (n)-[:LOVES]->(x:Person)
    RETURN x
  }
}
RETURN n.name AS name,
       n.age AS age
```

`Alice` and `Charlie` are both older than `40`, so they are returned by the `WHEN` branch, while `Bob` is returned by the `ELSE` branch. Note that some `Person` nodes in the graph are not matched in either branch of the conditional subquery, and are therefore not returned.

Table 21. Result

name	age
"Alice"	65
"Charlie"	61
"Bob"	25
Rows: 3	

Combining conditional queries with `UNION`

The results of multiple conditional queries can also be combined using `UNION [DISTINCT]` or `UNION ALL`; the former alternative removes duplicates from the result set, the latter does not (for more information, see Combined queries (`UNION`)).

If the conditional query begins with `WHEN` and involves `UNION`, the `WHEN` branches **must** be enclosed within curly braces, `{}` (the same `{}` can also be used to combine `UNION` and `UNION ALL`).

Syntax for combining standalone `WHEN` branches with `UNION` using `{}`

```
{
  WHEN predicate THEN [{]
    <conditionalQuery>
  [}]
  [WHEN ...]*
  [ELSE [{]
    <conditionalQuery>
[}]]
}
UNION [DISTINCT|ALL]
{
  WHEN predicate THEN [{]
   <conditionalQuery>
  [}]
  [WHEN ...]*
  [ELSE [{]
    <conditionalQuery>
[}]]
}
[UNION [DISTINCT|ALL] ...]*
```

Combining conditional branches with `UNION` using `{}`

```
{
  WHEN true THEN RETURN 1 AS x
  WHEN false THEN RETURN 2 AS x
  ELSE RETURN 3 AS x
}
UNION
{
  WHEN false THEN RETURN 4 AS x
  WHEN false THEN RETURN 5 AS x
  ELSE RETURN 6 AS x
}
```

Table 22. Result

x
1

x
6
Rows: 2

If the queries combined by the `UNION` do not begin with a `WHEN` branch, then enclosing curly braces are not necessary.

Example 7. Combining conditional logic with `UNION`

In the example below, `UNION` is used to combine the results of two conditional subqueries (the first `CALL`, enclosing the whole query, is necessary to enable the collection for each `person` from each conditional subquery part in the final results).

The query categorizes people based on two conditions: whether they have a `LOVES` relationship and whether they are younger or older than `40`. The final result provides a list of `status` messages for each `person`, showing the combined outcome of both conditions.

Combining conditional subqueries with `UNION`

```
MATCH (n:Person)
CALL (n) {
  OPTIONAL MATCH (n)-[r:LOVES]->(m:Person)
  CALL (*) {
    WHEN r IS NULL THEN {
      RETURN n.name AS person, "Loves no one" AS message
    }
    ELSE {
      RETURN n.name AS person, "Loves somebody" AS message
    }
  }
  RETURN person, message
  UNION
  CALL (*) {
    WHEN n.age < 40 THEN {
      RETURN n.name AS person, "Under 40" AS message
    }
    ELSE {
      RETURN n.name AS person, "40 or older" AS message
    }
  }
  RETURN person, message
}
RETURN person, collect(message) AS status
```

Table 23. Result

person	status
"Alice"	["Loves no one", "40 or older"]
"Charlie"	["Loves somebody", "40 or older"]
"Daniel"	["Loves no one", "Under 40"]
"Eskil"	["Loves no one", "Under 40"]
"Peter"	["Loves no one", "Under 40"]
"Bob"	["Loves somebody", "Under 40"]

person	status
Rows: 6	

Further reading

- Neo4j Developer Blog: Cypher Conditional Queries

Sequential queries (`NEXT`)

Introduced in Neo4j 2025.06 | Cypher 25 only

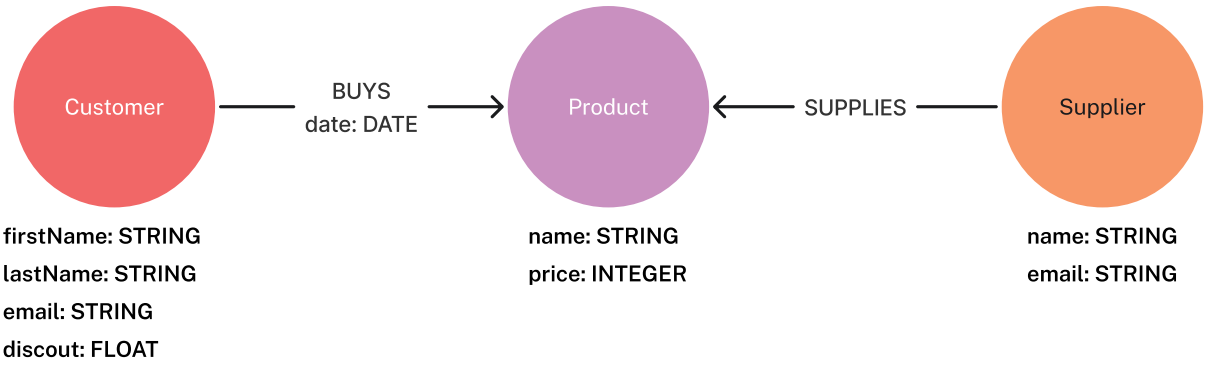
`NEXT` allows for linear composition of queries into a sequence of smaller, self-contained segments, passing the whole table of intermediate results from one segment to the next.

`NEXT` has the following benefits:

- `NEXT` can improve the modularity and readability of complex queries.
- `NEXT` can be used instead of WITH clause to construct complex queries.
- `NEXT` can improve the usability of Conditional queries (`WHEN`).
- `NEXT` allows for passing the full table of intermediate results into the branches of a `UNION`.

Example graph

The following graph is used for the examples on this page:



To recreate the graph, run the following query against an empty Neo4j database.

```
CREATE (techCorp:Supplier {name: 'TechCorp', email: 'contact@techcorp.com'}),
       (foodies:Supplier {name: 'Foodies Inc.', email: 'info@foodies.com'}),

       (laptop:Product {name: 'Laptop', price: 1000}),
       (phone:Product {name: 'Phone', price: 500}),
       (headphones:Product {name: 'Headphones', price: 250}),
       (chocolate:Product {name: 'Chocolate', price: 5}),
       (coffee:Product {name: 'Coffee', price: 10}),

       (amir:Customer {firstName: 'Amir', lastName: 'Rahman', email: 'amir.rahman@example.com', discount:
0.1}),
       (keisha:Customer {firstName: 'Keisha', lastName: 'Nguyen', email: 'keisha.nguyen@example.com',
discount: 0.2}),
```

```
       (mateo:Customer {firstName: 'Mateo', lastName: 'Ortega', email: 'mateo.ortega@example.com',
discount: 0.05}),
       (hannah:Customer {firstName: 'Hannah', lastName: 'Connor', email: 'hannah.connor@example.com',
discount: 0.15}),
       (leila:Customer {firstName: 'Leila', lastName: 'Haddad', email: 'leila.haddad@example.com',
discount: 0.1}),
       (niko:Customer {firstName: 'Niko', lastName: 'Petrov', email: 'niko.petrov@example.com', discount:
0.25}),
       (yusuf:Customer {firstName: 'Yusuf', lastName: 'Abdi', email: 'yusuf.abdi@example.com', discount:
0.1}),

       (amir)-[:BUYS {date: date('2024-10-09')}]->(laptop),
       (amir)-[:BUYS {date: date('2025-01-10')}]->(chocolate),
       (keisha)-[:BUYS {date: date('2023-07-09')}]->(headphones),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(chocolate),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(coffee),
       (mateo)-[:BUYS {date: date('2024-04-11')}]->(laptop),
       (hannah)-[:BUYS {date: date('2023-12-11')}]->(coffee),
       (hannah)-[:BUYS {date: date('2024-06-02')}]->(headphones),
       (leila)-[:BUYS {date: date('2023-05-17')}]->(laptop),
       (niko)-[:BUYS {date: date('2025-02-27')}]->(phone),
       (niko)-[:BUYS {date: date('2024-08-23')}]->(headphones),
       (niko)-[:BUYS {date: date('2024-12-24')}]->(coffee),
       (yusuf)-[:BUYS {date: date('2024-12-24')}]->(chocolate),
       (yusuf)-[:BUYS {date: date('2025-01-02')}]->(laptop),

       (techCorp)-[:SUPPLIES]->(laptop),
       (techCorp)-[:SUPPLIES]->(phone),
       (techCorp)-[:SUPPLIES]->(headphones),
       (foodies)-[:SUPPLIES]->(chocolate),
       (foodies)-[:SUPPLIES]->(coffee)
```

Syntax

`NEXT` syntax

```
<Query1>

NEXT

<Query2>

NEXT

<Query3>
```

Passing values to subsequent queries

`NEXT` passes the result table of a query to the subsequent query. In the following example, `NEXT` passes the variable `customer` to the second query:

Passing a variable to another query via `NEXT`

```
MATCH (c:Customer)
RETURN c AS customer

NEXT

MATCH (customer)-[:BUYS]->(:Product {name: 'Chocolate'})
RETURN customer.firstName AS chocolateCustomer
```

Result

chocolateCustomer
"Amir"
"Mateo"
"Yusuf"
Rows: 3

Passing multiple variables to another query via `NEXT`

```
MATCH (c:Customer)-[:BUYS]->(p:Product {name: 'Chocolate'})
RETURN c AS customer, p AS product

NEXT

RETURN customer.firstName AS chocolateCustomer,
       product.price * (1 - customer.discount) AS chocolatePrice
```

Result

chocolateCustomer	chocolatePrice
"Amir"	4.5
"Mateo"	4.75
"Yusuf"	4.5
Rows: 3	

> **Note:**
>
> When followed by `NEXT`, a `RETURN` clause may only contain variables or aliased expressions. Literals or unaliased expressions are not allowed. For example, `RETURN 1` and `RETURN 1 + 1` cannot precede `NEXT`, but `RETURN 1 AS one` and `RETURN 1 + 1 AS two` can.

Variables which are local to the query preceding `NEXT` and which are not explicitly returned as part of the result of that query are not accessible by subsequent queries in the context of `NEXT`. This allows you to control variable scope similarly to what you can do with `WITH`, see Control variables in scope.

Aggregation after `NEXT`

`NEXT` passes the result table as a whole to the subsequent query. This is particularly useful when aggregating values.

In the following example, `NEXT` passes the variable `customer` to the second query:

Aggregation after `NEXT`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
RETURN c AS customer, p AS product

NEXT

RETURN product.name AS product,
```

```
        COUNT(customer) AS numberOfCustomers
```

Result

product	numberOfCustomers
"Laptop"	4
"Chocolate"	3
"Headphones"	3
"Coffee"	3
"Phone"	1
Rows: 5	

Interactions with `UNION` queries

Using `UNION` after `NEXT`

If a `UNION` query follows a `NEXT` the full table of intermediate results is passed into all branches of the `UNION` query.

`UNION` after `NEXT`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
RETURN c, p

NEXT

RETURN c.firstName AS name, COLLECT(p.price * (1 - c.discount)) AS purchases, "discounted price" AS type
UNION
RETURN c.firstName AS name, COLLECT(p.price) AS purchases, "real price" AS type

NEXT

RETURN * ORDER BY name, type
```

Result

name	purchases	type
"Amir"	[900.0, 4.5]	"discounted price"
"Amir"	[1000, 5]	"real price"
"Hannah"	[212.5, 8.5]	"discounted price"
"Hannah"	[250, 10]	"real price"
"Keisha"	[200.0]	"discounted price"
"Keisha"	[250]	"real price"
"Leila"	[900.0]	"discounted price"
"Leila"	[1000]	"real price"
"Mateo"	[950.0, 4.75, 9.5]	"discounted price"

name	purchases	type
"Mateo"	[1000, 5, 10]	"real price"
"Niko"	[375.0, 187.5, 7.5]	"discounted price"
"Niko"	[500, 250, 10]	"real price"
"Yusuf"	[900.0, 4.5]	"discounted price"
"Yusuf"	[1000, 5]	"real price"
Rows: 14		

Using `UNION` before `NEXT`

If a `UNION` query precedes a `NEXT` the full result of the `UNION` is passed to the subsequent query.

`UNION` before `NEXT`

```
MATCH (c:Customer)-[:BUYS]->(:Product{name: "Laptop"})
RETURN c.firstName AS customer
UNION ALL
MATCH (c:Customer)-[:BUYS]-> (:Product{name: "Coffee"})
RETURN c.firstName AS customer

NEXT

RETURN customer AS customer, count(customer) as numberOfProducts
```

Result

customer	numberOfProducts
"Amir"	1
"Mateo"	2
"Leila"	1
"Yusuf"	1
"Hannah"	1
"Niko"	1
Rows: 6	

In this example, the list of customer names from the first segment has a duplicate entry for "Mateo" who bought both a laptop and coffee. The use of `UNION ALL` added him to the list twice. The second segment can access the list, because both parts of the `UNION` return a part of the list, aliased as `customer`. By using `count()`, the list aggregates the duplicate in the `RETURN` part of the query.

`NEXT` inside a `UNION` using `{}`

If a `UNION` query has a `NEXT` in any of its blocks, it is necessary to wrap that block with `{}`.

`NEXT` *inside* `UNION`

```
{
MATCH (c:Customer)-[:BUYS]->(:Product {name: 'Chocolate'})
RETURN c AS customer

NEXT

RETURN customer.firstName AS plantCustomer
}

UNION ALL

{
MATCH (c:Customer)-[:BUYS]->(:Product {name: 'Coffee'})
RETURN c AS customer

NEXT

RETURN customer.firstName AS plantCustomer
}
```

Result

plantCustomer
"Amir"
"Mateo"
"Yusuf"
"Mateo"
"Hannah"
"Niko"
Rows: 6

Interactions with `CALL` subqueries

`CALL` subqueries pass the table of intermediate results to the subquery row by row, while `NEXT` passes the table as a whole. If `NEXT` is wrapped in a `CALL` subquery, the result of the preceding query is passed to `NEXT` a single row at a time. This can be used to compute more complex aggregates in groups.

`NEXT` *inside* `CALL`

```
MATCH (p:Product) WHERE p.name <> "Coffee"
CALL (p) {
    MATCH (p)<-[:BUYS]-(c:Customer)-[:BUYS]->(otherProduct)
    RETURN c, otherProduct

    NEXT

    RETURN count(DISTINCT c) AS customers, 0 AS customersAlsoBuyingCoffee
    UNION
    FILTER otherProduct.name = "Coffee"
    RETURN 0 as customers, count(DISTINCT c) AS customersAlsoBuyingCoffee

    NEXT

    RETURN max(customers) AS customers, max(customersAlsoBuyingCoffee) AS customersAlsoBuyingCoffee
}
RETURN p.name AS product,
       round(toFloat(customersAlsoBuyingCoffee) * 100 / customers, 1) AS
```

```
percentageOfCustomersAlsoBuyingCoffee
  ORDER BY product
```

Result

product	percentageOfCustomersAlsoBuyingCoffee
"Chocolate"	33.3
"Headphones"	100.0
"Laptop"	33.3
"Phone"	100.0
Rows: 5	

This example computes the percentage of customers that also bought coffee for each non-coffee product. For each product `p` the subquery finds all pairs of a customer `c` of product `p` and another product `otherProduct` that the customer has also bought. The first `NEXT` passes these pairs as a whole into a `UNION`, so that the query can:

1. count all customers in the first branch of the union.
2. count the customers who also bought coffee in the second branch of the union.

The `UNION` produces two rows — one from each branch. The second `NEXT` passes these two rows as a whole to a query that aggregates them into a single row, which is the result of the `CALL` subquery.

Note:

`NEXT` cannot be used inside a `CALL` subquery that uses the (deprecated) importing `WITH` syntax.

Interactions with conditional queries

Using conditional queries before or after `NEXT`

Conditional queries act similar to `CALL` by processing the incoming table of intermediate result row by row. A conditional query following a `NEXT` acts equivalent to a conditional query wrapped in a `CALL` subquery.

Conditional query inside `NEXT`

```
MATCH (c:Customer)-[:BUYS]->(:Product)<-[:SUPPLIES]-(s:Supplier)
RETURN c.firstName AS customer, s.name AS supplier

NEXT

WHEN supplier = "TechCorp" THEN
  RETURN customer, "Tech enjoyer" AS personality
WHEN supplier = "Foodies Inc." THEN
  RETURN customer, "Tropical plant enjoyer" AS personality

NEXT

RETURN customer, collect(DISTINCT personality) AS personalities

NEXT
```

```
WHEN size(personalities) > 1 THEN
  RETURN customer, "Enjoyer of tech and plants" AS personality
ELSE
  RETURN customer, personalities[0] AS personality
```

Result

customer	personality
"Amir"	"Enjoyer of tech and plants"
"Mateo"	"Enjoyer of tech and plants"
"Yusuf"	"Enjoyer of tech and plants"
"Niko"	"Enjoyer of tech and plants"
"Hannah"	"Enjoyer of tech and plants"
"Leila"	"Tech enjoyer"
"Keisha"	"Tech enjoyer"

Rows: 7

In the query above, customers are assigned personality types based on the products they purchased. The second segment is a conditional query that returns different base personality types for different suppliers the customers purchased from. The third segment aggregates the personality types. Finally, the fourth segment is another conditional query which subsumes multiple base personality types, if present, to a new personality.

`NEXT` inside a conditional query using `{}`

If a conditional query has a `NEXT` in any of its `THEN` or `ELSE` blocks, it is necessary to wrap the part after `THEN` or `ELSE` with `{}`.

`NEXT` inside conditional query

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
RETURN c AS customer, sum(p.price) AS sum

NEXT

WHEN sum >= 1000 THEN {
  RETURN customer.firstName AS customer, "club 1000 plus" AS customerType, sum AS sum
}
ELSE {
  RETURN sum * (1 - customer.discount) AS finalSum

  NEXT

  RETURN customer.firstName AS customer, "club below 1000" AS customerType, finalSum AS sum
}
```

Result

customer	customerType	sum
"Amir"	"club 1000 plus"	1005

customer	customerType	sum
"Mateo"	"club 1000 plus"	1015
"Leila"	"club 1000 plus"	1000
"Yusuf"	"club 1000 plus"	1005
"Keisha"	"club below 1000"	200.0
"Hannah"	"club below 1000"	221.0
"Niko"	"club below 1000"	570.0
Rows: 3		

The query above calculates the total price of products purchased per customer and then only applies the customer discount to sums below 1000.

Known issues and fixes

`NEXT` initially did not correctly handle aggregations in the context of `UNION` branches and `CALL` subqueries. As of Neo4j 2025.08, this issue has been fixed. The table below summarizes the behavior changes across versions.

Neo4j version	Behavior
2025.06	`NEXT` yields incorrect results when using aggregations in the context of `UNION` branches and `CALL` subqueries.
2025.07	Using aggregations with `NEXT` in the context of `UNION` branches and `CALL` subqueries throws an error.
2025.08+	`NEXT` correctly supports aggregations in the context of `UNION` branches and `CALL` subqueries. See above for more details.

Clauses

This section contains information on all the clauses in the Cypher query language.

Reading clauses

These comprise clauses that read data from the database.

The flow of data within a Cypher query is an unordered sequence of maps with key-value pairs — a set of possible bindings between the variables in the query and values derived from the database. This set is refined and augmented by subsequent parts of the query.

Clause	Description
`FILTER`	Adds filters to queries. Cypher 25 only Introduced in Neo4j 2025.06
`MATCH`	Specify the patterns to search for in the database.
`OPTIONAL MATCH`	Specify the patterns to search for in the database while using `nulls` for missing parts of the pattern.

Projecting clauses

These comprise clauses that define which expressions to return in the result set. The returned expressions may all be aliased using `AS`.

Clause	Description
`FINISH`	Defines a query to have no result.
`LET`	Binds values to variables. Cypher 25 only Introduced in Neo4j 2025.06
`RETURN … [AS]`	Defines what to include in the query result set.
`UNWIND … [AS]`	Expands a list into a sequence of rows.
`FOR`	Expands a list into a sequence of rows (GQL-aligned syntax; same semantics as `UNWIND`). Cypher 25 only Introduced in Neo4j 2026.04
`WITH … [AS]`	Allows query parts to be chained together, piping the results from one to be used as starting points or criteria in the next.

Reading sub-clauses

These comprise sub-clauses that must operate as part of reading clauses.

Sub-clause	Description
`WHERE`	Adds constraints to the patterns in a `MATCH` or `OPTIONAL MATCH` clause or filters the results of a `WITH` clause.
`SEARCH`	A subclause following `MATCH` or `OPTIONAL MATCH`, filtering the results based on approximate nearest neighbor (ANN) vector search or full-text search.
`ORDER BY [ASC[ENDING\ | DESC[ENDING\]\]]`	A subclause following `RETURN` or `WITH`, specifying that the output should be sorted in either ascending (the default) or descending order.
`SKIP / OFFSET`	Defines from which row to start including the rows in the output.
`LIMIT`	Constrains the number of rows in the output.

Writing clauses

These comprise clauses that write the data to the database.

Clause	Description
`CREATE`	Create nodes and relationships.
`DELETE`	Delete nodes, relationships or paths. Any node to be deleted must also have all associated relationships explicitly deleted.
`DETACH DELETE`	Delete a node or set of nodes. All associated relationships will automatically be deleted.
`SET`	Update labels on nodes and properties on nodes and relationships.
`REMOVE`	Remove properties and labels from nodes and relationships.
`FOREACH`	Update data within a list, whether components of a path, or the result of aggregation.

Reading/Writing clauses

These comprise clauses that both read data from and write data to the database.

Clause	Description
`MERGE`	Ensures that a pattern exists in the graph. Either the pattern already exists, or it needs to be created.

Clause	Description
`--- ON CREATE`	Used in conjunction with `MERGE`, this write sub-clause specifies the actions to take if the pattern needs to be created.
`--- ON MATCH`	Used in conjunction with `MERGE`, this write sub-clause specifies the actions to take if the pattern already exists.
`CALL … [YIELD … ]`	Invokes a procedure deployed in the database and return any results.

Subquery clauses

Clause	Description
`CALL { … }`	Evaluates a subquery, typically used for post-union processing or aggregations.
`CALL { … } IN TRANSACTIONS`	Evaluates a subquery in separate transactions. Typically used when modifying or importing large amounts of data.

Set operations

Clause	Description
`UNION`	Combines the result of multiple queries into a single result set. Duplicates are removed.
`UNION ALL`	Combines the result of multiple queries into a single result set. Duplicates are retained.

Multiple graphs

Clause	Description
`USE`	Determines which graph a query, or query part, is executed against.

Importing data

Clause	Description
`LOAD CSV`	Use when importing data from CSV files.
`CALL { … } IN TRANSACTIONS`	This clause may be used to prevent an out-of-memory error from occurring when importing large amounts of data using `LOAD CSV`.

Index and schema clauses

These comprise clauses to manage indexes, constraints, and graph types.

Clause	Description
`CREATE | SHOW | DROP INDEX`	Create, show or drop an index.
`CREATE | SHOW | DROP CONSTRAINT`	Create, show or drop a constraint.
`ALTER CURRENT GRAPH TYPE SET | ADD | ALTER |DROP`	Set, extend, alter or drop elements in a graph type.
`SHOW CURRENT GRAPH TYPE`	Show the full graph type.

Reading hints

These comprise clauses used to specify planner hints when tuning a query. More details regarding the usage of these — and query tuning in general — can be found in Planner hints and the USING keyword.

Hint	Description
`USING INDEX`	Index hints are used to specify which index, if any, the planner should use as a starting point.
`USING INDEX SEEK`	Index seek hint instructs the planner to use an index seek for this clause.
`USING SCAN`	Scan hints are used to force the planner to do a label scan (followed by a filtering operation) instead of using an index.
`USING JOIN`	Join hints are used to enforce a join operation at specified points.

SHOW commands

For information about the `SHOW` command in general, see SHOW.

Clause	Description
`SHOW AUTH RULES` (Operations Manual)	Show all auth rules.
`SHOW ALIASES` (Operations Manual)	Show local and remote database aliases.
`SHOW CONSTRAINTS`.	Show the constraints set on database.
`SHOW CURRENT GRAPH TYPE`	Show the graph type for a database. Cypher 25 only Introduced in Neo4j 2026.02
`SHOW DATABASES` (Operations Manual)	Show the databases available in a Neo4j instance.

Clause	Description
`SHOW FUNCTIONS`	Show the available functions.
`SHOW INDEXES`	Show the indexes set on a database.
`SHOW PRIVILEGES` (Operations Manual)	Show privileges assigned or revoked from a role.
`SHOW PROCEDURES` (Operations Manual)	Show the available procedures.
`SHOW ROLES` (Operations Manual)	Show the available roles.
`SHOW SERVERS` (Operations Manual)	Show all servers visible to a cluster.
`SHOW SETTINGS` (Operations Manual)	Show all configuration settings.
`SHOW TRANSACTIONS` (Operations Manual)	Show all currently running transactions across an instance.
`SHOW USERS` (Operations Manual)	Show all users.

TERMINATE Commands

Clause	Description
`TERMINATE TRANSACTIONS (Operations Manual)`	Terminate transactions by their IDs.

Administration clauses

Cypher includes commands to manage databases, aliases, servers, and role-based access control. To learn more about each of these, see:

- Operations Manual → Database administration
- Operations Manual → Authentication and authorization
- Operations Manual → Clustering

Clause composition

This section describes the semantics of Cypher when composing different read and write clauses.

A query is made up from several clauses chained together. These are discussed in more detail in the chapter on Clauses.

The semantics of a whole query is defined by the semantics of its clauses. Each clause has as input the state of the graph and a table of intermediate results consisting of the current variables. The output of a clause is a new state of the graph and a new table of intermediate results, serving as input to the next clause. The first clause takes as input the state of the graph before the query and a table initialized with a single empty row for intermediate results. The output of the last clause is the result of the query.

Most clauses are applied to each row of the table of intermediate results in turn, producing zero, one, or

more output rows for each input row. The exceptions to this rule are projections such as `RETURN` or `WITH` with aggregation functions, which operate on the whole table of intermediate results at once.

> **Note:**
>
> Unless ORDER BY is used, Neo4j does not guarantee the row order of a query result.

Example 8. Table of intermediate results between read clauses

The following example graph is used throughout this section.

```
digraph L { node [shape=record style=rounded];
  N0 [
    label = "{Person|name = \'John\'\l}"
  ]
  N0 -> N3 [
    color = "#2e3436"
    fontcolor = "#2e3436"
    label = "FRIEND\n"
  ]
  N0 -> N1 [
    color = "#2e3436"
    fontcolor = "#2e3436"
    label = "FRIEND\n"
  ]
  N1 [
    label = "{Person|name = \'Joe\'\l}"
  ]
  N1 -> N2 [
    color = "#2e3436"
    fontcolor = "#2e3436"
    label = "FRIEND\n"
  ]
  N2 [
    label = "{Person|name = \'Steve\'\l}"
  ]
  N3 [
    label = "{Person|name = \'Sara\'\l}"
  ]
  N3 -> N4 [
    color = "#2e3436"
    fontcolor = "#2e3436"
    label = "FRIEND\n"
  ]
  N4 [
    label = "{Person|name = \'Maria\'\l}"
  ]
}
```

Now follows the table of intermediate results and the state of the graph after each clause for the following query:

```
MATCH (john:Person {name: 'John'})
MATCH (john)-[:FRIEND]->(friend)
RETURN friend.name AS friendName
```

The query only has read clauses, so the state of the graph remains unchanged and is therefore omitted below.

The table of intermediate results after each clause

Clause	Table of intermediate results after the clause
`MATCH (john:Person {name: 'John'})`	**john** `(:Person {name: 'John'})`

`MATCH (john)-[:FRIEND]->(friend)`	**john** / **friend**
	`(:Person {name: 'John'})` / `(:Person {name: 'Sara'})`
	`(:Person {name: 'John'})` / `(:Person {name: 'Joe'})`

`RETURN friend.name AS friendName`	**friendName**
	`'Sara'`
	`'Joe'`

The above example only looked at clauses that allow linear composition and omitted write clauses. The next section will explore these non-linear composition and write clauses.

Read-write queries

In a Cypher query, read and write clauses can take turns. The most important aspect of read-write queries is that the state of the graph also changes between clauses.

> A clause can never observe writes made by a later clause, and will observe all writes done by the previous clauses.

As of Cypher 25, read and write clauses can be combined in any order. That is, a write clause followed by a read clause no longer requires a separating `WITH` clause in order for the read clause to observe the changes made by a preceding write clause. For example, the following query, in which the changes made by a write clause (`SET`) are observed by a subsequent `MATCH` clause without an intermediate `WITH` clause, is valid using Cypher 25 but not Cypher 5.

Combine write and read clauses without a separating `WITH` clause

```
MATCH (j:Person {name: 'John'})-[:FRIEND]->(f)
SET f.degreesFromJohn = 1
MATCH (f)-[:FRIEND]->(f2)
SET f2.degreesFromJohn = f.degreesFromJohn + 1
RETURN f.name AS friendName,
       f.degreesFromJohn AS friendDegree,
       f2.name AS friendOfFriendName,
       f2.degreesFromJohn AS friendOfFriendDegree
```

Example 9. Table of intermediate results and state of the graph between read and write clauses

Using the same example graph as above, this example shows the table of intermediate results and the state of the graph after each clause for a query with a read clause before a write clause:

```
MATCH (j:Person) WHERE j.name STARTS WITH "J"
CREATE (j)-[:FRIEND]->(jj:Person {name: "Jay-jay"})
```

The query finds all nodes where the `name` property starts with "J" and for each such node it creates another node with the `name` property set to "Jay-jay".

Read-write: The table of intermediate results and the state of the graph after each clause+

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red

`MATCH (j:Person) WHERE j.name STARTS WITH "J"`	j
	`(:Person {name: 'John'})`
	`(:Person {name: 'Joe'})`
		`digraph L { node [shape=record style=rounded]; N0 [label = "{Person|name = \'John\'\l}"] N0 -> N3 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N0 -> N1 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N1 [label = "{Person|name = \'Joe\'\l}"] N1 -> N2 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N2 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Steve\'\l}"] N3 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Sara\'\l}"] N3 -> N4 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N4 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Maria\'\l}"] }`

<table>
<tr><th>Clause</th><th>Table of intermediate results after the clause</th><th>State of the graph after the clause, changes in red</th></tr>
<tr>
<td>

```
CREATE (j)-[:FRIEND]-
>(jj:Person {name: "Jay-
jay"})
```

</td>
<td>
<table>
<tr><th>j</th><th>jj</th></tr>
<tr><td>(:Person {name: 'John'})</td><td>(:Person {name: 'Jay-jay'})</td></tr>
<tr><td>(:Person {name: 'Joe'})</td><td>(:Person {name: 'Jay-jay'})</td></tr>
</table>
</td>
<td>

```
digraph L { node [shape=record
style=rounded];
N0 [
label = "{Person|name = \'John\'\l}"
]
N0 -> N3 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
N0 -> N5 [
color = "red"
fontcolor = "red"
label = "FRIEND\n"
]
N5 [
color = "red"
fontcolor = "red"
label = "{Person|name = \'Jay-
jay\'\l}"
]
N1 -> N6 [
color = "red"
fontcolor = "red"
label = "FRIEND\n"
]
N6 [
color = "red"
fontcolor = "red"
label = "{Person|name = \'Jay-
jay\'\l}"
]
}
```

</td>
</tr>
</table>

It is important to note that the `MATCH` clause does not find the `Person` nodes that are created by the `CREATE` clause, even though the name "Jay-jay" starts with "J". This is because the `CREATE` clause comes after the `MATCH` clause and thus the `MATCH` can not observe any changes to the graph made by the `CREATE`.

Example 10. Table of intermediate results and state of the graph between write and read clauses

On an empty graph, this example shows the table of intermediate results and the state of the graph after each clause for a query with a write clause before a read clause:

```
UNWIND ["Max", "Lune"] AS dogName
CREATE (n:Dog {name: dogName})
WITH n
MATCH (d:Dog)
RETURN COUNT(*)
```

This query creates two `Dog` nodes and returns the value `4`.

Write-read: The table of intermediate results and the state of the graph after each clause

<table>
<tr><th>Clause</th><th>Table of intermediate results after the clause</th><th>State of the graph after the clause, changes in red</th></tr>
<tr><td><pre>UNWIND ["Max", "Luna"] AS dogName</pre></td><td><table><tr><th>dogName</th></tr><tr><td>"Max"</td></tr><tr><td>"Luna"</td></tr></table></td><td></td></tr>
<tr><td><pre>CREATE (n:Dog {name: dogName})</pre></td><td><table><tr><th>dogName</th><th>n</th></tr><tr><td>"Max"</td><td>(:Dog {name: 'Max'})</td></tr><tr><td>"Luna"</td><td>(:Dog {name: 'Luna'})</td></tr></table></td><td><pre>digraph L { node [shape=record style=rounded];
N5 [
color = "red"
fontcolor = "red"
label = "{Dog|name = \'Max\'\l}"
]
N6 [
color = "red"
fontcolor = "red"
label = "{Dog|name = \'Luna\'\l}"
]
}</pre></td></tr>
<tr><td><pre>MATCH (d:Dog)</pre></td><td><table><tr><th>dogName</th><th>n</th><th>d</th></tr><tr><td>"Max"</td><td>(:Dog {name: 'Max'})</td><td>(:Dog {name: 'Max'})</td></tr><tr><td>"Max"</td><td>(:Dog {name: 'Max'})</td><td>(:Dog {name: 'Luna'})</td></tr><tr><td>"Luna"</td><td>(:Dog {name: 'Luna'})</td><td>(:Dog {name: 'Max'})</td></tr><tr><td>"Luna"</td><td>(:Dog {name: 'Luna'})</td><td>(:Dog {name: 'Luna'})</td></tr></table></td><td><pre>digraph L { node [shape=record style=rounded];
N5 [
label = "{Dog|name = \'Max\'\l}"
]
N6 [
label = "{Dog|name = \'Luna\'\l}"
]
}</pre></td></tr>
</table>

It is important to note that the `MATCH` clause reads all the `Dog` nodes that are created by the `CREATE` clause. This is because the `CREATE` clause comes before the `MATCH` clause and thus the `MATCH` observes all changes to the graph made by the `CREATE`. The `MATCH` clause is performed for every intermediate result, this leads to finding two nodes for both intermediate results.

Queries with `UNION`

`UNION` queries are slightly different because the results of two or more queries are put together, but each query starts with an empty table of intermediate results.

In a query with a `UNION` clause, any clause *before* the `UNION` cannot observe writes made by a clause *after* the `UNION`. Any clause *after* `UNION` can observe all writes made by a clause *before* the `UNION`. This means that the rule that a clause can never observe writes made by a later clause still applies in queries using `UNION`.

Example 11. Table of intermediate results and state of the graph in a query with `UNION`

Using the same example graph as above, this example shows the table of intermediate results and the state of the graph after each clause for the following query:

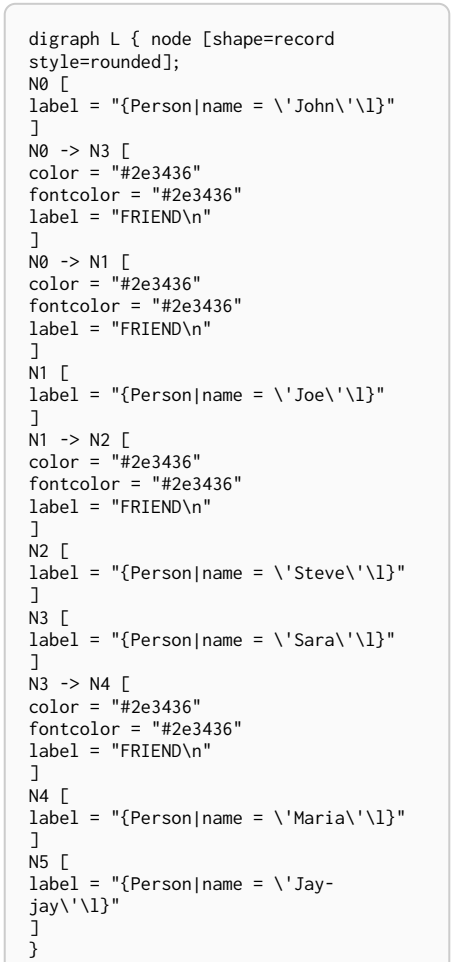
```
CREATE (jj:Person {name: "Jay-jay"})
RETURN count(*) AS count
  UNION
MATCH (j:Person) WHERE j.name STARTS WITH "J"
RETURN count(*) AS count
```

The table of intermediate results and the state of the graph after each clause

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red
`CREATE (jj:Person {name: "Jay-jay"})`	jj `(:Person {name: 'Jay-jay'})`	`digraph L { node [shape=record style=rounded]; N0 [label = "{Person|name = \'John\'\l}"] N0 -> N3 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N0 -> N1 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N1 [label = "{Person|name = \'Joe\'\l}"] N1 -> N2 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N2 [label = "{Person|name = \'Steve\'\l}"] N3 [label = "{Person|name = \'Sara\'\l}"] N3 -> N4 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N4 [label = "{Person|name = \'Maria\'\l}"] N5 [color = "red" fontcolor = "red" label = "{Person|name = \'Jay-jay\'\l}"] }`

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red
`RETURN count(*) AS count`	count 1	`digraph L { node [shape=record style=rounded]; N0 [label = "{Person|name = \'John\'\l}"] N0 -> N3 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N0 -> N1 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N1 [label = "{Person|name = \'Joe\'\l}"] N1 -> N2 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N2 [label = "{Person|name = \'Steve\'\l}"] N3 [label = "{Person|name = \'Sara\'\l}"] N3 -> N4 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N4 [label = "{Person|name = \'Maria\'\l}"] N5 [label = "{Person|name = \'Jay-jay\'\l}"] }`

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red
`MATCH (j:Person) WHERE j.name STARTS WITH "J"`	j (:Person {name: 'John'}) (:Person {name: 'Joe'}) (:Person {name: 'Jay-jay'})	`digraph L { node [shape=record style=rounded]; N0 [label = "{Person|name = \'John\'\l}"] N0 -> N3 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N0 -> N1 [color = "#2e3436" fontcolor = "#2e3436" label = "FRIEND\n"] N1 [label = "{Person|name = \'Joe\'\l}"] N1 -> N2 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N2 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Steve\'\l}"] N3 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Sara\'\l}"] N3 -> N4 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N4 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Maria\'\l}"] N5 [label = "{Person|name = \'Jay-jay\'\l}"] }`

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red
`RETURN count(*) AS count`	count 3	

It is important to note that the `MATCH` clause finds the `Person` node that is created by the `CREATE` clause. This is because the `CREATE` clause comes before the `MATCH` clause and thus the `MATCH` can observe any changes to the graph made by the `CREATE`.

Queries with `CALL {}` subqueries

Subqueries inside a `CALL {}` clause are evaluated for each incoming input row. This means that write clauses inside a subquery can get executed more than once. The different invocations of the subquery are executed in turn, in the order of the incoming input rows.

Later invocations of the subquery can observe writes made by earlier invocations of the subquery.

Example 12. Table of intermediate results and state of the graph in a query with `CALL {}`

Using the same example graph as above, this example shows the table of intermediate results and the state of the graph after each clause for the following query:

> The below query uses a variable scope clause to import variables into the `CALL` subquery.

```
MATCH (john:Person {name: 'John'})
SET john.friends = []
WITH john
MATCH (john)-[:FRIEND]->(friend)
WITH john, friend
CALL (john, friend) {
  WITH john.friends AS friends
  SET john.friends = friends + friend.name
}
```

The table of intermediate results and the state of the graph after each clause

Clause	Table of intermediate results after the clause	State of the graph after the clause, changes in red
`MATCH (john:Person {name: 'John'})`	john `(:Person {name: 'John'})`	`digraph L { node [shape=record style=rounded]; N0 [label = "{Person|name = \'John\'\l}"] N0 -> N3 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N0 -> N1 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N1 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Joe\'\l}"] N1 -> N2 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N2 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Steve\'\l}"] N3 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Sara\'\l}"] N3 -> N4 [color = "grey" fontcolor = "grey" label = "FRIEND\n"] N4 [color = "grey" fontcolor = "grey" label = "{Person|name = \'Maria\'\l}"] }`

<table>
<tr><th>Clause</th><th>Table of intermediate results after the clause</th><th>State of the graph after the clause, changes in red</th></tr>
<tr>
<td>

```
SET john.friends = []
```

</td>
<td>

john
(:Person {name: 'John', friends: []})

</td>
<td>

```
digraph L { node [shape=record
style=rounded];
N0 [
color = "red"
fontcolor = "red"
label = "{Person|name =
\'John\'\l|friends = []\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
}
```

</td>
</tr>
<tr>
<td>

```
MATCH (john)-[:FRIEND]-
>(friend)
```

</td>
<td>

john	friend
(:Person {name: 'John', friends: []})	(:Person {name: 'Sara'})
(:Person {name: 'John', friends: []})	(:Person {name: 'Joe'})

</td>
<td>

```
digraph L { node [shape=record
style=rounded];
N0 [
label = "{Person|name =
\'John\'\l|friends = []\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
color = "grey"
fontcolor = "grey"
label = "FRIEND\n"
]
N2 [
color = "grey"
fontcolor = "grey"
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
color = "grey"
fontcolor = "grey"
label = "FRIEND\n"
]
N4 [
color = "grey"
fontcolor = "grey"
label = "{Person|name = \'Maria\'\l}"
]
}
```

</td>
</tr>
</table>

<table>
<tr><th>Clause</th><th>Table of intermediate results after the clause</th><th>State of the graph after the clause, changes in red</th></tr>
<tr>
<td>First invocation of

<pre>WITH john.friends AS friends</pre></td>
<td>
<table>
<tr><th>john</th><th>friend</th><th>friends</th></tr>
<tr><td>(:Person {name: 'John', friends: []})</td><td>(:Person {name: 'Sara'})</td><td>[]</td></tr>
</table>
</td>
<td><pre>digraph L { node [shape=record style=rounded];
N0 [
label = "{Person|name = \'John\'\l|friends = []\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
}</pre></td>
</tr>
<tr>
<td>First invocation of

<pre>SET john.friends = friends + friend.name</pre></td>
<td>
<table>
<tr><th>john</th><th>friend</th><th>friends</th></tr>
<tr><td>(:Person {name: 'John', friends: ['Sara']})</td><td>(:Person {name: 'Sara'})</td><td>[]</td></tr>
</table>
</td>
<td><pre>digraph L { node [shape=record style=rounded];
N0 [
color = "red"
fontcolor = "red"
label = "{Person|name = \'John\'\l|friends = ['Sara']\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
}</pre></td>
</tr>
</table>

<table>
<tr><th>Clause</th><th>Table of intermediate results after the clause</th><th>State of the graph after the clause, changes in red</th></tr>
<tr>
<td>Second invocation of

<pre>WITH john.friends AS
friends</pre></td>
<td><table>
<tr><th>john</th><th>friend</th><th>friends</th></tr>
<tr><td>(:Person {name:
'John', friends:
['Sara']})</td><td>(:Person
{name:
'Joe'})</td><td>['Sara']</td></tr>
</table></td>
<td><pre>digraph L { node [shape=record
style=rounded];
N0 [
label = "{Person|name =
\'John\'\l|friends = ['Sara']\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
}</pre></td>
</tr>
<tr>
<td>Second invocation of

<pre>SET john.friends = friends
+ friend.name</pre></td>
<td><table>
<tr><th>john</th><th>friend</th><th>friends</th></tr>
<tr><td>(:Person {name:
'John', friends:
['Sara', 'Joe']})</td><td>(:Person
{name:
'Joe'})</td><td>['Sara']</td></tr>
</table></td>
<td><pre>digraph L { node [shape=record
style=rounded];
N0 [
color = "red"
fontcolor = "red"
label = "{Person|name =
\'John\'\l|friends = ['Sara',
'Joe']\l}"
]
N0 -> N3 [
label = "FRIEND\n"
]
N0 -> N1 [
color = "#2e3436"
fontcolor = "#2e3436"
label = "FRIEND\n"
]
N1 [
label = "{Person|name = \'Joe\'\l}"
]
N1 -> N2 [
label = "FRIEND\n"
]
N2 [
label = "{Person|name = \'Steve\'\l}"
]
N3 [
label = "{Person|name = \'Sara\'\l}"
]
N3 -> N4 [
label = "FRIEND\n"
]
N4 [
label = "{Person|name = \'Maria\'\l}"
]
}</pre></td>
</tr>
</table>

It is important to note that, in the subquery, the second invocation of the `WITH` clause could observe the writes made by the first invocation of the `SET` clause.

Notes on the implementation

An easy way to implement the semantics outlined above is to fully execute each clause and materialize the table of intermediate results before executing the next clause. This approach would consume a lot of memory for materializing the tables of intermediate results and would generally not perform well.

Instead, Cypher will in general try to interleave the execution of clauses. This is called lazy evaluation. It only materializes intermediate results when needed. In many read-write queries it is unproblematic to execute clauses interleaved, but when it is not, Cypher must ensure that the table of intermediate results gets materialized at the right time(s). This is done by inserting an `Eager` operator into the execution plan.

CALL

The `CALL` clause can be used to invoke a subquery or to call a procedure. For more information about either of these uses, refer to:

- CALL subqueries
- CALL subqueries in transactions
- Operations Manual → Call procedures

CREATE

The `CREATE` clause allows you to create nodes and relationships. To define these entities, `CREATE` uses a syntax similar to that of `MATCH`. However, while patterns only need to evaluate to either true or false, the syntax for `CREATE` needs to specify exactly what nodes and relationships to create.

Unlike `MERGE`, which first checks for a match and only creates anything if nothing is found, `CREATE` always creates the given pattern, regardless of whether matching patterns exist. For more information, see Comparing `CREATE` and `MERGE` below.

Create nodes

The `CREATE` clause allows you to create one or more nodes. Each node can be assigned labels and properties. You can bind each node to a variable that you can refer to later in the query. Multiple labels are separated by colons.

Query

```
CREATE (charlie:Person:Actor {name: 'Charlie Sheen'}), (oliver:Person:Director {name: 'Oliver Stone'})
```

Multiple labels can also be separated by an ampersand `&`, in the same manner as it is used in label expressions. Separation by colon `:` and ampersand `&` cannot be mixed in the same clause.

Query

```
CREATE (charlie:Person&Actor {name: 'Charlie Sheen'}), (oliver:Person&Director {name: 'Oliver Stone'})
```

Both of the above queries create two nodes, bound to the variables `charlie` and `oliver`, each with a

`Person` label and a `name` property. The node representing Charlie Sheen also has the label `Actor` while the node representing Oliver Stone is assigned the label `Director`.

Create relationships

Relationships can also be created using the `CREATE` clause. Unlike nodes, relationships always need exactly one relationship type and a direction. Similar to nodes, relationships can be assigned properties and relationship types and be bound to variables.

Query

```
CREATE (charlie:Person:Actor {name: 'Charlie Sheen'})-[:ACTED_IN {role: 'Bud Fox'}]->(wallStreet:Movie
{title: 'Wall Street'})<-[:DIRECTED]-(oliver:Person:Director {name: 'Oliver Stone'})
```

This query creates the `Person` nodes for Charlie Sheen and Oliver Stone and the `Movie` node for Wall Street. It also created the relationships of the types `ACTED_IN` and `DIRECTED` between them.

Comparing `CREATE` and `MERGE`

`CREATE` and `MERGE` are both used to add nodes and relationships to the graph, but they differ in how they handle existing data.

`CREATE` unconditionally creates a new node or relationship, without checking whether a matching one already exists. Running the same `CREATE` statement multiple times will therefore produce duplicate entries.

Two identical `CREATE` statements will create two separate but identical entities

```
CREATE (:Person {name: "Alice"})
CREATE (:Person {name: "Alice"})
```

`CREATE` does not check for existing matches, so it will create duplicates unless a uniqueness constraint or key constraint prevents it. With a uniqueness constraint on `:Person(name)`, the second `CREATE` statement above would fail instead.

`MERGE` first attempts to match an existing pattern in the graph. If a match is found, it uses the existing node or relationship; if not, it creates a new one. This makes `MERGE` idempotent; running it repeatedly with the same pattern will not create duplicates.

Two identical `MERGE` statements will result in only one entity, as the second matches the first

```
MERGE (:Person {name: "John"})
MERGE (:Person {name: "John"})
```

When to use `CREATE` and `MERGE`

	`CREATE`	`MERGE`
Duplicates	Always creates new data.	Creates only if no match exists; safe for incremental imports and re-runs.

	`CREATE`	`MERGE`
Performance	Fast — no lookup required.	Requires a lookup per row; without an index, this means a full label scan for every row.
Best for	• Loading into a fresh database with data you're certain is clean. • IDs guaranteed unique, import won't be re-run.	• Incremental imports or re-runnable jobs. • Ensuring uniqueness when data may already exist.

Reusing variables

The previous example created a path between the specified nodes. Note, that these newly created nodes and relationships are not connected to what was previously in the graph. To connect them to already existing data, bind the desired nodes and relationships to variables. These variables can then be passed along to subsequent clauses in a query that target pre-existing elements in the graph.

Query

```
MATCH (charlie:Person {name: 'Charlie Sheen'}), (oliver:Person {name: 'Oliver Stone'})
CREATE (charlie)-[:ACTED_IN {role: 'Bud Fox'}]->(wallStreet:Movie {title: 'Wall Street'})<-[:DIRECTED]-
(oliver)
```

In this example, the `MATCH` clause finds the nodes Charlie Sheen and Oliver Stone and binds them to the `charlie` and `oliver` variables respectively. These variables are then passed along to the subsequent `CREATE` clause, which creates new relationships from the bound nodes.

You can also reuse variables from the same `CREATE`, both in the same or a later clause. This way, you can, for example, define constructs that are more complex than just a linear path.

Query

```
CREATE p = (charlie:Person:Actor {name: 'Charlie Sheen'})-[:ACTED_IN {role: 'Bud Fox'}]->(wallStreet:Movie
{title: 'Wall Street'})<-[:DIRECTED]-(oliver:Person:Director {name: 'Oliver Stone'}), (wallStreet)<-
[:ACTED_IN {role: 'Gordon Gekko'}]-(michael:Person:Actor {name: 'Michael Douglas'})
RETURN length(p)
```

Creates all three nodes for Charlie Sheen, Oliver Stone and Michael Douglas and connects them all to the node representing the Wall Street movie. It then returns the length of the path from Charlie Sheen to Oliver Stone.

Note that when repeating a node's variable, you may not add labels or properties to the repetition.

Query

```
MATCH (charlie:Person {name: 'Charlie Sheen'})
CREATE (charlie:Actor)
```

This query will fail because the variable `charlie` has already been bound to a pre-existing node, and therefore it cannot be reused to create a new node. If you intend to add a label, use the `SET` clause instead.

Reusing variables in properties

The value that can be assigned to a node's or a relationship's property can be defined by an expression.

Query

```
MATCH (person:Person)
  WHERE person.name IS NOT NULL
CREATE (anotherPerson:Person {name: person.name, age: $age})
```

This example created a `Person` node with the same name as another person and the age from a parameter called `age`.

Such an expression may not contain a reference to a variable that is defined in the same `CREATE` statement. This is to ensure that the value of a property is always clear.

Query

```
CREATE (charlie {score: oliver.score + 1}), (oliver {score: charlie.score + 1})
```

This query tries to create nodes such that Charlie's score is higher than Oliver's and vice versa, which is a contradiction. The query therefore fails.

Use parameters with `CREATE`

Create node with a parameter for the properties

You can also create a graph entity from a map. All the key/value pairs in the map will be set as properties on the created relationship or node. In this case we add a `Person` label to the node as well.

Parameters

```
{
  "props": {
    "name": "Andy",
    "position": "Developer"
  }
}
```

Query

```
CREATE (n:Person $props)
RETURN n
```

Result

n
`Node[2]{name:"Andy",position:"Developer"}`
Rows: 1 Nodes created: 1 Properties set: 2 Labels added: 1

Create multiple nodes with a parameter for their properties

By providing Cypher an array of maps, it will create a node for each map.

Parameters

```
{
  "props": [ {
    "name": "Andy",
    "position": "Developer"
  }, {
    "name": "Michael",
    "position": "Developer"
  } ]
}
```

Query

```
UNWIND $props AS map
CREATE (n)
SET n = map
```

Result

```
(empty result)
```

Rows: 0

Nodes created: 2

Properties set: 4

Create using dynamic node labels and relationship types

Node labels and relationship types can be referenced dynamically in expressions, parameters, and variables when creating nodes and relationships. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

Syntax for creating nodes and relationships dynamically

```
CREATE (n:$(<expr>))
CREATE ()-[r:$(<expr>)]->()
```

The expression must evaluate to a `STRING NOT NULL | LIST<STRING NOT NULL> NOT NULL` value. Using a `LIST<STRING>` with more than one item when creating a relationship using dynamic relationship types will fail. This is because a relationship can only have exactly one type.

Parameters

```
{
  "nodeLabels": ["Person", "Director"],
  "relType": "DIRECTED",
  "movies": ["Ladybird", "Little Women", "Barbie"]
}
```

Create nodes and relationships using dynamic node labels and relationship types

```
CREATE (greta:$($nodeLabels) {name: 'Greta Gerwig'})
WITH greta
UNWIND $movies AS movieTitle
CREATE (greta)-[rel:$($relType)]->(m:Movie {title: movieTitle})
RETURN greta.name AS name, labels(greta) AS labels, type(rel) AS relType, collect(m.title) AS movies
```

Result

name	labels	relType	movies
"Greta Gerwig"	["Person", "Director"]	"DIRECTED"	["Ladybird", "Little Women", "Barbie"]

Rows: 1

`INSERT` as a synonym of `CREATE`

`INSERT` can be used as a synonym to `CREATE` for creating nodes and relationships, and was introduced as part of Cypher's GQL conformance. However, `INSERT` requires that multiple labels are separated by an ampersand `&` and not by colon `:`. Additionally, `INSERT` does not support using dynamic node labels and relationship types.

Query

```
INSERT (tom:Person&Actor&Director {name: 'Tom Hanks'})
```

Creates a node, bound to the variable `tom`, with the labels `Person`, `Actor`, and `Director` and a `name` property.

`CREATE` and concurrent transactions

Checking whether a node or relationship already exists before running `CREATE` does not prevent duplicates under concurrent transactions: concurrent transactions can each perform the check, find no existing node or relationship, and proceed to create identical entities.

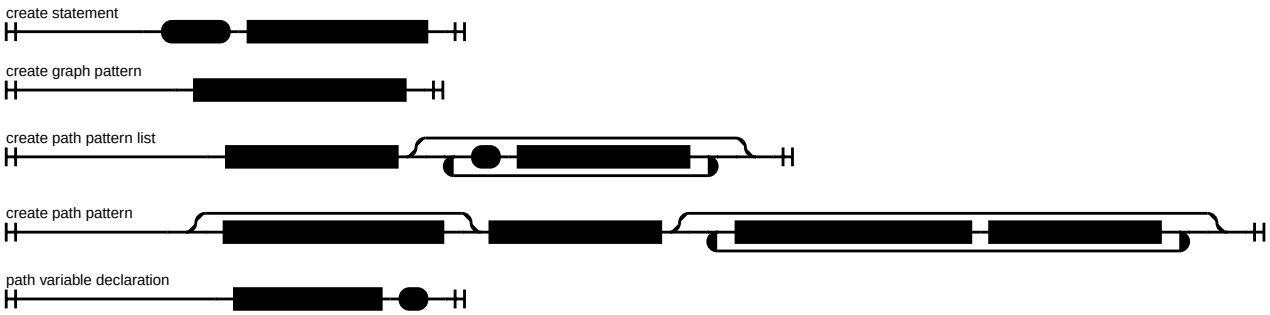
To reliably create an entity only if it does not already exist, use `MERGE` instead, combined with a uniqueness constraint on the merged property when merging nodes (this forces an index lock), or with both endpoint nodes bound by a preceding `MATCH` when merging relationships (forces a locking merge with locks taken on both endpoints).

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



DELETE

The `DELETE` clause is used to delete nodes, relationships or paths.

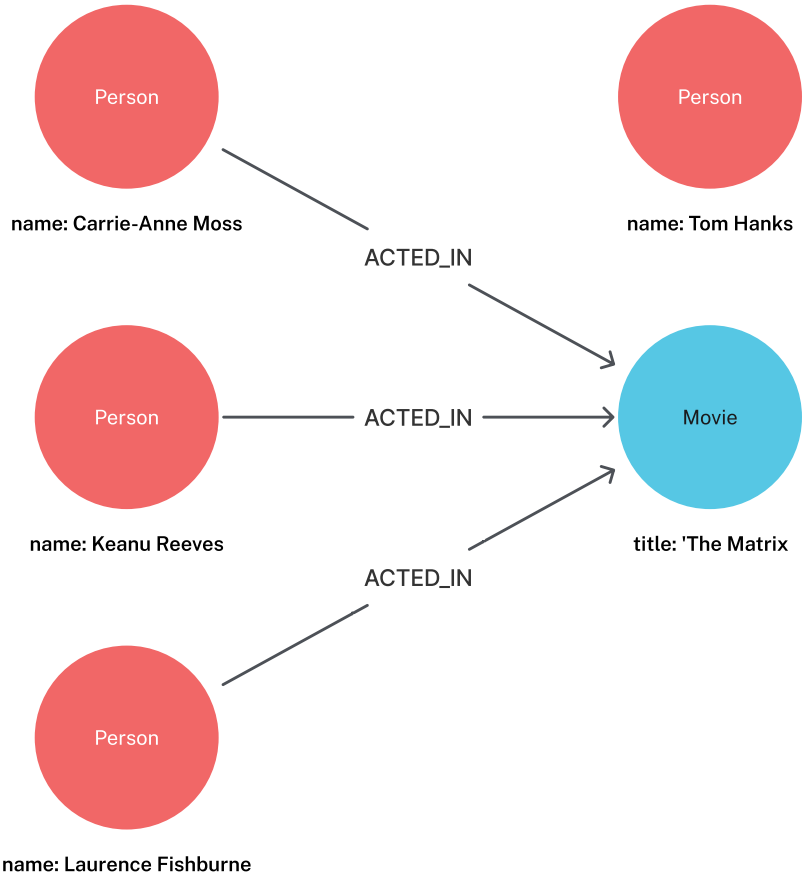
For removing properties and labels, see the REMOVE clause.

It is not possible to delete nodes with relationships connected to them without also deleting the relationships. This can be done by either explicitly deleting specific relationships, or by using the `DETACH DELETE` clause.

While the `DELETE` clause renders the deleted objects no longer accessible, the space occupied by the deleted nodes and relationships remain on the disk and is reserved for future transactions creating data. For information about how to clear and reuse the space occupied by deleted objects, see Operations Manual → Space reuse.

Example graph

The following graph is used for the examples below. It shows four actors, three of whom `ACTED_IN` the `Movie` `The Matrix` (`Keanu Reeves`, `Carrie-Anne Moss`, and `Laurence Fishburne`), and one actor who did not act in it (`Tom Hanks`).



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE
  (keanu:Person {name: 'Keanu Reever'}),
  (laurence:Person {name: 'Laurence Fishburne'}),
  (carrie:Person {name: 'Carrie-Anne Moss'}),
  (tom:Person {name: 'Tom Hanks'}),
  (theMatrix:Movie {title: 'The Matrix'}),
  (keanu)-[:ACTED_IN]->(theMatrix),
  (laurence)-[:ACTED_IN]->(theMatrix),
  (carrie)-[:ACTED_IN]->(theMatrix)
```

Delete single node

To delete a single node, use the `DELETE` clause:

Query

```
MATCH (n:Person {name: 'Tom Hanks'})
DELETE n
```

This deletes the `Person` node `Tom Hanks`. This query is only possible to run on nodes without any relationships connected to them.

Result

```
Deleted 1 node
```

NODETACH keyword

It is also possible to delete the single node using the `NODETACH DELETE` clause. Using the `NODETACH` keyword explicitly defines that relationships will not be detached and deleted from a node. The `NODETACH` keyword is a mirror of the already existing keyword DETACH, and it was introduced as part of Cypher's GQL conformance. Including it is functionally the same as using simple `DELETE`.

Query

```
MATCH (n:Person {name: 'Tom Hanks'})
NODETACH DELETE n
```

This also deletes the `Person` node `Tom Hanks`.

Delete relationships only

It is possible to delete a relationship while leaving the node(s) connected to that relationship otherwise unaffected.

Query

```
MATCH (n:Person {name: 'Laurence Fishburne'})-[r:ACTED_IN]->()
DELETE r
```

This deletes all outgoing `ACTED_IN` relationships from the `Person` node `Laurence Fishburne`, without deleting the node.

Result

```
Deleted 1 relationship
```

Delete a node with all its relationships

To delete nodes and any relationships connected them, use the `DETACH DELETE` clause.

Query

```
MATCH (n:Person {name: 'Carrie-Anne Moss'})
DETACH DELETE n
```

This deletes the `Person` node `Carrie-Anne Moss` and all relationships connected to it.

Result

```
Deleted 1 node, deleted 1 relationship
```

Note:

The `DETACH DELETE` clause may not be permitted to users with restricted security privileges. For more information, see Operations Manual → Fine-grained access control.

Delete all nodes and relationships

`DETACH DELETE` can be used to delete all nodes and relationships in a graph.

Delete all nodes and relationships

```
MATCH (n)
DETACH DELETE n
```

Result

```
Deleted 3 nodes, deleted 1 relationship
```

`DETACH DELETE` is useful when experimenting with small example datasets, but it is not suitable for deleting large amounts of data, nor does it delete indexes or any schema.

To delete large amounts of data without deleting indexes and any schema, use CALL subqueries in transactions instead.

Delete all nodes with a specific label and all relationships connected to them using `CALL` subqueries

```
MATCH (n:Person)
CALL (n) {
  DETACH DELETE n
} IN TRANSACTIONS
```

Delete all nodes and relationships using `CALL` subqueries

```
MATCH (n)
CALL (n) {
 DETACH DELETE n
} IN TRANSACTIONS
```

To remove all data, including indexes and constraints, you can either:

- Recreate the database using the `CREATE OR REPLACE DATABASE <database-name>` command. If you intend to recreate the existing indexes and constraints in the new database, first run `SHOW CONSTRAINTS` / `SHOW INDEXES` and return the specific `createStatement`.
- Delete a database using the `DROP DATABASE <database-name>` command, which deletes everything in a database, includes constraints and indexes, without recreating a new one.

Show the `createStatement` for constraints

```
SHOW CONSTRAINTS YIELD name, createStatement
```

Result

```
+-----------------------------------------------------------------------------------------------------+
| name              | createStatement                                                                 |
+-----------------------------------------------------------------------------------------------------+
| "person_name_key" | "CREATE CONSTRAINT `person_name_key` FOR (n:`Person`) REQUIRE (n.`name`) IS KEY" |
+-----------------------------------------------------------------------------------------------------+
```

Delete a database and recreate it

```
CREATE OR REPLACE DATABASE actors
```

Delete a database

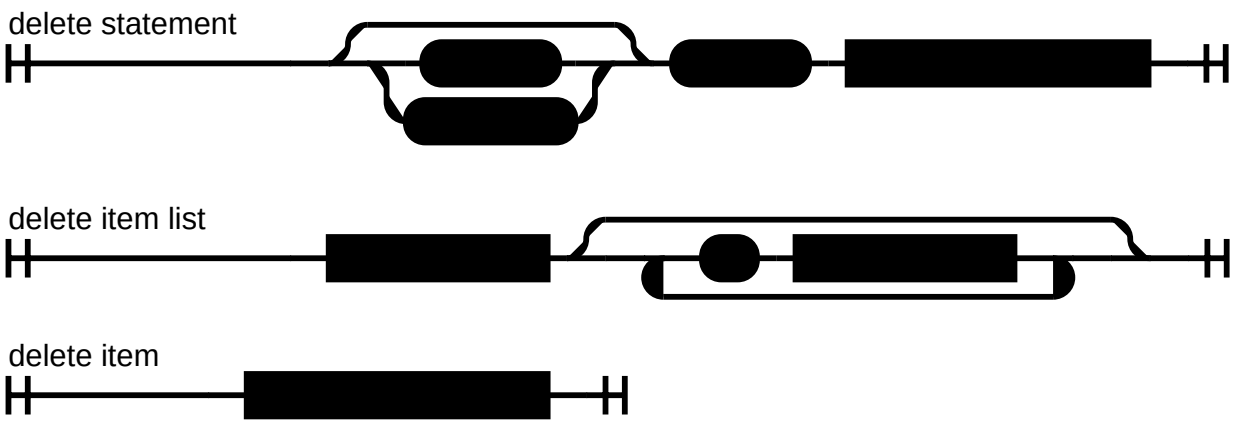
```
DROP DATABASE actors IF EXISTS
```

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



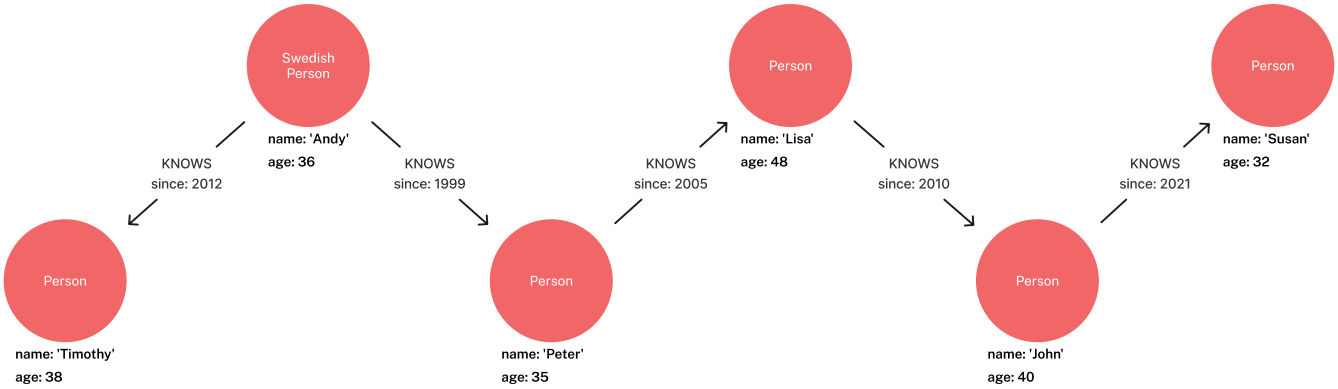
FILTER

Introduced in Neo4j 2025.06 | Cypher 25 only

`FILTER` is used to add filters to queries, similar to Cypher's `WHERE`. Unlike `WHERE`, `FILTER` is not a subclause, which means it can be used independently of the `MATCH`, `OPTIONAL MATCH`, and `WITH` clauses, but not within them.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (andy:Swedish:Person {name: 'Andy', age: 36}),
       (timothy:Person {name: 'Timothy', age: 38}),
       (peter:Person {name: 'Peter', age: 35}),
       (lisa:Person {name: 'Lisa', age: 48}),
       (john:Person {name: 'John', age: 40}),
       (susan:Person {name: 'Susan', age: 32}),
       (andy)-[:KNOWS {since: 2012}]->(timothy),
       (andy)-[:KNOWS {since: 1999}]->(peter),
       (peter)-[:KNOWS {since: 2005}]->(lisa),
       (lisa)-[:KNOWS {since: 2010}]->(john),
       (john)-[:KNOWS {since: 2021}]->(susan)
```

Basic filtering

Filter on a node label

```
MATCH (n)
FILTER n:Swedish
RETURN n.name AS name
```

Result

name
"Andy"
Rows: 1

Filter on a node property

```
MATCH (n:Person)
FILTER n.age < 35
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Susan"	32
Rows: 1	

Filter on a relationship property

```
MATCH (p:Person)-[r:KNOWS]->(n:Person)
FILTER r.since > 2010
RETURN p.name AS person,
       r.since AS knowsSince,
       n.name AS otherPerson
```

Result

person	knowsSince	otherPerson
"Andy"	2012	"Timothy"
"John"	2021	"Susan"
Rows: 2		

Filter on dynamic labels and types

To filter on a label or relationship type using a dynamically computed value, use `$(<expr>)` where <expr> is any Cypher expression:

Parameters

```
{
  "label": "Swedish"
}
```

Filter on a dynamically computed node label

```
MATCH (n)
FILTER n:$($label)
RETURN labels(n) AS labels
```

Result

labels
["Person", "Swedish"]
Rows: 1

Filter on dynamic properties

To filter on a property using a dynamically computed name, use square brackets `[]`:

Parameters

```
{
  "propname": "age"
}
```

Filter on a dynamically computed node property

```
MATCH (n:Person)
```

```
FILTER n[$propname] > 40
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Lisa"	48
Rows: 1	

Differences between `FILTER` and `WHERE`

`FILTER` and `WHERE` are both used to apply filter to queries. However, there are a number of important differences between them that arise from the fact that `FILTER` is a clause and `WHERE` is a subclause:

- `FILTER` acts on entities *after* they have been matched, whereas `WHERE` constrains what rows get matched *before* the match is performed.
- `FILTER` cannot be used within `MATCH`, `OPTIONAL MATCH`, or `WITH` clauses but only alongside them. This means that, unlike `WHERE`, `FILTER` cannot be used to add filters inside patterns.

Example 13. Distinction between `FILTER` and `WHERE`

This `OPTIONAL MATCH` example highlights the differences between the `WHERE` subclause and the `FILTER` clause.

`WHERE` *constraining an* `OPTIONAL MATCH` *pattern*

```
UNWIND [32,37,40] AS ageValue
OPTIONAL MATCH (p:Person)
WHERE p.age = ageValue
RETURN p.name AS name, p.age AS age
```

Result

name	age
"Susan"	32
NULL	NULL
"John"	40
Rows: 3	

Because `WHERE` is a subclause belonging to `OPTIONAL MATCH`, it only filters the matches, but does not affect the number of returned rows. In this case, `OPTIONAL MATCH` always keeps all rows from `UNWIND`, and `WHERE` does not remove any rows returning `NULL`. The same is not true if `WHERE` is exchanged for `FILTER`:

`FILTER` *adding post-filtering to* `OPTIONAL MATCH`

```
UNWIND [32,37,40] AS ageValue
OPTIONAL MATCH (p:Person)
FILTER p.age = ageValue
```

```
RETURN p.name AS name, p.age AS age
```

Result

name	age
"Susan"	32
"John"	40
Rows: 2	

Unlike `WHERE`, `FILTER` is not part of the `OPTIONAL MATCH` and so removes entire rows from the result set based on the condition provided within the expression. That is, when `OPTIONAL MATCH` fails to find a match and `p` is `NULL`, `FILTER p.age = ageValue` cannot be evaluated, causing the entire row to be removed.

Example 14. `FILTER` cannot be used within patterns

Because `WHERE` is a subclause qualifying a described pattern, it can be used inside patterns.

`WHERE` *inside a node pattern*

```
WITH 35 AS minAge
MATCH (a:Person WHERE a.name = 'Andy')-[:KNOWS]->(b:Person WHERE b.age > minAge)
RETURN b.name AS name
```

Result

name
"Timothy"
Rows: 1

The same is not true for `FILTER` which, as a clause, cannot be placed with a pattern.

Not allowed — `FILTER` inside a pattern

```
WITH 35 AS minAge
MATCH (a:Person FILTER a.name = 'Andy')-[:KNOWS]->(b:Person FILTER b.age > minAge)
RETURN b.name AS name
```

For more information about how to use `WHERE` in fixed-length and variable-length pattern matching, see `WHERE` → Filter patterns.

`FILTER` as a substitute for `WITH * WHERE`

Unlike `WHERE`, which relies on `MATCH`, `OPTIONAL MATCH`, or `WITH` to define its scope, `FILTER` can filter queries independently of these clauses. This can make some queries more concise.

For example, the following two queries are equivalent:

Filter using `WITH * WHERE`

```
UNWIND [1, 2, 3, 4, 5, 6] AS x
WITH x
WHERE x > 2
RETURN x
```

Filter using `FILTER`

```
UNWIND [1, 2, 3, 4, 5, 6] AS x
FILTER x > 2
RETURN x
```

As such, `FILTER` can be seen as a substitute for the `WITH * WHERE <predicate>` constructs in Cypher.

Example 15. Using `FILTER` *instead of* `WITH * WHERE` *in* `LOAD CSV`

The following two `LOAD CSV` commands are equivalent:

companies.csv

```
Id,Name,Location,Email,BusinessType
1,Neo4j,San Mateo,contact@neo4j.com,P
2,AAA,,info@aaa.com,
3,BBB,Chicago, info@ ,G
,CCC,Michigan,info@ccc.com,G
```

`LOAD CSV` *using* `WITH * WHERE`

```
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
WITH row
WHERE row.Id IS NOT NULL
MERGE (c:Company {id: row.Id})
```

`LOAD CSV` *using* `FILTER`

```
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
FILTER row.Id IS NOT NULL
MERGE (c:Company {id: row.Id})
```

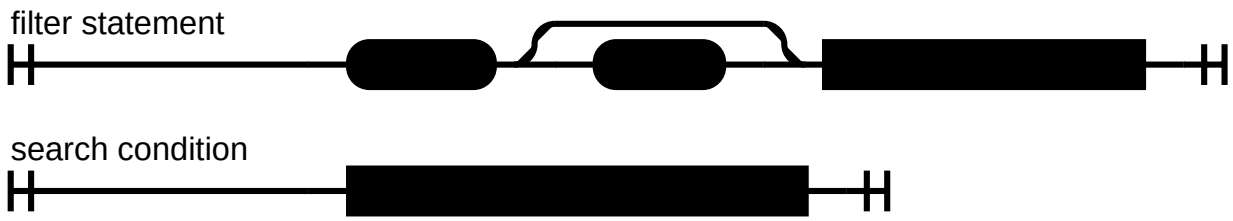
However, while `FILTER` can act as a substitute for `WITH * WHERE <predicate>` constructs, it does not include the ability of `WITH` to manipulate the variables in scope for subsequent clauses. Nor can `FILTER` alias or create new variables. In other words, `FILTER` only has the function of `WITH * WHERE <predicate>` and not `WITH <selectedVariable> AS <newVariableName> WHERE <predicate>`.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



FINISH

A query ending in `FINISH` — instead of `RETURN` — has no result but executes all its side effects. `FINISH` was introduced as part of Cypher's GQL conformance.

The following read query successfully executes but has no results:

Query

```
MATCH (p:Person)
FINISH
```

The following query has no result but creates one node with the label `Person`:

Query

```
CREATE (p:Person)
FINISH
```

It is equivalent to the following query:

Query

```
CREATE (p:Person)
```

FOR

The `FOR` clause makes it possible to transform any list back into individual rows. The list can be a parameter that is passed in, a previously collected result, or any other list expression.

`FOR` is the GQL conformant synonym of `UNWIND`. `FOR` is valid wherever `UNWIND` is valid and has the same semantics. `FOR` and `UNWIND` differ only in syntax.

```
FOR variable IN expression
```

For comparison, the Cypher `UNWIND` form is:

```
UNWIND expression AS variable
```

> **Note:**

Neo4j does not guarantee the row order produced by `FOR`. The only clause that guarantees a specific row order is ORDER BY.

Unwinding a list

We want to transform the literal list into rows named `x` and return them.

Query

```
FOR x IN [1, 2, 3, null]
RETURN x, 'val' AS y
```

Each value of the original list — including `null` — is returned as an individual row. The same outcome is shown here using the equivalent `UNWIND` form:

```
UNWIND [1, 2, 3, null] AS x
RETURN x, 'val' AS y
```

Result

x	y
1	"val"
2	"val"
3	"val"
<null>	"val"
Rows: 4	

Further behavior

All cases described for `UNWIND` apply to `FOR`: empty lists, nested lists, expressions that are not lists, parameters, and combinations with `WITH`, `RETURN`, subqueries, and so on.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

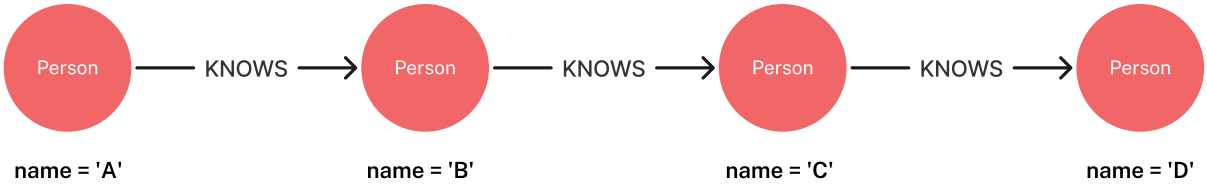
FOREACH

Lists and paths are key concepts in Cypher. The `FOREACH` clause can be used to update data, such as executing update commands on elements in a path, or on a list created by aggregation.

The variable context within the `FOREACH` parenthesis is separate from the one outside it. This means that if you `CREATE` a node variable within a `FOREACH`, you will *not* be able to use it outside of the foreach statement, unless you match to find it.

Within the `FOREACH` parentheses, you can do any of the updating commands — `SET`, `REMOVE`, `CREATE`, `MERGE`, `DELETE`, and `FOREACH`.

If you want to execute an additional `MATCH` for each element in a list then the `UNWIND` clause would be a more appropriate command.



Mark all nodes along a path

This query sets the property `marked` to `true` on all nodes along a path.

Query

```
MATCH p=(start)-[*]->(finish)
WHERE start.name = 'A' AND finish.name = 'D'
FOREACH (n IN nodes(p) | SET n.marked = true)
```

Result

```
(empty result)
```

Rows: 0

Properties set: 4

Mark all relationships along a path

This query sets the property `marked` to `true` on all relationships along a path.

```
MATCH p=(start)-[*]->(finish)
WHERE start.name = 'A' AND finish.name = 'D'
FOREACH ( r IN relationships(p) | SET r.marked = true )
```

Result

```
(empty result)
```

Rows: 0

Properties set: 3

Create new nodes from a list of name labels

This query creates a new node for each label in a list.

Query

```
WITH ['E', 'F', 'G'] AS names
FOREACH ( value IN names | CREATE (:Person {name: value}) )
```

Result

```
(empty result)
```

Rows: 0

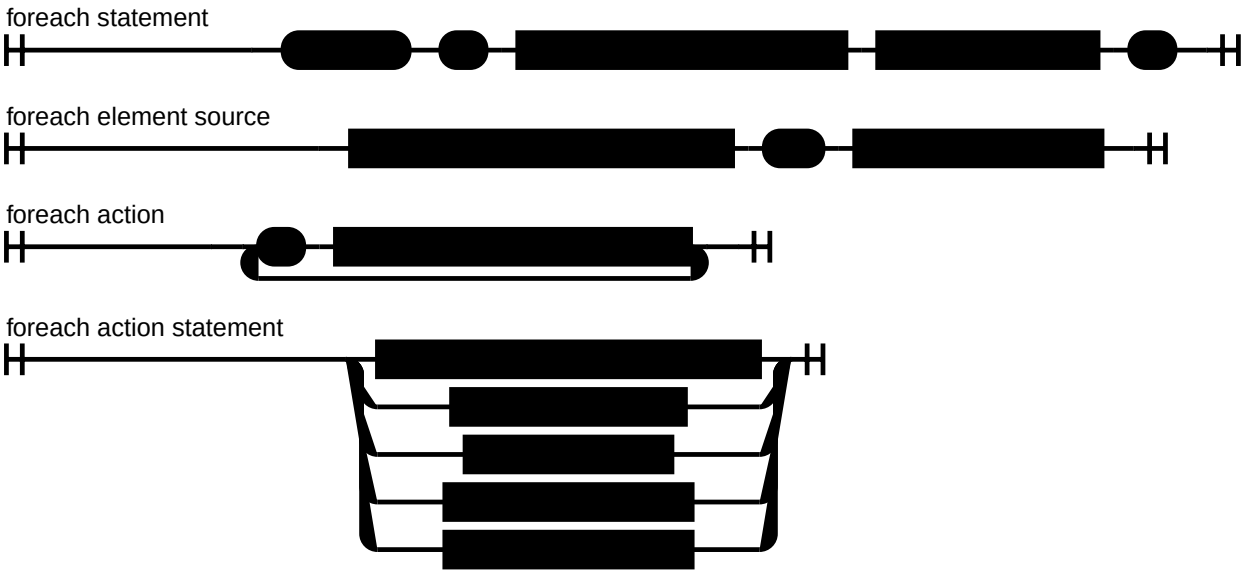
Nodes created: 3

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



GROUP BY

`GROUP BY` is a subclause that gives explicit control over the grouping keys of a `RETURN` or `WITH` clause. If no `GROUP BY` clause is present, Cypher uses implicit grouping: If you use an aggregation function, all non-aggregated columns automatically become grouping keys.

Cypher's `GROUP BY` is similar to SQL's `GROUP BY`. It arranges identically structured data into groups based on one or more grouping keys. For this reason, it combines well with aggregate functions like count() or avg() to perform calculations on each group of data.

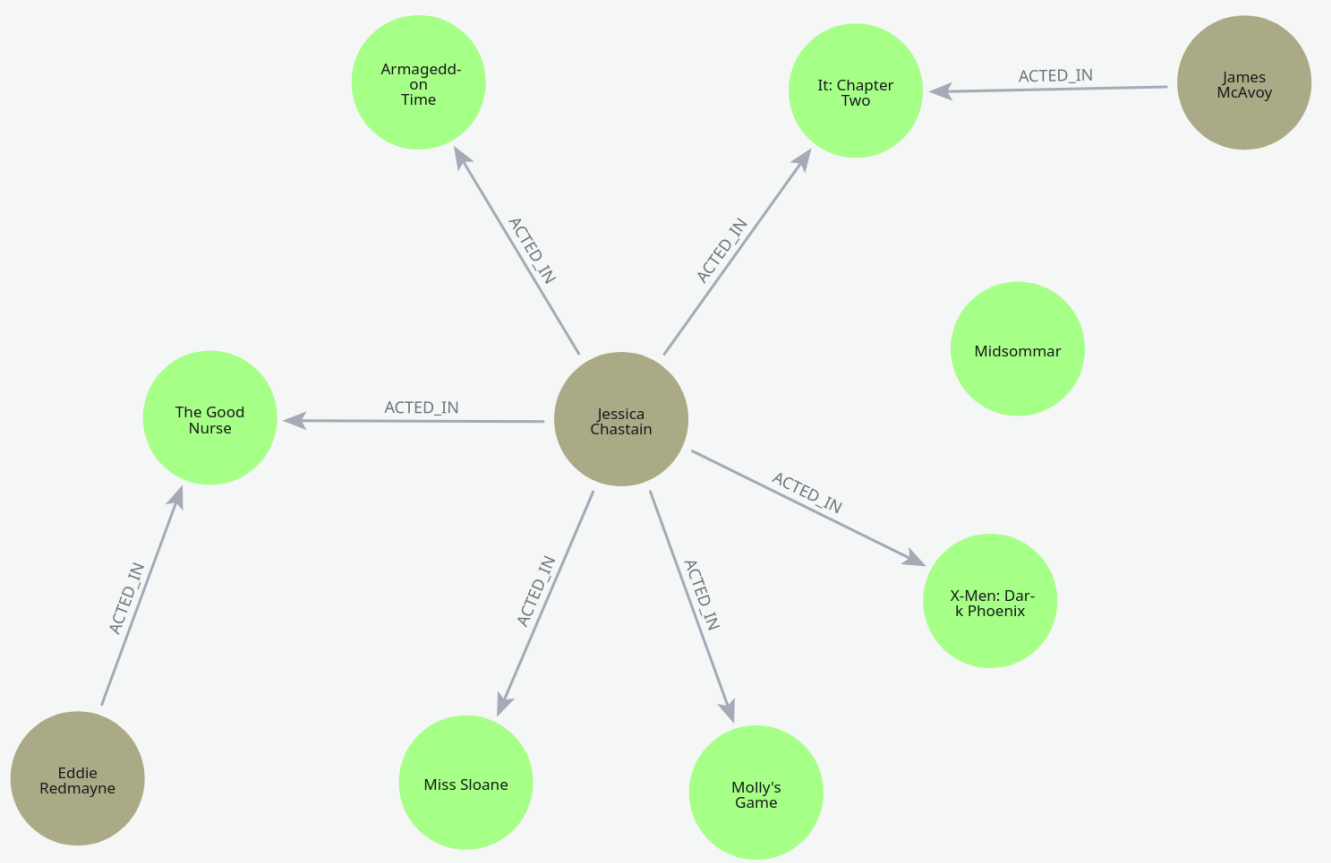
Grouping keys

The following rules apply to grouping keys:

- A single grouping key or multiple grouping keys are possible
- The set of grouping keys may be empty (`GROUP BY ()`)
- `GROUP BY ALL` makes every non-aggregating return item a grouping key
- If a projection clause contains a `GROUP BY` clause, then every projection item must be:
  - A leaf, that is:
    - An aggregation function like `sum(x)`
    - A constant like `0.1`
    - A parameter like `$param`
    - A local variable like the `x` in `count(*) + size([x IN range(1,10) | x ])`
    - A grouping key expression, that is a projected expression that doesn't contain an aggregation function
  - A sub-expression, where all operands are either a leaf or a sub-expression
  - An exact grouping key, equal to a grouping key expression
- Grouping keys which are not also return items are not projected to the output

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (jessica:Person {name: 'Jessica Chastain'}),
  (james:Person {name: 'James McAvoy'}),
  (eddie:Person {name: 'Eddie Redmayne'}),
  (missSloane:Movie {title: 'Miss Sloane', genre: 'Drama', rating: 4.5, year: 2016}),
  (mollysGame:Movie {title: "Molly's Game", genre: 'Drama', rating: 4.4, year: 2017}),
  (darkPhoenix:Movie {title: 'X-Men: Dark Phoenix', genre: 'Action', rating: 3.5, year: 2019}),
  (it2:Movie {title: 'It: Chapter Two', genre: 'Horror', rating: 3.6, year: 2019}),
  (midsommar:Movie {title: 'Midsommar', genre: 'Horror', rating: 3.6, year: 2019}),
  (armageddon:Movie {title: 'Armageddon Time', genre: 'Drama', rating: 3.8, year: 2022}),
  (goodnurse:Movie {title: 'The Good Nurse', genre: 'Drama', rating: 4.2, year: 2022}),
  (jessica)-[:ACTED_IN {role: 'Madeline Elizabeth Sloane'}]->(missSloane),
  (jessica)-[:ACTED_IN {role: 'Molly Bloom'}]->(mollysGame),
  (jessica)-[:ACTED_IN {role: 'Vuk'}]->(darkPhoenix),
  (jessica)-[:ACTED_IN {role: 'Beverly Marsh'}]->(it2),
  (jessica)-[:ACTED_IN {role: 'Maryanne Trump'}]->(armageddon),
  (jessica)-[:ACTED_IN {role: 'Amy Loughren'}]->(goodnurse),
  (james)-[:ACTED_IN {role: 'Bill Denbrough'}]->(it2),
  (eddie)-[:ACTED_IN {role: 'CHarlie Cullen'}]->(goodnurse)
```

`GROUP BY` clause template

Template for `GROUP BY` in a `RETURN` clause

```
RETURN <return statement body>
  GROUP BY { <grouping element> {, <grouping element> } … | () | ALL }
  <order by and page clause>
```

Template for `GROUP BY` in a `WITH` clause

```
WITH <return statement body>
  GROUP BY { <grouping element> {, <grouping element> } … | () | ALL }
  [<order by and page clause>]
  [<where clause>]
```

Basic examples

Example 16. `GROUP BY` with a single grouping key

```
MATCH (m:Movie)
RETURN m.genre AS genre, avg(m.rating) AS averageRating
  GROUP BY genre
```

Result

genre	averageRating
"Drama"	4.225
"Action"	3.5
"Horror"	3.6

Rows: 3

The movies are grouped by genre and the average rating for each genre is calculated. The four drama

movies have an average rating of `4.225`.

Example 17. `GROUP BY` *with two grouping keys*

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor_name, m.genre AS genre, avg(m.rating) AS averageRating
GROUP BY actor_name, genre
```

Result

actor_name	genre	averageRating
"Jessica Chastain"	"Drama"	4.225
"Jessica Chastain"	"Action"	3.5
"Jessica Chastain"	"Horror"	3.6
"James McAvoy"	"Horror"	3.6
"Eddie Redmayne"	"Drama"	4.2

Rows: 5

Grouping by actor and genre, the drama and horror movies have two entries each.

Example 18. `GROUP BY` *with no grouping keys*

```
MATCH (m:Movie)
RETURN collect(m.title) AS movieTitles, avg(m.rating) AS globalAverageRating
  GROUP BY ()
```

Result

movieTitles	globalAverageRating
["Miss Sloane", "Molly's Game", "X-Men: Dark Phoenix", "It: Chapter Two", "Midsommar", "Armageddon Time", "The Good Nurse"]	3.9428571428571426

Rows: 1

When using `GROUP BY` with `()`, you explicitly state that there are no grouping keys. This means that there can be no direct projections of returned entities. However, aggregations still work like in the showcased query.

Example 19. `GROUP BY ALL`

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor_name, m.genre AS genre, m.year AS year, avg(m.rating) AS averageRating
GROUP BY ALL
```

Result

actor_name	genre	year	averageRating
"Jessica Chastain"	"Drama"	2016	4.5
"Jessica Chastain"	"Drama"	2017	4.4
"Jessica Chastain"	"Action"	2019	3.5
"Jessica Chastain"	"Horror"	2019	3.6
"Jessica Chastain"	"Drama"	2022	4.0
"James McAvoy"	"Horror"	2019	3.6
"Eddie Redmayne"	"Drama"	2022	4.2

Rows: 7

`GROUP BY ALL` uses all projected expressions as grouping keys. In this case, only the two drama movies from 2022 starring Jessica Chastain actually calculate an average from two values.

GROUP BY and ORDER BY

`GROUP BY` and `ORDER BY` can be used in the same query. `GROUP BY` takes precedence and the query result can then be ordered by a subsequent `ORDER BY` clause.

Example 20. `GROUP BY` and `ORDER BY`

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor_name, m.genre AS genre, avg(m.rating) AS averageRating
GROUP BY actor_name, genre
ORDER BY genre
```

Result

actor_name	genre	averageRating
"Jessica Chastain"	"Action"	3.5
"Jessica Chastain"	"Drama"	4.225
"Eddie Redmayne"	"Drama"	4.2
"Jessica Chastain"	"Horror"	3.6
"James McAvoy"	"Horror"	3.6

Rows: 5

The results are ordered by genre after being grouped.

Note:

Neo4j 2026.07 features changes to the visibility of expressions in `WHERE` and `ORDER BY` clauses when using `GROUP BY`:

Variables named with `AS` in `RETURN` and `WITH` clauses are called *projected aliases*. Projected aliases make variables defined prior to `RETURN` or `WITH` inaccessible if they share the same name.

Special cases

There are some special cases for `GROUP BY`:

- `GROUP BY` can be used without aggregation.
- You can group by both aliased and unaliased return items.
- You can group by an item that is not returned.

Example 21. `GROUP BY` without aggregation

```
MATCH (m: Movie)
RETURN m.year AS year, m.rating AS rating
  GROUP BY year, rating
```

Result

year	rating
2016	4.5
2017	4.4
2019	3.5
2019	3.6
2022	3.8
2022	4.2

Rows: 6

In this query, `GROUP BY` is equivalent to using the `DISTINCT` keyword. Two movies with a rating of `3.6` exist for the year 2019, but the entry is not repeated.

Example 22. `GROUP BY` and aliasing

```
MATCH (m: Movie)
LET year = m.year
RETURN year, m.rating AS rating
  GROUP BY year, rating
```

Result

year	rating
2016	4.5
2017	4.4
2019	3.5
2019	3.6
2022	3.8
2022	4.2
Rows: 6	

You can group by aliased and unaliased return items (`year` is not aliased, `rating` is aliased).

Example 23. `GROUP BY` an item that is not returned

```
MATCH (m: Movie)
LET title = m.title
RETURN m.year AS year, m.rating AS rating
  GROUP BY title, year, rating
```

Result

year	rating
2016	4.5
2017	4.4
2019	3.5
2019	3.6
2019	3.6
2022	3.8
2022	4.2
Rows: 7	

`title` is not returned, but still used as a grouping key. Without it, the result for 2019 with a rating of `3.6` would not be repeated.

Errors

Example 24. Missing grouping element in `GROUP BY`

```
MATCH (m:Movie)
LET title = m.title
RETURN m.rating AS rating, avg(m.rating) AS avg_rating
```

```
GROUP BY title
```

GQLSTATUS error chain

42I18: error: syntax error or access rule violation - reference to non-grouping sub-expression. The expression contains a non-grouping sub-expression `m`. In an aggregating context only grouping sub-expressions and constants are allowed.

42001: error: syntax error or access rule violation - invalid syntax

`m` in `m.rating AS rating` is not a grouping key since it is not listed in the `GROUP BY` clause.

For more examples of aggregation errors, see Aggregation expressions and grouping keys → Examples.

Example 25. Inaccessible variables

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p AS person
GROUP BY person
ORDER BY m.year;
```

GQLSTATUS error chain

42N44: error: syntax error or access rule violation - inaccessible variable. It is not possible to access the variable `m` declared before the RETURN clause when using `DISTINCT`, an aggregation, or a `GROUP BY` clause.

42001: error: syntax error or access rule violation - invalid syntax

`ORDER BY` cannot access `m` because it has been declared before the `RETURN` clause and the query contains a `GROUP BY` clause.

For another example, see the last error described in Ordering aggregated or DISTINCT results.

Example 26. `GROUP BY` with ambiguous references in grouping elements

```
MATCH (m)
RETURN m AS a
GROUP BY a.year
```

GQLSTATUS error chain

42I80: error: syntax error or access rule violation - invalid grouping element. The grouping element 'a.year' is not a valid grouping key. A grouping element that references the projection item alias `a` must be a simple variable reference.

42001: error: syntax error or access rule violation - invalid syntax

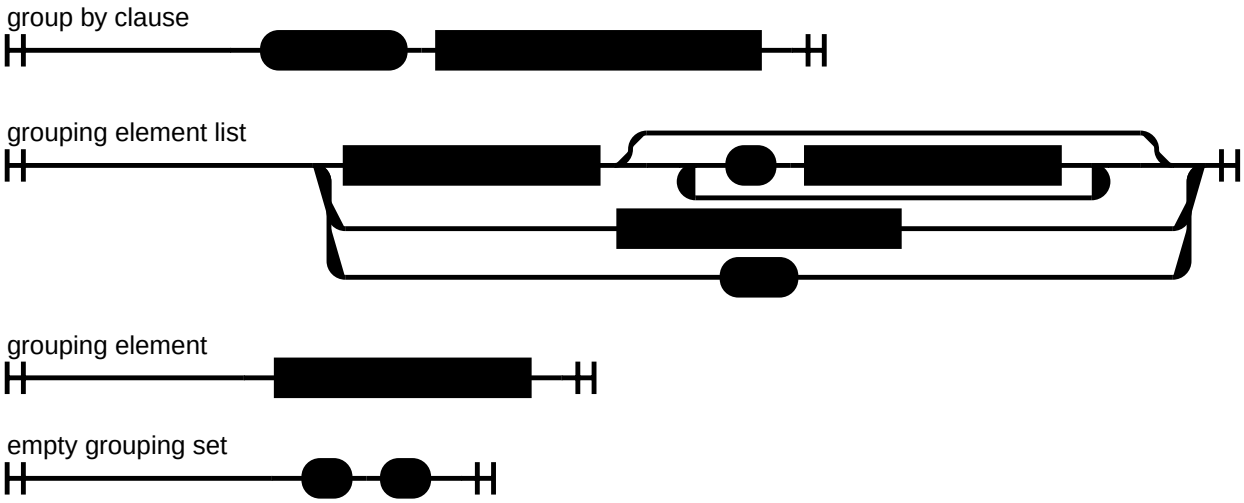
`a.year` cannot be used as a grouping key because the property access cannot be resolved.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



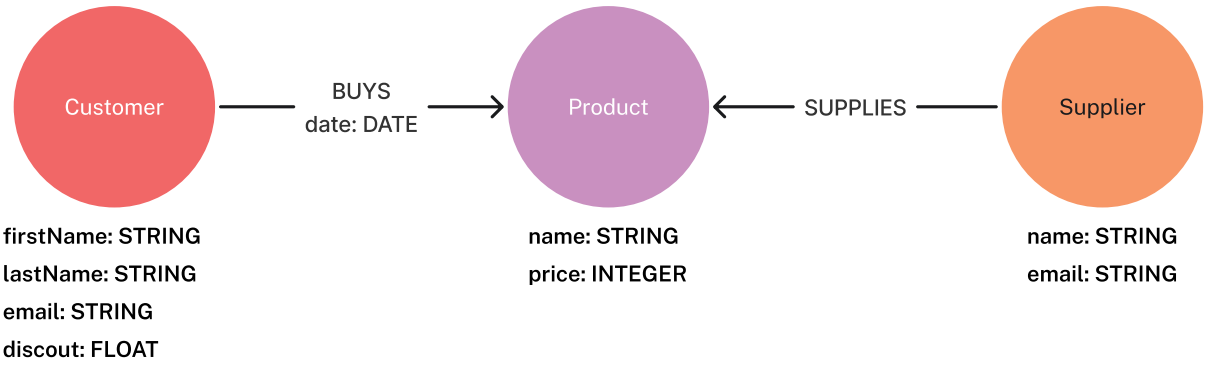
LET

Introduced in Neo4j 2025.06 | Cypher 25 only

`LET` binds expressions to variables. For queries involving several chained expressions, it can be a more succinct and readable alternative to `WITH`. Unlike `WITH`, `LET` does not drop variables from the scope of subsequent clauses. Nor can it be used for aggregations or in combination with `DISTINCT`; it can only be used to bind new variables.

Example graph

A graph with the following schema is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database.

```
CREATE (techCorp:Supplier {name: 'TechCorp', email: 'contact@techcorp.com'}),
       (foodies:Supplier {name: 'Foodies Inc.', email: 'info@foodies.com'}),
```

```
       (laptop:Product {name: 'Laptop', price: 1000}),
       (phone:Product {name: 'Phone', price: 500}),
       (headphones:Product {name: 'Headphones', price: 250}),
       (chocolate:Product {name: 'Chocolate', price: 5}),
       (coffee:Product {name: 'Coffee', price: 10}),

       (amir:Customer {firstName: 'Amir', lastName: 'Rahman', email: 'amir.rahman@example.com', discount:
0.1}),
       (keisha:Customer {firstName: 'Keisha', lastName: 'Nguyen', email: 'keisha.nguyen@example.com',
discount: 0.2}),
       (mateo:Customer {firstName: 'Mateo', lastName: 'Ortega', email: 'mateo.ortega@example.com',
discount: 0.05}),
       (hannah:Customer {firstName: 'Hannah', lastName: 'Connor', email: 'hannah.connor@example.com',
discount: 0.15}),
       (leila:Customer {firstName: 'Leila', lastName: 'Haddad', email: 'leila.haddad@example.com',
discount: 0.1}),
       (niko:Customer {firstName: 'Niko', lastName: 'Petrov', email: 'niko.petrov@example.com', discount:
0.25}),
       (yusuf:Customer {firstName: 'Yusuf', lastName: 'Abdi', email: 'yusuf.abdi@example.com', discount:
0.1}),

       (amir)-[:BUYS {date: date('2024-10-09')}]->(laptop),
       (amir)-[:BUYS {date: date('2025-01-10')}]->(chocolate),
       (keisha)-[:BUYS {date: date('2023-07-09')}]->(headphones),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(chocolate),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(coffee),
       (mateo)-[:BUYS {date: date('2024-04-11')}]->(laptop),
       (hannah)-[:BUYS {date: date('2023-12-11')}]->(coffee),
       (hannah)-[:BUYS {date: date('2024-06-02')}]->(headphones),
       (leila)-[:BUYS {date: date('2023-05-17')}]->(laptop),
       (niko)-[:BUYS {date: date('2025-02-27')}]->(phone),
       (niko)-[:BUYS {date: date('2024-08-23')}]->(headphones),
       (niko)-[:BUYS {date: date('2024-12-24')}]->(coffee),
       (yusuf)-[:BUYS {date: date('2024-12-24')}]->(chocolate),
       (yusuf)-[:BUYS {date: date('2025-01-02')}]->(laptop),

       (techCorp)-[:SUPPLIES]->(laptop),
       (techCorp)-[:SUPPLIES]->(phone),
       (techCorp)-[:SUPPLIES]->(headphones),
       (foodies)-[:SUPPLIES]->(chocolate),
       (foodies)-[:SUPPLIES]->(coffee)
```

Bind values to variables

`LET` is used to bind variables to the results of expressions.

Syntax

```
LET variable = expression, variable = expression
```

Using `LET` to bind a variable

```
MATCH (c:Customer)
LET fullName = c.firstName + ' ' + c.lastName
RETURN fullName
```

Result

fullName
"Amir Rahman"
"Keisha Nguyen"
"Mateo Ortega"

fullName
"Hannah Connor"
"Leila Haddad"
"Niko Petrov"
"Yusuf Abdi"
Rows: 7

Using `LET` to bind several variables

```
MATCH (s:Supplier)-[:SUPPLIES]->(p:Product)
LET supplier = s.name, product = p.name
RETURN supplier, product
```

Result

supplier	product
"TechCorp"	"Laptop"
"TechCorp"	"Phone"
"TechCorp"	"Headphones"
"Foodies Inc."	"Chocolate"
"Foodies Inc."	"Coffee"
Rows: 4	

Differences between `LET` and `WITH`

There are important differences between `LET` and `WITH` that can be divided into the following categories:

- Variables in scope
- Chaining expressions
- Aggregations and `DISTINCT`

Variables in scope

`LET` does not drop variables from the scope of subsequent clauses, while `WITH` does. As such, `LET <variable> = <expression>` is a substitute for `WITH *, <expression> AS <variable>`, not `WITH <expression> AS <variable>` (which would drop any variables present in the preceding clause not referenced in `<expression>`).

Example 27. Variables in scope: comparing `LET` and `WITH`

Any variable not explicitly referenced by `WITH` (or carried over by `WITH *`) is dropped from the scope of subsequent clauses.

Not allowed — Referencing a variable dropped by `WITH`

```
MATCH (s:Supplier)-[:SUPPLIES]->(p:Product)
WITH s.name AS supplier
RETURN supplier, p.name AS product
```

GQLSTATUS error chain

```
42N62: error: syntax error or access rule violation - variable not defined. Variable p not defined.


42001: error: syntax error or access rule violation - invalid syntax
```

`LET`, however, cannot regulate which variables are in scope. Replacing `WITH` with `LET` in the above query would, therefore, return results.

`LET` does not drop variables

```
MATCH (s:Supplier)-[:SUPPLIES]->(p:Product)
LET supplier =  s.name
RETURN supplier, p.name AS product
```

Result

supplier	product
"TechCorp"	"Laptop"
"TechCorp"	"Phone"
"TechCorp"	"Headphones"
"Foodies Inc."	"Chocolate"
"Foodies Inc."	"Coffee"

Rows: 5

Chaining expressions

The fact that `LET` does not drop variables means that it can be used to chain expressions in a clear and concise manner, where variables bound in one `LET` clause can be referenced by subsequent clauses.

Example 28. Chaining expressions: comparing `LET` and `WITH`

The below query shows that variables bound by a `LET` clause can be referenced by subsequent clauses without being explicitly carried over. Specifically, the variable `isExpensive` is created in the first `LET` clause and referenced again in the subsequent clauses. Note also that the variable `p`, bound in the `MATCH` clause, is available in the final `RETURN` clause despite not being referenced in any of `LET` clauses.

`LET` referencing variables assigned in previous a `LET`

```
MATCH (p:Product)
LET isExpensive = p.price >= 500
LET isAffordable = NOT isExpensive
```

```
LET discountCategory = CASE
    WHEN isExpensive THEN 'High-end'
    ELSE 'Budget'
END
RETURN p.name AS product, p.price AS price, isAffordable, discountCategory
ORDER BY price
```

Result

product	price	isAffordable	discountCategory
"Chocolate"	5	TRUE	'Budget'
"Coffee"	10	TRUE	'Budget'
"Headphones"	250	TRUE	'Budget'
"Phone"	500	FALSE	'High-end'
"Laptop"	1000	FALSE	'High-end'

Rows: 5

Using `WITH`, the same query would become less succinct, as `WITH` would have to explicitly carry over each variable in between clauses:

Less succinct `WITH` equivalent

```
MATCH (p:Product)
WITH p, p.price >= 500 AS isExpensive
WITH p, isExpensive, NOT isExpensive AS isAffordable
WITH p, isExpensive, isAffordable,
     CASE
         WHEN isExpensive THEN 'High-end'
         ELSE 'Budget'
     END AS discountCategory
RETURN p.name AS product, p.price AS price, isAffordable, discountCategory
ORDER BY price
```

Aggregations and `DISTINCT`

Unlike `WITH`, `LET` cannot perform aggregations or be combined with `DISTINCT`. For example, in the following query, `WITH` could not be replaced by `LET`:

Combining `WITH DISTINCT` and aggregations on expressions

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH DISTINCT c, sum(p.price) AS totalSpent
RETURN c.firstName AS customer, totalSpent
```

Result

customer	totalSpent
"Amir"	1005
"Keisha"	250
"Mateo"	1015

customer	totalSpent
"Hannah"	260
"Leila"	1000
"Niko"	760
"Yusuf"	1005
Rows: 7	

Combining `WITH` and `LET`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH DISTINCT c, sum(p.price) AS totalSpent
LET fullName = c.firstName + ' ' + c.lastName
RETURN fullName, totalSpent
```

Result

customer	totalSpent
"Amir Rahman"	1005
"Keisha Nguyen"	250
"Mateo Ortega"	1015
"Hannah Connor"	260
"Leila Haddad"	1000
"Niko Petrov"	760
"Yusuf Abdi"	1005
Rows: 7	

Advanced examples

The following examples demonstrates how `LET`, and its ability to chain expressions, can be used in more advanced queries:

Example 29. Scenario 1: Supplier notification on customer purchases and discounts

This example retrieves information about what `Product` a `Customer` has bought, and from what `Supplier`. It then calculates the `price` after applying the `discount` and constructs a `message` for each purchase, including the `fullName` of each `Customer` and the `effectivePrice` of a `Product` after a discount, sent to the `Supplier` `email`.

This example highlights that `LET` does not drop variables. All variables introduced in the `MATCH` and subsequent `LET` clauses are available in the final `RETURN` clause.

Retrieve `Customer` purchase details, including discount information

```
MATCH (c:Customer)-[b:BUYS]->(p:Product)<--(s:Supplier)
```

```
LET fullname = c.firstName + ' ' + c.lastName,
    effectivePrice = p.price * (1 - c.discount)
LET message = fullname + " bought " + p.name + " for $" + effectivePrice + " after a " + (c.discount
* 100) + "% discount"
RETURN b.date AS date, message, s.email AS toSupplier
ORDER BY date
```

Result

date	message	toSupplier
2023-05-17	"Leila Haddad bought Laptop for $900.0 after a 10.0% discount"	"contact@techcorp.com"
2023-07-09	"Keisha Nguyen bought Headphones for $200.0 after a 20.0% discount"	"contact@techcorp.com"
2023-12-11	"Hannah Connor bought Coffee for $8.5 after a 15.0% discount"	"info@foodies.com"
2024-04-11	"Mateo Ortega bought Laptop for $950.0 after a 5.0% discount"	"contact@techcorp.com"
2024-06-02	"Hannah Connor bought Headphones for $212.5 after a 15.0% discount"	"contact@techcorp.com"
2024-08-23	"Niko Petrov bought Headphones for $187.5 after a 25.0% discount"	"contact@techcorp.com"
2024-10-09	"Amir Rahman bought Laptop for $900.0 after a 10.0% discount"	"contact@techcorp.com"
2024-12-24	"Yusuf Abdi bought Chocolate for $4.5 after a 10.0% discount"	"info@foodies.com"
2024-12-24	"Niko Petrov bought Coffee for $7.5 after a 25.0% discount"	"info@foodies.com"
2025-01-02	"Yusuf Abdi bought Laptop for $900.0 after a 10.0% discount"	"contact@techcorp.com"
2025-01-10	"Amir Rahman bought Chocolate for $4.5 after a 10.0% discount"	"info@foodies.com"
2025-02-27	"Niko Petrov bought Phone for $375.0 after a 25.0% discount"	"contact@techcorp.com"
2025-03-05	"Mateo Ortega bought Chocolate for $4.75 after a 5.0% discount"	"info@foodies.com"
2025-03-05	"Mateo Ortega bought Coffee for $9.5 after a 5.0% discount"	"info@foodies.com"
Rows: 14		

Example 30. Scenario 2: Supplier gift card distribution based on customer spending

The example calculates the `customerRevenue` for each `Customer` after applying their `discount` on each `Product` they bought. Customers are then categorized into three groups based on their total

spending: Category `A` for those who spent more than `850`, Category `B` for those who spent more than `350` but less than or equal to `850`, and Category `C` for those who spent `350` or less. Category `C` customers are excluded from the results using the `FILTER` clause, leaving only Category `A` and `B` customers eligible for a gift card. The `amount` in the gift card is assigned based on the category, with with Category `A` receiving `20` and Category `B` receiving `10`. The details of the gift card are then sent to the `email` of the relevant customers.

This example highlights how `LET` can be used to succinctly chain expressions, and also that it cannot be used to perform aggregations.

Calculate customer gift card distribution based on spending

```
MATCH (customer:Customer)-[bought:BUYS]->(product:Product)
LET effectivePrice = product.price * (1 - customer.discount)
WITH customer, bought, sum(effectivePrice) AS customerRevenue
LET category = CASE
    WHEN customerRevenue > 850 THEN 'A'
    WHEN customerRevenue > 350 THEN 'B'
    ELSE 'C'
  END
FILTER category <> 'C'
LET amount = CASE
    WHEN category = 'A' THEN 20
    WHEN category = 'B' THEN 10
END
LET message = {
    type: 'giftcard',
    addressee: customer.firstName + ' ' + customer.lastName,
    amount: amount,
    year: bought.date.year
}
RETURN message, customer.email AS toCustomer, customerRevenue
ORDER BY amount
```

Result

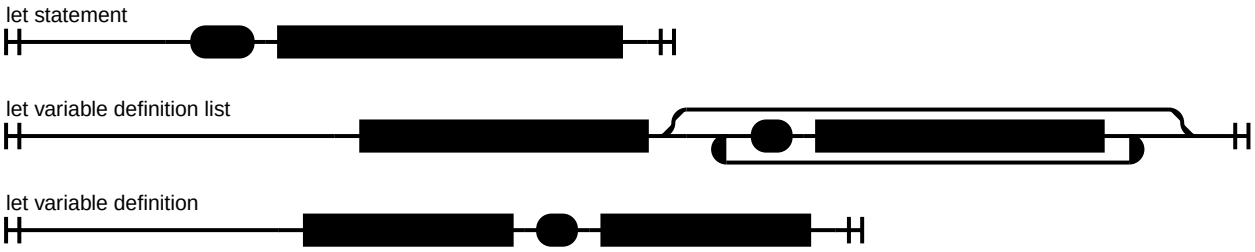
message	toCustomer	customerRevenue
{amount: 10, addressee: "Niko Petrov", type: "giftcard", year: 2025}	"niko.petrov@example.com"	375.0
{amount: 20, addressee: "Amir Rahman", type: "giftcard", year: 2024}	"amir.rahman@example.com"	900.0
{amount: 20, addressee: "Mateo Ortega", type: "giftcard", year: 2024}	"mateo.ortega@example.com"	950.0
{amount: 20, addressee: "Leila Haddad", type: "giftcard", year: 2023}	"leila.haddad@example.com"	900.0
{amount: 20, addressee: "Yusuf Abdi", type: "giftcard", year: 2025}	"yusuf.abdi@example.com"	900.0
Rows: 5		

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



LIMIT

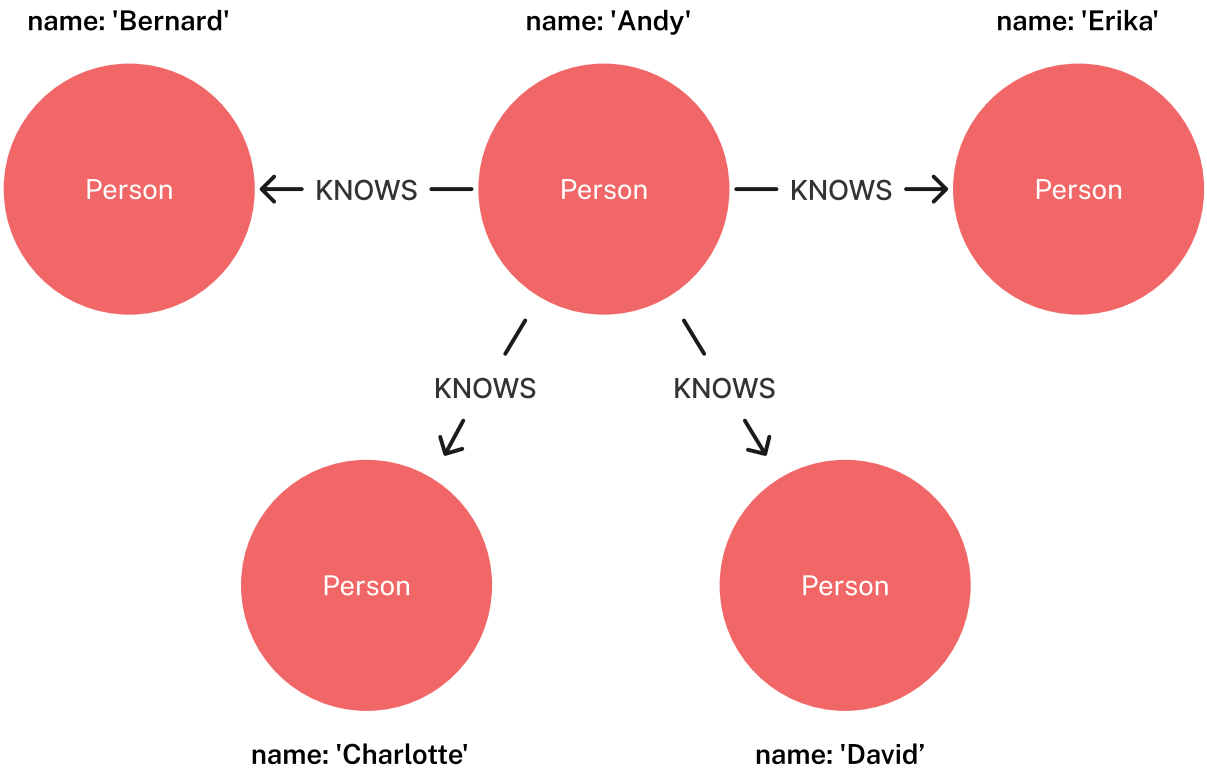
`LIMIT` constrains the number of returned rows.

`LIMIT` accepts any expression that evaluates to a non-negative `INTEGER`, does not contain any variables, and does not refer to nodes or relationships.

> Neo4j does not guarantee the results generated by `LIMIT`. The only clause that guarantees a specific row order is `ORDER BY`.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (andy: Person {name: 'Andy'}),
  (bernard: Person {name: 'Bernard'}),
  (charlotte: Person {name: 'Charlotte'}),
  (david: Person {name: 'David'}),
  (erika: Person {name: 'Erika'}),
  (andy)-[:KNOWS]->(bernard),
  (andy)-[:KNOWS]->(charlotte),
  (andy)-[:KNOWS]->(david),
  (andy)-[:KNOWS]->(erika)
```

Examples

Example 31. Return a limited subset of the rows

To return a limited subset of the rows, use this syntax:

Query

```
MATCH (n)
RETURN n.name
ORDER BY n.name
LIMIT 3
```

Limit to 3 rows by the example query.

Result

n.name
"Andy"
"Bernard"
"Charlotte"
Rows: 3

Example 32. Using an expression with `LIMIT` to return a subset of the rows

`LIMIT` accepts any expression that evaluates to a positive integer, as long as it can be statically calculated (i.e. calculated before the query is run).

Query

```
MATCH (n)
RETURN n.name
ORDER BY n.name
LIMIT 1 + toInteger(3 * rand())
```

Limit 1 row plus randomly 0, 1, or 2. So randomly limit to 1, 2, or 3 rows.

Result

n.name
"Andy"
"Bernard"
"Charlotte"
Rows: 3

`LIMIT` and side effects

The use of `LIMIT` in a query will not stop side effects, like `CREATE`, `DELETE`, or `SET`, from happening if the limit is in the same query part as the side effect.

Query

```
CREATE (n)
RETURN n
LIMIT 0
```

This query returns nothing, but creates one node:

Result

(empty result)
Rows: 0 Nodes created: 1

Query

```
MATCH (n {name: 'A'})
SET n.age = 60
RETURN n
LIMIT 0
```

This query returns nothing, but writes one property:

Result

(empty result)
Rows: 0 Properties set: 1

If we want to limit the number of updates we can split the query using the `WITH` clause:

Query

```
MATCH (n)
WITH n ORDER BY n.name LIMIT 1
SET n.locked = true
RETURN n
ORDER BY n.name
```

Writes `locked` property on one node and return that node:

Result

n
{locked:true,name:"Andy",age:60}
Rows: 1 Properties set: 1

Using `LIMIT` as a standalone clause

`LIMIT` can be used as a standalone clause, or in conjunction with `ORDER BY` or `SKIP`/`OFFSET`.

Standalone use of `LIMIT`

```
MATCH (n)
LIMIT 2
RETURN collect(n.name) AS names
```

Result

names
["Andy", "Bernard"]
Rows: 1

The following query orders all nodes by `name` descending, skips the two first rows and limits the results to two rows. It then collects the results in a list.

`LIMIT` used in conjunction with `ORDER BY` and `SKIP`

```
MATCH (n)
ORDER BY n.name DESC
SKIP 2
LIMIT 2
RETURN collect(n.name) AS names
```

Result

names
["David", "Charlotte"]
Rows: 1

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

LOAD CSV

`LOAD CSV` is used to import data from CSV files into a Neo4j database.

```
LOAD CSV FROM 'https://data.neo4j.com/bands/artists.csv'
AS row
MERGE (:Artist {name: row[1], year: toInteger(row[2])})
```

`FROM` takes a `STRING` containing the path where the CSV file is located.

The clause parses one row at a time, temporarily storing the current row in the variable specified with `AS`.

The `MERGE` clause accesses the `row` variable to insert data into the database.

`LOAD CSV` supports both local and remote URLs. Local paths are resolved relative to the Neo4j installation folder.

> **Note:**
>
> Loading CSV files requires load privileges.

Import CSV data into Neo4j

Import local files

Not available on Aura

You can store CSV files on the database server and then access them by using a `file:///` URL. By default, paths are resolved relative to the Neo4j import directory.

Example 33. Import artists name and year information from a local file

artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'file:///artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
```

```
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'

4 rows

Added 4 nodes, Set 8 properties, Added 4 labels

For ways of importing data into an Aura instance, see Aura → Importing data.

When using `file:///` URLs, spaces and other non-alphanumeric characters must be URL-encoded.

Configuration settings for file URLs

dbms.security.allow_csv_import_from_file_urls

This setting determines whether `file:///` URLs are allowed.

server.directories.import

This setting sets the root directory relative to which `file:///` URLs are parsed.

Import from a remote location

You can import data from a CSV file hosted on a remote path.

`LOAD CSV` supports accessing CSV files via HTTPS, HTTP, and FTP (with or without credentials). It also follows redirects, except those changing the protocol (for security reasons).

Important:

It is strongly recommended to permit resource loading only over secure protocols such as HTTPS instead of insecure protocols like HTTP. This can be done by limiting the load privileges to only trusted sources that use secure protocols. If allowing an insecure protocol is absolutely unavoidable, Neo4j takes measures internally to enhance the security of these requests within their limitations. However, this means that insecure URLs on virtual hosts will not function unless you add the JVM argument `-Dsun.net.http.allowRestrictedHeaders=true` to the configuration setting `server.jvm.additional`.

Example 34. Import artists name and year information from a remote file via HTTPS

https://data.neo4j.com/bands/artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'https://data.neo4j.com/bands/artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'

4 rows

Added 4 nodes, Set 8 properties, Added 4 labels

Example 35. Import artists name and year information from a remote file via FTP using credentials

ftp://<username>:<password>@<domain>/bands/artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'ftp://<username>:<password>@<domain>/bands/artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'

a.name	a.year
4 rows	
Added 4 nodes, Set 8 properties, Added 4 labels	

Import from cloud URIs

Enterprise Edition Not available on Aura

You can import data from a number of different cloud storages:

- Azure Cloud Storage
- Google Cloud Storage
- AWS S3

See Operations Manual → Load a dump from a cloud storage on how to set up access to cloud storages.

Import from an Azure Cloud Storage URI

You can import data from a CSV file hosted in an Azure Cloud Storage URI.

Example 36. Import artists name and year information from an Azure Cloud Storage URI

azb://azb-account/azb-container/artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'azb://azb-account/azb-container/artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'
4 rows	
Added 4 nodes, Set 8 properties, Added 4 labels	

Import from a Google Cloud Storage URI

You can import data from a CSV file hosted in a Google Cloud Storage URI.

Example 37. Import artists name and year information from a Google Cloud Storage URI

gs://gs-bucket/artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'gs://gs-bucket/artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'

4 rows

Added 4 nodes, Set 8 properties, Added 4 labels

Import from an AWS S3 URI

You can import data from a CSV file hosted in an AWS S3 URI.

Example 38. Import artists name and year information from an AWS S3 URI

s3://aws-bucket/artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 's3://aws-bucket/artists.csv' AS row
MERGE (a:Artist {name: row[1], year: toInteger(row[2])})
RETURN a.name, a.year
```

Result

a.name	a.year
'ABBA'	'1992'
'Roxette'	'1986'
'Europe'	'1979'
'The Cardigans'	'1992'

4 rows

Added 4 nodes, Set 8 properties, Added 4 labels

Import CSV files using dynamic columns

CSV columns can be referenced dynamically to map labels to nodes in the graph. This enables flexible data handling, allowing labels to be be populated from CSV column values without manually specifying each entry. It also mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

bands-with-headers.csv

```
Id,Label,Name
1,Band,The Beatles
2,Band,The Rolling Stones
3,Band,Pink Floyd
4,Band,Led Zeppelin
```

Query

```
LOAD CSV WITH HEADERS FROM 'file:///bands-with-headers.csv' AS line
MERGE (n:$(line.Label) {name: line.Name})
RETURN n AS bandNodes
```

Result

bandNodes
(:Band {name: 'The Beatles'})
(:Band {name: 'The Rolling Stones'})
(:Band {name: 'Pink Floyd'})
(:Band {name: 'Led Zeppelin'})

Rows: 4

Added 4 nodes, Set 4 properties, Added 4 labels

`MERGE` queries using dynamic values may not be as performant as those using static values. This is because the Cypher planner uses statically available information when planning queries to determine whether to use an index or not, and this is not possible when using dynamic values. For more information, see `MERGE` using dynamic node labels and relationship

types → Performance caveats.

Import compressed CSV files

`LOAD CSV` can read local CSV files compressed with ZIP or gzip. ZIP archives can have arbitrary directory structures but may only contain a single CSV file.

Import a CSV file from within a ZIP file

```
LOAD CSV FROM 'file:///artists.zip' AS row
MERGE (:Artist {name: row[1], year: toInteger(row[2])})
```

You can't load zipped CSV files from remote URLs.

Import data from relational databases

If the source data comes from a relational model, it's worth evaluating how to gain the most from moving to a graph data model. Before running the import, think about how the data can be modeled as a graph, and adapt its structure accordingly when running the import (see Graph data modeling).

Data from relational databases may consist of one or multiple CSV files, depending on the source database structure. A performant approach is to run multiple passes of `LOAD CSV` to import nodes separately from relationships.

Example 39. Import from a single CSV file

The source file books.csv contains information about both authors and books. From a graph perspective, these are nodes with different labels, so it takes different queries to load them.

The example executes multiple passes of `LOAD CSV` on that one file, and each pass focuses on the creation of one entity type.

books.csv

```
id,title,author,publication_year,genre,rating,still_in_print,last_purchased
19515,The Heights,Anne Conrad,2012,Comedy,5,true,2023/4/12 8:17:00
39913,Starship Ghost,Michael Tyler,1985,Science Fiction|Horror,4.2,false,2022/01/16 17:15:56
60980,The Death Proxy,Tim Brown,2002,Horror,2.1,true,2023/11/26 8:34:26
18793,Chocolate Timeline,Mary R. Robb,1924,Romance,3.5,false,2022/9/17 14:23:45
67162,Stories of Three,Eleanor Link,2022,Romance|Comedy,2,true,2023/03/12 16:01:23
25987,Route Down Below,Tim Brown,2006,Horror,4.1,true,2023/09/24 15:34:18
```

Query

```
// Create `Book` nodes
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/books.csv' AS row
MERGE (b:Book {id: row.id, title: row.title})
MERGE (a:Author {name: row.author});

// Create `WROTE` relationships
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/books.csv' AS row
MATCH (a:Author{name: row.author})
MATCH (b:Book{id: row.id})
MERGE (a)-[:WROTE]->(b);
```

Result

```
Added 11 nodes, Set 17 properties, Added 11 labels
Created 6 relationships
```

Example 40. Import from multiple CSV files

The file acted_in.csv contains data about the relationship between actors and the movies they acted in (from persons.csv and movies.csv). Actors and movies are linked through their ID columns `person_tmdbId` and `movieId`.

The file also contains the role the actor played in the movie, and it is imported in Neo4j as a relationship property.

acted_in.csv

```
movieId,person_tmdbId,role
1,12899,Slinky Dog (voice)
1,12898,Buzz Lightyear (voice)
...
```

It takes three `LOAD CSV` clauses to import this dataset: the first two create `Person` nodes from *persons.csv* and `Movie` nodes from *movies.csv*, and the third adds the `:ACTED_IN` relationship from *acted_in.csv*.

Query

```
// Create person nodes
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
MERGE (p:Person {name: row.name, tmdbId: row.person_tmdbId});

// Create movie nodes
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
MERGE (m:Movie {movieId: row.movieId, title: row.title});

// Create relationships
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/acted_in.csv' AS row
MATCH (p:Person {tmdbId: row.person_tmdbId})
MATCH (m:Movie {movieId: row.movieId})
MERGE (p)-[r:ACTED_IN {role: row.role}]->(m);
```

Result

```
Added 444 nodes, Set 888 properties, Added 444 labels
Added 93 nodes, Set 186 properties, Added 93 labels
Created 372 relationships, Set 372 properties
```

For a guide on importing the Northwind dataset from Postgres into Neo4j, see Tutorial: Import data from a relational database into Neo4j in the Getting Started Guide.

Create additional node labels

In Neo4j a node can have multiple labels, while in a relational setting it's not as straightforward to mix entities. For example, a node in Neo4j can be labeled both `Dog` and `Actor`, while in a relational model dogs

and actors are separate entities.

After a relational dataset has been imported, there may be further labels that can be added, depending on the use case. Additional labels can speed up pinpointing a node if you use them in your queries.

Example 41. Add extra `Actor` label on `Person` nodes

The `:ACTED_IN` relationship from *acted_in.csv* implicitly defines actors as a subset of people. The following queries adds an additional `Actor` label to all people who have an outgoing `:ACTED_IN` relationship.

Query

```
MATCH (p:Person)-[:ACTED_IN]->()
WITH DISTINCT p
SET p:Actor
```

Result

```
Added 353 labels
```

Pre-process the data during import

Cast CSV columns to Neo4j data types

`LOAD CSV` inserts all imported CSV data as `STRING` properties. However, Neo4j supports a range of data types, and storing data with appropriate types allows both to query it more effectively and to process it with type-specific Cypher functions.

Example 42. Import numeric and temporal data

The column `person_tmdbId` and `born` in the file persons.csv contains `INTEGER` and `DATE` values respectively. The functions `toInteger()` and `date()` allow to cast those values to the appropriate types before importing them.

persons.csv

```
person_tmdbId,bio,born,bornIn,died,person_imdbId,name,person_poster,person_url
3,"Legendary Hollywood Icon Harrison Ford was born on July 13, 1942 in Chicago, Illinois.   His
family history includes a strong lineage of actors, radio personalities, and models.   Harrison
attended public high school in Park Ridge, Illinois where he was a member of the school Radio Station
WMTH.  Harrison worked as the lead voice for sports reporting at WMTH for several years.   Acting
wasn't a major interest to Ford until his junior year at Ripon College when he first took an acting
class...",1942-07-13,"Chicago, Illinois, USA",,148,Harrison
Ford,https://image.tmdb.org/t/p/w440_and_h660_face/5M7oN3sznp99hWYQ9sX0xheswWX.jpg,https://themoviedb
.org/person/3
...
```

Query

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
MERGE (p:Person {tmdbId: toInteger(row.person_tmdbId)})
SET p.name = row.name, p.born = date(row.born)
RETURN
  p.name AS name,
```

```
  p.tmdbId AS tmdbId,
  p.born AS born
LIMIT 5
```

Result

name	tmdbId	born
'Harrison Ford'	3	1942-07-13
'Tom Hanks'	31	1956-07-09
'Robin Wright'	32	1966-04-08
'Sally Field'	35	1946-11-06
'Sean Bean'	48	1959-04-17

5 rows

Added 444 nodes, Set 1332 properties, Added 444 labels

For a list of type casting functions, see Casting data values.

Handle `null` values

Neo4j does not store `null` values. `null` or empty fields in a CSV files can be skipped or replaced with default values in `LOAD CSV`.

Example 43. Processing a file with `null` values

In the file `companies.csv`, some rows do not specify values for some columns. The examples show several options of how to handle `null` values.

> The queries in this example use `FILTER` (introduced in Neo4j 2025.06) as a replacement for `WITH * WHERE <predicate>`.

companies.csv

```
Id,Name,Location,Email,BusinessType
1,Neo4j,San Mateo,contact@neo4j.com,P
2,AAA,,info@aaa.com,
3,BBB,Chicago, ,G
,CCC,Michigan,info@ccc.com,G
```

Skip `null` values

```
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
FILTER row.Id IS NOT NULL
MERGE (c:Company {id: row.Id})
```

Provide a default for `null` values

```
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
FILTER row.Id IS NOT NULL
```

```
MERGE (c:Company {id: row.Id, hqLocation: coalesce(row.Location, "Unknown")})
```

Change empty `STRING` values to `null` values (not stored)

```
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
FILTER row.Id IS NOT NULL
MERGE (c:Company {id: row.Id})
SET c.email = nullIf(trim(row.Email), "")
```

`null` values are not stored in the database. A strategy for selectively getting rid of some values is to map them into `null` values. The empty `STRING` values from the last query serve as an example.

Split list values

The function `split()` allows to convert a `STRING` of elements into a list.

Example 44. Parse movies languages and genres as lists

The file movies.csv contains a header line and a total of 94 lines.

The columns `languages` and `genres` contain list-like values. Both are separated by a pipe `|`, and `split()` allows to make them into Cypher lists ahead of inserting them into the database.

movies.csv

```
movieId,title,budget,countries,movie_imdbId,imdbRating,imdbVotes,languages,plot,movie_poster,released
,revenue,runtime,movie_tmdbId,movie_url,year,genres
1,Toy Story,30000000.0,USA,114709,8.3,591836,English,A cowboy doll is profoundly threatened and
jealous when a new spaceman figure supplants him as top toy in a boy's
room.,https://image.tmdb.org/t/p/w440_and_h660_face/uXDfjJbdP4ijW5hWSBrPrlKpxab.jpg,1995-11-
22,373554033.0,81,862,https://themoviedb.org/movie/862,1995,Adventure|Animation|Children|Comedy|Fanta
sy
2,Jumanji,65000000.0,USA,113497,6.9,198355,English|French,"When two kids find and play a magical
board game, they release a man trapped for decades in it and a host of dangers that can only be
stopped by finishing the
game.",https://image.tmdb.org/t/p/w440_and_h660_face/vgpXmVaVyUL7GGiDeiK1mKEKzcX.jpg,1995-12-
15,262797249.0,104,8844,https://themoviedb.org/movie/8844,1995,Adventure|Children|Fantasy
...
```

Query

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
MERGE (m:Movie {id: toInteger(row.movieId)})
SET
    m.title = row.title,
    m.imdbId = toInteger(row.movie_imdbId),
    m.languages = split(row.languages, '|'),
    m.genres = split(row.genres, '|')
RETURN
  m.title AS title,
  m.imdbId AS imdbId,
  m.languages AS languages,
  m.genres AS genres
LIMIT 5
```

Result

title	imdbId	languages	genres
'Toy Story'	114709	['English']	['Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy']
'Jumanji'	113497	['English', 'French']	['Adventure', 'Children', 'Fantasy']
'Grumpier Old Men'	113228	['English']	['Comedy', 'Romance"]
'Waiting to Exhale'	114885	['English']	['Comedy', 'Romance', 'Drama']
'Father of the Bride Part II'	113041	['English']	['Comedy']

5 rows

Added 93 nodes, Set 465 properties, Added 93 labels

For more `STRING` manipulation functions, see String functions.

Recommendations

Create property uniqueness constraints

Always create property uniqueness constraints prior to importing data, to avoid duplicates or colliding entities. If the source file contains duplicated data and the right constraints are in place, Cypher raises an error.

Example 45. Create a node property uniqueness constraints on person ID

persons.csv

```
person_tmdbId,bio,born,bornIn,died,person_imdbId,name,person_poster,person_url
3,"Legendary Hollywood Icon Harrison Ford was born on July 13, 1942 in Chicago, Illinois.   His
family history includes a strong lineage of actors, radio personalities, and models.   Harrison
attended public high school in Park Ridge, Illinois where he was a member of the school Radio Station
WMTH.  Harrison worked as the lead voice for sports reporting at WMTH for several years.   Acting
wasn't a major interest to Ford until his junior year at Ripon College when he first took an acting
class...",1942-07-13,"Chicago, Illinois, USA",,148,Harrison
Ford,https://image.tmdb.org/t/p/w440_and_h660_face/5M7oN3sznp99hWYQ9sX0xheswWX.jpg,https://themoviedb
.org/person/3
...
```

Create a node property uniqueness constraint on person ID

```
CREATE CONSTRAINT Person_tmdbId IF NOT EXISTS
FOR (p:Person) REQUIRE p.tmdbId IS UNIQUE
```

Result

```
Added 1 constraints
```

Handle large amounts of data

`LOAD CSV` may run into memory issues with files containing a significant number of rows (approaching hundreds of thousands or millions). For large files, it's recommended to split the import process in several lighter transactions through the clause `CALL {…} IN TRANSACTIONS`.

Example 46. Load a large CSV file in several transactions

The file persons.csv contains a header line and a total of 869 lines. The example loads the `name` and `born` columns in transactions of 200 rows.

persons.csv

```
person_tmdbId,bio,born,bornIn,died,person_imdbId,name,person_poster,person_url
3,"Legendary Hollywood Icon Harrison Ford was born on July 13, 1942 in Chicago, Illinois.   His
family history includes a strong lineage of actors, radio personalities, and models.   Harrison
attended public high school in Park Ridge, Illinois where he was a member of the school Radio Station
WMTH.  Harrison worked as the lead voice for sports reporting at WMTH for several years.   Acting
wasn't a major interest to Ford until his junior year at Ripon College when he first took an acting
class...",1942-07-13,"Chicago, Illinois, USA",,148,Harrison
Ford,https://image.tmdb.org/t/p/w440_and_h660_face/5M7oN3sznp99hWYQ9sX0xheswWX.jpg,https://themoviedb
.org/person/3
...
```

The below query uses a variable scope clause to import variables into the `CALL` subquery.

Query

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
CALL (row) {
  MERGE (p:Person {tmdbId: row.person_tmdbId})
  SET p.name = row.name, p.born = row.born
} IN TRANSACTIONS OF 200 ROWS
```

Result

```
Added 444 nodes, Set 1332 properties, Added 444 labels
```

In case of errors, `CALL {…} IN TRANSACTIONS` may only import a part of the CSV data as the transactions are *committed*. For example, if the first 200 rows are error free, they are committed. If the next 200 rows contain data that causes an error, the second transaction fails, but leaves the first transaction unaffected.

LOAD CSV and Neo4j functions

Access line numbers with `linenumber()`

The `linenumber()` function provides the line number which `LOAD CSV` is operating on, or `null` if called outside of a `LOAD CSV` context.

A common use case for this function is to generate sequential unique IDs for CSV data that doesn't have a unique column already.

Example 47. `linenumber()`

artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'file:///artists.csv' AS row
RETURN linenumber() AS number, row
```

Result

number	row
1	["1","ABBA","1992"]
2	["2","Roxette","1986"]
3	["3","Europe","1979"]
4	["4","The Cardigans","1992"]
4 rows	

Access the CSV file path with `file()`

The `file()` function provides the absolute path of the file that `LOAD CSV` is operating on, or `null` if called out of a `LOAD CSV` context.

Example 48. `file()`

artists.csv

```
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV FROM 'file:///artists.csv' AS row
RETURN DISTINCT file() AS path
```

Result

path
'/artists.csv'

`file()` always returns a local path, even when loading remote CSV files. For remote resources, `file()` returns the temporary local path it was downloaded to.

CSV file format

The CSV file format and `LOAD CSV` interact as follows:

- The file character encoding must be UTF-8.
- The line terminator is system dependent (`\n` for Unix and `\r\n` for Windows).
- The default field delimiter is `,`. Change it with the option `FIELDTERMINATOR`.
- CSV files may contain quoted `STRING` values, and the quotes are dropped when `LOAD CSV` reads the data.
- If `dbms.import.csv.legacy_quote_escaping` is set to the default value of `true`, `\` is used as an escape character.
- A double quote must be in a quoted `STRING` and escaped, with either the escape character or a second double quote.

Headers

If the CSV file starts with a header row containing column names, each import row in the file acts as a map instead of an array.

You must indicate the presence of the header row by adding `WITH HEADERS` to the query. You can then access specific fields by their corresponding column name.

Example 49. Parsing a CSV as a list of maps

artists-with-headers.csv

```
Id,Name,Year
1,ABBA,1992
2,Roxette,1986
3,Europe,1979
4,The Cardigans,1992
```

Query

```
LOAD CSV WITH HEADERS FROM 'file:///artists-with-headers.csv' AS row
MERGE (a:Artist {name: row.Name, year: toInteger(row.Year)})
RETURN
  a.name AS name,
  a.year AS year
```

Result

name	year
"ABBA"	1992
"Roxette"	1986

name	year
"Europe"	1979
"The Cardigans"	1992

4 rows

Added 4 nodes, Set 8 properties, Added 4 labels

Field delimiter

The default field delimiter is `,`. Use the `FIELDTERMINATOR` option to specify a different field delimiter.

If you try to import a file that doesn't use `,` as field delimiter and you also don't specify a custom delimiter, `LOAD CSV` will interpret the CSV as having a single column.

Example 50. Import a CSV using `;` as field delimiter

artists-fieldterminator.csv

```
1;ABBA;1992
2;Roxette;1986
3;Europe;1979
4;The Cardigans;1992
```

Query

```
LOAD CSV FROM 'file:///artists-fieldterminator.csv' AS row FIELDTERMINATOR ';'
MERGE (:Artist {name: row[1], year: toInteger(row[2])})
```

Result

```
Added 4 nodes, Set 8 properties, Added 4 labels
```

You can use the hexadecimal representation of the unicode character for the field delimiter if you prepend `\u`. Write the encoding with four digits: for example, `\u003B` is equivalent to `;` (semicolon).

Quotes escaping

Quoted `STRING` values are allowed in the CSV file and the quotes are dropped when `LOAD CSV` reads the data. If quoted `STRING` values must contain quote characters `"`, there are two ways to escape them:

1. **Double quotes** — Use another quote `"` to escape a quote (for example, the CSV encoding of the `STRING` `The "Symbol"` is `"The ""Symbol"""`).
2. **Prefix with backslash `\`** — If the configuration setting `dbms.import.csv.legacy_quote_escaping` is set to `true` (the default value), `\` works as the escape character for quotes (for example, the CSV encoding of the `STRING` `The "Symbol"` is `"The \"Symbol\""`).

Example 51. Import a CSV with double-quotes escaping

artists-with-escaped-quotes.csv

```
"1","The ""Symbol""","1992"
"2","The \"Symbol\"","1992"
```

Query

```
LOAD CSV FROM 'file:///artists-with-escaped-quotes.csv' AS row
MERGE (a:Artist {id: toInteger(row[0]), name: row[1], year: toInteger(row[2])})
RETURN
  a.id AS id,
  a.name AS name,
  a.year AS year,
  size(a.name) AS size
```

Result

id	name	year	size
1	'The "Symbol"'	1992	12
2	'The "Symbol"'	1992	12
Added 2 nodes, Set 6 properties, Added 2 labels			

Note that `name` is a `STRING`, as it is wrapped in quotes in the output. The third column outputs the `STRING` length as `size`. The length only counts what is between the outer quotes, but not the quotes themselves.

Check source data quality

In case of a failed import, there are some elements to check to ensure the source file is not corrupted.

- **Inconsistent headers** — The CSV header may be inconsistent with the data. It can be missing, have too many columns or have a different delimiter. Verify that the header matches the data in the file. Adjust the formatting, delimiters or columns.
- **Extra or missing quotes** — Standalone double or single quotes in the middle of non-quoted text or non-escaped quotes in quoted text can cause issues reading the file. Either escape or remove stray quotes. See Quotes escaping.
- **Special or newline characters** — When dealing with special characters in a file, ensure they are quoted or remove them.
- **Inconsistent line breaks** — Ensure line breaks are consistent throughout your file.
- **Binary zeros, BOM byte order mark and other non-text characters** — Unusual characters or tool-specific formatting are sometimes hidden in application tools, but become apparent in plain-text editors. If you come across these types of characters in your file, either remove them or use Cypher's normalize function.

Inspect source files ahead of import

Before importing data into the database, you can use `LOAD CSV` to inspect a source file and get an idea of what form the imported data is going to have.

Example 52. Assert correct line count

```
// Assert correct line count
LOAD CSV FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS line
RETURN count(*);
```

Result

count(*)
445
1 row

Example 53. Check the first five lines with header sampling

```
// Check first 5 line-sample with header-mapping
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS line
RETURN line.person_tmdbId, line.name
LIMIT 5;
```

Result

line.person_tmdbId	line.name
'3'	'Harrison Ford'
'31'	'Tom Hanks'
'32'	'Robin Wright'
'35'	'Sally Field'
'48'	'Sean Bean'
5 rows	

Example

Erase current database and import the full movie dataset

```
// Clear data
MATCH (n) DETACH DELETE n;

// Create constraints
CREATE CONSTRAINT Person_tmdbId IF NOT EXISTS
FOR (p:Person) REQUIRE p.tmdbId IS UNIQUE;

CREATE CONSTRAINT Movie_movieId IF NOT EXISTS
FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;

// Create person nodes
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
```

```
MERGE (p:Person {tmdbId: toInteger(row.person_tmdbId)})
SET p.name = row.name, p.born = date(row.born);

// Create movie nodes
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
MERGE (m:Movie {id: toInteger(row.movieId)})
SET
    m.title = row.title,
    m.imdbId = toInteger(row.movie_imdbId),
    m.languages = split(row.languages, '|'),
    m.genres = split(row.genres, '|');

// Create relationships
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/acted_in.csv' AS row
MATCH (p:Person {tmdbId: toInteger(row.person_tmdbId)})
MATCH (m:Movie {id: toInteger(row.movieId)})
MERGE (p)-[r:ACTED_IN]->(m)
SET r.role = row.role;

// Set additional node label
MATCH (p:Person)-[:ACTED_IN]->()
WITH DISTINCT p
SET p:Actor;
```

Result

```
Added 1 constraints
Added 1 constraints
Added 444 nodes, Set 1332 properties, Added 444 labels
Added 93 nodes, Set 465 properties, Added 93 labels
Created 372 relationships, Set 372 properties
Added 353 labels
```

With increasing amounts of data, it is more efficient to create *all* nodes first, and then add relationships with a second pass.

Other ways of importing data

There are a few other tools to get CSV data into Neo4j.

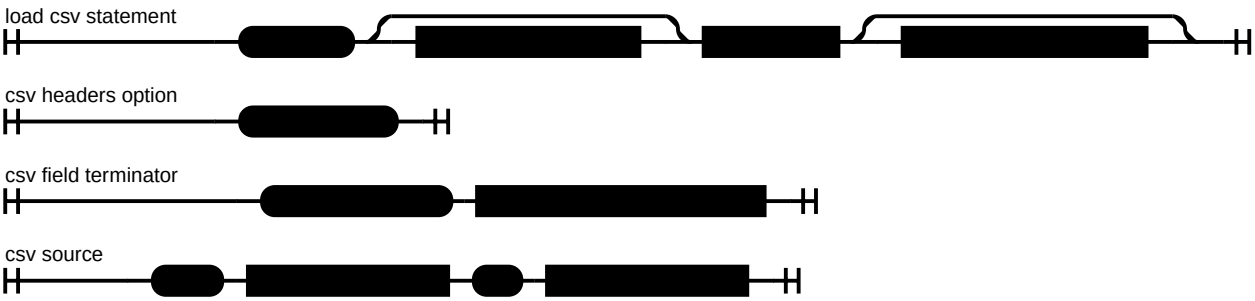
1. **The `neo4j-admin database import` command** is the most efficient way of importing large CSV files.
2. Use a language library to parse CSV data and run creation Cypher queries against a Neo4j database. Created as an extension library to provide common procedures and functions to developers. This library is especially helpful for complex transformations and data manipulations. Useful procedures include apoc.load.jdbc, apoc.load.json, and others.
3. The ETL Tool: allows to extract the schema from a relational database and turn it into a graph model. It then takes care of importing the data into Neo4j.
4. The Kettle import tool maps and executes steps for the data process flow and works well for very large datasets, especially if you are already familiar with using this tool.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

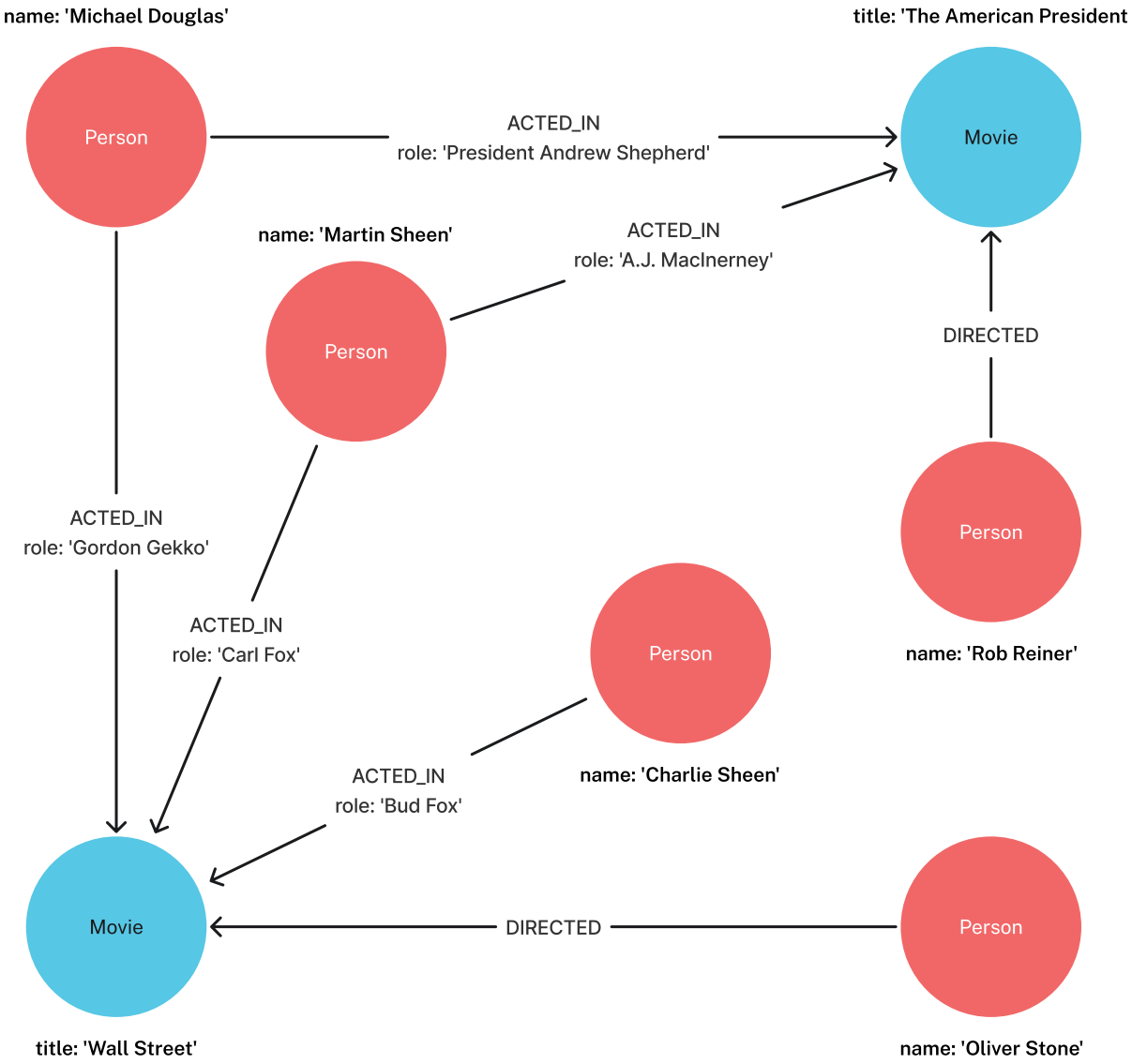


MATCH

The `MATCH` clause enables you to define specific patterns that the database will search for within its graph structure. The `MATCH` clause can specify the nodes, relationships, and properties in a pattern, allowing for queries that traverse the graph to retrieve relevant data.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (charlie:Person:Actor {name: 'Charlie Sheen'}),
       (martin:Person:Actor {name: 'Martin Sheen'}),
       (michael:Person:Actor {name: 'Michael Douglas'}),
       (oliver:Person:Director {name: 'Oliver Stone'}),
       (rob:Person:Director {name: 'Rob Reiner'}),
       (wallStreet:Movie {title: 'Wall Street'}),
       (charlie)-[:ACTED_IN {role: 'Bud Fox'}]->(wallStreet),
       (martin)-[:ACTED_IN {role: 'Carl Fox'}]->(wallStreet),
       (michael)-[:ACTED_IN {role: 'Gordon Gekko'}]->(wallStreet),
       (oliver)-[:DIRECTED]->(wallStreet),
       (thePresident:Movie {title: 'The American President'}),
       (martin)-[:ACTED_IN {role: 'A.J. MacInerney'}]->(thePresident),
       (michael)-[:ACTED_IN {role: 'President Andrew Shepherd'}]->(thePresident),
       (rob)-[:DIRECTED]->(thePresident)
```

Find nodes

The `MATCH` clause allows you to specify node patterns of varying complexity to retrieve from a graph. For more information about finding node patterns, see Patterns → Node patterns.

Find all nodes

By specifying a pattern with a single node and no labels, all nodes in the graph will be returned.

Find all nodes in a graph

```
MATCH (n)
RETURN n
```

Result

n
(:Person {"name":"Charlie Sheen"})
(:Person {"name":"Martin Sheen"})
(:Person {"name":"Michael Douglas"})
(:Person {"name":"Oliver Stone"})
(:Person {"name":"Rob Reiner"})
(:Movie {"title":"Wall Street"})
(:Movie {"title":"The American President"})
Rows: 7

Find nodes with a specific label

Find all nodes with the `Movie` label

```
MATCH (movie:Movie)
RETURN movie.title
```

Result

movie.title
"Wall Street"
"The American President"
Rows: 2

MATCH using node label expressions

Node pattern using the `OR` (`|`) label expression

```
MATCH (n:Movie|Person)
RETURN n.name AS name, n.title AS title
```

Result

name	title
"Charlie Sheen"	<null>
"Martin Sheen"	<null>
"Michael Douglas"	<null>
"Oliver Stone"	<null>
"Rob Reiner"	<null>
<null>	"Wall Street"
<null>	"The American President"
Rows: 7	

Node pattern using negation (`!`) label expression

```
MATCH (n:!Movie)
RETURN labels(n) AS label, count(n) AS labelCount
```

The above query uses the `labels()` and `count()` functions.

Result

label	labelCount
["Person", "Actor"]	3
["Person", "Director"]	2
Rows: 2	

For a list of all label expressions supported by Cypher, see Patterns → Label expressions.

Find relationships

The `MATCH` clause allows you to specify relationship patterns of varying complexity to retrieve from a graph. Unlike a node pattern, a relationship pattern cannot be used in a `MATCH` clause without node patterns at both ends. For more information about relationship patterns, see Patterns → Relationship patterns.

By default, Cypher will only match a relationship once inside a single pattern. This behavior can be bypassed using `REPEATABLE ELEMENTS`, which allows both nodes and relationships to occur more than once in a given `MATCH` result. For more information, see `REPEATABLE ELEMENTS`.

Empty relationship patterns

By applying `--`, a pattern will be matched for a relationship with any direction and without any filtering on relationship types or properties.

Find connected nodes using an empty relationship pattern

```
MATCH (:Person {name: 'Oliver Stone'})--(n)
RETURN n AS connectedNodes
```

Result

connectedNodes
(:Movie {title: "Wall Street"})
Rows: 1

Directed relationship patterns

The direction of a relationship in a pattern is indicated by arrows: `-->` or `<--`.

Find all nodes connected to `Oliver Stone` by an outgoing relationship.

```
MATCH (:Person {name: 'Oliver Stone'})-->(movie:Movie)
RETURN movie.title AS movieTitle
```

Result

movieTitle
"Wall Street"
Rows: 1

Relationship variables

It is possible to introduce a variable to a pattern, either for filtering on relationship properties or to return a relationship.

Find the types of an aliased relationship

```
MATCH (:Person {name: 'Oliver Stone'})-[r]->()
RETURN type(r) AS relType
```

The above query uses the `type()` function.

Result

relType
"DIRECTED"
Rows: 1

MATCH on an undirected relationship

When a pattern contains a bound relationship, and that relationship pattern does not specify direction, Cypher will match the relationship in both directions.

Relationship pattern without direction

```
MATCH (a)-[:ACTED_IN {role: 'Bud Fox'}]-(b)
RETURN a, b
```

Result

a	b
(:Movie {"title":"Wall Street"})	(:Person {"name":"Charlie Sheen"})
(:Person {"name":"Charlie Sheen"})	(:Movie {"title":"Wall Street"})
Rows: 2	

Filter on relationship types

It is possible to specify the type of a relationship in a relationship pattern by using a colon (`:`) before the relationship type.

Relationship pattern filtering on the `ACTED_IN` relationship type

```
MATCH (:Movie {title: 'Wall Street'})<-[:ACTED_IN]-(actor:Person)
RETURN actor.name AS actor
```

Result

actor
"Michael Douglas"
"Martin Sheen"
"Charlie Sheen"

actor
Rows: 3

MATCH using relationship type expressions

It is possible to match a pattern containing one of several relationship types using the `OR` symbol, `|`.

Relationship pattern including either `ACTED_IN` or `DIRECTED` relationship types

```
MATCH (:Movie {title: 'Wall Street'})<-[:ACTED_IN|DIRECTED]-(person:Person)
RETURN person.name AS person
```

Result

person
"Oliver Stone"
"Michael Douglas"
"Martin Sheen"
"Charlie Sheen"
Rows: 4

As relationships can only have exactly one type each, `()-[:A&B]→()` will never match a relationship.

For a list of all relationship type expressions supported by Cypher, see Patterns → Label expressions.

Find multiple relationships

A graph pattern can contain several relationship patterns.

Graph pattern including several relationship patterns

```
MATCH (:Person {name: 'Charlie Sheen'})-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(director:Person)
RETURN movie.title AS movieTitle, director.name AS director
```

Result

movieTitle	director
"Wall Street"	"Oliver Stone"
Rows: 1	

MATCH with WHERE predicates

The `MATCH` clause is often paired with a `WHERE` sub-clause, which adds predicates to refine the patterns, making them more specific. These predicates are part of the pattern itself, not just filters applied after matching. Thus, always place the `WHERE` clause with its corresponding `MATCH` clause.

Simple `WHERE` predicate

```
MATCH (charlie:Person)-[:ACTED_IN]->(movie:Movie)
WHERE charlie.name = 'Charlie Sheen'
RETURN movie.title AS movieTitle
```

Result

movieTitle
"Wall Street"
Rows: 1

More complex `WHERE` predicate

```
MATCH (martin:Person)-[:ACTED_IN]->(movie:Movie)
WHERE martin.name = 'Martin Sheen' AND NOT EXISTS {
    MATCH (movie)<-[:DIRECTED]-(director:Person {name: 'Oliver Stone'})
}
RETURN movie.title AS movieTitle
```

The above query uses an `EXISTS` subquery.

Result

movieTitle
"The American President"
Rows: 1

For more information, see the `WHERE` page.

MATCH with parameters

The `MATCH` clause can be used with parameters.

Parameters

```
{
  "movieTitle": "Wall Street",
  "actorRole": "Fox"
}
```

Find nodes using paramters

```
MATCH (:Movie {title: $movieTitle})<-[r:ACTED_IN]-(p:Person)
WHERE r.role CONTAINS $actorRole
RETURN p.name AS actor, r.role AS role
```

The above query uses the `CONTAINS` operator.

Result

actor	role
"Charlie Sheen"	"Bud Fox"
"Martin Sheen"	"Carl Fox"
Rows: 2	

For more information about how to set parameters, see Syntax → Parameters.

Find paths

The `MATCH` clause can also be used to bind whole paths to variables.

Find all paths matching a pattern

```
MATCH path = ()-[:ACTED_IN]->(movie:Movie)
RETURN path
```

Result

path
(:Person {name: "Charlie Sheen"})-[:ACTED_IN {role: "Bud Fox"}]→(:Movie {title: "Wall Street"})
(:Person {name: "Martin Sheen"})-[:ACTED_IN {role: "Carl Fox"}]→(:Movie {title: "Wall Street"})
(:Person {name: "Martin Sheen"})-[:ACTED_IN {role: "A.J. MacInerney"}]→(:Movie {title: "The American President"})
(:Person {name: "Michael Douglas"})-[:ACTED_IN {role: "Gordon Gekko"}]→(:Movie {title: "Wall Street"})
(:Person {name: "Michael Douglas"})-[:ACTED_IN {role: "President Andrew Shepherd"}]→(:Movie {title: "The American President"})
Rows: 5

Find paths matching a pattern including a `WHERE` predicate

```
MATCH path = (:Person)-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(:Person)
WHERE movie.title = 'Wall Street'
RETURN path
```

Result

path
(:Person {name: "Charlie Sheen"})-[:ACTED_IN {role: "Bud Fox"}]→(:Movie {title: "Wall Street"})←[:DIRECTED]-(:Person {name: "Oliver Stone"})
(:Person {name: "Martin Sheen"})-[:ACTED_IN {role: "Carl Fox"}]→(:Movie {title: "Wall Street"})←[:DIRECTED]-(:Person {name: "Oliver Stone"})
(:Person {name: "Michael Douglas"})-[:ACTED_IN {role: "Gordon Gekko"}]→(:Movie {title: "Wall Street"})←[:DIRECTED]-(:Person {name: "Oliver Stone"})
Rows: 3

For more information about how `MATCH` is used to find patterns of varying complexity (including quantified path patterns, quantified relationships, and the shortest paths between nodes), see the section on Patterns.

Multiple MATCH clauses, the WITH clause, and clause composition

In Cypher, the behavior of a query is defined by its clauses. Each clause takes the current graph state and a table of intermediate results, processes them, and passes the updated graph state and results to the next clause. The first clause starts with the graph's initial state and an empty table, while the final clause produces the query result.

Chaining consecutive `MATCH` clauses

```
MATCH (:Person {name: 'Martin Sheen'})-[:ACTED_IN]->(movie:Movie)
MATCH (director:Person)-[:DIRECTED]->(movie)
RETURN director.name AS director, movie.title AS movieTitle
```

The result of the first `MATCH` clause is the variable `movie` which holds all the `Movies` that `Martin Sheen` has `ACTED_IN`.

The second `MATCH` clause uses the `movie` variable to find any `Person` node with a `DIRECTED` relationship to those `Movie` nodes that `Martin Sheen` has `ACTED_IN`.

Result

director	movieTitle
"Oliver Stone"	"Wall Street"
"Rob Reiner"	"The American President"
Rows: 2	

A variable can be implicitly carried over to the following clause by being referenced in another operation. A variable can also be explicitly passed to the following clause using the `WITH` clause. If a variable is neither implicitly nor explicitly carried over to its following clause, it will be discarded and is not available for reference later in the query.

Using `WITH` and multiple `MATCH` clauses

```
MATCH (actors:Person)-[:ACTED_IN]->(movies:Movie)
WITH actors, count(movies) AS movieCount
ORDER BY movieCount DESC
LIMIT 1
MATCH (actors)-[:ACTED_IN]->(movies)
RETURN actors.name AS actor, movieCount, collect(movies.title) AS movies
```

The `Person` and `Movie` nodes matched in this step are stored in variables, which are then passed on to the second row of the query.

The `movies` variable is implicitly imported by its occurrence in the `count()` function. The `WITH` clause explicitly imports the `actors` variable.

An `ORDER BY` clause orders the results by `movieCount` in descending order, ensuring that the `Person` with the highest number of movies appears at the top, and `LIMIT 1` ensures that all other `Person` nodes are

discarded.

The second `MATCH` clause finds all `Movie` nodes associated with the `Person` nodes currently bound to the `actors` variable.

The above query uses the `collect()` function.

Result

actor	movieCount	movies
"Martin Sheen"	2	["Wall Street", "The American President"]

Rows: 1

For more information about how Cypher queries work, see Clause composition.

MATCH using dynamic node labels and relationship types

Node labels and relationship types can be referenced dynamically in expressions, parameters, and variables when matching nodes and relationships. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

Syntax for matching node labels dynamically

```
MATCH (n:$(<expr>))
MATCH (n:$any(<expr>))
MATCH (n:$all(<expr>))
```

`MATCH (n:$all(<expr>))` is functionally equivalent to `MATCH (n:$(<expr>))`.

Syntax for matching relationship types dynamically

```
MATCH ()-[r:$(<expr>)]->()
MATCH ()-[r:$any(<expr>)]->()
MATCH ()-[r:$all(<expr>)]->()
```

The expression must evaluate to a `STRING NOT NULL | LIST<STRING NOT NULL> NOT NULL` value. If you use a `LIST<STRING>` with more than one item in a relationship pattern with dynamic relationship types, no results will be returned. This is because a relationship can only have exactly one type.

Match labels dynamically

```
WITH ["Person", "Director"] AS labels
MATCH (directors:$(labels))
RETURN directors
```

Result

directors
(:Person:Director {name: "Oliver Stone"})
(:Person:Director {name: "Rob Reiner"})
Rows: 2

Match nodes dynamically using `any()`

```
MATCH (n:$any(["Movie", "Actor"]))
RETURN n AS nodes
```

Result

nodes
(:Person:Actor {name: "Charlie Sheen"})
(:Person:Actor {name: "Martin Sheen"})
(:Person:Actor {name: "Michael Douglas"})
(:Movie {title: "Wall Street"})
(:Movie {title: "The American President"})
Rows: 5

Parameter

```
{
  "label": "Movie"
}
```

Match nodes dynamically using a parameter

```
MATCH (movie:$($label))
RETURN movie.title AS movieTitle
```

Result

movieTitle
"Wall Street"
"The American President"
Rows: 2

Match relationships dynamically using a variable

```
CALL db.relationshipTypes()
YIELD relationshipType
MATCH ()-[r:$(relationshipType)]->()
RETURN relationshipType, count(r) AS relationshipCount
```

Result

relationshipType	relationshipCount
"ACTED_IN"	5
"DIRECTED"	2
Rows: 2	

Performance caveats

`MATCH` queries that use dynamic values may not perform as well as those with static values. Neo4j is actively working to improve the performance of these queries. The table below outlines performance caveats for specific Neo4j versions.

Neo4j versions and performance caveats

Neo4j versions	Performance caveat
5.26 — 2025.07	The Cypher planner is not able to leverage indexes with index scans or seeks and must instead utilize the `AllNodesScan` operator, which reads all nodes from the node store and is therefore more costly.
2025.08 — 2025.10	The Cypher planner is able to leverage token lookup indexes when matching node labels and relationship types dynamically. This is enabled by the introduction of three new query plan operators: `DynamicLabelNodeLookup`, `DynamicDirectedRelationshipTypeLookup`, and `DynamicUndirectedRelationshipTypeLookup`. It is not, however, able to use indexes on property values. For example, `MATCH (n:$(Label) {foo: bar})` will not use any indexes on `n.foo` but can use a `DynamicLabelNodeLookup` on `$(label)`.

2025.11 — current	The Cypher planner is able to leverage indexes on property values, however:
	• It only supports exact seeks on range indexes (no full text or spatial).
	• The index order cannot be leveraged, so the planner must insert separate ordering if required later on in the query.
	• Parallel runtime seeks and scans are single-threaded.
	• The planner does not combine multiple index seeks for a dynamic label query. If several labels passed to `$any` share an index on the same property, the planner picks just one of those indexes (based on selectivity) and filters the rest of the results manually. For example, the below `MATCH` query will only use one of the indexes on `birthYear` for either `Actor` or `Director`, but not both.
	`CREATE RANGE INDEX actor_has_birthyear FOR (a:Actor) ON (a.birthYear);`
	`CREATE RANGE INDEX director_has_birthyear FOR (d:Director) ON (d.birthYear);`
	`MATCH (p:$any(["Actor", "Director"]) { birthYear: 1983 })`
	`RETURN p.name`

Further reading

Neo4j Developer Blog: Cypher Dynamism: A Step Toward Simpler and More Secure Queries

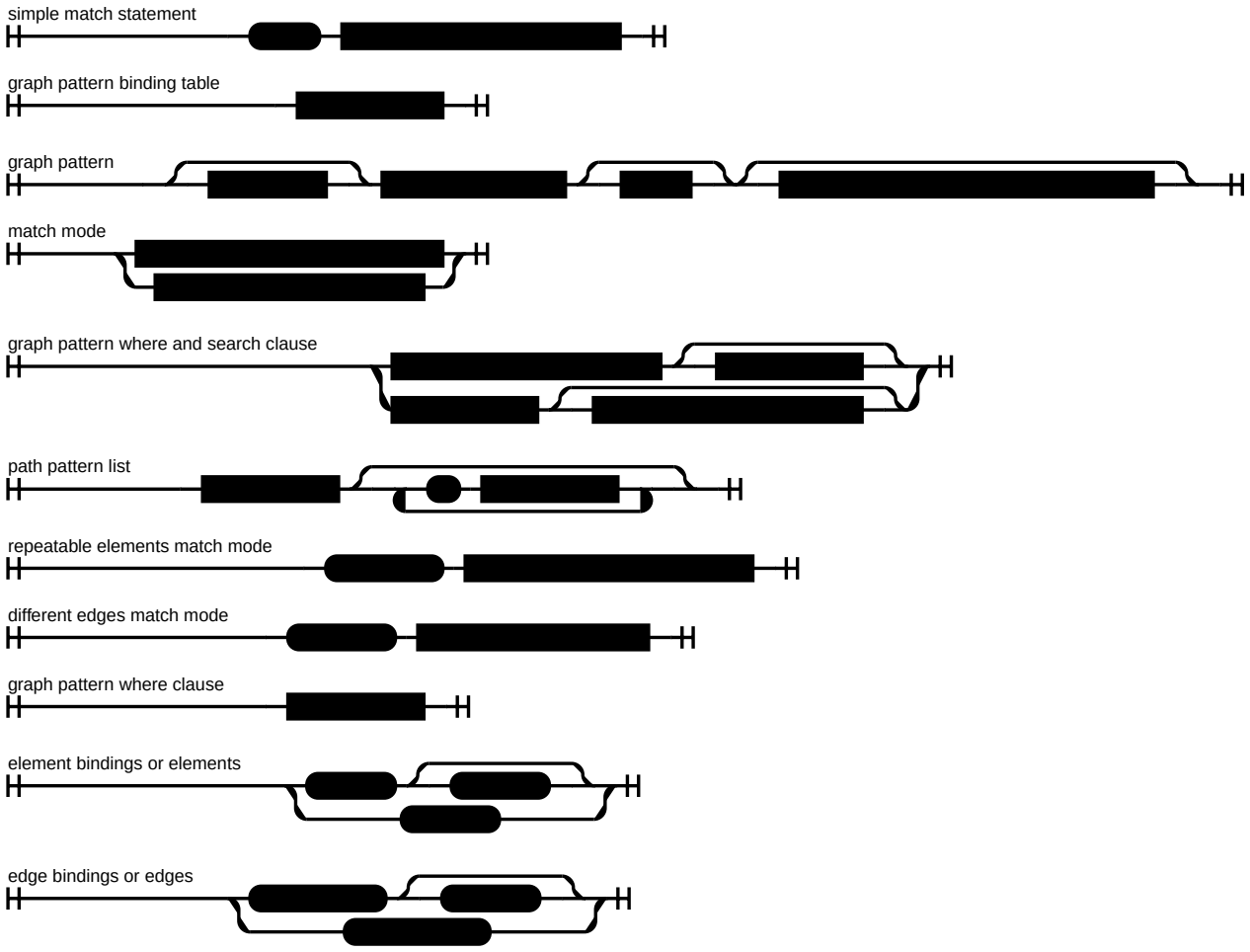
Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
```

```
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



MERGE

The `MERGE` clause combines the behavior of `MATCH` and `CREATE`. When merging a pattern:

- If the exact pattern is found, nothing is created; `MERGE` acts like `MATCH` and binds the found pattern.
- If the exact pattern is not found, `MERGE` acts like `CREATE` and creates the pattern.

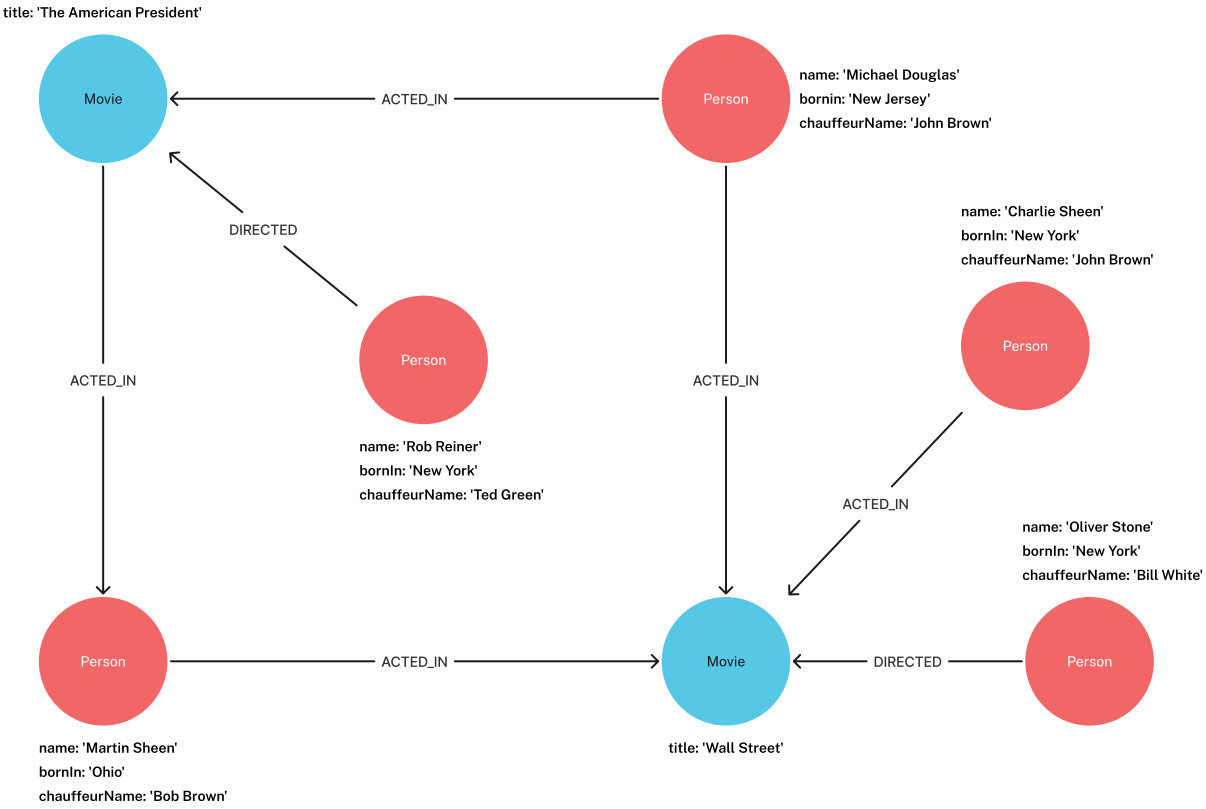
For more information about the differences between the two clauses, see `CREATE` → Comparing `CREATE` and `MERGE`.

`MERGE` also allows for specific actions depending on whether the specified pattern was matched (`ON MATCH`) or created (`ON CREATE`).

It is recommended to create constraints prior to merging data: they provide index-backed performance improvements (in the case of property uniqueness and key constraints) while also preventing the creation of data that differ in unintended ways from pre-existing data. For more information, see Using `MERGE` with constraints below.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (charlie:Person {name: 'Charlie Sheen', bornIn: 'New York', chauffeurName: 'John Brown'}),
       (martin:Person {name: 'Martin Sheen', bornIn: 'Ohio', chauffeurName: 'Bob Brown'}),
       (michael:Person {name: 'Michael Douglas', bornIn: 'New Jersey', chauffeurName: 'John Brown'}),
       (oliver:Person {name: 'Oliver Stone', bornIn: 'New York', chauffeurName: 'Bill White'}),
       (rob:Person {name: 'Rob Reiner', bornIn: 'New York', chauffeurName: 'Ted Green'}),
       (wallStreet:Movie {title: 'Wall Street'}),
       (theAmericanPresident:Movie {title: 'The American President'}),
       (charlie)-[:ACTED_IN]->(wallStreet),
       (martin)-[:ACTED_IN]->(wallStreet),
       (michael)-[:ACTED_IN]->(wallStreet),
       (martin)-[:ACTED_IN]->(theAmericanPresident),
       (michael)-[:ACTED_IN]->(theAmericanPresident),
       (oliver)-[:DIRECTED]->(wallStreet),
       (rob)-[:DIRECTED]->(theAmericanPresident)
```

`MERGE` existing patterns

If `MERGE` matches an existing pattern, it will not create anything. Instead, it will bind the pattern in the same way that `MATCH` does.

`MERGE` *an existing node*

```
MERGE (michael:Person {name: 'Michael Douglas'})
RETURN michael.name AS name,
       michael.bornIn AS bornIn
```

No new node is created, since there already exists an identical node in the database.

Result

name	bornIn
"Michael Douglas"	"New Jersey"
Rows: 1	

`MERGE` *an existing relationship*

```
MATCH (charlie:Person {name: 'Charlie Sheen'}),
      (wallStreet:Movie {title: 'Wall Street'})
MERGE (charlie)-[r:ACTED_IN]->(wallStreet)
RETURN charlie.name AS name,
       type(r) AS newRel,
       wallStreet.title AS title
```

`Charlie Sheen` had already been marked as acting in `Wall Street`, so the existing relationship is found and returned.

Result

name	newRel	title
"Charlie Sheen"	"ACTED_IN"	"Wall Street"
Rows: 1		

`null` properties

`MERGE` cannot be used on property values that are `null`.

For example, the following query will throw an error:

`MERGE` *on* `null` *property*

```
MERGE (martin:Person {name: 'Martin Sheen'})-[r:FATHER_OF {since: null}]->(charlie:Person {name: 'Charlie Sheen'})
RETURN type(r)
```

GQLSTATUS error chain

22N31: error: data exception - invalid properties in merge pattern. The relationship property `since` in '(martin)-[:FATHER_OF {since: null}]→(charlie)' is invalid. 'MERGE' cannot be used with a graph element property value that is null.

22G03: error: data exception - invalid value type

`MERGE` new patterns

If `MERGE` cannot match the exact pattern given in the query, it will create the pattern instead. To prevent the creation of nodes or relationships that differ in unintended ways from existing ones, it is therefore recommended to use constraints when merging data. For more information, see Using `MERGE` with constraints below.

`MERGE` *a new node with labels and properties*

```
MERGE (robert:Critic:Viewer {name: 'Robert Smith', occupation: 'Journalist'})
RETURN labels(robert) AS labels,
       properties(robert) AS properties
```

A new node is created because there are no identical nodes in the dataset.

Result

labels	properties
["Critic", "Viewer"]	{name: "Robert Smith", occupation: "Journalist"}
Rows: 1	

Multiple labels can also be separated by an ampersand `&`, in the same manner as it is used in label expressions. Separation by colon `:` and ampersand `&` cannot be mixed in the same clause.

`MERGE` *node with different property values*

```
MERGE (:Person {name: 'Charlie Sheen', bornIn: "New York", chauffeurName: 'Michael Black'})
MATCH (charlies:Person)
WHERE charlies.name = 'Charlie Sheen'
RETURN charlies.name AS name,
       charlies.chauffeurName AS chauffeur
```

A new node with the name `Charlie Sheen` is created since not all properties matched those set to the pre-existing `Charlie Sheen` node:

Result

name	chauffeur
"Charlie Sheen"	"John Brown"
"Charlie Sheen"	"Michael Black"
Rows: 2	

`MERGE` *a node derived from an existing node property*

```
MATCH (person:Person)
MERGE (location:Location {name: person.bornIn})
RETURN person.name AS name,
       person.bornIn AS bornIn,
       location.name AS locationNode
ORDER BY name
```

Three `Location` nodes are created, with `name` values of `New York`, `Ohio`, and `New Jersey`. Although `MATCH` returns multiple `Person` nodes with `bornIn: 'New York'`, only one `New York` location node is created. Subsequent matches bind to it rather than creating duplicates.

Result

name	bornIn	locationNode
"Charlie Sheen"	"New York"	"New York"
"Charlie Sheen"	"New York"	"New York"
"Martin Sheen"	"Ohio"	"Ohio"
"Michael Douglas"	"New Jersey"	"New Jersey"
"Oliver Stone"	"New York"	"New York"
"Rob Reiner"	"New York"	"New York"
Rows: 6		

`MERGE` can be used with preceding `MATCH` and `MERGE` clauses to create a relationship between two bound nodes.

`MERGE` *a relationship between two existing nodes*

```
MATCH (person:Person)
MERGE (location:Location {name: person.bornIn})
MERGE (person)-[r:BORN_IN]->(location)
RETURN startNode(r).name AS name,
       type(r) AS newRel,
       endNode(r).name AS location
ORDER BY name
```

The second `MERGE` creates a `BORN_IN` relationship between each person and their corresponding `Location` node.

Result

name	newRel	location
"Charlie Sheen"	"BORN_IN"	"New York"
"Charlie Sheen"	"BORN_IN"	"New York"
"Martin Sheen"	"BORN_IN"	"Ohio"
"Michael Douglas"	"BORN_IN"	"New Jersey"
"Oliver Stone"	"BORN_IN"	"New York"
"Rob Reiner"	"BORN_IN"	"New York"
Rows: 6		

`MERGE` can also create both a new node and a relationship in a single clause:

`MERGE` *a new node and relationship connected to an existing node*

```
MATCH (person:Person)
MERGE (person)-[r:HAS_CHAUFFEUR]->(chauffeur:Chauffeur {name: person.chauffeurName})
REMOVE person.chauffeurName
RETURN startNode(r) AS startNode,
       type(r) AS newRel,
       endNode(r).name AS newChauffeurNode
```

```
ORDER BY newChauffeurNode
```

As the graph contains no `Chauffeur` nodes and no `HAS_CHAUFFEUR` relationships, `MERGE` creates six `Chauffeur` nodes, each with a `name` corresponding to the matched `Person` node's `chauffeurName` property, and a `HAS_CHAUFFEUR` relationship between each pair.

This differs from the `Location` example above, where a preceding `MERGE` bound the `Location` nodes before the relationship was created. Here, `MERGE` matches the full pattern including the relationship, which doesn't match the existing `Person` nodes and instead creates new `Chauffeur` nodes, even if `Person` nodes with the same names already exist.

Result

startNode	newRel	newChauffeurNode
(:Person {name: "Oliver Stone", bornIn: "New York"})	"HAS_CHAUFFEUR"	"Bill White"
(:Person {name: "Martin Sheen", bornIn: "Ohio"})	"HAS_CHAUFFEUR"	"Bob Brown"
(:Person {name: "Charlie Sheen", bornIn: "New York"})	"HAS_CHAUFFEUR"	"John Brown"
(:Person {name: "Michael Douglas", bornIn: "New Jersey"})	"HAS_CHAUFFEUR"	"John Brown"
(:Person {name: "Rob Reiner", bornIn: "New York"})	"HAS_CHAUFFEUR"	"Ted Green"
(:Person {name: "Charlie Sheen", bornIn: "New York"})	"HAS_CHAUFFEUR"	"Michael Black"

Rows: 6

Undirected relationships

`MERGE` can be used without specifying relationship direction. Cypher tries to match the relationship in both directions first; if no match is found, it creates the relationship left to right.

`MERGE` *an undirected relationship*

```
MATCH (charlie:Person {name: 'Oliver Stone'}),
      (oliver:Person {name: 'Michael Douglas'})
MERGE (charlie)-[r:KNOWS]-(oliver)
RETURN startNode(r).name AS from,
       type(r) AS relationship,
       endNode(r).name AS to
```

As no `KNOWS` relationship exists between `Oliver Stone` and `Michael Douglas`, `MERGE` creates one left to right.

Result

from	relationship	to
"Oliver Stone"	"KNOWS"	"Michael Douglas"
Rows: 1		

`ON MATCH` and `ON CREATE`

Use `ON CREATE` and `ON MATCH` to conditionally execute clauses depending on whether a node or relationship is created or matched.

`MERGE` *with* `ON CREATE`

```
MERGE (keanu:Person {name: 'Keanu Reeves', bornIn: 'Beirut', chauffeurName: 'Eric Brown'})
ON CREATE
  SET keanu.created = timestamp()
RETURN keanu.name AS name,
       keanu.created AS creationTimestamp
```

Result

name	creationTimestamp
"Keanu Reeves"	1655200898563
Rows: 1	

`MERGE` *with* `ON MATCH`

```
MERGE (person:Person)
ON MATCH
  SET person.updatedAt = date()
RETURN person.name AS name,
       person.updatedAt AS updatedAt
ORDER BY name
```

Result

name	updatedAt
"Charlie Sheen"	2026-06-16
"Charlie Sheen"	2026-06-16
"Martin Sheen"	2026-06-16
"Michael Douglas"	2026-06-16
"Oliver Stone"	2026-06-16
"Rob Reiner"	2026-06-16
Rows: 6	

`MERGE` *with* `ON CREATE` *and* `ON MATCH`

```
MERGE (keanu:Person {name: 'Keanu Reeves'})
ON CREATE
  SET keanu.created = timestamp()
```

```
ON MATCH
  SET keanu.lastSeen = timestamp()
RETURN keanu.name AS name,
       keanu.created AS createdAt,
       keanu.lastSeen AS lastSeen
```

Because the `Person` node named `Keanu Reeves` already exists, this query does not create a new node. Instead, it adds a timestamp on the `lastSeen` property.

Result

name	createdAt	lastSeen
"Keanu Reeves"	1655200902354	1674655352124
Rows: 1		

Using map parameters with `MERGE`

Unlike `CREATE`, `MERGE` does not accept a map parameter directly as a property map (e.g. `CREATE (n:Person $param)`). Each property must be specified explicitly:

Parameters

```
{
  "param": {
    "name": "Keanu Reeves",
    "bornIn": "Beirut",
    "chauffeurName": "Eric Brown"
  }
}
```

`MERGE` *with parameters*

```
MERGE (person:Person {name: $param.name, bornIn: $param.bornIn, chauffeurName: $param.chauffeurName})
RETURN person.name AS name,
       person.bornIn AS bornIn,
      person.chauffeurName AS chauffeur
```

Result

name	bornIn	chauffeur
"Keanu Reeves"	"Beirut"	"Eric Brown"
Rows: 1		

For more information on parameters, see Parameters.

`MERGE` using dynamic node labels and relationship types

Node labels and relationship types can be referenced dynamically in expressions, parameters, and variables when merging nodes and relationships. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

Syntax for merging nodes and relationships dynamically

```
MERGE (n:$(<expr>))
MERGE ()-[r:$(<expr>)]->()
```

The expression must evaluate to a `STRING NOT NULL | LIST<STRING NOT NULL> NOT NULL` value. Using a `LIST<STRING>` with more than one item when merging a relationship using dynamic relationship types will fail. This is because a relationship can only have exactly one type.

Parameters

```
{
  "nodeLabels": ["Person", "Director"],
  "relType": "DIRECTED",
  "movies": ["Ladybird", "Little Women", "Barbie"]
}
```

`MERGE` *nodes and relationships using dynamic node labels and relationship types*

```
MERGE (greta:$($nodeLabels) {name: 'Greta Gerwig'})
WITH greta
UNWIND $movies AS movieTitle
MERGE (greta)-[rel:$($relType)]->(m:Movie {title: movieTitle})
RETURN greta.name AS name, labels(greta) AS labels, type(rel) AS relType, collect(m.title) AS movies
```

Result

name	labels	relType	movies
"Greta Gerwig"	["Person", "Director"]	"DIRECTED"	["Ladybird", "Little Women", "Barbie"]
Rows: 1			

Performance caveats

`MERGE` queries that use dynamic values may not perform as well as those with static values. Neo4j is actively working to improve the performance of these queries. The table below outlines performance caveats for specific Neo4j versions.

Neo4j versions and performance caveats

Neo4j versions	Performance caveat
5.26 — 2025.07	The Cypher planner is not able to leverage indexes with index scans or seeks and must instead utilize the `AllNodesScan` operator, which reads all nodes from the node store and is therefore more costly.
2025.08 — 2025.10	The Cypher planner is able to leverage token lookup indexes when matching node labels and relationship types dynamically. This is enabled by the introduction of three new query plan operators: `DynamicLabelNodeLookup`, `DynamicDirectedRelationshipTypeLookup`, and `DynamicUndirectedRelationshipTypeLookup`. It is not, however, able to use indexes on property values. For example, `MERGE (n:$(Label) {foo: bar})` will not use any indexes on `n.foo` but can use a `DynamicLabelNodeLookup` on `$(label)`.

Neo4j versions	Performance caveat
2025.11 — current	The Cypher planner is able to leverage indexes on property values, however: • It only supports exact seeks on range indexes (no full text or spatial). • The index order cannot be leveraged, so the planner must insert separate ordering if required later on in the query. • Parallel runtime seeks and scans are single-threaded.

Using `MERGE` with constraints

Constraints on identifying properties (such as `id`) when using `MERGE` prevents duplicating data that differ in unintended ways. They also protect against duplicate creation under concurrent loads, where `MERGE` alone only guarantees the existence of the pattern, not its uniqueness.

Enforce property existence and uniqueness with key constraints

Enterprise Edition

There are two ways to ensure the uniqueness of a property value in a database: property uniqueness constraints and key constraints. Both are backed by indexes, which provide significant performance improvements. Key constraints are the preferred option, however, since they both guarantee the existence and uniqueness of a property.

Create key constraint

```
CREATE CONSTRAINT FOR (n:Product) REQUIRE n.id IS NODE KEY
```

`MERGE` *a node that complies with key constraint*

```
MERGE (n:Product {id: 12, category: 'Electronics', type: 'Laptop'})
```

A subsequent `MERGE` with the same `id` but different properties will fail rather than create a duplicate:

`MERGE` *a node that violates key constraint*

```
MERGE (n:Product {id: 12, category: 'Computing', type: 'Laptop'})
```

GQLSTATUS error chain

22N80: error: data exception - index entry conflict. Index entry conflict: Node(23) already exists with label `Product` and property `id` = 12.

22N79: error: data exception - property uniqueness constraint violated. Property uniqueness constraint violated: Node(23) already exists with label `Product` and property `id` = 12.

Similarly, an attempt to `MERGE` a node with a missing `id` property would fail due to the key constraint:

`MERGE` *a node that violates key constraint*

```
MERGE (n:Product {category: 'Computing', type: 'Laptop'})
```

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. NODE (24) with label 'Product' must have the following properties: `id`.

Enforce property value types with property type constraints

Enterprise Edition

Property type constraints are useful to enforce type correctness when merging data.

Create property type constraint

```
CREATE CONSTRAINT FOR (n:Product) REQUIRE n.id IS :: INTEGER
```

`MERGE` *a node that complies with property type constraint*

```
MERGE (n:Product {id: 13, category: 'Electronics', type: 'Laptop'})
```

`MERGE` *a new node that violates property type constraint*

```
MERGE (n:Product {id: "13", category: 'Electronics', type: 'Laptop'})
```

GQLSTATUS error chain

22N78: error: data exception - property type verification failed. NODE (25) with label 'Product' must have the property `id` with value type INTEGER.

Cypher offers the following functions to cast data while merging:

- `toBoolean()` / `toBooleanOrNull()`
- `toFloat()` / `toFloatOrNull()`
- `toInteger()` / `toIntegerOrNull()`
- `toString()`/ `toStringOrNull()`
- `date()`, `datetime()`, `localdatetime()`, `time()`, `duration()`
- `toBooleanList()`, `toFloatList()`, `toIntegerList()`, `toStringList()`
- `vector()`

`MERGE` *a node that casts data type and complies with property type constraint*

```
MERGE (n:Product {id: toInteger("14"), category: 'Electronics', type: 'Laptop'})
```

Concurrent relationship merges

`MERGE` *a relationship between two previously bound nodes*

```
MATCH (keanu:Person { name: 'Keanu Reeves' })
MATCH (charlie:Person { name: 'Charlie Sheen' })
MERGE (keanu)-[:KNOWS]->(charlie)
```

If the above query is run sequentially, `MERGE` creates the relationship on the first execution and matches it on any subsequent run. If the same query is run concurrently, Neo4j preserves this behavior by acquiring exclusive locks on both end nodes when no match is found, preventing two transactions from creating the relationship simultaneously. After locking, `MERGE` performs a second `MATCH` to account for any race condition between the initial failed match and lock acquisition.

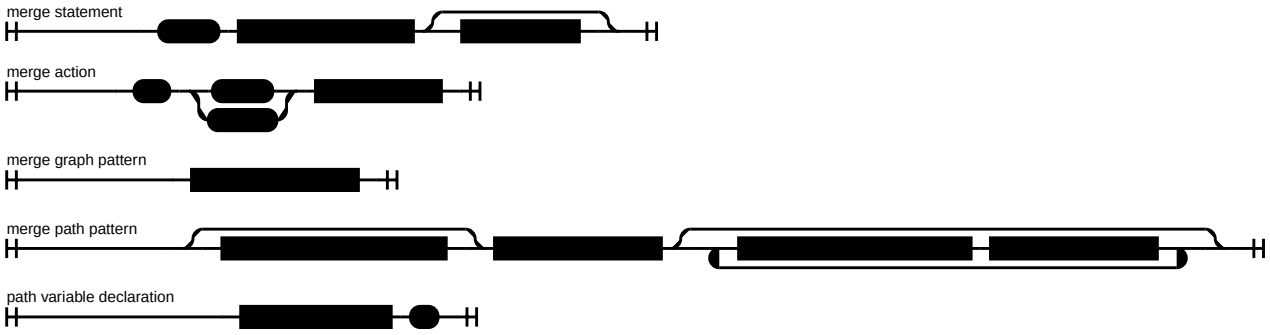
This serialization guarantee for concurrent updates does not rely on a uniqueness constraint. Cypher has no constraint that limits the number of relationships of a given type between two nodes, so the guarantee is provided by the locking behavior used by `MERGE`.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



OPTIONAL MATCH

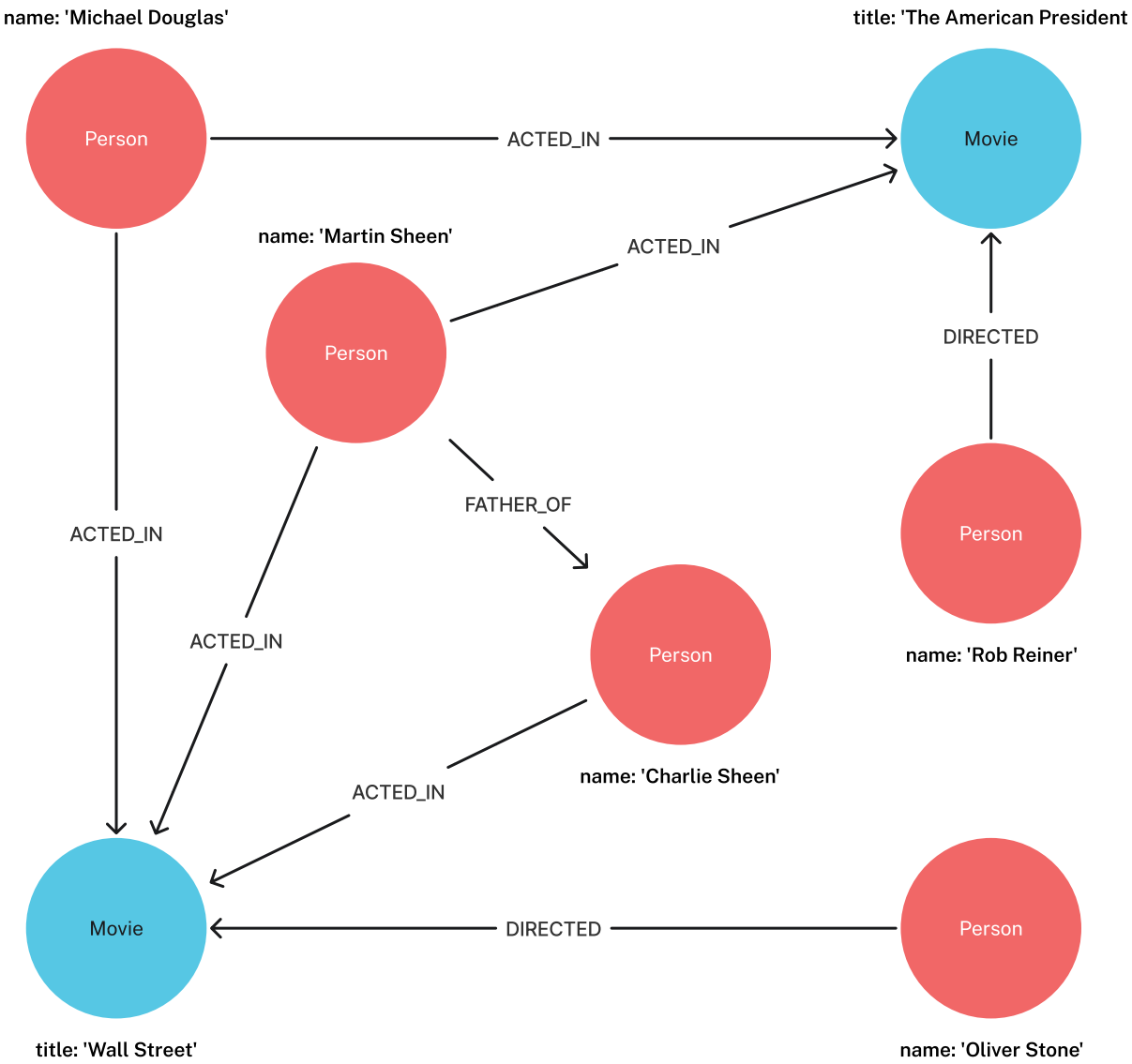
Introduction

`OPTIONAL MATCH` matches patterns against a graph database, just as `MATCH` does. The difference is that if no matches are found, `OPTIONAL MATCH` will use a `null` for missing parts of the pattern. `OPTIONAL MATCH` could therefore be considered the Cypher equivalent of the outer join in SQL.

When using `OPTIONAL MATCH`, either the whole pattern is matched, or nothing is matched. The `WHERE` clause is part of the pattern description, and its predicates will be considered while looking for matches, not after. This matters especially in the case of multiple (`OPTIONAL`) `MATCH` clauses, where it is crucial to put `WHERE` together with the `MATCH` it belongs to.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE
  (charlie:Person {name: 'Charlie Sheen'}),
  (martin:Person {name: 'Martin Sheen'}),
  (michael:Person {name: 'Michael Douglas'}),
  (oliver:Person {name: 'Oliver Stone'}),
  (rob:Person {name: 'Rob Reiner'}),
  (wallStreet:Movie {title: 'Wall Street'}),
  (charlie)-[:ACTED_IN]->(wallStreet),
  (martin)-[:ACTED_IN]->(wallStreet),
  (michael)-[:ACTED_IN]->(wallStreet),
  (oliver)-[:DIRECTED]->(wallStreet),
  (thePresident:Movie {title: 'The American President'}),
  (martin)-[:ACTED_IN]->(thePresident),
  (michael)-[:ACTED_IN]->(thePresident),
  (rob)-[:DIRECTED]->(thePresident),
  (martin)-[:FATHER_OF]->(charlie)
```

OPTIONAL MATCH in more detail

Like SQL, Cypher queries are constructed using various clauses which are chained together to feed intermediate results between each other. For example, the matching variables from one `MATCH` clause will

provide the context in which the next clause exists. However, there are two important differences between Neo4j and SQL which helps to explain `OPTIONAL MATCH` further.

1. While it is both possible and advised to enforce partial schemas using indexes and constraints, Neo4j offers a greater degree of schema flexibility than a relational database. Nodes and relationships in a Neo4j database do not have to have a specific property set to them because other nodes or relationships in the same graph have that property (unless there is a property existence constraint created on the specific property).
2. Queries in Cypher are run as pipelines. If a clause returns no results, it will effectively end the query as subsequent clauses will have no data to execute upon.

For example, the following query returns no results:

```
MATCH (a:Person {name: 'Martin Sheen'})
MATCH (a)-[r:DIRECTED]->()
RETURN a.name, r
```

```
(no changes, no records)
```

This is because the second `MATCH` clause returns no data (there are no `DIRECTED` relationships connected to `Martin Sheen` in the graph) to pass on to the `RETURN` clause.

However, replacing the second `MATCH` clause with `OPTIONAL MATCH` does return results. This is because, unlike `MATCH`, `OPTIONAL MATCH` enables the value `null` to be passed between clauses.

```
MATCH (p:Person {name: 'Martin Sheen'})
OPTIONAL MATCH (p)-[r:DIRECTED]->()
RETURN p.name, r
```

Result

p.name	r
"Martin Sheen"	<null>
Rows: 1	

`OPTIONAL MATCH` can therefore be used to check graphs for missing as well as existing values, and to pass on rows without any data to subsequent clauses in a query.

Optional relationships

If the existence of a relationship is optional, use the `OPTIONAL MATCH` clause. If the relationship exists, it is returned. If it does not, `null` is returned in its place.

```
MATCH (a:Movie {title: 'Wall Street'})
OPTIONAL MATCH (a)-->(x)
RETURN x
```

Returns `null`, since the `Movie` node `Wall Street` has no outgoing relationships.

Result

x
<null>
Rows: 1

On the other hand, the following query does not return `null` since the `Person` node `Charlie Sheen` has one outgoing relationship.

```
MATCH (a:Person {name: 'Charlie Sheen'})
OPTIONAL MATCH (a)-->(x)
RETURN x
```

Result

x
{"title":"Wall Street"}
Rows: 2

Properties on optional elements

If the existence of a property is optional, use the `OPTIONAL MATCH` clause. `null` will be returned if the specified property does not exist.

```
MATCH (a:Movie {title: 'Wall Street'})
OPTIONAL MATCH (a)-->(x)
RETURN x, x.name
```

Returns the element `x` (`null` in this query), and `null` for its `name` property, because the `Movie` node `Wall Street` has no outgoing relationships.

Result

x	x.name
<null>	<null>
Rows: 1	

The following query only returns `null` for the nodes which lack a `name` property.

```
MATCH (a:Person {name: 'Martin Sheen'})
OPTIONAL MATCH (a)-->(x)
RETURN x, x.name
```

Result

x	x.name
{"title":"Wall Street"}	<null>
{"name":"Charlie Sheen"}	"Charlie Sheen"
{"title":"The American President"}	<null>
Rows: 3	

Optional typed and named relationship

It is also possible to look for specific relationship types when using `OPTIONAL MATCH`:

```
MATCH (a:Movie {title: 'Wall Street'})
OPTIONAL MATCH (a)-[r:ACTED_IN]->()
RETURN a.title, r
```

This returns the title of the `Movie` node `Wall Street`, and since this node has no outgoing `ACTED_IN` relationships, `null` is returned for the relationship denoted by the variable `r`.

Result

a.title	r
"Wall Street"	<null>
Rows: 1	

However, the following query does not return `null` since it is looking for incoming relationships of the type `ACTED_IN` to the `Movie` node `Wall Street`.

```
MATCH (a:Movie {title: 'Wall Street'})
OPTIONAL MATCH (x)-[r:ACTED_IN]->(a)
RETURN a.title, x.name, type(r)
```

Result

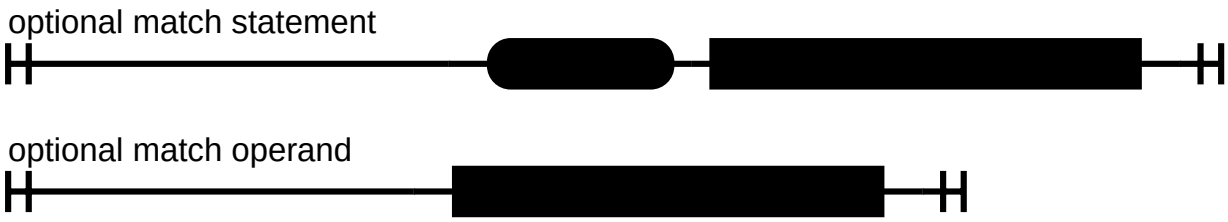
a.title	x.name	type(r)
"Wall Street"	"Michael Douglas"	"ACTED_IN"
"Wall Street"	"Martin Sheen"	"ACTED_IN"
"Wall Street"	"Charlie Sheen"	"ACTED_IN"
Rows: 3		

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



ORDER BY

`ORDER BY` is a subclause that determines how the results of a `RETURN` or `WITH` clause are ordered. If the query also contains a `GROUP BY` clause, `ORDER BY` can be used to order results after grouping, see `GROUP BY` and `ORDER BY`. It can also be used as a standalone clause, either on its own or in combination with `SKIP` / `OFFSET` or `LIMIT`.

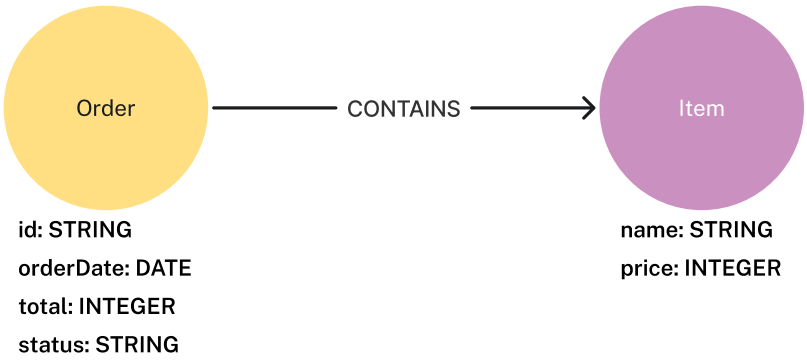
`ORDER BY` defaults to sorting results in an ascending order, though it can be modified to sort results in a descending order.

`ORDER BY` relies on comparisons to sort the output (see Equality, ordering, and comparison of value types for more details). You can sort on different values, such as node or relationship properties, IDs, or the result of expressions.

> Unless `ORDER BY` is used, Neo4j does not guarantee the row order of a query result.

Example graph

A graph with the following schema is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (o1:Order {id: 'ORD-001', orderDate: datetime('2024-05-01T10:00:00'), total: 550, status:
'shipped'}),
       (o2:Order {id: 'ORD-002', orderDate: datetime('2024-05-02T14:30:00'), total: 1000, status:
'pending'}),
       (o3:Order {id: 'ORD-003', orderDate: datetime('2024-05-03T09:15:00'), total: 550, status:
'pending'}),
       (o4:Order {id: 'ORD-004', orderDate: datetime('2024-05-04T12:45:00'), total: 200}),
       (o5:Order {id: 'ORD-005', orderDate: datetime('2024-05-05T15:00:00'), total: 800, status:
'shipped'}),

       (i1:Item {name: 'Phone', price: 500}),
       (i2:Item {name: 'Laptop', price: 1000}),
```

```
      (i3:Item {name: 'Headphones', price: 250}),
      (i4:Item {name: 'Charger', price: 50}),
      (i5:Item {name: 'Keyboard', price: 200}),

      (o1)-[:CONTAINS]->(i1),
      (o1)-[:CONTAINS]->(i4),
      (o2)-[:CONTAINS]->(i2),
      (o3)-[:CONTAINS]->(i1),
      (o3)-[:CONTAINS]->(i4),
      (o4)-[:CONTAINS]->(i5),
      (o5)-[:CONTAINS]->(i1),
      (o5)-[:CONTAINS]->(i3),
      (o5)-[:CONTAINS]->(i4)
```

Basic examples

Example 54. Order by property values

`ORDER BY` can be used to sort the result by property values.

Order by node property

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total AS total
  ORDER BY total
```

The nodes are returned, sorted by the value of the `total` properties in an ascending order.

Result

order	total
"ORD-004"	200
"ORD-001"	550
"ORD-003"	550
"ORD-005"	800
"ORD-002"	1000

Rows: 5

Example 55. Order by multiple property values

Order by multiple property values by listing two or more properties in the `ORDER BY` subclause. Cypher sorts by the first property, and if values are equal, it moves to the next property, and so on.

Order by multiple properties

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total AS total,
       o.orderDate AS orderDate
  ORDER BY total,
           orderDate
```

This returns the nodes, sorted first by their `total` property, and then, for equal values, by their `orderDate` property.

Result

order	total	orderDate
"ORD-004"	200	2024-05-04T12:45Z
"ORD-001"	550	2024-05-01T10:00Z
"ORD-003"	550	2024-05-03T09:15Z
"ORD-005"	800	2024-05-05T15:00Z
"ORD-002"	1000	2024-05-02T14:30Z

Rows: 5

Example 56. Order by ID

`ORDER BY` can be used to sort nodes or relationships by their ID (retrieved by either the `elementId()` or `id()` functions).

Order by element ID

```
MATCH (o:Order)
RETURN o.id AS order,
       elementId(o) AS elementId
  ORDER BY elementId
```

Result

order	elementId
"ORD-001"	"4:9350eddd-5a35-413d-8684-708b1da35d23:0"
"ORD-002"	"4:9350eddd-5a35-413d-8684-708b1da35d23:1"
"ORD-003"	"4:9350eddd-5a35-413d-8684-708b1da35d23:2"
"ORD-004"	"4:9350eddd-5a35-413d-8684-708b1da35d23:3"
"ORD-005"	"4:9350eddd-5a35-413d-8684-708b1da35d23:4"

Rows: 5

Neo4j reuses its internal IDs when nodes and relationships are deleted. Applications relying on internal Neo4j IDs are, as a result, brittle and can be inaccurate. It is recommended to use application-generated IDs instead.

Example 57. Order by expressions

`ORDER BY` can be used to sort according to the results of an expression. The below query calculates a

10% discount on each order's `total` property value, and then orders the results by the discounted total.

Order by an expression result

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total * 0.9 AS discountedTotal
  ORDER BY discountedTotal
```

Result

order	discountedTotal
"ORD-004"	180.0
"ORD-001"	495.0
"ORD-003"	495.0
"ORD-005"	720.0
"ORD-002"	900.0
Rows: 5	

This next query counts the number of items contained in each order and then orders the results by the item count.

Order by an expression result

```
MATCH (o:Order)
RETURN o.id AS order,
      COUNT { (o)-[:CONTAINS]->(:Item) } AS itemCount
  ORDER BY itemCount
```

Result

order	itemCount
"ORD-002"	1
"ORD-004"	1
"ORD-001"	2
"ORD-003"	2
"ORD-005"	3
Rows: 5	

Order by values not in the result

`ORDER BY` can sort by values that are not included in the result set. That is, the sort key does not need to be part of the preceding `RETURN` or `WITH` clause. For example, the query below sorts orders based on how many items they contain, even though that count is not returned.

Order by values not in the returned results

```
MATCH (o:Order)
RETURN o.id AS order
  ORDER BY COUNT { (o)-[:CONTAINS]->(:Item) }
```

Result

order
"ORD-002"
"ORD-004"
"ORD-001"
"ORD-003"
"ORD-005"
Rows: 5

Ascending and descending order

`ORDER BY` sorts results in an ascending order by default. To explicitly sort results in an ascending order, append `ASC[ENDING]`.

Explicitly sort results in an ascending order

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total AS total
  ORDER BY total ASC
```

Result

order	total
"ORD-004"	200
"ORD-001"	550
"ORD-003"	550
"ORD-005"	800
"ORD-002"	1000
Rows: 5	

To sort results in a descending order, append `DESC[ENDING]`.

Sort results in a descending order

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total AS total
  ORDER BY total DESC
```

Result

order	total
"ORD-002"	1000
"ORD-005"	800
"ORD-001"	550
"ORD-003"	550
"ORD-004"	200

Rows: 5

`ORDER BY` can combine ascending and descending ordering. In the example below, results are sorted first by `total` values in descending order and then by `orderDate` values in ascending order.

Combine ascending and descending result ordering

```
MATCH (o:Order)
RETURN o.id AS order,
       o.total AS total,
       o.orderDate AS orderDate
  ORDER BY total DESC,
           orderDate ASC
```

Result

order	total	orderDate
"ORD-002"	1000	2024-05-02T14:30Z
"ORD-005"	800	2024-05-05T15:00Z
"ORD-001"	550	2024-05-01T10:00Z
"ORD-003"	550	2024-05-03T09:15Z
"ORD-004"	200	2024-05-04T12:45Z

Rows: 5

ORDER BY and pattern matching

`ORDER BY` can be used to sort results before continuing with additional pattern matching. In the example below, it is combined with the `LIMIT` to first sort `Order` nodes by their `orderDate` property values, limit the result to the most recent `Order`, and then match any connected `Item` nodes. Also note that `ORDER BY` and `LIMIT` are used as standalone clauses and not as subclauses in this example.

Find the items contained in the most recently placed order

```
MATCH (o:Order)
ORDER BY o.orderDate DESC
LIMIT 1
MATCH (o)-[:CONTAINS]->(i:Item)
RETURN o.id AS order,
       o.total,
```

```
       collect(i.name) AS items
```

Result

order	total	items
"ORD-005"	800	["Phone", "Headphones", "Charger"]
Rows: 1		

Null values

When sorting, `null` values appear last in ascending order and first in descending order.

Sort on null property

```
MATCH (o:Order)
RETURN o.id AS order,
       o.status AS status
  ORDER BY status DESC
```

Result

order	status
"ORD-004"	null
"ORD-001"	"shipped"
"ORD-005"	"shipped"
"ORD-002"	"pending"
"ORD-003"	"pending"
Rows: 5	

ORDER BY and the WITH clause

When `ORDER BY` is present on a `WITH` clause, the immediately following clause will receive records in the specified order. This guaranteed order is useful for operations that rely on the sequence in which values are processed. For example, appending `ORDER BY` to a `WITH` clause can be used to control the order of items in the list produced by the `collect()` aggregating function. The `MERGE` and `SET` clauses also have ordering dependencies which can be controlled this way.

The below example uses `WITH` and `ORDER BY` to sort `Item` nodes by their `price` property, then the `collect()` in the subsequent `RETURN` clause builds an ordered list per order based on that sort.

`WITH`, `ORDER BY`, and `collect()`

```
MATCH (o:Order)-[:CONTAINS]->(i:Item)
WITH o, i
  ORDER BY i.price DESC
RETURN o.id AS order,
       collect(i.name || " ($" || toString(i.price) || ")") AS orderedListOfItems
```

Result

order	orderedListOfItems
"ORD-002"	["Laptop ($1000)"]
"ORD-001"	["Phone ($500)", "Charger ($50)"]
"ORD-003"	["Phone ($500)", "Charger ($50)"]
"ORD-005"	["Phone ($500)", "Headphones ($250)", "Charger ($50)"]
"ORD-004"	["Keyboard ($200)"]
Rows: 5	

Ordering aggregated or DISTINCT results

The variables available to `ORDER BY` depend on whether or not the preceding `RETURN` or `WITH` clause performs an aggregation to combine results or uses `DISTINCT` to remove duplicates.

- If the `RETURN` or `WITH` is not aggregating values or using `DISTINCT`, then `ORDER BY` can reference any variables referenced in the preceding `RETURN` or `WITH` clause.

`ORDER BY` *following a* `WITH` *clause excluding aggregation or* `DISTINCT`

```
MATCH (o:Order)-[:CONTAINS]->(i:Item)
WITH o.id AS order,
     i.name AS item
  ORDER BY o.orderDate
RETURN order, item
```

- If the `RETURN` or `WITH` performs an aggregation or uses `DISTINCT` only the projected variables from either operation are available to `ORDER BY`. This is because these operations alter the number of rows produced by the clause and any variables not explicitly projected are discarded.

`ORDER BY` *following a* `WITH` *clause projecting an aggregated value*

```
MATCH (o:Order)-[:CONTAINS]->(i:Item)
WITH collect(o.id) AS orders,
     i.name AS items
  ORDER BY o.orderDate
RETURN orders, items
```

GQLSTATUS error chain

42N44: error: syntax error or access rule violation - inaccessible variable. It is not possible to access the variable `o` declared before the WITH clause when using `DISTINCT` or an aggregation.

42001: error: syntax error or access rule violation - invalid syntax

ORDER BY and indexes

The performance of Cypher queries using `ORDER BY` on node properties can be influenced by the existence and use of an index for finding the nodes. If the index can provide the nodes in the order requested in the

query, Cypher can avoid the use of an expensive `Sort` operation. Read more about this capability in Range index-backed ORDER BY.

Using `ORDER BY` as a standalone clause

`ORDER BY` can be used as a standalone clause, or in conjunction with `SKIP`/`OFFSET` or `LIMIT`.

Standalone use of `ORDER BY`

```
MATCH (i:Item)
ORDER BY i.price
RETURN collect(i.name || " ($" || toString(i.price) || ")") AS orderedPriceList
```

Result

orderedPriceList
["Charger ($50)", "Keyboard ($200)", "Headphones ($250)", "Phone ($500)", "Laptop ($1000)"]
Rows: 1

`ORDER BY` used in conjunction with `SKIP` and `LIMIT`

```
MATCH (i:Item)
ORDER BY i.price DESC
SKIP 1
LIMIT 1
RETURN i.name AS secondMostExpensiveItem,
       i.price AS price
```

Result

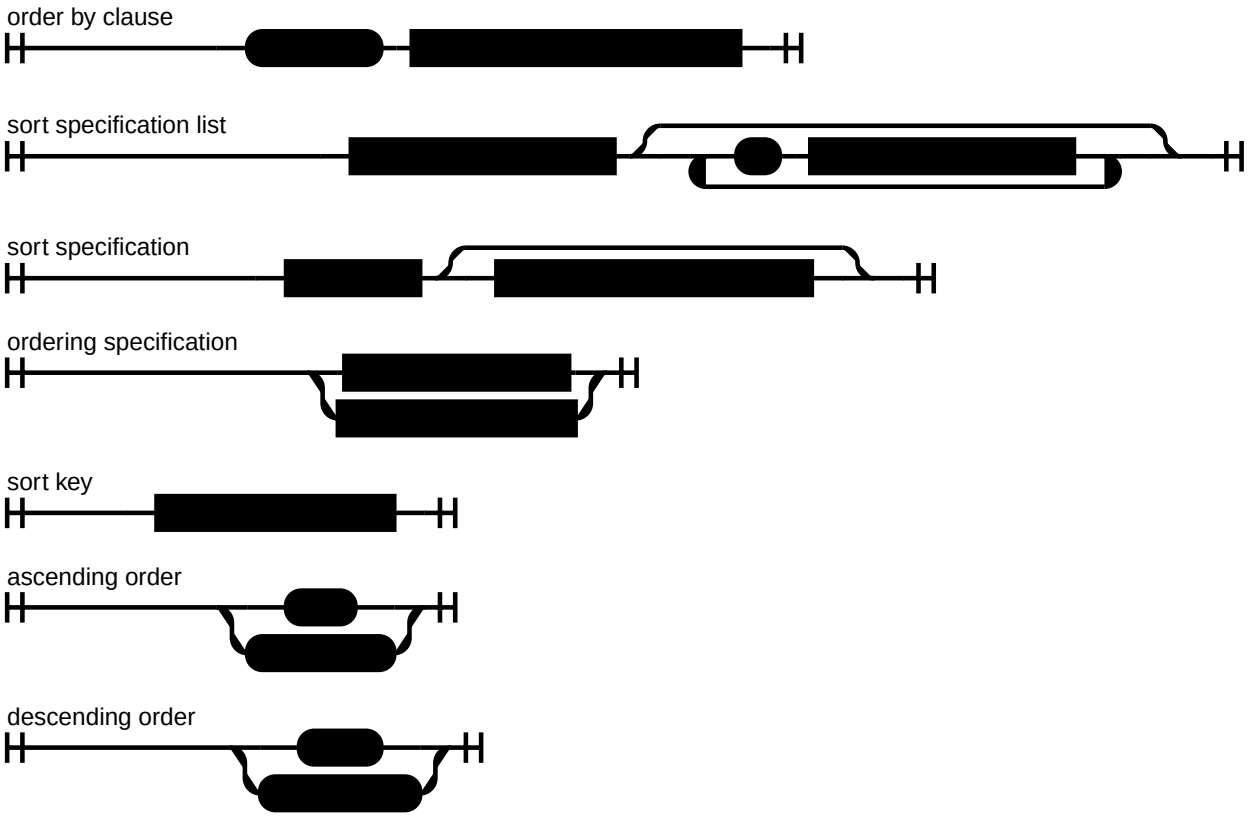
secondMostExpensiveItem	price
"Phone"	500
Rows: 1	

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



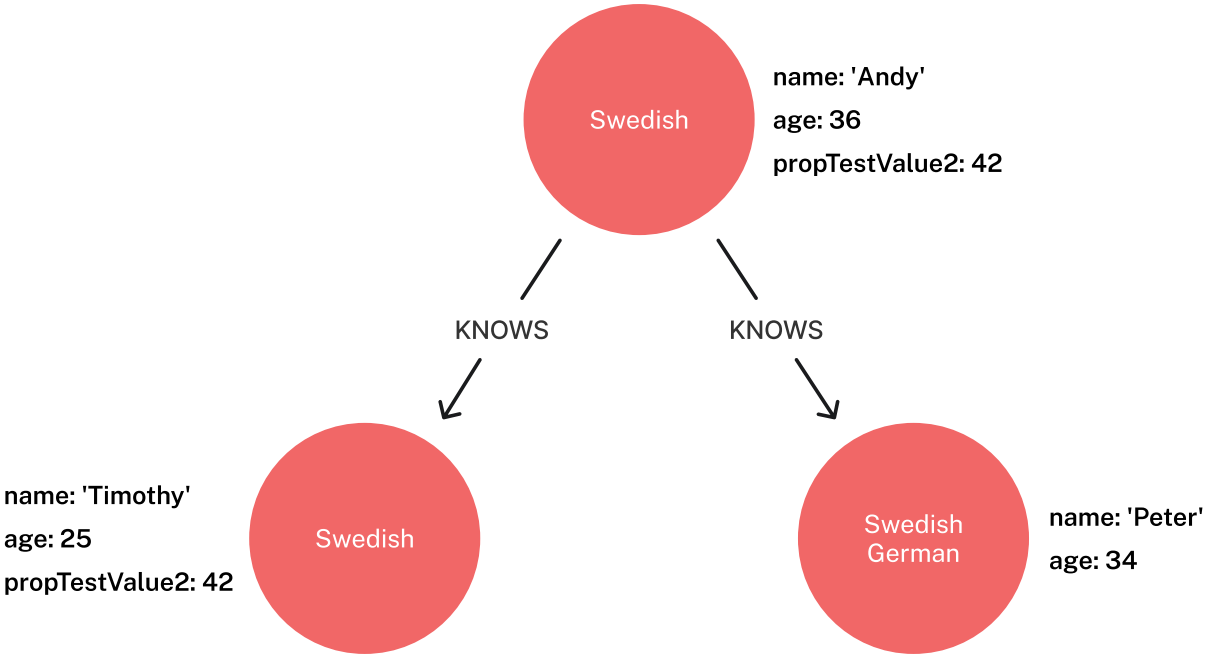
REMOVE

The `REMOVE` clause is used to remove properties from nodes and relationships, and to remove labels from nodes. To delete nodes and relationships, use the `DELETE` clause.

> Removing labels from a node is an idempotent operation: if you try to remove a label from a node that does not have that label on it, nothing happens. The query statistics will tell you if something needed to be done or not.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (a:Swedish {name: 'Andy', age: 36, propTestValue1: 42}),
  (t:Swedish {name: 'Timothy', age: 25, propTestValue2: 42}),
  (p:German:Swedish {name: 'Peter', age: 34}),
  (a)-[:KNOWS]->(t),
  (a)-[:KNOWS]->(p)
```

Remove a property

Neo4j doesn't allow storing `null` in properties. Instead, if no value exists, the property is just not there. So, `REMOVE` is used to remove a property value from a node or a relationship.

Query

```
MATCH (a {name: 'Andy'})
REMOVE a.age
RETURN a.name, a.age
```

The node is returned, and no property `age` exists on it.

Result

a.name	a.age
"Andy"	<null>
Rows: 1 Properties set: 1	

Remove all properties

`REMOVE` cannot be used to remove all existing properties from a node or relationship. Instead, using `SET` with `=` and an empty map as the right operand will clear all properties from the node or relationship.

Dynamically remove a property

`REMOVE` can be used to remove a property on a node or relationship even when the property key name is not statically known. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

```
REMOVE n[key]
```

The dynamically calculated key must evaluate to a `STRING` value. This query creates a copy of every property on the nodes:

Query

```
MATCH (n)
WITH n, [k IN keys(n) WHERE k CONTAINS "Test" | k] as propertyKeys
FOREACH (i IN propertyKeys | REMOVE n[i])
RETURN n.name, keys(n);
```

The keys() function retrieves all property keys of the matched nodes, and a list comprehension filters these keys to include only those that contain the substring "Test", assigning the resulting list to the variable `propertyKeys`.

The `FOREACH` clause iterates over each key in the `propertyKeys` list and removes the corresponding property using the `REMOVE` clause.

All properties with the word "Test" in them are removed:

Result

n.name	keys(n)
"Andy"	["name", "age"]
"Timothy"	["name", "age"]
"Peter"	["name", "age"]

Rows: 3

Properties set: 2

Remove a label from a node

To remove labels, you use `REMOVE`.

Query

```
MATCH (n {name: 'Peter'})
REMOVE n:German
RETURN n.name, labels(n)
```

Result

n.name	labels(n)
"Peter"	["Swedish"]

Rows: 1

Labels removed: 1

Dynamically remove a node label

`REMOVE` can be used to remove a label on a node even when the label is not statically known.

```
MATCH (n)
REMOVE n:$(expr)
```

The expression must evaluate to a `STRING NOT NULL | LIST<STRING NOT NULL> NOT NULL` value.

Query

```
MATCH (n {name: 'Peter'})
UNWIND labels(n) AS label
REMOVE n:$(label)
RETURN n.name, labels(n)
```

`UNWIND` is used here to transform the list of labels from the `labels()` function into separate rows, allowing subsequent operations to be performed on each label individually.

Result

n.name	labels(n)
"Peter"	[]

Rows: 1

Labels removed: 2

Remove multiple labels from a node

To remove multiple labels, you use `REMOVE`.

Query

```
MATCH (n {name: 'Peter'})
REMOVE n:German:Swedish
RETURN n.name, labels(n)
```

Result

n.name	labels(n)
"Peter"	[]

Rows: 1

Labels removed: 2

Remove multiple labels dynamically from a node

It is possible to remove multiple labels dynamically using a `LIST<STRING>` and/or by chaining them separately with a `:`:

Query

```
MATCH (n {name: 'Peter'})
REMOVE n:$(labels(n))
RETURN n.name, labels(n)
```

Result

n.name	labels(n)
"Peter"	[]

Rows: 1

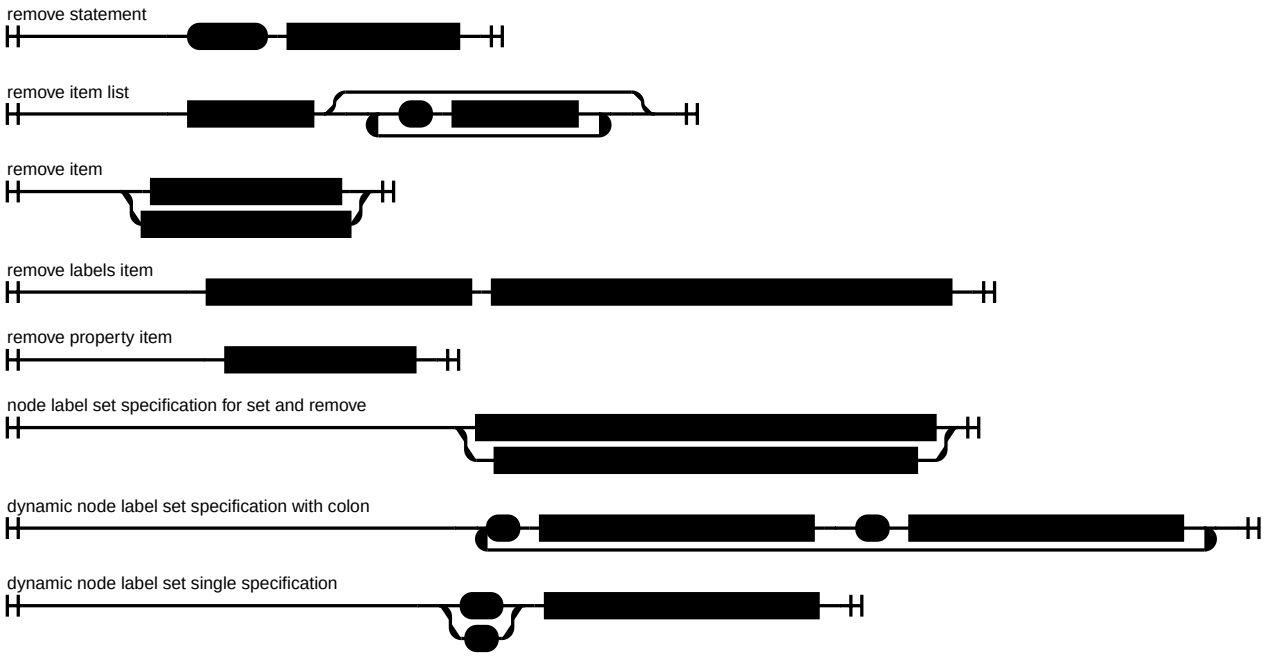
Labels removed: 2

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



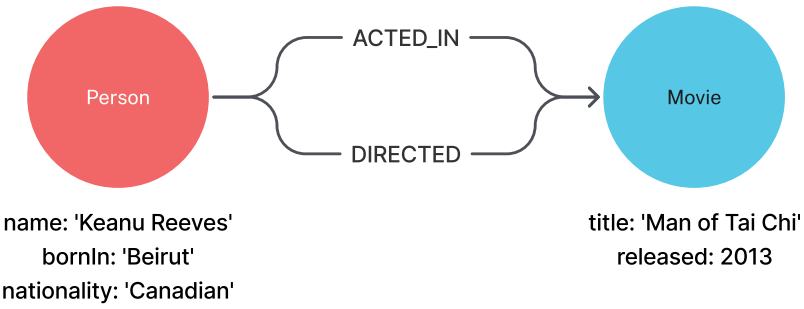
RETURN

Introduction

The `RETURN` clause defines the parts of a pattern (nodes, relationships, and/or properties) to be included in the query result.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database.

```
CREATE
  (keanu:Person {name: 'Keanu Reeves', bornIn: 'Beirut', nationality: 'Canadian'}),
  (taiChi:Movie {title: 'Man of Tai Chi', released: 2013}),
  (keanu)-[:ACTED_IN]->(taiChi),
  (keanu)-[:DIRECTED]->(taiChi)
```

Return nodes

To return a node, list it in the `RETURN` clause:

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})
RETURN p
```

Result

p
{"bornIn":"Beirut","nationality":"Canadian","name":"Keanu Reeves"}
Rows: 1

Return relationships

To return a relationship type, list it in the `RETURN` clause:

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})-[r:ACTED_IN]->(m)
RETURN type(r)
```

Result

type(r)
"ACTED_IN"
Rows: 1

Return property

To return a specific property, use the dot separator:

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})
RETURN p.bornIn
```

Result

p.bornIn
"Beirut"
Rows: 1

It is more performant to return properties rather than the full node or relationship.

Return all elements

To return all nodes, relationships and paths found in a query, use the `*` symbol:

Query

```
MATCH p = (keanu:Person {name: 'Keanu Reeves'})-[r]->(m)
RETURN *
```

This returns the two nodes, and the two possible paths between them.

Result

keanu	m	p	r
{"bornIn":"Beirut","nationality":"Canadian","name":"Keanu Reeves"}	{"title":"Man of Tai Chi","released":2013}	(:Person {bornIn: "Beirut",nationality: "Canadian",name: "Keanu Reeves"})-[:ACTED_IN]→(:Movie {title: "Man of Tai Chi",released: 2013})	{:ACTED_IN}

keanu	m	p	r
{"bornIn":"Beirut","nationality":"Canadian","name":"Keanu Reeves"}	{"title":"Man of Tai Chi","released":2013}	(:Person {bornIn: "Beirut",nationality: "Canadian",name: "Keanu Reeves"})-[:DIRECTED]→(:Movie {title: "Man of Tai Chi",released: 2013})	{:DIRECTED}
Rows: 1			

Variable with uncommon characters

To introduce a variable made up of characters not contained in the English alphabet, use `` ` `` to enclose the variable:

Query

```
MATCH (`/uncommon variable\`)
WHERE `/uncommon variable\`.name = 'Keanu Reeves'
RETURN `/uncommon variable\`.bornIn
```

The `bornIn` property of the node with the `name` property set to `'Keanu Reeves'` is returned:

Result

/uncommon variable\.bornIn
"Beirut"
Rows: 1

Column alias

Names of returned columns can be renamed using the `AS` operator:

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})
RETURN p.nationality AS citizenship
```

Returns the `nationality` property of `'Keanu Reeves'`, but the column is renamed to `citizenship`.

Result

citizenship
"Canadian"
Rows: 1

Optional properties

If the existence of a property is unknown, it can still be included in a `RETURN` clause. It will be treated as `null` if it is missing.

Query

```
MATCH (n)
RETURN n.bornIn
```

This example returns the `bornIn` properties for nodes that has that property, and `null` for those nodes missing the property.

Result

n.bornIn
"Beirut"
<null>
Rows: 2

Other expressions

Any expression can be used as a return item — literals, predicates, properties, functions, and so on.

Query

```
MATCH (m:Movie {title: 'Man of Tai Chi'})
RETURN m.released < 2012, "I'm a literal",[p=(m)--() | p] AS `(m)--()`
```

Returns a predicate, a literal and function call with a pattern expression parameter:

Result

m.released < 2012	"I'm a literal"	(m)--()
false	"I'm a literal"	[(:Movie {title: "Man of Tai Chi",released: 2013})<-[:DIRECTED]-(:Person {bornIn: "Beirut",nationality: "Canadian",name: "Keanu Reeves"}),(:Movie {title: "Man of Tai Chi",released: 2013})<-[:ACTED_IN]-(:Person {bornIn: "Beirut",nationality: "Canadian",name: "Keanu Reeves"})]
Rows: 1		

Unique results

`DISTINCT` retrieves only unique rows for the columns that have been selected for output.

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})-->(m)
RETURN DISTINCT m
```

The `Movie` node `'Man of Tai Chi'` is returned by the query, but only once (without the `DISTINCT` operator it would have been returned twice because there are two relationships going to it from `'Keanu Reeves'`):

Result

m
{"title":"Man of Tai Chi","released":2013}+
Rows: 1

RETURN ALL

Introduced in 2025.06

Returning all results can also be accomplished by explicitly including `ALL` in the `RETURN`. The `RETURN ALL` keyword was introduced as part of Cypher's GQL conformance, and using it is functionally the same as using simple `RETURN`.

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})-->(m)
RETURN ALL m
```

The same node is returned twice, as there are two relationships connecting to it from `'Keanu Reeves'`.

Result

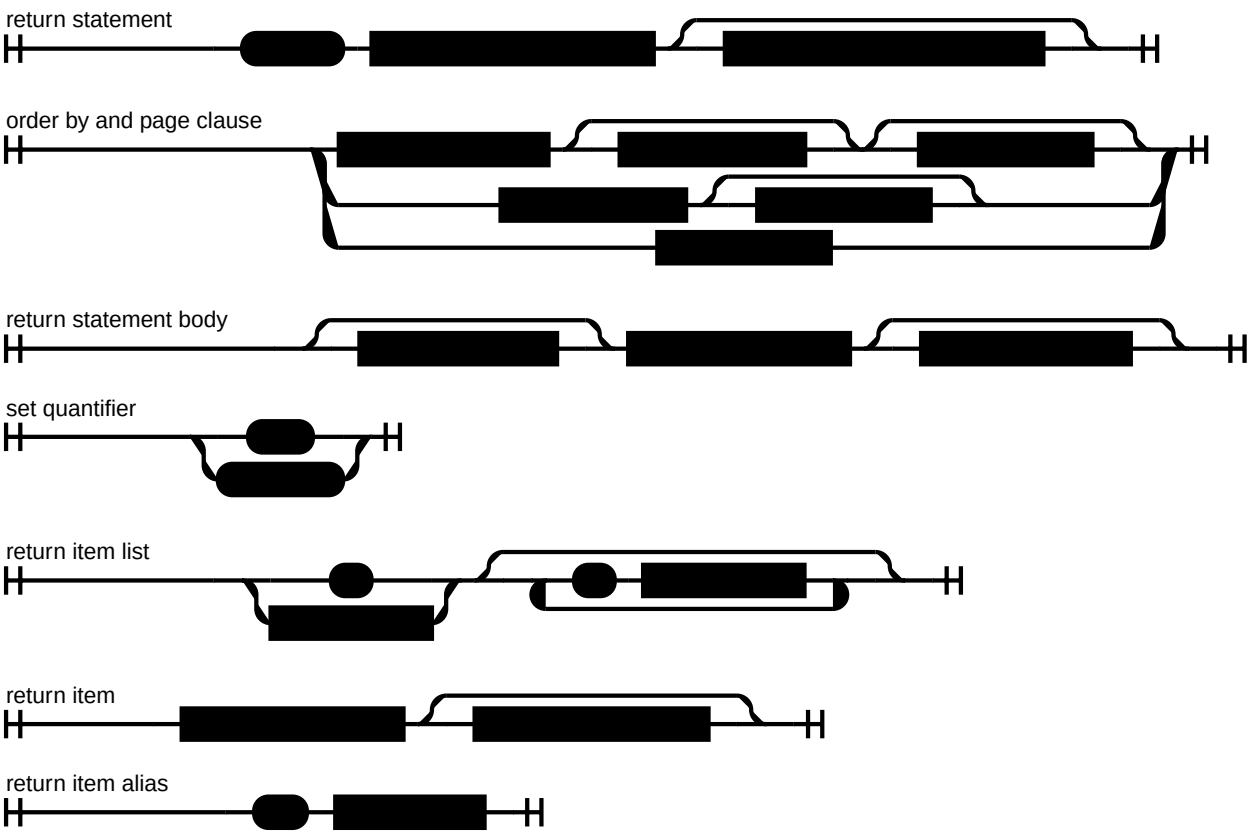
m
{"title":"Man of Tai Chi","released":2013}+
{"title":"Man of Tai Chi","released":2013}+
Rows: 1

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



SEARCH

The `SEARCH` clause is a subclause used within `MATCH` and `OPTIONAL MATCH` clauses to constrain their patterns using a Neo4j search index. Neo4j currently supports two types of search index: full-text indexes for lexical search and vector indexes for similarity search, commonly used for semantic search. See the search indexes documentation for details.

Vector index queries with `SEARCH` are available from Neo4j 2026.01, and full-text index queries from Neo4j 2026.09. Before using `SEARCH`, you must create a vector index or full-text index.

SEARCH clause template

The `SEARCH` clause has the following templates:

Example 58. `SEARCH` clause template for vector and full-text search

```
[OPTIONAL] MATCH pattern
  SEARCH binding_variable IN (
    VECTOR INDEX index_name
    FOR query_vector
    [WHERE ...]
    LIMIT top_k
  ) [SCORE AS score_alias]
```

The `SEARCH` clause constrains the `pattern` from the enclosing `MATCH` or `OPTIONAL MATCH` by only returning nodes or relationships included in the results of the vector index query. Below follows a short description of the different parts of the template:

- The `binding_variable` must be a node or relationship variable from the `pattern` of the enclosing `MATCH` or `OPTIONAL MATCH` clause.
- The `index_name` must be an identifier that matches the name of an existing vector index in the Neo4j database. The index must be a node vector index if the `binding_variable` refers to a node and a relationship vector index if the `binding_variable` refers to a relationship.
- The `query_vector` can be any expression that evaluates to a `VECTOR` or `LIST<INTEGER NOT NULL | FLOAT NOT NULL>`, for example a literal, a parameter or a property. The `SEARCH` clause returns a neighborhood of nodes or relationships based on the similarity between the `query_vector` and the corresponding property of those nodes or relationships.
- The optional `WHERE` subclause is only allowed in vector searches. It applies a filter to the vector index, and is a restricted version of the normal Cypher `WHERE` clause. For more information, see Predicates for vector search with filters.
- The `LIMIT` clause sets the number of approximate nearest neighbor nodes or relationships returned by the `SEARCH` clause. The `top_k` must be of type `INTEGER NOT NULL` and in the interval `0 <= top_k <= 2147483647` [1].
- The optional `SCORE` subclause makes the `SEARCH` clause return the similarity score for each node or relationship in addition to the node or relationship itself. In the result, the column for the similarity scores will be called `score_alias`.
- The `SEARCH` clause puts some restrictions on which `pattern` the enclosing `MATCH` or `OPTIONAL MATCH` clause is allowed to have. For more information, see Limitations of the `SEARCH` clause.

The `SCORE` value is the vector similarity score from the vector index query. Vector scores are meaningful within the result set from a vector query. When combining vector results with full-text results or other scored sources, rank each source independently rather than comparing raw score values directly. For a worked example, see Developer Guide → Hybrid search.

```
[OPTIONAL] MATCH pattern
  SEARCH binding_variable IN (
    FULLTEXT INDEX index_name
    FOR query_string [ WITH ANALYZER analyzer_string ]
    [ (SKIP | OFFSET) skip_integer ]
    LIMIT top_k
  ) [SCORE AS score_alias]
```

The `SEARCH` clause constrains the `pattern` from the enclosing `MATCH` or `OPTIONAL MATCH` by only returning nodes or relationships included in the results of the full-text index query. Below follows a short description of the different parts of the template:

- The `binding_variable` must be a node or relationship variable from the `pattern` of the enclosing `MATCH` or `OPTIONAL MATCH` clause.
- The `index_name` must be an identifier that matches the name of an existing full-text index in the Neo4j database, or a parameter that evaluates to the index name. The index must be a node full-text index if the `binding_variable` refers to a node and a relationship full-text index if the `binding_variable` refers to a relationship.
- The `query_string` can be any expression that evaluates to a `STRING`, for example a literal,

parameter, or property. The query string supports Lucene query syntax.

- The optional `WITH ANALYZER` part selects the analyzer to use for the query. The `analyzer_string` must evaluate to a `STRING NOT NULL`.
- The optional `SKIP` or `OFFSET` clause skips results before applying `LIMIT`. The `skip_integer` must evaluate to an `INTEGER NOT NULL` that is `0` or positive.
- The `LIMIT` clause sets the maximum number of full-text results returned by the `SEARCH` clause. The `top_k` must be of type `INTEGER NOT NULL` and in the interval `0 <= top_k <= 2147483647` [2].
- The optional `SCORE` subclause makes the `SEARCH` clause return the Lucene relevance score for each node or relationship in addition to the node or relationship itself. In the result, the column for the relevance scores will be called `score_alias`.
- Unlike vector search, full-text search does not support a `WHERE` subclause inside `SEARCH`. A `WHERE` subclause attached to the enclosing `MATCH` or `OPTIONAL MATCH` post-filters the full-text search results.
- The `SEARCH` clause puts some restrictions on which `pattern` the enclosing `MATCH` or `OPTIONAL MATCH` clause is allowed to have. For more information, see Limitations of the `SEARCH` clause.

The `SCORE` value is the Lucene relevance score from the full-text index query. Full-text scores are meaningful within the result set from a full-text query. When combining full-text results with vector results or other scored sources, rank each source independently rather than comparing raw score values directly. For a worked example, see Developer Guide → Hybrid search.

Vector search

The following graph is used for the examples below:

To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (:Movie {title: 'Aladdin', releaseDate: date('1992-11-08'), rating: 8.0}),
       (:Movie {title: 'Cinderella', embedding: vector([1, 3, 4], 3, INTEGER), releaseDate: date('1950-02-
15'), rating: 7.3}),
       (:Movie:Favorite {title: 'Frozen', embedding: [1, 3, 3], releaseDate: date('2013-11-10'), rating:
7.4}),
       (:Movie {title: 'Lilo & Stitch', embedding: vector([1, 1, 3], 3, INTEGER), releaseDate: date('2002-
06-16'), rating: 7.4}),
       (:Movie:Favorite {title: 'Lion King', embedding: vector([2, 5, 2], 3, INTEGER), releaseDate: date
('1994-06-12'), rating: 8.5}),
       (:Movie:Favorite {title: 'Mulan', embedding: [2, 3, 3], releaseDate: date('1998-06-05'), rating:
7.7}),
       (:Movie {title: 'Snow White', embedding: vector([1, 2, 3], 3, INTEGER), releaseDate: date('1937-12-
21'), rating: 7.6})
```

Each node in the example graph represents a movie with properties for its title, release date and rating. The majority of the nodes also have a `VECTOR` property called `embedding`, which is a vector representation of the plot of the movie. In a real-world example, these vector embeddings would typically be generated by a proprietary or open source embedding generator and have a dimension of for example 768 or 1536, but for simplicity this example dataset uses vector embeddings of dimension 3.

The examples use a vector index named `moviePlots` on the `embedding` property and `releaseDate` and `rating` as additional properties for filtering. Such a vector index can be created with the following Cypher query:

```
CREATE VECTOR INDEX moviePlots IF NOT EXISTS
FOR (m:Movie) ON m.embedding
WITH [m.releaseDate, m.rating]
OPTIONS {
  indexConfig: {
    `vector.similarity_function`: 'cosine',
    `vector.dimensions`: 3,
    `vector.quantization.type`: 'none'
  }
}
```

For more information about the creation of vector indexes, see Create vector indexes.

Basic vector search

Find the most similar nodes

In our example dataset, the movie with the title `Snow White` has the embedding `vector([1, 2, 3], 3, INTEGER)`. To find the 4 most similar movies to `Snow White`, you can run several different Cypher queries with the `SEARCH` clause.

Example 59. Using a vector literal as the query vector

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Snow White"
"Cinderella"
"Frozen"
"Mulan"
Rows: 4

Example 60. Using a list literal as the query vector

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR [1, 2, 3]
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Snow White"
"Cinderella"
"Frozen"
"Mulan"
Rows: 4

Example 61. Using a vector parameter as the query vector

Parameters

```
{
  snowWhiteEmbedding: vector([1, 2, 3], 3, INTEGER)
}
```

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR $snowWhiteEmbedding
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Snow White"
"Cinderella"
"Frozen"
"Mulan"
Rows: 4

Example 62. Using a property as the query vector

Query

```
MATCH (snowWhite:Movie {title: 'Snow White'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR snowWhite.embedding
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Snow White"
"Cinderella"
"Frozen"
"Mulan"
Rows: 4

All four queries above return the same result. The movie nodes are returned in decreasing similarity order. `Snow White` is the most similar movie to itself, while `Mulan` is the fourth most similar movie to `Snow White`. The movies `Lilo & Stitch` and `Lion King` are not returned as they are not among the top 4 most similar movies, while the movie `Aladdin` does not have an `embedding` property and is therefore not considered by the vector index at all.

Note:

The label predicate `:Movie` in the `MATCH` patterns above is not strictly required. As the vector index only contains nodes with label `:Movie`, the `SEARCH` clause adds this predicate implicitly.

Find the most similar nodes with score

To return the four most similar movies to `Snow White` together with their similarity score, you can include

the optional `SCORE` subclause in the `SEARCH` clause.

Example 63. Adding the similarity score to the output

Query

```
MATCH (snowWhite:Movie {title: 'Snow White'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR snowWhite.embedding
    LIMIT 4
  ) SCORE AS similarityScore
RETURN movie.title AS title, similarityScore
```

Result

title	similarityScore
"Snow White"	1.0
"Cinderella"	0.9979352951049805
"Frozen"	0.9905114769935608
"Mulan"	0.9843324422836304
Rows: 4	

Similarity scores have type `FLOAT NOT NULL` and are bounded between `0.0` and `1.0`. The closer to `1.0` the score is, the more similar the indexed `embedding` property is to the query vector. The movie `Snow White` has a similarity score of `1.0` as its `embedding` property is identical to the query vector.

Similarity scores are calculated using the similarity function that was chosen when the vector index was created, the default is cosine similarity. For more information on similarity functions, see Cosine and Euclidean similarity functions.

Additional predicates in the MATCH pattern

The pattern in the `MATCH` or `OPTIONAL MATCH` can have label and property predicates on the node or relationship with the binding variable.

Example 64. Finding the most similar movies with an extra label predicate

Query

```
MATCH (movie:Movie&Favorite)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Frozen"
"Mulan"
Rows: 2

Note:

The label predicate acts as a post-filtering step to the vector search. Even if the movie node with the title `Lion King` also has the label `Favorite`, it is not returned as it is not among the four most similar nodes to the query vector.

Example 65. Finding the most similar movies with an extra property predicate

Query

```
MATCH (movie:Movie {releaseDate: date('2013-11-10')})
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
"Frozen"
Rows: 2

Vector search with filters

A `MATCH` or `OPTIONAL MATCH` clause can have both a `WHERE` subclause and `SEARCH` subclause. Regardless of their order, the `WHERE` clause will act as a post-filtering step to the vector search, further filtering down the results.

Either of the queries below find the four most similar movies to `Snow White` and then filter down the result to only movies which were released after 1990.

Example 66. The order of WHERE and SEARCH does not affect the result

WHERE followed by SEARCH

```
MATCH (movie:Movie)
  WHERE movie.releaseDate > date('1990')
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
```

```
RETURN movie.title AS title, movie.releaseDate.year AS year
```

SEARCH followed by WHERE

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
  WHERE movie.releaseDate > date('1990')
RETURN movie.title AS title, movie.releaseDate.year AS year
```

Result

title	year
"Frozen"	2013
"Mulan"	1998
Rows: 2	

Often, it is preferable to do the filtering *while* performing the vector search, so called in-index filtering, instead of as a post-filtering step. This can be achieved with *vector search with filters*, where the `WHERE` is a subclause to the `SEARCH` clause instead of a subclause to the enclosing `MATCH` or `OPTIONAL MATCH` clause.

To find the four most similar movies to `Snow White` which were released after 1990 with a vector search with filters, use the following query.

Example 67. Vector search with filter

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE movie.releaseDate > date('1990')
    LIMIT 4
  )
RETURN movie.title AS title, movie.releaseDate.year AS year
```

Result

title	year
"Frozen"	2013
"Mulan"	1998
"Lilo & Stitch"	2002
"Lion King"	1994
Rows: 4	

> **Note:**
>
> When using vector search with filters, the vector search continues until it has found the requested number of result that also fulfills the other predicates. By contrast, using a normal `WHERE` clause in the `MATCH` can lead to significantly fewer results. Because of this, you might want to set a higher `LIMIT` in the `SEARCH` clause and complement it with a lower `LIMIT` in the `RETURN`. This approach is called over-fetching.

Combining vector search with filters and post-filtering

Vector search with filters and a normal `WHERE` clause can be combined. The following query finds the four most similar movies to `Snow White` which were released after 1990, and then does post-filtering to discard any result with a rating worse than 7.5.

Example 68. Vector search with filter and `WHERE` clause

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE movie.releaseDate > date('1990')
    LIMIT 4
  )
  WHERE movie.rating > 7.5
RETURN movie.title AS title, movie.releaseDate.year AS year, movie.rating AS rating
```

Result

title	year	rating
"Mulan"	1998	7.7
"Lion King"	1994	8.5
Rows: 2		

Predicates for vector search with filters

When used within `SEARCH`, `WHERE` supports the following predicates:

- Property predicates
- Multiple supported property predicates joined by the `AND` operator
- Property predicates using the `IN` operator Introduced in Neo4j 2026.06

`IN` is supported in vector search filter predicates from Neo4j 2026.06. Earlier versions do not support `IN` within the `WHERE` subclause of `SEARCH`.

Example 69. Vector search with filter with an AND predicate on the same property

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE movie.releaseDate > date('1948') AND date('2010') > movie.releaseDate
    LIMIT 4
  )
RETURN movie.title AS title, movie.releaseDate.year AS year
```

Result

title	year
"Cinderella"	1950
"Mulan"	1998
"Lilo & Stitch"	2002
"Lion King"	1994
Rows: 4	

Tip:

The `WHERE` subclause in the query above can also be written as `WHERE date('1948') < movie.releaseDate < date('2010')`.

Example 70. Vector search with filter with an AND predicate on separate properties

Query

```
MATCH (snowWhite:Movie {title:"Snow White"})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR snowWhite.embedding
    WHERE movie.releaseDate < date('2000') AND movie.rating >= snowWhite.rating
    LIMIT 4
  )
RETURN movie.title AS title, movie.releaseDate.year AS year, movie.rating AS rating
```

Result

title	year	rating
"Snow White"	1937	7.6
"Mulan"	1998	7.7
"Lion King"	1994	8.5
Rows: 4		

Note:

If the `LIMIT` in the `SEARCH` clause is larger than the total number of indexed vectors fulfilling the predicate, the vector search will return fewer results. In the example above, only three nodes are returned despite the `LIMIT` being four. Due to the approximate nature of vector search, this can also happen if the `LIMIT` is smaller than but close to the total number of indexed vectors fulfilling the predicate.

Example 71. Vector search with filter using `IN`

Introduced in Neo4j 2026.06

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE movie.rating IN [7.4, 7.7, 8.5]
    LIMIT 4
  )
RETURN movie.title AS title, movie.rating AS rating
```

Result

title	rating
"Frozen"	7.4
"Mulan"	7.7
"Lilo & Stitch"	7.4
"Lion King"	8.5
Rows: 4	

Note:

When the property in an `IN` predicate is of a numeric or string type, the list can be of any size. For other supported types, the list is limited to 256 elements.

The variable in the property predicate must be the same as the binding variable in the `SEARCH` clause.

Example 72. Not allowed — Mismatch of a property predicate variable and a binding variable in the SEARCH clause

Query

```
MATCH (m: Movie {name: "Snow White"})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE m.rating > 8
```

```
    LIMIT 4
  )
RETURN movie.title AS title
```

GQLSTATUS error chain

42I74: error: syntax error or access rule violation - wrong variable for vector search with filters. The variable `m` in a vector search filter property predicate must be the same as the search clause binding variable `movie`.

42001: error: syntax error or access rule violation - invalid syntax

All properties in the property predicate must have been added as additional properties for filtering when creating the vector index.

Example 73. Not allowed — A property of the property predicate wasn't defined as an additional property in the vector index

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE movie.rating > 8 AND movie.title > "L"
    LIMIT 4
  )
RETURN movie.title AS title, movie.rating AS rating
```

GQLSTATUS error chain

22ND3: error: data exception - wrong property for vector search with filters. The property `title` is not an additional property for vector search with filters on the vector index `moviePlots`.

For further limitations on the `WHERE` predicate, see Limitations of the `WHERE` predicate.

More on vector search

Using a query vector with different dimensions

If the query vector has a different dimension than the configured dimension of the vector index, the query will fail.

Example 74. Not allowed — Query vector with only two dimensions when three are being used

Query

```
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2], 2, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

GQLSTATUS error chain

51N65: error: system configuration or operation exception - vector index dimensionality mismatch. Vector index `moviePlots` has a configured dimensionality of 3, but the provided vector has a dimension 2.

Note:

If the vector index has no configured dimension, the query succeeds. However, if there are no vectors in the index of the same dimension as the query vector, the result of the query will be empty. For this reason, it is recommended to configure `vector.dimensions` when creating vector indexes.

Using a query vector which evaluates to null

If the query vector evaluates to `null`, for example because it refers to a non-existing property, a `SEARCH` clause inside a `MATCH` returns no results.

Example 75. A query vector which evaluates to null

Query

```
MATCH (snowWhite:Movie {title: 'Snow White'})
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR snowWhite.prop
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title

Rows: 0

If the `SEARCH` clause with a query vector evaluating to `null` is instead in an `OPTIONAL MATCH` clause, it returns `null` as per regular `OPTIONAL MATCH` semantics.

Example 76. A query vector which evaluates to null as per regular OPTIONAL MATCH semantics

Query

```
MATCH (snowWhite:Movie {title: 'Snow White'})
OPTIONAL MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR snowWhite.prop
    LIMIT 4
  )
RETURN movie.title AS title
```

Result

title
null
Rows: 1

Using a binding variable which is not from the enclosing clause

The binding variable in a `SEARCH` clause must be a node or relationship variable from the pattern of the enclosing `MATCH` or `OPTIONAL MATCH` clause. Trying to refer to a variable from an earlier clause in the query or introducing a new variable will cause the query to fail.

Example 77. Not allowed — Binding variable from earlier clause

Query

```
MATCH (movie:Movie)
MATCH (m: Movie {name: 'Snow White'})
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 4
  )
RETURN movie.title AS title
```

GQLSTATUS error chain

42I69: error: syntax error or access rule violation - invalid search variable reference. `movie` must reference a variable from the same MATCH statement.

42001: error: syntax error or access rule violation - invalid syntax

Example 78. Not allowed — Binding variable not previously defined

Query

```
MATCH (movie:Movie)
  SEARCH node IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

GQLSTATUS error chain

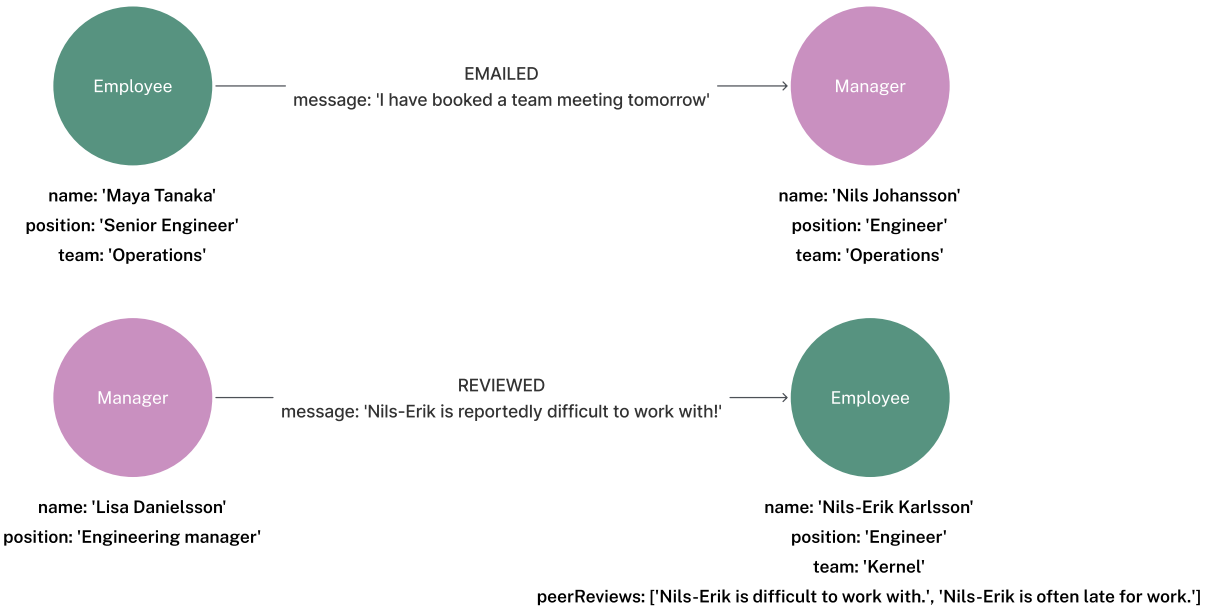
42N62: error: syntax error or access rule violation - variable not defined. Variable `node` not defined.

42001: error: syntax error or access rule violation - invalid syntax

Full-text search

Introduced in Neo4j 2026.09 Cypher 25 only

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (nilsE:Employee {name: "Nils-Erik Karlsson", position: "Engineer", team: "Kernel", peerReviews:
['Nils-Erik is difficult to work with.', 'Nils-Erik is often late for work.']}),
(lisa:Manager {name: "Lisa Danielsson", position: "Engineering manager"}),
(nils:Employee {name: "Nils Johansson", position: "Engineer", team: "Operations"}),
(maya:Employee {name: "Maya Tanaka", position: "Senior Engineer", team:"Operations"}),
(lisa)-[:REVIEWED {message: "Nils-Erik is reportedly difficult to work with."}]->(nilsE),
(maya)-[:EMAILED {message: "I have booked a team meeting tomorrow."}]->(nils)
```

The examples use the following full-text indexes:

```
CREATE FULLTEXT INDEX namesAndTeams IF NOT EXISTS
FOR (n:Employee|Manager) ON EACH [n.name, n.team];

CREATE FULLTEXT INDEX communications FOR ()-[r:REVIEWED|EMAILED]-() ON EACH [r.message]
OPTIONS {
  indexConfig: {
    `fulltext.analyzer`: 'english'
  }
};

CREATE FULLTEXT INDEX reviews FOR (n:Employee|Manager) ON EACH [n.peerReviews];
```

For more information about the creation of full-text indexes, see Create full-text indexes.

Full-text search examples

This query uses the `namesAndTeams` full-text index to look for "nils" among the employee and manager nodes:

Query a full-text index for a node property

```
MATCH (node)
  SEARCH node IN (
```

```
    FULLTEXT INDEX namesAndTeams
    FOR "nils"
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, score
```

Result

node.name	score
"Nils Johansson"	0.3300700783729553
"Nils-Erik Karlsson"	0.27725890278816223

Rows: 2

`SKIP` *full-text search results*

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR 'Nils'
    SKIP 1
    LIMIT 1
  )
RETURN node.name AS name,
       node.team AS team
```

Result

name	team
"Nils-Erik Karlsson"	"Kernel"

Rows: 2

This query uses the `communications` full-text index for any `message` containing "meeting":

Query a full-text index for a relationship property

```
MATCH ()-[relationship]->()
  SEARCH relationship IN (
    FULLTEXT INDEX communications
    FOR "meeting"
    LIMIT 5
  )
  SCORE AS score
RETURN type(relationship), relationship.message, score
```

Result

type(relationship)	relationship.message	score
"EMAILED"	"I have booked a team meeting tomorrow."	0.30376967787742615

Rows: 1

To search for a sequence of terms as a phrase, enclose the phrase in double quotation marks. This matches the terms in order; it does not require the entire indexed property to equal the query string.

Query a full-text index for a phrase

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR '"Nils-Erik"'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, score
```

Result

node.name	score
"Nils-Erik Karlsson"	0.7588480710983276
Rows: 1	

Query strings also support the use of the Lucene boolean operators (`AND`, `OR`, `NOT`, `+`, `-`):

Query a full-text index using logical operators

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR 'nils AND kernel'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, node.team, score
```

Result

node.name	node.team	score
"Nils-Erik Karlsson"	"Kernel"	0.723090410232544
Rows: 1		

It is possible to limit the search to specific properties, by prefixing `<propertyName>:` to the query string.

Query a full-text index for specific properties

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR 'team:"Operations"'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, node.team, score
```

Result

node.name	node.team	score
"Nils Johansson"	"Operations"	0.21363800764083862
"Maya Tanaka"	"Operations"	0.21363800764083862

node.name	node.team	score
Rows: 2		

Lists of `STRING` values

If the indexed property contains a list of `STRING` values, each entry is analyzed independently and all produced tokens are associated to the same property name. This means that when querying such an indexed node or relationship, there is a match if any of the list elements matches the query string. For scoring purposes, the full-text index treats it as a single-property value, and the score represents how close the query is to matching the entire list.

Query a full-text index for content present in a list of `STRING` properties

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX reviews
    FOR 'late'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, node.peerReviews, score
```

Result

node.name	node.peerReviews	score
"Nils-Erik Karlsson"	["Nils-Erik is difficult to work with.", "Nils-Erik is often late for work."]	0.13076457381248474

Using an analyzer

You can query full-text indexes with `SEARCH` and use an analyzer with `WITH ANALYZER`:

Query a full-text index using an analyzer

```
MATCH ()-[relationship]->()
  SEARCH relationship IN (
    FULLTEXT INDEX communications
    FOR "reportedly" WITH ANALYZER "english"
    LIMIT 5
  )
  SCORE AS score
RETURN type(relationship), relationship.message, score
```

Result

type(relationship)	relationship.message	score
"REVIEWED"	"Nils-Erik is reportedly difficult to work with."	0.3272368609905243
Rows: 1		

See tokenization and analyzers for more information on analyzers.

Limitations of the SEARCH clause

The tables below list some past and current limitations of the `SEARCH` clause. If applicable, the version which lifted the limitation is also listed.

The `SEARCH` clause puts restrictions on the pattern of the enclosing `MATCH` or `OPTIONAL MATCH` clause.

Limitations of the `MATCH` or `OPTIONAL MATCH` pattern

Limitation	Disallowed example	Lifted in
The pattern cannot have more than one bound variable.	`MATCH (movie:Movie)-[r]->()`	
The pattern cannot have predicates on other elements than the bound variable.	`MATCH (:Actor)-[]->(m:Movie)`	
The pattern cannot have a length longer than one relationship hop.	`MATCH ()-[r: KNOWS]->()-[]->()` `MATCH (movie:Movie)-[*1..3]->()`	
The pattern cannot have multiple pattern parts.	`MATCH (movie:Movie), ()`	
The pattern cannot have a path selector other than `ALL`.	`MATCH ANY 2 (movie: Movie)`	

The optional `WHERE` subclause used to do *vector search with filters* is a restricted version of the normal Cypher `WHERE` clause and only allows certain types of property predicates.

Limitations of the `WHERE` predicate

Limitation	Disallowed example	Lifted in
The comparison operator in the predicate cannot be `<>`.	`WHERE movie.rating <> 7.5`	
The predicate cannot include other boolean operators than `AND`.	`WHERE NOT movie.rating > 8` [3] `WHERE movie.rating > 8 OR movie.releaseDate <= date("2000")`	
Before Neo4j 2026.06, the predicate cannot include the `IN` operator.	`WHERE movie.rating IN [7, 7.5, 8]`	Neo4j 2026.06
The predicate cannot include a string operator.	`WHERE movie.rating STARTS WITH '8'`	
The predicate cannot include a type predicate expression.	`WHERE movie.rating IS :: FLOAT`	
The expression in the predicate cannot be of type `POINT`.	`WHERE movie.rating < point({x:2.3, y:4.5})`	
The expression in the predicate cannot be of type `VECTOR`.	`WHERE movie.rating < vector([8.2, 8.5], 2, FLOAT)`	
A `LIST` expression cannot be used with a comparison operator. Lists are supported only as the right-hand operand of `IN`.	`WHERE movie.rating < [8, 8.5]`	

Limitation	Disallowed example	Lifted in
The expression in the predicate is not allowed to reference the binding variable.	`WHERE movie.rating > movie.title`	
The predicate cannot contain multiple property predicates on the same property in the same direction.	`WHERE movie.rating > 8.2 AND movie.rating >= 8.5`	
The predicate cannot contain multiple property predicates on the same property where one is an equality and one is a range predicate.	`WHERE movie.rating > 8.2 AND movie.rating = 8.5`	

Other limitations of the SEARCH clause

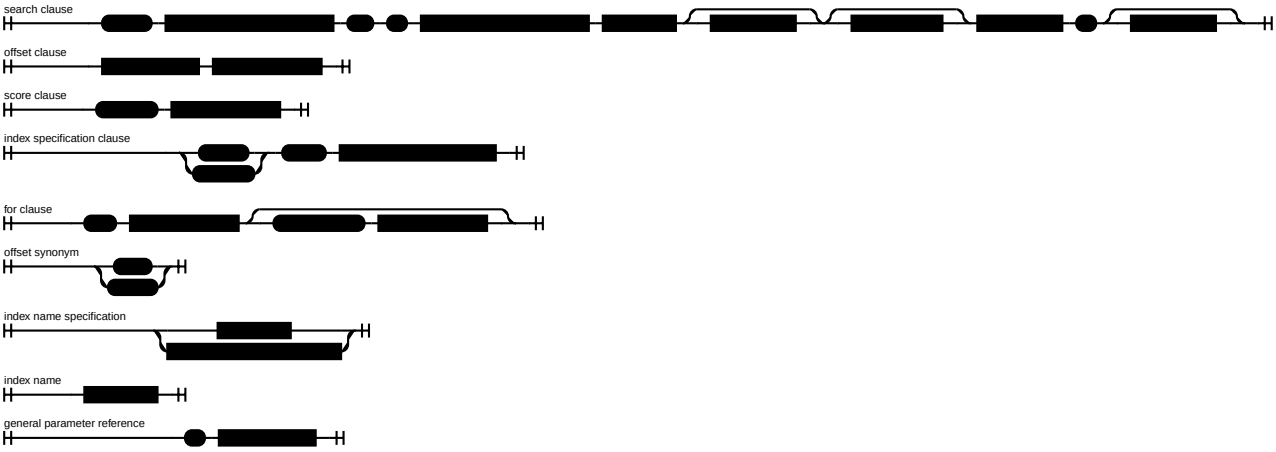
Limitation	Lifted in
Unlike in other index commands such as `CREATE VECTOR INDEX` and `DROP INDEX`, the index name is not allowed to be a parameter in the `VECTOR INDEX` subclause of `SEARCH`.	
The query vector is not allowed to reference the binding variable. *Not allowed* `MATCH (movie:Movie) SEARCH movie IN ( VECTOR INDEX moviePlots FOR movie.embedding LIMIT 4 ) RETURN movie.title AS title` *GQLSTATUS error chain* 42I75: error: syntax error or access rule violation - self-referencing in search. The expression 'movie.embedding' in the search clause may not depend on the search clause binding variable `movie`. 42001: error: syntax error or access rule violation - invalid syntax	

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



SET

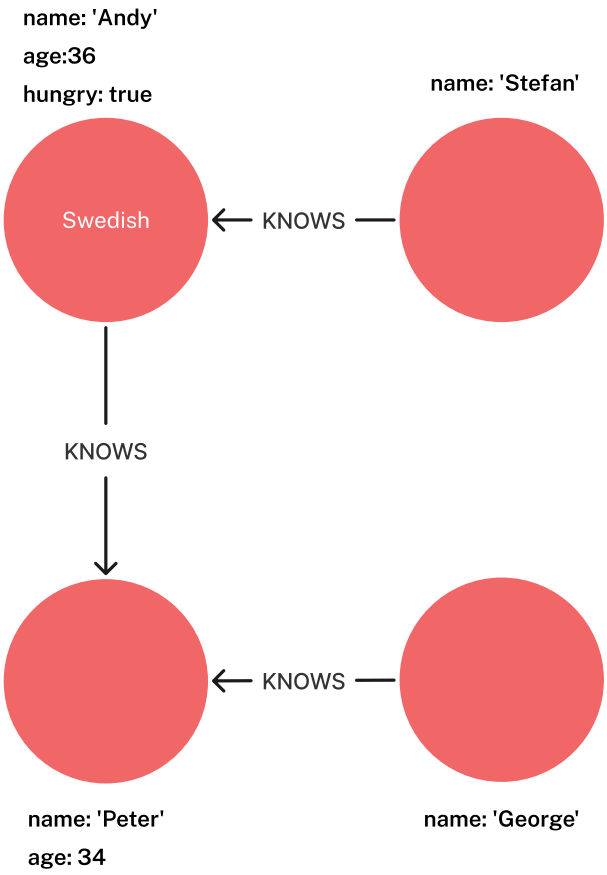
The `SET` clause is used to update labels on nodes and properties on nodes and relationships.

The `SET` clause can be used with a map — provided as a literal or a parameter — to set properties.

> **Note:**
>
> Setting labels on a node is an idempotent operation — nothing will occur if an attempt is made to set a label on a node that already has that label. The query statistics will state whether any updates actually took place.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (a:Swedish {name: 'Andy', age: 36, hungry: true}),
  (b {name: 'Stefan'}),
  (c {name: 'Peter', age: 34}),
  (d {name: 'George'}),
  (a)-[:KNOWS]->(c),
  (b)-[:KNOWS]->(a),
  (d)-[:KNOWS]->(c)
```

Set a property

Update a node property:

Query

```
MATCH (n {name: 'Andy'})
SET n.surname = 'Taylor'
RETURN n.name, n.surname
```

The newly-changed node is returned by the query.

Result

n.name	n.surname
"Andy"	"Taylor"
Rows: 1 Properties set: 1	

Update a relationship property:

Query

```
MATCH (n:Swedish {name: 'Andy'})-[r:KNOWS]->(m)
SET r.since = 1999
RETURN r, m.name AS friend
```

Result

r	friend
[:KNOWS {since: 1999}]	"Peter"

Rows: 1

Properties set: 1

It is possible to set a property on a node or relationship using more complex expressions. For instance, in contrast to specifying the node directly, the following query shows how to set a property for a node selected by an expression:

Query

```
MATCH (n {name: 'Andy'})
SET (CASE WHEN n.age = 36 THEN n END).worksIn = 'Malmo'
RETURN n.name, n.worksIn
```

Result

n.name	n.worksIn
"Andy"	"Malmo"

Rows: 1

Properties set: 1

No action will be taken if the node expression evaluates to `null`, as shown in this example:

Query

```
MATCH (n {name: 'Andy'})
SET (CASE WHEN n.age = 55 THEN n END).worksIn = 'Malmo'
RETURN n.name, n.worksIn
```

As no node matches the `CASE` expression, the expression returns a `null`. As a consequence, no updates occur, and therefore no `worksIn` property is set.

Result

n.name	n.worksIn
"Andy"	<null>

Rows: 1

Update a property

`SET` can be used to update a property on a node or relationship. This query forces a change of type in the `age` property:

Query

```
MATCH (n {name: 'Andy'})
SET n.age = toString(n.age)
RETURN n.name, n.age
```

The `age` property has been converted to the `STRING` `'36'`.

Result

n.name	n.age
"Andy"	"36"
Rows: 1 Properties set: 1	

Dynamically set or update a property

`SET` can be used to set or update a property on a node or relationship even when the property key name is not statically known. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

```
SET n[key] = expression
```

The dynamically calculated key must evaluate to a `STRING` value. This query creates a copy of every property on the nodes:

Query

```
MATCH (n)
FOREACH (k IN keys(n) | SET n[k + "Copy"] = n[k])
RETURN n.name, keys(n);
```

The `FOREACH` clause iterates over each property key `k` obtained from the `keys()` function. For each key, it sets a new property on the nodes with a key name of `k` + "Copy" and copies the value from the original property.

The nodes now have copies of all their properties.

Result

n.name	keys(n)
"Andy"	["name", "nameCopy", "age", "ageCopy", "hungry", "hungryCopy"]

n.name	keys(n)
"Stefan"	["name", "nameCopy"]
"Peter"	["name", "nameCopy", "age", "ageCopy"]
"George"	["name", "nameCopy"]

Rows: 4

Properties set: 6

Remove a property

Although `REMOVE` is normally used to remove a property, it is sometimes convenient to do it using the `SET` command. A case in point is if the property is provided by a parameter.

Query

```
MATCH (n {name: 'Andy'})
SET n.name = null
RETURN n.name, n.age
```

The `name` property is now missing.

Result

n.name	n.age
<null>	"36"

Rows: 1

Properties set: 1

Copy properties between nodes and relationships

`SET` can be used to copy all properties from one node or relationship to another using the `properties()` function. This will remove *all* other properties on the node or relationship being copied to.

Query

```
MATCH
  (at {name: 'Andy'}),
  (pn {name: 'Peter'})
SET at = properties(pn)
RETURN at.name, at.age, at.hungry, pn.name, pn.age
```

The `'Andy'` node has had all its properties replaced by the properties of the `'Peter'` node.

Result

at.name	at.age	at.hungry	pn.name	pn.age
"Peter"	34	<null>	"Peter"	34

at.name	at.age	at.hungry	pn.name	pn.age
Rows: 1 Properties set: 3				

Replace all properties using a map and `=`

The property replacement operator `=` can be used with `SET` to replace all existing properties on a node or relationship with those provided by a map:

Query

```
MATCH (p {name: 'Peter'})
SET p = {name: 'Peter Smith', position: 'Entrepreneur'}
RETURN p.name, p.age, p.position
```

This query updated the `name` property from `Peter` to `Peter Smith`, deleted the `age` property, and added the `position` property to the `'Peter'` node.

Result

p.name	p.age	p.position
"Peter Smith"	<null>	"Entrepreneur"
Rows: 1 Properties set: 3		

Remove all properties using an empty map and `=`

All existing properties can be removed from a node or relationship by using `SET` with `=` and an empty map as the right operand:

Query

```
MATCH (p {name: 'Peter'})
SET p = {}
RETURN p.name, p.age
```

This query removed all the existing properties — namely, `name` and `age` — from the `'Peter'` node.

Result

p.name	p.age
<null>	<null>
Rows: 1 Properties set: 2	

Mutate specific properties using a map and `+=`

The property mutation operator `+=` can be used with `SET` to mutate properties from a map in a fine-grained fashion:

- Any properties in the map that are not on the node or relationship will be *added*.
- Any properties not in the map that are on the node or relationship will be left as is.
- Any properties that are in both the map and the node or relationship will be *replaced* in the node or relationship. However, if any property in the map is `null`, it will be *removed* from the node or relationship.

Query

```
MATCH (p {name: 'Peter'})
SET p += {age: 38, hungry: true, position: 'Entrepreneur'}
RETURN p.name, p.age, p.hungry, p.position
```

This query left the `name` property unchanged, updated the `age` property from `34` to `38`, and added the `hungry` and `position` properties to the `'Peter'` node.

Result

p.name	p.age	p.hungry	p.position
"Peter"	38	true	"Entrepreneur"

Rows: 1

Properties set: 3

In contrast to the property replacement operator `=`, providing an empty map as the right operand to `=` will not remove any existing properties from a node or relationship. In line with the semantics detailed above, passing in an empty map with `+=` will have no effect:

Query

```
MATCH (p {name: 'Peter'})
SET p += {}
RETURN p.name, p.age
```

Result

p.name	p.age
"Peter"	34

Rows: 1

Set multiple properties using one `SET` clause

Set multiple properties at once by separating them with a comma:

Query

```
MATCH (n {name: 'Andy'})
SET n.position = 'Developer', n.surname = 'Taylor'
```

Result

```
(empty result)
```

Rows: 0

Properties set: 2

Set a property using a parameter

Use a parameter to set the value of a property:

Parameters

```
{
  "surname": "Taylor"
}
```

Query

```
MATCH (n {name: 'Andy'})
SET n.surname = $surname
RETURN n.name, n.surname
```

A `surname` property has been added to the `'Andy'` node.

Result

n.name	n.surname
"Andy"	"Taylor"

Rows: 1

Properties set: 1

Set all properties using a parameter

This will replace all existing properties on the node with the new set provided by the parameter.

Parameters

```
{
  "props" : {
    "name": "Andy",
    "position": "Developer"
  }
}
```

Query

```
MATCH (n {name: 'Andy'})
SET n = $props
RETURN n.name, n.position, n.age, n.hungry
```

The `'Andy'` node has had all its properties replaced by the properties in the `props` parameter.

Result

n.name	n.position	n.age	n.hungry
"Andy"	"Developer"	<null>	<null>

Rows: 1

Properties set: 4

Set a label on a node

Use `SET` to set a label on a node:

Query

```
MATCH (n {name: 'Stefan'})
SET n:German
RETURN n.name, labels(n) AS labels
```

The newly-labeled node is returned by the query.

Result

n.name	labels
"Stefan"	["German"]

Rows: 1

Labels added: 1

Dynamically set a node label

`SET` can be used to set a label on a node even when the label is not statically known.

```
MATCH (n)
SET n:$(<expr>)
```

Query

```
MATCH (n:Swedish)
SET n:$(n.name)
RETURN n.name, labels(n) AS labels
```

The newly-labeled node is returned by the query.

Result

n.name	labels
"Andy"	["Swedish", "Andy"]

Rows: 1

Labels added: 1

Set a label using a parameter

Use a parameter to set the value of a label:

Parameters

```
{
  "label": "Danish"
}
```

Query

```
MATCH (n {name: 'Stefan'})
SET n:$($label)
RETURN labels(n) AS labels
```

A `Danish` label has been added to the `'Stefan'` node.

Result

labels
['German', 'Danish']
Rows: 1 Labels added: 1

Set multiple labels on a node

Set multiple labels on a node with `SET` and use `:` to separate the different labels:

Query

```
MATCH (n {name: 'George'})
SET n:Swedish:Bossman
RETURN n.name, labels(n) AS labels
```

The newly-labeled node is returned by the query.

Result

n.name	labels
"George"	["Swedish","Bossman"]
Rows: 1 Labels added: 2	

Set multiple labels dynamically on a node

It is possible to set multiple labels dynamically using a `LIST<STRING>` and/or by chaining them separately with a `:`:

Query

```
WITH COLLECT { UNWIND range(0,3) AS id RETURN "Label" + id } as labels
MATCH (n {name: 'George'})
SET n:$(labels)
RETURN n.name, labels(n) AS labels
```

A `COLLECT` subquery aggregates the results of `UNWIND range(0,3) AS id RETURN "Label" + id`, which generates a `LIST<STRING>` strings ("Label0", "Label1", "Label2", "Label3"), and assigns it to the variable `labels`.

The newly-labeled node is returned by the query.

Result

n.name	labels
"George"	["Swedish","Bossman", "Label0", "Label1", "Label2", "Label3"]

Rows: 1

Labels added: 4

Set multiple labels using parameters

Use a parameter to set multiple labels:

Parameters

```
{
  "labels": ["Swedish", "German"]
}
```

Query

```
MATCH (n {name: 'Peter'})
SET n:$($labels)
RETURN labels(n) AS labels
```

A `Swedish` and a `German` label has been added to the `'Peter'` node.

Result

labels
['Swedish', 'German']

Rows: 1

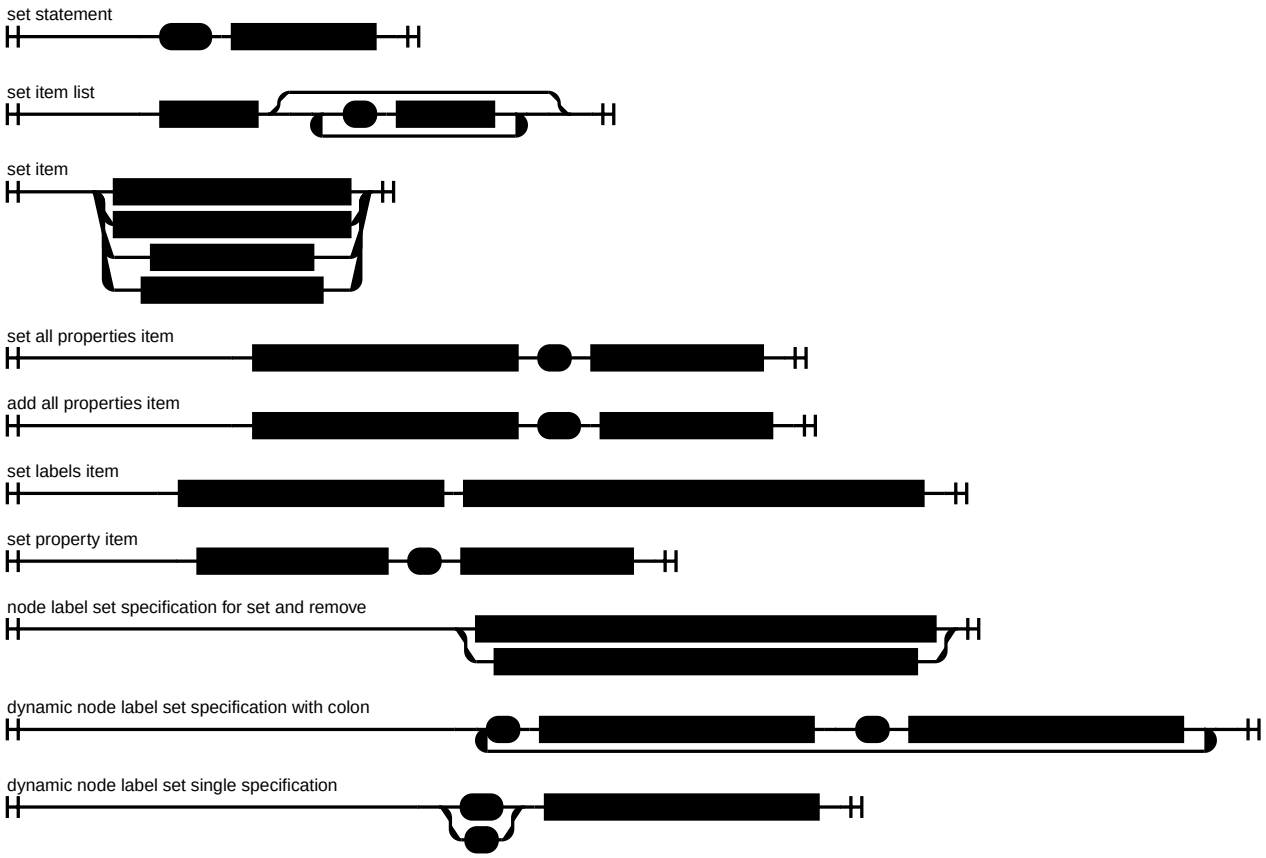
Labels added: 2

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



SHOW

The `SHOW` command can be used to show the metadata of a database or a DBMS. This page discusses common behavior to all `SHOW` commands. Rules pertaining to specific `SHOW` commands, as well as the different columns returned by them, are listed on other pages or in the Operations Manual as follows:

- `SHOW AUTH RULES` (Operations Manual)
- `SHOW ALIASES` (Operations Manual)
- `SHOW CONSTRAINTS`
- `SHOW CURRENT GRAPH TYPE`
- `SHOW DATABASES` (Operations Manual)
- `SHOW FUNCTIONS`
- `SHOW INDEXES`
- `SHOW PRIVILEGES` (Operations Manual)
- `SHOW PROCEDURES` (Operations Manual)
- `SHOW ROLES` (Operations Manual)

- `SHOW SERVERS` (Operations Manual)
- `SHOW SETTINGS` (Operations Manual)
- `SHOW TRANSACTIONS` (Operations Manual)
- `SHOW USERS` (Operations Manual)

Common behavior and rules

All `SHOW` commands have similar behavior and comply to the same rules.

Returning default and non-default columns using `SHOW` and `YIELD`

All `SHOW` commands have default return columns, and most have non-default return columns as well. For information about the specific columns, visit the documentation for the specific `SHOW` command listed above.

Using `YIELD` returns non-default columns, either specific columns or all available columns.

Show indexes — return only default columns

```
SHOW INDEXES
```

Show indexes — return specific columns with `YIELD`

```
SHOW INDEXES
YIELD name, type, indexProvider
```

Show indexes — return all available columns using `YIELD *`

```
SHOW INDEXES
YIELD *
```

Using `SHOW` commands with `RETURN`

If the `RETURN` clause is used in any `SHOW` command, `YIELD` is mandatory.

`RETURN` after `YIELD`

```
SHOW INDEXES
YIELD name, type, indexProvider AS provider, options, createStatement
RETURN name, type, provider, options.indexConfig AS config, createStatement
```

Filter `SHOW` commands using `WHERE`

All `SHOW` commands can be filtered using the `WHERE` clause. This allows you to narrow down results based on specific conditions, such as names or status values.

Filter `SHOW` command using `WHERE`

```
SHOW SETTINGS YIELD name, value, description
WHERE name STARTS WITH 'server'
```

```
RETURN name, value, description
```

Composable `SHOW` commands

Introduced in 2026.05 | Cypher 25 only

The following `SHOW` commands can be combined with each other and other Cypher clauses; see their respective sections for examples:

- `SHOW CONSTRAINTS`
- `SHOW CURRENT GRAPH TYPE`
- `SHOW CURRENT USER` (Operations Manual) Introduced in Neo4j 2026.10
- `SHOW DATABASES` (Operations Manual)
- `SHOW FUNCTIONS`
- `SHOW INDEXES`
- `SHOW PROCEDURES` (Operations Manual)
- `SHOW SETTINGS` (Operations Manual)
- `SHOW TRANSACTIONS` (Operations Manual) — it has been possible to combine `SHOW TRANSACTIONS` with `TERMINATE TRANSACTIONS` since Neo4j 5.0.

> **Note:**
>
> When combining a `SHOW` command with other clauses, the `YIELD` clause is mandatory and must not be omitted. `YIELD` must explicitly list the yielded columns. `YIELD *` is not permitted. The query must also end with a valid last clause (like a `RETURN` or an updating clause).

Return all functions and procedures

```
SHOW PROCEDURES
YIELD name, signature
RETURN name, signature, 'procedure' AS type
UNION
SHOW FUNCTIONS
YIELD name, signature
RETURN name, signature, 'function' AS type
```

Return all functions with more than one signature

```
SHOW FUNCTIONS
YIELD name, signature
WITH name, collect(signature) AS signatures
FILTER size(signatures) > 1
RETURN name, signatures
```

SKIP

`SKIP` (and its synonym `OFFSET`) defines from which row to start including the rows in the output.

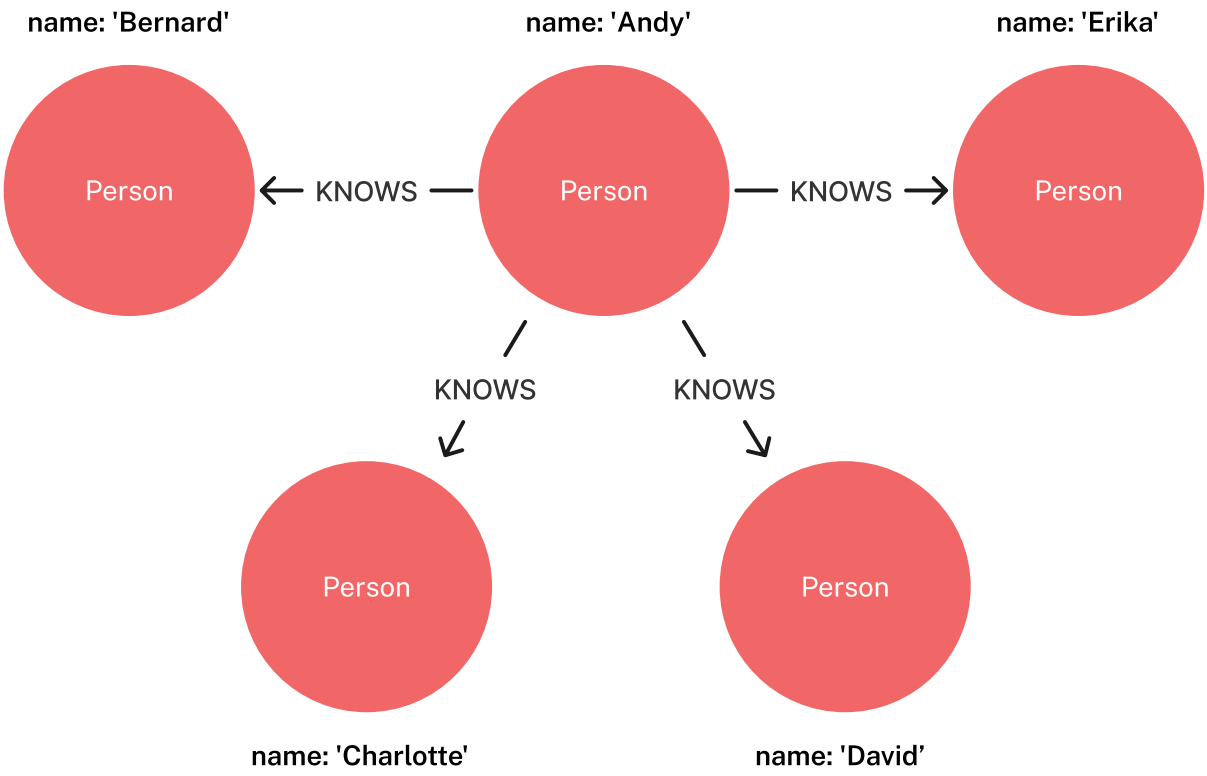
By using `SKIP`, the result set will get trimmed from the top.

`SKIP` accepts any expression that evaluates to a non-negative `INTEGER`, does not contain any variables, and does not refer to nodes or relationships.

> Neo4j does not guarantee the results generated by `SKIP`/`OFFSET`. The only clause that guarantees a specific row order is `ORDER BY`.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (andy: Person {name: 'Andy'}),
  (bernard: Person {name: 'Bernard'}),
  (charlotte: Person {name: 'Charlotte'}),
  (david: Person {name: 'David'}),
  (erika: Person {name: 'Erika'}),
  (andy)-[:KNOWS]->(bernard),
  (andy)-[:KNOWS]->(charlotte),
  (andy)-[:KNOWS]->(david),
  (andy)-[:KNOWS]->(erika)
```

Examples

Example 79. Skip the first three rows

The following query returns a subset of the result, starting from the fourth result.

Query

```
MATCH (n)
RETURN n.name
ORDER BY n.name
SKIP 3
```

Result

n.name
"David"
"Erika"
Rows: 2

Example 80. Return the middle two rows

The following query returns the middle two rows, with `SKIP` skipping the first and `LIMIT` removing the final two.

Query

```
MATCH (n)
RETURN n.name
ORDER BY n.name
SKIP 1
LIMIT 2
```

Result

n.name
"Bernard"
"Charlotte"
Rows: 2

Example 81. Using an expression with `SKIP` to return a subset of the rows

`SKIP` accepts any expression that evaluates to a positive `INTEGER`, as long as it can be statically calculated (i.e. calculated before the query is run).

This query skips the first row and then randomly skips an additional 0, 1, or 2 rows, resulting in skipping a total of 1, 2, or 3 rows before returning the remaining names.

Query

```
MATCH (n)
RETURN n.name
ORDER BY n.name
SKIP 1 + toInteger(3 * rand())
```

Result

n.name
"Bernard"
"Charlotte"
"David"
"Erika"
Rows: 4

Using `SKIP` as a standalone clause

`SKIP` can be used as a standalone clause, or in conjunction with `ORDER BY` or `LIMIT`.

Standalone use of `SKIP`

```
MATCH (n)
SKIP 2
RETURN collect(n.name) AS names
```

Result

names
["Charlotte", "David", "Erika"]
Rows: 1

The following query orders all nodes by `name`, skips the two first rows and limits the results to two rows. It then collects the results in a list.

`SKIP` *used in conjunction with* `ORDER BY` *and* `LIMIT`

```
MATCH (n)
ORDER BY n.name
SKIP 2
LIMIT 2
RETURN collect(n.name) AS names
```

Result

names
["Charlotte", "David"]
Rows: 1

`OFFSET` as a synonym to `SKIP`

`OFFSET` was introduced as part of Cypher's GQL conformance and can be used as a synonym to `SKIP`.

Query

```
MATCH (n)
ORDER BY n.name
OFFSET 2
LIMIT 2
RETURN collect(n.name) AS names
```

Result

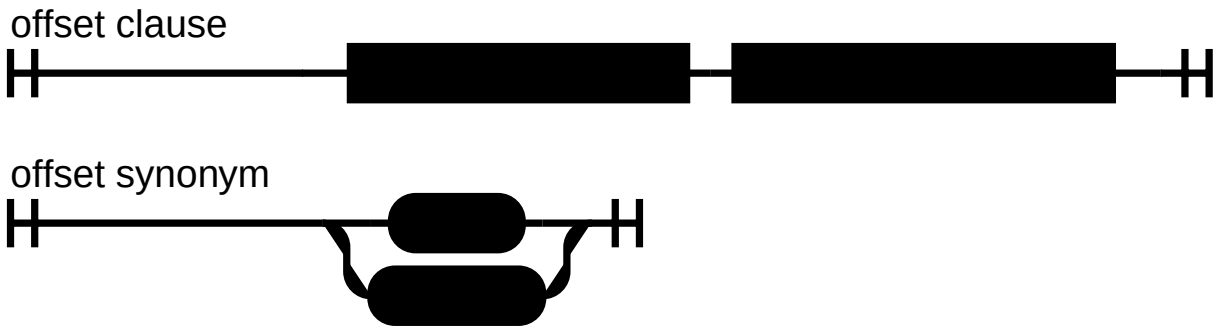
names
["Charlotte", "David"]
Rows: 1

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-filename}.bnf[]
```

▼ *Railroad diagram of the snippet*



UNWIND

The `UNWIND` clause makes it possible to transform any list back into individual rows. These lists can be parameters that were passed in, previously `collect`-ed result, or other list expressions.

> **Note:**
>
> As of Neo4j 2026.04, the same semantics are also available with GQL conformant syntax: `FOR variable IN expression`. `UNWIND` remains fully supported; the two forms are equivalent for list unnesting.

> **Note:**
>
> Neo4j does not guarantee the row order produced by `UNWIND`. The only clause that guarantees a specific row order is ORDER BY.

Common usage of the `UNWIND` clause:

- Create distinct lists.
- Create data from parameter lists that are provided to the query.

Note:

The `UNWIND` clause requires you to specify a new name for the inner values.

Unwinding a list

We want to transform the literal list into rows named `x` and return them.

Query

```
UNWIND [1, 2, 3, null] AS x
RETURN x, 'val' AS y
```

Each value of the original list — including `null` — is returned as an individual row.

Result

x	y
1	"val"
2	"val"
3	"val"
<null>	"val"
Rows: 4	

Creating a distinct list

We want to transform a list of duplicates into a set using `DISTINCT`.

Query

```
WITH [1, 1, 2, 2] AS coll
UNWIND coll AS x
WITH DISTINCT x
RETURN collect(x) AS setOfVals
```

Each value of the original list is unwound and passed through `DISTINCT` to create a unique set.

Result

setOfVals
[1,2]
Rows: 1

Using `UNWIND` with any expression returning a list

Any expression that returns a list may be used with `UNWIND`.

Query

```
WITH
  [1, 2] AS a,
  [3, 4] AS b
UNWIND (a + b) AS x
RETURN x
```

The two lists — a and b — are concatenated to form a new list, which is then operated upon by `UNWIND`.

Result

x
1
2
3
4
Rows: 4

Using `UNWIND` with a list of lists

Multiple `UNWIND` clauses can be chained to unwind nested list elements.

Query

```
WITH [[1, 2], [3, 4], 5] AS nested
UNWIND nested AS x
UNWIND x AS y
RETURN y
```

The first `UNWIND` results in three rows for `x`, each of which contains an element of the original list (two of which are also lists); namely, `[1, 2]`, `[3, 4]`, and `5`. The second `UNWIND` then operates on each of these rows in turn, resulting in five rows for `y`.

Result

y
1
2
3
4
5
Rows: 5

Using `UNWIND` with an empty list

Using an empty list with `UNWIND` will produce no rows, irrespective of whether or not any rows existed beforehand, or whether or not other values are being projected.

Essentially, `UNWIND []` reduces the number of rows to zero, and thus causes the query to cease its execution, returning no results. This has value in cases such as `UNWIND v`, where `v` is a variable from an earlier clause that may or may not be an empty list — when it is an empty list, this will behave just as a `MATCH` that has no results.

Query

```
UNWIND [] AS empty
RETURN 'literal_that_is_not_returned'
```

Result

```
(empty result)
```

Rows: 0

To avoid inadvertently using `UNWIND` on an empty list, `CASE` may be used to replace an empty list with a `null`:

```
WITH [] AS list
UNWIND
  CASE
    WHEN list = [] THEN [null]
    ELSE list
  END AS emptylist
RETURN emptylist
```

Using `UNWIND` with an expression that is not a list

Using `UNWIND` on an expression that does not return a list, will return the same result as using `UNWIND` on a list that just contains that expression. As an example, `UNWIND 5` is effectively equivalent to `UNWIND[5]`. The exception to this is when the expression returns `null` — this will reduce the number of rows to zero, causing it to cease its execution and return no results.

Query

```
UNWIND null AS x
RETURN x, 'some_literal'
```

Result

```
(empty result)
```

Rows: 0

Creating nodes from a list parameter

Create a number of nodes and relationships from a parameter-list without using `FOREACH`.

Parameters

```
{
  "events" : [ {
    "year" : 2014,
    "id" : 1
  }, {
    "year" : 2014,
    "id" : 2
  } ]
}
```

Query

```
UNWIND $events AS event
MERGE (y:Year {year: event.year})
MERGE (y)<-[:IN]-(e:Event {id: event.id})
RETURN e.id AS x ORDER BY x
```

Each value of the original list is unwound and passed through `MERGE` to find or create the nodes and relationships.

Result

x
1
2

Rows: 2

Nodes created: 3

Relationships created: 2

Properties set: 3

Labels added: 3

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

USE

The `USE` clause determines which graph a query, or query part, is executed against. It is supported for queries and schema commands.

Syntax

```
USE <graph reference>
<other clauses>
```

A graph reference can be described using:

- Direct graph references: `USE db1`.
- The graph function `graph.byElementId()`, to access a graph of a given element: `USE graph.byName(<element-id-string>)`.

When connected to a composite database, a graph reference may additionally be passed with:

- The graph function `graph.byName()`, which allows the graph reference to be resolved dynamically: `USE graph.byName(<string-expression>)`.

A more detailed description of how and when a graph references needs to be quoted and/or escaped is defined here.

USE clause when connected to a standard or system database

All databases and aliases are valid graph reference targets, except for sharded property database. Targeting multiple databases is not allowed, unless connected to a composite database.

Position of use clauses

When connected to a non-composite database, the `USE` clause can only appear as the prefix of schema commands, or as the first clause of queries. There may be multiple `USE` clauses as long as they target the same database.

USE clause when connected to a composite database

When executing queries against a composite database, the `USE` clause must only refer to graphs that are part of the current composite database. The constituents can be listed either with `RETURN graph.names()` when connected to the composite database or `SHOW DATABASES YIELD name, constituents RETURN *`.

Position of use clauses

When running queries against a composite database, the `USE` clause can appear as the first clause of:

- A query (similar to how it is used when connected to a non-composite database).
- Union parts:

  ```
  USE <graph>
  <other clauses>
    UNION
  USE <graph>
  <other clauses>
  ```

- Subqueries:

```
CALL () {
  USE <graph>
  <other clauses>
}
```

In subqueries, a `USE` clause may appear directly following the variable scope clause: `CALL () { … }`.

Examples

Query a graph

This example assumes that the DBMS contains a database named `myDatabase`:

Query

```
USE myDatabase
MATCH (n) RETURN n
```

Query a composite database constituent graph

In this example it is assumed that the DBMS contains a composite database named `myComposite`, which includes an alias named `myConstituent`:

Query

```
USE myComposite.myConstituent
MATCH (n) RETURN n
```

Query a composite database constituent graph dynamically

The `graph.byName()` function can be used in the `USE` clause to resolve a constituent graph from a `STRING` value containing the qualified name of a constituent.

This example uses a composite database named `myComposite` that includes an alias named `myConstituent`:

Query

```
USE graph.byName('myComposite.myConstituent')
MATCH (n) RETURN n
```

The argument can be any expression that evaluates to the name of a constituent graph - for example a parameter:

Query

```
USE graph.byName($graphName)
MATCH (n) RETURN n
```

Query a composite database constituent using elementId

The `graph.byElementId()` function can be used in the `USE` clause to resolve a constituent graph to which a

given element id belongs.

In the below example, it is assumed that the DBMS contains the database corresponding to the given element id. If you are connected to a composite database it needs to be a element id to a constituent database, which is a standard database in the DBMS.

Query

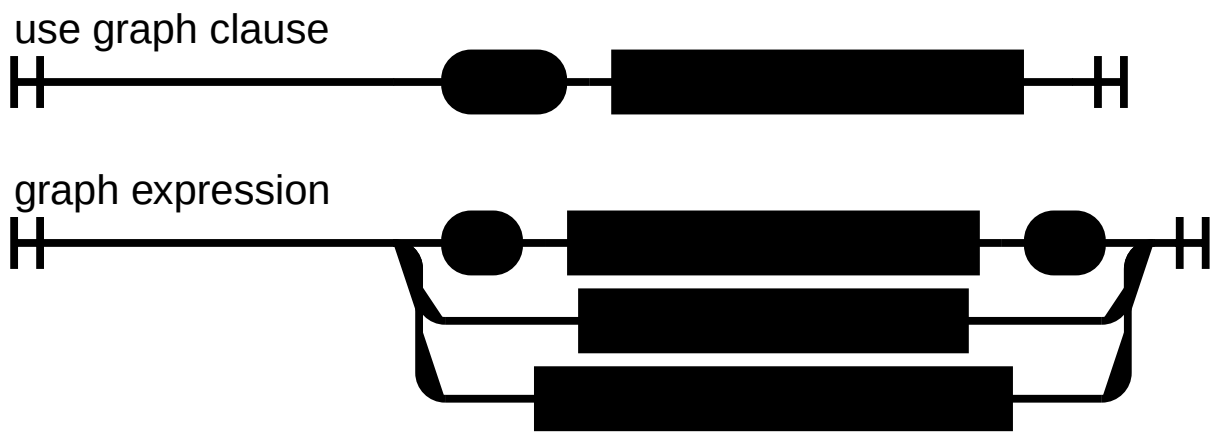
```
USE graph.byElementId("4:c0a65d96-4993-4b0c-b036-e7ebd9174905:0")
MATCH (n) RETURN n
```

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ Railroad diagram of the snippet



WHERE

The `WHERE` clause is not a clause in its own right — rather, it is subclause used with `MATCH`, `OPTIONAL MATCH`, and `WITH` clauses.

When used with `MATCH` and `OPTIONAL MATCH`, `WHERE` adds constraints to the patterns described. *It should not be seen as a filter after the matching is finished.*

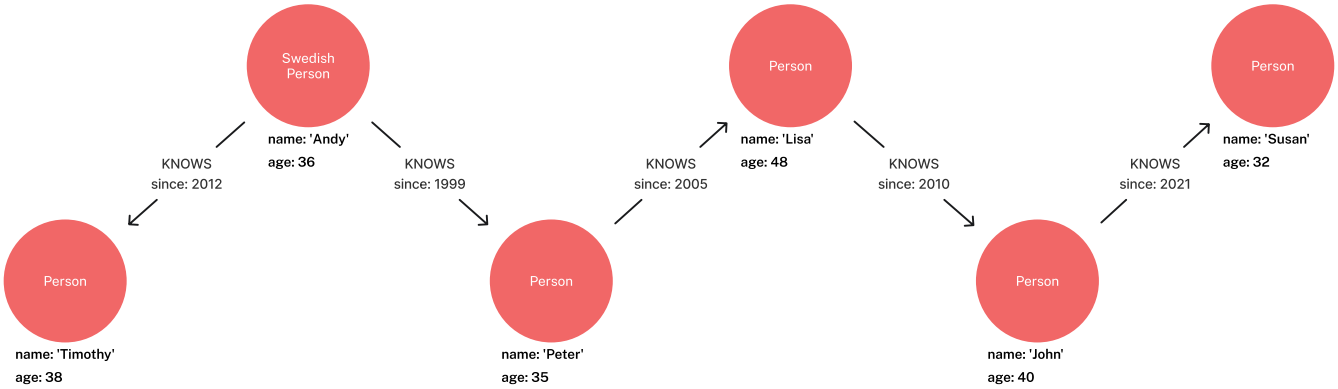
In the case of multiple `MATCH` / `OPTIONAL MATCH` clauses, the predicate in `WHERE` is always a part of the patterns in the directly preceding `MATCH` / `OPTIONAL MATCH`. Both results and performance may be impacted if `WHERE` is put inside the wrong `MATCH` clause.

When used after `WITH`, `WHERE` simply filters the results.

For more uses of `WHERE`, see Predicates.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (andy:Swedish:Person {name: 'Andy', age: 36}),
       (timothy:Person {name: 'Timothy', age: 38}),
       (peter:Person {name: 'Peter', age: 35}),
       (lisa:Person {name: 'Lisa', age: 48}),
       (john:Person {name: 'John', age: 40}),
       (susan:Person {name: 'Susan', age: 32}),
       (andy)-[:KNOWS {since: 2012}]->(timothy),
       (andy)-[:KNOWS {since: 1999}]->(peter),
       (peter)-[:KNOWS {since: 2005}]->(lisa),
       (lisa)-[:KNOWS {since: 2010}]->(john),
       (john)-[:KNOWS {since: 2021}]->(susan)
```

Basic filtering

Filter on a node label

```
MATCH (n)
WHERE n:Swedish
RETURN n.name AS name
```

Result

name
"Andy"
Rows: 1

Filter on a node property

```
MATCH (n:Person)
WHERE n.age < 35
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Susan"	32
Rows: 1	

Filter on a relationship property

```
MATCH (:Person {name:'Andy'})-[k:KNOWS]->(f)
WHERE k.since < 2000
RETURN f.name AS oldFriend
```

Result

oldFriend
"Peter"
Rows: 1

Filter on dynamic labels and types

To filter on a label or relationship type using a dynamically computed value, use `$(<expr>)` where <expr> is any Cypher expression:

Parameters

```
{
  "label": "Swedish"
}
```

Filter on a dynamically computed node label

```
MATCH (n)
WHERE n:$($label)
RETURN labels(n) AS labels
```

Result

labels
["Person", "Swedish"]
Rows: 1

Filter on dynamic properties

To filter on a property using a dynamically computed name, use square brackets `[]`:

Parameters

```
{
  "propname": "age"
}
```

Filter on a dynamically computed node property

```
MATCH (n:Person)
WHERE n[$propname] > 40
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Lisa"	48
Rows: 1	

Using `WHERE` after `WITH`

`WITH` can be used to manipulate the output of a clause before it is passed on to subsequent query parts. Once such a manipulation has occurred, the original clause output is not available to subsequent clauses. For example, in the below query, `WITH` manipulates the output of the preceding `MATCH` in such a way that the succeeding `RETURN` no longer has access to the variable `n` declared in the `MATCH`.

`WITH` *only retains explicitly listed variables; others become inaccessible*

```
MATCH (n:Person)
WITH n.name as name
RETURN n
```

The above query would work if the `RETURN` clause instead referenced the `name` variable produced by `WITH`. However, because `WHERE` is a subclause and not a clause, its scope is not limited by immediately preceding `WITH` clauses.

`WHERE` *is not limited by an immediately preceding* `WITH`

```
MATCH (n:Person)
WITH n.name as name
WHERE n.age = 38
RETURN name
```

Result

name
"Timothy"
Rows: 1

The name for `Timothy` is returned because the `WHERE` clause still acts as a filter on the `MATCH`. However, `WITH` still reduces the scope for the rest of the query moving forward. In this case, `name` is the only variable in scope for the `RETURN` clause.

Filter patterns

`WHERE` clauses can be added to fixed-length and variable-length patterns in order to specify additional constraints.

Fixed-length patterns

`WHERE` *inside a node pattern*

```
WITH 35 AS minAge
MATCH (a:Person WHERE a.name = 'Andy')-[:KNOWS]->(b:Person WHERE b.age > minAge)
RETURN b.name AS name
```

Result

name
"Timothy"
Rows: 1

When used this way, predicates in `WHERE` can reference the node variable that the `WHERE` clause belongs to, but not other elements of the `MATCH` pattern.

The same rule applies to pattern comprehensions:

`WHERE` *inside a pattern comprehension*

```
MATCH (a:Person {name: 'Andy'})
RETURN [(a)-->(b WHERE b:Person) | b.name] AS friends
```

Result

friends
["Peter","Timothy"]
Rows: 1

`WHERE` can also appear inside a relationship pattern in a `MATCH` clause:

`WHERE` *inside relationship pattern*

```
WITH 2000 AS minYear
MATCH (a:Person)-[r:KNOWS WHERE r.since < minYear]->(b:Person)
RETURN a.name AS person, b.name AS friend, r.since AS knowsSince
```

Result

name	friend	knowsSince
"Andy"	"Peter"	1999
Rows: 1		

Relationship pattern predicates can also be used inside pattern comprehensions, where the same caveats apply:

Relationship pattern predicate inside a pattern comprehension

```
WITH 2000 AS minYear
MATCH (a:Person {name: 'Andy'})
RETURN [(a)-[r:KNOWS WHERE r.since < minYear]->(b:Person) | r.since] AS years
```

Result

years
[1999]
Rows: 1

Variable-length patterns

If matching for variable length patterns, `WHERE` can only be used together with the quantified path pattern or quantified relationships syntax.

Allowed - `WHERE` predicate inside a quantified relationship

```
MATCH p = (a:Person {name: "Andy"})-[r:KNOWS WHERE r.since < 2011]->{1,4}(:Person)
RETURN [n IN nodes(p) | n.name] AS paths
```

Note that any paths including `Timothy` and `Susan` are excluded by the `WHERE` predicate, since their incoming `KNOWS` relationships both have a `since` value that is higher than `2011.`

Result

paths
["Andy", "Peter"]
["Andy", "Peter", "Lisa"]
["Andy", "Peter", "Lisa", "John"]
Rows: 3

For more information about using `WHERE` predicates in quantified path patterns, see Predicates in quantified path patterns.

`WHERE` is not allowed in variable-length patterns using the non-GQL compliant variable-length relationship syntax.

Not allowed - `WHERE` inside a variable-length relationship

```
MATCH p = (a:Person {name: 'Andy'})-[r:KNOWS*1..4 WHERE r.since < 2011]->(b:Person)
RETURN [n IN nodes(p) | n.name] AS path
```

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

WITH

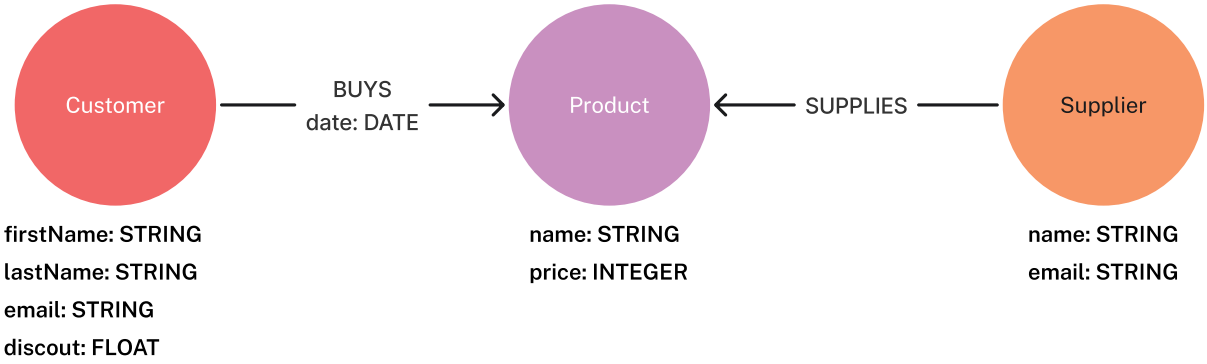
The `WITH` clause serves multiple purposes in Cypher:

- Create new variables
- Control variables in scope
- Bind the results of expressions to new variables
- Perform aggregations
- Remove duplicate values
- Order and paginate results
- Filter results

> As of Cypher 25, `WITH` is no longer required as a separator between a write and a read clause.

Example graph

A graph with the following schema is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database.

```
CREATE (techCorp:Supplier {name: 'TechCorp', email: 'contact@techcorp.com'}),
       (foodies:Supplier {name: 'Foodies Inc.', email: 'info@foodies.com'}),

       (laptop:Product {name: 'Laptop', price: 1000}),
       (phone:Product {name: 'Phone', price: 500}),
       (headphones:Product {name: 'Headphones', price: 250}),
       (chocolate:Product {name: 'Chocolate', price: 5}),
       (coffee:Product {name: 'Coffee', price: 10}),
```

```
       (amir:Customer {firstName: 'Amir', lastName: 'Rahman', email: 'amir.rahman@example.com', discount:
0.1}),
       (keisha:Customer {firstName: 'Keisha', lastName: 'Nguyen', email: 'keisha.nguyen@example.com',
discount: 0.2}),
       (mateo:Customer {firstName: 'Mateo', lastName: 'Ortega', email: 'mateo.ortega@example.com',
discount: 0.05}),
       (hannah:Customer {firstName: 'Hannah', lastName: 'Connor', email: 'hannah.connor@example.com',
discount: 0.15}),
       (leila:Customer {firstName: 'Leila', lastName: 'Haddad', email: 'leila.haddad@example.com',
discount: 0.1}),
       (niko:Customer {firstName: 'Niko', lastName: 'Petrov', email: 'niko.petrov@example.com', discount:
0.25}),
       (yusuf:Customer {firstName: 'Yusuf', lastName: 'Abdi', email: 'yusuf.abdi@example.com', discount:
0.1}),

       (amir)-[:BUYS {date: date('2024-10-09')}]->(laptop),
       (amir)-[:BUYS {date: date('2025-01-10')}]->(chocolate),
       (keisha)-[:BUYS {date: date('2023-07-09')}]->(headphones),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(chocolate),
       (mateo)-[:BUYS {date: date('2025-03-05')}]->(coffee),
       (mateo)-[:BUYS {date: date('2024-04-11')}]->(laptop),
       (hannah)-[:BUYS {date: date('2023-12-11')}]->(coffee),
       (hannah)-[:BUYS {date: date('2024-06-02')}]->(headphones),
       (leila)-[:BUYS {date: date('2023-05-17')}]->(laptop),
       (niko)-[:BUYS {date: date('2025-02-27')}]->(phone),
       (niko)-[:BUYS {date: date('2024-08-23')}]->(headphones),
       (niko)-[:BUYS {date: date('2024-12-24')}]->(coffee),
       (yusuf)-[:BUYS {date: date('2024-12-24')}]->(chocolate),
       (yusuf)-[:BUYS {date: date('2025-01-02')}]->(laptop),

       (techCorp)-[:SUPPLIES]->(laptop),
       (techCorp)-[:SUPPLIES]->(phone),
       (techCorp)-[:SUPPLIES]->(headphones),
       (foodies)-[:SUPPLIES]->(chocolate),
       (foodies)-[:SUPPLIES]->(coffee)
```

Create new variables

`WITH` can be used in combination with the `AS` keyword to bind new variables which can then be passed to subsequent clauses.

Create a new variable

```
WITH [1, 2, 3] AS list
RETURN list
```

Result

list
[1, 2, 3]
Rows: 1

In the below example, the `WITH` clause binds all matched `Customer` nodes to a new variable, `customers`. The bound nodes are `MAP` values which can then be referenced from the new variable.

Create a new variable bound to matched nodes

```
MATCH (c:Customer)-[:BUYS]->(:Product {name: 'Chocolate'})
WITH c AS customers
RETURN customers.firstName AS chocolateCustomers
```

Result

chocolateCustomers
"Amir"
"Mateo"
"Yusuf"
Rows: 3

Control variables in scope

`WITH` can be used to control which variables remain within the scope of a query. Any variable that is referenced by a `WITH` clause remains with the scope of the query and is available to subsequent clauses. If a variable is re-named in a `WITH` clause, it can only be referenced by its new name by subsequent clauses. If a variable is not explicitly referenced in a `WITH` clause, it is dropped from the scope of the query and cannot be referenced by subsequent clauses. To retain all variables in the scope of the query, use `WITH *`.

In the below query, the `WITH` clause de-scopes the `p` variable. As a result, it is not available to the subsequent `RETURN` clause. Nor would the `c` variable be available — only `chocolateCustomers` is available due to the preceding `WITH` clause.

De-scoping a variable

```
MATCH (c:Customer)-[:BUYS]->(p:Product {name: 'Chocolate'})
WITH c.name AS chocolateCustomers
RETURN chocolateCustomers,
       p.price AS chocolatePrice
```

GQLSTATUS error chain

42N62: error: syntax error or access rule violation - variable not defined. Variable `p` not defined.

42001: error: syntax error or access rule violation - invalid syntax

Retain all variables with `WITH *`

```
MATCH (supplier:Supplier)-[r]->(product:Product)
WITH *
RETURN supplier.name AS company,
       type(r) AS relType,
       product.name AS product
```

Result

company	relType	product
"TechCorp"	"SUPPLIES"	"Laptop"
"TechCorp"	"SUPPLIES"	"Phone"
"TechCorp"	"SUPPLIES"	"Headphones"
"Foodies Inc."	"SUPPLIES"	"Chocolate"

company	relType	product
"Foodies Inc."	"SUPPLIES"	"Coffee"
Rows: 5		

`WITH` cannot de-scope variables imported to a `CALL` subquery, because variables imported to a subquery are considered global to its inner scope. More specifically, a variable imported into a `CALL` subquery will be available to subsequent clauses even if a preceding `WITH` clause does not reference it.

In the below example, the `x` variable is imported to the inside scope of a `CALL` subquery, and is successfully referenced by the `RETURN` clause even though the preceding `WITH` neglects to list it.

Variables cannot be de-scoped in the inner scope of a subquery

```
WITH 11 AS x
CALL (x) {
  UNWIND [2, 3] AS y
  WITH y
  RETURN x*y AS a
}
RETURN x, a
```

Result

x	a
11	22
11	33
Rows: 2	

For more information, see `CALL` subqueries → Import variables.

Bind values to variables

`WITH` can be used to assign the values of expressions to variables. In the below query, the value of the `STRING` concatenation expression is bound to a new variable `customerFullName`, and the value from the expression `chocolate.price * (1 - customer.discount)` is bound to `chocolateNetPrice`, both of which are then available in the `RETURN` clause.

Bind values to variables

```
MATCH (customer:Customer)-[:BUYS]->(chocolate:Product {name: 'Chocolate'})
WITH customer.firstName || ' ' || customer.lastName AS customerFullName,
     chocolate.price * (1 - customer.discount) AS chocolateNetPrice
RETURN customerFullName,
       chocolateNetPrice
```

Result

customerFullName	chocolateNetPrice
"Amir Rahman"	4.5

customerFullName	chocolateNetPrice
"Mateo Ortega"	4.75
"Yusuf Abdi"	4.5
Rows: 3	

Because `WITH` can be used to assign variables to the values of expressions, it can be used to chain expressions.

Chain expressions using `WITH`

```
MATCH (p:Product)
WITH p, p.price >= 500 AS isExpensive
WITH p, isExpensive, NOT isExpensive AS isAffordable
WITH p, isExpensive, isAffordable,
     CASE
         WHEN isExpensive THEN 'High-end'
         ELSE 'Budget'
     END AS discountCategory
RETURN p.name AS product,
       p.price AS price,
       isAffordable,
       discountCategory
ORDER BY price
```

Result

product	price	isAffordable	discountCategory
"Chocolate"	5	true	'Budget'
"Coffee"	10	true	'Budget'
"Headphones"	250	true	'Budget'
"Phone"	500	false	'High-end'
"Laptop"	1000	false	'High-end'
Rows: 5			

The `LET` clause can be used to assign values to variables and to chain expressions more clearly and concisely than `WITH`. For more information, see `LET` → Chaining expressions.

Aggregations

The `WITH` clause can perform aggregations and bind the results to new variables. In this example, the `sum()` function is used to calculate the total spent by each customer, and the value for each is bound to the new variable `totalSpent`. The `collect()` function is used to collect each product into `LIST` values bound to the `productsBought` variable.

`WITH` *performing aggregations*

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH c.firstName AS customer,
     sum(p.price) AS totalSpent,
```

```
     collect(p.name) AS productsBought
RETURN customer,
       totalSpent,
       productsBought
ORDER BY totalSpent DESC
```

Result

customer	totalSpent	productsBought
"Mateo"	1015	["Laptop", "Chocolate", "Coffee"]
"Amir"	1005	["Laptop", "Chocolate"]
"Yusuf"	1005	["Laptop", "Chocolate"]
"Leila"	1000	["Laptop"]
"Niko"	760	["Phone", "Headphones", "Coffee"]
"Hannah"	260	["Headphones", "Coffee"]
"Keisha"	250	["Headphones"]

Rows: 7

Remove duplicate values

`WITH` can be used to remove duplicate values from the result set if appended with the modifier `DISTINCT`.

In the below query, `WITH DISTINCT` is used to remove any duplicate `discount` property values from `Customer` nodes.

`WITH DISTINCT` to *remove duplicate values*

```
MATCH (c:Customer)
WITH DISTINCT c.discount AS discountRates
RETURN discountRates
ORDER BY discountRates
```

Result

discountRates
0.05
0.1
0.15
0.2
0.25

Rows: 5

Explicitly project values

Introduced in Neo4j 2025.06 | Cypher 25 only

`WITH ALL` can be used to explicitly project all values bound to a variable. Using it is functionally the same as using simple `WITH`.

Explicit result projection using `WITH ALL`

```
MATCH (c:Customer)
WITH ALL c.discount AS discountRates
RETURN discountRates
ORDER BY discountRates
```

Result

discountRates
0.05
0.1
0.1
0.1
0.15
0.2
0.25
Rows: 7

Ordering and pagination

`WITH` can order and paginate results if used together with the `ORDER BY`, `LIMIT`, and `SKIP` subclauses. If so, these subclauses should be understood as part of the result manipulation performed by `WITH` — not as standalone clauses — before results are passed on to subsequent clauses.

In the below query, the results are ordered in a descending order by which `Customer` has spent the most using `ORDER BY` before they are passed on to the final `RETURN` clause.

Order results with `ORDER BY`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH c,
     sum(p.price) AS totalSpent
  ORDER BY totalSpent DESC
RETURN c.firstName AS customer, totalSpent
```

Result

customer	totalSpent
"Mateo"	1015
"Amir"	1005

customer	totalSpent
"Yusuf"	1005
"Leila"	1000
"Niko"	760
"Hannah"	260
"Keisha"	250

Rows: 7

In the next example, `LIMIT` is used to only retain the top 3 customers with the highest `totalSpent` values in the result set after ordering. Then, the `SET` assigns a new property (`topSpender = true`) to those customers who have spent the most.

Limit results with `LIMIT`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH c,
     sum(p.price) AS totalSpent
  ORDER BY totalSpent DESC
  LIMIT 3
SET c.topSpender = true
RETURN c.firstName AS customer,
       totalSpent,
       c.topSpender AS topSpender
```

customer	totalSpent	topSpender
"Mateo"	1015	true
"Amir"	1005	true
"Yusuf"	1005	true

Rows: 3

`SKIP` can be used after a `WITH` clause to discard rows from the result set. Below, `SKIP` excludes the first 3 rows in the ordered result set (i.e. the 3 `Customer` nodes with highest `totalSpent` value) and assigns a `false` value to the new `topSpender` property of the remaining `Customer` nodes.

Exclude results with `SKIP`

```
MATCH (c:Customer)-[:BUYS]->(p:Product)
WITH c,
     sum(p.price) AS totalSpent
  ORDER BY totalSpent DESC
  SKIP 3
SET c.topSpender = false
RETURN c.firstName AS customer,
       totalSpent,
       c.topSpender AS topSpender
```

customer	totalSpent	topSpender
"Leila"	1000	false

customer	totalSpent	topSpender
"Niko"	760	false
"Hannah"	260	false
"Keisha"	250	false
Rows: 4		

`ORDER BY`, `LIMIT`, and `SKIP` can also be used after a `WITH` clause to narrow down the set of rows before continuing with further pattern matching. In the query below, all products supplied by `Foodies Inc.` are matched first. `WITH` passes those products forward, `ORDER BY` sorts them by descending `price`, and `LIMIT` retains only the most expensive one. The second `MATCH` clause then matches only from that single product to find all customers who bought it.

Control pattern matching scope with ordering and pagination

```
MATCH (:Supplier {name: 'Foodies Inc.'})-[:SUPPLIES]->(p:Product)
WITH p
  ORDER BY p.price DESC
  LIMIT 1
MATCH (p)<-[:BUYS]-(c:Customer)
RETURN p.name AS product,
       p.price AS price,
       collect(c.firstName) AS customers
```

product	price	customers
"Coffee"	10	["Mateo", "Hannah", "Niko"]
Rows: 1		

Filter results

`WITH` can be followed by the `WHERE` subclause to filter results. Similar to the subclauses used for ordering and pagination, `WHERE` should be understood as part of the result manipulation performed by `WITH` — not as a standalone clause — before the results are passed on to subsequent clauses. For more information, see Using `WHERE` after `WITH`.

Filter using `WITH` and `WHERE`

```
UNWIND [1, 2, 3, 4, 5, 6] AS x
WITH x
  WHERE x > 2
RETURN x
```

x
3
4
5
6

x
Rows: 4

In the below query, `WITH` and `WHERE` are used to filter out any `Supplier` nodes whose `totalSales` is less than `1000`. Note the use of `DISTINCT` inside `collect()` to remove any duplicate `Customer` nodes.

Filter property values using `WITH` and `WHERE`

```
MATCH (s:Supplier)-[:SUPPLIES]->(p:Product)<-[:BUYS]-(c:Customer)
WITH s,
     sum(p.price) AS totalSales,
     count(DISTINCT c) AS uniqueCustomers
  WHERE totalSales > 1000
RETURN s.name AS supplier,
       totalSales,
       uniqueCustomers
```

supplier	totalSales	uniqueCustomers
"TechCorp"	5250	7
Rows: 1		

The `FILTER` clause can be used as a more concise alternative to `WITH * WHERE <predicate>` constructs. For more information, see `FILTER` as a substitute for `WITH * WHERE`.

Syntax

Syntax snippet extracted from the full Cypher grammar.

```
Unresolved directive in ../../partials/syntax.adoc - include::{pdf-example-path}syntax/{syntax-
filename}.bnf[]
```

▼ *Railroad diagram of the snippet*

[1] 2147483647 is the `Integer.MAX_VALUE` from Java
[2] 2147483647 is the `Integer.MAX_VALUE` from Java
[3] `NOT` is allowed for boolean properties. For example, given that `seen` has been added as an additional property for filtering when the vector index was created, `WHERE NOT movie.seen` is allowed and will be rewritten to `WHERE movie.seen = false` internally.

Subqueries

A Cypher subquery is called from an enclosing outer query, and executes within its own scope, as defined by `{` and `}`.

For more information, see the following sections:

- CALL subqueries
- CALL subqueries in transactions
- EXISTS subqueries
- COUNT subqueries
- COLLECT subqueries

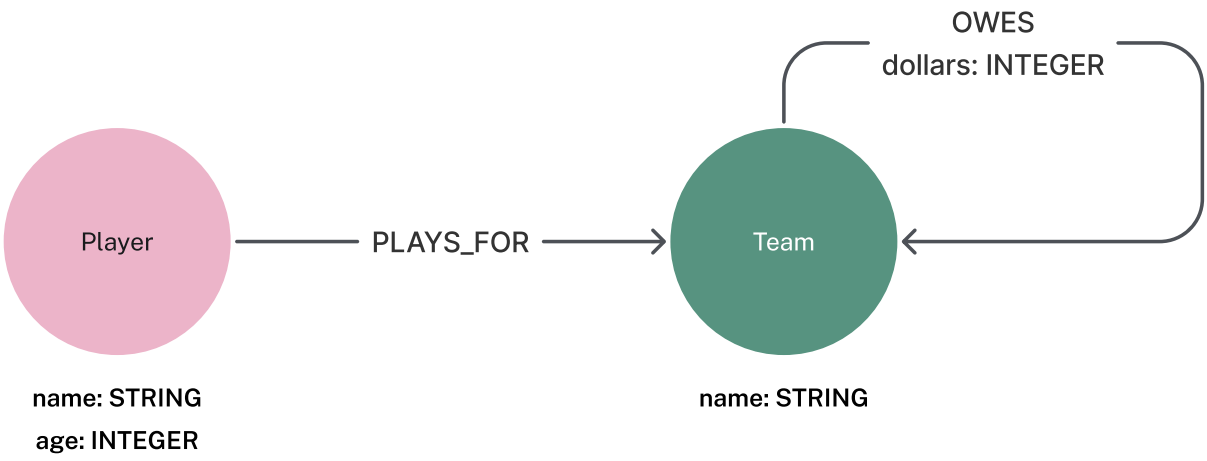
CALL subqueries

The `CALL` clause can be used to invoke subqueries that execute operations within a defined scope, thereby optimizing data handling and query efficiency. Unlike other subqueries in Cypher, `CALL` subqueries can be used to perform changes to the database (e.g. CREATE new nodes).

> The `CALL` clause is also used for calling procedures. For descriptions of the `CALL` clause in this context, refer to the `CALL` procedure.

Example graph

A graph with the following schema is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (teamA:Team {name: 'Team A'}),
       (teamB:Team {name: 'Team B'}),
       (teamC:Team {name: 'Team C'}),
       (playerA:Player {name: 'Player A', age: 21}),
       (playerB:Player {name: 'Player B', age: 23}),
       (playerC:Player {name: 'Player C', age: 19}),
       (playerD:Player {name: 'Player D', age: 30}),
```

```
(playerE:Player {name: 'Player E', age: 25}),
(playerF:Player {name: 'Player F', age: 35}),
(playerA)-[:PLAYS_FOR]->(teamA),
(playerB)-[:PLAYS_FOR]->(teamA),
(playerD)-[:PLAYS_FOR]->(teamB),
(playerE)-[:PLAYS_FOR]->(teamC),
(playerF)-[:PLAYS_FOR]->(teamC),
(teamA)-[:OWES {dollars: 1500}]->(teamB),
(teamA)-[:OWES {dollars: 3000}]->(teamB),
(teamB)-[:OWES {dollars: 1700}]->(teamC),
(teamC)-[:OWES {dollars: 5000}]->(teamB)
```

Semantics and performance

A `CALL` subquery is executed once for each incoming row. The variables returned in a subquery are available to the outer scope of the enclosing query.

Example 82. Basic example

In this example, the `CALL` subquery executes three times, one for each row that the `UNWIND` clause outputs.

Query

```
UNWIND [0, 1, 2] AS x
CALL () {
  RETURN 'hello' AS innerReturn
}
RETURN innerReturn
```

Result

innerReturn
'hello'
'hello'
'hello'
Rows: 3

Each execution of a `CALL` subquery can observe changes from previous executions. This allows for the accumulation of results and the progressive transformation of data within a single Cypher query.

Example 83. Incremental updates

In this example, each iteration of the `CALL` subquery adds 1 to the `age` of `Player A` and the returned `newAge` reflects the `age` after each increment.

Incrementally update the age property of a Player

```
UNWIND [1, 2, 3] AS x
CALL () {
    MATCH (p:Player {name: 'Player A'})
    SET p.age = p.age + 1
    RETURN p.age AS newAge
}
```

```
MATCH (p:Player {name: 'Player A'})
RETURN x AS iteration, newAge, p.age AS totalAge
```

Result

iteration	newAge	totalAge
1	22	24
2	23	24
3	24	24

Rows: 3

The scoping effect of a `CALL` subquery means that the work performed during each execution of each row can be cleaned up as soon its execution ends, before proceeding to the next row. This allows for efficient resource management and reduces memory overhead by ensuring that temporary data structures created during the subquery execution do not persist beyond their usefulness. As a result, `CALL` subqueries can help maintain optimal performance and scalability, especially in complex or large-scale queries.

Example 84. Performance

In this example, a `CALL` subquery is used to `collect` a `LIST` containing all players who play for a particular team.

Collect a list of all players playing for a particular team

```
MATCH (t:Team)
CALL (t) {
  MATCH (p:Player)-[:PLAYS_FOR]->(t)
  RETURN collect(p) as players
}
RETURN t AS team, players
```

Result

team	players
(:Team {name: "Team A"})	(:Player {name: "Player B", age: 23}), (:Player {name: "Player A", age: 24})]
(:Team {name: "Team B"})	[(:Player {name: "Player D", age: 30})]
(:Team {name: "Team C"})	[(:Player {name: "Player F", age: 35}), (:Player {name: "Player E", age: 25})]

Rows: 3

The `CALL` subquery ensures that each `Team` is processed separately (one row per `Team` node), rather than having to hold every `Team` and `Player` node in heap memory simultaneously before collecting them into lists. Using a `CALL` subquery can therefore reduce the amount of heap memory required for an operation.

Importing variables

Variables from the outer scope must be explicitly imported into the inner scope of the `CALL` subquery, either by using a variable scope clause or an importing `WITH` clause (deprecated). As the subquery is evaluated for each incoming input row, the imported variables are assigned the corresponding values from that row.

The variable scope clause

Variables can be imported into a `CALL` subquery using a scope clause: `CALL (<variable>)`. Using the scope clause disables the deprecated importing `WITH` clause.

A scope clause can be used to import all, specific, or none of the variables from the outer scope.

Example 85. Import specific variables from the outer scope

This example only imports the `p` variable from the outer scope and uses it to create a new, randomly generated, `rating` property for each `Player` node. It then returns the `Player` node with the highest `rating`.

Import one variable from the outer scope

```
MATCH (p:Player), (t:Team)
CALL (p) {
  WITH rand() AS random
  SET p.rating = random
  RETURN p.name AS playerName, p.rating AS rating
}
RETURN playerName, rating, t AS team
ORDER BY rating
LIMIT 1
```

Result

playerName	rating	team
"Player C"	0.9307432039870395	"Team A"
Rows: 1		

To import additional variables, include them within the parentheses after `CALL`, separated by commas. For example, to import both variables from the `MATCH` clause in the above query, modify the scope clause accordingly: `CALL (p, t)`.

Example 86. Import all variables

To import all variables from the outer scope, use `CALL (*)`. This example imports both the `p` and `t` variables and sets a new `lastUpdated` property on both.

Import all variables from the outer scope

```
MATCH (p:Player), (t:Team)
CALL (*) {
  SET p.lastUpdated = timestamp()
```

```
  SET t.lastUpdated = timestamp()
}
RETURN p.name AS playerName,
       p.lastUpdated AS playerUpdated,
       t.name AS teamName,
       t.lastUpdated AS teamUpdated
LIMIT 1
```

Result

playerName	playerUpdated	teamName	teamUpdated
"Player A"	1719304206653	"Team A"	1719304206653

Rows: 1

Example 87. Import no variables

To import no variables from the outer scope, use `CALL ()`.

Import no variables from the outer scope

```
MATCH (t:Team)
CALL () {
  MATCH (p:Player)
  RETURN count(p) AS totalPlayers
}
RETURN count(t) AS totalTeams, totalPlayers
```

Result

totalTeams	totalPlayers
3	6

Rows: 1

`CALL` subqueries without a variable scope clause are deprecated.

Deprecated

```
MATCH (t:Team)
CALL {
  MATCH (p:Player)
  RETURN count(p) AS totalPlayers
}
RETURN count(t) AS totalTeams, totalPlayers
```

Rules

- The scope clause's variables can be globally referenced in the subquery. A subsequent `WITH` within the subquery cannot descope an imported variable. The deprecated importing `WITH` clause behaves differently because imported variables can only be referenced from the first line and can be descoped by subsequent clauses.

- Variables cannot be aliased in the scope clause. Only simple variable references are allowed.

Not allowed

```
MATCH (t:Team)
CALL (t AS teams) {
  MATCH (p:Player)-[:PLAYS_FOR]->(teams)
  RETURN collect(p) as players
}
RETURN t AS teams, players
```

- The scope clause's variables cannot be re-declared in the subquery.

Not allowed

```
MATCH (t:Team)
CALL (t) {
  WITH 'New team' AS t
  MATCH (p:Player)-[:PLAYS_FOR]->(t)
  RETURN collect(p) as players
}
RETURN t AS team, players
```

- The subquery cannot return a variable name which already exists in the outer scope. To return imported variables they must be renamed.

Not allowed

```
MATCH (t:Team)
CALL () {
  RETURN 1 AS t
}
RETURN t
```

Importing `WITH` clause

Deprecated

Variables can also be imported into a `CALL` subquery using an importing `WITH` clause. Note that this syntax is not GQL conformant.

Variables imported by `WITH` clause

```
MATCH (t:Team)
CALL {
  WITH t
  MATCH (p:Player)-[:PLAYS_FOR]->(t)
  RETURN collect(p) as players
}
RETURN t AS teams, players
```

▼ *Click to read more about importing variables using the `WITH` clause*

- Just as when using a variable scope clause, a subquery using an importing `WITH` clause cannot return a variable name which already exists in the outer scope. To return imported variables they must be renamed.
- The importing `WITH` clause must be the first clause of a subquery (or the second clause, if directly

following a `USE` clause).

- It is not possible to follow an importing `WITH` clause with any of the following clauses: `DISTINCT`, `ORDER BY`, `WHERE`, `SKIP`, and `LIMIT`.
- When using a `UNION [ALL]` directly within the `CALL` any imported variables must be imported per `UNION` branch.

Attempting any of the above, will throw an error. For example, the following query using a `WHERE` clause after an importing `WITH` clause will throw an error:

Not Allowed

```
UNWIND [[1,2],[1,2,3,4],[1,2,3,4,5]] AS l
CALL {
  WITH l
  WHERE size(l) > 2
  RETURN l AS largeLists
}
RETURN largeLists
```

GQLSTATUS error chain

42I28: error: syntax error or access rule violation - invalid use of importing WITH. Importing WITH can consist only of direct references to outside variables. 'WHERE' is not allowed.

42001: error: syntax error or access rule violation - invalid syntax

A solution to this restriction, necessary for any filtering or ordering of an importing `WITH` clause, is to declare a second `WITH` clause after the importing `WITH` clause. This second `WITH` clause will act as a regular `WITH` clause. For example, the following query will not throw an error:

Allowed

```
UNWIND [[1-2,2],[1,2,3,4],[1,2,3,4,5]] AS l
CALL {
  WITH l
    WITH l
    WHERE size(l) > 2
  RETURN l AS largeLists
}
RETURN largeLists
```

Result

largeLists
[1, 2, 3, 4]
[1, 2, 3, 4, 5]
Rows: 2

Attempting to use an outer variable only imported into the first `UNION` branch in the second throws an error:

Not Allowed

```
WITH "Team A" AS teamName
CALL {
  WITH teamName
  MATCH (p:Player)-[:PLAYS_FOR]->(t {name: teamName})
  RETURN p.name AS associatedName, "Player" AS relationshipType
  UNION
  MATCH (t {name: teamName})-[:OWES]->(other:Team)
  RETURN other.name AS associatedName, "Owed Team" AS relationshipType
}
RETURN teamName, associatedName, relationshipType
```

GQLSTATUS error chain

42N62: error: syntax error or access rule violation - variable not defined. Variable `teamName` not defined.

42001: error: syntax error or access rule violation - invalid syntax

To fix this, import the needed variables separately using an importing `WITH` in all branches.

Allowed

```
WITH "Team A" AS teamName
CALL {
  WITH teamName // Import teamName here
  MATCH (p:Player)-[:PLAYS_FOR]->(t {name: teamName})
  RETURN p.name AS associatedName, "Player" AS relationshipType
  UNION
  WITH teamName // Import teamName here as well
  MATCH (t {name: teamName})-[:OWES]->(other:Team)
  RETURN other.name AS associatedName, "Owed Team" AS relationshipType
}
RETURN teamName, associatedName, relationshipType
```

Result

teamName	associatedName	relationshipType
"Team A"	"Player A"	"Player"
"Team A"	"Player B"	"Player"
"Team A"	"Team B"	"Owed Team"

Rows: 3

Optional subquery calls

`OPTIONAL CALL` allows for optional execution of a `CALL` subquery. Similar to `OPTIONAL MATCH` any empty rows produced by the `OPTIONAL CALL` subquery will return `null`.

Example 88. Difference between using `CALL` and `OPTIONAL CALL`

This example, which finds the team that each `Player` plays for, highlights the difference between using `CALL` and `OPTIONAL CALL`.

Regular subquery `CALL`

```
MATCH (p:Player)
CALL (p) {
    MATCH (p)-[:PLAYS_FOR]->(team:Team)
    RETURN team
}
RETURN p.name AS playerName, team.name AS team
```

Result

playerName	team
"Player A"	"Team A"
"Player B"	"Team A"
"Player D"	"Team B"
"Player E"	"Team C"
"Player F"	"Team C"
Rows: 5	

Note that no results are returned for `Player C`, since they are not connected to any `Team` with a `PLAYS_FOR` relationship.

Query using regular `OPTIONAL CALL`

```
MATCH (p:Player)
OPTIONAL CALL (p) {
    MATCH (p)-[:PLAYS_FOR]->(team:Team)
    RETURN team
}
RETURN p.name AS playerName, team.name AS team
```

Now all `Player` nodes, regardless of whether they have any `PLAYS_FOR` relationships connected to a `Team`, are returned.

Result

playerName	team
"Player A"	"Team A"
"Player B"	"Team A"
"Player C"	null
"Player D"	"Team B"
"Player E"	"Team C"
"Player F"	"Team C"
Rows: 6	

Conditional `CALL` subqueries

Introduced in Neo4j 2025.06 | Cypher 25 only

`WHEN` can be used inside `CALL` subqueries to execute branches conditionally when a predicate evaluates to `true`. Note that the names and the number of columns returned by the different `WHEN` branches must be identical. For more information, see Conditional queries → Conditional subqueries.

Example 89. Conditional `CALL` subqueries

This example uses conditional logic to categorize players based on age by executing `WHEN`/`ELSE` branches, setting the `ageGroup` property as either `"Senior"` or `"Junior"` for each player depending on their age.

Categorize players by age using a conditional WHEN and ELSE for each team.

```
MATCH (t:Team)
OPTIONAL MATCH (p:Player)-[:PLAYS_FOR]->(t)
CALL (*) {
  WHEN p.age > 25 THEN {
    SET p.ageGroup = "Senior"
    RETURN p.name AS player, p.ageGroup AS ageGroup
  }
  ELSE {
     SET p.ageGroup = "Junior"
     RETURN p.name AS player, p.ageGroup AS ageGroup
  }
}
RETURN player, ageGroup
```

Result

player	ageGroup
"Player A"	"Junior"
"Player B"	"Junior"
"Player D"	"Senior"
"Player E"	"Junior"
"Player F"	"Senior"
Rows: 5	

Execution order of CALL subqueries

The order in which rows from the outer scope are passed into subqueries is not defined. If the results of the subquery depend on the order of these rows, use an `ORDER BY` clause before the `CALL` clause to guarantee a specific processing order for the rows.

Example 90. Ordering results before `CALL` subquery

This example creates a linked list of all `Player` nodes in order of ascending `age`.

The `CALL` clause is relying on the incoming row ordering to ensure that a correctly ordered linked list is created, thus the incoming rows must be ordered with a preceding `ORDER BY` clause.

Order results before a `CALL` subquery

```
MATCH (player:Player)
WITH player
ORDER BY player.age ASC LIMIT 1
  SET player:ListHead
WITH *
MATCH (nextPlayer: Player&!ListHead)
WITH nextPlayer
ORDER BY nextPlayer.age
CALL (nextPlayer) {
  MATCH (current:ListHead)
    REMOVE current:ListHead
    SET nextPlayer:ListHead
    CREATE(current)-[:IS_YOUNGER_THAN]->(nextPlayer)
  RETURN current AS from, nextPlayer AS to
}
RETURN
  from.name AS name,
  from.age AS age,
  to.name AS closestOlderName,
  to.age AS closestOlderAge
```

Result

name	age	closestOlderName	closestOlderAge
"Player C"	19	"Player B"	23
"Player B"	23	"Player A"	24
"Player A"	24	"Player E"	25
"Player E"	25	"Player D"	30
"Player D"	30	"Player F"	35

Rows: 5

Post-union processing

Call subqueries can be used to further process the results of a `UNION` query.

Example 91. Using `UNION` within a `CALL` subquery

This example query finds the youngest and the oldest `Player` in the graph.

Find the oldest and youngest players

```
CALL () {
  MATCH (p:Player)
  RETURN p
  ORDER BY p.age ASC
  LIMIT 1
UNION
  MATCH (p:Player)
  RETURN p
  ORDER BY p.age DESC
  LIMIT 1
```

```
}
RETURN p.name AS playerName, p.age AS age
```

Result

playerName	age
"Player C"	19
"Player F"	35

Rows: 2

If different parts of a result should be matched differently, with some aggregation over the whole result, subqueries need to be used. The example below query uses a `CALL` subquery in combination with `UNION ALL` to determine how much each `Team` in the graph owes or is owed.

Find how much every team is owed

```
MATCH (t:Team)
CALL (t) {
  OPTIONAL MATCH (t)-[o:OWES]->(other:Team)
  RETURN o.dollars * -1 AS moneyOwed
  UNION ALL
  OPTIONAL MATCH (other)-[o:OWES]->(t)
  RETURN o.dollars AS moneyOwed
}
RETURN t.name AS team, sum(moneyOwed) AS amountOwed
ORDER BY amountOwed DESC
```

Result

team	amountOwed
"Team B"	7800
"Team C"	-3300
"Team A"	-4500

Rows: 3

Aggregations

Returning subqueries change the number of results of the query. The result of the `CALL` subquery is the combined result of evaluating the subquery for each input row.

Example 92. `CALL` subquery changing returned rows of outer query

The following example finds the name of each `Player` and the team they play for. No rows are returned for `Player C`, since they are not connected to a `Team` with a `PLAYS_FOR` relationship. The number of results of the subquery thus changed the number of results of the enclosing query.

Find the friends of players

```
MATCH (p:Player)
```

```
CALL (p) {
  MATCH (p)-[:PLAYS_FOR]->(team:Team)
  RETURN team.name AS team
}
RETURN p.name AS playerName, team
```

Result

playerName	team
"Player A"	"Team A"
"Player B"	"Team A"
"Player D"	"Team B"
"Player E"	"Team C"
"Player F"	"Team C"

Rows: 5

Example 93. `CALL` subqueries and isolated aggregations

Subqueries can also perform isolated aggregations. The below example uses the sum() function to count how much money is owed between the `Team` nodes in the graph. Note that the `owedAmount` for `Team A` is the aggregated results of two `OWES` relationships to `Team B`.

Find how much each team owes

```
MATCH (t:Team)
CALL (t) {
  MATCH (t)-[o:OWES]->(t2:Team)
  RETURN sum(o.dollars) AS owedAmount, t2.name AS owedTeam
}
RETURN t.name AS owingTeam, owedAmount, owedTeam
```

Result

owingTeam	owedAmount	owedTeam
"Team A"	4500	"Team B"
"Team B"	1700	"Team C"
"Team C"	5000	"Team B"

Rows: 4

Note on returning subqueries and unit subqueries

The examples above have all used subqueries which end with a `RETURN` clause. These subqueries are called *returning subqueries*.

A subquery is evaluated for each incoming input row. Every output row of a returning subquery is combined with the input row to build the result of the subquery. That means that a returning subquery will

influence the number of rows. If the subquery does not return any rows, there will be no rows available after the subquery.

Subqueries without a `RETURN` statement are called *unit subqueries*. Unit subqueries are used for their ability to alter the graph with clauses such as `CREATE`, `MERGE`, `SET`, and `DELETE`. They do not explicitly return anything, and this means that the number of rows present after the subquery is the same as was going into the subquery.

Unit subqueries

Unit subqueries are used for their ability to alter the graph with updating clauses. They do not impact the amount of rows returned by the enclosing query.

This example query creates 3 clones of each existing `Player` node in the graph. As the subquery is a unit subquery, it does not change the number of rows of the enclosing query.

Create cloned nodes

```
MATCH (p:Player)
CALL (p) {
  UNWIND range (1, 3) AS i
  CREATE (:Person {name: p.name})
}
RETURN count(*)
```

Result

count(*)
6

Rows: 1

Nodes created: 18

Properties set: 18

Labels added: 18

Summary

- `CALL` subqueries optimize data handling and query efficiency, and can perform changes to the database.
- `CALL` subqueries allow for row-by-row data transformation and enable the accumulation of results across multiple rows, facilitating complex operations that depend on intermediate or aggregated data.
- `CALL` subqueries can only refer to variables from the enclosing query if they are explicitly imported by either a variable scope clause or an importing `WITH` clause (deprecated).
- All variables that are returned from a `CALL` subquery are afterwards available in the enclosing query.
- Returning subqueries (with `RETURN` clause) influence the number of output rows, while unit subqueries (without `RETURN` clause) perform graph updates without changing the number of rows.
- An `ORDER BY` clause can be used before `CALL` subqueries to ensure a specific order.
- `CALL` subqueries can be used in combination with `UNION` to process and aggregate different parts of a

query result.

CALL subqueries in transactions

`CALL` subqueries can be made to execute in separate, inner transactions, producing intermediate commits. This can be useful when doing large write operations, like batch updates, imports, and deletes.

To execute a `CALL` subquery in separate transactions, add the modifier `IN TRANSACTIONS` after the subquery. An outer transaction is opened to report back the accumulated statistics for the inner transactions (created and deleted nodes, relationships, etc.) and it will succeed or fail depending on the results of those inner transactions. By default, inner transactions group together batches of 1000 rows. Cancelling the outer transaction will cancel the inner ones as well.

> `CALL { … } IN TRANSACTIONS` is only allowed in implicit transactions.
> If you are using Neo4j Browser, you must prepend any queries using `CALL { … } IN TRANSACTIONS` with `:auto`.

> The examples on this page use a variable scope clause to import variables into the `CALL` subquery.

Syntax

```
CALL {
    subQuery
} IN [[concurrency] CONCURRENT] TRANSACTIONS
[OF batchSize ROW[S]]
[DISJOINT BY {NONE | AUTO | (<expr>, ...)}]
[REPORT STATUS AS statusVar]
[ON ERROR {CONTINUE | BREAK | FAIL | RETRY [FOR] [duration SEC[OND[S]]] [THEN {CONTINUE | BREAK | FAIL}]}]
```

Loading CSV data

This example uses a CSV file and the `LOAD CSV` clause to import data into the database. It creates nodes in separate transactions using `CALL { … } IN TRANSACTIONS`:

friends.csv

```
1,Bill,26
2,Max,27
3,Anna,22
4,Gladys,29
5,Summer,24
```

Query

```
LOAD CSV FROM 'file:///friends.csv' AS line
CALL (line) {
  CREATE (:Person {name: line[1], age: toInteger(line[2])})
} IN TRANSACTIONS
```

Result

```
(empty result)
```

Rows: 0

Nodes created: 5

Properties set: 10

Labels added: 5

Transactions committed: 1

As the size of the CSV file in this example is small, only a single separate transaction is started and committed.

Deleting a large volume of data

Using `CALL { … } IN TRANSACTIONS` is the recommended way of deleting a large volume of data.

Example 94. DETACH DELETE on all nodes

Query

```
MATCH (n)
CALL (n) {
  DETACH DELETE n
} IN TRANSACTIONS
```

Result

```
(empty result)
```

Rows: 0

Nodes deleted: 5

Relationships deleted: 2

Transactions committed: 1

Example 95. DETACH DELETE on only some nodes

The `CALL { … } IN TRANSACTIONS` subquery should not be modified.

Any necessary filtering can be done before the subquery.

Query

```
MATCH (n:Label) WHERE n.prop > 100
CALL (n) {
  DETACH DELETE n
} IN TRANSACTIONS
```

Result

```
(empty result)
```

Rows: 0

Note:

The batching is performed on the input rows fed into `CALL { … } IN TRANSACTIONS`, so the data must be supplied from outside the call in order for the batching to have an effect. That is why the nodes are matched outside the subqueries in the examples above. If the `MATCH` clause were inside the subquery, the data deletion would run as one single transaction.

Batching

The amount of work to do in each separate transaction can be specified in terms of how many input rows to process before committing the current transaction and starting a new one. The number of input rows is set with the modifier `OF n ROWS` (or `OF n ROW`). If omitted, the default batch size is `1000` rows. The number of rows can be expressed using any expression that evaluates to a positive integer and does not refer to nodes or relationships.

This example loads a CSV file with one transaction for every `2` input rows:

friends.csv

```
1,Bill,26
2,Max,27
3,Anna,22
4,Gladys,29
5,Summer,24
```

Query

```
LOAD CSV FROM 'file:///friends.csv' AS line
CALL (line) {
  CREATE (:Person {name: line[1], age: toInteger(line[2])})
} IN TRANSACTIONS OF 2 ROWS
```

Result

```
(empty result)
```

Rows: 0

Nodes created: 5

Properties set: 10

Labels added: 5

Transactions committed: 3

The query now starts and commits three separate transactions:

1. The first two executions of the subquery (for the first two input rows from `LOAD CSV`) take place in the first transaction.
2. The first transaction is then committed before proceeding.
3. The next two executions of the subquery (for the next two input rows) take place in a second transaction.
4. The second transaction is committed.

5. The last execution of the subquery (for the last input row) takes place in a third transaction.
6. The third transaction is committed.

You can also use `CALL { … } IN TRANSACTIONS OF n ROWS` to delete all your data in batches in order to avoid a huge garbage collection or an `OutOfMemory` exception. For example:

Query

```
MATCH (n)
CALL (n) {
  DETACH DELETE n
} IN TRANSACTIONS OF 2 ROWS
```

Result

```
(empty result)
```

Rows: 0

Nodes deleted: 9

Relationships deleted: 2

Transactions committed: 5

> **Note:**
>
> Up to a point, using a larger batch size will be more performant. The batch size of `2 ROWS` is an example given the small data set used here. For larger data sets, you might want to use larger batch sizes, such as `10000 ROWS`.

Composite databases

`CALL { … } IN TRANSACTIONS` can be used with composite databases.

Even though composite databases allow accessing multiple graphs in a single query, only one graph can be modified in a single transaction. `CALL { … } IN TRANSACTIONS` offers a way of constructing queries which modify multiple graphs.

While the previous examples are generally valid for composite databases, there's a few extra factors that come into play when working with composite databases in subqueries. The following examples show how you can use `CALL { … } IN TRANSACTIONS` on a composite database.

> **Important:**
>
> Your application needs to be connected to the composite database when submitting queries involving its constituents. Select the composite database in UI tools or when running queries via client libraries.

Example 96. Import a CSV file on all constituents

friends.csv

```
1,Bill,26
2,Max,27
3,Anna,22
4,Gladys,29
5,Summer,24
```

Create `Person` nodes on all constituents, drawing data from friends.csv

```
UNWIND graph.names() AS graphName
LOAD CSV FROM 'file:///friends.csv' AS line
CALL (graphName, line) {
  USE graph.byName( graphName )
  CREATE (:Person {name: line[1], age: toInteger(line[2])})
} IN TRANSACTIONS
```

Example 97. Remove all nodes and relationships from all constituents

Query

```
UNWIND graph.names() AS graphName
CALL (graphName) {
  USE graph.byName( graphName )
  MATCH (n)
  RETURN elementId(n) AS id
}
CALL (graphName, id) {
  USE graph.byName( graphName )
  WITH id
  MATCH (n)
  WHERE elementId(n) = id
  DETACH DELETE n
} IN TRANSACTIONS
```

Since the batching is performed on the input rows fed into `CALL { … } IN TRANSACTIONS`, the data must be supplied from outside the subquery in order for the batching to have an effect. That is why the nodes are matched in a subquery *preceding* the one that actually deletes the data. If the `MATCH` clause were inside the second subquery, the data deletion would run as one single transaction.

Note:

There is currently a known issue. When an error occurs during `CALL { … } IN TRANSACTIONS` processing, the error message includes information about how many transactions have been committed. That information is inaccurate on composite databases, as it always reports `(Transactions committed: 0)`.

Note:

`CALL { … } IN CONCURRENT TRANSACTIONS` currently provides no extra parallelism for

composite databases. The subquery is executed in parallel on each target graph by default.

Batch size in composite databases

Because `CALL { … } IN TRANSACTIONS` subqueries targeting different graphs can't be interleaved, if a `USE` clause evaluates to a different target than the current one, the current batch is committed and the next batch is created.

The batch size declared with `IN TRANSACTIONS OF … ROWS` represents an upper limit of the batch size, but the real batch size depends on how many input rows target one database **in sequence**. Every time the target database changes, the batch is committed.

Example 98. Behavior of `IN TRANSACTIONS OF ROWS` on composite databases

The next example assumes the existence of two constituents `remoteGraph1` and `remoteGraph2` for the composite database `composite`.

While the declared batch size is 3, only the first 2 rows act on `composite.remoteGraph1`, so the batch size for the first transaction is 2. That is followed by 3 rows on `composite.remoteGraph2`, 1 on `composite.remoteGraph2` and finally 2 on `composite.remoteGraph1`.

Query

```
WITH ['composite.remoteGraph1', 'composite.remoteGraph2'] AS graphs
UNWIND [0, 0, 1, 1, 1, 1, 0, 0] AS i
WITH graphs[i] AS g
CALL (g) {
  USE graph.byName( g )
  CREATE ()
} IN TRANSACTIONS OF 3 ROWS
```

Error behavior

`CALL { … } IN TRANSACTIONS` has four different behavioral options in case an error occurs in any of the inner transactions: `ON ERROR CONTINUE`, `ON ERROR BREAK`, `ON ERROR FAIL`, and `ON ERROR RETRY`.

If an error occurs, any inner transactions that were successfully committed remain unchanged and are not rolled back. However, any inner transactions that failed are fully rolled back. This behavior applies regardless of which `ON ERROR` option is used.

`ON ERROR CONTINUE`

`ON ERROR CONTINUE` ignores recoverable errors and continues the execution of subsequent inner transactions. The outer transaction succeeds. When an inner query fails, `ON ERROR CONTINUE` ensures the outer transaction continues, returning `null` for the failed inner query.

Example 99. `ON ERROR CONTINUE`

In the below query, the last subquery execution in the second inner transaction fails due to division by

zero:

Subquery with failing transaction

```
UNWIND [4, 2, 1, 0] AS i
CALL (i) {
  CREATE (:Person {num: 100/i}) // Note, fails when i = 0
} IN TRANSACTIONS OF 2 ROWS
RETURN i
```

GQLSTATUS error chain

22N28: error: data exception - overflow error. The result of the operation '/ by zero' has caused an overflow.

22012: error: data exception - division by zero

Since the failure occurred after the first transaction was committed, the database retains the successfully created nodes.

Return nodes created prior to failed transaction

```
MATCH (e:Person)
RETURN e.num
```

Result

e.num
25
50
Rows: 2

In the following example, `ON ERROR CONTINUE` is used after a failed inner transaction to execute the remaining inner transactions and not fail the outer transaction:

Transactions batched in 1 row with `ON ERROR CONTINUE`

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR CONTINUE
RETURN n.num
```

Result

n.num
100
null
50

n.num
25
Rows: 4

Note the difference in results when batching in transactions of 2 rows:

Transactions batched in 2 rows with `ON ERROR CONTINUE`

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i})
  RETURN n
} IN TRANSACTIONS
  OF 2 ROWS
  ON ERROR CONTINUE
RETURN n.num
```

Result

n.num
null
null
50
25
Rows: 4

In this case, the first inner transaction includes `i = 1` and `i = 0`. Since `i = 0` causes an error, the entire transaction is rolled back, resulting in `null` for both elements.

`ON ERROR BREAK`

`ON ERROR BREAK` ignores recoverable errors and stops the execution of subsequent inner transactions. The outer transaction succeeds. When an inner query fails, `ON ERROR BREAK` ensures the outer transaction continues but stops executing any further inner transactions. Expected variables from the failed inner query are bound as `null` for all subsequent transactions, including the failed one.

Example 100. `ON ERROR BREAK`

In this example, `ON ERROR BREAK` ensures that once an inner transaction fails `(i = 0)`, no further inner transactions execute, while the outer transaction remains successful.

Transactions batched in 1 row with `ON ERROR BREAK`

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR BREAK
```

```
RETURN n.num
```

Result

n.num
100
null
null
null
Rows: 4

When batching in transactions of 2 rows, the first transaction (`i = 1, 0`) encounters an error at `i = 0`, causing execution to stop immediately. As a result, all remaining transactions also return null.

Transactions batched in 2 rows with `ON ERROR BREAK`

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i})
  RETURN n
} IN TRANSACTIONS
  OF 2 ROWS
  ON ERROR BREAK
RETURN n.num
```

Result

n.num
null
null
null
null
Rows: 4

`ON ERROR FAIL`

`ON ERROR FAIL` acknowledges a recoverable error and stops the execution of subsequent inner transactions, causing the outer transaction to fail.

`ON ERROR FAIL` is the default behavior if no error-handling flag is explicitly specified.

Example 101. `ON ERROR FAIL`

In the following example, `ON ERROR FAIL` is used after the failed inner transaction to prevent the remaining inner transactions from executing and to cause the outer transaction to fail as well.

Transactions batched in 1 row with `ON ERROR FAIL`

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR FAIL
RETURN n.num
```

GQLSTATUS error chain

22N28: error: data exception - overflow error. The result of the operation '/ by zero' has caused an overflow.

22012: error: data exception - division by zero

`ON ERROR RETRY`

`ON ERROR RETRY` uses an exponential delay between retry attempts for transaction batches that fail due to transient errors (i.e. errors where retrying a transaction can be expected to give a different result), with an optional maximum retry duration. If the transaction still fails after the maximum duration, the failure is handled according to an optionally specified fallback error handling mode (`THEN CONTINUE`, `THEN BREAK`, `THEN FAIL` (default)).

`ON ERROR RETRY` increases query robustness by handling transient errors without manual intervention. It is particularly suitable for concurrent transactions, reducing the likelihood of a query failing due to deadlocks. For concurrent transactions, `PROFILE` reports the number of retries. For more information, see Measuring concurrency with `PROFILE`.

Example 102. Basic retry with default duration

The below example demonstrates a basic retry scenario. If a transient error occurs during the creation of a `User` node, the transaction will be retried for the default maximum retry duration (`30` seconds). If the retry succeeds, the query continues.

If the retry fails after the default duration, the query fails because it behaves like `ON ERROR FAIL` (the default fallback).

`ON ERROR RETRY`

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY
```

Specify a maximum retry duration

The default maximum retry duration is `30` seconds. A new default can be set using `dbms.cypher.transactions.default_subquery_retry_timeout`.

A maximum retry duration can be specified for individual queries as follows: `ON ERROR RETRY [FOR] <duration> SEC[OND[S]]`, where <duration> is an `INTEGER` or `FLOAT` value representing seconds that is greater than or equal to `0`. Decimal values are allowed, and the `<duration>` can be set with a parameter. Note that this `<duration>` overrides the default value.

The duration timer starts when the first retry is scheduled. As a result, regardless of the specified duration, a minimum of one retry will be attempted. If the transaction still fails with a transient error, a new attempt with made unless the duration has expired. For example, a value of `0` (or close to `0`) will result in a retry, but guarantees that only a single retry will be attempted.

Example 103. Set a maximum time limit for retries

In this example, the retry duration is explicitly set to `2.5` seconds. This means that the transaction will be retried until it succeeds or until `2.5` seconds have elapsed.

`ON ERROR RETRY` *with a set duration*

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY FOR 2.5 SECONDS
```

Parameter

```
{
  "duration": 10
}
```

`ON ERROR RETRY` *using a parameter for the duration*

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY FOR $duration SECONDS
```

Fallback error handling options

`ON ERROR RETRY` can be combined with the other `ON ERROR` options via the `THEN` clause to specify a fallback behavior. The fallback behavior specifies what will happen if a transaction has not succeeded within the time limit. Specifically:

- `ON ERROR RETRY … THEN CONTINUE`: the query will ignore recoverable errors and continue with the execution of subsequent inner transactions. The outer transaction succeeds, and `null` will be returned for any failed inner transactions. See `ON ERROR CONTINUE` for more information about this behavior.
- `ON ERROR RETRY … THEN BREAK`: the query will ignore recoverable errors and stop the execution of subsequent inner transactions. The outer transaction succeeds, and `null` will be returned for the failed inner transaction and all subsequent ones. See `ON ERROR BREAK` for more information about this behavior.
- `ON ERROR RETRY … THEN FAIL` (default): the query will acknowledge a recoverable error and stop the

execution of subsequent inner transactions, causing the outer transaction to fail. See `ON ERROR FAIL` for more information about this behavior.

Because `THEN FAIL` is the default fallback option it does not have to be specified.

ON ERROR RETRY THEN CONTINUE

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY FOR 1 SECOND THEN CONTINUE
```

ON ERROR RETRY THEN BREAK

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY FOR 1 SECOND THEN BREAK
```

ON ERROR RETRY THEN FAIL

```
UNWIND range(1, 100) AS i
CALL (i) {
    MERGE (u:User {id: i})
    ON CREATE SET u.created = timestamp()
} IN TRANSACTIONS ON ERROR RETRY FOR 1 SECOND THEN FAIL
```

Status report

Users can also report the execution status of the inner transactions by using `REPORT STATUS AS var`. This flag is disallowed for `ON ERROR FAIL`. For more information, see Error behavior.

After each execution of the inner query finishes (successfully or not), a status value is created that records information about the execution and the transaction that executed it:

- If the inner execution produces one or more rows as output, then a binding to this status value is added to each row, under the selected variable name.
- If the inner execution fails then a single row is produced containing a binding to this status value under the selected variable, and null bindings for all variables that should have been returned by the inner query (if any).

The status value is a map value with the following fields:

- `started`: `true` when the inner transaction was started, `false` otherwise.
- `committed`, `true` when the inner transaction changes were successfully committed, `false` otherwise.
- `transactionId`: the inner transaction id, or `null` if the transaction was not started.
- `errorMessage`, the inner transaction error message, or `null` in case of no error.

Example of reporting status with `ON ERROR CONTINUE`:

Query

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR CONTINUE
  REPORT STATUS AS s
RETURN n.num, s
```

Result

n.num	s
100	{"committed": true, "errorMessage": null, "started": true, "transactionId": "neo4j-transaction-835" }
null	{"committed": false, "errorMessage": "/ by zero", "started": true, "transactionId": "neo4j-transaction-836" }
50	{"committed": true, "errorMessage": null, "started": true, "transactionId": "neo4j-transaction-837" }
25	{"committed": true, "errorMessage": null, "started": true, "transactionId": "neo4j-transaction-838" }

Rows: 4

Example of reporting status with `ON ERROR BREAK`:

Query

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR BREAK
  REPORT STATUS AS s
RETURN n.num, s.started, s.committed, s.errorMessage
```

Result

n.num	s.started	s.committed	s.errorMessage
100	true	true	null
null	true	false	"/ by zero"
null	false	false	null
null	false	false	null

Rows: 4

Reporting status with `ON ERROR FAIL` is disallowed:

Query

```
UNWIND [1, 0, 2, 4] AS i
CALL (i) {
  CREATE (n:Person {num: 100/i}) // Note, fails when i = 0
  RETURN n
} IN TRANSACTIONS
  OF 1 ROW
  ON ERROR FAIL
  REPORT STATUS AS s
RETURN n.num, s.errorMessage
```

Error

```
REPORT STATUS can only be used when specifying ON ERROR CONTINUE or ON ERROR BREAK
```

Concurrent transactions

By default, `CALL { … } IN TRANSACTIONS` is single-threaded; one CPU core is used to sequentially execute batches.

However, `CALL` subqueries can also execute batches in parallel by appending `IN [n] CONCURRENT TRANSACTIONS`, where `n` is a concurrency value used to set the maximum number of transactions that can be executed in parallel. This allows `CALL` subqueries to utilize multiple CPU cores simultaneously, which can significantly reduce the time required to execute a large, outer transaction.

The concurrency value is optional. If not specified, a default value based on the amount of available CPU cores will be chosen. If a negative number is specified (which can only be done through a parameter), the concurrency will be the number of available CPU cores reduced by the absolute value of that number. To measure the concurrency a query achieves, see Measuring concurrency with `PROFILE`.

Queries using `CONCURRENT TRANSACTIONS` can currently only use the slotted runtime.

When using `CONCURRENT TRANSACTIONS` on a composite database, there is no additional parallelism in the query execution. See CALL subqueries in transactions for composite databases for more information.

Example 104. Load a CSV file in concurrent transactions

`CALL { … } IN CONCURRENT TRANSACTIONS` is particularly suitable for importing data without dependencies. This example creates `Person` nodes from a unique `tmdbId` value assigned to each person row in the CSV file (444 in total) in 3 concurrent transactions.

`CALL` *subquery run in* `CONCURRENT TRANSACTIONS`

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
CALL (row) {
  CREATE (p:Person {tmdbId: row.person_tmdbId})
  SET p.name = row.name, p.born = row.born
} IN 3 CONCURRENT TRANSACTIONS OF 10 ROWS
RETURN count(*) AS personNodes
```

Result

personNodes
444
Rows: 1

Concurrency and non-deterministic results

`CALL { … } IN TRANSACTIONS` uses *ordered* semantics by default, where batches are committed in a sequential row-by-row order. For example, in `CALL { <I> } IN TRANSACTIONS`, any writes done in the execution of `<I`$_1$`>` must be observed by `<I`$_2$`>`, and so on.

In contrast, `CALL { … } IN CONCURRENT TRANSACTIONS` uses *concurrent* semantics, where both the number of rows committed by a particular batch and the order of committed batches is undefined. That is, in `CALL { <I> } IN CONCURRENT TRANSACTIONS`, writes committed in the execution of `<I`$_1$`>` may or may not be observed by `<I`$_2$`>`, and so on.

The results of `CALL` subqueries executed in concurrent transactions may, therefore, not be deterministic. To guarantee deterministic results, ensure that the results of committed batches are not dependent on each other.

Using `CALL { … } IN CONCURRENT TRANSACTIONS` can impact error behavior. Specifically, when using `ON ERROR BREAK` or `ON ERROR FAIL` and one transaction fails, then any concurrent transactions *may* not be interrupted and rolled back (though subsequent ones would). This is because no timing guarantees can be given for concurrent transactions. That is, an ongoing transaction may or may not commit successfully in the time window when the error is being handled. Use the status report to determine which batches were committed and which failed or did not start.

Deadlocks

When a write transaction occurs, Neo4j takes locks to preserve data consistency while updating. For example, when creating or deleting a relationship, a write lock is taken on both the specific relationship and its connected nodes.

A deadlock happens when two transactions are blocked by each other because they are attempting to concurrently modify a node or a relationship that is locked by the other transaction (for more information about locks and deadlocks in Neo4j, see Operations Manual → Concurrent data access.

A deadlock may occur when using `CALL { … } IN CONCURRENT TRANSACTIONS` if the transactions for two or more batches try to take the same locks in an order that results in a circular dependency between them. If so, the impacted transactions are always rolled back, and an error is thrown unless the query is appended with one of the following error options:

- `ON ERROR CONTINUE`
- `ON ERROR BREAK`

- `ON ERROR RETRY`

The latter is particularly suited for concurrent transactions, because it retries recoverable transient errors with exponential backoff between retries until the maximum retry duration has been reached.

> Deadlock detection and transaction retries can be time-consuming. When importing data that includes a significant number of relationships to be merged between the same nodes but processed in different batches, increasing the concurrency may not enhance performance. On the contrary, it could slow down the import process.

Example 105. Finding and retrying deadlocked transactions

The following query tries to create `Movie` and `Year` nodes connected by a `RELEASED_IN` relationship. Note that there are only three different years in the CSV file, meaning that only three `Year` nodes should be created.

Query with concurrent transaction causing a deadlock

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
CALL (row) {
    MERGE (m:Movie {movieId: row.movieId})
    MERGE (y:Year {year: row.year})
    MERGE (m)-[r:RELEASED_IN]->(y)
} IN 2 CONCURRENT TRANSACTIONS OF 10 ROWS
```

The deadlock occurs because the two transactions are simultaneously trying to lock and merge the same `Year`.

GQLSTATUS error chain

50N05: error: general processing exception - deadlock detected. Deadlock detected while trying to acquire locks. See log for more details.

Query using `ON ERROR CONTINUE` to ignore deadlocks and complete outer transaction

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
CALL (row) {
   MERGE (m:Movie {movieId: row.movieId})
   MERGE (y:Year {year: row.year})
   MERGE (m)-[r:RELEASED_IN]->(y)
} IN 2 CONCURRENT TRANSACTIONS OF 10 ROWS ON ERROR CONTINUE REPORT STATUS as status
WITH status
WHERE status.errorMessage IS NOT NULL
RETURN status.transactionId AS transaction, status.committed AS commitStatus, status.errorMessage AS
errorMessage
```

Result

```
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------+
| transaction             | commitStatus | errorMessage
|
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------+
```

```
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
```

```
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
| "neo4j-transaction-486" | false        | "ForsetiClient[transactionId=486, clientId=8] can't
acquire ExclusiveLock{owner=ForsetiClient[transactionId=485, clientId=13]} on
NODE_RELATIONSHIP_GROUP_DELETE(18) because holders of that lock are waiting for
ForsetiClient[transactionId=486, clientId=8].
|
|                          |              \  Wait list:ExclusiveLock[
|
|                          |              \ Client[485] waits for [ForsetiClient[transactionId=486,
clientId=8]]]"
|
+-------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------+
```

These are transient errors, meaning that re-running the transactions may be successful. To retry the any failed inner transactions, use the error option `ON ERROR RETRY`, which retries any failing transactions until the maximum retry duration has been reached.

The following query uses `ON ERROR RETRY … THEN CONTINUE` to retry the above query for a maximum of `3` seconds and then continue the execution of subsequent inner transactions by ignoring any recoverable errors.

Query using `ON ERROR RETRY … THEN CONTINUE` to retry deadlocked inner transactions and complete outer transaction

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
CALL (row) {
   MERGE (m:Movie {movieId: row.movieId})
   MERGE (y:Year {year: row.year})
   MERGE (m)-[r:RELEASED_IN]->(y)
} IN 2 CONCURRENT TRANSACTIONS OF 10 ROWS ON ERROR RETRY FOR 3 SECONDS THEN CONTINUE REPORT STATUS AS
status
RETURN status.transactionId as transaction, status.committed AS successfulTransaction
```

The result shows that all transactions are now successful:

Result

```
+-------------------------------------------------+
| transaction             | successfulTransaction |
+-------------------------------------------------+
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-500" | true                  |
| "neo4j-transaction-501" | true                  |
| "neo4j-transaction-501" | true                  |
| "neo4j-transaction-501" | true                  |
| "neo4j-transaction-501" | true                  |
| "neo4j-transaction-501" | true                  |
```

```
| "neo4j-transaction-501" | true                |
| "neo4j-transaction-501" | true                |
| "neo4j-transaction-501" | true                |
| "neo4j-transaction-501" | true                |
| "neo4j-transaction-501" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-502" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-504" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-503" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-505" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-506" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-507" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-508" | true                |
| "neo4j-transaction-509" | true                |
| "neo4j-transaction-509" | true                |
```

```
| "neo4j-transaction-509" | true                 |
+-------------------------------------------------+
```

Deadlock resolution and transaction retries are time-consuming, however. The preferred strategy for dealing with deadlocks should instead be one of prevention.

Batch scheduling and deadlock prevention with `DISJOINT BY`

Introduced in Neo4j 2026.06 | Cypher 25 only

Batch scheduling prevents deadlocks in concurrent transactions by ensuring that batches sharing resources never run at the same time, while batches that do not share resources run concurrently. This can be particularly useful when importing relationships. To check how much batch scheduling limits the concurrency of a query, see Measuring concurrency with `PROFILE`.

The default batch scheduling strategy for all queries run against a database can be configured with the `dbms.cypher.transactions.default_subquery_batch_strategy` setting. Accepted values for this setting are `NONE` (no scheduling), `AUTO` (automatic inference), and `DEFAULT` (the product default, currently `NONE`).

Alternatively, `DISJOINT BY` can be added to individual `CALL { … } IN CONCURRENT TRANSACTIONS` statements to control batch scheduling at the query level.

There are three forms of `DISJOINT BY`:

- `DISJOINT BY (<expr>, …)`: declares lock-prone resources explicitly per row.
- `DISJOINT BY AUTO`: the runtime infers lock-prone resources automatically.
- `DISJOINT BY NONE`: disables scheduling explicitly, overriding any database-level default.

Note:

- `DISJOINT BY` overrides the database-wide setting. For example, if `dbms.cypher.transactions.default_subquery_batch_strategy` is set to `AUTO`, a query with `DISJOINT BY NONE` will run without batch scheduling.
- `DISJOINT BY` can only be run together with concurrent transactions; it is not valid in a regular `CALL { … } IN TRANSACTIONS` subquery.

Declaring lock expressions with `DISJOINT BY`

`DISJOINT BY (<expr>, …)` allows you to declare the expressions identifying locked resources for a `CALL { … } IN CONCURRENT TRANSACTIONS` statement.

The expressions must use only variables imported from the outer query, since they are evaluated before the subquery runs; for example, `elementId(n)` or `n.id`, where `n` is an imported variable. They act as constraints on batch composition: the runtime ensures that no two concurrent batches share a value for any of the declared expressions.

For example, given `DISJOINT BY (n.id, n.category)`, two batches are scheduled concurrently only if they

share no values for either `n.id` or `n.category`. Any row in one batch that shares a value for either expression with a row in another batch causes those batches to be run sequentially. With a batch size of `2` (i.e. `IN CONCURRENT TRANSACTIONS OF 2 ROWS DISJOINT BY (n.id, n.category)`), the batching and execution could look something like this:

Simplified batch scheduling model

```
row 1: id=1, category=A
row 2: id=2, category=A
row 3: id=3, category=A
row 4: id=4, category=A
row 5: id=5, category=B
row 6: id=6, category=B

Batches:
  Batch A: rows 1, 2  (category=A)
  Batch B: rows 3, 4  (category=A)
  Batch C: rows 5, 6  (category=B)

Execution:
  Batch A ------->
                  Batch B -------->
  Batch C ------------------------>
```

Batch A and Batch B share `category=A` and are therefore serialized; Batch C shares no values with either and runs concurrently throughout.

No automatic lock expression inference is performed when using `DISJOINT BY (<expr>, …)`; the supplied list is the entire lock expression set for that subquery.

Example 106. `DISJOINT BY (<expr>, …)`: importing nodes and relationships

This example imports movies and their release years from a CSV file, creating `Movie` and `Year` nodes and `RELEASED_IN` relationships between them. For this import, key constraints are in place to ensure the existence of unique `movieId` properties for `Movie` nodes and `year` properties for `Year` nodes, and to provide index-backed lookups for these properties (key constraints are backed by indexes).

With these constraints and indexes, two concurrent batches attempting to `MERGE` the same `Movie` or `Year` node simultaneously will contend on the key constraint lock, causing a deadlock.

Create node key constraints

```
CREATE CONSTRAINT movie_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.movieId IS KEY;
CREATE CONSTRAINT year IF NOT EXISTS FOR (y:Year) REQUIRE y.year IS KEY;
```

Import with explicit lock expressions

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS line
CALL (line) {
  MERGE (m:Movie {movieId: line.movieId})
  MERGE (y:Year {year: line.year})
  MERGE (m)-[:RELEASED_IN]->(y)
} IN CONCURRENT TRANSACTIONS
  DISJOINT BY (line.year, line.movieId)
  ON ERROR RETRY THEN CONTINUE
```

`DISJOINT BY (line.year, line.movieId)` declares that `line.year` and `line.movieId` identify the locked resources for each batch.

`ON ERROR RETRY THEN CONTINUE` provides a fallback for transient errors.

Automatic lock inference with `DISJOINT BY AUTO`

`DISJOINT BY AUTO` instructs the runtime to infer lock expressions automatically from the subquery body, without requiring the user to declare them explicitly. This is useful when the locked resources are straightforward to infer statically; for example, `MERGE` patterns on uniquely constrained properties. Indexes on the merged properties are required in order for `DISJOINT BY AUTO` to infer locking expressions. If the runtime cannot infer any lock expressions, the subquery executes with no scheduling restriction.

To see which lock expressions (if any) were inferred by the planner, prepend the statement with `EXPLAIN`.

Example 107. `DISJOINT BY AUTO`: importing actors and movies

This example imports actors and their roles from a CSV file, creating `Person` and `Movie` nodes and `ACTS_IN` relationships between them. With key constraints (and backing indexes) on the `name` properties of `Person` nodes and the `title` properties of `Movie` nodes, the runtime infers the lock expressions `(line.person, line.movie)` automatically from the `MERGE` patterns as can be seen in the `TransactionForEach` operator `Details` column in the query plan produced by prepending the statement with `EXPLAIN` (the same expressions a user would write manually with `DISJOINT BY (line.person, line.movie)`).

Create node key constraints

```
CREATE CONSTRAINT person_name IF NOT EXISTS FOR (p:Person) REQUIRE p.name IS KEY;
CREATE CONSTRAINT movie_title IF NOT EXISTS FOR (m:Movie) REQUIRE m.title IS KEY;
```

Import with automatic lock inference

```
EXPLAIN
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/intro/movies/actors.csv' AS line
CALL (line) {
  MERGE (p:Person {name: line.person})
  MERGE (m:Movie  {title: line.movie})
  CREATE (p)-[:ACTS_IN {role: line.roles}]->(m)
} IN CONCURRENT TRANSACTIONS
  DISJOINT BY AUTO
```

Query plan

```
+----------------------+----
+-------------------------------------------------------------------------------------------
+----------------+
| Operator             | Id | Details
| Estimated Rows |
+----------------------+----
+-------------------------------------------------------------------------------------------
+----------------+
| +ProduceResults      |  0 |
|             10 |
| |                    +----
+-------------------------------------------------------------------------------------------
+----------------+
| +EmptyResult         |  1 |
|             10 |
| |                    +----
+-------------------------------------------------------------------------------------------
+----------------+
| +TransactionForeach  |  2 | IN CONCURRENT TRANSACTIONS OF 1000 ROWS DISJOINT BY (line.person,
```

```
line.movie) ON ERROR FAIL |            10 |
| |\                  +----
+-----------------------------------------------------------------------------------------------
+----------------+
| | +Create           |  3 | (p)-[anon_0:ACTS_IN {role: line.roles}]->(m)
|             10 |
| | |                 +----
+-----------------------------------------------------------------------------------------------
+----------------+
| | +Apply            |  4 |
|             10 |
| | |\                +----
+-----------------------------------------------------------------------------------------------
+----------------+
| | | +MergeUniqueNode |  5 | UNIQUE m:Movie(title) WHERE title = line.movie,
|             10 |
| | |                 +----
+-----------------------------------------------------------------------------------------------
+----------------+
| | +MergeUniqueNode  |  6 | UNIQUE p:Person(name) WHERE name = line.person,
|             10 |
| |                   +----
+-----------------------------------------------------------------------------------------------
+----------------+
| +LoadCSV          |  7 | line
|             10 |
+---------------------+----
+-----------------------------------------------------------------------------------------------
+----------------+
```

Disabling batch scheduling with `DISJOINT BY NONE`

`DISJOINT BY NONE` explicitly disables batch scheduling, overriding `dbms.cypher.transactions.default_subquery_batch_strategy`.

Example 108. `DISJOINT BY NONE`: importing independent nodes

This example imports movies from a CSV file, creating `Movie` nodes. With a key constraint on the `movieId` property of `Movie` nodes, each row is guaranteed to operate on a distinct node; concurrent batches cannot contend on the same resource, making batch scheduling unnecessary.

Importing nodes without batch scheduling

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS line
CALL (line) {
  MERGE (m:Movie {movieId: line.movieId})
} IN CONCURRENT TRANSACTIONS
  DISJOINT BY NONE
```

Deadlock prevention by task division

Deadlock prevention can also be achieved by dividing a task into two distinct subqueries: a data-independent subquery run in parallel with maximum concurrency, and a data-dependent subquery executed serially (i.e. one transaction at a time) to avoid deadlocks. In the below example, nodes and properties are created in a concurrent subquery, while the relationships connecting those nodes are created in a serial subquery. This method benefits from the performance of concurrent transactions while avoiding deadlocks.

Avoiding deadlocks by dividing an import task into a data-independent concurrent subquery followed by a data-dependent serial subquery

```
LOAD CSV WITH HEADERS FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
CALL (row) {
   MERGE (m:Movie {movieId: row.movieId})
   MERGE (y:Year {year: row.year})
   RETURN m, y
} IN CONCURRENT TRANSACTIONS OF 10 ROWS ON ERROR RETRY THEN CONTINUE REPORT STATUS AS nodeStatus
CALL (m, y) {
   MERGE (m)-[r:RELEASED_IN]->(y)
} IN TRANSACTIONS OF 10 ROWS ON ERROR RETRY THEN CONTINUE REPORT STATUS AS relationshipStatus
RETURN nodeStatus.transactionId as nodeTransaction,
       nodeStatus.committed AS successfulNodeTransaction,
       relationshipStatus.transactionId as relationshipTransaction,
       relationshipStatus.committed AS successfulRelationshipTransaction
```

Measuring concurrency with `PROFILE`

Introduced in Neo4j 2026.10

The concurrency value in `IN [n] CONCURRENT TRANSACTIONS` sets the maximum number of batches that can run at the same time. The concurrency a query actually achieves can be lower. For example, the input rows may be produced too slowly to keep all transactions busy, or batch scheduling with `DISJOINT BY` may hold back batches that conflict with batches already running.

To see how much concurrency a query achieved, and what limited it, prepend the query with `PROFILE`. The `TransactionApply` or `TransactionForeach` operator that runs the concurrent transactions then records concurrency statistics. The query plan shows the peak and average number of batches running at the same time in the `Concurrency` column, for example `peak 4 / avg 3.64`. The average shows how well the concurrency was sustained.

The `Concurrency` column shows only the peak and the average. The complete set of statistics is included in the profiled plan returned to drivers and the HTTP API, as a map in the `concurrent-transactions` argument of the operator. Statistics are only recorded for `IN CONCURRENT TRANSACTIONS`, not for `IN TRANSACTIONS`.

The statistics follow each batch through two states:

- *Queued*: the batch is complete and waiting to start, because the concurrency limit is reached or because the batch conflicts with a running batch.
- *Running*: the batch is being executed in its inner transaction, until that transaction has committed or rolled back.

A batch that is retried with `ON ERROR RETRY` passes through both states again, and each attempt is counted separately. The delay between retry attempts is not counted as queued time.

Concurrency statistics

Statistic	Description
`concurrencyLimit`	The maximum number of batches allowed to run at the same time. This is the concurrency value `n` of `IN [n] CONCURRENT TRANSACTIONS`, resolved to an actual number if `n` is omitted or negative.

Statistic	Description
`peakConcurrency`	The highest number of batches running at the same time.
`avgConcurrency`	The time-weighted average number of batches running at the same time, measured from when the first batch started until the last batch finished.
`peakQueuedBatches`	The highest number of batches queued at the same time.
`avgQueuedBatches`	The time-weighted average number of queued batches, measured from when the first batch was queued until the last batch finished.
`peakStartGapNanos`	The longest time, in nanoseconds, that a batch spent queued before it started running.
`avgStartGapNanos`	The average time, in nanoseconds, that a batch spent queued before it started running.
`batchesQueued`	The number of batch attempts that were queued. With `DISJOINT BY`, batch scheduling can start a batch before it is full, so this can be higher than the number of rows divided by the batch size.
`batchesStarted`	The number of batch attempts that started running. This is lower than `batchesQueued` if the query stopped before all queued batches had run, for example because of an error.
`innerTransactionsStarted`	The number of inner transactions that were started.
`innerTransactionsCommitted`	The number of inner transactions that were committed successfully. The difference from `innerTransactionsStarted` is the number of inner transactions that failed and were rolled back.
`retries`	The number of times that a failed batch was scheduled to run again by `ON ERROR RETRY`. A retry is counted when it is scheduled, even if the query ends before the retried batch runs.

Read the statistics against `concurrencyLimit`:

- **`avgConcurrency` close to `concurrencyLimit`**: the transactions were busy almost all the time. If `avgQueuedBatches` is also high, a higher concurrency value may make the query faster if CPU cores are available.
- **Low `avgConcurrency` and `avgQueuedBatches` close to `0`**: batches were not produced fast enough to keep the transactions busy. The bottleneck is the part of the query that produces the input rows, for example `LOAD CSV`.
- **Low `avgConcurrency` and high `avgQueuedBatches`**: batches were ready but could not start. This happens when many batches share values for the `DISJOINT BY` expressions, because such batches cannot run at the same time, or when the part of the query after the subquery consumes the returned rows too slowly for new batches to be started.
- **High `retries`**: many batches failed with transient errors, such as deadlocks, and were run again. Batch scheduling with `DISJOINT BY` can prevent deadlocks between concurrent batches.

Example 109. Measure the concurrency of an import with `PROFILE`

This example creates 50,000 `TRANSFER` relationships between 1,000 `Account` nodes in batches of 100 rows, with 4 concurrent transactions. Because many rows refer to the same accounts, `DISJOINT BY (sourceId, targetId)` ensures that batches that share an account never run at the same time.

Create a key constraint

```
CREATE CONSTRAINT account_id IF NOT EXISTS FOR (a:Account) REQUIRE a.id IS KEY
```

Profile a concurrent import

```
PROFILE
UNWIND range(1, 50000) AS i
WITH i, i % 1000 AS sourceId, (i * 7) % 1000 AS targetId
CALL (i, sourceId, targetId) {
  MERGE (a:Account {id: sourceId})
  MERGE (b:Account {id: targetId})
  CREATE (a)-[:TRANSFER {seq: i}]->(b)
} IN 4 CONCURRENT TRANSACTIONS OF 100 ROWS
  DISJOINT BY (sourceId, targetId)
  ON ERROR RETRY
```

Query plan

```
Cypher 25

Planner COST

Runtime SLOTTED

Runtime version 2026.10

+----------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| Operator             | Id | Details
| Estimated Rows | Rows  | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Concurrency       |
+----------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| +ProduceResults      |  0 |
|             10 |     0 |       0 |              0 |                    0/0 |                   |
| |                    +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| +EmptyResult         |  1 |
|             10 |     0 |       0 |                |                    0/0 |                   |
| |                    +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| +TransactionForeach  |  2 | IN $autoint_6 CONCURRENT TRANSACTIONS OF $autoint_5 ROWS DISJOINT BY
(sourceId, targetId) ON ERROR R |             10 | 50000 |       0 |         661416 |
297433/4 | peak 4 / avg 3.64 |
| |                    |    | ETRY THEN FAIL
|                |       |         |                |                        |                   |
| |\                   +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| | +Create            |  3 | (a)-[anon_0:TRANSFER {seq: i}]->(b)
|             10 | 50000 |  150000 |                |               193715/0 |                   |
| | |                  +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| | +Apply             |  4 |
|             10 | 50000 |       0 |                |                    0/0 |                   |
| | |\                 +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| | | +MergeUniqueNode |  5 | UNIQUE b:Account(id) WHERE id = targetId,
|             10 | 50000 |  100000 |                |               100384/0 |                   |
| | |                  +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
| | +Apply             |  6 |
|             10 | 50000 |       0 |                |                    0/0 |                   |
| | |\                 +----
+-------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+-------------------+
```

```
| | | +MergeUniqueNode |  7 | UNIQUE a:Account(id) WHERE id = sourceId,
|             10 | 50000 |  100000 |                |               100408/0 |                    |
| | |                 +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+--------------------+
| | +Argument          |  8 | i, sourceId, targetId
|             10 | 50000 |       0 |                |                    0/0 |                    |
| |                 +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+--------------------+
| +Projection          |  9 | i % $autoint_2 AS sourceId, (i * $autoint_3) % $autoint_4 AS targetId
|             10 | 50000 |       0 |                |                    0/0 |                    |
| |                 +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+--------------------+
| +Unwind              | 10 | range($autoint_0, $autoint_1) AS i
|             10 | 50000 |       0 |                |                    0/0 |                    |
+----------------------+----
+-----------------------------------------------------------------------------------------------------------
--+----------------+-------+---------+----------------+------------------------+--------------------+

Total database accesses: 350000, total allocated memory: 2883584
```

The `Concurrency` column of the `TransactionForeach` operator shows `peak 4 / avg 3.64`: at some point all four transactions were running, and on average 3.64 batches ran at the same time. The query therefore stayed close to its concurrency limit of `4` for most of its execution, and batch scheduling did not noticeably restrict it. The exact values vary between runs.

Restrictions

These are the restrictions on queries that use `CALL { … } IN TRANSACTIONS`:

- A nested `CALL { … } IN TRANSACTIONS` inside a `CALL { … }` clause is not supported.
- A `CALL { … } IN TRANSACTIONS` in a `UNION` is not supported.
- A `CALL { … } IN TRANSACTIONS` after a write clause is not supported, unless that write clause is inside a `CALL { … } IN TRANSACTIONS`.

Further reading

- Neo4j Developer Blog: Introducing Concurrent Writes to Cypher Subqueries

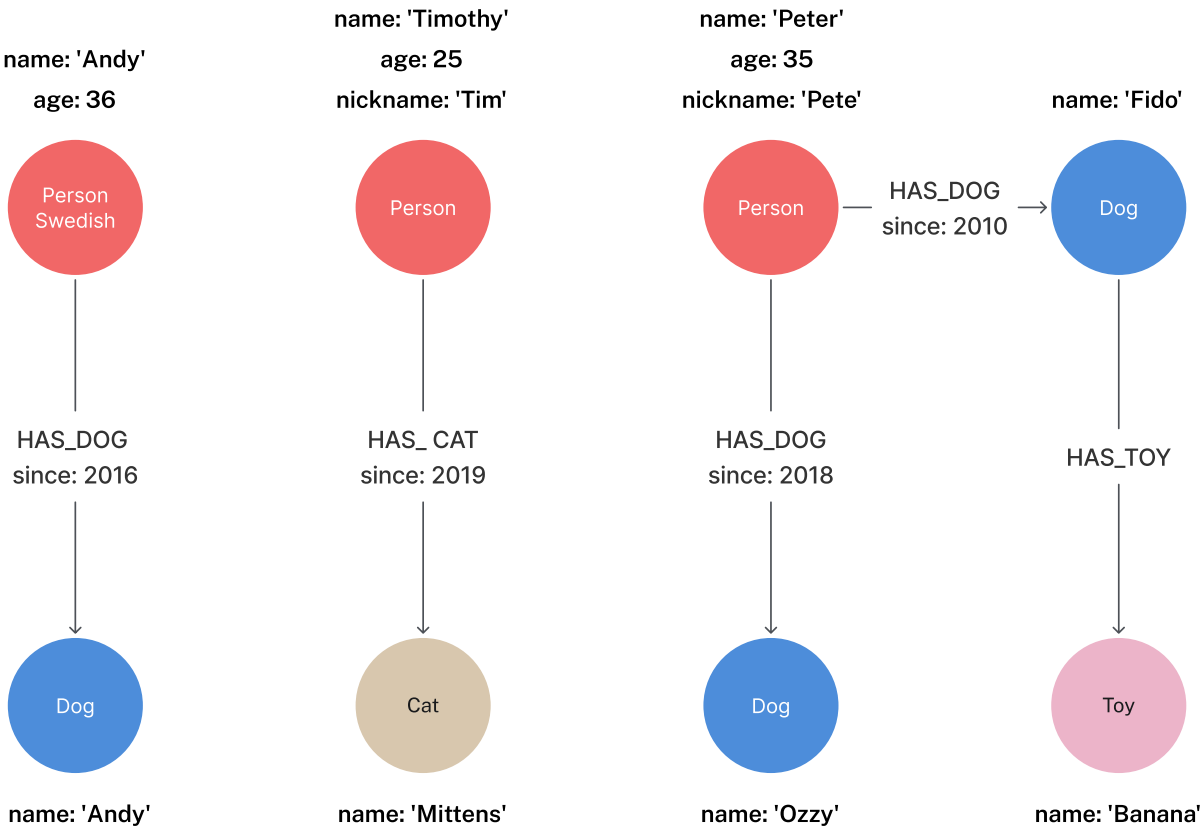
COLLECT subqueries

A `COLLECT` subquery expression can be used to create a list with the rows returned by a given subquery.

`COLLECT` subqueries differ from `COUNT` and `EXISTS` subqueries in that the final `RETURN` clause is mandatory. The `RETURN` clause must return exactly one column.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
(andy:Swedish:Person {name: 'Andy', age: 36}),
(timothy:Person {name: 'Timothy', nickname: 'Tim', age: 25}),
(peter:Person {name: 'Peter', nickname: 'Pete', age: 35}),
(andy)-[:HAS_DOG {since: 2016}]->(:Dog {name:'Andy'}),
(timothy)-[:HAS_CAT {since: 2019}]->(:Cat {name:'Mittens'}),
(fido:Dog {name:'Fido'})<-[:HAS_DOG {since: 2010}]-(peter)-[:HAS_DOG {since: 2018}]->(:Dog {name:'Ozzy'}),
(fido)-[:HAS_TOY]->(:Toy{name:'Banana'})
```

Simple `COLLECT` subquery

Variables introduced by the outside scope can be used in the `COLLECT` subquery without importing them. In this regard, `COLLECT` subqueries are different from `CALL` subqueries, which do require importing. The following query exemplifies this and outputs the owners of the dog named `Ozzy`:

```
MATCH (person:Person)
WHERE 'Ozzy' IN COLLECT { MATCH (person)-[:HAS_DOG]->(dog:Dog) RETURN dog.name }
RETURN person.name AS name
```

name
"Peter"
Rows: 1

`COLLECT` subquery with `WHERE` clause

A `WHERE` clause can be used inside the `COLLECT` subquery. Variables introduced by the `MATCH` clause and the

outside scope can be used in the inner scope.

```
MATCH (person:Person)
RETURN person.name as name, COLLECT {
  MATCH (person)-[r:HAS_DOG]->(dog:Dog)
  WHERE r.since > 2017
  RETURN dog.name
} as youngDogs
```

name	youngDogs
"Andy"	[]
"Timothy"	[]
"Peter"	["Ozzy"]
Rows: 3	

Conditional `COLLECT` subquery

Introduced in Neo4j 2025.06 | Cypher 25 only

`WHEN` can be used inside `COLLECT` subqueries to execute branches conditionally when a predicate evaluates to `true`. Note that the names and the number of columns returned by the different `WHEN` branches must be identical. For more information, see Conditional queries → Conditional subqueries.

In the example below query, the `WHEN` branch is executed if the person has a dog, while the `ELSE` branch runs for people without a dog.

Conditional `COLLECT` subquery

```
MATCH (n:Person)
RETURN n.name AS name,
       COLLECT {
         WHEN exists((n)-[:HAS_DOG]->(:Dog)) THEN {
          RETURN 'Dog owner' AS petStatus
         }
         ELSE {
          RETURN 'Cat owner' AS petStatus
         }
       } AS petStatus
```

name	petStatus
"Andy"	["Dog owner"]
"Timothy"	["Cat owner"]
"Peter"	["Dog owner"]
Rows: 3	

`COLLECT` subquery with a `UNION`

`COLLECT` can be used with a `UNION` clause. The below example shows the collection of pet names each

person has by using a `UNION` clause:

```
MATCH (person:Person)
RETURN
    person.name AS name,
    COLLECT {
        MATCH (person)-[:HAS_DOG]->(dog:Dog)
        RETURN dog.name AS petName
        UNION
        MATCH (person)-[:HAS_CAT]->(cat:Cat)
        RETURN cat.name AS petName
    } AS petNames
```

name	petNames
"Andy"	["Andy"]
"Timothy"	["Mittens"]
"Peter"	["Ozzy", "Fido"]

Rows: 3

`COLLECT` subquery with `WITH`

Variables from the outside scope are visible for the entire subquery, even when using a `WITH` clause. To avoid confusion, shadowing of these variables is not allowed. An outside scope variable is shadowed when a newly introduced variable within the inner scope is defined with the same variable. In the example below, the outer variable `name` is shadowed and will therefore throw an error.

```
WITH 'Peter' as name
MATCH (person:Person {name: name})
RETURN COLLECT {
    WITH 'Ozzy' AS name
    MATCH (person)-[r:HAS_DOG]->(d:Dog {name: name})
    RETURN d.name
} as dogsOfTheYear
```

GQLSTATUS error chain

42N07: error: syntax error or access rule violation - variable shadowing. The variable `name` is shadowing a variable with the same name from the outer scope and needs to be renamed.

42001: error: syntax error or access rule violation - invalid syntax

New variables can be introduced into the subquery, as long as they use a different identifier. In the example below, a `WITH` clause introduces a new variable. Note that the outer scope variable `person` referenced in the main query is still available after the `WITH` clause.

```
MATCH (person:Person)
RETURN person.name AS name, COLLECT {
    WITH 2018 AS yearOfTheDog
    MATCH (person)-[r:HAS_DOG]->(d:Dog)
    WHERE r.since = yearOfTheDog
    RETURN d.name
} as dogsOfTheYear
```

name	dogsOfTheYear
"Andy"	[]
"Timothy"	[]
"Peter"	["Ozzy"]
Rows: 3	

Using `COLLECT` subqueries inside other clauses

`COLLECT` can be used in any position in a query, with the exception of administration commands, where the `COLLECT` expression is restricted. See a few examples below of how `COLLECT` can be used in different positions within a query:

Using `COLLECT` in `RETURN`

```
MATCH (person:Person)
RETURN person.name,
       COLLECT {
            MATCH (person)-[:HAS_DOG]->(d:Dog)
            MATCH (d)-[:HAS_TOY]->(t:Toy)
            RETURN t.name
       } as toyNames
```

person.name	toyNames
"Andy"	[]
"Timothy"	[]
"Peter"	["Banana"]
Rows: 3	

Using `COLLECT` in `SET`

```
MATCH (person:Person) WHERE person.name = "Peter"
SET person.dogNames = COLLECT { MATCH (person)-[:HAS_DOG]->(d:Dog) RETURN d.name }
RETURN person.dogNames as dogNames
```

dogNames
["Ozzy", "Fido"]
Rows: 1 Properties set: 1

Using `COLLECT` in `CASE`

```
MATCH (person:Person)
RETURN
```

```
  CASE
    WHEN COLLECT { MATCH (person)-[:HAS_DOG]->(d:Dog) RETURN d.name } = []  THEN "No Dogs " + person.name
    ELSE person.name
  END AS result
```

result
"Andy"
"No Dogs Timothy"
"Peter"
Rows: 3

Using `COLLECT` as a grouping key

The following query collects all persons by their dogs' names, and then calculates the average age for each group.

```
MATCH (person:Person)
RETURN COLLECT { MATCH (person)-[:HAS_DOG]->(d:Dog) RETURN d.name } AS dogNames,
       avg(person.age) AS averageAge
 ORDER BY dogNames
```

dogNames	averageAge
[]	25.0
["Andy"]	36.0
["Ozzy", "Fido"]	35.0
Rows: 3	

Using `COLLECT` vs `collect()`

`COLLECT` does not handle `null` values in the same way that the aggregating function `collect()` does. The `collect()` function automatically removes `null` values. `COLLECT` will not remove `null` values automatically. However, they can be removed by adding a filtering step in the subquery.

The following queries illustrate these differences:

```
MATCH (p:Person)
RETURN collect(p.nickname) AS names
```

names
["Pete", "Tim"]
Rows: 1

```
RETURN COLLECT {
```

```
        MATCH (p:Person)
        RETURN p.nickname ORDER BY p.nickname
      } AS names
```

names
["Pete", "Tim", null]
Rows: 1

```
RETURN COLLECT {
        MATCH (p:Person)
        WHERE p.nickname IS NOT NULL
        RETURN p.nickname ORDER BY p.nickname
      } AS names
```

name
["Pete", "Tim"]
Rows: 1

Rules

The following is true for `COLLECT` subqueries:

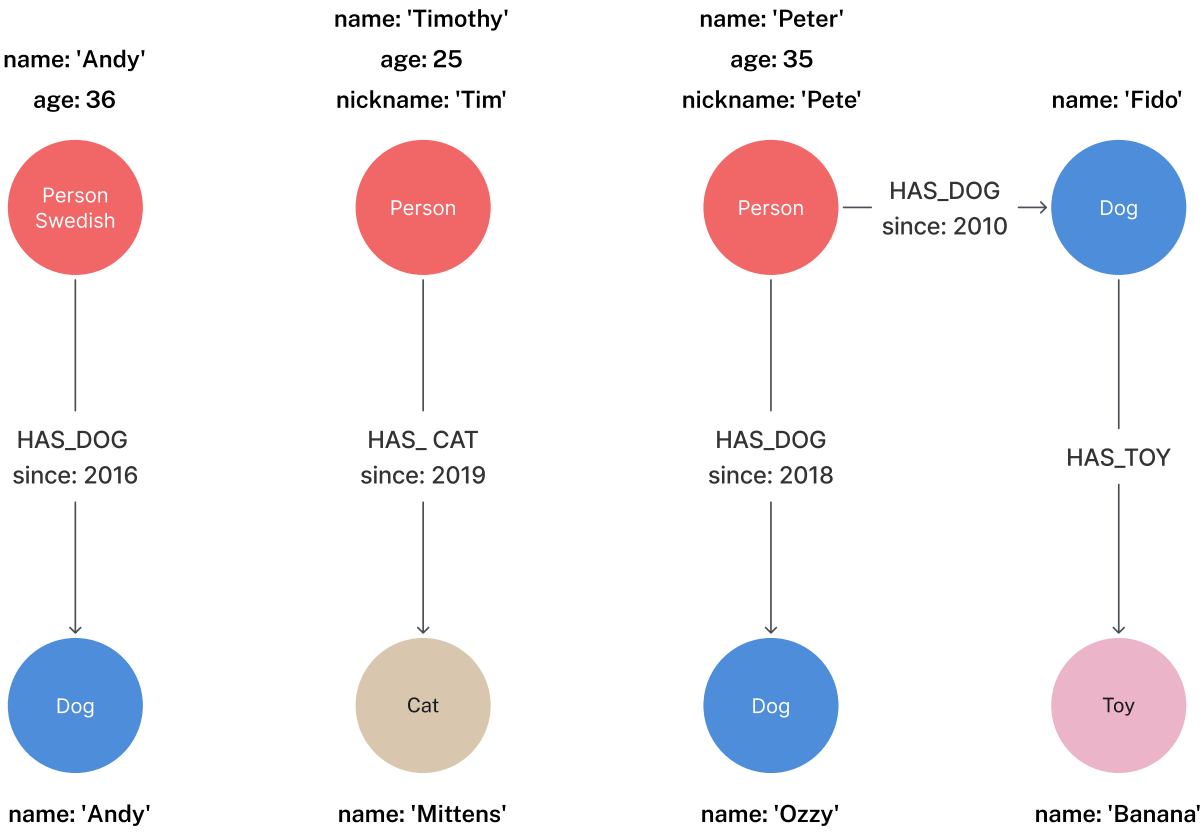
- Any non-writing query is allowed.
- The final `RETURN` clause is mandatory when using a `COLLECT` subquery. The `RETURN` clause must return exactly one column.
- A `COLLECT` subquery can appear anywhere in a query that an expression is valid.
- Any variable that is defined in the outside scope can be referenced inside the `COLLECT` subquery's own scope.
- Variables introduced inside the `COLLECT` subquery are not part of the outside scope and therefore cannot be accessed on the outside.

COUNT subqueries

A `COUNT` subquery can be used to count the number of rows returned by the subquery.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
(andy:Swedish:Person {name: 'Andy', age: 36}),
(timothy:Person {name: 'Timothy', nickname: 'Tim', age: 25}),
(peter:Person {name: 'Peter', nickname: 'Pete', age: 35}),
(andy)-[:HAS_DOG {since: 2016}]->(:Dog {name:'Andy'}),
(timothy)-[:HAS_CAT {since: 2019}]->(:Cat {name:'Mittens'}),
(fido:Dog {name:'Fido'})<-[:HAS_DOG {since: 2010}]-(peter)-[:HAS_DOG {since: 2018}]->(:Dog {name:'Ozzy'}),
(fido)-[:HAS_TOY]->(:Toy{name:'Banana'})
```

Simple `COUNT` subquery

Variables introduced by the outside scope can be used in the `COUNT` subquery without importing them. In this regard, `COUNT` subqueries are different from `CALL` subqueries, which do require importing. The following query exemplifies this and outputs the owners of more than one dog:

```
MATCH (person:Person)
WHERE COUNT { (person)-[:HAS_DOG]->(:Dog) } > 1
RETURN person.name AS name
```

name
"Peter"
Rows: 1

`COUNT` subquery with `WHERE` clause

A `WHERE` clause can be used inside the `COUNT` pattern. Variables introduced by the `MATCH` clause and the

outside scope can be used in this scope.

```
MATCH (person:Person)
WHERE COUNT {
  (person)-[:HAS_DOG]->(dog:Dog)
  WHERE person.name = dog.name
} = 1
RETURN person.name AS name
```

name
"Andy"
Rows: 1

Conditional `COUNT` subquery

Introduced in Neo4j 2025.06 | Cypher 25 only

`WHEN` can be used inside `COUNT` subqueries to execute branches conditionally when a predicate evaluates to `true`. Note that the names and the number of columns returned by the different `WHEN` branches must be identical. For more information, see Conditional queries → Conditional subqueries.

In the example below, the `WHEN` branch is executed if a person has no cat (`c IS NULL`), counting their dogs. The `ELSE` branch runs for those who have a cat.

Conditional `COUNT` subquery

```
MATCH (p:Person)
OPTIONAL MATCH (p)-[:HAS_CAT]->(c)
RETURN p.name AS person, c IS NOT NULL AS hasCat,
       COUNT {
          WHEN c IS NULL THEN {
           MATCH (p)-[:HAS_DOG]->(dog)
           RETURN dog.name AS petOwner
          }
          ELSE {
           MATCH (p)-[:HAS_CAT]->(cat)
           RETURN cat.name AS petOwner
          }
         } AS howManyPets
```

name	has Cat	howManyPets
"Andy"	FALSE	1
"Timothy"	TRUE	1
"Peter"	FALSE	2
Rows: 3		

`COUNT` subquery with a `UNION`

`COUNT` can be used with a `UNION` clause. If the `UNION` clause is distinct, the `RETURN` clause is required. `UNION ALL` clauses do not require the `RETURN` clause. However, it is worth noting that if one branch has a `RETURN`

clause, then all require one. The below example shows the count of pets each person has by using a `UNION` clause:

```
MATCH (person:Person)
RETURN
    person.name AS name,
    COUNT {
        MATCH (person)-[:HAS_DOG]->(dog:Dog)
        RETURN dog.name AS petName
        UNION
        MATCH (person)-[:HAS_CAT]->(cat:Cat)
        RETURN cat.name AS petName
    } AS numPets
```

name	numPets
"Andy"	1
"Timothy"	1
"Peter"	2
Rows: 3	

`COUNT` subquery with `WITH`

Variables from the outside scope are visible for the entire subquery, even when using a `WITH` clause. To avoid confusion, shadowing of these variables is not allowed. An outside scope variable is shadowed when a newly introduced variable within the inner scope is defined with the same variable. In the example below, the outer variable `name` is shadowed and will therefore throw an error.

```
WITH 'Peter' as name
MATCH (person:Person {name: name})
WHERE COUNT {
    WITH "Ozzy" AS name
    MATCH (person)-[:HAS_DOG]->(d:Dog)
    WHERE d.name = name
} = 1
RETURN person.name AS name
```

GQLSTATUS error chain

42N07: error: syntax error or access rule violation - variable shadowing. The variable `name` is shadowing a variable with the same name from the outer scope and needs to be renamed.

42001: error: syntax error or access rule violation - invalid syntax

New variables can be introduced into the subquery, as long as they use a different identifier. In the example below, a `WITH` clause introduces a new variable. Note that the outer scope variable `person` referenced in the main query is still available after the `WITH` clause.

```
MATCH (person:Person)
WHERE COUNT {
    WITH "Ozzy" AS dogName
    MATCH (person)-[:HAS_DOG]->(d:Dog)
    WHERE d.name = dogName
} = 1
```

```
RETURN person.name AS name
```

name
"Peter"
Rows: 1

Using `COUNT` subqueries inside other clauses

`COUNT` can be used in any position in a query, with the exception of administration commands, where it is restricted. See a few examples below:

Using `COUNT` in `RETURN`

```
MATCH (person:Person)
RETURN person.name, COUNT { (person)-[:HAS_DOG]->(:Dog) } as howManyDogs
```

person.name	howManyDogs
"Andy"	1
"Timothy"	0
"Peter"	2
Rows: 3	

Using `COUNT` in `SET`

```
MATCH (person:Person) WHERE person.name ="Andy"
SET person.howManyDogs = COUNT { (person)-[:HAS_DOG]->(:Dog) }
RETURN person.howManyDogs as howManyDogs
```

howManyDogs
1
Rows: 1 Properties set: 1

Using `COUNT` in `CASE`

```
MATCH (person:Person)
RETURN
   CASE
     WHEN COUNT { (person)-[:HAS_DOG]->(:Dog) } > 1 THEN "Doglover " + person.name
     ELSE person.name
   END AS result
```

result
"Andy"
"Timothy"
"Doglover Peter"
Rows: 3

Using `COUNT` as a grouping key

The following query groups all persons by how many dogs they own, and then calculates the average age for each group.

```
MATCH (person:Person)
RETURN COUNT { (person)-[:HAS_DOG]->(:Dog) } AS numDogs,
       avg(person.age) AS averageAge
 ORDER BY numDogs
```

numDogs	averageAge
0	25.0
1	36.0
2	35.0
Rows: 3	

`COUNT` subquery with `RETURN`

`COUNT` subqueries do not require a `RETURN` clause at the end of the subquery. If one is present, it does not need to be aliased. This is a difference compared to `CALL` subqueries. Any variables returned in a `COUNT` subquery will not be available after the subquery.

```
MATCH (person:Person)
WHERE COUNT {
    MATCH (person)-[:HAS_DOG]->(:Dog)
    RETURN person.name
} = 1
RETURN person.name AS name
```

name
"Andy"
Rows: 1

Rules

The following is true for `COUNT` subqueries:

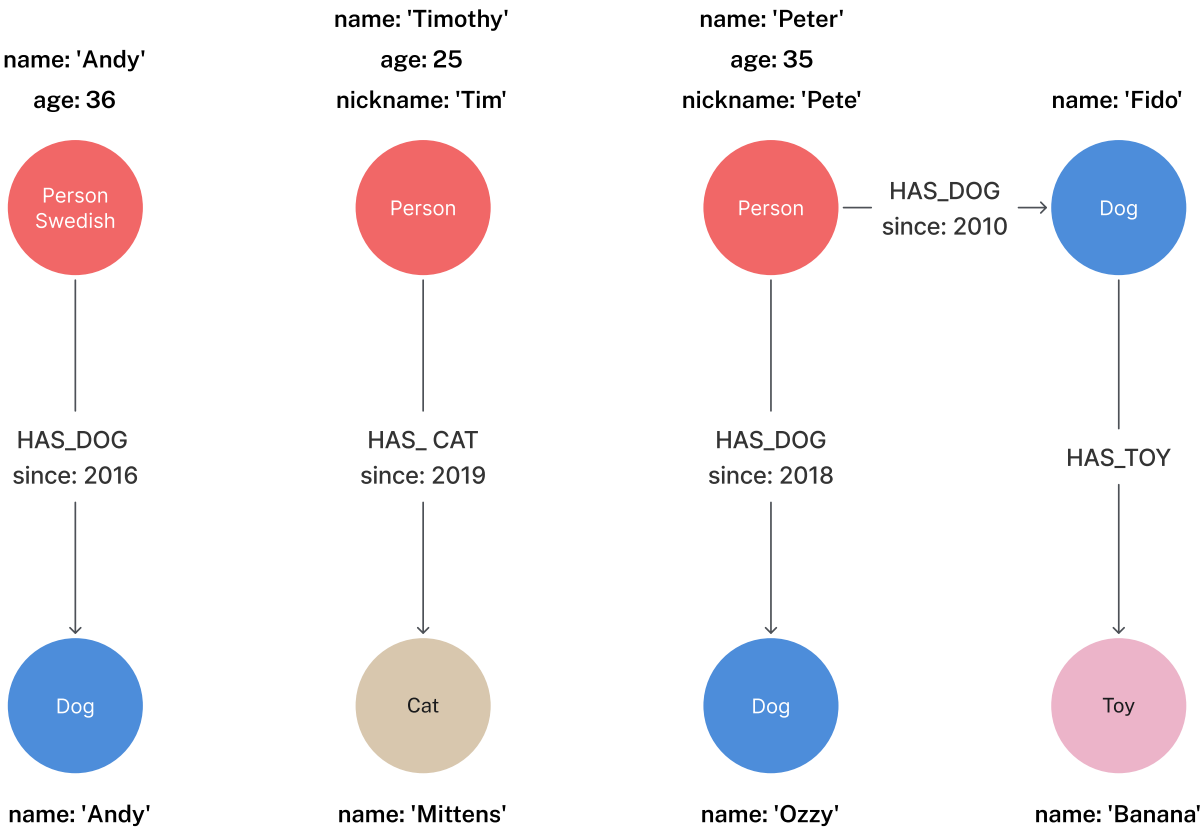
- Any non-writing query is allowed.
- The final `RETURN` clause may be omitted, as any variable defined within the subquery will not be available outside of the expression, even if a final `RETURN` clause is used. One exception to this is that for a `DISTINCT UNION` clause, the `RETURN` clause is still mandatory.
- The `MATCH` keyword can be omitted in subqueries in cases where the `COUNT` consists of only a pattern and an optional `WHERE` clause.
- A `COUNT` subquery can appear anywhere in a query that an expression is valid.
- Any variable that is defined in the outside scope can be referenced inside the `COUNT` subquery's own scope.
- Variables introduced inside the `COUNT` subquery are not part of the outside scope and therefore cannot be accessed on the outside.

EXISTS subqueries

An `EXISTS` subquery can be used to find out if a specified pattern exists at least once in the graph. It serves the same purpose as a path pattern but it is more powerful because it allows you to use `MATCH` and `WHERE` clauses internally.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
```

```
(andy:Swedish:Person {name: 'Andy', age: 36}),
(timothy:Person {name: 'Timothy', nickname: 'Tim', age: 25}),
(peter:Person {name: 'Peter', nickname: 'Pete', age: 35}),
(andy)-[:HAS_DOG {since: 2016}]->(:Dog {name:'Andy'}),
(timothy)-[:HAS_CAT {since: 2019}]->(:Cat {name:'Mittens'}),
(fido:Dog {name:'Fido'})<-[:HAS_DOG {since: 2010}]-(peter)-[:HAS_DOG {since: 2018}]->(:Dog {name:'Ozzy'}),
(fido)-[:HAS_TOY]->(:Toy{name:'Banana'})
```

Simple `EXISTS` subquery

Variables introduced by the outside scope can be used in the `EXISTS` subquery without importing them. In this regard, `EXISTS` subqueries are different from `CALL` subqueries, which do require importing. The following example shows this:

```
MATCH (person:Person)
WHERE EXISTS {
    (person)-[:HAS_DOG]->(:Dog)
}
RETURN person.name AS name
```

name
"Andy"
"Peter"
Rows: 2

`EXISTS` subquery with `WHERE` clause

A `WHERE` clause can be used in conjunction to the `MATCH`. Variables introduced by the `MATCH` clause and the outside scope can be used in this scope.

```
MATCH (person:Person)
WHERE EXISTS {
  MATCH (person)-[:HAS_DOG]->(dog:Dog)
  WHERE person.name = dog.name
}
RETURN person.name AS name
```

name
"Andy"
Rows: 1

Conditional `EXISTS` subqueries

Introduced in Neo4j 2025.06 | Cypher 25 only

`WHEN` can be used inside `EXISTS` subqueries to execute branches conditionally when a predicate evaluates to `true`. Note that the names and the number of columns returned by the different `WHEN` branches must be identical. For more information, see Conditional queries → Conditional subqueries.

The example below filters people based on their age and pet ownership. If someone is over 35, it checks for dog ownership, and if 35 or younger, it checks for cat ownership. Note that `Peter`, being 35 and a dog owner, is excluded from the result.

Conditional `EXISTS` subquery

```
MATCH (n:Person)
WHERE EXISTS {
  WHEN n.age > 35 THEN {
    MATCH (n)-[:HAS_DOG]->(:Dog)
    RETURN n AS petOwner
  }
  ELSE {
    MATCH (n)-[:HAS_CAT]->(:Cat)
    RETURN n AS petOwner
  }
}
RETURN n.name AS name,
       n.age AS age
```

name	age
"Andy"	36
"Timothy"	25
Rows: 2	

Nesting `EXISTS` subqueries

`EXISTS` subqueries can be nested like the following example shows. The nesting also affects the scopes. That means that it is possible to access all variables from inside the subquery which are either from the outside scope or defined in the very same subquery.

```
MATCH (person:Person)
WHERE EXISTS {
  MATCH (person)-[:HAS_DOG]->(dog:Dog)
  WHERE EXISTS {
    MATCH (dog)-[:HAS_TOY]->(toy:Toy)
    WHERE toy.name = 'Banana'
  }
}
RETURN person.name AS name
```

name
"Peter"
Rows: 1

`EXISTS` subquery outside of a `WHERE` clause

`EXISTS` subquery expressions can appear anywhere that an expression is valid. Here the result is a boolean that shows whether the subquery can find the given pattern.

```
MATCH (person:Person)
RETURN person.name AS name, EXISTS {
```

```
  MATCH (person)-[:HAS_DOG]->(:Dog)
} AS hasDog
```

name	hasDog
"Andy"	true
"Timothy"	false
"Peter"	true
Rows: 3	

`EXISTS` subquery with a `UNION`

`Exists` can be used with a `UNION` clause, and the `RETURN` clauses are not required. It is worth noting that if one branch has a `RETURN` clause, then all branches require one. The below example demonstrates that if one of the `UNION` branches was to return at least one row, the entire `EXISTS` expression will evaluate to true.

```
MATCH (person:Person)
RETURN
    person.name AS name,
    EXISTS {
        MATCH (person)-[:HAS_DOG]->(:Dog)
        UNION
        MATCH (person)-[:HAS_CAT]->(:Cat)
    } AS hasPet
```

name	hasPet
"Andy"	true
"Timothy"	true
"Peter"	true
Rows: 3	

`EXISTS` subquery with `WITH`

Variables from the outside scope are visible for the entire subquery, even when using a `WITH` clause. To avoid confusion, shadowing of these variables is not allowed. An outside scope variable is shadowed when a newly introduced variable within the inner scope is defined with the same variable. In the example below, the outer variable `name` is shadowed and will therefore throw an error.

```
WITH 'Peter' as name
MATCH (person:Person {name: name})
WHERE EXISTS {
    WITH "Ozzy" AS name
    MATCH (person)-[:HAS_DOG]->(d:Dog)
    WHERE d.name = name
}
RETURN person.name AS name
```

GQLSTATUS error chain

42N07: error: syntax error or access rule violation - variable shadowing. The variable `name` is shadowing a variable with the same name from the outer scope and needs to be renamed.

42001: error: syntax error or access rule violation - invalid syntax

New variables can be introduced into the subquery, as long as they use a different identifier. In the example below, a `WITH` clause introduces a new variable. Note that the outer scope variable `person` referenced in the main query is still available after the `WITH` clause.

```
MATCH (person:Person)
WHERE EXISTS {
    WITH "Ozzy" AS dogName
    MATCH (person)-[:HAS_DOG]->(d:Dog)
    WHERE d.name = dogName
}
RETURN person.name AS name
```

name
"Peter"
Rows: 1

`EXISTS` subquery with `RETURN`

`EXISTS` subqueries do not require a `RETURN` clause at the end of the subquery. If one is present, it does not need to be aliased, which is different compared to `CALL` subqueries. Any variables returned in an `EXISTS` subquery will not be available after the subquery.

```
MATCH (person:Person)
WHERE EXISTS {
    MATCH (person)-[:HAS_DOG]->(:Dog)
    RETURN person.name
}
RETURN person.name AS name
```

name
"Andy"
"Peter"
Rows: 2

Rules

The following is true for `EXISTS` subqueries:

- Any non-writing query is allowed.
- If the `EXISTS` subquery evaluates to at least one row, the whole expression will become `true`. This also

means that the system only needs to evaluate if there is at least one row and can skip the rest of the work.

- `EXISTS` subqueries differ from regular queries in that the final `RETURN` clause may be omitted, as any variable defined within the subquery will not be available outside of the expression, even if a final `RETURN` clause is used.
- The `MATCH` keyword can be omitted in subqueries in cases where the `EXISTS` consists of only a pattern and an optional `WHERE` clause.
- An `EXISTS` subquery can appear anywhere in a query that an expression is valid.
- Any variable that is defined in the outside scope can be referenced inside the subquery's own scope.
- Variables introduced inside the subquery are not part of the outside scope and therefore cannot be accessed on the outside.

Patterns

Graph pattern matching sits at the very core of Cypher. It is the mechanism used to navigate, describe and extract data from a graph by applying a declarative pattern. Inside a MATCH clause, you specify graph patterns to search for, and return the paths that match them. Graph pattern matching can also be used without a `MATCH` clause, in the subqueries EXISTS, COUNT, and COLLECT.

> **Note:**
>
> - Pattern: a queryable specification of a graph structure.
> - Path: an actual sequence of nodes and relationships in the graph that matches a pattern.

A graph pattern describes data using a syntax that is similar to how the nodes and relationships of a property graph are drawn on a whiteboard. On a whiteboard, nodes are drawn as circles and relationships are drawn as arrows. Cypher represents the circles as a pair of parentheses, and the arrows as dashes and greater-than or less-than symbols:

```
()-->()<--()
```

These simple patterns for nodes and relationships form the building blocks of path patterns that can match paths of a fixed length. As well as discussing simple fixed-length patterns, this chapter covers more complex patterns, showing how to match patterns of a variable or unknown length, find the shortest paths between a given set of nodes, add inline filters for improved query performance, and add cycles and non-linear shapes to path patterns. It also discusses Cypher's default behavior of not allowing relationships to be traversed in a match pattern, and how to alter this default, either by allowing the re-traversal of relationships using `REPEATABLE ELEMENTS` or by disallowing nodes to be repeated in a path with the `ACYCLIC` keyword.

This chapter includes the following sections:

- Primer: a short primer on how to get started with using graph pattern matching in Cypher.
- Fixed-length paths: information about node, relationship, and path patterns.
- Variable-length paths: information about quantified path patterns, quantified relationships, and group variables.
- Non-linear paths: information about equijoins and graph patterns (combined path patterns).
- Shortest paths: information about finding the `SHORTEST` path patterns.
- Paths with unique relationships: information about Cypher's default behavior of not re-traversing relationships in graph pattern matches and the `DIFFERENT RELATIONSHIPS` keyword.
- Paths with repeated nodes and relationships: information about how to enable re-traversal of relationships in graph pattern matches using `REPEATABLE ELEMENTS`. Introduced in Neo4j 2025.06 Cypher 25 only
- Acyclic paths: information about `ACYCLIC` paths. Introduced in Neo4j 2026.03 Cypher 25 only

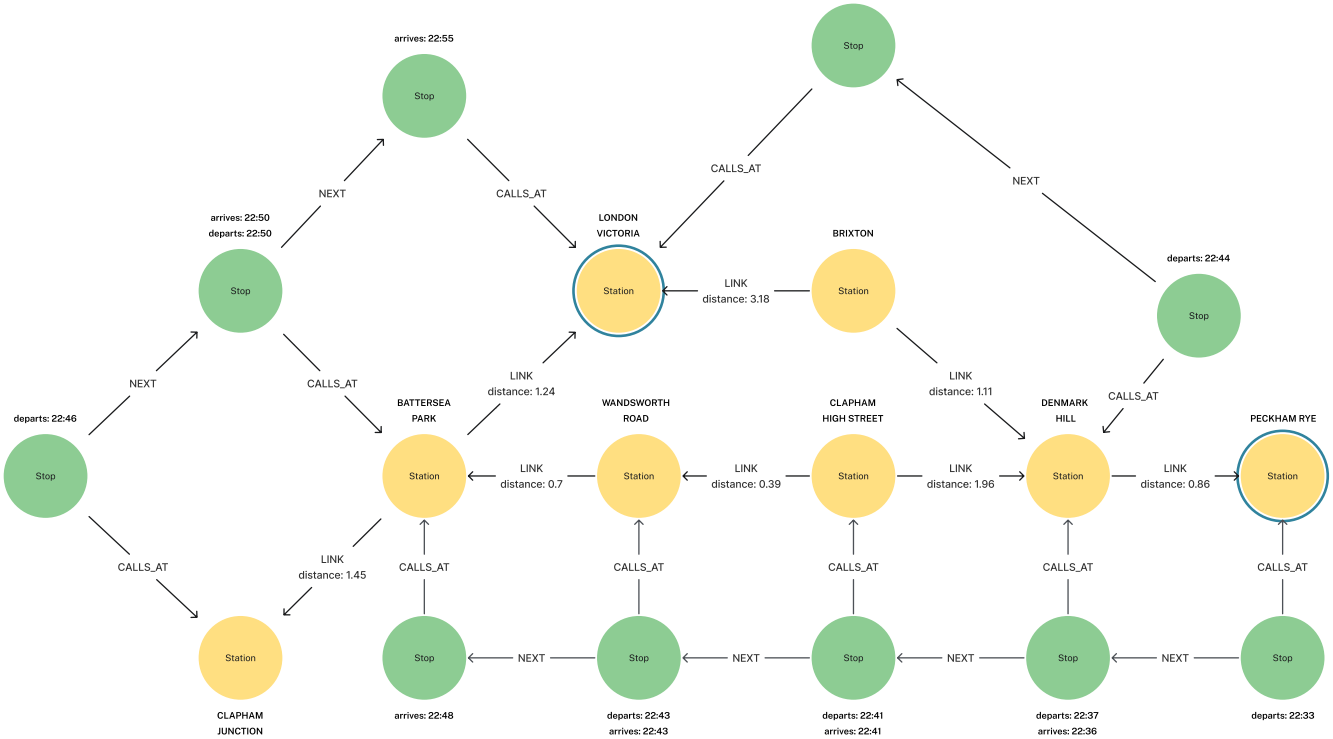
The model data in the examples used in this chapter are based on the UK national rail network, using publicly available datasets.

Primer

This section contains a primer covering some fundamental features of graph pattern matching with Cypher queries.

Example graph

The example graph used in this tutorial is a model of train `Stations`, and different train services with `Stops` that call at the `Stations`.



To recreate the graph, run the following query against an empty Neo4j database:

Query

```
CREATE (n1:Station {name: 'Denmark Hill'}),
(n5:Station {name: 'Battersea Park'}),
(n6:Station {name: 'Wandsworth Road'}),
(n15:Station {name: 'Clapham High Street'}),
(n16:Station {name: 'Peckham Rye'}),
(n17:Station {name: 'Brixton'}),
(n14:Station {name: 'London Victoria'}),
(n18:Station {name: 'Clapham Junction'}),
(p10:Stop {departs: time('22:37'), arrives: time('22:36')}),
(p0:Stop {departs: time('22:41'), arrives: time('22:41')}),
(p2:Stop {departs: time('22:43'), arrives: time('22:43')}),
(p17:Stop {arrives: time('22:50'), departs: time('22:50')}),
(p18:Stop {arrives: time('22:46'), departs: time('22:46')}),
(p19:Stop {departs: time('22:33'), arrives: time('22:31')}),
(p21:Stop {arrives: time('22:55')}),
(p20:Stop {departs: time('22:44'), arrives: time('22:43')}),
(p22:Stop {arrives: time('22:55')}),
(p23:Stop {arrives: time('22:48')}),
(n15)-[:LINK {distance: 1.96}]->(n1)-[:LINK {distance: 0.86}]->(n16),
(n15)-[:LINK {distance: 0.39}]->(n6)<-[:LINK {distance: 0.7}]-(n5)-[:LINK {distance: 1.24}]->(n14), (n5)-
```

```
[:LINK {distance: 1.45}]->(n18),
(n14)<-[:LINK {distance: 3.18}]-(n17)-[:LINK {distance: 1.11}]->(n1),
(p2)-[:CALLS_AT]->(n6), (p17)-[:CALLS_AT]->(n5), (p19)-[:CALLS_AT]->(n16),
(p22)-[:CALLS_AT]->(n14), (p18)-[:CALLS_AT]->(n18), (p0)-[:CALLS_AT]->(n15), (p23)-[:CALLS_AT]->(n5), (
p20)-[:CALLS_AT]->(n1),
(p21)-[:CALLS_AT]->(n14), (p10)-[:CALLS_AT]->(n1), (p19)-[:NEXT]->(p10)-[:NEXT]->(p0)-[:NEXT]->(p2)-[
:NEXT]->(p23),
(p22)<-[:NEXT]-(p17)<-[:NEXT]-(p18), (p21)<-[:NEXT]-(p20)
```

Matching fixed-length paths

An empty pair of parentheses is a node pattern that will match any node. This example gets a count of all the nodes in the graph:

```
MATCH ()
RETURN count(*) AS numNodes
```

Result

numNodes
18
Rows: 1

Adding a label to the node pattern will filter on nodes with that label (see label expressions). The following query gets a count of all the nodes with the label `Stop`:

```
MATCH (:Stop)
RETURN count(*) AS numStops
```

Result

numStops
10
Rows: 1

Path patterns can match relationships and the nodes they connect. The following query gets the arrival time of all trains calling at `Denmark Hill`:

```
MATCH (s:Stop)-[:CALLS_AT]->(:Station {name: 'Denmark Hill'})
RETURN s.arrives AS arrivalTime
```

Result

arrivalTime
"22:36:00Z"
"22:43:00Z"
Rows: 2

Path patterns can include inline `WHERE` clauses. The following query gets the next calling point of the service that departs `Denmark Hill` at `22:37`:

```
MATCH (n:Station {name: 'Denmark Hill'})<-[:CALLS_AT]-
       (s:Stop WHERE s.departs = time('22:37'))-[:NEXT]->
       (:Stop)-[:CALLS_AT]->(d:Station)
RETURN d.name AS nextCallingPoint
```

Result

nextCallingPoint
"Clapham High Street"
Rows: 1

For more information, see Fixed-length paths.

Matching variable-length paths

Variable-length paths that only traverse relationships with a specified type can be matched with quantified relationships. Any variable declared in the relationship pattern will return a list of the relationships traversed. The following query returns the total distance traveled via all `LINK`s connecting the stations `Peckham Rye` and `Clapham Junction`:

Query

```
MATCH (:Station {name: 'Peckham Rye'})-[link:LINK]-+
       (:Station {name: 'Clapham Junction'})
RETURN reduce(acc = 0.0, l IN link | round(acc + l.distance, 2)) AS
        totalDistance
```

Result

totalDistance
7.84
5.36
Rows: 2

> **Note:**
>
> `+` is a quantified relationship. It is composed of a relationship pattern that matches relationships going in either direction, and a quantifier `+` means that it matches one or more relationships. As no node patterns are included with quantified relationships, they match any intermediate nodes.

Variable-length paths can also be matched with quantified path patterns, which allow both `WHERE` clauses and accessing the nodes traversed by the path. The following query returns a list of calling points on routes from `Peckham Rye` to `London Victoria`, where no distance between stations is greater than two miles:

Query

```
MATCH (:Station {name: 'Peckham Rye'})
      (()-[link:LINK]-(s) WHERE link.distance <= 2)+
      (:Station {name: 'London Victoria'})
UNWIND s AS station
RETURN station.name AS callingPoint
```

Result

callingPoint
"Denmark Hill"
"Clapham High Street"
"Wandsworth Road"
"Battersea Park"
"London Victoria"
Rows: 5

`WHERE` clauses inside node patterns can themselves include path patterns. The following query using an EXISTS subquery to anchor on the last `Stop` in a sequence of `Stops`, and returns the departure times, arrival times and final destination of all services calling at `Denmark Hill`:

Query

```
MATCH (:Station {name: 'Denmark Hill'})<-[:CALLS_AT]-(s1:Stop)-[:NEXT]->+
        (sN:Stop WHERE NOT EXISTS { (sN)-[:NEXT]->(:Stop) })-[:CALLS_AT]->
        (d:Station)
RETURN s1.departs AS departure, sN.arrives AS arrival,
       d.name AS finalDestination
```

Result

departure	arrival	finalDestination
'22:37:00Z'	'22:48:00Z'	"Battersea Park"
'22:44:00Z'	'22:55:00Z'	"London Victoria"
Rows: 2		

Node variables declared inside quantified path patterns become bound to lists of nodes, which can be unwound and used in subsequent `MATCH` clauses. The following query lists the calling points of the `Peckham Rye` to `Battersea Park` train service:

Query

```
MATCH (:Station {name: 'Peckham Rye'})<-[:CALLS_AT]-(:Stop)
      (()-[:NEXT]->(s:Stop))+
      ()-[:CALLS_AT]->(:Station {name: 'Battersea Park'})
UNWIND s AS stop
MATCH (stop)-[:CALLS_AT]->(station:Station)
RETURN stop.arrives AS arrival, station.name AS callingPoint
```

Result

arrival	callingPoint
"22:36:00Z"	"Denmark Hill"
"22:41:00Z"	"Clapham High Street"
"22:43:00Z"	"Wandsworth Road"
"22:48:00Z"	"Battersea Park"

Rows: 4

Repeating a node variable in a path pattern enables the same node to be bound more than once in a path (see equijoins). The following query finds all stations that are on a cycle (i.e., pass through the same `Station` more than once) formed by the `LINK` between `Stations`:

Query

```
MATCH (n:Station)-[:LINK]-+(n)
RETURN DISTINCT n.name AS station
```

Result

station
"Denmark Hill"
"Battersea Park"
"Wandsworth Road"
"Clapham High Street"
"Brixton"
"London Victoria"

Rows: 6

Complex, non-linear paths can be matched using graph patterns, a comma separated list of path patterns that are connected via repeated node variables, i.e. equijoins. For example, a passenger is traveling from `Denmark Hill` and wants to join the train service to `London Victoria` that leaves `Clapham Junction` at `22:46`. The following query finds the departure time from `Denmark Hill` as well as the changeover `Station` and time of arrival:

Query

```
MATCH (:Station {name: 'Denmark Hill'})<-[:CALLS_AT]-
        (s1:Stop)-[:NEXT]->+(s2:Stop)-[:CALLS_AT]->
        (c:Station)<-[:CALLS_AT]-(x:Stop),
       (:Station {name: 'Clapham Junction'})<-[:CALLS_AT]-
         (t1:Stop)-[:NEXT]->+(x)-[:NEXT]->+(:Stop)-[:CALLS_AT]->
         (:Station {name: 'London Victoria'})
WHERE t1.departs = time('22:46')
      AND s2.arrives < x.departs
RETURN s1.departs AS departure, s2.arrives AS changeArrival,
       c.name AS changeAt
```

Result

departure	changeArrival	changeAt
"22:37:00Z"	"22:48:00Z"	"Battersea Park"

Rows: 1

For more information, see Variable-length paths.

Matching shortest paths

The shortest path between two nodes can be found using the `SHORTEST` keyword:

Query

```
MATCH p = SHORTEST 1
  (:Station {name: "Brixton"})
  (()-[:LINK]-(:Station))+
  (:Station {name: "Clapham Junction"})
RETURN [station IN nodes(p) | station.name] AS route
```

Result

route
["Brixton", "London Victoria", "Battersea Park", "Clapham Junction"]

Rows: 1

To find all shortest paths, the `ALL SHORTEST` keywords can be used:

Query

```
MATCH p = ALL SHORTEST
  (:Station {name: "Denmark Hill"})
  (()-[:LINK]-(:Station))+
  (:Station {name: "Clapham Junction"})
RETURN [station IN nodes(p) | station.name] AS route
```

Result

route
["Denmark Hill", "Clapham High Street", "Wandsworth Road", "Battersea Park", "Clapham Junction"]
["Denmark Hill", "Brixton", "London Victoria", "Battersea Park", "Clapham Junction"]

Rows: 2

In general, `SHORTEST k` can be used to return the `k` shortest paths. The following returns the two shortest paths:

Query

```
MATCH p = SHORTEST 2
  (:Station {name: "Denmark Hill"})
```

```
  (()-[:LINK]-(:Station))+
  (:Station {name: "Clapham High Street"})
RETURN [station IN nodes(p) | station.name] AS route
```

Result

route
["Denmark Hill", "Clapham High Street"]
["Denmark Hill", "Brixton", "London Victoria", "Battersea Park", "Wandsworth Road", "Clapham High Street"]
Rows: 2

For more information, see Shortest paths.

Fixed-length paths

The most basic form of graph pattern matching in Cypher involves the matching of fixed-length patterns. This includes node patterns, relationship patterns, and path patterns.

Node patterns

Every graph pattern contains at least one node pattern. The simplest graph pattern is a single, empty node pattern:

```
MATCH ()
```

The empty node pattern matches every node in a property graph. In order to obtain a reference to the nodes matched, a variable needs to be declared in the node pattern:

```
MATCH (n)
```

With this reference, node properties can be accessed:

```
MATCH (n)
RETURN n.name
```

Adding a label expression to the node pattern means only nodes with labels that match will be returned. The following matches nodes that have the `Stop` label:

```
MATCH (n:Stop)
```

The following more complex label expression matches all nodes that are either a `TrainStation` and a `BusStation` or `StationGroup`:

```
MATCH (n:(TrainStation&BusStation)|StationGroup)
```

A map of property names and values can be used to match on node properties based on equality with the

specified values. The following matches nodes that have their `mode` property equal to `Rail`:

```
MATCH (n { mode: 'Rail' })
```

More general predicates can be expressed with a `WHERE` clause. The following matches nodes whose name property starts with `Preston`:

```
MATCH (n:Station WHERE n.name STARTS WITH 'Preston')
RETURN n
```

See the node patterns reference section for more details.

Relationship patterns

The simplest possible relationship pattern is a pair of dashes:

```
--
```

This pattern matches a relationship with any direction and does not filter on any relationship type or property. Unlike a node pattern, a relationship pattern cannot be used in a `MATCH` clause without node patterns at both ends. See path patterns for more details.

In order to obtain a reference to the relationships matched by the pattern, a relationship variable needs to be declared in the pattern by adding the variable name in square brackets in between the dashes:

```
-[r]-
```

To match a specific direction, add `<` or `>` to the left or right hand side respectively:

```
-[r]->
```

To match on a relationship type, add the type name after a colon:

```
-[:CALLS_AT]->
```

Similar to node patterns, a map of property names and values can be added to filter on properties of the relationship based on equality with the specified values:

```
-[{ distance: 0.24, duration: 'PT4M' }]->
```

A `WHERE` clause can be used for more general predicates:

```
-[r WHERE time() + duration(r.duration) < time('22:00') ]->
```

See the relationship patterns reference section for more details.

Path patterns

Any valid path starts and ends with a node, with relationships between each node (if there is more than one node). Fixed-length path patterns have the same restrictions, and for all valid path patterns the following are true:

- They have at least one node pattern.
- They begin and end with a node pattern.
- They alternate between nodes and relationships.

These are all valid path patterns:

```
()
```

```
(s)--(e)
```

```
(:Station)--()<--(m WHERE m.departs > time('12:00'))-->()-[:NEXT]->(n)
```

These are invalid path patterns:

```
-->
```

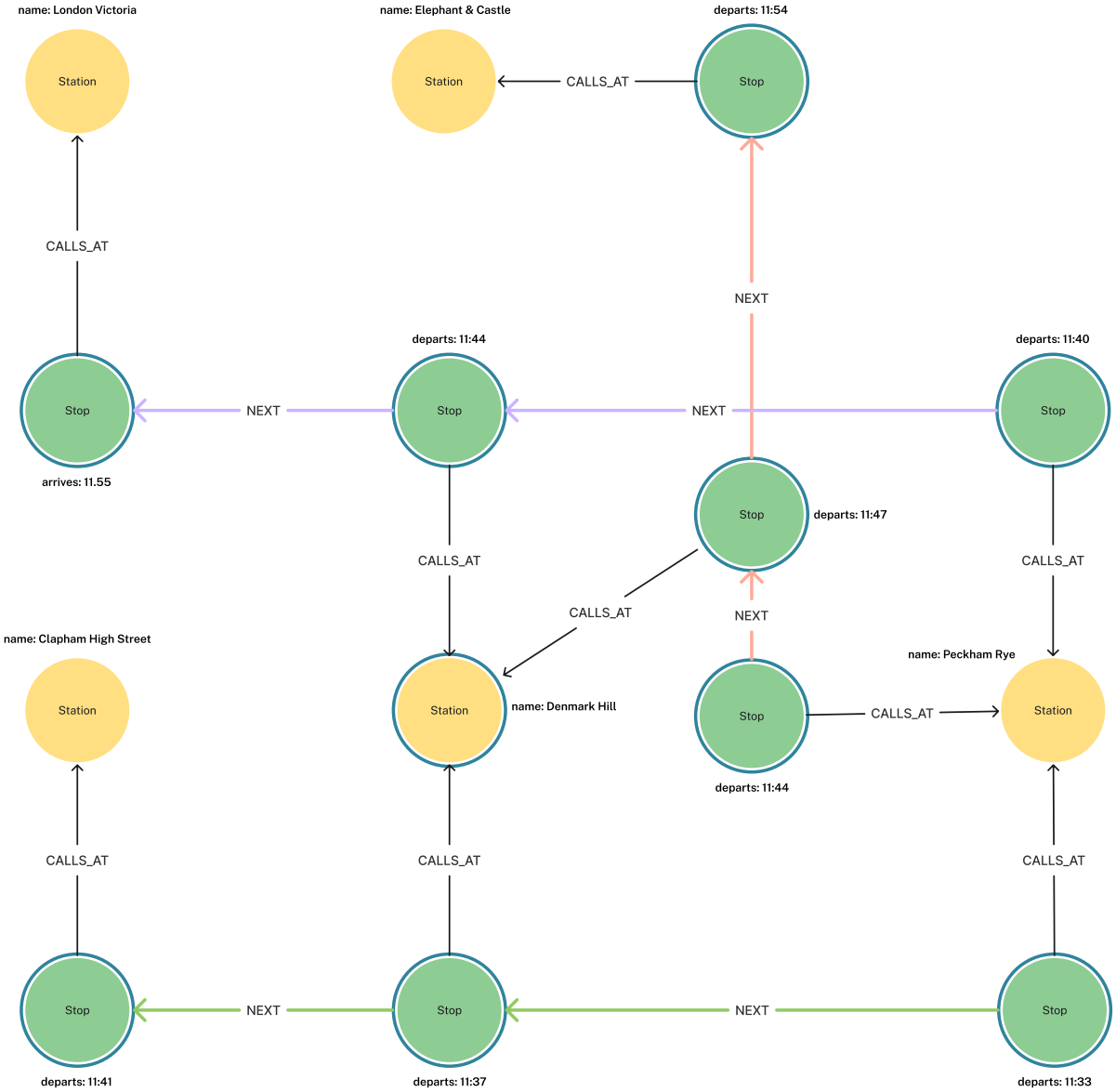
```
()-->
```

```
()-->-->()
```

Path pattern matching

This section contains an example of matching a path pattern to paths in a property graph.

It uses the following graph:



To recreate the graph, run the following query against an empty Neo4j database:

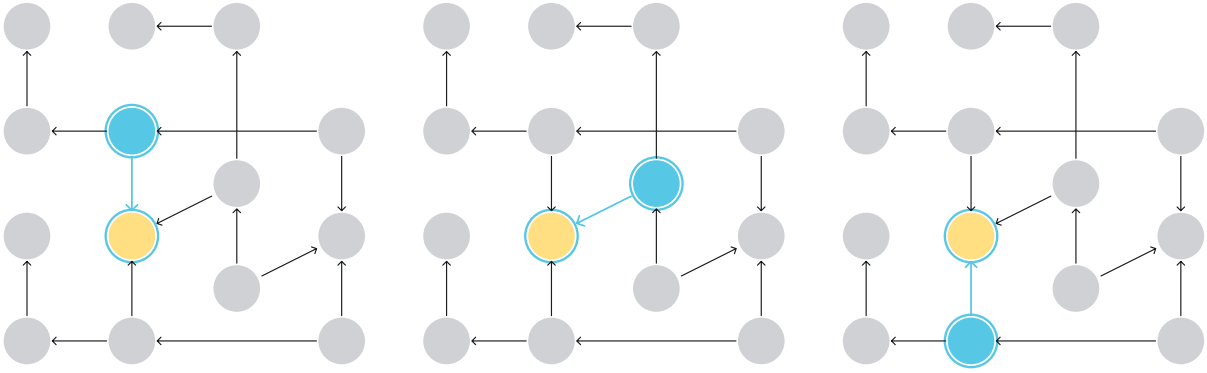
```
CREATE (pmr:Station {name: 'Peckham Rye'}),
  (dmk:Station {name: 'Denmark Hill'}),
  (vic:Station {name: 'London Victoria'}),
  (clp:Station {name: 'Clapham High Street'}),
  (eph:Station {name: 'Elephant & Castle'}),
  (vic)<-[:CALLS_AT]-(s1:Stop {departs: time('11:55')}),
  (dmk)<-[:CALLS_AT]-(s2:Stop {departs: time('11:44')})-[:NEXT]->(s1),
  (pmr)<-[:CALLS_AT]-(s3:Stop {departs: time('11:40')})-[:NEXT]->(s2),
  (clp)<-[:CALLS_AT]-(s4:Stop {departs: time('11:41')}),
  (dmk)<-[:CALLS_AT]-(s5:Stop {departs: time('11:37')})-[:NEXT]->(s4),
  (pmr)<-[:CALLS_AT]-(s6:Stop {departs: time('11:33')})-[:NEXT]->(s5),
  (eph)<-[:CALLS_AT]-(s7:Stop {departs: time('11:54')}),
  (dmk)<-[:CALLS_AT]-(s8:Stop {departs: time('11:47')})-[:NEXT]->(s7),
  (pmr)<-[:CALLS_AT]-(s9:Stop {departs: time('11:44')})-[:NEXT]->(s8)
```

The graph contains a number of train `Stations` and `Stops`. A `Stop` represents the arrival and departure of a train that `CALLS_AT` a `Station`. Each `Stop` forms part of a sequence of `Stops` connected by relationships with the type `NEXT`, representing the order of calling points made by a train service.

The graph shows three chains of `Stops` that represent different train services. Each of these services calls at the `Station` with the name `Denmark Hill`.

To return all `Stops` that call at the `Station` `Denmark Hill`, the following *motif* is used (the term motif is used to describe the pattern looked for in the graph):

In this case, three paths in the graph match the structure of the motif (plus the predicate anchoring to the `Station` `Denmark Hill`):



In order to return the name of each `Stop` that calls at a `Station`, declare a variable in the `Stop` node pattern. The results will then have a row containing the departs value of each `Stop` for each match shown above:

Query

```
MATCH (s:Stop)-[:CALLS_AT]->(:Station {name: 'Denmark Hill'})
RETURN s.departs AS departureTime
```

Result

departureTime
"11:44:00Z"
"11:47:00Z"
"11:37:00Z"
Rows: 3

Variable-length paths

Cypher can be used to match patterns of a variable or an unknown length. Such patterns can be found using quantified path patterns and quantified relationships. This page also discusses how variables work when declared in quantified path patterns (group variables), and how to use predicates in quantified path patterns.

Quantified path patterns

This section considers how to match paths of *varying* length by using *quantified path patterns*, allowing you to search for paths whose lengths are unknown or within a specific range.

Quantified path patterns can be useful when, for example, searching for all nodes that can be reached from an anchor node, finding all paths connecting two nodes, or when traversing a hierarchy that may have differing depths.

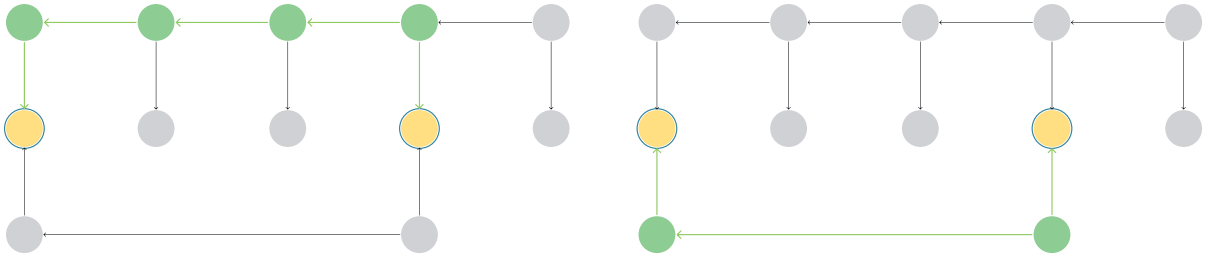
This example uses a new graph:

To recreate the graph, run the following query against an empty Neo4j database:

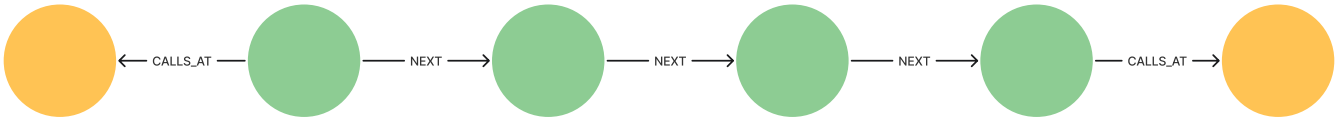
```
CREATE (pmr:Station {name: 'Peckham Rye'}),
  (dmk:Station {name: 'Denmark Hill'}),
  (clp:Station {name: 'Clapham High Street'}),
  (wwr:Station {name: 'Wandsworth Road'}),
  (clj:Station {name: 'Clapham Junction'}),
  (s1:Stop {arrives: time('17:19'), departs: time('17:20')}),
  (s2:Stop {arrives: time('17:12'), departs: time('17:13')}),
  (s3:Stop {arrives: time('17:10'), departs: time('17:11')}),
  (s4:Stop {arrives: time('17:06'), departs: time('17:07')}),
  (s5:Stop {arrives: time('16:58'), departs: time('17:01')}),
  (s6:Stop {arrives: time('17:17'), departs: time('17:20')}),
  (s7:Stop {arrives: time('17:08'), departs: time('17:10')}),
  (clj)<-[:CALLS_AT]-(s1), (wwr)<-[:CALLS_AT]-(s2),
  (clp)<-[:CALLS_AT]-(s3), (dmk)<-[:CALLS_AT]-(s4),
  (pmr)<-[:CALLS_AT]-(s5), (clj)<-[:CALLS_AT]-(s6),
  (dmk)<-[:CALLS_AT]-(s7),
  (s5)-[:NEXT {distance: 1.2}]->(s4),(s4)-[:NEXT {distance: 0.34}]->(s3),
  (s3)-[:NEXT {distance: 0.76}]->(s2), (s2)-[:NEXT {distance: 0.3}]->(s1),
  (s7)-[:NEXT {distance: 1.4}]->(s6)
```

Each `Stop` on a service `CALLS_AT` one `Station`. Each `Stop` has the properties `arrives` and `departs` that give the times the train is at the `Station`. Following the `NEXT` relationship of a `Stop` will give the next `Stop` of the service.

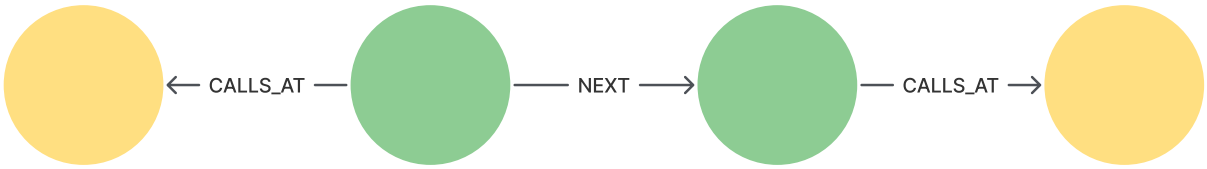
For this example, a path pattern is constructed to match each of the services that allow passengers to travel from `Denmark Hill` to `Clapham Junction`. The following shows the two paths that the path pattern should match:



The following motif represents a fixed-length path pattern that matches the service that departs from `Denmark Hill` station at `17:07`:



To match the second train service, leaving `Denmark Hill` at `17:10`, a shorter path pattern is needed:



Translating the motifs into Cypher, and adding predicates to match the origin and destination `Stations`, yields the following two path patterns respectively:

```
(:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(:Stop)
  -[:NEXT]->(:Stop)
  -[:NEXT]->(:Stop)
  -[:NEXT]->(:Stop)-[:CALLS_AT]->
(:Station { name: 'Clapham Junction' })
```

```
(:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(:Stop)
  -[:NEXT]->(:Stop)-[:CALLS_AT]->
(:Station { name: 'Clapham Junction' })
```

To return both solutions in the same query using these fixed-length path patterns, a UNION of two `MATCH` statements would be needed. For example, the following query returns the `departure` of the two services:

Query

```
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
        -[:NEXT]->(:Stop)
        -[:NEXT]->(:Stop)
        -[:NEXT]->(a:Stop)-[:CALLS_AT]->
      (:Station { name: 'Clapham Junction' })
RETURN d.departs AS departureTime, a.arrives AS arrivalTime
UNION
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
        -[:NEXT]->(a:Stop)-[:CALLS_AT]->
      (:Station { name: 'Clapham Junction' })
RETURN d.departs AS departureTime, a.arrives AS arrivalTime
```

Result

departureTime	arrivalTime
"17:07:00Z"	"17:19:00Z"

departureTime	arrivalTime
"17:10:00Z"	"17:17:00Z"
Rows: 2	

The problem with this solution is that not only is it verbose, it can only be used where the lengths of the target paths are known in advance. Quantified path patterns solve this problem by extracting repeating parts of a path pattern into parentheses and applying a **quantifier**. That quantifier specifies a range of possible repetitions of the extracted pattern to match on. For the current example, the first step is identifying the repeating pattern, which in this case is the sequence of alternating `Stop` nodes and `NEXT` relationships, representing one segment of a `Service`:

```
(:Stop)-[:NEXT]->(:Stop)
```

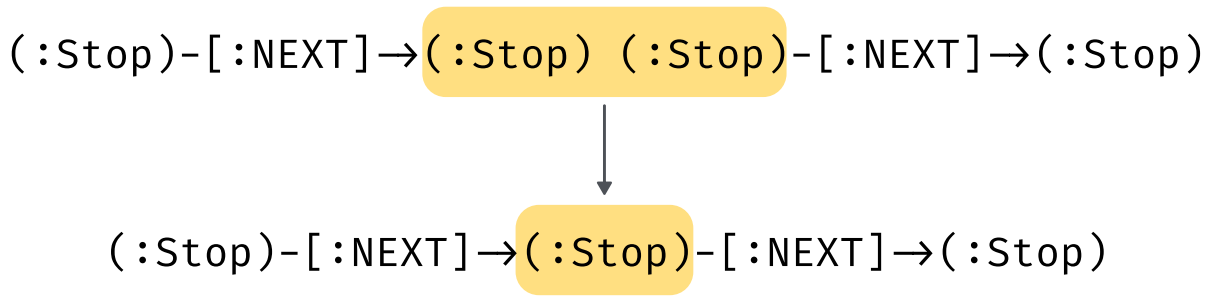
The shortest path has one instance of this pattern, the longest three. So the quantifier applied to the wrapper parentheses is the range one to three, expressed as `{1,3}`:

```
((:Stop)-[:NEXT]->(:Stop)){1,3}
```

This also includes repetitions of two, but in this case this repetition will not return matches. To understand the semantics of this pattern, it helps to work through the expansion of the repetitions. Here are the three repetitions specified by the quantifier, combined into a union of path patterns:

```
(:Stop)-[:NEXT]->(:Stop) |
(:Stop)-[:NEXT]->(:Stop)(:Stop)-[:NEXT]->(:Stop) |
(:Stop)-[:NEXT]->(:Stop)(:Stop)-[:NEXT]->(:Stop)(:Stop)-[:NEXT]->(:Stop)
```

The union operator (`|`) and placing node patterns next to each other are used here for illustration only; using it this way is not part of Cypher syntax. Where two node patterns are next to each other in the expansion above, they must necessarily match the same node: the next segment of a `Service` starts where the previous segment ends. As such they can be rewritten as a single node pattern with any filtering condition combined conjunctively. In this example this is trivial, because the filtering applied to those nodes is just the label `Stop`:



With this, the union of path patterns simplifies to:

```
(:Stop)-[:NEXT]->(:Stop) |
(:Stop)-[:NEXT]->(:Stop)-[:NEXT]->(:Stop) |
(:Stop)-[:NEXT]->(:Stop)-[:NEXT]->(:Stop)-[:NEXT]->(:Stop)
```

The segments of the original path pattern that connect the `Stations` to the `Stops` can also be rewritten. Here is what those segments look like when concatenated with the first repetition:

```
(:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(:Stop)
(:Stop)-[:NEXT]->(:Stop)
(:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
```

The original `MATCH` clause now has the following three parts:

```
(:Station {name:"Denmark Hill"})←[:CALLS_AT]-(:Stop)
```

```
(:Stop)-[:NEXT]→(:Stop) |
(:Stop)-[:NEXT]→(:Stop)-[:NEXT]→( :Stop) |
(:Stop)-[:NEXT]→(:Stop)-[:NEXT]→(:Stop)-[:NEXT]→(:Stop)
```

```
(:Stop)-[:CALLS_AT]→(:Station {name:"Clapham Junction"})
```

Translating the union of fixed-length path patterns into a quantified path pattern results in a pattern that will return the correct paths. The following query adds a `RETURN` clause that yields the departure and arrival times of the two services:

Query

```
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
      ((:Stop)-[:NEXT]->(:Stop)){1,3}
      (a:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
RETURN d.departs AS departureTime, a.arrives AS arrivalTime
```

Result

departureTime	arrivalTime
"17:10Z"	"17:17Z"
"17:07Z"	"17:19Z"
Rows: 2	

Quantified relationships

Quantified relationships allow some simple quantified path patterns to be re-written in a more succinct way. Continuing with the example of `Stations` and `Stops` from the previous section, consider the following query:

Query

```
MATCH (d:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(n:Stop)
      ((:Stop)-[:NEXT]->(:Stop)){1,10}
      (m:Stop)-[:CALLS_AT]->(a:Station { name: 'Clapham Junction' })
WHERE m.arrives < time('17:18')
RETURN n.departs AS departureTime
```

If the relationship `NEXT` only connects `Stop` nodes, the `:Stop` label expressions can be removed:

Query

```
MATCH (d:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(n:Stop)
      (()-[:NEXT]->()){1,10}
      (m:Stop)-[:CALLS_AT]->(a:Station { name: 'Clapham Junction' })
WHERE m.arrives < time('17:18')
RETURN n.departs AS departureTime
```

When the quantified path pattern has one relationship pattern, it can be abbreviated to a *quantified relationship*. A quantified relationship is a relationship pattern with a postfix quantifier. Below is the previous query rewritten with a quantified relationship:

Query

```
MATCH (d:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-
       (n:Stop)-[:NEXT]->{1,10}(m:Stop)-[:CALLS_AT]->
       (a:Station { name: 'Clapham Junction' })
WHERE m.arrives < time('17:18')
RETURN n.departs AS departureTime
```

The scope of the quantifier `{1,10}` is the relationship pattern `-[:NEXT]->` and not the node patterns abutting it. More generally, where a path pattern contained in a quantified path pattern has the following form:

```
(() <relationship pattern> ()) <quantifier>
```

then it can be re-written as follows:

```
<relationship pattern> <quantifier>
```

Note:

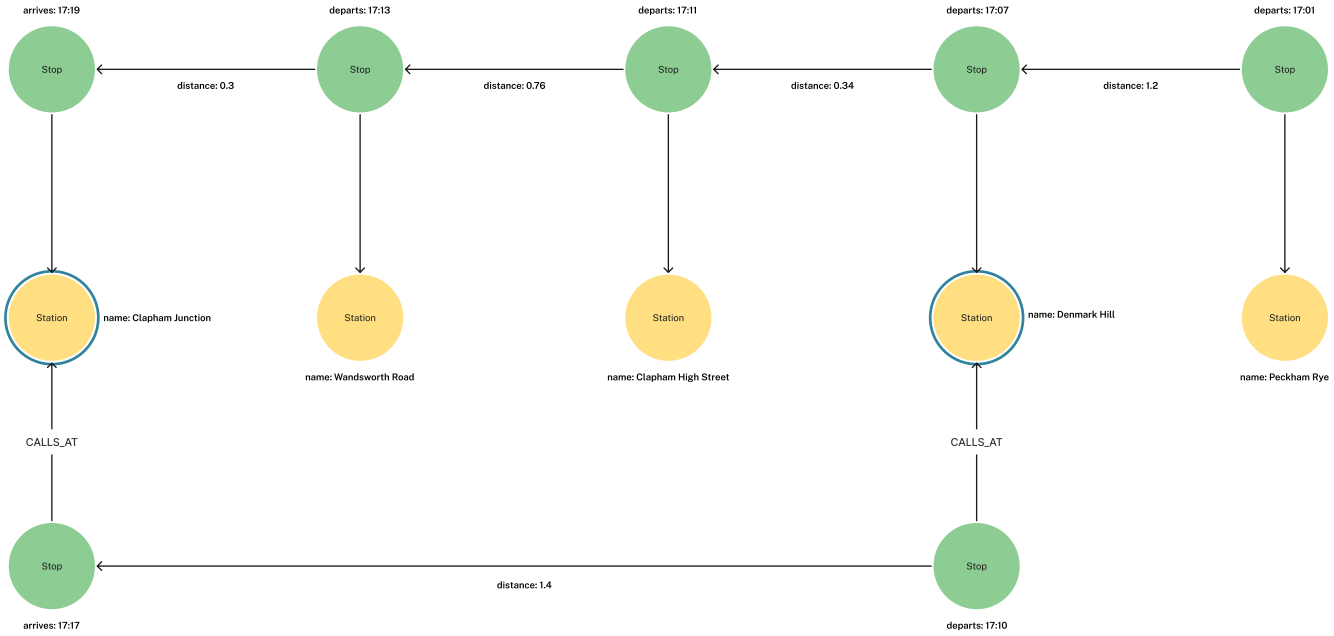
Prior to the introduction of quantified path patterns and quantified relationships, the only method in Cypher to match paths of a variable length was through variable-length relationships. This syntax is still available but it is not GQL conformant. It is very similar to the syntax for quantified relationships, with the following differences:

- Position and syntax of quantifier.
- Semantics of the asterisk symbol.
- Type expressions are limited to the disjunction operator.
- The WHERE clause is not allowed.

For more information, see the reference section on variable-length relationships.

Group variables

This section uses the example of `Stations` and `Stops` used in the previous section, but with an additional property `distance` added to the `NEXT` relationships:



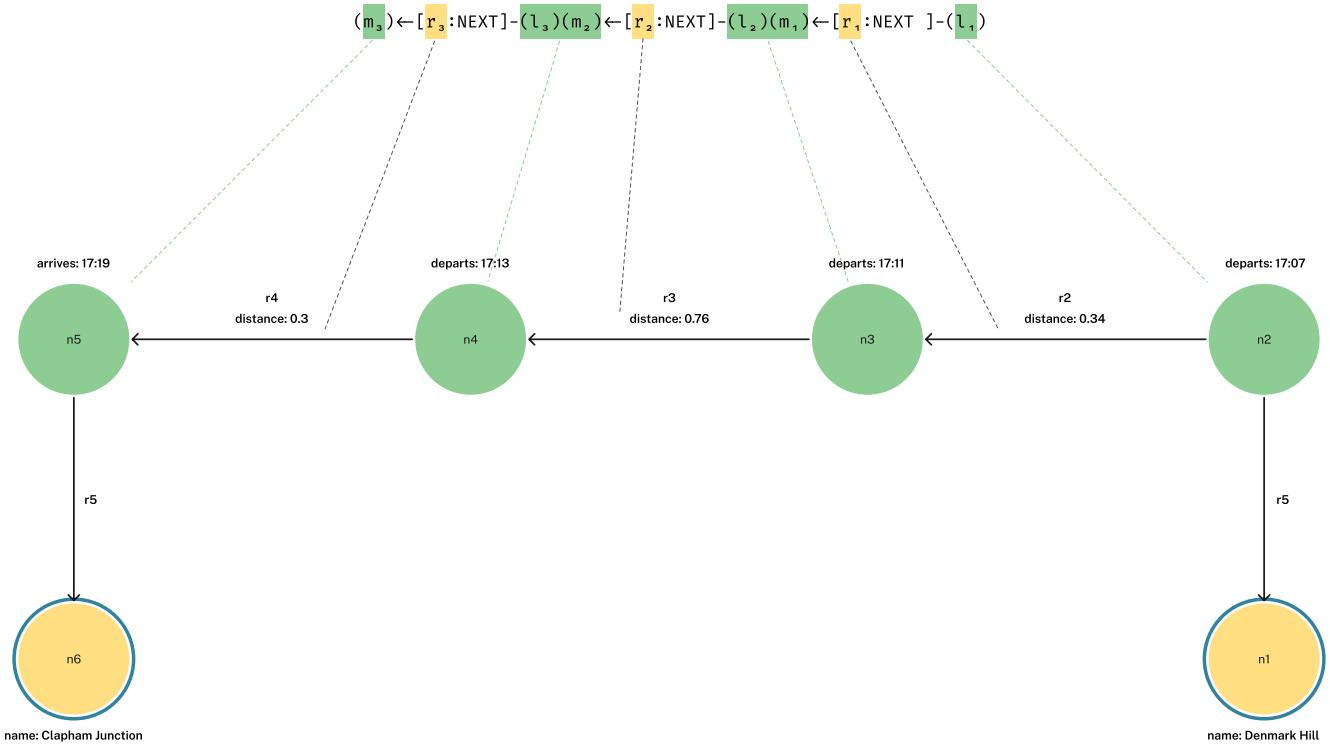
As the name suggests, this property represents the distance between two `Stops`. To return the total distance for each service connecting a pair of `Stations`, a variable referencing each of the relationships traversed is needed. Similarly, to extract the `departs` and `arrives` properties of each `Stop`, variables referencing each of the nodes traversed is required. In this example of matching services between `Denmark Hill` and `Clapham Junction`, the variables `l` and `m` are declared to match the `Stops` and `r` is declared to match the relationships. The variable origin only matches the first `Stop` in the path:

```
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(origin)
      ((l)-[r:NEXT]->(m)){1,3}
      ()-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
```

Variables that are declared inside quantified path patterns are known as *group variables*. They are so called because, when referred outside of the quantified path pattern, they are lists of the nodes or relationships they are bound to in the match. To understand how to think about the way group variables are bound to nodes or relationships, it helps to expand the quantified path pattern, and observe how the different variables match to the elements of the overall matched path. Here the three different expansions for each value in the range given by the quantifier `{1,3}`:

```
(l1)-[r1:NEXT]->(m1) |
(l1)-[r1:NEXT]->(m1)(l2)-[r2:NEXT]->(m2) |
(l1)-[r1:NEXT]->(m1)(l2)-[r2:NEXT]->(m2)(l3)-[r3:NEXT]->(m3)
```

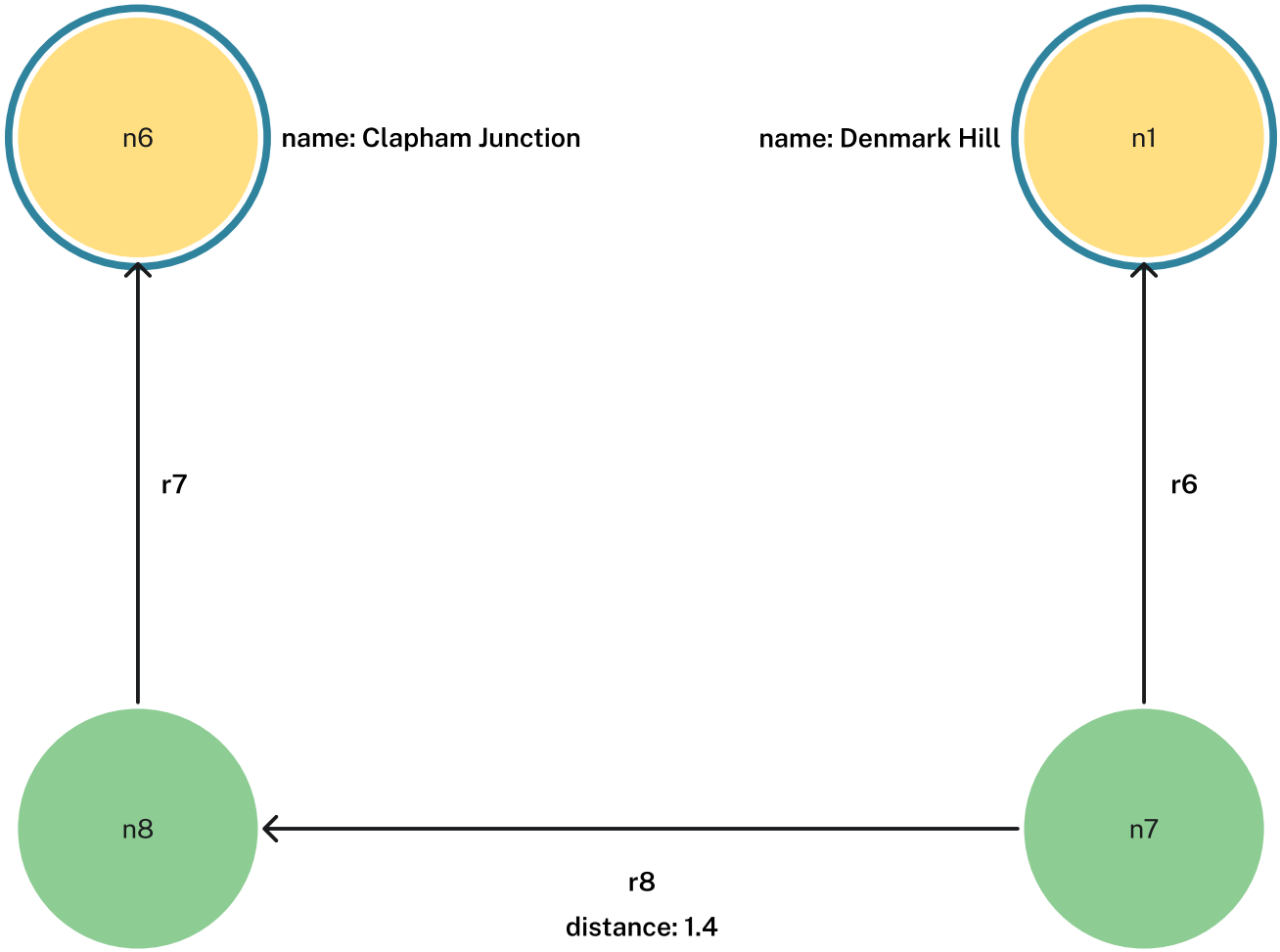
The subscript of each variable indicates which instance of the path pattern repetition they belong to. The following diagram shows the variable bindings of the path pattern with three repetitions, which matches the service that departs `Denmark Hill` at `17:07`. It traces the node or relationship that each indexed variable is bound to. Note that the index increases from right to left as the path starts at `Denmark Hill`:



For this matched path, the group variables have the following bindings:

```
l => [n2, n3, n4]
r => [r2, r3, r4]
m => [n3, n4, n5]
```

The second solution is the following path:



The following table shows the bindings for both matches, including the variable origin. In contrast to the group variables, `origin` is a singleton variable due to being declared outside the quantification. Singleton variables bind at most to one node or relationship.

origin	l	r	m
n2	[n2, n3, n4]	[r2, r3, r4]	[n3, n4, n5]
n7	[n7]	[r8]	[n8]

Returning to the original goal, which was to return the sequence of depart times for the `Stops` and the total distance of each service, the final query exploits the compatibility of group variables with list comprehensions and list functions such as reduce():

Query

```
MATCH (:Station {name: 'Denmark Hill'})<-[:CALLS_AT]-(origin)
      ((l)-[r:NEXT]->(m)){1,3}
      ()-[:CALLS_AT]->(:Station {name: 'Clapham Junction'})
RETURN origin.departs + [stop in m | stop.departs] AS departureTimes,
       reduce(acc = 0.0, next in r | round(acc + next.distance, 2)) AS totalDistance
```

Result

departureTimes	totalDistance
["17:10:00Z", "17:20:00Z"]	1.4

departureTimes	totalDistance
["17:07:00Z", "17:11:00Z", "17:13:00Z", "17:20:00Z"]	1.4
Rows: 2	

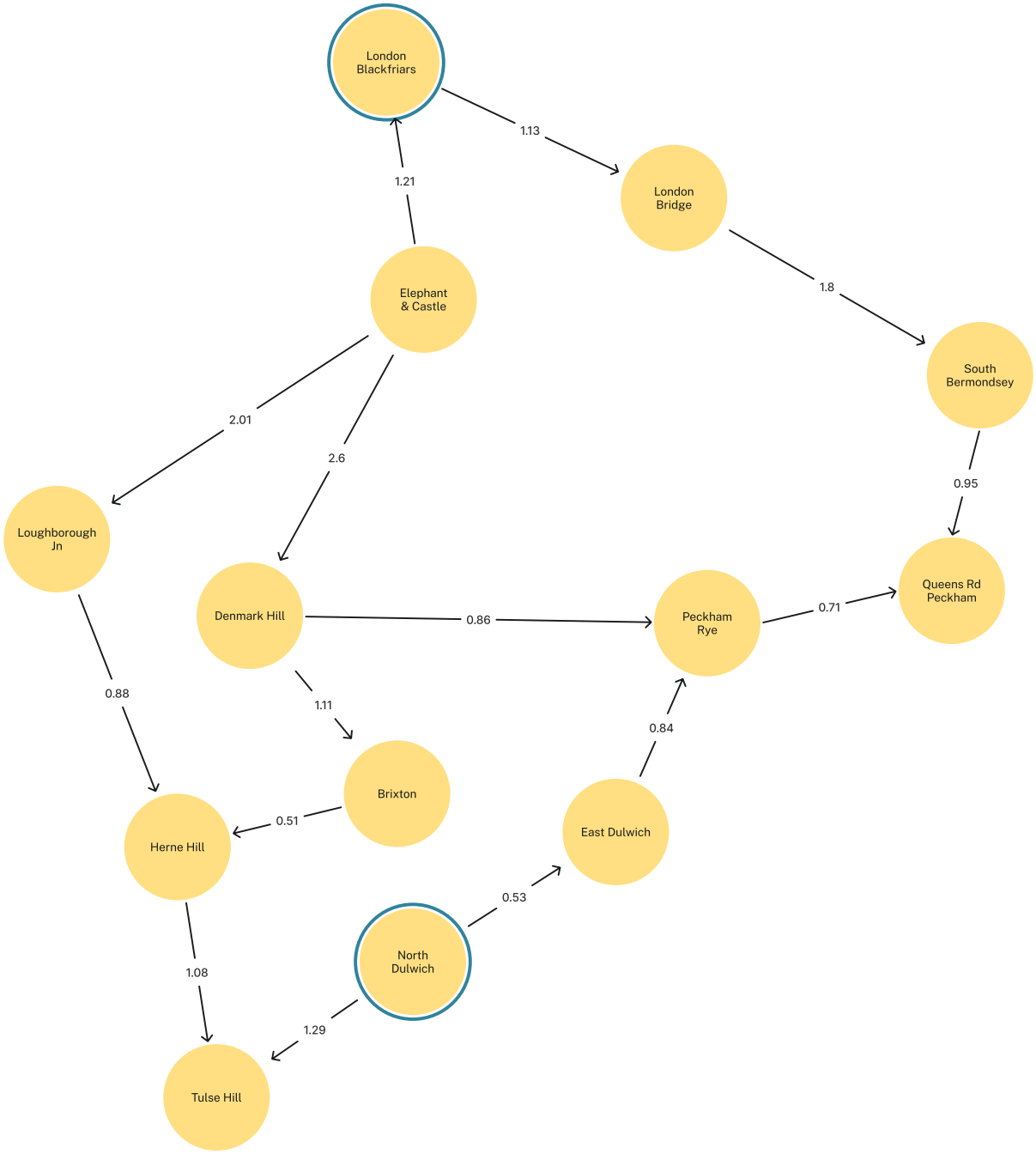
Predicates in quantified path patterns

One of the pitfalls of quantified path patterns is that, depending on the graph, they can end up matching very large numbers of paths, resulting in a slow query performance. This is especially true when searching for paths with a large maximum length or when the pattern is too general. However, by using inline predicates that specify precisely which nodes and relationships should be included in the results, unwanted results will be pruned as the graph is traversed.

Here are some examples of the types of constraints you can impose on quantified path pattern traversals:

- Nodes must have certain combinations of labels. For example, all nodes must be an `Employee`, but not a `Contractor`.
- Relationships must have certain types. For example, all relationships in the path must be of type `EMPLOYED_BY`.
- Nodes or relationships must have properties satisfying some condition. For example, all relationships must have the property `distance > 10`.
- For every iteration of the quantified path pattern, an aggregated value over the constructed path so far must satisfy a predicate. For example, the sum of the property `distance` of the relationships in the path must be less than 50 for every step in the construction of the path. See allReduce for more information about this predicate.

To demonstrate the utility of predicates in quantified path patterns, this section considers an example of finding the shortest path by physical distance and compares that to the results yielded by using the `SHORTEST` keyword. The graph in this example continues with `Station` nodes, but adds both a geospatial `location` property to the `Stations`, as well as `LINK` relationships with a `distance` property representing the distance between pairs of `Stations`:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (lbg:Station {name: "London Bridge"}),
    (bfr:Station {name: "London Blackfriars"}),
    (eph:Station {name: "Elephant & Castle"}),
    (dmk:Station {name: "Denmark Hill"}),
    (pmr:Station {name: "Peckham Rye"}),
    (qrp:Station {name: "Queens Rd Peckham"}),
    (sbm:Station {name: "South Bermondsey"}),
    (lgj:Station {name: "Loughborough Jn"}),
    (hnh:Station {name: "Herne Hill"}),
    (tuh:Station {name: "Tulse Hill"}),
    (ndl:Station {name: "North Dulwich"}),
    (edw:Station {name: "East Dulwich"}),
    (brx:Station {name: "Brixton"})
 SET lbg.location = point({longitude: -0.08609, latitude: 51.50502}),
    bfr.location = point({longitude: -0.10333, latitude: 51.51181}),
    eph.location = point({longitude: -0.09873, latitude: 51.49403}),
    dmk.location = point({longitude: -0.08936, latitude: 51.46820}),
    pmr.location = point({longitude: -0.06941, latitude: 51.47003}),
    qrp.location = point({longitude: -0.05731, latitude: 51.47357}),
    sbm.location = point({longitude: -0.05468, latitude: 51.48814}),
    lgj.location = point({longitude: -0.10218, latitude: 51.46630}),
```

```
    hnh.location = point({longitude: -0.10229, latitude: 51.45331}),
    tuh.location = point({longitude: -0.10508, latitude: 51.43986}),
    ndl.location = point({longitude: -0.08792, latitude: 51.45451}),
    edw.location = point({longitude: -0.08057, latitude: 51.46149}),
    brx.location = point({longitude: -0.11418, latitude: 51.46330})
CREATE (lbg)<-[:LINK {distance: 1.13}]-(bfr),
    (bfr)<-[:LINK {distance: 1.21}]-(eph),
    (eph)-[:LINK {distance: 2.6}]->(dmk),
    (dmk)-[:LINK {distance: 0.86}]->(pmr),
    (pmr)-[:LINK {distance: 0.71}]->(qrp),
    (qrp)<-[:LINK {distance: 0.95}]-(sbm),
    (sbm)<-[:LINK {distance: 1.8}]-(lbg),
    (lgj)-[:LINK {distance: 0.88}]->(hnh),
    (hnh)-[:LINK {distance: 1.08}]->(tuh),
    (tuh)<-[:LINK {distance: 1.29}]-(ndl),
    (ndl)-[:LINK {distance: 0.53}]->(edw),
    (edw)-[:LINK {distance: 0.84}]->(pmr),
    (eph)-[:LINK {distance: 2.01}]->(lgj),
    (dmk)-[:LINK {distance: 1.11}]->(brx),
    (brx)-[:LINK {distance: 0.51}]->(hnh)
```

The following query finds the path length and total distance for `ALL SHORTEST` paths between `London Blackfriars` to `North Dulwich`:

Query

```
MATCH (bfr:Station {name: 'London Blackfriars'}),
      (ndl:Station {name: 'North Dulwich'})
MATCH p = ALL SHORTEST (bfr)-[:LINK]-+(ndl)
RETURN [n in nodes(p) | n.name] AS stops,
       length(p) as stopCount,
       reduce(acc = 0, r in relationships(p) | round(acc + r.distance, 2)) AS distance
```

Result

stops	stopCount	distance
["London Blackfriars", "Elephant & Castle", "Denmark Hill", "Peckham Rye", "East Dulwich", "North Dulwich"]	5	6.04
["London Blackfriars", "Elephant & Castle", "Loughborough Jn", "Herne Hill", "Tulse Hill", "North Dulwich"]	5	6.47

Rows: 2

`ALL SHORTEST` finds all shortest paths by number of hops, and as the result shows, there are two paths in the graph tied for the shortest path. Whether any of these paths corresponds to the shortest path by distance can be checked by looking at each path between the two end `Stations` and returning the first result after ordering by `distance`:

Query

```
MATCH (bfr:Station {name: 'London Blackfriars'}),
      (ndl:Station {name: 'North Dulwich'})
MATCH p = (bfr)-[:LINK]-+(ndl)
RETURN reduce(acc = 0, r in relationships(p) | round(acc + r.distance, 2))
    AS distance
ORDER BY distance LIMIT 1
```

Result

distance
5.96
Rows: 1

This shows that there is a route with a shorter distance than either of the paths with fewer `Stations` returned using `ALL SHORTEST`. But to get this result, the query had to first find all paths from `London Blackfriars` to `North Dulwich` before it could select the shortest one. The following query shows the number of possible paths:

Query

```
MATCH (bfr:Station {name: 'London Blackfriars'}),
      (ndl:Station {name: 'North Dulwich'})
MATCH p = (bfr)-[:LINK]-+(ndl)
RETURN count(*) AS numPaths
```

Result

numPaths
7
Rows: 1

For a small dataset like this, finding all the paths will be fast. But as the size of the graph grows, the execution time will increase exponentially. For a real dataset, such as the entire rail network of the UK, it might be unacceptably long.

One approach to avoid the exponential explosion in paths is to put a finite upper bound to the quantified path pattern (e.g. `{,10}`) to limit the amount of path iterations returned. This works fine where the solution is known to lie within some range of hops. But in cases where this is not known, one alternative would be to make the pattern more specific by, for example, adding node labels, or by specifying a relationship direction. Another alternative would be to add an inline predicate to the quantified path pattern.

In this example, an inline predicate can be added that takes advantage of the geospatial `location` property of the `Stations`: for each pair of `Stations` on the path, the second `Station` will be closer to the endpoint (not always true, but is assumed here to keep the example simple). To compose the predicate, the point.distance() function is used to compare the distance between the left-hand `Station` (`a`) and the right-hand `Station` (`b`) for each node-pair along the path to the destination `North Dulwich`:

Query

```
MATCH (bfr:Station {name: "London Blackfriars"}),
      (ndl:Station {name: "North Dulwich"})
MATCH p = (bfr)
          ((a)-[:LINK]-(b:Station)
            WHERE point.distance(a.location, ndl.location) >
              point.distance(b.location, ndl.location))+ (ndl)
RETURN reduce(acc = 0, r in relationships(p) | round(acc + r.distance, 2))
  AS distance
```

Result

distance
5.96
Rows: 1

This query avoids having to find all possible paths and then imposing a `LIMIT 1` to find the shortest one by distance. It also shows that there is only one path to solving the query (a number that remains constant even if the data from the rest of the UK railway network was included). Using inline predicates or making quantified path patterns more specific where possible can thus greatly improve query performance.

An alternative is to use an upper bound for the total distance, for example `6.05`. Once a path exceeds this upper bound, you can prune it and continue searching other paths. The allReduce predicate function expresses this as follows:

Query

```
MATCH (bfr:Station {name: "London Blackfriars"}),
      (ndl:Station {name: "North Dulwich"})
MATCH p = (bfr) ((a)-[l:LINK]-(b:Station))+ (ndl)
WHERE allReduce(
  pathLength = 0,
  link IN l | pathLength + link.distance,
  pathLength < 6.05
)
RETURN reduce(acc = 0, r in relationships(p) | round(acc + r.distance, 2))
  AS distance
ORDER BY distance LIMIT 1
```

Further reading

- Neo4j Developer Blog: Getting From Denmark Hill to Gatwick Airport With Quantified Path Patterns
- Neo4j Developer Blog: Did You Take the Neo4j 5 Cypher Bullet Train?

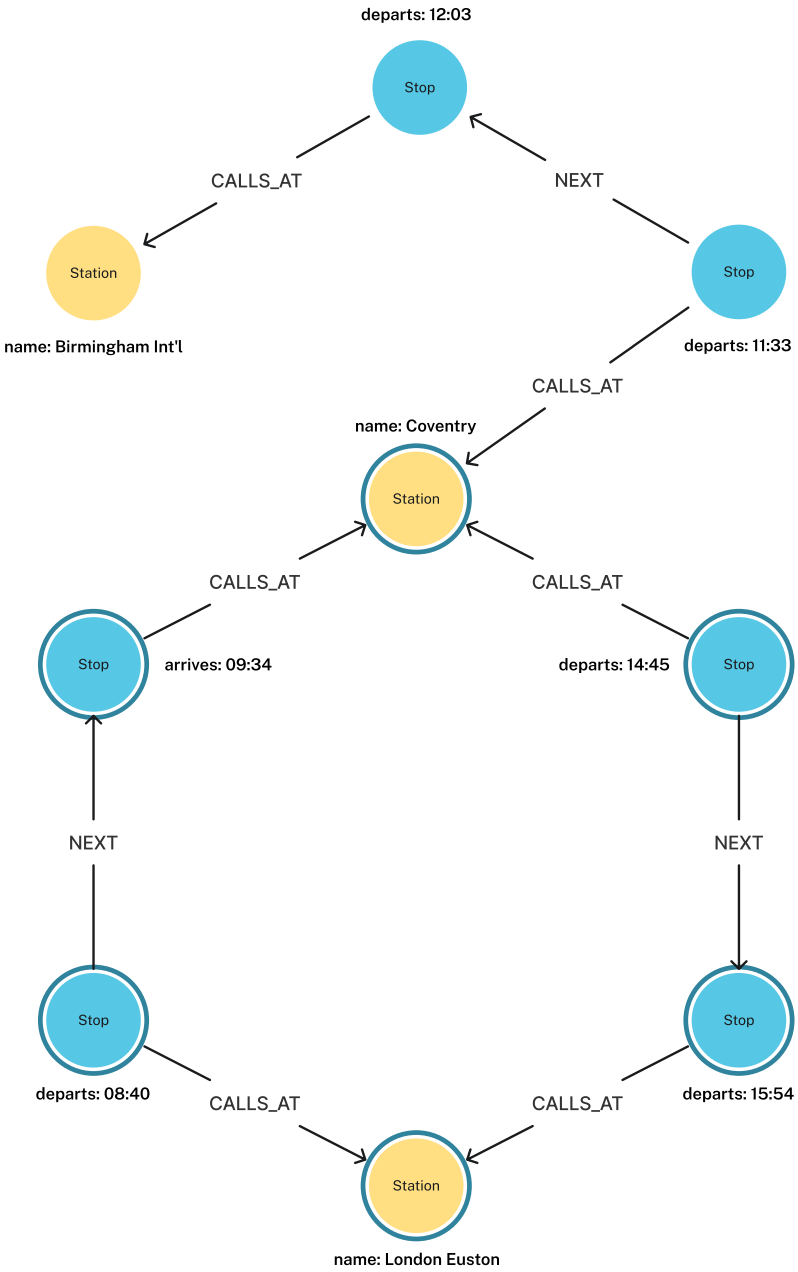
Non-linear paths

Cypher can be used to express non-linear patterns, either by equijoins (an operation in which more than one node relationship in a path is the same) or by more complicated graph patterns consisting of multiple path patterns.

Equijoins

An equijoin is an operation on paths that requires more than one of the nodes or relationships of the paths to be the same. The equality between the nodes or relationships is specified by declaring the same variable in multiple node patterns or relationship patterns. An equijoin allows cycles to be specified in a path pattern.

This section uses the following graph:



To recreate the graph, run the following query against an empty Neo4j database:

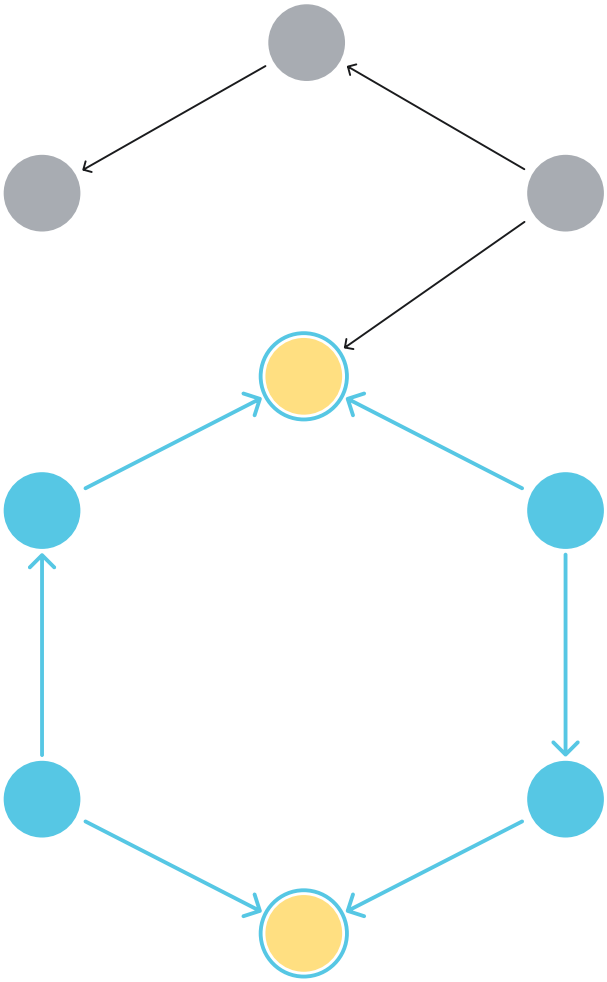
```
CREATE (bhi:Station {name: 'Birmingham International'}),
  (cov:Station {name: 'Coventry'}),
  (eus:Station  {name: 'London Euston'}),
  (bhi)<-[:CALLS_AT]-(s1:Stop {departs: time('12:03')}),
  (cov)<-[:CALLS_AT]-(s2:Stop {departs: time('11:33')})-[:NEXT]->(s1),
  (eus)<-[:CALLS_AT]-(s3:Stop {departs: time('15:54')}),
  (cov)<-[:CALLS_AT]-(s4:Stop {departs: time('14:45')})-[:NEXT]->(s3),
  (cov)<-[:CALLS_AT]-(s5:Stop {departs: time('09:34')}),
  (eus)<-[:CALLS_AT]-(s6:Stop {departs: time('08:40')})-[:NEXT]->(s5)
```

To illustrate how equijoins work, we will use the problem of finding a round trip between two `Stations`.

In this example scenario, a passenger starts their outbound journey at `London Euston` `Station` and ends at `Coventry` `Station`. The return journey will be the reverse order of those `Stations`.

The graph has three different services, two of which would compose the desired round trip, and a third which would send the passenger to `Birmingham International`.

The solution is the following path with a cycle:



If unique properties exist on the node where the cycle "join" occurs in the path, then it is possible to repeat the node pattern with a predicate matching on the unique property. The following motif demonstrates how that can be achieved, repeating a `Station` node pattern with the name `London Euston`:

The path pattern equivalent is:

```
(:Station {name: 'London Euston'})<-[:CALLS_AT]-(:Stop)-[:NEXT]->(:Stop)
  -[:CALLS_AT]->(:Station {name: 'Coventry'})<-[:CALLS_AT]-(:Stop)
  -[:NEXT]->(:Stop)-[:CALLS_AT]->(:Station {name: 'London Euston'})
```

There may be cases where a unique predicate is not available. In this case, an equijoin can be used to define the desired cycle by using a repeated node variable. In the current example, if you declare the same node variable `n` in both the first and last node patterns, then the node patterns *must* match the same node:

Putting this path pattern with an equijoin in a query, the times of the outbound and return journeys can be returned:

Query

```
MATCH (n:Station {name: 'London Euston'})<-[:CALLS_AT]-(s1:Stop)
  -[:NEXT]->(s2:Stop)-[:CALLS_AT]->(:Station {name: 'Coventry'})
  <-[:CALLS_AT]-(s3:Stop)-[:NEXT]->(s4:Stop)-[:CALLS_AT]->(n)
RETURN s1.departs+'-'+s2.departs AS outbound,
  s3.departs+'-'+s4.departs AS `return`
```

Result

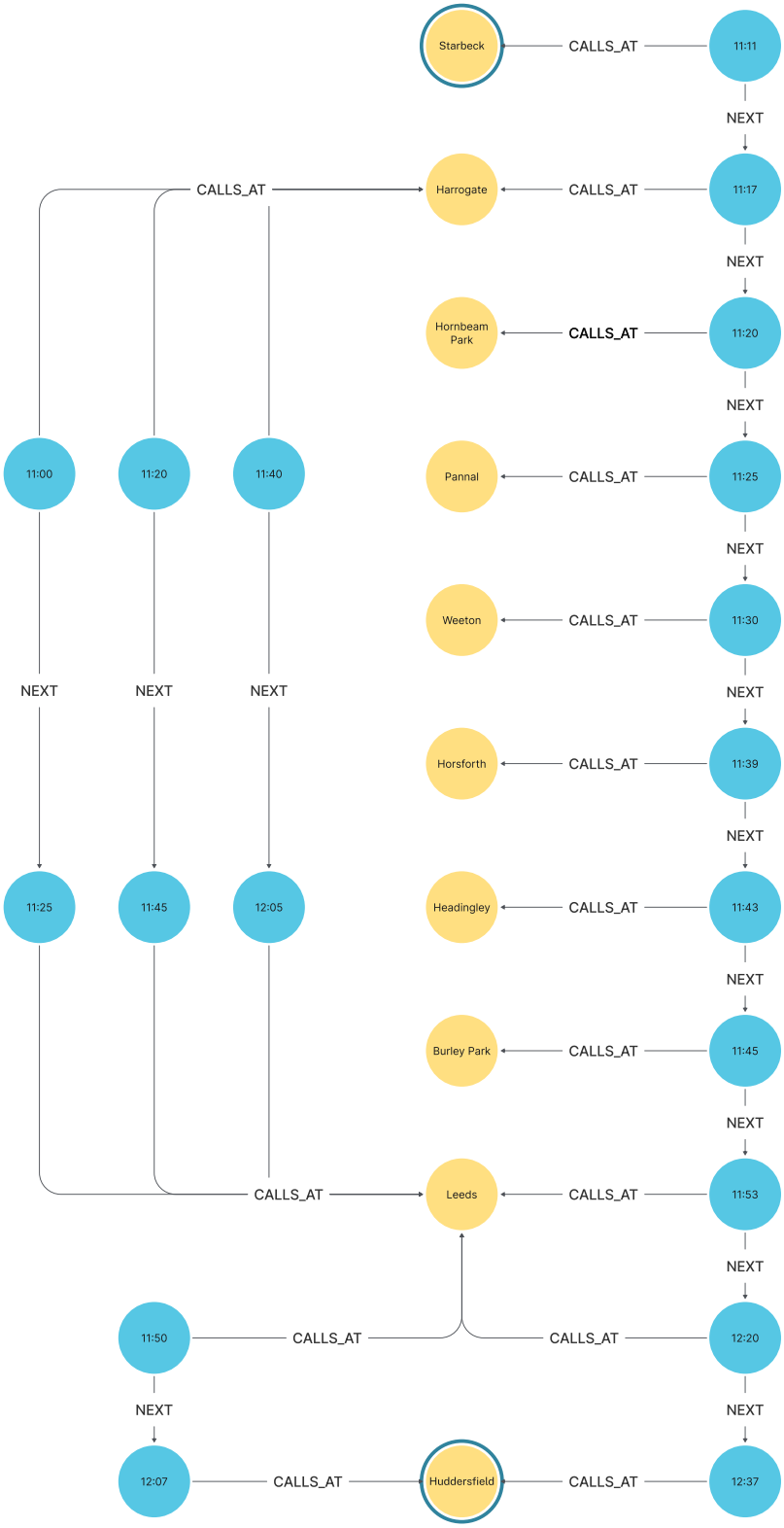
outbound	return
"08:40:00Z-09:34:00Z"	"14:45:00Z-15:54:00Z"
Rows: 1	

Graph patterns

In addition to the single path patterns, multiple path patterns can be combined in a comma-separated list to form a graph pattern. In a graph pattern, each path pattern is matched separately, and where node variables are repeated in the separate path patterns, the solutions are reduced via equijoins. If there are no equijoins between the path patterns, the result is a Cartesian product between the separate solutions.

The benefit of joining multiple path patterns in this way is that it allows the specification of more complex patterns than the linear paths allowed by a single path pattern. To illustrate this, another example drawn from the railway model will be used. In this example, a passenger is traveling from `Starbeck` to `Huddersfield`, changing trains at `Leeds`. To get to `Leeds` from `Starbeck`, the passenger can take a direct service that stops at all stations on the way. However, there is an opportunity to change at one of the stations (`Harrogate`) on the way to `Leeds`, and catch an express train, which may enable the passenger to catch an earlier train leaving from `Leeds`, reducing the overall journey time.

This section uses the following graph, showing the train services the passenger can use:



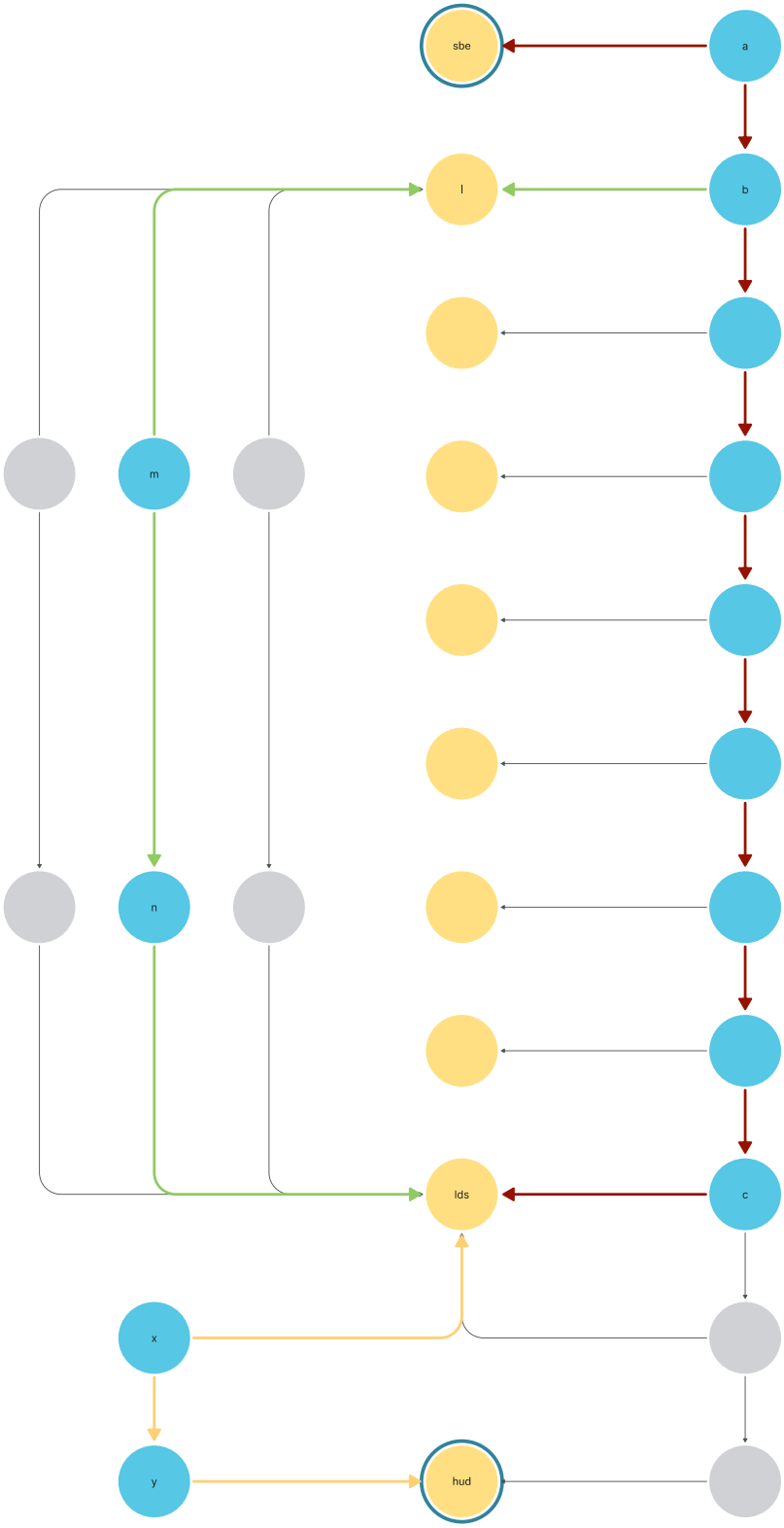
To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (hgt:Station {name: 'Harrogate'}), (lds:Station {name: 'Leeds'}),
(sbe:Station {name: 'Starbeck'}), (hbp:Station {name: 'Hornbeam Park'}),
(wet:Station {name: 'Weeton'}), (hrs:Station {name: 'Horsforth'}),
(hdy:Station {name: 'Headingley'}), (buy:Station {name: 'Burley Park'}),
(pnl:Station {name: 'Pannal'}), (hud:Station {name: 'Huddersfield'}),
(s9:Stop {arrives: time('11:53')}),
(s8:Stop {arrives: time('11:44'), departs: time('11:45')}),
(s7:Stop {arrives: time('11:40'), departs: time('11:43')}),
(s6:Stop {arrives: time('11:38'), departs: time('11:39')}),
(s5:Stop {arrives: time('11:29'), departs: time('11:30')}),
(s4:Stop {arrives: time('11:24'), departs: time('11:25')}),
```

```
(s3:Stop {arrives: time('11:19'), departs: time('11:20')}),
(s2:Stop {arrives: time('11:16'), departs: time('11:17')}),
(s1:Stop {departs: time('11:11')}), (s21:Stop {arrives: time('11:25')}),
(s211:Stop {departs: time('11:00')}), (s10:Stop {arrives: time('11:45')}),
(s101:Stop {departs: time('11:20')}), (s11:Stop {arrives: time('12:05')}),
(s111:Stop {departs: time('11:40')}), (s12:Stop {arrives: time('12:07')}),
(s121:Stop {departs: time('11:50')}), (s13:Stop {arrives: time('12:37')}),
(s131:Stop {departs: time('12:20')}),
(lds)<-[:CALLS_AT]-(s9), (buy)<-[:CALLS_AT]-(s8)-[:NEXT]->(s9),
(hdy)<-[:CALLS_AT]-(s7)-[:NEXT]->(s8), (hrs)<-[:CALLS_AT]-(s6)-[:NEXT]->(s7),
(wet)<-[:CALLS_AT]-(s5)-[:NEXT]->(s6), (pnl)<-[:CALLS_AT]-(s4)-[:NEXT]->(s5),
(hbp)<-[:CALLS_AT]-(s3)-[:NEXT]->(s4), (hgt)<-[:CALLS_AT]-(s2)-[:NEXT]->(s3),
(sbe)<-[:CALLS_AT]-(s1)-[:NEXT]->(s2), (lds)<-[:CALLS_AT]-(s21), (hgt)<-[:CALLS_AT]-(s211)-[:NEXT]->(s21),
(lds)<-[:CALLS_AT]-(s10), (hgt)<-[:CALLS_AT]-(s101)-[:NEXT]->(s10), (lds)<-[:CALLS_AT]-(s11), (hgt)<-
[:CALLS_AT]-(s111)-[:NEXT]->(s11), (hud)<-[:CALLS_AT]-(s12), (lds)<-[:CALLS_AT]-(s121)-[:NEXT]->(s12),
(hud)<-[:CALLS_AT]-(s13), (lds)<-[:CALLS_AT]-(s131)-[:NEXT]->(s13)
```

The solution to the problem assembles a set of path patterns matching the following three parts: the stopping service; the express service; and the final leg of the journey from `Leeds` to `Huddersfield`. Each changeover, from stopping to express service and from express to onward service, has to respect the fact that the arrival time of a previous leg has to be before the departure time of the next leg. This will be encoded in a single `WHERE` clause.

The following image visualizes the three legs with different colors, and identifies the node variables used to create the equijoins and anchoring:



For the stopping service, it is assumed that the station the passenger needs to change at is unknown. As a result, the pattern needs to match a variable number of stops before and after the `Stop` `b`, the `Stop` that calls at the changeover station `l`. This is achieved by placing the quantified relationship `-[:NEXT]->*` on each side of node `b`. The ends of the path also needs to be anchored at a `Stop` departing from `Starbeck` at `11:11`, as well as at a `Stop` calling at `Leeds`:

```
(:Station {name: 'Starbeck'})<-[:CALLS_AT]-
  (a:Stop {departs: time('11:11')})-[:NEXT]->*(b)-[:NEXT]->*
  (c:Stop)-[:CALLS_AT]->(lds:Station {name: 'Leeds'})
```

For the express service, the ends of the path are anchored at the stop `b` and `Leeds` station, which `lds` will be bound to by the first leg. Although in this particular case there are only two stops on the service, a more general pattern that can match any number of stops is used:

```
(b)-[:CALLS_AT]->(l:Station)<-[:CALLS_AT]-(m:Stop)-[:NEXT]->*
  (n:Stop)-[:CALLS_AT]->(lds)
```

By default, Cypher only allows a relationship to be traversed once in a given match for a graph pattern. (The same is not true for queries using `REPEATABLE ELEMENTS`.) As a result, the first and second legs are guaranteed to be different train services. Similarly, another quantified relationship that bridges the stops calling at `Leeds` station and `Huddersfield` station is added:

```
(lds)<-[:CALLS_AT]-(x:Stop)-[:NEXT]->*(y:Stop)-[:CALLS_AT]->
  (:Station {name: 'Huddersfield'})
```

The other node variables are for the `WHERE` clause or for returning data. Putting this together, the resulting query returns the earliest arrival time achieved by switching to an express service:

Query

```
MATCH (:Station {name: 'Starbeck'})<-[:CALLS_AT]-
        (a:Stop {departs: time('11:11')})-[:NEXT]->*(b)-[:NEXT]->*
        (c:Stop)-[:CALLS_AT]->(lds:Station {name: 'Leeds'}),
      (b)-[:CALLS_AT]->(l:Station)<-[:CALLS_AT]-(m:Stop)-[:NEXT]->*
        (n:Stop)-[:CALLS_AT]->(lds),
      (lds)<-[:CALLS_AT]-(x:Stop)-[:NEXT]->*(y:Stop)-[:CALLS_AT]->
        (:Station {name: 'Huddersfield'})
WHERE b.arrives < m.departs AND n.arrives < x.departs
RETURN a.departs AS departs,
       l.name AS changeAt,
       m.departs AS changeDeparts,
       y.arrives AS arrives
ORDER BY y.arrives LIMIT 1
```

Result

departs	changeAt	changeDeparts	arrives
"11:11:00Z"	"Harrogate"	"11:20:00Z"	"12:07:00Z"

Rows: 1

Shortest paths

The Cypher keyword `SHORTEST` is used to find variations of the shortest paths between nodes. This includes the ability to look for the shortest, second-shortest (and so on) paths, all available shortest paths, and groups of paths containing the same pattern length. The `ANY` keyword, which can be used to test the reachability of nodes from a given node(s), is also explained, as is how to apply filters in queries using `SHORTEST`.

`SHORTEST` functionally replaces and extends the `shortestPath()` and `allShortestPaths()` functions. Both functions can still be used, but they are not GQL conformant. For more information, see Syntax and semantics → The `shortestPath()` and `allShortestPaths()`

functions.

Note on Cypher and GDS shortest paths

Both Cypher and Neo4j's Graph Data Science (GDS) library can be used to find variations of the shortest paths between nodes.

Use Cypher if:

- You need to specify complex graph navigation via quantified path patterns.
- Creating a graph projection takes too long.
- GDS is not available in your instance, or the size of the GDS projection is too large for your instance.

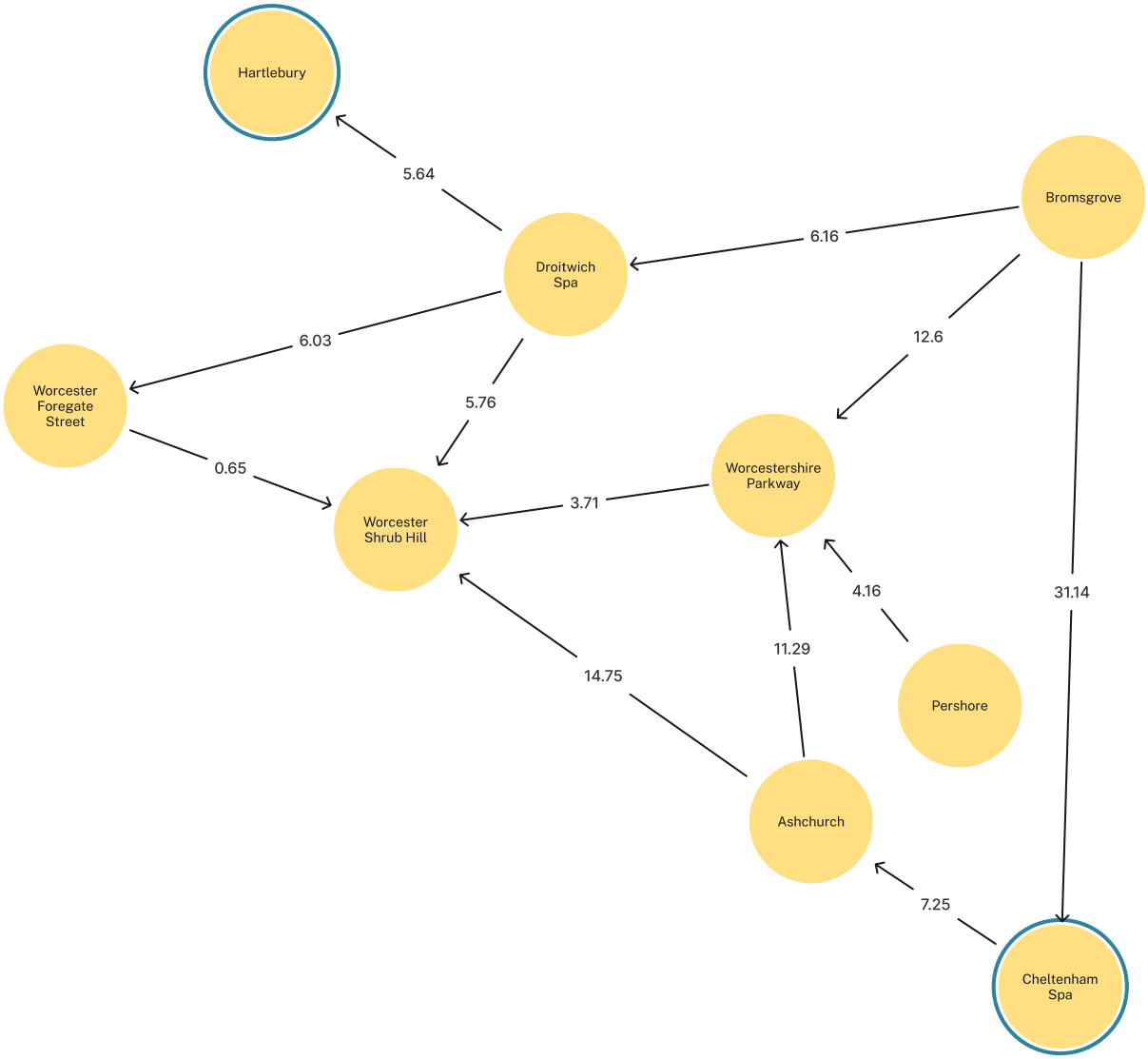
Use GDS if:

- You need to compute a weighted shortest path.
- You need a specific algorithm like A* or Yen's.
- You need to transform the graph with a projection before finding shortest path.
- You need to use shortest paths in conjunction with other GDS algorithms in the pipeline.

To read more about the shortest path algorithms included in the GDS library, see GDS Graph algorithms → Path finding.

`SHORTEST k`

This section uses the following graph:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (asc:Station {name:"Ashchurch"}),
  (bmv:Station {name:"Bromsgrove"}),
  (cnm:Station {name:"Cheltenham Spa"}),
  (dtw:Station {name:"Droitwich Spa"}),
  (hby:Station {name:"Hartlebury"}),
  (psh:Station {name:"Pershore"}),
  (wop:Station {name:"Worcestershire Parkway"}),
  (wof:Station {name:"Worcester Foregate Street"}),
  (wos:Station {name:"Worcester Shrub Hill"})
CREATE (asc)-[:LINK {distance: 7.25}]->(cnm),
  (asc)-[:LINK {distance: 11.29}]->(wop),
  (asc)-[:LINK {distance: 14.75}]->(wos),
  (bmv)-[:LINK {distance: 31.14}]->(cnm),
  (bmv)-[:LINK {distance: 6.16}]->(dtw),
  (bmv)-[:LINK {distance: 12.6}]->(wop),
  (dtw)-[:LINK {distance: 5.64}]->(hby),
  (dtw)-[:LINK {distance: 6.03}]->(wof),
  (dtw)-[:LINK {distance: 5.76}]->(wos),
  (psh)-[:LINK {distance: 4.16}]->(wop),
  (wop)-[:LINK {distance: 3.71}]->(wos),
  (wof)-[:LINK {distance: 0.65}]->(wos)
```

The paths matched by a path pattern can be restricted to only the shortest (by number of hops) by including the keyword `SHORTEST k`, where `k` is the number of paths to match, and can be either an `INTEGER` literal or, as of Neo4j 2025.06, a parameter which resolves to an `INTEGER`. For example, the following

example uses `SHORTEST 1` to return the length of the shortest path between `Worcester Shrub Hill` and `Bromsgrove`:

Query

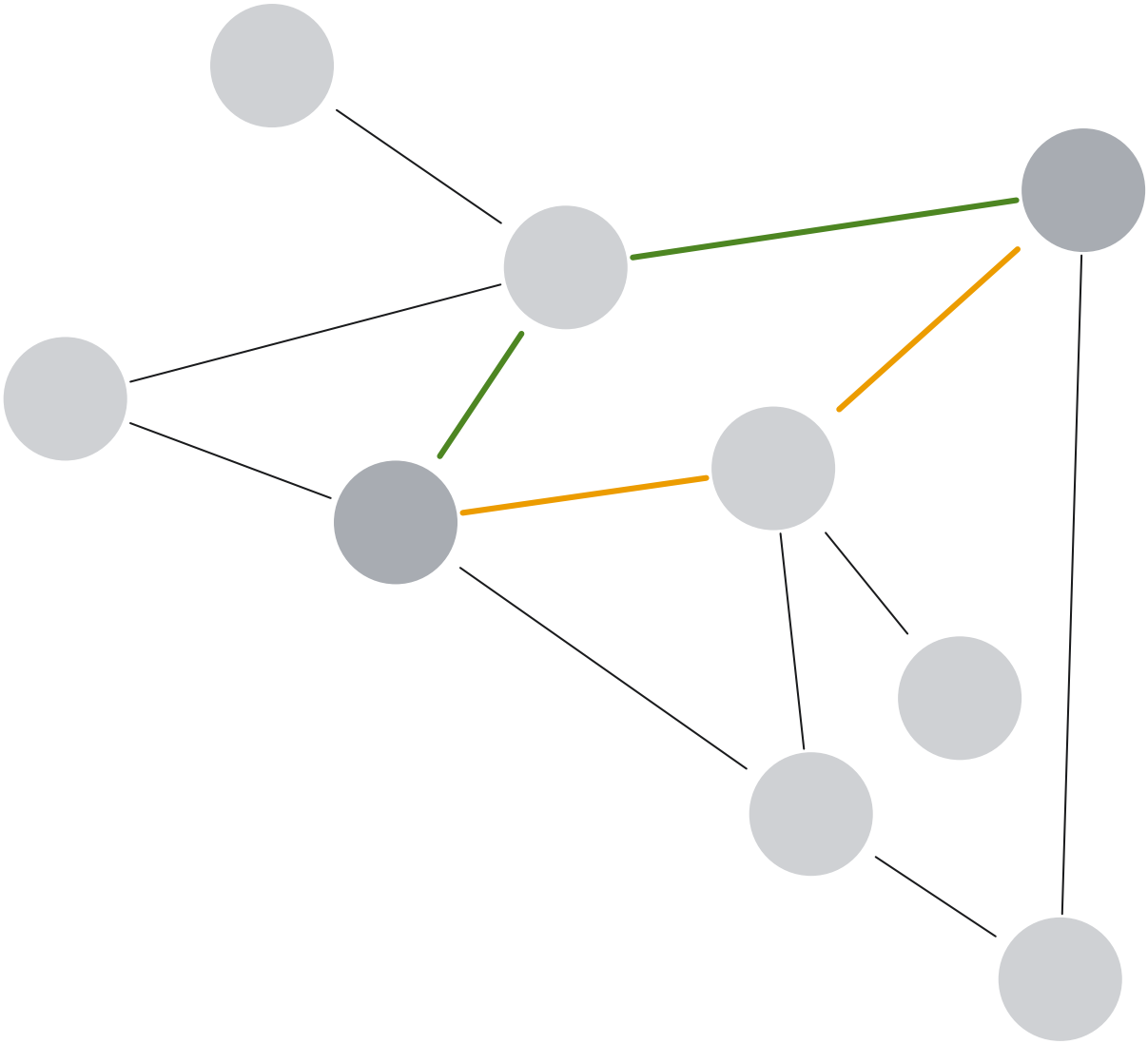
```
MATCH p = SHORTEST 1 (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN length(p) AS result
```

Note that this and the following examples in this section use a quantified relationship `-[:LINK]-+`, which is composed of a relationship pattern `-[:LINK]-` and a postfix quantifier `+`. The relationship pattern is only concerned with following relationships with type `LINK`, and will otherwise traverse any node along the way. There is no arrowhead `<` or `>` on the relationship pattern, allowing the pattern to match relationships going in either direction. This represents the fact that trains can go in both directions along the `LINK` relationships between Stations. The `+` quantifier means that one or more relationships should be matched. For more information, see Syntax and semantics - quantified relationships.

Result

result
2
Rows: 1

Although the query returned a single result, there are in fact two paths that are tied for shortest:



Because `1` was specified in `SHORTEST`, only one of the paths is returned. Which one is returned is non-deterministic.

If instead `SHORTEST 2` is specified, all shortest paths in this example would be returned, and the result would be deterministic:

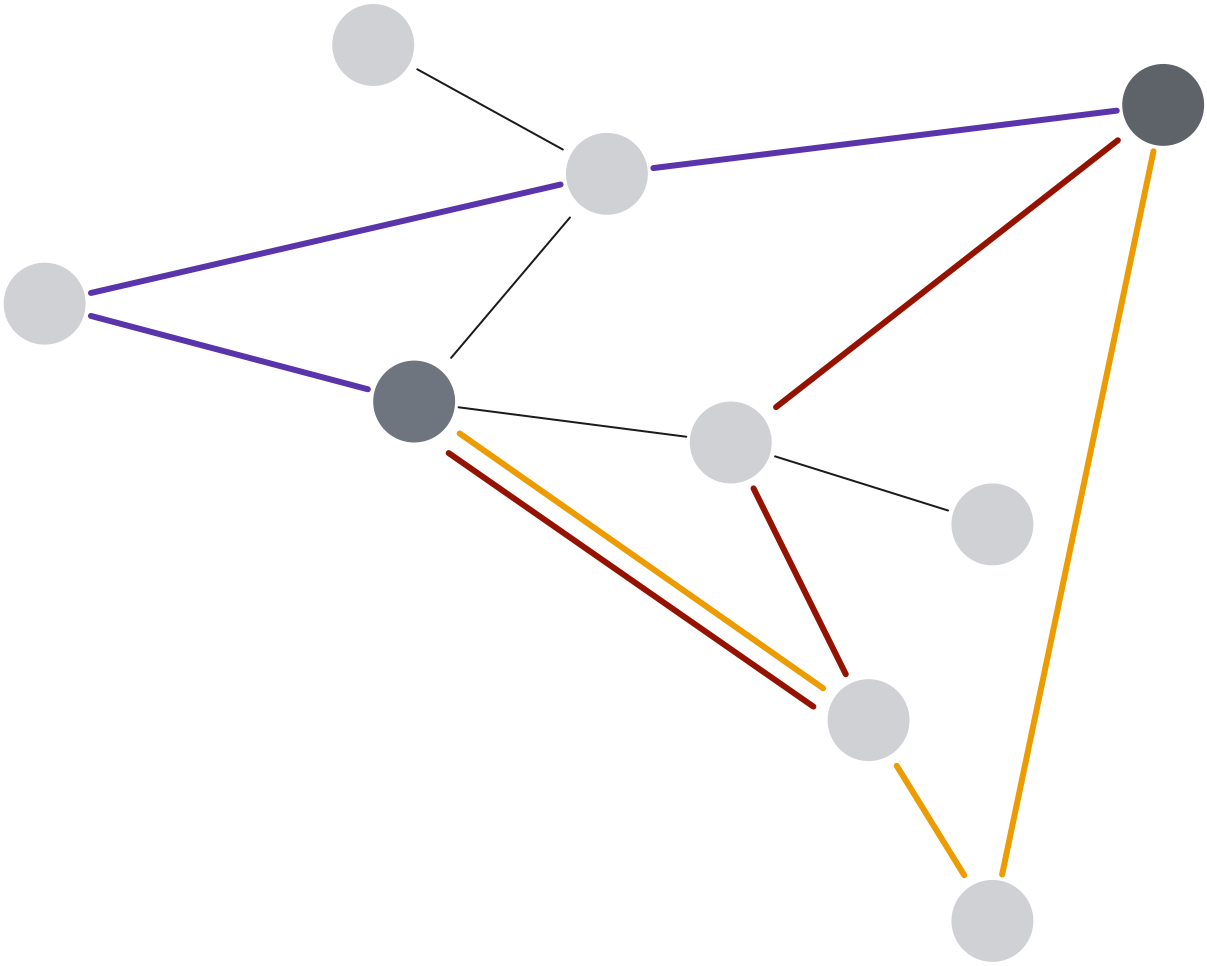
Query

```
MATCH p = SHORTEST 2 (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN [n in nodes(p) | n.name] AS stops
```

Result

stops
["Worcester Shrub Hill", "Droitwich Spa", "Bromsgrove"]
["Worcester Shrub Hill", "Worcestershire Parkway", "Bromsgrove"]
Rows: 2

Increasing the number of paths will return the next shortest paths. Three paths are tied for the second shortest:



The following query returns all three of the second shortest paths, along with the two shortest paths:

Query

```
MATCH p = SHORTEST 5 (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN [n in nodes(p) | n.name] AS stops
```

Result

stops
["Worcester Shrub Hill", "Droitwich Spa", "Bromsgrove"]
["Worcester Shrub Hill", "Worcestershire Parkway", "Bromsgrove"]
["Worcester Shrub Hill", "Worcester Foregate Street", "Droitwich Spa", "Bromsgrove"]
["Worcester Shrub Hill", "Ashchurch", "Worcestershire Parkway", "Bromsgrove"]
["Worcester Shrub Hill", "Ashchurch", "Cheltenham Spa", "Bromsgrove"]
Rows: 5

If there had been only four possible paths between the two Stations, then only those four would have been returned.

`ALL SHORTEST`

To return all paths that are tied for shortest length, use the keywords `ALL SHORTEST`:

Query

```
MATCH p = ALL SHORTEST (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN [n in nodes(p) | n.name] AS stops
```

Result

stops
["Worcester Shrub Hill", "Droitwich Spa", "Bromsgrove"]
["Worcester Shrub Hill", "Worcestershire Parkway", "Bromsgrove"]
Rows: 2

SHORTEST k GROUPS

To return all paths that are tied for first, second, and so on up to the kth shortest length, use `SHORTEST k GROUPS`. For example, the following returns the first and second shortest length paths between `Worcester Shrub Hill` and `Bromsgrove`:

Query

```
MATCH p = SHORTEST 2 GROUPS (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN [n in nodes(p) | n.name] AS stops, length(p) AS pathLength
```

Result

stops	pathLength
["Worcester Shrub Hill", "Droitwich Spa", "Bromsgrove"]	2
["Worcester Shrub Hill", "Worcestershire Parkway", "Bromsgrove"]	2
["Worcester Shrub Hill", "Worcester Foregate Street", "Droitwich Spa", "Bromsgrove"]	3
["Worcester Shrub Hill", "Ashchurch", "Worcestershire Parkway", "Bromsgrove"]	3
["Worcester Shrub Hill", "Ashchurch", "Cheltenham Spa", "Bromsgrove"]	3
Rows: 5	

The first group includes the two shortest paths with `pathLength = 2` (as seen in the first two rows of the results), and the second group includes the three second shortest paths with `pathLength = 3` (as seen in the last three rows of the results).

If more groups are specified than exist in the graph, only those paths that exist are returned. For example, if the paths equal to one of the eight shortest paths are specified for `Worcester Shrub Hill` to `Bromsgrove`, only seven groups are returned:

Query

```
MATCH p = SHORTEST 8 GROUPS (wos:Station)-[:LINK]-+(bmv:Station)
WHERE wos.name = "Worcester Shrub Hill" AND bmv.name = "Bromsgrove"
RETURN length(p) AS pathLength, count(*) AS numPaths
```

Result

pathLength	numPaths
2	2
3	3
4	1
5	4
6	8
7	10
8	6

Rows: 7

ANY

The `ANY` keyword can be used to test the reachability of nodes from a given node(s). It returns the same as `SHORTEST 1`, but by using the `ANY` keyword the intent of the query is clearer. For example, the following query shows that there exists a route from `Pershore` to `Bromsgrove` where the distance between each pair of stations is less than 10 miles:

Query

```
MATCH path = ANY
  (:Station {name: 'Pershore'})-[l:LINK WHERE l.distance < 10]-+(b:Station {name: 'Bromsgrove'})
RETURN [r IN relationships(path) | r.distance] AS distances
```

Result

distances
[4.16, 3.71, 5.76, 6.16]

Rows: 1

Partitions

When there are multiple start or end nodes matching a path pattern, the matches are partitioned into distinct pairs of start and end nodes prior to selecting the shortest paths; a partition is one distinct pair of start node and end node. The selection of shortest paths is then done from all paths that join the start and end node of a given partition. The results are then formed from the union of all the shortest paths found for each partition.

For example, if the start nodes of matches are bound to either `Droitwich Spa` or `Hartlebury`, and the end

nodes are bound to either `Ashchurch` or `Cheltenham Spa`, there will be four distinct pairs of start and end nodes, and therefore four partitions:

Start node	End node
`Droitwich Spa`	`Ashchurch`
`Droitwich Spa`	`Cheltenham Spa`
`Hartlebury`	`Ashchurch`
`Hartlebury`	`Cheltenham Spa`

The following query illustrates how these partitions define the sets of results within which the shortest paths are selected. It uses a pair of `UNWIND` clauses to generate a Cartesian product of the names of the `Stations` (all possible pairs of start node and end node), followed by the `MATCH` clause to find the shortest two groups of paths for each pair of distinct start and end `Stations`:

Query

```
UNWIND ["Droitwich Spa", "Hartlebury"] AS a
UNWIND ["Ashchurch", "Cheltenham Spa"] AS b
MATCH SHORTEST 2 GROUPS (o:Station {name: a})-[l]-+(d:Station {name: b})
RETURN o.name AS start, d.name AS end,
    size(l) AS pathLength, count(*) AS numPaths
ORDER BY start, end, pathLength
```

Result

start	end	pathLength	numPaths
"Droitwich Spa"	"Ashchurch"	2	1
"Droitwich Spa"	"Ashchurch"	3	4
"Droitwich Spa"	"Cheltenham Spa"	2	1
"Droitwich Spa"	"Cheltenham Spa"	3	1
"Hartlebury"	"Ashchurch"	3	1
"Hartlebury"	"Ashchurch"	4	4
"Hartlebury"	"Cheltenham Spa"	3	1
"Hartlebury"	"Cheltenham Spa"	4	1

Rows: 8

Each partition appears twice: once for the group of shortest paths and once for the group of second shortest paths. For example, for the partition of `Droitwich Spa` as the `start` and `Ashchurch` as the `end`, the shortest path group (paths with length `2`) has one path, and the second shortest path group (paths with length `3`) has four paths.

Pre-filters and post-filters

The position of a filter in a shortest path query will affect whether it is applied before or after selecting the shortest paths. To see the difference, first consider a query that returns the shortest path from `Hartlebury`

to `Cheltenham Spa`:

Query

```
MATCH SHORTEST 1
  (:Station {name: 'Hartlebury'})
  (()--(n))+
  (:Station {name: 'Cheltenham Spa'})
RETURN [stop in n[..-1] | stop.name] AS stops
```

Result

stops
["Droitwich Spa", "Bromsgrove"]
Rows: 1

Note that `n[..-1]` is a slicing operation that returns all elements of `n` except the last. If instead, the query uses a `WHERE` clause at the `MATCH` level to filter out routes that go via Bromsgrove, the filtering is applied after the shortest paths are selected. This results in the only solution being removed, and no results being returned:

Query

```
MATCH SHORTEST 1
  (:Station {name: 'Hartlebury'})
  (()--(n:Station))+
  (:Station {name: 'Cheltenham Spa'})
WHERE none(stop IN n[..-1] WHERE stop.name = 'Bromsgrove')
RETURN [stop in n[..-1] | stop.name] AS stops
```

Result

stops
Rows: 0

There are two ways to turn a post-filter without solutions into a pre-filter that returns solutions. One is to inline the predicate into the path pattern:

Query

```
MATCH SHORTEST 1
  (:Station {name: 'Hartlebury'})
  (()--(n:Station WHERE n.name <> 'Bromsgrove'))+
  (:Station {name: 'Cheltenham Spa'})
RETURN [stop in n[..-1] | stop.name] AS stops
```

Result

stops
["Droitwich Spa", "Worcester Shrub Hill", "Ashchurch"]
Rows: 1

The shortest journey that avoids `Bromsgrove` is now returned.

An alternative is to wrap the path pattern and filter in parentheses (leaving the `SHORTEST` keyword on the outside):

Query

```
MATCH SHORTEST 1
  ( (:Station {name: 'Hartlebury'})
    (()--(n:Station))+
    (:Station {name: 'Cheltenham Spa'})
     WHERE none(stop IN n[..-1] WHERE stop.name = 'Bromsgrove') )
RETURN [stop in n[..-1] | stop.name] AS stops
```

Result

stops
["Droitwich Spa", "Worcester Shrub Hill", "Ashchurch"]
Rows: 1

Pre-filter with a path variable

The previous section showed how to apply a filter before the shortest path selection by the use of parentheses. Placing a path variable declaration before the shortest path keywords, however, places it outside the scope of the parentheses. To reference a path variable in a pre-filter, it has to be declared inside the parentheses.

To illustrate, consider this example that returns all shortest paths from `Hartlebury` to each of the other `Stations`:

Query

```
MATCH p = SHORTEST 1 (:Station {name: 'Hartlebury'})--+(b:Station)
RETURN b.name AS destination, length(p) AS pathLength
ORDER BY pathLength, destination
```

Result

destination	pathLength
"Droitwich Spa"	1
"Bromsgrove"	2
"Worcester Foregate Street"	2
"Worcester Shrub Hill"	2
"Ashchurch"	3
"Cheltenham Spa"	3
"Worcestershire Parkway"	3
"Pershore"	4

destination	pathLength
Rows: 8	

If the query is altered to only include routes that have an even number of stops, adding a `WHERE` clause at the `MATCH` level will not work, because it would be a post-filter. It would return the results of the previous query with all routes with an odd number of stops removed:

Query

```
MATCH p = SHORTEST 1 (:Station {name: 'Hartlebury'})--+(b:Station)
WHERE length(p) % 2 = 0
RETURN b.name AS destination, length(p) AS pathLength
ORDER BY pathLength, destination
```

Result

destination	pathLength
"Bromsgrove"	2
"Worcester Foregate Street"	2
"Worcester Shrub Hill"	2
"Pershore"	4
Rows: 4	

To move the predicate to a pre-filter, the path variable should be referenced from within the parentheses, and the shortest routes with an even number of stops will be returned for all the destinations:

Query

```
MATCH SHORTEST 1
  (p = (:Station {name: 'Hartlebury'})--+(b:Station)
    WHERE length(p) % 2 = 0 )
RETURN b.name AS destination, length(p) AS pathLength
ORDER BY pathLength, destination
```

Result

destination	pathLength
"Bromsgrove"	2
"Worcester Foregate Street"	2
"Worcester Shrub Hill"	2
"Ashchurch"	4
"Cheltenham Spa"	4
"Droitwich Spa"	4
"Pershore"	4
"Worcestershire Parkway"	4

destination	pathLength
Rows: 8	

Performance and planning

Shortest path queries often perform better when the Cypher planner can identify a single source-target node pair for a shortest path. This is because it allows the planner to use a bidirectional search from the source and target nodes and terminate when the shortest path between them is found, rather than traversing the whole graph for potential target nodes. However, while there are strategies to enforce this optimization, forcing Cypher to use them does not always improve performance.

If the planner estimates a single source-target node pair, Cypher uses either the `ShortestPath` or the `StatefulShortestPath(Into)` operators; otherwise it uses `StatefulShortestPath(All)`. Each of these operators, and the criteria for their use, is outlined below.

For readers not familiar with Cypher execution plans and operators, it is recommended to first read Understanding query plans.

Example graph

This section uses a graph structured as a tree with a branching factor of three and a depth of nine. This means the graph starts with a single root node, and each node creates three child nodes, continuing this pattern for nine levels. In total, the graph consists of almost 90 000 nodes.

Each node (`N`) has a `trail` property that acts as a "breadcrumb" list, tracing the path from the root of the tree to that node (the root node has an empty list for its `trail` property). The trail records the choice of `"A"`, `"B"`, or `"C"` taken to reach each node, building a sequence that increases in length with each level. This ensures that every `trail` value is unique in the graph.

To recreate the graph, run the following query against an empty Neo4j database:

```
WITH 9 AS depth, 3 AS branching
CREATE (:N {level: 0, trail: []})
WITH *
UNWIND range(0, depth) AS level
CALL (branching, level) {
   MATCH (n {level: level})
   UNWIND ["A", "B", "C"] AS branch
   CREATE (n)-[:R]->(:N {level: level + 1, trail: n.trail + [branch]})
};
CREATE RANGE INDEX level_index FOR (n:N) ON n.level
```

Enforcing a single source-target node pair with `CALL` subqueries

One way for Cypher to enforce a single-source node pair for a shortest path is to rewrite the query to include a `CALL` subquery. This is because, with `CALL`, each incoming row already has the source and target nodes bound, so the cardinality (i.e. count) for each is known to be 1. Without `CALL`, the planner does not know how many targets there are per source node, even though it still runs the shortest-path search once per source, and this may generate less efficient queries.

Example 110. `CALL` subqueries and shortest path query performance

Find a shortest path between two nodes

```
PROFILE
MATCH p = ANY SHORTEST
  (source:N {trail: ["C", "C", "A", "C", "A", "B", "B", "B", "A"]})--{2,}
  (target:N {trail: ["A", "B", "C", "A", "B", "C", "A", "B", "C"]})
RETURN length(p) AS pathLength
```

Result

pathLength
18
Rows: 1

The plan generated by this query shows that the planner cannot ascertain the existence of a single source-target node pair, even though the `trail` property is unique to each node in the graph (no index has been created on this property yet). It, therefore, must exhaust all possible target nodes before determining the shortest path from the source using the `StatefulShortestPath(All)` operator.

Query Plan

```
+-----------------------------------+----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator                          | Id | Details
| Estimated Rows | Rows  | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------------+----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults                   |  0 | pathLength
|       19612941 |     1 |       0 |              0 |                    0/0 |     0.088 |
|
| |                                 +----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+                     |
| +Projection                       |  1 | length((source)-[anon_39*]-(target)) AS pathLength
|       19612941 |     1 |      17 |               |                    9/0 |     0.091 |
|
| |                                 +----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+                     |
| +StatefulShortestPath(All, Trail) |  2 | SHORTEST 1 (source) ((`anon_35`)-[`anon_36`]-
(`anon_37`)){2, } (target) |       19612941 |     1 |  354300 |       65608284 |
85662/0 |   158.664 | In Pipeline 1       |
| |                                 |    |         expanding from: source
|                |       |         |               |                         |           |
|
| |                                 |    |     inlined predicates: target.trail = $autolist_1
|                |       |         |               |                         |           |
|
| |                                 |    |                         target:N
|                |       |         |               |                         |           |
|
| |                                 +----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------------+
| +Filter                           |  3 | source.trail = $autolist_0
|           4429 |     1 |  177146 |               |                         |           |
|
| |                                 +----
+-------------------------------------------------------------------------------+----------------+-------
+---------+----------------+                       |           |                     |
| +NodeByLabelScan                  |  4 | source:N
```

```
|         88573 | 88573 |   88574 |           376 |                1853/0 |    53.311 | Fused in
Pipeline 0 |
+-----------------------------------+----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+--------------------+

Total database accesses: 620037, total allocated memory: 65608620

1 row
ready to start consuming query after 45 ms, results consumed after another 214 ms
```

However, since each `trail` property is unique, rewriting the query to use a `CALL` subquery yields a more efficient plan. This is because it forces the planner to use the `StatefulShortestPath(Into)` operator, which expands from the source node until it finds its specific target node, and ensures that `ANY SHORTEST` is executed once per source-target pair.

Shortest path query rewritten with a `CALL` subquery

```
PROFILE
MATCH (start:N {trail: ["C", "C", "A", "C", "A", "B", "B", "B", "A"]}),
      (end:N {trail: ["A", "B", "C", "A", "B", "C", "A", "B", "C"]})
CALL (start, end) {
   MATCH p = ANY SHORTEST (start)--{2,}(end)
   RETURN p
}
RETURN length(p) AS pathLength
```

The result is a significantly faster query (down from 45 to 4 milliseconds):

Query Plan

```
+-----------------------------------+----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+--------------------+
| Operator                          | Id | Details
| Estimated Rows | Rows  | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------------+----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+--------------------+
| +ProduceResults                   |  0 | pathLength
|       19612941 |     1 |       0 |              0 |                   0/0 |     0.080 |
|
| |                                 +----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+                    |
| +Projection                       |  1 | length(p) AS pathLength
|       19612941 |     1 |       0 |                |                   0/0 |     0.023 |
|
| |                                 +----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+                    |
| +Projection                       |  2 | (start)-[anon_15*]-(end) AS p
|       19612941 |     1 |      17 |                |                   9/0 |     0.063 |
|
| |                                 +----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+                    |
| +StatefulShortestPath(Into, Trail) |  3 | SHORTEST 1 (start) ((`anon_11`)-[`anon_12`]-
(`anon_13`)){2, } (end) |       19612941 |     1 |    1952 |        1337576 |                  209/0
|     1.938 | In Pipeline 3       |
| |                                 +----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+--------------------+
| +CartesianProduct                 |  4 |
|       19612941 |     1 |       0 |           9040 |                     |     0.054 | In
Pipeline 2       |
| |\                                +----
+-------------------------------------------------------------------------+---------------+-------
+---------+---------------+----------------------+-----------+--------------------+
```

```
| | +Filter                          |  5 | end.trail = $autolist_1
|          22143 |     1 |  177146 |               |                       |           |
|
| | |                                +----
+-----------------------------------------------------------------------+----------------+-------
+---------+----------------+                       |           |                     |
| | +NodeByLabelScan                |  6 | end:N
|          442865 | 88573 |   88574 |            392 |                1853/0 |    44.047 | Fused in
Pipeline 1 |
| |                                  +----
+-----------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------------+
| +Filter                          |  7 | start.trail = $autolist_0
|           4429 |     1 |  177146 |               |                       |           |
|
| |                                  +----
+-----------------------------------------------------------------------+----------------+-------
+---------+----------------+                       |           |                     |
| +NodeByLabelScan                 |  8 | start:N
|          88573 | 88573 |   88574 |            376 |                1853/0 |    57.818 | Fused in
Pipeline 0 |
+----------------------------------+----
+-----------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------------+

Total database accesses: 533409, total allocated memory: 1346888

1 row
ready to start consuming query after 4 ms, results consumed after another 106 ms
```

Enforcing a single source-target node pair with indexes and constraints

Another way to inform the planner of the uniqueness of the target node in a shortest path is to create an index or property uniqueness/key constraint (both of which are index-backed) on a property belonging to the matched nodes in the shortest path. This will accurately inform the planner of node cardinality and thereby enable more efficient query planning (assuming the graph contains uniquely identifying node properties).

Example 111. Impact of indexes and constraints

Create a property uniqueness constraint on the `trail` property

```
CREATE CONSTRAINT unique_trail FOR (n:N) REQUIRE n.trail IS UNIQUE
```

This constraint will inform the planner of the uniqueness of `trail` values up front. As a result, the simpler shortest path query (without a `CALL` subquery) will now generate a faster plan (using the `StatefulShortestPath(Into)` operator) with a cardinality of 1 for both the source and target nodes of the shortest path.

Find a shortest path between two nodes

```
PROFILE
MATCH p = ANY SHORTEST
  (source:N {trail: ["C", "C", "A", "C", "A", "B", "B", "B", "A"]})--{2,}
  (target:N {trail: ["A", "B", "C", "A", "B", "C", "A", "B", "C"]})
RETURN length(p) AS pathLength
```

Query Plan

```
+----------------------------------+----
+-------------------------------------------------------------------------------------------------------
```

```
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| Operator                          | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +ProduceResults                   |  0 | pathLength
|              1 |    1 |       0 |              0 |                    0/0 |     0.058 |
|
| |                                 +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +Projection                       |  1 | length((source)-[anon_55*]-(target)) AS pathLength
|              1 |    1 |      17 |                |                    9/0 |     0.081 |
|
| |                                 +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +StatefulShortestPath(Into, Trail) |  2 | SHORTEST 1 (source) ((`anon_51`)-[`anon_52`]-
(`anon_53`)){2, } (target)                          |              1 |    1 |    1952 |
1337568 |                 209/0 |     2.213 | In Pipeline 1 |
| |                                 +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +MultiNodeIndexSeek               |  3 | UNIQUE source:N(trail) WHERE trail = $autolist_0, UNIQUE
target:N(trail) WHERE trail = $autolist_1 |              1 |    1 |       4 |            376 |
1/5 |     0.400 | In Pipeline 0 |
+-----------------------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+

Total database accesses: 1957, total allocated memory: 990608

1 row
ready to start consuming query after 48 ms, results consumed after another 3 ms
```

Limitations of enforcing a single source-target node pair

Enforcing a single source-target node pair is not always preferable. With one source and many targets, rewriting a shortest path query using a `CALL` subquery forces the planner to use either the `ShortestPath` or `StatefulShortestPath(Into)` operator, which runs once per target node. While this is efficient for a single pair, it can become slower as the number of targets increases because it forces the planner to traverse the graph for each individual pair of source-target nodes. In such cases, it may be more efficient to let the planner use `StatefulShortestPath(All)`, which expands across the graph once and returns all matches.

Example 112. Efficient planning for multi-target shortest paths

Consider the following query, which does not specify a unique target node and generates a total of 19682 shortest paths from the source nodes:

Find a shortest path to many target nodes

```
PROFILE
MATCH p = ANY SHORTEST
  (start:N {trail: ["C", "C", "A", "C", "A", "B", "B", "B", "A"]})--{2,}
  (end:N {level: 9})
RETURN count(*) AS pathCount
```

Result

pathLength
19682
Rows: 1

Due to the existence of multiple target nodes without a specified, unique property value (there are 19683 nodes in the graph with a `level` property value of `9`), the planner will default to using the `StatefulShortestPath(All)` operator, which expands once from the source node until all valid shortest paths have been found.

Query Plan

```
+-----------------------------------+----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------+
| Operator                          | Id | Details
| Estimated Rows | Rows  | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------------+----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------+
| +ProduceResults                   |  0 | pathCount
|              1 |     1 |       0 |              0 |                   0/0 |     0.082 |
|
| |                                 +----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+               |
| +EagerAggregation                 |  1 | count(*) AS pathCount
|              1 |     1 |       0 |             40 |                   0/0 |     1.151 | In
Pipeline 2 |
| |                                 +----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------+
| +StatefulShortestPath(All, Trail) |  2 | SHORTEST 1 (start) ((`anon_19`)-[`anon_20`]-
(`anon_21`)){2, } (end) |          8052 | 19682 |  373982 |       82162292 |                65705/0
|   455.921 | In Pipeline 1 |
| |                                 |    |         expanding from: start
|                |       |         |               |                          |           |
|
| |                                 |    |     inlined predicates: end.level = $autoint_1
|                |       |         |               |                          |           |
|
| |                                 |    |                          end:N
|                |       |         |               |                          |           |
|
| |                                 +----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------+
| +NodeUniqueIndexSeek              |  3 | UNIQUE start:N(trail) WHERE trail = $autolist_0
|              1 |     1 |       2 |            376 |                   3/0 |     0.509 | In
Pipeline 0 |
+-----------------------------------+----
+-------------------------------------------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+---------------+

Total database accesses: 373984, total allocated memory: 82162652

1 row
ready to start consuming query after 83 ms, results consumed after another 592 ms
```

If the query is rewritten with a `CALL` subquery the planner will use the `StatefulShortestPath(Into)` operator which performs separate traversals for each individual source-target node pairs.

Multi-target shortest path query rewritten with a `CALL` subquery

```
PROFILE
MATCH (start:N {trail: ["C", "C", "A", "C", "A", "B", "B", "B", "A"]}),
      (end:N {level: 9})
CALL (start, end) {
   MATCH p = ANY SHORTEST (start)--{2,}(end)
   RETURN p
}
RETURN count(*) AS pathCount
```

Query Plan

```
+-----------------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------
+---------------+
| Operator                          | Id | Details
| Estimated Rows | Rows  | DB Hits  | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------
+---------------+
| +ProduceResults                   |  0 | pathCount
|              1 |     1 |        0 |              0 |                    0/0 |     0.185 |
|
| |                                 +----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------+
|
| +EagerAggregation                 |  1 | count(*) AS pathCount
|              1 |     1 |        0 |             40 |                    0/0 |     1.571 | In
Pipeline 2 |
| |                                 +----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------
+---------------+
| +Projection                       |  2 | (start)-[anon_15*]-(end) AS p
|           8052 | 19682 |   314930 |                |               189543/0 |    67.007 |
|
| |                                 +----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------+
|
| +StatefulShortestPath(Into, Trail) |  3 | SHORTEST 1 (start) ((`anon_11`)-[`anon_12`]-
(`anon_13`)){2, } (end)                              |           8052 | 19682 | 32986690 |
221424672 |             3775866/0 | 22680.410 | In Pipeline 1 |
| |                                 +----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------
+---------------+
| +MultiNodeIndexSeek               |  4 | UNIQUE start:N(trail) WHERE trail = $autolist_0, RANGE
INDEX end:N(level) WHERE level = $autoint_1 |           8052 | 19683 |    19686 |            376 |
57/0 |    12.979 | In Pipeline 0 |
+-----------------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+-------+----------+----------------+------------------------+-----------
+---------------+

Total database accesses: 33321306, total allocated memory: 221425168

1 row
ready to start consuming query after 49 ms, results consumed after another 22774 ms
```

As the plan shows, in this scenario it is not more efficient to enforce a single source-target node pair. On the contrary, doing so ensures that the `StatefulShortestPath(Into)` operator is executed `19682` times, once for each source-target node pair, thereby generating a more expensive query.

Shortest path operators

Cypher uses three different operators to plan shortest path queries. The criteria for when each is used is outlined in the table below.

Operator	Description	Criteria
`ShortestPath`	Performs bidirectional breadth-first searches (BFS) from target and source node. Terminates when a shortest path is found between them.	Used when: • The `shortestPath()` or `allShortestPaths()` functions are used. Or, when the estimated cardinality of the source and target nodes in a shortest path are 1 or less and the following are all true: • Selector is one of: `SHORTEST 1`, `ANY`, `ANY SHORTEST`, `ALL SHORTEST`, `SHORTEST GROUP`, or `SHORTEST 1 GROUP`. • There is only one relationship pattern. • If the pattern is a quantified path pattern in the form of `(()-[]-())`, or is a quantified-relationship, and it uses a filter that can be applied directly to the relationship. • If the pattern is a quantified path pattern or a quantified relationship and the lower bound of the quantifier is 0 or 1. • In the case of a quantified path pattern, there are no node variables declared inside the quantified path pattern that are referenced elsewhere.
`StatefulShortestPath(Into)`	Performs bidirectional BFS from target and source node. Terminates when a shortest path is found between them.	Used when the estimated cardinality of the source and target nodes in a shortest path are 1 or less and either of the following are true: • Selector is one of: `ANY/SHORTEST k` or `SHORTEST k GROUPS` and `k` is larger than `1`. • There is more than one relationship pattern.
`StatefulShortestPath(All)`	Performs unidirectional BFS to find shortest paths from a source node to all nodes matching the target node conditions.	Used when the planner estimates more than one source-target node pair in a shortest path.

`StatefulShortestPath(Into)` and `StatefulShortestPath(All)` can match more complex shortest paths than `ShortestPath`. As a result, queries using these operators may be slower and more costly.

`ShortestPath` operator: fast vs. exhaustive search

Queries planned with the `ShortestPath` operator (see the table above for when this operator is used), use

two different search algorithms depending on the predicates in the query.

If the predicate can be checked as the search progresses (for example, requiring every relationship in the path to have a specific property), the planner can exclude invalid paths early. In such cases, a fast bidirectional breadth-first search (BFS) algorithm is used.

Fast search-algorithm

```
MATCH (start:N {level: 1}), (end:N {level: 5})
MATCH p = shortestPath((start)-[r*]-(end))
WHERE all(rel IN r WHERE rel.flag IS NULL)
RETURN p
```

If the predicate requires inspecting the entire path after it has been matched (such as checking whether the path length exceeds a certain value), the planner cannot exclude paths early. In such cases, a slower, exhaustive search-algorithm is used. Exhaustive searches may be very time consuming in certain cases, such as when there is no shortest path between two nodes (to disallow exhaustive searches, set `dbms.cypher.forbid_exhaustive_shortestpath` to `true`).

Exhaustive search-algorithm

```
MATCH (start:N {level: 1}), (end:N {level: 5})
MATCH p = shortestPath((start)-[*]-(end))
WHERE length(p) > 3
RETURN p
```

For queries that would otherwise trigger an exhaustive search, a practical workaround is to first bind the matched path and then filter it using a `FILTER` clause (`FILTER` is a separate clause that performs a post-match filter, unlike `WHERE` which adds constraints to the pattern matched by the `MATCH` clause). This allows the planner to use a fast search-algorithm while finding the shortest path, and only afterwards apply the filter. Note that, because the filter is applied after the fast algorithm runs, it may eliminate all candidate paths and return no results.

Query rewritten to use fast search-algorithm

```
MATCH (start:N {level: 1}), (end:N {level: 5})
MATCH p = shortestPath((start)-[*]-(end))
FILTER length(p) > 3
RETURN p
```

Paths with unique relationships

By default Cypher does not allow the same relationship to be traversed more than once in a given `MATCH` result, regardless of the direction it is traversed in. The same restriction does not hold for nodes, which may be re-traversed any number of times in a matched path.

Note:

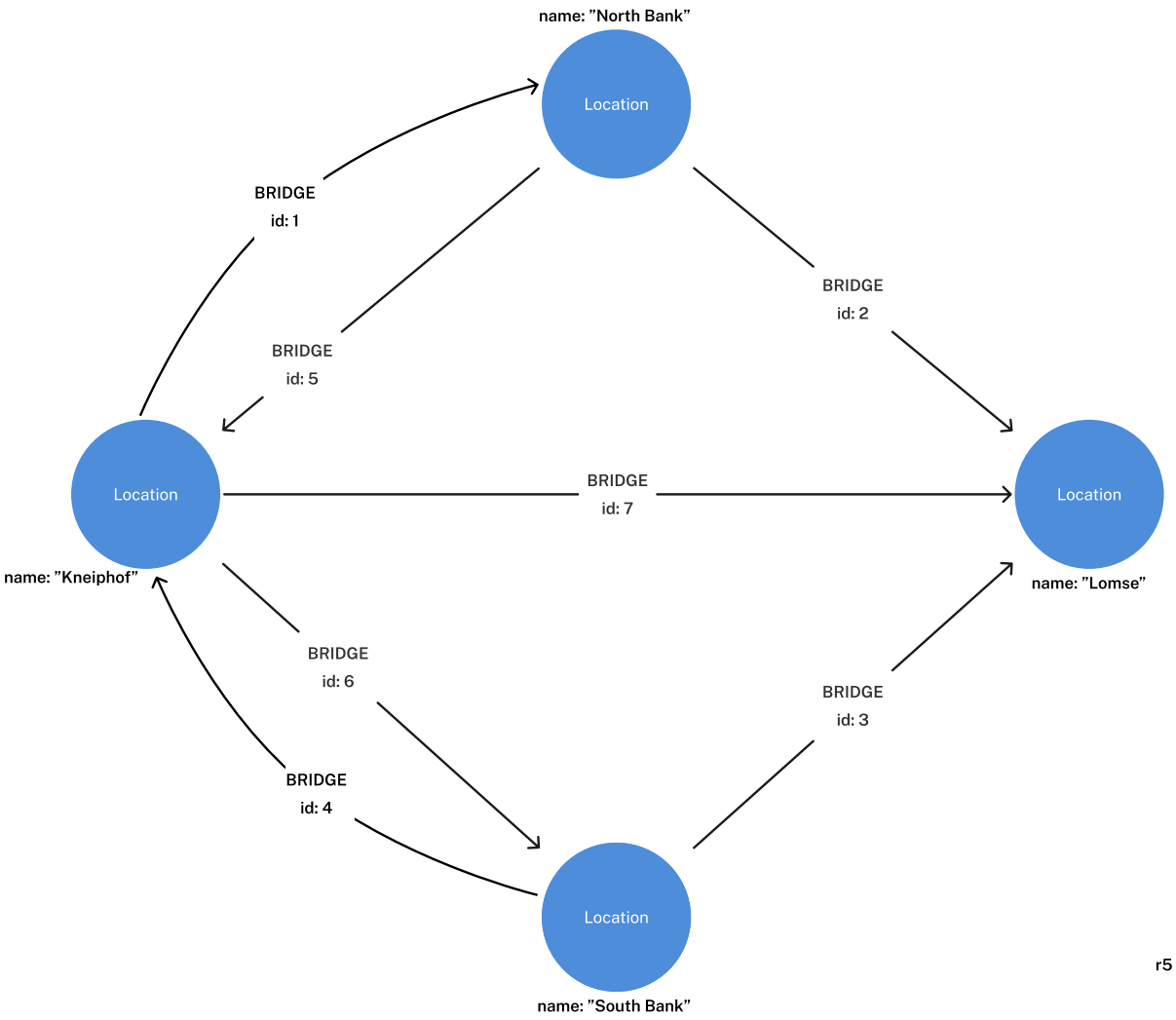
Cypher's default pattern matching behavior can be explicitly specified using the `DIFFERENT RELATIONSHIPS` keyword (released in Neo4j 2025.06 and only available in Cypher 25) after `MATCH`. Specifying `DIFFERENT RELATIONSHIPS` does not alter how a graph pattern is matched. For example: `MATCH DIFFERENT RELATIONSHIPS p = (start)--{,2}(end)` is functionally

equivalent to `MATCH p = (start)--{,2}(end)`.

`DIFFERENT RELATIONSHIPS` is a GQL conformant match mode. Match modes determine whether relationships can appear more than once in a graph pattern match. For more information, see Syntax and semantics → Match modes.

Example graph

To explain Cypher's default behavior, which allows for the re-traversal of nodes but not relationships in the same `MATCH` result, this page will use the scenario of the Seven bridges of Königsberg, a problem studied by Leonhard Euler in 1736 to determine if one could cross all seven bridges of Königsberg exactly once in a single walk.



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (kneiphof:Location {name: "Kneiphof"}),
       (northBank:Location {name: "North Bank"}),
       (southBank:Location {name: "South Bank"}),
       (lomse:Location {name: "Lomse"}),

       (kneiphof)-[:BRIDGE {id: 1}]->(northBank),
       (kneiphof)-[:BRIDGE {id: 6}]->(southBank),
       (kneiphof)-[:BRIDGE {id: 7}]->(lomse),
       (northBank)-[:BRIDGE {id: 5}]->(kneiphof),
       (northBank)-[:BRIDGE {id: 2}]->(lomse),
       (southBank)-[:BRIDGE {id: 4}]->(kneiphof),
```

```
      (southBank)-[:BRIDGE {id: 3}]->(lomse)
```

Cypher's default traversal rules and the Königsberg bridges problem

Cypher's default `MATCH` behavior is ideal to showcase the Königsberg bridges problem: in the graph model, not traversing a relationship means not traversing a bridge, the problem's key constraint.

Consider the following query, which uses a quantified relationship to search for paths exactly `5` hops away from the start `Location` node, `Kneiphof`. The relationships used in this query are directional, meaning paths are traversed following specific directions for each bridge (`-[:BRIDGE]->`).

Find paths with a length of 5 relationships from a start node

```
MATCH p = (:Location {name: 'Kneiphof'})-[:BRIDGE]->{5}()
RETURN [n IN nodes(p) | n.name] AS locations,
       [r IN relationships(p) | r.id] AS crossedBridges
```

The list comprehensions iterate over nodes and relationships in a path and return specific properties (the `name` property from nodes, and the `id` property from relationships).

As the results show, a node may be traversed several times in the same path, but a relationship (bridge) is never re-traversed (for example, the path `[6, 4, 1, 5, 6]`, which traverses the same bridge more than once, cannot be matched using the `DIFFERENT RELATIONSHIPS` match mode).

Result

locations	crossedBridges
`["Kneiphof", "North Bank", "Kneiphof", "South Bank", "Kneiphof", "Lomse"]`	`[1, 5, 6, 4, 7]`
`["Kneiphof", "South Bank", "Kneiphof", "North Bank", "Kneiphof", "Lomse"]`	`[6, 4, 1, 5, 7]`
Rows: 2	

In the next example, the direction is removed from the pattern, and the path length is increased to `6`. It shows that there are `48` valid paths with this length, where no relationships (bridges) are re-traversed. The query returns one of those paths as a `samplePath`, showing both the nodes and relationships traversed therein.

Find paths with a length of 6 relationships from a start node

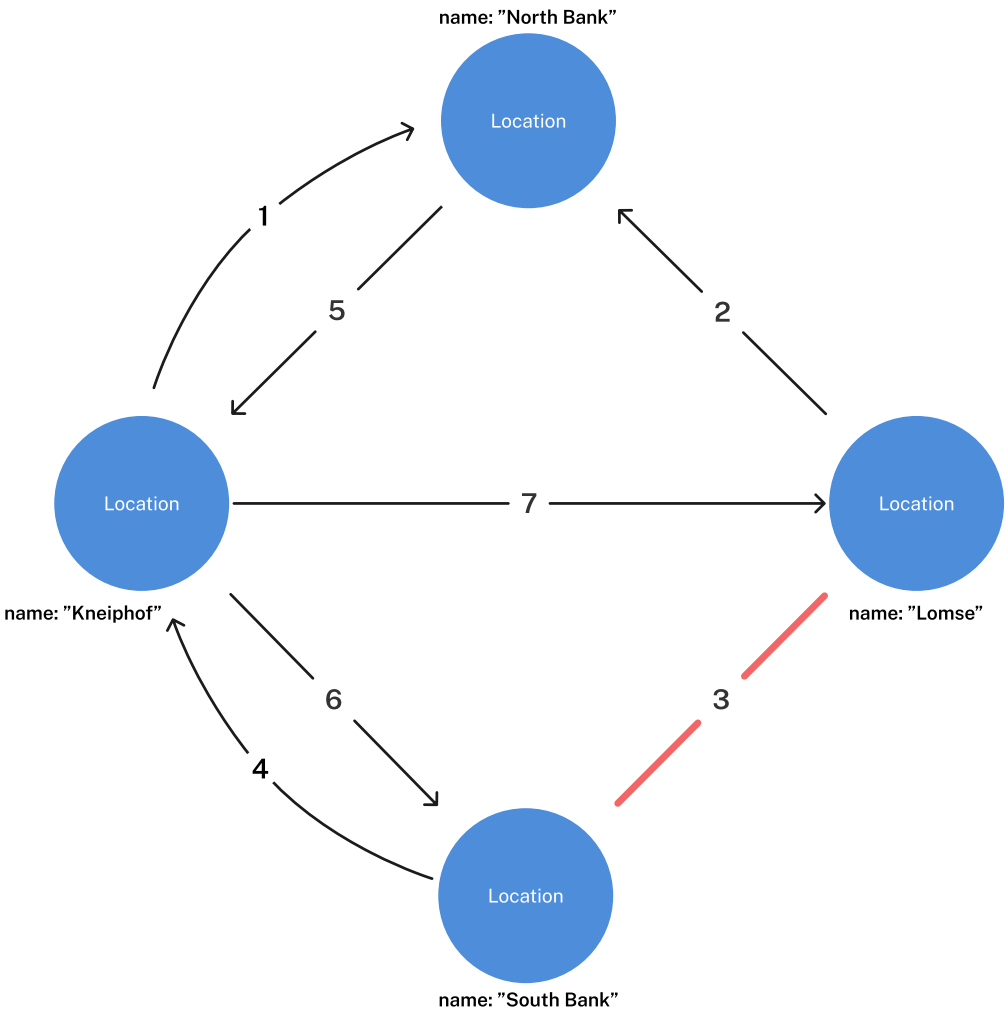
```
MATCH p = (:Location {name: 'Kneiphof'})-[:BRIDGE]-{6}()
WITH count(p) AS pathCount, collect(p)[0] AS samplePath
ORDER BY pathCount
RETURN pathCount,
       [n IN nodes(samplePath) | n.name] AS samplePathLocations,
       [r IN relationships(samplePath) | r.id] AS samplePathBridges
```

The `collect()` function collects all paths and `[0]` takes the first entry as the `samplePath`.

Result

pathCount	samplePathLocations	samplePathBridges
48	["Kneiphof", "North Bank", "Kneiphof", "South Bank", "Kneiphof", "Lomse", "North Bank"]	[1, 5, 6, 4, 7, 2]

Rows: 1



In the `samplePath` with a length of `6` returned by `DIFFERENT RELATIONSHIPS`, all bridges are traversed except for bridge `3`.

However, if the relationship count is increased to `7`, `0` paths are returned.

Find paths with a length of 7 relationships

```
MATCH p = (:Location {name: 'Kneiphof'})--{7}()
RETURN count(p) AS pathCount
```

Result

pathCount
0

Rows: 1

Using Cypher's default `MATCH` behavior, each step must use a unique bridge. After the first step, one bridge is used, leaving six remaining bridges to be used in the subsequent steps. After the sixth bridge is traversed, a seventh step would require re-traversing a bridge, which is not allowed.

This reflects the conclusion of Euler's Seven bridges of Königsberg problem: the impossibility of crossing each bridge exactly once in a single walk.

For more information about Cypher's default `MATCH` behavior, see Syntax & semantics → Match modes.

Non-default `MATCH` behavior

Cypher's default `MATCH` behavior is applicable to most use cases. It is, however, possible to alter how paths are traversed to solve specific problems, either by permitting re-traversal of relationships using `REPEATABLE ELEMENTS` or by not allowing the re-traversal of nodes in the same path with the `ACYCLIC` keyword.

For more information, see:

- Paths with repeated nodes and relationships
- Acyclic paths

Paths with repeated nodes and relationships

Introduced in Neo4j 2025.06 | Cypher 25 only

By default, Cypher does not allow relationships to be re-traversed in the same graph pattern match. This constraint can be bypassed using the `REPEATABLE ELEMENTS` keyword, which ensures there are no restrictions on how many times a node or relationship can occur for a given `MATCH` result.

The `REPEATABLE ELEMENTS` keyword must be specified after the `MATCH` clause.

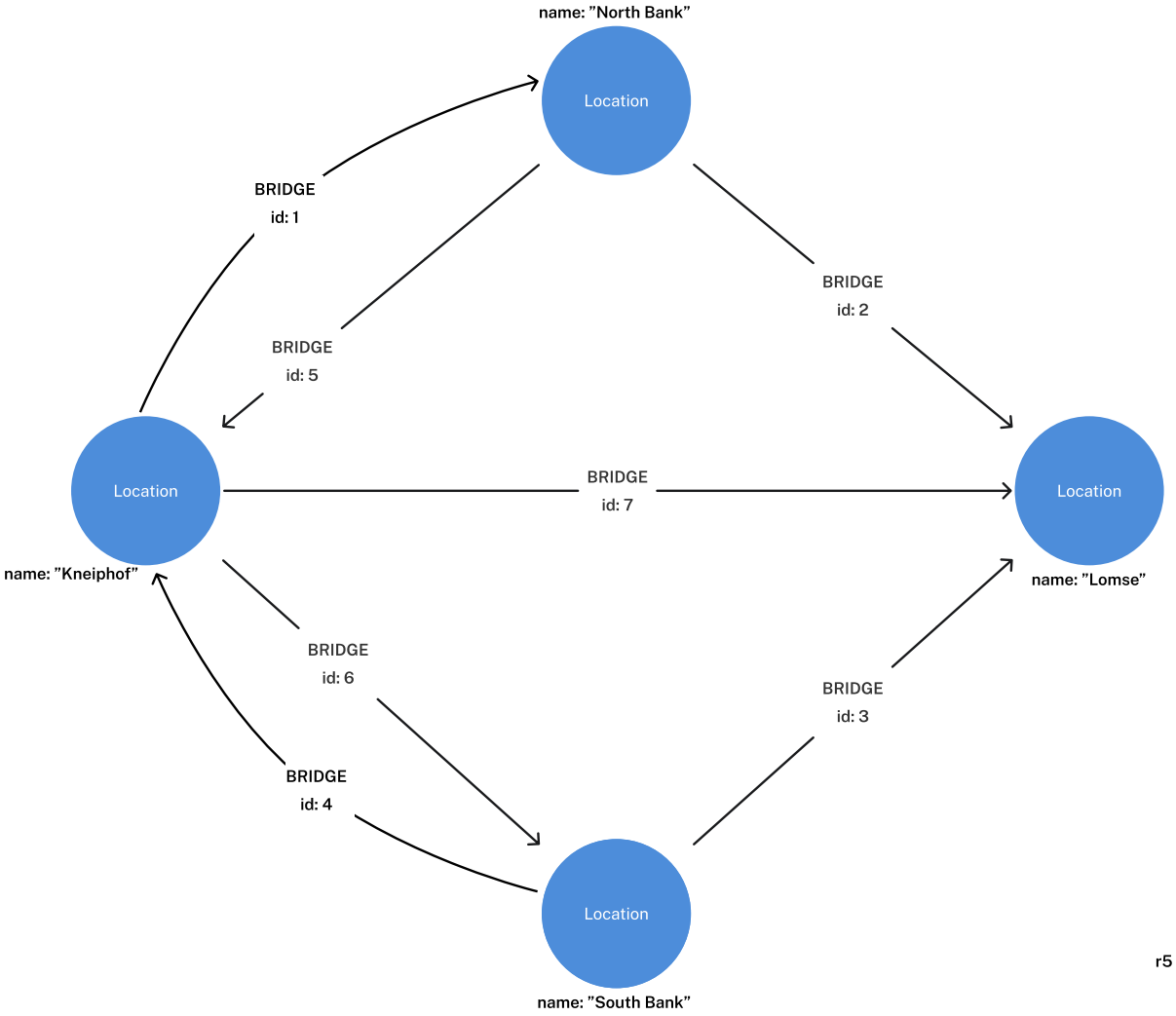
Query using `REPEATABLE ELEMENTS`

```
MATCH REPEATABLE ELEMENTS p = (start)--{,2}(end)
RETURN p
```

> `REPEATABLE ELEMENTS` is a GQL conformant match mode. Match modes determine whether relationships can appear more than once in a graph pattern match. For more information, see Syntax and semantics → Match modes.

Example graph

This page will use the scenario of the Seven bridges of Königsberg, a problem studied by Leonhard Euler in 1736 to determine if one could cross all seven bridges of Königsberg exactly once in a single walk.



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (kneiphof:Location {name: "Kneiphof"}),
       (northBank:Location {name: "North Bank"}),
       (southBank:Location {name: "South Bank"}),
       (lomse:Location {name: "Lomse"}),

       (kneiphof)-[:BRIDGE {id: 1}]->(northBank),
       (kneiphof)-[:BRIDGE {id: 6}]->(southBank),
       (kneiphof)-[:BRIDGE {id: 7}]->(lomse),
       (northBank)-[:BRIDGE {id: 5}]->(kneiphof),
       (northBank)-[:BRIDGE {id: 2}]->(lomse),
       (southBank)-[:BRIDGE {id: 4}]->(kneiphof),
       (southBank)-[:BRIDGE {id: 3}]->(lomse)
```

`REPEATABLE ELEMENTS` and the Königsberg bridges problem

The section on Cypher's default traversal rules and the Königsberg bridges problem showed the impossibility of crossing each of the seven bridges in Königsberg exactly once in a single walk.

`REPEATABLE ELEMENTS` does not solve this impossibility. However, the ability to re-traverse relationships does allow Cypher to return paths when testing Euler's hypothesis.

The following query matches the graph for paths with a length of `7` relationships using `REPEATABLE ELEMENTS` and returns a sample path.

Find a path with a length of 7 relationships using `REPEATABLE ELEMENTS`

```
MATCH REPEATABLE ELEMENTS p = (:Location {name: 'Kneiphof'})-[:BRIDGE]-{7}()
WITH collect(p)[0] AS samplePath
RETURN [n IN nodes(samplePath) | n.name] AS samplePathLocations,
       [r IN relationships(samplePath) | r.id] AS samplePathBridges
```

Result

samplePathLocations	samplePathBridges
["Kneiphof", "North Bank", "Kneiphof", "North Bank", "Kneiphof", "North Bank", "Kneiphof", "North Bank"]	[1, 5, 1, 5, 1, 5, 1]
Rows: 1	

While no paths with a length of `7` can be returned using Cypher's default `MATCH` behavior, the query now returns a `samplePath` that shows the same two bridges (`1` and `5`) being repeatedly re-traversed until the maximum path length is reached.

The next example matches paths starting and ending at the same `Location` (`Kneiphof`), where each bridge is crossed at least once, using `REPEATABLE ELEMENTS`:

Find a path traversing all bridges at least once before returning to the starting `Location` *using* `REPEATABLE ELEMENTS`

```
MATCH REPEATABLE ELEMENTS p = (start:Location {name: 'Kneiphof'})-[:BRIDGE]-{,9}(start)
WHERE all(bridge IN range(1,7) WHERE bridge IN [r IN relationships(p) | r.id])
RETURN [n IN nodes(p) | n.name] AS visitedLocations,
       [r IN relationships(p) | r.id] AS crossedBridges
ORDER BY length(p), crossedBridges
LIMIT 1
```

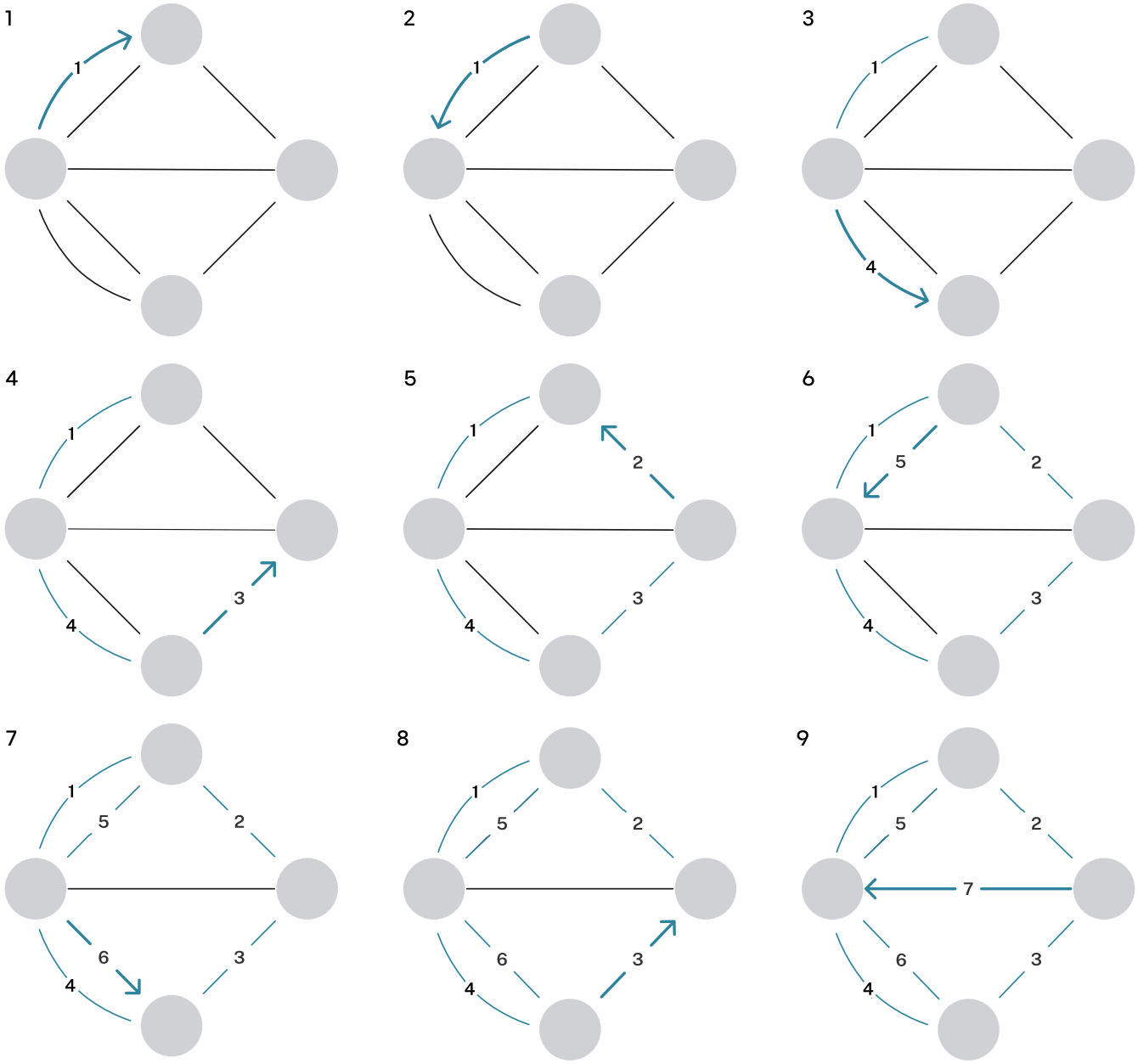
Paths with a length of less than 9 relationships return no results because they do not allow enough moves to traverse all seven bridges at least once while forming a cycle back to `Kneiphof`.

The `all()` function ensures that each bridge must be traversed once in the path.

Result

visitedLocations	crossedBridges
["Kneiphof", "North Bank", "Kneiphof", "South Bank", "Lomse", "North Bank", "Kneiphof", "South Bank", "Lomse", "Kneiphof"]	[1, 1, 4, 3, 2, 5, 6, 3, 7]
Rows: 1	

The order of the bridges traversed in the path returned demonstrates that bridges `1` and `3` were crossed twice in order to return to `Kneiphof`.



Sequence of bridge traversals when using `REPEATABLE ELEMENTS`.

For more information about `REPEATABLE ELEMENTS`, see Syntax and semantics → Repeatable relationship and node matching.

Bounded path length

When using Cypher's default `MATCH` behavior, the number of returned paths is limited because relationships cannot be re-traversed. However, because `REPEATABLE ELEMENTS` allows for re-traversing relationships, it enables full cycle re-traversals and paths that can double back on relationships. As a result, `REPEATABLE ELEMENTS` can generate a very large, or even infinite, number of valid paths.

The below query shows that there are `10583` valid paths with a length of `7` relationships starting from `Kneiphof` using `REPEATABLE ELEMENTS`:

Count the paths traversing all 7 bridges at least once using `REPEATABLE ELEMENTS`

```
MATCH REPEATABLE ELEMENTS p = (:Location {name: 'Kneiphof'})-[:BRIDGE]-{7}()
```

```
RETURN count(p) AS pathCount
```

Result

pathCount
10583
Rows: 1

If the path length is increased to exactly `10`, a total of `490047` paths are returned, `11` hops returns `1818159` paths, `12` hops returns `6498397` paths, and so on, without limit.

To ensure that a `MATCH` will return a finite number of solutions in a finite amount of time, unbounded quantifiers that do not impose an upper bound on a pattern, such as `*`, `+`, or `{1,}` are not allowed in combination with `REPEATABLE ELEMENTS`. Also, users should take into account that the number of results returned for a high number of repetitions can significantly impact both memory usage and the speed of the query.

Not allowed: find paths of an unbounded length using `REPEATABLE ELEMENTS`

```
MATCH REPEATABLE ELEMENTS p = (start:Location {name: 'Kneiphof'})-[:BRIDGE]-+(start)
WHERE all(bridge IN range(1,7) WHERE bridge IN [r IN relationships(p) | r.id])
RETURN count(p) AS pathCount
```

GQLSTATUS error chain

42N53: error: syntax error or access rule violation - unsafe usage of repeatable elements. The quantified path pattern may yield an infinite number of rows under match mode 'REPEATABLE ELEMENTS'. Add an upper bound to the quantified path pattern.

42001: error: syntax error or access rule violation - invalid syntax

Further reading

- Neo4j Developer Blog: I Would Walk 500 Miles
- Neo4j Developer Blog: Query, Chomp, Repeat

Acyclic paths

Introduced in Neo4j 2026.03 | Cypher 25 only

By default, Cypher only prevents relationships being repeated in a `MATCH` result, not nodes (see Paths with unique relationships for more information). The `ACYCLIC` keyword adds a further restriction by not allowing nodes to be repeated within a path, though they can still be repeated across paths enabling the formation of equijoins.

The `ACYCLIC` keyword must be specified before a path pattern:

Query using `ACYCLIC`

```
MATCH p = ACYCLIC (:Router {name: 'A'})-[:LINK]-+(:Router {name: 'Z'})
RETURN p
```

> `ACYCLIC` is a GQL conformant path mode. Path modes are levels of constraints that determine whether nodes or relationships can appear more than once in a path pattern match. Cypher also supports explicitly specifying the GQL path modes `TRAIL` and `WALK` though these do not alter the constraints already imposed by `DIFFERENT RELATIONSHIPS` or `REPEATABLE ELEMENTS` respectively. For more information, see Syntax and semantics → Path modes.

Example graph

To illustrate the practical application of `ACYCLIC` pruning, consider a router network used to model packet forwarding. In networking, identifying loop-free transmission routes is critical for avoiding broadcast storms and network congestion. The following graph contains several routing loops and bidirectional links between routers.

Figure 1. Router network graph

To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (a:Router {name: 'A'}), (b:Router {name: 'B'}), (c:Router {name: 'C'}),
  (d:Router {name: 'D'}), (e:Router {name: 'E'}), (f:Router {name: 'F'}),
  (g:Router {name: 'G'}), (h:Router {name: 'H'}), (i:Router {name: 'I'}),
  (j:Router {name: 'J'}), (k:Router {name: 'K'}), (z:Router {name: 'Z'}),

  (a)-[:LINK]->(b), (a)-[:LINK]->(c), (a)-[:LINK]->(d), (c)-[:LINK]->(b),
  (c)-[:LINK]->(d), (b)-[:LINK]->(e), (c)-[:LINK]->(e), (d)-[:LINK]->(f),
  (e)-[:LINK]->(g), (f)-[:LINK]->(g), (h)-[:LINK]->(i), (h)-[:LINK]->(j),
  (i)-[:LINK]->(j), (i)-[:LINK]->(z), (g)-[:LINK]->(j), (g)-[:LINK]->(k),
  (j)-[:LINK]->(k), (j)-[:LINK]->(z), (k)-[:LINK]->(z)
```

Relationship uniqueness in packet forwarding

In many graph contexts, the default relationship uniqueness mode of Cypher is sufficient. However, in packet forwarding, a packet should not visit the same router twice, even if it uses a different link. Relationship uniqueness only ensures that no relationship is reused — it does not prevent nodes from being revisited, meaning cycles are still possible. In the router network graph, for example, there are paths where no relationships are repeated but nodes are revisited, such as the cycles involving Router `B`, `C`, and `D`.

For example, the following query uses Cypher's default behavior, which disallows repeated relationships but allows repeated nodes:

Find cyclic paths (with repeated routers) from 'A' to 'Z', using inline predicates to exclude start and target nodes

```
MATCH p = (:Router {name: 'A'})
          ((l WHERE l.name <> 'Z')-[:LINK]-(m WHERE m.name <> 'A'))+
          (:Router {name: 'Z'})
WHERE size(nodes(p)) > COUNT { UNWIND nodes(p) AS n RETURN DISTINCT n } AND length(p) = 8
RETURN [n IN nodes(p) | n.name] AS routeWithCycles
ORDER BY length(p), p LIMIT 1
```

A quantified path pattern with inline predicates excluding paths that either return to the start router or pass through the target router.

Filter for paths that have cycles, where the number of nodes in the path is greater than the number of distinct nodes in the path.

List comprehension converting the sequence of nodes to a compact list of their names.

Result

routeWithCycles
["A", "B", "E", "G", "J", "I", "H", "J", "Z"]
Rows: 1

Notice how the `J` node is visited twice.

`ACYCLIC` forwarding paths

To ensure that no node is repeated, or, in this case, prevent packet forwarding that involves revisiting the same routers, use the `ACYCLIC` keyword.

```
MATCH p = ACYCLIC (:Router {name: 'A'})-[:LINK]-+(:Router {name: 'Z'})
RETURN [n IN nodes(p) | n.name] AS routeWithoutCycles
ORDER BY p LIMIT 1
```

Note that, with `ACYCLIC`, the predicates to exclude visiting the start or target nodes are redundant.

Result

routeWithoutCycles
["A", "B", "E", "G", "J", "I", "Z"]

routeWithoutCycles
Rows: 1

You can also use `ACYCLIC` to examine the number of possible routes from the start to the target router that pass through each of the intermediate routers. By grouping the paths based on the intermediate routers and calculating the percentage of total paths that include each router, it is possible to determine the network's sensitivity to a failure at a particular router.

Determining the percentage of acyclic paths from 'A' to 'Z' that pass through each intermediate router

```
LET middleNodesOfPaths = COLLECT {
  MATCH p = ACYCLIC (start:Router {name: 'A'})-[:LINK]-+(end:Router {name: 'Z'})
  RETURN nodes(p)[1..-1]
}
LET numPaths = size(middleNodesOfPaths)
UNWIND coll.flatten(middleNodesOfPaths) AS node
WITH node.name AS node, count(*) AS numPathsWithNode, numPaths
RETURN node, round(100.0 * numPathsWithNode / numPaths, 1) AS `% of paths with node`
ORDER BY numPathsWithNode DESC
```

Collects all the `ACYCLIC` paths to count the total number of paths from 'A' to 'Z'.

List slicing is used to remove the first and last node from each path.

The number of lists of middle nodes equals the number of paths.

The `coll.flatten()` function unnests `middleNodesOfPaths` (a list of lists) into a single list of nodes.

As the paths are `ACYCLIC`, counting the number of times a particular node appears in the list of lists tells you in how many paths that node appears.

Result

"node"	"% of paths with node"
"G"	100.0
"J"	87.5
"C"	80.0
"E"	70.0
"K"	62.5
"D"	60.0
"B"	60.0
"I"	50.0
"F"	40.0
"H"	25.0
Rows: 10	

This shows that router `G` is the weakest point in the network, with 100% of traffic from `A` to `Z` passing through it.

Performance improvement with `ACYCLIC`

Using `ACYCLIC` is significantly more efficient than using manual post-filters for node uniqueness.

Prior to the release of the `ACYCLIC`, Cypher would return paths satisfying the pattern, including those with nodes repeated. Only after the generation of these paths could a `WHERE` filter discard those containing cycles. For the below query, the following is true:

- **Evaluation:** The engine generates every possible path, including those that loop infinitely or visit the same nodes multiple times.
- **Overhead:** The engine must compute and hold a massive set of cyclic solutions in memory before the filter can discard them.

Using `WHERE` to discard paths containing cycles

```
MATCH p = (start:User {id: 1})-[:FRIEND]->+(target:User)
  WHERE size(nodes(p)) = COUNT { UNWIND nodes(p) AS n RETURN DISTINCT n }
RETURN p
```

However, `ACYCLIC` applies the uniqueness constraint during evaluation. This produces a significantly more efficient query. More specifically:

- **Evaluation:** As the engine traverses the graph, it monitors the nodes already present in the path.
- **Pruning:** If the engine attempts to move to a node that has already been visited in the current path, it prunes that specific candidate solution immediately.
- **Efficiency:** Because invalid paths are never fully generated or held in memory, execution is faster and uses fewer resources.

Applying uniqueness constraints during query evaluation with `ACYCLIC`

```
MATCH p = ACYCLIC (start:User {id: 1})-[:FRIEND]->+(target:User)
RETURN p
```

Syntax and semantics

Node and relationship patterns

Node patterns

A node pattern is a pattern that matches a single node. It can be used on its own in a clause such as `MATCH` or `EXIST`, or form part of a path pattern.

See also node pattern concepts.

Syntax

```
nodePattern ::= "(" [ nodeVariable ] [ labelExpression ]
    [ propertyKeyValueExpression ] [ elementPatternWhereClause] ")"

elementPatternWhereClause ::= "WHERE" booleanExpression
```

For rules on valid node variable names, see the Cypher naming rules.

Rules

Predicates

Three types of predicate can be specified inside a node pattern:

- label expressions
- property key-value expressions
- WHERE clauses

The boolean expression of the `WHERE` clause can reference any variables within scope of the node pattern. A node variable needs to be declared in the node pattern in order to reference it in the boolean expression.

If no predicates are specified, then the node pattern matches any node.

Variable binding

If a variable has not been declared elsewhere in the query, it will become bound to nodes when the matching of its containing path pattern is executed. If it has been bound in a previous clause, then no new nodes will be bound to the variable; any previously bound nodes that do not match in the current path pattern will lead to the match being eliminated from the results. See the section on clause composition for more details on the passing of results between clauses.

Examples

Matches all nodes with the label `A` and binds them to the variable `n`:

```
(n:A)
```

Matches all nodes with the label `B` and a property `departs` with the `time` value `11:11`:

```
(:B { departs: time('11:11') })
```

Matches all nodes with the property `departs` with a value equal to the current time plus 30 minutes:

```
(n WHERE n.departs > time() + duration('PT30M'))
```

Relationship patterns

A relationship pattern is a pattern that matches a single relationship. It can only be used with node patterns on either side of it.

A relationship pattern followed immediately by a quantifier is an abbreviated quantified path pattern called a quantified relationship.

See also relationship pattern concepts.

Syntax

```
relationshipPattern ::= fullPattern | abbreviatedRelationship

fullPattern ::=
    "<-[" patternFiller "]-"
  | "-[" patternFiller "]->"
  | "-[" patternFiller "]-"

abbreviatedRelationship ::= "<--" | "--" | "-->"

patternFiller ::= [ relationshipVariable ] [ typeExpression ]
    [ propertyKeyValueExpression ] [ elementPatternWhereClause ]

elementPatternWhereClause ::= "WHERE" booleanExpression
```

Note that the syntax for type expressions in relationship patterns is the same as for label expressions in node patterns (although unlike node labels, relationships must have exactly one type).

For rules on valid relationship variable names, see the Cypher naming rules.

Rules

Predicates

The following three types of predicate can be specified in the pattern filler of a full relationship pattern (i.e. a pattern with the square brackets):

- label expressions
- property key-value expressions
- WHERE clauses

A fourth type of predicate specifies the directionality of the relationship with respect to the overall path pattern, using the less-than or greater-than symbols to form arrows (`<` and `>`). If a relationship pattern has

no arrows, it will match relationships of any direction.

The boolean expression of the `WHERE` clause can reference any variables within scope of the relationship pattern. A relationship variable needs to be declared in the pattern in order to reference it in the boolean expression.

If no predicates are specified then the pattern matches all relationships.

Variable binding

If the variable has not been declared elsewhere in the query, it will become bound to relationships when the matching of its containing path pattern is executed. If it has been bound in a previous clause, then no new relationships will be bound to the variable; if any previously bound relationships do not match in the current path pattern, then those matches will be eliminated from the results.

See the chapter on clause composition for more details on the passing of results between clauses.

Examples

Matches all relationships with the type `R` and binds them to the variable `r`:

```
()-[r:R]->()
```

Matches all relationships with type `R` and property `distance` equal to `100`:

```
()-[:R {distance: 100}]->()
```

Matches all relationships where property `distance` is between `10` and `100`:

```
()-[r WHERE 10 < r.distance < 100]->()
```

Matches all relationships that connect nodes with label `A` as their source and nodes with label `B` as their target:

```
(:A)-->(:B)
```

Matches all relationships that connect nodes with label `A` and nodes with label `B`, irrespective of their direction:

```
(:A)--(:B)
```

Label expressions

The following applies to both the label expressions of node patterns and the type expressions of relationship patterns.

A label expression is a boolean predicate composed from label names and a wildcard symbol using disjunction, conjunction, negation and grouping. A label expression returns true when it matches the set of

labels for a node.

The same grammar appears in label expression predicates in expression context: the colon form `element:labelExpression`, and as of Neo4j 2026.04, `element IS [NOT] LABELED labelExpression` (see `IS [NOT] LABELED`]).

Although relationships have a type rather than labels, the syntax for expressions matching a relationship type is identical to that of label expressions.

Syntax

```
labelExpression ::=
    ":" labelTerm
  | "IS" [ "NOT" ] "LABELED" labelTerm
  | "IS NOT" labelTerm

labelTerm ::=
    labelIdentifier
  | labelTerm "&" labelTerm
  | labelTerm "|" labelTerm
  | "!" labelTerm
  | "%"
  | "(" labelTerm ")"
```

For valid label identifiers, see the Cypher naming rules.

Rules

The following table lists the symbols used in label expressions:

Symbol	Description	Precedence
`%`	Wildcard. Evaluates to `true` if the label set is non-empty	
`()`	Contained expression is evaluated before evaluating the outer expression the group is contained in.	1 (highest)
`!`	Negation	2
`&`	Conjunction	3
`|`	Disjunction	4 (lowest)

Associativity is left-to-right.

Examples

In the following table, a tick is shown where the label expression matches the node with the labels shown:

	Node							
Node pattern	`()`	`(:A)`	`(:B)`	`(:C)`	`(:A:B)`	`(:A:C)`	`(:B:C)`	`(:A:B:C)`
`()`	✔	✔	✔	✔	✔	✔	✔	✔

`(:A)`		✓			✓	✓		✓
`(:A&B)`					✓			✓
`(:A|B)`		✓	✓		✓	✓	✓	✓
`(:!A)`	✓		✓	✓			✓	
`(:!!A)`		✓			✓	✓		✓
`(:A&!A)`								
`(:A|!A)`	✓	✓	✓	✓	✓	✓	✓	✓
`(:%)`		✓	✓	✓	✓	✓	✓	✓
`(:!%)`	✓							
`(:%|!%)`	✓	✓	✓	✓	✓	✓	✓	✓
`(:%&!%)`								
`(:A&%)`		✓			✓	✓		✓
`(:A|%)`		✓	✓	✓	✓	✓	✓	✓
`(:(A&B)&!(B&C))`					✓			
`(:!A&%)`			✓	✓			✓	

As relationships have exactly one type each, this expression will never match a relationship:

```
-[:A&B]->
```

Similarly, the following will always match a relationship:

```
-[:%]->
```

The use of negation can make the conjunction useful in relationship patterns. The following matches relationships that have a type that is neither `A` nor `B`:

```
-[:!A&!B]->
```

Dynamic label and type expressions

Node labels and relationship types can be referenced dynamically in expressions, parameters, and variables within label expressions.

Syntax for creating nodes and relationships dynamically

```
dynamicLabelExpression ::= ":$(" + valueExpression ")"
```

The `valueExpression` must evaluate to a `STRING NOT NULL | LIST<STRING NOT NULL> NOT NULL` value.

Property key-value expressions

Syntax

```
propertyKeyValueExpression ::=
  "{" propertyKeyValuePairList "}"

propertyKeyValuePairList ::=
  propertyKeyValuePair [ "," propertyKeyValuePair ]

propertyKeyValuePair ::= propertyName ":" valueExpression
```

Rules

The property key-value expression is treated as a conjunction of equalities on the properties of the element that the containing pattern matches.

For example, the following node pattern:

```
({ p: valueExp1, q: valueExp2 })
```

is equivalent to the following node pattern with a `WHERE` clause:

```
(n WHERE n.p = valueExp1 AND n.q = valueExp2)
```

The value expression can be any expression as listed in the section on expressions, except for path patterns (which will throw a syntax error) and regular expressions (which will be treated as string literals). An empty property key-value expression matches all elements. Property key-value expressions can be combined with a `WHERE` clause.

Examples

Matches all nodes with property `p` = `10`:

```
({ p: 10 })
```

Matches all relationships with property `p` = `10` and `q` equal to date `2023-02-10`:

```
()-[{ p: 10, q: date('2023-02-10') }]-()
```

Matches all relationships with its property `p` equal to the property `p` of its source node:

```
(s)-[{ p: s.p }]-()
```

Matches all nodes with property `p` = `10` and property `q` greater than `100`:

```
(n { p: 10 } WHERE n.q > 100)
```

Path patterns and graph patterns

Path patterns

A path pattern is the top level pattern that is matched against paths in a graph.

Syntax

```
pathPattern ::= [ pathVariableDeclaration ] [ pathPatternPrefix ] pathPatternExpression

pathVariableDeclaration ::= pathVariable "="

pathPatternPrefix ::= pathModePrefix | shortestPathSelector

pathPatternExpression ::= { parenthesizedPathPatternExpression | pathPatternPhrase }

parenthesizedPathPatternExpression ::=
  "("
    [ subpathVariableDeclaration ]
    pathPatternExpression
    [ parenthesizedPathPatternWhereClause ]
")"

subpathVariableDeclaration ::= pathVariable "="

pathPatternPhrase ::= [{ simplePathPattern | quantifiedPathPattern }]+

simplePathPattern ::= nodePattern
  [ { relationshipPattern | quantifiedRelationship } nodePattern ]*

parenthesizedPathPatternWhereClause ::= "WHERE" booleanExpression
```

See the related section for each of the syntax rules:

`shortestPathSelector`	Shortest paths
`quantifiedPathPattern`	Quantified path patterns
`nodePattern`	Node patterns
`relationshipPattern`	Relationship patterns
`quantifiedRelationship`	Quantified relationships

Rules

The minimum number of elements in the path pattern must be greater than zero. For example, a path pattern that is a quantified path pattern with a quantifier that has a lower bound of zero is not allowed:

Not allowed

```
((n)-[r]->(m)){0,10}
```

A path pattern must always begin and end with a node pattern.

Not allowed

```
(n)-[r]->(m)-[s]-
```

A path pattern may be composed of a concatenation of simple and quantified path patterns. Two simple path patterns, however, may not be placed next to each other.

Not allowed

```
(a)<-[s]-(b) (c)-[t]->(d)
```

When a path pattern is matched to paths in a graph, nodes can be revisited but relationships cannot.

A subpath variable (a path variable declared inside a parenthesized path pattern expression) can only be used if a shortest paths mode is also specified.

Allowed

```
MATCH SHORTEST 1 (p = (a)-[r]->+(b) WHERE length(p) > 3)
```

Not allowed

```
MATCH (:A)-[:S]->(:B) (p = ((a)-[r]->(b))+)
```

See graph patterns for rules on declaring variables multiple times.

Examples

A single node pattern is allowed as it has at least one element:

```
(n)
```

A simple path pattern with more than one element:

```
(a:A)<-[{p: 30}]-(b)-[t WHERE t.q > 0]->(c:C)
```

A quantified path pattern can have a lower bound of zero in its quantifier as long as it abuts other patterns that have at least one element:

```
(:A) ((:X)-[:R]-()){0,10} (:B)
```

A quantified relationship can also have a lower bound of zero as long as the overall path pattern has at least one element:

```
(:A)-[:R]->{0,10}(:B)
```

A concatenation of simple and quantified path patterns:

```
(a)<-[s]-(b)-[t]->(c) ((n)-[r]->(m)){0,10} (:X)
```

Referencing non-local node variable in a simple path pattern:

```
(a)<-[s:X WHERE a.p = s.p]-(b)
```

Referencing a non-local relationship variable within a quantified path pattern:

```
(:A) ((a)<-[s:X WHERE a.p = s.p]-(b)){,5}
```

A variable that was introduced in a previous clause can be referenced as long as that variable was defined outside of a quantified path pattern:

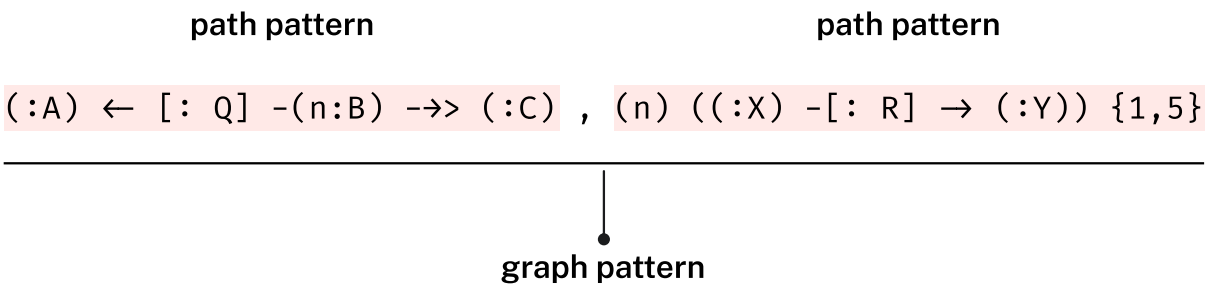
```
MATCH (n)
MATCH ()-[r WHERE r.q = n.q]-() (()<-[s:X WHERE n.p = s.p]-()){2,3}
```

Paths matched by the path pattern can be assigned to a variable:

```
MATCH p = ()-[r WHERE r.q = n.q]-()
```

Graph patterns

A graph pattern is a comma separated list of one or more path patterns. It is the top level construct provided to `MATCH`. Whether or not a graph pattern can match a relationship more than once is governed by the selected match mode.



Syntax

```
graphPattern ::=
  pathPattern [ "," pathPattern ]* [ graphPatternWhereClause ]

graphPatternWhereClause ::= "WHERE" booleanExpression
```

Rules

The rules for path patterns apply to each constituent path pattern of a graph pattern.

Variable references

If a variable is declared inside a quantified path pattern, then it can be treated as a singleton only from within the quantified path pattern it was declared in. Outside of that quantified path pattern, it must be treated as a group variable.

Allowed

```
((n)-[r]->(m WHERE r.p = m.q))+
```

Allowed

```
(n)-[r]->+(m WHERE all(rel in r WHERE rel.q > m.q))
```

Not allowed

```
(n)-[r]->+(m WHERE r.p = m.q)
```

Equijoin

If a node variable is declared more than once in a path pattern, it is expressing an equijoin. This is an operation that requires that each node pattern with the same node variable be bound to the same node. For example, the following pattern refers to the same node twice with the variable `a`, forming a cycle:

```
(a)-->(b)-->(c)-->(a)
```

The following pattern refers to the same node with variable `b` in different path patterns of the same graph pattern, forming a "T" shaped pattern:

```
(a)-->(b)-->(c), (b)-->(e)
```

Equijoins can only be made using variables outside of quantified path patterns. The following would not be a valid equijoin:

Not allowed

```
(a)-->(b)-->(c), ((b)-->(e))+ (:X)
```

If no equijoin exists between path patterns in a graph pattern, then a Cartesian join is formed from the sets of matches for each path pattern. An equijoin can be expressed between relationship patterns by declaring a relationship variable multiple times. However, as relationships can only be traversed once in a given match, no solutions would be returned.

Examples

The `WHERE` clause can refer to variables inside and outside of quantified path patterns:

```
(a)-->(b)-->(c), (b) ((d)-->(e))+ WHERE any(n in d WHERE n.p = a.p)
```

An equijoin can be formed to match "H" shaped graphs:

```
(:A)-->(x)--(:B), (x)-[:R]->+(y), (:C)-->(y)-->(:D)
```

With no variables in common, this graph pattern will result in a Cartesian join between the sets of matches

for the two path patterns:

```
(a)-->(b)-->(c), ((d)-->(e))+
```

Multiple equijoins can be formed between path patterns:

```
(:X)-->(a:A)-[!:R]->+(b:B)-->(:Y), (a)-[:R]->+(b)
```

Variables declared in a previous `MATCH` can be referenced inside of a quantified path pattern:

```
MATCH (n {p = 'ABC'})
MATCH (n)-->(m:A)-->(:B), (m) (()-[r WHERE r.p <> n.p]->())+ (:C)
```

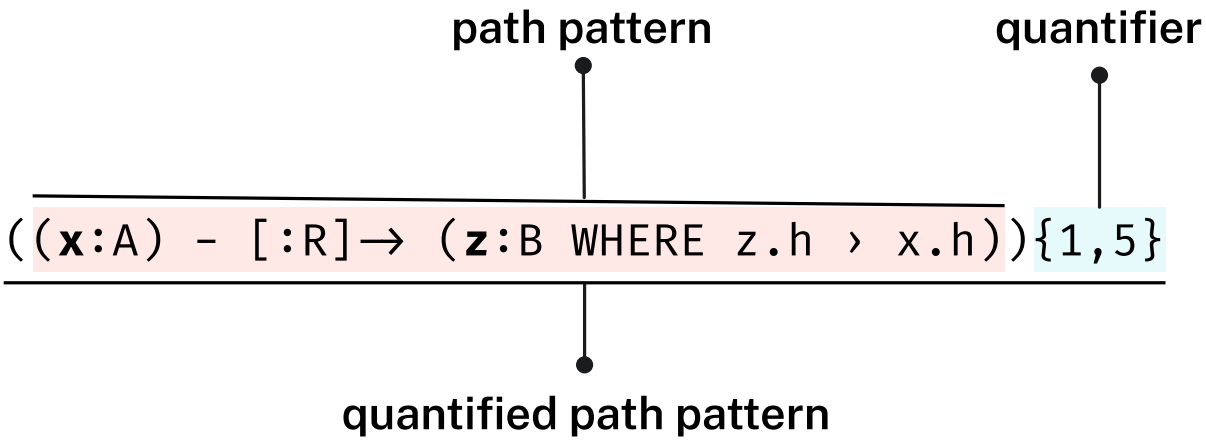
The repetition of a relationship variable in the following yields no solutions due to Cypher enforcing relationship uniqueness within a match for a graph pattern:

```
MATCH ()-[r]->()-->(), ()-[r]-()
```

Variable length paths

Quantified path patterns

A quantified path pattern represents a path pattern repeated a number of times in a given range. It is composed of a path pattern, representing the path section to be repeated, followed by a quantifier, constraining the number of repetitions between a lower bound and an upper bound.



For information about an alternative version of patterns for matching paths of variable length, see variable-length relationships.

Syntax

```
quantifiedPathPattern ::=
  parenthesizedPathPatternExpression quantifier

fixedPath ::= nodePattern [ relationshipPattern nodePattern ]+
```

Rules

Minimum pattern length

The path pattern being quantified must have a length greater than zero. In other words, it must contain at least one relationship. A single node pattern cannot be quantified.

Not allowed

```
((x:A)){2,4}
```

Nesting of quantified path patterns

The nesting of quantified path patterns is not allowed. For example, the following nesting of a quantified relationship in a quantified path pattern is not allowed:

Not allowed

```
(:A) (()-[:R]->+()){2,3} (:B)
```

A quantified path pattern that is part of the boolean expression within a quantified path pattern would not count as nested and is permitted.

Allowed

```
MATCH ((n:A)-[:R]->({p: 30}) WHERE EXISTS { (n)-->+(:X) }){2,3}
```

Group variables

Variables introduced inside of a quantified path pattern are said to be exposed as *group variables* outside of the definition of the pattern. As a group variable, they will be bound to either a list of nodes or a list of relationships. By contrast, variables can be treated as singletons inside the quantified path pattern where they are declared. The difference can be seen in the following query:

```
MATCH ((x)-[r]->(z WHERE z.p > x.p)){2,3}
RETURN [n in x | n.p] AS x_p
```

In the boolean expression `z.p > x.p` both `z` and `x` are singletons; in the `RETURN` clause, `x` is a group variable that can be iterated over like a list. Note that this means that the `WHERE` clause `z.p > x.p` above needs to be inside the quantified path pattern. The following would throw a syntax error because it is treating `z` and `p` as singletons:

Not allowed

```
MATCH ((x)-[r]->(z)){2,3} WHERE z.p > x.p
```

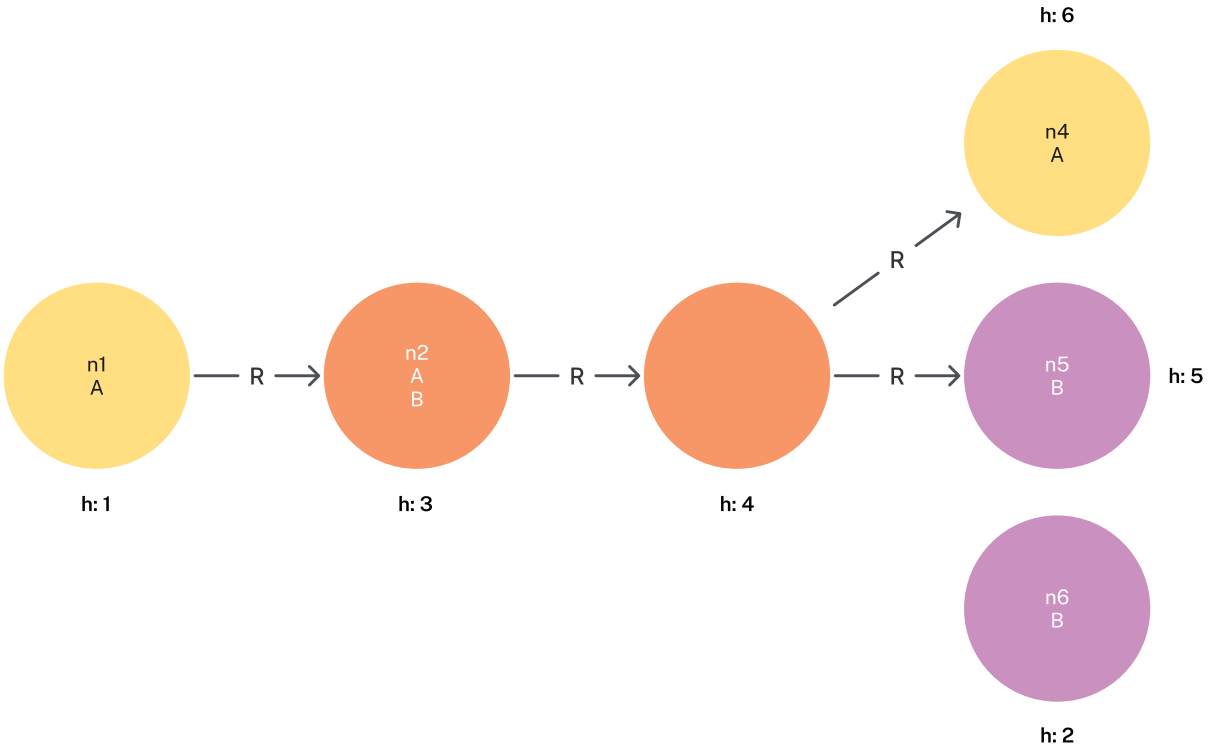
It is possible, however, to position the `WHERE` clause outside of the node pattern:

Allowed

```
MATCH ((x)-[r]->(z) WHERE z.p > x.p){2,3}
```

Matching

The mechanics of matching a quantified path pattern against paths is best explained with an example. For the example, the following simple graph will be used:



First, consider the following simple path pattern:

```
(x:A)-[:R]->(z:B WHERE z.h > 2)
```

This matches three different paths in the graph above. The resulting bindings for `x` and `z` for each match are the following (the captions `n1` etc indicate the identity of the nodes in the diagram above):

x	z
n1	n2
n2	n3
n3	n5

If the quantifier `{2}` is affixed to the simple path pattern, the result is the following quantified path pattern:

```
((x:A)-[:R]->(z:B WHERE z.h > 2)){2}
```

This is equivalent to chaining together two iterations of the pattern, where the rightmost node of the first iteration is merged with the leftmost node of the second one. (See node pattern pairs for more details.)

```
(x:A)-[:R]->(z:B WHERE z.h > 2) (x:A)-[:R]->(z:B WHERE z.h > 2)
```

To avoid introducing equijoins between the two instances of `x`, and between the two instances of `z`, the variables are replaced with a set of fresh variables inside each iteration:

```
(x1:A)-[:R]->(z1:B WHERE z1.h > 2) (x2:A)-[:R]->(z2:B WHERE z2.h > 2)
```

Then the node variables in adjoining node patterns are merged:

```
(x1:A)-[:R]->({z1,x2}:A&B WHERE z1.h > 2)-[:R]->(z2:B WHERE z2.h > 2)
```

The fact that variables `x2` and `z1` are bound to matches of the same node pattern is represented with the notation `{z1,x2}`. Outside of the pattern, the variables `x` and `z` will be group variables that contain lists of nodes.

Consider the quantified path pattern in the following query:

```
MATCH ((x:A)-[:R]->(z:B WHERE z.h > 2)){2}
RETURN [n in x | n.h] AS x_h, [n in z | n.h] AS z_h
```

This yields the following results:

x_h	z_h
[1, 3]	[3, 4]
[3, 4]	[4, 5]

Now the quantifier is changed to match lengths from one to five:

```
MATCH ((x:A)-[:R]->(z:B WHERE z.h > 2)){1,5}
RETURN [n in x | n.h] AS x_h, [n in z | n.h] AS z_h
```

Compared to the fixed-length quantifier `{2}`, this also matches paths of length one and three, but no matches exist for length greater than three:

x_h	z_h
[1]	[3]
[3]	[4]
[4]	[5]
[1, 3]	[3, 4]
[3, 4]	[4, 5]
[1, 3, 4]	[3, 4, 5]

Quantified relationships

Syntax

```
quantifiedRelationship ::= relationshipPattern quantifier
```

Rules

A quantified relationship is an abbreviated form of a quantified path pattern, with only a single relationship pattern specified.

For example, the following quantified relationship:

```
(:A)-[r:R]->{m,n}(:B)
```

It matches paths with between `m` and `n` relationships of type `R` pointing left-to-right, starting at a node of type `A`, ending at a node of type `B`, and matching any nodes in-between. It is equivalent to the following quantified path pattern:

```
(:A) (()-[r:R]->()){m,n} (:B)
```

With the expanded form of a quantified path pattern, it is possible to add predicates inside. Conversely, if the only predicate is a relationship type expression, query syntax can be more concise using a quantified relationship.

Note that, unlike a quantified path pattern, a quantified relationship must always have a node pattern on each side.

Examples

Matches one or more relationships with type `R` and of any direction, and any nodes connecting those relationships:

```
()-[:R]-+()
```

Matches paths consisting of two inbound subpaths, one with relationships of type `R` and one with relationships of type `S`, meeting at a node with label `A`:

```
()-[:R]->+(:A)<-[:S]-+()
```

Matches paths with any nodes and with one or more relationships of any direction and any type:

```
()--+()
```

Matches paths starting with nodes labeled `A` and ending with nodes labeled `B`, that traverse between two and three relationships of type `R`, matching any intermediate nodes:

```
(:A)-[r:R]->{2,3}(:B)
```

Quantifiers

The quantifiers here only refer to those used in quantified path patterns and quantified relationships.

Syntax

```
quantifier ::=
  "*" | "+" | fixedQuantifier | generalQuantifier

fixedQuantifier ::= "{" unsignedDecimalInteger "}"

generalQuantifier ::= "{" lowerBound "," upperBound "}"

lowerBound ::= unsignedDecimalInteger

upperBound ::= unsignedDecimalInteger

unsignedDecimalInteger ::= [0-9]+
```

Note:

The `unsignedDecimalInteger` must not be larger than the Java constant `Long.MAX_VALUE` (2^{63} -1).

Rules

The absence of an upper bound in the general quantifier syntax means there is no upper bound. The following table shows variants of the quantifier syntax and their canonical form:

Variant	Canonical form	Description
`{m,n}`	`{m,n}`	Between m and n iterations.
`+`	`{1,}`	1 or more iterations.
`*`	`{0,}`	0 or more iterations.
`{n}`	`{n,n}`	Exactly n iterations.
`{m,}`	`{m,}`	m or more iterations.
`{,n}`	`{0,n}`	Between 0 and n iterations.
`{,}`	`{0,}`	0 or more iterations.

Note that a quantified path pattern with the quantifier `{1}` is not equivalent to a fixed-length path pattern. Although the resulting quantified path pattern will match on the same paths the fixed-length path contained in it would without the quantifier, the presence of the quantifier means that all variables within the path pattern will be exposed as group variables.

Variable-length relationships

Prior to the introduction of the syntax for quantified path patterns and quantified relationships, the only way in Cypher to match paths of variable length was with a variable-length relationship. This syntax is still available, but it is not GQL conformant. It is equivalent to the syntax for quantified relationships, with the

following differences:

- Position and syntax of quantifier.
- Semantics of the asterisk symbol.
- Type expressions are limited to the disjunction operator.
- The WHERE clause is not allowed.

Syntax

```
varLengthRelationship ::=
    "<-[" varLengthFiller "]-"
  | "-[" varLengthFiller "]->"
  | "-[" varLengthFiller "]-"

varLengthFiller ::= [ relationshipVariable ] [ varLengthTypeExpression ]
    [ varLengthQuantifier ] [ propertyKeyValueExpression ]

varLengthTypeExpression ::= ":" varLengthTypeTerm

varLengthTypeTerm ::=
    typeIdentifier
  | varLengthTypeTerm "|" varLengthTypeTerm

varLengthQuantifier ::= varLengthVariable | varLengthFixed

varLengthVariable ::= "*" [ [ lowerBound ] ".." [ upperBound ] ]

varLengthFixed ::= "*" fixedBound

fixedBound ::= unsignedDecimalInteger

lowerBound ::= unsignedDecimalInteger

upperBound ::= unsignedDecimalInteger

unsignedDecimalInteger ::= [0-9]+
```

Note:

The `unsignedDecimalInteger` must not be larger than the Java constant `Long.MAX_VALUE` (2^{63} -1).

For rules on valid relationship variable names, see Cypher naming rules.

Rules

The following table shows variants of the variable-length quantifier syntax and their equivalent quantifier form (the form used by quantified path patterns):

Variant	Description	Quantified relationship equivalent	Quantified path pattern equivalent
`-[*]->`	1 or more iterations.	`-[]->+`	`(()-[]->())+`
`-[*n]->`	Exactly n iterations.	`-[]->{n}`	`(()-[]->()){n}`
`-[*m..n]->`	Between m and n iterations.	`-[]->{m,n}`	`(()-[]->()){m,n}`

Variant	Description	Quantified relationship equivalent	Quantified path pattern equivalent
`-[*m..]->`	m or more iterations.	`-[]->{m,}`	`(()-[]->()){m,}`
`-[*0..]->`	0 or more iterations.	`-[]->*`	`(()-[]->())*`
`-[*..n]->`	Between 1 and n iterations.	`-[]->{1,n}`	`(()-[]->()){1,n}`
`-[*0..n]->`	Between 0 and n iterations.	`-[]->{,n}`	`(()-[]->()){,n}`

Note that `*` used here on its own is not the same as the Kleene star (an operator that represents zero or more repetitions), as the Kleene star has a lower bound of zero. The above table can be used to translate the quantifier used in variable-length relationships. The rules given for quantified path patterns would apply to the translation.

This table shows some examples:

Variable-length relationship	Equivalent quantified path pattern
`(a)-[*2]->(b)`	`(a) (()-[]->()){2,2} (b)`
`(a)-[:KNOWS*3..5]->(b)`	`(a) (()-[:KNOWS]->()){3,5} (b)`
`(a)-[r*..5 {name: 'Filipa'}]->(b)`	`(a) (()-[r {name: 'Filipa'}]->()){1,5} (b)`

Equijoins

The variable of a variable-length relationship can be used in subsequent patterns to refer to the list of relationships the variable is bound to. This is the same as the equijoin for variables bound to single nodes or relationships.

This section uses the following graph:

To recreate the graph, run the following query against an empty Neo4j database:

Query

```
CREATE ({name: 'Filipa'})-[:KNOWS]->({name:'Anders'})-[:KNOWS]->
        ({name:'Dilshad'})
```

In the following query, the node variables will be bound to the same nodes:

Query

```
MATCH (a {name: 'Dilshad'})<-[r*1..2]-(b)
MATCH (c)<-[r*1..2]-(d)
```

```
RETURN a = c, b = d, size(r)
```

Result

a = c	b = d	size(r)
true	true	1
true	true	2
Rows: 2		

The list of relationships keeps its order. This means that in the following query, where the direction of the variable-length relationship in the second `MATCH` is switched, the equijoin will only match once, when there is a single relationship:

Query

```
MATCH (a {name: 'Dilshad'})<-[r*1..2]-(b)
MATCH (c)-[r*1..2]->(d)
RETURN a = c, b = d, size(r)
```

Result

a = c	b = d	size(r)
false	false	1
Rows: 1		

The variable `r` can be reversed in order like any list, and made to match the switch in relationship pattern direction:

Query

```
MATCH (a {name: 'Dilshad'})<-[r*1..2]-(b)
WITH a, b, reverse(r) AS s
MATCH (c)-[s*1..2]->(d)
RETURN a = d, b = c, size(s)
```

Result

a = d	b = c	size()
true	true	1
true	true	2
Rows: 2		

Changing the bounds on subsequent `MATCH` statements will mean that only the overlapping lengths of the quantifier bounds will produce results:

Query

```
MATCH (a {name: 'Dilshad'})<-[r*1..2]-(b)
MATCH (c)<-[r*2..3]-(d)
```

```
RETURN a = c, b = d, size(r)
```

Result

a = c	b = d	size(r)
true	true	2
Rows: 1		

By default, only allows paths to traverse a relationship once, repeating a variable-length relationship in the same graph pattern will yield no results. (The same is not true if a query uses `REPEATABLE ELEMENTS`). For example, this `MATCH` clause will never pass on any intermediate results to subsequent clauses:

```
MATCH (x)-[r*1..2]->(y)-[r*1..2]->(z)
```

Attempting to repeat a variable-length relationship in a single relationship pattern will raise an error. For example, the following pattern raises an error because the variable `r` appears in both a variable-length relationship and a fixed-length relationship:

```
MATCH (x)-[r*1..2]->(y)-[r]->(z)
```

Examples

The following pattern matches paths starting with nodes labeled `A` and ending with nodes labeled `B`, that traverse between two and three relationships of type `R`:

```
(:A)-[r:R*2..3]->(:B)
```

The following traverses relationships of type `R` or `S` or `T` exactly five times:

```
()-[r:R|S|T*5]->()
```

The following traverses any relationship between zero and five times, with the path beginning at nodes labeled `A` and ending at nodes labeled `B`. Note that this will also return all nodes that have both labels `A` and `B` for the case where zero relationships are traversed:

```
(:A)-[*0..5]->(:B)
```

If the lower bound is removed, it will default to one, and will no longer return paths of length zero, i.e. single nodes:

```
(:A)-[*..5]->(:B)
```

The following pattern traverses one or more relationships of any direction that have property `p` = `$param`:

```
()-[* {p: $param}]-()
```

Node pattern pairs

It is not valid syntax to write a pair of node patterns next to each other. For example, all of the following would raise a syntax error:

```
(a:A)(b:B)
```

```
(a:A)(b:B)<-[r:R]-(c:C)
```

```
(a:A)<--(b:B)(c:C)-->(d:C)
```

However, the placing of pairs of node patterns next to each other is valid where it results indirectly from the expansion of quantified path patterns.

Iterations of quantified path patterns

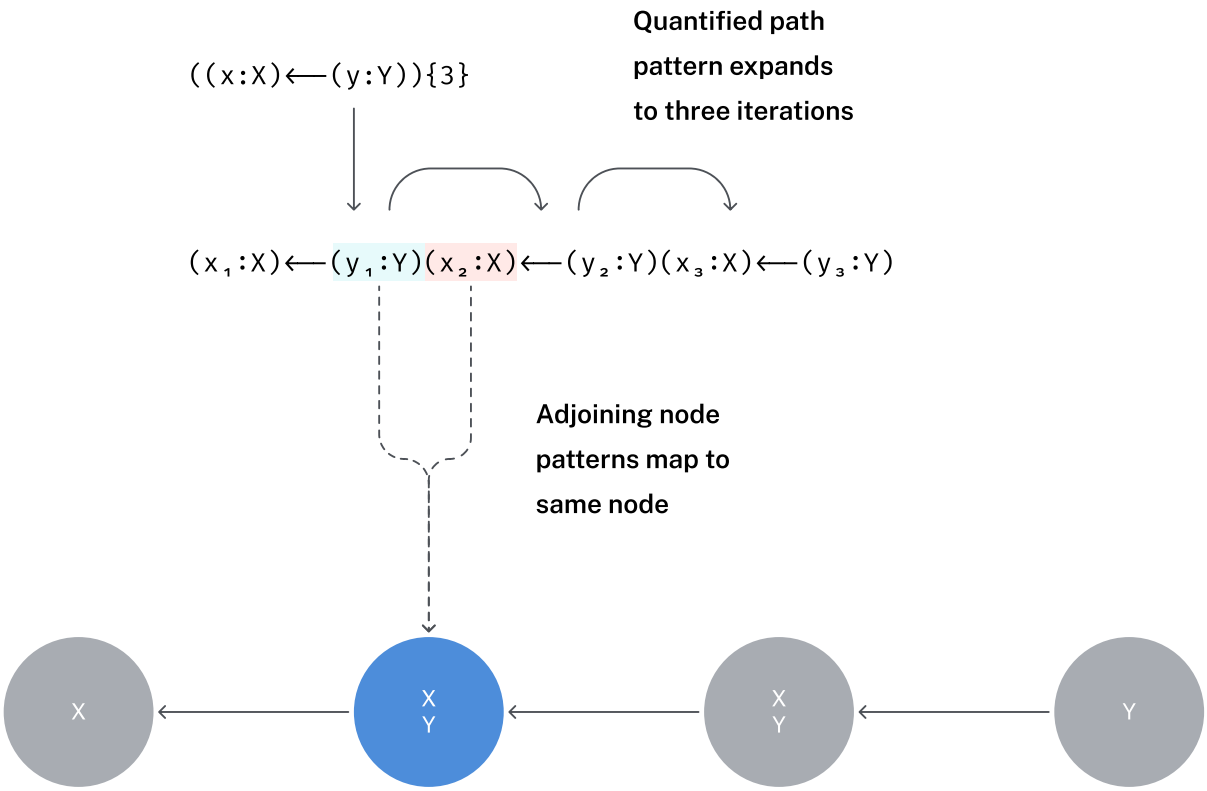
When a quantified path pattern is expanded, the fixed path pattern contained in its parentheses is repeated and chained. This results in pairs of node patterns sitting next to each other. Take the following quantified path pattern as an example:

```
((x:X)<--(y:Y)){3}
```

This is expanded by repeating the fixed path pattern `(x:X)<-(y:Y)` three times, with indices on the variables to show that no equijoin is implied (see equijoins for more information):

```
(x1:X)<--(y1:Y)(x2:X)<--(y2:Y)(x3:X)<--(y3:Y)
```

The result is that two pairs of node patterns end up adjoining each other, `(y1:Y)(x2:X)` and `(y2:Y)(x3:X)`. During the matching process, each pair of node patterns will match the same nodes, and those nodes will satisfy the conjunction of the predicates in the node patterns. For example, in the first pair both `y1` and `x2` will bind to the same node, and that node must have labels `X` and `Y`. This expansion and binding is depicted in the following diagram:



Simple path patterns and quantified path patterns

Pairs of node patterns are also generated when a simple path pattern is placed next to a quantified path. For example, consider the following path pattern:

```
(:A)-[:R]->(:B) ((:X)<--(:Y)){1,2}
```

After expanding the iterations of the quantified path pattern, the right-hand node pattern `(:B)` adjoins the left-hand node pattern `(:X)`. The result will match the same paths as the union of matches of the following two path patterns:

```
(:A)-[:R]->(:B&X)<--(:Y)
```

```
(:A)-[:R]->(:B&X)<--(:Y&X)<--(:Y)
```

If the simple path pattern is on the right of the quantified path pattern, its leftmost node `(:A)` adjoins the rightmost node `(:Y)` of the last iteration of the quantified path pattern. For example, the following:

```
((:X)<--(:Y)){1,2} (:A)-[:R]->(:B)
```

will match the same paths as the union of the following two path patterns:

```
(:X)<--(:Y&A)-[:R]->(:B)
```

```
(:X)<--(:Y&X)<--(:Y&A)-[:R]->(:B)
```

Pairs of quantified path patterns

When two quantified path patterns adjoin, the rightmost node of the last iteration of the first pattern is merged with the leftmost node of the first iteration of the second pattern. For example, the following adjoining patterns:

```
((:A)-[:R]->(:B)){2} ((:X)<--(:Y)){1,2}
```

will match the same set of paths as the union of the paths matched by these two path patterns:

```
(:A)-[:R]->(:B&A)-[:R]->(:B&X)<--(:Y)
```

```
(:A)-[:R]->(:B&A)-[:R]->(:B&X)<--(:Y&X)<--(:Y)
```

Zero iterations

If the quantifier allows for zero iterations of a pattern, for example `{0,3}`, then the 0th iteration of that pattern results in the node patterns on either side pairing up.

For example, the following path pattern:

```
(:X) ((a:A)-[:R]->(b:B)){0,1} (:Y)
```

will match the same set of paths as the union of the paths matched by the following two path patterns:

```
(:X&Y)
```

```
(:X&A)-[:R]->(:B&Y)
```

Shortest paths

Shortest paths

Path selectors are modes that are specified at the path pattern level. There are five types of path selectors:

- `SHORTEST k`
- `ALL SHORTEST`
- `SHORTEST k GROUPS`
- `ANY k`
- `ALL`

The first three are shortest path selectors, and they specify which shortest paths should be returned, with

the exact method of selection depending on the path selector type and the value of `k`.

See path patterns for details on where in the path pattern to include the path selector.

Syntax

```
shortestPathSelector ::=
  { allPathSearch | anyPathSearch | shortestPathSearch }


shortestPathSearch ::=
  { allShortest | anyShortest | countedShortestPaths |
     countedShortestGroups }


allShortest ::= "ALL SHORTEST" [ pathOrPaths ]


anyShortest ::= "ANY SHORTEST" [ pathOrPaths ]


countedShortestPaths ::=
  "SHORTEST" numberOfPaths [ pathOrPaths ]


countedShortestGroups ::=
  "SHORTEST" [ numberOfGroups ] [ pathOrPaths ] { "GROUP" | "GROUPS" }


allPathSearch ::= "ALL" [ pathOrPaths ]


anyPathSearch ::= "ANY" [ numberOfPaths ] [ pathOrPaths ]


pathOrPaths ::= { "PATH" | "PATHS" }


numberOfPaths ::= unsignedDecimalInteger | parameter


numberOfGroups ::= unsignedDecimalInteger | parameter
```

Note:

The `unsignedDecimalInteger` must not be larger than the Java constant `Long.MAX_VALUE` (2^{63} -1).

Rules

Selective path selectors

The following path selector types are *selective* path selectors:

- `SHORTEST k`
- `SHORTEST k GROUPS`
- `ALL SHORTEST`
- `ANY k`

This means that they reduce the number of matches produced by the path pattern. `ALL` returns all matches

produced, and so is not selective.

Order of path selection

When a path selector is specified, the following steps are followed to find solutions to the path pattern:

1. All paths matching the path pattern are found.
2. Any paths not satisfying the predicates contained in the path pattern are removed. This is a *pre*-filter.
3. Paths are selected according to the path selector specified.
4. Any paths not satisfying the predicates in the graph pattern `WHERE` clause are removed. This is a *post*-filter.

Examples of pre-filters

```
// node pattern WHERE clause
MATCH p = SHORTEST 2 (a WHERE a.p < 42)-+()

// relationship pattern property key-value expression
MATCH p = SHORTEST 2 (a)-[{p: 42}]-+()

// label and type expressions
MATCH p = SHORTEST 2 (:A)-[:R]-+(:B)

// quantified path pattern
MATCH SHORTEST 2 ((a)--(b) WHERE a.p < b.p)+

// parenthesized path pattern expression
// note the position of the parentheses!
MATCH SHORTEST 2 ( p = ()--+() WHERE any(n IN nodes(p) WHERE n.p < 42) )
```

Examples of post-filters

```
// graph pattern WHERE clause
MATCH p = SHORTEST 2 (a)--+()
WHERE all(n IN nodes(p) WHERE n.p < 42)
```

Partitions

If there are multiple start and end nodes matching a path pattern, then, when a selective path selector is specified, the paths matched by the path pattern are partitioned by distinct pairs of start and end nodes. Within each partition, they are put in ascending order of path length. The partitioned paths are then selected according to the specified path selector as follows:

Selective path selector	Partitioning
`SHORTEST k`	First `k` paths, starting with the shortest. Where more than one path could be picked for a given length, there is no particular order in which that selection is done. If `k` is greater than the number of paths in the partition, then all of the paths in that partition are returned. `ANY SHORTEST` is equivalent to `SHORTEST 1`.
`SHORTEST k GROUPS`	The paths are further grouped by path length, and those groups are put in ascending order by path length. Then the paths from the first `k` groups are selected. If `k` is greater than the number of path length groups within a partition, then all paths in that partition are returned. `GROUP` and `GROUPS` can be used interchangeably.

Selective path selector	Partitioning
`ALL SHORTEST`	All paths tied for the shortest. Equivalent to `SHORTEST 1 GROUP`.
`ANY k`	Any `k` paths are returned. `ANY` is the same as `ANY 1`. This is useful to determine the reachability of nodes.
`ALL`	All paths are returned. This is the same as not specifying any path selector.

For a selective path selector, if `k` is greater than `N`, the number of paths matching the path pattern, then all `N` paths are returned.

Only one path pattern allowed

When a selective shortest path selector is specified for a path pattern, it must be the only path pattern in the graph pattern.

Not allowed

```
MATCH p = SHORTEST 2 (:A)--+(a)--+(:B), q = ANY 2 (a)-->{,2}(:C)
RETURN p, q

MATCH ALL SHORTEST (n:A) (()-->(:B))+, (:X)--(n)--(:Y)
RETURN n
```

Allowed

```
MATCH p = SHORTEST 2 (:A)--+(a)--+(:B)
MATCH q = ANY 2 (a)-->{,2}(:C)
RETURN p, q

MATCH ALL SHORTEST (n:A) (()-->(:B))+
MATCH (:X)--(n)--(:Y)
RETURN n

MATCH ALL (n:A) (()-->(:B))+, (:X)--(n)--(:Y)
RETURN n
```

`PATH`/`PATHS`

The keywords `PATH` and `PATHS` are optional and can be added to the end of the path selector (but before `GROUP` or `GROUPS`). Including either of them does not change path selection. For example, the following two `MATCH` clauses are equivalent:

```
MATCH ALL SHORTEST PATHS (n:A) (()-->(:B))+
MATCH ALL SHORTEST (n:A) (()-->(:B))+
```

Match modes and path modes

When combined with a match mode such as `REPEATABLE ELEMENTS`, `SHORTEST` selective path selectors are placed after the specified match mode. This is because a match mode applies to the whole graph pattern match.

Combine `SHORTEST` with the `REPEATABLE ELEMENTS` match mode

```
MATCH REPEATABLE ELEMENTS p = SHORTEST 1 (:A)--{,20}(:B)--{,20}(:C)
```

`SHORTEST 1` returns only a single path. However, because `REPEATABLE ELEMENTS` can generate an infinite number of paths due to its ability to re-traverse relationships, and because a predicate that no path satisfies would therefore cause the search to never halt, an upper quantifier bound is required when combining `SHORTEST` selective path selectors and `REPEATABLE ELEMENTS`.

When combined with a path mode such as `ACYCLIC`, `SHORTEST` selective path selectors are placed before the path mode. This is because, although the selective path selector and path mode keywords are written left-to-right, they are evaluated right-to-left. In the below example, `ACYCLIC` pre-filters paths to only valid ones and then `SHORTEST 1` selects among those.

Combine `SHORTEST` with the `ACYCLIC` path mode

```
MATCH p = SHORTEST 1 ACYCLIC (:A)--{,20}(:B)
```

Examples

Return a single shortest path for each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH SHORTEST 1 (:A)-[:R]->{0,10}(:B)
```

Return any two paths connecting each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH p = ANY 2 (:A)-[:R]->{0,10}(:B)
```

Return all paths equal to the shortest path length for each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH ALL SHORTEST (:A)-[:R]->{0,10}(:B)
```

Return all paths equal to the two shortest path lengths for each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH SHORTEST 2 GROUPS (:A)-[:R]->{0,10}(:B)
```

Return a single shortest path for each distinct pair of nodes, where the path length is an even number:

```
MATCH SHORTEST 1 (p = ()--+() WHERE length(p) % 2 = 0)
```

For every single shortest path connecting each distinct pair of nodes, only return those that have path length that is an even number (so fewer results than the previous example):

```
MATCH p = SHORTEST 2 ()--+()
WHERE length(p) % 2 = 0
```

The `shortestPath()` and `allShortestPaths()` functions

Prior to the introduction of keyword-based specification of shortest path selection, the only available syntax for shortest paths were the functions `shortestPath()` and `allShortestPaths()`. They are similar to `SHORTEST 1` and `ALL SHORTEST`, but with several differences:

- The path pattern is passed as an argument to the functions.
- The path pattern is limited to a single relationship pattern.
- To return results where the first and last node in the path are the same requires a change to the configuration setting `dbms.cypher.forbid_shortestpath_common_nodes`.
- The minimum path length, also called the lower bound of the variable-length relationship pattern, should be 0 or 1.

Both functions will continue to be available, but they are not GQL conformant.

Syntax

```
pathSelectorFunction ::=
  { shortestPathFunction | allShortestPathsFunction }

shortestPathFunction ::=
  "shortestPath(" + oneHopPathPatternExpression + ")"

allShortestPathsFunction ::=
  "allShortestPaths(" + oneRelPathPatternExpression + ")"

oneRelPathPatternExpression ::=
  nodePattern varLengthRelationship nodePattern
```

Note that it is possible to pass a fixed-length path pattern (with a single relationship) to the path selector function, but doing so would not serve any purpose in discovering a shortest path.

Rules

Restricted to variable-length relationships

The pattern in the path selector function must be a variable-length relationship and not a quantified path pattern.

Not allowed

```
shortestPath(((a)-[:R]-(b)){1,5})

shortestPath((:A)-->+(:B))
```

Path pattern length

There must be exactly one relationship pattern in the path pattern, and the lower bound should be 0 or 1.

Allowed

```
shortestPath((a)-[:R*1..5]-(b))
```

Not allowed

```
shortestPath((a)-[:R*1..5]-(b)-->(:X))

shortestPath((a)-[:R*2..5]-(b))

shortestPath((:A))

allShortestPaths((a:A)-[:S*]->(:B), (a)-[:R*1..3]->(:C))
```

Pre and post filtering

If the `MATCH` clause of the `shortestPath()` function includes a `WHERE` clause, this condition will act as a pre-filter: paths satisfying the `WHERE` clause are first found, and from those paths the shortest path is selected.

Examples of pre-filters:

```
MATCH p = shortestPath(()-[*]-())
WHERE all(n in nodes(p) WHERE n.p < 42)

MATCH p = shortestPath(()-[*]-())
WHERE all(r in relationships(p) WHERE r.p < 42)
```

These contrast with `WHERE` clauses that are not part of the same `MATCH` clause.

Example of post-filters

```
MATCH p = shortestPath(()-[*]-())
WITH nodes(p) AS N
WHERE all(n in N WHERE n.p < 42)
```

Non-determinism

If there is more than one path with minimum length, then `shortestPath()` function will return one of those paths non-deterministically. As `allShortestPaths()` would return all of those paths, its results are deterministic.

Examples

Return a single shortest path for each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH shortestPath((:A)-[:R*0..10]->(:B))
```

Return all paths equal to the shortest path length for each distinct pair of nodes matching `(:A)` and `(:B)`:

```
MATCH allShortestPaths((:A)-[:R*0..10]->(:B))
```

Match modes and path modes

Match modes

A match mode specifies whether or not a relationship can be matched more than once when matching graph patterns. Cypher contains two match modes:

- `DIFFERENT RELATIONSHIPS` (default)
- `REPEATABLE ELEMENTS` Cypher 25 only Introduced in Neo4j 2025.06

Syntax

```
graphPattern ::=
  [ matchMode ] pathPattern [ "," pathPattern ]* [ graphPatternWhereClause ]

matchMode ::=
  { REPEATABLE ELEMENT[S] | DIFFERENT RELATIONSHIPS }
```

Rules

`DIFFERENT RELATIONSHIPS`

`DIFFERENT RELATIONSHIPS` is a restrictive match mode. It requires that all relationships matched across all constituent path patterns in a graph pattern be unique. More specifically:

- A relationship may only occur once for a given `MATCH` result.
- A given relationship can only be bound to at most one relationship variable (e.g. `-[r]-`) or an anonymous relationship pattern (`-[]-`).
- A relationship can only be bound once within a relationship group variable.

No solutions can be returned if a relationship variable occurs more than once across a graph pattern. For example, the following patterns, where the `r` variable acts as an equijoin requiring all occurrences to match the same relationship, cannot return paths:

```
MATCH (a)-[r]-(b)-[f]-(c)-[r]-(d)  // r repeated twice
MATCH (a)-[r]-(b), (c)-[r]-(d)     // r repeated twice
```

The `DIFFERENT RELATIONSHIPS` keyword was released in Neo4j 2025.06. It is only available in Cypher 25 and can be used to explicitly specify Cypher's default match mode after `MATCH`. Specifying the `DIFFERENT RELATIONSHIPS` keyword is functionally equivalent to not specifying a match mode. For example, the following two `MATCH` clauses are equivalent:

```
MATCH (a)--{2}(b)
MATCH DIFFERENT RELATIONSHIPS (a)--{2}(b)
```

`REPEATABLE ELEMENTS`

Introduced in Neo4j 2025.06 Cypher 25 only

`REPEATABLE ELEMENTS` is a non-restrictive match mode. It allows for relationships matched across all constituent path patterns in a graph pattern to be repeatedly traversed. More specifically:

- A relationship may occur more than once for a given `MATCH` result.
- A given relationship can be bound to more than one relationship variable or anonymous relationship pattern.

- A relationship can be bound more than once within a relationship group variable.

Queries utilizing this match mode must add the keyword `REPEATABLE ELEMENTS` between the `MATCH` clause and the graph pattern.

Solutions can be returned if a relationship variable occurs more than once across a graph pattern. For example, the following two graph patterns, where the `r` variable acts as an equijoin requiring all occurrences to match the same relationship, can return paths:

```
MATCH REPEATABLE ELEMENTS (a)-[r]-(b)-[f]-(c)-[r]-(d)  // r repeated twice
MATCH REPEATABLE ELEMENTS (a)-[r]-(b), (c)-[r]-(d)     // r repeated twice
```

Quantifier bounds

The following applies to the use of variable-length patterns quantifiers in different match modes:

- `DIFFERENT RELATIONSHIPS` does not require an upper bound on a quantifier. The following are valid: `+`, `{n, }`, `{ , }`, `*`.
- `REPEATABLE ELEMENTS` requires all quantifiers to have an upper bound. The following are not valid: `+`, `{n, }`, `{ , }`, `*`, but these are valid: `{n}`, `{n, m}`, `{,m}`. This restriction is required to prevent a a potentially infinite number of returned paths.

Clauses and subqueries

Match modes can only be specified after `MATCH`. They cannot be applied to a `MERGE` clause.

Match modes are allowed in the `COLLECT`, `COUNT`, and `EXISTS` subqueries after a `MATCH` clause. If so, the match mode applies to patterns matched within the subquery and is separate from that applied to the top level graph pattern. In the following example, `REPEATABLE ELEMENTS` only applies to the pattern matched within the `COUNT` subquery:

```
MATCH (a)-[:R]->(b)
WHERE COUNT {
    MATCH REPEATABLE ELEMENTS (a)-[:S]-{,10}(b)
    RETURN *
} > 5
RETURN *
```

Selective path selectors

The following applies to match modes and the number of path patterns with selective path selectors (`SHORTEST k`, `SHORTEST k GROUPS`, `ALL SHORTEST`, and `ANY k`).

- If the match mode is `DIFFERENT RELATIONSHIPS`, there can only be one path pattern if it has a selective path selector.
- If the match mode is `REPEATABLE ELEMENTS`, there can be multiple path patterns even if one or more have a selective path selector.

Allowed: `DIFFERENT RELATIONSHIPS` *with selective path selector and one path pattern*

```
MATCH SHORTEST 2 ((a)--(b) WHERE a.p < b.p)+
```

Allowed: `REPEATABLE ELEMENTS` *with multiple path patterns and selective path selectors*

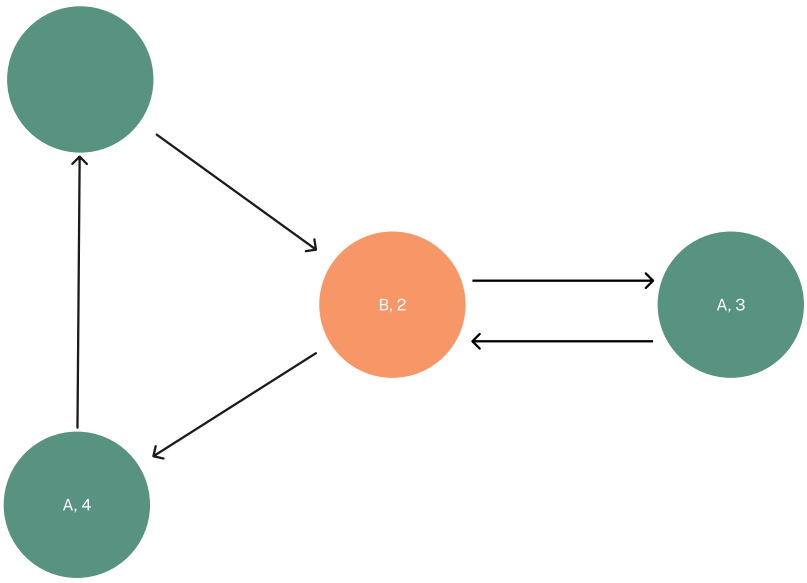
```
MATCH REPEATABLE ELEMENTS SHORTEST 2 (x) ((a)--(b) WHERE a.p < b.p){,10},
      ANY (x)-->{,10}(:X)
```

Not allowed: `DIFFERENT RELATIONSHIPS` *with more than one path pattern and a selective path selector*

```
MATCH ALL SHORTEST (x) ((a)--(b) WHERE a.p < b.p){,10},
      ANY (x)-->{,10}(:X)
```

Examples

This section uses the following graph:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (n1:A {q: 1}),
       (n2:B {q: 2}),
       (n3:A {q: 3}),
       (n4:A {q: 4}),

       (n1)-[:R]->(n2),
       (n2)-[:R]->(n3),
       (n3)-[:R]->(n2),
       (n2)-[:R]->(n4),
       (n4)-[:R]->(n1)
```

To demonstrate the basic concept behind `REPEATABLE ELEMENTS`, and how it differs from the `DIFFERENT RELATIONSHIPS` match mode, consider the following pattern, which attempts to form a relationship equijoin: `(a)-[r]-(b)-[r]-(c)`. For this pattern to return any results, the same relationship must be re-traversed. That is, the same relationship connecting `a` to `b` must be the same as that connecting `b` to `c`.

If tested using `DIFFERENT RELATIONSHIPS`, no results are returned due to the restrictions of the match mode.

Find paths re-traversing the same relationship using `DIFFERENT RELATIONSHIPS`

```
MATCH  p = (:A)-[r]-(:B)-[r]-(:A)
RETURN [n IN nodes(p) | n.q] AS nodes
```

Result

```
(no results)
```

However, if the same query uses the `REPEATABLE ELEMENTS` match mode, four paths are returned in which the same relationship is traversed more than once.

Find paths re-traversing the same relationship using `REPEATABLE ELEMENTS`

```
MATCH  REPEATABLE ELEMENTS p = (:A)-[r]-(:B)-[r]-(:A)
RETURN [n IN nodes(p) | n.q] AS nodes
```

Result

nodes
[3, 2, 3]
[4, 2, 4]
[3, 2, 3]
[1, 2, 1]
Rows: 4

The `DIFFERENT RELATIONSHIPS` match mode allows only nodes to recur in a path.

Match a pattern with a length of `5` *relationships using the* `DIFFERENT RELATIONSHIPS` *match mode*

```
MATCH p = (:B)-->{5}()
RETURN [n IN nodes(p) | n.q] AS nodes
```

Result

nodes
[2, 3, 2, 4, 1, 2]
[2, 4, 1, 2, 3, 2]
Rows: 2

The `REPEATABLE ELEMENTS` match mode allows both nodes and relationships to be repeatedly traversed in paths.

Match a pattern with a length of `5` *relationships using the* `REPEATABLE ELEMENTS` *match mode*

```
MATCH REPEATABLE ELEMENTS p = (:B)-->{5}()
RETURN [n IN nodes(p) | n.q] AS nodes
```

Result

nodes
[2, 3, 2, 3, 2, 3]
[2, 3, 2, 3, 2, 4]
[2, 3, 2, 4, 1, 2]
[2, 4, 1, 2, 3, 2]
[2, 4, 1, 2, 4, 1]
Rows: 2

Path modes

Introduced in Neo4j 2026.03 | Cypher 25 only

Path modes define the uniqueness constraints for nodes and relationships within a single path pattern. This contrasts with match modes which constrain relationship uniqueness across an entire `MATCH` clause.

Cypher supports three path modes:

- `WALK`: no constraints imposed; both nodes and relationships can be repeated within the path.
- `TRAIL`: relationships cannot be repeated, but nodes can be repeated within the path.
- `ACYCLIC`: nodes cannot be repeated within a path. A node can be repeated across paths, enabling the formation of equijoins.

Syntax

```
pathModePrefix ::= pathMode [ pathKeywords ]

pathMode ::= WALK | TRAIL | ACYCLIC

pathKeywords ::= PATH | PATHS
```

Rules

The table below shows the effective uniqueness constraint for each combination of match mode and path mode.

Effective uniqueness constraints for match mode and path mode combinations

Match mode	Path mode	Effective uniqueness constraint	Notes
`DIFFERENT RELATIONSHIPS`	`WALK`	Relationship uniqueness	Constraint enforced by match mode.
`DIFFERENT RELATIONSHIPS`	`TRAIL`	Relationship uniqueness	Path mode does not alter the constraint imposed by the match mode.

Match mode	Path mode	Effective uniqueness constraint	Notes
`DIFFERENT RELATIONSHIPS`	`ACYCLIC`	Node uniqueness and relationship uniqueness	Path mode constraint stricter than match mode; prevents node repetition.
`REPEATABLE ELEMENTS`	`WALK`	No uniqueness	Both match mode and path mode allow repetitions of nodes and relationships.

Note that `TRAIL` and `ACYCLIC` cannot be combined with the `REPEATABLE ELEMENTS` match mode.

In practice, the syntax needed to achieve each uniqueness constraint is:

Path pattern uniqueness constraints and the corresponding syntax

Desired constraint	Syntax	Notes
No uniqueness constraint	`MATCH REPEATABLE ELEMENTS …`	
Relationship uniqueness	No keywords required	
Node uniqueness	`MATCH ACYCLIC …`	`ACYCLIC` must be specified at the start of each path pattern.

As the above tables show, `ACYCLIC` is the only path mode that alters the effective uniqueness constraint of the path pattern match. For more information about `ACYCLIC`, see Acyclic paths.

All path modes in a `MATCH` must be the same

All path patterns in a single `MATCH` clause must use the same path mode. For example, you cannot mix `ACYCLIC` with a pattern that uses the `TRAIL` mode:

```
MATCH p = ACYCLIC (a)-->(b), q = TRAIL (c)-->(d)
RETURN p, q
```

GQLSTATUS error chain

42N61: error: syntax error or access rule violation - unsupported mixing of different path modes. Mixing ACYCLIC and TRAIL in the same graph pattern is not supported. Split the pattern into separate MATCH clauses instead.

42001: error: syntax error or access rule violation - invalid syntax

Only `WALK` can be combined with the `REPEATABLE ELEMENTS` match mode

Explicit path modes like `ACYCLIC` or `TRAIL` cannot be used with the `REPEATABLE ELEMENTS` match mode. For example:

```
MATCH REPEATABLE ELEMENTS p = ACYCLIC (n)-->{1, 10}(m)
RETURN p
```

GQLSTATUS error chain

42N60: error: syntax error or access rule violation - unsupported combination of match mode and path mode. REPEATABLE ELEMENTS with ACYCLIC path mode is not supported.

42001: error: syntax error or access rule violation - invalid syntax

Mixing restrictive path selectors and explicit path modes

Neo4j versions and rules for mixing restrictive path selectors and explicit path modes

Neo4j versions	Behavior
2026.03 - 2026.4	Explicit path modes cannot be combined with restrictive path selectors (e.g., `ANY`, `SHORTEST`) or with legacy shortest path functions like `shortestPath()`.

2026.5 - current	Explicit path modes can be combined with restrictive path selectors, but not with legacy shortest path functions.
	
	Supported
	`MATCH p = ANY SHORTEST ACYCLIC (a:A)-->+(b:!A)`
	`RETURN p`

Path modes and variable-length paths

Explicit path modes cannot be used with the variable-length relationships syntax (e.g., `-[*]->`). Use quantified path patterns or quantified relationships instead.

Supported — explicit path mode with quantified relationship syntax

```
MATCH ACYCLIC (start)-[:R]->{4,7}(target)
RETURN start, target
```

Not supported — explicit path mode with variable-length relationship syntax

```
MATCH ACYCLIC (start)-[*4..7]->(target)
RETURN start, target
```

GQLSTATUS error chain

51N26: error: system configuration or operation exception - not supported in this version. Using explicit path modes on a pattern containing a variable-length relationship is not available. This implementation of Cypher does not support `ACYCLIC` on variable-length relationships.

42001: error: syntax error or access rule violation - invalid syntax

Path modes cannot be applied to `MERGE` clauses

For example, this is not allowed:

```
MERGE ACYCLIC (robert:Critic)-[:P]->(c:Critic)
```

GQLSTATUS error chain

42I04: error: syntax error or access rule violation - invalid expression. 'ACYCLIC' cannot be used in a MERGE clause.

42001: error: syntax error or access rule violation - invalid syntax

Repeated node variables within a path pattern result in no matches

Declaring a node variable more than once within a single restrictive path pattern (like `ACYCLIC`) results in no matches. Because the variable must bind to the same node element and the path mode forbids repeated nodes, there is no solution. The below query finds no matches because the node `x` is repeated.

```
MATCH p = ACYCLIC (x:Router)-[:LINK]->(x)
RETURN p
```

Repeated node variables across different path patterns specify an equijoin

When a node variable is repeated in separate path patterns within the same `MATCH` clause, it specifies an equijoin. In the below example, `x` is repeated across `p` and `q`, creating an `equijoin` between two acyclic paths. This is allowed because each individual path pattern remains acyclic.

```
MATCH p = ACYCLIC (a:Router)-[:LINK]->+(x:Router),
  q = ACYCLIC (x)-[:LINK]->+(z:Router)
RETURN p, q
```

Path mode scope in subqueries

The path mode of a subquery (such as an `EXISTS` or `COUNT` expression) is independent of the path mode in the outer query. This allows you to perform secondary checks with different uniqueness constraints. In the below example, the outer query finds path using relationship uniqueness (default `WALK` path mode + default match mode `DIFFERENT RELATIONSHIPS`). The subquery then checks for the existence of an `ACYCLIC` path connecting `a` and `b`.

```
MATCH p = (a:Router {name: 'A'})-[:LINK]->+(b:Router {name: 'D'})
  WHERE EXISTS { MATCH ACYCLIC (a)-[:LINK]->+(b) }
RETURN [n IN nodes(p) | n.name] AS route
```

Values and types

Cypher supports a range of data values. When writing Cypher queries, it is not possible to declare a data type. Rather, Cypher will automatically infer the data type of a given value.

More information about the data values and types supported by Cypher can be found in the following sections:

- Property, structural, and constructed values
- Boolean, numeric, and string literals
- Temporal values
- Spatial values
- Lists
- Maps
- UUIDs Introduced in Neo4j 2026.08
- Vectors Introduced in Neo4j 2025.10
- Graph references
- Working with `null`
- Casting data values
- Equality, ordering, and comparison of value types

For information about how to check the type of a value, see Type predicate expressions

Property, structural, and constructed values

Cypher provides first class support for a number of data value types. These fall into the following three categories: **property**, **structural**, and **constructed**. This section will first provide a brief overview of each type, and then go into more detail about the property data type.

Property types

A property type value is one that can be stored as a node or relationship property.

Property types are the most primitive types in Cypher and include the following: `BOOLEAN`, `DATE`, `DURATION`, `FLOAT`, `INTEGER`, `LIST`, `LOCAL DATETIME`, `LOCAL TIME`, `POINT`, `STRING`, `UUID`, `VECTOR`, `ZONED DATETIME`, and `ZONED TIME`.

- Property types can be returned from Cypher queries.
- Property types can be used as parameters.
- Property types can be stored as properties.
- Property types can be constructed with Cypher literals.

Homogeneous lists of simple types can be stored as properties (with the exeption of `VECTOR` types, which cannot be stored in lists), although lists in general (see Constructed types) cannot be stored as properties. Lists stored as properties cannot contain `null` values.

Storing `VECTOR` type values as properties is only supported in the Neo4j Enterprise Edition using block format or on Aura instances. This functionality is not available in Neo4j Community Edition.

Cypher also provides pass-through support for byte arrays, which can be stored as property values. Byte arrays are supported for performance reasons, since using Cypher's generic data type, `LIST<INTEGER>` (where each `INTEGER` has a 64-bit representation), would be too costly. However, byte arrays are *not* considered a first class data type by Cypher, so they do not have a literal representation.

Structural types

The following data types are included in the structural types category: `NODE`, `RELATIONSHIP`, and `PATH`.

- Structural types can be returned from Cypher queries.
- Structural types cannot be used as parameters.
- Structural types cannot be stored as properties.
- Structural types cannot be constructed with Cypher literals.

The `NODE` data type includes: id, label(s), and a map of properties. Note that labels are not values, but a form of pattern syntax.

The `RELATIONSHIP` data type includes: id, relationship type, a map of properties, start node id, and end node id.

The `PATH` data type is an alternating sequence of nodes and relationships.

> **Note:**
>
> Nodes, relationships, and paths are returned as a result of pattern matching. In Neo4j, all relationships have a direction. However, you can have the notion of undirected relationships at query time.

Constructed types

The following data types are included in the constructed types category: `LIST` and `MAP`.

- Constructed types can be returned from Cypher queries.
- Constructed types can be used as parameters.
- Constructed types cannot be stored as properties (with the exception of homogenous lists).
- Constructed types can be constructed with Cypher literals.

The `LIST` data type can be either a homogenous collection of simple values, or a heterogeneous, ordered collection of values, each of which can have any property, structural or constructed type.

The `MAP` data type is a heterogeneous, unordered collection of (*Key*, *Value*) pairs, where *Key* is a literal and *Value* can have any property, structural, or constructed type.

Constructed type values can also contain `null`. For more details, see working with null.

Types and their synonyms

The table below shows the types and their syntactic synonyms.

These types (and their synonyms) can be used in type predicate expressions and in property type constraints. They are also returned as a `STRING` value when using the valueType() function.

However, not all types can be used in all places.

Type	Synonyms
`ANY`	`ANY VALUE`
`BOOLEAN`	`BOOL`
`DATE`	
`DURATION`	
`FLOAT`	
`INTEGER`	`INT`, `SIGNED INTEGER`
`LIST<INNER_TYPE>`	`ARRAY<INNER_TYPE>`, `INNER_TYPE LIST`, `INNER_TYPE ARRAY`
`LOCAL DATETIME`	`TIMESTAMP WITHOUT TIME ZONE`, `TIMESTAMP WITHOUT TIMEZONE`
`LOCAL TIME`	`TIME WITHOUT TIME ZONE`, `TIME WITHOUT TIMEZONE`
`MAP`	
`NODE`	`ANY NODE`, `VERTEX`, `ANY VERTEX`
`NOTHING`	
`NULL`	
`PATH`	
`POINT`	
`PROPERTY VALUE`	`ANY PROPERTY VALUE`
`RELATIONSHIP`	`ANY RELATIONSHIP`, `EDGE`, `ANY EDGE`
`STRING`	`VARCHAR`
`UUID`	
`VECTOR<TYPE>(DIMENSION)`	`VECTOR`, `VECTOR<TYPE>`, `VECTOR(DIMENSION)`, `VECTOR(DIMENSION, TYPE)`
`ZONED DATETIME`	`TIMESTAMP WITH TIME ZONE`, `TIMESTAMP WITH TIMEZONE`
`ZONED TIME`	`TIME WITH TIME ZONE`, `TIME WITH TIMEZONE`
`INNER_TYPE_1 | INNER_TYPE_2…`	`ANY<INNER_TYPE_1 | INNER_TYPE_2…>`

All Cypher types contain the `null` value. To make them not nullable, `NOT NULL` can be appended to the end

of the type (e.g. `BOOLEAN NOT NULL`, `LIST<FLOAT NOT NULL>`). A shorthand syntax equivalent for `NOT NULL` is to use an exclamation mark `!` (e.g. `INTEGER!`, `LIST<STRING!>`). Note that closed dynamic types (`INNER_TYPE_1 | INNER_TYPE_2…`) cannot be appended with `NOT NULL`: all inner types must be nullable, or all appended with `NOT NULL`.

Type normalization

Cypher runs a normalization algorithm on all input types, simplifying the given type to a deterministic representation for equivalent types. Types are simplified to their default name (e.g. `BOOL` is simplified to `BOOLEAN`). Encompassing types are absorbed (e.g. `LIST<BOOLEAN> | LIST<BOOLEAN | INTEGER>` is normalized to `LIST<BOOLEAN | INTEGER>`). Types are also ordered.

The type `PROPERTY VALUE` is expanded to a closed dynamic union of all valid property types, and if all types are represented, then the normalization would simplify to `ANY`.

For example, given the closed dynamic type `BOOL | LIST<INT> | BOOLEAN | LIST<FLOAT | INT>`, the normalized type would be: `BOOLEAN | LIST<INTEGER | FLOAT>`.

This normalization is run on types used in type predicate expressions, and in property type constraints. Type normalization is also used to ensure the consistency of the output for the valueType() function.

Vector type normalization

The following rules apply to `VECTOR` normalization:

- A `VECTOR` type without a defined coordinate type or dimension encompasses all others.
- A `VECTOR` type with a defined coordinate type but no dimension encompasses `VECTOR` types with the same coordinate type and any dimension.
- A `VECTOR` with a defined dimension but no coordinate type encompasses `VECTOR` types with the same dimension and any coordinate type.
- `VECTOR` types with different coordinate types or dimensions cannot be normalized.

`VECTOR` *type normalization*

Compared types	Normalized Result	Reason
`VECTOR | VECTOR<FLOAT32>(12345)`	`VECTOR`	`VECTOR` is the supertype and encompasses all others.
`VECTOR<FLOAT32> | VECTOR(12345)`	`VECTOR<FLOAT32> | VECTOR(12345)`	Cannot be normalized — one vector lacks a dimension, the other lacks a coordinate type.
`VECTOR<FLOAT32> | VECTOR<FLOAT32>(12345)`	`VECTOR<FLOAT32>`	Same type, the vector without a defined dimension encompasses the other.
`VECTOR(12345) | VECTOR<FLOAT32>(12345)`	`VECTOR(12345)`	Same dimension, the vector without a defined coordinate type encompasses the other.
`VECTOR<FLOAT32> | VECTOR<INTEGER64>`	`VECTOR<FLOAT32> | VECTOR<INTEGER64>`	Cannot be normalized — coordinate types differ.

Compared types	Normalized Result	Reason
`VECTOR(1234) | VECTOR(12345)`	`VECTOR(1234) | VECTOR(12345)`	Cannot be normalized — dimensions differ.
`VECTOR<FLOAT32>(1234) | VECTOR<FLOAT64>(1234)`	`VECTOR<FLOAT32>(1234) | VECTOR<FLOAT64>(1234)`	Cannot be normalized — coordinate types differ, even if dimensions match.

Ordering of types

The ordering of types is as follows:

- Predefined types
  - `NOTHING`
  - `NULL`
  - `BOOLEAN`
  - `STRING`
  - `UUID`
  - `INTEGER8`*
  - `INTEGER16`*
  - `INTEGER32`*
  - `INTEGER` (`INTEGER64`)
  - `FLOAT32`*
  - `FLOAT` (`FLOAT64`)
  - `DATE`
  - `LOCAL TIME`
  - `ZONED TIME`
  - `LOCAL DATETIME`
  - `ZONED DATETIME`
  - `DURATION`
  - `POINT`
  - `NODE`
  - `RELATIONSHIP`
  - `VECTOR` (ordered by coordinate type and dimension)

* *Vector-only coordinate type.*

- Constructed types
  - `MAP`
  - `LIST<INNER_TYPE>` (ordered by the inner type)

    - `PATH`
- Dynamic union types
    - `INNER_TYPE_1 | INNER_TYPE_2…` (ordered by specific rules for closed dynamic union type)
    - `ANY`

Subtypes are always ordered before any enclosing types (e.g. `LIST<INTEGER>` is ordered before `LIST<INTEGER | FLOAT>`). This also means that the `NOT NULL` variants of each type comes before the nullable variant.

The order between two closed dynamic unions `A` and `B` is determined as followed:

- If `A` has fewer inner types than `B`, `A` is ordered first.
- If `A` and `B` have the same number of inner types, they are ordered according to the order of the first inner type that differ (lexicographic order).

The resulting order is deterministic.

Property type details

The below table provides more detailed information about the various property types that Cypher supports. Note that Cypher types are implemented using Java, and that below table references Java value constants.

Type	Min. value	Max. value	Precision
`BOOLEAN`	`False`	`True`	-
`DATE`	`-999_999_999-01-01`	`+999_999_999-12-31`	Days
`DURATION`	`P-292471208677Y-6M-15DT-15H-36M-32S`	`P292471208677Y6M15DT15H36M32.999999999S`	Nanoseconds
`FLOAT32`*	`Float.MIN_VALUE` [1]	`Float.MAX_VALUE`	32 bit
`FLOAT` (`FLOAT64`)	`Double.MIN_VALUE`	`Double.MAX_VALUE`	64 bit
`INTEGER8`*	`Byte.MIN_VALUE`	`Byte.MAX_VALUE`	8 bit
`INTEGER16`*	`Short.MIN_VALUE`	`Short.MAX_VALUE`	16 bit
`INTEGER32`*	`Integer.MIN_VALUE`	`Integer.MAX_VALUE`	32 bit
`INTEGER` (`INTEGER64`)	`Long.MIN_VALUE`	`Long.MAX_VALUE`	64 bit
`LOCAL DATETIME`	`-999_999_999-01-01T00:00:00`	`+999_999_999-12-31T23:59:59.999999999`	Nanoseconds
`LOCAL TIME`	`00:00:00`	`23:59:59.999999999`	Nanoseconds

Type	Min. value	Max. value	Precision
`POINT`	**Cartesian**: (`-Double.MAX_VALUE`, `-Double.MAX_VALUE`) **Cartesian_3D**: (`-Double.MAX_VALUE`, `-Double.MAX_VALUE`, `-Double.MAX_VALUE`) **WGS_84**: (`-180`, `-90`) **WGS_84_3D**: (`-180`, `-90`, `-Double.MAX_VALUE`)	**Cartesian**: (`Double.MAX_VALUE`, `Double.MAX_VALUE`) **Cartesian_3D**: (`Double.MAX_VALUE`, `Double.MAX_VALUE`, `Double.MAX_VALUE`) **WGS_84**: (`180`, `90`) **WGS_84_3D**: (`180`, `90`, `Double.MAX_VALUE`)	The precision of each coordinate of the `POINT` is 64 bit as they are floats.
`STRING`	-	-	-
`VECTOR<TYPE>(DIMENSION)`	Minimum of the coordinate type at the given dimension.	Maximum of the coordinate type at the given dimension.	Precision of the coordinate type.
`UUID`	`uuid(Long.MIN_VALUE, Long.MIN_VALUE)`	`uuid(Long.MAX_VALUE, Long.MAX_VALUE)`	128 bit (a `UUID` value is comprised of two 64 bit components; the least and most significant bit).
`ZONED DATETIME`	`-999_999_999-01-01T00:00:00+18:00`	`+999_999_999-12-31T23:59:59.999999999-18:00`	Nanoseconds
`ZONED TIME`	`00:00:00+18:00`	`23:59:59.999999999-18:00`	Nanoseconds

* *Vector-only coordinate type.*

Java value details

Name	Value
`Byte.MAX_VALUE`	127
`Byte.MIX_VALUE`	-128
`Double.MAX_VALUE`	1.7976931348623157e+308
`Double.MIN_VALUE`	4.9e-324
`Float.MAX_VALUE`	3.4028235E38
`Float.MIN_VALUE`	1.4E-45

Name	Value
`Integer.MAX_VALUE`	2147483647
`Integer.MIN_VALUE`	-2147483648
`Long.MAX_VALUE`	2^63-1
`Long.MIN_VALUE`	-2^63
`SHORT.MAX_VALUE`	32767
`SHORT.MIN_VALUE`	-32768

Boolean, numeric, and string literals

This page describes the literal forms of numerical (`INTEGER` and `FLOAT`), `STRING`, and `BOOLEAN` values in expressions.

Expressions containing unsanitized user input may make your application vulnerable to Cypher injection. Consider using parameters instead. For more information, see Neo4j Knowledge Base → Protecting against Cypher Injection.

Boolean

Boolean literals represent `BOOLEAN` values. A boolean literal may be written as:

- The literal `true`
- The literal `false`

Numerical

Numerical literals can represent `INTEGER` or `FLOAT` values. A numerical literal may be written as:

- A decimal `INTEGER` literal: `13`, `-40000`
- A hexadecimal `INTEGER` literal (prefix `0x`): `0x13af`, `0xFC3A9`, `-0x66eff`
- An octal `INTEGER` literal (prefix `0o`): `0o1372`, `-0o5671`
- A `FLOAT` literal in common notation: `3.14`
- A `FLOAT` literal in scientific notation: `6.022E23`, `1e-9`
- Literals for special `FLOAT` values: `Inf`, `Infinity`, `NaN`

Any numeric literal may contain an underscore `_` between digits. There may be an underscore between the `0x` or `0o` and the digits for hexadecimal and octal literals. For example: `1_000_000`, `0x_FC3A9`, and `0o_1372`.

String

String literals represent `STRING` values. They are written using single(`'`) or double quotes (`"`) and may contain escape sequences using a backslash (`\`). A string literal may be written as:

- A `STRING` quoted with single quotes: `'Hello, 42'`
- A `STRING` quoted with double quotes: `"Hello, 42"`
- A `STRING` with whitespace: `' hello '`
- A `STRING` with escape sequences: `'Line 1\nLine 2'`, `'Tab\tseparated'`
- A `STRING` containing Unicode characters: `'          '`
- A `STRING` using a Unicode code point: `'Name: \u004Aohn'` (produces `'Name: John'`)

String literal escape sequences

String literals can contain the following escape sequences:

Escape sequence	Character
`\t`	Tab
`\b`	Backspace
`\n`	Newline
`\r`	Carriage return
`\f`	Form feed
`\'`	Single quote
`\"`	Double quote
`\\`	Backslash
`\uxxxx`	Unicode UTF-16 code point (4 hex digits must follow the `\u`)

String interpolation

Introduced in Neo4j 2026.08 | Cypher 25 only

String interpolation lets one or more expressions be embedded directly inside a string literal. Each embedded expression is evaluated and inserted into the resulting `STRING`, so the string does not need to be built up manually with the string concatenation operators.

An interpolated string is written by prefixing a quoted string with `s` or `S`, and wrapping each expression to be evaluated in curly braces (`{` and `}`):

- `s"Hello, {p.name}"`
- `S'Hello, {p.name}'`

Interpolated strings can be quoted with either single or double quotes, following the same quoting rules as string literals, and may contain any number of `{expression}` placeholders.

Every embedded expression is automatically converted to a `STRING` using `toString()`. Because `toString()` does not support them, expressions that evaluate to `MAP`, `LIST`, `NODE`, `PATH`, or `RELATIONSHIP` values cannot be used inside an interpolated expression.

To include a literal curly brace in an interpolated string, escape it with a backslash: `\{` or `\}`.

> **Note:**
>
> Take care when using string interpolation to build Cypher strings that get passed into plugin procedures which execute dynamic Cypher, such as `apoc.cypher.run()`. Interpolating unsanitized values into such strings increases the risk of Cypher injection. Where possible, use parameters instead.

Examples

Interpolating a node property

```
CREATE (p:Person {name: 'Alice'})
RETURN s"Hello, {p.name}!" AS greeting
```

Result

greeting
"Hello, Alice!"
Rows: 1

Multiple placeholders in a single interpolated string

```
WITH 'Keanu' AS firstName, 'Reeves' AS lastName
RETURN s"{firstName} {lastName}" AS fullName
```

Result

fullName
"Keanu Reeves"
Rows: 1

Non-`STRING` values are automatically converted using `toString()`.

Interpolating a non-`STRING` value

```
WITH 42 AS age
RETURN s"Age: {age}" AS result
```

Result

result
"Age: 42"
Rows: 1

Escaping literal curly braces

```
RETURN s"Use \{curly\} braces" AS result
```

Result

result
"Use {curly} braces"
Rows: 1

Interpolated strings can be nested, since an embedded expression may itself be an interpolated string.

Nested string interpolation

```
WITH 'World' AS name
RETURN s"Outer: {s'Inner, {name}!'}" AS result
```

Result

result
"Outer: Inner, World!"
Rows: 1

Temporal values

Cypher has built-in support for handling temporal values, which can be stored as properties on nodes and relationships in Neo4j databases. This section will discuss how Cypher handles time zones, before exploring temporal values in more detail.

Note:

- Refer to Temporal functions - instant types for information regarding temporal *functions* allowing for the creation and manipulation of temporal values.
- Refer to Temporal operators for information regarding temporal operators.
- Refer to Equality, ordering, and comparison of value types for information regarding the comparison and ordering of temporal values.

Temporal value types

The following table lists the temporal value types and their supported components:

Type	Date support	Time support	Time zone support
`DATE`	✔		
`LOCAL TIME`		✔	
`ZONED TIME`		✔	✔
`LOCAL DATETIME`	✔	✔	
`ZONED DATETIME`	✔	✔	✔
`DURATION`	-	-	-

`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, and `ZONED DATETIME` are *temporal instant* types. A temporal instant value expresses a point in time with varying degrees of precision.

By contrast, `DURATION` is not a temporal instant type. A `DURATION` represents a temporal amount, capturing the difference in time between two instants, and can be negative. `DURATION` captures the amount of time between two instants, it does not capture a start time and end time.

Time zones

Time zones are represented either as an offset from UTC, or as a logical identifier of a *named time zone* (these are based on the IANA time zone database). In either case, the time is stored as UTC internally, and the time zone offset is only applied when the time is presented. This means that temporal instants can be ordered without taking time zone into account. If, however, two times are identical in UTC, then they are ordered by timezone.

When creating a time using a named time zone, the offset from UTC is computed from the rules in the time zone database to create a time instant in UTC, and to ensure the named time zone is a valid one.

It is possible for time zone rules to change in the IANA time zone database. For example, there could be alterations to the rules for daylight savings time in a certain area. If this occurs after the creation of a temporal instant, the presented time could differ from the originally-entered time, insofar as the local timezone is concerned. However, the absolute time in UTC would remain the same.

There are three ways of specifying a time zone in Cypher:

- Specifying the offset from UTC in hours and minutes (ISO 8601).
- Specifying a named time zone.
- Specifying both the offset and the time zone name (with the requirement that these match).

See specifying time zones for examples.

The named time zone form uses the rules of the IANA time zone database to manage *daylight savings time* (DST).

The default time zone of the database can be configured using the configuration option `db.temporal.timezone`. This configuration option influences the creation of temporal types for the following functions:

- Getting the current date and time without specifying a time zone.
- Creating a temporal type from its components without specifying a time zone.
- Creating a temporal type by parsing a `STRING` without specifying a time zone.
- Creating a temporal type by combining or selecting values that do not have a time zone component, and without specifying a time zone.
- Truncating a temporal value that does not have a time zone component, and without specifying a time zone.

Temporal instants

Specifying temporal instants

A temporal instant consists of three parts; the `date`, the `time`, and the `timezone`. These parts can be combined to produce the various temporal value types. The character `T` is a literal character.

Temporal instant type	Composition of parts
`DATE`	`<date>`
`LOCAL TIME`	`<time>` or `T<time>`
`ZONED TIME`	`<time><timezone>` or `T<time><timezone>`
`LOCAL DATETIME`*	`<date>T<time>`
`ZONED DATETIME`*	`<date>T<time><timezone>`

*When `date` and `time` are combined, `date` must be complete; i.e. fully identify a particular day.

Specifying dates

Component	Format	Description
Year	`YYYY`	Specified with at least four digits (special rules apply in certain cases).
Month	`MM`	Specified with a double digit number from `01` to `12`.
Week	`ww`	Always prefixed with `W` and specified with a double digit number from `01` to `53`.
Quarter	`q`	Always prefixed with `Q` and specified with a single digit number from `1` to `4`.
Day of the month	`DD`	Specified with a double digit number from `01` to `31`.

Component	Format	Description
Day of the week	`D`	Specified with a single digit number from `1` to `7`.
Day of the quarter	`DD`	Specified with a double digit number from `01` to `92`.
Ordinal day of the year	`DDD`	Specified with a triple digit number from `001` to `366`.

If the year is before `0000` or after `9999`, the following additional rules apply:

- Minus sign, `-` must prefix any year before `0000`, (e.g. `-3000-01-01`).
- Plus sign, `+` must prefix any year after `9999`, (e.g. `+11000-01-01`).
- The year must be separated with `-` from the next component:
  - if the next component is month, (e.g. `+11000-01`).
  - if the next component is day of the year, (e.g. `+11000-123`).

If the year component is prefixed with either `-` or `+`, and is separated from the next component, `Year` is allowed to contain up to nine digits. Thus, the allowed range of years is between -999,999,999 and +999,999,999. For all other cases, i.e. the year is between `0000` and `9999` (inclusive), `Year` must have exactly four digits (the year component is interpreted as a year of the Common Era (CE)).

The following formats are supported for specifying dates:

Format	Description	Example	Interpretation of example
`YYYY-MM-DD`	Calendar date: `Year-Month-Day`	`2015-07-21`	`2015-07-21`
`YYYYMMDD`	Calendar date: `Year-Month-Day`	`20150721`	`2015-07-21`
`YYYY-MM`	Calendar date: `Year-Month`	`2015-07`	`2015-07-01`
`YYYYMM`	Calendar date: `Year-Month`	`201507`	`2015-07-01`
`YYYY-Www-D`	Week date: `Year-Week-Day`	`2015-W30-2`	`2015-07-21`
`YYYYWwwD`	Week date: `Year-Week-Day`	`2015W302`	`2015-07-21`
`YYYY-Www`	Week date: `Year-Week`	`2015-W30`	`2015-07-20`
`YYYYWww`	Week date: `Year-Week`	`2015W30`	`2015-07-20`
`YYYY-Qq-DD`	Quarter date: `Year-Quarter-Day`	`2015-Q2-60`	`2015-05-30`
`YYYYQqDD`	Quarter date: `Year-Quarter-Day`	`2015Q260`	`2015-05-30`

Format	Description	Example	Interpretation of example
`YYYY-Qq`	Quarter date: `Year-Quarter`	`2015-Q2`	`2015-04-01`
`YYYYQq`	Quarter date: `Year-Quarter`	`2015Q2`	`2015-04-01`
`YYYY-DDD`	Ordinal date: `Year-Day`	`2015-202`	`2015-07-21`
`YYYYDDD`	Ordinal date: `Year-Day`	`2015202`	`2015-07-21`
`YYYY`	Year	`2015`	`2015-01-01`

The smallest components can be omitted. Cypher will assume omitted components to have their lowest possible value. For example, `2013-06` will be interpreted as being the same date as `2013-06-01`.

Specifying times

Component	Format	Description
`Hour`	`HH`	Specified with a double digit number from `00` to `23`.
`Minute`	`MM`	Specified with a double digit number from `00` to `59`.
`Second`	`SS`	Specified with a double digit number from `00` to `59`.
`fraction`	`sssssssss`	Specified with a number from `0` to `999999999`. It is not required to specify leading zeros. `fraction` is an optional, sub-second component of `Second`. This can be separated from `Second` using either a full stop (`.`) or a comma (`,`). The `fraction` is in addition to the two digits of `Second`.

Cypher does not support leap seconds; UTC-SLS (*UTC with Smoothed Leap Seconds*) is used to manage the difference in time between UTC and TAI (*International Atomic Time*).

The following formats are supported for specifying times:

Format	Description	Example	Interpretation of example
`HH:MM:SS.sssssssss`	`Hour:Minute:Second.fraction`	`21:40:32.142`	`21:40:32.142`
`HHMMSS.sssssssss`	`Hour:Minute:Second.fraction`	`214032.142`	`21:40:32.142`
`HH:MM:SS`	`Hour:Minute:Second`	`21:40:32`	`21:40:32.000`

Format	Description	Example	Interpretation of example
`HHMMSS`	`Hour:Minute:Second`	`214032`	`21:40:32.000`
`HH:MM`	`Hour:Minute`	`21:40`	`21:40:00.000`
`HHMM`	`Hour:Minute`	`2140`	`21:40:00.000`
`HH`	`Hour`	`21`	`21:00:00.000`

The smallest components can be omitted. For example, a time may be specified with `Hour` and `Minute`, leaving out `Second` and `fraction`. On the other hand, specifying a time with `Hour` and `Second`, while leaving out `Minute`, is not possible.

Specifying time zones

The time zone is specified in one of the following ways:

- As an offset from UTC.
- Using the `Z` shorthand for the UTC (`±00:00`) time zone.

When specifying a time zone as an offset from UTC, the rules below apply:

- The time zone always starts with either a plus (`+`) or minus (`-`) sign.
  - Positive offsets, i.e. time zones beginning with `+`, denote time zones east of UTC.
  - Negative offsets, i.e. time zones beginning with `-`, denote time zones west of UTC.
- A double-digit hour offset follows the `+`/`-` sign.
- An optional double-digit minute offset follows the hour offset, optionally separated by a colon (`:`).
- The time zone of the International Date Line is denoted either by `+12:00` or `-12:00`, depending on country.

When creating values of the `ZONED DATETIME` temporal instant type, the time zone may also be specified using a named time zone, using the names from the IANA time zone database. This may be provided either in addition to, or in place of the offset. The named time zone is given last and is enclosed in square brackets (`[]`). Should both the offset and the named time zone be provided, the offset must match the named time zone.

The following formats are supported for specifying time zones:

Format	Description	Example	Supported for `ZONED DATETIME`	Supported for `ZONED TIME`
`Z`	UTC	`Z`	✔	✔
`±HH:MM`	`Hour:Minute`	`+09:30`	✔	✔
`±HH:MM[ZoneName]`	`Hour:Minute[ZoneName]`	`+08:45[Australia/Eucla]`	✔	
`±HHMM`	`Hour:Minute`	`+0100`	✔	✔
`±HHMM[ZoneName]`	`Hour:Minute[ZoneName]`	`+0200[Africa/Johannesburg]`	✔	

Format	Description	Example	Supported for `ZONED DATETIME`	Supported for `ZONED TIME`
`±HH`	`Hour`	`-08`	✔	✔
`±HH[ZoneName]`	`Hour[ZoneName]`	`+08[Asia/Singapore]`	✔	
`[ZoneName]`	`[ZoneName]`	`[America/Regina]`	✔	

Components of temporal instants

Components of temporal instant values can be accessed as properties.

Components of temporal instant values and where they are supported

Component	Description	Type	Range/Format	`DATE`	`ZONED DATETIME`	`LOCAL DATETIME`	`ZONED TIME`	`LOCAL TIME`
`instant.year`	The `year` component represents the astronomical year number of the instant.[2]	`INTEGER`	At least 4 digits. For more information, see the rules for using the `Year` component.	✔	✔	✔		
`instant.quarter`	The *quarter-of-the-year* component.	`INTEGER`	`1` to `4`.	✔	✔	✔		
`instant.month`	The *month-of-the-year* component.	`INTEGER`	`1` to `12`.	✔	✔	✔		
`instant.week`	The *week-of-the-year* component.[3]	`INTEGER`	`1` to `53`.	✔	✔	✔		
`instant.weekYear`	The *year* that the *week-of-year* component belongs to.[4]	`INTEGER`	At least 4 digits. For more information, see the rules for using the `Year` component.	✔	✔	✔		
`instant.dayOfQuarter`	The *day-of-the-quarter* component.	`INTEGER`	`1` to `92`.	✔	✔	✔		
`instant.quarterDay`	The *day-of-the-quarter* component (alias for `instant.dayOfQuarter`).	`INTEGER`	`1` to `92`.	✔	✔	✔		

Component	Description	Type	Range/Format	`DATE`	`ZONED DATETIME`	`LOCAL DATETIME`	`ZONED TIME`	`LOCAL TIME`
`instant.day`	The *day-of-the-month* component.	`INTEGER`	`1` to `31`.	✔	✔	✔		
`instant.ordinalDay`	The *day-of-the-year* component.	`INTEGER`	`1` to `366`.	✔	✔	✔		
`instant.dayOfWeek`	The *day-of-the-week* component (the first day of the week is *Monday*).	`INTEGER`	`1` to `7`.	✔	✔	✔		
`instant.weekDay`	The *day-of-the-week* component (alias for `instant.dayOfWeek`).	`INTEGER`	`1` to `7`.	✔	✔	✔		
`instant.hour`	The *hour* component.	`INTEGER`	`0` to `23`.		✔	✔	✔	✔
`instant.minute`	The *minute* component.	`INTEGER`	`0` to `59`.		✔	✔	✔	✔
`instant.second`	The *second* component.[5]	`INTEGER`	`0` to `59`.		✔	✔	✔	✔
`instant.millisecond`	The *millisecond* component.	`INTEGER`	`0` to `999`.		✔	✔	✔	✔
`instant.microsecond`	The *microsecond* component.	`INTEGER`	`0` to `999999`.		✔	✔	✔	✔
`instant.nanosecond`	The *nanosecond* component.	`INTEGER`	`0` to `999999999`.		✔	✔	✔	✔
`instant.timezone`	The *timezone* component.	`STRING`	Depending on how the time zone was specified, this is either a time zone name or an offset from UTC in the format `±HHMM`.		✔		✔	
`instant.offset`	The *timezone offset*.	`STRING`	In the format `±HHMM`.		✔		✔	
`instant.offsetMinutes`	The *timezone offset in minutes*.	`INTEGER`	`-1080` to `+1080`.		✔		✔	

Component	Description	Type	Range/Format	DATE	ZONED DATETIME	LOCAL DATETIME	ZONED TIME	LOCAL TIME
instant.offsetSeconds	The timezone offset in seconds.	INTEGER	-64800 to +64800.		✔		✔	
instant.epochMillis	The number of milliseconds between 1970-01-01T00:00:00+0000 and the instant.[6]	INTEGER	Positive for instants after and negative for instants before 1970-01-01T00:00:00+0000.		✔			
instant.epochSeconds	The number of seconds between 1970-01-01T00:00:00+0000 and the instant.[7]	INTEGER	Positive for instants after and negative for instants before 1970-01-01T00:00:00+0000.		✔			

Examples

To work with a particular temporal instant type, its corresponding function must be used. For example, in order to create a property value of type ZONED DATETIME, the datetime() function must be used.

For specific examples, see:

- DATE
- LOCAL TIME
- ZONED TIME
- LOCAL DATETIME
- ZONED DATETIME
- Truncating temporal values

DATE

To work with DATE values, including creating, parsing, and extracting components, use the date() function.

Example 113. DATE

Create a DATE property value

```
CREATE (n:Label)
SET n.date = date("2025-02-18")
RETURN n.date AS date, valueType(n.date) AS temporalValueType
```

Result

date	temporalValueType
2025-02-18	"DATE NOT NULL"
Rows: 1	

Create a `DATE` property value using components

```
CREATE (n:Label)
SET n.date = date({year: 2025, month: 2, day: 18})
RETURN n.date AS date, valueType(n.date) AS temporalValueType
```

Result

date	temporalValueType
2025-02-18	"DATE NOT NULL"
Rows: 1	

Parse a `DATE` using the week date format:

```
RETURN date('+2015-W13-4') AS theDate
```

Result

theDate
2015-03-26
Rows: 1

Get the components of a `DATE` value

```
WITH date({year: 1984, month: 10, day: 11}) AS d
RETURN d.year, d.quarter, d.month, d.week, d.weekYear, d.day, d.ordinalDay, d.dayOfWeek,
d.dayOfQuarter
```

Result

d.year	d.quarter	d.month	d.week	d.weekYear	d.day	d.ordinalDay	d.dayOfWeek	d.dayOfQuarter
1984	4	10	41	1984	11	285	4	11
Rows: 1								

`LOCAL TIME`

To work with `LOCAL TIME` values, including creating, parsing, and extracting components, use the `localtime()` function.

Example 114. `LOCAL TIME`

Create a `LOCAL TIME` *property value*

```
CREATE (n:Label)
SET n.localTime = localtime("12:34:56.789")
RETURN n.localTime AS localTime, valueType(n.localTime) AS temporalValueType
```

Result

localTime	temporalValueType
12:34:56.789	"LOCAL TIME NOT NULL"
Rows: 1	

Create a `LOCAL TIME` *property value using components*

```
CREATE (n:Label)
SET n.localTime = localtime({hour: 12, minute: 34, second: 56, millisecond: 789})
RETURN n.localTime AS localTime, valueType(n.localTime) AS temporalValueType
```

Result

localTime	temporalValueType
12:34:56.789	"LOCAL TIME NOT NULL"
Rows: 1	

Get the components of a `LOCAL TIME` *value*

```
WITH localtime({hour: 12, minute: 34, second: 56, millisecond: 789}) AS t
RETURN t.hour, t.minute, t.second, t.millisecond
```

Result

t.hour	t.minute	t.second	t.millisecond
12	34	56	789
Rows: 1			

`ZONED TIME`

To work with `ZONED TIME` values, including creating, parsing, and extracting components, use the `time()` function.

Example 115. `ZONED TIME`

Create a `ZONED TIME` *property value with an offset*

```
CREATE (n:Label)
SET n.zonedTime = time("12:34:56.789+02:00")
RETURN n.zonedTime AS zonedTime, valueType(n.zonedTime) AS temporalValueType
```

Result

zonedTime	temporalValueType
12:34:56.789+02:00	"ZONED TIME NOT NULL"

Rows: 1

Create a `ZONED TIME` property value with components and a time zone

```
CREATE (n:Label)
SET n.zonedTime = time({hour: 12, minute: 34, second: 56, millisecond: 789, timezone:
'Europe/Stockholm'})
RETURN n.zonedTime AS zonedTime, valueType(n.zonedTime) AS temporalValueType
```

Result

time	temporalValueType
12:34:56.789+01:00	"ZONED TIME NOT NULL"

Rows: 1

Get the components of a `ZONED TIME` value

```
WITH time("12:34:56.789+02:00") AS t
RETURN t.hour, t.minute, t.second, t.millisecond, t.offset
```

Result

t.hour	t.minute	t.second	t.millisecond	t.offset
12	34	56	789	"+02:00"

Rows: 1

`LOCAL DATETIME`

To work with `LOCAL DATETIME` values, including creating, parsing, and extracting components, use the `localdatetime()` function.

Example 116. `LOCAL DATETIME`

Create a `LOCAL DATETIME` property value using the ordinal date format:

```
CREATE (n:Label)
SET n.localDateTime = localdatetime("2025-02-18T12:34:56")
RETURN n.localDateTime AS localDateTime, valueType(n.localDateTime) AS temporalValueType
```

Result

localDateTime	temporalValueType
2025-02-18T12:34:56	"LOCAL DATETIME NOT NULL"

localDateTime	temporalValueType
Rows: 1	

Create a `LOCAL DATETIME` property value using temporal components

```
CREATE (n:Label)
SET n.localDateTime = localdatetime({year: 2025, month: 2, day: 18, hour: 12, minute: 34, second: 56,
millisecond: 789})
RETURN n.localDateTime AS localDateTime, valueType(n.localDateTime) AS temporalValueType
```

Result

localDatetime	temporalValueType
2025-02-18T12:34:56.789	"LOCAL DATETIME NOT NULL"
Rows: 1	

Parse a `LOCAL DATETIME` using the ordinal date format:

```
RETURN localdatetime('2015185T19:32:24') AS theLocalDateTime
```

Result

theLocalDateTime
2015-07-04T19:32:24
Rows: 1

Get the components of a `LOCAL DATETIME` value

```
WITH localdatetime({year: 2025, month: 2, day: 19, hour: 12, minute: 34, second: 56, millisecond:
789}) AS t
RETURN t.year, t.month, t.day, t.hour, t.minute, t.second, t.millisecond
```

Result

t.year	t.month	t.day	t.hour	t.minute	t.second	t.millisecond
2025	2	19	12	34	56	789
Rows: 1						

`ZONED DATETIME`

To work with `ZONED DATETIME` values, including creating, parsing, and extracting components, use the `datetime()` function.

Example 117. `ZONED DATETIME`

Create a `ZONED DATETIME` property value using calendar date format

```
CREATE (n:Label)
SET n.zonedDateTime = datetime("2025-02-18T12:34:56.789+02:00")
RETURN n.zonedDateTime AS zonedDateTime, valueType(n.zonedDateTime) AS temporalValueType
```

Result

zonedDateTime	temporalValueType
2025-02-18T12:34:56.789+02:00	"ZONED DATETIME NOT NULL"

Rows: 1

Create a `ZONED DATETIME` property value using temporal components

```
CREATE (n:Label)
SET n.zonedDateTime = datetime({year: 2025, month: 2, day: 18, hour: 12, minute: 34, second: 56,
millisecond: 789, timezone: 'Europe/Stockholm'})
RETURN n.zonedDateTime, valueType(n.zonedDateTime) AS temporalValueType
```

Result

zonedDateTime	temporalValueType
2025-02-18T12:34:56.789+01:00[Europe/Stockholm]	"ZONED DATETIME NOT NULL"

Rows: 1

Create a `ZONED DATETIME` property value using a timezone

```
CREATE (n:Label)
SET n.zonedDateTime = datetime({timezone: 'Europe/Stockholm'})
RETURN n.zonedDateTime AS zonedDateTime, valueType(n.zonedDateTime) AS temporalValueType
```

Result

zonedDateTime	temporalValueType
2025-02-18T15:22:48.227+01:00[Europe/Stockholm]	"ZONED DATETIME NOT NULL"

Rows: 1

Parse a `ZONED DATETIME` using the calendar date format

```
RETURN datetime('2015-06-24T12:50:35.556+0100') AS theDateTime
```

Result

theDateTime
2015-06-24T12:50:35.556+01:00

Rows: 1

Get the date-related components of a `ZONED DATETIME` value:

```
WITH datetime({
  year: 1984, month: 11, day: 11,
  hour: 12, minute: 31, second: 14, nanosecond: 645876123,
  timezone: 'Europe/Stockholm'
}) AS d
RETURN d.year, d.quarter, d.month, d.week, d.weekYear, d.day, d.ordinalDay, d.dayOfWeek,
d.dayOfQuarter
```

Result

d.year	d.quarter	d.month	d.week	d.weekYear	d.day	d.ordinalDay	d.dayOfWeek	d.dayOfQuarter
1984	4	11	45	1984	11	316	7	42

Rows: 1

Get the time-related components of a `ZONED DATETIME` value:

```
WITH datetime({
  year: 1984, month: 11, day: 11,
  hour: 12, minute: 31, second: 14, nanosecond: 645876123,
  timezone: 'Europe/Stockholm'
}) AS d
RETURN d.hour, d.minute, d.second, d.millisecond, d.microsecond, d.nanosecond
```

Result

d.hour	d.minute	d.second	d.millisecond	d.microsecond	d.nanosecond
12	31	14	645	645876	645876123

Rows: 1

Get the epoch time and timezone-related components of a `ZONED DATETIME` value:

```
WITH datetime({
  year: 1984, month: 11, day: 11,
  hour: 12, minute: 31, second: 14, nanosecond: 645876123,
  timezone: 'Europe/Stockholm'
}) AS d
RETURN d.timezone, d.offset, d.offsetMinutes, d.epochSeconds, d.epochMillis
```

Result

d.timezone	d.offset	d.offsetMinutes	d.epochSeconds	d.epochMillis
"Europe/Stockholm"	"+01:00"	60	469020674	469020674645

Rows: 1

Truncating temporal values

The truncate functions in Neo4j allow you to reduce the precision of temporal values by truncating them to a specified component such as `year`, `month`, or `second`.

Example 118. Truncate `DATE` values

To truncate `DATE` values, use the `date.truncate()` function.

Get the first day of the current year:

```
RETURN date.truncate('year') AS firstDay
```

Result

firstDay
2022-01-01
Rows: 1

Get the date of the Thursday in the week of a specific date:

```
RETURN date.truncate('week', date('2019-10-01'), {dayOfWeek: 4}) AS thursday
```

Result

thursday
2019-10-03
Rows: 1

Example 119. Truncate `LOCAL TIME` values

To truncate `LOCAL TIME` values, use the `localtime.truncate()` function.

Get the start of the current minute for a specific `LOCAL TIME` value

```
RETURN localtime.truncate('minute', localtime("12:34:56.789")) AS truncatedMinute
```

Result

truncatedMinute
12:34
Rows: 1

Get the start of the current second using the system's `LOCAL TIME`

```
RETURN localtime.truncate('second') AS currentSecond
```

Result

currentSecond
10:12:28

currentSecond
Rows: 1

Example 120. Truncate `ZONED TIME` values

To truncate `ZONED TIME` values, use the `time.truncate()` function.

Get the start of the current minute for a specific `ZONED TIME` value

```
RETURN time.truncate('minute', time("12:34:56.789+02:00")) AS truncatedMinute
```

Result

truncatedMinute
12:34+02:00
Rows: 1

Get the current `ZONED TIME` truncated to the nearest minute

```
RETURN time.truncate('minute', time()) AS currentTime
```

Result

truncatedTime
10:20Z
Rows: 1

Example 121. Truncate `LOCAL DATETIME` values

To truncate `LOCAL DATETIME` values, use the `localdatetime.truncate()` function.

Get the start of the hour for a specific `LOCAL DATETIME`

```
RETURN localdatetime.truncate('hour', localdatetime("2025-02-18T12:34:56.789")) AS truncatedHour
```

Result

truncatedHour
2025-02-18T12:00
Rows: 1

Get the start of the current month in `LOCAL DATETIME`

```
RETURN localdatetime.truncate('month') AS truncatedMonth
```

Result

truncatedMonth
2025-02-01T00:00
Rows: 1

Example 122. Truncate `ZONED DATETIME` values

To truncate `ZONED DATETIME` values, use the `datetime.truncate()` function.

Get the start of the minute for a specific `ZONED DATETIME`

```
RETURN datetime.truncate('minute', datetime("2025-02-18T12:34:56.789+02:00")) AS
truncatedZonedDateTime
```

Result

truncatedZonedDateTime
2025-02-18T12:34+02:00
Rows: 1

Get the start of Wednesday for the current week in `ZONED DATETIME`

```
RETURN datetime.truncate('week', datetime(), {dayOfWeek: 3}) AS startOfWednesday
```

Result

startOfWednesday
2025-02-19T00:00Z
Rows: 1

Durations

Specifying durations

A `DURATION` represents a temporal amount, capturing the difference in time between two instants, and can be negative.

The specification of a `DURATION` is prefixed with a `P`, and can use either a *unit-based form* or a *date-and-time-based form*:

- Unit-based form: `P[nY][nM][nW][nD][T[nH][nM][nS]]`
  - The square brackets (`[]`) denote an optional component (components with a zero value may be omitted).

- The `n` denotes a numeric value within the bounds of a 64-bit integer.
- The value of the last — and smallest — component may contain a decimal fraction.
- Each component must be suffixed by a component identifier denoting the unit.
- The unit-based form uses `M` as a suffix for both months and minutes. Therefore, time parts must always be preceded with `T`, even when no components of the date part are given.
- The maximum total length of a duration is bounded by the number of seconds that can be held in a 64-bit integer.

- Date-and-time-based form: `P<date>T<time>`.
  - Unlike the unit-based form, this form requires each component to be within the bounds of a valid `LOCAL DATETIME`.

The following table lists the component identifiers for the unit-based form:

Component identifier	Description	Comments
`Y`	Years	
`M`	Months	Must be specified before `T`.
`W`	Weeks	
`D`	Days	
`H`	Hours	
`M`	Minutes	Must be specified after `T`.
`S`	Seconds	

Components of durations

A `DURATION` can have several components, each categorized into *Months*, *Days*, and *Seconds* groups.

Components of `DURATION` values are truncated within their component groups as follows:

First order `DURATION` *components*

Component Group	Component	Description	Type	Details
Months	`duration.years`	The total number of *years*.	`INTEGER`	Each set of `4` *quarters* is counted as `1` *year*; each set of `12` *months* is counted as `1` *year*.
	`duration.quarters`	The total number of *quarters*.	`INTEGER`	Each *year* is counted as `4` *quarters*; each set of `3` *months* is counted as `1` *quarter*.
	`duration.months`	The total number of *months*.	`INTEGER`	Each *year* is counted as `12` *months*; each_quarter_ is counted as `3` *months*.
Days	`duration.weeks`	The total number of *weeks*.	`INTEGER`	Each set of `7` *days* is counted as `1` *week*.
	`duration.days`	The total number of *days*.	`INTEGER`	Each *week* is counted as `7` *days*.
Seconds	`duration.hours`	The total number of *hours*.	`INTEGER`	Each set of `60` *minutes* is counted as `1` *hour*; each set of `3600` *seconds* is counted as `1` *hour*.
	`duration.minutes`	The total number of *minutes*.	`INTEGER`	Each *hour* is counted as `60` *minutes*; each set of `60` *seconds* is counted as `1` *minute*.
	`duration.seconds`	The total number of *seconds*.	`INTEGER`	Each *hour* is counted as `3600` *seconds*; each *minute* is counted as `60` *seconds*.
	`duration.milliseconds`	The total number of *milliseconds*	`INTEGER`	Each set of `1000` *milliseconds* is counted as `1` *second*.
	`duration.microseconds`	The total number of *microseconds*.	`INTEGER`	Each *millisecond* is counted as `1000` *microseconds*.
	`duration.nanoseconds`	The total number of *nanoseconds*.	`INTEGER`	Each *microsecond* is counted as `1000` *nanoseconds*.

Note:

Please note that:

- Cypher uses UTC-SLS when handling leap seconds.
- There are not always `24` *hours* in `1` *day*; when switching to/from daylight savings time, a *day* can have `23` or `25` *hours*.

- There are not always the same number of days in a *month*.
- Due to leap years, there are not always the same number of days in a *year*.

It is also possible to access the second order components of a component group bounded by the first order component of the group:

Second order `DURATION` *components*

Component	Component Group	Description	Type
`duration.quartersOfYear`	Months	The number of *quarters* in the group that do not make a whole *year*.	`INTEGER`
`duration.monthsOfYear`	Months	The number of *months* in the group that do not make a whole *year*.	`INTEGER`
`duration.monthsOfQuarter`	Months	The number of *months* in the group that do not make a whole *quarter*.	`INTEGER`
`duration.daysOfWeek`	Days	The number of *days* in the group that do not make a whole *week*.	`INTEGER`
`duration.minutesOfHour`	Seconds	The number of *minutes* in the group that do not make a whole *hour*.	`INTEGER`
`duration.secondsOfMinute`	Seconds	The number of *seconds* in the group that do not make a whole *minute*.	`INTEGER`
`duration.millisecondsOfSecond`	Seconds	The number of *milliseconds* in the group that do not make a whole *second*.	`INTEGER`
`duration.microsecondsOfSecond`	Seconds	The number of *microseconds* in the group that do not make a whole *second*.	`INTEGER`
`duration.nanosecondsOfSecond`	Seconds	The number of *nanoseconds* in the group that do not make a whole *second*	`INTEGER`

Examples

Below are examples of parsing durations using the `duration()` function. More information can be found here.

Example 123. Return a duration of `14` *days,* `16` *hours, and* `12` *minutes*

Query

```
RETURN duration('P14DT16H12M') AS theDuration
```

Result

theDuration
P14DT16H12M
Rows: 1

Example 124. Return a duration of `5` months, `1` day, and `12` hours

Query

```
RETURN duration('P5M1.5D') AS theDuration
```

Result

theDuration
P5M1DT12H
Rows: 1

Example 125. Return a duration of `45` seconds

Query

```
RETURN duration('PT0.75M') AS theDuration
```

Result

theDuration
PT45S
Rows: 1

Example 126. Return a duration of `2` weeks, `3` days, and `12` hours

Query

```
RETURN duration('P2.5W') AS theDuration
```

Result

theDuration
P17DT12H
Rows: 1

Example 127. Get the month-based components of a `DURATION` value

Query

```
WITH duration({years: 1, months: 5, days: 111, minutes: 42}) AS d
RETURN d.years, d.quarters, d.quartersOfYear, d.months, d.monthsOfYear, d.monthsOfQuarter
```

Result

d.years	d.quarters	d.quartersOfYear	d.months	d.monthsOfYear	d.monthsOfQuarter
1	5	1	17	5	2

Rows: 1

`d.quarters` has a value of `5` because the year of the duration has four quarters and there is another quarter in the five months. `d.months` has a value of `17` because it adds the 12 months in the year of the duration to the five months. `d.quartersOfYear` is the remaining quarter, counting towards the next full year. Similarly, `d.monthsOfYear` and `d.monthsOfQuarter` count towards the next full year and quarter respectively. See tables *First order* `DURATION` *components* and *Second order* `DURATION` *components* in Components of durations.

Example 128. Get the days-based components of a `DURATION` value

Query

```
WITH duration({months: 5, days: 25, hours: 1}) AS d
RETURN d.weeks, d.days, d.daysOfWeek
```

Result

d.weeks	d.days	d.daysOfWeek
3	25	4

Rows: 1

`d.weeks` has a value of 3 because the 25 days from the query are three full weeks (or 21 days). `d.daysOfWeek` are the remaining days, counting towards the next full week. See tables *First order* `DURATION` *components* and *Second order* `DURATION` *components* in Components of durations.

Example 129. Get the first order seconds-based components of a `DURATION` value

Query

```
WITH duration({
  years: 1, months:1, days:1, hours: 1,
  minutes: 1, seconds: 1, nanoseconds: 111111111
}) AS d
RETURN d.hours, d.minutes, d.seconds, d.milliseconds, d.microseconds, d.nanoseconds
```

Result

d.hours	d.minutes	d.seconds	d.milliseconds	d.microseconds	d.nanoseconds
1	61	3661	3661111	3661111111	3661111111111

Rows: 1

`d.minutes` is the sum of 60 minutes of the hour and the one minute from the query as both `duration.hours` and `duration.minutes` are both seconds-based components. Similarly, `d.seconds`, `d.milliseconds`, `d.microseconds` and `d.nanoseconds` are sum values of the relevant seconds-based components from the query.

`d.hours` does not take the day from the query into account because `duration.days` is a days-based component.

See table *First order* `DURATION` *components* in Components of durations.

Example 130. Get the second order seconds-based components of a `DURATION` *value*

Query

```
WITH duration({
  years: 1, months:1, days:1,
  hours: 1, minutes: 1, seconds: 1, nanoseconds: 111111111
}) AS d
RETURN d.minutesOfHour, d.secondsOfMinute, d.millisecondsOfSecond, d.microsecondsOfSecond,
d.nanosecondsOfSecond
```

Result

d.minutesOfHour	d.secondsOfMinute	d.millisecondsOfSecond	d.microsecondsOfSecond	d.nanosecondsOfSecond
1	1	111	111111	111111111

Rows: 1

The returned values all count towards the next full hour, minute or second respectively. For example, `d.microsecondsOfSecond` has a value of `111111` because it is the `111111111` nanoseconds from the query in microseconds (rounded down) but it is not another full second.

See table *Second order* `DURATION` *components* in Components of durations.

Example 131. Create a duration representing 1.5 days

Query

```
RETURN duration({days: 1, hours: 12}) AS theDuration
```

Result

theDuration
P1DT12H
Rows: 1

Example 132. Compute the `DURATION` between two temporal instants

Query

```
RETURN duration.between(date('1984-10-11'), date('2015-06-24')) AS theDuration
```

Result

theDuration
P30Y8M13D
Rows: 1

Example 133. Compute the number of days between two `DATE` values

Query

```
RETURN duration.inDays(date('2014-10-11'), date('2015-08-06')) AS theDuration
```

Result

theDuration
P299D
Rows: 1

Example 134. Get the `DATE` of the last day of the next month

Query

```
RETURN date.truncate('month', date() + duration('P2M')) - duration('P1D') AS lastDay
```

Result

lastDay
2022-07-31
Rows: 1

Example 135. Add a `DURATION` to a `DATE`

Query

```
RETURN time('13:42:19') + duration({days: 1, hours: 12}) AS theTime
```

Result

theTime
01:42:19.000000000+00:00
Rows: 1

Example 136. Add two `DURATION` values

Query

```
RETURN duration({days: 2, hours: 7}) + duration({months: 1, hours: 18}) AS theDuration
```

Result

theDuration
P1M2DT25H
Rows: 1

Example 137. Multiply a `DURATION` by a number

Query

```
RETURN duration({hours: 5, minutes: 21}) * 14 AS theDuration
```

Result

theDuration
PT74H54M
Rows: 1

Example 138. Divide a `DURATION` by a number

Query

```
RETURN duration({hours: 3, minutes: 16}) / 2 AS theDuration
```

Result

theDuration
PT1H38M
Rows: 1

Example 139. Examine whether two instants are less than one day apart

Query

```
WITH
  datetime('2015-07-21T21:40:32.142+0100') AS date1,
  datetime('2015-07-21T17:12:56.333+0100') AS date2
RETURN
CASE
  WHEN date1 < date2 THEN date1 + duration("P1D") > date2
  ELSE date2 + duration("P1D") > date1
END AS lessThanOneDayApart
```

Result

lessThanOneDayApart
true
Rows: 1

Example 140. Return the abbreviated name of the current month

Query

```
RETURN ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"][date
().month-1] AS month
```

Result

month
"Jun"
Rows: 1

Temporal indexing

All temporal types can be indexed, and thereby support exact lookups for equality predicates. Indexes for temporal instant types additionally support range lookups.

Formatting temporal values

Using Cypher's `format()`, you can create dynamically formatted string representations of temporal instance and duration types.

Example 141. String representations of an instance type

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "MM/dd/yyyy") AS US, format(dt, "dd/MM/yyyy") AS EU
```

Result

US	EU
"11/18/1986"	"18/11/1986"
Rows: 1	

Example 142. A string representation of a duration type

Query

```
WITH duration({years: 1, months: 4, weeks: 3, days: 4, hours: 5, minutes: 6, seconds: 7,
milliseconds: 8,  microseconds: 9, nanoseconds: 10}) AS d
RETURN format(d, "y 'years' q 'quarters' M 'months' w 'weeks' d 'days' h 'hours' m 'minutes' s
'seconds' N 'nanos'") AS x
```

Result

x
"1 years 1 quarters 1 months 3 weeks 4 days 5 hours 6 minutes 7 seconds 8009010 nanos"
Rows: 1

For more examples and details on how to construct string patterns, see instance types and duration types.

Spatial values

Cypher has built-in support for handling spatial values (`POINT` values), which can be stored as properties on nodes and relationships in Neo4j databases.

This section begins with an explanation of the `POINT` type. It then proceeds to discuss Cypher's support of Coordinate Reference Systems, and how to work with spatial instants in Cypher, including how spatial point instants work with Cypher indexing. Finally, it briefly explains comparability and orderability with regard to spatial instants.

Note:

For more information about spatial functions, allowing for the creation and manipulation of spatial values, see the section on Spatial functions.

For more information about the comparison and ordering of spatial values, see the section

on Ordering spatial and temporal values.

The POINT type

Neo4j supports the `POINT` type for values of spatial geometry.

Values with the `POINT` type have the following characteristics:

- Each point can have either 2 or 3 dimensions. This means it contains either 2 or 3 64-bit `FLOAT` values, which together are called the *Coordinate*.
- Each point will also be associated with a specific Coordinate Reference System (CRS) that determines the meaning of the values in the *Coordinate*.
- Instances of `POINT` and `LIST<POINT>` can be assigned to node and relationship properties.
- Nodes and relationships with `POINT` or `LIST<POINT>` properties can be indexed using a point index. This is true for all CRSs (and for both 2D and 3D).
- The distance function will work on points in all CRS and in both 2D and 3D, but only if the two points have the same CRS (and therefore also same dimension).

Coordinate Reference Systems

Four Coordinate Reference Systems (CRS) are supported, each of which falls within one of two types: *geographic coordinates*, modeling points on the earth, or *Cartesian coordinates*, modeling points in euclidean space:

Data within different coordinate systems are entirely incomparable, and cannot be implicitly converted from one to the other. This is true even if they are both Cartesian or both geographic but of a different dimension. For example, if you search for 3D points using a 2D range, you will get no results. However, they can be ordered, as discussed in more detail in the section about Ordering spatial and temporal values.

Geographic coordinate reference systems

Two Geographic Coordinate Reference Systems (CRS) are supported, modeling points on the earth:

- WGS 84 2D
  - A 2D geographic point in the *WGS 84* CRS is specified in one of two ways:
    - `longitude` and `latitude` (if these are specified, and the `crs` is not, then the `crs` is assumed to be `WGS-84`).
    - `x` and `y` (in this case the `crs` must be specified, or will be assumed to be Cartesian).
  - Specifying this CRS can be done using either the name 'wgs-84' or the SRID 4326 as described in point() - WGS 84 2D.
- WGS 84 3D
  - A 3D geographic point in the *WGS 84* CRS is specified one of in two ways:
    - `longitude`, `latitude` and either `height` or `z` (if these are specified, and the `crs` is not, then the `crs` is assumed to be `WGS-84-3D`).

        - `x`, `y` and `z` (in this case the `crs` must be specified, or will be assumed to be Cartesian-3D).
    - Specifying this CRS can be done using either the name 'wgs-84-3d' or the SRID 4979 as described in point() - WGS 84 3D.

Converting coordinate units

The units of the `latitude` and `longitude` fields are in decimal degrees, and need to be specified as floating point numbers using Cypher literals. It is not possible to use any other format, such as 'degrees, minutes, seconds'. The units of the `height` field are in meters. When geographic points are passed to the distance function, the result will always be in meters. If the coordinates are in any other format or unit than those supported, it is necessary to explicitly convert them.

For example, if the incoming `$height` is a `STRING` field in kilometers, it would be necessary to add `height: toFloat($height) * 1000` to the query. Likewise if the results of the `distance` function are expected to be returned in kilometers, an explicit conversion is required. The below query is an example of this conversion:

Query

```
WITH
  point({latitude: toFloat('13.43'), longitude: toFloat('56.21')}) AS p1,
  point({latitude: toFloat('13.10'), longitude: toFloat('56.41')}) AS p2
RETURN toInteger(point.distance(p1, p2)/1000) AS km
```

Result

km
42
Rows: 1

Cartesian coordinate reference systems

Two Cartesian Coordinate Reference Systems (CRS) are supported, modeling points in euclidean space:

- **Cartesian 2D**
    - A 2D point in the *Cartesian* CRS is specified with a map containing `x` and `y` coordinate values
    - Specifying this CRS can be done using either the name 'cartesian' or the SRID 7203 as described in point() - Cartesian 2D
- **Cartesian 3D**
    - A 3D point in the *Cartesian* CRS is specified with a map containing `x`, `y` and `z` coordinate values
    - Specifying this CRS can be done using either the name 'cartesian-3d' or the SRID 9157 as described in point() - Cartesian 3D)

The units of the `x`, `y`, and `z` fields are unspecified. This means that when two Cartesian points are passed to the `distance` function, the resulting value will be in the same units as the original coordinates. This is true for both 2D and 3D points, as the *Pythagoras* equation used is generalized to any number of dimensions. However, just as you cannot compare geographic points to Cartesian points, you cannot calculate the distance between a 2D point and a 3D point. If you need to do that, explicitly transform the one type into

the other. For example:

Query

```
WITH
  point({x: 3, y: 0}) AS p2d,
  point({x: 0, y: 4, z: 1}) AS p3d
RETURN
  point.distance(p2d, p3d) AS bad,
  point.distance(p2d, point({x: p3d.x, y: p3d.y})) AS good
```

Result

bad	good
<null>	5.0
Rows: 1	

Spatial instants

All `POINT` types are created from two components:

- The *Coordinate* containing either 2 or 3 `FLOAT` values (64-bit).
- The Coordinate Reference System (or CRS) defining the meaning (and possibly units) of the values in the *Coordinate*.

For most use cases, it is not necessary to specify the CRS explicitly as it will be deduced from the keys used to specify the coordinate. Two rules are applied to deduce the CRS from the coordinate:

- Choice of keys:
  - If the coordinate is specified using the keys `latitude` and `longitude` the CRS will be assumed to be *Geographic* and therefor either `WGS-84` or `WGS-84-3D`.
  - If instead `x` and `y` are used, then the default CRS would be `Cartesian` or `Cartesian-3D`.
- Number of dimensions:
  - If there are 2 dimensions in the coordinate, `x` & `y` or `longitude` & `latitude` the CRS will be a 2D CRS.
  - If there is a third dimensions in the coordinate, `z` or `height` the CRS will be a 3D CRS.

All fields are provided to the `point` function in the form of a map of explicitly named arguments. Neo4j does not support an ordered list of coordinate fields because of the contradictory conventions between geographic and cartesian coordinates, where geographic coordinates normally list `y` before `x` (`latitude` before `longitude`).

The following query which returns points created in each of the four supported CRSs. Take particular note of the order and keys of the coordinates in the original `point` function, and how those values are displayed in the results:

Query

```
RETURN
  point({x: 3, y: 0}) AS cartesian_2d,
  point({x: 0, y: 4, z: 1}) AS cartesian_3d,
  point({latitude: 12, longitude: 56}) AS geo_2d,
  point({latitude: 12, longitude: 56, height: 1000}) AS geo_3d
```

Result

cartesian_2d	cartesian_3d	geo_2d	geo_3d
`point({srid:7203, x: 3.0, y: 0.0})`	`point({srid:9157, x: 0.0, y: 4.0, z: 1.0})`	`point({srid:4326, x: 56.0, y: 12.0})`	`point({rid:4979, x: 56.0, y: 12.0, z: 1000.0})`
Rows: 1			

For the geographic coordinates, it is important to note that the `latitude` value should always lie in the interval `[-90, 90]`. Any other value outside this range will throw an exception. The `longitude` value should always lie in the interval `[-180, 180]`. Any other value outside this range will be wrapped around to fit in this range. The `height` value and any Cartesian coordinates are not explicitly restricted. Any value within the allowed range of the signed 64-bit floating point type will be accepted.

Components of points

Components of `POINT` values can be accessed as properties.

Components of `POINT` instances and where they are supported

Component	Description	Type	Range/Format	WGS-84	WGS-84-3D	Cartesian	Cartesian-3D
`instant.x`	The first element of the *Coordinate*	`FLOAT`	Number literal, range depends on CRS	✔	✔	✔	✔
`instant.y`	The second element of the *Coordinate*	`FLOAT`	Number literal, range depends on CRS	✔	✔	✔	✔
`instant.z`	The third element of the *Coordinate*	`FLOAT`	Number literal, range depends on CRS		✔		✔

Component	Description	Type	Range/Format	WGS-84	WGS-84-3D	Cartesian	Cartesian-3D
`instant.longitude`	The first element of the Coordinate for geographic CRSs, degrees East of the prime meridian	`FLOAT`	Number literal, `-180.0` to `180.0`	✔	✔		
`instant.latitude`	The second element of the Coordinate for geographic CRS, degrees North of the equator	`FLOAT`	Number literal, `-90.0` to `90.0`	✔	✔		
`instant.height`	The third element of the Coordinate for geographic CRSs, meters above the ellipsoid defined by the datum (WGS-84)	`FLOAT`	Number literal, range limited only by the underlying 64-bit floating point type		✔		
`instant.crs`	The name of the CRS	`STRING`	One of `wgs-84`, `wgs-84-3d`, `cartesian`, `cartesian-3d`	✔	✔	✔	✔
`instant.srid`	The internal Neo4j ID for the CRS	`INTEGER`	One of `4326`, `4979`, `7203`, `9157`	✔	✔	✔	✔

Examples

The following query shows how to extract the components of a *Cartesian 2D* `POINT` value:

Query

```
WITH point({x: 3, y: 4}) AS p
RETURN
  p.x AS x,
  p.y AS y,
```

```
  p.crs AS crs,
  p.srid AS srid
```

Result

x	y	crs	srid
3.0	4.0	"cartesian"	7203

Rows: 1

The following query shows how to extract the components of a *WGS-84 3D* `POINT` value:

Query

```
WITH point({latitude: 3, longitude: 4, height: 4321}) AS p
RETURN
  p.latitude AS latitude,
  p.longitude AS longitude,
  p.height AS height,
  p.x AS x,
  p.y AS y,
  p.z AS z,
  p.crs AS crs,
  p.srid AS srid
```

Result

latitude	longitude	height	x	y	z	crs	srid
3.0	4.0	4321.0	4.0	3.0	4321.0	"wgs-84-3d"	4979

Rows: 1

Spatial values and indexes

If there is a range or point index on a particular node or relationship property, and a spatial point is assigned to that property on a node or relationship, the node or relationship will be indexed.

In a point index, Neo4j uses space filling curves in 2D or 3D over an underlying generalized B+Tree. Point indexes are optimized for distance and bounding box queries. For more information, see Create indexes → Point indexes.

In a range index, the points will be sorted according to their lexicographic ordering per coordinate reference system. For point values, this index has support for equality checks. For more information, see Create indexes → Range indexes.

Comparability and orderability

Cypher does not support comparing spatial values using the inequality operators, `<`, `<=`, `>`, and `>=`. Attempting to do so will return `null`.

To compare spatial points within a specific range, instead use the spatial functions point.distance or point.withinBBox.

Lists

Cypher includes comprehensive support for lists.

For information about the list predicate operator `IN`, which checks for list membership, see Expressions → Predicates → List operators. For information list concatenation (`+` and `||`), list element access and slicing (`[]`), as well as list and pattern comprehensions, see List expressions.

Lists in general

A literal list is created by using brackets and separating the elements in the list with commas.

Query

```
RETURN [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] AS list
```

Result

list
[0,1,2,3,4,5,6,7,8,9]
Rows: 1

A list can consist of different value types.

Query

```
RETURN [0, "hello", 3.14, null] AS list
```

Result

list
[0, "hello", 3.14, null]
Rows: 1

Lists containing `VECTOR` values as nested entries cannot be stored as properties.

Lists are indexed by 0 in Cypher. To access individual elements in a list, use square brackets. This extracts from the start index and up to, but not including, the end index.

For example:

Query

```
WITH [5,1,7] AS list
RETURN list[2]
```

Result

list[2]
7
Rows: 1

List range and size

The below examples use the `range` function to create lists. This function returns a list containing all numbers between given start and end numbers. The range is inclusive in both ends.

Query

```
RETURN range(0, 10)[3] AS element
```

Result

element
3
Rows: 1

It is also possible to use negative numbers, to start from the end of the list instead.

Query

```
RETURN range(0, 10)[-3] AS element
```

Result

element
8
Rows: 1

Finally, it is possible to use ranges inside the brackets to return ranges of the list. The list range operator (`[]`) is inclusive of the first value, but exclusive of the last value.

Query

```
RETURN range(0, 10)[0..3] AS list
```

Result

list
[0,1,2]
Rows: 1

Query

```
RETURN range(0, 10)[0..-5] AS list
```

Result

list
[0,1,2,3,4,5]
Rows: 1

Query

```
RETURN range(0, 10)[-5..] AS list
```

Result

list
[6,7,8,9,10]
Rows: 1

Query

```
RETURN range(0, 10)[..4] AS list
```

Result

list
[0,1,2,3]
Rows: 1

Out-of-bound slices are simply truncated, but out-of-bound single elements return `null`.

Query

```
RETURN range(0, 10)[15] AS list
```

Result

list
<null>
Rows: 1

Query

```
RETURN range(0, 10)[5..15] AS list
```

Result

list
[5,6,7,8,9,10]
Rows: 1

The `size` of a list can be obtained as follows:

Query

```
RETURN size(range(0, 10)[0..3]) AS list
```

Result

list
3
Rows: 1

Storing lists as properties

It is possible to store homogenous lists of simple values as properties.

Allowed - store homogenous list as a property

```
CREATE (n:Label)
SET n.listProperty = [1, 2, 3]
RETURN n.listProperty AS homogenousListProperty
```

Result

homogenousListProperty
[1, 2, 3]
Rows: 1

It is not, however, possible to store heterogeneous lists as properties.

Not allowed - store heterogenous list as a property

```
CREATE (n:Label)
SET n.listProperty = [1, "hello", .45, date()]
RETURN n.listProperty AS heterogenousListProperty
```

GQLSTATUS error chain

22N39: error: data exception - unsupported property value type. Value String("hello") cannot be stored in properties.

22G03: error: data exception - invalid value type

Maps

Cypher supports the construction of maps.

> **Note:**
>
> For information about the property access operators `.` and `[]`, and map comprehension, see Map expressions.
> For information about how the `[]` operator behaves with respect to `null`, see Working with `null` → The `[\` operator and `null`].

Literal maps

The key names in a map must be literals. If returned through an HTTP API call, a JSON object will be returned. If returned in Java, an object of type `java.util.Map<String,Object>` will be returned.

Query

```
RETURN {key: 'Value', listKey: [{inner: 'Map1'}, {inner: 'Map2'}]} AS map
```

Result

map
{'listKey': [{'inner': 'Map1'}, {'inner': 'Map2'}], 'key': 'Value'}
Rows: 1

UUIDs

Introduced in Neo4j 2026.08 | Enterprise Edition

`UUID` values can be created and stored as properties on nodes and relationships.

The UUID type

The `UUID` type is a 128-bit identifier, commonly used to uniquely identify objects within a distributed system. In Cypher, `UUID` values are stored as a dedicated type and can be represented as a hexadecimal string in the format `8-4-4-4-12` (e.g., `550e8400-e29b-41d4-a716-446655440000`).

An example `UUID` value

```
uuid("550e8400-e29b-41d4-a716-446655440000")
```

In this example, the string `"550e8400-e29b-41d4-a716-446655440000"` is a valid representation of a UUID.

> Cypher's `UUID` type does not guarantee a specific UUID version. See the RFC 9562 Universally Unique IDentifiers (UUIDs) documentation for details on the different UUID

versions.

See also Equality, ordering, and comparison of value types → Ordering UUID values.

You can convert `UUID` values into `STRING` values with the `toString()` function.

Valid values

A `UUID` value must follow the standard representation of 32 hexadecimal digits, displayed in five groups separated by hyphens (e.g., `8-4-4-4-12`).

Construct UUID values

To construct a `UUID` value, use the `uuid()` function.

Example 143. Create a `UUID` value from a `STRING` value.

Query

```
RETURN uuid("550e8400-e29b-41d4-a716-446655440000") AS uuid
```

Result

uuid
uuid("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

Example 144. Create a `UUID` value using parameters

Parameters (`STRING`)

```
{
  "uuidAsString": "550e8400-e29b-41d4-a716-446655440000"
}
```

Query

```
RETURN uuid($uuidAsString) AS uuid
```

Result

uuid
uuid("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

Store UUID values as properties

Enterprise Edition

To store a `UUID` value as a node or relationship property, use the `uuid()` function and a write clause.

> Storing `UUID` values requires the database to use block format. This is the default on Aura instances. Community Edition cannot store `UUID` values as properties.

Example 145. Create a node with a `UUID` property

Query

```
CREATE (n:Label {uuidProp: uuid("550e8400-e29b-41d4-a716-446655440000")})
RETURN n.uuidProp AS uuidProp
```

Result

uuidProp
uuid("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

Example 146. Create a node with a `UUID` property using parameters

Parameters

```
{
  "uuidString": "550e8400-e29b-41d4-a716-446655440000"
}
```

Query

```
CREATE (n:Label {uuidProp: uuid($uuidString)})
RETURN n.uuidProp AS uuidProp
```

Result

uuidProp
uuid("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

UUIDs and client libraries (drivers)

Working with uuids via Neo4j's client libraries results in a different behavior depending on the library version.

- **Versions >= 6.2** — UUIDs are fully supported and mapped into client types (see the *Data types* page of each language manual).
- **Versions < 6.2** — UUIDs can be created, attached, and manipulated in queries via Cypher functions, but cannot be returned nor created in the application.
  It is possible to create and store values as shown in the Store UUID values as properties examples, but *returning* a `UUID` results in a placeholder `MAP` value and a warning.

  Result of returning a `UUID` with a driver older than 6.2

```
+-----------------------------------------------------------------------------------------------------
-+
| n.uuid
|
+-----------------------------------------------------------------------------------------------------
-+
| {originalType: "UUID("550e8400-e29b-41d4-a716-446655440000")", reason: "UNKNOWN_TYPE"}
|
+-----------------------------------------------------------------------------------------------------
-+
warn: One or more values returned could not be handled by this version of the driver and were replaced
with placeholder map values. Please upgrade your driver!
03N95 (Neo.ClientNotification.UnknownType)
```

Vectors

Introduced in Neo4j 2025.10 | Enterprise Edition

`VECTOR` values can be created and stored as embedding properties on nodes and relationships, and used for efficient semantic retrieval using vector indexes and the GenAI plugin. `VECTOR` values can also be measured and compared (in terms of similarity, distance, and norm) using vector functions.

The vector type

The `VECTOR` type is a fixed-length, ordered collection of numeric values (`INTEGER` or `FLOAT`) stored as a single unit. The type of a value is defined by:

- **Dimension** — The number of values it contains.
- **Coordinate type** — The data type of the entries, determining precision and storage size.

An example `VECTOR` value

```
vector([1.05, 0.123, 5], 3, FLOAT32)
```

In this example, `[1.05, 0.123, 5]` is the list of values, `3` its dimension, and `FLOAT32` the data type of the individual entries.
Each number in the list can also be seen as a coordinate along one of the vector's dimensions.

You can convert vectors into lists via the functions `toIntegerList()` and `toFloatList()`.

Valid values

- A `VECTOR` value must have a dimension and a coordinate type (see table below for supported types).
- The dimension of a `VECTOR` value must be larger than `0` and less than or equal to `4096`.
- Vectors cannot contain lists as elements.

Supported coordinate types

Default name	Alias
`FLOAT`	`FLOAT64`
`FLOAT32`	
`INTEGER`	`INT`, `INT64`, `INTEGER64`, `SIGNED INTEGER`
`INTEGER32`	`INT32`
`INTEGER16`	`INT16`
`INTEGER8`	`INT8`

Construct vector values

To construct a `VECTOR` value, use the `vector()` function.

Example 147. Create a three-dimensional vector with `INTEGER` values

Query

```
RETURN vector([1, 2, 3], 3, INTEGER) AS vector
```

Result

vector
`vector([1, 2, 3], 3, INTEGER NOT NULL)`
Rows: 1

Example 148. Create a `VECTOR` value using parameters

Parameters (`LIST<INTEGER>`)

```
{
  "integerList": [1, 2, 3, 4, 5]
}
```

Query

```
RETURN vector($integerList, 5, INTEGER8) AS vector
```

Result

vector
vector([1, 2, 3, 4, 5], 5, INTEGER8 NOT NULL)
Rows: 1

Store vector values as properties

Enterprise Edition

To store a `VECTOR` value as a node or relationship property, use the `vector()` function and a write clause.

> Storing `VECTOR` values requires the database to use block format. This is the default on Aura instances. Community Edition cannot store `VECTOR` values as properties. In Community Edition, store embeddings as `LIST<INTEGER | FLOAT>` properties and index them with vector indexes.

Example 149. Create a node and a `VECTOR` property

Query

```
CREATE (n:Label {vectorProp: vector([1,2,3], 3, INTEGER)})
RETURN n.vectorProp AS vectorProp
```

Result

vectorProp
vector([1, 2, 3], 3, INTEGER NOT NULL)
Rows: 1

Example 150. Create a node and a `VECTOR` property using parameters

Parameters

```
{
  "floatList": [1.3, 2.4, 3.1, 4.4, 5.9]
}
```

Query

```
CREATE (n:Label {vectorProp: vector($floatList, 5, FLOAT)})
RETURN n.vectorProp AS vectorProp
```

Result

vectorProp
vector([1.3, 2.4, 3.1, 4.4, 5.9], 5, FLOAT NOT NULL)

vectorProp
Rows: 1

Lists containing `VECTOR` values as nested entries cannot be stored as properties.

Vectors and client libraries (drivers)

Working with vectors via Neo4j's client libraries results in a different behavior depending on the library version.

- **Versions >= 6.0** — Vectors are fully supported and mapped into client types (see the *Data types* page of each language manual).
- **Versions < 6.0** — Vectors can be created, attached, and manipulated in queries via Cypher functions, but cannot be returned nor created in the application.
  It is possible to create and store values as shown in the Store vector values as properties examples, but *returning* a `VECTOR` results in a placeholder `MAP` value and a warning.

  Result of returning a `VECTOR` with a driver older than 6.0

```
+-------------------------------------------------------------+
| n.vector                                                    |
+-------------------------------------------------------------+
| {originalType: "VECTOR(1, INTEGER64)", reason: "UNKNOWN_TYPE"} |
+-------------------------------------------------------------+
warn: One or more values returned could not be handled by this version of the driver and were replaced
with placeholder map values. Please upgrade your driver!
03N95 (Neo.ClientNotification.UnknownType)
```

Type coercion

Coercion is the action of forcing entries of a different (implicit) type into a vector with a different coordinate type.

When the coordinate type is the same as the type of the given elements, no coercion is done. When the coordinate type differs, coercion may be done or an error may be raised depending on the situation.

An error is raised if a value does not fit into the coordinate type. If the coordinate type is an `INTEGER` type and all the coordinate values are `INTEGER` values, then an error will be raised if and only if one of the coordinate types does not fit into the size of the specified type. The same applies for `FLOAT` vector types: if the elements are all `FLOAT` values then an error will only be raised if one value does not fit into the specified type.

```
1 RETURN VECTOR([128], 1, INT8)
2 // 22N28: data exception - overflow error. The result of the operation 'vector()' has caused an
  overflow.
3 //   22003: data exception - numeric value out of range. The numeric value 128 is outside the required
  range.
```

Coercion (i.e. lossy conversion) is allowed when:

- The list contains `INTEGER` values and the specified vector type is of a `FLOAT` type. Precision will be lost for values at the higher end of the range (see the Java type specification), but an error will be raised only if the value were to overflow/underflow.

```
RETURN VECTOR([987374677], 1, FLOAT32)
// vector([9.8737466E8], 1, FLOAT32 NOT NULL)
```

- The list contains `FLOAT` values and the specified type is of an `INTEGER` type. Information may be lost, as all values after the decimal point will be truncated, but an error will be raised only if the value were to overflow/underflow.

```
RETURN VECTOR([1.2], 1, INT)
// vector([1], 1, INTEGER NOT NULL)
```

Supertypes

`VECTOR` is a supertype of `VECTOR<TYPE>(DIMENSION)` types. The same applies for `VECTOR` types with only a coordinate type or a dimension:

- `VECTOR` with only a defined dimension is a supertype of all `VECTOR` values of that dimension, regardless of the coordinate type. For example, `VECTOR(4)` is a supertype of `VECTOR<FLOAT>(4)` and `VECTOR<INT8>(4)`.
- `VECTOR` with only a defined coordinate type is a supertype of all `VECTOR` values with that coordinate type, regardless of the dimension. For example, `VECTOR<INT>` is a supertype of `VECTOR<INT>(3)` and `VECTOR<INT>(1024)`.

All of these supertypes can be used in type predicate expressions. For more information, see:

- Equality, ordering, and comparison of value types → Ordering vector types
- Property, structural, and constructed values → Vector type normalization

Lists, vector embeddings, and vector indexes

`VECTOR` and `LIST` values are similar and can both be indexed and searched through using vector indexes, but have a few key differences:

- Elements in a `LIST` can be accessed individually, whereas operations on a `VECTOR` must operate on the entire `VECTOR`: it is not possible to access or slice individual elements.
- Storing vector embeddings as `VECTOR` properties with a defined coordinate type allows them to be stored more efficiently. Moreover, reducing a vector's coordinate type (e.g., from `INTEGER16` to `INTEGER8`) downsizes storage requirements and improves performance, provided all values remain within the range supported by the smaller type.

You can convert vectors into lists via the functions `toIntegerList()` and `toFloatList()`. This is necessary if you want to access individual elements in a vector.

For information about how to store embeddings as `VECTOR` values with the GenAI plugin, see GenAI

documentation → Create and store embeddings in a Neo4j database.

Graph references

Graph references are values used to specify which graph is targeted in a query. They are principally used with the `USE` clause, graph functions, such as `graph.byName()`, and when referring to databases and aliases in administration commands.

The `name` column output from `SHOW ALIASES FOR DATABASE` can be used as identifiers for graph references.

When referring to a constituent in a composite database, the identifier is defined as `<composite-name>.<alias-name>`, where the dot functions as a separator between the composite database name and the alias name. Although creating databases or aliases containing dots is not recommended, it is possible to refer to the alias `some.alias` in the composite `some.composite` as `some.composite.some.alias`.

Static graph references

A static graph reference is a direct reference to a graph in a database. These include:

- Database and alias names when managing databases and aliases: `CREATE DATABASE <symbolic-name>`.
- Database and alias names when selecting a graph with the `USE` clause: `USE <symbolic-name>` or `USE <symbolic-name>.<symbolic-name>` (for constituents of a composite databases).

Dynamic graph references

A dynamic graph reference is a reference to a graph using values that are evaluated during query execution using a function. These include:

- Dynamic graph reference with the `graph.byName()` function, to access a graph of a given name: `USE graph.byName(<string-expression>)`.
- Dynamic graph references with the `graph.byElementId()` function, to access a graph of a given node or relationship: `USE graph.byElementId(<element-id-string>)`.
- Retrieving properties of a graph with the `graph.propertiesByName()` function: `graph.propertiesByName(<string-expression>)`.

Rules

The following rules apply for graph references when using identifiers (static graph references, administration commands) or the `graph.byName` function.

`USE graph.byName(<graph-reference>)` is typically equivalent to `USE <graph-reference>`. There may be differences if the name contains special characters:

- Graph references passed as identifiers such as `USE <graph-reference>` apply the escaping rules of symbolic names. Graph names can either be unquoted (e.g. , `USE neo4j`) or need to be quoted in its entirety (e.g., `USE `db-with-dash``).
- Graph references passed as graph functions such as `USE graph.byName(<graph-reference>)` apply the

escaping rules of string literals. Note that quotes are considered as part of the name, e.g. `USE graph.byName(``db-with-dash``)` would try to resolve the graph reference `` `db-with-dash` ``.

For example, the graph reference `db-with-dash` can be passed as:

- `USE `db-with-dash``
- `USE graph.byName('db-with-dash')`
- `USE graph.byName($param)` with parameter `db-with-dash`

Working with `null`

In Cypher, `null` is used to represent missing or undefined values. All data types in Cypher are nullable. This means that type predicate expressions always return `true` for `null` values.

Conceptually, `null` means **a missing or unknown value**, and it is treated somewhat differently from other values. For example, returning a property from a node that does not have said property produces `null`. Most expressions that take `null` as input will produce `null`. In the case of a predicate used in a `WHERE` clause, anything that is not `true` is interpreted as being false.

`null` is not equal to `null`. Not knowing two values does not imply that they are the same value. This means that the expression `null` = `null` yields `null`, and not `true`.

Logical operations with `null`

The boolean operators (`AND`, `OR`, `XOR`, `NOT`) treat `null` as the **unknown value** of three-valued logic.

Truth table for logical operators

a	b	a `AND` b	a `OR` b	a `XOR` b	`NOT` a
`false`	`false`	`false`	`false`	`false`	`true`
`false`	`null`	`false`	`null`	`null`	`true`
`false`	`true`	`false`	`true`	`true`	`true`
`true`	`false`	`false`	`true`	`true`	`false`
`true`	`null`	`null`	`true`	`null`	`false`
`true`	`true`	`true`	`true`	`false`	`false`
`null`	`false`	`false`	`null`	`null`	`null`
`null`	`null`	`null`	`null`	`null`	`null`
`null`	`true`	`null`	`true`	`null`	`null`

The `IN` operator and `null`

The `IN` operator follows similar logic. If Cypher can ascertain that something exists in a list, the result will be `true`. Any list that contains a `null` and does not have a matching element will return `null`. Otherwise,

the result will be `false`.

examples of expressions containing the IN operator

Expression	Result
`2 IN [1, 2, 3]`	`true`
`2 IN [1, null, 3]`	`null`
`2 IN [1, 2, null]`	`true`
`2 IN [1]`	`false`
`2 IN []`	`false`
`null IN [1, 2, 3]`	`null`
`null IN [1, null, 3]`	`null`
`null IN []`	`false`

Using `all`, `any`, `none`, and `single` follows a similar rule. If the result can be calculated definitively, `true` or `false` is returned. Otherwise `null` is produced.

The `[]` operator and `null`

Accessing a list or a map with `null` will result in `null`:

Expression	Result
`[1, 2, 3][null]`	`null`
`[1, 2, 3, 4][null..2]`	`null`
`[1, 2, 3][1..null]`	`null`
`{age: 25}[null]`	`null`

Using parameters to pass in the bounds, such as `a[$lower..$upper]`, may result in a `null` for the lower or upper bound (or both). The following workaround will prevent this from happening by setting the absolute minimum and maximum bound values:

```
a[coalesce($lower,0)..coalesce($upper,size(a))]
```

Expressions that return `null`

- Getting a missing element from a list: `[][0]`, `head([])`.
- Trying to access a property that does not exist on a node or relationship: `n.missingProperty`.
- Comparisons when either side is `null`: `1 < null`.
- Arithmetic expressions containing `null`: `1 + null`.
- Some function calls where any argument is `null`: e.g., `sin(null)`.

Using `IS NULL` and `IS NOT NULL`

Testing any value against `null`, either with the `=` operator or with the `<>` operator, always evaluates to `null`. Therefore, use the special equality operators `IS NULL` or `IS NOT NULL`. Using `IS NOT NULL` on a property (e.g. `n.prop IS NOT NULL`) tests that the property exists and is not null.

Casting data values

Cypher supports a number of functions to cast values to different data types. This section will provide an overview of those functions, as well examples of how to use them in practice.

Functions for converting data values

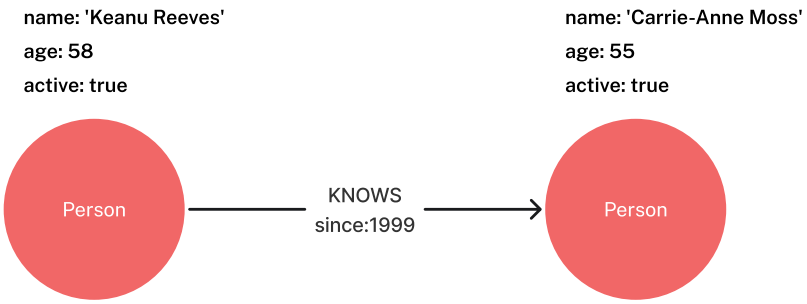
The following functions are available for casting data values:

Function	Description
`toBoolean()`	Converts a `STRING`, `INTEGER`, or `BOOLEAN` value to a `BOOLEAN` value.
`toBooleanList()`	Converts a `LIST<ANY>` and returns a `LIST<BOOLEAN>` values. If any values are not convertible to `BOOLEAN` they will be `null` in the `LIST<BOOLEAN>` returned.
`toBooleanOrNull()`	Converts a `STRING`, `INTEGER` or `BOOLEAN` value to a `BOOLEAN` value. For any other input value, `null` will be returned.
`toFloat()`	Converts an `INTEGER`, `FLOAT`, or a `STRING` value to a `FLOAT` value. Otherwise `null` is returned.
`toFloatList()`	Converts a `LIST<ANY>` or, as of Neo4j 2025.10, additionally `VECTOR` and returns a `LIST<FLOAT>` values. If any values are not convertible to `FLOAT` they will be `null` in the `LIST<FLOAT>` returned.
`toFloatOrNull()`	Converts an `INTEGER`, `FLOAT`, or a `STRING` value to a `FLOAT`. For any other input value, `null` will be returned.
`toInteger()`	Converts a `BOOLEAN`, `INTEGER`, `FLOAT` or a `STRING` value to an `INTEGER` value.
`toIntegerList()`	Converts a `LIST<ANY>` or, as of Neo4j 2025.10, additionally `VECTOR` to a `LIST<INTEGER>` values. If any values are not convertible to `INTEGER` they will be null in the `LIST<INTEGER>` returned.
`toIntegerOrNull()`	Converts a `BOOLEAN`, `INTEGER`, `FLOAT` or a `STRING` value to an `INTEGER` value. For any other input value, `null` will be returned.
`toString()`	Converts a value to a `STRING`.
`toStringList()`	Converts a `LIST<ANY>` and returns a `LIST<STRING>`.
`toStringOrNull()`	Converts a value to a `STRING`, or `null` if the value cannot be converted.

More information about these, and many other functions, can be found in the section on Functions.

Examples

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (keanu:Person {name:'Keanu Reeves', age: 58, active:true}),
       (carrieAnne:Person  {name:'Carrie-Anne Moss', age: 55, active:true}),
       (keanu)-[r:KNOWS {since:1999}]->(carrieAnne)
```

Returning converted values

In the below query, the function `toFloat` is used to cast two `STRING` values. It shows that `null` is returned if the data casting is not possible.

```
MATCH (keanu:Person {name:'Keanu Reeves'})
RETURN toFloat(keanu.age), toInteger(keanu.name)
```

Result

toFloat(keanu.age)	toInteger(keanu.name)
58.0	null

If the function `toFloat` is passed an unsupported value (such as a `DATE` value), it will throw an error:

Query

```
WITH date({
  year: 2023, month: 5, day: 2
}) AS d
RETURN toFloat(d)
```

GQLSTATUS error chain

22N27: error: data exception - invalid entity type. Invalid input 'DATE' for argument at index 0 of function toFloat(). Expected to be FLOAT, INTEGER or STRING.

22G03: error: data exception - invalid value type

However, if the same value is passed to the function `toFloatOrNull`, `null` will be returned.

Query

```
WITH date({
  year: 2023, month: 5, day: 2
}) AS d
RETURN toFloatOrNull(d)
```

Result

toFloatOrNull(d)
null

It is also possible to return casted values as a list. The below query uses the `toStringList` to cast all passed values into `STRING` values, and return them in as a `LIST<STRING>`:

```
MATCH (keanu:Person {name:'Keanu Reeves'})
RETURN toStringList([keanu.name, keanu.age]) AS keanuList
```

Result

keanuList
["Keanu Reeves", "58"]

Updating property value types

The functions to cast data values can be used to update property values on nodes and relationships. The below query casts the `age` (`INTEGER`), `active` (`BOOLEAN`), and `since` (`INTEGER`) properties to `STRING` values:

```
MATCH (keanu:Person {name:'Keanu Reeves'})-[r:KNOWS]-()
SET keanu.age = toString(keanu.age),
    keanu.active = toString(keanu.active),
    r.since = toString(r.since)
RETURN keanu.age, keanu.active, r.since
```

Result

keanu.age	keanu.active	r.since
"58"	"true"	"1999"

Equality, ordering, and comparison of value types

This page explains how Cypher compares and orders different value types, including equality and inequality rules.

Equality and comparison of values

The equality (`=`) and inequality (`<>`) operators allows for comparing the equality of different values.

Values of the same type are only equal if they are the same identical value (e.g. `3 = 3` and `"x" <> "xy"`).

Maps are only equal if they map exactly the same keys to equal values and lists are only equal if they contain the same sequence of equal values (e.g. `[3, 4] = [1+2, 8/2]`).

`UUID` values support equality (`=`) and inequality (`<>`) comparisons. However, the ordering operators `<` and `>` are not defined for `UUID` values and always return `null`. The operators `<=` and `>=` return `true` if the two `UUID` values are equal, and `null` otherwise.

Values of different types are considered as equal according to the following rules:

- `PATH` values are treated as lists of alternating nodes and relationships and are equal to all lists that contain an identical sequence of nodes and relationships.
- Testing any value against `null` with either the `=` or `<>` operator always evaluates to `null`. This includes `null = null` and `null <> null`. To reliably test if a value is `null` use `v IS NULL` or `v IS NOT NULL` (the latter is equivalent to `NOT(v IS NULL)`).

Other combinations of value types cannot be compared with each other. For example, nodes, relationships, and literal maps cannot be compared to one another. Comparing incomparable values will throw an error.

Ordering and comparison of values

`ORDER BY` requires that all values are orderable. The following points explain how comparisons are made when using the `<=`, `<`, `>=`, `>` operators.

- Numerical values are compared for ordering using numerical order (e.g. `3 < 4` is `true`).
- All comparability tests with `java.lang.Double.NaN` evaluate as `false`. For example, `1 > b` and `1 < b` are both `false` when `b` is NaN.
- String values are compared for ordering using lexicographic order (e.g. `"x" < "xy"`).
- Boolean values are compared for ordering such that `false < true`.
- When comparing values for ordering, if one of the arguments is `null`, the result is always `null`.
- Spatial values and `VECTOR` values cannot be compared using the operators `<=`, `<`, `>=`, `>`. To compare spatial values within a specific range, use either the `point.withinBBox()` or the `point()` function.

For the ordering of `VECTOR` types, see Ordering `VECTOR` types.

Hierarchy of values

Values of different types are ordered based on a predefined hierarchy, from least to greatest, as outlined in the following list:

- `MAP`
- `NODE`
- `RELATIONSHIP`
- `LIST`
- `PATH`
- `VECTOR`
- `POINT`
- `ZONED DATETIME`
- `LOCAL DATETIME`
- `DATE`

- `ZONED TIME`
- `LOCAL TIME`
- `DURATION`
- `STRING`
- `BOOLEAN`
- `UUID`
- Numbers: `INTEGER`, `FLOAT`

`null` is ordered after all other values.

Sorting rules for mixed Cypher types

```
WITH [42, "hello", null, true, {name: "Alice"}, [1, 2, 3], date("2024-02-10")] AS v
UNWIND v AS values
RETURN values
ORDER BY values
```

Result

values
{name: "Alice"}
[1, 2, 3]
2024-02-10
"hello"
true
42
null
Rows: 7

Ordering list and map values

Lists are compared in lexicographic order:

- Elements are compared pairwise from the start to the end of the list. The order is defined by the first pair where the elements differ. For example, `[1, 'foo', 3]` is less than `[1, 2, 'bar']` because `'foo'` is less than `2`.
- If one list is shorter, it is padded with an empty element, which is considered less than any other value (including `null`). For example, `[1, 'foo']` is less than `[1, 'foo', 3]`, and `[1]` is less than `[1, null]`.

Maps are ordered by size, keys, and values:

- Maps are compared primarily by size: the smallest map has the fewest entries. For example, `{a: 1}` is less than `{a: 0, b: 'foo'}`.

- Maps of equal size are compared by the alphabetical order of their keys. For example, `{b: 100, a: 'foo'}` is less than `{a: '', c: null}` because `['a', 'b']` is less than `['a', 'c']`.
- For maps with identical key sets, the comparison is based on the values. After sorting keys alphabetically, values are compared pairwise. For example, `{b: 100, a: 'foo'}` is less than `{a: 'foo', b: null}` because `['foo', 100]` is less than `['foo', null]`.

For more information about `LIST` and `MAP` values, see Values and types → Constructed types.

Ordering path values

`PATH` values are compared element-by-element (nodes and relationships alternating); the first differing element determines the order. For example, `()-[:A]->()` is ordered before `()-[:B]->()`.

If one path is a prefix of another, the shorter path is considered smaller. For example, `()-[:REL]->()` is ordered before `()-[:REL]->()-[:REL]->()`.

Ordering spatial and temporal values

The following applies to the ordering of spatial types:

- `POINT` values are ordered after lists and before temporal types.
- `POINT` values of different coordinate reference systems (CRS) are ordered by the CRS code (the value of SRID field). For the supported set of CRS, the following ascending order applies: `4326`, `4979`, `7302`, `9157`.
- `POINT` values with the same CRS are ordered by each coordinate value in turn; first `x`, then `y`, and finally `z`.
- Note that this ordering is different to the order returned by the spatial index, which follows the space filling curve.

The following applies to the ordering of temporal types:

- Temporal types are ordered after spatial types but before strings.
- Temporal values follow a chronological order. For example, `2023-01-01` comes before `2024-01-01`.
- Temporal values are first sorted by type, then by value. For example, `DATETIME` is considered "greater" than a `DATE` and `2023-02-10T12:00:00` comes before `2023-02-10T15:00:00` because it is chronologically earlier.
- Since there is no perfect way to compare duration values (because months and years have varying lengths), Cypher defines a specific rule for sorting them in `ORDER BY`:
  - 1 year is treated as 365.2425 days (to account for leap years).
  - 1 month is treated as 30.436875 days (which is 1/12 of a year).
  - 1 day is always 24 hours.

The following applies to the comparison of temporal types:

- Temporal instant values (like `DATETIME` and `DATE`) can be compared if they are of the same type. An

earlier instant is considered smaller (less than) compared to a later instant.

- Instants at the same point in time but with different time zones are not considered equal. To ensure consistent ordering, Cypher sorts them first by their actual point in time. If two instants have the same time but different time zones, they are ordered by their UTC offset (west to east, meaning negative offsets come first). If they have the same time and offset but different named time zones, they are sorted alphabetically by the time zone name.
- Duration values cannot be directly compared. Since the length of a day, month, or year varies, Cypher does not define a strict ordering for durations. As a result, comparing two durations `(e.g, duration1 < duration2)` will always return `null`.

Ordering UUID values

`UUID` values are ordered by comparing their most significant bits first (the first 64 bits of the 128-bit value), then their least significant bits (the last 64 bits), treating each as an unsigned `INTEGER` (per RFC 9562). For example, `uuid('00000000-0000-0000-0000-000000000001')` is ordered before `uuid('00000000-0001-0000-0000-000000000000')`, because the most significant bits differ (`0000…0000` versus `0000…0001`) and that comparison is decisive regardless of the least significant bits.

Ordering `VECTOR` types

The `VECTOR` supertypes are ordered in the following ascending order:

- `VECTOR`
- `VECTOR<TYPE>`
- `VECTOR(DIMENSION)`

`VECTOR` types with both a defined coordinate type and dimension are ordered according to the ordering of the vector coordinate types, listed in ascending order below:

- `INTEGER8`
- `INTEGER16`
- `INTEGER32`
- `INTEGER64`
- `FLOAT32`
- `FLOAT64`

Within the same coordinate type, `VECTOR` types are ordered by their dimension, with smaller values first. `VECTOR` types of the same coordinate type and dimension are then ordered pairwise.

Ordering rules for `VECTOR` values

A	B	Ordered as	Reason
`VECTOR<FLOAT32>(12345)`	`VECTOR<FLOAT32>(123456)`	A < B	Same coordinate type: compare by dimension ascending.

A	B	Ordered as	Reason
`VECTOR<INTEGER32>(1234)`	`VECTOR<FLOAT32>(1234)`	A < B	Coordinate type order: `INTEGER32` < `FLOAT32`
`VECTOR<INTEGER8>`	`VECTOR(3)`	A < B	Coordinate type defined and no dimension < dimension defined and no coordinate type
`VECTOR<INTEGER64>(123456)`	`VECTOR<FLOAT32>(3)`	A < B	Coordinate type order: `INTEGER64` < `FLOAT32`, compare coordinate type first.
`VECTOR<FLOAT64>(1234)`	`VECTOR<FLOAT32>(1234)`	B < A	Coordinate type order: `FLOAT32` < `FLOAT64`
`VECTOR<FLOAT32>([1, 2])`	`VECTOR<FLOAT32>([2, 1])`	A < B	Same coordinate type and dimension, pairwise value comparison.
`VECTOR<INTEGER16>(1234)`	`LIST<INTEGER>`	A < B	`VECTOR` values are ordered before `LIST` values.

[1] For both `FLOAT32` and `FLOAT64`, the minimum value represents the minimum positive value, i.e. the closest value to zero. Negative values are also possible.
[2] This is in accordance with the Gregorian calendar; i.e. years AD/CE start at year 1, and the year before that (year 1 BC/BCE) is 0, while year 2 BCE is -1 etc.
[3] The first week of any year is the week that contains the first Thursday of the year, and thus always contains January 4.
[4] For dates from December 29, this could be the next year, and for dates until January 3 this could be the previous year, depending on how week 1 begins.
[5] Cypher does not support leap seconds; UTC-SLS (UTC with Smoothed Leap Seconds) is used to manage the difference in time between UTC and TAI (International Atomic Time).
[6] The expression datetime().epochMillis returns the equivalent value of the timestamp() function.
[7] For the nanosecond part of the epoch offset, the regular nanosecond component (instant.nanosecond) can be used.

Expressions

A Cypher expression is any part of a query that evaluates to a value. For details and examples of specific expressions, see the following sections:

- Predicates
  - Boolean operators: `AND`, `OR`, `XOR`, `NOT`
  - Comparison operators: `=`, `<>`, `<`, `>`, `<=`, `>=`, `IS NULL`, `IS NOT NULL`
  - List operators: `IN`
  - String operators: `STARTS WITH`, `ENDS WITH`, `CONTAINS`, `IS NORMALIZED`, `IS NOT NORMALIZED`, `=~`
  - Label expression predicates: information about how to test whether a node or relationship matches a label expression or not.
  - Path pattern expressions: information about filtering queries with path pattern expressions.
  - Type predicate expressions: information about how to verify the value type of a Cypher expression.
- Node and relationship operators: information about how to access `NODE` and `RELATIONSHIP` property values with `.` and `[]`.
- Mathematical operators: `+`, `-`, `*`, `/`, `%`, `^`.
- String concatenation operators: `+`, `||`
- Temporal operators: `+`, `-`, `*`, `/`
- List expressions: information about list concatenation operators (`||`, `+`), list element access, list slicing, and list as well as pattern comprehensions.
- Map expressions: information about map operators (`.`, `[]`) and map projection.
- Conditional expressions (CASE)

The following expressions are documented elsewhere in the Cypher Manual:

- Label expressions
- Function calls
- Subquery expressions: `COLLECT`, `COUNT`, and `EXISTS`
- Value literals (see Values and types)
- Graph references

Expressions containing unsanitized user input may make your application vulnerable to Cypher injection. Consider using parameters instead. For more information, see Neo4j Knowledge Base → Protecting against Cypher Injection.

Predicates

Predicates evaluate to a `BOOLEAN` value (`true`, `false`, or `null`), and are frequently used for filtering in `WHERE` subclauses.

This chapter is divided into the following sections:

- Boolean operators: `AND`, `OR`, `XOR`, `NOT`
- Comparison operators: `=`, `<>`, `<`, `>`, `<=`, `>=`, `IS NULL`, `IS NOT NULL`
- List operators: `IN`
- String operators: `STARTS WITH`, `ENDS WITH`, `CONTAINS`, `IS NORMALIZED`, `IS NOT NORMALIZED`, `=~`
- Label expression predicates: `:`, `IS [NOT] LABELED`
- Path pattern expressions: information about filtering queries with path pattern expressions.
- Type predicate expressions: information about how to verify the value type of a Cypher expression.

The following can also serve as a predicates if they result in a `BOOLEAN` value (but are documented elsewhere):

- Variables
- Properties (See also Values and types → property types)
- Predicate functions
- Conditional expressions (CASE)

Boolean operators

Boolean operators are used to combine or evaluate logical conditions. Cypher contains the following boolean operators:

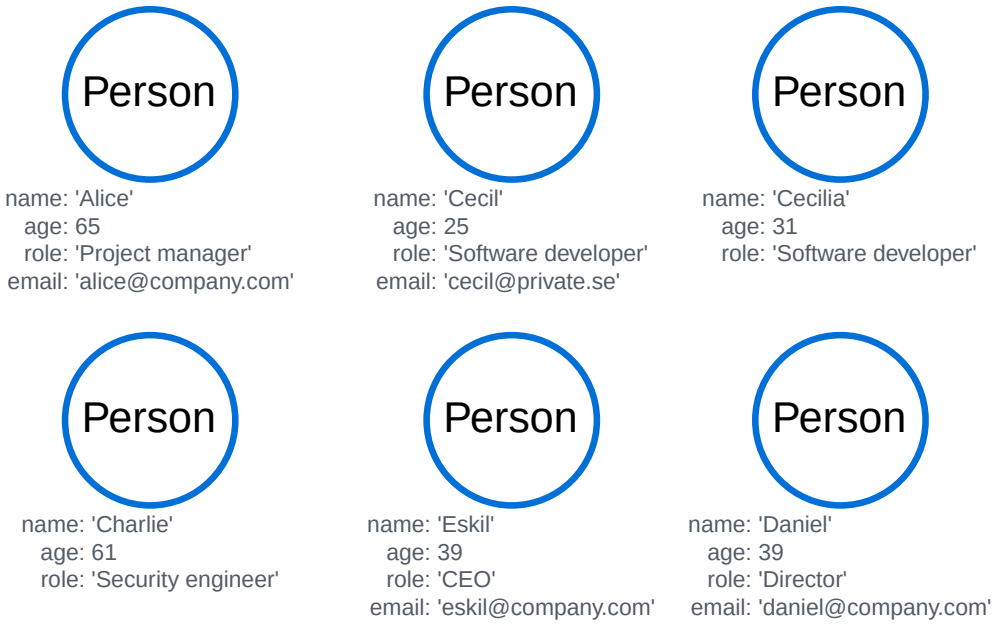
- Conjunction: `AND`
- Disjunction: `OR`
- Exclusive disjunction: `XOR`
- Negation: `NOT`

Truth table for boolean operators

a	b	a `AND` b	a `OR` b	a `XOR` b	`NOT` a
`false`	`false`	`false`	`false`	`false`	`true`
`false`	`null`	`false`	`null`	`null`	`true`
`false`	`true`	`false`	`true`	`true`	`true`
`true`	`false`	`false`	`true`	`true`	`false`
`true`	`null`	`null`	`true`	`null`	`false`
`true`	`true`	`true`	`true`	`false`	`false`
`null`	`false`	`false`	`null`	`null`	`null`
`null`	`null`	`null`	`null`	`null`	`null`
`null`	`true`	`null`	`true`	`null`	`null`

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager', email: 'alice@company.com'}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer', email: 'cecil@private.se'}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer'}),
       (charlie:Person {name: 'Charlie', age: 61, role: 'Security engineer'}),
       (daniel:Person {name: 'Daniel', age: 39, role: 'Director', email: 'daniel@company.com'}),
       (eskil:Person {name: 'Eskil', age: 39, role: 'CEO', email: 'eskil@company.com'})
```

Examples

Example 151. Boolean operators

`AND` operator

```
MATCH (n:Person)
WHERE n.age > 30 AND n.role = 'Software developer'
RETURN n.name AS name, n.age AS age, n.role AS role
```

Result

name	age	role
"Cecilia"	31	"Software developer"

Rows: 1

`OR` operator

```
MATCH (n:Person)
WHERE n.age < 30 OR n.role = 'Software developer'
RETURN n.name AS name, n.age AS age, n.role AS role
```

Result

name	age	role
"Cecilia"	31	"Software developer"
"Cecil"	25	"Software developer"

Rows: 2

`XOR` operator

```
MATCH (n:Person)
WHERE n.age > 30 XOR n.role = 'Software developer'
RETURN n.name AS name, n.age AS age, n.role AS role
```

Result

name	age	role
"Alice"	65	"Project manager"
"Cecil"	25	"Software developer"
"Charlie"	61	"Security engineer"
"Eskil"	39	"CEO"
"Daniel"	39	"Director"

Rows: 5

`NOT` operator

```
MATCH (n:Person)
WHERE NOT n.age = 39
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Alice"	65
"Cecil"	25
"Cecilia"	31
"Charlie"	61

Rows: 4

Operator precedence

Note:

Operator precedence defines how Cypher groups expressions when parentheses are not present. Within predicates, the following order is relevant (higher precedence first):

- Comparison operators: `=`, `<>`, `<`, `>`, `<=`, `>=`, `IS NULL`, `IS NOT NULL`, `IN`, `STARTS WITH`, `ENDS WITH`, `CONTAINS`, `=~`
- `NOT`
- `AND`
- `XOR`
- `OR`

This means `NOT` applies to the result of a comparison unless you add parentheses.

Examples:

- `NOT true AND false` is valid and equivalent to `(NOT true) AND false`.
- `true AND NOT false` is valid and equivalent to `true AND (NOT false)`.
- `NOT true = false` is valid and equivalent to `NOT (true = false)`.
- `true = NOT false` is not valid. Use `true = (NOT false)` instead.

When in doubt, add parentheses to make the intended grouping explicit.

Combining boolean operators

```
MATCH (n:Person)
WHERE n.role = 'Software developer' XOR (n.age > 60 AND n.role = 'Security engineer') OR NOT (n.role
= 'Director' OR n.name = 'Eskil')
RETURN n.name AS name, n.age AS age, n.role AS role
```

Result

name	age	role
"Alice"	65	"Project manager"
"Cecil"	25	"Software developer"
"Cecilia"	31	"Software developer"
"Charlie"	61	"Security engineer"

Rows: 4

Comparison operators

Comparison operators are used to compare values. Cypher contains the following comparison operators:

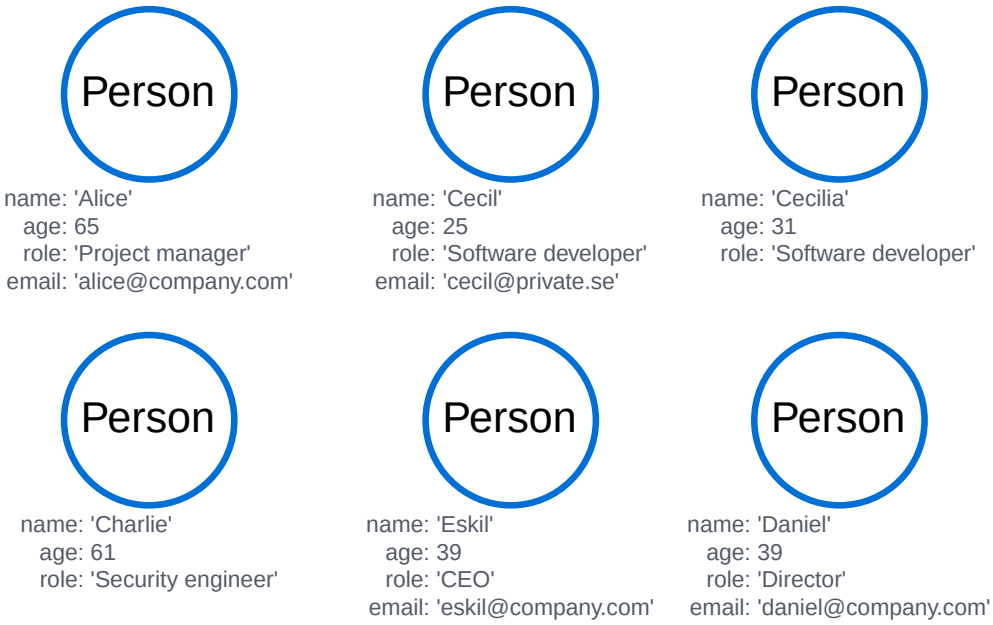
- Equality: `=`
- Inequality: `<>`
- Less than: `<`
- Greater than: `>`
- Less than or equal to: `<=`

- Greater than or equal to: `>=`
- `IS NULL`
- `IS NOT NULL` as well as the `PROPERTY_EXISTS` predicate

> For more information about how Cypher orders and compares different value types, see Values and types → Equality, ordering, and comparison of value types

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager', email: 'alice@company.com'}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer', email: 'cecil@private.se'}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer'}),
       (charlie:Person {name: 'Charlie', age: 61, role: 'Security engineer'}),
       (daniel:Person {name: 'Daniel', age: 39, role: 'Director', email: 'daniel@company.com'}),
       (eskil:Person {name: 'Eskil', age: 39, role: 'CEO', email: 'eskil@company.com'})
```

Examples

Example 152. Comparison operators

Equality operator (`=`)

```
MATCH (n:Person)
WHERE n.role = 'Software developer'
RETURN n.name AS name, n.role AS role
```

Result

name	role
"Cecil"	"Software developer"
"Cecilia"	"Software developer"
Rows: 2	

Inequality operator (`<>`)

```
MATCH (n:Person)
WHERE n.role <> 'Software developer'
RETURN n.name AS name, n.role AS role
```

Result

name	role
"Alice"	"Project manager"
"Charlie"	"Security engineer"
"Daniel"	"Director"
"Eskil"	"CEO"
Rows: 4	

Less than operator (`<`)

```
MATCH (n:Person)
WHERE n.age < 39
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Cecil"	25
"Cecilia"	31
Rows: 2	

Less than or equal operator (`<=`)

```
MATCH (n:Person)
WHERE n.age <= 39
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Cecil"	25
"Cecilia"	31

name	age
"Daniel"	39
"Eskil"	39
Rows: 4	

Greater than operator (`>`)

```
MATCH (n:Person)
WHERE n.age > 39
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Alice"	65
"Charlie"	61
Rows: 2	

Greater than or equal operator (`>=`)

```
MATCH (n:Person)
WHERE n.age >= 39
RETURN n.name AS name, n.age AS age
```

Result

name	age
"Alice"	65
"Charlie"	61
"Daniel"	39
"Eskil"	39
Rows: 4	

`IS NULL` *operator*

```
MATCH (n:Person)
WHERE n.email IS NULL
RETURN n.name AS name
```

Result

name
"Cecilia"
"Charlie"

name
Rows: 2

`IS NOT NULL` operator

```
MATCH (n:Person)
WHERE n.email IS NOT NULL
RETURN n.name AS name, n.email AS email
```

Result

name	email
"Alice"	"alice@company.com"
"Cecil"	"cecil@private.se"
"Daniel"	"daniel@company.com"
"Eskil"	"eskil@company.com"
Rows: 4	

The `IS NOT NULL` predicate tests whether an expression evaluates to a non-`NULL` value. For the GQL property existence predicate (GQL feature G115), see `PROPERTY_EXISTS` predicate.

`PROPERTY_EXISTS` predicate

The optional GQL feature G115 defines a `PROPERTY_EXISTS(element, propertyName)` predicate for testing whether a property exists on a node or relationship and is not `NULL`.

As of Neo4j 2026.03, Cypher supports the GQL syntax:

```
MATCH (n:Person)
WHERE PROPERTY_EXISTS(n, email)
RETURN n.name AS name, n.email AS email
```

For node and relationship properties, this is equivalent to:

```
MATCH (n:Person)
WHERE n.email IS NOT NULL
RETURN n.name AS name, n.email AS email
```

Notes and differences:

- The second argument must be a property name identifier (for example, `email`). String literals and expressions are not allowed.
- If the element argument is `NULL`, `PROPERTY_EXISTS` returns `NULL` (consistent with GQL semantics).
- `PROPERTY_EXISTS` applies to node and relationship properties only. `IS NOT NULL` is more general and can be applied to any expression.

Chaining comparison operators

There is no limit on how many comparisons can be chained together. If chaining two or more comparison operators, each comparison is effectively separated by an `AND` operator (though this `AND` is not required syntactically). For example, if `a, b, c, …, z` are expressions and `op1, op2, …, opN` are comparison operators, then the following expressions are equivalent:

Equivalent expressions

```
a op1 b op2 c ... y opN z;
a op1 b AND b op2 c AND ... y opN z
```

Note that `a op1 b op2 c` does not imply any kind of comparison between `a` and `c`. For example, in `x < y > z`, `x` and `z` are not compared.

Chaining equality operators

Chains of `=` and `<>` are treated in a special way in Cypher. Specifically, `1=1=true` is equivalent to `1=1 AND 1=true` and not to `(1=1)=true` or `1=(1=true)`. For example, the following expressions are equivalent.

Equivalent expressions

```
a < b = c <= d <> e;
a < b AND b = c AND c <= d AND d <> e
```

List operators

List operators are used to perform operations on `LIST` values. Cypher contains the following list operator:

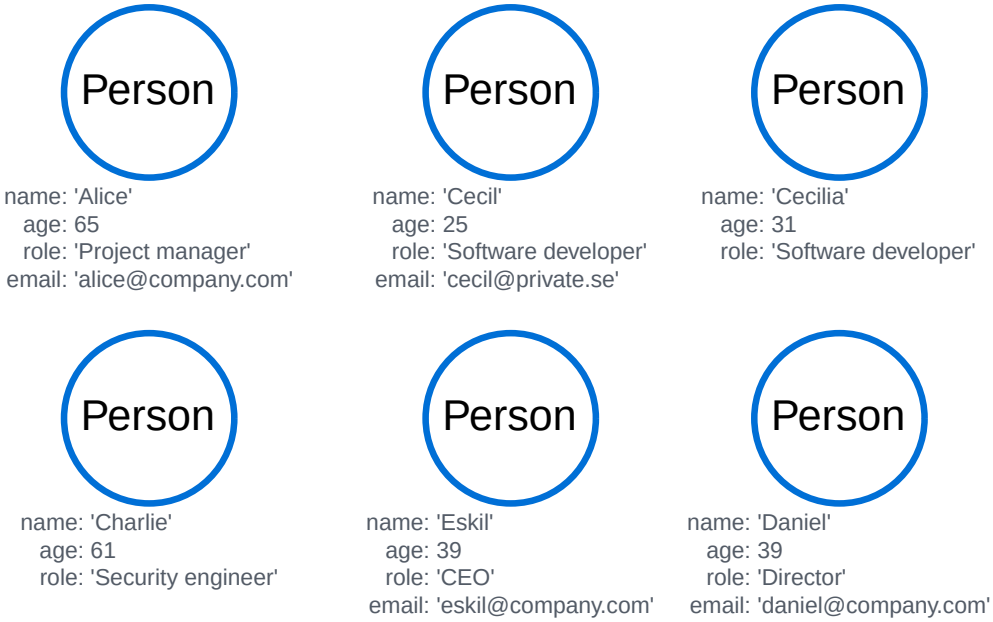
- Membership: `IN`

For additional list predicates, see:

- The following predicate functions: `all()`, `any()`, `none()`, and `single()`.
- The `isEmpty()` function.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager', email: 'alice@company.com'}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer', email: 'cecil@private.se'}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer'}),
       (charlie:Person {name: 'Charlie', age: 61, role: 'Security engineer'}),
       (daniel:Person {name: 'Daniel', age: 39, role: 'Director', email: 'daniel@company.com'}),
       (eskil:Person {name: 'Eskil', age: 39, role: 'CEO', email: 'eskil@company.com'})
```

Examples

Example 153. Basic `LIST` membership checks

`IN` operator

```
MATCH (n:Person)
WHERE n.role IN ['Software developer', 'Project manager']
RETURN n.name AS name, n.role AS role
```

Result

name	role
"Alice"	"Project manager"
"Cecil"	"Software developer"
"Cecilia"	"Software developer"

Rows: 3

Checking membership in a dynamic `LIST`

```
MATCH (p:Person)
WITH p, ['Software developer', 'CEO'] AS roles
WHERE p.role IN roles
RETURN p.name AS name, p.role AS role
```

Result

name	role
"Cecil"	"Software developer"
"Cecilia"	"Software developer"
"Eskil"	"CEO"

Rows: 3

The below query finds `Person` nodes that share the `role` of `Cecil` or `Eskil` but have a different `name`.

Multiple `IN` operators

```
WITH ['Cecil', 'Eskil'] AS names
MATCH (ce:Person)
WHERE ce.name IN names
WITH collect(ce.role) AS roles, names
MATCH (p:Person)
WHERE p.role IN roles AND NOT p.name IN names
RETURN p.name AS name, p.role AS role
```

Only `Cecilia` is returned because she shares a `role` with `Cecil` (no `Person` node in the graph shares a `role` with `Eskil`).

Result

name	role
"Cecilia"	"Software developer"

Rows: 1

Example 154. Checking membership in `LIST` values containing duplicates

The presence of duplicate values in a `LIST` does not affect the result of the `IN` operator, which checks whether an element appears at least once. For example, the below query returns `true` (two `Person` nodes are `39` years old).

Membership check in a `LIST` with duplicate values

```
MATCH (p:Person)
WITH collect(p.age) AS allAges
RETURN 39 IN allAges AS listWithDuplicates
```

Result

listWithDuplicates
true

Rows: 1

`null` behavior

When `null` is involved in a membership check, the result is will be `null`.

Using the `IN` operator with `null`

```
RETURN null IN [1, 2, null] AS nullInList, 123 IN null AS valueInNull
```

Result

nullInList	valueInNull
null	null
Rows: 1	

To check if `null` is a member of a `LIST`, use the `any()` function:

Checking if `null` is a member of a `LIST`

```
RETURN any(x IN [1, 2, null] WHERE x IS NULL) AS containsNull
```

Result

containsNull
true
Rows: 1

Nested lists

When used with nested `LIST` values, the `IN` operator evaluates whether a `LIST` is an exact match to any of the nested `LIST` values that are part of an outer `LIST`. Partial matches of individual elements within a nested `LIST` will return `false`.

Checking for membership in nested `LIST` values

```
RETURN [0, 2] IN [[1-3, 2], [3, 4]] AS listInNestedList
```

Result

listInNestedList
false
Rows: 1

Checking for membership in nested `LIST` values

```
RETURN [3, 4] IN [[1-4, 2], [3, 4]] AS listInNestedList
```

Result

listInNestedList
true
Rows: 1

Checking for partial membership in nested `LIST` values

```
RETURN [1] IN [[1-5, 2], [3, 4]] AS listInNestedTest
```

Result

listInNestedList
false
Rows: 1

List subsets

A subset check verifies if all elements of one `LIST` exist in another. The `all()` function is used to ensure that every element in the first `LIST` is found in the second `LIST`.

Subset check

```
WITH [1,3,4] AS sub, [3,5,1,7,6,2,8,4] AS list
RETURN all(x IN sub WHERE x IN list) AS subInList
```

This returns `true` because all elements in `sub` are part of `list`.

Result

subInList
true
Rows: 1

Subset check

```
WITH [1,3,9] AS sub, [3,5,1,7,6,2,8,4] AS list
RETURN all(x IN sub WHERE x IN list) AS subInList
```

This returns `false` because all elements in `sub` are not part of `list`.

Result

subInList
false
Rows: 1

String operators

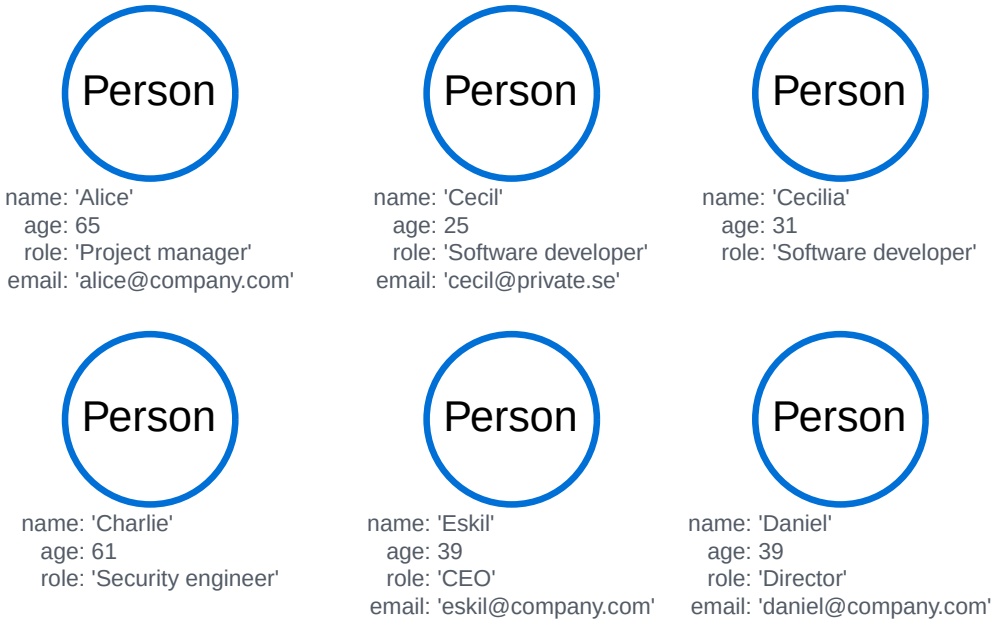
String operators are used to perform operations on `STRING` values. Cypher contains the following string operators:

- Prefix: `STARTS WITH` (case sensitive)
- Suffix: `ENDS WITH` (case sensitive)
- substring: `CONTAINS` (case sensitive)
- Regular expression: `=~`
- `IS NORMALIZED`
- `IS NOT NORMALIZED`

These operators perform *case-sensitive* matching. Attempting to use them on values which are not `STRING` values will return `null`.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager', email: 'alice@company.com'}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer', email: 'cecil@private.se'}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer'}),
       (charlie:Person {name: 'Charlie', age: 61, role: 'Security engineer'}),
       (daniel:Person {name: 'Daniel', age: 39, role: 'Director', email: 'daniel@company.com'}),
       (eskil:Person {name: 'Eskil', age: 39, role: 'CEO', email: 'eskil@company.com'})
```

Examples

Example 155. Prefix, suffix, and substring operators

`STARTS WITH` operator

```
MATCH (n:Person)
WHERE n.name STARTS WITH 'C'
RETURN n.name AS name
```

Result

name
"Cecil"
"Cecilia"
"Charlie"
Rows: 3

`ENDS WITH` operator

```
MATCH (n:Person)
WHERE n.role ENDS WITH 'developer'
RETURN n.name AS name, n.role AS role
```

Result

name	role
"Cecil"	"Software developer"
"Cecilia"	"Software developer"
Rows: 2	

`CONTAINS` operator

```
MATCH (n:Person)
WHERE n.role CONTAINS 'eng'
RETURN n.name AS name, n.role AS role
```

Result

name	role
"Charlie"	"Security engineer"
Rows: 1	

Regular expressions

Cypher supports filtering using regular expressions. The regular expression syntax is inherited from the Java regular expressions. This includes support for flags that change how `STRING` values are matched, including the case-insensitive `(?i)`, multiline `(?m)`, and dotall `(?s)` flags. Flags are given at the beginning of

the regular expression.

Example 156. Regular expressions

Regular expression (`=~`)

```
MATCH (n:Person)
WHERE n.email =~ '.*@company.com'
RETURN n.name AS name, n.email AS email
```

Result

name	email
"Alice"	"alice@company.com"
"Daniel"	"daniel@company.com"
"Eskil"	"eskil@company.com"
Rows: 3	

By pre-pending a regular expression with the flag `(?i)`, the whole expression becomes case-insensitive:

Case-insensitive regular expression `(?i)`

```
MATCH (n:Person)
WHERE n.name =~ '(?i)CEC.*'
RETURN n.name
```

The names of both `Cecil` and `Cecilia` are returned because their name starts with `'CEC'` regardless of casing:

Result

name
"Cecil"
"Cecilia"
Rows: 2

Escaping in regular expressions

Characters such as `.` or `*` have special meaning in a regular expression. To use these as ordinary characters without special meaning, escape them.

Escaped characters in a regular expression

```
MATCH (n:Person)
WHERE n.email =~ '.*\\.se'
RETURN n.name AS name, n.email AS email
```

`Cecil` is returned because his email ends with `'.se'`:

Result

name	email
"Cecil"	"cecil@private.se"
Rows: 1	

Note that the regular expression constructs in Java regular expressions are applied only after resolving the escaped character sequences in the given string literal. It is sometimes necessary to add additional backslashes to express regular expression constructs. This list clarifies the combination of these two definitions, containing the original escape sequence and the resulting character in the regular expression:

String literal sequence	Resulting Regex sequence	Regex match
`\t`	Tab	Tab
`\\t`	`\t`	Tab
`\b`	Backspace	Backspace
`\\b`	`\b`	Word boundary
`\n`	Newline	NewLine
`\\n`	`\n`	Newline
`\r`	Carriage return	Carriage return
`\\r`	`\r`	Carriage return
`\f`	Form feed	Form feed
`\\f`	`\f`	Form feed
`\'`	Single quote	Single quote
`\"`	Double quote	Double quote
`\\`	Backslash	Backslash
`\\\`	`\\`	Backslash
`\uxxxx`	Unicode UTF-16 code point (4 hex digits must follow the `\u`)	Unicode UTF-16 code point (4 hex digits must follow the `\u`)
`\\uxxxx`	`\uxxxx`	Unicode UTF-16 code point (4 hex digits must follow the `\u`)

Note:

Using regular expressions with unsanitized user input makes you vulnerable to Cypher

injection. Consider using parameters instead.

String normalization operators

The `IS NORMALIZED` operator is used to check whether the given `STRING` is in the `NFC` Unicode normalization form:

> **Note:**
>
> Unicode normalization is a process that transforms different representations of the same string into a standardized form. For more information, see the documentation for Unicode normalization forms.

`IS NORMALIZED` operator

```
RETURN 'the \u212B char' IS NORMALIZED AS normalized
```

Result

normalized
false
Rows: 1

Because the given `STRING` contains a non-normalized Unicode character (`\u212B`), `false` is returned.

To normalize a `STRING`, use the normalize() function.

Note that the `IS NORMALIZED` operator returns `null` when used on a non-`STRING` value. For example, `RETURN 1 IS NORMALIZED` returns `null`.

The `IS NOT NORMALIZED` operator is used to check whether the given `STRING` is not in the `NFC` Unicode normalization form:

`IS NOT NORMALIZED`

```
RETURN 'the \u212B char' IS NOT NORMALIZED AS notNormalized
```

Result

notNormalized
TRUE
Rows: 1

Because the given `STRING` contains a non-normalized Unicode character (`\u212B`), and is not normalized, `true` is returned.

Note that the `IS NOT NORMALIZED` operator returns `null` when used on a non-`STRING` value. For example, `RETURN 1 IS NOT NORMALIZED` returns `null`.

Using `IS NORMALIZED` with a specified normalization type

It is possible to define which Unicode normalization type is used (the default is `NFC`).

The available normalization types are:

- `NFC`
- `NFD`
- `NFKC`
- `NFKD`

Query

```
WITH 'the \u00E4 char' as myString
RETURN myString IS NFC NORMALIZED AS nfcNormalized,
    myString IS NFD NORMALIZED AS nfdNormalized
```

The given `STRING` contains the Unicode character: `\u00E4`, which is considered normalized in `NFC` form, but not in `NFD` form.

Result

nfcNormalized	nfdNormalized
true	false
Rows: 2	

It is also possible to specify the normalization form when using the negated normalization operator. For example, `RETURN "string" IS NOT NFD NORMALIZED`.

Label expression predicates

You can use a label expression predicate to verify that the labels of a node or the relationship type of a relationship match a given label expression.

Cypher supports two label expression alternatives:

- Colon form (`<expr>:<label-expression>`)
- `IS [NOT] LABELED` form Introduced in Neo4j 2026.04 Cypher 25 only

Syntax

Colon form

```
<expr>:<label-expression>
```

Where `<expr>` is any Cypher expression and `<label-expression>` is any Cypher label expression.

`IS [NOT] LABELED` predicate

Introduced in Neo4j 2026.04 | Cypher 25 only

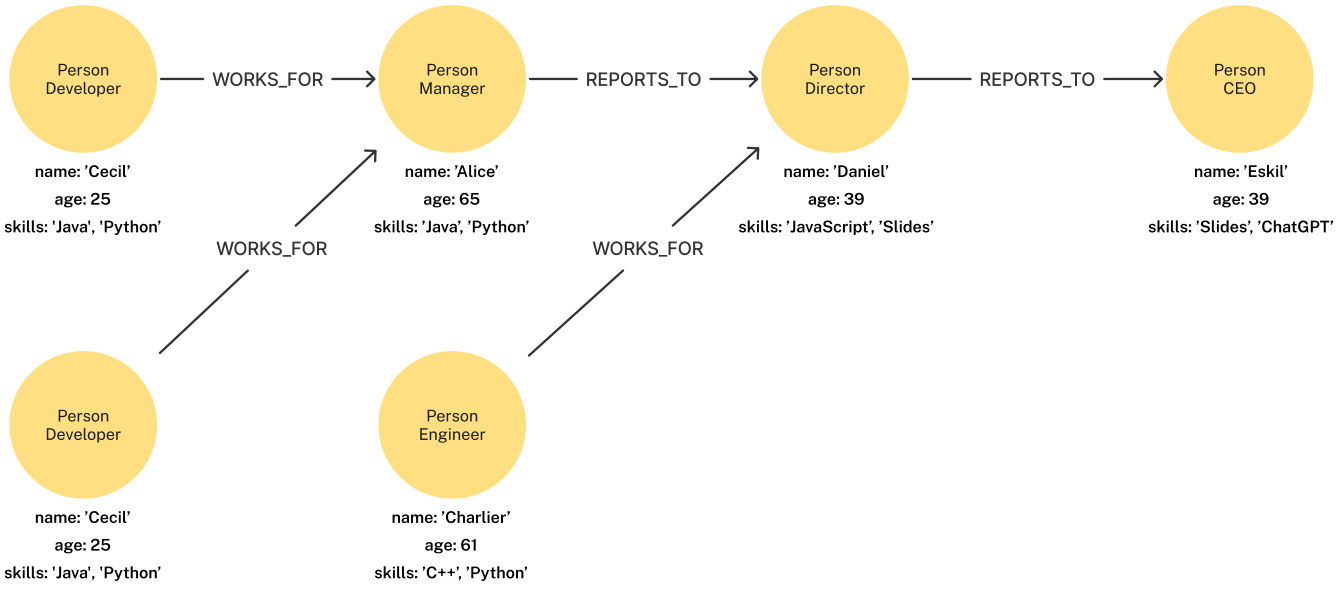
```
<expr> IS LABELED <label-expression>
<expr> IS NOT LABELED <label-expression>
<expr> IS NOT <label-expression>
```

- If `<expr>` evaluates to `null`, the predicate result is `null`.
- Otherwise, the label expression is evaluated against the node's label set or the relationship's single type, and the result is `true` or `false` equivalent to the colon form.
- `IS NOT LABELED <label-expression>` negates the result of the entire label expression. This is different from using `!` **inside** the label expression. For example:
  - `n IS LABELED !A&B` means the node has label `B`, and does not have label `A`.
  - `n IS NOT LABELED A&B` means the node does not have both label `A` and label `B`.

> **Note:**
>
> `IS [NOT] LABELED` is only valid in expression contexts, (for example, `WHERE`, `RETURN`, `WITH`, on the right-hand side of `SET` property assignments, or in `CASE` branches). It cannot be used in place of a label expression inside a node pattern or relationship pattern (for example `MATCH (n IS LABELED Person)` is invalid), nor as `SET` / `REMOVE` label specification syntax. In patterns and when attaching or removing labels, use the colon form instead (for example `(n:Person)`).

Example graph

The following graph is used for the examples on this page:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (alice:Person&Manager {name:'Alice', age: 65, skills: ['Java', 'Python']}),
       (cecil:Person&Developer {name: 'Cecil', age: 25, skills: ['Java', 'Python']}),
       (cecilia:Person&Developer {name: 'Cecilia', age: 31, skills: ['JavaScript', 'TypeScript']}),
       (charlie:Person&Engineer {name: 'Charlie', age: 61, skills: ['C++', 'Python']}),
       (daniel:Person&Director {name: 'Daniel', age: 39, skills: ['JavaScript', 'Slides']}),
       (eskil:Person&CEO {name: 'Eskil', age: 39, skills: ['Slides', 'ChatGPT']}),

       (cecil)-[:WORKS_FOR]->(alice),
       (cecilia)-[:WORKS_FOR]->(alice),
       (charlie)-[:WORKS_FOR]->(daniel),
       (alice)-[:REPORTS_TO]->(daniel),
       (daniel)-[:REPORTS_TO]->(eskil)
```

Test whether a node has a certain label

Given that `p` is a node, `p:Manager` tests whether `p` has the label `Manager` or not and results in `true` or `false`, respectively.

```
MATCH (p:Person)
RETURN p.name AS name, p:Manager AS isManager
```

name	isManager
"Alice"	true
"Cecil"	false
"Cecilia"	false
"Charlie"	false
"Daniel"	false
"Eskil"	false

Rows: 6

Given that `p` is a node and a more complex label expression, e.g, `Manager|Director|CEO`, `p:Manager|Director|CEO` tests whether node `p` matches the label expression `Manager|Director|CEO` or not and results in `true` or `false`, respectively. More specifically, `p:Manager|Director|CEO` tests whether `p` has at least one of the three labels `Manager`, `Director`, and `CEO`.

```
MATCH (p:Person)
RETURN p.name AS name,
       p:Manager|Director|CEO AS isManager
```

name	isManager
"Alice"	true
"Cecil"	false
"Cecilia"	false
"Charlie"	false
"Daniel"	true

name	isManager
"Eskil"	true
Rows: 6	

`p:!CEO` tests whether `p` does not have the label `CEO`.

```
MATCH (p:Person)-[r]->(m)
RETURN p.name AS name,
       m:!CEO AS doesNotWorkDirectlyForTheCEO
```

name	doesNotWorkDirectlyForTheCEO
"Alice"	true
"Cecil"	true
"Cecilia"	true
"Charlie"	true
"Daniel"	false
Rows: 5	

When `<expr>` evaluates to `null`, the label expression predicate results in `null`; for example if `p` is `null`, then `p:!CEO` results in `null`.

```
MATCH (p:Person)
OPTIONAL MATCH (p)-[r]->(m)
RETURN p.name AS name,
       m:!CEO AS doesNotWorkDirectlyForTheCEO
```

name	doesNotWorkDirectlyForTheCEO
"Alice"	true
"Cecil"	true
"Cecilia"	true
"Charlie"	true
"Daniel"	false
"Eskil"	null
Rows: 6	

The function coalesce allows to turn `null` into a default value:

```
MATCH (p:Person)
OPTIONAL MATCH (p)-[r]->(m)
RETURN p.name AS name,
       coalesce(m:!CEO, false) AS doesNotWorkDirectlyForTheCEO
```

name	doesNotWorkDirectlyForTheCEO
"Alice"	true
"Cecil"	true
"Cecilia"	true
"Charlie"	true
"Daniel"	false
"Eskil"	false

Rows: 6

Test whether a node matches a dynamic label expression

Introduced in Neo4j 2025.07 | Cypher 25 only

If you don't know the manager labels, you can infer a list of `managerLabels` by collecting all non-`Person` labels from people with a direct report. You can test nodes with the label expression predicate to match a dynamic label expression. `p:$any(managerLabels)` tests whether a node `p` has at least on of the label in the list `managerLabels`.

```
MATCH ()-[r]->(p:Person)
UNWIND labels(p) AS label
FILTER label <> "Person"
RETURN COLLECT(label) AS managerLabels
NEXT
MATCH (p)
RETURN p.name AS name, p:$any(managerLabels) AS isManager
```

name	isManager
"Alice"	true
"Cecil"	false
"Cecilia"	false
"Charlie"	false
"Daniel"	true
"Eskil"	true

Rows: 6

Examples using `IS [NOT] LABELED`

Introduced in Neo4j 2026.04 | Cypher 25 only

Test a node with `IS LABELED`

```
MATCH (p:Person)
```

```
RETURN p.name AS name, p IS LABELED Manager AS isManager
```

name	isManager
"Alice"	true
"Cecil"	false
"Cecilia"	false
"Charlie"	false
"Daniel"	false
"Eskil"	false

Rows: 6

Simple `CASE` with `IS LABELED` branches

```
MATCH (p:Person)
RETURN p.name AS name,
       CASE p
         WHEN IS LABELED CEO THEN 'executive'
         WHEN IS LABELED Manager|Director THEN 'lead'
         ELSE 'ic'
       END AS role
ORDER BY name
```

name	role
"Alice"	"lead"
"Cecil"	"ic"
"Cecilia"	"ic"
"Charlie"	"ic"
"Daniel"	"lead"
"Eskil"	"executive"

Rows: 6

Test whether a relationship has a certain relationship type

Given that `r` is a relationship, `r:WORKS_FOR` tests whether `r` has the relationship type `WORKS_FOR` or not and result in `true` or `false`, respectively.

```
MATCH (p:Person)-[r]->()
RETURN p.name AS name,
       r:WORKS_FOR AS isNotManager
```

name	isNotManager
"Alice"	false

name	isNotManager
"Cecil"	true
"Cecilia"	true
"Charlie"	true
"Daniel"	false
Rows: 5	

If `r` is `null`, then the label expression predicate, e.g. `r:WORKS_FOR|REPORTS_TO`, results in `null`. The function coalesce allows to turn `null` into a default value.

```
MATCH (p:Person)
OPTIONAL MATCH ()-[r]->(p)
RETURN DISTINCT
  p.name AS name,
  coalesce(r:WORKS_FOR|REPORTS_TO, false) AS hasReports
```

name	hasReports
"Alice"	true
"Cecil"	false
"Cecilia"	false
"Charlie"	false
"Daniel"	true
"Eskil"	true
Rows: 6	

Path pattern expressions

Similar to `EXISTS` subqueries, path pattern expressions can be used to assert whether a specified path exists at least once in a graph. While existential subqueries are more powerful and capable of performing anything achievable with path pattern expressions, path pattern expressions are more concise.

For more information about graph pattern matching in Cypher, see Patterns.

Rules

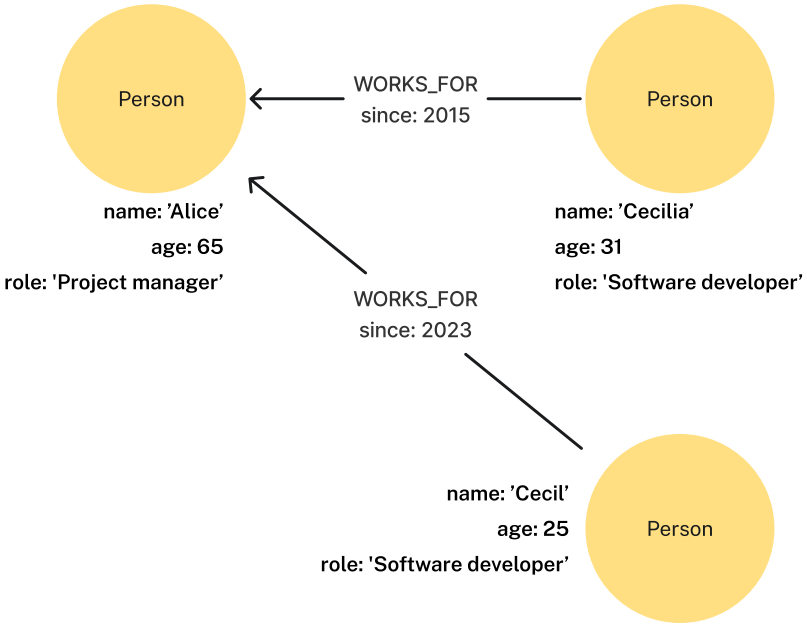
Path pattern expressions have the following restrictions (use cases that require extended functionality should consider using existential subqueries instead):

- Path pattern expressions may only use a subset of graph pattern semantics.
- A path pattern expression must be a path pattern of length greater than zero. In other words, it must contain at least one relationship or variable-length relationship.
- Path pattern expressions may not declare new variables. They can only reference existing variables.

- Path pattern expressions may only be used in positions where a boolean expression is expected. The following sections will demonstrate how to use path pattern expressions in a `WHERE` clause.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query in an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager'}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer'}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer'}),
       (cecil)-[:WORKS_FOR {since: 2023}]->(alice),
       (cecilia)-[:WORKS_FOR {since: 2015}]->(alice)
```

Examples

Simple path pattern expression

```
MATCH (employee:Person)
WHERE (employee)-[:WORKS_FOR]->(:Person {name: 'Alice'})
RETURN employee.name AS employee
```

Result

employee
"Cecil"
"Cecilia"
Rows: 2

Path pattern expression including boolean operators `NOT` *and* `AND`

```
MATCH (employee:Person)
WHERE NOT employee.name = 'Cecil' AND (employee)-[:WORKS_FOR]->(:Person {name: 'Alice'})
```

```
RETURN employee.name AS employee
```

Result

employee
"Cecilia"
Rows: 1

For more information about Cypher's boolean operators, see Predicate expressions → Boolean operators.

Patterns can be placed inside expressions.

Pattern inside an expression

```
RETURN NOT (:Person {name: "Alice"})<-[:WORKS_FOR {since: 2023}]-(:Person) AS patternCheck
```

Result

patternCheck
false
Rows: 1

`NOT` is required in the above example to make the query a valid predicate expression. Without it, the query would be invalid because the expression would not return a `BOOLEAN` value.

The `exists()` function can be used to check for the presence of a pattern. Note that this function not as versatile as the `EXISTS` subqueries.

Using the `exists()` function to check if a pattern exists

```
RETURN exists((:Person)-[:WORKS_FOR {since: 2023}]->(:Person {name: "Alice"})) AS patternCheck
```

Result

patternCheck
TRUE
Rows: 1

Expressions similar to path pattern expressions

Since any expression can be wrapped in parentheses, some are very similar to path pattern expressions. For example:

`(p:Person)` is not a path pattern expression since it does not have at least one relationship or variable-length relationship. Instead, `(p:Person)` is the label expression predicate `p:Person` in parentheses.

Parenthesized label expression predicate

```
MATCH (employee:Person)-[:WORKS_FOR]->(p)
  WHERE (p:Person)
RETURN employee.name AS employee, (p:Person) AS workForAPeron
```

The query result however is the same for both `p:Person` and `(p:Person)`:

```
MATCH (employee:Person)-[:WORKS_FOR]->(p)
  WHERE p:Person
RETURN employee.name AS employee, p:Person AS workForAPerson
```

Result

employee	workForAPerson
"Cecil"	true
"Cecilia"	true
Rows: 2	

`(p)` is not a path pattern expression since it does not have at least one relationship or variable-length relationship. Instead, `(p)` is the variable `p` in parentheses.

`(p)` does result in the value of variable `p`. If `p` is not a Boolean, then `(p)` does not result in a Boolean either. Hence, the following query throws an error:

Parenthesized node variable

```
MATCH (employee:Person)-[:WORKS_FOR]->(p)
  WHERE (p)
RETURN employee.name AS employee
```

```
Invalid input 'Node' for `p`. Expected to be Boolean.
```

Type predicate expressions

A type predicate expression can be used to verify the type of a variable, literal, property or other Cypher expression.

Syntax

```
<expr> IS :: <TYPE>
```

Where `<expr>` is any Cypher expression and `<TYPE>` is a Cypher type. For all available Cypher types, see the section on types and their synonyms.

Verify the type of a Cypher expression

```
UNWIND [42, true, 'abc', null] AS val
```

```
RETURN val, val IS :: INTEGER AS isInteger
```

val	isInteger
42	true
true	false
'abc'	false
null	true

Rows: 4

Vector values and supertypes

Introduced in Neo4j 2025.10 | Cypher 25 only

The following examples use `VECTOR` values (constructed using the `vector()` function) with `3` dimensions and `INTEGER8` coordinate type.

Verify a `VECTOR<TYPE>(DIMENSION)` value against a `VECTOR` the same coordinate type and dimension

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR<INTEGER8>(3) AS isVector
```

Result

isVector
true

Rows: 1

Although it is not possible to store `VECTOR` values without a coordinate type and dimension, it is possible to use the `VECTOR` supertype (encompassing all combinations of `VECTOR<TYPE>(DIMENSION)`) in type predicate expressions.

Verify a `VECTOR<TYPE>(DIMENSION)` value against the `VECTOR` supertype

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR AS isVector
```

Result

isVector
true

Rows: 1

The same is true for the supertypes `VECTOR<TYPE>` (which encompasses all `VECTOR` values with the same coordinate type regardless of the dimension) and `VECTOR(DIMENSION)` (which encompasses all `VECTOR` values with the same dimension regardless of the coordinate type).

Verify a `VECTOR<TYPE>(DIMENSION)` value against the `VECTOR<TYPE>` supertype

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR<INTEGER8> AS isInteger8Vector
```

Result

isInteger8Vector
true
Rows: 1

Verify a `VECTOR<TYPE>(DIMENSION)` value against the `VECTOR(DIMENSION)` supertype

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR(3) AS isDimension3Vector
```

Result

isDimension3Vector
true
Rows: 1

If the coordinate type or dimension in the `VECTOR` does not match what is specified in the type predicate expression, the result will be `false`.

Verify a `VECTOR<TYPE>(DIMENSION)` value against a `VECTOR` with a different dimension

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR(5) AS isDimension5Vector
```

Result

isDimension5Vector
false
Rows: 1

Verify a `VECTOR<TYPE>(DIMENSION)` value against a `VECTOR` with a different coordinate type

```
WITH vector([1, 2, 3], 3, INTEGER8) AS vector
RETURN vector IS :: VECTOR<INTEGER16> AS isInteger16Vector
```

Result

isInteger16Vector
false
Rows: 1

Type predicate expressions with NOT

It is also possible to verify that a Cypher expression is not of a certain type, using the negated type predicate expression `IS NOT ::`.

```
UNWIND [42, true, 'abc', null] AS val
RETURN val, val IS NOT :: STRING AS notString
```

val	notString
42	true
true	true
'abc'	false
null	false
Rows: 4	

Type predicate expressions for null

All Cypher types includes the `null` value. Type predicate expressions can be appended with `NOT NULL`. This means that `IS ::` returns `true` for all expressions evaluating to `null`, unless `NOT NULL` is appended.

```
RETURN
  NULL IS :: BOOLEAN AS isBoolean,
  NULL IS :: BOOLEAN NOT NULL AS isNotNullBoolean
```

isBoolean	isNotNullBoolean
true	false
Rows: 1	

Likewise, `IS NOT ::` returns `false` for all expressions evaluating to `null`, unless the type is appended with `NOT NULL`.

```
RETURN
  (null + 1) IS NOT :: DATE AS isNotDate,
  (null + 1) IS NOT :: DATE NOT NULL AS isNotNotNullDate
```

isNotDate	isNotNotNullDate
false	true
Rows: 1	

It is also possible to check whether a value is the only `null` value using the `null` type.

```
RETURN NULL IS :: NULL AS isNull
```

isNull
true
Rows: 1

Closed dynamic union types (`INNER_TYPE_1 | INNER_TYPE_2...`) cannot be declared as `NOT NULL`. Instead, all the inner types should be individually declared as not nullable to achieve this behavior.

Note that all inner types in a closed dynamic union must be either nullable, or not nullable. This is because `null` values cannot be attributed to a specific type. A syntax error will be raised if the inner types are not of the same nullability.

```
RETURN 1 IS :: INTEGER NOT NULL | FLOAT
```

GQLSTATUS error chain

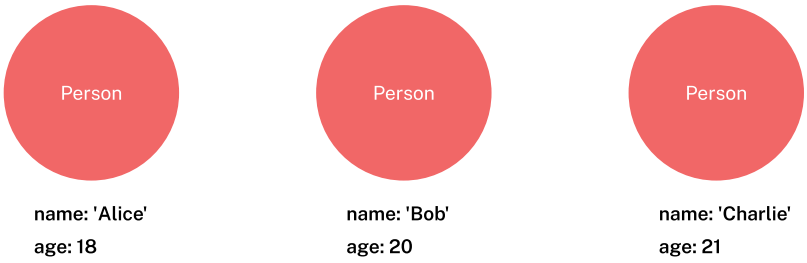
42N63: error: syntax error or access rule violation - inner type with different nullability. All inner types in a Closed Dynamic Union must be nullable, or be appended with 'NOT NULL'.

42001: error: syntax error or access rule violation - invalid syntax

Type predicate expression for properties

Type predicate expressions can also be used to filter out nodes or relationships with properties of a certain type.

A graph containing the following nodes is used for the example below:



The following query finds all `Person` nodes with an `age` property that is an `INTEGER` with a value greater than `18`.

```
MATCH (n:Person)
WHERE n.age IS :: INTEGER AND n.age > 18
RETURN n.name AS name, n.age AS age
```

name	age
'Charlie'	21
Rows: 1	

The type `PROPERTY VALUE` can also be used to check whether a type is storable as a property. Types not

storable in properties, such as `MAP`, will return `false` when checked with `IS :: PROPERTY VALUE`.

Type predicate expressions for numbers of different sizes

For numerical values passed in as parameters, Cypher does not take the size of the number into account. Cypher will therefore regard any exact numerical parameter as an `INTEGER` regardless of its declared size. For example, an `INT16` or an `INT32` passed through from a language library will both be treated by Cypher as an `INTEGER`. Note that any exact numerical parameter used must fit within the range of an `INT64`.

```
RETURN $int16param IS :: INTEGER AS isInteger
```

isInteger
true
Rows: 1

More information about parameters can be found here.

The vector-only coordinate types cannot be used as the numeric type to verify against in type predicate expressions.

Syntactical variations of type predicate expressions

Type predicate expressions allow for some alternative syntax:

```
<expr> IS TYPED <TYPE>
```

```
<expr> :: <TYPE>
```

For verifying that an expression is not of a certain type, the following alternative syntax is supported:

```
<expr> IS NOT TYPED <TYPE>
```

Use of `ANY` and `NOTHING` types

`ANY` is a supertype which matches values of all types. `NOTHING` is a type containing an empty set of values. This means that it returns `false` for all values.

```
RETURN 42 IS :: ANY AS isOfTypeAny, 42 IS :: NOTHING AS isOfTypeNothing
```

isOfTypeAny	isOfTypeNothing
true	false
Rows: 1	

Closed Dynamic Unions

Closed dynamic union types allow for the testing of multiple types in the same predicate.

```
UNWIND [42, 42.0, "42"] as val
RETURN val, val IS :: INTEGER | FLOAT AS isNumber
```

val	isNumber
42	true
42.0	true
"42"	false
Rows: 3	

Sized numeric values (such as `INT8`, `INT16`, `FLOAT32`) are not valid in type predicate expressions testing multiple types. Only the base types `INTEGER` and `FLOAT` are supported. For example, `RETURN 1 IS :: FLOAT | INTEGER` is valid, but `RETURN 1 IS :: FLOAT | INT8` is not.

List Types

Type predicate expressions can be used for `LIST` types, where the inner type of the elements in the list must be specified. If the inner type is not relevant, then the `ANY` type may be used.

For a `LIST` type check to return `true`, all values in the list must match the inner type.

```
UNWIND [[42], [42, null], [42, 42.0]] as val
RETURN val, val IS :: LIST<INTEGER> AS isIntList
```

val	isIntList
[42]	true
[42, null]	true
[42, 42.0]	false
Rows: 3	

An empty list will match on all inner types, even the `NOTHING` type.

```
RETURN
    [] IS :: LIST<NOTHING> AS isNothingList,
    [] IS :: LIST<INTEGER> AS isIntList,
    [] IS :: LIST<FLOAT NOT NULL> AS isFloatNotNullList
```

isNothingList	isIntList	isFloatNotNullList
true	true	true
Rows: 1		

Lists can be combined with closed dynamic union types to create tests for heterogeneous lists.

```
WITH [1, 0, true, false] AS booleanList
RETURN booleanList IS :: LIST<BOOLEAN | INTEGER> as isMixedList
```

isMixedList
true
Rows: 1

Node and relationship operators

Node and relationship operators allow you to manipulate and query `NODE` and `RELATIONSHIP` property values. Cypher contains the following node and relationship operators:

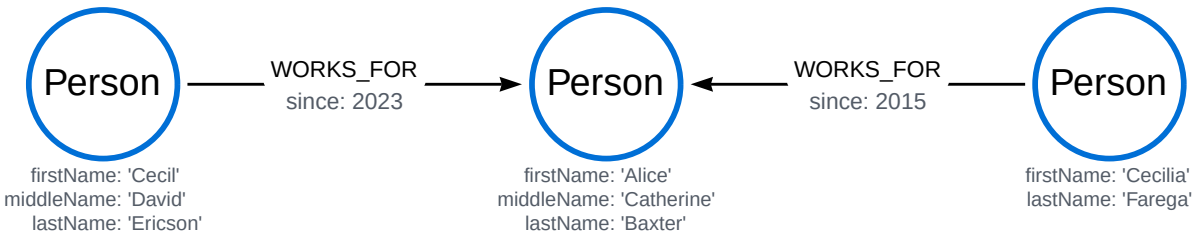
- Static property access: dot operator (`.`)
- Dynamic property access: subscript operator (`[]`)

For functions that return metadata about `NODE` and `RELATIONSHIP` values, see:

- `elementId()`
- `endNode()`
- `id()`
- `keys()`
- `labels()`
- `properties()`
- `startNode()`
- `type()`

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (alice:Person {firstName:'Alice', middleName: 'Catherine', lastName: 'Baxter'}),
       (cecil:Person {firstName: 'Cecil', middleName: 'David', lastName: 'Ericson'}),
       (cecilia:Person {firstName: 'Cecilia', lastName: 'Farega'}),
       (cecil)-[:WORKS_FOR {since: 2023}]->(alice),
       (cecilia)-[:WORKS_FOR {since: 2015}]->(alice)
```

Static property access

Property values can be accessed statically by specifying a property name after the `.` operator.

Access node properties statically

```
MATCH (p:Person)
RETURN p.firstName AS name
```

Result

name
"Alice"
"Cecil"
"Cecilia"
Rows: 3

Access node and relationship properties statically

```
MATCH (employee:Person)-[r:WORKS_FOR]->(manager:Person)
RETURN employee.firstName AS employee,
       r.since AS employedSince,
       manager.firstName AS manager
```

Result

employee	employedSince	manager
"Cecil"	2023	"Alice"
"Cecilia"	2015	"Alice"
Rows: 2		

Dynamic property access

Property values can be accessed dynamically by using the subscript operator, `[]`.

Access properties dynamically using a variable

```
LET nodeProperty = 'lastName'
MATCH (p:Person)
RETURN p[nodeProperty] AS lastName
```

Result

lastName
"Baxter"
"Ericson"
"Farega"
Rows: 3

Parameters

```
{
  "propertyName": "middleName"
}
```

Access properties dynamically using a parameter

```
MATCH (p:Person)
RETURN p[$propertyName] AS middleName
```

Result

middleName
"Catherine"
"David"
null
Rows: 3

Handling `null` values

If a property (or property value) is missing in an expression that uses tries to access a property statically or dynamically, the whole expression will evaluate to `null`. The query below performs a string concatentation on the `firstName`, `middleName`, and `lastName` properties on `Person` nodes. Note that `null` is returned for `Cecilia`, who lacks a `middleName` property.

String concatenation using node properties

```
MATCH (p:Person)
RETURN p.firstName || ' ' || p.middleName || ' ' || p.lastName AS fullName
```

Result

fullName
"Alice Catherine Baxter"
"Cecil David Ericson"
null

fullName
Rows: 3

The `coalesce()` function can be used to skip the first `null` values in an expression. In the below example, it replaces the first `null` value found with an empty `STRING`.

Use the `coalesce()` function to skip `null` values

```
MATCH (p:Person)
RETURN p.firstName || coalesce(' ' + p.middleName, '') || ' ' || p.lastName AS fullName
```

Result

fullName
"Alice Catherine Baxter"
"Cecil David Ericson"
"Cecilia Farega"
Rows: 3

Mathematical operators

Cypher contains the following mathematical operators:

- Addition or unary addition: `+`
- Subtraction or unary minus: `-`
- Multiplication: `*`
- Division: `/`
- Modulo division: `%`
- Exponentiation: `^`

For additional mathematical expressions, see:

- Logarithmic functions
- Numeric functions
- Trigonometric functions

Examples

Addition operator (`+`)

```
RETURN 10 + 5 AS result
```

Result

result
15
Rows: 1

Unary addition operator (`+`)

```
RETURN + 5 AS result
```

Result

result
5
Rows: 1

Subtraction operator (`-`)

```
RETURN 10 - 5 AS result
```

Result

result
5
Rows: 1

Unary subtraction operator (`-`)

```
RETURN - 5 AS result
```

Result

result
-5
Rows: 1

Multiplication operator (`*`)

```
RETURN 10 * 5 AS result
```

Result

result
50
Rows: 1

Division operator (`/`)

```
RETURN 10 / 5 AS result
```

Result

result
2
Rows: 1

Modulo division operator (`%`)

```
RETURN 10 % 3 AS result
```

Result

result
1
Rows: 1

Exponentiation operator (`^`)

```
RETURN 10 ^ 5 AS result
```

Result

result
100000.0
Rows: 1

Modifying properties using mathematical operators

```
CREATE (p:Product {price: 10})
SET p.discountPrice = p.price * (1 - 0.15)
RETURN p.discountPrice AS discountPrice
```

Result

discountPrice
8.5
Rows: 1

Division by zero and `null` operands

The behavior of division (`/`) and modulo division (`%`) with a zero divisor or a `null` operand depends on the operand types.

- Integer division or modulo by zero throws an error.
- If either operand is a float, the result is `Infinity`, `-Infinity`, or `NaN` and no error is thrown.
- If either operand is `null`, the result is `null`.

Division and modulo by zero or `null`

Expression	Result
`1 / 0`	Error: division by zero
`1 % 0`	Error: division by zero
`1.0 / 0`	`Infinity`
`-1 / 0.0`	`-Infinity`
`0.0 / 0`	`NaN`
`1.0 % 0`	`NaN`
`null / 0`	`null`
`null / 0.0`	`null`
`null % 0`	`null`
`1 / null`	`null`

Prior to Neo4j 2026.08, `null / 0` throws a division-by-zero error instead of returning `null`.

Order of precedence

The following table details the order of precedence of the mathematical operators in ascending order. Note, the lower the precedence level, the higher the binding power of the operands. Also note, if parentheses `()` are used, any operation within them takes precedence, overriding the default order of precedence.

Mathematical operators: order of precedence

Precedence	Operators	Associativity
1	Unary negation (`-`), unary positive (`+`)	Right to left
2	Exponentiation (`^`)	Right to left
3	Multiplication (`*`), division (`/`), modulo division (`%`)	Left to right
4	Addition (`+`), subtraction (`-`)	Left to right

Operators within the same precedence group are evaluated based on associativity.

The order of precedence ensures that the following two expressions return the same result.

Expression with several different mathematical operations

```
RETURN -50 + 6 * 3 - 100 / 5 ^ 2 % 12 AS result1,
       (((-50) + (6 * 3)) - ((100 / (5 ^ 2)) % 12)) AS result2
```

Result

result1	result2
`-36.0`	`-36.0`
Rows: 1	

Order of evaluation

Precedence	Operation	Result
1	Unary negation (`-50`)	`-50`
2	Exponentiation (`5 ^ 2`)	`25`
3	Multiplication (`6 * 3`)	`18`
3	Division (`100 / 25`)	`4`
3	Modulo division (`4 % 12`)	`4`
4	Addition (`-50 + 18`)	`-32`
4	Subtraction (`-32 - 4`)	`-36`

Only bracketing some of the operations within parentheses will change the order of precedence and may, therefore, change the result of an expression.

Parenthesizing single operation

```
RETURN (-50 + 6) * 3 - 100 / 5 ^ 2 % 12 AS result
```

Result

result
`-136.0`
Rows: 1

Changed order of evaluation

Precedence	Operation	Result
1	Parenthesized operation (`-50 + 6`)	`-44`
2	Exponentiation (`5 ^ 2`)	`25`
3	Multiplication (`-44 * 3`)	`132`
3	Division (`100 / 25`)	`4`
3	Modulo division (`4 % 12`)	`4`
4	Subtraction (`-132 - 4`)	`-136`

String concatenation operators

Cypher contains two operators for the concatenation of `STRING` values:

- `||`
- `+`

The two operators are functionally equivalent on `STRING` values. However, `||` is GQL conformant, while `+` is not.

When `+` is used to concatenate a `STRING` with a non-`STRING` value (excluding `LIST` values), the non-`STRING` value is implicitly converted to a `STRING` using `toString()`. `||` does not perform this implicit conversion, so a `STRING` can only be concatenated with another `STRING` when using `||`; non-`STRING` values must first be explicitly converted using `toString()`.

Implicit `toString()` conversion with `+`

```
RETURN 'The number is: ' + 42 AS result
```

Result

result
"The number is: 42"
Rows: 1

For additional expressions that evaluate to `STRING` values, see String functions.

Expressions can also be embedded directly inside a string using string interpolation, which is an alternative to concatenation for building strings.

Examples

`||` and `+`

```
RETURN 'Neo' || '4j' AS result1,
       'Neo' + '4j' AS result2
```

Result

result1	result2
"Neo4j"	"Neo4j"
Rows: 1	

The `toString()` function can be used to concatenate non-`STRING` values into a `STRING` value.

Concatenation using `toString()` function

```
RETURN 'The number is: ' || toString(42) AS result
```

Result

result
"The number is: 42"
Rows: 1

Cypher does not insert spaces when concatenating `STRING` values.

Adding whitespaces in `STRING` concatenation

```
RETURN 'Alpha' || 'Beta' AS result1,
       'Alpha' || ' ' || 'Beta' AS result2
```

Result

result1	result2
"AlphaBeta"	"Alpha Beta"
Rows: 1	

Concatenating `STRING` properties

```
CREATE (p:Person {firstName: 'Keanu', lastName: 'Reeves'})
SET p.fullName = p.firstName || ' ' || p.lastName
RETURN p.fullName AS fullName
```

Result

fullName
"Keanu Reeves"
Rows: 1

Adding separators in `STRING` concatenation

```
RETURN 'Hello' || ', ' || 'World' AS result
```

Result

result
"Hello, World"
Rows: 1

Adding prefix, suffix, and separators in `STRING` concatenation

```
RETURN 'My favorite fruits are: ' || 'apples' || ', ' || 'bananas' || ', and ' || 'oranges' || '.' AS
result
```

Result

result
"My favorite fruits are: apples, bananas, and oranges."
Rows: 1

String concatenation, `LIST` values, and `null`

`STRING` values in a `LIST` can be concatenated using the `reduce()` function.

Concatenate `STRING` values in a `LIST`

```
WITH ['Neo', '4j'] AS list
RETURN reduce(acc = '', item IN list| acc || item) AS result
```

Result

result
"Neo4j"
Rows: 1

The following query uses the `head()` function to start the accumulator with the first `item` in the `list`, while the `tail()` function returns the remaining items in the `list`. The `reduce()` function then concatenates these items with commas.

Add prefix and a separator (,) in a `STRING` concatenated from `STRING` values in a `LIST`

```
WITH ['Apples', 'Bananas', 'Oranges'] AS list
RETURN 'My favorite fruits are: ' || reduce(acc = head(list), item IN tail(list) | acc || ', ' || item) ||
'.' AS result
```

Result

result
"My favorite fruits are: Apples, Bananas, Oranges."
Rows: 1

Concatenating a `STRING` value with `null` returns `null`. To skip the first `null` value in a list of expressions, use the `coalesce()` function.

In the following query, `coalesce()` is used with `reduce()` to replace each `null` value in the `LIST` with an empty `STRING` (`''`). This ensures that all `null` values are effectively skipped, allowing the `reduce()` function to concatenate the remaining `STRING` values.

Using `reduce()` and `coalesce()` to skip `null` values when concatenating a `LIST`

```
WITH ['Apples', null, 'Bananas', null, 'Oranges', null] AS list
RETURN 'My favorite fruits are: ' || reduce(acc = head(list), item IN tail(list) | acc || coalesce(', ' ||
item, '')) || '.' AS result
```

Result

result
"My favorite fruits are: Apples, Bananas, Oranges."
Rows: 1

If a `LIST` is empty, `reduce()` will return `null` because there are no elements to process. In such cases, `coalesce()` can be used to replace the `null` with a default value (e.g., `'none'`).

Using `reduce()` and `coalesce()` to handle empty `LIST` values

```
UNWIND [['Apples', 'Bananas', 'Oranges'], ['Pears'], []] AS list
RETURN 'My favorite fruits are: ' || coalesce(reduce(acc = head(list), item IN tail(list) | acc || ', ' ||
item), 'none') || '.' AS result
```

Result

result
"My favorite fruits are: Apples, Bananas, Oranges."
"My favorite fruits are: Pears."
"My favorite fruits are: none."
Rows: 3

Additionally, list comprehension allows concatenating a `STRING` value to each item in a `LIST` to generate a new `LIST` of modified `STRING` values.

List comprehension with `STRING` concatenation on `LIST` items

```
WITH ['Apples', 'Bananas', 'Oranges'] AS list
RETURN [item IN list | 'Eat more ' || item || '!'] AS result
```

Result

result
["Eat more Apples!", "Eat more Bananas!", "Eat more Oranges!"]
Rows: 1

Temporal operators

Cypher contains the following temporal operators:

- Adding a `DURATION` to either a temporal instant value or another `DURATION`: `+`
- Subtracting a `DURATION` from either a temporal instant value or another `DURATION`: `-`
- Multiplying a `DURATION` with a number (`INTEGER` or `FLOAT`): `*`
- Dividing a `DURATION` by a number: `/`

For additional expressions that evaluate to a temporal value type, see:

- Temporal functions — instant types
- Temporal functions — duration types

Value returned from temporal operators

Operator	Left-hand operand	Right-hand operand	Result value type
`+`	Temporal instant	`DURATION`	The type of the temporal instant
`+`	`DURATION`	Temporal instant	The type of the temporal instant
`-`	Temporal instant	`DURATION`	The type of the temporal instant
`+`	`DURATION`	`DURATION`	`DURATION`
`-`	`DURATION`	`DURATION`	`DURATION`
`*`	`DURATION`	Number	`DURATION`
`*`	Number	`DURATION`	`DURATION`
`/`	`DURATION`	Number	`DURATION`

Adding and subtracting `DURATION` values

`DURATION` values can be added and subtracted from temporal instant values, such as `LOCAL DATETIME`. In the below example, the `localdatetime()` function is used to create a `LOCAL DATETIME` value, and the `duration()` function is used to create a `DURATION` value.

Add and subtract a `DURATION` value to/from a `LOCAL DATETIME`

```
WITH localdatetime({year:1984, month:10, day:11, hour:12, minute:31, second:14}) AS aDateTime,
     duration({years: 12, nanoseconds: 2}) AS aDuration
RETURN aDateTime + aDuration AS addition,
       aDateTime - aDuration AS subtraction
```

Result

addition	subtraction
1996-10-11T12:31:14.000000002	1972-10-11T12:31:13.999999998
Rows: 1	

When adding or subtracting a `DURATION` that results in a non-existing date, Cypher truncates the date to the nearest valid date. For example, if adding 1 month to January 31st, the result will not be February 31st (an invalid date) but February 28th (or 29th in a leap year).

Add 1 month to January 31st

```
RETURN date("2011-01-31") + duration("P1M") AS truncatedDate
```

Result

truncatedDate
2011-02-28
Rows: 1

When adding two `DURATION` values to a temporal instant value, the order in which the durations are applied affects the result.

Add two `DURATION` values to a `DATE`

```
RETURN (date("2011-01-31") + duration("P1M")) + duration("P12M") AS date1,
       date("2011-01-31") + (duration("P1M") + duration("P12M")) AS date2
```

In `date1`, durations are added one after the other, so the date is truncated after adding the first month, resulting in `2012-02-28`. In `date2`, the durations are combined first, and truncation only happens once, resulting in `2012-02-29`.

Result

date1	date2
2012-02-28	2012-02-29
Rows: 1	

Ignored components

When adding or subtracting a `DURATION` value to a temporal instant value, any `DURATION` components that do not apply to that specific type are ignored. (For information about what components are supported by temporal instant values, see Components of temporal instants). For example, when adding a `DURATION` to a `DATE`, only the `year`, `month`, and `day` components of a `DURATION` value are considered, while `hour`, `minute`, `second`, and `nanosecond` are ignored. This behavior also applies to `LOCAL TIME` and `ZONED TIME`.

Add and subtract a `DURATION` to/from a `DATE`

```
WITH date({year:1984, month:10, day:11}) AS aDate,
     duration({years: 12, nanoseconds: 2}) AS aDuration
RETURN aDate + aDuration AS addition,
       aDate - aDuration AS subtraction
```

Result

addition	subtraction
1996-10-11	1972-10-11
Rows: 1	

Multiplying and dividing `DURATION` values

When multiplying or dividing a `DURATION`, each component is handled separately. In multiplication, the value of each component is multiplied by the given factor, while in division, each component is divided by the given number. If the result of the division does not fit into the original components, it overflows into smaller components (e.g. converting days into hours). This overflow also occurs when multiplying with fractions.

Multiply and divide a `DURATION` value

```
WITH duration({days: 14, minutes: 12, seconds: 70, nanoseconds: 1}) AS aDuration
RETURN aDuration,
       aDuration * 2 AS multipliedDuration,
       aDuration / 3 AS dividedDuration
```

Result

aDuration	multipliedDuration	dividedDuration
P14DT13M10.000000001S	P28DT26M20.000000002S	P4DT16H4M23.333333333S
Rows: 1		

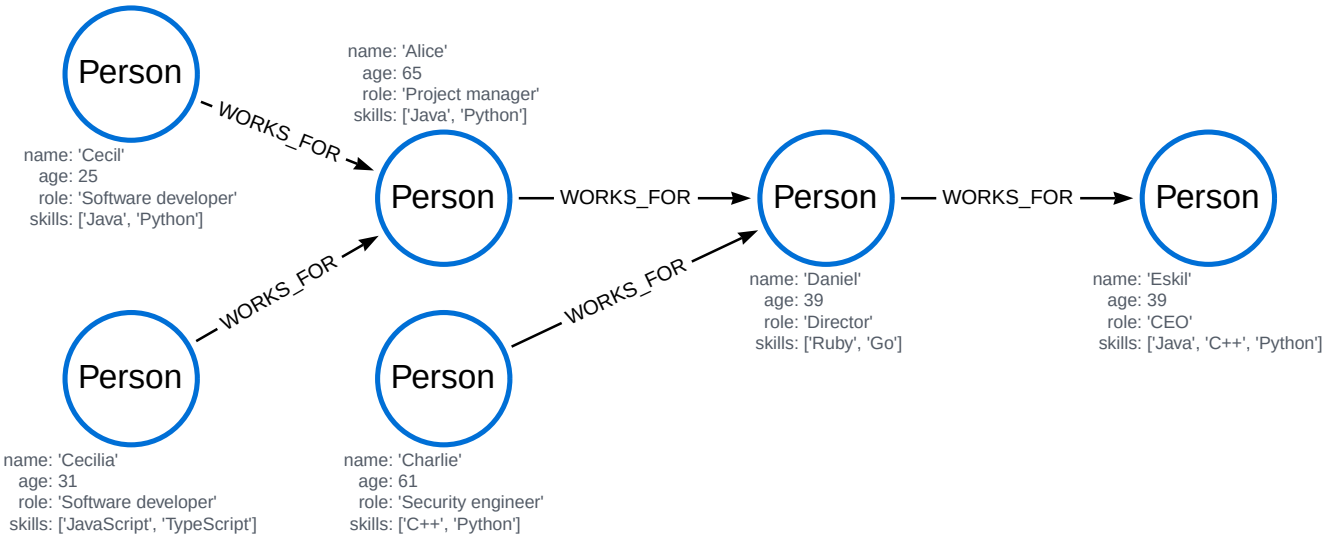
List expressions

List expressions allow you to manipulate and query `LIST` values in Cypher.

For more expressions that evaluate to a `LIST` value, see List functions.
For information about how to check membership in a `LIST` using the `IN` operator, see Predicates → List operators.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (alice:Person {name:'Alice', age: 65, role: 'Project manager', skills: ['Java', 'Python']}),
       (cecil:Person {name: 'Cecil', age: 25, role: 'Software developer', skills: ['Java', 'Python']}),
       (cecilia:Person {name: 'Cecilia', age: 31, role: 'Software developer', skills: ['JavaScript',
'TypeScript']}),
       (charlie:Person {name: 'Charlie', age: 61, role: 'Security engineer', skills: ['C++', 'Python']}),
       (daniel:Person {name: 'Daniel', age: 39, role: 'Director', skills: ['Ruby', 'Go']}),
       (eskil:Person {name: 'Eskil', age: 39, role: 'CEO', skills: ['Java', 'C++', 'Python']}),

       (cecil)-[:WORKS_FOR]->(alice),
       (cecilia)-[:WORKS_FOR]->(alice),
       (charlie)-[:WORKS_FOR]->(daniel),
       (alice)-[:WORKS_FOR]->(daniel),
       (daniel)-[:WORKS_FOR]->(eskil)
```

List element access

The subscript operator, `[]`, can be used to access specific elements in a `LIST`. `[0]` refers to the first element in a `LIST`, `[1]` to the second, and so on. `[-1]` refers to the last element in a `LIST`, `[-2]` to the penultimate element, and so on.

Access individual elements in a `LIST`

```
WITH [1, 2, 3, 4] AS list
RETURN list[0] AS firstElement,
       list[2] AS thirdElement,
       list[-1] AS finalElement
```

Result

firstElement	thirdElement	finalElement
1	3	4

Rows: 1

The index of the element in the `LIST` can be parameterized.

Parameters

```
{
  "myIndex" : 1
}
```

Access `LIST` elements with a parameter

```
WITH [1, 2, 3, 4] AS list
RETURN list[$myIndex] AS secondElement
```

Result

secondElement
2

Rows: 1

Access a `LIST` within a nested `LIST`

```
WITH [[1-6, 2], [3, 4], [5, 6]] AS nestedList
RETURN nestedList[1] AS secondList
```

Result

secondList
[3, 4]
Rows: 1

Access specific elements in a nested `LIST`

```
WITH [[1-7, 2], [3, 4], [5, 6]] AS nestedList
RETURN nestedList[1] AS secondList,
       nestedList[1][0] AS firstElementOfSecondList
```

Result

secondList	firstElementOfSecondList
[3, 4]	3
Rows: 1	

The index in the list element access can be any expression, including a variable.

Access list elements using a variable as an index

```
WITH [[1-8, 2], [3, 4], [5, 6]] AS nestedList, 2 AS listIndex
RETURN nestedList[listIndex] AS thirdList,
       nestedList[listIndex][listIndex - 1] AS secondElementOfThirdList
```

Result

thirdList	secondElementOfThirdList
[5, 6]	6
Rows: 1	

The `IN` operator, which checks for `LIST` membership, can be used together with `[]` to test whether an element exists in a nested `LIST`.

Check for membership in a nested `LIST`

```
WITH [[1-9, 2, 3], [4, 5, 6]] AS nestedList
RETURN 3 IN nestedList[0] AS elementPresent
```

Result

elementPresent
TRUE

elementPresent
Rows: 1

Attempting to reference an element outside the bounds of the `LIST` will return `null`, as will attempting to access elements from an empty `LIST`.

Out-of-bounds and empty `LIST` access

```
WITH [1, 2, 3, 4] AS list, [] AS emptyList
RETURN list[5] AS outOfBound, emptyList[0] AS emptyAccess
```

Result

outOfBound	emptyAccess
`null`	`null`
Rows: 1	

List slicing

`LIST` values can be sliced if a range is provided within the subscript operator. The bounds of the range are separated using two dots (`..`). This allows for extracting a subset of a `LIST` rather than a single element. List slicing is inclusive at the start of the range, but exclusive at the end (e.g. `list[start..end]` includes `start`, but excludes `end`).

Slice operations on a `LIST`

```
WITH [1, 2, 3, 4, 5, 6] AS list
RETURN list[2..4] AS middleElements,
       list[..2] AS noLowerBound,
       list[2..] AS noUpperBound
```

Result

middleElements	noLowerBound	noUpperBound
`[3, 4]`	`[1, 2]`	`[3, 4, 5, 6]`
Rows: 1		

Negative indexing in list slicing references elements from the end of the `LIST`; `..-1` excludes the last element, `..-2` excludes the last two elements, and so on.

Negative indexing and list slicing

```
WITH [1, 2, 3, 4, 5, 6] AS list
RETURN list[..-1] AS finalElementRemoved,
       list[..-2] AS finalTwoElementsRemoved,
       list[-3..-1] AS removedFirstThreeAndLast
```

Result

finalElementRemoved	finalTwoElementsRemoved	removedFirstThreeAndLast
[1, 2, 3, 4, 5]	[1, 2, 3, 4]	[4, 5]
Rows: 1		

When slicing nested `LIST` values, it is important to specify which level is sliced. The below example slices the outer `LIST` and returns the first two nested `LIST` values.

Slice outer `LIST`

```
WITH [[1-10, 2, 3], [4, 5, 6], [7, 8, 9]] AS nestedList
RETURN nestedList[0..2] AS slicedNestedList
```

Result

slicedNestedList
[[1-11, 2, 3], [4, 5, 6]]
Rows: 1

Slicing inner `LIST` values require two `[]` operators; the first `[]` accesses elements from the outer `LIST`, while the second slices or accesses elements from the inner `LIST`.

Slice inner `LIST`

```
WITH [[1-12, 2, 3], [4, 5, 6], [7, 8, 9]] AS nestedList
RETURN nestedList[1][0..2] AS slicedInnerList
```

Result

slicedInnerList
[4, 5]
Rows: 1

Accessing specific elements or a range of elements can also be used in combination with the `+` operator to create a new `LIST` with values inserted into specific sections of an existing `LIST` value.

Insert elements into specific positions of a `LIST`

```
WITH [1, 3, 4] AS list
RETURN list[0] + [2] + list[1..] AS newList
```

Result

newList
[1, 2, 3, 4]
Rows: 1

List concatenation

Cypher contains two list concatenation operators: `||` and `+`. `||` is GQL conformant, while `+` is not.

List concatenation using `||` and `+`

```
RETURN [1,2] || [3,4] AS list1,
       [1,2] + [3,4] AS list2
```

Result

list1	list2
`[1, 2, 3, 4]`	`[1, 2, 3, 4]`
Rows: 1	

Concatenate two `LIST` properties

```
MATCH (cecil:Person {name: 'Cecil'}), (cecilia:Person {name: 'Cecilia'})
RETURN cecil.skills || cecilia.skills AS combinedSkills
```

Result

combinedSkills
`["Java", "Python", "JavaScript", "TypeScript"]`
Rows: 1

If `null` is part of a concatenated `LIST`, `null` will be a part of the new `LIST`.

Concatenate `LIST` including `null`

```
RETURN [1, 2] || [3, null] AS listWithNull
```

Result

listWithNull
`[1, 2, 3, null]`
Rows: 1

For removing `null` values when concatenating `LIST` values, see List comprehension.

Add elements to a list

The `+` operator can add elements to the beginning or end of a `LIST` value. This is not possible using the `||` operator.

Add elements to the beginning and end of a `LIST`

```
WITH [1, 2, 3, 4] AS list
```

```
RETURN 0 + list AS newBeginning,
       list + 5 AS newEnd
```

Result

newBeginning	newEnd
[0, 1, 2, 3, 4]	[1, 2, 3, 4, 5]
Rows: 1	

To insert a `LIST` value into a nested `LIST`, the added `LIST` must itself be nested. If the added `LIST` is not nested, its elements are treated as individual elements, whereas if it is nested, it maintains the `LIST` structure within the nested `LIST`.

Add `LIST` values to a nested `LIST`

```
WITH [[1-13, 2], [3, 4]] AS nestedList
RETURN nestedList + [5, 6] AS nonNestedAddition,
       nestedList + [[5, 6]] AS nestedAddition
```

Result

nonNestedAddition	nestedAddition
[[1-14, 2], [3, 4], 5, 6]	[[1, 2], [3, 4], [5, 6]]
Rows: 1	

Add elements to the beginning of a `LIST` property

```
MATCH (cecil:Person {name: 'Cecil'})
SET cecil.skills = "Cypher" + cecil.skills
RETURN cecil.skills AS skillsList
```

Result

skillsList
["Cypher", "Java", "Python"]
Rows: 1

List comprehension

List comprehension is used to create new `LIST` values by iterating over existing `LIST` values and transforming the elements based on certain conditions or operations. This process effectively maps each element in the original `LIST` to a new value. The result is a new `LIST` that consists of the transformed elements.

Syntax

```
[item IN list [WHERE predicate] | [expression]]
```

The iteration step (`item IN list`) ensures that each element of a `list` is accessed one by one, while the `expression` optionally applies a transformation to each `item`, creating new `LIST` values containing the modified elements.

List comprehension without an expression

```
RETURN [x IN range(0,10) WHERE x % 2 = 0] AS result
```

Result

result
[0, 2, 4, 6, 8, 10]
Rows: 1

List comprehension without filtering

```
RETURN [x IN range(0,5) | x * 10] AS result
```

Result

result
[0, 10, 20, 30, 40, 50]
Rows: 1

List comprehension with both an expression and filtering

```
WITH [1, 2, 3, 4, 5] AS list
RETURN [n IN list WHERE n > 2 | n] AS filteredList
```

Result

filteredList
[3, 4, 5]
Rows: 1

The next example shows how to map a `LIST` using its indexes with a list comprehension. The `range()` function is used to generate the indexes from `0` to the last valid index of the `LIST`, and then each index is combined with its corresponding `LIST` value into a `STRING` value. The result is a `LIST` of `STRING` values formatted as `'index: value'`.

Map list elements using indexes

```
WITH [1,2,3,4] AS list
RETURN [listIndex IN range(0, size(list)-1) | toString(listIndex) || ': ' || toString(list[listIndex])] AS
mappedListElements
```

Result

mappedListElements
["0: 1", "1: 2", "2: 3", "3: 4"]
Rows: 1

The below query iterates over the `skills` property of each `Person` node and creates a new `LIST` by concatenating the `STRING` `" expert"` to each element in `skills`.

Modify `LIST` properties using list comprehension

```
MATCH (p:Person) WHERE p.skills IS NOT NULL
ORDER BY p.name
RETURN p.name AS name,
       [skill IN p.skills | skill + " expert"] AS modifiedSkills
```

Result

name	modifiedSkills
"Alice"	["Java expert", "Python expert"]
"Cecil"	["Cypher expert", "Java expert", "Python expert"]
"Cecilia"	["JavaScript expert", "TypeScript expert"]
"Charlie"	["C++ expert", "Python expert"]
"Daniel"	["Ruby expert", "Go expert"]
"Eskil"	["Java expert", "C++ expert", "Python expert"]
Rows: 6	

The next query uses the `collect()` function to gather all `Person` nodes into a `LIST`, and the `WHERE 'Python' IN person.skills` predicate filters that list to include only those nodes whose `skills` property contains `Python`.

List comprehension with a `WHERE` predicate

```
MATCH (p:Person)
RETURN [person IN collect(p) WHERE 'Python' IN person.skills | person.name] AS pythonExperts
```

Result

pythonExperts
["Alice", "Cecil", "Charlie", "Eskil"]
Rows: 1

List comprehension can be used to remove any unknown `null` values when concatenating `LIST` values.

List comprehension to remove `null` values during list concatenation

```
RETURN [x IN ([1, null, 3] || [null, 5, null]) WHERE x IS NOT NULL] AS listWithoutNull
```

Result

listWithoutNull
`[1, 3, 5]`
Rows: 1

Pattern comprehension

Pattern comprehension is used to create new `LIST` values by matching graph patterns and applying conditions to the matched elements, returning custom projections.

Syntax

```
[pattern [WHERE predicate] | expression]
```

The below query retrieves a list of names of people who work for `Alice` by using pattern comprehension extract the names of `employees` into a `LIST`.

Pattern comprehension on a fixed-length pattern

```
MATCH (alice:Person {name: 'Alice'})
RETURN [(employee:Person)-[:WORKS_FOR]->(alice) | employee.name] AS employees
```

Result

employees
`["Cecil", "Cecilia"]`
Rows: 1

Pattern comprehensions can include `WHERE` predicates.

Pattern comprehension including a `WHERE` predicate

```
MATCH (alice:Person {name: 'Alice'})
RETURN [(employee:Person)-[:WORKS_FOR]->(alice) WHERE employee.age > 30 | employee.name || ', ' ||
toString(employee.age)] AS employeesAbove30
```

Result

employeesAbove30
`["Cecilia, 31"]`
Rows: 1

Pattern comprehension can also match for variable-length patterns. However, pattern comprehension does not support the GQL conformant quantifier syntax.

Not allowed: variable-length pattern comprehension using GQL quantifier syntax

```
MATCH (cecil:Person {name: 'Cecil'})
RETURN [(cecil)-[:WORKS_FOR]->+(superior:Person) | superior.skills] AS superiorsSkills
```

Pattern comprehension only supports only the variable-length relationships syntax. The below query uses a pattern comprehension to collect the skills of all superiors in the chain above `Cecil`. The `reduce()` function concatenates these skills into a single `LIST`, and `UNWIND` is used to flatten this `LIST` before returning the distinct skills in a new `LIST`.

Allowed: variable-length pattern comprehension using variable-length relationship syntax

```
MATCH (cecil:Person {name: 'Cecil'})
WITH [(cecil)-[:WORKS_FOR*]->(superior:Person) | superior.skills] AS allSuperiorsSkills
WITH reduce(accumulatedSkills = [], superiorSkills IN allSuperiorsSkills | accumulatedSkills ||
superiorSkills) AS allSkills
UNWIND allSkills AS superiorsSkills
RETURN collect(DISTINCT superiorsSkills) AS distinctSuperiorsSkills
```

Result

distinctSuperiorsSkills
["Java", "Python", "Ruby", "Go", "C++"]
Rows: 1

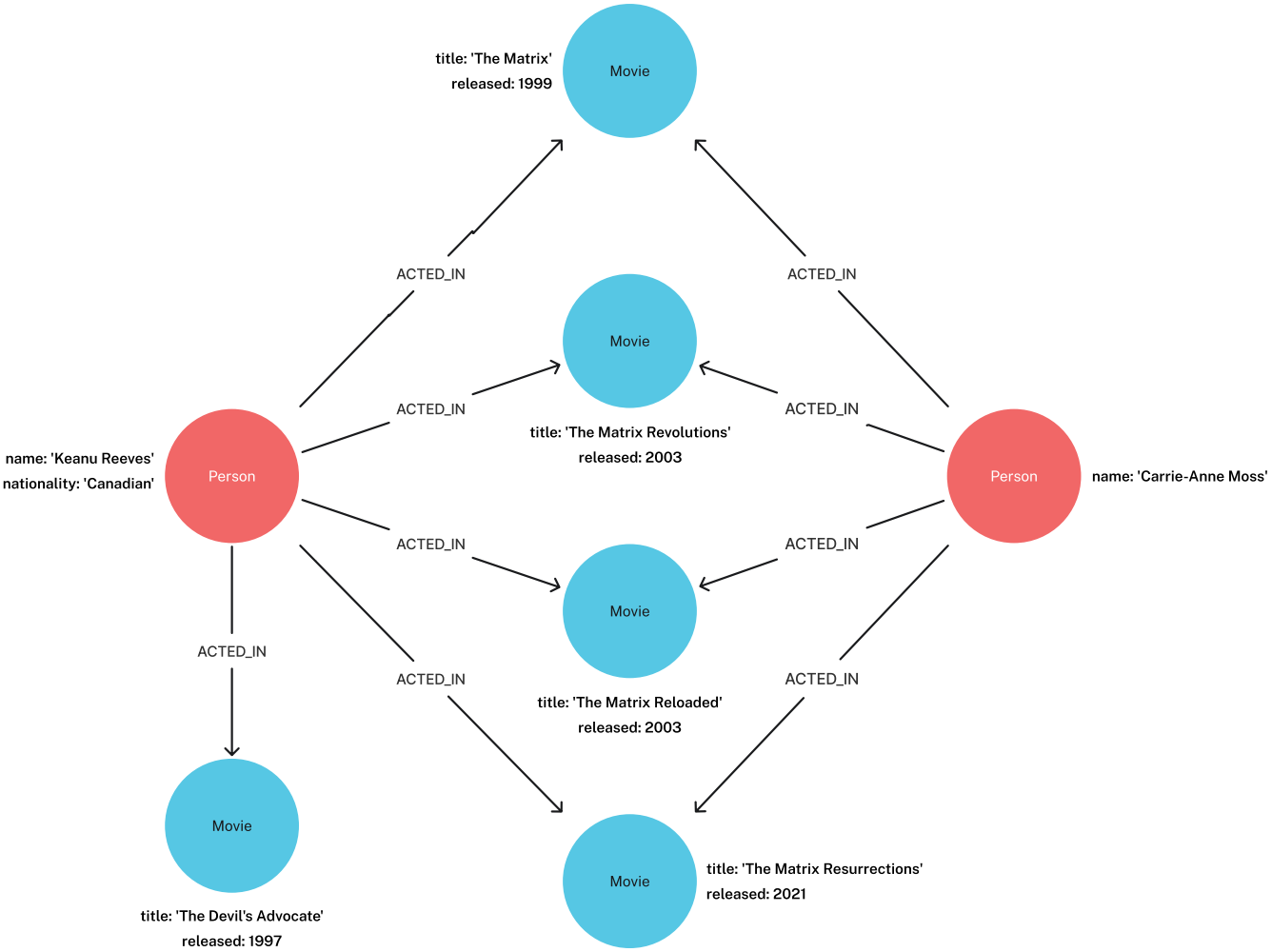
Map expressions

Map expressions allow you to manipulate and query `MAP` values in Cypher.

The `keys()` function can be used to return a `LIST` of all the keys in a `MAP`.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE (keanu:Person {name: 'Keanu Reeves', nationality: 'Canadian'}),
       (carrieAnne:Person {name: 'Carrie-Anne Moss'}),
       (theMatrixRevolutions:Movie {title: 'The Matrix Revolutions', released: 2003}),
       (theMatrixReloaded:Movie {title: 'The Matrix Reloaded', released: 2003}),
       (theMatrix:Movie {title: 'The Matrix', released: 1999}),
       (theDevilsAdvocate:Movie {title: 'The Devils Advocate', released: 1997}),
       (theMatrixResurrections:Movie {title: 'The Matrix Resurrections', released: 2021}),

       (keanu)-[:ACTED_IN]->(theMatrix),
       (keanu)-[:ACTED_IN]->(theMatrixRevolutions),
       (keanu)-[:ACTED_IN]->(theMatrixReloaded),
       (keanu)-[:ACTED_IN]->(theMatrixResurrections),
       (keanu)-[:ACTED_IN]->(theDevilsAdvocate),

       (carrieAnne)-[:ACTED_IN]->(theMatrix),
       (carrieAnne)-[:ACTED_IN]->(theMatrixRevolutions),
       (carrieAnne)-[:ACTED_IN]->(theMatrixReloaded),
       (carrieAnne)-[:ACTED_IN]->(theMatrixResurrections)
```

Map operators

Cypher contains two map operators:

- Statically access a value by key: dot operator (`.`).
- Dynamically access a value by key: subscript operator (`[]`).

Statically access a `MAP` value

`MAP` values can be accessed statically by specifying a key after the `.` operator.

Static `MAP` value access

```
WITH {a: 10, b: 20, c: 30} AS map
RETURN map.a AS firstValue,
       map.c AS lastValue
```

Result

firstValue	lastValue
10	30
Rows: 1	

To statically access a value in a nested `MAP`, use chained `.` operators. Each `.` operator traverses one level deeper into the nested structure.

Static access of a nested `MAP` value

```
WITH {a: 10, b: 20, c: 30, innerMap: {x: 100, y: 200, z: 300}} AS map
RETURN map.a AS firstOuterValue,
       map.innerMap.y AS secondInnerValue
```

Result

firstOuterValue	secondInnerValue
10	200
Rows: 1	

Node and relationship properties are stored as key-value pairs in a `MAP`. Properties can, therefore, be statically accessed using the `.` operator.

Statically access a node property

```
MATCH (p:Person)
RETURN p AS nodePropertyMaps,
       p.name AS accessedMapValue
```

Result

nodePropertyMaps	accessedMapValue
(:Person {nationality: "Canadian", name: "Keanu Reeves"})	"Keanu Reeves"
(:Person {name: "Carrie-Anne Moss"})	"Carrie-Anne Moss"
Rows: 2	

Dynamically access a `MAP` value

To dynamically access a `MAP` value, use the subscript operator, `[]`. The key can be provided by a variable or a parameter.

Dynamically access a `MAP` value using a variable

```
WITH {a: 10, b: 20, c: 30} AS map,
     'a' AS dynamicKey
RETURN map[dynamicKey] AS dynamicValue
```

Result

dynamicValue
10
Rows: 1

Parameters

```
{
  "dynamicKey" : "a"
}
```

Dynamically access a `MAP` value using a parameter

```
WITH {a: 10, b: 20, c: 30} AS map
RETURN map[$dynamicKey] AS dynamicValue
```

Result

dynamicValue
10
Rows: 1

Dynamically access a nested `MAP` value

```
WITH {a: 10, b: 20, c: 30, innerMap: {x: 100, y: 200, z: 300}} AS map,
     'z' AS dynamicInnerKey
RETURN map.innerMap[dynamicInnerKey] AS dynamicInnerValue
```

Result

dynamicInnerValue
300
Rows: 1

It is possible to dynamically access multiple values from a `MAP` by using list comprehension. This query iterates over the `LIST` `dynamicKeys` and retrieves their corresponding values from the `map`.

Dynamic `MAP` value access and list comprehension

```
WITH {a: 10, b: 20, c: 30} AS map,
     ['a', 'c'] AS dynamicKeys
RETURN [key IN dynamicKeys | map[key]] AS dynamicValue
```

Result

dynamicValue
[10, 30]
Rows: 1

Dynamically access a node property value

```
MATCH (p:Person)
LET dynamicValue = 'name'
RETURN p[dynamicValue] AS names
```

Result

names
"Keanu Reeves"
"Carrie-Anne Moss"
Rows: 2

Referencing a non-existent key in the `MAP` with the `[]` operator will return `null`.

Dynamically referencing a non-existent `MAP` key

```
WITH {a: 10, b: 20, c: 30} AS map,
     'z' AS dynamicKey
RETURN map[dynamicKey] AS dynamicValue
```

Result

dynamicValue
null
Rows: 1

Merging `MAP` values

Introduced in Neo4j 2026.10 | Cypher 25 only

Two `MAP` values can be merged together using the `+` symbol. They follow a "last update wins" logic for any repeated keys.

Map addition

```
WITH {a: 10, b: 20, c: 30} AS map1, {d: 40, e: 50, f: 60} AS map2
```

```
RETURN map1 + map2 AS mergedMaps
```

Result

mergedMaps
{a: 10, b: 20, c: 30, d: 40, e: 50, f: 60}
Rows: 1

If either `MAP` value is `null`, then the result will be `null`:

Map addition with null value

```
WITH {a: 10, b: 20, c: 30} AS map1, null AS map2
RETURN map1 + map2 AS mergedMaps
```

Result

mergedMaps
null
Rows: 1

If the second map contains keys that are present in the first map, the result will favor the second map's keys using a "last update wins" logic:

Map addition with matching keys

```
WITH {a: 10, b: 20, c: 30} AS map1, {a: 40, b: 50} AS map2
RETURN map1 + map2 AS mergedMaps
```

Result

mergedMaps
{a: 40, b: 50, c: 30}
Rows: 1

Map projection

Cypher supports map projections, which allow the creation of `MAP` values from nodes, relationships, and other `MAP` values.

A map projection begins with the variable bound to the `MAP` to be projected from, and contains a body of comma-separated `MAP` keys, enclosed by curly braces, `{}`. Each map element specifies a key-value pair that is included in the resulting map projection.

Syntax

```
mapVariable {mapElement [, ...n]}
```

`mapElement` can be one of the following:

- An **key selector** that references a key in a `MAP` and pulls the corresponding value.
- A **literal entry** that uses a custom key-value pair where the key is specified and the value is defined using an expression.
- A **variable selector** that projects the value of a variable, using the variable name as the key, and the value it references as the value.
- An **all-map projection** that projects all key-value pairs from `mapVariable`.

The key names in a map must be of type `STRING`.
If `mapVariable` is `null`, the projection evaluates to `null`.

Example 157. Map projection with a key selector

Map projections with a key selector are useful when you need to extract specific key-value pairs from a `MAP`.

Basic example

```
WITH {a: 10, b: 20, c: 30} AS map
RETURN map{.a, .c} AS projectedMap
```

Result

projectedMap
{a: 10, c: 30}
Rows: 1

In the below query, a map projection with property selectors is used to `collect` only the `title` and `release` year of each movie.

Map projection with a key selector

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor, collect(m{.title, .released}) AS movies
```

Result

actor	movies
"Keanu Reeves"	[{title: "The Matrix Resurrections", released: 2021}, {title: "The Matrix Revolutions", released: 2003}, {title: "The Matrix Reloaded", released: 2003}, {title: "The Matrix", released: 1999}, {title: "The Devils Advocate", released: 1997}]

actor	movies
"Carrie-Anne Moss"	[{title: "The Matrix Resurrections", released: 2021}, {title: "The Matrix Revolutions", released: 2003}, {title: "The Matrix Reloaded", released: 2003}, {title: "The Matrix", released: 1999}]

Rows: 2

Example 158. Map projection with a literal entry

Map projections with a literal entry are useful when you want to add custom values to a projected `MAP` value without modifying the original data structure.

Basic example

```
WITH {a: 10, b: 20, c: 30} AS map
RETURN map{a: map.a, valueSum: map.a + map.b + map.c} AS projectedMap
```

Result

projectedMap
{a: 10, valueSum: 60}

Rows: 1

This query uses a map projection with a literal entry, where the `size(movies)` expression calculates the total number of movies `Keanu Reeves` has acted in.

Map projection with a literal entry

```
MATCH (keanu:Person {name: 'Keanu Reeves'})-[:ACTED_IN]->(movie:Movie)
WITH keanu, collect(movie) AS movies
RETURN keanu {.name, totalMovies: size(movies)} AS keanuDetails
```

Result

keanuDetails
{name: "Keanu Reeves", totalMovies: 5}

Rows: 1

Example 159. Map projection with a variable selector

Map projection with a variable selector is useful when you want to project values based on a variable name.

Basic example

```
MATCH (keanu:Person {name: 'Keanu Reeves'})
LET dob = date('1964-09-02'), birthPlace =  'Beirut, Lebanon'
```

```
RETURN keanu{.name, dob, birthPlace} AS projectedKeanu
```

Result

projectedKeanu
{name: "Keanu Reeves", birthPlace: "Beirut, Lebanon", dob: 1964-09-02}
Rows: 1

The below query finds all `Person` nodes in the graph that have one or more relationships with the type `ACTED_IN` connected to `Movie` nodes. It uses the count() function to count how many `Movie` nodes are connected to each `Person` node in this way, and uses a variable selector to project the value of the count.

Map projection with a variable selector

```
MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)
WITH actor, count(movie) AS totalMovies
RETURN actor{totalMovies, .name} AS nameAndMovies
```

Result

nameAndMovies
{name: "Keanu Reeves", totalMovies: 5}
{name: "Carrie-Anne Moss", totalMovies: 4}
Rows: 2

Example 160. Map projection with an all-map projection

Map projection with an all-map projection is useful when you want to project all key-value pairs from a `MAP` without explicitly listing them.

Basic example

```
WITH {a: 10, b: 20, c: 30} AS map
RETURN map{.*} AS projectedMap
```

Result

projectedMap
{a: 10, b: 20, c: 30}
Rows: 1

The below query returns all properties from the `Keanu Reeves` node. An all-map selector is used to project all the node properties.

Map projection with all-map selector

```
MATCH (keanu:Person {name: 'Keanu Reeves'})
RETURN keanu{.*} AS allKeanuProperties
```

Result

AllKeanuProperties
{nationality: "Canadian", name: "Keanu Reeves"}
Rows: 1

Map comprehension

Introduced in Neo4j 2026.09 | Cypher 25 only

Map comprehension is used to create new `MAP` values by iterating over the elements of an existing `MAP` or `LIST`, and transforming them into key-value pairs. The result is a new `MAP` where each entry is generated by the specified key and value expressions.

Map comprehension follows a similar syntax to list comprehension.

Syntax (map variant)

```
{key: value IN map [WHERE predicate] | keyExpression : valueExpression}
```

The iteration step (`key: value IN map`) ensures that each key, value pair in a `map` is accessed one by one, while the `keyExpression` and `valueExpression` optionally applies a transformation to each `item`, creating new `MAP` of key, value pairs containing the modified elements.

Syntax (list variant)

```
{item IN list [WHERE predicate] | keyExpression : valueExpression}
```

The iteration step (`item IN list`) ensures that each element of a `list` is accessed one by one, while the `keyExpression` and `valueExpression` apply a transformation to each `item`, creating new `MAP` of key-value pairs containing the modified elements.

Considerations

Variables introduced inside a map comprehension (such as `key`, `value`, or `item`) shadow any outer variables with the same name and are not accessible outside the map comprehension.
As `MAP` values cannot have duplicate keys, map comprehension follows a "last update wins" logic. If multiple elements result in the same key, the value from the last element processed will overwrite any previous value for that key. However, the order in which elements are processed is not guaranteed, so which value is treated as "last" should not be relied upon.
Unlike list comprehension, the `keyExpression` and `valueExpression` (and the `|` separator) are not optional.
The `keyExpression` must evaluate to a `STRING` value or an error will be thrown.

Simple map comprehension

```
WITH {a: 1, b: 2, c: 3} AS map
RETURN {k: v IN map | k : v * 10} AS result
```

Result

result
{a: 10, b: 20, c: 30}
Rows: 1

Filter a `MAP`

```
WITH {a: 1, b: 2, c: 3, d: null, e: null} AS map
RETURN {k: v IN map WHERE v IS NOT NULL | k : v} AS result
```

Result

result
{a: 1, b: 2, c: 3}
Rows: 1

Last write wins

```
WITH {a: 1, b: 2, c: 3} AS map
RETURN {k: v IN map | 'a' : v * 10} AS result
```

Result

result
{a: 30}
Rows: 1

The value in the result comes from whichever `MAP` entry is processed last (in this case, `{a:30}`). That entry is not guaranteed, however, because Cypher does not define an order in which `MAP` entries are processed.

Nested map comprehension

```
WITH {a: {aa: 1, ab: 2}, b: {ba: 1, bb: 2}} AS nestedMap
RETURN {k: v IN nestedMap | k : {innerK: innerV IN v | innerK : innerV * 10}} AS result
```

Result

result
{a: {aa: 10, ab: 20}, b: {ba: 10, bb: 20}}
Rows: 1

Simple map comprehension (list variant)

```
WITH ['a', 'b', 'c'] AS list
RETURN {item IN list | item : item + '!' } AS result
```

Result

result
{a: "a!", b: "b!", c: "c!"}
Rows: 1

Filter a `LIST` and transform into a `MAP`

```
WITH range(0, 10) AS list
RETURN {i IN list WHERE i % 2 = 0 | toString(i) : i * 10} AS result
```

Result

result
{"0": 0, "2": 20, "4": 40, "6": 60, "8": 80, "10": 100}
Rows: 1

Conditional expressions (CASE)

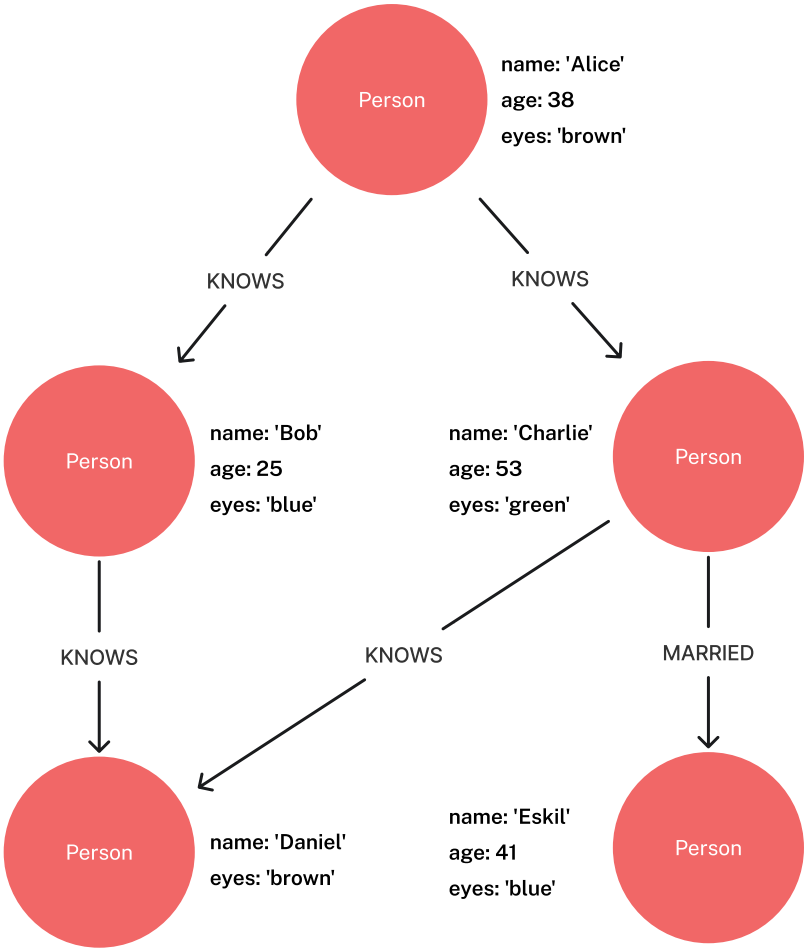
Generic conditional expressions can be expressed in Cypher using the `CASE` construct. Two variants of `CASE` exist within Cypher: the *simple* form, to compare a single expression against multiple values, and the *generic* form, to express multiple conditional statements.

Note:

`CASE` can only be used as part of RETURN or WITH if you want to use the result in a subsequent clause.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (alice:Person {name:'Alice', age: 38, eyes: 'brown'}),
  (bob:Person {name: 'Bob', age: 25, eyes: 'blue'}),
  (charlie:Person {name: 'Charlie', age: 53, eyes: 'green'}),
  (daniel:Person {name: 'Daniel', eyes: 'brown'}),
  (eskil:Person {name: 'Eskil', age: 41, eyes: 'blue'}),
  (alice)-[:KNOWS]->(bob),
  (alice)-[:KNOWS]->(charlie),
  (bob)-[:KNOWS]->(daniel),
  (charlie)-[:KNOWS]->(daniel),
  (bob)-[:MARRIED]->(eskil)
```

Simple `CASE`

The simple `CASE` form is used to compare a single expression against multiple values, and is analogous to the `switch` construct of programming languages. The expressions are evaluated by the `WHEN` operator until a match is found. If no match is found, the expression in the `ELSE` operator is returned. If there is no `ELSE` case and no match is found, `null` will be returned.

Syntax

```
CASE test
  WHEN value [, value]* THEN result
  [WHEN ...]
  [ELSE default]
END
```

Arguments:

Name	Description
`test`	An expression.
`value`	An expression whose result will be compared to `test`.
`result`	The expression returned as output if `value` matches `test`
`default`	The expression to return if no value matches the test expression.

Example

```
MATCH (n:Person)
RETURN
CASE n.eyes
  WHEN 'blue'  THEN 1
  WHEN 'brown', 'hazel' THEN 2
  ELSE 3
END AS result, n.eyes
```

result	n.eyes
2	"brown"
1	"blue"
3	"green"
2	"brown"
1	"blue"
Rows: 5	

Extended Simple `CASE`

The extended simple `CASE` form allows the comparison operator to be specified explicitly. The simple `CASE` uses an implied equals (`=`) comparator.

The supported comparators are:

- Regular Comparison Operators: `=`, `<>`, `<`, `>`, `<=`, `>=`
- `IS [NOT\` NULL]
- Type predicate expressions: `IS [NOT] TYPED <TYPE>` (Note that the form `IS [NOT] :: <TYPE>` is not accepted)
- Normalization Predicate Expression: `IS [NOT] NORMALIZED`
- String Comparison Operators: `STARTS WITH`, `ENDS WITH`, `=~` (regex matching)

Syntax

```
CASE test
  WHEN [comparisonOperator] value [, [comparisonOperator] value ]* THEN result
  [WHEN ...]
  [ELSE default]
END
```

Arguments:

Name	Description
`test`	An expression.
`comparisonOperator`	One of the supported comparison operators.
`value`	An expression whose result is compared to `test` using the given comparison operator.
`result`	The expression returned as output if `value` matches `test`.
`default`	The expression to return if no value matches the test expression.

Example

```
MATCH (n:Person)
RETURN n.name,
CASE n.age
  WHEN IS NULL, IS NOT TYPED INTEGER | FLOAT THEN "Unknown"
  WHEN = 0, = 1, = 2 THEN "Baby"
  WHEN <= 13 THEN "Child"
  WHEN < 20 THEN "Teenager"
  WHEN < 30 THEN "Young Adult"
  WHEN > 1000 THEN "Immortal"
  ELSE "Adult"
END AS result
```

n.name	result
"Alice"	"Adult"
"Bob"	"Young Adult"
"Charlie"	"Adult"
"Daniel"	"Unknown"
"Eskil"	"Adult"
Rows: 5	

Generic `CASE`

The generic `CASE` expression supports multiple conditional statements, and is analogous to the `if-elseif-else` construct of programming languages. Each row is evaluated in order until a `true` value is found. If no match is found, the expression in the `ELSE` operator is returned. If there is no `ELSE` case and no match is found, `null` will be returned.

Syntax

```
CASE
  WHEN predicate THEN result
  [WHEN ...]
  [ELSE default]
END
```

Arguments:

Name	Description
`predicate`	A predicate is an expression that evaluates to a `BOOLEAN` value. In this case, the predicate is tested to find a valid alternative.
`result`	The expression returned as output if `predicate` evaluates to `true`.
`default`	If no match is found, `default` is returned.

Example

```
MATCH (n:Person)
RETURN
CASE
  WHEN n.eyes = 'blue' THEN 1
  WHEN n.age < 40      THEN 2
  ELSE 3
END AS result, n.eyes, n.age
```

result	n.eyes	n.age
2	"brown"	38
1	"blue"	25
3	"green"	53
3	"brown"	null
1	"blue"	41

Rows: 5

`CASE` with `null` values

When working with `null` values, you may be forced to use the generic `CASE` form. The two examples below use the `age` property of the `Daniel` node (which has a `null` value for that property) to clarify the difference.

Simple `CASE`

```
MATCH (n:Person)
RETURN n.name,
CASE n.age
  WHEN null THEN -1
  ELSE n.age - 10
END AS age_10_years_ago
```

`n.age` is the expression being evaluated. Note that the node `Daniel` has a `null` value as age.

This branch is skipped, because `null` does not equal any other value, including `null` itself.

The execution takes the `ELSE` branch, which outputs `null` because `n.age - 10` equals `null`.

n.name	age_10_years_ago
"Alice"	28
"Bob"	15
"Charlie"	43
"Daniel"	null
"Eskil"	31

Rows: 5

Generic `CASE`

```
MATCH (n:Person)
RETURN n.name,
CASE
  WHEN n.age IS NULL THEN -1
  ELSE n.age - 10
END AS age_10_years_ago
```

If no expression is provided after `CASE`, it acts in its generic form, supporting predicate expressions in each branch.

This predicate expression evaluates to `true` for the node `Daniel`, so the result from this branch is returned.

n.name	age_10_years_ago
"Alice"	28
"Bob"	15
"Charlie"	43
"Daniel"	-1
"Eskil"	31

Rows: 5

For more information about `null`, see Working with `null`.

`CASE` expressions and succeeding clauses

The results of a `CASE` expression can be used to set properties on a node or relationship.

```
MATCH (n:Person)
WITH n,
CASE n.eyes
  WHEN 'blue'  THEN 1
```

```
  WHEN 'brown' THEN 2
  ELSE 3
END AS colorCode
SET n.colorCode = colorCode
RETURN n.name, n.colorCode
```

n.name	n.colorCode
"Alice"	2
"Bob"	1
"Charlie"	3
"Daniel"	2
"Eskil"	1
Rows: 5	

For more information about using the `SET` clause, see SET.

Further considerations

`CASE` result branches are statically checked prior to execution. This means that if a branch is not semantically correct, it will still throw an exception, even if that branch may never be executed during runtime.

In the following example, `date` is statically known to be a `STRING` value, and therefore would fail if treated as a `DATE` value.

Not allowed

```
WITH "2024-08-05" AS date, "string" AS type
RETURN CASE type
    WHEN "string" THEN datetime(date)
    WHEN "date" THEN datetime({year: date.year, month: date.month, day: date.day})
    ELSE datetime(date)
END AS dateTime
```

GQLSTATUS error chain

22N27: error: data exception - invalid entity type. Invalid input 'STRING' for `date`. Expected to be MAP, NODE, RELATIONSHIP, POINT, DURATION, DATE, ZONED TIME, LOCAL TIME, LOCAL DATETIME or ZONED DATETIME.

22G03: error: data exception - invalid value type

Functions

This section contains a summary of all functions in Cypher.

To list all functions, run the following query:

List all functions

```
SHOW FUNCTIONS
```

For more information about this command, see Show functions.

> **Note:**
>
> Functions taking a `STRING` as input all operate on *Unicode characters* rather than on a standard `char[]`. For example, the `size()` function applied to any *Unicode character* will return `1`, even if the character does not fit in the 16 bits of one `char`.

Aggregating functions

These functions take multiple values as arguments, and calculate and return an aggregated value from them.

Function	Signature	Description
`avg()`	`avg(input :: INTEGER | FLOAT | DURATION) :: INTEGER | FLOAT | DURATION`	Returns the average of a set of `INTEGER`, `FLOAT`, or `DURATION` values.
`collect()`	`collect(input :: ANY) :: LIST<ANY>`	Returns a list containing the values returned by an expression.
`collect_list()`	`collect_list(input :: ANY) :: LIST<ANY>`	Returns a list containing the values returned by an expression. This function is an alias to the `collect()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`count()`	`count(input :: ANY) :: INTEGER`	Returns the number of values or rows.
`max()`	`max(input :: ANY) :: ANY`	Returns the maximum value in a set of values.
`min()`	`min(input :: ANY) :: ANY`	Returns the minimum value in a set of values.
`percentileCont()`	`percentileCont(input :: FLOAT, percentile :: FLOAT) :: FLOAT`	Returns the percentile of a value over a group using linear interpolation.

Function	Signature	Description
`percentile_cont()`	`percentile_cont(input :: FLOAT, percentile :: FLOAT) :: FLOAT`	Returns the percentile of a value over a group using linear interpolation. This function is an alias to the `percentileCont()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`percentileDisc()`	`percentileDisc(input :: INTEGER | FLOAT, percentile :: FLOAT) :: FLOAT`	Returns the nearest `INTEGER` or `FLOAT` value to the given percentile over a group using a rounding method.
`percentile_disc()`	`percentile_disc(input :: INTEGER | FLOAT, percentile :: FLOAT) :: INTEGER | FLOAT`	Returns the nearest `INTEGER` or `FLOAT` value to the given percentile over a group using a rounding method. This function is an alias to the `percentileDisc()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`stDev()`	`stDev(input :: FLOAT) :: FLOAT`	Returns the standard deviation for the given value over a group for a sample of a population.
`stdev_samp()`	`stdev_samp(input :: FLOAT) :: FLOAT`	Returns the standard deviation for the given value over a group for a sample of a population. This function is an alias to the `stDev()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`stDevP()`	`stDevP(input :: FLOAT) :: FLOAT`	Returns the standard deviation for the given value over a group for an entire population.
`stdev_pop()`	`stdev_pop(input :: FLOAT) :: FLOAT`	Returns the standard deviation for the given value over a group for an entire population. This function is an alias to the `stDevP()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02

Function	Signature	Description
`sum()`	`sum(input :: INTEGER | FLOAT | DURATION) :: INTEGER | FLOAT | DURATION`	Returns the sum of a set of `INTEGER`, `FLOAT`, or `DURATION` values.

Database functions

Database functions provide information about databases.

Function	Signature	Description
`db.nameFromElementId()`	`db.nameFromElementId(elementId :: STRING) :: STRING`	Resolves the database name from the given element id.

Graph functions

Graph functions provide information about the constituent graphs in composite databases.

Function	Signature	Description
`graph.byElementId()`	`USE graph.byElementId(elementId :: STRING)`	Resolves the constituent graph to which a given element id belongs.
`graph.byName()`	`USE graph.byName(name :: STRING)`	Resolves a constituent graph by name.
`graph.names()`	`graph.names() :: LIST<STRING>`	Returns a list containing the names of all graphs in the current composite database.
`graph.propertiesByName()`	`graph.propertiesByName(name :: STRING) :: MAP`	Returns a map containing the properties associated with the given graph.

List functions

These functions return lists of other values. Further details and examples of lists may be found in Lists.

Function	Signature	Description
`coll.distinct()`	`coll.distinct(list :: LIST<ANY>) :: LIST<ANY>`	Returns the given list with all duplicate values removed. Cypher 25 only Introduced in Neo4j 2025.11
`coll.flatten()`	`coll.flatten(list :: LIST<ANY>, depth = 1 :: INTEGER) :: LIST<ANY>`	Returns a list flattened to the given depth. Cypher 25 only Introduced in Neo4j 2025.11

Function	Signature	Description
`coll.indexOf()`	`coll.indexOf(list :: LIST<ANY>, value :: ANY) :: INTEGER`	Returns the index of the first match of value in the given list, if the value is no present, -1 is returned. Cypher 25 only Introduced in Neo4j 2025.11
`coll.insert()`	`coll.insert(list :: LIST<ANY>, index :: INTEGER, value :: ANY) :: LIST<ANY>`	Returns a list with the given value inserted at the given index. Cypher 25 only Introduced in Neo4j 2025.11
`coll.max()`	`coll.max(list :: LIST<ANY>) :: ANY`	Returns the largest value. Cypher 25 only Introduced in Neo4j 2025.11
`coll.min()`	`coll.min(list :: LIST<ANY>) :: ANY`	Returns the smallest value. Cypher 25 only Introduced in Neo4j 2025.11
`coll.remove()`	`coll.remove(list :: LIST<ANY>, index :: INTEGER) :: LIST<ANY>`	Returns a list with the value at the given index removed. Cypher 25 only Introduced in Neo4j 2025.11
`coll.sort()`	`coll.sort(list :: LIST<ANY>) :: LIST<ANY>`	Returns a sorted list. Cypher 25 only Introduced in Neo4j 2025.11
`keys()`	`keys(input :: NODE | RELATIONSHIP | MAP) :: LIST<STRING>`	Returns a `LIST<STRING>` containing the `STRING` representations for all the property names of a `MAP`, `NODE`, or `RELATIONSHIP`.
`labels()`	`labels(input :: NODE) :: LIST<STRING>`	Returns a `LIST<STRING>` containing the `STRING` representations for all the labels of a `NODE`.
`nodes()`	`nodes(input :: PATH) :: LIST<NODE>`	Returns a `LIST<NODE>` containing all the `NODE` values in a `PATH`.
`range()`	`range(start :: INTEGER, end :: INTEGER [, step :: INTEGER]) :: LIST<INTEGER>`	Returns a `LIST<INTEGER>` comprising all `INTEGER` values within a specified range, optionally specifying a step length.
`reduce()`	`reduce(accumulator :: VARIABLE = initial :: ANY, variable :: VARIABLE IN list :: LIST<ANY> expression :: ANY) :: ANY`	Runs an expression against individual elements of a `LIST<ANY>`, storing the result of the expression in an accumulator.
`relationships()`	`relationships(input :: PATH) :: LIST<RELATIONSHIP>`	Returns a `LIST<RELATIONSHIP>` containing all the `RELATIONSHIP` values in a `PATH`.

Function	Signature	Description
`reverse()`	`reverse(input :: LIST<ANY>) :: LIST<ANY>`	Returns a `LIST<ANY>` in which the order of all elements in the given `LIST<ANY>` have been reversed.
`tail()`	`tail(input :: LIST<ANY>) :: LIST<ANY>`	Returns all but the first element in a `LIST<ANY>`.
`toBooleanList()`	`toBooleanList(input :: LIST<ANY>) :: LIST<BOOLEAN>`	Converts a `LIST<ANY>` of values to a `LIST<BOOLEAN>` values. If any values are not convertible to `BOOLEAN` they will be null in the `LIST<BOOLEAN>` returned.
`toFloatList()`	`toFloatList(input :: VECTOR | LIST<ANY>) :: LIST<FLOAT>`	Converts a `LIST<ANY>` to a `LIST<FLOAT>` values. If any values are not convertible to `FLOAT` they will be null in the `LIST<FLOAT>` returned.
`toIntegerList()`	`toIntegerList(input :: VECTOR | LIST<ANY>) :: LIST<INTEGER>`	Converts a `LIST<ANY>` to a `LIST<INTEGER>` values. If any values are not convertible to `INTEGER` they will be null in the `LIST<INTEGER>` returned.
`toStringList()`	`toStringList(input :: LIST<ANY>) :: LIST<STRING>`	Converts a `LIST<ANY>` to a `LIST<STRING>` values.

LOAD CSV functions

LOAD CSV functions can be used to get information about the file that is processed by `LOAD CSV`.

Function	Signature	Description
`file()`	`file() :: STRING`	Returns the absolute path of the file that LOAD CSV is using.
`linenumber()`	`linenumber() :: INTEGER`	Returns the line number that LOAD CSV is currently using.

Mathematical functions

Logarithmic functions

These functions all operate on numerical expressions only, and will return an error if used on any other values.

Function	Signature	Description
`e()`	`e() :: FLOAT`	Returns the base of the natural logarithm, e.

Function	Signature	Description
`exp()`	`exp(input :: FLOAT) :: FLOAT`	Returns e^n, where e is the base of the natural logarithm, and n is the value of the argument expression.
`log()`	`log(input :: FLOAT) :: FLOAT`	Returns the natural logarithm of a `FLOAT`.
`ln()`	`ln(input :: FLOAT) :: FLOAT`	Returns the natural logarithm of a `FLOAT`. This function is an alias to the `log()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`log10()`	`log10(input :: FLOAT) :: FLOAT`	Returns the common logarithm (base 10) of a `FLOAT`.
`sqrt()`	`sqrt(input :: FLOAT) :: FLOAT`	Returns the square root of a `FLOAT`.

Numeric functions

These functions all operate on numerical expressions only, and will return an error if used on any other values.

Function	Signature	Description
`abs()`	`abs(input :: INTEGER | FLOAT) :: INTEGER | FLOAT`	Returns the absolute value of an `INTEGER` or `FLOAT`.
`ceil()`	`ceil(input :: FLOAT) :: FLOAT`	Returns the smallest `FLOAT` that is greater than or equal to a number and equal to an `INTEGER`.
`ceiling()`	`ceiling(input :: FLOAT) :: FLOAT`	Returns the smallest `FLOAT` that is greater than or equal to a number and equal to an `INTEGER`. This function is an alias to the `ceil()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`floor()`	`floor(input :: FLOAT) :: FLOAT`	Returns the largest `FLOAT` that is less than or equal to a number and equal to an `INTEGER`.
`isNaN()`	`isNaN(input :: INTEGER | FLOAT) :: BOOLEAN`	Returns `true` if the floating point number is `NaN`.

Function	Signature	Description
`rand()`	`rand() :: FLOAT`	Returns a random `FLOAT` in the range from 0 (inclusive) to 1 (exclusive).
`round()`	`round(input :: FLOAT [, precision :: INTEGER | FLOAT, mode :: STRING]) :: FLOAT`	Returns the value of a number rounded to the nearest `INTEGER`, optionally using a specified precision and rounding mode.
`sign()`	`sign(input :: INTEGER | FLOAT) :: INTEGER`	Returns the signum of an `INTEGER` or `FLOAT`: 0 if the number is 0, -1 for any negative number, and 1 for any positive number.

Trigonometric functions

These functions all operate on numerical expressions only, and will return an error if used on any other values.

All trigonometric functions operate on radians, unless otherwise specified.

Function	Signature	Description
`acos()`	`acos(input :: FLOAT) :: FLOAT`	Returns the arccosine of a `FLOAT` in radians.
`asin()`	`asin(input :: FLOAT) :: FLOAT`	Returns the arcsine of a `FLOAT` in radians.
`atan()`	`atan(input :: FLOAT) :: FLOAT`	Returns the arctangent of a `FLOAT` in radians.
`atan2()`	`atan2(y :: FLOAT, x :: FLOAT) :: FLOAT`	Returns the arctangent2 of a set of coordinates in radians.
`cos()`	`cos(input :: FLOAT) :: FLOAT`	Returns the cosine of a `FLOAT`.
`cosh()`	`cosh(input :: FLOAT) :: FLOAT`	Returns the hyperbolic cosine of a `FLOAT`. Cypher 25 only Introduced in Neo4j 2025.06
`cot()`	`cot(input :: FLOAT) :: FLOAT`	Returns the cotangent of a `FLOAT`.
`coth()`	`coth(input :: FLOAT) :: FLOAT`	Returns the hyperbolic cotangent of a `FLOAT`. Cypher 25 only Introduced in Neo4j 2025.06
`degrees()`	`degrees(input :: FLOAT) :: FLOAT`	Converts radians to degrees.
`haversin()`	`haversin(input :: FLOAT) :: FLOAT`	Returns half the versine of a number.

Function	Signature	Description
`pi()`	`pi() :: FLOAT`	Returns the mathematical constant pi.
`radians()`	`radians(input :: FLOAT) :: FLOAT`	Converts degrees to radians.
`sin()`	`sin(input :: FLOAT) :: FLOAT`	Returns the sine of a `FLOAT`.
`sinh()`	`sinh(input :: FLOAT) :: FLOAT`	Returns the hyperbolic sine of a `FLOAT`. Cypher 25 only Introduced in Neo4j 2025.06
`tan()`	`tan(input :: FLOAT) :: FLOAT`	Returns the tangent of a `FLOAT`.
`tanh()`	`tanh(input :: FLOAT) :: FLOAT`	Returns the hyperbolic tangent of a `FLOAT`. Cypher 25 only Introduced in Neo4j 2025.06

Predicate functions

These functions return either true or false for the given arguments.

Function	Signature	Description
`all()`	`all(variable :: ANY, list :: LIST<ANY>, predicate :: ANY) :: BOOLEAN`	Returns true if the predicate holds for all elements in the given `LIST<ANY>`.
`allReduce()`	`allReduce(accumulator = initial, stepVariable IN list | reductionFunction, predicate) :: BOOLEAN`	Returns true if, during the stepwise evaluation of a value across the elements in a given `LIST<ANY>`, the accumulated result satisfies a specified predicate at every step. Where that list is a group variable defined in a quantified path pattern, it allows for the early pruning of paths that do not satisfy the predicate. Cypher 25 only Introduced in Neo4j 2025.08
`any()`	`any(variable :: ANY, list :: LIST<ANY>, predicate :: ANY) :: BOOLEAN`	Returns true if the predicate holds for at least one element in the given `LIST<ANY>`.
`exists()`	`exists(input :: ANY) :: BOOLEAN`	Returns `true` if a match for the pattern exists in the graph.
`isEmpty()`	`isEmpty(input :: LIST<ANY> | MAP | STRING ) :: BOOLEAN`	Checks whether the given `LIST<ANY>`, `MAP`, or `STRING` is empty.

Function	Signature	Description
`none()`	`none(variable :: ANY, list :: LIST<ANY>, predicate :: ANY) :: BOOLEAN`	Returns true if the predicate holds for no element in the given `LIST<ANY>`.
`single()`	`single(variable :: ANY, list :: LIST<ANY>, predicate :: ANY) :: BOOLEAN`	Returns true if the predicate holds for exactly one of the elements in the given `LIST<ANY>`.

Scalar functions

These functions return a single value.

Function	Signature	Description
`cardinality()`	`cardinality(input :: MAP | LIST<ANY> | PATH) :: INTEGER`	Returns the number of keys in a `MAP`, the number of elements in a `LIST<ANY>`, or the number of nodes and relationships in a `PATH`. Cypher 25 only Introduced in Neo4j 2026.07
`char_length()`	`char_length(input :: STRING) :: INTEGER`	Returns the number of Unicode characters in a `STRING`.
`character_length()`	`character_length(input :: STRING) :: INTEGER`	Returns the number of Unicode characters in a `STRING`.
`coalesce()`	`coalesce(input :: ANY) :: ANY`	Returns the first non-null value in a list of expressions.
`elementId()`	`elementId(input :: NODE | RELATIONSHIP) :: STRING`	Returns a node or relationship identifier, unique within a specific transaction and DBMS.
`endNode()`	`endNode(input :: RELATIONSHIP) :: NODE`	Returns the end `NODE` of a `RELATIONSHIP`.
`head()`	`head(list :: LIST<ANY>) :: ANY`	Returns the first element in a `LIST<ANY>`.
`id()`	`id(input :: NODE | RELATIONSHIP) :: INTEGER`	Deprecated Returns the id of a `NODE` or a `RELATIONSHIP`. Replaced by `elementId()`.
`last()`	`last(list :: LIST<ANY>) :: ANY`	Returns the last element in a `LIST<ANY>`.
`length()`	`length(input :: PATH) :: INTEGER`	Returns the length of a `PATH`.

Function	Signature	Description
`path_length()`	`path_length(input :: PATH) :: INTEGER`	Returns the length of a `PATH`. This function is an alias to the `length()` function, and it was introduced as part of Cypher's GQL conformance. Cypher 25 only Introduced in Neo4j 2026.02
`nullIf()`	`nullIf(v1 :: ANY, v2 :: ANY) :: ANY`	Returns `null` if the two given parameters are equivalent, otherwise returns the value of the first parameter.
`properties()`	`properties(input :: NODE | RELATIONSHIP | MAP) :: MAP`	Returns a `MAP` containing all the properties of a `NODE` or `RELATIONSHIP`.
`randomUUID()`	`randomUUID() :: STRING`	Generates a random UUID.
`size()`	`size(input STRING | LIST<ANY>) :: INTEGER`	Returns the number of items in a `LIST<ANY>` or the number of Unicode characters in a `STRING`.
`startNode()`	`startNode(input :: RELATIONSHIP) :: NODE`	Returns the start `NODE` of a `RELATIONSHIP`.
`timestamp()`	`timestamp() :: INTEGER`	Returns the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.
`toBoolean()`	`toBoolean(input :: BOOLEAN | STRING | INTEGER) :: BOOLEAN`	Converts a `BOOLEAN`, `STRING`, or an `INTEGER` value to a `BOOLEAN` value.
`toBooleanOrNull()`	`toBooleanOrNull(input :: ANY) :: BOOLEAN`	Converts a value to a `BOOLEAN` value, or null if the value cannot be converted.
`toFloat()`	`toFloat(input :: STRING | INTEGER | FLOAT) :: FLOAT`	Converts a `STRING` or `INTEGER` value to a `FLOAT` value.
`toFloatOrNull()`	`toFloatOrNull(input :: ANY) :: FLOAT`	Converts a value to a `FLOAT` value, or null if the value cannot be converted.
`toInteger()`	`toInteger(input :: BOOLEAN | STRING | INTEGER | FLOAT) :: INTEGER`	Converts a `BOOLEAN`, `STRING`, or `FLOAT` value to an `INTEGER` value.
`toIntegerOrNull()`	`toIntegerOrNull(input :: ANY) :: INTEGER`	Converts a value to an `INTEGER` value, or null if the value cannot be converted.
`type()`	`type(input :: RELATIONSHIP) :: STRING`	Returns a `STRING` representation of the `RELATIONSHIP` type.

Function	Signature	Description
`valueType()`	`valueType(input :: ANY) :: STRING`	Returns a `STRING` representation of the most precise value type that the given expression evaluates to.

Security functions

Security functions are used to evaluate user attributes in the condition of an attribute-based access control (ABAC) auth rule. These functions cannot be called directly in queries; they can only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command.

Function	Signature	Description
`abac.native.user_tags()`	`abac.native.user_tags() :: LIST<STRING>`	Returns the list of tags for the native user. Cypher 25 only Introduced in Neo4j 2026.06
`abac.oidc.user_attribute()`	`abac.oidc.user_attribute(claimKey :: STRING) :: ANY`	Returns the value of the specified user attribute from the OIDC claims. Cypher 25 only Introduced in Neo4j 2026.03

String functions

These functions are used to manipulate `STRING` values or to create a `STRING` representation of another value.

Function	Signature	Description
`btrim()`	`btrim(original :: STRING [, trimCharacterString :: STRING ]) :: STRING`	Returns the given `STRING` with leading and trailing whitespace removed, optionally specifying a `trimCharacterString` value to remove.
`left()`	`left(original :: STRING, length :: INTEGER) :: STRING`	Returns a `STRING` containing the specified number (`INTEGER`) of leftmost characters in the given `STRING`.
`lower()`	`lower(input :: STRING) :: STRING`	Returns the given `STRING` in lowercase. This function is an alias to the `toLower()` function, and it was introduced as part of Cypher's GQL conformance.
`ltrim()`	`ltrim(input :: STRING [, trimCharacterString :: STRING]) :: STRING`	Returns the given `STRING` with leading whitespace removed, optionally specifying a `trimCharacterString` to remove.

Function	Signature	Description
`normalize()`	`normalize(input :: STRING [,normalForm = NFC :: [NFC, NFD, NFKC, NFKD]]) :: STRING`	Normalizes a `STRING`, optionally specifying a normalization form.
`replace()`	`replace(original :: STRING, search :: STRING, replace :: STRING) :: STRING`	Returns a `STRING` in which all occurrences of a specified search `STRING` in the given `STRING` have been replaced by another (specified) replacement `STRING`.
`reverse()`	`reverse(input :: STRING) :: STRING`	Returns a `STRING` in which the order of all characters in the given `STRING` have been reversed.
`right()`	`right(original :: STRING, length :: INTEGER) :: STRING`	Returns a `STRING` containing the specified number of rightmost characters in the given `STRING`.
`rtrim()`	`rtrim(input :: STRING [, trimCharacterString :: STRING]) :: STRING`	Returns the given `STRING` with trailing whitespace removed, optionally specifying a `trimCharacterString` of characters to remove.
`split()`	`split(original :: STRING, splitDelimiters :: LIST<STRING>) :: LIST<STRING>`	Returns a `LIST<STRING>` resulting from the splitting of the given `STRING` around matches of any of the given delimiters.
`string.indexOf()`	`string.indexOf(input :: STRING, value :: STRING) :: INTEGER`	Returns the index of the first occurrence of the given `value` within the `input` `STRING`, or -1 if no match is found. Cypher 25 only Introduced in Neo4j 2026.05
`string.join()`	`string.join(input :: STRING, delimiter : STRING) :: STRING`	Concatenates the elements in `input`, inserting `delimiter` between each element, and returns the resulting `STRING`. Cypher 25 only Introduced in Neo4j 2026.05
`string.regexReplace()`	`string.regexReplace(original :: STRING, regex :: STRING, replacement :: STRING) :: STRING`	Returns a `STRING` where all matches of `regex` in the `original` `STRING` are replaced with the `replacement` `STRING`. Cypher 25 only Introduced in Neo4j 2026.05
`substring()`	`substring(original :: STRING, start :: INTEGER length :: INTEGER) :: STRING`	Returns a substring of a given `length` from the given `STRING`, beginning with a 0-based index start.

Function	Signature	Description
`toLower()`	`toLower(input :: STRING) :: STRING`	Returns the given `STRING` in lowercase.
`toString()`	`toString(input :: ANY) :: STRING`	Converts a value to a `STRING`.
`toStringOrNull()`	`toStringOrNull(input :: ANY) :: STRING`	Converts a value to a `STRING`, or `null` if the value cannot be converted.
`toUpper()`	`toUpper(input :: STRING) :: STRING`	Returns the given `STRING` in uppercase.
`trim()`	`trim([[LEADING | TRAILING | BOTH] [trimCharacterString :: STRING] FROM] input :: STRING) :: STRING`	Returns the given `STRING` with the leading and/or trailing `trimCharacterString` character removed.
`upper()`	`upper(input :: STRING) :: STRING`	Returns the given `STRING` in uppercase. This function is an alias to the `toUpper()` function, and it was introduced as part of Cypher's GQL conformance.

Spatial functions

These functions are used to specify 2D or 3D points in a geographic or cartesian Coordinate Reference System and to calculate the geodesic distance between two points.

Function	Signature	Description
`point()`	`point(input :: MAP) :: POINT`	Returns a 2D or 3D point object, given two or respectively three coordinate values in the Cartesian coordinate system or WGS 84 geographic coordinate system.
`point.distance()`	`point.distance(from :: POINT, to :: POINT) :: FLOAT`	Returns a `FLOAT` representing the distance between any two points in the same CRS. If the points are in the WGS 84 CRS, the function returns the geodesic distance (i.e., the shortest path along the curved surface of the Earth). If the points are in a Cartesian CRS, the function returns the Euclidean distance (i.e., the shortest straight-line distance in a flat, planar space).

Function	Signature	Description
`point.withinBBox()`	`point.withinBBox(point :: POINT, lowerLeft :: POINT, upperRight :: POINT) :: BOOLEAN`	Returns `true` if the provided point is within the bounding box defined by the two provided points, `lowerLeft` and `upperRight`.

Temporal functions

Duration functions

`DURATION` values of the temporal types can be created manipulated using the following functions:

Function	Signature	Description
`duration()`	`duration(input :: ANY) :: DURATION`	Constructs a `DURATION` value.
`duration.between()`	`duration.between(from :: ANY, to :: ANY) :: DURATION`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in logical units.
`duration_between()`	`duration_between(from :: ANY, to :: ANY) :: DURATION`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in logical units. This function is an alias to the `duration.between()` function, and it was introduced as part of Cypher's GQL conformance.
`duration.inDays()`	`duration.inDays(from :: ANY, to :: ANY) :: DURATION`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in days.
`duration.inMonths()`	`duration.inMonths(from :: ANY, to :: ANY) :: DURATION`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in months.
`duration.inSeconds()`	`duration.inSeconds(from :: ANY, to :: ANY) :: DURATION`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in seconds.

Instant type functions

Values of the temporal types — `DATE`, `ZONED TIME`, `LOCAL TIME`, `ZONED DATETIME`, and `LOCAL DATETIME` — can be created manipulated using the following functions:

Function	Signature	Description
`date()`	`date(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: DATE`	Creates a `DATE` instant.
`date.realtime()`	`date.realtime(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: DATE`	Returns the current `DATE` instant using the realtime clock.
`date.statement()`	`date.statement(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: DATE`	Returns the current `DATE` instant using the statement clock.
`date.transaction()`	`date.transaction(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: DATE`	Returns the current `DATE` instant using the transaction clock.
`date.truncate()`	`date.truncate(unit :: STRING, input = DEFAULT_TEMPORAL_ARGUMENT :: ANY, fields = null :: MAP) :: DATE`	Truncates the given temporal value to a `DATE` instant using the specified unit.
`datetime()`	`datetime(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED DATETIME`	Creates a `ZONED DATETIME` instant.
`zoned_datetime()`	`zoned_datetime(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED DATETIME`	Creates a `ZONED DATETIME` instant. This function is an alias to the `datetime()` function, and it was introduced as part of Cypher's GQL conformance.
`datetime.fromEpoch()`	`datetime.fromEpoch(seconds :: INTEGER | FLOAT, nanoseconds :: INTEGER | FLOAT) :: ZONED DATETIME`	Creates a `ZONED DATETIME` given the seconds and nanoseconds since the start of the epoch.
`datetime.fromEpochMillis()`	`datetime.fromEpochMillis(milliseconds :: INTEGER | FLOAT) :: ZONED DATETIME`	Creates a `ZONED DATETIME` given the milliseconds since the start of the epoch.
`datetime.realtime()`	`datetime.realtime(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED DATETIME`	Returns the current `ZONED DATETIME` instant using the realtime clock.
`datetime.statement()`	`datetime.statement(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED DATETIME`	Returns the current `ZONED DATETIME` instant using the statement clock.
`datetime.transaction()`	`datetime.transaction(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED DATETIME`	Returns the current `ZONED DATETIME` instant using the transaction clock.
`datetime.truncate()`	`datetime.truncate(unit :: STRING, input = DEFAULT_TEMPORAL_ARGUMENT :: ANY, fields = null :: MAP) :: ZONED DATETIME`	Truncates the given temporal value to a `ZONED DATETIME` instant using the specified unit.

Function	Signature	Description
`localdatetime()`	`localdatetime(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL DATETIME`	Creates a `LOCAL DATETIME` instant.
`local_datetime()`	`local_datetime(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL DATETIME`	Creates a `LOCAL DATETIME` instant. This function is an alias to the `localdatetime()` function, and it was introduced as part of Cypher's GQL conformance.
`localdatetime.realtime()`	`localdatetime.realtime(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL DATETIME`	Returns the current `LOCAL DATETIME` instant using the realtime clock.
`localdatetime.statement()`	`localdatetime.statement(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL DATETIME`	Returns the current `LOCAL DATETIME` instant using the statement clock.
`localdatetime.transaction()`	`localdatetime.transaction(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL DATETIME`	Returns the current `LOCAL DATETIME` instant using the transaction clock.
`localdatetime.truncate()`	`localdatetime.truncate(unit :: STRING, input = DEFAULT_TEMPORAL_ARGUMENT :: ANY, fields = null :: MAP) :: LOCAL DATETIME`	Truncates the given temporal value to a `LOCAL DATETIME` instant using the specified unit.
`localtime()`	`localtime(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL TIME`	Creates a `LOCAL TIME` instant.
`local_time()`	`local_time(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL TIME`	Creates a `LOCAL TIME` instant. This function is an alias to the `localtime()` function, and it was introduced as part of Cypher's GQL conformance.
`localtime.realtime()`	`localtime.realtime(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL TIME`	Returns the current `LOCAL TIME` instant using the realtime clock.
`localtime.statement()`	`localtime.statement(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL TIME`	Returns the current `LOCAL TIME` instant using the statement clock.
`localtime.transaction()`	`localtime.transaction(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: LOCAL TIME`	Returns the current `LOCAL TIME` instant using the transaction clock.
`localtime.truncate()`	`localtime.truncate(unit :: STRING, input = DEFAULT_TEMPORAL_ARGUMENT :: ANY, fields = null :: MAP) :: LOCAL TIME`	Truncates the given temporal value to a `LOCAL TIME` instant using the specified unit.

Function	Signature	Description
`time()`	`time(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED TIME`	Creates a `ZONED TIME` instant.
`zoned_time()`	`zoned_time(input = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED TIME`	Creates a `ZONED TIME` instant. This function is an alias to the `time()` function, and it was introduced as part of Cypher's GQL conformance.
`time.realtime()`	`time.realtime(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED TIME`	Returns the current `ZONED TIME` instant using the realtime clock.
`time.statement()`	`time.statement(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED TIME`	Returns the current `ZONED TIME` instant using the statement clock.
`time.transaction()`	`time.transaction(timezone = DEFAULT_TEMPORAL_ARGUMENT :: ANY) :: ZONED TIME`	Returns the current `ZONED TIME` instant using the transaction clock.
`time.truncate()`	`time.truncate(unit :: STRING, input = DEFAULT_TEMPORAL_ARGUMENT :: ANY, fields = null :: MAP) :: ZONED TIME`	Truncates the given temporal value to a `ZONED TIME` instant using the specified unit.

Format functions

Cypher 25 only | Introduced in Neo4j 2025.09

These functions format temporal instance and duration values:

Function	Signature	Description
`format()`	`format(value :: DATE | LOCAL TIME | ZONED TIME | LOCAL DATETIME | ZONED DATETIME | DURATION[, pattern :: STRING]) :: STRING`	Returns the temporal value as an ISO-formatted `STRING` or as a `STRING` formatted by the provided pattern.

User-defined functions

User-defined functions are written in Java, deployed into the database and are called in the same way as any other Cypher function. There are two main types of functions that can be developed and used:

Type	Description	Usage	Developing
Scalar	For each row the function takes parameters and returns a result.	Using UDF	Extending Neo4j (UDF)

Type	Description	Usage	Developing
Aggregating	Consumes many rows and produces an aggregated result.	Using aggregating UDF	Extending Neo4j (Aggregating UDF)

UUID functions

UUID functions allow you to create `UUID` values and return their most/least significant types.

Function	Signature	Description
`uuid()`	`uuid() :: UUID`	Returns a randomly generated `UUID`. Cypher 25 only Introduced in Neo4j 2026.08
`uuid()`	`uuid(name :: STRING) :: UUID`	The `STRING` to convert to a `UUID`. Converts a `STRING` value to a `UUID`. Cypher 25 only Introduced in Neo4j 2026.08
`uuid()`	`uuid(mostSigBits :: INTEGER, leastSigBits :: INTEGER) :: UUID`	Converts two `INTEGER` values to a `UUID`. Cypher 25 only Introduced in Neo4j 2026.08
`uuid.leastSignificantBits()`	`leastSignificantBits(uuid :: UUID) :: INTEGER`	Returns the least significant bits from the given `UUID`. Cypher 25 only Introduced in Neo4j 2026.08
`uuid.mostSignificantBits()`	`mostSignificantBits(uuid :: UUID) :: INTEGER`	Returns the most significant bits from the given `UUID`. Cypher 25 only Introduced in Neo4j 2026.08

Vector functions

Vector functions allow you to compute the similarity scores of vector pairs.

Function	Signature	Description
`vector()`	`vector(vectorValue :: STRING | LIST<INTEGER | FLOAT>, dimension :: INTEGER, coordinateType :: [INTEGER64, INTEGER32, INTEGER16, INTEGER8, FLOAT64, FLOAT32]) :: VECTOR`	Constructs a `VECTOR` value. Cypher 25 only Introduced in Neo4j 2025.10

Function	Signature	Description
`vector.similarity.cosine()`	`vector.similarity.cosine(a :: VECTOR | LIST<INTEGER | FLOAT>, b :: VECTOR | LIST<INTEGER | FLOAT>) :: FLOAT`	Returns a `FLOAT` representing the similarity between the argument vectors based on their cosine.
`vector.similarity.euclidean()`	`vector.similarity.euclidean(a :: VECTOR | LIST<INTEGER | FLOAT>, b :: VECTOR | LIST<INTEGER | FLOAT>) :: FLOAT`	Returns a `FLOAT` representing the similarity between the argument vectors based on their Euclidean distance.
`vector_dimension_count()`	`(vector :: VECTOR) :: INTEGER`	Returns the dimension of a `VECTOR`. Cypher 25 only Introduced in Neo4j 2025.10
`vector_distance()`	`(vector1 :: VECTOR, vector2 :: VECTOR, vectorDistanceMetric :: [EUCLIDEAN, EUCLIDEAN_SQUARED, MANHATTAN, COSINE, DOT, HAMMING]) :: FLOAT`	Returns a `FLOAT` representing the distance between the two vector values based on the selected `vectorDistanceMetric` algorithm. Cypher 25 only Introduced in Neo4j 2025.10
`vector_norm()`	`vector_norm(vector :: VECTOR, vectorDistanceMetric :: [EUCLIDEAN, MANHATTAN]) :: FLOAT`	Returns a `FLOAT` representing the distance between the given vector and a vector of the same dimension with all coordinates set to zero, calculated using the specified `vectorDistanceMetric`. Cypher 25 only Introduced in Neo4j 2025.10

Show functions

Showing the available functions can be done with `SHOW FUNCTIONS`. For general information about the `SHOW` command, see the `SHOW` page.

Syntax

For full details about the syntax descriptions, see the Operations Manual → Administration command syntax.

Action	Syntax

Show functions, either all or only built-in or user-defined	`SHOW [ALL|BUILT IN|USER DEFINED] FUNCTION[S]`
	`[YIELD { * | field[, ...] } [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`
	`[WHERE expression]`
	`[RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`

Action	Syntax

Show functions that the current user can execute	`SHOW [ALL|BUILT IN|USER DEFINED] FUNCTION[S] EXECUTABLE [BY CURRENT USER]`
	`[YIELD { * | field[, ...] } [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`
	`[WHERE expression]`
	`[RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`

Show functions that a specified user can execute	`SHOW [ALL|BUILT IN|USER DEFINED] FUNCTION[S] EXECUTABLE BY username`
	`[YIELD { * | field[, ...] } [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`
	`[WHERE expression]`
	`[RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]`
	
	Note:
	Requires the `SHOW USER` privilege. This command cannot be used for LDAP users.

Note:

When using the `RETURN` clause, the `YIELD` clause is mandatory and must not be omitted.

Examples

- Show all functions
- Return specific columns
- Show functions with filtering
- Combine `SHOW FUNCTIONS` with other Cypher clauses

Show all functions

To show all functions with the default output columns, use the `SHOW FUNCTIONS` command. If all columns are required, use `SHOW FUNCTIONS YIELD *`.

Show all functions and return default columns

```
SHOW FUNCTIONS
```

Show all functions and return all columns

```
SHOW FUNCTIONS YIELD *
```

Return specific columns

It is possible to return specific columns of the available functions using the `YIELD` clause:

Show all functions and return specific columns only

```
SHOW FUNCTIONS YIELD name, signature
```

Show functions with filtering

The `SHOW FUNCTIONS` command can be filtered using the `WHERE` clause.

Filter `SHOW FUNCTIONS` using `WHERE`

```
SHOW FUNCTIONS YIELD name
WHERE name CONTAINS 'vector'
RETURN count(name) AS numVectorFunctions
```

`SHOW FUNCTIONS` can also be filtered with the `BUILT IN` and `USER DEFINED` keywords.

Filter `SHOW FUNCTIONS` using `WHERE` and the `BUILT IN` keyword

```
SHOW BUILT IN FUNCTIONS YIELD name, isBuiltIn
WHERE name STARTS WITH 'a'
```

Functions can also be filtered on whether a user can execute them. This filtering is only available through the `EXECUTABLE` clause and not through the `WHERE` clause. This is due to using the user's privileges instead of filtering on the available output columns.

There are two options, how to use the `EXECUTABLE` clause. The first option, is to filter for the current user:

Show all functions executable by the current user

```
SHOW FUNCTIONS EXECUTABLE BY CURRENT USER YIELD name, category, description, rolesExecution,
rolesBoostedExecution
LIMIT 5
```

Result

name	category	description	rolesExecution	rolesBoostedExecution
"abs"	"Numeric"	"Returns the absolute value of an `INTEGER` or `FLOAT`."	<null>	<null>
"acos"	"Trigonometric"	"Returns the arccosine of a `FLOAT` in radians."	<null>	<null>
"all"	"Predicate"	"Returns true if the predicate holds for all elements in the given `LIST<ANY>`."	<null>	<null>
"allReduce"	"Predicate"	"Evaluates `expression` against each element of `LIST<ANY>`, with the result stored in `accumulator`. Returns true if `predicate` holds for each iteration of `accumulator`."	<null>	<null>

name	category	description	rolesExecution	rolesBoostedExecution
"any"	"Predicate"	"Returns true if the predicate holds for at least one element in the given LIST<ANY>."	<null>	<null>
Rows: 5				

Notice that the two `roles` columns are empty due to missing the `SHOW ROLE` privilege.

The second option, is to filter for a specific user:

Show all functions executable by a specific user

```
SHOW FUNCTIONS EXECUTABLE BY jake
```

Combine `SHOW FUNCTIONS` with other Cypher clauses

Introduced in 2026.05 | Cypher 25 only

`SHOW FUNCTIONS` can be combined with other Cypher clauses to form a single query.

> **Note:**
>
> When combining `SHOW FUNCTIONS` with other clauses, the `YIELD` clause is mandatory and must not be omitted. `YIELD` must explicitly list the yielded columns. `YIELD *` is not permitted. The query must also end with a valid last clause (like a `RETURN` or an updating clause).

Return all functions with more than one signature

```
SHOW FUNCTIONS
YIELD name, signature
WITH name, collect(signature) AS signatures
FILTER size(signatures) > 1
RETURN name, signatures
```

Return columns

`SHOW FUNCTIONS` will produce a table with the following columns:

Show functions output

Column	Description	Type
`name`	The name of the function. Default Output	`STRING`
`category`	The function category, for example `scalar` or `string`. Default Output	`STRING`
`description`	The function description. Default Output	`STRING`

Column	Description	Type
`signature`	The signature of the function.	`STRING`
`isBuiltIn`	Whether the function is built-in or user-defined.	`BOOLEAN`
`argumentDescription`	List of the arguments for the function, as map of strings and booleans with `name`, `type`, `default`, `isDeprecated`, and `description`.	`LIST<MAP>`
`returnDescription`	The return value type.	`STRING`
`aggregating`	Whether the function is aggregating or not.	`BOOLEAN`
`rolesExecution`	List of roles permitted to execute this function. Is `null` without the `SHOW ROLE` privilege.	`LIST<STRING>`
`rolesBoostedExecution`	List of roles permitted to use boosted mode when executing this function. Is `null` without the `SHOW ROLE` privilege.	`LIST<STRING>`
`isDeprecated`	Whether the function is deprecated.	`BOOLEAN`
`deprecatedBy`	The replacement function to use in case of deprecation; otherwise `null`.	`STRING`

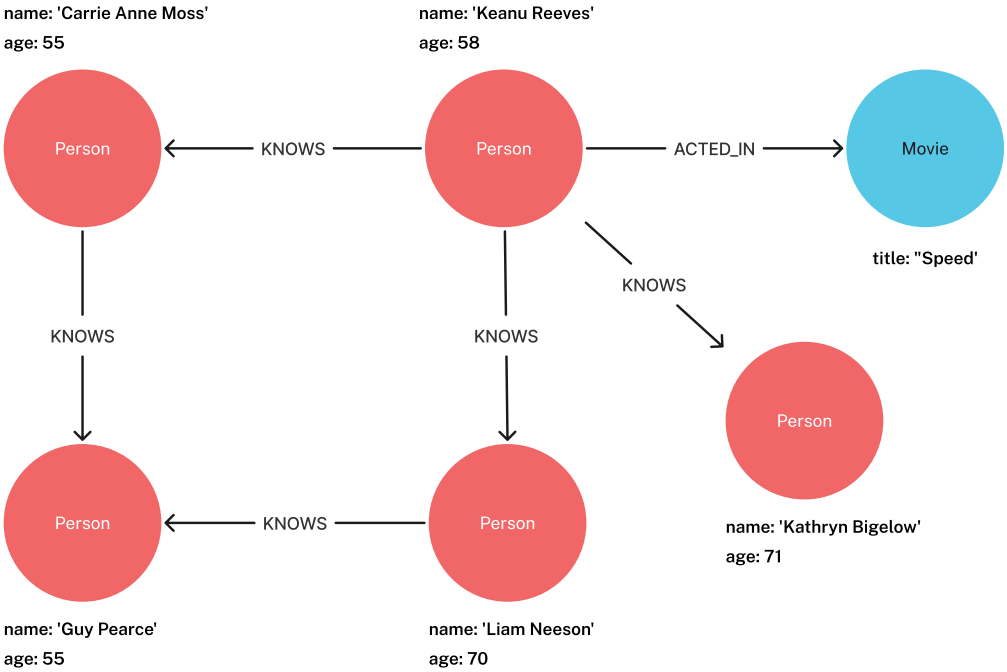
Aggregating functions

An aggregating function performs a calculation over a set of values, returning a single value. Aggregation can be computed over all the matching paths, or it can be further divided by introducing grouping keys.

To learn more about how Cypher handles aggregations performed on zero rows, visit Neo4j Knowledge Base → Understanding aggregations on zero rows.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (keanu:Person {name: 'Keanu Reeves', age: 58}),
  (liam:Person {name: 'Liam Neeson', age: 70}),
  (carrie:Person {name: 'Carrie Anne Moss', age: 55}),
  (guy:Person {name: 'Guy Pearce', age: 55}),
  (kathryn:Person {name: 'Kathryn Bigelow', age: 71}),
  (speed:Movie {title: 'Speed'}),
  (keanu)-[:ACTED_IN]->(speed),
  (keanu)-[:KNOWS]->(carrie),
  (keanu)-[:KNOWS]->(liam),
  (keanu)-[:KNOWS]->(kathryn),
  (carrie)-[:KNOWS]->(guy),
  (liam)-[:KNOWS]->(guy)
```

avg()

Details

Syntax	`avg(input)`		
Description	Returns the average of a set of `INTEGER`, `FLOAT` or `DURATION` values.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT | DURATION`	A value aggregated to form an average.
Returns	`INTEGER | FLOAT | DURATION`		

Considerations

Any `null` values are excluded from the calculation.
`avg(null)` returns `null`.

Example 161. avg() - numerical values

Query

```
MATCH (p:Person)
RETURN avg(p.age)
```

The average of all the values in the property `age` is returned:

Result

avg(p.age)
61.8
Rows: 1

Example 162. avg() - duration values

Query

```
UNWIND [duration('P2DT3H'), duration('PT1H45S')] AS dur
RETURN avg(dur)
```

The average of the two supplied `DURATION` values is returned:

Result

avg(dur)
P1DT2H22.5S
Rows: 1

collect()

Details

Syntax	`collect(input)`		
Description	Returns a list containing the values returned by an expression.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value aggregated into a list.
Returns	`LIST<ANY>`		

Considerations

Any `null` values are ignored and will not be added to the list.

`collect(null)` returns an empty list.

Example 163. collect()

Query

```
MATCH (p:Person)
RETURN collect(p.age)
```

All the values are collected and returned in a single list:

Result

collect(p.age)
[58, 70, 55, 55, 71]
Rows: 1

collect_list()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`collect_list(input)`		
Description	Returns a list containing the values returned by an expression.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value aggregated into a list.
Returns	`LIST<ANY>`		

This function is an alias to the `collect()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

Any `null` values are ignored and will not be added to the list.
`collect_list(null)` returns an empty list.

Example 164. collect_list()

Query

```
MATCH (p:Person)
RETURN collect_list(p.age)
```

All the values are collected and returned in a single list:

Result

collect_list(p.age)
`[58, 70, 55, 55, 71]`
Rows: 1

count()

Details

Syntax	`count(input)`		
Description	Returns the number of values or rows.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be aggregated.
Returns	`INTEGER`		

Considerations

`count(*)` includes rows returning `null`.
`count(input)` ignores `null` values.
`count(null)` returns `0`.

Neo4j maintains a transactional count store for holding count metadata, which can significantly increase the speed of queries using the `count()` function. For more information about the count store, refer to Neo4j Knowledge Base → Fast counts using the count store.

Using `count(*)` to return the number of nodes

The function `count(*)` can be used to return the number of nodes; for example, the number of nodes connected to a node `n`.

Example 165. count()

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})-->(x)
RETURN labels(p), p.age, count(*)
```

The labels and `age` property of the start node `Keanu Reeves` and the number of nodes related to it are returned:

Result

labels(p)	p.age	count(*)
["Person"]	58	4
Rows: 1		

Using `count(*)` to group and count relationship types

The function `count(*)` can be used to group the type of matched relationships and return the number of types.

Example 166. count()

Query

```
MATCH (p:Person {name: 'Keanu Reeves'})-[r]->()
RETURN type(r), count(*)
```

The type of matched relationships are grouped and the group count of relationship types is returned:

Result

type(r)	count(*)
"ACTED_IN"	1
"KNOWS"	3
Rows: 2	

Counting non-`null` values

Instead of simply returning the number of rows with `count(*)`, the function `count(expression)` can be used to return the number of non-`null` values returned by the expression.

Example 167. count()

Query

```
MATCH (p:Person)
RETURN count(p.age)
```

The number of nodes with the label `Person` and a property `age` is returned: (To calculate the sum, use `sum(n.age)`)

Result

count(p.age)
5

count(p.age)
Rows: 1

Counting with and without duplicates

The default behavior of the `count` function is to count all matching results, including duplicates. To avoid counting duplicates, use the `DISTINCT` keyword.

It is also possible to use the `ALL` keyword with aggregating functions. This will count all results, including duplicates, and is functionally the same as not using the `DISTINCT` keyword. The `ALL` keyword was introduced as part of Cypher's GQL conformance.

This example tries to find all friends of friends of `Keanu Reeves` and count them. It shows the behavior of using both the `ALL` and the `DISTINCT` keywords:

Example 168. count()

Query

```
MATCH (p:Person)-->(friend:Person)-->(friendOfFriend:Person)
WHERE p.name = 'Keanu Reeves'
RETURN friendOfFriend.name, count(friendOfFriend), count(ALL friendOfFriend), count(DISTINCT
friendOfFriend)
```

The nodes `Carrie Anne Moss` and `Liam Neeson` both have an outgoing `KNOWS` relationship to `Guy Pearce`. The `Guy Pearce` node will, therefore, get counted twice when not using `DISTINCT`.

Result

friendOfFriend.name	count(friendOfFriend)	count(ALL friendOfFriend)	count(DISTINCT friendOfFriend)
"Guy Pearce"	2	2	1
Rows: 1			

max()

Details

Syntax	`max(input)`		
Description	Returns the maximum value in a set of values.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be aggregated.
Returns	`ANY`		

Considerations

Any `null` values are excluded from the calculation.
In a mixed set, any numeric value is always considered to be higher than any `STRING` value, and any `STRING` value is always considered to be higher than any `LIST<ANY>`.
Lists are compared in dictionary order, i.e. list elements are compared pairwise in ascending order from the start of the list to the end.
`max(null)` returns `null`.

Example 169. max()

Query

```
UNWIND [1, 'a', null, 0.2, 'b', '1', '99'] AS val
RETURN max(val)
```

The highest of all the values in the mixed set — in this case, the numeric value `1` — is returned:

Result

max(val)
1
Rows: 1

Note:

The value `'99'` (a `STRING`), is considered to be a lower value than `1` (an `INTEGER`), because `'99'` is a `STRING`.

Example 170. max()

Query

```
UNWIND [[1-15, 'a', 89], [1, 2]] AS val
RETURN max(val)
```

The highest of all the lists in the set — in this case, the list `[1, 2]` — is returned, as the number `2` is considered to be a higher value than the `STRING` `'a'`, even though the list `[1, 'a', 89]` contains more elements.

Result

max(val)
[1,2]
Rows: 1

Example 171. max()

Query

```
MATCH (p:Person)
RETURN max(p.age)
```

The highest of all the values in the property `age` is returned:

Result

max(p.age)
71
Rows: 1

min()

Details

Syntax	`min(input)`		
Description	Returns the minimum value in a set of values.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be aggregated.
Returns	`ANY`		

Considerations

Any `null` values are excluded from the calculation.
In a mixed set, any `STRING` value is always considered to be lower than any numeric value, and any `LIST<ANY>` is always considered to be lower than any `STRING`.
Lists are compared in dictionary order, i.e. list elements are compared pairwise in ascending order from the start of the list to the end.
`min(null)` returns `null`.

Example 172. min()

Query

```
UNWIND [1, 'a', null, 0.2, 'b', '1', '99'] AS val
RETURN min(val)
```

The lowest of all the values in the mixed set — in this case, the `STRING` value `"1"` — is returned. Note that the (numeric) value `0.2`, which may *appear* at first glance to be the lowest value in the list, is

considered to be a higher value than `"1"` as the latter is a `STRING`.

Result

min(val)
"1"
Rows: 1

Example 173. min()

Query

```
UNWIND ['d', [1, 2], ['a', 'c', 23]] AS val
RETURN min(val)
```

The lowest of all the values in the set — in this case, the list `['a', 'c', 23]` — is returned, as (i) the two lists are considered to be lower values than the `STRING` `"d"`, and (ii) the `STRING` `"a"` is considered to be a lower value than the numerical value `1`.

Result

min(val)
["a","c",23]
Rows: 1

Example 174. min()

Query

```
MATCH (p:Person)
RETURN min(p.age)
```

The lowest of all the values in the property `age` is returned:

Result

min(p.age)
55
Rows: 1

percentileCont()

Details

Syntax	`percentileCont(input, percentile)`

Description	Returns the percentile of a value over a group using linear interpolation.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to be aggregated.
	`percentile`	`FLOAT`	A percentile between 0.0 and 1.0.
Returns	`FLOAT`		

Considerations

Any `null` values are excluded from the calculation.
`percentileCont(null, percentile)` returns `null`.

Example 175. percentileCont()

Query

```
MATCH (p:Person)
RETURN percentileCont(p.age, 0.4)
```

The 40th percentile of the values in the property `age` is returned, calculated with a weighted average:

Result

percentileCont(p.age, 0.4)
56.8
Rows: 1

percentile_cont()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`percentile_cont(input, percentile)`		
Description	Returns the percentile of a value over a group using linear interpolation.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to be aggregated.
	`percentile`	`FLOAT`	A percentile between 0.0 and 1.0.
Returns	`FLOAT`		

This function is an alias to the `percentileCont()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

Any `null` values are excluded from the calculation.
`percentile_cont(null, percentile)` returns `null`.

Example 176. percentile_cont()

Query

```
MATCH (p:Person)
RETURN percentile_cont(p.age, 0.4)
```

The 40th percentile of the values in the property `age` is returned, calculated with a weighted average:

Result

percentile_cont(p.age, 0.4)
56.8
Rows: 1

percentileDisc()

Details

Syntax	`percentileDisc(input, percentile)`		
Description	Returns the nearest `INTEGER` or `FLOAT` value to the given percentile over a group using a rounding method.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT`	A value to be aggregated.
	`percentile`	`FLOAT`	A percentile between 0.0 and 1.0.
Returns	`INTEGER | FLOAT`		

Considerations

Any `null` values are excluded from the calculation.
`percentileDisc(null, percentile)` returns `null`.

Example 177. percentileDisc()

Query

```
MATCH (p:Person)
RETURN percentileDisc(p.age, 0.5)
```

The 50th percentile of the values in the property `age` is returned:

Result

percentileDisc(p.age, 0.5)
58
Rows: 1

percentile_disc()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`percentile_disc(input, percentile)`		
Description	Returns the nearest `INTEGER` or `FLOAT` value to the given percentile over a group using a rounding method.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT`	A value to be aggregated.
	`percentile`	`FLOAT`	A percentile between 0.0 and 1.0.
Returns	`INTEGER | FLOAT`		

This function is an alias to the `percentileDisc()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

Any `null` values are excluded from the calculation.
`percentile_disc(null, percentile)` returns `null`.

Example 178. percentile_disc()

Query

```
MATCH (p:Person)
RETURN percentile_disc(p.age, 0.5)
```

The 50th percentile of the values in the property `age` is returned:

Result

percentile_disc(p.age, 0.5)
58
Rows: 1

stDev()

Details

Syntax	`stDev(input)`		
Description	Returns the standard deviation for the given value over a group for a sample of a population.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	The value to calculate the standard deviation of.
Returns	`FLOAT`		

Considerations

Any `null` values are excluded from the calculation.
As of Neo4j 2026.05, `stDev(null)` returns `null`. In earlier versions, it returns `0`.

Example 179. stDev()

Query

```
MATCH (p:Person)
WHERE p.name IN ['Keanu Reeves', 'Liam Neeson', 'Carrie Anne Moss']
RETURN stDev(p.age)
```

The standard deviation of the values in the property `age` is returned:

Result

stDev(p.age)
7.937253933193772
Rows: 1

stdev_samp()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`stdev_samp(input)`		
Description	Returns the standard deviation for the given value over a group for a sample of a population.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	The value to calculate the standard deviation of.
Returns	`FLOAT`		

This function is an alias to the `stDev()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

Any `null` values are excluded from the calculation.
As of Neo4j 2026.05, `stdev_samp(null)` returns `null`. In earlier versions, it returns `0`.

Example 180. stdev_samp()

Query

```
MATCH (p:Person)
WHERE p.name IN ['Keanu Reeves', 'Liam Neeson', 'Carrie Anne Moss']
RETURN stdev_samp(p.age)
```

The standard deviation of the values in the property `age` is returned:

Result

stdev_samp(p.age)
7.937253933193772
Rows: 1

stDevP()

Details

Syntax	`stDevP(input)`		
Description	Returns the standard deviation for the given value over a group for an entire population.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	The value to calculate the population standard deviation of.

Returns	`FLOAT`

Considerations

Any `null` values are excluded from the calculation.
As of Neo4j 2026.05, `stDevP(null)` returns `null`. In earlier versions, it returns `0`.

Example 181. stDevP()

Query

```
MATCH (p:Person)
WHERE p.name IN ['Keanu Reeves', 'Liam Neeson', 'Carrie Anne Moss']
RETURN stDevP(p.age)
```

The population standard deviation of the values in the property `age` is returned:

Result

stDevP(p.age)
`6.48074069840786`
Rows: 1

stdev_pop()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`stdev_pop(input)`		
Description	Returns the standard deviation for the given value over a group for an entire population.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	The value to calculate the population standard deviation of.
Returns	`FLOAT`		

This function is an alias to the `stDevP()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

Any `null` values are excluded from the calculation.

As of Neo4j 2026.05, `stdev_pop(null)` returns `null`. In earlier versions, it returns `0`.

Example 182. stdev_pop()

Query

```
MATCH (p:Person)
WHERE p.name IN ['Keanu Reeves', 'Liam Neeson', 'Carrie Anne Moss']
RETURN stdev_pop(p.age)
```

The population standard deviation of the values in the property `age` is returned:

Result

stdev_pop(p.age)
6.48074069840786
Rows: 1

sum()

Details

Syntax	`sum(input)`		
Description	Returns the sum of a set of `INTEGER`, `FLOAT` or `DURATION` values		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT | DURATION`	A value to be aggregated.
Returns	`INTEGER | FLOAT | DURATION`		

Considerations

Any `null` values are excluded from the calculation.
`sum(null)` returns `0`.

Example 183. sum() - numeric values

Query

```
MATCH (p:Person)
RETURN sum(p.age)
```

The sum of all the values in the property `age` is returned:

Result

sum(p.age)
309
Rows: 1

Example 184. sum() - duration values

Query

```
UNWIND [duration('P2DT3H'), duration('PT1H45S')] AS dur
RETURN sum(dur)
```

The sum of the two supplied durations is returned:

Result

sum(dur)
P2DT4H45S
Rows: 1

Aggregating expressions and grouping keys

Aggregating expressions are expressions which contain one or more aggregating functions. A simple aggregating expression consists of a single aggregating function. For instance, `sum(x.a)` is an aggregating expression that only consists of the aggregating function `sum( )` with `x.a` as its argument. Aggregating expressions are also allowed to be more complex, where the result of one or more aggregating functions are input arguments to other expressions. For instance, `0.1 * (sum(x.a) / count(x.b))` is an aggregating expression that contains two aggregating functions, `sum( )` with `x.a` as its argument and `count( )` with `x.b` as its argument. Both are input arguments to the division expression.

Grouping keys are non-aggregating expressions that are used to group the values going into the aggregating functions. Grouping keys are implicit in Cypher unless you control them explicitly with the `GROUP BY` clause.

For example, given the following query containing two return expressions, `n` and `count(*)`:

```
RETURN n, count(*)
```

The first, `n` is not an aggregating function, so it will be the grouping key. The latter, `count(*)` is an aggregating function. The matching paths will be divided into different buckets, depending on the grouping key. The aggregating function will then be run on these buckets, calculating an aggregate value per bucket.

The input expression of an aggregating function can contain any expression, including expressions that are not grouping keys. However, not all expressions can be composed with aggregating functions. The example below will throw an error since `n.x`, which is not a grouping key, is combined with the

aggregating function `count(*)`.

```
RETURN n.x + count(*)
```

To sort the result set using aggregating functions, the aggregation must be included in the `ORDER BY` subclause following the `RETURN` clause. For aggregating subclause expressions, the same rules apply as for aggregating expressions in `RETURN` and `WITH`.

Examples

Example 185. Simple aggregation without any grouping keys

Query

```
MATCH (p:Person)
RETURN max(p.age)
```

Result

max(p.age)
71
Rows: 1

Example 186. Addition of an aggregation and a constant, without any grouping keys

Query

```
MATCH (p:Person)
RETURN max(p.age) + 1
```

Result

max(p.age) + 1
72
Rows: 1

Example 187. Subtraction of a property access and an aggregation

Note that `p` is a grouping key:

Query

```
MATCH (p:Person{name:'Keanu Reeves'})-[:KNOWS]-(f:Person)
RETURN p, p.age - max(f.age)
```

Result

p	p.age - max(f.age)
{{"name":"Keanu Reeves","age":58}}	-13
Rows: 1	

Example 188. Subtraction of a property access and an aggregation.

Note that `p.age` is a grouping key:

Query

```
MATCH (p:Person {name:'Keanu Reeves'})-[:KNOWS]-(f:Person)
RETURN p.age, p.age - max(f.age)
```

Result

p.age	p.age - max(f.age)
58	-13
Rows: 1	

Grouping keys themselves can be complex expressions. Cypher only recognizes a sub-expression in aggregating expressions as a grouping key if the grouping key is:

- A variable - e.g. the `p` in `RETURN p, p.age - max(f.age)`.
- A property access - e.g. the `p.age` in `RETURN p.age, p.age - max(f.age)`.
- A map access - e.g. the `p.age` in `WITH {name:'Keanu Reeves', age:58} AS p RETURN p.age, p.age - max(p.age)`.

These are called *recognizable grouping key expressions*.

If you need more complex grouping keys as operands in aggregating expressions, you can project them with `AS` in `RETURN` and `WITH`.

Using the property `p.age` will throw an exception, since `p.age` is not a grouping key. Therefore, it cannot be used in the expressions which contain the aggregating function. The below two queries would consequently return the same error message:

Query

```
MATCH (p:Person {name:'Keanu Reeves'})-[:KNOWS]-(f:Person)
RETURN p.age - max(f.age)
```

Query

```
MATCH (p:Person {name:'Keanu Reeves'})-[:KNOWS]-(f:Person)
RETURN p.age + p.age, p.age + p.age - max(f.age)
```

GQLSTATUS error chain

42I18: error: syntax error or access rule violation - reference to non-grouping sub-expression. The expression contains a non-grouping sub-expression `p`. In an aggregating context only grouping sub-expressions and constants are allowed.

42001: error: syntax error or access rule violation - invalid syntax

However, the latter query would work if rewritten to:

Query

```
MATCH (p:Person {name:'Keanu Reeves'})-[:KNOWS]-(f:Person)
WITH p.age + p.age AS groupingKey, f
RETURN groupingKey, groupingKey - max(f.age)
```

Result

groupingKey	groupingKey - max(f.age)
116	45
Rows: 1	

Rules for aggregating expressions

For aggregating expressions to be correctly computable for the buckets formed by the grouping key(s), they have to fulfill some requirements. Specifically, each sub-expression in an aggregating expression has to be either:

- an aggregating function, e.g. `sum(x.a)`.
- a constant, e.g. `0.1`.
- a parameter, e.g. `$param`.
- a grouping key, e.g. the `a` in `RETURN a, count(*)`.
- a local variable, e.g. the `x` in `count(*) + size([ x IN range(1, 10) | x ])`.
- a sub-expression, all operands of which have to be allowed in an aggregating expression.

Database functions

db.nameFromElementId()

Details

Syntax	`db.nameFromElementId(elementId)`
Description	Resolves the database name for the given element id.

Arguments	Name	Type	Description
	`elementId`	`STRING`	An element id of a node or relationship.
Returns	`STRING`		

Considerations

The name of the database can only be returned if the provided element id belongs to a standard database in the DBMS.
The function only returns the name of the database if one or more of the following are true: • the user has access to the database • the database is a standard database and the user has `DATABASE MANAGEMENT`, `CREATE DATABASE`, `DROP DATABASE`, `ALTER DATABASE` or `SET DATABASE ACCESS` privileges • the database is a composite database and the user has `COMPOSITE DATABASE MANAGEMENT`, `CREATE COMPOSITE DATABASE`, `DROP COMPOSITE DATABASE` or `ALTER COMPOSITE DATABASE` privileges

Example 189. db.nameFromElementId()

Query

```
WITH "2:efc7577d-022a-107c-a736-dbcdfc189c03:0" AS eid
RETURN db.nameFromElementId(eid) AS name
```

Returns the name of the database which the element id belongs to.

Result

name
"neo4j"
Rows: 1

Graph functions

graph.names()

Details

Syntax	`graph.names()`
Description	Lists the names of graphs in the current database.
Returns	`LIST<STRING>`

Considerations

`graph.names()` is only supported on composite databases.

`graph.names()` only returns the name of a graph reference if the user has access to it.

Example 190. graph.names()

Setup

```
CREATE DATABASE dba;
CREATE DATABASE dbb;
CREATE DATABASE dbc;
CREATE COMPOSITE DATABASE composite;
CREATE ALIAS composite.first FOR DATABASE dba;
CREATE ALIAS composite.second FOR DATABASE dbb;
CREATE ALIAS composite.third FOR DATABASE dbc;
```

Query

```
RETURN graph.names() AS name
```

The names of all graphs on the current composite database are returned.

Result

name
"composite.first"
"composite.second"
"composite.third"
Rows: 3

graph.propertiesByName()

Details

Syntax	`graph.propertiesByName(graphName)`		
Description	Returns the `MAP` of properties associated with a graph.		
Arguments	Name	Type	Description
	`graphName`	`STRING`	The name of the graph from which all associated properties will be returned.
Returns	`MAP`		

Considerations

`graph.propertiesByName()` is only supported on composite databases.
The properties in the returned `MAP` are set on the alias that adds the graph as a constituent of a composite database.

`graph.propertiesByName()` only returns the properties of a graph reference if the user has access to it.

Example 191. graph.propertiesByName()

Setup

```
CREATE DATABASE dba;
CREATE DATABASE dbb;
CREATE DATABASE dbc;
CREATE COMPOSITE DATABASE composite;
CREATE ALIAS composite.first FOR DATABASE dba
  PROPERTIES {number: 1, tags: ['A', 'B']};
CREATE ALIAS composite.second FOR DATABASE dbb
  PROPERTIES {number: 0, tags: ['A']};
CREATE ALIAS composite.third FOR DATABASE dbc
  PROPERTIES {number: 2, tags: ['B', 'C']};
```

Query

```
UNWIND graph.names() AS name
RETURN name, graph.propertiesByName(name) AS props
```

Properties for all graphs on the current composite database are returned.

Result

name	props
"composite.first"	{number: 1, tags: ["A", "B"]}
"composite.second"	{number: 0, tags: ["A"]}
"composite.third"	{number: 2, tags: ["B", "C"]}

Rows: 3

Query

```
UNWIND graph.names() AS name
WITH name, graph.propertiesByName(name) AS props
WHERE "A" IN props.tags
CALL () {
  USE graph.byName(name)
  MATCH (n)
  RETURN n
}
RETURN n
```

Returns all nodes from a subset of graphs that have a `tags` property containing `"A"`.

The above query uses an empty variable scope clause: `CALL () { … }` (introduced in Neo4j 5.23). If you are using an older version of Neo4j, use `CALL { … }` instead. For more information, see CALL subqueries → Importing variables.

graph.byName()

Details

Syntax	`graph.byName(name)`		
Description	Returns the graph reference of the given name.		
Arguments	Name	Type	Description
	`name`	`STRING`	The name of the graph to be resolved.
Returns	`GRAPH`		

Considerations

`graph.byName()` is only supported in the `USE` clause, on composite databases.
`graph.byName()` only returns a graph reference if the user has access to it.

Example 192. graph.byName()

Query

```
UNWIND graph.names() AS graphName
CALL () {
  USE graph.byName(graphName)
  MATCH (n)
  RETURN n
}
RETURN n
```

Returns all nodes from all graphs on the current composite database.

graph.byElementId()

Details

Syntax	`graph.byElementId(elementId)`		
Description	Returns the graph reference with the given element id.		
Arguments	Name	Type	Description
	`elementId`	`STRING`	An element id of a node or relationship.
Returns	`GRAPH`		

Considerations

If the constituent database is not a standard database in the DBMS, an error will be thrown.
`graph.byElementId()` is only supported in the `USE` clause.

`graph.byElementId()` is supported on both standard and composite databases.
On a standard database, a `USE` clause with `graph.byElementId()` cannot be combined with other `USE` clauses unless the subsequent `USE` clauses reference the same element id.
`graph.byElementId()` only returns a graph reference if the user has access to it.

Example 193. graph.byElementId()

In this example, it is assumed that the DBMS contains a composite database constituent, which contains the element id `4:c0a65d96-4993-4b0c-b036-e7ebd9174905:0`.

Query

```
USE graph.byElementId("4:c0a65d96-4993-4b0c-b036-e7ebd9174905:0")
MATCH (n) RETURN n
```

List functions

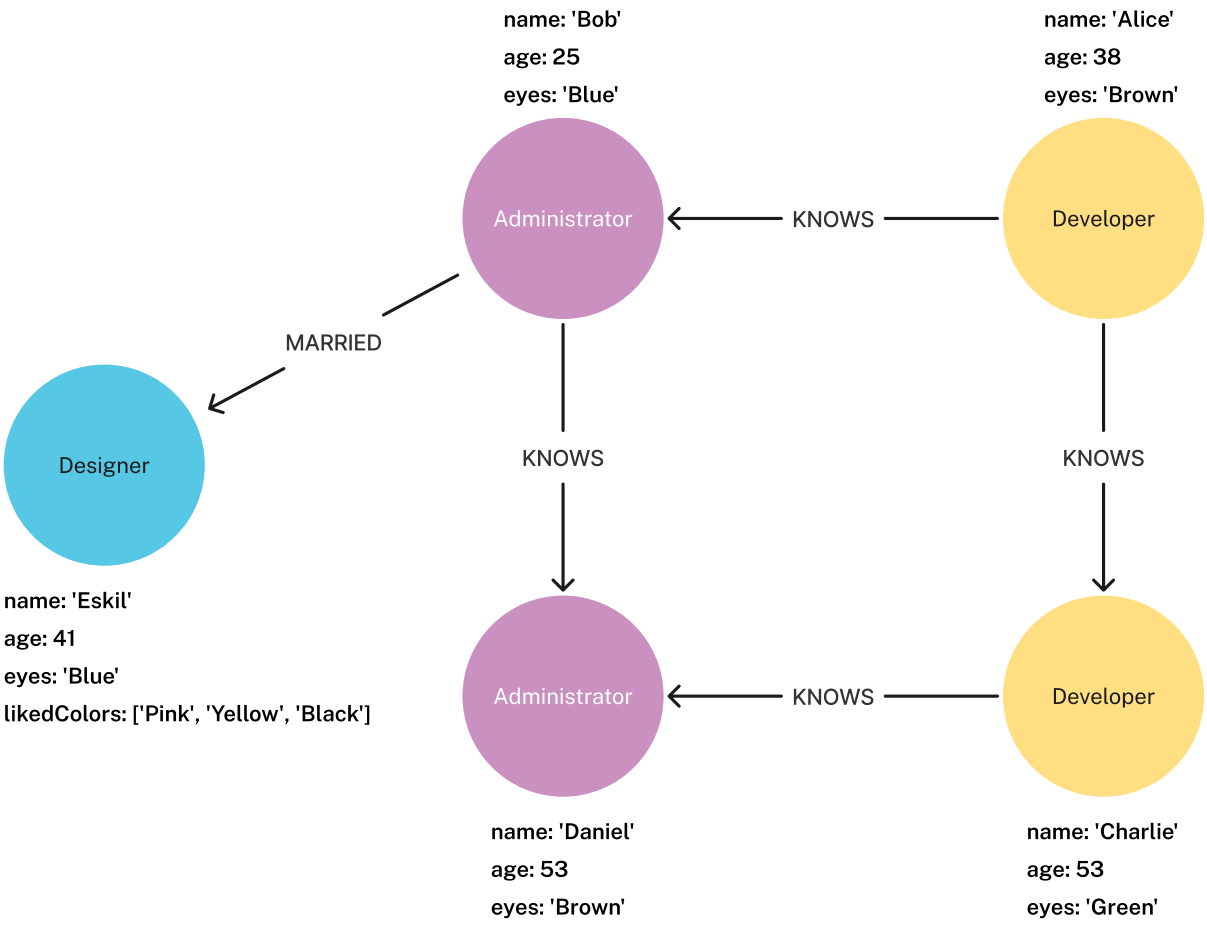
List functions return lists of different data entities.

For more information about working with `LIST` values, see:

- Values and types → Lists
- Predicates → List operators
- List expressions

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (alice:Developer {name:'Alice', age: 38, eyes: 'Brown'}),
  (bob:Administrator {name: 'Bob', age: 25, eyes: 'Blue'}),
  (charlie:Administrator {name: 'Charlie', age: 53, eyes: 'Green'}),
  (daniel:Administrator {name: 'Daniel', age: 54, eyes: 'Brown'}),
  (eskil:Designer {name: 'Eskil', age: 41, eyes: 'blue', likedColors: ['Pink', 'Yellow', 'Black']}),
  (alice)-[:KNOWS]->(bob),
  (alice)-[:KNOWS]->(charlie),
  (bob)-[:KNOWS]->(daniel),
  (charlie)-[:KNOWS]->(daniel),
  (bob)-[:MARRIED]->(eskil)
```

coll.distinct()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.distinct(list)`		
Description	Returns the given list with all duplicate values removed.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to be deduplicated.
Returns	`LIST<ANY>`		

Considerations

`coll.distinct(null)` returns `null`.
The order of the first occurrences of the distinct values is preserved.

Example 194. coll.distinct()

Deduplicate a list of integer values

```
RETURN coll.distinct([1, 3, 2, 4, 2, 3, 1])
```

A `LIST<ANY>` containing only unique values is returned.

Result

coll.distinct([1, 3, 2, 4, 2, 3, 1])
`[1, 3, 2, 4]`
Rows: 1

Deduplicate a list of mixed values, including null

```
RETURN coll.distinct([1, true, true, null, 'a', false, true, 1, null])
```

A `LIST<ANY>` containing only unique values is returned.

Result

coll.distinct([1, true, true, null, 'a', false, true, 1, null])
`[1, true, null, 'a', false]`
Rows: 1

coll.flatten()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.flatten(list [, depth])`		
Description	Returns a list flattened to the given nesting depth.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A nested list to be flattened.
	`depth`	`INTEGER`	The maximum depth to flatten to (default value: 1).

Returns	`LIST<ANY>`

Considerations

Preserves the original order of non-list items.
If `depth` is 0, the input list is returned unchanged.
`depth` must be a non-negative integer.
`coll.flatten(null)` returns `null`.
`coll.flatten(list, null)` returns `null`.
`coll.flatten(null, depth)` returns `null`.
`coll.flatten(null, null)` returns `null`.

Example 195. coll.flatten()

Flatten a list containing nesting to level 2

```
RETURN coll.flatten(['a', ['b', ['c']]], 2)
```

A `LIST<ANY>` with any inner lists up to a nesting depth of 2 flattened.

Result

coll.flatten(['a', ['b', ['c']]], 2)
`['a', 'b', 'c']`
Rows: 1

Flatten a list to default of depth 1

```
RETURN coll.flatten(['a', ['b', ['c']]])
```

A `LIST<ANY>` with any inner lists up to a nesting depth of 1 (default depth) flattened.

Result

coll.flatten(['a', ['b', ['c']]])
`['a', 'b', ['c']]`
Rows: 1

coll.indexOf()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.indexOf(list, value)`		
Description	Returns the index of the first match of a value in the given list or -1 if the value is not present.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to be searched.
	`value`	`ANY`	A value to search for.
Returns	`INTEGER`		

Considerations

Indexing is 0-based.
Returns -1 if the value is not present in the list.
`coll.indexOf(null, null)` returns `null`.
`coll.indexOf(null, value)` returns `null`.
`coll.indexOf(list, null)` returns `null`.

Example 196. coll.indexOf()

Find the first index of a given value

```
RETURN coll.indexOf(['a', 'b', 'c', 'c'], 'c')
```

An `INTEGER` representing the index of the item in the list.

Result

coll.indexOf(['a', 'b', 'c', 'c'], 'c')
2
Rows: 1

Search for a value not present in the given list

```
RETURN coll.indexOf([1, 'b', false], 4.3)
```

The `INTEGER` -1 representing that the value is not found.

Result

coll.indexOf([1, 'b', false], 4.3)
-1

coll.indexOf([1, 'b', false], 4.3)
Rows: 1

coll.insert()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.insert(list, index, value)`		
Description	Returns a list with the given value inserted at the given index.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to add to.
	`index`	`INTEGER`	The index to add the given value at.
	`value`	`ANY`	The value to add into the list.
Returns	`LIST<ANY>`		

Considerations

Indexing is 0-based.
`coll.insert(null, null, value)` returns `null`.
`coll.insert(list, null, value)` returns `null`.
`coll.insert(null, 1, value)` returns `null`.
If the given `index` is negative or larger than the size of the given list, an error is returned.

Example 197. coll.insert()

Query

```
RETURN coll.insert([true, 'a', 1, 5.4], 1, false)
```

The original list with a new value inserted at the given index, shifting all the values following by 1.

Result

coll.insert([true, 'a', 1, 5.4], 1, false)
`[true, false, 'a', 1, 5.4]`
Rows: 1

coll.max()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.max(list)`		
Description	Returns the largest value present in the given list.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to be searched.
Returns	`ANY`		

Considerations

`coll.max(null)` returns `null`.
`coll.max([])` returns `null`.
The maximum is determined using Cypher's standard value ordering; see Equality, ordering, and comparison of value types.

Example 198. coll.max()

Query

```
RETURN coll.max([true, 'a', 1, 5.4])
```

The maximum value found in the given list based on Cypher's ordering.

Result

coll.max([true, 'a', 1, 5.4])
5.4
Rows: 1

coll.min()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.min(list)`
Description	Returns the smallest value present in the given list.

Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to be searched.
Returns	`ANY`		

Considerations

`coll.min(null)` returns `null`.
`coll.min([])` returns `null`.
The minimum is determined using Cypher's standard value ordering; see Equality, ordering, and comparison of value types.

Example 199. coll.min()

Query

```
RETURN coll.min([true, 'a', 1, 5.4])
```

The minimum value found in the given list based on Cypher's ordering.

Result

coll.min([true, 'a', 1, 5.4])
`true`
Rows: 1

coll.remove()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.remove(list, index)`		
Description	Returns a list with the value at the given index removed.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list to remove a value from.
	`index`	`INTEGER`	The index of the value to be removed.
Returns	`LIST<ANY>`		

Considerations

Indexing is 0-based.

`coll.remove(null, null)` returns `null`.
`coll.remove(null, index)` returns `null`.
`coll.remove(list, null)` returns `null`.
If the given `index` is negative or larger than the size of the given list, an error is returned.

Example 200. coll.remove()

Query

```
RETURN coll.remove([true, 'a', 1, 5.4], 1)
```

The original list with the value at the given index removed and all values following shifted by 1.

Result

coll.remove([true, 'a', 1, 5.4], 1)
`[true, 1, 5.4]`
Rows: 1

coll.sort()

Introduced in Neo4j 2025.11 | Cypher 25 only

Details

Syntax	`coll.sort(list)`		
Description	Returns a sorted list.		
Arguments	**Name**	**Type**	**Description**
	`list`	`LIST<ANY>`	A list to be sorted.
Returns	`LIST<ANY>`		

Considerations

Sorting follows Cypher's standard value ordering; see Equality, ordering, and comparison of value types.
`coll.sort(null)` returns `null`.

Example 201. coll.sort()

Query

```
RETURN coll.sort([true, 'a', 1, 2])
```

The list sorted using Cypher's ordering in ascending order.

Result

coll.sort([true, 'a', 1, 2])
`['a', true, 1, 2]`
Rows: 1

keys()

Details

Syntax	`keys(input)`		
Description	Returns a `LIST<STRING>` containing the `STRING` representations for all the property names of a `NODE`, `RELATIONSHIP` or `MAP`.		
Arguments	Name	Type	Description
	`input`	`NODE | RELATIONSHIP | MAP`	A node or relationship from which the names of all properties will be returned.
Returns	`LIST<STRING>`		

Considerations

`keys(null)` returns `null`.

Example 202. keys()

Query

```
MATCH (a) WHERE a.name = 'Alice'
RETURN keys(a)
```

A `LIST<STRING>` containing the names of all the properties on the node bound to `a` is returned.

Result

keys(a)
`["eyes", "name", "age"]`
Rows: 1

labels()

Details

Syntax	`labels(input)`		
Description	Returns a `LIST<STRING>` containing the `STRING` representations for all the labels of a `NODE`.		
Arguments	Name	Type	Description
	`input`	`NODE`	A node whose labels will be returned.
Returns	`LIST<STRING>`		

Considerations

`labels(null)` returns `null`.
The order of the returned labels is not guaranteed when using the `labels()` function.

Example 203. labels()

Query

```
MATCH (a) WHERE a.name = 'Alice'
RETURN labels(a)
```

A `LIST<STRING>` containing all the labels of the node bound to `a` is returned.

Result

labels(a)
["Developer"]
Rows: 1

nodes()

Details

Syntax	`nodes(input)`		
Description	Returns a `LIST<NODE>` containing all the `NODE` values in a `PATH`.		
Arguments	Name	Type	Description
	`input`	`PATH`	A path whose nodes will be returned.
Returns	`LIST<NODE>`		

Considerations

The `NODE` values in the returned `LIST` are guaranteed to be in the exact order they appear along the path traversal, from the starting node to the end node.
`nodes(null)` returns `null`.

Example 204. nodes()

Query

```
MATCH p = (a)-->(b)-->(c)
WHERE a.name = 'Alice' AND c.name = 'Eskil'
RETURN nodes(p)
```

A `LIST<NODE>` containing all the nodes in the path `p` is returned.

Result

nodes(p)
`[(:Developer {name: "Alice", eyes: "Brown", age: 38}), (:Administrator {name: "Bob", eyes: "Blue", age: 25}), (:Designer {name: "Eskil", likedColors: ["Pink", "Yellow", "Black"], eyes: "blue", age: 41})]`
Rows: 1

range()

Details

Syntax	`range(start, end [, step])`		
Description	Returns a `LIST<INTEGER>` comprising all `INTEGER` values within a specified range created with step length, optionally specifying a step length.		
Arguments	Name	Type	Description
	`start`	`INTEGER`	The start value of the range.
	`end`	`INTEGER`	The end value of the range.
	`step`	`INTEGER`	The size of the increment (default value: 1).
Returns	`LIST<INTEGER>`		

Considerations

To create ranges with decreasing `INTEGER` values, use a negative value `step`.
The range is inclusive for non-empty ranges, and the arithmetic progression will therefore always contain `start` and — depending on the values of `start`, `step` and `end` — `end`. The only exception where the range does not contain `start` are empty ranges.

An empty range will be returned if the value `step` is negative and `start - end` is positive, or vice versa, e.g. `range(0, 5, -1)`.

Example 205. range()

Query

```
RETURN range(0, 10), range(2, 18, 3), range(0, 5, -1)
```

Three lists of numbers in the given ranges are returned.

Result

range(0, 10)	range(2, 18, 3)	range(0, 5, -1)
`[0,1,2,3,4,5,6,7,8,9,10]`	`[2,5,8,11,14,17]`	`[]`
Rows: 1		

reduce()

Details

Syntax	`reduce(accumulator = initial, variable IN list | expression)`		
Description	Runs an expression against individual elements of a `LIST<ANY>`, storing the result of the expression in an accumulator.		
Arguments	Name	Type	Description
	`accumulator`	`ANY`	A variable that holds the result as the `list` is iterated. Starts with an `initial` value.
	`initial`	`ANY`	The starting value of the `accumulator`.
	`variable`	`ANY`	A variable that represents each element in the `list` during iteration.
	`list`	`LIST<ANY>`	The `list` that is being iterated over.
	`expression`	`ANY`	An expression that updates the `accumulator` with each iteration.
Returns	`ANY`		

Considerations

`reduce()` differs from most Cypher functions because it iterates over a list, incrementally updating an accumulator with each element based on an expression, rather than returning a result from a single evaluation. As such, Cypher's `reduce()` is analogous to the `fold` or `reduce` methods in functional languages such as Lisp and Scala.

Example 206. reduce()

Query

```
MATCH p = (a)-->(b)-->(c)
WHERE a.name = 'Alice' AND b.name = 'Bob' AND c.name = 'Daniel'
RETURN reduce(totalAge = 0, n IN nodes(p) | totalAge + n.age) AS reduction
```

The `age` property of all `NODE` values in the `PATH` are summed and returned as a single value.

Result

reduction
117
Rows: 1

relationships()

Details

Syntax	`relationships(input)`		
Description	Returns a `LIST<RELATIONSHIP>` containing all the `RELATIONSHIP` values in a `PATH`.		
Arguments	Name	Type	Description
	`input`	`PATH`	The path from which all relationships will be returned.
Returns	`LIST<RELATIONSHIP>`		

Considerations

`relationships(null)` returns `null`.

Example 207. relationships()

Query

```
MATCH p = (a)-->(b)-->(c)
WHERE a.name = 'Alice' AND c.name = 'Eskil'
RETURN relationships(p)
```

A `LIST<RELATIONSHIP>` containing all the `RELATIONSHIP` values in the `PATH` `p` is returned.

Result

relationships(p)
[[:KNOWS], [:MARRIED]]
Rows: 1

reverse()

Details

Syntax	`reverse(input)`		
Description	Returns a `STRING` or `LIST<ANY>` in which the order of all characters or elements in the given `STRING` or `LIST<ANY>` have been reversed.		
Arguments	Name	Type	Description
	`input`	`STRING | LIST<ANY>`	The string or list to be reversed.
Returns	`STRING | LIST<ANY>`		

Considerations

Any `null` element in `original` is preserved.
See also String functions → `reverse()`.

Example 208. reverse()

Query

```
WITH [4923,'abc',521, null, 487] AS ids
RETURN reverse(ids)
```

Result

reverse(ids)
[487,<null>,521,"abc",4923]
Rows: 1

tail()

Details

Syntax	`tail(input)`
Description	Returns all but the first element in a `LIST<ANY>`.

Arguments	Name	Type	Description
	`input`	`LIST<ANY>`	A list from which all but the first element will be returned.
Returns	`LIST<ANY>`		

Example 209. tail()

Query

```
MATCH (a) WHERE a.name = 'Eskil'
RETURN a.likedColors, tail(a.likedColors)
```

The property named `likedColors` and a `LIST<ANY>` comprising all but the first element of the `likedColors` property are returned.

Result

a.likedColors	tail(a.likedColors)
`["Pink", "Yellow", "Black"]`	`["Yellow", "Black"]`
Rows: 1	

toBooleanList()

Details

Syntax	`toBooleanList(input)`		
Description	Converts a `LIST<ANY>` of values to a `LIST<BOOLEAN>` values. If any values are not convertible to `BOOLEAN` they will be null in the `LIST<BOOLEAN>` returned.		
Arguments	Name	Type	Description
	`input`	`LIST<ANY>`	A list of values to be converted into a list of booleans.
Returns	`LIST<BOOLEAN>`		

Considerations

Any `null` element in `input` is preserved.
Any `BOOLEAN` value in `input` is preserved.
If the `input` is `null`, `null` will be returned.
If the `input` is not a `LIST<ANY>`, an error will be returned.

The conversion for each value in `input` is done according to the `toBooleanOrNull()` function.

Example 210. toBooleanList()

Query

```
RETURN toBooleanList(null) as noList,
toBooleanList([null, null]) as nullsInList,
toBooleanList(['a string', true, 'false', null, ['A','B']]) as mixedList
```

Result

noList	nullsInList	mixedList
<null>	[<null>,<null>]	[<null>,true,false,<null>,<null>]
Rows: 1		

toFloatList()

Details

Syntax	`toFloatList(input)`		
Description	Converts a `LIST<ANY>` or a `VECTOR` to a `LIST<FLOAT>` values. If any values are not convertible to `FLOAT` they will be null in the `LIST<FLOAT>` returned.		
Arguments	Name	Type	Description
	`input`	`VECTOR` | `LIST<ANY>`	A list of values or a vector value to be converted into a list of floats.
Returns	`LIST<FLOAT>`		

Considerations

Any `null` element in `input` is preserved.
Any `FLOAT` value in `input` is preserved.
If the `input` is `null`, `null` will be returned.
If the `input` is not a `VECTOR` or `LIST<ANY>`, an error will be returned.
`input` accepts `VECTOR` values as of Neo4j 2025.10
The conversion for each value in `input` is done according to the `toFloatOrNull()` function.

Example 211. toFloatList()

Query

```
RETURN toFloatList(null) as noList,
toFloatList([null, null]) as nullsInList,
toFloatList(['a string', 2.5, '3.14159', null, ['A','B']]) as mixedList
```

Result

noList	nullsInList	mixedList
<null>	[<null>,<null>]	[<null>,2.5,3.14159,<null>,<null>]
Rows: 1		

toIntegerList()

Details

Syntax	`toIntegerList(input)`		
Description	Converts a `LIST<ANY>` or a `VECTOR` to a `LIST<INTEGER>` values. If any values are not convertible to `INTEGER` they will be null in the `LIST<INTEGER>` returned.		
Arguments	Name	Type	Description
	`input`	`VECTOR` | `LIST<ANY>`	A list of values or a vector value to be converted into a list of integers.
Returns	`LIST<INTEGER>`		

Considerations

Any `null` element in `input` is preserved.
Any `INTEGER` value in `input` is preserved.
If the `input` is `null`, `null` will be returned.
If the `input` is not a `VECTOR` or a `LIST<ANY>`, an error will be returned.
`input` accepts `VECTOR` values as of Neo4j 2025.10
The conversion for each value in `list` is done according to the `toIntegerOrNull()` function.

Example 212. toIntegerList()

Query

```
RETURN toIntegerList(null) as noList,
toIntegerList([null, null]) as nullsInList,
```

```
toIntegerList(['a string', 2, '5', null, ['A','B']]) as mixedList
```

Result

noList	nullsInList	mixedList
<null>	[<null>,<null>]	[<null>,2,5,<null>,<null>]
Rows: 1		

toStringList()

Details

Syntax	`toStringList(input)`		
Description	Converts a `LIST<ANY>` to a `LIST<STRING>` values.		
Arguments	Name	Type	Description
	`input`	`LIST<ANY>`	A list of values to be converted into a list of strings.
Returns	`LIST<STRING>`		

Considerations

Any `null` element in `input` is preserved.
Any `STRING` value in `input` is preserved.
If the `input` is `null`, `null` will be returned.
If the `input` is not a `LIST<ANY>`, an error will be returned.
The conversion for each value in `input` is done according to the `toStringOrNull()` function.
Prior to Neo4j 2026.09, this function did not accept `LIST`, `MAP`, `NODE`, `RELATIONSHIP`, and `PATH` types.

Example 213. toStringList()

Query

```
RETURN toStringList(null) as noList,
toStringList([null, null]) as nullsInList,
toStringList(['already a string', 2, date({year:1955, month:11, day:5}), null, ['A','B']]) as
mixedList
```

Result

noList	nullsInList	mixedList
NULL	["null", "null"]	["already a string", "2", "1955-11-05", "null", "[A, B]"]

noList	nullsInList	mixedList
Rows: 1		

LOAD CSV functions

LOAD CSV functions can be used to get information about the file that is processed by the `LOAD CSV` clause.

The functions described on this page are only useful when run on a query that uses `LOAD CSV`. In all other contexts they always return `null`. If there are multiple `LOAD CSV` clauses in a query, the functions return information about the most recently executed `LOAD CSV` clause.

linenumber()

Details

Syntax	`linenumber()`
Description	Returns the line number that LOAD CSV is currently using.
Returns	`INTEGER`

Considerations

`null` will be returned if this function is called without a `LOAD CSV` context.
If the CSV file contains headers, the headers will be `linenumber` 1 and the 1st row of data will have a `linenumber` of 2.

file()

Details

Syntax	`file()`
Description	Returns the absolute path of the file that LOAD CSV is using.
Returns	`STRING`

Considerations

`null` will be returned if this function is called without a `LOAD CSV` context.

Mathematical functions

Logarithmic functions

Logarithmic mathematical functions operate on numeric expressions only, and will return an error if used on any other values. See also Mathematical operators.

e()

Details

Syntax	`e()`
Description	Returns the base of the natural logarithm, e.
Returns	`FLOAT`

Example 214. e()

Query

```
RETURN e()
```

The base of the natural logarithm, `e`, is returned.

Result

e()
2.718281828459045
Rows: 1

exp()

Details

Syntax	`exp(input)`		
Description	Returns e^n, where e is the base of the natural logarithm, and n is the value of the argument expression.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to which the base of the natural logarithm, e, will be raised.
Returns	`FLOAT`		

Considerations

`exp(null)` returns `null`.
`exp()` returns `Infinity` when the return value is greater than the largest `FLOAT` value (Java Double.MAX_VALUE).

Example 215. exp()

Query

```
RETURN exp(2)
```

`e` to the power of `2` is returned.

Result

exp(2)
7.38905609893065
Rows: 1

log()

Details

Syntax	`log(input)`		
Description	Returns the natural logarithm of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value for which the natural logarithm will be returned.
Returns	`FLOAT`		

Considerations

`log(null)` returns `null`.
`log(0)` returns `-Infinity`.
If (`input` < 0), then (`log(input)`) returns `NaN`.

Example 216. log()

Query

```
RETURN log(27)
```

The natural logarithm of `27` is returned.

Result

log(27)
3.295836866004329
Rows: 1

ln()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`ln(input)`		
Description	Returns the natural logarithm of a `FLOAT`.		
Arguments	**Name**	**Type**	**Description**
	`input`	`FLOAT`	A value for which the natural logarithm will be returned.
Returns	`FLOAT`		

This function is an alias to the `log()` function, and it was introduced as part of Cypher's GQL conformance.

Differences from GQL

In GQL, the natural logarithm function must raise an exception when the input is zero or negative. In Cypher, `ln(0)` returns `-Infinity` and `ln(negative)` returns `NaN`. Changing Cypher to raise an error for these inputs would be a breaking change, so the difference is documented here and in the GQL conformance section.

Considerations

`ln(null)` returns `null`.
`ln(0)` returns `-Infinity`.
If (`input` < 0), then (`ln(input)`) returns `NaN`.

Example 217. ln()

Query

```
RETURN ln(27)
```

The natural logarithm of `27` is returned.

Result

ln(27)
3.295836866004329
Rows: 1

log10()

Details

Syntax	`log10(input)`		
Description	Returns the common logarithm (base 10) of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value for which the common logarithm (base 10) will be returned.
Returns	`FLOAT`		

Considerations

`log10(null)` returns `null`.
`log10(0)` returns `-Infinity`.
If (`input` < 0), then (`log10(input)`) returns `NaN`.

Example 218. log10()

Query

```
RETURN log10(27)
```

The common logarithm of `27` is returned.

Result

log10(27)
1.4313637641589874
Rows: 1

sqrt()

Details

Syntax	`sqrt(input)`

Description	Returns the square root of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	The value to calculate the square root of.
Returns	`FLOAT`		

Considerations

`sqrt(null)` returns `null`.
If (`input` < 0), then (`sqrt(input)`) returns `NaN`.

Example 219. sqrt()

Query

```
RETURN sqrt(256)
```

The square root of `256` is returned.

Result

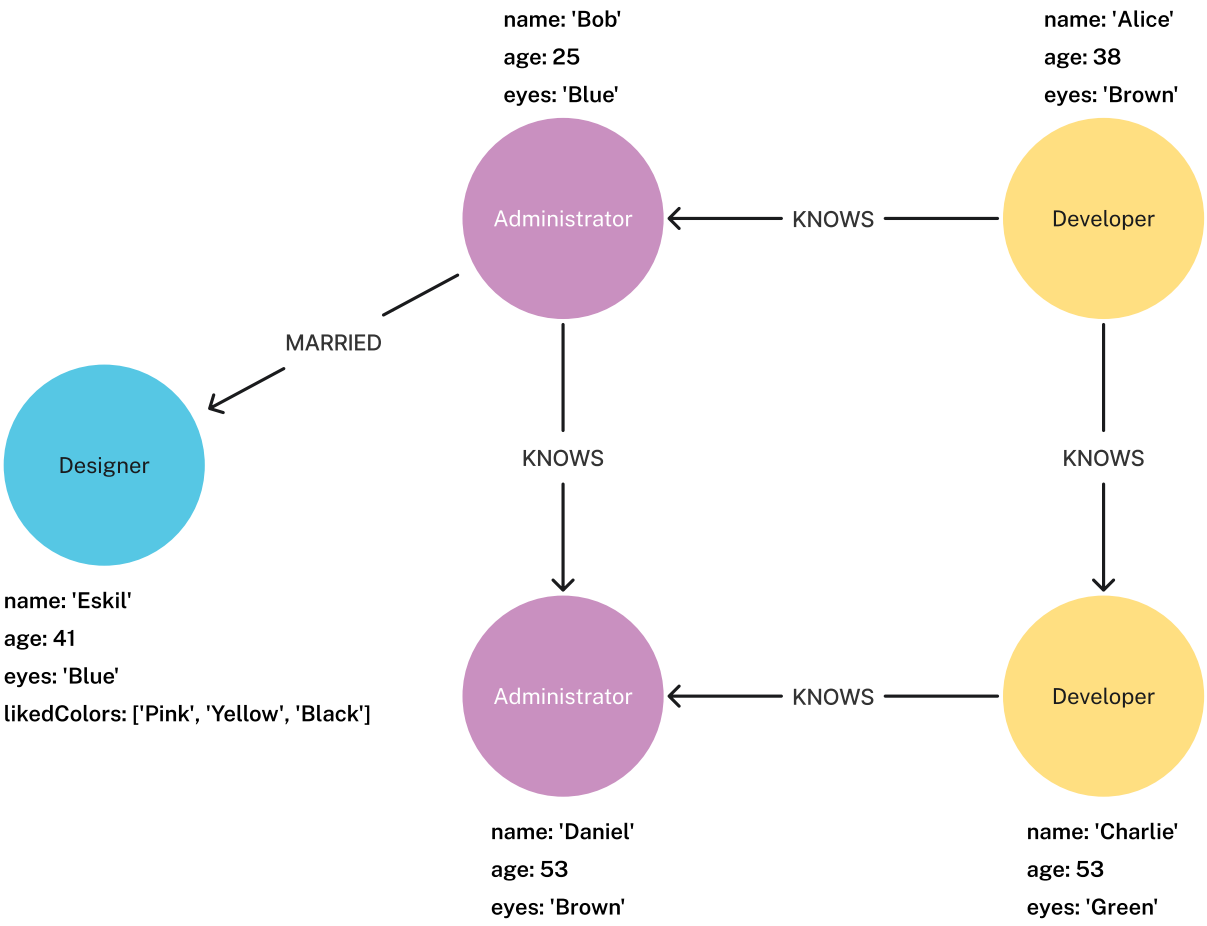
sqrt(256)
16.0
Rows: 1

Numeric functions

Numeric mathematical functions operate on numeric expressions only, and will return an error if used on any other values. See also Mathematical operators.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (alice:Developer {name:'Alice', age: 38, eyes: 'Brown'}),
  (bob:Administrator {name: 'Bob', age: 25, eyes: 'Blue'}),
  (charlie:Administrator {name: 'Charlie', age: 53, eyes: 'Green'}),
  (daniel:Adminstrator {name: 'Daniel', age: 54, eyes: 'Brown'}),
  (eskil:Designer {name: 'Eskil', age: 41, eyes: 'blue', likedColors: ['Pink', 'Yellow', 'Black']}),
  (alice)-[:KNOWS]->(bob),
  (alice)-[:KNOWS]->(charlie),
  (bob)-[:KNOWS]->(daniel),
  (charlie)-[:KNOWS]->(daniel),
  (bob)-[:MARRIED]->(eskil)
```

abs()

Details

Syntax	`abs(input)`		
Description	Returns the absolute value of an `INTEGER` or `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT`	A numeric value from which the absolute number will be returned.
Returns	`INTEGER | FLOAT`		

Considerations

`abs(null)` returns `null`.
If `input` is negative, `-(input)` (i.e. the *negation* of `input`) is returned.

Example 220. abs()

Query

```
MATCH (a), (e) WHERE a.name = 'Alice' AND e.name = 'Eskil'
RETURN a.age, e.age, abs(a.age - e.age)
```

The absolute value of the age difference is returned.

Result

a.age	e.age	abs(a.age - e.age)
38	41	3
Rows: 1		

ceil()

Details

Syntax	`ceil(input)`		
Description	Returns the smallest `FLOAT` that is greater than or equal to a number and equal to an `INTEGER`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to be rounded to the nearest higher integer.
Returns	`FLOAT`		

Considerations

`ceil(null)` returns `null`.

Example 221. ceil()

Query

```
RETURN ceil(0.1)
```

The ceil of `0.1` is returned.

Result

ceil(0.1)
1.0
Rows: 1

ceiling()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`ceiling(input)`		
Description	Returns the smallest `FLOAT` that is greater than or equal to a number and equal to an `INTEGER`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to be rounded to the nearest higher integer.
Returns	`FLOAT`		

This function is an alias to the `ceil()` function, and it was introduced as part of Cypher's GQL conformance. The underlying `ceil()` function is already GQL compliant; only the GQL naming alias is added.

Considerations

`ceiling(null)` returns `null`.

Example 222. ceiling()

Query

```
RETURN ceiling(0.1)
```

The ceiling of `0.1` is returned.

Result

ceiling(0.1)
1.0
Rows: 1

floor()

Details

Syntax	`floor(input)`		
Description	Returns the largest `FLOAT` that is less than or equal to a number and equal to an `INTEGER`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value to be rounded to the nearest lower integer.
Returns	`FLOAT`		

Considerations

`floor(null)` returns `null`.

Example 223. floor()

Query

```
RETURN floor(0.9)
```

The floor of `0.9` is returned.

Result

floor(0.9)
`0.0`
Rows: 1

isNaN()

Details

Syntax	`isNaN(input)`		
Description	Returns whether the given `INTEGER` or `FLOAT` is NaN.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT`	A numeric value to be compared against `NaN`.
Returns	`BOOLEAN`		

Considerations

`isNaN(null)` returns `null`.

Example 224. isNaN()

Query

```
RETURN isNaN(0/0.0)
```

`true` is returned since the value is `NaN`.

Result

isNaN(0/0.0)
true
Rows: 1

rand()

Details

Syntax	`rand()`
Description	Returns a random `FLOAT` in the range from 0 (inclusive) to 1 (exclusive).
Returns	`FLOAT`

Example 225. rand()

Query

```
RETURN rand()
```

A random number is returned.

Result

rand()
0.5460251846326871
Rows: 1

round()

Details

Syntax	`round(value[, precision, mode])`
Description	Returns the value of a rounded number, optionally using a specified precision and rounding mode.

Arguments	Name	Type	Description
	`value`	`FLOAT`	A value to be rounded.
	`precision`	`INTEGER | FLOAT`	The rounding precision (default: `0`).
	`mode`	`STRING`	A precision rounding mode (`UP`, `DOWN`, `CEILING`, `FLOOR`, `HALF_UP`, `HALF_DOWN`, `HALF_EVEN`) (default: `UP`).
Returns	`FLOAT`		

Modes

`mode`	Description
`UP`	Round away from zero.
`DOWN`	Round towards zero.
`CEILING`	Round towards positive infinity.
`FLOOR`	Round towards negative infinity.
`HALF_UP`	Round towards closest value of given precision, with ties always being rounded away from zero.
`HALF_DOWN`	Round towards closest value of given precision, with ties always being rounded towards zero.
`HALF_EVEN`	Round towards closest value of given precision, with ties always being rounded to the even neighbor.

Considerations

For the rounding modes, a tie means that the two closest values of the given precision are at the same distance from the given value. E.g. for precision 1, 2.15 is a tie as it has equal distance to 2.1 and 2.2, while 2.151 is not a tie, as it is closer to 2.2.

`round()` returns `null` if any of its input parameters are `null`.

Example 226. round()

Query

```
RETURN round(3.141592)
```

`3.0` is returned.

Result

round(3.141592)
3.0
Rows: 1

Example 227. round() of negative number with tie

Query

```
RETURN round(-1.5)
```

Ties are rounded towards positive infinity, therefore `-1.0` is returned.

Result

round(-1.5)
-1.0
Rows: 1

round() with precision

Example 228. round() with precision

Query

```
RETURN round(3.141592, 3)
```

`3.142` is returned.

Result

round(3.141592, 3)
3.142
Rows: 1

Example 229. round() with precision 0 and tie

Query

```
RETURN round(-1.5, 0)
```

To align with `round(-1.5)`, `-1.0` is returned.

Result

round(-1.5, 0)
-1.0
Rows: 1

Example 230. round() with precision 1 and tie

Query

```
RETURN round(-1.55, 1)
```

The default is to round away from zero when there is a tie, therefore `-1.6` is returned.

Result

round(-1.55, 1)
-1.6
Rows: 1

round() with precision and rounding mode

Example 231. round() with precision and UP rounding mode

Query

```
RETURN round(1.249, 1, 'UP') AS positive,
round(-1.251, 1, 'UP') AS negative,
round(1.25, 1, 'UP') AS positiveTie,
round(-1.35, 1, 'UP') AS negativeTie
```

The rounded values using precision 1 and rounding mode `UP` are returned.

Result

positive	negative	positiveTie	negativeTie
1.3	-1.3	1.3	-1.4
Rows: 1			

Example 232. round() with precision and DOWN rounding mode

Query

```
RETURN round(1.249, 1, 'DOWN') AS positive,
round(-1.251, 1, 'DOWN') AS negative,
round(1.25, 1, 'DOWN') AS positiveTie,
round(-1.35, 1, 'DOWN') AS negativeTie
```

The rounded values using precision 1 and rounding mode `DOWN` are returned.

Result

positive	negative	positiveTie	negativeTie
1.2	-1.2	1.2	+-1.3

Rows: 1

Example 233. round() with precision and CEILING rounding mode

Query

```
RETURN round(1.249, 1, 'CEILING') AS positive,
round(-1.251, 1, 'CEILING') AS negative,
round(1.25, 1, 'CEILING') AS positiveTie,
round(-1.35, 1, 'CEILING') AS negativeTie
```

The rounded values using precision 1 and rounding mode `CEILING` are returned.

Result

positive	negative	positiveTie	negativeTie
1.3	-1.2	1.3	-1.3

Rows: 1

Example 234. round() with precision and FLOOR rounding mode

Query

```
RETURN round(1.249, 1, 'FLOOR') AS positive,
round(-1.251, 1, 'FLOOR') AS negative,
round(1.25, 1, 'FLOOR') AS positiveTie,
round(-1.35, 1, 'FLOOR') AS negativeTie
```

The rounded values using precision 1 and rounding mode `FLOOR` are returned.

Result

positive	negative	positiveTie	negativeTie
1.2	-1.3	1.2	-1.4

Rows: 1

Example 235. round() with precision and HALF_UP rounding mode

Query

```
RETURN round(1.249, 1, 'HALF_UP') AS positive,
round(-1.251, 1, 'HALF_UP') AS negative,
```

```
round(1.25, 1, 'HALF_UP') AS positiveTie,
round(-1.35, 1, 'HALF_UP') AS negativeTie
```

The rounded values using precision 1 and rounding mode `HALF_UP` are returned.

Result

positive	negative	positiveTie	negativeTie
1.2	-1.3	1.3	-1.4

Rows: 1

Example 236. round() with precision and HALF_DOWN rounding mode

Query

```
RETURN round(1.249, 1, 'HALF_DOWN') AS positive,
round(-1.251, 1, 'HALF_DOWN') AS negative,
round(1.25, 1, 'HALF_DOWN') AS positiveTie,
round(-1.35, 1, 'HALF_DOWN') AS negativeTie
```

The rounded values using precision 1 and rounding mode `HALF_DOWN` are returned.

Result

positive	negative	positiveTie	negativeTie
1.2	-1.3	1.2	-1.3

Rows: 1

Example 237. round() with precision and HALF_EVEN rounding mode

Query

```
RETURN round(1.249, 1, 'HALF_EVEN') AS positive,
round(-1.251, 1, 'HALF_EVEN') AS negative,
round(1.25, 1, 'HALF_EVEN') AS positiveTie,
round(-1.35, 1, 'HALF_EVEN') AS negativeTie
```

The rounded values using precision 1 and rounding mode `HALF_EVEN` are returned.

Result

positive	negative	positiveTie	negativeTie
1.2	-1.3	1.2	-1.4

Rows: 1

sign()

Details

Syntax	`sign(input)`		
Description	Returns the signum of an `INTEGER` or `FLOAT`: 0 if the number is 0, -1 for any negative number, and 1 for any positive number.		
Arguments	Name	Type	Description
	`input`	`INTEGER | FLOAT`	A positive or negative number.
Returns	`INTEGER`		

Considerations

`sign(null)` returns `null`.

Example 238. sign()

Query

```
RETURN sign(-17), sign(0.1)
```

The signs of `-17` and `0.1` are returned.

Result

sign(-17)	sign(0.1)
-1	1
Rows: 1	

Trigonometric functions

Trigonometric mathematical functions operate on numeric expressions only, and will return an error if used on any other values. See also Mathematical operators.

acos()

Details

Syntax	`acos(input)`		
Description	Returns the arccosine of a `FLOAT` in radians.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`acos(null)` returns `null`.
If (`input` < -1) or (`input` > 1), then (`acos(input)`) returns `NaN`.

Example 239. acos()

Query

```
RETURN acos(0.5)
```

The arccosine of `0.5` is returned.

Result

acos(0.5)
1.0471975511965979
Rows: 1

asin()

Details

Syntax	`asin(input)`		
Description	Returns the arcsine of a `FLOAT` in radians.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`asin(null)` returns `null`.
If (`input` < -1) or (`input` > 1), then (`asin(input)`) returns `NaN`.

Example 240. asin()

Query

```
RETURN asin(0.5)
```

The arcsine of `0.5` is returned.

Result

asin(0.5)
0.5235987755982989
Rows: 1

atan()

Details

Syntax	`atan(input)`		
Description	Returns the arctangent of a `FLOAT` in radians.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`atan(null)` returns `null`.

Example 241. atan()

Query

```
RETURN atan(0.5)
```

The arctangent of `0.5` is returned.

Result

atan(0.5)
0.4636476090008061
Rows: 1

atan2()

Details

Syntax	`atan2(y, x)`
Description	Returns the arctangent2 of a set of coordinates in radians.

Arguments	Name	Type	Description
	`y`	`FLOAT`	A y angle in radians.
	`x`	`FLOAT`	An x angle in radians.
Returns	`FLOAT`		

Considerations

`atan2(null, null)`, `atan2(null, x)` and `atan(y, null)` all return `null`.

Example 242. atan2()

Query

```
RETURN atan2(0.5, 0.6)
```

The arctangent2 of `0.5` and `0.6` is returned.

Result

atan2(0.5, 0.6)
`0.6947382761967033`
Rows: 1

cos()

Details

Syntax	`cos(input)`		
Description	Returns the cosine of a `FLOAT` in radians.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`cos(null)` returns `null`.

Example 243. cos()

Query

```
RETURN cos(0.5)
```

The cosine of `0.5` is returned.

Result

cos(0.5)
0.8775825618903728
Rows: 1

cosh()

Introduced in Neo4j 2025.06 | Cypher 25 only

Details

Syntax	`cosh(input)`		
Description	Returns the hyperbolic cosine of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value.
Returns	`FLOAT`		

Considerations

`cosh(null)` returns `null`.

Example 244. cosh()

Query

```
RETURN cosh(0.7)
```

The hyperbolic cosine of `0.7` is returned.

Result

cosh(0.7)
1.255169005630943
Rows: 1

cot()

Details

Syntax	`cot(input)`

Description	Returns the cotangent of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`cot(null)` returns `null`.
`cot(0)` returns `Infinity`.

Example 245. cot()

Query

```
RETURN cot(0.5)
```

The cotangent of `0.5` is returned.

Result

cot(0.5)
1.830487721712452
Rows: 1

coth()

Introduced in Neo4j 2025.06 | Cypher 25 only

Details

Syntax	`coth(input)`		
Description	Returns the hyperbolic cotangent of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value.
Returns	`FLOAT`		

Considerations

`coth(null)` returns `null`.
`coth(0)` returns `NaN`.

The calculation of the hyperbolic cotangent, `coth(x)`, is done using the `cosh(x) / sinh(x)` method.

Example 246. coth()

Query

```
RETURN coth(0.7)
```

The hyperbolic cotangent of `0.7` is returned.

Result

coth(0.7)
1.6546216358026298
Rows: 1

degrees()

Details

Syntax	`degrees(input)`		
Description	Converts radians to degrees.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`degrees(null)` returns `null`.

Example 247. degrees

Query

```
RETURN degrees(3.14159)
```

The number of degrees in something close to *pi* is returned.

Result

degrees(3.14159)
179.9998479605043
Rows: 1

haversin()

Details

Syntax	`haversin(input)`		
Description	Returns half the versine of a number.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`haversin(null)` returns `null`.

Example 248. haversin()

Query

```
RETURN haversin(0.5)
```

The haversine of `0.5` is returned.

Result

haversin(0.5)
0.06120871905481362
Rows: 1

Spherical distance using the `haversin()` function

The `haversin()` function may be used to compute the distance on the surface of a sphere between two points (each given by their latitude and longitude).

Example 249. haversin()

In this example the spherical distance (in km) between Berlin in Germany (at lat 52.5, lon 13.4) and San Mateo in California (at lat 37.5, lon -122.3) is calculated using an average earth radius of 6371 km.

Query

```
CREATE (ber:City {lat: 52.5, lon: 13.4}), (sm:City {lat: 37.5, lon: -122.3})
RETURN 2 * 6371 * asin(sqrt(haversin(radians( sm.lat - ber.lat ))
  + cos(radians( sm.lat )) * cos(radians( ber.lat )) *
  haversin(radians( sm.lon - ber.lon )))) AS dist
```

The estimated distance between **'Berlin'** and **'San Mateo'** is returned.

Result

dist
9129.969740051658
Rows: 1

pi()

Details

Syntax	`pi()`
Description	Returns the mathematical constant pi.
Returns	`FLOAT`

Example 250. pi()

Query

```
RETURN pi()
```

The constant *pi* is returned.

Result

pi()
3.141592653589793
Rows: 1

radians()

Details

Syntax	`radians(input)`		
Description	Converts degrees to radians.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in degrees.
Returns	`FLOAT`		

Considerations

`radians(null)` returns `null`.

Example 251. radians()

Query

```
RETURN radians(180)
```

The number of radians in `180` degrees is returned (pi).

Result

radians(180)
3.141592653589793
Rows: 1

sin()

Details

Syntax	`sin(input)`		
Description	Returns the sine of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`sin(null)` returns `null`.

Example 252. sin()

Query

```
RETURN sin(0.5)
```

The sine of `0.5` is returned.

Result

sin(0.5)
0.479425538604203
Rows: 1

sinh()

Introduced in Neo4j 2025.06 Cypher 25 only

Details

Syntax	`sinh(input)`		
Description	Returns the hyperbolic sine of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value.
Returns	`FLOAT`		

Considerations

`sinh(null)` returns `null`.

Example 253. sinh()

Query

```
RETURN sinh(0.7)
```

The hyperbolic sine of `0.7` is returned.

Result

sinh(0.7)
0.7585837018395334
Rows: 1

tan()

Details

Syntax	`tan(input)`		
Description	Returns the tangent of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	An angle in radians.
Returns	`FLOAT`		

Considerations

`tan(null)` returns `null`.

Example 254. tan()

Query

```
RETURN tan(0.5)
```

The tangent of `0.5` is returned.

Result

tan(0.5)
0.5463024898437905
Rows: 1

tanh()

Introduced in Neo4j 2025.06 | Cypher 25 only

Details

Syntax	`tanh(input)`		
Description	Returns the hyperbolic tangent of a `FLOAT`.		
Arguments	Name	Type	Description
	`input`	`FLOAT`	A value.
Returns	`FLOAT`		

Considerations

`tanh(null)` returns `null`.

Example 255. tanh()

Query

```
RETURN tanh(0.7)
```

The hyperbolic tangent of `0.7` is returned.

Result

tanh(0.7)
0.6043677771171636
Rows: 1

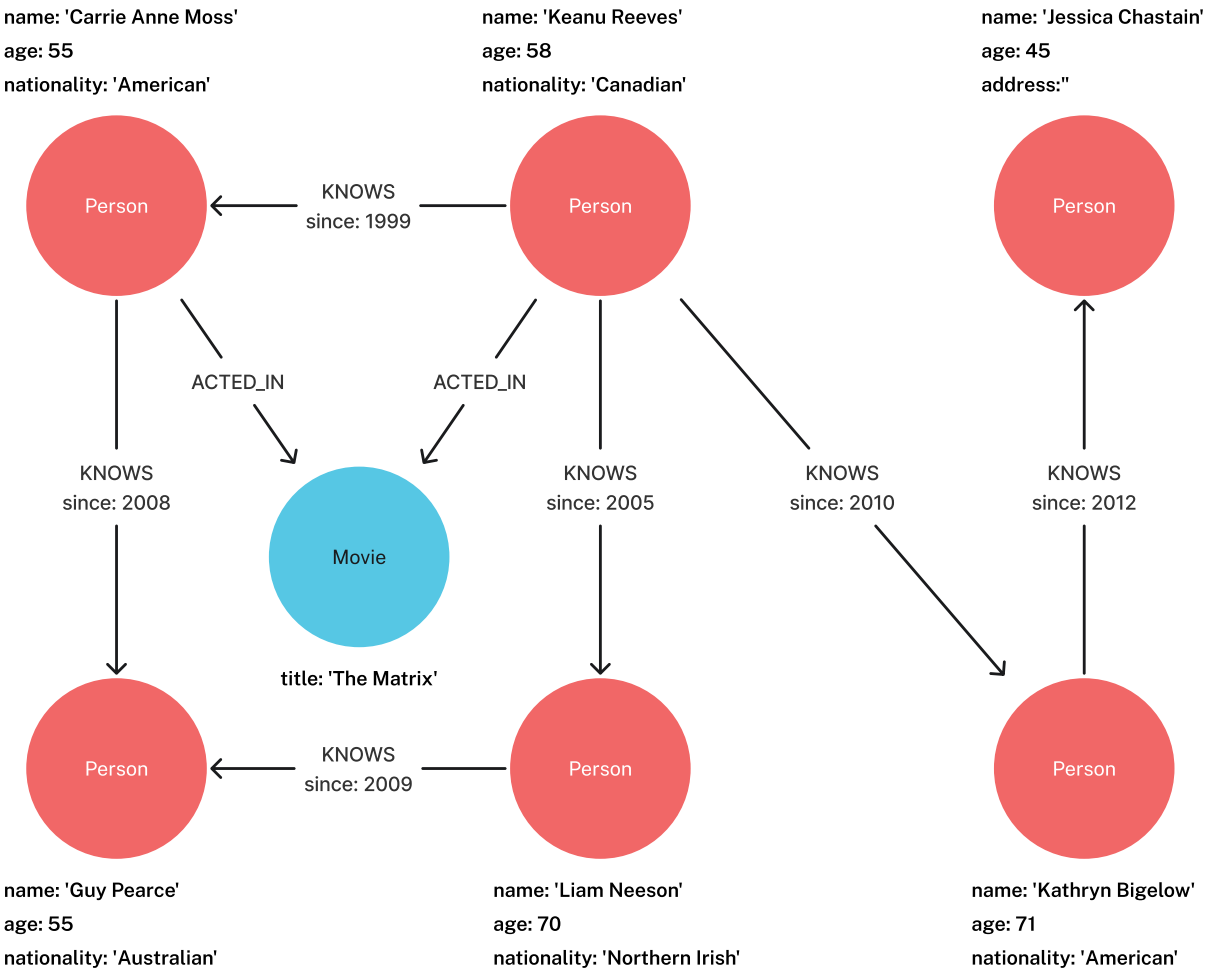
Predicate functions

Introduction

Predicates are boolean functions that return `true` or `false` for a given set of non-`null` input. They are most commonly used to filter out paths in the `WHERE` part of a query.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE
  (keanu:Person {name:'Keanu Reeves', age:58, nationality:'Canadian'}),
  (carrie:Person {name:'Carrie Anne Moss', age:55, nationality:'American'}),
  (liam:Person {name:'Liam Neeson', age:70, nationality:'Northern Irish'}),
  (guy:Person {name:'Guy Pearce', age:55, nationality:'Australian'}),
  (kathryn:Person {name:'Kathryn Bigelow', age:71, nationality:'American'}),
  (jessica:Person {name:'Jessica Chastain', age:45, address:''}),
  (theMatrix:Movie {title:'The Matrix'}),
  (keanu)-[:KNOWS {since: 1999}]->(carrie),
  (keanu)-[:KNOWS {since: 2005}]->(liam),
  (keanu)-[:KNOWS {since: 2010}]->(kathryn),
  (kathryn)-[:KNOWS {since: 2012}]->(jessica),
  (carrie)-[:KNOWS {since: 2008}]->(guy),
  (liam)-[:KNOWS {since: 2009}]->(guy),
  (keanu)-[:ACTED_IN]->(theMatrix),
```

```
(carrie)-[:ACTED_IN]->(theMatrix)
```

all()

Details

Syntax	`all(variable IN list WHERE predicate)`		
Description	Returns true if the predicate holds for all elements in the given `LIST<ANY>`.		
Arguments	Name	Type	Description
	`variable`	`ANY`	A variable that can be used within the `WHERE` clause.
	`list`	`LIST<ANY>`	A predicate must hold for all elements in this list for the function to return `true`.
	`predicate`	`ANY`	A predicate that is tested against all items in the given list.
Returns	`BOOLEAN`		

Considerations

`all()` differs from most Cypher functions because it iterates over a list, evaluating an expression for each element, rather than returning a result from a single evaluation.
`null` is returned if the `list` is `null` or if the `predicate` evaluates to `null` for at least one element and does not evaluate to false for any other element.
`all()` returns `true` if `list` is empty because there are no elements to falsify the `predicate`.

Example 256. all()

Find paths where all nodes meet a given property value

```
MATCH p = (a:Person {name: 'Keanu Reeves'})-[]-{2,}()
WHERE all(x IN nodes(p) WHERE x.age < 60)
RETURN [n IN nodes(p) | n.name] AS actorsList
```

All nodes in the returned paths have an `age`property below `60`:

Result

actorsList
`["Keanu Reeves", "Carrie Anne Moss", "Guy Pearce"]`
Rows: 2

`all()` on an empty `LIST`

```
WITH [] as emptyList
RETURN all(i in emptyList WHERE true) as allTrue, all(i in emptyList WHERE false) as allFalse
```

Result

allTrue	allFalse
TRUE	TRUE
Rows: 1	

allReduce()

Introduced in 2025.08 | Cypher 25 only

Details

Syntax	`allReduce(accumulator = initial, stepVariable IN list | reductionFunction, predicate)`
Description	Returns true if, during the stepwise evaluation of a value across the elements in a given `LIST<ANY>`, the accumulated result satisfies a specified predicate at every step. If that list is a group variable defined in a quantified path pattern, its predicate is inlined where applicable. This inlining allows for early pruning of the search space by discarding paths as soon as the predicate is not satisfied. Note that `allReduce()` predicates are not inlined when used in a shortest path pattern, and therefore do not benefit from this pruning.

Arguments	Name	Type	Description
	`accumulator`	`ANY`	A variable that holds the result of the `reductionFunction` as the `list` is iterated. It is initialized with the value of `initial`.
	`initial`	`ANY`	The value of the `accumulator` for the first evaluation of `reductionFunction`.
	`stepVariable`	`ANY`	A variable that holds the value of each element of `list` during iteration.
	`list`	`LIST<ANY>`	The list that is being iterated over.
	`reductionFunction`	`ANY`	An expression whose return value becomes the next value of the `accumulator`. The return type must match the return type of `initial`.
	`predicate`	`BOOLEAN`	A predicate that is evaluated for each iteration. It has access to both the `accumulator` and `stepVariable` variables.
Returns	`BOOLEAN`		

Considerations

`allReduce()` differs from most Cypher functions because it iterates over a list, evaluating an expression for each element, rather than returning a result from a single evaluation.
`allReduce()` combines the functionality of the `all()` and `reduce()` functions.
If all evaluations of `predicate` are `true`, `allReduce()` will return `true`.
If any evaluations of `predicate` are `false`, `allReduce()` will return `false`.
`allReduce()` returns `true` if `list` is empty because there are no elements to falsify the `predicate`.
`null` is returned if the `list` is `null` or if the `predicate` evaluates to `null` for at least one element and does not evaluate to `false` for any other element.

Example 257. allReduce()

The below query finds `KNOWS` paths with a length of `3` where the `accumulator` begins with first node's `age` and the accumulated `age` values of all nodes in the path never exceeds `230`. Paths that do not meet this requirement are excluded, such as the path with the sequence `["Keanu Reeves (58)", "Carrie Anne Moss (55)", "Guy Pearce (55)", "Liam Neeson (70)"]` which has an aggregated `age` value of `238`.

Find aggregated ages within a boundary

```
MATCH (s) (()-[:KNOWS]-(n)){3}
WHERE allReduce(
  acc = s.age,
  node IN n | acc + node.age,
  acc < 230
)
RETURN [i IN [s] + n | i.name || " (" + toString(i.age) || ")"] AS ageSequence,
       reduce(acc = 0, node IN [s] + n | acc + node.age) AS aggregatedAges
ORDER BY aggregatedAges
```

Result

ageSequence	aggregatedAges
["Carrie Anne Moss (55)", "Keanu Reeves (58)", "Kathryn Bigelow (71)", "Jessica Chastain (45)"]	229
["Jessica Chastain (45)", "Kathryn Bigelow (71)", "Keanu Reeves (58)", "Carrie Anne Moss (55)"]	229

Rows: 2

The next query uses `allReduce()` to compare neighboring relationships. It finds `KNOWS` paths with a length of at least `3` where each relationship's `since` value is greater than the previous one and above `2000`.

Find paths where a relationship property must be above a value and increase along a path

```
MATCH path = ()-[r:KNOWS]-{3,}()
WHERE allReduce(
  span = {},
  rel IN r | { previous: span.current, current: rel.since },
  (span.previous IS NULL OR span.previous < span.current) AND span.current > 2000
)
LET people = nodes(path)
RETURN [actor IN people | actor.name] AS connectedActors,
       [rel IN r | rel.since] AS sinceYears
ORDER BY sinceYears
```

Result

connectedActors	sinceYears
["Liam Neeson", "Keanu Reeves", "Kathryn Bigelow", "Jessica Chastain"]	[2005, 2010, 2012]

Rows: 1

any()

Details

Syntax	`any(variable IN list WHERE predicate)`		
Description	Returns true if the predicate holds for at least one element in the given `LIST<ANY>`.		
Arguments	Name	Type	Description
	`variable`	`ANY`	A variable that can be used within the `WHERE` clause.
	`list`	`LIST<ANY>`	A predicate must hold for all elements in this list for the function to return `true`.
	`predicate`	`ANY`	A predicate that is tested against all items in the given list.
Returns	`BOOLEAN`		

Considerations

`any()` differs from most Cypher functions because it iterates over a list, evaluating an expression for each element, rather than returning a result from a single evaluation.
`null` is returned if the `list` is `null` or if the `predicate` evaluates to `null` for at least one element and does not evaluate to false for any other element.
`any()` returns `false` if `list` is empty because there are no elements to satisfy the `predicate`.

Example 258. any()

Find paths where at least one relationship property is above a given threshold

```
MATCH p = (n:Person {name: 'Keanu Reeves'})-[:KNOWS]-{3}()
WHERE any(rel IN relationships(p) WHERE rel.since < 2000)
RETURN [person IN nodes(p) | person.name] AS connectedActors,
       [rel IN relationships(p) | rel.since] AS sinceYears
```

Result

connectedActors	sinceYears
`["Keanu Reeves", "Carrie Anne Moss", "Guy Pearce", "Liam Neeson"]`	`[1999, 2008, 2009]`
Rows: 1	

`any()` on an empty `LIST`

```
WITH [] as emptyList
```

```
RETURN any(i IN emptyList WHERE true) as anyTrue, any(i IN emptyList WHERE false) as anyFalse
```

Result

anyTrue	anyFalse
false	false
Rows: 1	

exists()

Details

Syntax	`exists(input)`		
Description	Returns true if a match for the pattern exists in the graph.		
Arguments	Name	Type	Description
	`input`	`ANY`	A pattern to verify the existence of.
Returns	`BOOLEAN`		

Considerations

`null` is returned if `input` is `null`.

> **Note:**
>
> To check if a property is not `null` use the `IS NOT NULL` predicate.

Example 259. exists()

Query

```
MATCH (p:Person)
RETURN p.name AS name,
       exists((p)-[:ACTED_IN]->()) AS has_acted_in_rel
```

This query returns the `name` property of every `Person` node, along with a boolean (`true` or `false`) indicating if those nodes have an `ACTED_IN` relationship in the graph.

Result

name	has_acted_in_rel
"Carrie Anne Moss"	true
"Keanu Reeves"	true

name	has_acted_in_rel
"Liam Neeson"	false
"Guy Pearce"	false
"Kathryn Bigelow"	false
"Jessica Chastain"	false

Rows: 6

Note:

For information about the `EXISTS` subquery, which is more versatile than the `exists()` function, see EXISTS subqueries.

isEmpty()

Details

Syntax	`isEmpty(input)`		
Description	Checks whether a `STRING`, `MAP` or `LIST<ANY>` is empty.		
Arguments	Name	Type	Description
	`input`	`STRING | MAP | LIST<ANY>`	A value to be checked for emptiness.
Returns	`BOOLEAN`		

Example 260. isEmpty(list)

Query

```
MATCH (p:Person)
WHERE NOT isEmpty(p.nationality)
RETURN p.name, p.nationality
```

This query returns every `Person` node in the graph with a set `nationality` property value (i.e., all `Person` nodes except for `Jessica Chastain`):

Result

p.name	p.nationality
"Keanu Reeves"	"Canadian"
"Carrie Anne Moss"	"American"
"Liam Neeson"	"Northern Irish"

p.name	p.nationality
"Guy Pearce"	"Australian"
"Kathryn Bigelow"	"American"
Rows: 5	

Example 261. isEmpty(map)

Query

```
MATCH (n)
WHERE isEmpty(properties(n))
RETURN n
```

Because the example graph contains no empty nodes, nothing is returned:

Result

```
(no changes, no records)
```

Example 262. isEmpty(string)

Query

```
MATCH (p:Person)
WHERE isEmpty(p.address)
RETURN p.name AS name
```

The `name` property of each node that has an empty `STRING` `address` property is returned:

Result

name
"Jessica Chastain"
Rows: 1

Note:

The function `isEmpty()`, like most other Cypher functions, returns `null` if `null` is passed in to the function. That means that a predicate `isEmpty(n.address)` will filter out all nodes where the `address` property is not set. Thus, `isEmpty()` is not suited to test for `null`-values. `IS NULL` or `IS NOT NULL` should be used for that purpose.

none()

Details

Syntax	`none(variable IN list WHERE predicate)`		
Description	Returns true if the predicate holds for no element in the given `LIST<ANY>`.		
Arguments	Name	Type	Description
	`variable`	`ANY`	A variable that can be used within the `WHERE` clause.
	`list`	`LIST<ANY>`	A predicate must hold for all elements in this list for the function to return `true`.
	`predicate`	`ANY`	A predicate that is tested against all items in the given list.
Returns	`BOOLEAN`		

Considerations

`none()` differs from most Cypher functions because it iterates over a list, evaluating an expression for each element, rather than returning a result from a single evaluation.
`null` is returned if the `list` is `null`, or if the `predicate` evaluates to `null` for at least one element and does not evaluate to `true` for any other element.
`none()` returns `true` if `list` is empty because there are no elements to violate the `predicate`.

Example 263. none()

Find paths where no node exceeds a given property value

```
MATCH p = (n:Person {name: 'Keanu Reeves'})-[]-{2}()
WHERE none(x IN nodes(p) WHERE x.age > 60)
RETURN [x IN nodes(p) | x.name] AS connectedActors
```

No nodes in the returned paths have an `age` property with a greater value than `60`:

Result

connectedActors
`["Keanu Reeves", "Carrie Anne Moss", "Guy Pearce"]`
Rows: 1

`none()` on an empty `LIST`

```
WITH [] as emptyList
RETURN none(i IN emptyList WHERE true) as noneTrue, none(i IN emptyList WHERE false) as noneFalse
```

Result

noneTrue	noneFalse
TRUE	TRUE
Rows: 1	

single()

Details

Syntax	`single(variable IN list WHERE predicate)`		
Description	Returns true if the predicate holds for exactly one of the elements in the given `LIST<ANY>`.		
Arguments	Name	Type	Description
	`variable`	`ANY`	A variable that can be used within the `WHERE` clause.
	`list`	`LIST<ANY>`	A predicate must hold for all elements in this list for the function to return `true`.
	`predicate`	`ANY`	A predicate that is tested against all items in the given list.
Returns	`BOOLEAN`		

Considerations

`single()` differs from most Cypher functions because it iterates over a list, evaluating an expression for each element, rather than returning a result from a single evaluation.

`null` is returned if the `list` is `null`, or if the `predicate` evaluates to `null` for at least one element and does not evaluate to `true` for any other element.

`single()` returns `false` if `list` is empty because there is not exactly one element satisfying the `predicate`.

Example 264. single()

Find paths where exactly one node has a given property value

```
MATCH p = (n:Person {name: 'Keanu Reeves'})-[:KNOWS]-+(b)
WHERE single(x IN [b] WHERE x.nationality = 'Northern Irish')
RETURN [person IN nodes(p) | person.name + " (" + person.nationality + ")"] AS northernIrishPaths
ORDER BY length(p)
```

Result

northernIrishPaths
["Keanu Reeves (Canadian)", "Liam Neeson (Northern Irish)"]
["Keanu Reeves (Canadian)", "Carrie Anne Moss (American)", "Guy Pearce (Australian)", "Liam Neeson (Northern Irish)"]
Rows: 2

`single()` on an empty `LIST`

```
WITH [] as emptyList
RETURN single(i IN emptyList WHERE true) as singleTrue, single(i IN emptyList WHERE false) as
singleFalse
```

Result

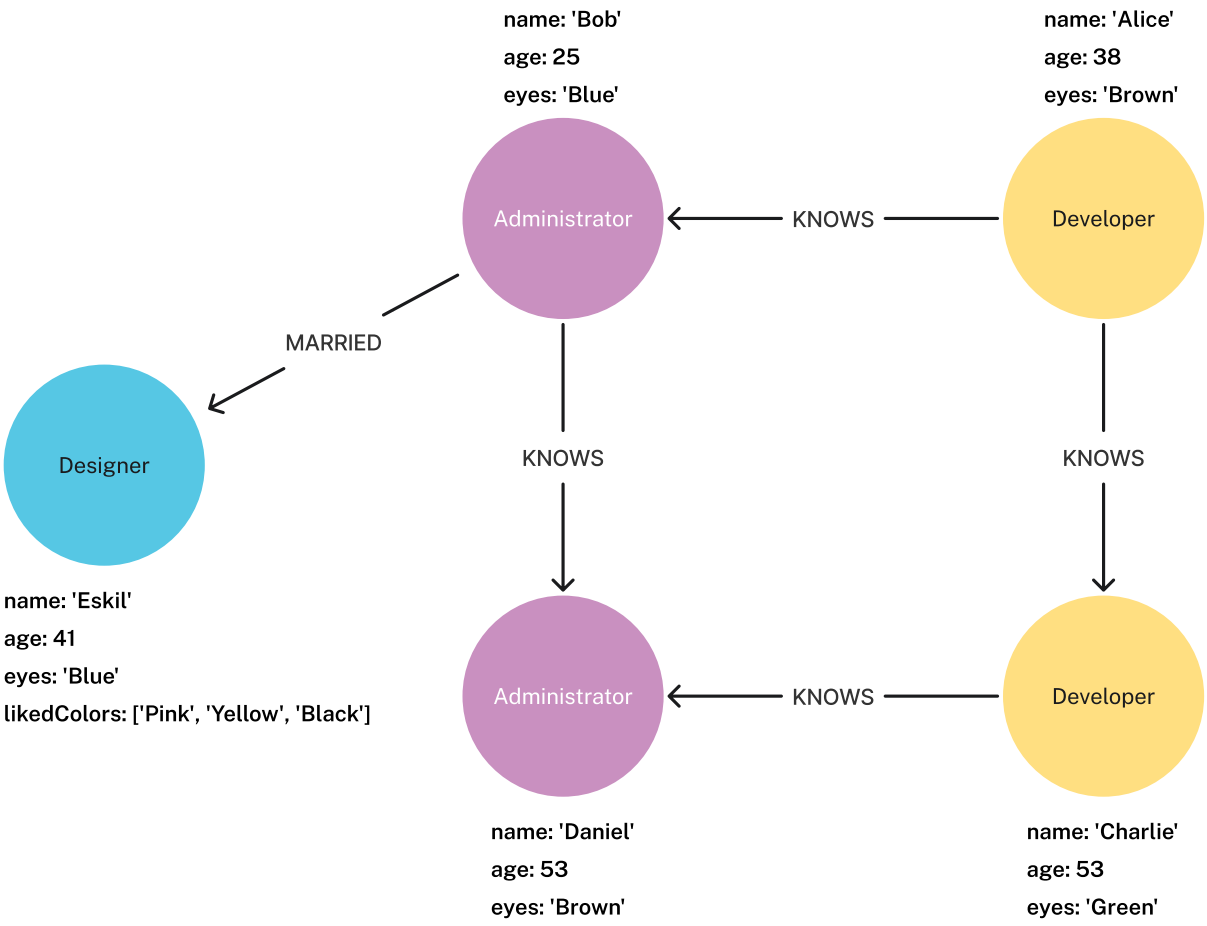
singleTrue	singleFalse
false	false
Rows: 1	

Scalar functions

Scalar functions return a single value.

Example graph

The following graph is used for the examples below:



To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (alice:Developer {name:'Alice', age: 38, eyes: 'Brown'}),
  (bob:Administrator {name: 'Bob', age: 25, eyes: 'Blue'}),
  (charlie:Administrator {name: 'Charlie', age: 53, eyes: 'Green'}),
  (daniel:Adminstrator {name: 'Daniel', age: 54, eyes: 'Brown'}),
  (eskil:Designer {name: 'Eskil', age: 41, eyes: 'blue', likedColors: ['Pink', 'Yellow', 'Black']}),
  (alice)-[:KNOWS]->(bob),
  (alice)-[:KNOWS]->(charlie),
  (bob)-[:KNOWS]->(daniel),
  (charlie)-[:KNOWS]->(daniel),
  (bob)-[:MARRIED]->(eskil)
```

cardinality()

Introduced in Neo4j 2026.07 | Cypher 25 only

Details

Syntax	`cardinality(input)`		
Description	Returns the number of keys in a `MAP`, the number of elements in a `LIST<ANY>`, or the number of nodes and relationships in a `PATH`.		
Arguments	Name	Type	Description
	`input`	`MAP | LIST<ANY> | PATH`	A list, path or map whose cardinality is to be calculated.

Returns	`INTEGER`

Considerations

`cardinality(null)` returns `null`.
For a `MAP`, `cardinality()` returns the same result as `size(keys(map))`.
For a `LIST<ANY>`, `cardinality()` returns the same result as `size()`.
For a `LIST<ANY>`, only top-level elements are counted; nested lists count as a single element.
For a non-empty path, `cardinality(p) = 2 * length(p) + 1`.

To calculate the length of a `PATH` as a relationship count only, use `length()` or `path_length()`.

Example 265. cardinality() applied to lists

Query

```
RETURN cardinality([1, 2, 3]) AS result
```

Result

result
3
Rows: 1

The number of elements in the list is returned.

Example 266. cardinality() applied to maps

Query

```
RETURN cardinality({a: 1, b: 2}) AS result,
       cardinality({a: 1, b: 2}) = size(keys({a: 1, b: 2})) AS sameAsSizeKeys
```

Result

result	sameAsSizeKeys
2	true
Rows: 1	

The number of keys in the map is returned.

Example 267. cardinality() applied to paths

Query

```
MATCH p = (a:Developer)-[:KNOWS]->(b:Administrator)
WHERE a.name = 'Alice' AND b.name = 'Bob'
RETURN cardinality(p) AS cardinality, length(p) AS length
```

The cardinality of the returned path is `3` (2 nodes and 1 relationship), while its length is `1` (for the single relationship traversed).

Result

cardinality	length
3	1
Rows: 1	

char_length()

Details

Syntax	`char_length(input)`		
Description	Returns the number of Unicode characters in a `STRING`.		
Arguments	Name	Type	Description
	`input`	`STRING`	A string value whose length in characters is to be calculated.
Returns	`INTEGER`		

This function is an alias of the `size()` function, and was introduced as part of Cypher's GQL conformance.

Considerations

`char_length(null)` returns `null`.

Example 268. char_length()

Query

```
RETURN char_length('Alice')
```

Result

char_length('Alice')
5

char_length('Alice')
Rows: 1

The number of Unicode characters in the `STRING` is returned.

character_length()

Details

Syntax	`character_length(input)`		
Description	Returns the number of Unicode characters in a `STRING`.		
Arguments	Name	Type	Description
	`input`	`STRING`	A string value whose length in characters is to be calculated.
Returns	`INTEGER`		

This function is an alias of the `size()` function, and was introduced as part of Cypher's GQL conformance.

Considerations

`character_length(null)` returns `null`.

Example 269. character_length()

Query

```
RETURN character_length('Alice')
```

Result

character_length('Alice')
5
Rows: 1

The number of Unicode characters in the `STRING` is returned.

coalesce()

Details

Syntax	`coalesce(input)`

Description	Returns the first non-null value in a list of expressions.		
Arguments	Name	Type	Description
	`input`	`ANY`	If this is the first non-`null` value, it will be returned.
Returns	`ANY`		

Considerations

`null` will be returned if all the arguments are `null`.

Example 270. coalesce()

Query

```
MATCH (a)
WHERE a.name = 'Alice'
RETURN coalesce(a.hairColor, a.eyes)
```

Result

coalesce(a.hairColor, a.eyes)
"Brown"
Rows: 1

elementId()

Details

Syntax	`elementId(input)`		
Description	Returns the element id of a `NODE` or `RELATIONSHIP`.		
Arguments	Name	Type	Description
	`input`	`NODE | RELATIONSHIP`	An element id of a node or a relationship.
Returns	`STRING`		

There are important considerations to bear in mind when using `elementId()`:

1. Every node and relationship is guaranteed an element ID. This ID is unique among both nodes and relationships across all databases in the same DBMS within the scope of a single transaction. However, no guarantees are given regarding the order of the returned ID values or the length of the ID `STRING` values. Outside of the scope of a single transaction, no guarantees are given about the mapping between ID values and elements.

2. Neo4j reuses its internal IDs when nodes and relationships are deleted. Applications relying on internal Neo4j IDs are, as a result, brittle and can be inaccurate. It is therefore recommended to use application-generated IDs.

Considerations

`elementId(null)` returns `null`.
`elementId` on values other than a `NODE`, `RELATIONSHIP`, or `null` will fail the query.

Example 271. elementId() for nodes

Query

```
MATCH (n:Developer)
RETURN elementId(n)
```

The identifier for each `Developer` node is returned.

Result

elementId(n)
"4:d8d172ec-96d8-4364-8f5d-9353d776aeb3:0"
Rows: 1

Example 272. elementId() for relationships

Query

```
MATCH (:Developer)-[r]-()
RETURN elementId(r)
```

The identifier for each relationship connected to a `Developer` node is returned.

Result

elementId(r)
"5:d8d172ec-96d8-4364-8f5d-9353d776aeb3:0"
"5:d8d172ec-96d8-4364-8f5d-9353d776aeb3:1"
Rows: 2

endNode()

Details

Syntax	`endNode(input)`

Description	Returns the end `NODE` of a `RELATIONSHIP`.		
Arguments	Name	Type	Description
	`input`	`RELATIONSHIP`	A relationship.
Returns	`NODE`		

Considerations

`endNode(null)` returns `null`.

Example 273. endNode()

Query

```
MATCH (x:Developer)-[r]-()
RETURN endNode(r)
```

Result

endNode(r)
{name: "Bob", age: 25, eyes: "Blue"}
{name: "Charlie", age: 53, eyes: "Green"}
Rows: 2

head()

Details

Syntax	`head(list)`		
Description	Returns the first element in a `LIST<ANY>`.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list from which the first element will be returned.
Returns	`ANY`		

Considerations

`head(null)` returns `null`.
`head([])` returns `null`.
If the first element in `list` is `null`, `head(list)` will return `null`.

Example 274. head()

Query

```
MATCH (a)
WHERE a.name = 'Eskil'
RETURN a.likedColors, head(a.likedColors)
```

The first element in the list is returned.

Result

a.likedColors+	+head(a.likedColors)
["Pink", "Yellow", "Black"]	"Pink"
Rows: 1	

id()

Deprecated

It is recommended to use `elementId()` instead.

Details

Syntax	`id(input)`		
Description	Returns the id of a `NODE` or `RELATIONSHIP`.		
Arguments	Name	Type	Description
	`input`	`NODE | RELATIONSHIP`	A node or a relationship.
Returns	`INTEGER`		

Considerations

`id(null)` returns `null`.

There are important considerations to bear in mind when using `id()`:

1. The function `id()` returns a node or a relationship identifier, unique by an object type and a database. Therefore, `id()` can return the same value for both nodes and relationships in the same database.
2. Neo4j implements the ID so that every node and relationship in a database has an identifier. The identifier for a node or relationship is guaranteed to be unique among other nodes' and relationships' identifiers in the same database, within the scope of a single transaction.
3. Neo4j reuses its internal IDs when nodes and relationships are deleted. Applications relying on internal Neo4j IDs are, as a result, brittle and can be inaccurate. It is therefore recommended to use application-generated IDs instead.

Note:

On a composite database, the `id()` function should be used with caution. It is recommended to use `elementId()` instead.

When called in database-specific subqueries, the resulting ID value for a node or relationship is local to that database. The local ID for nodes or relationships from different databases may be the same.

When called from the root context of a query, the resulting value is an extended ID for the node or relationship. The extended ID is likely different from the local ID for the same node or relationship.

Example 275. id()

Query

```
MATCH (a)
RETURN id(a)
```

The node identifier for each of the nodes is returned.

Result

id(a)
0
1
2
3
4
Rows: 5

last()

Details

Syntax	`last(list)`		
Description	Returns the last element in a `LIST<ANY>`.		
Arguments	Name	Type	Description
	`list`	`LIST<ANY>`	A list from which the last element will be returned.
Returns	`ANY`		

Considerations:

`last(null)` returns `null`.
`last([])` returns `null`.
If the last element in `list` is `null`, `last(list)` will return `null`.

Example 276. last()

Query

```
MATCH (a)
WHERE a.name = 'Eskil'
RETURN a.likedColors, last(a.likedColors)
```

The last element in the list is returned.

Result

a.liked_colors	last(a.liked_colors)
["Pink", "Yellow", "Black"]	"Black"
Rows: 1	

length()

Details

Syntax	`length(input)`		
Description	Returns the length of a `PATH`.		
Arguments	Name	Type	Description
	`input`	`PATH`	A path whose relationships will be counted.
Returns	`INTEGER`		

Considerations

`length(null)` returns `null`.

To calculate the length of a `LIST` of the number of Unicode characters in a `STRING`, see `size()`

Example 277. length()

Query

```
MATCH p = (a)-->(b)-->(c)
WHERE a.name = 'Alice'
RETURN length(p)
```

The length of the path `p` is returned.

Result

length(p)
2
2
2
Rows: 3

path_length()

Introduced in Neo4j 2026.02 | Cypher 25 only

This function is an alias to the `length()` function when applied to a path, and it was introduced as part of Cypher's GQL conformance. Apart from naming and GQL's path value expression/concatenation rules, the function is GQL compliant.

Details

Syntax	`path_length(input)`		
Description	Returns the length of a `PATH`.		
Arguments	Name	Type	Description
	`input`	`PATH`	A path whose relationships will be counted.
Returns	`INTEGER`		

Considerations

`path_length(null)` returns `null`.

> To calculate the length of a `LIST` or the number of Unicode characters in a `STRING`, see `size()`

Example 278. path_length()

Query

```
MATCH p = (a)-->(b)-->(c)
WHERE a.name = 'Alice'
RETURN path_length(p)
```

The length of the path `p` is returned.

Result

path_length(p)
2
2
2
Rows: 3

nulllf()

Details

Syntax	`nullIf(v1, v2)`		
Description	Returns null if the two given parameters are equivalent, otherwise returns the value of the first parameter.		
Arguments	Name	Type	Description
	`v1`	`ANY`	A first value to be returned if the second value is not equivalent.
	`v2`	`ANY`	A second value against which the first value is compared.
Returns	`ANY`		

This function is the opposite of the coalesce() function, which returns a default value if the given value is null.

Example 279. nulllf()

Query

```
RETURN nullIf(4, 4)
```

The null value is returned as the two parameters are equivalent.

Result

nulllf(4, 4)
null
Rows: 1

Example 280. nulllf()

Query

```
RETURN nullIf("abc", "def")
```

The first parameter, "abc", is returned, as the two parameters are not equivalent.

Result

nulllf("abc", "def")
"abc"
Rows: 1

Example 281. nulllf()

The `nullIf()` function can be used in conjunction with the `coalesce()` function for transforming one data value into another value:

Query

```
MATCH (a)
RETURN a.name AS name, coalesce(nullIf(a.eyes, "Brown"), "Hazel") AS eyeColor
```

Result

name	eyeColor
"Alice"	"Hazel"
"Bob"	"Blue"
"Charlie"	"Green"
"Daniel"	"Hazel"
"Eskil"	"Blue"
Rows: 5	

properties()

Details

Syntax	`properties(input)`		
Description	Returns a `MAP` containing all the properties of a `NODE`, `RELATIONSHIP` or `MAP`.		
Arguments	Name	Type	Description
	`input`	`NODE | RELATIONSHIP | MAP`	An entity to return the properties from.
Returns	`MAP`		

Considerations

`properties(null)` returns `null`.
If `input` is already a `MAP`, it is returned unchanged.

Example 282. properties()

Query

```
CREATE (p:Person {name: 'Stefan', city: 'Berlin'})
RETURN properties(p)
```

Result

properties(p)
{"city": "Berlin", "name": "Stefan"}
Rows: 1

randomUUID()

Details

Syntax	`randomUUID()`
Description	Generates a random UUID.
Returns	`STRING`

A Universally Unique Identified (UUID), also known as a Globally Unique Identifier (GUID), is a 128-bit value with strong guarantees of uniqueness.

Example 283. randomUUID()

Query

```
RETURN randomUUID() AS uuid
```

Result

uuid
"9f4c297d-309a-4743-a196-4525b96135c1"
Rows: 1

A randomly-generated UUID is returned.

size()

Details

Syntax	`size(input)`		
Description	Returns the number of items in a `LIST<ANY>`, the number of Unicode characters in a `STRING` or the dimension of a `VECTOR`.		
Arguments	Name	Type	Description
	`input`	`STRING | LIST<ANY> | VECTOR`	A value whose length is to be calculated.
Returns	`INTEGER`		

To calculate the length of a `PATH`, see `length()`.

Considerations

`size(null)` returns `null`.

Example 284. size() applied to lists

Query

```
RETURN size(['Alice', 'Bob'])
```

Result

size(['Alice', 'Bob'])
2
Rows: 1

The number of elements in the list is returned.

Example 285. size() applied to pattern comprehensions

Query

```
MATCH (a)
WHERE a.name = 'Alice'
RETURN size([p=(a)-->()-->() | p]) AS fof
```

Result

fof
3
Rows: 1

The number of paths matching the pattern expression is returned. (The size of the list of paths).

Example 286. size() applied to strings

Query

```
MATCH (a)
WHERE size(a.name) > 6
RETURN size(a.name)
```

Result

size(a.name)
7
Rows: 1

The number of characters in the `STRING` `'Charlie'` is returned.

Example 287. size() applied to vectors

Query

```
RETURN size(vector([1, 2, 3], 3, INTEGER8)) AS size
```

Result

size
3
Rows: 1

The dimension of the vector is returned.

You can also use the `vector_dimension_count()` function to return the dimension of a `VECTOR` value.

startNode()

Details

Syntax	`startNode(input)`		
Description	Returns the start `NODE` of a `RELATIONSHIP`.		
Arguments	Name	Type	Description
	`input`	`RELATIONSHIP`	A relationship.
Returns	`NODE`		

Considerations

`startNode(null)` returns `null`.

Example 288. startNode()

Query

```
MATCH (x:Developer)-[r]-()
RETURN startNode(r)
```

Result

startNode(r)
{name: "Alice", age: 38, eyes: "Brown"}
{name: "Alice", age: 38, eyes: "Brown"}
Rows: 2

timestamp()

Details

Syntax	`timestamp()`
Description	Returns the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.
Returns	`INTEGER`

`datetime.fromEpochMillis()` is a functional equivalent to `timestamp()`.

Considerations

`timestamp()` will return the same value during one entire query, even for long-running queries.

Example 289. timestamp()

Query

```
RETURN timestamp()
```

The time in milliseconds is returned.

Result

timestamp()
1655201331965
Rows: 1

toBoolean()

Details

Syntax	`toBoolean(input)`		
Description	Converts a `BOOLEAN`, `STRING` or `INTEGER` value to a `BOOLEAN` value. For `INTEGER` values, 0 is defined to be false and any other `INTEGER` is defined to be true.		
Arguments	Name	Type	Description
	`input`	`BOOLEAN | STRING | INTEGER`	A value to be converted into a boolean.
Returns	`BOOLEAN`		

Considerations

`toBoolean(null)` returns `null`.
If `input` is a `BOOLEAN` value, it will be returned unchanged.
If the parsing fails, `null` will be returned.
If `input` is the `INTEGER` value `0`, `false` will be returned. For any other `INTEGER` value `true` will be returned.
This function will return an error if provided with an expression that is not a `STRING`, `INTEGER` or `BOOLEAN` value.

Example 290. toBoolean()

Query

```
RETURN toBoolean('true'), toBoolean('not a boolean'), toBoolean(0)
```

Result

toBoolean('true')	toBoolean('not a boolean')	toBoolean(0)
true	<null>	false
Rows: 1		

toBooleanOrNull()

Details

Syntax	`toBooleanOrNull(input)`		
Description	Converts a value to a `BOOLEAN` value, or null if the value cannot be converted.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be converted into a boolean or null.
Returns	`BOOLEAN`		

Considerations

`toBooleanOrNull(null)` returns `null`.
If `input` is a `BOOLEAN` value, it will be returned unchanged.
If the parsing fails, `null` will be returned.
If `input` is the `INTEGER` value `0`, `false` will be returned. For any other `INTEGER` value `true` will be returned.
If the `input` is not a `STRING`, `INTEGER` or `BOOLEAN` value, `null` will be returned.

Example 291. toBooleanOrNull()

Query

```
RETURN toBooleanOrNull('true'), toBooleanOrNull('not a boolean'), toBooleanOrNull(0),
toBooleanOrNull(1.5)
```

Result

toBooleanOrNull('true')	toBooleanOrNull('not a boolean')	toBooleanOrNull(0)	toBooleanOrNull(1.5)
`true`	`<null>`	`false`	`<null>`
Rows: 1			

toFloat()

Details

Syntax	`toFloat(input)`		
Description	Converts a `STRING`, `INTEGER` or `FLOAT` value to a `FLOAT` value.		
Arguments	Name	Type	Description
	`input`	`STRING | INTEGER | FLOAT`	A value to be converted into a float.
Returns	`FLOAT`		

Considerations

`toFloat(null)` returns `null`.
If `input` is a `FLOAT`, it will be returned unchanged.
If the parsing fails, `null` will be returned.
This function will return an error if provided with an expression that is not an `INTEGER`, `FLOAT` or a `STRING` value.

Example 292. toFloat()

Query

```
RETURN toFloat('11.5'), toFloat('not a number')
```

Result

toFloat('11.5')	toFloat('not a number')
`11.5`	`<null>`
Rows: 1	

toFloatOrNull()

Details

Syntax	`toFloatOrNull(input)`

Description	Converts a value to a `FLOAT` value, or null if the value cannot be converted.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be converted into a float or null.
Returns	`FLOAT`		

Considerations

`toFloatOrNull(null)` returns `null`.
If `input` is a `FLOAT`, it will be returned unchanged.
If the parsing fails, `null` will be returned.
If the `input` is not an `INTEGER`, `FLOAT` or a `STRING` value, `null` will be returned.

Example 293. toFloatOrNull()

Query

```
RETURN toFloatOrNull('11.5'), toFloatOrNull('not a number'), toFloatOrNull(true)
```

Result

toFloatOrNull('11.5')	toFloatOrNull('not a number')	toFloatOrNull(true)
11.5	<null>	<null>
Rows: 1		

toInteger()

Details

Syntax	`toInteger(input)`		
Description	Converts a `BOOLEAN`, `STRING`, `INTEGER` or `FLOAT` value to an `INTEGER` value. For `BOOLEAN` values, true is defined to be 1 and false is defined to be 0.		
Arguments	Name	Type	Description
	`input`	`BOOLEAN | STRING | INTEGER | FLOAT`	A value to be converted into an integer.
Returns	`INTEGER`		

Considerations

`toInteger(null)` returns `null`.

If `input` is an `INTEGER` value, it will be returned unchanged.
If the parsing fails, `null` will be returned.
If `input` is the boolean value `false`, `0` will be returned.
If `input` is the boolean value `true`, `1` will be returned.
This function will return an error if provided with an expression that is not a `BOOLEAN`, `FLOAT`, `INTEGER` or a `STRING` value.

Example 294. toInteger()

Query

```
RETURN toInteger('42'), toInteger('not a number'), toInteger(true)
```

Result

toInteger('42')	toInteger('not a number')	toInteger(true)
42	<null>	1
Rows: 1		

toIntegerOrNull()

Details

Syntax	`toIntegerOrNull(input)`		
Description	Converts a value to an `INTEGER` value, or null if the value cannot be converted.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be converted into an integer or null.
Returns	`INTEGER`		

Considerations

`toIntegerOrNull(null)` returns `null`.
If `input` is an `INTEGER` value, it will be returned unchanged.
If the parsing fails, `null` will be returned.
If `input` is the `BOOLEAN` value `false`, `0` will be returned.
If `input` is the `BOOLEAN` value `true`, `1` will be returned.
If the `input` is not a `BOOLEAN`, `FLOAT`, `INTEGER` or a `STRING` value, `null` will be returned.

Example 295. toIntegerOrNull()

Query

```
RETURN toIntegerOrNull('42'), toIntegerOrNull('not a number'), toIntegerOrNull(true),
toIntegerOrNull(['A', 'B', 'C'])
```

Result

toIntegerOrNull('42')	toIntegerOrNull('not a number')	toIntegerOrNull(true)	toIntegerOrNull(['A', 'B', 'C'])
42	<null>	1	<null>
Rows: 1			

type()

Details

Syntax	`type(input)`		
Description	Returns a `STRING` representation of the `RELATIONSHIP` type.		
Arguments	Name	Type	Description
	`input`	`RELATIONSHIP`	A relationship.
Returns	`STRING`		

Considerations

`type(null)` returns `null`.

Example 296. type()

Query

```
MATCH (n)-[r]->()
WHERE n.name = 'Alice'
RETURN type(r)
```

The relationship type of `r` is returned.

Result

type(r)
"KNOWS"
"KNOWS"
Rows: 2

valueType()

Details

Syntax	`valueType(input)`		
Description	Returns a `STRING` representation of the most precise value type that the given expression evaluates to.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to return the type of.
Returns	`STRING`		

The output is deterministic and makes use of Type Normalization.

Considerations:

Future releases of Cypher may include updates to the current type system. This can include the introduction of new types and subtypes of already supported types. If a new type is introduced, it will be returned by the `valueType()` function as soon as it is released. However, if a more precise subtype of a previously supported type is introduced, it would be considered a breaking change.

For example, the function currently returns `"FLOAT"`, but if a more specific `FLOAT` type was added, e.g. `FLOAT32`, this would be considered more specific and not be returned until the next major release of Neo4j. As a result, `"FLOAT"` would continue to be returned for any `FLOAT32` values until the next major release.

The below list contains all currently supported types displayed by the `valueType()` function:

- Predefined types
  - `NOTHING`
  - `null`
  - `BOOLEAN`
  - `STRING`
  - `INTEGER`
  - `FLOAT`
  - `DATE`
  - `LOCAL TIME`
  - `ZONED TIME`
  - `LOCAL DATETIME`
  - `ZONED DATETIME`
  - `DURATION`
  - `POINT`
  - `NODE`

- `RELATIONSHIP`

- Constructed types
  - `MAP`
  - `LIST<INNER_TYPE>` (ordered by the inner type)
  - `PATH`
- Dynamic union types
  - `INNER_TYPE_1 \| INNER_TYPE_2…` (ordered by specific rules for closed dynamic union type)
  - `ANY`

This should be taken into account when relying on the output of the `valueType()` function.

See the type predicate expression for an alternative way of testing type values.

Example 297. valueType()

Query

```
UNWIND ["abc", 1, 2.0, true, [date()]] AS value
RETURN valueType(value) AS result
```

Result

result
"STRING NOT NULL"
"INTEGER NOT NULL"
"FLOAT NOT NULL"
"BOOLEAN NOT NULL"
"LIST<DATE NOT NULL> NOT NULL"
Rows: 5

Security functions

Cypher 25 only

Security functions are used to evaluate user attributes in the condition of an attribute-based access control (ABAC) auth rule.

Important:

The functions on this page cannot be called directly in queries. They can only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command. For more information about defining auth rules and more extensive examples, see Operations Manual → Attribute-based

access control.

abac.native.user_tags()

Introduced in Neo4j 2026.06 | Cypher 25 only

Details

Syntax	`abac.native.user_tags()`
Description	Returns the list of tags for the native user.
Returns	`LIST<STRING>`

Considerations

`abac.native.user_tags()` cannot be called directly in queries. Prior to Neo4j 2026.10 it could only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command.
As of Neo4j 2026.10, `abac.native.user_tags()` can also be used in an `IN` predicate of the `WHERE` clause of a property-based privilege `GRANT` command, in addition to its existing use in auth rule conditions. This allows a single privilege to grant different users different data access based on their tags, as long as the native provider is configured for ABAC. For more information, see the Operations Manual → Grant a property-based privilege using native user tags.
The function returns an empty list if the user has no tags.
When evaluating the function in `dbms.security.require_local_user` is set to `true`, in which case the absence of a native user will prevent the user from authenticating).

Example 298. abac.native.user_tags()

The following auth rule grants its role when the native user has the `admin` tag.

Grant a role using native user tags

```
CREATE AUTH RULE nativeAdminRule
SET CONDITION 'admin' IN abac.native.user_tags()
```

Grant a property-based privilege using native user tags

```
GRANT READ {*} ON GRAPH * FOR (n:Document)
WHERE n.classification IN abac.native.user_tags() TO regularUsers
```

abac.oidc.user_attribute()

Introduced in Neo4j 2026.03 | Cypher 25 only

Details

Syntax	`abac.oidc.user_attribute(claimKey)`		
Description	Returns the value of the specified user attribute from the OIDC claims. Returns `null` if the attribute is not present.		
Arguments	Name	Type	Description
	`claimKey`	`STRING`	The key of the OIDC claim whose value is returned.
Returns	`ANY`		

Considerations

`abac.oidc.user_attribute()` cannot be called directly in queries. It can only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command.
The returned value can be an arbitrarily nested map object that represents the value of the user's `claimKey` claim.
`null` is returned if the specified attribute is not present in the user's OIDC claims.

Example 299. abac.oidc.user_attribute()

The following auth rule grants its role when the `department` claim in the user's OIDC token is equal to `sales`.

Query

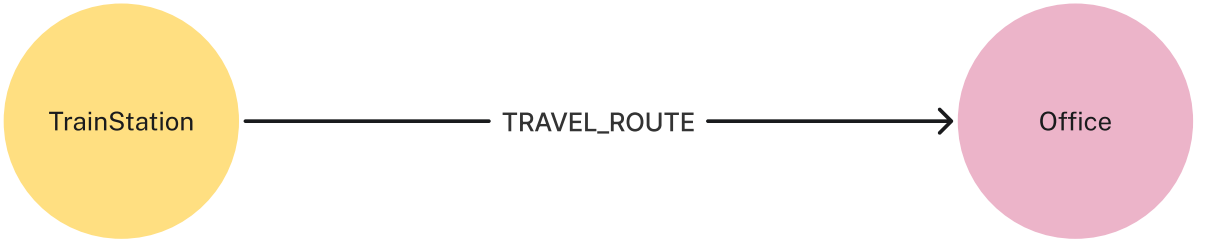
```
CREATE AUTH RULE salesRule
SET CONDITION abac.oidc.user_attribute('department') = 'sales'
```

Spatial functions

Spatial functions are used to specify 2D or 3D `POINT` values in a Coordinate Reference System (CRS) and to calculate the geodesic distance between two `POINT` values.

Example graph

The following graph is used for some of the examples below.



latitude: 55.672874
ongitude: 12.564590
city: 'Copenhagen'

latitude: 55.611784
longitude: 12.994341
city: 'Malmö'

To recreate the graph, run the following query against an empty Neo4j database:

```
CREATE
  (copenhagen:TrainStation {latitude: 55.672874, longitude: 12.564590, city: 'Copenhagen'}),
  (malmo:Office {latitude: 55.611784, longitude: 12.994341, city: 'Malmö'}),
  (copenhagen)-[:TRAVEL_ROUTE]->(malmo)
```

point()

Details

Syntax	`point(input)`		
Description	Returns a 2D or 3D point object, given two or respectively three coordinate values in the Cartesian coordinate system or WGS 84 geographic coordinate system.		
Arguments	Name	Type	Description
	`input`	`MAP`	Cartesian 2D: `{ x :: FLOAT, y :: FLOAT, crs = "cartesian" :: STRING, srid = 7203 :: INTEGER }` Cartesian 3D: `{ x :: FLOAT, y :: FLOAT, z :: FLOAT, crs = "cartesian-3D" :: STRING, srid = 9157 :: INTEGER }` WGS 84 2D: `{ longitude | x :: FLOAT latitude | y :: FLOAT crs = "WGS-84-2D" :: STRING srid = 4326 :: INTEGER }` WGS 84 3D: `{ longitude | x :: FLOAT, latitude | y :: FLOAT, height | z :: FLOAT, crs = "WGS-84-3D" :: STRING, srid = 4979 :: INTEGER }`
Returns	`POINT`		

Considerations

If any argument provided to `point()` is `null`, `null` will be returned.
If the coordinates are specified using `latitude` and `longitude`, the `crs` or `srid` fields are optional and inferred to be `'WGS-84'` (`srid:4326`) for 2D points or `'WGS-84-3D'` (`srid:4979`) for 3D points.
If the coordinates are specified using `x` and `y`, then either the `crs` or `srid` field is required if a geographic CRS is desired.
If the `height/z` key and value is not provided, a 2D `POINT` will be returned in either the *WGS 84* or *Cartesian* CRS, depending on the coordinate system used.
The `crs` or `srid` fields are optional and default to the *Cartesian* CRS (which means `srid:7203`) for 2D points or the *3D Cartesian* CRS (which means `srid:9157`) for 3D points.

Example 300. point() - WGS 84 2D

Query

```
RETURN point({longitude: 56.7, latitude: 12.78}) AS point
```

A 2D `POINT` with a `longitude` of `56.7` and a `latitude` of `12.78` in the *WGS 84* CRS is returned.

Result

point
`point({srid:4326, x:56.7, y:12.78})`
Rows: 1

Example 301. point() - WGS 84 2D

Query

```
RETURN point({x: 2.3, y: 4.5, crs: 'WGS-84'}) AS point
```

`x` and `y` coordinates may be used in the *WGS 84* CRS instead of `longitude` and `latitude`, respectively, providing `crs` is set to `'WGS-84'`, or `srid` is set to `4326`.

Result

point
`point({srid:4326, x:2.3, y:4.5})`
Rows: 1

Example 302. point() - WGS 84 2D

Query

```
MATCH (p:Office)
RETURN point({longitude: p.longitude, latitude: p.latitude}) AS officePoint
```

A 2D `POINT` representing the coordinates of the city of Malmo in the *WGS 84* CRS is returned.

Result

officePoint
point({srid:4326, x:12.994341, y:55.611784})
Rows: 1

Example 303. point() - WGS 84 3D

Query

```
RETURN point({longitude: 56.7, latitude: 12.78, height: 8}) AS point
```

A 3D `POINT` with a `longitude` of `56.7`, a `latitude` of `12.78` and a height of `8` meters in the *WGS 84* CRS is returned.

Result

point
point({srid:4979, x:56.7, y:12.78, z:8.0})
Rows: 1

Example 304. point() - Cartesian 2D

Query

```
RETURN point({x: 2.3, y: 4.5}) AS point
```

A 2D `POINT` with an `x` coordinate of `2.3` and a `y` coordinate of `4.5` in the *Cartesian* CRS is returned.

Result

point
point({srid:7203, x:2.3, y:4.5})
Rows: 1

Example 305. point() - Cartesian 3D

Query

```
RETURN point({x: 2.3, y: 4.5, z: 2}) AS point
```

A 3D `POINT` with an `x` coordinate of `2.3`, a `y` coordinate of `4.5` and a `z` coordinate of `2` in the Cartesian CRS is returned.

Result

point
point({srid:9157, x:2.3, y:4.5, z:2.0})
Rows: 1

Example 306. point() - null

Query

```
RETURN point(null) AS p
```

If `null` is provided as the argument, `null` is returned.

Result

p
<null>
Rows: 1

point.distance()

Details

Syntax	`point.distance(from, to)`		
Description	Returns a `FLOAT` representing the distance between any two points in the same CRS. If the points are in the WGS 84 CRS, the function returns the geodesic distance (i.e., the shortest path along the curved surface of the Earth). If the points are in a Cartesian CRS, the function returns the Euclidean distance (i.e., the shortest straight-line distance in a flat, planar space).		
Arguments	Name	Type	Description
	`from`	`POINT`	A start point.
	`to`	`POINT`	An end point in the same CRS as the start point.

Returns	`FLOAT`

- If the `POINT` values are in the *Cartesian* CRS (2D or 3D), then the units of the returned distance will be the same as the units of the points, calculated using Pythagoras' theorem.
- If the `POINT` values are in the *WGS-84* CRS (2D), then the units of the returned distance will be meters, based on the haversine formula over a spherical Earth approximation.
- If the `POINT` values are in the *WGS-84* CRS (3D), then the units of the returned distance will be meters.
  - The distance is calculated in two steps.
    - First, a haversine formula over a spherical Earth is used, at the average height of the two points.
    - To account for the difference in height, Pythagoras' theorem is used, combining the previously calculated spherical distance with the height difference.
  - This formula works well for points close to the earth's surface; for instance, it is well-suited for calculating the distance of an airplane flight. It is less suitable for greater heights, however, such as when calculating the distance between two satellites.

Considerations

`point.distance(null, null)` return `null`.
`point.distance(null, to)` return `null`.
`point.distance(from, null)` return `null`.
Attempting to use points with different Coordinate Reference Systems (such as WGS 84 2D and WGS 84 3D) will return `null`.

Example 307. point.distance()

Query

```
WITH
  point({x: 2.3, y: 4.5, crs: 'cartesian'}) AS p1,
  point({x: 1.1, y: 5.4, crs: 'cartesian'}) AS p2
RETURN point.distance(p1,p2) AS dist
```

The distance between two 2D points in the *Cartesian* CRS is returned.

Result

dist
1.5
Rows: 1

Example 308. point.distance()

Query

```
WITH
  point({longitude: 12.78, latitude: 56.7, height: 100}) AS p1,
  point({latitude: 56.71, longitude: 12.79, height: 100}) AS p2
RETURN point.distance(p1, p2) AS dist
```

The distance between two 3D points in the WGS 84 CRS is returned.

Result

dist
1269.9148706779097
Rows: 1

Example 309. point.distance()

Query

```
MATCH (t:TrainStation)-[:TRAVEL_ROUTE]->(o:Office)
WITH
  point({longitude: t.longitude, latitude: t.latitude}) AS trainPoint,
  point({longitude: o.longitude, latitude: o.latitude}) AS officePoint
RETURN round(point.distance(trainPoint, officePoint)) AS travelDistance
```

The distance between the train station in Copenhagen and the Neo4j office in Malmo is returned.

Result

travelDistance
27842.0
Rows: 1

Example 310. point.distance()

Query

```
RETURN point.distance(null, point({longitude: 56.7, latitude: 12.78})) AS d
```

If `null` is provided as one or both of the arguments, `null` is returned.

Result

d
null
Rows: 1

point.withinBBox()

Details

Syntax	`point.withinBBox(point, lowerLeft, upperRight)`		
Description	Returns true if the provided point is within the bounding box defined by the two provided points.		
Arguments	Name	Type	Description
	`point`	`POINT`	A point to be confirmed in the bounding box.
	`lowerLeft`	`POINT`	The lower left side point of the bounding box.
	`upperRight`	`POINT`	The upper right side point of the bounding box.
Returns	`BOOLEAN`		

Considerations

`point.withinBBox(point, lowerLeft, upperRight)` will return `null` if any of the arguments evaluate to `null`.
Attempting to use `POINT` values with different Coordinate Reference Systems (such as WGS 84 2D and WGS 84 3D) will return `null`.
`point.withinBBox` will handle crossing the 180th meridian in geographic coordinates.
Switching the longitude of the `lowerLeft` and `upperRight` in geographic coordinates will switch the direction of the resulting bounding box.
Switching the latitude of the `lowerLeft` and `upperRight` in geographic coordinates so that the former is north of the latter will result in an empty range.

Example 311. point.withinBBox()

Query

```
WITH
  point({x: 0, y: 0, crs: 'cartesian'}) AS lowerLeft,
  point({x: 10, y: 10, crs: 'cartesian'}) AS upperRight
RETURN point.withinBBox(point({x: 5, y: 5, crs: 'cartesian'}), lowerLeft, upperRight) AS result
```

Checking if a point in *Cartesian* CRS is contained in the bounding box.

Result

result
true

result
Rows: 1

Example 312. point.withinBBox()

Query

```
WITH
  point({longitude: 12.53, latitude: 55.66}) AS lowerLeft,
  point({longitude: 12.614, latitude: 55.70}) AS upperRight
MATCH (t:TrainStation)
WHERE point.withinBBox(point({longitude: t.longitude, latitude: t.latitude}), lowerLeft, upperRight)
RETURN count(t)
```

Finds all train stations contained in a bounding box around Copenhagen.

Result

count(t)
1
Rows: 1

Example 313. point.withinBBox()

Query

```
WITH
  point({longitude: 179, latitude: 55.66}) AS lowerLeft,
  point({longitude: -179, latitude: 55.70}) AS upperRight
RETURN point.withinBBox(point({longitude: 180, latitude: 55.66}), lowerLeft, upperRight) AS result
```

A bounding box that crosses the 180th meridian.

Result

result
true
Rows: 1

Example 314. point.withinBBox()

Query

```
RETURN
  point.withinBBox(
    null,
    point({longitude: 56.7, latitude: 12.78}),
    point({longitude: 57.0, latitude: 13.0})
  ) AS in
```

If `null` is provided as any of the arguments, `null` is returned.

Result

in
`null`
Rows: 1

String functions

String functions operate on string expressions only, and will return an error if used on any other values. The exception to this rule is `toString()`, which also accepts numbers, booleans and temporal values (i.e. `DATE`, `ZONED TIME` ` `LOCAL TIME`, `ZONED DATETIME`, `LOCAL DATETIME` or `DURATION` values).

Functions taking a `STRING` as input all operate on *Unicode characters* rather than on a standard `char[]`. For example, the `size()` function applied to any *Unicode character* will return `1`, even if the character does not fit in the 16 bits of one `char`.

Note:

When `toString()` is applied to a temporal value, it returns a `STRING` representation suitable for parsing by the corresponding temporal functions. This `STRING` will therefore be formatted according to the ISO 8601 format.

See also String concatenation operators.

btrim()

Details

Syntax	`btrim(input[, trimCharacterString])`		
Description	Returns the given `STRING` with leading and trailing whitespace removed, optionally specifying a `trimCharacterString` to remove.		
Arguments	**Name**	**Type**	**Description**
	`input`	`STRING`	A value from which the leading and trailing trim character will be removed.
	`trimCharacterString`	`STRING`	A character to be removed from the start and end of the given string.
Returns	`STRING`		

Considerations

`btrim(null)` returns `null`.
`btrim(null, null)` returns `null`.
`btrim("hello", null)` returns `null`.
`btrim(null, ' ')` returns `null`.
If `trimCharacterString` is not specified then all leading and trailing whitespace will be removed.

Example 315. btrim()

Query

```
RETURN btrim('   hello    '), btrim('xxyyhelloxyxy', 'xy')
```

Result

btrim(' hello')	btrim('xxyyhelloxyxy', 'xy')
"hello"	"hello"
Rows: 1	

left()

Details

Syntax	`left(original, length)`		
Description	Returns a `STRING` containing the specified number (`INTEGER`) of leftmost characters in the given `STRING`.		
Arguments	Name	Type	Description
	`original`	`STRING`	A string value whose rightmost characters will be trimmed.
	`length`	`INTEGER`	An integer between `0` and `2_147_483_647` (2^31-1) that specifies the length of the leftmost characters to be returned.
Returns	`STRING`		

Considerations

`left(null, length)` return `null`.

`left(null, null)` return `null`.
`left(original, null)` will raise an error.
If `length` is a negative `INTEGER` or larger than `2_147_483_647` (2^31-1), an error is raised.
If `length` exceeds the size of `original`, `original` is returned.

Example 316. left()

Query

```
RETURN left('hello', 3)
```

Result

left('hello', 3)
"hel"
Rows: 1

lower()

Details

Syntax	`lower(input)`		
Description	Returns the given `STRING` in lowercase.		
Arguments	Name	Type	Description
	`input`	`STRING`	A string to be converted into lowercase.
Returns	`STRING`		

This function is an alias to the `toLower()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

`lower(null)` returns `null`.

Example 317. lower()

Query

```
RETURN lower('HELLO')
```

Result

lower('HELLO')
"hello"
Rows: 1

ltrim()

Details

Syntax	`ltrim(input[, trimCharacterString])`		
Description	Returns the given `STRING` with leading whitespace removed, optionally specifying a `trimCharacterString` to remove.		
Arguments	Name	Type	Description
	`input`	`STRING`	A value from which the leading trim character will be removed.
	`trimCharacterString`	`STRING`	A character to be removed from the start of the given string.
Returns	`STRING`		

Considerations

`ltrim(null)` returns `null`.
`ltrim(null, null)` returns `null`.
`ltrim("hello", null)` returns `null`.
`ltrim(null, ' ')` returns `null`.
If `trimCharacterString` is not specified all leading whitespace will be removed.

Example 318. ltrim()

Query

```
RETURN ltrim('   hello'), ltrim('xxyyhelloxyxy', 'xy')
```

Result

ltrim(' hello')	ltrim('xxyyhelloxyxy', 'xy')
"hello"	"helloxyxy"
Rows: 1	

normalize()

Details

Syntax	`normalize(input[, normalForm])`		
Description	Normalize a `STRING`, optionally specifying a normalization form.		
Arguments	Name	Type	Description
	`input`	`STRING`	A value to be normalized.
	`normalForm`	`[NFC, NFD, NFKC, NFKD]`	A keyword specifying any of the normal forms; NFC, NFD, NFKC or NFKD.
Returns	`STRING`		

> **Note:**
>
> Unicode normalization is a process that transforms different representations of the same string into a standardized form. For more information, see the documentation for Unicode normalization forms.

The `normalize()` function is useful for converting `STRING` values into comparable forms. When comparing two `STRING` values, it is their Unicode codepoints that are compared. In Unicode, a codepoint for a character that looks the same may be represented by two, or more, different codepoints. For example, the character `<` can be represented as `\uFE64` (﹤) or `\u003C` (<). To the human eye, the characters may appear identical. However, if compared, Cypher will return false as `\uFE64` does not equal `\u003C`. Using the `normalize()` function, it is possible to normalize the codepoint `\uFE64` to `\u003C`, creating a single codepoint representation, allowing them to be successfully compared.

Considerations

`normalize(null)` returns `null`.

Example 319. normalize()

Query

```
RETURN normalize('\u212B') = '\u00C5' AS result
```

Result

result
true
Rows: 1

To check if a `STRING` is normalized, use the `IS NORMALIZED` operator.

normalize() with specified normal form

There are two main types of normalization forms:

- **Canonical equivalence**: The `NFC` (default) and `NFD` are forms of canonical equivalence. This means that codepoints that represent the same abstract character will be normalized to the same codepoint (and have the same appearance and behavior). The `NFC` form will always give the **composed** canonical form (in which the combined codes are replaced with a single representation, if possible). The`NFD` form gives the **decomposed** form (the opposite of the composed form, which converts the combined codepoints into a split form if possible).
- **Compatability normalization**: `NFKC` and `NFKD` are forms of compatibility normalization. All canonically equivalent sequences are compatible, but not all compatible sequences are canonical. This means that a character normalized in `NFC` or `NFD` should also be normalized in `NFKC` and `NFKD`. Other characters with only slight differences in appearance should be compatibly equivalent.

For example, the Greek Upsilon with Acute and Hook Symbol `ϓ` can be represented by the Unicode codepoint: `\u03D3`.

- Normalized in `NFC`: `\u03D3` Greek Upsilon with Acute and Hook Symbol (ϓ)
- Normalized in `NFD`: `\u03D2\u0301` Greek Upsilon with Hook Symbol + Combining Acute Accent (ϓ)
- Normalized in `NFKC`: `\u038E` Greek Capital Letter Upsilon with Tonos (Ύ)
- Normalized in `NFKD`: `\u03A5\u0301` Greek Capital Letter Upsilon + Combining Acute Accent (Υ)

In the compatibility normalization forms (`NFKC` and `NFKD`) the character is visibly different as it no longer contains the hook symbol.

Example 320. normalize() with specified normalization form

Query

```
RETURN normalize('\uFE64', NFKC) = '\u003C' AS result
```

Result

result
true
Rows: 1

replace()

Details

Syntax	`replace(original, search, replace [, limit])`

Description	Returns a `STRING` in which all occurrences of a specified search `STRING` in the given `STRING` have been replaced by another (specified) replacement `STRING`.		
Arguments	Name	Type	Description
	`original`	`STRING`	The string to be modified.
	`search`	`STRING`	The value to replace in the original string.
	`replace`	`STRING`	The value to be inserted in the original string.
	`limit`	`STRING`	The maximum amount of times the search value should be replaced in the string, starting from the left. Introduced in Neo4j 2025.06
Returns	`STRING`		

Considerations

If any argument is `null`, `null` will be returned.
If `search` is not found in `original`, `original` will be returned.
If `limit` is not defined, all occurrences of the search will be replaced.

Example 321. replace()

Query

```
RETURN replace("hello", "l", "w")
```

Result

replace("hello", "l", "w")
"hewwo"
Rows: 1

Query with limit

```
RETURN replace("hello", "l", "w", 1)
```

Result

replace("hello", "l", "w", 1)
"hewlo"
Rows: 1

reverse()

Details

Syntax	`reverse(input)`		
Description	Returns a `STRING` or `LIST<ANY>` in which the order of all characters or elements in the given `STRING` or `LIST<ANY>` have been reversed.		
Arguments	Name	Type	Description
	`input`	`STRING | LIST<ANY>`	The string or list to be reversed.
Returns	`STRING | LIST<ANY>`		

Considerations

`reverse(null)` returns `null`.
See also List functions → `reverse()`.

Example 322. reverse

Query

```
RETURN reverse('palindrome')
```

Result

reverse('palindrome')
"emordnilap"
Rows: 1

right()

Details

Syntax	`right(original, length)`
Description	Returns a `STRING` containing the specified number of rightmost characters in the given `STRING`.

Arguments	Name	Type	Description
	`original`	`STRING`	A string value whose leftmost characters will be trimmed.
	`length`	`INTEGER`	An integer between `0` and `2_147_483_647` (2^31-1) that specifies the length of the rightmost characters to be returned.
Returns	`STRING`		

Considerations

`right(null, length)` return `null`.
`right(null, null)` return `null`.
`right(original, null)` will raise an error.
If `length` is a negative `INTEGER` or larger than `2_147_483_647` (2^31-1), an error is raised.
If `length` exceeds the size of `original`, `original` is returned.

Example 323. right()

Query

```
RETURN right('hello', 3)
```

Result

right('hello', 3)
"llo"
Rows: 1

rtrim()

Details

Syntax	`rtrim(input[, trimCharacterString])`
Description	Returns the given `STRING` with trailing whitespace characters to remove, optionally specifying a `trimCharacterString` of characters to remove.

Arguments	Name	Type	Description
	`input`	`STRING`	A value from which the leading and trailing trim character will be removed.
	`trimCharacterString`	`STRING`	A character to be removed from the start and end of the given string.
Returns	`STRING`		

Considerations

`rtrim(null)` returns `null`.
`rtrim(null, null)` returns `null`.
`rtrim("hello", null)` returns `null`.
`rtrim(null, ' ')` returns `null`.
If `trimCharacterString` is not specified all leading whitespace will be removed.

Example 324. rtrim()

Query

```
RETURN rtrim('hello   '), rtrim('xxyyhelloxyxy', 'xy')
```

Result

rtrim('hello ')	rtrim('xxyyhelloxyxy', 'xy')
"hello"	"xxyyhello"
Rows: 1	

split()

Details

Syntax	`split(original, splitDelimiters)`
Description	Returns a `LIST<STRING>` resulting from the splitting of the given `STRING` around matches of the given delimiter(s).

Arguments	Name	Type	Description
	`original`	`STRING`	The string to be split.
	`splitDelimiters`	`STRING | LIST<STRING>`	The string with which to split the original string.
Returns	`LIST<STRING>`		

Considerations

`split(null, splitDelimiter)` return `null`.
`split(original, null)` return `null`

Example 325. split()

Query

```
RETURN split('one,two', ',')
```

Result

split('one,two', ',')
`["one","two"]`
Rows: 1

string.indexOf()

Introduced in Neo4j 2026.05 | Cypher 25 only

Details

Syntax	`string.indexOf(input, value)`		
Description	Returns the index of the first occurrence of the given `value` within the `input` `STRING`, or -1 if no match is found.		
Arguments	Name	Type	Description
	`input`	`STRING`	The string to be searched.
	`value`	`STRING`	A value to search for.
Returns	`INTEGER`		

Considerations

`string.indexOf(null, value)` returns `null`.

`string.indexOf(input, null)` returns `null`.
If `value` is not present in `input`, `-1` is returned.

Example 326. string.indexOf()

Query

```
RETURN string.indexOf('hello', 'l')
```

Result

string.indexOf('hello', 'l')
2
Rows: 1

string.join()

Introduced in Neo4j 2026.05 | Cypher 25 only

Details

Syntax	`string.join(input, delimiter)`		
Description	Concatenates the elements in `input`, inserting `delimiter` between each element, and returns the resulting `STRING`.		
Arguments	Name	Type	Description
	`input`	`LIST<STRING>`	A list of strings to be joined.
	`delimiter`	`STRING`	The joining delimiter.
Returns	`STRING`		

Considerations

`string.join(null, delimiter)` returns `null`.
`string.join(input, null)` returns `null`.

Example 327. string.join()

Query

```
RETURN string.join(['one', 'two'], ', ')
```

Result

string.join(['one', 'two'], ', ')
"one, two"
Rows: 1

string.regexReplace()

Introduced in Neo4j 2026.05 | Cypher 25 only

Details

Syntax	`string.regexReplace(original, regex, replacement)`		
Description	Returns a `STRING` where all matches of `regex` in the `original` `STRING` are replaced with the `replacement` `STRING`.		
Arguments	Name	Type	Description
	`original`	`STRING`	The string to be modified.
	`regex`	`STRING`	The regex pattern to replace in the original `STRING`.
	`replacement`	`STRING`	The value to be inserted in the original string.
Returns	`STRING`		

For more information on how the regular expression pattern works, see Regular Expressions.

Considerations

If any argument is `null`, `null` will be returned.

Example 328. string.regexReplace()

Query

```
RETURN string.regexReplace("hello", "l.", "w")
```

Result

string.regexReplace("hello", "l.", "w")
"hewo"
Rows: 1

substring()

Details

Syntax	`substring(original, start, length)`		
Description	Returns a substring of a given `length` from the given `STRING`, beginning with a 0-based index start.		
Arguments	Name	Type	Description
	`original`	`STRING`	The string to be shortened.
	`start`	`INTEGER`	An integer between `0` and `2_147_483_647` (2^31-1) that specifies the start position of the new string.
	`length`	`INTEGER`	An integer between `0` and `2_147_483_647` (2^31-1) that specifies the length of the new string.
Returns	`STRING`		

Considerations

`start` uses a zero-based index.
If `length` is omitted, the function returns the substring starting at the position given by `start` and extending to the end of `original`.
If `original` is `null`, `null` is returned.
If either `start` or `length` is `null`, a negative `INTEGER` or larger than `2_147_483_647` (2^31-1), an error is raised.
If `start` is `0`, the substring will start at the beginning of `original`.
If `length` is `0`, the empty `STRING` will be returned.

Example 329. substring()

Query

```
RETURN substring('hello', 1, 3), substring('hello', 2)
```

Result

substring('hello', 1, 3)	substring('hello', 2)
"ell"	"llo"
Rows: 1	

toLower()

Details

Syntax	`toLower(input)`		
Description	Returns the given `STRING` in lowercase.		
Arguments	Name	Type	Description
	`input`	`STRING`	A string to be converted into lowercase.
Returns	`STRING`		

Considerations

`toLower(null)` returns `null`.

Example 330. toLower()

Query

```
RETURN toLower('HELLO')
```

Result

toLower('HELLO')
"hello"
Rows: 1

toString()

Details

Syntax	`toString(input)`		
Description	Converts a value to a `STRING`.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be converted into a string.
Returns	`STRING`		

Considerations

`toString(null)` returns `null`.

If `input` is a `STRING`, it will be returned unchanged.
The following stringification rules apply: • A `LIST` is all its types recursively stringified, surrounded by square brackets `[]` and comma-separated, `null` values will be stringified inside the list. For example: `['a', 1, true, null]` becomes `"[a, 1, true, null]"`. An empty `LIST` becomes `"[]"`. • A `MAP` has its keys sorted alphabetically, and all values stringified, including `null` values. For example: `{a: 'hi', b: true, d: 1, c: 1.5, e: null}` becomes `"{a: hi, b: true, c: 1.5, d: 1, e: null}"`. An empty `MAP` becomes `"{}"`. • A `NODE` is represented by parentheses around its labels sorted alphabetically and colon separated. For example: `(:LabelB:LabelA)` becomes `"(:LabelA:LabelB)"`. • A `RELATIONSHIP` is represented by square brackets surrounding its type. For example: `[:TYPE]` becomes `"[:TYPE]"`. • A `PATH` is a combination of the above, using Cypher's ascii art arrow syntax. For example: `"(:LabelA)-[:TYPE]→(:LabelB)"`.
Prior to Neo4j 2026.09, this function did not accept `LIST`, `MAP`, `NODE`, `RELATIONSHIP`, and `PATH` types.

Example 331. toString()

Query

```
RETURN
  toString(11.5),
  toString('already a string'),
  toString(true),
  toString(date({year: 1984, month: 10, day: 11})) AS dateString,
  toString(datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14, millisecond:
341, timezone: 'Europe/Stockholm'})) AS datetimeString,
  toString(duration({minutes: 12, seconds: -60})) AS durationString,
  toString([1, 'a', true]) AS listString,
  toString({a: 1, b: 'hi'}) AS mapString,
  toString(null) AS nullValue
```

Result

toString(11.5)	toString('already a string')	toString(true)	dateString	datetimeString	durationString	listString	mapString	nullValue
"11.5"	"already a string"	"true"	"1984-10-11"	"1984-10-11T12:31:14.341+01:00[Europe/Stockholm]"	"PT11M"	"[1, a, true]"	"{a: 1, b: hi}"	null

Rows: 1

toStringOrNull()

Details

Syntax	`toStringOrNull(input)`

Description	Converts a value to a `STRING`, or `null` if the value cannot be converted.		
Arguments	Name	Type	Description
	`input`	`ANY`	A value to be converted into a string or null.
Returns	`STRING`		

Considerations

`toStringOrNull(null)` returns `null`.
Prior to Neo4j 2026.09, this function did not accept `LIST`, `MAP`, `NODE`, `RELATIONSHIP`, and `PATH` types.

Example 332. toStringOrNull()

Query

```
RETURN toStringOrNull(11.5),
toStringOrNull('already a string'),
toStringOrNull(true),
toStringOrNull(date({year: 1984, month: 10, day: 11})) AS dateString,
toStringOrNull(datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14,
millisecond: 341, timezone: 'Europe/Stockholm'})) AS datetimeString,
toStringOrNull(duration({minutes: 12, seconds: -60})) AS durationString,
toStringOrNull([1, 'a', true]) AS list,
toStringOrNull({a: 1, b: 'hi'}) AS map,
toStringOrNull(null) AS nullValue
```

Result

toStringOrNull(11.5)	toStringOrNull('already a string')	toStringOrNull(true)	dateString	datetimeString	durationString	list	map	nullValue
"11.5"	"already a string"	"true"	"1984-10-11"	"1984-10-11T12:31:14.341+01:00[Europe/Stockholm]"	"PT11M"	"[1, a, true]"	"{a: 1, b: hi}"	null
Rows: 1								

toUpper()

Details

Syntax	`toUpper(input)`
Description	Returns the given `STRING` in uppercase.

Arguments	Name	Type	Description
	`input`	`STRING`	A string to be converted into uppercase.
Returns	`STRING`		

Considerations

`toUpper(null)` returns `null`.

Example 333. toUpper()

Query

```
RETURN toUpper('hello')
```

Result

toUpper('hello')
"HELLO"
Rows: 1

trim()

Details

Syntax	`trim([[LEADING | TRAILING | BOTH] [trimCharacterString] FROM] input)`		
Description	Returns the given `STRING` with leading and/or trailing `trimCharacterString` removed.		
Arguments	Name	Type	Description
	`trimSpecification`	`[LEADING, TRAILING, BOTH]`	The parts of the string to trim; LEADING, TRAILING, BOTH
	`trimCharacterString`	`STRING`	The characters to be removed from the start and/or end of the given string.
	`input`	`STRING`	A value from which all leading and/or trailing trim characters will be removed.
Returns	`STRING`		

Considerations

`trim(null)` returns `null`.

`trim(null FROM "hello")` returns `null`.
`trim(" " FROM null)` returns `null`.
`trim(BOTH null FROM null)` returns `null`.
`trim(INVALID null FROM null)` throws an invalid trimSpecification error.
If `trimSpecification` and a `trimCharacterString` are not specified all leading and/or trailing whitespace will be removed.

Example 334. trim()

Query

```
RETURN trim('   hello   '), trim(BOTH 'x' FROM 'xxxhelloxxx')
```

Result

trim(' hello ')	trim(BOTH 'x' FROM 'xxxhelloxxx')
"hello"	"hello"
Rows: 1	

upper()

Details

Syntax	`upper(input)`		
Description	Returns the given `STRING` in uppercase.		
Arguments	Name	Type	Description
	`input`	`STRING`	A string to be converted into uppercase.
Returns	`STRING`		

This function is an alias to the `toUpper()` function, and it was introduced as part of Cypher's GQL conformance.

Considerations

`upper(null)` returns `null`.

Example 335. upper()

Query

```
RETURN upper('hello')
```

Result

upper('hello')
"HELLO"
Rows: 1

Temporal functions

Duration functions

Duration functions allow for the creation and manipulation of temporal `DURATION` values.

Note:

See also Temporal values and Temporal operators.

duration()

Details

Syntax	`duration(input[, pattern])`		
Description	Creates a `DURATION` value.		
Arguments	**Name**	**Type**	**Description**
	`input`	`ANY`	Either a string representation of a duration value or a map optionally containing the following keys: 'years', 'months', 'weeks', 'days', 'hours', 'minutes', 'seconds', 'milliseconds', 'microseconds', or 'nanoseconds'.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`DURATION`		

Considerations

If `input` is not a string, at least one component must be provided (`duration()` and `duration({})` are invalid).
There is no constraint on how many components are provided.
It is possible to have a `DURATION` where the amount of a smaller unit (e.g. `seconds`) exceeds the threshold of a larger unit (e.g. `days`).
The values of the components may be expressed as decimal fractions.
The values of components may be arbitrarily large.
The values of components may be negative.

The components of `DURATION` objects are individually accessible.
The `pattern` argument is constructed from Allowed characters for duration types.

Example 336. duration() using duration components

Query

```
UNWIND [
duration({days: 14, hours:16, minutes: 12}),
duration({months: 5, days: 1.5}),
duration({months: 0.75}),
duration({weeks: 2.5}),
duration({minutes: 1.5, seconds: 1, milliseconds: 123, microseconds: 456, nanoseconds: 789}),
duration({minutes: 1.5, seconds: 1, nanoseconds: 123456789})
] AS aDuration
RETURN aDuration
```

Result

aDuration
P14DT16H12M
P5M1DT12H
P22DT19H51M49.5S
P17DT12H
PT1M31.123456789S
PT1M31.123456789S
Rows: 6

Example 337. duration() using STRING values

Query

```
UNWIND [
duration("P14DT16H12M"),
duration("P5M1.5D"),
duration("P0.75M"),
duration("PT0.75M"),
duration("P2012-02-02T14:37:21.545"),
duration("5 hours 6 minutes", "h 'hours' m 'minutes'")
] AS aDuration
RETURN aDuration
```

Result

aDuration
P14DT16H12M
P5M1DT12H
P22DT19H51M49.5S

aDuration
PT45S
P2012Y2M2DT14H37M21.545S
PT5H6M
Rows: 6

duration.between()

Details

Syntax	`duration.between(from, to)`		
Description	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in logical units.		
Arguments	Name	Type	Description
	`from`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the starting instant.
	`to`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the ending instant.
Returns	`DURATION`		

Considerations

If `to` occurs earlier than `from`, the resulting `DURATION` will be negative.
If `from` has a time component and `to` does not, the time component of `to` is assumed to be midnight, and vice versa.
If `from` has a time zone component and `to` does not, the time zone component of `to` is assumed to be the same as that of `from`, and vice versa.
If `to` has a date component and `from` does not, the date component of `from` is assumed to be the same as that of `to`, and vice versa.

Example 338. duration.between()

Query

```
UNWIND [
```

```
duration.between(date("1984-10-11"), date("1985-11-25")),
duration.between(date("1985-11-25"), date("1984-10-11")),
duration.between(date("1984-10-11"), datetime("1984-10-12T21:40:32.142+0100")),
duration.between(date("2015-06-24"), localtime("14:30")),
duration.between(localtime("14:30"), time("16:30+0100")),
duration.between(localdatetime("2015-07-21T21:40:32.142"), localdatetime("2016-07-21T21:45:22.142")),
duration.between(datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/Stockholm'}),
datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/London'}))
] AS aDuration
RETURN aDuration
```

Result

aDuration
P1Y1M14D
P-1Y-1M-14D
P1DT21H40M32.142S
PT14H30M
PT2H
P1YT4M50S
PT1H
Rows: 7

duration_between()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`duration_between(from, to)`		
Description	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in logical units.		
Arguments	Name	Type	Description
	`from`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the starting instant.
	`to`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the ending instant.

Returns	`DURATION`

This function is an alias to the `duration.between()` function, and it was introduced as part of Cypher's GQL conformance.

Differences from GQL

In GQL, if either argument is not a temporal instant type, a data exception is raised (22G03). In Cypher, a different error (e.g. invalid call signature) may be reported, but with the same severity. Cypher's `duration.between()` returns the date difference in years, months, and days; GQL's `duration_between()` returns the date difference in days only. For the same two dates the results can therefore differ (e.g. for 2026-02-01 to 2026-03-02, Cypher returns P1M1D and GQL returns P29D). Whenever the span includes one or more full months, the Cypher result may differ from the GQL result.

Considerations

If `to` occurs earlier than `from`, the resulting `DURATION` will be negative.
If `from` has a time component and `to` does not, the time component of `to` is assumed to be midnight, and vice versa.
If `from` has a time zone component and `to` does not, the time zone component of `to` is assumed to be the same as that of `from`, and vice versa.
If `to` has a date component and `from` does not, the date component of `from` is assumed to be the same as that of `to`, and vice versa.

Example 339. duration_between()

Query

```
RETURN duration_between(date("1984-10-11"), date("1985-11-25")) AS aDuration
```

Result

aDuration
P1Y1M14D
Rows: 1

duration.inDays()

Details

Syntax	`duration.inDays(from, to)`
Description	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in days.

Arguments	Name	Type	Description
	`from`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the starting instant.
	`to`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the ending instant.
Returns	`DURATION`		

Considerations

If `to` occurs earlier than `from`, the resulting `DURATION` will be negative.
If `from` has a time component and `to` does not, the time component of `to` is assumed to be midnight, and vice versa.
If `from` has a time zone component and `to` does not, the time zone component of `to` is assumed to be the same as that of `from`, and vice versa.
If `from` has a date component and `to` does not, the date component of `to` is assumed to be the same as that of `from`, and vice versa.
Any difference smaller than a whole day is disregarded.
Get the total number of days in a `DURATION` by returning the `days` component. For more information, see Components of durations.

Example 340. duration.inDays()

Query

```
UNWIND [
duration.inDays(date("1984-10-11"), date("1985-11-25")),
duration.inDays(date("1985-11-25"), date("1984-10-11")),
duration.inDays(date("1984-10-11"), datetime("1984-10-12T21:40:32.142+0100")),
duration.inDays(date("2015-06-24"), localtime("14:30")),
duration.inDays(localdatetime("2015-07-21T21:40:32.142"), localdatetime("2016-07-21T21:45:22.142")),
duration.inDays(datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/Stockholm'}),
datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/London'}))
] AS aDuration
RETURN aDuration
```

Result

aDuration
P410D
P-410D

aDuration
P1D
PT0S
P366D
PT0S
Rows: 6

duration.inMonths()

Details

Syntax	`duration.inMonths(from, to)`		
Description	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in months.		
Arguments	Name	Type	Description
	`from`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the starting instant.
	`to`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the ending instant.
Returns	`DURATION`		

Considerations

If `to` occurs earlier than `from`, the resulting `DURATION` will be negative.
If `from` has a time component and `to` does not, the time component of `to` is assumed to be midnight, and vice versa.
If `from` has a time zone component and `to` does not, the time zone component of `to` is assumed to be the same as that of `from`, and vice versa.
If `from` has a date component and `to` does not, the date component of `to` is assumed to be the same as that of `from`, and vice versa.
Any difference smaller than a whole month is disregarded.

Get the total number of months in a `DURATION` by returning the `months` component. For more information, see Components of durations.

Example 341. duration.inMonths()

Query

```
UNWIND [
duration.inMonths(date("1984-10-11"), date("1985-11-25")),
duration.inMonths(date("1985-11-25"), date("1984-10-11")),
duration.inMonths(date("1984-10-11"), datetime("1984-10-12T21:40:32.142+0100")),
duration.inMonths(date("2015-06-24"), localtime("14:30")),
duration.inMonths(localdatetime("2015-07-21T21:40:32.142"), localdatetime("2016-07-
21T21:45:22.142")),
duration.inMonths(datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/Stockholm'}),
datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/London'}))
] AS aDuration
RETURN aDuration
```

Result

aDuration
P1Y1M
P-1Y-1M
PT0S
PT0S
P1Y
PT0S
Rows: 6

duration.inSeconds()

Details

Syntax	`duration.inSeconds(from, to)`
Description	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in seconds.

Arguments	Name	Type	Description
	`from`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the starting instant.
	`to`	`ANY`	A temporal instant type (`DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME`, `ZONED DATETIME`) representing the ending instant.
Returns	`DURATION`		

Considerations

If `to` occurs earlier than `from`, the resulting `DURATION` will be negative.
If `from` has a time component and `to` does not, the time component of `to` is assumed to be midnight, and vice versa.
If `from` has a time zone component and `to` does not, the time zone component of `to` is assumed to be the same as that of `from`, and vice versa.
If `from` has a date component and `to` does not, the date component of `to` is assumed to be the same as that of `from`, and vice versa.
Get the total seconds of days in a `DURATION` by returning the `seconds` component. For more information, see Components of durations.

Example 342. duration.inSeconds()

Query

```
UNWIND [
duration.inSeconds(date("1984-10-11"), date("1984-10-12")),
duration.inSeconds(date("1984-10-12"), date("1984-10-11")),
duration.inSeconds(date("1984-10-11"), datetime("1984-10-12T01:00:32.142+0100")),
duration.inSeconds(date("2015-06-24"), localtime("14:30")),
duration.inSeconds(datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/Stockholm'}),
datetime({year: 2017, month: 10, day: 29, hour: 0, timezone: 'Europe/London'}))
] AS aDuration
RETURN aDuration
```

Result

aDuration
PT24H
PT-24H
PT25H32.142S

aDuration
PT14H30M
PT1H
Rows: 5

Instant type functions

Temporal functions allow for the creation and manipulation of values for each temporal type — `DATE`, `ZONED TIME`, `LOCAL TIME`, `ZONED DATETIME`, and `LOCAL DATETIME`.

The following functions are included on this page:

`DATE`	`ZONED DATETIME`	`LOCAL DATETIME`	`LOCAL TIME`	`ZONED TIME`
date()	datetime()	localdatetime()	localtime()	time()
	zoned_datetime()	local_datetime()	local_time()	zoned_time()
	datetime.fromEpoch()			
	datetime.fromEpochMillis()			
date.realtime()	datetime.realtime()	localdatetime.realtime()	localtime.realtime()	time.realtime()
date.statement()	datetime.statement()	localdatetime.statement()	localtime.statement()	time.statement()
date.transaction()	datetime.transaction()	localdatetime.transaction()	localtime.transaction()	time.transaction()

Note:

See also Temporal (Date/Time) values and Temporal operators.

Temporal instant types

An overview of temporal instant type creation

Each function bears the same name as the type, and construct the type they correspond to in one of four ways:

- Capturing the current time.
- Composing the components of the type.
- Parsing a `STRING` representation of the temporal value.
- Selecting and composing components from another temporal value by

- either combining temporal values (such as combining a `DATE` with a `ZONED TIME` to create a `ZONED DATETIME`), or
- selecting parts from a temporal value (such as selecting the `DATE` from a `ZONED DATETIME`); the *extractors* — groups of components which can be selected — are:
  - `date` — contains all components for a `DATE` (conceptually *year*, *month* and *day*).
  - `time` — contains all components for a `ZONED TIME` (*hour*, *minute*, *second*, and sub-seconds; namely *millisecond*, *microsecond* and *nanosecond*). If the type being created and the type from which the time component is being selected both contain `timezone` (and a `timezone` is not explicitly specified) the `timezone` is also selected.
  - `datetime` — selects all components, and is useful for overriding specific components. Analogously to `time`, if the type being created and the type from which the time component is being selected both contain `timezone` (and a `timezone` is not explicitly specified) the `timezone` is also selected.
- In effect, this allows for the *conversion* between different temporal types, and allowing for 'missing' components to be specified.

Temporal instant type creation functions

Function	`DATE`	`ZONED TIME`	`LOCAL TIME`	`ZONED DATETIME`	`LOCAL DATETIME`
Getting the current value.	✔	✔	✔	✔	✔
Creating a calendar-based (Year-Month-Day) value.	✔			✔	✔
Creating a week-based (Year-Week-Day) value.	✔			✔	✔
Creating a quarter-based (Year-Quarter-Day) value.	✔			✔	✔
Creating an ordinal (Year-Day) value.	✔			✔	✔
Creating a value from time components.		✔	✔		

Function	`DATE`	`ZONED TIME`	`LOCAL TIME`	`ZONED DATETIME`	`LOCAL DATETIME`
Creating a value from other temporal values using extractors (i.e. converting between different types).	✔	✔	✔	✔	✔
Creating a value from a `STRING`.	✔	✔	✔	✔	✔
Creating a value from a timestamp.				✔	

Note:

All the temporal instant types — including those that do not contain timezone information support such as `DATE`, `LOCAL TIME` and `ZONED DATETIME` — allow for a timezone to specified for the functions that retrieve the current instant. This allows for the retrieval of the current instant in the specified timezone.

Controlling which clock to use

The functions which create temporal instant values based on the current instant use the `statement` clock as default. However, there are three different clocks available for more fine-grained control:

- `transaction`: The same instant is produced for each invocation within the same transaction. A different time may be produced for different transactions.
- `statement`: The same instant is produced for each invocation within the same statement. A different time may be produced for different statements within the same transaction.
- `realtime`: The instant produced will be the live clock of the system.

The following table lists the different sub-functions for specifying the clock to be used when creating the current temporal instant value:

Type	default	transaction	statement	realtime
Date	date()	date.transaction()	date.statement()	date.realtime()
Time	time()	time.transaction()	time.statement()	time.realtime()
LocalTime	localtime()	localtime.transaction()	localtime.statement()	localtime.realtime()
DateTime	datetime()	datetime.transaction()	datetime.statement()	datetime.realtime()

Type	default	transaction	statement	realtime
LocalDateTime	localdatetime()	localdatetime.transaction()	localdatetime.statement()	localdatetime.realtime()

Truncating temporal values

A temporal instant value can be created by truncating another temporal instant value at the nearest preceding point in time at a specified component boundary (namely, a *truncation unit*). A temporal instant value created in this way will have all components which are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of overriding the default values which would otherwise have been set for these smaller components.

The following table lists the supported truncation units and the corresponding sub-functions:

Truncation unit	`DATE`	`ZONED TIME`	`LOCAL TIME`	`ZONED DATETIME`	`LOCAL DATETIME`
`millennium`	date.truncate('millennium', input)			datetime.truncate('millennium', input)	localdatetime.truncate('millennium', input)
`century`	date.truncate('century', input)			datetime.truncate('century', input)	localdatetime.truncate('century', input)
`decade`	date.truncate('decade', input)			datetime.truncate('decade', input)	localdatetime.truncate('decade', input)
`year`	date.truncate('year', input)			datetime.truncate('year', input)	localdatetime.truncate('year', input)
`weekYear`	date.truncate('weekYear', input)			datetime.truncate('weekYear', input)	localdatetime.truncate('weekYear', input)
`quarter`	date.truncate('quarter', input)			datetime.truncate('quarter', input)	localdatetime.truncate('quarter', input)
`month`	date.truncate('month', input)			datetime.truncate('month', input)	localdatetime.truncate('month', input)
`week`	date.truncate('week', input)			datetime.truncate('week', input)	localdatetime.truncate('week', input)
`day`	date.truncate('day', input)	time.truncate('day', input)	localtime.truncate('day', input)	datetime.truncate('day', input)	localdatetime.truncate('day', input)
`hour`		time.truncate('hour', input)	localtime.truncate('hour', input)	datetime.truncate('hour', input)	localdatetime.truncate('hour',input)

Truncation unit	`DATE`	`ZONED TIME`	`LOCAL TIME`	`ZONED DATETIME`	`LOCAL DATETIME`
`minute`		time.truncate('minute', input)	localtime.truncate('minute', input)	datetime.truncate('minute', input)	localdatetime.truncate('minute', input)
`second`		time.truncate('second', input)	localtime.truncate('second', input)	datetime.truncate('second', input)	localdatetime.truncate('second', input)
`millisecond`		time.truncate('millisecond', input)	localtime.truncate('millisecond', input)	datetime.truncate('millisecond', input)	localdatetime.truncate('millisecond', input)
`microsecond`		time.truncate('microsecond', input)	localtime.truncate('microsecond', input)	datetime.truncate('microsecond', input)	localdatetime.truncate('microsecond', input)

date()

Details

Syntax	`date([ input, pattern] )`		
Description	Creates a `DATE` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	Either a string representation of a temporal value, a map containing the single key 'timezone', or a map containing temporal values ('date', 'year', 'month', 'day', 'week', 'dayOfWeek', 'quarter', 'dayOfQuarter', 'ordinalDay') as components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`DATE`		

Temporal components

Name	Description
`date`	A `DATE` value.
`year`	An expression consisting of at least four digits that specifies the year.

Name	Description
`month`	An integer between `1` and `12` that specifies the month.
`day`	An integer between `1` and `31` that specifies the day of the month.
`week`	An integer between `1` and `53` that specifies the week.
`dayOfWeek`	An integer between `1` and `7` that specifies the day of the week.
`quarter`	An integer between `1` and `4` that specifies the quarter.
`dayOfQuarter`	An integer between `1` and `92` that specifies the day of the quarter.
`ordinalDay`	An integer between `1` and `366` that specifies the ordinal day of the year.

Considerations

If no parameters are provided, `date()` must be invoked (`date({})` is invalid).
If no timezone is specified, the local timezone will be used.
The *day of the month* component will default to `1` if `day` is omitted.
The *month* component will default to `1` if `month` is omitted.
If `month` is omitted, `day` must also be omitted.
The *day of the week* component will default to `1` if `dayOfWeek` is omitted.
The *week* component will default to `1` if `week` is omitted.
If `week` is omitted, `dayOfWeek` must also be omitted.
The *day of the quarter* component will default to `1` if `dayOfQuarter` is omitted.
The *quarter* component will default to `1` if `quarter` is omitted.
If `quarter` is omitted, `dayOfQuarter` must also be omitted.
The *ordinal day of the year* component will default to `1` if `ordinalDay` is omitted.
String representations of temporal values must comply with the format defined for dates.
String representations of temporal values must denote a valid date; i.e. a temporal value denoting `30 February 2001` is invalid.
`date(null)` returns `null`.
If any of the optional parameters are provided, these will override the corresponding components of `date`.
`date(dd)` may be written instead of `date({date: dd})`.

The `pattern` argument is constructed from Allowed characters for instance types.

Example 343. date() to get the current time (no parameters provided)

Query

```
RETURN date() AS currentDate
```

The current date is returned.

Result

currentDate
2022-06-14
Rows: 1

Example 344. date() with provided timezone

Query

```
RETURN date({timezone: 'America/Los Angeles'}) AS currentDateInLA
```

The current date in California is returned.

Result

currentDateInLA
2022-06-14
Rows: 1

Creating `DATE` values

Example 345. date() - Creating a calendar (Yeay-Month-Day) `DATE`

Query

```
UNWIND [
date({year: 1984, month: 10, day: 11}),
date({year: 1984, month: 10}),
date({year: 1984})
] AS theDate
RETURN theDate
```

Result

theDate
1984-10-11

theDate
1984-10-01
1984-01-01
Rows: 3

Example 346. date() - Creating a week (Year-Week-Day) `DATE`

Query

```
UNWIND [
date({year: 1984, week: 10, dayOfWeek: 3}),
date({year: 1984, week: 10}),
date({year: 1984})
] AS theDate
RETURN theDate
```

Result

theDate
1984-03-07
1984-03-05
1984-01-01
Rows: 3

Example 347. date() - Creating a quarter (Year-Quarter-Day) `DATE`

Query

```
UNWIND [
date({year: 1984, quarter: 3, dayOfQuarter: 45}),
date({year: 1984, quarter: 3}),
date({year: 1984})
] AS theDate
RETURN theDate
```

Result

theDate
1984-08-14
1984-07-01
1984-01-01
Rows: 3

Example 348. date() - Creating an ordinal (Year-Day) `DATE`

Query

```
UNWIND [
date({year: 1984, ordinalDay: 202}),
date({year: 1984})
] AS theDate
RETURN theDate
```

The date corresponding to `11 February 1984` is returned.

Result

theDate
1984-07-20
1984-01-01
Rows: 2

Example 349. date() - Creating a `DATE` using other temporal values as components

Query

```
UNWIND [
date({year: 1984, month: 11, day: 11}),
localdatetime({year: 1984, month: 11, day: 11, hour: 12, minute: 31, second: 14}),
datetime({year: 1984, month: 11, day: 11, hour: 12, timezone: '+01:00'})
] AS dd
RETURN date({date: dd}) AS dateOnly, date({date: dd, day: 28}) AS dateDay
```

Result

dateOnly	+dateDay
1984-11-11	1984-11-28
1984-11-11	1984-11-28
1984-11-11	1984-11-28
Rows: 3	

Example 350. date() - Creating a `DATE` from a `STRING`

Query

```
UNWIND [
date('2015-07-21'),
date('2015-07'),
date('201507'),
date('2015-W30-2'),
date('2015202'),
date('2015'),
date('11/18/1986', "MM/dd/yyyy")
] AS theDate
```

```
RETURN theDate
```

Result

theDate
2015-07-21
2015-07-01
2015-07-01
2015-07-21
2015-07-21
2015-01-01
1986-11-18
Rows: 7

date.realtime()

Details

Syntax	`date.realtime([ timezone ])`		
Description	Returns the current `DATE` instant using the realtime clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`DATE`		

This returned `DATE` will be the live clock of the system.

Example 351. date.realtime()

Query

```
RETURN date.realtime() AS currentDate
```

Result

currentDate
2022-06-14
Rows: 1

Example 352. date.realtime()

Query

```
RETURN date.realtime('America/Los Angeles') AS currentDateInLA
```

Result

currentDateInLA
2022-06-14
Rows: 1

date.statement()

Details

Syntax	`date.statement([ timezone ])`		
Description	Returns the current `DATE` instant using the statement clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`DATE`		

This returned `DATE` will be the same for each invocation within the same statement. However, a different value may be produced for different statements within the same transaction.

Example 353. date.statement()

Query

```
RETURN date.statement() AS currentDate
```

Result

currentDate
2022-06-14
Rows: 1

date.transaction()

Details

Syntax	`date.transaction([ timezone ])`

Description	Returns the current `DATE` instant using the transaction clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`DATE`		

The returned `DATE` will be the same for each invocation within the same transaction. However, a different value may be produced for different transactions.

Example 354. date.transaction()

Query

```
RETURN date.transaction() AS currentDate
```

Result

currentDate
2022-06-14
Rows: 1

date.truncate()

Details

Syntax	`date.truncate(unit [, input, fields])`		
Description	Truncates the given temporal value to a `DATE` instant using the specified unit.		
Arguments	Name	Type	Description
	`unit`	`STRING`	A string representing one of the following: 'day', 'week', 'month', 'weekYear', 'quarter', 'year', 'decade', 'century', 'millennium'.
	`input`	`ANY`	The date to be truncated using either `ZONED DATETIME`, `LOCAL DATETIME`, or `DATE`.
	`fields`	`MAP`	A list of time components smaller than those specified in `unit` to preserve during truncation.

Returns	`DATE`

`date.truncate()` returns the `DATE` value obtained by truncating a specified temporal instant value at the nearest preceding point in time at the specified component boundary (which is denoted by the truncation unit passed as a parameter to the function). In other words, the `DATE` returned will have all components that are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of *overriding* the default values which would otherwise have been set for these smaller components. For example, `day` — with some value `x` — may be provided when the truncation unit `STRING` is `'year'` in order to ensure the returned value has the *day* set to `x` instead of the default *day* (which is `1`).

Considerations

Any component that is provided in `fields` must be smaller than `unit`; i.e. if `unit` `STRING` is `'day'`, `fields` cannot contain information pertaining to a *month*.
Any component that is not contained in `fields` and which is smaller than `unit` will be set to its minimal value.
If `fields` is not provided, all components of the returned value which are smaller than `unit` will be set to their default values.
If `input` is not provided, it will be set to the current date, i.e. `date.truncate(unit)` is equivalent of `date.truncate(unit, date())`.

Example 355. date.truncate()

Query

```
WITH
  datetime({
    year: 2017, month: 11, day: 11,
    hour: 12, minute: 31, second: 14, nanosecond: 645876123,
    timezone: '+01:00'
  }) AS d
RETURN
  date.truncate('millennium', d) AS truncMillenium,
  date.truncate('century', d) AS truncCentury,
  date.truncate('decade', d) AS truncDecade,
  date.truncate('year', d, {day: 5}) AS truncYear,
  date.truncate('weekYear', d) AS truncWeekYear,
  date.truncate('quarter', d) AS truncQuarter,
  date.truncate('month', d) AS truncMonth,
  date.truncate('week', d, {dayOfWeek: 2}) AS truncWeek,
  date.truncate('day', d) AS truncDay
```

Result

truncMillenium	truncCentury	truncDecade	truncYear	truncWeekYear	truncQuarter	truncMonth	truncWeek	truncDay
2000-01-01	2000-01-01	2010-01-01	2017-01-05	2017-01-02	2017-10-01	2017-11-01	2017-11-07	2017-11-11
Rows: 1								

datetime()

Details

Syntax	`datetime([ input, pattern ])`		
Description	Creates a `ZONED DATETIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	Either a string representation of a temporal value, a map containing the single key 'timezone', or a map containing temporal values ('year', 'month', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond', 'nanosecond', 'timezone') as components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`ZONED DATETIME`		

Temporal components

Name	Description
`year`	An expression consisting of at least four digits that specifies the year.
`month`	An integer between `1` and `12` that specifies the month.
`day`	An integer between `1` and `31` that specifies the day of the month.
`hour`	An integer between `0` and `23` that specifies the hour of the day.
`minute`	An integer between `0` and `59` that specifies the number of minutes.
`second`	An integer between `0` and `59` that specifies the number of seconds.
`millisecond`	An integer between `0` and `999` that specifies the number of milliseconds.

Name	Description
`microsecond`	An integer between `0` and `999,999` that specifies the number of microseconds.
`nanosecond`	An integer between `0` and `999,999,999` that specifies the number of nanoseconds.
`timezone`	An expression that specifies the timezone.
`epochSeconds`	A numeric value representing the number of seconds from the UNIX epoch in the UTC timezone.
`epochMillis`	A numeric value representing the number of milliseconds from the UNIX epoch in the UTC timezone.

Considerations

If no parameters are provided, `datetime()` must be invoked (`datetime({})` is invalid).
The *month* component will default to `1` if `month` is omitted.
The *day of the month* component will default to `1` if `day` is omitted.
The *hour* component will default to `0` if `hour` is omitted.
The *minute* component will default to `0` if `minute` is omitted.
The *second* component will default to `0` if `second` is omitted.
Any missing `millisecond`, `microsecond` or `nanosecond` values will default to `0`.
The *timezone* component will default to the configured default timezone if `timezone` is omitted.
If `millisecond`, `microsecond` and `nanosecond` are given in combination (as part of the same set of parameters), the individual values must be in the range `0` to `999`.
The smallest components in the set `year`, `month`, `day`, `hour`, `minute`, and `second` may be omitted; i.e. it is possible to specify only `year`, `month` and `day`, but specifying `year`, `month`, `day` and `minute` is not permitted.
One or more of `millisecond`, `microsecond` and `nanosecond` can only be specified as long as `second` is also specified.
String representations of temporal values must comply with the format defined for dates, times and time zones.
String representations of temporal values must denote a valid date; i.e. a temporal value denoting `30 February 2001` is invalid.
If any of the optional parameters are provided, these will override the corresponding components of `datetime`, `date` and/or `time`.
`datetime(dd)` may be written instead of `datetime({datetime: dd})`.
Selecting a `ZONED TIME` or `ZONED DATETIME` value as the `time` component also selects its timezone. If a `LOCAL TIME` or `LOCAL DATETIME` is selected instead, the default timezone is used. In any case, the timezone can be overridden explicitly.

Selecting a `ZONED DATETIME` as the `datetime` component and overwriting the timezone will adjust the local time to keep the same point in time.
Selecting a `ZONED DATETIME` or `ZONED TIME` as the `time` component and overwriting the timezone will adjust the local time to keep the same point in time.
`epochSeconds` / `epochMillis` may be used in conjunction with `nanosecond`.
`datetime(null)` returns null.
The `pattern` argument is constructed from Allowed characters for instance types.

Example 356. .datetime() to get the current datetime (no parameters provided)

Query

```
RETURN datetime() AS currentDateTime
```

The current date and time using the local timezone is returned.

Result

currentDateTime
2022-06-14T10:02:28.192Z
Rows: 1

Example 357. datetime() with provided timezone

Query

```
RETURN datetime({timezone: 'America/Los Angeles'}) AS currentDateTimeInLA
```

The current date and time of day in California is returned.

Result

currentDateTimeInLA
2022-06-14T03:02:28.238-07:00[America/Los_Angeles]
Rows: 1

Creating `ZONED DATETIME` values

Example 358. datetime() - Creating a calendar (Year-Month-Day) `ZONED DATETIME`

Query

```
UNWIND [
```

```
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14, millisecond: 123,
microsecond: 456, nanosecond: 789}),
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14, millisecond: 645,
timezone: '+01:00'}),
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14, nanosecond: 645876123,
timezone: 'Europe/Stockholm'}),
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14, timezone: '+01:00'}),
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, second: 14}),
datetime({year: 1984, month: 10, day: 11, hour: 12, minute: 31, timezone: 'Europe/Stockholm'}),
datetime({year: 1984, month: 10, day: 11, hour: 12, timezone: '+01:00'}),
datetime({year: 1984, month: 10, day: 11, timezone: 'Europe/Stockholm'})
] AS theDate
RETURN theDate
```

Result

theDate
1984-10-11T12:31:14.123456789Z
1984-10-11T12:31:14.645+01:00
1984-10-11T12:31:14.645876123+01:00[Europe/Stockholm]
1984-10-11T12:31:14+01:00
1984-10-11T12:31:14Z
1984-10-11T12:31+01:00[Europe/Stockholm]
1984-10-11T12:00+01:00
1984-10-11T00:00+01:00[Europe/Stockholm]
Rows: 8

Example 359. datetime() - Creating a week (Year-Week-Day) `ZONED DATETIME`

Query

```
UNWIND [
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, minute: 31, second: 14, millisecond: 645}),
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, minute: 31, second: 14, microsecond: 645876,
timezone: '+01:00'}),
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, minute: 31, second: 14, nanosecond:
645876123, timezone: 'Europe/Stockholm'}),
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, minute: 31, second: 14, timezone:
'Europe/Stockholm'}),
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, minute: 31, second: 14}),
datetime({year: 1984, week: 10, dayOfWeek: 3, hour: 12, timezone: '+01:00'}),
datetime({year: 1984, week: 10, dayOfWeek: 3, timezone: 'Europe/Stockholm'})
] AS theDate
RETURN theDate
```

Result

theDate
1984-03-07T12:31:14.645Z
1984-03-07T12:31:14.645876+01:00
1984-03-07T12:31:14.645876123+01:00[Europe/Stockholm]

theDate
1984-03-07T12:31:14+01:00[Europe/Stockholm]
1984-03-07T12:31:14Z
1984-03-07T12:00+01:00
1984-03-07T00:00+01:00[Europe/Stockholm]
Rows: 7

Example 360. datetime() - Creating a quarter (Year-Quarter-Day) `ZONED DATETIME`

Query

```
UNWIND [
datetime({year: 1984, quarter: 3, dayOfQuarter: 45, hour: 12, minute: 31, second: 14, microsecond:
645876}),
datetime({year: 1984, quarter: 3, dayOfQuarter: 45, hour: 12, minute: 31, second: 14, timezone:
'+01:00'}),
datetime({year: 1984, quarter: 3, dayOfQuarter: 45, hour: 12, timezone: 'Europe/Stockholm'}),
datetime({year: 1984, quarter: 3, dayOfQuarter: 45})
] AS theDate
RETURN theDate
```

Result

theDate
1984-08-14T12:31:14.645876Z
1984-08-14T12:31:14+01:00
1984-08-14T12:00+02:00[Europe/Stockholm]
1984-08-14T00:00Z
Rows: 4

Example 361. datetime() - Creating an ordinal (Year-Day) `ZONED DATETIME`

Query

```
UNWIND [
datetime({year: 1984, ordinalDay: 202, hour: 12, minute: 31, second: 14, millisecond: 645}),
datetime({year: 1984, ordinalDay: 202, hour: 12, minute: 31, second: 14, timezone: '+01:00'}),
datetime({year: 1984, ordinalDay: 202, timezone: 'Europe/Stockholm'}),
datetime({year: 1984, ordinalDay: 202})
] AS theDate
RETURN theDate
```

Result

theDate
1984-07-20T12:31:14.645Z

theDate
1984-07-20T12:31:14+01:00
1984-07-20T00:00+02:00[Europe/Stockholm]
1984-07-20T00:00Z
Rows: 4

Example 362. datetime() - Creating a `ZONED DATETIME` from a `STRING`

Query

```
UNWIND [
datetime('2015-07-21T21:40:32.142+0100'),
datetime('2015-W30-2T214032.142Z'),
datetime('2015T214032-0100'),
datetime('20150721T21:40-01:30'),
datetime('2015-W30T2140-02'),
datetime('2015202T21+18:00'),
datetime('2015-07-21T21:40:32.142[Europe/London]'),
datetime('2015-07-21T21:40:32.142-04[America/New_York]'),
datetime('Tuesday, November 18, AD 1986', 'EEEE, MMMM d, G uuuu')
] AS theDate
RETURN theDate
```

Result

theDate
2015-07-21T21:40:32.142+01:00
2015-07-21T21:40:32.142Z
2015-01-01T21:40:32-01:00
2015-07-21T21:40-01:30
2015-07-20T21:40-02:00
2015-07-21T21:00+18:00
2015-07-21T21:40:32.142+01:00[Europe/London]
2015-07-21T21:40:32.142-04:00[America/New_York]
1986-11-18T00:00Z
Rows: 9

Example 363. datetime() - Creating a `ZONED DATETIME` using other temporal values as components

The following query shows the various usages of `datetime({date [, year, ..., timezone]})`.

Query

```
WITH date({year: 1984, month: 10, day: 11}) AS dd
RETURN
```

```
  datetime({date: dd, hour: 10, minute: 10, second: 10}) AS dateHHMMSS,
  datetime({date: dd, hour: 10, minute: 10, second: 10, timezone:'+05:00'}) AS dateHHMMSSTimezone,
  datetime({date: dd, day: 28, hour: 10, minute: 10, second: 10}) AS dateDDHHMMSS,
  datetime({date: dd, day: 28, hour: 10, minute: 10, second: 10, timezone:'Pacific/Honolulu'}) AS
dateDDHHMMSSTimezone
```

Result

dateHHMMSS	dateHHMMSSTimezone	dateDDHHMMSS	dateDDHHMMSSTimezone
1984-10-11T10:10:10Z	1984-10-11T10:10:10+05:00	1984-10-28T10:10:10Z	1984-10-28T10:10:10-10:00[Pacific/Honolulu]

Rows: 1

Example 364. datetime() - Creating a `ZONED DATETIME` using other temporal values as components

The following query shows the various usages of `datetime({time [, year, …, timezone]})`.

Query

```
WITH time({hour: 12, minute: 31, second: 14, microsecond: 645876, timezone: '+01:00'}) AS tt
RETURN
  datetime({year: 1984, month: 10, day: 11, time: tt}) AS YYYYMMDDTime,
  datetime({year: 1984, month: 10, day: 11, time: tt, timezone:'+05:00'}) AS YYYYMMDDTimeTimezone,
  datetime({year: 1984, month: 10, day: 11, time: tt, second: 42}) AS YYYYMMDDTimeSS,
  datetime({year: 1984, month: 10, day: 11, time: tt, second: 42, timezone: 'Pacific/Honolulu'}) AS
YYYYMMDDTimeSSTimezone
```

Result

YYYYMMDDTime	+YYYYMMDDTimeTimezone	YYYYMMDDTimeSS	YYYYMMDDTimeSSTimezone
1984-10-11T12:31:14.645876+01:00	1984-10-11T16:31:14.645876+05:00	1984-10-11T12:31:42.645876+01:00	1984-10-11T01:31:42.645876-10:00[Pacific/Honolulu]

Rows: 1

Example 365. datetime() - Creating a `ZONED DATETIME` using other temporal values as components

The following query shows the various usages of `datetime({date, time [, year, ..., timezone]})`; i.e. combining a `DATE` and a `ZONED TIME` value to create a single `ZONED DATETIME` value.

Query

```
WITH
  date({year: 1984, month: 10, day: 11}) AS dd,
  localtime({hour: 12, minute: 31, second: 14, millisecond: 645}) AS tt
RETURN
  datetime({date: dd, time: tt}) AS dateTime,
  datetime({date: dd, time: tt, timezone: '+05:00'}) AS dateTimeTimezone,
  datetime({date: dd, time: tt, day: 28, second: 42}) AS dateTimeDDSS,
  datetime({date: dd, time: tt, day: 28, second: 42, timezone: 'Pacific/Honolulu'}) AS
dateTimeDDSSTimezone
```

Result

dateTime	dateTimeTimezone	dateTimeDDSS	dateTimeDDSSTimezone
1984-10-11T12:31:14.645Z	1984-10-11T12:31:14.645+05:00	1984-10-28T12:31:42.645Z	1984-10-28T12:31:42.645-10:00[Pacific/Honolulu]

Rows: 1

Example 366. datetime() - Creating a `ZONED DATETIME` using other temporal values as components

The following query shows the various usages of `datetime({datetime [, year, ..., timezone]})`.

Query

```
WITH
  datetime({
    year: 1984, month: 10, day: 11,
    hour: 12,
    timezone: 'Europe/Stockholm'
  }) AS dd
RETURN
  datetime({datetime: dd}) AS dateTime,
  datetime({datetime: dd, timezone: '+05:00'}) AS dateTimeTimezone,
  datetime({datetime: dd, day: 28, second: 42}) AS dateTimeDDSS,
  datetime({datetime: dd, day: 28, second: 42, timezone: 'Pacific/Honolulu'}) AS dateTimeDDSSTimezone
```

Result

dateTime	dateTimeTimezone	dateTimeDDSS	dateTimeDDSSTimezone
1984-10-11T12:00+01:00[Europe/Stockholm]	1984-10-11T16:00+05:00	1984-10-28T12:00:42+01:00[Europe/Stockholm]	1984-10-28T01:00:42-10:00[Pacific/Honolulu]

Rows: 1

Example 367. datetime() - Creating a `ZONED DATETIME` from UNIX epoch (`epocSeconds`)

`datetime()` returns the `ZONED DATETIME` value at the specified number of *seconds* or *milliseconds* from the UNIX epoch in the UTC timezone.

Conversions to other temporal instant types from UNIX epoch representations can be achieved by transforming a `ZONED DATETIME` value to one of these types.

Query

```
RETURN datetime({epochSeconds: timestamp() / 1000, nanosecond: 23}) AS theDate
```

Result

theDate
2022-06-14T10:02:30.000000023Z

theDate
Rows: 1

Example 368. datetime() - Creating a `ZONED DATETIME` from UNIX epoch (`epocMillis`)

Query

```
RETURN datetime({epochMillis: 424797300000}) AS theDate
```

Result

theDate
1983-06-18T15:15Z
Rows: 1

zoned_datetime()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`zoned_datetime([ input, pattern ])`		
Description	Creates a `ZONED DATETIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	A temporal value or a map containing temporal components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`ZONED DATETIME`		

This function is an alias to the `datetime()` function, and it was introduced as part of Cypher's GQL conformance. For details on supported components and parsing behavior, see `datetime()`.

Differences from GQL

In GQL, invalid sets of datetime (or timezone) field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument for parsing; this alias supports it as well. The GQL keyword `CURRENT_TIMESTAMP` is not introduced as a reserved keyword in Cypher, as that would break existing queries using that name as a variable.

Example 369. zoned_datetime()

Query

```
RETURN zoned_datetime('2022-06-14T10:02:30Z') AS theDateTime
```

Result

theDateTime
2022-06-14T10:02:30Z
Rows: 1

datetime.fromEpoch()

Details

Syntax	`datetime.fromEpoch(seconds, nanoseconds)`		
Description	Creates a `ZONED DATETIME` given the seconds and nanoseconds since the start of the epoch.		
Arguments	Name	Type	Description
	`seconds`	`INTEGER | FLOAT`	The number of seconds from the UNIX epoch in the UTC timezone.
	`nanoseconds`	`INTEGER | FLOAT`	The number of nanoseconds from the UNIX epoch in the UTC timezone. This can be added to seconds.
Returns	`ZONED DATETIME`		

Example 370. datetime.fromEpoch()

Query

```
WITH datetime.fromEpoch(1683000000, 123456789) AS dateTimeFromEpoch
RETURN dateTimeFromEpoch
```

Result

dateTimeFromEpoch
2023-05-02T04:00:00.123456789Z
Rows: 1

datetime.fromEpochMillis()

Details

Syntax	`datetime.fromEpochMillis(milliseconds)`		
Description	Creates a `ZONED DATETIME` given the milliseconds since the start of the epoch.		
Arguments	Name	Type	Description
	`milliseconds`	`INTEGER | FLOAT`	The number of milliseconds from the UNIX epoch in the UTC timezone.
Returns	`ZONED DATETIME`		

Example 371. datetime.fromEpochMillis()

Query

```
WITH datetime.fromEpochMillis(1724198400000) AS dateTimeFromMillis
RETURN dateTimeFromMillis
```

Result

dateTimeFromMillis
2024-08-21T00:00Z
Rows: 1

datetime.realtime()

Details

Syntax	`datetime.realtime([ timezone ])`		
Description	Returns the current `ZONED DATETIME` instant using the realtime clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`ZONED DATETIME`		

The returned `ZONED DATETIME` will be the live clock of the system.

Example 372. datetime.realtime()

Query

```
RETURN datetime.realtime() AS currentDateTime
```

Result

currentDateTime
2022-06-14T10:02:28.494444Z
Rows: 1

datetime.statement()

Details

Syntax	`datetime.statement([ timezone ])`		
Description	Returns the current `ZONED DATETIME` instant using the statement clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`ZONED DATETIME`		

This returned `ZONED DATETIME` will be the same for each invocation within the same statement. However, a different value may be produced for different statements within the same transaction.

Example 373. datetime.statement()

Query

```
RETURN datetime.statement() AS currentDateTime
```

Result

currentDateTime
2022-06-14T10:02:28.395Z
Rows: 1

datetime.transaction()

Details

Syntax	`datetime.transaction([ timezone ])`		
Description	Returns the current `ZONED DATETIME` instant using the transaction clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.

Returns	`ZONED DATETIME`

The returned `ZONED DATETIME` value will be the same for each invocation within the same transaction. However, a different value may be produced for different transactions.

Example 374. datetime.transaction()

Query

```
RETURN datetime.transaction() AS currentDateTime
```

Result

currentDateTime
2022-06-14T10:02:28.290Z
Rows: 1

Example 375. datetime.transaction()

Query

```
RETURN datetime.transaction('America/Los Angeles') AS currentDateTimeInLA
```

Result

currentDateTimeInLA
2022-06-14T03:02:28.338-07:00[America/Los_Angeles]
Rows: 1

datetime.truncate()

Details

Syntax	`datetime.truncate(unit [, input, fields])`
Description	Truncates the given temporal value to a `ZONED DATETIME` instant using the specified unit.

Arguments	Name	Type	Description
	`unit`	`STRING`	A string representing one of the following: 'microsecond', 'millisecond', 'second', 'minute', 'hour', 'day', 'week', 'month', 'weekYear', 'quarter', 'year', 'decade', 'century', 'millennium'.
	`input`	`ANY`	The date to be truncated using either `ZONED DATETIME`, `LOCAL DATETIME`, or `DATE`.
	`fields`	`MAP`	A list of time components smaller than those specified in `unit` to preserve during truncation.
Returns	`ZONED DATETIME`		

`datetime.truncate()` returns the `ZONED DATETIME` value obtained by truncating a specified temporal instant value at the nearest preceding point in time at the specified component boundary (which is denoted by the truncation unit passed as a parameter to the function). In other words, the `ZONED DATETIME` returned will have all components that are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of *overriding* the default values which would otherwise have been set for these smaller components. For example, `day` — with some value `x` — may be provided when the truncation unit `STRING` is `'year'` in order to ensure the returned value has the *day* set to `x` instead of the default *day* (which is `1`).

Considerations

`input` cannot be a `DATE` value if `unit` is one of: `'hour'`, `'minute'`, `'second'`, `'millisecond'`, `'microsecond'`.
The timezone of `input` may be overridden; for example, `datetime.truncate('minute', input, {timezone: '+0200'})`.
If `input` is one of `ZONED TIME`, `ZONED DATETIME` — a value with a timezone — and the timezone is overridden, no time conversion occurs.
If `input` is one of `LOCAL DATETIME`, `DATE` — a value without a timezone — and the timezone is not overridden, the configured default timezone will be used.
Any component that is provided in `fields` must be smaller than `unit`; i.e. if `unit` is `'day'`, `fields` cannot contain information pertaining to a *month*.
Any component that is not contained in `fields` and which is smaller than `unit` will be set to its minimal value.
If `fields` is not provided, all components of the returned value which are smaller than `unit` will be set to their default values.

If `input` is not provided, it will be set to the current date, time and timezone, i.e. `datetime.truncate(unit)` is equivalent of `datetime.truncate(unit, datetime())`.

Example 376. datetime.truncate()

Query

```
WITH
  datetime({
    year:2017, month:11, day:11,
    hour:12, minute:31, second:14, nanosecond: 645876123,
    timezone: '+03:00'
  }) AS d
RETURN
  datetime.truncate('millennium', d, {timezone: 'Europe/Stockholm'}) AS truncMillenium,
  datetime.truncate('year', d, {day: 5}) AS truncYear,
  datetime.truncate('month', d) AS truncMonth,
  datetime.truncate('day', d, {millisecond: 2}) AS truncDay,
  datetime.truncate('hour', d) AS truncHour,
  datetime.truncate('second', d) AS truncSecond
```

Result

truncMillenium	truncYear	truncMonth	truncDay	truncHour	truncSecond
2000-01-01T00:00+01:00[Europe/Stockholm]	2017-01-05T00:00+03:00	2017-11-01T00:00+03:00	2017-11-11T00:00:00.002+03:00	2017-11-11T12:00+03:00	2017-11-11T12:31:14+03:00

Rows: 1

localdatetime()

Details

Syntax	`localdatetime([ input, pattern ])`
Description	Creates a `LOCAL DATETIME` instant.

Arguments	Name	Type	Description
	`input`	`ANY`	Either a string representation of a temporal value, a map containing the single key 'timezone', or a map containing temporal values ('year', 'month', 'day', 'hour', 'minute', 'second', 'millisecond', 'microsecond', 'nanosecond') as components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`LOCAL DATETIME`		

Temporal components

Name	Description
`A single map consisting of the following:`	
`year`	An expression consisting of at least four digits that specifies the year.
`month`	An integer between `1` and `12` that specifies the month.
`day`	An integer between `1` and `31` that specifies the day of the month.
`hour`	An integer between `0` and `23` that specifies the hour of the day.
`minute`	An integer between `0` and `59` that specifies the number of minutes.
`second`	An integer between `0` and `59` that specifies the number of seconds.
`millisecond`	An integer between `0` and `999` that specifies the number of milliseconds.
`microsecond`	An integer between `0` and `999,999` that specifies the number of microseconds.
`nanosecond`	An integer between `0` and `999,999,999` that specifies the number of nanoseconds.

Considerations

If no parameters are provided, `localdatetime()` must be invoked (localdatetime({}) is invalid).
The *month* component will default to `1` if `month` is omitted.
The *day of the month* component will default to `1` if `day` is omitted.
The *hour* component will default to `0` if `hour` is omitted.
The *minute* component will default to `0` if `minute` is omitted.
The *second* component will default to `0` if `second` is omitted.
Any missing `millisecond`, `microsecond` or `nanosecond` values will default to `0`.
If `millisecond`, `microsecond` and `nanosecond` are given in combination (as part of the same set of parameters), the individual values must be in the range `0` to `999`.
The smallest components in the set `year`, `month`, `day`, `hour`, `minute`, and `second` may be omitted; i.e. it is possible to specify only `year`, `month` and `day`, but specifying `year`, `month`, `day` and `minute` is not permitted.
One or more of `millisecond`, `microsecond` and `nanosecond` can only be specified as long as `second` is also specified.
String representations of temporal values must comply with the format defined for dates and times.
String representations of temporal values must denote a valid date; i.e. a temporal value denoting `30 February 2001` is invalid.
`localdatetime(null)` returns null.
If any of the optional parameters are provided, these will override the corresponding components of `datetime`, `date` and/or `time`.
`localdatetime(dd)` may be written instead of `localdatetime({datetime: dd})`.
The `pattern` argument is constructed from Allowed characters for instance types.

Example 377. localdatetime() - to get current local date and time (no parameters)

Query

```
RETURN localdatetime() AS now
```

The current local date and time (i.e. in the local timezone) is returned.

Result

now
2022-06-14T10:02:30.447
Rows: 1

Example 378. localdatetime() with timezone

Query

```
RETURN localdatetime({timezone: 'America/Los Angeles'}) AS now
```

The current local date and time in California is returned.

Result

now
2022-06-14T03:02:30.482
Rows: 1

Creating `LOCAL DATETIME` values

Example 379. localdatetime() - Creating a calendar (Year-Month-Day) `LOCAL DATETIME`

Query

```
RETURN
  localdatetime({
    year: 1984, month: 10, day: 11,
    hour: 12, minute: 31, second: 14, millisecond: 123, microsecond: 456, nanosecond: 789
  }) AS theDate
```

Result

theDate
1984-10-11T12:31:14.123456789
Rows: 1

Example 380. localdatetime() - Creating a week (Year-Week-Day) `LOCAL DATETIME`

Query

```
RETURN
  localdatetime({
    year: 1984, week: 10, dayOfWeek: 3,
    hour: 12, minute: 31, second: 14, millisecond: 645
  }) AS theDate
```

Result

theDate
1984-03-07T12:31:14.645
Rows: 1

Example 381. localdatetime() - Creating a quarter (Year-Quarter-Day) `ZONED DATETIME`

Query

```
RETURN
  localdatetime({
    year: 1984, quarter: 3, dayOfQuarter: 45,
    hour: 12, minute: 31, second: 14, nanosecond: 645876123
  }) AS theDate
```

Result

theDate
1984-08-14T12:31:14.645876123
Rows: 1

Example 382. localdatetime() - Creating an ordinal (Year-Day) `LOCAL DATETIME`

Query

```
RETURN
  localdatetime({
    year: 1984, ordinalDay: 202,
    hour: 12, minute: 31, second: 14, microsecond: 645876
  }) AS theDate
```

Result

theDate
1984-07-20T12:31:14.645876
Rows: 1

Example 383. localdatetime() - Creating a `LOCAL DATETIME` from a `STRING`

Query

```
UNWIND [
localdatetime('2015-07-21T21:40:32.142'),
localdatetime('2015-W30-2T214032.142'),
localdatetime('2015-202T21:40:32'),
localdatetime('2015202T21'),
localdatetime('Tuesday, November 18, AD 1986', 'EEEE, MMMM d, G uuuu')
] AS theDate
RETURN theDate
```

Result

theDate
2015-07-21T21:40:32.142
2015-07-21T21:40:32.142

theDate
2015-07-21T21:40:32
2015-07-21T21:00
1986-11-18T00:00Z
Rows: 5

Example 384. localdatetime() - Creating a `LOCAL DATETIME` using other temporal values as components

The following query shows the various usages of `localdatetime({date [, year, ..., nanosecond]})`.

Query

```
WITH date({year: 1984, month: 10, day: 11}) AS dd
RETURN
  localdatetime({date: dd, hour: 10, minute: 10, second: 10}) AS dateHHMMSS,
  localdatetime({date: dd, day: 28, hour: 10, minute: 10, second: 10}) AS dateDDHHMMSS
```

Result

dateHHMMSS	dateDDHHMMSS
1984-10-11T10:10:10	1984-10-28T10:10:10
Rows: 1	

Example 385. localdatetime() - Creating a `LOCAL DATETIME` using other temporal values as components

The following query shows the various usages of `localdatetime({time [, year, ..., nanosecond]})`.

Query

```
WITH time({hour: 12, minute: 31, second: 14, microsecond: 645876, timezone: '+01:00'}) AS tt
RETURN
  localdatetime({year: 1984, month: 10, day: 11, time: tt}) AS YYYYMMDDTime,
  localdatetime({year: 1984, month: 10, day: 11, time: tt, second: 42}) AS YYYYMMDDTimeSS
```

Result

YYYYMMDDTime	YYYYMMDDTimeSS
1984-10-11T12:31:14.645876	1984-10-11T12:31:42.645876
Rows: 1	

Example 386. localdatetime() - Creating a `LOCAL DATETIME` using other temporal values as components

The following query shows the various usages of `localdatetime({date, time [, year, ...,`

`nanosecond]})`; i.e. combining a `DATE` and a `ZONED TIME` value to create a single `LOCAL DATETIME` value.

Query

```
WITH
  date({year: 1984, month: 10, day: 11}) AS dd,
  time({hour: 12, minute: 31, second: 14, microsecond: 645876, timezone: '+01:00'}) AS tt
RETURN
  localdatetime({date: dd, time: tt}) AS dateTime,
  localdatetime({date: dd, time: tt, day: 28, second: 42}) AS dateTimeDDSS
```

Result

dateTime	dateTimeDDSS
1984-10-11T12:31:14.645876	1984-10-28T12:31:42.645876
Rows: 1	

Example 387. localdatetime() - Creating a `LOCAL DATETIME` using other temporal values as components

The following query shows the various usages of `localdatetime({datetime [, year, ..., nanosecond]})`.

Query

```
WITH
  datetime({
    year: 1984, month: 10, day: 11,
    hour: 12,
    timezone: '+01:00'
  }) AS dd
RETURN
  localdatetime({datetime: dd}) AS dateTime,
  localdatetime({datetime: dd, day: 28, second: 42}) AS dateTimeDDSS
```

Result

dateTime	dateTimeDDSS
1984-10-11T12:00	1984-10-28T12:00:42
Rows: 1	

local_datetime()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`local_datetime([ input, pattern ])`		
Description	Creates a `LOCAL DATETIME` instant.		
Arguments	Name	Type	Description

	`input`	`ANY`	A temporal value or a map containing temporal components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`LOCAL DATETIME`		

This function is an alias to the `localdatetime()` function, and it was introduced as part of Cypher's GQL conformance. For details on supported components and parsing behavior, see `localdatetime()`.

Differences from GQL

In GQL, invalid sets of datetime field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument for parsing; this alias supports it as well. The GQL keyword `LOCAL_TIMESTAMP` is not introduced as a reserved keyword in Cypher, as that would break existing queries using that name as a variable.

Example 388. local_datetime()

Query

```
RETURN local_datetime('2015-07-21T21:40:32.142') AS theDateTime
```

Result

theDateTime
2015-07-21T21:40:32.142
Rows: 1

localdatetime.realtime()

Details

Syntax	`localdatetime.realtime([ timezone ])`		
Description	Returns the current `LOCAL DATETIME` instant using the realtime clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL DATETIME`		

The returned `LOCAL DATETIME` will be the live clock of the system.

Example 389. localdatetime.realtime()

Query

```
RETURN localdatetime.realtime() AS now
```

Result

now
2022-06-14T10:02:30.647817
Rows: 1

Example 390. localdatetime.realtime()

Query

```
RETURN localdatetime.realtime('America/Los Angeles') AS nowInLA
```

Result

nowInLA
2022-06-14T03:02:30.691099
Rows: 1

localdatetime.statement()

Details

Syntax	`localdatetime.statement([ timezone ])`		
Description	Returns the current `LOCAL DATETIME` instant using the statement clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL DATETIME`		

The returned `LOCAL DATETIME` will be the same for each invocation within the same statement. However, a different value may be produced for different statements within the same transaction.

Example 391. localdatetime.statement()

Query

```
RETURN localdatetime.statement() AS now
```

Result

now
2022-06-14T10:02:30.570
Rows: 1

localdatetime.transaction()

Details

Syntax	`localdatetime.transaction([ timezone ])`		
Description	Returns the current `LOCAL DATETIME` instant using the transaction clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL DATETIME`		

The returned `LOCAL DATETIME` will be the same for each invocation within the same transaction. However, a different value may be produced for different transactions.

Example 392. localdatetime.transaction()

Query

```
RETURN localdatetime.transaction() AS now
```

Result

now
2022-06-14T10:02:30.532
Rows: 1

localdatetime.truncate()

Details

Syntax	`localdatetime.truncate(unit [, input, fields])`
Description	Truncates the given temporal value to a `LOCAL DATETIME` instant using the specified unit.

Arguments	Name	Type	Description
	`unit`	`STRING`	A string representing one of the following: 'microsecond', 'millisecond', 'second', 'minute', 'hour', 'day', 'week', 'month', 'weekYear', 'quarter', 'year', 'decade', 'century', 'millennium'.
	`input`	`ANY`	The date to be truncated using either `ZONED DATETIME`, `LOCAL DATETIME`, or `DATE`.
	`fields`	`MAP`	A list of time components smaller than those specified in `unit` to preserve during truncation.
Returns	`LOCAL DATETIME`		

`localdatetime.truncate()` returns the `LOCAL DATETIME` value obtained by truncating a specified temporal instant value at the nearest preceding point in time at the specified component boundary (which is denoted by the truncation unit passed as a parameter to the function). In other words, the `LOCAL DATETIME` returned will have all components that are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of *overriding* the default values which would otherwise have been set for these smaller components. For example, `day` — with some value `x` — may be provided when the truncation unit `STRING` is `'year'` in order to ensure the returned value has the *day* set to `x` instead of the default *day* (which is `1`).

Considerations

`input` cannot be a `DATE` value if `unit` is one of: `'hour'`, `'minute'`, `'second'`, `'millisecond'`, `'microsecond'`.
Any component that is provided in `fields` must be smaller than `input`; i.e. if `input` is `'day'`, `fields` cannot contain information pertaining to a *month*.
Any component that is not contained in `fields` and which is smaller than `unit` will be set to its minimal value.
If `fields` is not provided, all components of the returned value which are smaller than `unit` will be set to their default values.
If `input` is not provided, it will be set to the current date and time, i.e. `localdatetime.truncate(unit)` is equivalent of `localdatetime.truncate(unit, localdatetime())`.

Example 393. localdatetime.truncate()

Query

```
WITH
  localdatetime({
    year: 2017, month: 11, day: 11,
    hour: 12, minute: 31, second: 14, nanosecond: 645876123
  }) AS d
RETURN
  localdatetime.truncate('millennium', d) AS truncMillenium,
  localdatetime.truncate('year', d, {day: 2}) AS truncYear,
  localdatetime.truncate('month', d) AS truncMonth,
  localdatetime.truncate('day', d) AS truncDay,
  localdatetime.truncate('hour', d, {nanosecond: 2}) AS truncHour,
  localdatetime.truncate('second', d) AS truncSecond
```

Result

truncMillenium	truncYear	truncMonth	truncDay	truncHour	truncSecond
2000-01-01T00:00	2017-01-02T00:00	2017-11-01T00:00	2017-11-11T00:00	2017-11-11T12:00:00.000000002	2017-11-11T12:31:14

Rows: 1

localtime()

Details

Syntax	`localtime([ input, pattern ])`		
Description	Creates a `LOCAL TIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	Either a string representation of a temporal value, a map containing the single key 'timezone', or a map containing temporal values ('hour, 'minute', 'second', 'millisecond', 'microsecond', 'nanosecond' as components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`LOCAL TIME`		

Temporal components

Name	Description
`hour`	An integer between `0` and `23` that specifies the hour of the day.
`minute`	An integer between `0` and `59` that specifies the number of minutes.
`second`	An integer between `0` and `59` that specifies the number of seconds.
`millisecond`	An integer between `0` and `999` that specifies the number of milliseconds.
`microsecond`	An integer between `0` and `999,999` that specifies the number of microseconds.
`nanosecond`	An integer between `0` and `999,999,999` that specifies the number of nanoseconds.

Considerations

If no parameters are provided, `localtime()` must be invoked (`localtime({})` is invalid).
The *hour* component will default to `0` if `hour` is omitted.
The *minute* component will default to `0` if `minute` is omitted.
The *second* component will default to `0` if `second` is omitted.
Any missing `millisecond`, `microsecond` or `nanosecond` values will default to `0`.
If `millisecond`, `microsecond` and `nanosecond` are given in combination (as part of the same set of parameters), the individual values must be in the range `0` to `999`.
The smallest components in the set `hour`, `minute`, and `second` may be omitted; i.e. it is possible to specify only `hour` and `minute`, but specifying `hour` and `second` is not permitted.
One or more of `millisecond`, `microsecond` and `nanosecond` can only be specified as long as `second` is also specified.
String representations of temporal values must comply with the format defined for times.
String representations of temporal values must denote a valid time; i.e. a temporal value denoting `13:46:64` is invalid.
`localtime(null)` returns null.
If any of the optional parameters are provided, these will override the corresponding components of `time`.
`localtime(tt)` may be written instead of `localtime({time: tt})`.
The `pattern` argument is constructed from Allowed characters for instance types.

Example 394. localtime() to get the current time (no parameters)

Query

```
RETURN localtime() AS now
```

The current local time (i.e. in the local timezone) is returned.

Result

now
10:02:31.596
Rows: 1

Example 395. localtime() with timezone

Query

```
RETURN localtime({timezone: 'America/Los Angeles'}) AS nowInLA
```

The current local time in California is returned.

Result

nowInLA
03:02:31.629
Rows: 1

Creating `LOCAL TIME` values

Example 396. localtime()

Query

```
UNWIND [
localtime({hour: 12, minute: 31, second: 14, nanosecond: 789, millisecond: 123, microsecond: 456}),
localtime({hour: 12, minute: 31, second: 14}),
localtime({hour: 12})
] AS theTime
RETURN theTime
```

Result

theTime
12:31:14.123456789
12:31:14

theTime
12:00
Rows: 3

Example 397. localtime() - Creating a `LOCAL TIME` from a `STRING`

Query

```
UNWIND [
localtime('21:40:32.142'),
localtime('214032.142'),
localtime('21:40'),
localtime('21'),
localtime('6:04', 'k:mm')
] AS theTime
RETURN theTime
```

Result

theTime
21:40:32.142
21:40:32.142
21:40
21:00
06:04
Rows: 5

Example 398. localtime() - Creating a `LOCAL TIME` using other temporal values as components

Query

```
WITH time({hour: 12, minute: 31, second: 14, microsecond: 645876, timezone: '+01:00'}) AS tt
RETURN
  localtime({time: tt}) AS timeOnly,
  localtime({time: tt, second: 42}) AS timeSS
```

Result

timeOnly	timeSS
12:31:14.645876	12:31:42.645876
Rows: 1	

local_time()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`local_time([ input, pattern ])`		
Description	Creates a `LOCAL TIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	A temporal value or a map containing temporal components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`LOCAL TIME`		

This function is an alias to the `localtime()` function, and it was introduced as part of Cypher's GQL conformance. For details on supported components and parsing behavior, see `localtime()`.

Differences from GQL

In GQL, invalid sets of time field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument for parsing; this alias supports it as well. The GQL keyword `LOCAL_TIME` (parameterless) is not introduced as a reserved keyword in Cypher, as that would break existing queries that use `LOCAL_TIME` as a variable name.

Example 399. local_time()

Query

```
RETURN local_time('21:40:32.142') AS theTime
```

Result

theTime
21:40:32.142
Rows: 1

localtime.realtime()

Details

Syntax	`localtime.realtime([ timezone ])`
Description	Returns the current `LOCAL TIME` instant using the realtime clock.

Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL TIME`		

The returned `LOCAL TIME` will be the live clock of the system.

Example 400. localtime.realtime()

Query

```
RETURN localtime.realtime() AS now
```

Result

now
10:02:31.806895
Rows: 1

localtime.statement()

Details

Syntax	`localtime.statement([ timezone ])`		
Description	Returns the current `LOCAL TIME` instant using the statement clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL TIME`		

This returned `LOCAL TIME` will be the same for each invocation within the same statement. However, a different value may be produced for different statements within the same transaction.

Example 401. localtime.statement()

Query

```
RETURN localtime.statement() AS now
```

Result

now
10:02:31.697

now
Rows: 1

Example 402. localtime.statement()

Query

```
RETURN localtime.statement('America/Los Angeles') AS nowInLA
```

Result

nowInLA
03:02:31.737
Rows: 1

localtime.transaction()

Details

Syntax	`localtime.transaction([ timezone ])`		
Description	Returns the current `LOCAL TIME` instant using the transaction clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`LOCAL TIME`		

The returned `LOCAL TIME` will be the same for each invocation within the same transaction. However, a different value may be produced for different transactions.

Example 403. localtime.transaction()

Query

```
RETURN localtime.transaction() AS now
```

Result

now
10:02:31.662
Rows: 1

localtime.truncate()

Details

Syntax	`localtime.truncate(unit [, input, fields])`		
Description	Truncates the given temporal value to a `LOCAL TIME` instant using the specified unit.		
Arguments	Name	Type	Description
	`unit`	`STRING`	A string representing one of the following: 'day', 'week', 'month', 'weekYear', 'quarter', 'year', 'decade', 'century', 'millennium'.
	`input`	`ANY`	The date to be truncated using either `ZONED DATETIME`, `LOCAL DATETIME`, `LOCAL TIME`, or `DATE`.
	`fields`	`MAP`	A list of time components smaller than those specified in `unit` to preserve during truncation.
Returns	`LOCAL TIME`		

`localtime.truncate()` returns the `LOCAL TIME` value obtained by truncating a specified temporal instant value at the nearest preceding point in time at the specified component boundary (which is denoted by the truncation unit passed as a parameter to the function). In other words, the `LOCAL TIME` returned will have all components that are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of *overriding* the default values which would otherwise have been set for these smaller components. For example, `minute` — with some value `x` — may be provided when the truncation unit string is `'hour'` in order to ensure the returned value has the *minute* set to `x` instead of the default *minute* (which is `1`).

Considerations

Truncating time to day — i.e. `unit` is `'day'` — is supported, and yields midnight at the start of the day (`00:00`), regardless of the value of `input`. However, the timezone of `input` is retained.
Any component that is provided in `fields` must be smaller than `unit`; i.e. if `unit` is `'second'`, `fields` cannot contain information pertaining to a *minute*.
Any component that is not contained in `fields` and which is smaller than `unit` will be set to its minimal value.
If `fields` is not provided, all components of the returned value which are smaller than `unit` will be set to their default values.

If `input` is not provided, it will be set to the current time, i.e. `localtime.truncate(unit)` is equivalent of `localtime.truncate(unit, localtime())`.

Example 404. localtime.truncate()

Query

```
WITH time({hour: 12, minute: 31, second: 14, nanosecond: 645876123, timezone: '-01:00'}) AS t
RETURN
  localtime.truncate('day', t) AS truncDay,
  localtime.truncate('hour', t) AS truncHour,
  localtime.truncate('minute', t, {millisecond: 2}) AS truncMinute,
  localtime.truncate('second', t) AS truncSecond,
  localtime.truncate('millisecond', t) AS truncMillisecond,
  localtime.truncate('microsecond', t) AS truncMicrosecond
```

Result

truncDay	truncHour	truncMinute	truncSecond	truncMillisecond	truncMicrosecond
00:00:00	12:00:00	12:31:00.002000000	12:31:14	12:31:14.645000000	12:31:14.645876000
Rows: 1					

time()

Details

Syntax	`time([ input, pattern ])`		
Description	Creates a `ZONED TIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	Either a string representation of a temporal value, a map containing the single key 'timezone', or a map containing temporal values ('hour', 'minute', 'second', 'millisecond', 'microsecond', 'nanosecond', 'timezone') as components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`ZONED TIME`		

Temporal components

Name	Description
`hour`	An integer between `0` and `23` that specifies the hour of the day.
`minute`	An integer between `0` and `59` that specifies the number of minutes.
`second`	An integer between `0` and `59` that specifies the number of seconds.
`millisecond`	An integer between `0` and `999` that specifies the number of milliseconds.
`microsecond`	An integer between `0` and `999,999` that specifies the number of microseconds.
`nanosecond`	An integer between `0` and `999,999,999` that specifies the number of nanoseconds.
`timezone`	An expression that specifies the timezone.

Considerations

If no parameters are provided, `time()` must be invoked (`time({})` is invalid).
The *hour* component will default to `0` if `hour` is omitted.
The *minute* component will default to `0` if `minute` is omitted.
The *second* component will default to `0` if `second` is omitted.
Any missing `millisecond`, `microsecond` or `nanosecond` values will default to `0`.
The *timezone* component will default to the configured default timezone if `timezone` is omitted.
If `millisecond`, `microsecond` and `nanosecond` are given in combination (as part of the same set of parameters), the individual values must be in the range `0` to `999`.
The smallest components in the set `hour`, `minute`, and `second` may be omitted; i.e. it is possible to specify only `hour` and `minute`, but specifying `hour` and `second` is not permitted.
One or more of `millisecond`, `microsecond` and `nanosecond` can only be specified as long as `second` is also specified.
String representations of temporal values must comply with the format defined for times and time zones.
The *timezone* component will default to the configured default timezone if it is omitted.
String representations of temporal values must denote a valid time; i.e. a `temporalValue` denoting `15:67` is invalid.
`time(null)` returns `null`.
If any of the optional parameters are provided, these will override the corresponding components of `time`.
`time(tt)` may be written instead of `time({time: tt})`.

Selecting a `ZONED TIME` or `ZONED DATETIME` value as the `time` component also selects its timezone. If a `LOCAL TIME` or `LOCAL DATETIME` is selected instead, the default timezone is used. In any case, the timezone can be overridden explicitly.
Selecting a `ZONED DATETIME` or `ZONED TIME` as the `time` component and overwriting the timezone will adjust the local time to keep the same point in time.
The `pattern` argument is constructed from Allowed characters for instance types.

Example 405. time() to get the current time (no parameters)

Query

```
RETURN time() AS currentTime
```

The current time of day using the local timezone is returned.

Result

currentTime
10:02:32.192Z
Rows: 1

Example 406. time() with timezone

Query

```
RETURN time({timezone: 'America/Los Angeles'}) AS currentTimeInLA
```

The current time of day in California is returned.

Result

currentTimeInLA
03:02:32.233-07:00
Rows: 1

Creating `ZONED TIME` values

Example 407. time()

Query

```
UNWIND [
time({hour: 12, minute: 31, second: 14, millisecond: 123, microsecond: 456, nanosecond: 789}),
time({hour: 12, minute: 31, second: 14, nanosecond: 645876123}),
time({hour: 12, minute: 31, second: 14, microsecond: 645876, timezone: '+01:00'}),
time({hour: 12, minute: 31, timezone: '+01:00'}),
time({hour: 12, timezone: '+01:00'})
] AS theTime
```

```
RETURN theTime
```

Result

theTime
12:31:14.123456789Z
12:31:14.645876123Z
12:31:14.645876000+01:00
12:31:00+01:00
12:00:00+01:00
Rows: 5

Example 408. time() - Creating a `ZONED TIME` from a `STRING`

Query

```
UNWIND [
time('21:40:32.142+0100'),
time('214032.142Z'),
time('21:40:32+01:00'),
time('214032-0100'),
time('21:40-01:30'),
time('2140-00:00'),
time('2140-02'),
time('22+18:00'),
time('6:04', 'k:mm')
] AS theTime
RETURN theTime
```

Result

theTime
21:40:32.142000000+01:00
21:40:32.142000000Z
21:40:32+01:00
21:40:32-01:00
21:40:00-01:30
21:40:00Z
21:40:00-02:00
22:00:00+18:00
06:04:00Z
Rows: 9

Example 409. time() - Creating a `ZONED TIME` using other temporal values as components

Query

```
WITH localtime({hour: 12, minute: 31, second: 14, microsecond: 645876}) AS tt
RETURN
  time({time: tt}) AS timeOnly,
  time({time: tt, timezone: '+05:00'}) AS timeTimezone,
  time({time: tt, second: 42}) AS timeSS,
  time({time: tt, second: 42, timezone: '+05:00'}) AS timeSSTimezone
```

Result

timeOnly	timeTimezone	timeSS	timeSSTimezone
12:31:14.645876Z	12:31:14.645876+05:00	12:31:42.645876Z	12:31:42.645876+05:00
Rows: 1			

zoned_time()

Introduced in Neo4j 2026.02 | Cypher 25 only

Details

Syntax	`zoned_time([ input, pattern ])`		
Description	Creates a `ZONED TIME` instant.		
Arguments	Name	Type	Description
	`input`	`ANY`	A temporal value or a map containing temporal components.
	`pattern`	`STRING`	A pattern used to parse the input. If a pattern is provided, `input` must be `STRING`.
Returns	`ZONED TIME`		

This function is an alias to the `time()` function, and it was introduced as part of Cypher's GQL conformance. For details on supported components and parsing behavior, see `time()`.

Differences from GQL

In GQL, invalid sets of time (or timezone) field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument for parsing; this alias supports it as well. The GQL keyword `CURRENT_TIME` is not introduced as a reserved keyword in Cypher, as that would break existing queries using that name as a variable.

Example 410. zoned_time()

Query

```
RETURN zoned_time('21:40:32.142+01:00') AS theTime
```

Result

theTime
21:40:32.142+01:00
Rows: 1

time.realtime()

Details

Syntax	`time.realtime([ timezone ])`		
Description	Returns the current `ZONED TIME` instant using the realtime clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`ZONED TIME`		

The returned `ZONED TIME` will be the live clock of the system.

Example 411. time.realtime()

Query

```
RETURN time.realtime() AS currentTime
```

Result

currentTime
10:02:32.436948Z
Rows: 1

time.statement()

Details

Syntax	`time.statement([ timezone ])`

Description	Returns the current `ZONED TIME` instant using the statement clock.		
Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`ZONED TIME`		

The returned `ZONED TIME` will be the same for each invocation within the same statement. However, a different value may be produced for different statements within the same transaction.

Example 412. time.statement()

Query

```
RETURN time.statement() AS currentTime
```

Result

currentTime
10:02:32.317Z
Rows: 1

Example 413. time.statement()

Query

```
RETURN time.statement('America/Los Angeles') AS currentTimeInLA
```

Result

currentTimeInLA
03:02:32.351-07:00
Rows: 1

time.transaction()

Details

Syntax	`time.transaction([ timezone ])`
Description	Returns the current `ZONED TIME` instant using the transaction clock.

Arguments	Name	Type	Description
	`timezone`	`ANY`	A string value representing a timezone.
Returns	`ZONED TIME`		

The returned `ZONED TIME` will be the same for each invocation within the same transaction. However, a different value may be produced for different transactions.

Example 414. time.transaction()

Query

```
RETURN time.transaction() AS currentTime
```

Result

currentTime
`10:02:32.276Z`
Rows: 1

time.truncate()

Details

Syntax	`time.truncate(unit [, input, fields])`		
Description	Truncates the given temporal value to a `ZONED TIME` instant using the specified unit.		
Arguments	Name	Type	Description
	`unit`	`STRING`	A string representing one of the following: 'microsecond', 'millisecond', 'second', 'minute', 'hour', 'day'.
	`input`	`ANY`	The date to be truncated using either `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, or `LOCAL TIME`.
	`fields`	`MAP`	A list of time components smaller than those specified in `unit` to preserve during truncation.
Returns	`ZONED TIME`		

`time.truncate()` returns the `ZONED TIME` value obtained by truncating a specified temporal instant value at the nearest preceding point in time at the specified component boundary (which is denoted by the truncation unit passed as a parameter to the function). In other words, the `ZONED TIME` returned will have all components that are smaller than the specified truncation unit set to their default values.

It is possible to supplement the truncated value by providing a map containing components which are smaller than the truncation unit. This will have the effect of *overriding* the default values which would otherwise have been set for these smaller components. For example, `minute` — with some value `x` — may be provided when the truncation unit `STRING` is `'hour'` in order to ensure the returned value has the *minute* set to `x` instead of the default *minute* (which is `1`).

Considerations

Truncating time to day — i.e. `unit` is `'day'` — is supported, and yields midnight at the start of the day (`00:00`), regardless of the value of `input`. However, the timezone of `input` is retained.
The timezone of `input` may be overridden; for example, `time.truncate('minute', input, {timezone: '+0200'})`.
If `input` is one of `ZONED TIME`, `ZONED DATETIME` — a value with a timezone — and the timezone is overridden, no time conversion occurs.
If `input` is one of `LOCAL TIME`, `LOCAL DATETIME`, `DATE` — a value without a timezone — and the timezone is not overridden, the configured default timezone will be used.
Any component that is provided in `fields` must be smaller than `unit`; i.e. if `unit` is `'second'`, `fields` cannot contain information pertaining to a *minute*.
Any component that is not contained in `fields` and which is smaller than `unit` will be set to its minimal value.
If `fields` is not provided, all components of the returned value which are smaller than `unit` will be set to their default values.
If `input` is not provided, it will be set to the current time and timezone, i.e. `time.truncate(unit)` is equivalent of `time.truncate(unit, time())`.

Example 415. time()

Query

```
WITH time({hour: 12, minute: 31, second: 14, nanosecond: 645876123, timezone: '-01:00'}) AS t
RETURN
  time.truncate('day', t) AS truncDay,
  time.truncate('hour', t) AS truncHour,
  time.truncate('minute', t) AS truncMinute,
  time.truncate('second', t) AS truncSecond,
  time.truncate('millisecond', t, {nanosecond: 2}) AS truncMillisecond,
  time.truncate('microsecond', t) AS truncMicrosecond
```

Result

truncDay	truncHour	truncMinute	truncSecond	truncMillisecond	truncMicrosecond
00:00:00-01:00	12:00:00-01:00	12:31:00-01:00	12:31:14-01:00	12:31:14.645000002-01:00	12:31:14.645876000-01:00

Rows: 1

Format functions

format()

Cypher 25 only | Introduced in Neo4j 2025.09

The `format()` function creates dynamically formatted string representations of temporal instance and duration types.

Details

Syntax	`format(value[, pattern])`		
Description	Returns the temporal value as an ISO-formatted `STRING` or as a `STRING` formatted by the provided pattern.		
Arguments	**Name**	**Type**	**Description**
	`value`	`DATE | LOCAL TIME | ZONED TIME | LOCAL DATETIME | ZONED DATETIME | DURATION`	A temporal value to be formatted.
	`pattern`	`STRING`	A pattern used to format the temporal value. If the pattern is not provided the value is in most cases formatted according to ISO 8601.
Returns	`STRING`		

Considerations

The output format can be customized via the `pattern` parameter.
The `pattern` parameter follows the Java DateTimeFormatter.
If no pattern is specified, the function returns an ISO-formatted string with some exceptions where the Cypher implementation is more lenient than ISO 8601, for instance when using seconds in a timezone.
Most characters yield a different output when they are repeated.
Some characters cannot be applied to certain types, for instance, `u` cannot be used to construct a string for a `LOCAL TIME` because it represents a year which is not part of the temporal value.
Any character that is not reserved, other than `[`, `]`, `{`, `}`, `#`, and `'`, are output directly. To ensure future compatibility it is recommended to wrap all characters that you want to output directly with single quotes.

Instance types

Cypher's instance types are `DATE`, `LOCAL TIME`, `ZONED TIME`, `LOCAL DATETIME` and `ZONED DATETIME`. For more information, see Values and types → temporal instants.

Use the characters in String pattern characters to create a string pattern for instance types.

Examples

Example 416. Instance formatting, US American and European dates

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "MM/dd/yyyy") AS US, format(dt, "dd/MM/yyyy") AS EU
```

Result

US	EU
"11/18/1986"	"18/11/1986"

Rows: 1

Example 417. Instance formatting with day-of-week, month-of-year, day-of-month, era and year

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "EEEE, MMMM d, G uuuu") AS instanceString
```

Result

instanceString
"Tuesday, November 18, AD 1986"

Rows: 1

Four occurrences of `E` and `M` (text presentations) output the full form of the day and month.

Example 418. Instance formatting with day-of-year and localized day-of-week

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "DDD'nd day of the year,' c'rd day of the week'") AS instanceString
```

Result

instanceString
"322nd day of the year, 3rd day of the week"

Rows: 1

Example 419. Instance formatting with clock-hour-of-day (1-24), minute-of-hour and time-zone name

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "k:mm z") AS CET, format(dt, "K:mm O") AS GMT
```

Result

CET	GMT
"6:04 CET"	"6:04 GMT+1"
Rows: 1	

Example 420. Instance formatting with month-of-year, day-of-month, milli-of-day, minute-of-hour and second-of-minute

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "LLL d,' minute 'm', second 's', millisecond of the day 'A") AS instanceString
```

Result

instanceString
"Nov 18, minute 4, second 45, millisecond of the day 21885123"
Rows: 1

Three occurrences of `L` (number/text presentation) output the short form ("Nov").

Example 421. Instance formatting with pad next and week-based-year

Query

```
WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt
RETURN format(dt, "pppYY") AS instanceString
```

Result

instanceString
" 86"
Rows: 1

The three occurrences of `p` add one space character of padding to the two digit form output by two occurrences of `Y`.

String pattern characters

Allowed characters for instance type string patterns

Character	Meaning	Presentation	Examples
`G`	era	text	AD; Anno Domini; A
`u`	year	year	2004; 04
`y`	year-of-era	year	2004; 04
`D`	day-of-year	number	189
`M` / `L`	month-of-year	number/text	7; 07; Jul; July; J
`d`	day-of-month	number	10
`g`	modified-julian-day	number	2451334
`Q` / `q`	quarter-of-year	number/text	3; 03; Q3; 3rd quarter
`Y`	week-based-year	year	1996; 96
`w`	week-of-week-based-year	number	27
`W`	week-of-month	number	4
`E`	day-of-week	text	Tue; Tuesday; T
`e` / `c`	localized day-of-week	number/text	2; 02; Tue; Tuesday; T
`F`	aligned-week-of-month	number	3
`a`	am-pm-of-day	text	PM
`B`	period-of-day	text	in the morning
`h`	clock-hour-of-am-pm (1-12)	number	12
`K`	hour-of-am-pm (0-11)	number	0
`k`	clock-hour-of-day (1-24)	number	24
`H`	hour-of-day (0-23)	number	0
`m`	minute-of-hour	number	30
`s`	second-of-minute	number	55
`S`	fraction-of-second	fraction	978
`A`	milli-of-day	number	1234
`n`	nano-of-second	number	987654321
`N`	nano-of-day	number	1234000000

Character	Meaning	Presentation	Examples
V	time-zone ID	zone-id	America/Los_Angeles; Z; -08:30
v	generic time-zone name	zone-name	Pacific Time; PT
z	time-zone name	zone-name	Pacific Standard Time; PST
O	localized zone-offset	offset-O	GMT+8; GMT+08:00; UTC-08:00
X	zone-offset 'Z' for zero	offset-X	Z; -08; -0830; -08:30; -083015; -08:30:15
x	zone-offset	offset-x	+0000; -08; -0830; -08:30; -083015; -08:30:15
Z	zone-offset	offset-Z	+0000; -0800; -08:00
p	pad next	pad modifier	1
'	escape for text	delimiter	
''	single quote	literal	

Duration types

Use the characters in String pattern characters to create a string pattern for duration types.

Cypher's duration type `DURATION` has components and component groups. For more information, see see Values and types → components of durations.

If the string pattern contains a character from a component group but does not contain a character denoting a longer duration from the same group, `format()` converts the longer duration to the equivalent duration with the character that is present. For example a missing `y` (year) will be converted to four quarters, if `q` is present in the string pattern. This is because without a reference point, there is no way to determine the specifics of a duration.

Examples

Example 422. Duration formatting, years converted to quarters

Query

```
WITH duration({years: 1, months: 4}) AS d
RETURN format(d, "y 'years' q 'quarters' M 'months'") AS withYears, format(d, "q 'quarters' M
'months'") AS withoutYears
```

Result

withYears	withoutYears
"1 years 1 quarters 1 months"	"5 quarters 1 months"
Rows: 1	

Example 423. Duration formatting, weeks converted to days

Query

```
WITH duration({weeks: 3, days: 4}) AS d
RETURN format(d, "w 'weeks' d 'days'") AS withWeeks, format(d, "d 'days'") AS withoutWeeks
```

Result

withWeeks	withoutWeeks
"3 weeks 4 days"	"25 days"
Rows: 1	

Example 424. Duration formatting, hours converted to minutes

Query

```
WITH duration({days: 4, hours: 5, minutes: 6, seconds: 7}) AS d
RETURN format(d, "h 'hours' m 'minutes'") AS withHours, format(d, "m 'minutes'") AS withoutHours
```

Result

withHours	withoutHours
"5 hours 6 minutes"	"306 minutes"
Rows: 1	

Note how the four days cannot be converted to hours or minutes and do not affect the query result. Days are in a different component group than hours and minutes, see Allowed characters for a duration type string pattern.

String pattern characters

Allowed characters for a duration type string pattern

Component Group	Characters	Presentation
Months	`y` / `Y` / `u`	years
	`q` / `Q`	quarters
	`M` / `L`	months

Component Group	Characters	Presentation
Days	`w` / `W`	weeks
	`d` / `D`	days
Seconds	`h` / `H` / `k` / `K`	hours
	`m`	minutes
	`s`	seconds
	`n` / `S`	fraction-of-second
	`A`	milliseconds
	`N`	nanoseconds

User-defined functions

User-defined functions are written in Java, deployed into the database and are called in the same way as any other Cypher function.

There are two main types of functions that can be developed and used:

Type	Description	Usage	Developing
Scalar	For each row the function takes parameters and returns a result.	Using UDF	Extending Neo4j (UDF)
Aggregating	Consumes many rows and produces an aggregated result.	Using aggregating UDF	Extending Neo4j (Aggregating UDF)

User-defined scalar functions

For each incoming row the function takes parameters and returns a single result.

For developing and deploying user-defined functions in Neo4j, see Extending Neo4j → User-defined functions.

Example 425. Call a user-defined function

This example shows how you invoke a user-defined function called `join` from Cypher.

This calls the user-defined function `org.neo4j.procedure.example.join()`.

Query

```
MATCH (n:Member)
```

```
RETURN org.neo4j.function.example.join(collect(n.name)) AS members
```

Result

members
"John,Paul,George,Ringo"
Rows: 1

User-defined aggregation functions

Aggregating functions consume many rows and produces a single aggregated result.

Example 426. Call a user-defined aggregation function

This example shows how you invoke a user-defined aggregation function called `longestString` from Cypher.

This calls the user-defined function `org.neo4j.function.example.longestString()`.

Query

```
MATCH (n:Member)
RETURN org.neo4j.function.example.longestString(n.name) AS member
```

Result

member
"George"
Rows: 1

UUID functions

Introduced in Neo4j 2026.08 | Cypher 25 only

UUID functions allow you to construct `UUID` values, and fetch their most/least significant bits.

uuid()

Details

Syntax	`uuid()`		
Description	Returns a randomly generated `UUID`.		
Arguments	Name	Type	Description

Returns	`UUID`

Details

Syntax	`uuid(name :: STRING)`		
Description	Converts a `STRING` value to a `UUID`.		
Arguments	Name	Type	Description
	`name`	`STRING`	The `STRING` must be 32 hexadecimal digits displayed in 5 groups, separated by 4 hyphens.
Returns	`UUID`		

Details

Syntax	`uuid(mostSigBits :: INTEGER, leastSigBits :: INTEGER)`		
Description	Converts two `INTEGER` values to a `UUID`.		
Arguments	Name	Type	Description
	`mostSigBits`	`INTEGER`	The most significant bits.
	`leastSigBits`	`INTEGER`	The least significant bits.
Returns	`UUID`		

Considerations

The randomly generated `UUID` value is not intended for cryptographic use.
`UUID` values can be stored as properties.
A `null` `name`, `mostSigBits`, or `leastSigBits` value will return `null`.

Example 427. Construct a random `UUID` value

Query

```
RETURN uuid() AS randomUUID
```

Result

randomUUID
UUID("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

Example 428. Convert a `STRING` value to a `UUID`

Query

```
RETURN uuid("550e8400-e29b-41d4-a716-446655440000") AS uuid
```

Result

uuid
uuid("550e8400-e29b-41d4-a716-446655440000")
Rows: 1

Example 429. Convert two `INTEGER` values to a `UUID`

Query

```
RETURN uuid(42, 42) AS uuid
```

Result

uuid
uuid("00000000-0000-002a-0000-00000000002a")
Rows: 1

Example 430. `null` values

Query

```
RETURN uuid(null) AS nullUUIDValue,
       uuid(42, null) AS nullMostSigBits,
       uuid(null, 42) AS nullLeastSigBits,
       uuid(null, null) AS nullMostLeastSigBits
```

Result

nullUUIDValue	nullMostSigBits	nullLeastSigBits	nullMostLeastSigBits
null	null	null	null
Rows: 1			

uuid.leastSignificantBits()

Details

Syntax	`uuid.leastSignificantBits(uuid)`

Description	Returns the least significant bits from the given `UUID`.		
Arguments	Name	Type	Description
	`uuid`	`UUID`	The `UUID` to return the least significant bits from.
Returns	`INTEGER`		

Considerations

`uuid.leastSignificantBits(null)` returns `null`.
The least significant bits represents the lower 64 bits of a `UUID` value (the least significant half).

Query

```
WITH uuid("550e8400-e29b-41d4-a716-446655440000") AS uuid
RETURN uuid.leastSignificantBits(uuid) AS leastSignificantBits
```

Result

leastSignificantBits
-6406858213580079104
Rows: 1

uuid.mostSignificantBits()

Details

Syntax	`uuid.mostSignificantBits(uuid)`		
Description	Returns the most significant bits from the given `UUID`.		
Arguments	Name	Type	Description
	`uuid`	`UUID`	The `UUID` to return the most significant bits from.
Returns	`INTEGER`		

Considerations

`uuid.mostSignificantBits(null)` returns `null`.
The most significant bits represents the upper 64 bits of a `UUID` value (the most significant half).

Query

```
WITH uuid("550e8400-e29b-41d4-a716-446655440000") AS uuid
```

```
RETURN uuid.mostSignificantBits(uuid) AS mostSignificantBits
```

Result

mostSignificantBits
6128981282234515924
Rows: 1

Vector functions

Vector functions allow you to construct `VECTOR` values, compute the similarity and distance of vector pairs, and calculate the size of a vector.

vector()

Introduced in Neo4j 2025.10 | Cypher 25 only

Details

Syntax	`vector(vectorValue, dimension, coordinateType)`		
Description	Constructs a `VECTOR` value.		
Arguments	Name	Type	Description
	`vectorValue`	`STRING` | `LIST<INTEGER | FLOAT>`	The numeric values to create the vector coordinates from.
	`dimension`	`INTEGER`	The dimension (number of coordinates) of the vector.
	`coordinateType`	`[INTEGER64, INTEGER32, INTEGER16, INTEGER8, FLOAT64, FLOAT32]`	The type of each coordinate in the vector.
Returns	`VECTOR`		

Considerations

`VECTOR` values can be stored as properties.
If a `STRING` is used in `vectorValue`, it must start and end with square brackets (`[]`). The values inside the brackets must be comma-separated numbers, represented in either decimal or scientific notation.
`null`, `NaN`, and `Infinity` values are not allowed as coordinate values.

If `vectorValue` contains elements that are not of the specified `coordinateType`, they will be coerced to that coordinate type if possible. This includes the potential of lossy conversion in cases where a larger type, e.g. `INTEGER64` does not fit into the specified type, e.g. `FLOAT32`.

`dimension` must be greater than `0` and less than or equal to `4096`.

A `null` `vectorValue` or `dimension` will return `null`.

Example 431. Construct a `VECTOR` value

Query

```
RETURN vector([1, 2, 3], 3, INTEGER) AS vector
```

Result

vector
vector([1, 2, 3], 3, INTEGER NOT NULL)
Rows: 1

Example 432. Construct a `VECTOR` value with a `STRING` as `vectorValue`

Query

```
RETURN vector("[1.05000e+00, 0.123, 5]", 3, FLOAT) AS vector
```

Result

vector
vector([1.05, 0.123, 5.0], 3, FLOAT NOT NULL)
Rows: 1

Example 433. `null` values

Query

```
RETURN vector(null, 3, FLOAT32) AS nullVectorValue,
       vector([1, 2, 3], null, INTEGER8) AS nullDimension
```

Result

nullVectorValue	nullDimension
null	null
Rows: 1	

vector.similarity.cosine()

Details

Syntax	`vector.similarity.cosine(a, b)`		
Description	Returns a `FLOAT` representing the similarity between the argument vectors based on their cosine.		
Arguments	Name	Type	Description
	`a`	`VECTOR` | `LIST<INTEGER | FLOAT>`	A vector or list value representing the first vector.
	`b`	`VECTOR` | `LIST<INTEGER | FLOAT>`	A vector or list value representing the second vector.
Returns	`FLOAT`		

Considerations

`vector.similarity.cosine(null, null)` returns `null`.
`vector.similarity.cosine(null, b)` returns `null`.
`vector.similarity.cosine(a, null)` returns `null`.
Both vectors must be of the same dimension.
Both vectors must be valid with respect to cosine similarity.
The implementation is the same of the latest vector index provider (`vector-2.0`).
The similarity score ranges from `0` and `1`, with scores closer to `1` indicating a higher degree of similarity.
The input arguments `a` and `b` accept `VECTOR` values as of Neo4j 2025.10.
Floating point operations are performed with `float32` arithmetic.

For more details, see the vector index documentation.

vector.similarity.euclidean()

Details

Syntax	`vector.similarity.euclidean(a, b)`
Description	Returns a `FLOAT` representing the similarity between the argument vectors based on their Euclidean distance.

Arguments	Name	Type	Description
	`a`	`VECTOR` | `LIST<INTEGER | FLOAT>`	A vector or list value representing the first vector.
	`b`	`VECTOR` | `LIST<INTEGER | FLOAT>`	A vector or list value representing the second vector.
Returns	`FLOAT`		

Considerations

`vector.similarity.euclidean(null, null)` returns `null`.
`vector.similarity.euclidean(null, b)` returns `null`.
`vector.similarity.euclidean(a, null)` returns `null`.
Both vectors must be of the same dimension.
Both vectors must be valid with respect to Euclidean similarity.
The implementation is the same of the latest available vector index provider (`vector-2.0`).
The similarity score ranges from `0` and `1`, with scores closer to `1` indicating a higher degree of similarity.
The input arguments `a` and `b` accept `VECTOR` values as of Neo4j 2025.10.
Floating point operations are performed with `float32` arithmetic.

For more details, see the vector index documentation.

Example 434. k-Nearest Neighbors

k-nearest neighbor queries return the *k* entities with the highest similarity scores based on comparing their associated vectors with a query vector. Such queries can be run against vector indexes in the form of *approximate* *k*-nearest neighbor (k-ANN) queries, whose returned entities have a high probability of being among the true *k* nearest neighbors. However, they can also be expressed as an exhaustive search using vector similarity functions directly. While this is typically significantly slower than using an index, it is exact rather than approximate and does not require an existing index. This can be useful for one-off queries on small datasets.

To create the graph used in this example, run the following query on an empty Neo4j database:

```
CREATE
  (:Node { id: 1, vector: vector([1.0, 4.0, 2.0], 3, FLOAT32) }),
  (:Node { id: 2, vector: vector([3.0, -2.0, 1.0], 3, FLOAT32) }),
  (:Node { id: 3, vector: vector([2.0, 8.0, 3.0], 3, FLOAT32) });
```

Given a parameter `query` (here set to `[4.0, 5.0, 6.0]`), you can query for the two nearest neighbors by Euclidean distance. This is achieved by matching on all candidate vectors and ordering by the similarity score:

```
MATCH (node:Node)
WITH node, vector.similarity.euclidean($query, node.vector) AS score
RETURN node, score
ORDER BY score DESCENDING
LIMIT 2
```

This returns the two nearest neighbors.

node	score
(:Node {vector: vector([2.0, 8.0, 3.0], 3, FLOAT32 NOT NULL), id: 3})	0.043478261679410934
(:Node {vector: vector([1.0, 4.0, 2.0], 3, FLOAT32 NOT NULL), id: 1})	0.03703703731298447
Rows: 2	

vector_dimension_count()

Introduced in Neo4j 2025.10 | Cypher 25 only

Details

Syntax	`vector_dimension_count(vector)`		
Description	Returns the dimension of a `VECTOR`.		
Arguments	Name	Type	Description
	`vector`	`VECTOR`	The vector to calculate the dimension of.
Returns	`INTEGER`		

You can also use the `size()` function to return the dimension of a `VECTOR` value.

Example 435. Calculate the size of a `VECTOR`

Query

```
RETURN vector_dimension_count(vector([1, 2, 3], 3, INTEGER8)) AS size
```

Result

size
3
Rows: 1

vector_distance()

Introduced in Neo4j 2025.10 | Cypher 25 only

Details

Syntax	`vector_distance(vector1, vector2, vectorDistanceMetric)`		
Description	Returns a `FLOAT` representing the distance between the two vector values based on the selected `vectorDistanceMetric` algorithm.		
Arguments	Name	Type	Description
	`vector1`	`VECTOR`	The first vector.
	`vector2`	`VECTOR`	The second vector.
	`vectorDistanceMetric`	`[EUCLIDEAN, EUCLIDEAN_SQUARED, MANHATTAN, COSINE, DOT, HAMMING]`	The vector distance algorithm to calculate the distance by.
Returns	`FLOAT`		

Supported `vectorDistanceMetric` algorithms

Distance Type	Formula
`EUCLIDEAN`	$\sqrt{((A_1 - B_1)^2 + (A_2 - B_2)^2 + \ldots + (A_n - B_n)^2)}$
`EUCLIDEAN_SQUARED`	$(A_1 - B_1)^2 + (A_2 - B_2)^2 + \ldots + (A_n - B_n)^2$
`MANHATTAN`	$\lvert A_1 - B_1 \rvert + \lvert A_2 - B_2 \rvert + \ldots + \lvert A_n - B_n \rvert$
`COSINE`	$1 - ((A_1 \times B_1 + A_2 \times B_2 + \ldots + A_n \times B_n) / (\sqrt{(A_1^2 + A_2^2 + \ldots + A_n^2)} \times \sqrt{(B_1^2 + B_2^2 + \ldots + B_n^2)}))$
`DOT`	$-(A_1 \times B_1 + A_2 \times B_2 + \ldots + A_n \times B_n)$
`HAMMING`	Number of dimensions in which `vector1` and `vector2` differ.

Considerations

The smaller the returned number, the more similar the vectors; the larger the number, the more distant the vectors. This is in contrast to the similarity functions where the closer to `1` the result is the higher the degree of similarity.
Floating point operations are performed with `float32` arithmetic.

Example 436. Calculate the distance between two vectors using the `COSINE` distance

Query

```
RETURN vector_distance(vector([1, 2, 3], 3, INTEGER8), vector([1, 2, 4], 3, INTEGER8), COSINE) AS
distance
```

Result

distance
0.008539855480194092
Rows: 1

Example 437. Calculate the distance between two vectors using the `EUCLIDEAN` distance

Query

```
RETURN vector_distance(vector([1.0, 5.0, 3.0, 6.7], 4, FLOAT32), vector([5.0, 2.5, 3.1, 9.0], 4,
FLOAT32), EUCLIDEAN)
```

Result

distance
5.248809337615967
Rows: 1

vector_norm()

Introduced in Neo4j 2025.20 | Cypher 25 only

Details

Syntax	`vector_norm(vector, vectorDistanceMetric)`		
Description	Returns a `FLOAT` representing the distance between the given vector and an origin vector, which is a vector with the same dimension with all coordinates set to zero, calculated using the specified `vectorDistanceMetric`.		
Arguments	Name	Type	Description
	`vector`	`VECTOR`	A vector for which the norm to the origin vector will be computed.
	`vectorDistanceMetric`	`[EUCLIDEAN, MANHATTAN]`	The vector distance algorithm to calculate the distance by.
Returns	`FLOAT`		

Supported `vectorDistanceMetric` algorithms

Distance Type	Formula
`EUCLIDEAN`	$\sqrt{((A_1 - B_1)^2 + (A_2 - B_2)^2 + ... + (A_n - B_n)^2)}$

Distance Type	Formula
`MANHATTAN`	$\|A_1 - B_1\| + \|A_2 - B_2\| + \ldots + \|A_n - B_n\|$

Example 438. Measure the norm between a vector and an origin vector using the `EUCLIDEAN` distance

Considerations

Floating point operations are performed with `float32` arithmetic.

Query

```
RETURN vector_norm(vector([1.0, 5.0, 3.0, 6.7], 4, FLOAT32), EUCLIDEAN) AS norm
```

Result

norm
8.93812084197998
Rows: 1

Example 439. Measure the norm between a vector and an origin vector using the `EUCLIDEAN` distance

Query

```
RETURN vector_norm(vector([1.0, 5.0, 3.0, 6.7], 4, FLOAT32), MANHATTAN) AS norm
```

Result

norm
15.699999809265137
Rows: 1

Indexes

An index is a copy of specified primary data in a Neo4j database, such as nodes, relationships, or properties. The data stored in an index provides an access path to data in primary storage. Depending on the index type, this access path can accelerate ordinary Cypher predicates or support explicit ranked search.

Once an index has been created, it is automatically populated and updated by the DBMS.

Neo4j supports two categories of indexes:

- Performance indexes are selected automatically by the Cypher planner to speed up data retrieval for query predicates. This category includes range, text, point, and token lookup indexes.
- Search indexes are queried explicitly to retrieve and rank matches. This category includes full-text indexes for lexical search and vector indexes for similarity search, commonly used for semantic search.

Performance indexes

Performance indexes allow the Cypher planner to evaluate predicates more efficiently without changing what those predicates mean. There are four different performance indexes available in Neo4j:

- **Range indexes**: Neo4j's default index. Supports most types of predicates.
- **Text indexes**: solves predicates operating on `STRING` values. Optimized for queries filtering with the `STRING` operators `CONTAINS` and `ENDS WITH`.
- **Point indexes**: solves predicates on spatial `POINT` values. Optimized for queries filtering on distance or within bounding boxes.
- **Token lookup indexes**: only solves node label and relationship type predicates (i.e. they cannot solve any predicates filtering on properties). Two token lookup indexes (one for node labels and one for relationship types) are present when a database is created in Neo4j.

To learn more about creating, listing, and deleting these indexes, as well as more details about the predicates supported by each index type, see Create indexes, Show indexes and Drop indexes.

For information about how performance indexes are used in Cypher queries, how they impact their performance, as well as some heuristics for when to use (and not to use) a performance index, see The impact of indexes on query performance.

Performance indexes are used automatically, and if several indexes are available, the Cypher planner tries to use the index (or indexes) that can most efficiently solve a particular predicate. It is, however, possible to explicitly force a query to use a particular index with the `USING` keyword. For more information, see Index hints for the Cypher planner.

Create indexes

Creating an index is done with the `CREATE [index_type] INDEX [index_name]` command. If no index type is specified in the create command a range index is created.

The following index types are included in this category:

- Range indexes
- Text indexes
- Point indexes
- Token lookup indexes

It is recommended to give the index a name when it is created. If the index is not explicitly named, it gets an auto-generated name.

> **Note:**
>
> The index name must be unique among both indexes and constraints.

The `CREATE INDEX` command is optionally idempotent. This mean that its default behavior is to throw an error if an attempt is made to create the same index twice. If `IF NOT EXISTS` is appended to the command, no error is thrown and nothing happens should an index with the same name or same schema and index type already exist. It may still throw an error if conflicting constraints exist, such as constraints with the same name or schema and backing index type. Instead, an informational notification is returned showing the existing index which blocks the creation.

Index configuration settings can be specified using the `OPTIONS` clause. However, not all indexes have available configuration settings. In those cases, nothing needs to be specified and the `OPTIONS` map should be omitted from the query.

> **Note:**
>
> Creating an index requires the `CREATE INDEX` privilege.

> **Note:**
>
> An index cannot be used while its `state` is `POPULATING`, which occurs immediately after it is created. To check the `state` of an index — whether it is `ONLINE` (usable) or `POPULATING` (still being built; the `populationPercent` column shows the progress of the index creation) — run the following command: `SHOW INDEXES`.

Create a range index

Creating a range index can be done with the `CREATE INDEX` command. Note that the index name must be unique.

Range indexes have no supported index configuration.

Supported predicates

Range indexes support most types of predicates:

Predicate	Syntax
Equality check.	`n.prop = value`
List membership check.	`n.prop IN list`
Existence check.	`n.prop IS NOT NULL`
Range search.	`n.prop > value`
Prefix search.	`STARTS WITH`

Examples

- Create a single-property range index for nodes
- Create a single-property range index for relationships
- Create a composite range index for nodes
- Create a composite range index for relationships
- Create a range index using a parameter
- Create a range index only if it does not already exist

Create a single-property range index for nodes

The following statement creates a named range index on all nodes labeled with `Person` and which have the `surname` property.

Creating a node range index on a single property

```
CREATE INDEX node_range_index_name FOR (n:Person) ON (n.surname)
```

Create a single-property range index for relationships

The following statement creates a named range index on all relationships with relationship type `KNOWS` and property `since`.

Creating a relationship range index on a single property

```
CREATE INDEX rel_range_index_name FOR ()-[r:KNOWS]-() ON (r.since)
```

Create a composite range index for nodes

A range index on multiple properties is also called a composite index. For node range indexes, only nodes with the specified label and that contain all the specified properties are added to the index.

The following statement creates a named composite range index on all nodes labeled with `Person` and which have both an `age` and `country` property.

Creating a composite node range index on multiple properties

```
CREATE INDEX composite_range_node_index_name FOR (n:Person) ON (n.age, n.country)
```

Create a composite range index for relationships

A range index on multiple properties is also called a composite index. For relationship range indexes, only relationships with the specified type and that contain all the specified properties are added to the index.

The following statement creates a named composite range index on all relationships labeled with `PURCHASED` and which have both a `date` and `amount` property.

Creating a composite relationship range index on multiple properties

```
CREATE INDEX composite_range_rel_index_name FOR ()-[r:PURCHASED]-() ON (r.date, r.amount)
```

Create a range index using a parameter

The following statement creates a named range index on all nodes with a `Person` label and a `firstname` property using a parameter for the index name.

Parameters

```
{
  "name": "range_index_param"
}
```

Creating a node range index on a single property

```
CREATE INDEX $name FOR (n:Person) ON (n.firstname)
```

Create a range index only if it does not already exist

If it is not known whether an index exists or not, add `IF NOT EXISTS` to ensure it does.

Creating a range index with `IF NOT EXISTS`

```
CREATE INDEX node_range_index_name IF NOT EXISTS
FOR (n:Person) ON (n.surname)
```

The index is not created if there already exists an index with the same schema and type, same name or both. Instead an informational notification is returned.

Notification

```
`CREATE RANGE INDEX node_range_index_name IF NOT EXISTS FOR (e:Person) ON (e.surname)` has no effect.
`RANGE INDEX node_range_index_name FOR (e:Person) ON (e.surname)` already exists.
```

Create a text index

Creating a text index can be done with the `CREATE TEXT INDEX` command. Note that the index name must be unique.

Text indexes have no supported index configuration.

See Trigram indexing for information about how text indexes process `STRING` values.

Supported predicates

Text indexes only solve predicates operating on `STRING` values.

The following predicates that only operate on `STRING` values are always solvable by a text index:

- `STARTS WITH`
- `ENDS WITH`
- `CONTAINS`

However, other predicates are only used when it is known that the property is compared to a `STRING`:

- `n.prop = "string"`
- `n.prop IN ["a", "b", "c"]`

This means that a text index is not able to solve, for example, e.g. `a.prop = b.prop`, unless a property type constraint also exists on the property.

Text indexes support the following predicates:

Predicate	Syntax
Equality check.	`n.prop = 'example_string'`
List membership check.	`n.prop IN ['abc', 'example_string', 'neo4j']`
Prefix search.	`STARTS WITH`
Suffix search.	`ENDS WITH`
Substring search.	`CONTAINS`

The above set of predicates can be extended with the use of type constraints. See the section about index compatibility and type constraints for more information.

Text indexes are only used for exact query matches. To match terms within `STRING` values and compute a relevance score, use full-text indexes instead.

Trigram indexing

Text indexes uses trigram indexing. This means that `STRING` values are indexed into overlapping trigrams, each containing three Unicode code points. For example, the word `"developer"` would be indexed by the following trigrams: `["dev", "eve", "vel", "elo", "lop", "ope", "per"]`.

This makes text indexes particularly suitable for substring (`CONTAINS`) and suffix (`ENDS WITH`) searches, as well as prefix searches (`STARTS WITH`). For example, searches like `CONTAINS "vel"` or `ENDS WITH "per"` can be efficiently performed by directly looking up the relevant trigrams in the index. By comparison, range indexes, which indexes `STRING` values lexicographically (see Range index-backed `ORDER BY` for more information) and are therefore more suited for prefix searches, would need to scan through all indexed values to check if `"vel"` existed anywhere within the text. For more information, see The impact of indexes on query performance → Text indexes.

Examples

- Create a node text index
- Create a relationship text index
- Create a text index using a parameter
- Create a text index only if it does not already exist

Create a node text index

The following statement creates a named text index on all nodes labeled with `Person` and which have the `nickname` `STRING` property.

Creating a node text index on a single property

```
CREATE TEXT INDEX node_text_index_nickname FOR (n:Person) ON (n.nickname)
```

Create a relationship text index

The following statement creates a named text index on all relationships with relationship type `KNOWS` and `STRING` property `interest`.

Creating a relationship text index on a single property

```
CREATE TEXT INDEX rel_text_index_name FOR ()-[r:KNOWS]-() ON (r.interest)
```

Create a text index using a parameter

The following statement creates a named text index on all nodes with the `Person` label the `favoriteColor` `STRING` property using a parameter for the index name.

Parameters

```
{
  "name": "text_index_param"
}
```

Creating a node text index on a single property

```
CREATE TEXT INDEX $name FOR (n:Person) ON (n.favoriteColor)
```

Create a text index only if it does not already exist

If it is not known whether an index exists or not, add `IF NOT EXISTS` to ensure it does.

The following statement attempts to create a named text index on all nodes labeled with `Person` and which have the `nickname` `STRING` property.

Creating a text index with `IF NOT EXISTS`

```
CREATE TEXT INDEX node_index_name IF NOT EXISTS FOR (n:Person) ON (n.nickname)
```

Note that the index is not created if there already exists an index with the same schema and type, same name or both. Instead, an informational notification is returned.

Notification

```
`CREATE TEXT INDEX node_index_name IF NOT EXISTS FOR (e:Person) ON (e.nickname)` has no effect.
`TEXT INDEX node_text_index_nickname FOR (e:Person) ON (e.nickname)` already exists.
```

Create a point index

Creating a point index can be done with the `CREATE POINT INDEX` command. Note that the index name must be unique.

Point indexes have supported index configuration.

Supported predicates

Point indexes only solve predicates operating on `POINT` values.

Point indexes support the following predicates:

Predicate	Syntax
Property point value.	`n.prop = point({x: value, y: value})`
Within bounding box.	`point.withinBBox(n.prop, lowerLeftCorner, upperRightCorner)`
Distance.	`point.distance(n.prop, center) < = distance`

The above set of predicates can be extended with the use of type constraints. See The impact of indexes on query performances → Property type constraints for more information.

To learn more about the spatial data types supported by Cypher, see the page about Spatial values.

Examples

- Create a node point index

- Create a relationship point index
- Create a point index using a parameter
- Create a point index only if it does not already exist
- Create a point index specifying the index configuration

Create a node point index

The following statement creates a named point index on all nodes labeled with `Person` and which have the `sublocation` `POINT` property.

Creating a node point index on a single property

```
CREATE POINT INDEX node_point_index_name FOR (n:Person) ON (n.sublocation)
```

Create a relationship point index

The following statement creates a named point index on all relationships with relationship type `STREET` and `POINT` property `intersection`.

Creating a relationship point index on a single property

```
CREATE POINT INDEX rel_point_index_name FOR ()-[r:STREET]-() ON (r.intersection)
```

Create a point index using a parameter

The following statement creates a named point index on all relationships with relationship type `STREET` and `POINT` property `coordinate` using a parameter for the index name.

Parameters

```
{
  "name": "point_index_param"
}
```

Creating a relationship point index on a single property

```
CREATE POINT INDEX $name FOR ()-[r:STREET]-() ON (r.coordinate)
```

Create a point index only if it does not already exist

If it is not known whether an index exists or not, add `IF NOT EXISTS` to ensure it does.

Creating a point index with `IF NOT EXISTS`

```
CREATE POINT INDEX node_point_index IF NOT EXISTS
FOR (n:Person) ON (n.sublocation)
```

Note that the index is not created if there already exists an index with the same schema and type, same name or both. Instead, an informational notification is returned.

Notification

```
`CREATE POINT INDEX node_point_index IF NOT EXISTS FOR (e:Person) ON (e.sublocation)` has no effect.
`POINT INDEX node_point_index_name FOR (e:Person) ON (e.sublocation)` already exists.
```

Create a point index specifying the index configuration

To create a point index with a specific index configuration, the `indexConfig` settings in the `OPTIONS` clause. The valid configuration settings are:

- `spatial.cartesian.min` (default value: [`-1000000.0`, `-1000000.0`])
- `spatial.cartesian.max` (default value: [`1000000.0`, `1000000.0`])
- `spatial.cartesian-3d.min` (default value: [`-1000000.0`, `-1000000.0`, `-1000000.0`])
- `spatial.cartesian-3d.max` (default value: [`1000000.0`, `1000000.0`, `1000000.0`])
- `spatial.wgs-84.min` (default value: [`-180.0`, `-90.0`])
- `spatial.wgs-84.max` (default value: [`180.0`, `90.0`])
- `spatial.wgs-84-3d.min` (default value: [`-180.0`, `-90.0`, `-1000000.0`])
- `spatial.wgs-84-3d.max` (default value: [`180.0`, `90.0`, `1000000.0`])

The following statement creates a point index specifying the `spatial.cartesian.min` and `spatial.cartesian.max` settings.

Creating a point index with index configuration

```
CREATE POINT INDEX point_index_with_config
FOR (n:Label) ON (n.prop2)
OPTIONS {
  indexConfig: {
    `spatial.cartesian.min`: [-100.0, -100.0],
    `spatial.cartesian.max`: [100.0, 100.0]
  }
}
```

Note that the wgs-84 and 3D cartesian settings, which are not specified in this example, are set with their respective default values.

Create a token lookup index

Two token lookup indexes are created by default when creating a Neo4j database (one node label lookup index and one relationship type lookup index). Only one node label and one relationship type lookup index can exist at the same time.

If a token lookup index has been dropped, it can be recreated with the `CREATE LOOKUP INDEX` command. Note that the index name must be unique.

Token lookup indexes have no supported index configuration.

Supported predicates

Token lookup indexes are present by default and solve only node label and relationship type predicates:

Predicate	Syntax (example)

Node label predicate.	`MATCH (n:Label)`
	
	`MATCH (n)`
	`WHERE n:Label`

Relationship type predicate.	`MATCH ()-[r:REL]->()`
	
	`MATCH ()-[r]->()`
	`WHERE r:REL`

Warning:

Token lookup indexes improve the performance of Cypher queries and the population of other indexes. Dropping these indexes may lead to severe performance degradation.

Examples

- Create a node label lookup index
- Create a relationship type lookup index
- Create a token lookup index only if it does not already exist

Create a node label lookup index

The following statement creates a named node label lookup index on all nodes with one or more labels:

Creating a node label lookup index

```
CREATE LOOKUP INDEX node_label_lookup_index FOR (n) ON EACH labels(n)
```

Note:

Only one node label lookup index can exist at a time.

Create a relationship type lookup index

The following statement creates a named relationship type lookup index on all relationships with any relationship type.

Creating a relationship type lookup index

```
CREATE LOOKUP INDEX rel_type_lookup_index FOR ()-[r]-() ON EACH type(r)
```

Note:

Only one relationship type lookup index can exist at a time.

Create a token lookup index only if it does not already exist

If it is not known whether an index exists or not, add `IF NOT EXISTS` to ensure it does.

Creating a node label lookup index with `IF NOT EXISTS`

```
CREATE LOOKUP INDEX node_label_lookup IF NOT EXISTS FOR (n) ON EACH labels(n)
```

The index is not created if there already exists an index with the same schema and type, same name or both. Instead, an informational notification is returned.

Notification

```
`CREATE LOOKUP INDEX node_label_lookup IF NOT EXISTS FOR (e) ON EACH labels(e)` has no effect.
`LOOKUP INDEX node_label_lookup_index FOR (e) ON EACH labels(e)` already exists.
```

Creating an index when a conflicting index or constraint exists

- Failure to create an already existing index
- Failure to create an index with the same name as an already existing index
- Failure to create an index when a constraint already exists
- Failure to create an index with the same name as an already existing constraint

Failure to create an already existing index

Create an index on the property `title` on nodes with the `Book` label, when that index already exists.

Creating a duplicate index

```
CREATE INDEX bookTitleIndex FOR (book:Book) ON (book.title)
```

In this case, the index can not be created because it already exists.

GQLSTATUS error chain

```
22N70: error: data exception - equivalent index already exists. An equivalent index already exists: '(:Book {title})'
```

Using `IF NOT EXISTS` when creating the index would result in no error and would not create a new index.

Failure to create an index with the same name as an already existing index

Create a named index on the property `numberOfPages` on nodes with the `Book` label, when an index with the given name already exists. The index type of the existing index does not matter.

Creating an index with a duplicated name

```
CREATE INDEX indexOnBooks FOR (book:Book) ON (book.numberOfPages)
```

In this case, the index cannot be created because there already exists an index with the given name.

GQLSTATUS error chain

22N71: error: data exception - duplicated index name. An index with the same name already exists: `indexOnBooks`

Using `IF NOT EXISTS` when creating the index would result in no error and would not create a new index.

Failure to create an index when a constraint already exists

Create an index on the property `isbn` on nodes with the `Book` label, when an index-backed constraint already exists on that schema. This is only relevant for range indexes.

Creating a range index on same schema as existing index-backed constraint

```
CREATE INDEX bookIsbnIndex FOR (book:Book) ON (book.isbn)
```

In this case, the index can not be created because an index-backed constraint already exists on that label and property combination.

GQLSTATUS error chain

22N70: error: data exception - equivalent index already exists. An equivalent index already exists: '(:Book {isbn})'

22N74: error: data exception - index conflicts with existing constraint. Index conflicts with already existing index owned by constraint `uniqueBookIsbn`.

Failure to create an index with the same name as an already existing constraint

Create a named index on the property `numberOfPages` on nodes with the `Book` label, when a constraint with the given name already exists.

Creating an index with same name as an existing constraint

```
CREATE INDEX bookRecommendations FOR (book:Book) ON (book.recommendations)
```

In this case, the index can not be created because there already exists a constraint with the given name.

GQLSTATUS error chain

22N67: error: data exception - duplicated constraint name. A constraint with the same name already exists: `bookRecommendations`

Show indexes

Showing indexes can be done with `SHOW INDEXES`. For general information about the `SHOW` command, see the `SHOW` page.

> **Note:**

Showing indexes requires the `SHOW INDEX` privilege.

Examples

- Show all indexes
- Return specific columns
- Show indexes with filtering
- Combine `SHOW INDEXES` with other Cypher clauses

Show all indexes

To show all indexes with the default output columns, the `SHOW INDEXES` command can be used. If all columns are required, use `SHOW INDEXES YIELD *`.

Show all indexes

```
SHOW INDEXES
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
----+
| id | name                              | state    | populationPercent | type     | entityType     |
labelsOrTypes | properties         | indexProvider      | owningConstraint | lastRead                 |
readCount |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
----+
| 3  | "composite_range_node_index_name" | "ONLINE" | 100.0             | "RANGE"  | "NODE"         |
["Person"]    | ["age", "country"] | "range-1.0"        | NULL              | NULL                       | 0
|
| 4  | "composite_range_rel_index_name"  | "ONLINE" | 100.0             | "RANGE"  | "RELATIONSHIP" |
["PURCHASED"] | ["date", "amount"] | "range-1.0"        | NULL              | 2023-03-13T11:41:44.537Z | 1
|
| 16 | "example_index"                   | "ONLINE" | 100.0             | "RANGE"  | "NODE"         |
["Book"]      | ["title"]          | "range-1.0"        | NULL              | 2023-04-10T15:41:44.537Z | 2
|
| 17 | "indexOnBooks"                    | "ONLINE" | 100.0             | "TEXT"   | "NODE"         |
["Label1"]    | ["prop1"]          | "text-2.0"         | NULL              | NULL                       | 0
|
| 14 | "node_label_lookup_index"         | "ONLINE" | 100.0             | "LOOKUP" | "NODE"         | NULL
| NULL               | "token-lookup-1.0" | NULL              | 2023-04-13T08:11:15.537Z | 10         |
| 10 | "node_point_index_name"           | "ONLINE" | 100.0             | "POINT"  | "NODE"         |
["Person"]    | ["sublocation"]    | "point-1.0"        | NULL              | 2023-04-05T16:21:44.692Z | 1
|
| 1  | "node_range_index_name"           | "ONLINE" | 100.0             | "RANGE"  | "NODE"         |
["Person"]    | ["surname"]        | "range-1.0"        | NULL              | 2022-12-30T02:01:44.537Z | 6
|
| 6  | "node_text_index_nickname"        | "ONLINE" | 100.0             | "TEXT"   | "NODE"         |
["Person"]    | ["nickname"]       | "text-2.0"         | NULL              | 2023-04-13T11:41:44.537Z | 2
|
| 12 | "point_index_param"               | "ONLINE" | 100.0             | "POINT"  | "RELATIONSHIP" |
["STREET"]    | ["coordinate"]     | "point-1.0"        | NULL              | NULL                       | 0
|
| 13 | "point_index_with_config"         | "ONLINE" | 100.0             | "POINT"  | "NODE"         |
["Label"]     | ["prop2"]          | "point-1.0"        | NULL              | NULL                       | 0
|
| 5  | "range_index_param"               | "ONLINE" | 100.0             | "RANGE"  | "NODE"         |
["Person"]    | ["firstname"]      | "range-1.0"        | NULL              | 2023-12-13T08:23:53.338Z | 2
|
| 11 | "rel_point_index_name"            | "ONLINE" | 100.0             | "POINT"  | "RELATIONSHIP" |
["STREET"]    | ["intersection"]   | "point-1.0"        | NULL              | 2023-03-03T13:37:42.537Z | 2
|
| 2  | "rel_range_index_name"            | "ONLINE" | 100.0             | "RANGE"  | "RELATIONSHIP" |
```

```
["KNOWS"]     | ["since"]          | "range-1.0"        | NULL             | 2023-04-12T10:41:44.692Z | 5
|
| 7  | "rel_text_index_name"             | "ONLINE" | 100.0             | "TEXT"   | "RELATIONSHIP" |
["KNOWS"]     | ["interest"]       | "text-2.0"         | NULL             | 2023-04-01T10:40:44.537Z | 3
|
| 15 | "rel_type_lookup_index"           | "ONLINE" | 100.0             | "LOOKUP" | "RELATIONSHIP" | NULL
| NULL               | "token-lookup-1.0" | NULL             | 2023-04-12T21:41:44.537Z | 7         |
| 8  | "text_index_param"                | "ONLINE" | 100.0             | "TEXT"   | "NODE"         |
["Person"]    | ["favoriteColor"]  | "text-2.0"         | NULL             | NULL                       | 0
|
| 18 | "uniqueBookIsbn"                  | "ONLINE" | 100.0             | "RANGE"  | "NODE"         |
["Book"]      | ["isbn"]           | "range-1.0"        | "uniqueBookIsbn" | 2023-04-13T11:41:44.692Z | 6
|
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
----+
18 rows
```

One of the output columns from `SHOW INDEXES` is the name of the index. This can be used to drop the index with the `DROP INDEX` command.

Return specific columns

It is possible to return only specific columns of the available indexes using the `YIELD` clause:

Returning specific columns for all indexes

```
SHOW INDEXES
YIELD name, type, indexProvider AS provider, options, createStatement
RETURN name, type, provider, options.indexConfig AS config, createStatement
```

Result

```
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-----------------------------------------+
| name                              | type     | provider           | config
| createStatement
|
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-----------------------------------------+
| "composite_range_node_index_name" | "RANGE"  | "range-1.0"        | {}
| "CREATE RANGE INDEX `composite_range_node_index_name` FOR (n:`Person`) ON (n.`age`, n.`country`)"
|
| "composite_range_rel_index_name"  | "RANGE"  | "range-1.0"        | {}
| "CREATE RANGE INDEX `composite_range_rel_index_name` FOR ()-[r:`PURCHASED`]-() ON (r.`date`,
r.`amount`)"
|
| "example_index"                   | "RANGE"  | "range-1.0"        | {}
| "CREATE RANGE INDEX `example_index` FOR (n:`Book`) ON (n.`title`)"
|
| "indexOnBooks"                    | "TEXT"   | "text-2.0"         | {}
| "CREATE TEXT INDEX `indexOnBooks` FOR (n:`Label1`) ON (n.`prop1`)"
|
| "index_343aff4e"                  | "LOOKUP" | "token-lookup-1.0" | {}
| "CREATE LOOKUP INDEX `index_343aff4e` FOR (n) ON EACH labels(n)"
```

```
|
| "index_f7700477"                  | "LOOKUP" | "token-lookup-1.0" | {}
| "CREATE LOOKUP INDEX `index_f7700477` FOR ()-[r]-() ON EACH type(r)"
|
| "node_point_index_name"           | "POINT"  | "point-1.0"        | {`spatial.cartesian.min`: [-
1000000.0, -1000000.0], `spatial.wgs-84.min`: [-180.0, -90.0], `spatial.wgs-84.max`: [180.0, 90.0],
`spatial.cartesian.max`: [1000000.0, 1000000.0], `spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],
`spatial.cartesian-3d.min`: [-1000000.0, -1000000.0, -1000000.0], `spatial.cartesian-3d.max`: [1000000.0,
1000000.0, 1000000.0], `spatial.wgs-84-3d.min`: [-180.0, -90.0, -1000000.0]} | "CREATE POINT INDEX
`node_point_index_name` FOR (n:`Person`) ON (n.`sublocation`) OPTIONS {indexConfig: {`spatial.cartesian-
3d.max`: [1000000.0, 1000000.0, 1000000.0],`spatial.cartesian-3d.min`: [-1000000.0, -1000000.0,
-1000000.0],`spatial.cartesian.max`: [1000000.0, 1000000.0],`spatial.cartesian.min`: [-1000000.0,
-1000000.0],`spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],`spatial.wgs-84-3d.min`: [-180.0, -90.0,
-1000000.0],`spatial.wgs-84.max`: [180.0, 90.0],`spatial.wgs-84.min`: [-180.0, -90.0]}}"       |
| "node_range_index"                | "RANGE"  | "range-1.0"        | {}
| "CREATE RANGE INDEX `node_range_index` FOR (n:`Person`) ON (n.`surname`)"
|
| "node_text_index_nickname"        | "TEXT"   | "text-2.0"         | {}
| "CREATE TEXT INDEX `node_text_index_nickname` FOR (n:`Person`) ON (n.`nickname`)"
|
| "point_index_with_config"         | "POINT"  | "point-1.0"        | {`spatial.cartesian.min`: [-100.0,
-100.0], `spatial.wgs-84.min`: [-180.0, -90.0], `spatial.wgs-84.max`: [180.0, 90.0],
`spatial.cartesian.max`: [100.0, 100.0], `spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],
`spatial.cartesian-3d.min`: [-1000000.0, -1000000.0, -1000000.0], `spatial.cartesian-3d.max`: [1000000.0,
1000000.0, 1000000.0], `spatial.wgs-84-3d.min`: [-180.0, -90.0, -1000000.0]}                 | "CREATE
POINT INDEX `point_index_with_config` FOR (n:`Label`) ON (n.`prop2`) OPTIONS {indexConfig:
{`spatial.cartesian-3d.max`: [1000000.0, 1000000.0, 1000000.0],`spatial.cartesian-3d.min`: [-1000000.0,
-1000000.0, -1000000.0],`spatial.cartesian.max`: [100.0, 100.0],`spatial.cartesian.min`: [-100.0,
-100.0],`spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],`spatial.wgs-84-3d.min`: [-180.0, -90.0,
-1000000.0],`spatial.wgs-84.max`: [180.0, 90.0],`spatial.wgs-84.min`: [-180.0, -90.0]}}"
|
| "rel_point_index_name"            | "POINT"  | "point-1.0"        | {`spatial.cartesian.min`: [-
1000000.0, -1000000.0], `spatial.wgs-84.min`: [-180.0, -90.0], `spatial.wgs-84.max`: [180.0, 90.0],
`spatial.cartesian.max`: [1000000.0, 1000000.0], `spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],
`spatial.cartesian-3d.min`: [-1000000.0, -1000000.0, -1000000.0], `spatial.cartesian-3d.max`: [1000000.0,
1000000.0, 1000000.0], `spatial.wgs-84-3d.min`: [-180.0, -90.0, -1000000.0]} | "CREATE POINT INDEX
`rel_point_index_name` FOR ()-[r:`STREET`]-() ON (r.`intersection`) OPTIONS {indexConfig:
{`spatial.cartesian-3d.max`: [1000000.0, 1000000.0, 1000000.0],`spatial.cartesian-3d.min`: [-1000000.0,
-1000000.0, -1000000.0],`spatial.cartesian.max`: [1000000.0, 1000000.0],`spatial.cartesian.min`: [-
1000000.0, -1000000.0],`spatial.wgs-84-3d.max`: [180.0, 90.0, 1000000.0],`spatial.wgs-84-3d.min`: [-180.0,
-90.0, -1000000.0],`spatial.wgs-84.max`: [180.0, 90.0],`spatial.wgs-84.min`: [-180.0, -90.0]}}" |
| "rel_range_index_name"            | "RANGE"  | "range-1.0"        | {}
| "CREATE RANGE INDEX `rel_range_index_name` FOR ()-[r:`KNOWS`]-() ON (r.`since`)"
|
| "rel_text_index_name"             | "TEXT"   | "text-2.0"         | {}
| "CREATE TEXT INDEX `rel_text_index_name` FOR ()-[r:`KNOWS`]-() ON (r.`interest`)"
|
| "uniqueBookIsbn"                  | "RANGE"  | "range-1.0"        | {}
| "CREATE CONSTRAINT `uniqueBookIsbn` FOR (n:`Book`) REQUIRE (n.`isbn`) IS UNIQUE"
|
+-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------
-------------------------------------------+
```

Note that `YIELD` is mandatory if the `RETURN` clause is used. `RETURN` is not, however, mandatory when the YIELD clause is used.

Show indexes with filtering

The `SHOW INDEX` command can be filtered in various ways.

For example, to show only range indexes, use `SHOW RANGE INDEXES`.

Another more flexible way of filtering the output is to use the `WHERE` clause. An example is to only show indexes not belonging to constraints.

To show only range indexes that does not belong to a constraint we can combine the filtering versions.

Show range indexes

```
SHOW RANGE INDEXES WHERE owningConstraint IS NULL
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------+
| id | name                              | state    | populationPercent | type    | entityType     |
labelsOrTypes | properties         | indexProvider | owningConstraint | lastRead                 |
readCount |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------+
| 3  | "composite_range_node_index_name" | "ONLINE" | 100.0             | "RANGE" | "NODE"         |
["Person"]    | ["age", "country"] | "range-1.0"   | NULL             | NULL                     | 0
|
| 4  | "composite_range_rel_index_name"  | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" |
["PURCHASED"] | ["date", "amount"] | "range-1.0"   | NULL             | 2023-03-13T11:41:44.537Z | 1
|
| 16 | "example_index"                   | "ONLINE" | 100.0             | "RANGE" | "NODE"         |
["Book"]      | ["title"]          | "range-1.0"   | NULL             | 2023-04-10T15:41:44.537Z | 2
|
| 1  | "node_range_index_name"           | "ONLINE" | 100.0             | "RANGE" | "NODE"         |
["Person"]    | ["surname"]        | "range-1.0"   | NULL             | 2022-12-30T02:01:44.537Z | 6
|
| 5  | "range_index_param"               | "ONLINE" | 100.0             | "RANGE" | "NODE"         |
["Person"]    | ["firstname"]      | "range-1.0"   | NULL             | 2023-12-13T08:23:53.338Z | 2
|
| 2  | "rel_range_index_name"            | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" |
["KNOWS"]     | ["since"]          | "range-1.0"   | NULL             | 2023-04-12T10:41:44.692Z | 5
|
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------+
6 rows
```

This returns only the default output columns.

To get all columns, use:

```
SHOW RANGE INDEXES YIELD * WHERE owningConstraint IS NULL
```

Combine `SHOW INDEXES` with other Cypher clauses

Cypher 25 only | Introduced in Neo4j 2026.05

The `SHOW INDEXES` command can be combined with general Cypher clauses in a single query.

For example, returning a string of drop index commands for all indexes except token lookup indexes and indexes backing constraints.

Return commands to drop all indexes as a single string

```
SHOW INDEXES
YIELD name, type, owningConstraint
WHERE type <> 'LOOKUP' AND owningConstraint IS NULL
WITH "DROP INDEX `" + name + "`;" AS dropCommand
WITH collect(dropCommand) AS dropCommands
RETURN reduce(acc = head(dropCommands), dc IN tail(dropCommands) | acc + " " + dc) AS result
```

Result

```
+-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------+
| result
|
+-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------+
| "DROP INDEX `composite_range_node_index_name`; DROP INDEX `composite_range_rel_index_name`; DROP INDEX
`example_index`; DROP INDEX `indexOnBooks`; DROP INDEX `node_point_index_name`; DROP INDEX
`node_range_index_name`; DROP INDEX `node_text_index_nickname`; DROP INDEX `point_index_param`; DROP INDEX
`point_index_with_config`; DROP INDEX `range_index_param`; DROP INDEX `rel_point_index_name`; DROP INDEX
`rel_range_index_name`; DROP INDEX `rel_text_index_name`; DROP INDEX `text_index_param`;" |
+-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------+
1 row
```

> **Note:**
>
> When combining `SHOW INDEXES` with other clauses, the `YIELD` clause is mandatory and must not be omitted. `YIELD` must explicitly list the yielded columns. `YIELD *` is not permitted. The query must also end with a valid last clause (like a `RETURN` or an updating clause).

Return columns

The below table contains the full information about all columns returned by the `SHOW INDEXES YIELD *` command. Any returned statistics are local to each instance and may differ for each member in a cluster.

Show indexes output

Column	Description	Type
`id`	The id of the index. Default Output	`INTEGER`
`name`	Name of the index (explicitly set by the user or automatically assigned). Default Output	`STRING`
`state`	Current state of the index. Default Output	`STRING`
`populationPercent`	% of index population. Default Output	`FLOAT`
`type`	The IndexType of this index (`FULLTEXT`, `LOOKUP`, `POINT`, `RANGE`, `TEXT`, or `VECTOR`). Default Output	`STRING`
`entityType`	Type of entities this index represents (`NODE` or `RELATIONSHIP`). Default Output	`STRING`
`labelsOrTypes`	The labels or relationship types of this index. Default Output	`LIST<STRING>`

Column	Description	Type
`properties`	The properties of this index. For vector indexes the vector property is the first element of the list, followed by the additional properties for filtering. Default Output	`LIST<STRING>`
`indexProvider`	The index provider for this index. Default Output	`STRING`
`owningConstraint`	The name of the constraint the index is associated with or `null` if the index is not associated with any constraint. Default Output	`STRING`
`lastRead`	The last time the index was used for reading. Returns `null` if the index has not been read since `trackedSince`, or if the statistics are not tracked. Default Output	`ZONED DATETIME`
`readCount`	The number of read queries that have been issued to this index since `trackedSince`, or `null` if the statistics are not tracked. Default Output	`INTEGER`
`trackedSince`	The time when usage statistics tracking started for this index, or `null` if the statistics are not tracked.	`ZONED DATETIME`
`options`	Information retrieved from the `OPTIONS` map about the configuration settings for an index. If neither is specified when creating the index, this column returns the default values.	`MAP`
`failureMessage`	The failure description of a failed index.	`STRING`
`createStatement`	Statement used to create the index, or `null` if the index cannot be created through `CREATE INDEX` (for example in case of version incompatibilities).	`STRING`

Drop indexes

An index can be dropped (removed) using the name with the `DROP INDEX index_name` command. This command can drop indexes of any type, except those backing constraints.

It is possible to drop indexes created in a different Cypher version than the one in use. For example, although `VECTOR` indexes on multiple labels or with additional properties were added in Cypher 25, such indexes can still be dropped using Cypher 5.

The name of the index can be found using the `SHOW INDEXES` command, given in the output column `name`.

```
DROP INDEX index_name [IF EXISTS]
```

The `DROP INDEX` command is optionally idempotent. This means that its default behavior is to throw an error if an attempt is made to drop the same index twice. With `IF EXISTS`, no error is thrown and nothing happens should the index not exist. Instead, an informational notification is returned detailing that the

index does not exist.

> **Note:**
>
> Dropping an index requires the `DROP INDEX` privilege.

Examples

- Drop an index
- Drop an index using a parameter
- Failure to drop an index backing a constraint
- Drop a non-existing index

Drop an index

The following statement attempts to drop the index named `example_index`.

Dropping an index

```
DROP INDEX example_index
```

If an index with that name exists it is removed, if not the command fails.

Drop an index using a parameter

The following statement attempts to drop the index named `range_index_param` using a parameter for the index name.

Parameters

```
{
  "name": "range_index_param"
}
```

Dropping an index

```
DROP INDEX $name
```

If an index with that name exists it is removed, if not the command fails.

Failure to drop an index backing a constraint

It is not possible to drop indexes that back constraints.

Dropping an index backing a constraint

```
DROP INDEX uniqueBookIsbn
```

GQLSTATUS error chain

```
22NBC: error: data exception - index belongs to constraint. Index belongs to constraint 'uniqueBookIsbn'.

50N10: error: general processing exception - index drop failed. Unable to drop 'uniqueBookIsbn'.
```

Dropping the index-backed constraint also removes the backing index. For more information, see Drop a constraint by name.

Drop a non-existing index

If it is uncertain if an index exists and you want to drop it if it does but not get an error should it not, use `IF EXISTS`.

The following statement attempts to drop the index named `missing_index_name`.

Dropping an index with `IF EXISTS`

```
DROP INDEX missing_index_name IF EXISTS
```

If an index with that name exists it is removed, if not the command does nothing and an informational notification is instead returned.

Notification

```
`DROP INDEX missing_index_name IF EXISTS` has no effect. `missing_index_name` does not exist.
```

The impact of indexes on query performance

Performance indexes enable quicker and more efficient pattern matching by solving a particular combination of node label/relationship type and property predicate. They are used automatically by the Cypher planner in `MATCH` clauses, usually at the start of a query, to scan the graph for the most appropriate place to start the pattern-matching process.

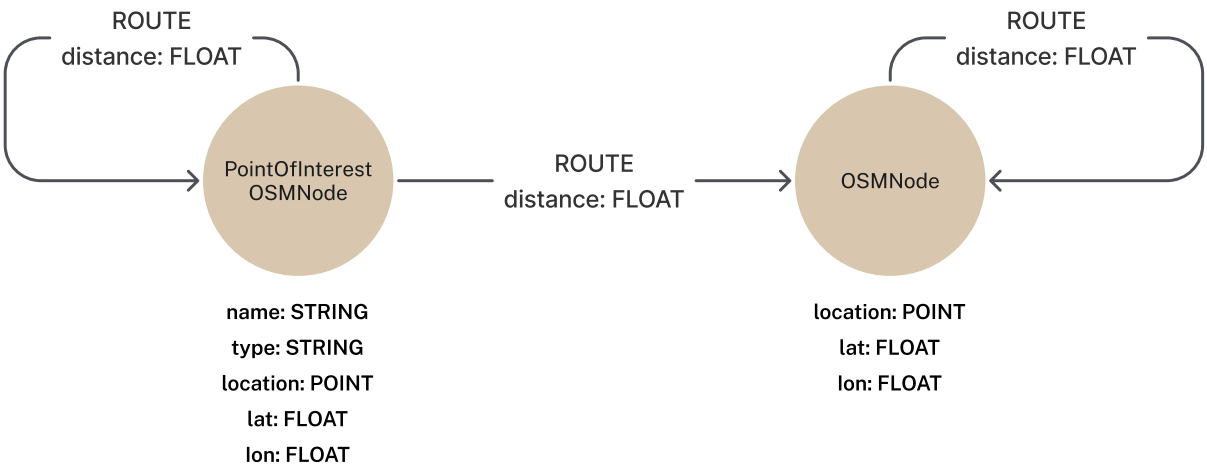
By examining query execution plans, this page will explain the scenarios in which the various performance indexes are used to improve the performance of Cypher queries. It will also provide some general heuristics for when to use indexes, and advice about how to avoid over-indexing.

Example graph

The examples on this page center around finding routes and points of interest in Central Park, New York, based on data provided by OpenStreetMap. The data model contains two node labels:

- `OSMNode` (Open Street Map Node) — a junction node with geo-spatial properties linking together routes from specific points.
- `PointOfInterest` — a subcategory of `OSMNode`. In addition to geospatial properties, these nodes also contain information about specific points of interest, such as statues, baseball courts, etc. in Central Park.

The data model also contains one relationship type: `ROUTE`, which specifies the distance in meters between the nodes in the graph.



In total, the graph contains 69165 nodes (of which 188 have the label `PointOfInterest`) and 152077 `ROUTE` relationships.

To recreate the graph, download and import the 5.0 dump file to an empty Neo4j database. Dump files can be imported for both on-prem instances.

Token lookup indexes

Two token lookup indexes are present by default when creating a Neo4j database. They store copies of all node labels and relationship types in the database and only solve node label and relationship type predicates.

The following query [1], which counts the number of `PointOfInterest` nodes that have value `baseball` for the `type` property, will access the node label lookup index:

Query

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.type = 'baseball'
RETURN count(n)
```

Result

count(n)
26
Rows: 1

Execution plan

```
+-------------------+----+-----------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| Operator          | Id | Details               | Estimated Rows | Rows | DB Hits | Memory (Bytes) |
Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------+----+-----------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| +ProduceResults   |  0 | `count(n)`            |              1 |    1 |       0 |              0 |
0/0 |     0.075 | In Pipeline 1       |
| |                 +----+-----------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| +EagerAggregation |  1 | count(n) AS `count(n)` |              1 |    1 |       0 |             32 |
|           |                    |
```

```
| |                +----+-----------------------+---------------+------+---------+---------------+
|           |                     |
| +Filter           |  2 | n.type = $autostring_0 |             9 |   26 |     376 |               |
|           |                     |
| |                +----+-----------------------+---------------+------+---------+---------------+
|           |                     |
| +NodeByLabelScan  |  3 | n:PointOfInterest      |           188 |  188 |     189 |           376 |
116/0 |     8.228 | Fused in Pipeline 0 |
+------------------+----+-----------------------+---------------+------+---------+---------------
+----------------------+-----------+--------------------+

Total database accesses: 565, total allocated memory: 472
```

The following details are worth highlighting in the execution plan:

- The `NodeByLabelScan` operator accesses the node label lookup index and produces 188 rows, representing the 188 nodes with the `PointOfInterest` label in the database.
- The query required 565 DB hits (each DB hit represents an instance when the query required access to the database).
- The query completed in just over 8 milliseconds.

Token lookup indexes are very important because they improve the performance of Cypher queries and the population of other indexes, and **deleting them will lead to severe performance degradation**. In the above example, had a node label lookup index not existed, the `NodeByLabelScan` operator would have been replaced with `AllNodesScan`, which would have had to read all 69165 nodes from the database before returning a result.

While useful, token lookup indexes will rarely be sufficient for applications querying databases of a non-trivial size because they cannot solve any property-related predicates.

For more information about the predicates supported by token lookup indexes, see Create indexes → Create a token lookup index → Supported predicates.

Range indexes

Range indexes solve most types of predicates, and they are used for efficiently retrieving data based on a range of values. They are particularly useful for dealing with properties that have ordered, comparable values.

The following example first creates a relevant index on the type property for `PointOfInterest` nodes, and then runs the above query again, counting the number of `PointOfInterest` nodes that have a `baseball` `type` value:

Create a range index

```
CREATE INDEX range_index_type FOR (n:PointOfInterest) ON (n.type)
```

> If no index type is specified when creating an index, Neo4j will default to create a range index. For more information about creating indexes, see Create indexes.

Rerun query after the creation of a relevant index

```
PROFILE
```

```
MATCH (n:PointOfInterest)
WHERE n.type = 'baseball'
RETURN count(n)
```

Execution plan

```
+-------------------+----+-----------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator          | Id | Details                                                                | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+-----------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults   |  0 | `count(n)`                                                             |              1
|    1 |       0 |              0 |                    0/0 |     0.057 | In Pipeline 1       |
| |                 +----+-------------------------------------------------------------------
+-------------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +EagerAggregation |  1 | count(n) AS `count(n)`                                                 |              1
|    1 |       0 |             32 |                      |           |                     |
| |                 +----+-------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexSeek    |  2 | RANGE INDEX n:PointOfInterest(type) WHERE type = $autostring_0 |              5
|   26 |      27 |            376 |                    0/1 |     0.945 | Fused in Pipeline 0 |
+-------------------+----+-----------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 27, total allocated memory: 472
```

Comparing this query plan with the plan generated before the creation of a relevant range index, the following has changed:

- NodeByLabelScan has been replaced by NodeIndexSeek. This only produces 26 rows (representing the 26 `PointOfInterest` nodes in the database with a `type` value set to `baseball`).
- The query now only requires 27 DB hits.
- The query completed in less than 1 millisecond - almost 8 times faster than it took the query to complete without a range index.

These points all illustrate the fundamental point that performance indexes can significantly improve the performance of Cypher queries.

For more information about the predicates supported by range indexes, see Create indexes → Create a range index → Supported predicates.

Text indexes

Text indexes are used for queries filtering on `STRING` properties.

If there exists both a range and a text index on a given `STRING` property, the text index will only be used by the Cypher planner for queries filtering with the `CONTAINS` or `ENDS WITH` operators. In all other cases, the range index will be used.

To show this behavior, it is necessary to create a text index and a range index on the same property:

Create a text index

```
CREATE TEXT INDEX text_index_name FOR (n:PointOfInterest) ON (n.name)
```

Create a range index

```
CREATE INDEX range_index_name FOR (n:PointOfInterest) ON (n.name)
```

The following query filters all `PointOfInterest` nodes with a `name` property that `CONTAINS` `'William'`:

Query filtering on what a `STRING` property `CONTAINS`

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name CONTAINS 'William'
RETURN n.name AS name, n.type AS type
```

Result

name	type
"William Shakespeare"	"statue"
"William Tecumseh Sherman"	"equestrian statue"
Rows: 2	

Execution plan

```
+-----------------------+----+-----------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator              | Id | Details                                                                    |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------+----+-----------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults       |  0 | name, type                                                                 |
1 |    2 |       0 |             0 |                       |           |                       |
| |                     +----+----------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +Projection           |  1 | cache[n.name] AS name, cache[n.type] AS type                               |
1 |    2 |       0 |               |                       |           |                       |
| |                     +----+----------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +CacheProperties      |  2 | cache[n.type], cache[n.name]                                               |
1 |    2 |       6 |               |                       |           |                       |
| |                     +----+----------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +NodeIndexContainsScan |  3 | TEXT INDEX n:PointOfInterest(name) WHERE name CONTAINS $autostring_0 |
1 |    2 |       3 |           248 |                   4/0 |    53.297 | Fused in Pipeline 0 |
+-----------------------+----+-----------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 9, total allocated memory: 312
```

The plan shows that the query uses the text index to find all relevant nodes. If, however, the query is changed to use the `STARTS WITH` operator instead of `CONTAINS`, the query will use the range index instead:

Query filtering on what a `STRING` property `STARTS WITH`

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name STARTS WITH 'William'
RETURN n.name, n.type
```

Execution plan

```
+-----------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator              | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults       |  0 | `n.name`, `n.type`
|              1 |    2 |       0 |              0 |                        |           |
|
| |                     +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Projection           |  1 | cache[n.name] AS `n.name`, n.type AS `n.type`
|              1 |    2 |       4 |                |                        |           |
|
| |                     +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexSeekByRange |  2 | RANGE INDEX n:PointOfInterest(name) WHERE name STARTS WITH $autostring_0,
cache[n.name] |              1 |    2 |       3 |            248 |                    4/1 |     1.276 |
Fused in Pipeline 0 |
+-----------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 7, total allocated memory: 312
```

This is because range indexes store `STRING` values alphabetically. This means that, while they are very efficient for retrieving exact matches of a `STRING`, or for prefix matching, they are less efficient for suffix and contains searches, where they have to scan all relevant properties to filter any matches. Text indexes do not store `STRING` properties alphabetically, and are instead optimized for suffix and contains searches (for more information, see Create a text index → Trigram indexing). That said, if no range index had been present on the name property, the previous query would still have been able to utilize the text index. It would have done so less efficiently than a range index, but it still would have been useful.

For more information about range index ordering, see the section on Range index-backed ORDER BY.

Text indexes are only used for exact query matches. To match terms within `STRING` values and compute a relevance score, use full-text indexes instead.

For more information about the predicates supported by text indexes, see Create indexes → Create a text index → Supported predicates.

Ensuring text index use

In order for the planner to use text indexes, it must be able to confirm that the properties included in the predicate are `STRING` values. This is not possible when accessing property values within nodes or relationships, or values within a `MAP`, since Cypher does not store the type information of these values. To ensure text indexes are used in these cases, the `toString` function should be used.

Text index not used

```
WITH {name: 'William Shakespeare'} AS varName
MERGE (:PointOfInterest {name:varName.name})
```

Text index used

```
WITH {name: 'William Shakespeare'} AS varName
MERGE (:PointOfInterest {name: toString(varName.name)})
```

For information about how to ensure the use of text indexes when predicates may contain `null` values, see Indexes and `null` values.

Text indexes and `STRING` sizes

The size of the indexed `STRING` properties is also relevant to the planner's selection between range and text indexes.

Range indexes have a maximum key size limit of around 8 kb. This means that range indexes cannot be used to index `STRING` values larger than 8 kb. Text indexes, on the other hand, have a maximum key size limit of around 32 kb. As a result, they can be used to index `STRING` values up to that size.

For information about calculating the size of indexes, see Neo4j Knowledge Base → A method to calculate the size of an index in Neo4j.

Point indexes

Point indexes solve predicates operating on spatial `POINT` values. Point indexes are optimized for queries filtering for the distance between property values, or for property values within a bounding box.

The following example creates a point index which is then used in a query returning the `name` and `type` of all `PointOfInterest` nodes within a set bounding box:

Create a point index

```
CREATE POINT INDEX point_index_location FOR (n:PointOfInterest) ON (n.location)
```

Query using the `point.withinBBox()` function

```
PROFILE
MATCH (n:PointOfInterest)
WHERE point.withinBBox(
  n.location,
  point({srid: 4326, x: -73.9723702, y: 40.7697989}),
  point({srid: 4326, x: -73.9725659, y: 40.770193}))
RETURN n.name AS name, n.type AS type
```

Result

name	type
"Heckscher Ballfield 3"	"baseball"
"Heckscher Ballfield 4"	"baseball"
"Heckscher Ballfield 1"	"baseball"
"Robert Burns"	"statue"
"Christopher Columbus"	"statue"
"Walter Scott"	"statue"
"William Shakespeare"	"statue"
"Balto"	"statue"
Rows: 8	

Execution plan

```
+-----------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator              | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults       |  0 | `n.name`, `n.type`
|              4 |    8 |       0 |              0 |                        |           |
|
| |                     +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Projection           |  1 | cache[n.name] AS `n.name`, cache[n.type] AS `n.type`
|              4 |    8 |       0 |                |                        |           |
|
| |                     +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +CacheProperties      |  2 | cache[n.type], cache[n.name]
|              4 |    8 |      24 |                |                        |           |
|
| |                     +----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexSeekByRange |  3 | POINT INDEX n:PointOfInterest(location) WHERE point.withinBBox(location,
point($autoint_0, $autodoub |              4 |    8 |      10 |            248 |                  302/0 |
2.619 | Fused in Pipeline 0 |
|                       |    | le_1, $autodouble_2), point($autoint_3, $autodouble_4, $autodouble_5))
|                |      |         |                |                        |           |
|
+-----------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 34, total allocated memory: 312
```

For more information about the predicates supported by point indexes, see Create indexes → Create a point index → Supported predicates.

Point index configuration settings

It is possible to configure point indexes to only index properties within a specific geographical area. This is done by specifying either of the following settings in the `indexConfig` part of the `OPTIONS` clause when creating a point index:

- `spatial.cartesian.min` and `spatial.cartesian.max`: used for Cartesian 2D coordinate systems.
- `spatial.cartesian-3d.min` and `spatial.cartesian-3d.max`: used for Cartesian 3D coordinate systems.
- `spatial.wgs-84.min` and `spatial.wgs-84.max`: used for WGS-84 2D coordinate systems.
- `spatial.wgs-84-3d.min` and `spatial.wgs-84-3d.max`: used for WGS-84 3D coordinate systems.

The `min` and `max` of each setting define the minimum and maximum bounds for the spatial data in each coordinate system.

For example, the following index would only store `OSMNodes` in the northern half of Central Park:

Create point index with configuration settings

```
CREATE POINT INDEX central_park_north
FOR (o:OSMNode) ON (o.location)
OPTIONS {
  indexConfig: {
    `spatial.wgs-84.min`:[40.7714, -73.9743],
    `spatial.wgs-84.max`:[40.7855, -73.9583]
  }
}
```

Restricting the geographic area of a point index improves the performance of spatial queries. This is especially beneficial when dealing with complex, large geo-spatial data, and when spatial queries are a significant part of an application's functionality.

Composite indexes

It is possible to create a range index on a single property or multiple properties (text and point indexes are single-property only). The latter are called composite indexes and can be useful if queries against a database frequently filter on *all* the properties indexed by the composite index.

The following example first creates a composite index on `PointOfInterest` nodes for the properties `name` and `type`, and then queries the graph using the shortestPath function to determine both the path length (in terms of traversed relationships in the graph) and geographical distance between the `Zoo School` and its nearest `tennis pitch` (note that there are 32 unique `PointOfInterest` `tennis pitch` nodes in the graph):

Create composite index

```
CREATE INDEX composite_index FOR (n:PointOfInterest) ON (n.name, n.type)
```

Query with a filter on both properties indexed by the composite index

```
PROFILE
MATCH (tennisPitch: PointOfInterest {name: 'pitch', type: 'tennis'})
WITH tennisPitch
MATCH path = shortestPath((tennisPitch)-[:ROUTE*]-(:PointOfInterest {name: 'Zoo School'}))
WITH path, relationships(path) AS relationships
ORDER BY length(path) ASC
```

```
LIMIT 1
UNWIND relationships AS rel
RETURN length(path) AS pathLength, sum(rel.distance) AS geographicalDistance
```

Result

pathLength	geographicalDistance
25	2410.4495689536334

Rows: 1

Execution plan

```
+---------------------+----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------
+------------------+---------------------+
| Operator            | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline            |
+---------------------+----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------
+------------------+---------------------+
| +ProduceResults     |  0 | pathLength, geographicalDistance
|              1 |    1 |       0 |              0 |                    0/0 |     0.065 |
|                     |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------+
|                     |
| +OrderedAggregation |  1 | length(path) AS pathLength, sum(rel.distance) AS geographicalDistance
|              1 |    1 |      50 |           5140 |                   31/0 |     4.097 | pathLength ASC
| In Pipeline 3       |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------
+------------------+---------------------+
| +Unwind             |  2 | relationships AS rel
|              1 |   25 |       0 |           3112 |                    0/0 |     0.180 |
| In Pipeline 2       |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------+
+---------------------+
| +Projection         |  3 | relationships(path) AS relationships
|              0 |    1 |       0 |                |                    0/0 |     0.050 |
|                     |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------+
|                     |
| +Top                |  4 | `length(path)` ASC LIMIT 1
|              0 |    1 |       0 |          57472 |                    0/0 |     1.763 | length(path) ASC
| In Pipeline 1       |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+------------------------+-----------
+------------------+---------------------+
| +Projection         |  5 | length(path) AS `length(path)`
|              0 |   32 |       0 |                |                        |           |
|                     |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+                        |
+------------------+                    |
| +ShortestPath       |  6 | path = (tennisPitch)-[anon_0:ROUTE*]-(anon_1)
|              0 |   32 |  181451 |          70080 |                        |           |
|                     |
| |                   +----
+-----------------------------------------------------------------------------------------------------------
+-----------------+------+---------+----------------+                        |
+------------------+                    |
```

```
| +MultiNodeIndexSeek |  7 | RANGE INDEX tennisPitch:PointOfInterest(name, type) WHERE name =
$autostring_0 AND type = $autostrin |              0 |   31 |       0 |            376 |
131215/1 |   188.723 |                  | Fused in Pipeline 0 |
|                     |    | g_1, RANGE INDEX anon_1:PointOfInterest(name) WHERE name = $autostring_2
|                |      |         |                |                         |           |
|                    |
+--------------------+----
+-----------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+-----------------------+-----------
+-----------------+--------------------+

Total database accesses: 181501, total allocated memory: 116040
```

The query plan shows the composite index being used, and not the previously created range index on the `type` property. This is because the composite index solves the queried predicate simultaneously, while the single propertied index would only be able to solve part of the predicate.

Property order and query planning

Like single-property range indexes, composite indexes support all predicates:

- Equality check: `n.prop = value`
- List membership check: `n.prop IN [value, …]`
- Existence check: `n.prop IS NOT NULL`
- Range search: `n.prop > value`
- Prefix search: `n.prop STARTS WITH value`

However, the order in which properties are defined when creating a composite index impacts how the planner will use the index to solve predicates. For example, a composite index on `(n.prop1, n.prop2, n.prop3)` will generate a different query plan than a composite index created on `(n.prop3, n.prop2, n.prop1)`.

The following example shows how composite indexes on the same properties defined in a different order will generate different execution plans:

Create a composite index on three properties

```
CREATE INDEX composite_2 FOR (n:PointOfInterest) ON (n.lat, n.name, n.type)
```

Note the order in which the properties are defined when creating the index, with `lat` first, `name` second, and `type` last.

Query with a filter on the three indexed properties

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.lat = 40.7697989 AND n.name STARTS WITH 'William' AND n.type IS NOT NULL
RETURN n.name AS name
```

Result

name
"William Shakespeare"

name
Rows: 1

Execution plan

```
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator        | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults |  0 | name
|              0 |    0 |       0 |              0 |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Projection     |  1 | cache[n.name] AS name
|              0 |    0 |       0 |                |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexSeek  |  2 | RANGE INDEX n:PointOfInterest(lat, name, type) WHERE lat = $autodouble_0 AND name
STARTS WITH $autos |              0 |    0 |       1 |            248 |                     0/2 |     1.276
| Fused in Pipeline 0 |
|                 |    | tring_1 AND type IS NOT NULL, cache[n.name]
|                |      |         |                |                        |           |
|
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 1, total allocated memory: 312
```

The plan shows the recently created composite index is used. It also shows that the predicates are filtered as specified in the query (i.e. an equality check on the `lat` property, a prefix search on the `name` property, and an existence check on the `type` property).

However, if the order of the properties is altered when creating the index, a different query plan will be generated. To demonstrate this behavior, it is first necessary to drop the recently created `composite_2` index and create a new composite index on the same properties defined in a different order:

Drop index

```
DROP INDEX composite_2
```

Create a composite index on same three properties defined in a different order

```
CREATE INDEX composite_3 FOR (n:PointOfInterest) ON (n.name, n.type, n.lat)
```

Note that the order of the properties has changed: the `name` property is now the first property defined in the composite index, and the `lat` property is indexed last.

Rerun query after the creation of a different composite index

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.lat = 40.769798 AND n.name STARTS WITH 'William' AND n.type IS NOT NULL
RETURN n.name AS name
```

Execution plan

```
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator        | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults |  0 | name
|              0 |    0 |       0 |              0 |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Projection     |  1 | cache[n.name] AS name
|              0 |    0 |       0 |                |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter         |  2 | cache[n.lat] = $autodouble_0
|              0 |    0 |       0 |                |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexSeek  |  3 | RANGE INDEX n:PointOfInterest(name, type, lat) WHERE name STARTS WITH
$autostring_1 AND type IS NOT |              0 |    2 |       3 |            248 |                    2/0
|     0.807 | Fused in Pipeline 0 |
|                 |    | NULL AND lat IS NOT NULL, cache[n.name], cache[n.lat]
|                |      |         |                |                        |           |
|
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 3, total allocated memory: 312
```

This plan now shows that, while a prefix search has been used to solve the `name` property predicate, the `lat` property predicate is no longer solved with an equality check, but rather with an existence check and an explicit filter operation afterward. Note that if the `composite_2` index had not been dropped before the query was rerun, the planner would have used it instead of the `composite_3` index.

This is because, when using composite indexes, any predicate after a prefix search will automatically be planned as an existence check predicate.

Composite index rules

- If a query contains an equality check or a list membership check predicates, they need to be for the first properties defined when creating the composite index.

- Queries utilizing a composite index can contain up to one range search or prefix search predicate.
- There can be any number of existence check predicates.
- Any predicates following a prefix search or an existence check will be planned as existence checks.
- Suffix and substring search predicates can utilize composite indexes. However, they are always planned as an existence check and any subsequent query predicates will accordingly also be planned as such. Note that if these predicates are used, and a text index also exists on any of the indexed (`STRING`) properties, the planner will use the text index instead of a composite index.

These rules can be important when creating composite indexes, as some checks are more efficient than others. For instance, it is generally more efficient for the planner to perform an equality check on a property than an existence check. Depending on the queries and the application, it may, therefore, be cost-effective to consider the order in which properties are defined when creating a composite index.

Additionally, it bears repeating that composite indexes can only be used if a predicate filters on all the properties indexed by the composite index, and that composite indexes can only be created for range indexes.

Range index-backed ORDER BY

Range indexes store properties in ascending order (alphabetically for `STRING` values, and numerically for `FLOAT` and `INTEGER` values). This can have important implications for query performance, because the planner may be able to take advantage of a pre-existing index order and therefore not have to perform an expensive `Sort` operation later in the query.

To demonstrate this behavior, the following query will filter out any `ROUTE` relationships with a `distance` property less than `30`, and return the `distance` property of the matched relationships in ascending numerical order using the ORDER BY clause.

Query to return order of results without a relevant index

```
PROFILE
MATCH ()-[r:ROUTE]-()
WHERE r.distance < 30
RETURN r.distance AS distance
ORDER BY distance
```

Execution plan

```
+----------------------------------+----+--------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+-------------+----------------------+
| Operator                         | Id | Details                        | Estimated Rows | Rows  | DB Hits
| Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by   | Pipeline            |
+----------------------------------+----+--------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+-------------+----------------------+
| +ProduceResults                  |  0 | distance                       |           3013 |  6744 |       0
|              0 |                     0/0 |    14.397 |              |                     |
| |                                |----+--------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+             |                      |
| +Sort                            |  1 | distance ASC                   |           3013 |  6744 |       0
|         540472 |                     0/0 |    16.844 | distance ASC | In Pipeline 1       |
| |                                |----+--------------------------------+----------------+-------
+---------+----------------+-----------------------+-----------+-------------+----------------------+
| +Projection                      |  2 | cache[r.distance] AS distance  |           3013 |  6744 |       0
|                |                                |           |             |                     |
| |                                |----+--------------------------------+----------------+-------
+---------+----------------+                       |           +-------------+                      |
| +Filter                          |  3 | cache[r.distance] < $autoint_0 |           3013 |  6744 |   10041
```

```
|               |                       |          |             |                    |
| |                             +----+-------------------------------+----------------+-------
+---------+---------------+                       |          +--------------+                    |
| +UndirectedRelationshipTypeScan |  4 | (anon_0)-[r:ROUTE]-(anon_1)    |          10044 | 10041 |    5023
|            376 |                   84/0 |    22.397 |              | Fused in Pipeline 0 |
+---------------------------------+----+-------------------------------+----------------+-------
+---------+---------------+-----------------------+----------+-------------+---------------------+

Total database accesses: 15064, total allocated memory: 540808
```

This plan shows two important points about indexes and the ordering of results:

- Only the relationship type lookup index was used in this query (accessed by the `UndirectedRelationshipTypeScan` operator, which fetches all relationships and their start and end nodes from the relationship type index).
- As a result, the planner has to perform a `Sort` operation to order the results by the distance property (in this case, it required 540472 bytes of memory).

To see how an index could impact the query plan, it is first necessary to create a range index on the `distance` property:

Create a range index on a relationship type property

```
CREATE INDEX range_index_relationships FOR ()-[r:ROUTE]-() ON (r.distance)
```

Re-running the query, it now generates a different plan:

Rerun query after the creation of a relevant index

```
PROFILE
MATCH ()-[r:ROUTE]-()
WHERE r.distance < 30
RETURN r.distance AS distance
ORDER BY distance
```

Execution plan

```
+----------------------------------------+----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+----------------
+---------------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by     |
Pipeline            |
+----------------------------------------+----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+----------------
+---------------------+
| +ProduceResults                         |  0 | distance
|            301 | 6744 |       0 |              0 |                        |           |                |
|
| |                                       +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |                |
|
| +Projection                             |  1 | cache[r.distance] AS distance
|            301 | 6744 |       0 |                |                        |           | distance ASC   |
|
| |                                       +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           +----------------+
|
| +UndirectedRelationshipIndexSeekByRange |  2 | RANGE INDEX (anon_0)-[r:ROUTE(distance)]-(anon_1) WHERE
distance < $autoint_0, cache[r.distance] |            301 | 6744 |    3373 |            248 |
2361/10 |    76.542 | r.distance ASC | Fused in Pipeline 0 |
```

```
+----------------------------------------+----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------+---------------
+--------------------+

Total database accesses: 3373, total allocated memory: 312
```

Focusing on the same two points in the plan, the following has changed:

- The recently created range index on the relationship type property `distance` is now used.
- As a result, the plan no longer needs to perform a `Sort` operation to order the results (because the `distance` property is already ordered by the index), and this substantially reduces the cost of the query (the total memory cost of the query is now 312 bytes).

In the same way, the order of a range index can significantly improve queries using the `max()` and `min()` functions.

Multiple index use

Indexes are principally used to find the starting points of patterns. If a query contains one `MATCH` clause, then, as a general rule, only the index that best suits the predicates in that clause will be selected by the planner. If, however, a query contains two or more `MATCH` clauses, it is possible to use several indexes.

To show multiple indexes used in one query, the following example will first create a new index on the `lon` (longitude) property for `PointOfInterest` nodes. It then uses a query that finds all `PointOfInterest` nodes north of the `William Shakespeare` statue in Central Park.

Create a range index on the longitude property

```
CREATE INDEX range_index_lon FOR (n:PointOfInterest) ON (n.lon)
```

Query to find all `PointOfInterest` nodes north of the `William Shakespeare` statue

```
PROFILE
MATCH (ws:PointOfInterest {name:'William Shakespeare'})
WITH ws
MATCH (poi:PointOfInterest)
WHERE poi.lon > ws.lon
RETURN poi.name AS name
```

Execution plan

```
+------------------------+----+-----------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| Operator               | Id | Details                                                              |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+------------------------+----+-----------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| +ProduceResults        |  0 | name                                                                 |
9 |  143 |       0 |              0 |                       |           |                     |
| |                        +----+----------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +Projection            |  1 | poi.name AS name                                                     |
9 |  143 |     283 |                |                       |           |                     |
| |                        +----+----------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
```

```
|
| +Apply                  |  2 |                                                                                  |
9 |  143 |       0 |                |                          |           |                     |
| |\                      +----+----------------------------------------------------------------------------------
+----------------+------+---------+----------------+                          |           |
|
| | +NodeIndexSeekByRange |  3 | RANGE INDEX poi:PointOfInterest(lon) WHERE lon > ws.lon          |
9 |  143 |     146 |           2280 |                  233/1 |     1.460 | Fused in Pipeline 1 |
| |                       +----+----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +NodeIndexSeek          |  4 | RANGE INDEX ws:PointOfInterest(name) WHERE name = $autostring_0 |
2 |    1 |       2 |            376 |                    1/0 |     0.635 | In Pipeline 0       |
+-------------------------+----+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 431, total allocated memory: 2616
```

This plan shows that a separate index is used to improve the performance of each `MATCH` clause (first by utilizing the index on the `name` property to find the `William Shakespeare` node, and then by using the index on the `lon` property to find all nodes with a greater longitudinal value).

Indexes and `null` values

Neo4j indexes do not store `null` values. This means that the planner must be able to rule out the possibility of `null` values in order for queries to use an index.

The following query demonstrates the incompatibility between `null` values and indexes by counting all `PointOfInterest` nodes with an unset `name` property:

Query to count nodes with a `null` `name` value

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS NULL
RETURN count(n) AS nodes
```

Result

nodes
3
Rows:1

Execution plan

```
+-------------------+----+-------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| Operator          | Id | Details           | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+-------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +ProduceResults   |  0 | nodes             |              1 |    1 |       0 |              0 |
0/0 |     0.012 | In Pipeline 1       |
| |                 +----+-------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +EagerAggregation |  1 | count(n) AS nodes |              1 |    1 |       0 |             32 |
|           |                   |
| |                 +----+-------------------+----------------+------+---------+----------------+
|           |                   |
| +Filter           |  2 | n.name IS NULL    |            141 |    3 |     373 |                |
|           |                   |
```

```
| |                +----+------------------+---------------+------+---------+---------------+
|           |                    |
| +NodeByLabelScan  |  3 | n:PointOfInterest |            188 |  188 |     189 |           376 |
115/0 |     0.769 | Fused in Pipeline 0 |
+-------------------+----+------------------+---------------+------+---------+---------------
+-----------------------+-----------+--------------------+

Total database accesses: 562, total allocated memory: 472
```

The plan shows that neither of the two available indexes (range and text) on the `name` property is used to solve the predicate.

However, if a query predicate is added which is able to exclude the presence of any `null` values, then an index can be used. The following query shows this by adding a substring predicate to the above query:

Query to count nodes with a `null` `name` value or nodes with a `name` property containing `'William'`

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS NULL OR n.name CONTAINS 'William'
RETURN count(n) AS nodes
```

Result

nodes
5
Rows: 1

The query result now includes both the three nodes with an unset `name` value found in the previous query and the two nodes with a `name` value containing `William` (`William Shakespeare` and `William Tecumseh Sherman`).

Execution plan

```
+--------------------------+----+-----------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator                 | Id | Details                                                                |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------+----+-----------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults          |  0 | nodes                                                                  |
1 |    1 |       0 |              0 |                   0/0 |     0.010 | In Pipeline 3       |
| |                        +----+------------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +EagerAggregation        |  1 | count(n) AS nodes                                                      |
1 |    1 |       0 |             32 |                       |           |                     |
| |                        +----+------------------------------------------------------------------------
+----------------+------+---------+---------------+                       |           |
|
| +Distinct                |  2 | n                                                                      |
141 |    5 |       0 |            352 |                       |           |                     |
| |                        +----+------------------------------------------------------------------------
+----------------+------+---------+---------------+                       |           |
|
| +Union                   |  3 |                                                                        |
142 |    5 |       0 |            352 |                   0/0 |     0.220 | Fused in Pipeline 2 |
| |\                       +----+------------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| | +NodeIndexContainsScan |  4 | TEXT INDEX n:PointOfInterest(name) WHERE name CONTAINS $autostring_0 |
```

```
1 |    2 |       3 |           376 |                   4/0 |     0.456 | In Pipeline 1       |
| |                       +----+-------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +Filter                 |  5 | n.name IS NULL                                                          |
141 |    3 |     373 |               |                         |           |                     |
| |                       +----+-------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +NodeByLabelScan        |  6 | n:PointOfInterest                                                       |
188 |  188 |     189 |           376 |                 115/0 |     0.673 | Fused in Pipeline 0 |
+-------------------------+----+-------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 565, total allocated memory: 1352
```

This plan shows that an index is only used to solve the second part of the `WHERE` clause, which excludes the presence of `null` values.

The presence of `null` values within an indexed property therefore does not negate the use of an index. Index use is only negated if the planner is unable to rule out the inclusion of any unset properties in the matching process.

The presence of `null` values may not be known in advance, and this can cause unexpected instances of indexes not being used. There are, however, a few strategies to ensure that an index will be used.

Property existence checks

One method to ensure an index is used is to explicitly filter out any `null` values by appending `IS NOT NULL` to the queried property. The following example uses the same query as above but exchanges `IS NULL` with `IS NOT NULL` in the `WHERE` clause:

Query to count `PointOfInterest` nodes without a `null` `name` value

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS NOT NULL
RETURN count(n) AS nodes
```

Result

nodes
185
Rows: 1

Execution plan

```
+-------------------+----+-----------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| Operator          | Id | Details                                                         | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------+----+-----------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| +ProduceResults   |  0 | nodes                                                           |              1 |
1 |       0 |              0 |                   0/0 |     0.013 | In Pipeline 1      |
| |                 +----+-----------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| +EagerAggregation |  1 | count(n) AS nodes                                               |              1 |
1 |       0 |             32 |                       |           |                    |
```

```
| |                +----+-------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                    |
| +NodeIndexScan    |  2 | RANGE INDEX n:PointOfInterest(name) WHERE name IS NOT NULL |            185 |
185 |     186 |            376 |                    0/1 |     0.691 | Fused in Pipeline 0 |
+-------------------+----+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 186, total allocated memory: 472
```

This plan shows that the previously created range index on the `name` property is now used to solve the predicate.

Text indexes and type predicate expressions

Text indexes require that predicates only include `STRING` properties.

To use text indexes in situations where any of the queried properties may be either of an incompatible type or `null` rather than a `STRING` value, add the type predicate expression `IS :: STRING NOT NULL` (or its alias, `IS :: STRING!`) to the query. This will enforce both the existence of a property and its `STRING` type, discarding any rows where the property is missing or not of type `STRING`, and thereby enable the use of text indexes.

For example, if the `WHERE` predicate in the previous query is altered to instead append `IS :: STRING NOT NULL`, then the text index rather than the range index is used (range indexes do not support type predicate expressions):

Query using a type predicate expression

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS :: STRING NOT NULL
RETURN count(n) AS nodes
```

Execution plan

```
+-------------------+----+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| Operator          | Id | Details                                                     | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------+----+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults   |  0 | nodes                                                       |              1 |
1 |       0 |              0 |                   0/0 |     0.009 | In Pipeline 1      |
| |                 +----+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| +EagerAggregation |  1 | count(n) AS nodes                                           |              1 |
1 |       0 |             32 |                       |           |                    |
| |                 +----+-------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                    |
| +NodeIndexScan    |  2 | TEXT INDEX n:PointOfInterest(name) WHERE name IS NOT NULL |            185 |
185 |     186 |            376 |                   0/0 |     0.343 | Fused in Pipeline 0 |
+-------------------+----+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 186, total allocated memory: 472
```

The `toString` function can also be used to convert an expression to `STRING` values, and thereby help the planner to select a text index.

Property type constraints

For indexes that are compatible only with specific types (i.e. text and point indexes), the Cypher planner needs to deduce that a predicate will evaluate to `null` for non-compatible values in order to use the index. If a predicate is not explicitly defined as the required type (`STRING` or `POINT`), this can lead to situations where a text or point index is not used.

Since property type constraints guarantee that a property is always of the same type, they can be used to extend the scenarios in which text and point indexes are compatible with a predicate.

To show this, the following example will first drop the existing range index on the `name` property (this is necessary because property type constraints only extend the compatibility of type-specific indexes - range indexes are not limited by a value type). It will then run the same query with a `WHERE` predicate on the `name` property (for which there exists a previously created text index) before and after creating a property type constraint, and compare the resulting execution plans.

Drop range index

```
DROP INDEX range_index_name
```

Query to count `PointOfInterest` nodes without a `null` `name` value

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS NOT NULL
RETURN count(n) AS nodes
```

Execution plan

```
+-------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| Operator          | Id | Details            | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +ProduceResults   |  0 | nodes              |              1 |    1 |       0 |              0 |
0/0 |     0.012 | In Pipeline 1       |
| |                 +----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +EagerAggregation |  1 | count(n) AS nodes  |              1 |    1 |       0 |             32 |
|           |                     |
| |                 +----+--------------------+----------------+------+---------+----------------+
|           |                     |
| +Filter           |  2 | n.name IS NOT NULL |            187 |  185 |     373 |                |
|           |                     |
| |                 +----+--------------------+----------------+------+---------+----------------+
|           |                     |
| +NodeByLabelScan  |  3 | n:PointOfInterest  |            188 |  188 |     189 |            376 |
259/0 |     0.363 | Fused in Pipeline 0 |
+-------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+

Total database accesses: 562, total allocated memory: 472
```

This plan shows that the available text index on the `name` property was not used to solve the predicate. This is because the planner was not able to deduce that all `name` values are of type `STRING`.

However, if a property type constraint is created to ensure that all `name` properties have a `STRING` value, a different query plan is generated.

Create `STRING` type constraint on the `name` property

```
CREATE CONSTRAINT type_constraint
FOR (n:PointOfInterest) REQUIRE n.name IS :: STRING
```

Rerun the query after the creation of a property type constraint

```
PROFILE
MATCH (n:PointOfInterest)
WHERE n.name IS NOT NULL
RETURN count(n) AS nodes
```

Execution plan

```
+-------------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| Operator          | Id | Details                                                   | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults   |  0 | nodes                                                     |              1 |
1 |       0 |               0 |                   0/0 |     0.013 | In Pipeline 1      |
| |                 +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| +EagerAggregation |  1 | count(n) AS nodes                                         |              1 |
1 |       0 |              32 |                       |           |                    |
| |                 +----+-----------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                    |
| +NodeIndexScan    |  2 | TEXT INDEX n:PointOfInterest(name) WHERE name IS NOT NULL |            187 |
185 |     186 |             376 |                   0/0 |     0.328 | Fused in Pipeline 0 |
+-------------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 186, total allocated memory: 472
```

Because of the property type constraint on the `name` property, the planner is now able to deduce that all `name` properties are of type `STRING`, and therefore use the available text index.

Point indexes can be extended in the same way if a property type constraint is created to ensure that all properties are `POINT` values.

Note that property existence constraints do not currently leverage index use in the same way.

Heuristics: deciding what to index

While it is impossible to give exact directions on when a performance index might be beneficial for a particular use-case, the following points provide some useful heuristics for when creating an index might improve query performance:

- **Frequent property-based queries**: if some properties are used frequently for filtering or matching, consider creating an index on them.
- **Performance optimization**: If certain queries are too slow, re-examine the properties that are filtered on, and consider creating indexes for those properties that may cause bottlenecking.
- **High cardinality properties**: high cardinality properties have many distinct values (e.g., unique identifiers, timestamps, or user names). Queries that seek to retrieve such properties will likely benefit from indexing.
- **Complex queries**: if queries traverse complex paths in a graph (for example, by involving multiple hops

and several layers of filtering), adding indexes to the properties used in those queries can improve query performance.

- **Experiment and test**: It is good practice to experiment with different indexes and query patterns, and to measure the performance of critical queries with and without different indexes to evaluate their effectiveness.

Over-indexing: considerations and solutions

Performance indexes can significantly improve query performance. They should, however, be used judiciously for the following reasons:

- **Storage space**: because each index is a secondary copy of the data in the primary database, each index essentially doubles the amount of storage space occupied by the indexed data.
- **Slower write queries**: adding indexes impacts the performance of write queries. This is because indexes are updated with each write query. If a system needs to perform a lot of writes quickly, it may be counterproductive to have an index on the affected data entities. In other words, if write performance is crucial for a particular use case, it may be beneficial to only add indexes where they are necessary for read purposes.

As a result of these two points, deciding what to index (and what not to index) is an important and non-trivial task.

Keeping track of index-use: `lastRead`, `readCount`, and `trackedSince`

Unused indexes take up unnecessary storage space and it may be beneficial to remove them. Knowing which indexes are most frequently used by the queries against a database can, however, be difficult. There are three relevant columns returned by the `SHOW INDEX` command which can help identify redundant indexes:

- **`lastRead`**: returns the last time the index was used for reading.
- **`readCount`**: returns the number of read queries issued to the index.
- **`trackedSince`** returns the time when usage statistics tracking started for an index.[2]

To return these values (along with other relevant information) for the indexes in a database, run the following query:

Query to identify redundant indexes

```
SHOW INDEX YIELD name, type, entityType, labelsOrTypes, properties, lastRead, readCount, trackedSince
```

If any unused indexes are identified, it may be beneficial to delete them using the `DROP INDEX` command.

Summary

- Range indexes can be used to solve most predicates.
- Text indexes are used over range indexes for `CONTAINS` and `ENDS WITH` predicates on `STRING` properties, and if the queried `STRING` properties exceed 8 kb.

- Point indexes are used when queries filter on distances and bounding boxes.
- Token lookup indexes only solve node label and relationship type predicates. They do not solve any property predicates. Deleting token lookup indexes will negatively impact query performance.
- Composite indexes are only used if the query filters on all properties indexed by the composite index. The order in which the properties are defined when creating a composite index impacts how the planner solves query predicates.
- Queries ordering results using `ORDER BY` can leverage the pre-existing order in range indexes and thereby improve query performance.
- A Cypher query can use several indexes if the planner deems it beneficial to the performance of a query.
- Neo4j indexes do not store `null` values, and the planner must be able to rule out any entities with properties containing `null` values in order to use an index. There are several strategies to ensure the use of indexes.
- The columns `lastRead`, `readCount`, and `trackedSince` returned by the `SHOW INDEX` command can be used to identify redundant indexes that take up unnecessary space.

Index hints for the Cypher planner

A planner hint is used to influence the decisions of the planner when building an execution plan for a query. Planner hints are specified in a query with the `USING` keyword.

> **Caution:**
>
> Forcing planner behavior is an advanced feature, and should be used with caution by experienced developers and/or database administrators only, as it may cause queries to perform poorly.

When executing a query, Neo4j needs to decide where in the query graph to start matching. This is done by looking at the `MATCH` clause and the `WHERE` conditions and using that information to find useful indexes, or other starting points.

However, the selected index might not always be the best choice. Sometimes multiple indexes are possible candidates, and the query planner picks the suboptimal one from a performance point of view. Moreover, in some circumstances (albeit rarely) it is better not to use an index at all.

Neo4j can be forced to use a specific starting point through the `USING` keyword. This is called giving a planner hint.

There are three types of planner hints:

- Index hints.
- Scan hints.
- Join hints.

Setup

To replicate results from this page, execute the following query on an empty database.

```
FOREACH(i IN range(1, 100) |
  CREATE (:Scientist {born: 1800 + i})-[:RESEARCHED]->
         (:Science)<-[:INVENTED_BY {year: 530 + (i % 50), location: 'Location' + i}]-
         (:Pioneer {born: 500 + (i % 50)})-[:LIVES_IN]->
         (:City)-[:PART_OF]->
         (:Country {formed: 400 + i, name:'Country' + i})
);

CREATE RANGE INDEX FOR (s:Scientist) ON (s.born);
CREATE RANGE INDEX FOR (p:Pioneer) ON (p.born);
CREATE RANGE INDEX FOR (c:Country) ON (c.formed);
CREATE RANGE INDEX FOR (c:Country) ON (c.name);
CREATE TEXT INDEX FOR (c:Country) ON (c.name);
CREATE RANGE INDEX FOR ()-[i:INVENTED_BY]-() ON (i.year);
CREATE RANGE INDEX FOR ()-[i:INVENTED_BY]-() ON (i.location);
CREATE TEXT INDEX FOR ()-[i:INVENTED_BY]-() ON (i.location);
CALL db.awaitIndexes;
```

The following query is used in some of the examples on this page. Without any hints, one index and no join is used.

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator        | Details                                                   | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults | c, cc, i, p, s, sc                                        |              0 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter         | s.born = $autoint_0 AND s:Scientist                       |              0 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Expand(All)    | (sc)<-[anon_0:RESEARCHED]-(s)                             |              0 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter         | i.year = $autoint_1 AND sc:Science                        |              0 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Expand(All)    | (p)-[i:INVENTED_BY]->(sc)                                 |              0 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter         | p.born = $autoint_2 AND p:Pioneer                         |              0 |    0 |
2 |                |                       |           |                     |
```

```
| |              +-------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +Expand(All)    | (c)<-[anon_1:LIVES_IN]-(p)                          |             1 |    1 |
3 |                 |                         |           |                    |
| |              +-------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +Filter         | c:City                                              |             1 |    1 |
2 |                 |                         |           |                    |
| |              +-------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +Expand(All)    | (cc)<-[anon_2:PART_OF]-(c)                          |             1 |    1 |
2 |                 |                         |           |                    |
| |              +-------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +NodeIndexSeek  | RANGE INDEX cc:Country(formed) WHERE formed = $autoint_3 |             1 |    1 |
2 |             120 |                     6/1 |     0.506 | Fused in Pipeline 0 |
+-----------------+-------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+

Total database accesses: 11, total allocated memory: 208
```

Index hints

Index hints are used to specify which index the planner should use as a starting point. This can be beneficial in cases where the index statistics are not accurate for the specific values that the query at hand is known to use, which would result in the planner picking a non-optimal index. An index hint is supplied after an applicable `MATCH` clause.

Available index hints are:

Hint	Fulfilled by plans
`USING [RANGE | TEXT | POINT] INDEX variable:Label(property)`	`NodeIndexScan`, `NodeIndexSeek`
`USING [RANGE | TEXT | POINT] INDEX SEEK variable:Label(property)`	`NodeIndexSeek`
`USING [RANGE | TEXT | POINT] INDEX variable:RELATIONSHIP_TYPE(property)`	`DirectedRelationshipIndexScan`, `UndirectedRelationshipIndexScan`, `DirectedRelationshipIndexSeek`, `UndirectedRelationshipIndexSeek`
`USING [RANGE | TEXT | POINT] INDEX SEEK variable:RELATIONSHIP_TYPE(property)`	`DirectedRelationshipIndexSeek`, `UndirectedRelationshipIndexSeek`

When specifying an index type for a hint, e.g. `RANGE`, `TEXT`, or `POINT`, the hint can only be fulfilled when an index of the specified type is available. When no index type is specified, the hint can be fulfilled by any index types.

> **Caution:**
>
> Using a hint must never change the result of a query. Therefore, a hint with a specified index type is only fulfillable when the planner knows that using an index of the specified type does not change the results. Please refer to The use of indexes for more details.

It is possible to supply several index hints, but keep in mind that several starting points will require the use

of a potentially expensive join later in the query plan.

Query using a node index hint

The query above can be tuned to pick a different index as the starting point.

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
USING INDEX p:Pioneer(born)
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------+------------------------------------------------------+----------------+------+---------
+----------------+----------------------+-----------+--------------------+
| Operator       | Details                                              | Estimated Rows | Rows | DB Hits
| Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+----------------+------------------------------------------------------+----------------+------+---------
+----------------+----------------------+-----------+--------------------+
| +ProduceResults | c, cc, i, p, s, sc                                  |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Filter         | cc.formed = $autoint_3 AND cc:Country               |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Expand(All)    | (c)-[anon_2:PART_OF]->(cc)                          |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Filter         | c:City                                              |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Expand(All)    | (p)-[anon_1:LIVES_IN]->(c)                          |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Filter         | s.born = $autoint_0 AND s:Scientist                 |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Expand(All)    | (sc)<-[anon_0:RESEARCHED]-(s)                       |              0 |    0 |       0
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Filter         | i.year = $autoint_1 AND sc:Science                  |              0 |    0 |       2
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +Expand(All)    | (p)-[i:INVENTED_BY]->(sc)                           |              2 |    2 |       6
|                |                      |           |                    |
| |              +------------------------------------------------------+----------------+------+---------
+----------------+                      |           |                    |
| +NodeIndexSeek  | RANGE INDEX p:Pioneer(born) WHERE born = $autoint_2 |              2 |    2 |       3
|            120 |                  4/1 |     0.491 | Fused in Pipeline 0 |
```

```
+----------------+-------------------------------------------------------+---------------+------+---------
+---------------+----------------------+-----------+--------------------+

Total database accesses: 11, total allocated memory: 208
```

Query using a node text index hint

The following query can be tuned to pick a text index.

Query

```
PROFILE
MATCH (c:Country)
USING TEXT INDEX c:Country(name)
WHERE c.name = 'Country7'
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------+-------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+--------------------+
| Operator        | Details                                               | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+----------------+-------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+--------------------+
| +ProduceResults | c                                                   |              1 |    1 |
0 |                |                        |           |                    |
| |              +-----------------------------------------------------+---------------+------
+---------+---------------+                        |           |                    |
| +NodeIndexSeek  | TEXT INDEX c:Country(name) WHERE name = $autostring_0 |              1 |    1 |
2 |            120 |                    2/0 |     0.949 | Fused in Pipeline 0 |
+----------------+-------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+--------------------+

Total database accesses: 2, total allocated memory: 184
```

Query using a relationship index hint

The query above can be tuned to pick a relationship index as the starting point.

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
USING INDEX i:INVENTED_BY(year)
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED
```

```
Runtime version 2026.10

Batch size 128

+-------------------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                      | Details                                                                |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-------------------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults               | c, cc, i, p, s, sc                                                     |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                       | cc.formed = $autoint_3 AND cc:Country                                  |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Expand(All)                  | (c)-[anon_2:PART_OF]->(cc)                                             |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                       | c:City                                                                 |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Expand(All)                  | (p)-[anon_1:LIVES_IN]->(c)                                             |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                       | s.born = $autoint_0 AND s:Scientist                                    |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Expand(All)                  | (sc)<-[anon_0:RESEARCHED]-(s)                                          |
0 |    0 |       0 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                       | p.born = $autoint_2 AND sc:Science AND p:Pioneer                       |
0 |    0 |       4 |                |                        |           |                      |
| |                               +------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +DirectedRelationshipIndexSeek | RANGE INDEX (p)-[i:INVENTED_BY(year)]->(sc) WHERE year = $autoint_1 |
2 |    2 |       3 |            120 |                    5/1 |     0.461 | Fused in Pipeline 0 |
+-------------------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 7, total allocated memory: 208
```

Query using a relationship text index hint

The following query can be tuned to pick a text index.

Query

```
PROFILE
MATCH ()-[i:INVENTED_BY]->()
USING TEXT INDEX i:INVENTED_BY(location)
WHERE i.location = 'Location7'
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------
+-----------------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+
| Operator                       | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-------------------------------
+-----------------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults                | i
|              1 |    1 |       0 |                |                       |           |
|
| |
+-----------------------------------------------------------------------------------------------+----------------
+------+---------+----------------+                       |           |                     |
| +DirectedRelationshipIndexSeek | TEXT INDEX (anon_0)-[i:INVENTED_BY(location)]->(anon_1) WHERE location
= $autostring_0 |              1 |    1 |       2 |            120 |                   3/0 |     1.079 |
Fused in Pipeline 0 |
+-------------------------------
+-----------------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+

Total database accesses: 2, total allocated memory: 184
```

Query using multiple index hints

Supplying one index hint changed the starting point of the query, but the plan is still linear, meaning it only has one starting point. If we give the planner yet another index hint, we force it to use two starting points, one at each end of the match. It will then join these two branches using a join operator.

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
USING INDEX s:Scientist(born)
USING INDEX cc:Country(formed)
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+-----------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator         | Details                                                         | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+-----------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
```

```
| +ProduceResults  | c, cc, i, p, s, sc                                                 |              0 |    0 |
0 |                  |                    0/0 |     0.000 |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+                                                    |
| +NodeHashJoin    | sc                                                                 |              0 |    0 |
0 |            432 |                        |           | In Pipeline 2                                      |
| |\               +-------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| | +Expand(All)   | (s)-[anon_0:RESEARCHED]->(sc)                                      |              1 |    0 |
0 |                  |                          |           |                                                    |
| | |              +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| | +NodeIndexSeek | RANGE INDEX s:Scientist(born) WHERE born = $autoint_0              |              1 |    0 |
0 |            120 |                    0/0 |     0.000 | Fused in Pipeline 1                                |
| |                +-------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +Filter          | i.year = $autoint_1 AND sc:Science                                 |              0 |    0 |
0 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +Expand(All)     | (p)-[i:INVENTED_BY]->(sc)                                          |              0 |    0 |
0 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +Filter          | p.born = $autoint_2 AND p:Pioneer                                  |              0 |    0 |
2 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +Expand(All)     | (c)<-[anon_1:LIVES_IN]-(p)                                         |              1 |    1 |
3 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +Filter          | c:City                                                             |              1 |    1 |
2 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +Expand(All)     | (cc)<-[anon_2:PART_OF]-(c)                                         |              1 |    1 |
2 |                  |                          |           |                                                    |
| |                  +-------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                                                    |
| +NodeIndexSeek   | RANGE INDEX cc:Country(formed) WHERE formed = $autoint_3           |              1 |    1 |
2 |            120 |                    7/0 |     0.494 | Fused in Pipeline 0                                |
+-----------------+-------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+

Total database accesses: 11, total allocated memory: 768
```

Query using multiple index hints with a disjunction

Supplying multiple index hints can also be useful if the query contains a disjunction (`OR`) in the `WHERE` clause. This makes sure that all hinted indexes are used and the results are joined together with a `Union` and a `Distinct` afterwards.

Query

```
PROFILE
MATCH (country:Country)
USING INDEX country:Country(name)
USING INDEX country:Country(formed)
WHERE country.formed = 500 OR country.name STARTS WITH "A"
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+----------------------+------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| Operator             | Details                                                                |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------+------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| +ProduceResults      | country                                                                |
1 |    1 |       0 |              |                         |           |                      |
| |                    +------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +Distinct            | country                                                                |
1 |    1 |       0 |          224 |                         |           |                      |
| |                    +------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +Union               |                                                                        |
2 |    1 |       0 |           80 |                    1/0 |     0.213 | Fused in Pipeline 2 |
| |\                   +------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| | +NodeIndexSeek     | RANGE INDEX country:Country(formed) WHERE formed = $autoint_0          |
1 |    1 |       2 |          120 |                    1/0 |     0.101 | In Pipeline 1       |
| |                    +------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+
| +NodeIndexSeekByRange | RANGE INDEX country:Country(name) WHERE name STARTS WITH $autostring_1 |
1 |    0 |       1 |          120 |                    0/1 |     0.307 | In Pipeline 0       |
+----------------------+------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+--------------------+

Total database accesses: 3, total allocated memory: 320
```

Cypher will usually provide a plan that uses all indexes for a disjunction without hints. It may, however, decide to plan a `NodeByLabelScan` instead, if the predicates appear to be not very selective. In this case, the index hints can be useful.

Scan hints

If your query matches large parts of an index, it might be faster to scan the label or relationship type and filter out rows that do not match. To do this, you can use `USING SCAN variable:Label` after the applicable `MATCH` clause for node indexes, and `USING SCAN variable:RELATIONSHIP_TYPE` for relationship indexes. This will force Cypher to not use an index that could have been used, and instead do a label scan/relationship type scan. You can use the same hint to enforce a starting point where no index is applicable.

Hinting a label scan

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
USING SCAN s:Scientist
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| Operator        | Details                                                   | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| +ProduceResults | c, cc, i, p, s, sc                                        |              0 |    0 |
0 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Filter         | cc.formed = $autoint_3 AND cc:Country                     |              0 |    0 |
0 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Expand(All)    | (c)-[anon_2:PART_OF]->(cc)                                |              0 |    0 |
0 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Filter         | c:City                                                    |              0 |    0 |
0 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Expand(All)    | (p)-[anon_1:LIVES_IN]->(c)                                |              0 |    0 |
0 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Filter         | i.year = $autoint_1 AND p.born = $autoint_2 AND p:Pioneer |              0 |    0 |
1 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Expand(All)    | (sc)<-[i:INVENTED_BY]-(p)                                 |              1 |    1 |
3 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Filter         | sc:Science                                                |              1 |    1 |
2 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Expand(All)    | (s)-[anon_0:RESEARCHED]->(sc)                             |              1 |    1 |
2 |                |                        |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +Filter         | s.born = $autoint_0                                       |              1 |    1 |
200 |               |                         |           |                    |
| |               +------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +NodeByLabelScan | s:Scientist                                              |            100 |  100 |
101 |            120 |                  11/0 |     0.512 | Fused in Pipeline 0 |
+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+

Total database accesses: 309, total allocated memory: 216
```

Hinting a relationship type scan

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
```

```
USING SCAN i:INVENTED_BY
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator                     | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults              | c, cc, i, p, s, sc
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter                      | cc.formed = $autoint_3 AND cc:Country
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Expand(All)                 | (c)-[anon_2:PART_OF]->(cc)
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter                      | c:City
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Expand(All)                 | (p)-[anon_1:LIVES_IN]->(c)
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter                      | s.born = $autoint_0 AND s:Scientist
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Expand(All)                 | (sc)<-[anon_0:RESEARCHED]-(s)
|              0 |    0 |       0 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Filter                      | i.year = $autoint_1 AND p.born = $autoint_2 AND sc:Science AND p:Pioneer
|              0 |    0 |     204 |               |                     |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +DirectedRelationshipTypeScan | (p)-[i:INVENTED_BY]->(sc)
|            100 |  100 |     101 |           120 |                   9/0 |     0.910 | Fused in
Pipeline 0 |
+-------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
```

```
Total database accesses: 305, total allocated memory: 208
```

Query using multiple scan hints with a disjunction

Supplying multiple scan hints can also be useful if the query contains a disjunction (`OR`) in the `WHERE` clause. This makes sure that all involved label predicates are solved by a `UnionNodeByLabelsScan`.

Query

```
PROFILE
MATCH (person)
USING SCAN person:Pioneer
USING SCAN person:Scientist
WHERE person:Pioneer OR person:Scientist
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+--------------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| Operator              | Details                  | Estimated Rows | Rows | DB Hits | Memory (Bytes) |
Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------+--------------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| +ProduceResults       | person                   |            180 |  200 |       0 |                |
|           |                     |
| |                     +--------------------------+----------------+------+---------+----------------+
|           |                     |
| +UnionNodeByLabelsScan | person:Pioneer|Scientist |            180 |  200 |     202 |            120 |
6/0 |     1.740 | Fused in Pipeline 0 |
+-----------------------+--------------------------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+

Total database accesses: 202, total allocated memory: 184
```

Cypher will usually provide a plan that uses scans for a disjunction without hints. It may, however, decide to plan an `AllNodeScan` followed by a `Filter` instead, if the label predicates appear to be not very selective. In this case, the scan hints can be useful.

Join hints

Join hints are the most advanced type of hints, and are not used to find starting points for the query execution plan, but to enforce that joins are made at specified points. This implies that there has to be more than one starting point (leaf) in the plan, in order for the query to be able to join the two branches ascending from these leaves. Due to this nature, joins, and subsequently join hints, will force the planner to look for additional starting points, and in the case where there are no more good ones, potentially pick a very bad starting point. This will negatively affect query performance. In other cases, the hint might force the planner to pick a *seemingly* bad starting point, which in reality proves to be a very good one.

Hinting a join on a single node

In the example above using multiple index hints, we saw that the planner chose to do a join, but not on the `p` node. By supplying a join hint in addition to the index hints, we can enforce the join to happen on the `p` node.

Query

```
PROFILE
MATCH
  (s:Scientist {born: 1850})-[:RESEARCHED]->
  (sc:Science)<-[i:INVENTED_BY {year: 560}]-
  (p:Pioneer {born: 525})-[:LIVES_IN]->
  (c:City)-[:PART_OF]->
  (cc:Country {formed: 411})
USING INDEX s:Scientist(born)
USING INDEX cc:Country(formed)
USING JOIN ON p
RETURN *
```

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+-------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator         | Details                                                                 | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+-------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults  | c, cc, i, p, s, sc                                                      |              0 |
0 |       0 |               |                   0/0 |     0.000 |                     |
| |                +-------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                     |
| +NodeHashJoin    | p                                                                       |              0 |
0 |       0 |           432 |                       |           | In Pipeline 2       |
| |\               +-------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| | +Filter        | cache[p.born] = $autoint_2                                              |              1 |
0 |       0 |               |                       |           |                     |
| | |              +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| | +Expand(All)   | (c)<-[anon_1:LIVES_IN]-(p)                                              |              1 |
0 |       0 |               |                       |           |                     |
| | |              +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| | +Filter        | c:City                                                                  |              1 |
0 |       0 |               |                       |           |                     |
| | |              +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| | +Expand(All)   | (cc)<-[anon_2:PART_OF]-(c)                                              |              1 |
0 |       0 |               |                       |           |                     |
| | |              +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| | +NodeIndexSeek | RANGE INDEX cc:Country(formed) WHERE formed = $autoint_3                |              1 |
0 |       0 |           120 |                   0/0 |     0.000 | Fused in Pipeline 1 |
| |                +-------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +Filter          | i.year = $autoint_1 AND cache[p.born] = $autoint_2 AND p:Pioneer        |              0 |
0 |       1 |               |                       |           |                     |
| |                +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| +Expand(All)     | (sc)<-[i:INVENTED_BY]-(p)                                               |              1 |
1 |       3 |               |                       |           |                     |
| |                +-------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| +Filter          | sc:Science                                                              |              1 |
1 |       2 |               |                       |           |                     |
| |                +-------------------------------------------------------------------------+----------------
```

```
+------+---------+---------------+                          |          |                    |
| +Expand(All)     | (s)-[anon_0:RESEARCHED]->(sc)                                  |                    |             1 |
1 |       2 |               |                          |          |                    |
| |               +--------------------------------------------------------------------+----------------
+------+---------+---------------+                          |          |                    |
| +NodeIndexSeek   | RANGE INDEX s:Scientist(born) WHERE born = $autoint_0          |                    |             1 |
1 |       2 |           120 |                    6/1 |     0.515 | Fused in Pipeline 0 |
+------------------+--------------------------------------------------------------------+----------------
+------+---------+---------------+----------------------+----------+---------------------+

Total database accesses: 10, total allocated memory: 768
```

Hinting a join for an OPTIONAL MATCH

A join hint can also be used to force the planner to pick a `NodeLeftOuterHashJoin` or `NodeRightOuterHashJoin` to solve an `OPTIONAL MATCH`. In most cases, the planner will rather use an `OptionalExpand`.

Query

```
PROFILE
MATCH (s:Scientist {born: 1850})
OPTIONAL MATCH (s)-[:RESEARCHED]->(sc:Science)
RETURN *
```

Without any hint, the planner did not use a join to solve the `OPTIONAL MATCH`.

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+-------------------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+
| Operator             | Details                                               | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+----------------------+-------------------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+
| +ProduceResults      | s, sc                                                 |              1 |    1 |
0 |               |                        |           |                    |
| |                    +-------------------------------------------------------+----------------+------
+---------+---------------+                        |           |                    |
| +OptionalExpand(All) | (s)-[anon_0:RESEARCHED]->(sc) WHERE sc:Science        |              1 |    1 |
4 |               |                        |           |                    |
| |                    +-------------------------------------------------------+----------------+------
+---------+---------------+                        |           |                    |
| +NodeIndexSeek       | RANGE INDEX s:Scientist(born) WHERE born = $autoint_0 |              1 |    1 |
2 |           120 |                    6/0 |     0.560 | Fused in Pipeline 0 |
+----------------------+-------------------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+

Total database accesses: 6, total allocated memory: 184
```

Query

```
PROFILE
MATCH (s:Scientist {born: 1850})
OPTIONAL MATCH (s)-[:RESEARCHED]->(sc:Science)
USING JOIN ON s
RETURN *
```

Now the planner uses a join to solve the `OPTIONAL MATCH`.

Query plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| Operator              | Details                                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults       | s, sc                                               |              1 |    1 |
0 |               |                   2/0 |     0.213 |                    |
| |                     +-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+                    |
| +NodeLeftOuterHashJoin | s                                                  |              1 |    1 |
0 |          3112 |                       |     0.650 | In Pipeline 2      |
| |\                    +-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| | +Expand(All)        | (sc)<-[anon_0:RESEARCHED]-(s)                       |            100 |  100 |
300 |               |                       |           |                    |
| | |                   +-----------------------------------------------------+----------------+------
+---------+---------------+                       |           |                    |
| | +NodeByLabelScan    | sc:Science                                          |            100 |  100 |
101 |            120 |                   4/0 |     0.786 | Fused in Pipeline 1 |
| |                     +-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| +NodeIndexSeek        | RANGE INDEX s:Scientist(born) WHERE born = $autoint_0 |              1 |    1 |
2 |            120 |                   1/0 |     0.214 | In Pipeline 0      |
+-----------------------+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 403, total allocated memory: 3192
```

Search indexes

Search indexes retrieve ranked matches and return a score that indicates how closely each result matches a query. Unlike performance indexes, search indexes are queried explicitly rather than selected automatically by the Cypher planner. The Cypher `SEARCH` clause supports vector indexes from Neo4j 2026.01 and full-text indexes from Neo4j 2026.09. Built-in procedures remain available for compatibility with earlier Neo4j versions and Cypher 5.

Two search indexes are available in Neo4j:

- Full-text indexes provide lexical search over the content of `STRING` properties and return relevance scores.
- Vector indexes provide approximate nearest-neighbor similarity search over vectors. Vector search is commonly used for semantic search when the indexed vectors encode semantic meaning.

Full-text and vector indexes can also be used together for hybrid search. This is useful when an application needs both lexical matches, such as exact terms, names, acronyms, or identifiers, and semantic matches from vector search over embeddings. When combining results from different search indexes, rank each source independently rather than comparing raw score values directly. For a worked example, see Developer Guide → Hybrid search.

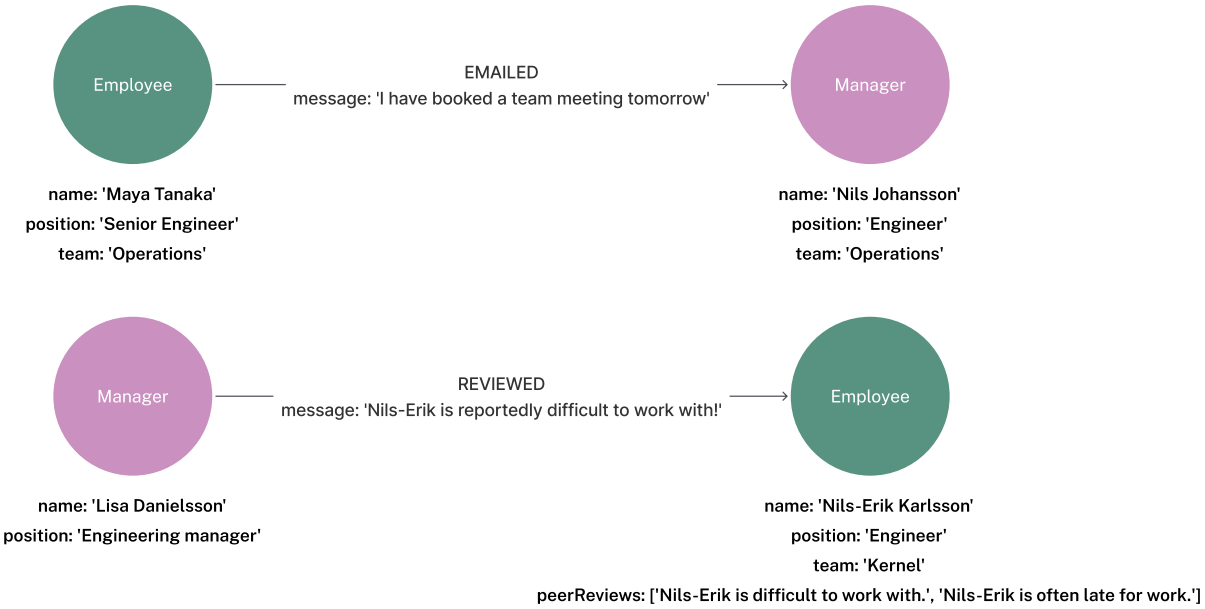
Full-text indexes

A full-text index is used to index nodes and relationships by `STRING` properties. Unlike range and text indexes, which can only perform limited `STRING` matching (exact, prefix, substring, or suffix matches), full-text indexes store individual words in any given `STRING` property. This means that full-text indexes can be used to match within the *content* of a `STRING` property. Full-text indexes also return a Lucene relevance score that indicates how closely each result matches the query.

Full-text indexes are powered by the Apache Lucene indexing and search library.

Example graph

The following graph is used for the examples below:



To recreate it, run the following query against an empty Neo4j database:

```
CREATE (nilsE:Employee {name: "Nils-Erik Karlsson", position: "Engineer", team: "Kernel", peerReviews:
['Nils-Erik is difficult to work with.', 'Nils-Erik is often late for work.']}),
(lisa:Manager {name: "Lisa Danielsson", position: "Engineering manager"}),
(nils:Employee {name: "Nils Johansson", position: "Engineer", team: "Operations"}),
(maya:Employee {name: "Maya Tanaka", position: "Senior Engineer", team:"Operations"}),
(lisa)-[:REVIEWED {message: "Nils-Erik is reportedly difficult to work with."}]->(nilsE),
(maya)-[:EMAILED {message: "I have booked a team meeting tomorrow."}]->(nils)
```

Create full-text indexes

Full-text indexes are created with the `CREATE FULLTEXT INDEX [index_name]` command. It is recommended to give the index a name when it is created. If no name is given when created, a random name will be assigned to the full-text index.

The `CREATE FULLTEXT INDEX` command is optionally idempotent. This means that its default behavior is to throw an error if an attempt is made to create the same index twice. If `IF NOT EXISTS` is appended to the command, no error is thrown and nothing happens should an index with the same name or a full-text index on the same schema already exist. Instead, an informational notification is returned showing the existing index which blocks the creation. The index name can also be given as a parameter, `CREATE`

`FULLTEXT INDEX $name FOR …`.

> Creating a full-text index requires the `CREATE INDEX` privilege.

When creating a full-text index, you need to specify the labels/relationship types and property names it should apply to.

This statement creates a full-text index named `namesAndTeams` on each `name` and `team` property for nodes with the label `Employee` or `Manager`:

Create a full-text index on a node label and property combination

```
CREATE FULLTEXT INDEX namesAndTeams IF NOT EXISTS
FOR (n:Employee|Manager) ON EACH [n.name, n.team]
```

This query highlights two key differences between full-text and Performance indexes:

- Full-text indexes can be applied to more than one node label.
- Full-text indexes can be applied to more than one property, but unlike composite performance indexes, a full-text index stores entities that have *at least* one of the indexed labels or relationship types, and *at least* one of the indexed properties.

Similarly, though a relationship can have only one type, a full-text index can store multiple relationship types. In that case, all types matching *at least one* of the relationship types and *at least one* of the indexed properties will be included.

This statement creates a full-text index named `communications` on the `message` property for the relationship types `REVIEWED` and `EMAILED`:

Create a full-text index on a relationship type and property combination

```
CREATE FULLTEXT INDEX communications FOR ()-[r:REVIEWED|EMAILED]-() ON EACH [r.message]
```

Tokenization and analyzers

Full-text indexes store individual words in a `STRING` property. This is achieved by a *tokenizer*, which breaks up a stream of characters into individual tokens (usually individual words). How a `STRING` is tokenized is determined by what *analyzer* the full-text index is configured with. The default analyzer (`standard-no-stop-words`) analyzes both the indexed values and the query string.

> **Note:**
>
> Stop words are common words in a language that can be filtered out during information retrieval tasks since they are considered to be of little use when determining the meaning of a string. These words are typically short and frequently used across various contexts.
>
> For example, the following stop words are included in Lucene's `english` analyzer: "a", "an", "and", "are", "as", "at", "be", "but", "by", "for", "if", "in", "into", "is", "it", "no", "not", "of", "on", "or", "such", "that", "the", "their", "then", "there", "these", "they", "this", "to", "was",

"will", and "with".

Removing stop words can help reduce the size of stored data and thereby improve the efficiency of data retrieval.

In some cases, using different analyzers for the indexed values and query string is more appropriate. For example, if handling `STRING` values written in Swedish, it may be beneficial to select the `swedish` analyzer, which knows how to tokenize Swedish words, and will avoid indexing Swedish stop words.

The `db.index.fulltext.listAvailableAnalyzers()` procedure shows all available analyzers.

Neo4j also supports the use of custom analyzers. For more information, see the Java Reference Manual → Full-text index analyzer providers.

Configuration settings

The `CREATE FULLTEXT INDEX` command takes an optional `OPTIONS` clause, where the `indexConfig` can be specified. The following statement creates a full-text index using a parameter for nodes with the label `Employee` or `Manager`.

Parameters

```
{
  "name": "peerReviews"
}
```

Create a full-text index using `OPTIONS`

```
CREATE FULLTEXT INDEX $name FOR (n:Employee|Manager) ON EACH [n.peerReviews]
OPTIONS {
  indexConfig: {
    `fulltext.analyzer`: 'english',
    `fulltext.eventually_consistent`: true
  }
}
```

The `fulltext.analyzer` setting can be used to configure an index-specific analyzer. In this case, it is set to the `english` analyzer. The possible values for the `fulltext.analyzer` setting can be listed with the `db.index.fulltext.listAvailableAnalyzers()` procedure. The `fulltext.eventually_consistent` setting, if set to `true`, will put the index in an *eventually consistent* update mode. This means that updates will be applied in a background thread "as soon as possible", instead of during a transaction commit, which is true for other indexes.

For more information on how to configure full-text indexes, refer to the Operations Manual → Indexes to support full-text search.

Query full-text indexes

Unlike performance text indexes, full-text search indexes perform lexical search over analyzed terms in `STRING` properties. They return a relevance score that indicates how closely the indexed terms match the query.

As of Neo4j 2026.09, the preferred way of querying full-text indexes is by using the SEARCH clause. As

an alternative, use either the `db.index.fulltext.queryRelationships()` procedure. These procedures are deprecated in Cypher 25 as of Neo4j 2026.10.

> An index cannot be used while its `state` is `POPULATING`, which occurs immediately after it is created. To check the `state` of a full-text index — whether it is `ONLINE` (usable) or `POPULATING` (still being built; the `populationPercent` column shows the progress of the index creation) — run the following command: `SHOW FULLTEXT INDEXES`.

Querying a full-text index using the SEARCH clause

Introduced in Neo4j 2026.09 | Cypher 25 only

This query uses the previously created full-text index `namesAndTeams` to look for "nils" among the employee and manager nodes:

Query a full-text index for a node property

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR "nils"
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, score
```

Result

node.name	score
"Nils Johansson"	0.3300700783729553
"Nils-Erik Karlsson"	0.27725890278816223

Rows: 2

For Neo4j versions before 2026.09, or when using Cypher 5, use the `db.index.fulltext.queryNodes()` procedure:

Query a full-text index for a node property Deprecated in Cypher 25 as of Neo4j 2026.10

```
CALL db.index.fulltext.queryNodes("namesAndTeams", "nils") YIELD node, score
RETURN node.name, score
```

> Many full-text index analyzers (including Neo4j's default analyzer) normalize tokens to lower case. Full-text indexes are therefore case-insensitive by default when used on Neo4j.

The `score` column represents the relevance of each result to the query string. Both indexed property values and queries are processed by the analyzer. Full-text search therefore matches analyzed terms rather than requiring the complete property value to equal the query string.

The `score` results are always returned in *descending* score order, where the best matching result entry is

put first. Full-text scores are meaningful within the result set from a full-text query. When combining full-text results with vector results or other scored sources, rank each source independently rather than comparing raw score values directly. For a worked example, see Developer Guide → Hybrid search.

Querying a relationship full-text index

This query uses the previously created `communications` full-text index to search for any `message` containing "meeting":

Query a full-text index for a relationship property

```
MATCH ()-[relationship]->()
  SEARCH relationship IN (
    FULLTEXT INDEX communications
    FOR "meeting"
    LIMIT 5
  )
  SCORE AS score
RETURN type(relationship), relationship.message, score
```

Result

type(relationship)	relationship.message	score
"EMAILED"	"I have booked a team meeting tomorrow."	0.3239005506038666

Rows: 1

To search for a sequence of terms as a phrase, enclose the phrase in double quotation marks. This matches the terms in order; it does not require the entire indexed property to equal the query string.

Query a full-text index for a phrase

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR '"Nils-Erik"'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, score
```

Result

node.name	score
"Nils-Erik Karlsson"	0.7588480710983276

Rows: 1

Query strings also support the use of the Lucene boolean operators (`AND`, `OR`, `NOT`, `+`, `-`):

Query a full-text index using logical operators

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR 'nils AND kernel'
```

```
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, node.team, score
```

Result

node.name	node.team	score
"Nils-Erik Karlsson"	"Kernel"	0.723090410232544

Rows: 1

It is possible to limit the search to specific properties, by prefixing `<propertyName>:` to the query string.

Query a full-text index for specific properties

```
MATCH (node)
  SEARCH node IN (
    FULLTEXT INDEX namesAndTeams
    FOR 'team:"Operations"'
    LIMIT 5
  )
  SCORE AS score
RETURN node.name, node.team, score
```

Result

node.name	node.team	score
"Nils Johansson"	"Operations"	0.21363800764083862
"Maya Tanaka"	"Operations"	0.21363800764083862

Rows: 2

A complete description of the Lucene query syntax can be found in the Lucene documentation.

Lists of `STRING` values

If the indexed property contains a list of `STRING` values, each entry is analyzed independently and all produced tokens are associated to the same property name. This means that when querying such an indexed node or relationship, there is a match if any of the list elements matches the query string. For scoring purposes, the full-text index treats it as a single-property value, and the score will represent how close the query is to matching the entire list.

Query a full-text index for content present in a list of `STRING` properties Deprecated in Cypher 25 as of Neo4j 2026.10

```
CALL db.index.fulltext.queryNodes('peerReviews', 'late') YIELD node, score
RETURN node.name, node.peerReviews, score
```

Result

node.name	node.peerReviews	score
"Nils-Erik Karlsson"	["Nils-Erik is difficult to work with.", "Nils-Erik is often late for work."]	0.13076457381248474

Show full-text indexes

To show all full-text indexes in a database, use the `SHOW FULLTEXT INDEXES` command:

Show all full-text indexes in a database

```
SHOW FULLTEXT INDEXES
```

Result

```
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------+
| id | name             | state    | populationPercent | type       | entityType     | labelsOrTypes
| properties       | indexProvider  | owningConstraint | lastRead                | readCount |
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------+
| 4  | "communications" | "ONLINE" | 100.0             | "FULLTEXT" | "RELATIONSHIP" | ["REVIEWED",
"EMAILED"] | ["message"]      | "fulltext-2.0" | NULL              | 2023-10-31T15:06:10.270Z | 2         |
| 3  | "namesAndTeams"  | "ONLINE" | 100.0             | "FULLTEXT" | "NODE"         | ["Employee",
"Manager"] | ["name", "team"] | "fulltext-2.0" | NULL              | 2023-10-31T15:07:48.874Z | 5         |
| 6  | "peerReviews"    | "ONLINE" | 100.0             | "FULLTEXT" | "NODE"         | ["Employee",
"Manager"] | ["peerReviews"]  | "fulltext-2.0" | NULL              | 2023-10-31T15:09:05.391Z | 3         |
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------+
```

The `SHOW` command can be filtered for particular columns:

Show full-text indexes using filtering

```
SHOW FULLTEXT INDEXES WHERE name CONTAINS "Team"
```

Result

```
+----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------+
| id | name             | state    | populationPercent | type       | entityType | labelsOrTypes
| properties       | indexProvider  | owningConstraint | lastRead | readCount |
+----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------+
| 5  | "namesAndTeams" | "ONLINE" | 100.0             | "FULLTEXT" | "NODE"     | ["Employee", "Manager"]
| ["name", "team"] | "fulltext-2.0" | NULL              | NULL     | 0         |
+----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------+
```

To return full index details, use the `YIELD` clause. For example:

Show all full-text indexes and all return columns

```
SHOW FULLTEXT INDEXES YIELD *
```

Result

```
+----------------------------------------------------------------------------------------------------
```

```
+--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
---------------------+
| id | name             | state    | populationPercent | type       | entityType     | labelsOrTypes
| properties       | indexProvider  | owningConstraint | lastRead | readCount | trackedSince             |
options                                                                                                      |
failureMessage | createStatement
|
+--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
---------------------+
| 4  | "communications" | "ONLINE" | 100.0            | "FULLTEXT" | "RELATIONSHIP" | ["REVIEWED",
"EMAILED"] | ["message"]      | "fulltext-2.0" | NULL             | NULL     | 0         | 2023-11-
01T09:27:57.024Z | {indexConfig: {`fulltext.analyzer`: "standard-no-stop-words",
`fulltext.eventually_consistent`: FALSE}} | ""             | "CREATE FULLTEXT INDEX `communications` FOR
()-[r:`REVIEWED`|`EMAILED`]-() ON EACH [r.`message`] OPTIONS {indexConfig: {`fulltext.analyzer`:
'standard-no-stop-words',`fulltext.eventually_consistent`: false}}"  |
| 5  | "namesAndTeams"  | "ONLINE" | 100.0            | "FULLTEXT" | "NODE"         | ["Employee",
"Manager"] | ["name", "team"] | "fulltext-2.0" | NULL             | NULL     | 0         | 2023-11-
01T12:24:48.002Z | {indexConfig: {`fulltext.analyzer`: "standard-no-stop-words",
`fulltext.eventually_consistent`: FALSE}} | ""             | "CREATE FULLTEXT INDEX `namesAndTeams` FOR
(n:`Employee`|`Manager`) ON EACH [n.`name`, n.`team`] OPTIONS {indexConfig: {`fulltext.analyzer`:
'standard-no-stop-words',`fulltext.eventually_consistent`: false}}"  |
| 6  | "peerReviews"    | "ONLINE" | 100.0            | "FULLTEXT" | "NODE"         | ["Employee",
"Manager"] | ["peerReviews"]  | "fulltext-2.0" | NULL             | NULL     | 0         | 2023-11-
01T12:25:41.495Z | {indexConfig: {`fulltext.analyzer`: "english", `fulltext.eventually_consistent`: TRUE}}
| ""             | "CREATE FULLTEXT INDEX `peerReviews` FOR (n:`Employee`|`Manager`) ON EACH
[n.`peerReviews`] OPTIONS {indexConfig: {`fulltext.analyzer`: 'english',`fulltext.eventually_consistent`:
true}}"                       |
+--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
--------------------------------------------------------------------------------------------------------------
---------------------+
```

For a full description of all return columns, see Performance indexes → Show indexes → Return columns.

Drop full-text indexes

A full-text node index is dropped by using the same command as for other indexes, `DROP INDEX`.

In the following example, the previously created `communications` full-text index is deleted from the database:

Drop a full-text index

```
DROP INDEX communications
```

The index name can also be given as a parameter when dropping an index: `DROP INDEX $name`.

List of full-text index procedures

The procedures for full-text indexes are listed in the table below:

Usage	Procedure/Command	Description
Eventually consistent indexes.	`db.index.fulltext.awaitEventuallyConsistentIndexRefresh()`	Wait for the updates from recently committed transactions to be applied to any eventually-consistent full-text indexes.
List available analyzers.	`db.index.fulltext.listAvailableAnalyzers()`	List the available analyzers that the full-text indexes can be configured with.
Use full-text node index.	`db.index.fulltext.queryNodes()`	Query the given full-text index. Returns the matching nodes and their Lucene query score, ordered by score. This procedure is deprecated in Cypher 25 as of Neo4j 2026.10.
Use full-text relationship index.	`db.index.fulltext.queryRelationships()`	Query the given full-text index. Returns the matching relationships and their Lucene query score, ordered by score. This procedure is deprecated in Cypher 25 as of Neo4j 2026.10.

Summary

- Full-text indexes support the indexing of both nodes and relationships.
- Full-text indexes include only property values of types `STRING` or `LIST<STRING>`.
- Full-text indexes can be queried with the `SEARCH` clause or Cypher procedures.
- Full-text indexes return the score for each result from a query.
- Full-text indexes support configuring custom analyzers, including analyzers that are not included with Lucene itself.
- Full-text indexes can be queried using the Lucene query language.
- Full-text indexes are kept up to date automatically, as nodes and relationships are added, removed, and modified.
- Full-text indexes will automatically populate newly created indexes with the existing data in a store.
- Full-text indexes can be checked by the consistency checker, and they can be rebuilt if there is a problem with them.
- Newly created full-text indexes get automatically populated with the existing data in the database.
- Full-text indexes can support any number of properties in a single index.
- Full-text indexes are created, dropped, and updated transactionally, and are automatically replicated throughout a cluster.
- Full-text indexes can be configured to be *eventually consistent*, in which index updating is moved from the commit path to a background thread. Using this feature, it is possible to work around the slow Lucene writes from the performance-critical commit process, thus removing the main bottlenecks for

Neo4j write performance.

Vector indexes

Vector indexes enable similarity searches and complex analytical queries by representing nodes or properties as vectors in a multidimensional space. Vector search is often used on its own for semantic matching, but it can also be combined with full-text search or other ranked result sources for hybrid search. For a worked example, see Developer Guide → Hybrid search.

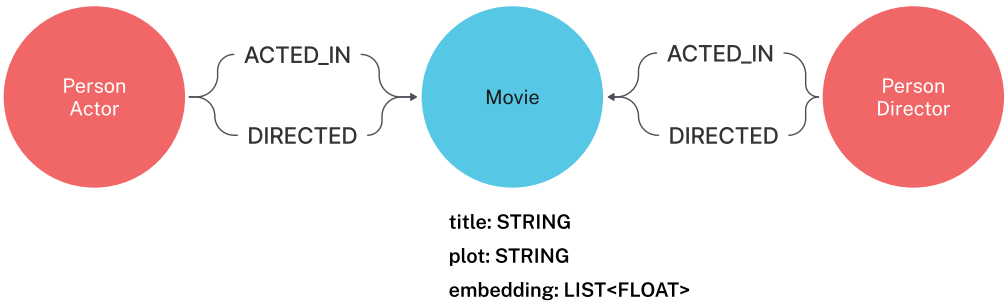
The following resources provide hands-on tutorials for working with LLMs and vector indexes in Neo4j:

- GraphAcademy → Neo4j & GenerativeAI Fundamentals
- GraphAcademy → Introduction to Vector Indexes and Unstructured Data
- GenAI documentation → Embeddings & Vector Indexes Tutorial

Neo4j vector indexes are powered by the Apache Lucene indexing and search library.[3]

Example graph

The examples on this page use the Neo4j movie recommendations dataset, focusing on the `plot` and `embedding` properties of `Movie` nodes. The `embedding` property consists of a 1536-dimension vector embedding of the `plot` and `title` property combined.



The graph contains 28863 nodes and 332522 relationships.

To recreate the graph, download and import this recommendations-5.26.dump file to an empty Neo4j database.

- Aura instances: select *Restore from file* and drag and drop the dump file. For more information, see Aura Documentation → Restore from backup file.
- On-prem instances: Use the `neo4j-admin database load` command to import the dump file and the `neo4j-admin database migrate` command to migrate it to the Neo4j version your instance is running. Then create a new database with the **same name** as the dump file.

```
bin/neo4j-admin database load recommendations-5.26 --from-path=./
bin/neo4j-admin database migrate recommendations-5.26
bin/cypher-shell -u neo4j -p <databasePassword> -d system 'CREATE DATABASE `recommendations-5.26`
WAIT'
```

The dump file used to load the dataset contains embeddings generated by OpenAI, using

the model `text-embedding-ada-002`.

Vectors and embeddings in Neo4j

Vector indexes allow you to query vector embeddings from large datasets. An embedding is a numerical representation of a data object, such as a text, image, or document. Each word or token in a text is typically represented as high-dimensional vector where each dimension represents a certain aspect of the word's meaning.

The embedding for a particular data object can be created by both proprietary (such as Vertex AI or OpenAI) and open source (such as sentence-transformers) embedding generators, which can produce vector embeddings with dimensions such as 256, 768, 1536, and 3072. Embeddings can also represent graph structure, not only the semantic content of text, images, or documents. For example, Neo4j Graph Data Science node embedding algorithms can write embeddings that capture graph topology to node properties. Those properties can then be indexed with a vector index and queried with Neo4j vector search as a structural search signal, either on their own or as one source in a hybrid search query.

Vector embeddings are stored as `LIST<INTEGER | FLOAT>` properties on a node or relationship. As of Neo4j 2025.10, they can also be more efficiently stored as `VECTOR` properties using Cypher 25.

Note:

Vector indexes are available in both Neo4j Enterprise Edition and Community Edition. Community Edition can index embeddings stored as `LIST<INTEGER | FLOAT>` properties. Storing embeddings as native `VECTOR` properties requires block format, which is available in Enterprise Edition and Aura.

Note:

For information about how embeddings can be generated and stored as properties, see:

- GenAI documentation → Create and store embeddings in a Neo4j database
- GenAI documentation → Embeddings & Vector Indexes Tutorial

For example, the movie The Godfather, has the following `plot`: `"The aging patriarch of an organized crime dynasty transfers control of his clandestine empire to his reluctant son."` This is its 1536-dimensional `embedding` property, where each coordinate in the `LIST` or `VECTOR` represents a particular aspect of the plot's meaning:

```
[0.005967312026768923, -0.03817005082964897, 0.0014667075593024492, -0.03868866711854935,
-0.006505374796688557, 0.020900176838040352, -0.0027551413513720036, -0.0024731445591896772,
-0.03734026849269867, -0.02228747308254242, 0.028783122077584267, 0.017905177548527718,
0.011396560817956924, 0.014235977083444595, 0.023143187165260315, -0.014184115454554558,
0.029846282675862312, -0.011928141117095947, 0.018838683143258095, -0.0019172541797161102,
0.0033483069855719805, 0.009497134014964104, -0.03516208380460739, 0.0021441481076180935,
0.002657901030033827, 0.0030760341323912144, 0.004255882930010557, -0.020809419453144073,
0.02358401007950306, -0.013808120042085648, 0.01064456906169653, -0.006975369527935982,
0.007318951655179262, -0.013872946612536907, 0.005905726458877325, -0.010689947754144669,
0.0020225979387760162, -0.016245609149336815, -0.00038815077277831733, -0.007163367234170437,
0.027668101713061333, 0.007215228863060474, -0.009380445815622807, -0.02956104464828968,
-0.000863007502630353, 0.012142069637775421, 0.0012957267463207245, -0.027953339740633965,
-0.016414159908890724, 0.008453421294689178, -0.0010777463903650641, 0.03311355784535408,
```

```
-0.013639570213854313, -0.052457891404628754, 0.0010242642601951957, 0.0034390646032989025,
-0.01049546804279089, 0.006456754636019468, 0.003970644902437925, -0.011629937216639519,
0.005280147306621075, -0.023402493447065353, -0.014689764939248562, -0.007623638026416302,
-0.002453696448355913, 0.02290981076657772, 0.0017989451298490167, 0.0013427261728793383,
-0.001776255783624947, -0.002414800226688385, 0.04833490028977394, 0.031142819672822952,
-0.0033013075590133667, 0.017879245802760124, 0.0070077828131616116, -0.016154851764440536,
-0.005772831384092569, 0.019875913858413696, -0.018008900806307793, 0.012764407321810722,
0.0055232481099665165, -0.027901478111743927, -0.0034909259993582964, 0.0307279285043478,
0.006472961511462927, 0.008861830458045006, -0.01802186481654644, 0.018281172960996628,
-0.014223011210560799, -0.00018313586770091206, 0.0026352116838097572, 0.0006754148053005338,
0.014975002966821194, 0.024361930787563324, -0.017166150733828545, 0.0028880364261567593,
0.011824417859315872, 0.01710132323205471, -0.0005003822734579444, -0.018890544772148132,
-0.002192768268287182, -0.0018264965619891882, 0.011033530347049236, -0.009095207788050175,
-0.022689398378133774, -0.004281813744455576, 0.007092057727277279, -0.015247276052832603,
0.024115590378642082, 0.002996621420606971, -0.02834230102598667, 0.030546413734555244,
0.02350621670484543, -0.020511215552687645, 0.010190781205892563, -0.016582708805799484,
0.028238577768206596, -0.011967036873520188, 0.011623455211520195, -0.02797926962375641,
0.0026254875119775534, 0.018307102844119072, 0.0038701631128787994, -0.03850715234875679,
0.006246067117899656, -0.0006312514888122678, 0.010352848097682, -0.02358401007950306,
-0.026708664372563362, -0.002863726345822215, 0.035862214863300323, 0.009860164485871792,
-0.01726987399160862, 0.004275330808013678, -0.02663087099790573, 0.009140586480498314,
-0.013872946612536907, 0.019136887043714523, -0.020835351198911667, -0.0250879917293787,
0.03044269047677517, 0.026280807331204414, -0.013406192883849144, 0.006683648563921452,
-0.01216800045222044, 0.007824601605534554, 0.031505849212408066, 0.023726629093289375,
0.0294832531362772, -0.013678465969860554, 0.033891480416059494, 0.009211895987391472,
0.017088359221816063, -0.02183368429541588, 0.01847565360367298, 0.004644844215363264,
-0.009834233671426773, -0.011344699189066887, -0.0006725785788148642, 0.00012691882147919387,
0.015338033437728882, 0.025736261159181595, -0.0039674034344216976, -0.007312469184398651,
-0.01312743779271841, 0.02350621670484543, -0.0006843284936621785, -0.011785522103309631,
0.006570201832801104, -0.004187814891338348, -0.0070013003423810005, 0.0165178831666708,
-0.004537879955023527, 0.022715330123901367, -0.0025120405480265617, 0.025580676272511482,
0.005053253378719091, -0.0020063910633325577, -0.039285074919462204, -0.001816772622987628,
0.0007224142318591475, 0.0161029901355505, 0.040866848081350 33, 0.03536953032016754, 0.009626788087189198,
-0.023571044206619263, -0.009607339277863503, 0.011085391975939274, 0.020835351198911667,
-0.0009027139167301357, -0.007584741804748774, 0.016958704218268394, 0.011130770668387413,
-0.016829051077365875, -0.6712950468063354, -0.006511857267469168, -0.024854615330696106,
-0.02663087099790573, -0.00008933950448408723, 0.0061779990792274475, 0.004605947993695736,
0.013231161050498486, -0.020187081769108772, 0.007986668849732399, -0.001847565290518105,
0.04086684808135033, 0.007519915234297514, 0.0040808506309986115, -0.034021131694316864,
-0.01997963711619377, -0.004972219467163086, -0.023220978677272797, 0.012129104696214199,
0.0018329792656004429, -0.011649386025965214, 0.028446022421121597, -0.0010356089333072305,
-0.006223377771675587, 0.021211346611380577, 0.004006299655884504, 0.021937407553195953,
-0.02927580662071705, -0.01129283756017685, -0.009296170435845852, -0.01864420250058174,
0.027175417169928 55, -0.0003555347793735564, 0.0021700789220631123, 0.048360832035541534,
-0.002277043182402849, -0.009049829095602036, 0.033969271928071976, 0.004557327833026648,
0.018916476517915726, -0.000779542897362262, -0.00638544512912631, 0.022183749824762344,
-0.012757924385368824, -0.027149485424160957, -0.012278205715119839, 0.0238303504884243,
-0.02963883802294731, 0.005218561738729477, -0.004434156697243452, 0.013665501028299332,
-0.0024520757142454386, 0.002124700229614973, -0.007273572962731123, -0.0035654769744724035,
-0.002862105611711740 5, 0.020640870556235313, 0.01091684214770794, -0.0006867594784125686,
-0.011694764718413353, 0.011215046048164368, 0.016504917293787003, 0.00827838946133852,
-0.0044471221044659615, 0.010676982812583447, 0.027771824970841408, -0.0133802630007267,
0.029820352792739868, 0.008349698968231678, -0.014573076739907265, -0.009017415344715118,
0.011655868031084538, -0.0061066895723342896, -0.013082059100270271, 0.004353123251348734,
0.00672254478558898, 0.01773662678897381, 0.012433790601789951, 0.023843316361308098,
0.015221345238387585, -0.0046221548691391945, -0.00026214358513243496, -0.016582708805799484,
0.016504917293787003, 0.028005201369524002, 0.005516765173524618, -0.043096892535568649,
0.013743292540311813, -0.0064308238215744495, -0.007176332641392946, 0.019110957160592 08,
0.00446332897990942, -0.012971853837370872, -0.016919808462262154, 0.010048162192106247,
0.0032769974786788225, -0.021548446267843246, 0.001816772622987628, 0.018566410988569 26,
-0.04804966226220131, 0.007286538369953632, -0.007299503777176142, -0.014080392196774483,
0.008952588774263859, 0.023908143863081932, 0.012932957150042057, -0.008433973416686058,
0.012783855199813843, 0.0430709607899189, -0.01015836838632822, 0.03534360229969025,
-0.007584741804748774, -0.016453055664896965, -0.005720969755202532, -0.014871280640363693,
-0.026540113613009453, 0.005228285677731037, 0.0004019264888484031, 0.005931657273322344,
-0.02533433400094509, -0.018825719133019447, 0.0023353875149041414, 0.0014059323584660888,
-0.02020004764199257, 0.022481953725218773, 0.034980569034814835, -0.02709762565791607,
-0.022974636405706406, -0.025023166090250015, 0.006417858414351 94, -0.0001982283574761822 8,
-0.004845807328820229, 0.0003723492263816297, -0.010132437571883202, 0.014987968839704 99,
0.001948046963661909, -0.0020161152351647615, -0.008842382580041885, 0.0223652645945549,
-0.013574742712080479, -0.002369421534240246, 0.003275376744568348, 0.005879795644432306,
0.005789037793874741, 0.006359514314681292, -0.035499185323715 21, 0.003118171589449048,
-0.026993902400135994, -0.01614188589155674, 0.011578075587749481, 0.000852473138365894 6,
-0.013367297127842903, 0.004194297362118959, 0.019331367686390877, 0.006152068264782429,
-0.015208380296826363, -0.0018005658639594913, -0.015714028850197792, -0.01681608520448208,
-0.028990568593144417, 0.010676982812583447, 0.024595309048891068, -0.045560311526060104,
-0.0009262136882171035, 0.014845349825918674, -0.020887212827801704, 0.015739960595965385,
0.011727177537977695, 0.0012560202740132809, -0.023052429780364037, 0.001424570102244615 6,
-0.013062611222267151, -0.011299320496618748, 0.022274507209658623, 0.011338216252624989,
-0.007908876053988934, 0.010339883156120777, -0.006132620386779308, 0.012479169294238 09,
```

```
-0.007947771809995174, -0.0025347298942506313, -0.011416008695960045, 0.011027047410607338,
0.004521673079580069, 0.04880165681242943, 0.0012543996563181281, 0.02115948498249054, 0.0165178831666708,
-0.025373229756951332, 0.026125222444534302, -0.0031262750271707773, 0.007669016718864441,
0.003821542952209711, -0.021561412140727043, 0.008187631145119667, 0.02358401007950306,
0.022494917735457 42, 0.015247276052832603, -0.004560569301247597, 0.030753860250115395,
0.031090958043932915, -0.021457688882946968, 0.027694031596183777, -0.004823117982596159,
0.0049171168357133865, -0.018346000462770462, -0.0030355174094438553, -0.011176149360835552,
0.024102624505758286, 0.006923507899045944, 0.010009266436100006, -0.00510187353938818,
0.0007916979375295341, -0.004722636193037033, 0.019914809614419937, 0.026190048083662987,
-0.013289504684507847, 0.006346548907458782, -0.015415825881063938, -0.026734594255685806,
0.003623821074143052, 0.005325525999069214, -0.003922024741768837, -0.00640813447535038,
-0.014624938368797302, -0.0065021333284676075, 0.007435640320181847, -0.002808623481541872,
0.010138919577002525, -0.033813685178756714, -0.0032008260022848845, 0.01614188589155674,
-0.018994268029928207, 0.008135770447552204, -0.008596041239798069, -0.015662167221307755,
0.004310985561460257, -0.014663834124803543, 0.014962038025259972, -0.03479905426502228,
0.013114472851157188, 0.01341915875673294, 0.05092797800898552, -0.011908693239092827,
0.005532008935511112, -0.013367297127842903, 0.02501020021736622, -0.00029678543796762824,
-0.024543447420001 03, 0.003152205841615796, -0.015454721637070179, 0.010028714314103127,
-0.02102983184158802, -0.0032624113373458385, 0.03583628311753273, -0.015026864595711231,
0.00672254478558898, 0.000010907877367571928, 0.019875913858413696, 0.020161151885986328,
0.014054462313652039, -0.005675591062754393, -0.009224860928952694, 0.014793488197028637,
0.03687351569533348, -0.005442214198410511, 0.005633453372865915, -0.0030436208471655846,
-0.012615305371582508, -0.009075759910047054, 0.017192082479596138, -0.002220319816842675,
0.005798762198537588, -0.0007568534929305315, 0.010378778912127018, 0.005908967927098274,
-0.0158825796097517, 0.0088812792673707, 0.007766257040202618, -0.0030209312681108713,
-0.013561777770519257, -0.035395462065935135, 0.022391194477677345, -0.0027049004565924406,
0.004748567007482052, -0.020433424040675163, -0.00028706141165457666, -0.005092149134725332,
-0.018371930345892906, 0.006009449250996113, -0.00645027169957757, 0.015286171808838844,
-0.012343033216893673, -0.008628454059362411, -0.010605673305690289, 0.009192448109388351,
0.007500466890633106, -0.013535846956074238, 0.003831267124041915, -0.02956104464828968,
0.0009724028059281409, 0.0034585127141326666, -0.00004074468961334787, -0.025139853358268738,
0.012278205715119839, 0.023519182577729225, -0.012913509272038937, -0.006301170215010643,
0.0037178201600909233, 0.004716153722256422, -0.017905177548527718, 0.009769407100975513,
-0.019746258854866028, -0.011675315909087658, 0.007409709505736828, -0.022676432505249977,
-0.013406192883849144, 0.003922024741768837, 0.03925914317369461, -0.011325251311063766,
-0.014611972495913506, -0.022404160350561142, -0.03311355784535408, 0.0024634203873574734,
0.1057974249124527, 0.014145219698548317, 0.025956671684980392, 0.006878129206597805,
-0.019914809614419937, -0.019162818789482117, -0.009231343865394592, -0.04423784464597702,
0.012018898501992226, -0.00921837892383337, 0.02408965863287449, -0.026501217857003212,
0.020225977525115013, 0.005014357157051563, 0.02053714729845524, 0.014521215111017227,
-0.002670866437256336, -0.020433424040675163, -0.0015372068155556917, -0.031168751418590546,
0.0051213214173913, 0.006865163799375296, 0.010048162192106247, 0.003795612370595336,
-0.009749959222972393, -0.024063728749752045, 0.026449356228113174, 0.00967864878475666,
-0.009049829095602036, -0.012284688651561737, -0.02475089207291603, 0.0034844432957470417,
-0.00928320549428463, 0.011772556230425835, -0.01811262220144272, -0.01918874867260456,
0.009043346159160137, 0.023843316361308098, 0.02580108679831028, 0.005980277433991432,
0.029327668249607086, -0.008103356696665287, 0.008083908818662167, -0.005490834359079599,
0.021146519109606743, -0.0023499734234064817, -0.03298390284180641, 0.005283388774842024,
-0.00043352958164177835, -0.024271173402667046, 0.03181701898574829, -0.000028944177756784484,
-0.004479535389691591, -0.002066355897113681, 0.017995934933423996, -0.012783855199813843,
0.013859981670975685, -0.006615580525249243, -0.0008403180981986225, 0.025489918887615204,
-0.01789221167564392, -0.03189481049776077, 0.00028949242550879717, -0.03251715004444122,
0.03588814660906792, -0.03500650078058243, -0.007869980297982693, -0.024361930787563324,
0.00451519014313817, -0.018177449703216553, 0.020627904683351517, 0.003249445930123329,
0.010962220840156078, -0.005299595184624195, 0.048023734241724014, -0.0033094107639044523,
0.012971853837370872, -0.02290981076657772, 0.017918141558766365, -0.016245609149336815,
-0.013179299421608448, -0.020589008927345276, 0.0037469922099262476, -0.029327668249607086,
-0.007383778691291809, 0.013017232529819012, 0.006327101029455662, -0.02689017914235592,
-0.004385536536574364, 0.005789037793874741, -0.005597798619419336, -0.004152160137891769,
0.012719028629362583, -0.008220044896006584, -0.01702353172004223, -0.011506766080856323,
0.0042980206198990345, 0.0018702547531574965, -0.0032964455895125866, 0.007267090491950512,
-0.009581409394741058, -0.005818210076540708 5, -0.005429248791188002, -0.008829417638480663,
-0.0030403793789446354, -0.01194110605865717, -0.002591453492641449, 0.005756624508649111,
-0.01618078351020813, -0.009821268729865551, -0.00021210535487625748, -0.01768476516008377,
-0.0005562954465858638, -0.017451388761401176, -0.015545479021966457, 0.02332470193505287,
0.015960371121764183, 0.02208002656698227, 0.01369143184274435, -0.014495284296572208,
-0.007701430004090071, -0.0005567006301134825, 0.027590308338403702, 0.05188741534948349,
-0.023609939962625504, -0.017957039177417755, 0.015999266877770424, -0.020900176838040352,
0.0030387586448341 61, 0.021042795851826668, -0.009814785793423653, 0.0014083633432164788,
0.010897394269704819, -0.0167253278195858, -0.020135220140218735, -0.005273664370179176,
-0.009788854978978634, -0.002986897248774767, -0.008764590136706829, -0.006729027256369591,
-0.018449721857905388, -0.009166517294943333, -0.002651418326422572, 0.008245975710451603,
0.0034358231350779533, -0.028757192194461823, 0.01511762198060751, -0.008544179610908031,
0.005344973877072334, 0.013924808241426945, -0.003299686824902892, -0.04143732413649559,
-0.000840318098 1986225, 0.010949255898594856, -0.013600673526525497, -0.03448788449168205,
-0.007863497361540794, -0.01809965819120407, -0.00444063963368535, 0.004920358303934336,
0.0330357663333416, -0.008816451765596867, 0.006683648563921452, 0.00823301076889038,
-0.015947405248880386, 0.02608632668852806, 0.0037243026308715343, -0.007623638026416302,
-0.028031131252646446, 0.027123555541038513, 0.01843675784766674, 0.016712361946702003,
0.040374163538217545, -0.0021538722794502974, 0.01885164901614189, -0.011740143410861492,
```

```
0.017490284517407417, -0.0004517621418926865, -0.00034439266892150044, -0.026190048083662987,
-0.021729961037635803, 0.0020209772046655416, -0.014521215111017227, -0.01467679999768734,
0.002505557844415307, -0.010618634247251 51, 0.015623271465301514, -0.010087057948112488,
-0.0031748951878398657, 0.01631043665111065, 0.016375262290239334, -0.013257091864943504,
0.010741809383034706, -0.012932957150042057, -0.002484489232301712, 0.0027324517723172903,
0.00897203665226698, -0.004793945699930191, 0.0043466403149068356, -0.0020047705620527267,
0.0021538722794502974, 0.021263208240270615, -0.0269679706543684, -0.024115590378642082,
-0.0025833500549197197, 0.030598275363445282, 0.002772968728095293, 0.015843683853745 46,
0.006981851998716593, -0.0037113374564796686, -0.01273199450224638, -0.026280807331204414,
-0.02182071842253208, -0.049527715891599655, 0.021950373426079 75, -0.008628454059362411,
-0.004353123251348734, 0.01064456906169653, -0.009698097594082355, -0.04094463959336281,
0.0238303504884243, 0.0034649954177439213, 0.032802388072013855, 0.0002048123424174264,
0.022507883608341217, 0.03770329803228378, -0.010346366092562675, 0.0028588641434907913,
0.026410460472106934, 0.019085025414824486, 0.008848865516483784, 0.015830717980861664,
-0.004469811450690031, 0.013808120042085648, -0.012031864374876022, -0.02099093608558178,
-0.006054827943444252, -0.045638103038072586, -0.024050762876868248, 0.014417491853237152,
0.012187448330223 56, 0.0032413427252322435, -0.013302470557391644, -0.0003156257444061339,
0.006942956242710352, 0.005426007788628 34, -0.0034358231350779533, 0.022067060694098473,
-0.013847015798091888, -0.026942040771245956, -0.0334506556391716, -0.01835896447300911,
-0.0021036313846707344, -0.001962633104994893, 0.012615305371582508, -0.0186053067445755,
0.01572699472308159, -0.02542509138584137, 0.019422125071287155, -0.013950739055871964,
-0.002110114088281989, 0.020524181425571 44, -0.0014197081327438354, 0.0010485743405297399,
-0.004372571129351854, 0.0069299908354878426, -0.005105114541947842, -0.003756716148927808,
-0.015960371121764183, 0.025554746389389038, 0.003516856813803315, 0.005951105151325464,
0.009736993350088596, 0.043459922075271606, -0.008952588774263859, 0.021315069869160652,
-0.011318768374621868, -0.016375262290239334, -0.004560569301247597, -0.026656802743673325,
0.004842565860599279, 0.0004894427256658673, -0.023635871708393097, 0.007448605261743069,
-0.008965553715825081, 0.0026092808693647385, -0.01999260112643242, -0.007811635732650757,
0.012142069637775421, -0.013756258413195 61, -0.02102983184158802, -0.006806819699704647,
0.015869613736867905, -0.0074032265692949295, -0.001892944099381566, -0.0037016132846474648,
-0.005322284530848265, 0.032932043075561 52, -0.014430457726120949, 0.0418262854218483,
-0.012641236186027527, 0.018216345459222794, -0.028290439397096634, 0.02576219104230404,
0.008433973416686058, 0.013963703997433186, 0.030598275363445282, -0.01225227490067482,
0.012051312252879143, 0.0014553628861904144, -0.008822934702038765, 0.011001116596616232,
0.009860164485871792, -0.004388778004795313, -0.01685498282313347, 0.01091035921126604,
-0.00033223762875422835, -0.007850532419979572, -0.0006320617976598442, 0.002114976057782769,
-0.007532880175858736, 0.017101323232054 71, 0.015610306523740292, -0.009036863222718239,
0.008200597018003464, 0.012174483388662338, 0.004473052918910 98, 0.0186053067445755,
-0.019253576174378395, 0.010638087056577206, -0.02086128108203411, 0.022404160350561142,
0.010437123477458954, 0.0006920266896486282, -0.02128913812339306, -0.009296170435845852,
-0.004106780979782343, 0.044808320701122284, -0.013782189227640629, -0.003750233445316553,
-0.01181145291775465, 0.02764216996729374, 0.011960554867982864, -0.005043528974056244,
0.006155309733003378, -0.015584375709295273, 0.012433790601789951, -0.021600307896733284,
-0.04314875230193138, -0.012148552574217 32, -0.024776823818683624, 0.039077628403902054,
0.016271540895104408, 0.000348039175150916, -0.01511762198060751, 0.0014926382573321462,
-0.04068533331155777, -0.0020290804095566273, -0.006904060021042824, 0.02099093608558178,
0.017049461603164673, -0.006981851998716593, 0.007364330347627401, 0.007416191976517439,
0.00766253424808383, 0.021535480394959 45, -0.002995000686496496, 0.021574376 15096569,
-0.011312286369502544, -0.009685131721198559, 0.004148918669670 82, -0.009672166779637337,
-0.013088542036712 17, -0.003380720503628254, -0.003168412484228611, 0.013769223354756832,
-0.012615305371582508, 0.007973702624440193, 0.001315985107794404, -0.006139102857559919,
-0.028212646022439003, 0.0004906582762487233, 0.0006340876570902765, 0.013289504684507847,
-0.010359331034123898, -0.029561044648289 68, 0.026345632970333 1, 0.026215979829430 58,
0.005357939284294844, -0.022754225879907608, -0.009393410757184029, 0.007053161505609751,
-0.018086692318320274, -0.0012552099069580436, 0.003977127373218536, -0.010839049704372883,
-0.01584368385374546, 0.007753291632980108, 0.005951105151325464, 0.024789789691567 42,
-0.008589558303356 17, 0.007280055433511734, 0.013257091864943504, -0.0000065713156800484285,
0.007234676741063595, -0.004139194730669 26, -0.014676799997687 34, -0.018333034589886665,
-0.017658835276961327, -0.016816085204482 08, 0.005108356010168791, -0.007630120497196913,
0.008479352109134197, -0.027719963341951 37, 0.004567051772028208, -0.018579376861453056,
-0.003983610309660435, -0.0023110774345695972, 0.023065393790602684, 0.042811654508113 86,
-0.015273206867277622, -0.006696613971143961, 0.002272181212902069, -0.008356180973351002,
-0.014508250169456005, -0.0066090975888073444, 0.008278389461338 52, -0.016906842589378357,
0.003750233445316553, -0.008524730801582336, -0.0022802846506237984, -0.005156976170837879,
-0.009633270092308521, -0.035940006375312805, -0.004323950968682766, 0.027771824970841408,
0.19261354207992554, -0.014547145925462246, -0.006657717749476433, 0.013808120042085648,
-0.021340999752283096, 0.011869796551764011, 0.024115590378642082, 0.014080392196774483,
0.0023856281768530607, 0.0005133476224727929, -0.016206713393330574, 0.017230978235602 38,
0.008012599311769009, 0.0019723570439964533, 0.006560477428138256, -0.040996503084897995,
-0.010657534934580326, 0.0003703233960550 2784, -0.027875546365976334, -0.011727177537977695,
-0.00768198212608695, -0.007299503777176142, -0.011202080175280571, -0.01939619518816471,
0.039622172713279724, -0.011668833903968334, -0.015830717980861664, 0.016919808462262154,
0.03207632899284363, 0.015960371121764183, -0.010936290025711 06, -0.016842016950249672,
-0.008336733095347881, -0.013244125992059708, -0.011999450623989105, -0.020122256129980087,
-0.007422674912959337, -0.025010200217366 22, -0.008505282923579216, -0.005526489112526178,
-0.0011830900330096483, 0.017736626788973 81, 0.010709396563470364, -0.007267090491950512,
0.015999266877770424, 0.026047429069876 67, -0.013315435498952866, 0.016219679266214 37,
-0.020822385326027 87, -0.016893878579139 71, -0.043900746852159 5, -0.033580310642719 27,
0.000994281843304634, 0.037262476980686 19, -0.022080026566982 27, 0.000011990435268671717,
0.006949438713490963, 0.020433424040675163, 0.005153734702616 93, -0.031298406422138214,
```

```
0.0031116888858377934, 0.015701064839959145, -0.02813485451042652, -0.007377295754849911,
0.007461570668965578, 0.03985555097460747, -0.010975186713039875, -0.025697365403175354,
0.0397258959710598, -0.026319703087210655, -0.0030403793789446354, -0.010067610070109367,
-0.002486109733581543, -0.0088812792673707, 0.0017438423819839954, -0.001923736883327365,
0.017827384173870087, 0.006220136769115925, 0.010255607776343822, 0.001199296792037785,
-0.01772366091609001, 0.035136155784130096, -0.0061066895723342896, -0.010735327377915382,
-0.010651051998138428, -0.026151152327656746, 0.006981851998716593, 0.006622062996029854,
-0.010048162192106247, -0.0009124379721470177, -0.00419105589389801, -0.019668467342853546,
0.00012296844215597957, 0.004894427489489317, 0.006852198392152786, 0.010437123477458954,
0.005908967927098274, 0.0038247844204306602, -0.008103356696665287, -0.006456754636019468,
-0.028653468936681747, 0.018216345459222794, 0.032205980271101, 0.00022101905778981745,
-0.029664767906069756, -0.008155218325555325, 0.03871459513902664, 0.03394334018230438,
0.005860347766429186, -0.013600673526525497, -0.016958704218268394, 0.006372479721903801,
0.0012543996563181281, -0.01911095716059208, -0.010437123477458954, 0.008356180973351002,
-0.012855164706707, -0.008472870104014874, 0.019370263442397118, -0.029457321390509605,
0.0034487885423004627, -0.015415825881063938, -0.00047364120837301016, 0.008887761272490025,
-0.0020015290938317776, 0.010501950047910213, -0.007500466890633106, -0.0017470837337896228,
0.01717911660671234, -0.024063728749752045, 0.026734594255685806, -0.024556411430239677,
0.0013573121977970004, -0.00010007645323639736, -0.00450546620413661, 0.007513432297855616,
0.027201347053050995, 0.003426099196076393, -0.022183749824762344, 0.002813485451042652,
0.008064460940659046, 0.002243009163066745, 0.009899060241878033, 0.010988151654601097,
-0.004790704697370529, -0.004638361278921366, 0.006025656126439571, -0.010605673305690289,
-0.01625857502222061, -0.020342666655778885, -0.016090024262666702, -0.026410460472106934,
0.0121226217597723, -0.009406376630067825, 0.0023759042378515005, -0.0273828636854887,
-0.015260240994393826, -0.004832841921597719, -0.0007702240254729986, 0.01856641098856926,
-0.031039098277688026, 0.0073967440985143185, 0.018721995875239372, -0.023026498034596443,
-0.008200597018003464, -0.023480286821722984, -0.16450461745262146, 0.025710329413414,
0.01681608520448208, -0.009023898281157017, 0.023428425192832947, -0.022754225879907608,
0.027616240084171295, 0.015234310179948807, -0.009224860928952694, 0.005166700109839439,
0.0008131718495860696, 0.0038507150020450354, -0.03153178095817566, -0.0026757284067571163,
0.003335341578349471, 0.00672254478558898, -0.030546413734555244, 0.036277107894420624,
0.017256908118724823, 0.0010526260593906045, 0.0053125605918467045, -0.02091314271092415,
-0.0016555515749007463 5, -0.0012454859679564834, 0.023467320948839188, 0.009497134014964104,
0.0046351198107004166, 0.005380628630518913, -0.021691065281629562, -0.013062611222267151,
-0.048023734241724014, -0.0008427490829490125, 0.017321735620498657, 0.021340999752283096,
0.011740143410861492, 0.012219862081110477, -0.012984818778932095, 0.007020748220384121,
-0.015130587853491306, -0.016193747520446777, 0.007143919356167316 4, 0.03236156702041626,
0.024997234344482422, 0.01185683161020279, 0.010735327377915382, 0.04636416584253311,
0.014599007554352283, -0.009004450403153896, 0.019383229315280914, -0.009607339277863503,
-0.00414891866967082, -0.008336733095347881, -0.019888877868652344, -0.0005830365116707981,
0.02771996334195137, 0.005620488431304693, -0.00701426574960351, 0.013730327598750591,
0.014145219698548317, 0.011331734247505665, -0.021807754412293434, 0.022857949137687683,
0.01593444123864174, -0.0031343784648925066, 0.001282761339098215, -0.028627539053559303,
0.013354332186281681, 0.0034098925534635782, -0.014689764939248562, -0.004784221760928631,
-0.015208380296826363, -0.00796722061932087, -0.008693280629813671, -0.02311725541949272,
0.011629937216639519, -0.012323584407567978, -0.03243935853242874, 0.007643085904419422,
0.00766253424808383, 0.0028702090494334698, -0.017412493005394936, 0.026267841458320618,
0.010884428396821022, -0.03448788449168205, 0.004327192436903715, 0.018838683143258095,
-0.02228747308254242, -0.014702730812132359, -0.01020374707877636, -0.027694031596183777,
0.006122896447777748, -0.004252641461789608, -0.012686614878475666, -0.008829417638480663,
0.03319134935736656, 0.01789221167564392, 0.021250242367386818, -0.006683648563921452,
0.009412859566509724, -0.02294870652258396, 0.0009659201023168862, -0.008336733095347881,
-0.019603639841079712, -0.012116138823330402, 0.009775889106094837, 0.03993334248661995,
0.009892578236758709, 0.017153184860944748, 0.015545479021966457, -0.01288109552115202,
-0.020433424040675163, 0.013652535155415535, 0.022170783951878548, 0.024102624505758286,
-0.003623821074143052, 0.03230970352888107, 0.01852751523256302, -0.03132433444261551,
-0.017218012362718582, 0.011279872618615627, 0.052250444889068604, 0.005604281555861235,
0.010722361505031586, 0.006155309733003378, -0.016362298280000687, 0.0038020950742065907,
-0.1179330125451088, 0.0006563718779943883, 0.006923507899045944, 0.010300987400114536,
0.010313952341675758, 0.016128921881318092, 0.010683465749025345, 0.032050397247076035,
-0.0040808506309986115, 0.011318768374621868, -0.021016865968704224, -0.05787741392850876,
-0.018786821514368057, -0.03155771270394325, 0.010994634591042995, -0.00672254478558898,
0.010605673305690289, -0.029042430222034454, -0.018294138833880424, 0.01147435326129198,
-0.0367957204580307, 0.016906842589378357, 0.010385261848568916, -0.006301170215010643,
-0.014547145925462246, -0.004356364719569683, -0.03181701898574829, -0.0031505851075053215,
0.004936564713716507, 0.012952405028045177, 0.010398227721452713, -0.018164483830332756,
0.01794407330453396, -0.020977970212697983, -0.0003221084189135581, 0.007617155089974403,
0.0036173383705317974, -0.01026209071278572, 0.037651438266038895, -0.015662167221307755,
0.011156701482832432, -0.014599007554352283, -0.00417160801589489, -0.03412485495209694,
0.010022231377661228, 0.004084091633558273, -0.005659384187310934, -0.0012973473640158772,
0.02266346849501133, -0.015195414423942566, -0.021211346611380577, -0.01056029461324215,
-0.008284871466457844, -0.001761669758707285, 0.005711245816200972, 0.009205413050949574,
0.008997967466711998, -0.021470654755830765, -0.02379145473241806, 0.013872946612536907,
-0.004845807328820229, -0.016193747520446777, -0.003912300802767277, 0.00304200011305511,
-0.002687073079869151, -0.007617155089974403, 0.003983610309660435, -0.028575677424669266,
0.012939440086483955, -0.004434156697243452, -0.020381562411785126, 0.0030338966753333807,
-0.022131888195872307, 0.0012187449028715491, -0.020251909270882607, 0.003640027716755867,
-0.018449721857905388, 0.008557144552469254, 0.029042430222034454, -0.00807094294577837,
-0.025502884760499, -0.025852948427200317, 0.03057234361767769, -0.032205980271101, 0.011779039166867733,
0.007481019012629986, -0.010112988762557507, 0.017114289104938507, 0.011623455211520195,
```

```
-0.01751621626317501, -0.009341550059616566, -0.005238009616732597, -0.013561777770519257,
-0.005001391749829054, -0.014702730812132359, 0.008609006181359291, -0.009568443521857262,
0.00857011042535305, 0.0027049004565924406, 0.009101689793169498, -0.018294138833880424,
-0.0010202126577496529, -0.07286538183689117, 0.01214855257421732, -0.011403043754398823,
0.0032656528055667877, 0.0046221548691391945, -0.017568077892065048, 0.019136887043714523,
-0.022546779364347458, -0.0037826469633728266, -0.004913875367492437, -0.01572699472308159,
0.034306369721889496, 0.013509916141629219, -0.014158184640109539, -0.011014082469046116,
-0.010787188075482845, 0.005027322564274073, 0.002033942611888051, 0.017218012362718582,
0.02478978969156742, -0.010430640541017056, 0.0027437966782599688, 0.00802556425333023,
-0.01406742725521326, 0.0038442325312644243, 0.0035038914065808058, -0.003345065750181675,
0.02386924810707569, -0.00384747376665473, -0.012550478801131248, 0.0008500420954078436,
-0.012025381438434124, 0.015662167221307755, 0.019914809614419937, 0.029846282675862312,
-0.026656802743673325, -0.006203929893672466, -0.011033530347049236, 0.04947585612535477,
0.03575849160552025, -0.007960737682878971, -0.024984268471598625, 0.013548812828958035,
-0.02484164945781231, -0.01406742725521326, 0.011681798845529556, -0.022806087508797646,
0.0018378413515165448, 0.030961304903030396, 0.007928323931992054, 0.03251715004444122,
0.010884428396821022, 0.002808623481541872, -0.008466387167572975, 0.02379145473241806,
-0.019162818789482117, 0.05357291176915169, -0.0044049848802387714, -0.00405816081911325 45,
-0.013471020385622978, 0.020718662068247795, 0.00829783733934164, -0.01685498282313347,
-0.004100298509001732, -0.00002413208903232217, -0.014482319355010986, -0.03404706344008446,
0.007876462303102016, 0.0011855211341753602, -0.0405556783080101, -0.01225227490067482,
-0.006268756929785013, 0.015312102623283863, 0.015299137681722641, 0.013548812828958035,
0.014560110867023468, 0.01471569575369358, -0.002121458761394024, -0.01100111659616232,
0.00015933225222397596, 0.01965550146996975, -0.003769681556150317, -0.02826450765132904,
0.01413225382566452, 0.029198015108704567, 0.048568278551101685, -0.005461662542074919,
0.014274872839450836, 0.0036529931239783764, 0.005160217639058828, 0.0010145402047783136,
0.017243942245841026, -0.006793854292482138, 0.0005344163510017097, 0.01341915875673294,
0.019538814201951027, -0.003721061395481229, -0.010560294613242 15, 0.005033805035054684,
0.03562884032726288, -0.004832841921597719, 0.012997783720493317, -0.006443789228796959,
-0.015817752107977867, -0.015947405248880386, 0.023208012804389, -0.019590675830841064,
-0.0267605260014534, -0.008045012131333351, 0.0018005658639594913, 0.022935740649700165,
-0.006323859561234713, -0.0033677550964057446, 0.02027783915400505, -0.020433424040675163,
0.01689387857913971, -0.027019832283258438, -0.040166717022657394, -0.0001858707400970161,
0.009393410757184029, 0.0010169713059440255, 0.00733839999884367, 0.00923782680183649,
-0.011007599532604218, 0.022339332848787308, 0.01406742725521326, 0.01659567467868328,
-0.021794788539409637, -0.019085025414824486, -0.008245975710451603, 0.016323402523994446,
-0.0023759042378515005, -0.0077208783477544785, -0.0019626331049948 93, 0.0036108556669205427,
-0.0008832658641040325, -0.0267605260014534, 0.02873126231133938, -0.022935740649700165,
0.02412855438888073, -0.005513523705303669, 0.00507918419316411, 0.005951105151325464,
0.005173183046281338, 0.005001391749829054, -0.0177625585347414, 0.015130587853491306,
-0.01999260112643242, -0.0192406103014946, 0.02140582725405693, -0.007377295754849911,
0.02927580662071705, -0.015104657039046288, -0.01216800045222044, 0.0034552712459117174,
-0.003481202060356736, 0.013561777770519257, 0.002259215805679559 7, -0.01183090079575777,
0.013665501028299332, 0.017075393348932266, 0.008511765860021114, -0.007409709505736828,
-0.0014261907199397683, -0.003058206755667925, 0.0031424816697835922, 0.0021846650633960962,
-0.000989419873803854, -0.022144854068756104, 0.0036270625423640013, -0.006233102176338434,
-0.016504917293787003, 0.003999816719442606, 0.017931107431650162, -0.025956671684980392,
0.021626237779855728, 0.002192768268287182, -0.001260882243514061, 0.018721995875239372,
-0.023493250831961632, 0.030598275363445282, -0.011616972275078297, -0.019331367686390877,
0.01505279541015625, 0.01505279541015625, 0.004524914547801018, 0.0019042887724936008,
-0.014962038025259972]
```

Words that are semantically similar are often represented by vectors that are close to each other in this vector space. This allows for mathematical operations like addition and subtraction to carry semantic meaning. For example, the vector representation of "king" minus "man" plus "woman" should be close to the vector representation of "queen." In other words, vector embeddings are a numerical representation of a particular data object.

A vector index allows you to retrieve a neighborhood of nodes or relationships based on the similarity between the embedding properties of those nodes or relationships and the ones specified in the query.

Create vector indexes

A vector index is created by using the `CREATE VECTOR INDEX [index_name]` command. It is recommended to give the index a name when it is created. If no name is given when created, a random name will be assigned. The index name can also be given as a parameter: `CREATE VECTOR INDEX $name …`.

Vector index capabilities per Neo4j version

Neo4j version	Capabilities
5.13 (general availability)	Supports single-label, single-property index for nodes or single-relationship-type, single-property index for relationships. Vectors stored as `LIST <INTEGER | FLOAT>` properties.
2025.10	Adds support for `VECTOR` value properties validated against index dimensions and the configured vector similarity function.
2026.01	Adds support for multi-label and multi-relationship-type vector indexes with additional properties used for filtering. These properties can later be used in vector search with filters using `SEARCH … WHERE`, but they cannot be vector properties. A vector index can store only one vector property per node or relationship.
2026.07	Adds support for High Fidelity Quantized (HFQ) vector search, which can be enabled in the index configuration.

The index name must be unique among both indexes and constraints.
A newly created index is not immediately available but is created in the background.

Creating indexes requires the `CREATE INDEX` privilege.

Create vector index for `Movie` nodes on the `embedding` property

```
CREATE VECTOR INDEX moviePlots IF NOT EXISTS
FOR (m:Movie)
ON m.embedding
OPTIONS { indexConfig: {
 `vector.dimensions`: 1536,
 `vector.similarity_function`: 'cosine'
}}
```

The `CREATE VECTOR INDEX` command is optionally idempotent. This means that its default behavior is to throw an error if an attempt is made to create the same index twice. With `IF NOT EXISTS`, no error is thrown and nothing happens should an index with the same name, schema or both already exist. It may still throw an error should a constraint with the same name exist. An informational notification is returned when nothing happens, showing the existing index which blocks the creation.

To read more about the available configuration settings, see Configuration settings. In the above example, the vector dimension is explicitly set to `1536`. Providing dimensions when creating a vector index is recommended, as it ensures that only vectors of that size are indexed and makes dimension mismatches fail clearly at query time. The vector similarity function is set to `'cosine'`, which is generally the preferred similarity function for text embeddings. To read more about the available similarity functions, see Cosine and Euclidean similarity functions.

You can also create a vector index for relationships with a particular type on a given property using the following syntax:

Create a vector index for a relationship type on a single property

```
CREATE VECTOR INDEX name IF NOT EXISTS
FOR ()-[r:REL_TYPE]-() ON (r.embedding)
OPTIONS { indexConfig: {
 `vector.dimensions`: $dimension,
 `vector.similarity_function`: $similarityFunction
```

```
}}
```

Like full-text indexes, vector indexes may be created for multiple labels or relationship types. In that case, all nodes with *at least one* label matching *at least one* of the labels or a relationship whose type matches *at least one* of the relationship types are included as long as they have the vector property.

In contrast to multi-property full-text indexes, the vector property is the only property to determine if the node or relationship is included in the index. The additional properties will only be included in the index if they are of one of the types `INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`, `DATE`, `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, `LOCAL TIME` or `DURATION`. If the node or relationship does not have an additional property or if the additional property is of another type, it is treated as `null` by the index.

You can create a vector index on multiple labels with additional properties for filtering using the following query:

Create a vector index for multiple labels and with additional properties

```
CREATE VECTOR INDEX multiLabelAdditionalProperties IF NOT EXISTS
FOR (n:Movie|Actor)
ON n.embedding
WITH [n.title, n.plot, n.name, n.born]
OPTIONS { indexConfig: {
 `vector.dimensions`: 1536,
 `vector.similarity_function`: 'cosine'
}}
```

And similarly for relationships on multiple relationship types with additional properties using the following syntax:

Create a vector index for multiple relationship types and an additional property

```
CREATE VECTOR INDEX name IF NOT EXISTS
FOR ()-[r:REL_TYPE1|REL_TYPE2|REL_TYPE3]-()
ON (r.embedding)
WITH [r.property]
OPTIONS { indexConfig: {
 `vector.dimensions`: $dimension,
 `vector.similarity_function`: $similarityFunction
}}
```

Configuration settings

For more information about the values accepted by different index providers, see Vector index providers for compatibility.

`vector.dimensions`

The dimensions of the vectors to be indexed. For more information, see Vectors and embeddings in Neo4j. This setting can be omitted, and any `LIST<INTEGER | FLOAT>` or, as of Neo4j 2025.10, additionally `VECTOR` value can be indexed and queried, separated by their dimensions, *though only vectors of the same dimension can be compared*. Setting this value adds additional checks that ensure only vectors with the configured dimensions are indexed, and querying the index with a vector of a different dimension returns an error.

It is recommended to provide dimensions when creating a vector index. Doing so avoids the

confusing situation where the same index may contain side-by-side vectors of different dimensions, while still only allowing comparisons between vectors of the same dimension.

Accepted values

`INTEGER` between `1` and `4096` inclusively.

Default value

None.

`vector.similarity_function`

The name of the similarity function used to assess the similarity of two vectors. To read more about the available similarity functions, see Cosine and Euclidean similarity functions.

Accepted values

`STRING`: `'cosine'`, `'euclidean'`.

Default value

`'cosine'`.

`vector.default_search_expansion_factor`

Introduced in Neo4j 2026.07

Scales the number of nearest neighbors to query the vector index, while returning the originally requested number of nearest neighbors. Setting this value above `1.0` can increase accuracy at the cost of query time.

Accepted values

`FLOAT`: between `1.0` and `10_000.0` *inclusive*.

Default values

Dependent upon the value of `vector.quantization.type`.

Value of `vector.quantization.type`	Default value `vector.default_search_expansion_factor`
`'none'`	`1.0`
`'scalar'`	`1.5`
`'binary'`	`3.0`

`vector.quantization.enabled`

Deprecated in Neo4j 2026.06

Quantization is a technique to reduce the size of vector representations. Enabling quantization can accelerate search performance but can slightly decrease accuracy. It is recommended to enable quantization on machines with limited memory.

Accepted values

`BOOLEAN`: `true`, `false`.

Default value

`true`

`vector.quantization.type`

Introduced in Neo4j 2026.06

Quantization is a technique to reduce the size of vector representations. Setting a quantization type can accelerate search performance but decrease accuracy. Binary quantization is a more aggressive form of Scalar quantization, reducing each dimension to a single bit. If `vector.default_search_expansion_factor` is set larger than `1.0` with binary quantization, the returned nearest neighbors are rescored with their unquantized vector values to increase accuracy (High Fidelity Quantized vector search).

It is recommended to use quantization on machines with limited memory.

Accepted values

`STRING`: `'none'`, `'scalar'`, `'binary'`.

Default value

`'binary'`

Prior to the release of Neo4j 2026.08, the default value for this setting was `'scalar'`.

Advanced configuration settings

`vector.hnsw.m`

The `M` parameter controls the maximum number of connections each node has in the HNSW (Hierarchical Navigable Small Worlds) graph. Increasing this value may lead to greater accuracy at the expense of increased index population and update times, especially for vectors with high dimensionality.

Accepted values

`INTEGER` between `1` and `512` inclusively.

Default value

`16`

`vector.hnsw.ef_construction`

The number of nearest neighbors tracked during the insertion of vectors into the HNSW graph. Increasing this value increases the quality of the index, and may lead to greater accuracy (with diminishing returns) at the expense of increased index population and update times.

Accepted values

`INTEGER` between `1` and `3200` inclusively.

Default value

`100`

Query vector indexes

As of Neo4j 2026.01, the preferred way of querying vector indexes is by using the Cypher SEARCH clause. A less powerful alternative, which also works for earlier Neo4j versions, is using the procedures db.index.vector.queryNodes and db.index.vector.queryRelationships. These procedures are deprecated as of Neo4j 2026.04.

Vector index scores are meaningful within the result set from a vector query. When combining vector results with full-text results or other scored sources, rank each source independently rather than comparing raw score values directly. For more information, see Developer Guide → Hybrid search.

> An index cannot be used while its `state` is `POPULATING`, which occurs immediately after it is created. To check the `state` of a vector index — whether it is `ONLINE` (usable) or `POPULATING` (still being built; the `populationPercent` column shows the progress of the index creation) — run the following command: `SHOW VECTOR INDEXES`.

Querying a node vector index using the SEARCH clause

The `SEARCH` clause returns the neighborhood of nodes, optionally with their respective similarity scores, ordered by those scores. The scores are bounded between `0.0` and `1.0`, where the closer to `1.0` the score is, the more similar the indexed vector is to the query vector.

Find the five movies with the most similar plot to The Godfather

```
MATCH (m:Movie {title: 'Godfather, The'})
MATCH (movie: Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR m.embedding
    LIMIT 5
  ) SCORE AS score
RETURN movie.title AS title, movie.plot AS plot, score
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------+
| title                    | plot
| score             |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------+
| "Godfather, The"           | "The aging patriarch of an organized crime dynasty transfers control of his
clandestine empire to his reluctant son."
| 0.9999999403953552 |
| "Godfather: Part III, The" | "In the midst of trying to legitimize his business dealings in New York and
Italy in 1979, aging Mafia don Michael Corleone seeks to avow for his sins while taking a young protégé
under his wing."                     | 0.9648237824440002 |
| "Godfather: Part II, The"  | "The early life and career of Vito Corleone in 1920s New York is portrayed
while his son, Michael, expands and tightens his grip on his crime syndicate stretching from Lake Tahoe,
Nevada to pre-revolution 1958 Cuba." | 0.954778790473938  |
| "Scarface"                 | "An ambitious and near insanely violent gangster climbs the ladder of
success in the mob, but his weaknesses prove to be his downfall."
| 0.9367182850837708 |
| "Jane Austen's Mafia!"     | "Takeoff on the Godfather with the son of a mafia king taking over for his
```

```
dying father"
| 0.9366796016693115 |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------+
```

All movies returned have a plot centered around criminal family organizations. The `score` results are returned in *descending order*, where the best matching result entry is put first. `The Godfather` has a similarity score of `1.0` because the index was queried with this specific property. To exclude the query vector, add the predicate `WHERE score < 1`.

Querying a node vector index using the db.index.vector.queryNodes() procedure

Signature for `db.index.vector.queryNodes()` *Deprecated in Neo4j 2026.04*

```
db.index.vector.queryNodes(indexName :: STRING, numberOfNearestNeighbours :: INTEGER, query :: ANY) ::
(node :: NODE, score :: FLOAT)
```

- The `indexName` refers to the unique name of the vector index to query.
- The `numberOfNearestNeighbours` refers to the number of nearest neighbors to return.
- The `query` refers to the `LIST<INTEGER | FLOAT>` or, as of Neo4j 2025.10, additionally the `VECTOR` of which the neighborhood should be searched.

The procedure returns the neighborhood of nodes with their respective similarity scores, ordered by those scores. The scores are bounded between `0` and `1`, where the closer to `1` the score is, the more similar the indexed vector is to the query vector.

Find the 5 movies with the most similar plot to The Godfather

```
MATCH (m:Movie {title: 'Godfather, The'})
CALL db.index.vector.queryNodes('moviePlots', 5, m.embedding)
YIELD node AS movie, score
RETURN movie.title AS title, movie.plot AS plot, score
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------+
| title                       | plot
| score              |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------+
| "Godfather, The"            | "The aging patriarch of an organized crime dynasty transfers control of his
clandestine empire to his reluctant son."
| 0.9999999403953552 |
| "Godfather: Part III, The" | "In the midst of trying to legitimize his business dealings in New York and
Italy in 1979, aging Mafia don Michael Corleone seeks to avow for his sins while taking a young protégé
under his wing."                      | 0.9648237824440002 |
| "Godfather: Part II, The"  | "The early life and career of Vito Corleone in 1920s New York is portrayed
while his son, Michael, expands and tightens his grip on his crime syndicate stretching from Lake Tahoe,
Nevada to pre-revolution 1958 Cuba." | 0.954778790473938  |
| "Scarface"                  | "An ambitious and near insanely violent gangster climbs the ladder of
success in the mob, but his weaknesses prove to be his downfall."
| 0.9367182850837708 |
| "Jane Austen's Mafia!"      | "Takeoff on the Godfather with the son of a mafia king taking over for his
dying father"
| 0.9366796016693115 |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------+
```

Note that all movies returned have a plot centred around criminal family organizations. The `score` results are returned in *descending order*, where the best matching result entry is put first (in this case, `The Godfather` has a similarity score of `1.0`, which is to be expected as the index was queried with this specific property). If the query vector itself is not wanted, adding the predicate `WHERE score < 1` removes identical vectors.

Querying a relationship vector index

To query a relationship vector index, use the Cypher SEARCH clause or the `db.index.vector.queryRelationships` procedure in a similar manner as shown for node vector indexes above.

Signature for `db.index.vector.queryRelationships()` *Deprecated in Neo4j 2026.04*

```
db.index.vector.queryRelationships(indexName :: STRING, numberOfNearestNeighbours :: INTEGER, query ::
ANY) :: (relationship :: RELATIONSHIP, score :: FLOAT)
```

`db.index.vector.queryRelationships()` has the same argument descriptions as `db.index.vector.queryNodes()`.

Use Vector functions to compute the similarity score between two specific vector pairs without using a vector index.

Performance suggestions

Vector indexes can take advantage of the incubated Java Vector API for noticeable speed improvements. If you are using a compatible version of Java, you can add the following setting to your configuration settings:

Configuration settings

```
server.jvm.additional=--add-modules=jdk.incubator.vector
```

Starting with Neo4j 2026.08, the `jdk.incubator.vector` module is enabled by default. For details, see the Operations manual → Changes in Neo4j 2025-2026 series.

While the Neo4j 5 series requires at least Java 17, at *least* Java 20 is necessary to take advantage of this performance improvement in Neo4j 5.14-5.26. If you are using Neo4j 2025.01 or later, Java 21 is required. For more information about what Java versions are supported by different Neo4j versions, see the Operations Manual → System requirements → Java.

Show vector indexes

To show all vector indexes in a database, use the `SHOW VECTOR INDEXES` command. This is the same `SHOW` command as for other indexes, with the index type filtering on `VECTOR`.

Showing indexes requires the `SHOW INDEX` privilege.

Example 440. Show all vector indexes

Show vector indexes with the default return columns

```
SHOW VECTOR INDEXES
```

Result

```
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------+
| id | name                             | state    | populationPercent | type     | entityType |
labelsOrTypes      | properties                                    | indexProvider    |
owningConstraint | lastRead                 | readCount |
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------+
| 1  | "moviePlots"                     | "ONLINE" | 100.0             | "VECTOR" | "NODE"     |
["Movie"]          | ["embedding"]                                 | "vector-2026.08" | NULL
| 2026-08-24T12:10:35.169Z | 2         |
| 2  | "multiLabelAdditionalProperties" | "ONLINE" | 100.0             | "VECTOR" | "NODE"     |
["Movie", "Actor"] | ["embedding", "title", "plot", "name", "born"] | "vector-2026.08" | NULL
| NULL                     | 0         |
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------+
```

For a full description of all return columns, see Performance indexes → Show indexes → Return columns.

Example 441. Show vector indexes with full or filtered details

To return full vector index details, use `YIELD *`.

Show all vector indexes with all return columns

```
SHOW VECTOR INDEXES YIELD *
```

Result

```
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------+
| id | name                             | state    | populationPercent | type     | entityType |
labelsOrTypes      | properties                                    | indexProvider    |
owningConstraint | lastRead                 | readCount | trackedSince             | options
| failureMessage | createStatement
|
+----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------
```

```
+--------------------------------------------------------------------------------+
| 1  | "moviePlots"                    | "ONLINE" | 100.0             | "VECTOR" | "NODE"     |
["Movie"]          | ["embedding"]                                  | "vector-2026.08" | NULL
| 2026-08-24T12:10:35.169Z | 2         | 2026-08-24T12:08:41.009Z | {indexConfig:
{`vector.dimensions`: 1536, `vector.default_search_expansion_factor`: 3.0, `vector.hnsw.m`: 16,
`vector.quantization.type`: "BINARY", `vector.similarity_function`: "COSINE",
`vector.hnsw.ef_construction`: 100}} | ""             | "CREATE VECTOR INDEX `moviePlots` FOR
(n:`Movie`) ON (n.`embedding`) OPTIONS {indexConfig: {`vector.default_search_expansion_factor`:
3.0,`vector.dimensions`: 1536,`vector.hnsw.ef_construction`: 100,`vector.hnsw.m`:
16,`vector.quantization.type`: 'BINARY',`vector.similarity_function`: 'COSINE'}}"
|
| 2  | "multiLabelAdditionalProperties" | "ONLINE" | 100.0             | "VECTOR" | "NODE"     |
["Movie", "Actor"] | ["embedding", "title", "plot", "name", "born"] | "vector-2026.08" | NULL
| NULL                      | 0         | 2026-08-24T12:08:53.602Z | {indexConfig:
{`vector.dimensions`: 1536, `vector.default_search_expansion_factor`: 3.0, `vector.hnsw.m`: 16,
`vector.quantization.type`: "BINARY", `vector.similarity_function`: "COSINE",
`vector.hnsw.ef_construction`: 100}} | ""             | "CREATE VECTOR INDEX
`multiLabelAdditionalProperties` FOR (n:`Movie`|`Actor`) ON (n.`embedding`) WITH [n.`title`,
n.`plot`, n.`name`, n.`born`] OPTIONS {indexConfig: {`vector.default_search_expansion_factor`:
3.0,`vector.dimensions`: 1536,`vector.hnsw.ef_construction`: 100,`vector.hnsw.m`:
16,`vector.quantization.type`: 'BINARY',`vector.similarity_function`: 'COSINE'}}" |
+--------------------------------------------------------------------------------+
```

To return only specific details, specify the desired column name(s) after the `YIELD` clause.

Show all vector indexes with filtered return columns

```
SHOW VECTOR INDEXES YIELD name, type, entityType, labelsOrTypes, properties
```

Result

```
+--------------------------------------------------------------------------------+
| name                            | type     | entityType | labelsOrTypes      | properties
|
+--------------------------------------------------------------------------------+
| "moviePlots"                    | "VECTOR" | "NODE"     | ["Movie"]          | ["embedding"]
|
| "multiLabelAdditionalProperties" | "VECTOR" | "NODE"     | ["Movie", "Actor"] | ["embedding",
"title", "plot", "name", "born"] |
+--------------------------------------------------------------------------------+
```

Drop vector indexes

A vector index is dropped by using the same command as for other indexes, `DROP INDEX`. It is possible to drop vector indexes created in a different Cypher version than the one in use. For example, although the multiple labels or additional properties were added in Cypher 25, such indexes can still be dropped using Cypher 5.

The index name can also be given as a parameter when dropping an index: `DROP INDEX $name`.

> Dropping indexes requires the `DROP INDEX` privilege.

Drop a vector index

```
DROP INDEX moviePlots
```

Vector index providers for compatibility

Neo4j automatically selects the latest provider when creating a vector index: it is not possible to specify an index provider explicitly in Cypher 25. Indexes created with older providers will continue to function.

Identify the provider used by vector indexes in a database

```
SHOW VECTOR INDEXES YIELD name, type, indexProvider
```

▼ *Learn more about the* `vector-2026.08` *provider* **(latest)** Introduced in Neo4j 2026.08

Feature	Capability
Index schema	multi-label, multi-property index for nodes. multi-type, multi-property index for relationships. Only one vector property can be stored per node or relationship. Any additional properties included in the index must be one of the following types: `INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`, `DATE`, `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, `LOCAL TIME` or `DURATION`.
Indexed property value type	`VECTOR` or `LIST<INTEGER | FLOAT>`.
Indexed vector dimension	`INTEGER` between `1` and `4096` *inclusive*.
Indexed vector quantization	High Fidelity Quantized vector search for binary quantization. `'binary'` is the default quantization type.
Indexed hnsw settings	Supported

Feature	Capability
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 **double** precision. Its ℓ^2-norm is non-zero and can be represented finitely in IEEE 754 **double** precision. The ratio of each vector component with its ℓ^2-norm can be represented finitely in IEEE 754 ***single*** precision.

▼ *Learn more about the* `vector-2026.07` *provider* Introduced in Neo4j 2026.07

Feature	Capability
Index schema	multi-label, multi-property index for nodes. multi-type, multi-property index for relationships. Only one vector property can be stored per node or relationship. Any additional properties included in the index must be one of the following types: `INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`, `DATE`, `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, `LOCAL TIME` or `DURATION`.
Indexed property value type	`VECTOR` or `LIST<INTEGER | FLOAT>`.
Indexed vector dimension	`INTEGER` between `1` and `4096` *inclusive*.
Indexed vector quantization	High Fidelity Quantized vector search for scalar and binary quantization.
Indexed hnsw settings	Supported

Feature	Capability
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 **double** precision. Its ℓ^2 -norm is non-zero and can be represented finitely in IEEE 754 **double** precision. The ratio of each vector component with its ℓ^2 -norm can be represented finitely in IEEE 754 ***single*** precision.

▼ *Learn more about the* `vector-2026.06` *provider* *Introduced in Neo4j 2026.06*

Feature	Capability
Index schema	multi-label, multi-property index for nodes. multi-type, multi-property index for relationships. Only one vector property can be stored per node or relationship. Any additional properties included in the index must be one of the following types: `INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`, `DATE`, `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, `LOCAL TIME` or `DURATION`.
Indexed property value type	`VECTOR` or `LIST<INTEGER | FLOAT>`.
Indexed vector dimension	`INTEGER` between `1` and `4096` *inclusive*.
Indexed vector quantization	Scalar and binary quantization supported.
Indexed hnsw settings	Supported

Feature	Capability
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 **double** precision. Its ℓ^2 -norm is non-zero and can be represented finitely in IEEE 754 **double** precision. The ratio of each vector component with its ℓ^2 -norm can be represented finitely in IEEE 754 ***single*** precision.

▼ *Learn more about the* `vector-3.0` *provider* *Introduced in Neo4j 2025.09*

Feature	Capability
Index schema	multi-label, multi-property index for nodes. multi-type, multi-property index for relationships. Only one vector property can be stored per node or relationship. Any additional properties included in the index must be one of the following types: `INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`, `DATE`, `ZONED DATETIME`, `LOCAL DATETIME`, `ZONED TIME`, `LOCAL TIME` or `DURATION`.
Indexed property value type	`VECTOR` or `LIST<INTEGER | FLOAT>`
Indexed vector dimension	`INTEGER` between `1` and `4096` *inclusive*.
Indexed vector quantization	Scalar quantization supported.
Indexed hnsw settings	Supported

Feature	Capability
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 **double** precision. Its ℓ^2-norm is non-zero and can be represented finitely in IEEE 754 **double** precision. The ratio of each vector component with its ℓ^2-norm can be represented finitely in IEEE 754 ***single*** precision.

▼ *Learn more about the* `vector-2.0` *provider* Introduced in Neo4j 5.18

Feature	Capability
Index schema	Single-label, single-property index for nodes. Single-type, single-property index for relationships.
Indexed property value type	`VECTOR` or `LIST<INTEGER | FLOAT>`
Indexed vector dimension	`INTEGER` between `1` and `4096` *inclusive*.
Indexed vector quantization	Scalar quantization supported.
Indexed hnsw settings	Supported
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 **double** precision. Its ℓ^2-norm is non-zero and can be represented finitely in IEEE 754 **double** precision. The ratio of each vector component with its ℓ^2-norm can be represented finitely in IEEE 754 ***single*** precision.

▼ *Learn more about the* `vector-1.0` *provider* Introduced in Neo4j 5.11

Feature	Capability
Index schema	Single-label, single-property index for nodes. *No relationship support.*
Indexed property value type	`LIST<FLOAT>`
Indexed vector dimension	`INTEGER` between `1` and `2048` *inclusive*.
Indexed vector quantization	Not supported
Indexed hnsw settings	Not supported
Cosine similarity vector validity	All vector components can be represented finitely in IEEE 754 ***single*** precision. Its ℓ^2-norm is non-zero and can be represented finitely in IEEE 754 ***single*** precision.

Cosine and Euclidean similarity functions

The choice of similarity function affects which indexed vectors are considered similar, and which are valid. The semantic meaning of the vector may itself dictate which similarity function to choose. Refer to the documentation for the particular vector embedding model you are using, as it may suggest a preference for certain similarity functions. Otherwise, being able to differentiate between the various similarity functions can assist in making a more informed decision.

Similarity functions

Name	Case insensitive argument	Key similarity feature
Cosine	`"cosine"`	*angle*
Euclidean	`"euclidean"`	*distance*

For ℓ^2-normalized vectors (unit vectors), cosine and Euclidean similarity functions produce the same similarity ordering.

▼ *Learn more about the cosine similarity function*

Cosine similarity is used when the *angle* between the vectors is what determines how similar two vectors are.

A valid vector for a cosine vector index is when:

- All vector components can be represented finitely in IEEE 754 double precision.[4]
- Its ℓ^2-norm is non-zero and can be represented finitely in IEEE 754 double precision.

- The ratio of each vector component with its ℓ^2-norm can be represented finitely in IEEE 754 single precision.

Cosine similarity interprets the vectors in Cartesian coordinates. The measure is related to the angle between the two vectors. However, an angle can be described in many units, sign conventions, and periods. The trigonometric cosine of this angle is both agnostic to the aforementioned angle conventions and bounded. Cosine similarity rebounds the trigonometric cosine.

$$\begin{aligned}\operatorname{cosine}(\vec{v}, \vec{u}) &:= \tfrac{1}{2}\left(1 + \hat{v} \cdot \hat{u}\right) \\ &= \frac{1}{2}\left[1 + \frac{\vec{v} \cdot \vec{u}}{\|\vec{v}\| \, \|\vec{u}\|}\right] \qquad \in [0, 1]\end{aligned}$$

In the above equation the trigonometric cosine is given by the scalar product of the two unit vectors.

▼ *Learn more about the Euclidean similarity function*

Euclidean similarity is useful when the *distance* between the vectors is what determines how similar two vectors are.

A valid vector for a Euclidean vector index is when all vector components can be represented finitely in IEEE 754 single precision.

Euclidean interprets the vectors in Cartesian coordinates. The measure is related to the Euclidean distance, i.e., how far two points are from one another. However, that distance is unbounded and less useful as a similarity score. Euclidean similarity bounds the square of the Euclidean distance.

$$\operatorname{Euclidean}(\vec{v}, \vec{u}) := \frac{1}{1 + \|\vec{v} - \vec{u}\|^2} \qquad \in (0, 1]$$

Vector index procedures

Usage	Procedure	Description
Use node vector index.	`db.index.vector.queryNodes()`	Query the given node vector index. Returns the requested number of approximate nearest neighbor nodes and their similarity score, ordered by score. Replaced by Cypher's `SEARCH` clause. Deprecated in Neo4j 2026.04
Use relationship vector index.	`db.index.vector.queryRelationships()`	Query the given relationship vector index. Returns the requested number of approximate nearest neighbor relationships and their similarity score, ordered by score. Replaced by Cypher's `SEARCH` clause. Deprecated in Neo4j 2026.04

Usage	Procedure	Description
Set node vector property.	`db.create.setNodeVectorProperty()`	Update a given node property with the given vector in a more space-efficient way than directly using `SET`.
Set relationship vector property.	`db.create.setRelationshipVectorProperty()`	Update a given relationship property with the given vector in a more space-efficient way than directly using `SET`.

Limitations and known issues

The vector index is no longer a beta feature. It does, however, still contain some limitations:

- The query is an *approximate* nearest neighbor search. The requested *k* nearest neighbors may not be the exact *k* nearest, but close within the same wider neighborhood.
- For large requested nearest neighbors, *k*, close to the total number of indexed vectors, the search may retrieve fewer than *k* results.
- Only one vector index can be over a schema. For example, you cannot have one Euclidean and one cosine vector index on the same label-property key pair.
- Changes made within the same transaction are not visible to the index.

Vector indexes also contain some known issues. The following table lists the issues and, if fixed, the version in which they were fixed:

Known issues

Known issues	Fixed in
The creation of a vector index using the legacy procedure `db.index.vector.createNodeIndex()` may fail with an error in Neo4j 5.18 and later if the database was last written to with a version prior to Neo4j 5.11, and the legacy procedure is the first write operation used on the newer version. In Neo4j 5.20, the error was clarified. Using the `CREATE VECTOR INDEX` command instead avoids this issue. If the use of the procedure is unavoidable, performing any other write operation to the database on the newer binary before using the procedure will avoid the issue	
Procedure signatures from `SHOW PROCEDURES` render the vector arguments with a type of `ANY` rather than the semantically correct type of `VECTOR | LIST<INTEGER | FLOAT>`. The types are still enforced as `VECTOR | LIST<INTEGER | FLOAT>`.	

Syntax

This page contains the syntax for creating, listing, and dropping the indexes available in Neo4j. It also contains the signatures for the procedures necessary to call in order to use full-text and vector indexes.

More details about the syntax can be found in the Operations Manual → Cypher syntax for administration

commands.

CREATE INDEX

The general structure of the `CREATE INDEX` command is:

```
CREATE [index_type] INDEX [index_name] [IF NOT EXISTS]
FOR {node_pattern | relationship_pattern}
ON property_or_token_lookup_pattern
[OPTIONS "{" option: value[, ...] "}"]
```

The `CREATE … INDEX …` command is optionally idempotent. This means that its default behavior is to throw an error if an attempt is made to create an index with the same name twice. With `IF NOT EXISTS`, no error is thrown and nothing happens should an index with the same name or same schema and index type already exist (it may still throw an error if conflicting constraints exist, such as constraints with the same name or with the same schema and backing index). Instead, an informational notification is returned showing the existing index which blocks the creation.

The index name must be unique among both indexes and constraints. A random name will be assigned if no name is explicitly given when an index is created.

Index configuration settings can be specified using the `OPTIONS` clause. However, not all indexes have available configuration settings. In those cases, nothing needs to be specified and the `OPTIONS` map should be omitted from the query.

Creating an index requires the `CREATE INDEX` privilege.

Range indexes

Range indexes have no supported index configuration. The `OPTIONS` map has, therefore, been omitted from the syntax below.

Create a range index for a node label, either on a single property or composite

```
CREATE [RANGE] INDEX [index_name] [IF NOT EXISTS]
FOR (n:LabelName)
ON (n.propertyName_1[,
    n.propertyName_2,
    ...
    n.propertyName_n])
```

Create a range index for a relationship type, either on a single property or composite

```
CREATE [RANGE] INDEX [index_name] [IF NOT EXISTS]
FOR ()-"["r:TYPE_NAME"]"-()
ON (r.propertyName_1[,
    r.propertyName_2,
    ...
    r.propertyName_n])
```

For more information, see Create indexes → Create a range index.

Text indexes

Text indexes have no supported index configuration. The `OPTIONS` map has, therefore, been omitted from the syntax below.

Create a text index for a node label on a single property

```
CREATE TEXT INDEX [index_name] [IF NOT EXISTS]
FOR (n:LabelName)
ON (n.propertyName_1)
```

Create a text index for a relationship type on a single property

```
CREATE TEXT INDEX [index_name] [IF NOT EXISTS]
FOR ()-"["r:TYPE_NAME"]"-()
ON (r.propertyName_1)
```

It is not possible to create composite text indexes on multiple properties.

For more information, see Create indexes → Create a text index.

Point indexes

Create a point index for a node label on a single property

```
CREATE POINT INDEX [index_name] [IF NOT EXISTS]
FOR (n:LabelName)
ON (n.propertyName_1)
[OPTIONS "{" option: value[, ...] "}"]
```

Create a point index for a relationship type on a single property

```
CREATE POINT INDEX [index_name] [IF NOT EXISTS]
FOR ()-"["r:TYPE_NAME"]"-()
ON (r.propertyName_1)
[OPTIONS "{" option: value[, ...] "}"]
```

The following settings can be specified for point indexes:

- `spatial.cartesian.min`
- `spatial.cartesian.max`
- `spatial.cartesian-3d.min`
- `spatial.cartesian-3d.max`
- `spatial.wgs-84.min`
- `spatial.wgs-84.max`
- `spatial.wgs-84-3d.min`
- `spatial.wgs-84-3d.max`

It is not possible to create composite point indexes on multiple properties.

For more information, see Create indexes → Create a point index.

Token lookup indexes

Token lookup indexes have no supported index configuration. The `OPTIONS` map has, therefore, been omitted from the syntax below.

Create a node label lookup index

```
CREATE LOOKUP INDEX [index name] [IF NOT EXISTS]
FOR (n)
ON EACH labels(n)
```

Create a relationship type lookup index

```
CREATE LOOKUP INDEX [index name] [IF NOT EXISTS]
FOR ()-"["r"]"-()
ON [EACH] type(r)
```

Two token lookup indexes are present by default when creating a Neo4j database, and only one node label lookup index and one relationship type lookup index can exist at the same time.

For more information, see Create indexes → Create a token lookup index.

Full-text indexes

Create a full-text index for one or more node labels, either on a single property or multiple properties

```
CREATE FULLTEXT INDEX [index_name] [IF NOT EXISTS]
FOR (n:LabelName["|" ...])
ON EACH "[" n.propertyName[, ...] "]"
[OPTIONS "{" option: value[, ...] "}"]
```

Create a full-text index for one or more relationship types, either on a single property or multiple properties

```
CREATE FULLTEXT INDEX [index_name] [IF NOT EXISTS]
FOR ()-"["r:TYPE_NAME["|" ...]"]"-()
ON EACH "[" r.propertyName[, ...] "]"
[OPTIONS "{" option: value[, ...] "}"]
```

The following settings can be specified for full-text indexes:

- `fulltext.analyzer` - specifies what analyzer to use (the `db.index.fulltext.listAvailableAnalyzers()` procedure lists what analyzers are available).
- `fulltext.eventually_consistent` - specifies whether a full-text index is eventually consistent. If set to `true`, it will ensure that updates from committing transactions are applied in a background thread.

For more information, see Full-text indexes - Create full-text indexes.

Vector indexes

Create a vector index for a node label on a single property

```
CREATE VECTOR INDEX [index_name] [IF NOT EXISTS]
```

```
FOR (n:LabelName["|" ...])
ON (n.propertyName)
[WITH "[" n.additionalPropertyName[, ...] "]"]
[OPTIONS "{" option: value[, ...] "}"]
```

Create a vector index for a relationship type on a single property

```
CREATE VECTOR INDEX [index_name] [IF NOT EXISTS]
FOR ()-"["r:TYPE_NAME["|" ...]"]"-()
ON (r.propertyName)
[WITH "[" r.additionalPropertyName[, ...] "]"]
[OPTIONS "{" option: value[, ...] "}"]
```

```
OPTIONS {
  indexConfig: {
    `vector.dimensions`: $dimension,
    `vector.similarity_function`: $similarityFunction
  }
}
```

It is recommended to provide dimensions when creating a vector index. Doing so avoids the confusing situation where the same index may contain side-by-side vectors of different dimensions, while still only allowing comparisons between vectors of the same dimension.

For a full list of all vector index settings, see Vector index configuration settings.

As of Neo4j 2026.01, vector indexes support multiple labels, multiple relationship types, and additional properties for filtering. A vector index can index only one vector property, and the optional `WITH` clause adds non-vector properties for filtering; it does not create a composite vector index over multiple vector properties.

For more information, see Vector indexes - Create and configure vector indexes.

SHOW INDEX

Showing indexes requires the `SHOW INDEX` privilege.

Show indexes in the database (either all or filtered on index type)

```
SHOW [ALL | FULLTEXT | LOOKUP | POINT | RANGE | TEXT | VECTOR] INDEX[ES]
[YIELD { * | field[,...] } [ORDER BY field[,...]] [SKIP n] [LIMIT n]]
[WHERE expression]
[RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]
```

When using the `RETURN` clause, the `YIELD` clause is mandatory.

For more information, see Show indexes.

In Cypher 25 from 2026.05, it is possible to combine `SHOW INDEXES` with general Cypher clauses in a single query.

Query search indexes

Neo4j's search indexes are not used automatically in Cypher queries. To query them, explicitly use the

Cypher `SEARCH` clause. Built-in procedures remain available for compatibility with earlier Neo4j versions and Cypher 5. The `SEARCH` syntax and procedure signatures are shown below.

Full-text indexes

As of Neo4j 2026.09, the preferred way of querying full-text indexes is by using the SEARCH clause. As an alternative, use either the `db.index.fulltext.queryRelationships()` procedure. These procedures are deprecated as of Neo4j 2026.10.

Query full-text index on nodes using SEARCH

```
MATCH (n)
  [WHERE ...]
  SEARCH n IN (
    FULLTEXT INDEX index_name
    FOR query_string [ WITH ANALYZER analyzer_string ]
    [ (SKIP | OFFSET) skip_integer ]
    LIMIT limit
  ) [SCORE AS score_alias]
```

Query full-text index on relationships using SEARCH

```
MATCH ()-"["r"]"->()
  [WHERE ...]
  SEARCH r IN (
    FULLTEXT INDEX index_name
    FOR query_string [ WITH ANALYZER analyzer_string ]
    [ (SKIP | OFFSET) skip_integer ]
    LIMIT limit
  ) [SCORE AS score_alias]
```

Query full-text index on nodes: db.index.fulltext.queryNodes() Deprecated in Neo4j 2026.10

```
CALL db.index.fulltext.queryNodes(indexName :: STRING, queryString :: STRING, options = {} :: MAP)
```

Query full-text index on relationships: db.index.fulltext.queryRelationships() Deprecated in Neo4j 2026.10

```
CALL db.index.fulltext.queryRelationships(indexName :: STRING, queryString :: STRING, options = {} :: MAP)
```

The valid *key: value* pairs for the `options` map are:

- skip: <number> — skip the top N results.
- limit: <number> — limit the number of results returned.
- analyzer: <string> — use the specified analyzer as a search analyzer for this query.

The `options` map and all of the keys are optional.

For more information, see Full-text indexes - Query full-text indexes.

Vector indexes

As of Neo4j 2026.01, the preferred way of querying vector indexes is by using the Cypher SEARCH clause. A less powerful alternative, which also works for earlier Neo4j versions, is using the procedures db.index.vector.queryRelationships.

Query vector index on nodes using SEARCH

```
MATCH (n)
  [WHERE ...]
  SEARCH n IN (
    VECTOR INDEX index_name
    FOR query_vector
    [WHERE ...]
    LIMIT limit
  ) [SCORE AS score_alias]
```

Query vector index on relationships using SEARCH

```
MATCH ()-"["r"]"->()
  [WHERE ...]
  SEARCH r IN (
    VECTOR INDEX index_name
    FOR query_vector
    [WHERE ...]
    LIMIT limit
  ) [SCORE AS score_alias]
```

Query vector index on nodes using procedure: db.index.vector.queryNodes()

```
CALL db.index.vector.queryNodes(indexName :: STRING, numberOfNearestNeighbours :: INTEGER, query :: VECTOR
| LIST<INTEGER | FLOAT>)
```

Query vector index on relationships using procedure: db.index.vector.queryRelationships()

```
CALL db.index.vector.queryRelationships(indexName :: STRING, numberOfNearestNeighbours :: INTEGER, query
:: VECTOR | LIST<INTEGER | FLOAT>)
```

The `numberOfNearestNeighbours` refers to the number of nearest neighbors to return as the neighborhood. The `query` refers to the `LIST<INTEGER | FLOAT>` or, as of Neo4j 2025.10, additionally the `VECTOR` of which the neighborhood should be searched.

For more information, see Vector indexes - Query vector indexes.

DROP INDEX

The `DROP INDEX` command can drop indexes of all types using their name. The name of the index can be found using the `SHOW INDEXES` command, given in the output column `name`.

The `DROP INDEX` command is optionally idempotent. This means that its default behavior is to throw an error if an attempt is made to drop the same index twice. With `IF EXISTS`, no error is thrown and nothing happens should the index not exist. Instead, an informational notification is returned detailing that the index does not exist.

> Dropping indexes requires the `DROP INDEX` privilege.

Drop an index of any index type

```
DROP INDEX index_name [IF EXISTS]
```

For more information, see Drop indexes.

[1] The example queries on this page are prepended with `PROFILE`. This both runs the query and generates its execution plan. For more information, see Understanding query plans → `EXPLAIN` and `PROFILE` query plans.
[2] The `trackedSince` column is not part of the default return columns for the `SHOW INDEXES` command. To return this and all other non-default columns, use `SHOW INDEXES YIELD *`. For more information, see Create, show, and delete indexes → Return columns.
[3] Lucene implements a Hierarchical Navigable Small World (HNSW) Graph to perform a k approximate nearest neighbors (k-ANN) query over the vector fields. For more information, see Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs — Yury A. Malkov and Dmitry A. Yashunin
[4] IEEE Standard for Floating-Point Arithmetic

Schema

Neo4j is a natively schema-free database. This allows for the rapid development of graphs that can continue to evolve and expand by incorporating new sources of data.

You can, however, impose an open schema in Neo4j. An open schema applies constraints only to the data explicitly included in the schema. This approach maintains a flexible framework that makes it easy to adapt and grow your data model as new requirements emerge, while also allowing for enforcing the shape of the most stable and established part of the data model.

There are two methods of imposing an open schema in Neo4j:

- Graph types: enforces holistically an open schema, providing a defined structure for an entire graph. Cypher 25 only | Introduced in 2026.02 | Enterprise Edition
- Constraints: enforces specific constraints on node or relationship properties.

> **Note:**
>
> All constraints created using the older `CREATE CONSTRAINT` syntax will automatically be added to the graph type of a database. Not all constraint types can be created using this syntax, however, and maintaining individual constraints can become complicated over time as their number increases.
>
> **It is, therefore, recommended to define a schema using a graph type, which offers both additional, more sophisticated constraint types and a more holistic and simplified approach for constraining and maintaining the shape of the data in a graph.**

Graph types

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

> Graph types are generally available starting in Neo4j 2026.06. Prior to this release, the feature was in preview.

Graph types allow you to specify a graph schema for a database.

When a database is created, it has an empty graph type, meaning it is free from any constraints on what can be stored in a database. Setting a graph type, typically done following the creation of a database, enables users to impose a schema on the nodes, relationships, and properties included in the graph type, thereby ensuring the integrity and quality of the data in a graph.

Cypher supports open graph types, meaning that a graph type only constrains the data included by the graph type. It does not impose any constraints on nodes, relationships, and properties not included by the graph type.

For more information about working with graph types, see:

- Set graph types
- Extend graph types
- Alter element types
- Show graph types
- Drop graph type elements

For reference material about the Cypher commands used to manage graph types, see Syntax → Graph types.

Further reading

- Neo4j Developer Blog: GRAPH TYPE – Schema enforcement made easy (Preview)

Set graph types

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

A graph type defines the schema of a database and is set with the command `ALTER CURRENT GRAPH TYPE SET`. **Note that any previously defined graph types or constraints are overwritten and replaced when a graph type is set.**

Graph types can include:

- Node element types
- Relationship element types
- Property uniqueness and key constraints defined on identifying node labels or relationship types
- Property existence, property type, property uniqueness, and key constraints defined on non-identifying node labels or relationship types

The three first categories are discussed in the section Set a graph type on an empty database. The last category is discussed in the section Set a graph type on a populated database.

Note:

Setting a graph type requires the following privileges:

- `CREATE CONSTRAINT`
- `DROP CONSTRAINT`
- `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

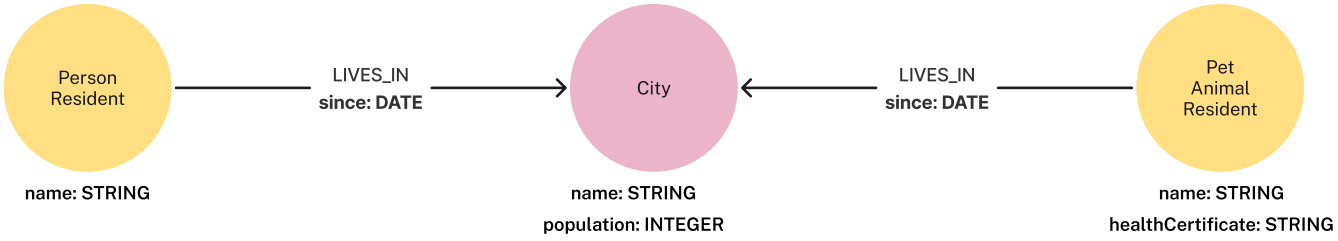
Selecting a data model

Before setting a graph type, it is crucial to develop a data model for the graph, as the graph type serves to

enforce a schema derived from that data model.

Modeling data in Neo4j is essential for representing complex, interconnected relationships using nodes, relationships, and properties. Unlike traditional databases, its graph structure mirrors real-world connections, making it easier to traverse and query (for more information about data modeling, see the Getting Started Guide → Data modeling).

The examples on this page have as their starting point a data model that identifies persons, pets, and the cities they reside in. Key entities and their attributes are defined, including the names of people, pets, and cities. Persons and pets are both classified as residents, and the data model captures how long both have lived in a particular city.



Set a graph type on an empty database

All databases are created with an empty graph type. Since setting a graph type can be a more involved process if a database already has pre-existing data, it is recommended to set the desired graph type before populating a graph with data.

Set a graph type to ensure the integrity of the data model in a database

```
ALTER CURRENT GRAPH TYPE SET {
    (:Person => :Resident {name :: STRING NOT NULL}),
    (:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),
    (:City => {name :: STRING NOT NULL, population :: INTEGER}),
    (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
}
```

Node element types

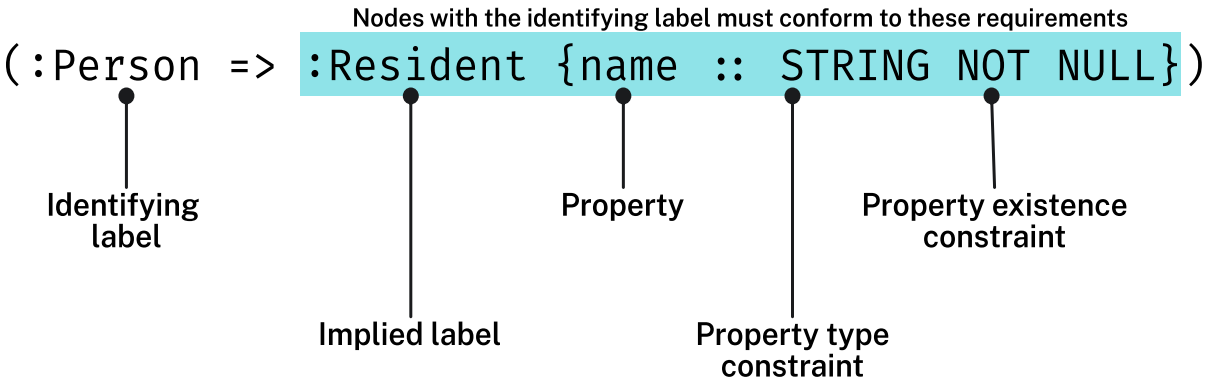


Figure 2. Node element type example

Node element types are defined by an **identifying label**. Node element types ensure that one or more of the following is true for all nodes with an identifying label:

- Certain additional labels must exist . The accompanying labels are referred to as **implied labels**.

- Certain properties must exist.
- Certain properties must be of a specific property type. For a list of accepted property types, see Values and types → Property types.

Node element types in graph type

```
(:Person => :Resident {name :: STRING NOT NULL}),
(:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),
(:City => {name :: STRING NOT NULL, population :: INTEGER})
```

Node element type ensuring that nodes with a `Person` label must also have a `Resident` label and that `Person` nodes must have a `name` property of type `STRING`. The `Person` label is the identifying label of this node element type, and `Resident` is the implied label.

Node element type ensuring that nodes with a `Pet` label must also have a `Resident` and an `Animal` label. The node element type also ensures that any `healthCertificate` and `name` properties on `Pet` nodes must be of type `STRING`. The `Pet` label is the identifying label of this node element type while `Resident` and `Animal` are implied labels.

Node element type ensuring nodes with a `City` label must have a `name` property of type `STRING`, and that any `population` properties are of type `INTEGER`. `City` is the identifying label of this node element type, and there are no implied labels.

A shorthand syntax equivalent for `NOT NULL` is to use an exclamation mark `!`. For example, `(:Person => :Resident {name :: STRING!})`, is the syntactical equivalent of `(:Person => :Resident {name :: STRING NOT NULL})`.

Node element type syntax

```
(:IdentifyingLabel => [:ImpliedLabel1[&...]]  ["{" {property1 :: <TYPE> [NOT NULL]}[, ...]"}"])
```

If `ANY` is specified as the `<TYPE>`, the node element type will allow all valid property types to be assigned to the given property (`ANY` is only allowed if the property's existence is enforced, that is if it is appended with `NOT NULL`).

Relationship element types

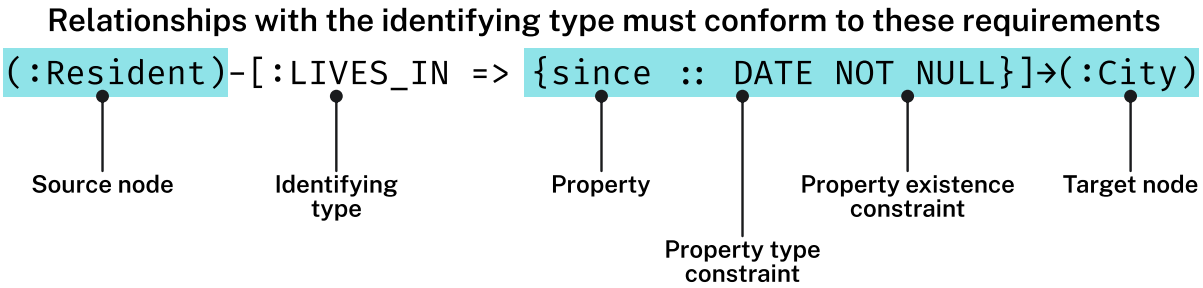


Figure 3. Relationship element type example

Relationship element types are defined by an **identifying type**. Relationship element types ensure that one or more of the following is true for relationships with an identifying type:

- The source node must have a specific label.

- The target node must have a specific label.
- Certain properties must exist.
- Certain properties must be of a specific property type. For a list of accepted property types, see Values and types → Property types.

Relationship element type in graph type

```
(:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
```

Relationship element type ensuring that all relationships with the identifying type `LIVES_IN` connect a `Resident` node to a `City` node, and have a `since` property of type `DATE`. `Resident` is the source node element type, and `City` is the target node element type.

Relationship element type syntax

```
([:SourceNodeLabel])-"[":IDENTIFYING_REL_TYPE => [ "{" {property1 :: <TYPE> [NOT NULL]}[, ...]"}" ] "]"-
>([:TargetNodeLabel])
```

Note:

Relationship element types do not require both end nodes to be defined. Specifically:

- If the relationship element type defines either a source or a target node, the other is not required (e.g., `(:Resident)-[:LIVES_IN =>]->()` is acceptable).
- If the relationship element type includes properties, both the source and target nodes are optional (e.g., `()-[:LIVES_IN => {since :: DATE NOT NULL}]->()` is acceptable).

Identifying node labels and relationship types

Node element types have identifying node labels and relationship element types have identifying relationship types. These identifying node labels and relationship types are used to uniquely identify a node or relationship element type in a graph type. For example, in a node element type definition like `(:Person => :Resident {...})`, the `Person` label is the identifying label, indicating that the node element type represents a node with a `Person` label.

You cannot have different element types with the same identifying node label or relationship type.

Allowed

```
(:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),
(:Person)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
(:Person)-[:BORN_IN => {dob :: DATE NOT NULL})]->(:City)
```

Not allowed — identifying relationship types among relationship element types must be unique

```
(:Person)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
(:Pet)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
```

Not allowed — identifying node labels among node element types must be unique

```
(:Person => :Resident {name :: STRING}),
(:Person => :Citizen {name :: STRING})
```

Implied node labels and their use in relationship element types

Node element types can also have one or more implied labels (relationship element types cannot have implied relationship types because relationships in Neo4j can only have one type). These labels further describe or refine the node element type, adding more information. For example, in a node element type definition like `(:Person => :Resident {...})`, `Resident` is the implied label, meaning that the node represents a `Person` who is also a `Resident`, but `Person` remains the main identifier.

Implied node labels, when shared by multiple element node types, can be useful to define relationship element types with multiple source or target element node types. For example, if `Resident` is an implied label in both the `Person` and `Pet` node element types, it can also be used as the source node for the relationship element type `LIVES_IN`.

Allowed

```
(:Person => :Resident {name :: STRING NOT NULL}),
(:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),
(:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
```

Not allowed — using a node label as both identifying and implied

```
(:Person => :Resident {name :: STRING NOT NULL}),
(:Resident => :Citizen {nationality :: STRING})
```

Not allowed — identifying relationship types among relationship element types must be unique

```
(:Person => :Resident {name :: STRING NOT NULL}),
(:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),
(:Person)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
(:Pet)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
```

Property uniqueness and key constraints on identifying node labels and relationship types

The examples so far have included node and relationship element types constraining the following:

- The types of specific properties on nodes with an identifying label and relationships with an identifying type.
- The existence of specific properties on nodes with an identifying label and relationships with an identifying type.
- The existence of implied labels on nodes with an identifying label.
- The source and target nodes of relationships with an identifying type.

However, node and relationship element types can also be defined to include property uniqueness and key constraints on their identifying node label or relationship type.

Set a graph type that includes key constraints on identifying node labels

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
      REQUIRE (p.name, p.ssn) IS KEY,
    (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
    (c:City => {name :: STRING NOT NULL, population :: INTEGER}),
    (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(c)
}
```

Node element type ensuring that nodes with a `Person` label must also have a `Resident` label and that any `name` and `ssn` (social security number) properties on `Person` nodes are of type `STRING` and `INTEGER` respectively. The `Person` node element type defines a node key constraint ensuring that `Person` nodes must have both a `name` and an `ssn` property and that their combined value must be unique.

Node element type ensuring that nodes with a `Pet` label must also have a `Resident` and an `Animal` label. The node element type defines a node key constraint requiring `Pet` nodes to have a unique `insuranceNumber` property of type `INTEGER`. The node element type also ensures that any `healthCertificate` properties on `Pet` nodes must be unique and of type `STRING` and that any `name` properties are of type `STRING`.

The example above sets a new graph type, **thus replacing the previously set graph type**. It also uses variables to bind node element types. Variables provide a concise way to reference elements specified in a graph type and are necessary when using the `REQUIRE` keyword.

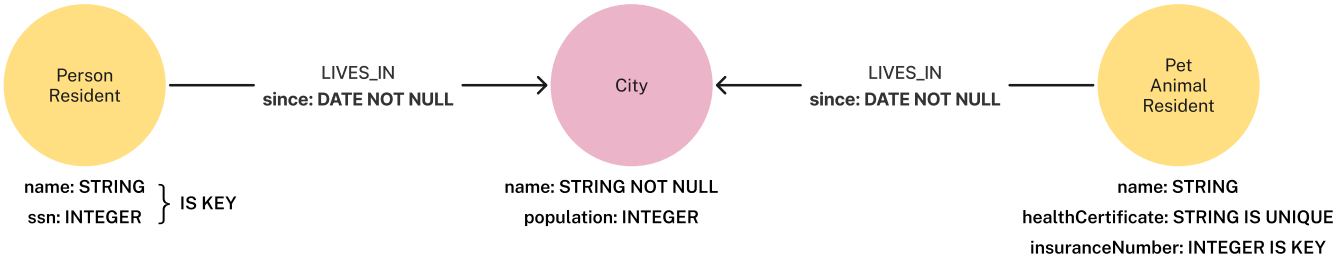


Figure 4. Updated data model, showing addition of key constraints on identifying node labels in graph type

The `REQUIRE` keyword and composite property uniqueness and key constraints

Single property uniqueness and key constraints can be defined in a node or relationship element type with or without the `REQUIRE` keyword.

Syntax for defining single property uniqueness or key constraints with and without the `REQUIRE` keyword

```
(n:Label => “{“property :: <TYPE> IS {UNIQUE | KEY} “}”),
(n:Label => “{“property :: <TYPE>”}”)
  REQUIRE n.property IS { UNIQUE | KEY }

()-”[“r:REL_TYPE => “{“property:: <TYPE> IS {UNIQUE | KEY} ”}”]->(),
()-”[“r:REL_TYPE => “{“property:: <TYPE>”}” “]”->()
  REQUIRE r.property IS { UNIQUE | KEY }
```

To define composite uniqueness and key constraints on more than one property in node and relationship element types it is necessary to use the `REQUIRE` keyword and the constrained properties must be parenthesized.

Syntax for defining composite property uniqueness or key constraints

```
(n:Label => “{“property1 :: <TYPE>, property2 :: <TYPE>[, ...]”}”)
  REQUIRE (n.property1, n.property2[, ...]) IS {UNIQUE | KEY}

()-”[“r:REL_TYPE => “{“property1:: <TYPE>, property2 :: <TYPE>[, ...] “}” “]”->()
  REQUIRE (r.property1, r.property2[, ...]) IS {UNIQUE | KEY}
```

Properties specified in a `REQUIRE` clause are not required to exist on the node or relationship element types to which the constraint applies. For example, the following is a valid element type key constraint defined against a node element type: `(n:Label => {property1 :: STRING, property2 :: INTEGER}) REQUIRE (n.property3, n.property4) IS KEY`.

The `REQUIRE` keyword can also be used as an alternative syntax to define several property uniqueness and key constraints within the same node or relationship element type.

Syntax for defining multiple property uniqueness or key constraints using the `REQUIRE` keyword

```
(n:Label => “{“property1 :: <TYPE>, property2 :: <TYPE>”}”)
    REQUIRE n.property1 IS {UNIQUE | KEY}
    REQUIRE n.property2 IS {UNIQUE | KEY}

()-”[“r:REL_TYPE => “{“r.property1 :: <TYPE>, r.property2 :: <TYPE> “}” “]”->()
    REQUIRE r.property1 IS {UNIQUE | KEY}
    REQUIRE r.property2 IS {UNIQUE | KEY}
```

Naming property uniqueness and key constraints

Property uniqueness and key constraints can also be created using the `CONSTRAINT … FOR … REQUIRE` syntax. If so, they can be given a name, which must be unique across both indexes and constraints (if a name is not explicitly given, one will be assigned automatically). The constraint name can be used in the `ALTER CURRENT GRAPH TYPE DROP` or `DROP CONSTRAINT` commands. For more information, see Drop constraints on identifying and non-identifying node labels and relationship types.

The following two examples define the same key constraint on an identifying node label. However, only in the latter, which uses the `CONSTRAINT … FOR … REQUIRE` syntax, is it explicitly named.

Define a key constraint without explicitly giving a name

```
(p:Person => :Resident {name :: STRING, ssn :: INTEGER})
  REQUIRE (p.name, p.ssn) IS KEY
```

Define an explicitly named key constraint

```
(:Person => :Resident {name :: STRING, ssn :: INTEGER}),
CONSTRAINT name_ssn_constraint FOR (p:Person) REQUIRE (p.name, p.ssn) IS KEY
```

Define a single property uniqueness or key constraint using the using the `CONSTRAINT … FOR … REQUIRE` syntax

```
(:Label => “{“property :: <TYPE>”}”),
CONSTRAINT [name] FOR (n:Label) REQUIRE n.property IS {UNIQUE | KEY}

()-”[“:REL_TYPE => “{“property:: <TYPE>”}” “]”->(),
CONSTRAINT [name] FOR ()-”[“r:REL_TYPE”]”->() REQUIRE r.property IS {UNIQUE | KEY}
```

Define a composite property uniqueness or key constraint using the `CONSTRAINT … FOR … REQUIRE` syntax

```
(:Label => “{“property1 :: <TYPE>, property2 :: <TYPE>”[, ...]}”),
CONSTRAINT [name] FOR (n:Label) REQUIRE (n.property1, n.property2[, ...]) IS {UNIQUE | KEY}

()-”[“:REL_TYPE => “{“property1 :: <TYPE>, property2 :: <TYPE>[, ...]”}” “]”->(),
CONSTRAINT [name] FOR ()-”[“r:REL_TYPE”]”->() REQUIRE (r.property1,r.property2[, ...]) IS {UNIQUE | KEY}
```

To define a constraint on node labels and relationship types that do not serve as identifying node labels or relationship types in a graph type it is necessary to use the `CONSTRAINT … FOR … REQUIRE` syntax. For more information, see Constraints on non-identifying node labels and relationship types below.

Create data after setting a graph type

The purpose of a graph type is to impose a schema on the nodes, relationships, and properties included within it. The data created in a graph must accordingly abide by the schema defined in the graph type.

Current graph type

```
(p:Person => :Resident {name :: STRING, ssn :: INTEGER})
  REQUIRE (p.name, p.ssn) IS KEY,
(:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE, name
:: STRING}),
(c:City => {name :: STRING NOT NULL, population :: INTEGER}),
(:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)
```

Create data complying with the graph type

```
CREATE (alice:Person:Resident {name: 'Alice Carlyle', ssn: 987654321}),
       (ben:Person:Resident {name: 'Benjamin Davis', ssn: 456789123}),
       (ozzy:Pet:Resident:Animal {insuranceNumber: 876543210, healthCertificate: 'HC789123', name:
'Ozzy'}),
       (zoey:Pet:Resident:Animal {insuranceNumber: 564738291, name: 'Zoey'}),
       (nyc:City {name: 'New York City', population: 8097282}),
       (la:City {name: 'Los Angeles'}),
       (alice)-[:LIVES_IN {since: date('2018-06-15')}]->(nyc),
       (ozzy)-[:LIVES_IN {since: date('2022-08-09')}]->(nyc),
       (ben)-[:LIVES_IN {since: date('1999-10-17')}]->(la),
       (zoey)-[:LIVES_IN {since: date('2015-08-11')}]->(la)
```

Note the absence of a `healthCertificate` property on the `Pet` node `'Zoey'`, and a `population` property on the `'Los Angeles'` `City` node. These omissions are allowed by the graph type, as neither node element type requires the existence of these properties -— only that if they are present, they conform to the specified property type.

Create data violating a node element type

```
CREATE (:Person:Resident {name: 'Alice Carlyle', ssn: 987654321})
CREATE (:Person {name: 'Alice Carlyle', ssn: 123456789})
CREATE (:Person:Resident {name: 'Alice Carlyle'})
CREATE (:Person:Resident {name: 'Alice Carlyle', ssn: 'HNB48182'})
```

This will fail because it would create a node with properties identical to an existing one, thereby violating the key constraint defined in the `Person` node element type.

This will fail because the `Person` node element type requires nodes with a `Person` label to also have a `Resident` label.

This will fail because not all properties constrained in the graph type key constraint (`name`, `ssn`) defined in the `Person` node element type are present.

This will fail because a type-constrained property (`ssn`) is of a different type (`STRING`) than the one defined for the property in the `Person` node element type (`INTEGER`).

Create data violating a relationship element type

```
CREATE (:Human {name: 'Carl Ericsson' ssn: 765498321})-[:LIVES_IN {since: date('1999-10-17')}]->(:City
{name: 'Toronto'})
MATCH (alice:Person:Resident {name: 'Alice Carlyle'})
CREATE (alice)-[:LIVES_IN {since: date('1999-10-17')}]->(:Country {name: 'Canada'})
CREATE (alice)-[:LIVES_IN]->(:City {name: 'Toronto'})
CREATE (alice)-[:LIVES_IN {since: '1999-10-17'}]->(:City {name: 'Toronto'})
```

This will fail because the source node defined in the `LIVES_IN` relationship element type must have a `Resident` label.

This will fail because the target node defined in the `LIVES_IN` relationship element type must have a `City` label.

This will fail because an existence-constrained property (`since`) defined in the `LIVES_IN` relationship element type is missing.

This will fail because a type-constrained property (`since`) is of a different type (`STRING`) than the one defined for the property in the `LIVES_IN` relationship element type (`DATE`).

Open graph type capabilities

The great benefit of working with open graph types is that they allow you to constrain specific data while remaining flexible to add or modify data not constrained by the graph type.

Create nodes and relationships with labels/types defined as identifying in the graph type, but with properties not defined in their respective node or relationship element types

```
CREATE (carl:Person:Resident {name: 'Carl Ericson', ssn: 162734679, born: date('1998-08-08')}),
       (molly:Pet:Resident:Animal {name: 'Molly', insuranceNumber: 672829172, healthCertificate:
'HT654987', breed: 'Alsatian'}),
       (atlanta:City {name: 'Atlanta', population: 2794356, country: 'USA'}),
       (carl)-[:LIVES_IN {since: date('2020-09-15'), address: '101, Jasper Avenue'}]->(atlanta)<-
[:LIVES_IN {since: date('2022-10-22')}]-(molly)
```

There is no `born` property defined in the `Person` node element type.

There is no `breed` property defined in the `Pet` node element type.

There is no `country` property defined in the `City` node element type.

There is no `address` property defined in the `LIVES_IN` relationship element type.

Create nodes with implied labels but no identifying labels

```
CREATE (:StrayAnimal:Animal:Resident {id: '24.09-172898'}),
       (:Robot:Resident {name: 'Gary', application: 'Veterinary medicine'})
```

Create nodes with node labels and relationship types not included in the graph type

```
MATCH (ben:Person {name: 'Benjamin Davis'}), (zoey:Pet {name: 'Zoey'}), (gary:Robot {name: 'Gary'})
CREATE (ben)-[:OWNER_OF {ownershipId: 'GTHD-985'}]->(zoey),
```

```
      (ben)-[:WORKS_FOR {role: 'Product Manager'}]->(healthyPets:Company {name: 'Healthy Pets Inc.',
address: '4567 Wellness Drive, Los Angeles, CA 90001'}),
      (gary)-[:INSTALLED_AT]->(healthyPets)
```

Set a graph type on a populated database

If a database contains data conflicting with the requirements of a graph type, it will only be possible to set the desired graph type once any conflicting data has been modified to abide by its schema. Setting a graph type on a populated database will, therefore, involve checks on the existing data. Note that any previously defined graph types are overwritten and replaced when a graph type is set.

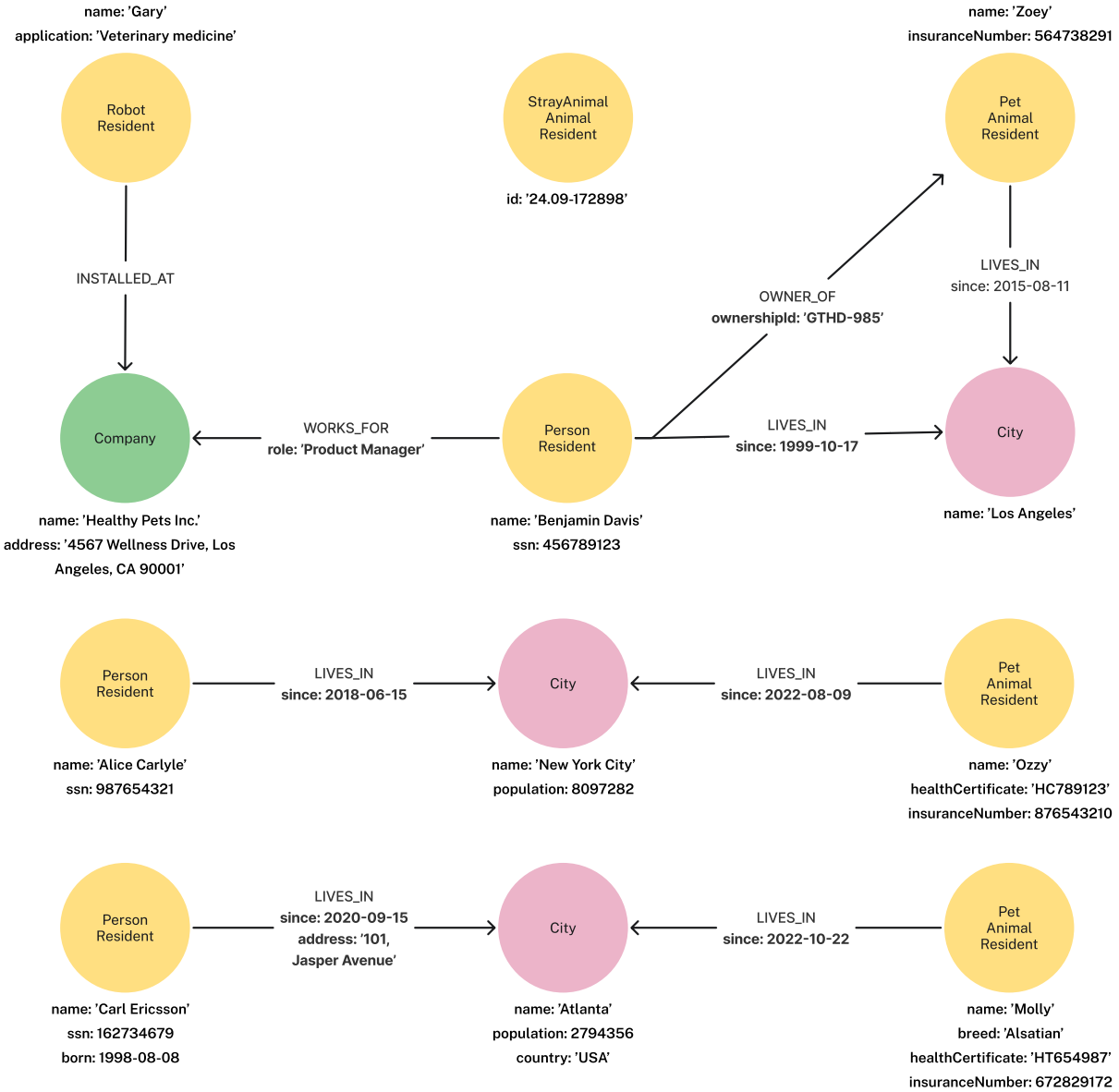


Figure 5. Data in the database

The following graph type could not be set in a database containing the data created in the previous section:

Not allowed — setting a graph type conflicting with existing data

```
ALTER CURRENT GRAPH TYPE SET {
  (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
    REQUIRE (p.name, p.ssn) IS KEY,
  (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
```

```
  (:City => {name :: STRING NOT NULL, population :: INTEGER}),
  (:Robot => :Resident {name :: STRING, id :: INTEGER IS KEY}),
  (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
  (:Person)-[r:OWNER_OF => {ownershipId :: INTEGER IS UNIQUE}]->(:Pet)
}
```

This node element type cannot be validated because it requires every `Robot` node to have a unique `id` property of type `INTEGER` (the previously created `Robot` node has no `id` property).

This relationship element type is invalid because it requires that any `ownershipId` properties on `OWNER_OF` relationships must be of type `INTEGER` (the previously created `OWNER_OF` relationship has an `ownershipId` property of type `STRING`).

When situations like the example above arise, users should migrate the existing data to the new desired model and then apply the new graph type that will keep the data consistent from then on.

Constraints on non-identifying node labels and relationship types

The examples thus far have included node and relationship element types, as well as property uniqueness and key constraints defined on identifying node labels and relationship types.

It is also possible to include constraints in a graph type on implied node labels, or on node labels and relationship types which are independent of a node or relationship element type, using the `CONSTRAINT … FOR … REQUIRE` syntax outlined above. This can be a useful tool if the data model requires certain attributes for nodes or relationships with non-identifying labels or types.

> Constraints on non-identifying node labels and relationship types can be explicitly named when defined. This can be useful when dropping constraints on identifying and non-identifying node labels and relationship types (if a name is not explicitly given, a name will be assigned). The given name must be unique among both indexes and constraints.

Property uniqueness and key constraints can be defined against any node label or relationship type in the graph, regardless of whether they serve as identifying node labels or relationship types (an example of a key constraint defined against the identifying node label of a node element type can be found above).

Set a graph type with key and property uniqueness constraints on non-identifying node labels

```
ALTER CURRENT GRAPH TYPE SET {
  (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
    REQUIRE (p.name, p.ssn) IS KEY,
  (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
  (:City => {name :: STRING NOT NULL, population :: INTEGER}),
  (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
  CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
  CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE
}
```

Node key constraint defined against a node label (`:Company`) which is not part of any element type (either as identifying or implied) in the graph type.

Property uniqueness constraint defined against a node label (`:Animal`) which serves as an implied label in the `Pet` node element type.

The example above sets a new graph type, **thus replacing the previously set graph type.**

Property existence and property type constraints can also be defined on non-identifying labels and relationship types. **Unlike property uniqueness and key constraints, however, they cannot constrain identifying node labels or relationship types if they are defined outside of the node or relationship element type.** This is because node and relationship element types can directly express the effects of property existence and property type constraints on the identifying node labels or relationship types.

Allowed — define property existence constraint in a node element type

```
(:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE, name
:: STRING, address :: ANY NOT NULL})
```

Not allowed — add property existence constraint on an identifying node label outside of a node element type

```
(:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE, name
:: STRING}),
CONSTRAINT pet_address FOR (pet:Pet) REQUIRE pet.address IS NOT NULL
```

Property existence and type constraints cannot be given a name if expressed in a node or relationship element type (they will always be assigned a generated name). For more information, see Show graph types → Graph type elements in `SHOW CONSTRAINTS`.

Most use cases can thus be served by the node and relationship element types in a graph type. However, adding property existence and type constraints on non-identifying node labels or relationship types can be useful for defining more specialized constraints, especially in an already populated database.

Set a graph type including property type constraint on non-identifying node label

```
ALTER CURRENT GRAPH TYPE SET {
  (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
    REQUIRE (p.name, p.ssn) IS KEY,
  (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
  (:Robot => :Resident {application :: STRING NOT NULL}),
  (:City => {name :: STRING NOT NULL, population :: INTEGER}),
  (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
  CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
  CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
  CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address IS :: STRING
}
```

Property type constraint ensuring that any address properties on `RESIDENT` nodes are of type `STRING`. This constraint is valid because `Resident` is not an identifying label in any of the above node element types.

The example above sets a new graph type, **thus replacing the previously set graph type.**

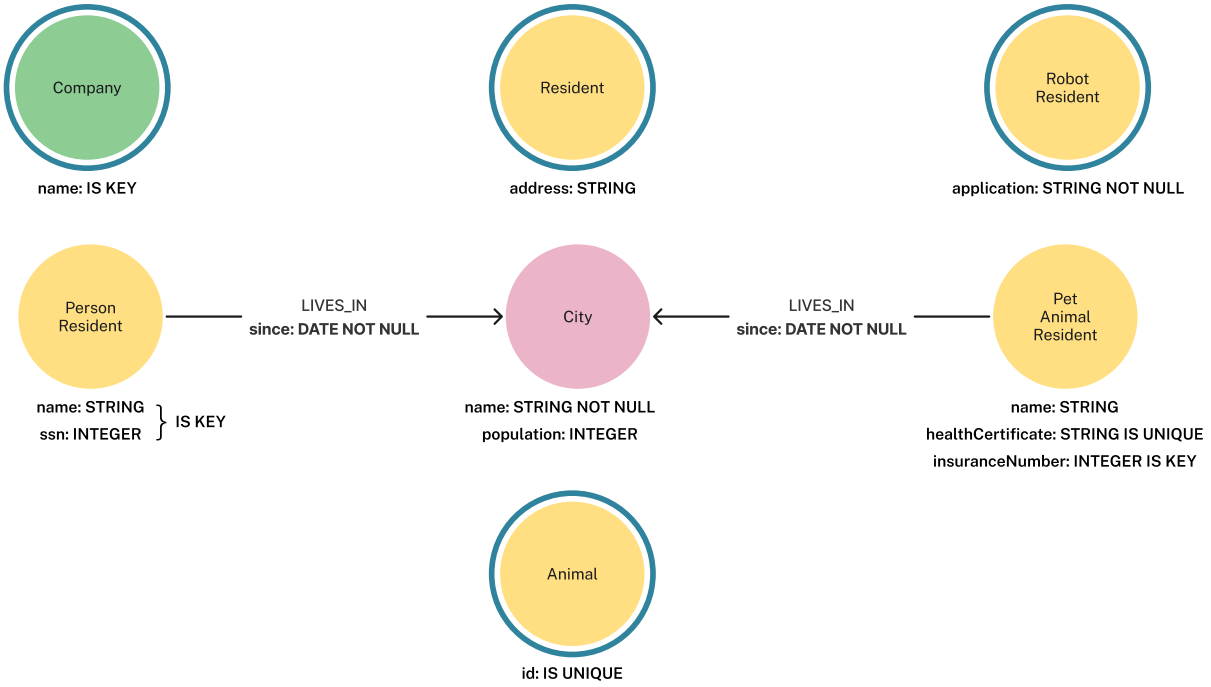


Figure 6. Updated data model, showing new `Robot` node element type and addition of constraints on non-identifying node labels in graph type

Graph type implementation: translation to constraints

Once a graph type is defined, it is translated into a set of constraints. These constraints ensure that the creation and development of a graph conforms to the open schema specified by the graph type.

The constraints that enforce a graph type are returned by the `SHOW CONSTRAINTS` command. For more information, see Show graph types → Graph type elements in `SHOW CONSTRAINTS`.

Current graph type

```
(p:Person => :Resident {name :: STRING, ssn :: INTEGER})
   REQUIRE (p.name, p.ssn) IS KEY,
(:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE, name
:: STRING}),
(:Robot => :Resident {application :: STRING NOT NULL}),
(:City => {name :: STRING NOT NULL, population :: INTEGER}),
(:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING
```

Node and relationship element types

Identifying node label/relationship type	Constraint type	Details
`(:Person)`	Node property type	`name :: STRING`
`(:Person)`	Node property type	`ssn :: INTEGER`
`(:Person)`	Node label existence	Nodes with a `Person` label must also have a `Resident` label.
`(:Pet)`	Node property type	`insuranceNumber :: INTEGER`

Identifying node label/relationship type	Constraint type	Details
`(:Pet)`	Node property type	`healthCertificate :: STRING`
`(:Pet)`	Node property type	`name :: STRING`
`(:Pet)`	Node label existence	Nodes with a `Pet` label must also have a `Resident` label.
`(:Pet)`	Node label existence	Nodes with a `Pet` label must also have an `Animal` label.
`(:Robot)`	Node property type	`application :: STRING`
`(:Robot)`	Node property existence	`application IS NOT NULL`
`(:Robot)`	Node label existence	Nodes with a `Robot` label must also have a `Resident` label.
`(:City)`	Node property type	`name :: STRING`
`(:City)`	Node property existence	`name IS NOT NULL`
`(:City)`	Node property type	`population :: INTEGER`
`()-[:LIVES_IN]->()`	Relationship property type	`since :: DATE`
`()-[:LIVES_IN]->()`	Relationship property existence	`since IS NOT NULL`
`()-[:LIVES_IN]->()`	Relationship source label	Source node must have the label `Resident`.
`()-[:LIVES_IN]->()`	Relationship target label	Target node must have the label `City`.

Key and property uniqueness constraints on node identifying node labels and relationship types

Identifying node label/relationship type	Constraint type	Details
`(:Person)`	Node key	`(name, ssn) IS KEY`
`(:Pet)`	Node key	`insuranceNumber IS KEY`
`(:Pet)`	Node property uniqueness	`healthCertificate IS UNIQUE`

Constraints on non-identifying node labels

Name	Node label/Relationship type	Constraint type	Details
`company_name`	`(:Company)`	Node key	`name IS KEY`
`animal_id`	`(:Animal)`	Node property uniqueness	`id IS UNIQUE`
`resident_address`	`(:Resident)`	Node property type	`name :: STRING`

Constraint types explained

Node label existence constraints ensure that a particular label exists on a node. This constraint will only be created if a node element type specifies implied labels, and **it cannot be created the `CREATE CONSTRAINT` command.**

Relationship source and target label constraints ensure that specific relationship types have target and source nodes with specific labels. A relationship source label constraint will only be created if a relationship element type specifies a source node. A relationship target label constraint will only be created if a relationship type specifies a target node. **Neither of these constraints can be created using the `CREATE CONSTRAINT` command.**

Property type constraints ensure a property has the required property type for all nodes with a specific label or all relationships with a specific type. To learn more about property type constraints and how to create them using the `CREATE CONSTRAINT` syntax, see Constraints → Create property type constraints.

Property existence constraints ensure a property exists on all nodes with a specific label or all relationships with a specific type. To learn more about property existence constraints and how to create them using the `CREATE CONSTRAINT` command, see Constraints → Create property existence constraints.

Property uniqueness constraints ensure that the combined property values are unique for all nodes with a specific label or all relationships with a specific type. To learn more about property uniqueness constraints, and how to create them using the `CREATE CONSTRAINT` command, see Constraints → Create property uniqueness constraints.

Key constraints ensure that all properties exist and that the combined property values are unique for all nodes with a specific label or all relationships with a specific type. To learn more about key constraints, and how to create them using the `CREATE CONSTRAINT` command, see Constraints → Create key constraints.

Extend graph types

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

A graph type can be extended with the command `ALTER CURRENT GRAPH TYPE ADD`. This command cannot alter existing elements in a graph type but adds entirely new elements to an existing graph type.

> **Note:**
>
> Extending a graph type requires the following privileges:
>
> - `CREATE CONSTRAINT`
> - `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

Set graph type

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
      REQUIRE (p.name, p.ssn) IS KEY,
    (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
    (:Robot => :Resident {application :: STRING NOT NULL}),
    (:City => {name :: STRING NOT NULL, population :: INTEGER}),
```

```
    (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
    CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
    CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
    CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING
}
```

For details about the different elements a graph type can contain, see Set graph types.

Extend a graph type by adding new element types

```
ALTER CURRENT GRAPH TYPE ADD {
    (:Company => {name :: STRING, address :: STRING IS UNIQUE}),
    (:Person)-[:WORKS_FOR => {role :: STRING}]->(:Company)
}
```

This adds a node element type with an identifying label also used by an existing key constraint in the graph type (`company_name`). This addition is possible for property uniqueness and key constraints originally not defined on an identifying label, but not for property existence and property type constraints.

The extension adds one node element type and one relationship element type to the graph type. It is implemented with the following constraints:

Identifying node label/relationship type	Constraint type	Details
`(:Company)`	Node property type	`name :: STRING`
`(:Company)`	Node property type	`address :: STRING`
`(:Company)`	Node property uniqueness	`address IS UNIQUE`
`()-[:WORKS_FOR]->()`	Relationship property type	`role :: STRING`
`()-[:WORKS_FOR]->()`	Relationship source label	Source node must have the label `Person`.
`()-[:WORKS_FOR]->()`	Relationship target label	Target node must have the label `Company`.

For more information about how graph types are implemented, see Set a graph type → Graph type implementation: translation to constraints.

The graph type now includes the added node and relationship element type:

Current graph type — with added node and relationship element types

```
(p:Person => :Resident {name :: STRING, ssn :: INTEGER})
   REQUIRE (p.name, p.ssn) IS KEY,
(:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE, name
:: STRING}),
(:Robot => Resident {application :: STRING NOT NULL}),
(:City => {name :: STRING NOT NULL, population :: INTEGER}),
(:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING,
(:Company => {name :: STRING, address :: STRING IS UNIQUE}),
(:Person)-[:WORKS_FOR => {role :: STRING}]->(:Company)
```

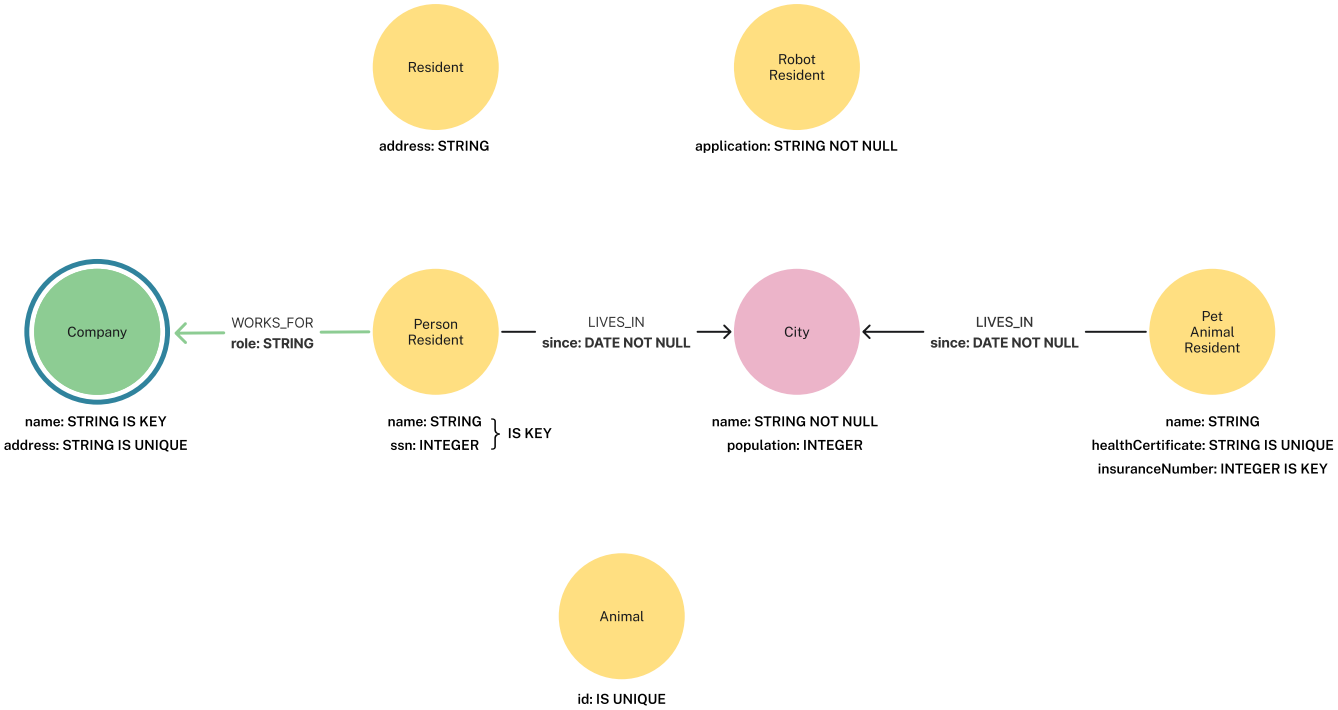


Figure 7. Updated data model, showing added node element type and relationship element type in graph type

Invalid graph type extensions

```
ALTER CURRENT GRAPH TYPE ADD {
  (p:Person => :Resident {name :: STRING, ssn :: INTEGER, nationality :: STRING})
    REQUIRE (p.name, p.ssn) IS KEY,
  CONSTRAINT pet_address FOR (pet:Pet) REQUIRE pet.address IS NOT NULL,
  (:Animal => {type :: STRING NOT NULL})
}
```

This attempts to extend the existing `Person` node element type by adding a new property (`nationality`). This is not valid.

This attempts to add a property type constraint on an identifying label in the existing graph type(`Pet`). This is not valid. The reverse (adding a node or relationship element type with an identifying label/type already defined by a property type or property existence constraint) is also not possible.

This attempts to add a node element type with an identifying label already used as an implied label in a node element type in the existing graph type (`Pet`). This is not valid. The reverse (adding a node element type with an implied label that is already used as an identifying label in an existing node element type) is also not possible.

Alter element types

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

The node and relationship element types in a graph type can be altered using the `ALTER CURRENT GRAPH TYPE ALTER` command. When altering a graph type, only those node or relationship element types to be modified should be included.

Note:

Altering element types in a graph type requires the following privileges:

- `CREATE CONSTRAINT`
- `DROP CONSTRAINT`
- `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

Set graph type

```
ALTER CURRENT GRAPH TYPE SET {
    (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
      REQUIRE (p.name, p.ssn) IS KEY,
    (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
    (:Robot => :Resident {application :: STRING NOT NULL}),
    (:City => {name :: STRING NOT NULL, population :: INTEGER}),
    (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City),
    (:Company => {name :: STRING, address :: STRING IS UNIQUE}),
    (:Person)-[:WORKS_FOR => {role :: STRING}]->(:Company),
    CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
    CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
    CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING
}
```

For details about the different elements a graph type can contain, see Set graph types.

Alter node and relationship element types in a graph type

```
ALTER CURRENT GRAPH TYPE ALTER {
    (:Robot => :Resident&Machine {application :: STRING NOT NULL, id :: INTEGER NOT NULL}),
    (:Resident)-[:LIVES_IN => {since :: ANY NOT NULL}]->(:City)
}
```

Nodes with a `Robot` label must now also have a `Machine` label. `Robot` nodes are also required to have an `id` property of type `INTEGER`.

`since` properties on `LIVES_IN` relationships can now be of `ANY` type (they were previously constrained to be of type `DATE`).

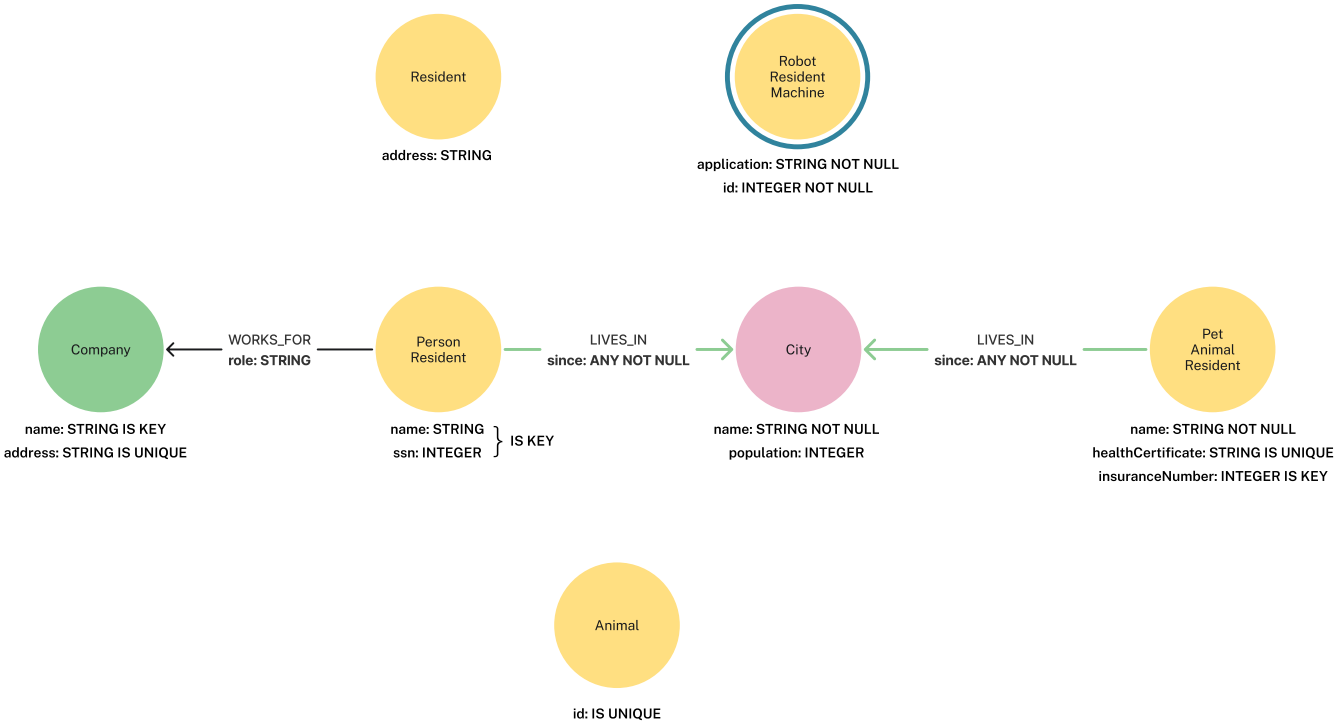


Figure 8. Updated data model, showing altered node element type and relationship element type in graph type

This alteration is implemented by the *creation* of the following constraints:

Identifying node label	Constraint type	Details
`(:Robot)`	Node property type	`id :: INTEGER`
`(:Robot)`	Node property existence	`id IS NOT NULL`
`(:Robot)`	Node label existence	Nodes with a `Robot` label must also have a `Machine` label.

It is also implemented by the *removal* of the following constraint:

Identifying relationship type	Constraint type	Details
`()-[:LIVES_IN]->()`	Relationship property type	`since :: DATE`

So long as the identifying node labels or relationship types are the same as those defined in the current graph type, `ALTER CURRENT GRAPH TYPE ALTER` will alter the element types into whatever is included in the alteration. **In other words, partially defining an element type will not leave the parts of the element type that are not included in the alteration unchanged. Instead, it will redefine the element type based on the reduced definition specified. Note, this does not apply to key and property uniqueness constraints defined on nodes with an identifying label or relationships with an identifying type; these cannot be modified by `ALTER` and will therefore remain unchanged.**

For more information about how graph types are implemented, see Set a graph type → Graph type implementation: translation to constraints.

Rules

The behavior of key and property uniqueness constraints on identifying node labels and relationship types, as well as constraints on non-identifying node labels and relationship types with regard to the `ALTER CURRENT GRAPH TYPE ALTER` command follows that of Neo4j constraints; they can be created and dropped, but they cannot be altered through a single command (they must, in that case, be dropped and recreated to fit the desired alteration).

It is not allowed to add key and property uniqueness constraints to node or relationship element types when altering a graph type. Nor can they be converted into another one through alteration.

Not allowed — alter a node or relationship element type to include a key constraint

```
ALTER CURRENT GRAPH TYPE ALTER {
    (:City => {name :: STRING NOT NULL, population :: INTEGER IS KEY})
}
```

This alteration is invalid because it modifies the `City` node element type to include a node key constraint.

Not allowed — alter a node or relationship element type by converting a key constraint to a property uniqueness constraint

```
ALTER CURRENT GRAPH TYPE ALTER {
(p:Person => :Resident {name :: STRING, ssn :: INTEGER})
    REQUIRE (p.name, p.ssn) IS UNIQUE
}
```

This alteration is not allowed because it attempts to modify the `Person` node element type by converting a node key constraint into a property uniqueness constraint.

Constraints cannot be altered using the `ALTER CURRENT GRAPH TYPE ALTER` command.

Not allowed — alter a constraint on a non-identifying relationship type in a graph type

```
ALTER CURRENT GRAPH TYPE ALTER {
    CONSTRAINT altered_animal_id FOR (a:Animal) REQUIRE a.id IS KEY
}
```

This alteration is not allowed because it tries to convert a property uniqueness constraint to a key constraint on a non-identifying node label. Also, the names of constraints cannot be altered.

Show graph types

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

The `SHOW CURRENT GRAPH TYPE` command allows you to show the graph type specification for the current database. It can either be shown as a string representation or, as of Neo4j 2026.07, a virtual graph representation. For general information about the `SHOW` command, see the `SHOW` page.

> Showing a graph type requires the `SHOW CONSTRAINTS` privilege.

Examples

- Show full graph type as a string
- Show full graph type as a virtual graph
- Combine `SHOW CURRENT GRAPH TYPE` with other Cypher clauses

The graph type used for the examples are:

Set graph type

```
ALTER CURRENT GRAPH TYPE SET {
   (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
     REQUIRE (p.name, p.ssn) IS KEY,
   (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
   (:Robot => :Resident&Machine {application :: STRING NOT NULL, id :: INTEGER NOT NULL}),
   (:City => {name :: STRING NOT NULL, population :: INTEGER}),
   (:Resident)-[:LIVES_IN => {since :: ANY NOT NULL}]->(:City),
   (:Company => {name :: STRING, address :: STRING IS UNIQUE}),
   (:Person)-[:WORKS_FOR => {role :: STRING}]->(:Company),
   CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
   CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
   CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING,
   CONSTRAINT stayed_in_startDateExistence FOR ()-[r:STAYED_IN]->() REQUIRE (r.startDate) IS NOT NULL,
   CONSTRAINT stayed_in_startDateType FOR ()-[r:STAYED_IN]->() REQUIRE (r.startDate) IS :: DATE,
   CONSTRAINT stayed_in_endDate FOR ()-[r:STAYED_IN]->() REQUIRE (r.endDate) IS :: DATE
}
```

Show full graph type as a string

Show the full graph type as a string

```
SHOW CURRENT GRAPH TYPE
```

Result

```
+-----------------------------------------------------------------------------------------------------------
--------------------+
| specification
|
+-----------------------------------------------------------------------------------------------------------
--------------------+
| "{
|
|    (:`City` => {`name` :: STRING NOT NULL, `population` :: INTEGER}),
|
|    (:`Company` => {`address` :: STRING, `name` :: STRING}),
|
|    (:`Person` => :`Resident` {`name` :: STRING, `ssn` :: INTEGER}),
|
|    (:`Pet` => :`Animal`&`Resident` {`healthCertificate` :: STRING, `insuranceNumber` :: INTEGER, `name`
:: STRING}),          |
|    (:`Robot` => :`Machine`&`Resident` {`application` :: STRING NOT NULL, `id` :: INTEGER NOT NULL}),
|
|    (:`Resident`)-[:`LIVES_IN` => {`since` :: ANY NOT NULL}]->(:`City` =>),
|
|    (:`Person` =>)-[:`WORKS_FOR` => {`role` :: STRING}]->(:`Company` =>),
|
|    CONSTRAINT `constraint_d7524c38` FOR (`n`:`Company` =>) REQUIRE (`n`.`address`) IS UNIQUE,
|
|    CONSTRAINT `company_name` FOR (`n`:`Company` =>) REQUIRE (`n`.`name`) IS KEY,
|
|    CONSTRAINT `constraint_bb1018eb` FOR (`n`:`Person` =>) REQUIRE (`n`.`name`, `n`.`ssn`) IS KEY,
|
|    CONSTRAINT `constraint_302a3693` FOR (`n`:`Pet` =>) REQUIRE (`n`.`healthCertificate`) IS UNIQUE,
|
```

```
|    CONSTRAINT `constraint_aa881a72` FOR (`n`:`Pet` =>) REQUIRE (`n`.`insuranceNumber`) IS KEY,
|
|    CONSTRAINT `animal_id` FOR (`n`:`Animal`) REQUIRE (`n`.`id`) IS UNIQUE,
|
|    CONSTRAINT `resident_address` FOR (`n`:`Resident`) REQUIRE (`n`.`address`) IS :: STRING,
|
|    CONSTRAINT `stayed_in_endDate` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`endDate`) IS :: DATE,
|
|    CONSTRAINT `stayed_in_startDateExistence` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`startDate`) IS
NOT NULL,             |
|    CONSTRAINT `stayed_in_startDateType` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`startDate`) IS ::
DATE                  |
| }"
|
+-------------------------------------------------------------------------------------------------------
--------------------+
```

Regardless of which syntax was used to define a property uniqueness or key constraint, they are all shown in the `specification` returned by `SHOW CURRENT GRAPH TYPE` using the `CONSTRAINT … FOR … REQUIRE` syntax.

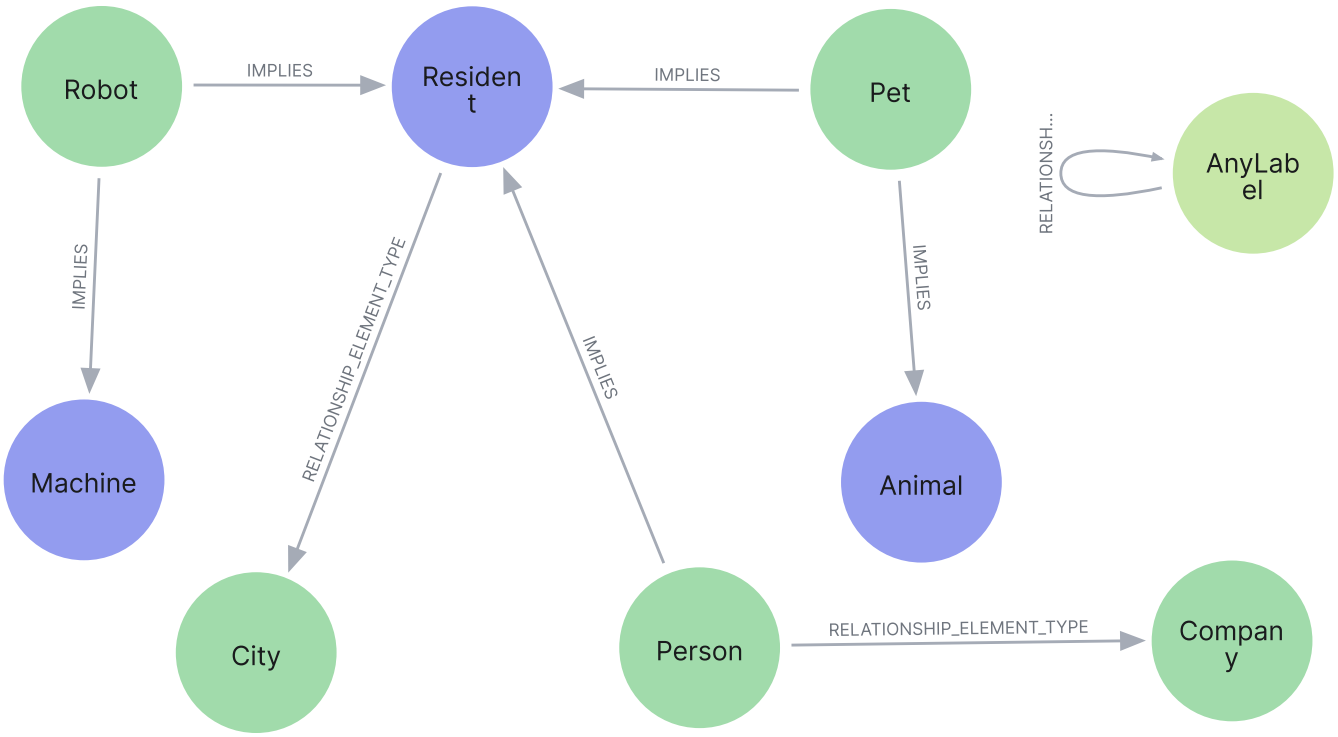
Show full graph type as a virtual graph

Introduced in Neo4j 2026.07

Show the full graph type as a virtual graph

```
SHOW CURRENT GRAPH TYPE AS GRAPH
```

The following graph is returned:



The following table is returned:

Result

```
+-------------------------------------------------------------------------------------------------------
```

```
-------------------------------------------------------------------------------------------------------------
-----------------------------------------+
| nodes
| relationships
|
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-----------------------------------------+
| [(:AnyLabel {}), (:NodeElementType {label: "City", properties: ["name :: STRING NOT NULL",
| [[:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:RELATIONSHIP_ELEMENT_TYPE
{relationshipType: "LIVES_IN",    |
| "population :: INTEGER"], constraints: []}), (:NodeElementType {label: "Company",
| properties: ["since :: ANY NOT NULL"], constraints: []}], [:RELATIONSHIP_ELEMENT_TYPE {relationshipType:
"WORKS_FOR", properties:         |
| properties: ["address :: STRING", "name :: STRING"], constraints: [
| ["role :: STRING"], constraints: []}], [:RELATIONSHIP_TYPE {relationshipType: "STAYED_IN", constraints:
[                                |
| "CONSTRAINT constraint_d7524c38 FOR (n:Company =>) REQUIRE n.address IS UNIQUE",
| "CONSTRAINT stayed_in_endDate FOR ()-[r:STAYED_IN]->() REQUIRE r.endDate IS :: DATE",
|
| "CONSTRAINT company_name FOR (n:Company =>) REQUIRE n.name IS KEY"]}), (:NodeElementType
| "CONSTRAINT stayed_in_startDateExistence FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS NOT NULL",
|
| {label: "Person", properties: ["name :: STRING", "ssn :: INTEGER"], constraints: [
| "CONSTRAINT stayed_in_startDateType FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS :: DATE"]}]]
|
| "CONSTRAINT constraint_bb1018eb FOR (n:Person =>) REQUIRE (n.name, n.ssn) IS KEY"]}), (:NodeElementType
|
|
| {label: "Pet", properties: ["healthCertificate :: STRING", "insuranceNumber :: INTEGER", "name ::
STRING"],      |
|
| constraints: ["CONSTRAINT constraint_302a3693 FOR (n:Pet =>) REQUIRE n.healthCertificate IS UNIQUE",
|
|
| "CONSTRAINT constraint_aa881a72 FOR (n:Pet =>) REQUIRE n.insuranceNumber IS KEY"]}), (:NodeElementType
|
|
| {label: "Robot", properties: ["application :: STRING NOT NULL", "id :: INTEGER NOT NULL"], constraints:
[]}),    |
|
| (:NodeLabel {label: "Animal", constraints: ["CONSTRAINT animal_id FOR (n:Animal) REQUIRE n.id IS
UNIQUE"]}),     |
|
| (:NodeLabel {label: "Machine", constraints: []}), (:NodeLabel {label: "Resident", constraints: [
|
|
| "CONSTRAINT resident_address FOR (n:Resident) REQUIRE n.address IS :: STRING"]})]
|
|
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-----------------------------------------+
```

The `nodes` column contains a list of virtual nodes representing the node element types and node labels defined in the graph type. The `relationships` column contains a list of virtual relationships representing the relationship element types, relationship constraints not belonging to any relationship element type, and label implications in node element types.

Four entities in the result are worth noting:

- `NodeElementType`: a virtual node representing a node element type in the graph type.
- `NodeLabel`: a virtual node representing a node label in the graph type.
- `AnyLabel`: relationship element types with unconstrained endpoints are represented by a virtual any-node with the label `AnyLabel` and no properties. Each relationship gets its own `AnyLabel` node.
- `IMPLIES`: when a node element type implies another label, the two are connected by a relationship of type `IMPLIES` with no properties.

For more information, see virtual nodes and virtual relationships below.

Together, the two lists returned in the `nodes` and `relationships` columns form the following graph:

```
(city:NodeElementType {label: "City", properties: ["name :: STRING NOT NULL", "population :: INTEGER"],
constraints: []}),
(company:NodeElementType {label: "Company", properties: ["address :: STRING", "name :: STRING"],
constraints: ["CONSTRAINT constraint_d7524c38 FOR (n:Company =>) REQUIRE n.address IS UNIQUE", "CONSTRAINT
company_name FOR (n:Company =>) REQUIRE n.name IS KEY"]}),
(person:NodeElementType {label: "Person", properties: ["name :: STRING", "ssn :: INTEGER"], constraints:
["CONSTRAINT constraint_bb1018eb FOR (n:Person =>) REQUIRE (n.name, n.ssn) IS KEY"]}),
(pet:NodeElementType {label: "Pet", properties: ["healthCertificate :: STRING", "insuranceNumber ::
INTEGER", "name :: STRING"], constraints: ["CONSTRAINT constraint_302a3693 FOR (n:Pet =>) REQUIRE
n.healthCertificate IS UNIQUE", "CONSTRAINT constraint_aa881a72 FOR (n:Pet =>) REQUIRE n.insuranceNumber
IS KEY"]}),
(robot:NodeElementType {label: "Robot", properties: ["application :: STRING NOT NULL", "id :: INTEGER NOT
NULL"], constraints: []}),
(animal:NodeLabel {label: "Animal", constraints: ["CONSTRAINT animal_id FOR (n:Animal) REQUIRE n.id IS
UNIQUE"]}),
(machine:NodeLabel {label: "Machine", constraints: []}),
(resident:NodeLabel {label: "Resident", constraints: ["CONSTRAINT resident_address FOR (n:Resident)
REQUIRE n.address IS :: STRING"]}),
(any:AnyLabel {}),
(person)-[:IMPLIES {}]->(resident),
(pet)-[:IMPLIES {}]->(animal),
(pet)-[:IMPLIES {}]->(resident),
(robot)-[:IMPLIES {}]->(machine),
(robot)-[:IMPLIES {}]->(resident),
(resident)-[:RELATIONSHIP_ELEMENT_TYPE {relationshipType: "LIVES_IN", properties: ["since :: ANY NOT
NULL"], constraints: []}]->(city),
(person)-[:RELATIONSHIP_ELEMENT_TYPE {relationshipType: "WORKS_FOR", properties: ["role :: STRING"],
constraints: []}]->(company),
(any)-[:RELATIONSHIP_TYPE {relationshipType: "STAYED_IN", constraints: ["CONSTRAINT stayed_in_endDate FOR
()-[r:STAYED_IN]->() REQUIRE r.endDate IS :: DATE", "CONSTRAINT stayed_in_startDateExistence FOR ()-
[r:STAYED_IN]->() REQUIRE r.startDate IS NOT NULL", "CONSTRAINT stayed_in_startDateType FOR ()-
[r:STAYED_IN]->() REQUIRE r.startDate IS :: DATE"]}]->(any)
```

Combine `SHOW CURRENT GRAPH TYPE` with other Cypher clauses

Introduced in Neo4j 2026.05

The `SHOW CURRENT GRAPH TYPE` command can be combined with general Cypher clauses in a single query.

For example, splitting the graph type specification into separate strings.

Return separate strings for each part of the graph type

```
SHOW CURRENT GRAPH TYPE
YIELD specification
FILTER specification <> '{}'
WITH split(specification, '\r\n') AS specificationLines
WITH specificationLines[1..-1] AS elementLines
WITH [line IN elementLines | rtrim(trim(line), ',')] AS graphTypeLines

UNWIND graphTypeLines AS graphTypeElement
RETURN graphTypeElement
```

Filter out the empty graph type.

Split `STRING` definition into separate lines.

Remove the opening and closing lines of the graph type (`{` and `}`).

Remove leading indention and trailing commas.

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------+
| graphTypeElement
|
+-----------------------------------------------------------------------------------------------------------
-----------+
| "(:`City` => {`name` :: STRING NOT NULL, `population` :: INTEGER})"
|
| "(:`Company` => {`address` :: STRING, `name` :: STRING})"
|
| "(:`Person` => :`Resident` {`name` :: STRING, `ssn` :: INTEGER})"
|
| "(:`Pet` => :`Animal`&`Resident` {`healthCertificate` :: STRING, `insuranceNumber` :: INTEGER, `name` ::
STRING})" |
| "(:`Robot` => :`Machine`&`Resident` {`application` :: STRING NOT NULL, `id` :: INTEGER NOT NULL})"
|
| "(:`Resident`)-[:`LIVES_IN` => {`since` :: ANY NOT NULL}]->(:`City` =>)"
|
| "(:`Person` =>)-[:`WORKS_FOR` => {`role` :: STRING}]->(:`Company` =>)"
|
| "CONSTRAINT `constraint_1324d6fc` FOR (`n`:`Company` =>) REQUIRE (`n`.`address`) IS UNIQUE"
|
| "CONSTRAINT `company_name` FOR (`n`:`Company` =>) REQUIRE (`n`.`name`) IS KEY"
|
| "CONSTRAINT `constraint_5d8979dd` FOR (`n`:`Person` =>) REQUIRE (`n`.`name`, `n`.`ssn`) IS KEY"
|
| "CONSTRAINT `constraint_bbcb9835` FOR (`n`:`Pet` =>) REQUIRE (`n`.`healthCertificate`) IS UNIQUE"
|
| "CONSTRAINT `constraint_550b510e` FOR (`n`:`Pet` =>) REQUIRE (`n`.`insuranceNumber`) IS KEY"
|
| "CONSTRAINT `animal_id` FOR (`n`:`Animal`) REQUIRE (`n`.`id`) IS UNIQUE"
|
| "CONSTRAINT `resident_address` FOR (`n`:`Resident`) REQUIRE (`n`.`address`) IS :: STRING"
|
+-----------------------------------------------------------------------------------------------------------
-----------+
14 rows
```

Note:

When combining `SHOW CURRENT GRAPH TYPE` with other clauses, the `YIELD` clause is mandatory and must not be omitted. `YIELD` must explicitly list the yielded columns. `YIELD *` is not permitted. The query must also end with a valid last clause (like a `RETURN` or an updating clause).

Return columns

The columns returned by `SHOW CURRENT GRAPH TYPE` and `SHOW CURRENT GRAPH TYPE AS GRAPH` are different.

`SHOW CURRENT GRAPH TYPE` columns

There is one return column for `SHOW CURRENT GRAPH TYPE`: `specification`. It is returned by default.

Show current graph type output

Column	Description	Type
`specification`	The graph type of the current database using the canonical form. This can be used in an `ALTER CURRENT GRAPH TYPE SET` command to recreate the same graph type. Default Output	`STRING`

`SHOW CURRENT GRAPH TYPE AS GRAPH` columns

Introduced in Neo4j 2026.07

There are two return columns for `SHOW CURRENT GRAPH TYPE AS GRAPH`: `nodes` and `relationships`. They are returned by default.

Show current graph type as graph output

Column	Description	Type
`nodes`	A list of virtual nodes representing the node element types, separate node labels, node constraints not belonging to any node element type, and nodes used to represent empty endpoints on relationships. Default Output	`LIST<NODE NOT NULL>`
`relationships`	A list of virtual relationships representing the relationship element types, relationship constraints not belonging to any relationship element type, and label implications in node element types. Default Output	`LIST<RELATIONSHIP NOT NULL>`

> The graph returned by `SHOW CURRENT GRAPH TYPE AS GRAPH` is a virtual graph. A virtual graph is a graph projection that exists only for the duration of a query. Virtual nodes and relationships are constructed purely to represent data in a graph-shaped format, are not persisted, and should not be written to. Nodes and relationships in a virtual graph have negative IDs.

Virtual nodes

There are three different virtual nodes used to represent parts of the graph type.

- *Node element type virtual nodes* represent a node element type in the graph type. They contain the identifying label, property specifications, and any key or uniqueness constraints tied to the node element type.
- *Node label virtual nodes* represent node labels that are not identifying a node element type. Either

node constraints that are not tied to any node element type, labels that are implied on node element types, or labels used as endpoints on relationship element types. The constraints are collected per label, meaning there is one node per group of constraints sharing a label, not one per constraint. They contain the label and any constraints belonging to it.

- *Empty relationship endpoint virtual nodes* represent the empty endpoints of relationships. Either an unconstrained endpoint of a relationship element type or the empty end point of a relationship constraint not belonging to a relationship element type. They contain no additional information.

Node element type virtual nodes all have the label `NodeElementType`, with their information stored in their properties. For example, `(:NodeElementType {label: "Person", properties: ["name :: STRING", "ssn :: INTEGER"], constraints: ["CONSTRAINT constraint_5d8979dd FOR (n:Person =>) REQUIRE (n.name, n.ssn) IS KEY"]})`.

Node element type virtual node properties

Property	Description	Type
`label`	The identifying label of the node element type.	`STRING`
`properties`	Any properties defined on the node element type, otherwise an empty list. The format of the strings are the same as given in the graph type definition, for example `"prop :: STRING NOT NULL"`.	`LIST<STRING NOT NULL>`
`constraints`	Any key or uniqueness constraints belonging to the node element type, otherwise an empty list. The constraints are represented by their string descriptions in the graph type definition.	`LIST<STRING NOT NULL>`

Node label virtual nodes all have the label `NodeLabel`, with their information stored in their properties. For example, `(:NodeLabel {label: "Animal", constraints: ["CONSTRAINT animal_id FOR (n:Animal) REQUIRE n.id IS UNIQUE"]})` and `(:NodeLabel {label: "Machine", constraints: []})`.

Node label virtual nodes properties

Property	Description	Type
`label`	The label of the node constraints or being referenced in an element type.	`STRING`
`constraints`	Any constraints belonging to the label, otherwise an empty list. The constraints are represented by their string descriptions in the graph type definition.	`LIST<STRING NOT NULL>`

Empty relationship endpoint virtual nodes have the label `AnyLabel` and no properties, as they store no additional information.

There are three different virtual relationships used to represent parts of the graph type.

- ***Relationship element type virtual relationships*** represent a relationship element type in the graph type. They contain the identifying relationship type, property specifications and any key or uniqueness constraints tied to the relationship element type.
- ***Relationship constraint virtual relationships*** represent relationship constraints that are not tied to any relationship element type. They are collected per relationship type, meaning there is one relationship per group of constraints sharing a relationship type, not one per constraint. They contain the relationship type and constraints belonging to it.
- ***Label implication virtual relationships*** represent a label implication of a node element type. They contain no additional information.

Relationship element type virtual relationships all have the relationship type `RELATIONSHIP_ELEMENT_TYPE`, with their information stored in their properties. For example, `[:RELATIONSHIP_ELEMENT_TYPE {relationshipType: "LIVES_IN", properties: ["since :: ANY NOT NULL"], constraints: []}]`.

Relationship element type virtual relationship properties

Property	Description	Type
`relationshipType`	The identifying relationship type of the relationship element type.	`STRING`
`properties`	Any properties defined on the relationship element type, otherwise an empty list. The format of the strings are the same as given in the graph type definition, for example `"prop :: STRING NOT NULL"`.	`LIST<STRING NOT NULL>`
`constraints`	Any key or uniqueness constraints belonging to the relationship element type, otherwise an empty list. The constraints are represented by their string descriptions in the graph type definition.	`LIST<STRING NOT NULL>`

Relationship constraint virtual relationships all have the relationship type `RELATIONSHIP_TYPE`, with their information stored in their properties. For example, `[:RELATIONSHIP_TYPE {relationshipType: "STAYED_IN", constraints: ["CONSTRAINT stayed_in_endDate FOR ()-[r:STAYED_IN]->() REQUIRE r.endDate IS :: DATE", "CONSTRAINT stayed_in_startDateExistence FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS NOT NULL", "CONSTRAINT stayed_in_startDateType FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS :: DATE"]}]`.

Relationship constraint virtual relationship properties

Property	Description	Type
`relationshipType`	The relationship type of the relationship constraints.	`STRING`
`constraints`	Any constraints belonging to the relationship, otherwise an empty list. The constraints are represented by their string descriptions in the graph type definition.	`LIST<STRING NOT NULL>`

Label implication virtual relationships have relationship type `IMPLIES` and no properties, as they store no additional information.

Constraints in `SHOW CURRENT GRAPH TYPE`

Constraints created using the `CREATE CONSTRAINT` command (i.e. not added with `ALTER CURRENT GRAPH TYPE SET`, `ADD`, or `ALTER`) are automatically added to the graph type. They will therefore appear in the `SHOW CURRENT GRAPH TYPE` result.

Create a node property uniqueness constraint with `CREATE CONSTRAINT` command

```
CREATE CONSTRAINT car_constraint FOR (c:Car) REQUIRE c.licensePlate IS UNIQUE
```

Show `specification` of the full graph type

```
SHOW CURRENT GRAPH TYPE YIELD specification
```

Result

```
+-----------------------------------------------------------------------------------------------------------------
-----------------+
| specification
|
+-----------------------------------------------------------------------------------------------------------------
-----------------+
| "{
|
|    (:`City` => {`name` :: STRING NOT NULL, `population` :: INTEGER}),
|
|    (:`Company` => {`address` :: STRING, `name` :: STRING}),
|
|    (:`Person` => :`Resident` {`name` :: STRING, `ssn` :: INTEGER}),
|
|    (:`Pet` => :`Animal`&`Resident` {`healthCertificate` :: STRING, `insuranceNumber` :: INTEGER, `name`
:: STRING}),     |
|    (:`Robot` => :`Machine`&`Resident` {`application` :: STRING NOT NULL, `id` :: INTEGER NOT NULL}),
|
|    (:`Resident`)-[:`LIVES_IN` => {`since` :: ANY NOT NULL}]->(:`City` =>),
|
|    (:`Person` =>)-[:`WORKS_FOR` => {`role` :: STRING}]->(:`Company` =>),
|
|    CONSTRAINT `constraint_d7524c38` FOR (`n`:`Company` =>) REQUIRE (`n`.`address`) IS UNIQUE,
|
|    CONSTRAINT `company_name` FOR (`n`:`Company` =>) REQUIRE (`n`.`name`) IS KEY,
|
|    CONSTRAINT `constraint_bb1018eb` FOR (`n`:`Person` =>) REQUIRE (`n`.`name`, `n`.`ssn`) IS KEY,
|
|    CONSTRAINT `constraint_302a3693` FOR (`n`:`Pet` =>) REQUIRE (`n`.`healthCertificate`) IS UNIQUE,
|
|    CONSTRAINT `constraint_aa881a72` FOR (`n`:`Pet` =>) REQUIRE (`n`.`insuranceNumber`) IS KEY,
```

```
|
|    CONSTRAINT `animal_id` FOR (`n`:`Animal`) REQUIRE (`n`.`id`) IS UNIQUE,
|
|    CONSTRAINT `car_constraint` FOR (`n`:`Car`) REQUIRE (`n`.`licensePlate`) IS UNIQUE,
|
|    CONSTRAINT `resident_address` FOR (`n`:`Resident`) REQUIRE (`n`.`address`) IS :: STRING,
|
|    CONSTRAINT `stayed_in_endDate` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`endDate`) IS :: DATE,
|
|    CONSTRAINT `stayed_in_startDateExistence` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`startDate`) IS
NOT NULL,         |
|    CONSTRAINT `stayed_in_startDateType` FOR ()-[`r`:`STAYED_IN`]->() REQUIRE (`r`.`startDate`) IS ::
DATE                |
| }"
|
+-----------------------------------------------------------------------------------------------------
-----------------+
```

Show the full graph type as a virtual graph Introduced in Neo4j 2026.06

```
SHOW CURRENT GRAPH TYPE AS GRAPH YIELD nodes, relationships
```

Four entities in the result are worth noting:

- `NodeElementType`: a virtual node representing a node element type in the graph type.
- `NodeLabel`: a virtual node representing a node label in the graph type.
- `AnyLabel`: relationship element types with unconstrained endpoints are represented by a virtual any-node with the label `AnyLabel` and no properties. Each relationship gets its own `AnyLabel` node.
- `IMPLIES`: when a node element type implies another label, the two are connected by a relationship of type `IMPLIES` with no properties.

For more information, see Show graph types → Virtual nodes and Virtual relationships.

Result

```
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-------------------------------------------------+
| nodes
| relationships
|
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-------------------------------------------------+
| [(:AnyLabel {}), (:NodeElementType {label: "City", properties: ["name :: STRING NOT NULL",
| [[:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:IMPLIES {}], [:RELATIONSHIP_ELEMENT_TYPE
{relationshipType: "LIVES_IN",    |
| "population :: INTEGER"], constraints: []}), (:NodeElementType {label: "Company",
| properties: ["since :: ANY NOT NULL"], constraints: []}], [:RELATIONSHIP_ELEMENT_TYPE {relationshipType:
"WORKS_FOR", properties:         |
| properties: ["address :: STRING", "name :: STRING"], constraints: [
| ["role :: STRING"], constraints: []}], [:RELATIONSHIP_TYPE {relationshipType: "STAYED_IN", constraints:
[                                |
| "CONSTRAINT constraint_d7524c38 FOR (n:Company =>) REQUIRE n.address IS UNIQUE",
| "CONSTRAINT stayed_in_endDate FOR ()-[r:STAYED_IN]->() REQUIRE r.endDate IS :: DATE",
|
| "CONSTRAINT company_name FOR (n:Company =>) REQUIRE n.name IS KEY"]}), (:NodeElementType
| "CONSTRAINT stayed_in_startDateExistence FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS NOT NULL",
|
| {label: "Person", properties: ["name :: STRING", "ssn :: INTEGER"], constraints: [
| "CONSTRAINT stayed_in_startDateType FOR ()-[r:STAYED_IN]->() REQUIRE r.startDate IS :: DATE"]}]]
|
| "CONSTRAINT constraint_bb1018eb FOR (n:Person =>) REQUIRE (n.name, n.ssn) IS KEY"]}), (:NodeElementType
|
|
| {label: "Pet", properties: ["healthCertificate :: STRING", "insuranceNumber :: INTEGER", "name ::
```

```
STRING"],            |
|
| constraints: ["CONSTRAINT constraint_302a3693 FOR (n:Pet =>) REQUIRE n.healthCertificate IS UNIQUE",
|
|
| "CONSTRAINT constraint_aa881a72 FOR (n:Pet =>) REQUIRE n.insuranceNumber IS KEY"]}), (:NodeElementType
|
|
| {label: "Robot", properties: ["application :: STRING NOT NULL", "id :: INTEGER NOT NULL"], constraints:
[]}),          |
|
| (:NodeLabel {label: "Animal", constraints: ["CONSTRAINT animal_id FOR (n:Animal) REQUIRE n.id IS
UNIQUE"]}),           |
|
| (:NodeLabel {label: "Car", constraints: ["CONSTRAINT car_constraint FOR (n:Car) REQUIRE n.licensePlate
IS UNIQUE"]}),  |
|
| (:NodeLabel {label: "Machine", constraints: []}), (:NodeLabel {label: "Resident", constraints: [
|
|
| "CONSTRAINT resident_address FOR (n:Resident) REQUIRE n.address IS :: STRING"]})]
|
|
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-------------------------------------------------+
```

Together, these two lists form the following graph:

```
(city:NodeElementType {label: "City", properties: ["name :: STRING NOT NULL", "population :: INTEGER"],
constraints: []}),
(company:NodeElementType {label: "Company", properties: ["address :: STRING", "name :: STRING"],
constraints: ["CONSTRAINT constraint_d7524c38 FOR (n:Company =>) REQUIRE n.address IS UNIQUE", "CONSTRAINT
company_name FOR (n:Company =>) REQUIRE n.name IS KEY"]}),
(person:NodeElementType {label: "Person", properties: ["name :: STRING", "ssn :: INTEGER"], constraints:
["CONSTRAINT constraint_bb1018eb FOR (n:Person =>) REQUIRE (n.name, n.ssn) IS KEY"]}),
(pet:NodeElementType {label: "Pet", properties: ["healthCertificate :: STRING", "insuranceNumber ::
INTEGER", "name :: STRING"], constraints: ["CONSTRAINT constraint_302a3693 FOR (n:Pet =>) REQUIRE
n.healthCertificate IS UNIQUE", "CONSTRAINT constraint_aa881a72 FOR (n:Pet =>) REQUIRE n.insuranceNumber
IS KEY"]}),
(robot:NodeElementType {label: "Robot", properties: ["application :: STRING NOT NULL", "id :: INTEGER NOT
NULL"], constraints: []}),
(animal:NodeLabel {label: "Animal", constraints: ["CONSTRAINT animal_id FOR (n:Animal) REQUIRE n.id IS
UNIQUE"]}),
(car:NodeLabel {label: "Car", constraints: ["CONSTRAINT car_constraint FOR (n:Car) REQUIRE n.licensePlate
IS UNIQUE"]}),
(machine:NodeLabel {label: "Machine", constraints: []}),
(resident:NodeLabel {label: "Resident", constraints: ["CONSTRAINT resident_address FOR (n:Resident)
REQUIRE n.address IS :: STRING"]}),
(any:AnyLabel {}),
(person)-[:IMPLIES {}]->(resident),
(pet)-[:IMPLIES {}]->(animal),
(pet)-[:IMPLIES {}]->(resident),
(robot)-[:IMPLIES {}]->(machine),
(robot)-[:IMPLIES {}]->(resident),
(resident)-[:RELATIONSHIP_ELEMENT_TYPE {relationshipType: "LIVES_IN", properties: ["since :: ANY NOT
NULL"], constraints: []}]->(city),
(person)-[:RELATIONSHIP_ELEMENT_TYPE {relationshipType: "WORKS_FOR", properties: ["role :: STRING"],
constraints: []}]->(company),
(any)-[:RELATIONSHIP_TYPE {relationshipType: "STAYED_IN", constraints: ["CONSTRAINT stayed_in_endDate FOR
()-[r:STAYED_IN]->() REQUIRE r.endDate IS :: DATE", "CONSTRAINT stayed_in_startDateExistence FOR ()-
[r:STAYED_IN]->() REQUIRE r.startDate IS NOT NULL", "CONSTRAINT stayed_in_startDateType FOR ()-
[r:STAYED_IN]->() REQUIRE r.startDate IS :: DATE"]}]->(any)
```

Graph type elements in `SHOW CONSTRAINTS`

Each element in a graph type is implemented with constraints. Whatever is added in a graph type is accordingly returned by the `SHOW CONSTRAINTS` command. For full information about this command, including its return columns, see Show constraints.

Showing constraints requires the `SHOW CONSTRAINTS` privilege.

Of particular interest are the `enforcedLabel` and `classification` columns returned by `SHOW CONSTRAINTS`. `enforcedLabel` returns the implied node labels belonging to a node element type and the source and target node labels belonging to a relationship element type. The `classification` describes the status of a constraint in a graph type, with three possible return values:

Values returned by the `classification` column

`classification`	Applies to
`dependent`	Property existence and property type constraints defined on an identifying node label/relationship type in a node or relationship element type, or the node label existence constraints, relationship target label constraints, or relationship source label constraints belonging to node or relationship element types.
`undesignated`	Key and property uniqueness constraints, regardless of whether defined on an identifying or non-identifying node label or relationship type.
`independent`	Property existence and property type constraints on non-identifying node labels or relationship types.

For more information, see Show constraints → Return columns.

Show constraints

```
SHOW CONSTRAINTS YIELD name, type, labelsOrTypes, enforcedLabel, classification
ORDER BY labelsOrTypes
```

Results

```
+-----------------------------------------------------------------------------------------------------
---+
| name                  | type                              | labelsOrTypes | enforcedLabel |
classification |
+-----------------------------------------------------------------------------------------------------
---+
| "animal_id"           | "NODE_PROPERTY_UNIQUENESS"        | ["Animal"]    | NULL          |
"undesignated" |
| "car_constraint"      | "NODE_PROPERTY_UNIQUENESS"        | ["Car"]       | NULL          |
"undesignated" |
| "constraint_22a8753a" | "NODE_PROPERTY_EXISTENCE"         | ["City"]      | NULL          | "dependent"
|
| "constraint_83085900" | "NODE_PROPERTY_TYPE"              | ["City"]      | NULL          | "dependent"
|
| "constraint_ff70b222" | "NODE_PROPERTY_TYPE"              | ["City"]      | NULL          | "dependent"
|
| "company_name"        | "NODE_KEY"                        | ["Company"]   | NULL          |
"undesignated" |
| "constraint_437f8bc9" | "NODE_PROPERTY_TYPE"              | ["Company"]   | NULL          | "dependent"
|
| "constraint_56444b08" | "NODE_PROPERTY_TYPE"              | ["Company"]   | NULL          | "dependent"
|
| "constraint_d7524c38" | "NODE_PROPERTY_UNIQUENESS"        | ["Company"]   | NULL          |
"undesignated" |
| "constraint_2fd51eaf" | "RELATIONSHIP_TARGET_LABEL"       | ["LIVES_IN"]  | "City"        | "dependent"
```

```
|
| "constraint_9dfe66aa" | "RELATIONSHIP_SOURCE_LABEL"       | ["LIVES_IN"]  | "Resident"    | "dependent"
|
| "constraint_f26c0c3d" | "RELATIONSHIP_PROPERTY_EXISTENCE" | ["LIVES_IN"]  | NULL          | "dependent"
|
| "constraint_77504979" | "NODE_PROPERTY_TYPE"              | ["Person"]    | NULL          | "dependent"
|
| "constraint_82630a6c" | "NODE_LABEL_EXISTENCE"            | ["Person"]    | "Resident"    | "dependent"
|
| "constraint_96f964fb" | "NODE_PROPERTY_TYPE"              | ["Person"]    | NULL          | "dependent"
|
| "constraint_bb1018eb" | "NODE_KEY"                        | ["Person"]    | NULL          |
"undesignated" |
| "constraint_302a3693" | "NODE_PROPERTY_UNIQUENESS"        | ["Pet"]       | NULL          |
"undesignated" |
| "constraint_aa881a72" | "NODE_KEY"                        | ["Pet"]       | NULL          |
"undesignated" |
| "constraint_afe0a8de" | "NODE_LABEL_EXISTENCE"            | ["Pet"]       | "Resident"    | "dependent"
|
| "constraint_c2c72e5b" | "NODE_LABEL_EXISTENCE"            | ["Pet"]       | "Animal"      | "dependent"
|
| "constraint_c79712fe" | "NODE_PROPERTY_TYPE"              | ["Pet"]       | NULL          | "dependent"
|
| "constraint_e6253a9f" | "NODE_PROPERTY_TYPE"              | ["Pet"]       | NULL          | "dependent"
|
| "constraint_f44978e8" | "NODE_PROPERTY_TYPE"              | ["Pet"]       | NULL          | "dependent"
|
| "resident_address"    | "NODE_PROPERTY_TYPE"              | ["Resident"]  | NULL          |
"independent"  |
| "constraint_12d19793" | "NODE_LABEL_EXISTENCE"            | ["Robot"]     | "Machine"     | "dependent"
|
| "constraint_5d3b927"  | "NODE_PROPERTY_TYPE"              | ["Robot"]     | NULL          | "dependent"
|
| "constraint_6e2d3d78" | "NODE_LABEL_EXISTENCE"            | ["Robot"]     | "Resident"    | "dependent"
|
| "constraint_8993edce" | "NODE_PROPERTY_TYPE"              | ["Robot"]     | NULL          | "dependent"
|
| "constraint_94c37fdc" | "NODE_PROPERTY_EXISTENCE"         | ["Robot"]     | NULL          | "dependent"
|
| "constraint_e5820d7e" | "NODE_PROPERTY_EXISTENCE"         | ["Robot"]     | NULL          | "dependent"
|
| "constraint_57239b68" | "RELATIONSHIP_SOURCE_LABEL"       | ["WORKS_FOR"] | "Person"      | "dependent"
|
| "constraint_d7a3733b" | "RELATIONSHIP_TARGET_LABEL"       | ["WORKS_FOR"] | "Company"     | "dependent"
|
| "constraint_df70c64c" | "RELATIONSHIP_PROPERTY_TYPE"      | ["WORKS_FOR"] | NULL          | "dependent"
|
+-----------------------------------------------------------------------------------------------------------
---+
```

All dependent node label existence, relationship source/target label, property existence, and property types constraints have a generated name. This is because they cannot be named when defined in their respective node and relationship element types. The `createStatement` column will also return `null` for these constraints, because they cannot be created through a `CREATE CONSTRAINT` command.

Drop graph type elements

Cypher 25 only | Introduced in Neo4j 2026.02 | Enterprise Edition

The `ALTER CURRENT GRAPH TYPE DROP` command allows you to drop element types and constraints from a graph type.

Dropping element types and constraints from a graph type requires the `DROP CONSTRAINT`

privilege.

Set graph type

```
ALTER CURRENT GRAPH TYPE SET {
   (p:Person => :Resident {name :: STRING, ssn :: INTEGER})
     REQUIRE (p.name, p.ssn) IS KEY,
   (:Pet => :Resident&Animal {insuranceNumber :: INTEGER IS KEY, healthCertificate :: STRING IS UNIQUE,
name :: STRING}),
   (:Robot => :Resident&Machine {application :: STRING NOT NULL, id :: INTEGER NOT NULL}),
   (:City => {name :: STRING NOT NULL, population :: INTEGER}),
   (:Resident)-[:LIVES_IN => {since :: ANY NOT NULL}]->(:City),
   (:Company => {name :: STRING, address :: STRING IS UNIQUE}),
   (:Person)-[:WORKS_FOR => {role :: STRING}]->(:Company),
   CONSTRAINT company_name FOR (c:Company) REQUIRE c.name IS KEY,
   CONSTRAINT animal_id FOR (a:Animal) REQUIRE a.id IS UNIQUE,
   CONSTRAINT resident_address FOR (resident:Resident) REQUIRE resident.address :: STRING
}
```

Drop node and relationship element types

To fully drop a node or relationship element type, it is only necessary to define its identifying node label/relationship type appended with `=>`.

Drop node and relationship element types

```
ALTER CURRENT GRAPH TYPE DROP {
    (:Pet => ),
    ()-[:LIVES_IN => ]->()
}
```

You can also define the full element type when dropping an element type. Partial definitions, however, are not allowed.

Allowed — identifying node label/relationship type only	Allowed — full definition	Not allowed — partial definition

`ALTER CURRENT GRAPH TYPE DROP {`
`(:Pet => )`
`}`
	`ALTER CURRENT GRAPH TYPE DROP {`
	`(:Pet => :Resident&Animal`
	`{insuranceNumber :: INTEGER,`
	`healthCertificate :: STRING, name ::`
	`STRING})`
	`}`
		`ALTER CURRENT GRAPH TYPE DROP {`
		`(:Pet => :Resident&Animal)`
		`}`

Dropping a node or relationship element type in this way can remove the following constraints:

- Node label existence constraints
- Source and target node label constraints
- Property type and property existence constraints defined against a node or relationship element type.

These constraints cannot be removed separately from the node or relationship element type they support.

Not allowed — drop a dependent constraint from a node or relationship element type

```
ALTER CURRENT GRAPH TYPE DROP {
   CONSTRAINT constraint_22a8753a
}
```

`constraint_22a8753a` is a property existence constraint defined on the `City` node element type (constraint names are returned by the `SHOW CONSTRAINTS` command).

Removing a node or relationship element type does not, however, remove any property uniqueness or key constraints defined against a node or relationship element type. Rather, it leaves them as constraints on non-identifying node-labels and relationship types. These must therefore be removed separately, using their constraint name. For more information, see Drop constraints on identifying and non-identifying node labels and relationship types below.

If the identifying label of a node element type is also constrained as a source/target node label in a relationship element type, then the node element type cannot be dropped without also dropping or first altering the relationship element type.

Not allowed — drop a node element type whose identifying label is also the source/target node label in a relationship element type

```
ALTER CURRENT GRAPH TYPE DROP {
    (:Person => )
}
```

Error message

```
22NCE: data exception - node element type in use. The node element type identified by the label `Person`
is referenced in the graph type element '()-[:`WORKS_FOR` =>]->()' and cannot be dropped.
```

Drop constraints on identifying and non-identifying node labels and relationship types

Key and property uniqueness constraints can only be dropped by using the `CONSTRAINT name` syntax in the `ALTER CURRENT GRAPH TYPE DROP` command, regardless if they were defined on an identifying or non-identifying label. Property existence and property type constraints defined on non-identifying node labels and relationship types are also dropped in this way. The names of constraints are returned by the `SHOW CONSTRAINTS` command or as part of their definitions in `SHOW CURRENT GRAPH TYPE`.

Drop explicitly named and generated name key and property uniqueness constraint

```
ALTER CURRENT GRAPH TYPE DROP {
    CONSTRAINT animal_id,
    CONSTRAINT constraint_302a3693
}
```

Note:

`constraint_302a3693` is the name of the uniqueness constraint of the `healthCertificate` established in Set graph types > Property uniqueness and key constraints on identifying node labels and relationship types.

These constraints can also be dropped from the graph type using the `DROP CONSTRAINT` command.

Drop property type constraint on a non-identifying node label

```
DROP CONSTRAINT resident_address
```

Drop a full graph type

To drop a full graph type, use the `ALTER CURRENT GRAPH TYPE SET` command. This will replace the existing graph type with a new (possibly empty) graph type.

Drop full graph type and replace it with an empty graph type

```
ALTER CURRENT GRAPH TYPE SET { }
```

Drop full graph type and replace with a new graph type

```
ALTER CURRENT GRAPH TYPE SET {
    (n:Label => :LabelTwo {property :: STRING})
}
```

Constraints

Neo4j offers several constraints to ensure the quality and integrity of data in a graph. The following constraints are available in Neo4j:

- Property uniqueness constraints: ensure that the combined property values are unique for all nodes with a specific label or all relationships with a specific type.
- Property existence constraints: ensure that a property exists either for all nodes with a specific label or for all relationships with a specific type. Enterprise Edition
- Property type constraints: ensure that a property has the required property type for all nodes with a specific label or for all relationships with a specific type. Enterprise Edition
- Key constraints: ensure that all properties exist and that the combined property values are unique for all nodes with a specific label or all relationships with a specific type. Enterprise Edition

For more information about index-backed constraints, constraint creation failures and data violation scenarios, as well as creating, listing and dropping constraints, see:

- Create constraints
- Show constraints
- Drop constraints

For reference material about the Cypher commands used to manage constraints, see Syntax → Constraints.

> **Note:**
>
> All constraints created using the older `CREATE CONSTRAINT` syntax will automatically be added to the graph type of a database. Not all constraint types can be created using this syntax, however, and maintaining individual constraints can become complicated over time

as their number increases.

It is, therefore, recommended to define a schema using a graph type, which offers both additional, more sophisticated constraint types and a more holistic and simplified approach for constraining and maintaining the shape of the data in a graph.

For more information, see Graph types.

Create constraints

Constraints are created with the `CREATE CONSTRAINT` command. When creating a constraint, it is recommended to provide a constraint name. This name must be unique among both indexes and constraints. If a name is not explicitly given, a unique name will be auto-generated.

Creating a constraint requires the `CREATE CONSTRAINT` privilege.

Adding constraints is an atomic operation that can take a while — all existing data has to be scanned before a Neo4j DBMS can use a constraint.

Create property uniqueness constraints

Property uniqueness constraints ensure that the property values are unique for all nodes with a specific label or all relationships with a specific type. For composite property uniqueness constraints on multiple properties, it is the combination of property values that must be unique. Queries that try to add duplicated property values will fail.

Property uniqueness constraints do not require all nodes or relationships to have values for the properties listed in the constraint. Only nodes or relationships that contain all properties specified in the constraint are subject to the uniqueness rule. Nodes or relationships missing one or more of the specified properties are not subject to this rule.

Create a single property uniqueness constraint

Single property uniqueness constraints are created with the following commands:

- Node property uniqueness constraints: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE n.property IS UNIQUE`.
- Relationship property uniqueness constraints: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE r.property IS UNIQUE`.

For the full command syntax to create a property uniqueness constraint, see Syntax → Create property uniqueness constraints.

Example 442. Create a node property uniqueness constraint on a single property

Create a constraint requiring `Book` nodes to have unique `isbn` properties

```
CREATE CONSTRAINT book_isbn
FOR (book:Book) REQUIRE book.isbn IS UNIQUE
```

Result

```
Added 1 constraint.
```

Example 443. Create a relationship property uniqueness constraint on a single property

Create a constraint requiring `SEQUEL_OF` relationships to have unique `order` properties

```
CREATE CONSTRAINT sequels
FOR ()-[sequel:SEQUEL_OF]-() REQUIRE sequel.order IS UNIQUE
```

Result

```
Added 1 constraint.
```

Create a composite property uniqueness constraint

Constraints created for multiple properties are called composite constraints. Note that the constrained properties must be parenthesized when creating composite property uniqueness constraints.

- Node property uniqueness constraints: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE (n.propertyName_1, …, n.propertyName_n) IS UNIQUE`.
- Relationship property uniqueness constraints: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE (r.propertyName_1, …, r.propertyName_n) IS UNIQUE`.

For the full command syntax to create a property uniqueness constraint, see Syntax → Create property uniqueness constraints.

Example 444. Create a composite node property uniqueness constraint on several properties

Create a constraint requiring `Book` nodes to have unique combinations of `title` and `publicationYear` properties

```
CREATE CONSTRAINT book_title_year
FOR (book:Book) REQUIRE (book.title, book.publicationYear) IS UNIQUE
```

Result

```
Added 1 constraint.
```

Example 445. Create a composite relationship property uniqueness constraint on several properties

Create a constraint requiring `PREQUEL_OF` relationships to have unique combinations of `order` and `author` properties

```
CREATE CONSTRAINT prequels
FOR ()-[prequel:PREQUEL_OF]-() REQUIRE (prequel.order, prequel.author) IS UNIQUE
```

Result

```
Added 1 constraint.
```

Create data that complies with existing property uniqueness constraints

Example 446. Create a node that complies with existing property uniqueness constraints

Create a `Book` node with a unique `isbn` property

```
CREATE (book:Book {isbn: '1449356265', title: 'Graph Databases'})
```

Result

```
Added 1 label, created 1 node, set 2 properties
```

Example 447. Create a relationship that complies with existing property uniqueness constraints

Create a `SEQUEL_OF` relationship with a unique `order` property

```
CREATE (:Book {title: 'Spirit Walker'})-[:SEQUEL_OF {order: 1, seriesTitle: 'Chronicles of Ancient
Darkness'}]->(:Book {title: 'Wolf Brother'})
```

Result

```
Added 2 labels, created 2 nodes, set 4 properties, created 1 relationship.
```

Create property existence constraints

Enterprise Edition

Property existence constraints ensure that a property exists either for all nodes with a specific label or for all relationships with a specific type. Queries that try to create new nodes of the specified label, or relationships of the specified type, without the constrained property will fail. The same is true for queries that try to remove the mandatory property.

Create a single property existence constraint

Property existence constraints on single properties are created with the following commands:

- Node property existence constraint: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE n.property IS NOT NULL`.
- Relationship property existence constraint: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE r.property IS NOT NULL`.

For the full command syntax to create an existence constraint, see Syntax → Create property existence constraints.

It is not possible to create composite existence constraints on several properties.

Example 448. Create a node property existence constraint

Create a constraint requiring `Author` nodes to have a `name` property

```
CREATE CONSTRAINT author_name
FOR (author:Author) REQUIRE author.name IS NOT NULL
```

Result

```
Added 1 constraint.
```

Example 449. Create a relationship property existence constraint

Create a constraint requiring `WROTE` relationships to have a `year` property

```
CREATE CONSTRAINT wrote_year
FOR ()-[wrote:WROTE]-() REQUIRE wrote.year IS NOT NULL
```

Result

```
Added 1 constraint.
```

Create data that complies with existing property existence constraints

Example 450. Create a node that complies with existing node property existence constraints

Create an `Author` node with a `name` property:

```
CREATE (author:Author {name:'Virginia Woolf', surname: 'Woolf'})
```

Result

```
Added 1 label, created 1 node, set 2 properties
```

Example 451. Create a relationship that complies with existing relationship property existence constraints

Create a `WROTE` relationship with a `year` property

```
CREATE (author:Author {name: 'Emily Brontë', surname: 'Brontë'})-[wrote:WROTE {year: 1847, location:
'Haworth, United Kingdom', published: true}]->(book:Book {title:'Wuthering Heights', isbn:
9789186579296})
```

Result

```
Added 2 labels, created 2 nodes, set 7 properties, created 1 relationship
```

Create property type constraints

Enterprise Edition

Property type constraints ensure that a property has the required data type for all nodes with a specific label or for all relationships with a specific type. Queries that attempt to add this property with the wrong data type or modify this property in a way that changes its data type for nodes of the specified label or relationships of the specified type will fail.

Property type constraints do not require all nodes or relationships to have the property. Nodes or relationships without the constrained property are not subject to this rule.

Create a single property type constraint

Property type constraints are created with the following commands:

- Node property type constraints: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE n.property IS :: <TYPE>`.
- Relationship property type constraints: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE r.property IS :: <TYPE>`.

`<TYPE>` refers to a specific Cypher data type, such as `STRING` or `INTEGER`. For the types that properties can be constrained by, see Allowed types, and for information about different data types in Cypher, see Values and types. For the full command syntax to create a property type constraint, see Syntax → Create property type constraints.

It is not possible to create composite property type constraints on several properties.

Example 452. Create a node property type constraint

Create a constraint requiring `title` properties on `Movie` nodes to be of type `STRING`

```
CREATE CONSTRAINT movie_title
FOR (movie:Movie) REQUIRE movie.title IS :: STRING
```

Result

```
Added 1 constraint.
```

Example 453. Create a relationship property type constraint

Create a constraint requiring `order` properties on `PART_OF` relationships to be of type `INTEGER`

```
CREATE CONSTRAINT part_of
FOR ()-[part:PART_OF]-() REQUIRE part.order IS :: INTEGER
```

Result

```
Added 1 constraint.
```

Example 454. Create `VECTOR` property type constraints Cypher 25 only Introduced in Neo4j 2025.10

It is necessary to specify both the dimension and the coordinate type of any constrained `VECTOR` properties. The dimension must be greater than `0` and less or equal to `4096`. For more information, see Values and types → Vectors.

Create a constraint requiring `embedding` properties on `Movie` nodes to be of type `VECTOR<INT32>(42)`

```
CREATE CONSTRAINT node_vector_constraint
FOR (n:Movie) REQUIRE n.embedding IS :: VECTOR<INT32>(42)
```

Result

```
Added 1 constraint.
```

Create a constraint requiring `embedding` properties on `CONTAINS` relationships to be of type `VECTOR<FLOAT32>(1536)`

```
CREATE CONSTRAINT rel_vector_constraint
FOR ()-[r:CONTAINS]->() REQUIRE r.embedding IS :: VECTOR<FLOAT32>(1536)
```

Result

```
Added 1 constraint.
```

Create property type constraints with a union type

A closed dynamic union allows a node or relationship property to maintain some type flexibility whilst preventing unexpected values from being stored.

Example 455. Create a node property type constraint with a union type

Create a constraint requiring `tagline` properties on `Movie` nodes to be either of type `STRING` or `LIST<STRING NOT NULL>`

```
CREATE CONSTRAINT movie_tagline
FOR (movie:Movie) REQUIRE movie.tagline IS :: STRING | LIST<STRING NOT NULL>
```

Result

```
Added 1 constraint.
```

Example 456. Create a relationship property type constraint with a union type

Create a constraint requiring `tags` properties on `PART_OF` relationships to either of type `STRING` or `LIST<STRING NOT NULL>`

```
CREATE CONSTRAINT part_of_tags
FOR ()-[part:PART_OF]-() REQUIRE part.tags IS :: STRING | LIST<STRING NOT NULL>
```

Result

```
Added 1 constraint.
```

Allowed types

The allowed property types for property type constraints are:

- `BOOLEAN`
- `STRING`
- `INTEGER`
- `FLOAT`
- `DATE`
- `LOCAL TIME`
- `ZONED TIME`
- `LOCAL DATETIME`
- `ZONED DATETIME`
- `DURATION`
- `POINT`
- `UUID` Cypher 25 only Introduced in Neo4j 2026.08
- `VECTOR<TYPE>(DIMENSION)` Cypher 25 only Introduced in Neo4j 2025.10
- `LIST<BOOLEAN NOT NULL>`
- `LIST<STRING NOT NULL>`

- `LIST<INTEGER NOT NULL>`
- `LIST<FLOAT NOT NULL>`
- `LIST<DATE NOT NULL>`
- `LIST<LOCAL TIME NOT NULL>`
- `LIST<ZONED TIME NOT NULL>`
- `LIST<LOCAL DATETIME NOT NULL>`
- `LIST<ZONED DATETIME NOT NULL>`
- `LIST<DURATION NOT NULL>`
- `LIST<POINT NOT NULL>`
- Any closed dynamic union of the above types, e.g. `INTEGER | FLOAT | STRING`.

Because storing lists of `VECTOR` values is not supported, property type constraints cannot be created for `LIST<VECTOR<TYPE>(DIMENSION) NOT NULL>`. Additionally, `VECTOR` property type constraints must be created with a specific dimension and coordinate value, where the dimension must be greater than `0` and less than or equal to `4096`. For more information, see Values and types → Vectors.

For a complete reference describing all types available in Cypher, see the section on types and their synonyms.

Creating property type constraints on invalid types will fail

Example 457. Create a node property type constraint with an invalid type

Create a constraint requiring `imdbScore` properties on `Movie` nodes to be of type `MAP`

```
CREATE CONSTRAINT score FOR (movie:Movie) REQUIRE movie.imdbScore IS :: MAP
```

GQLSTATUS error chain

22N90: error: data exception - property type unsupported in constraint. MAP is not supported in property type constraints.

50N11: error: general processing exception - constraint creation failed. Unable to create 'node property type constraint'.

Create data that complies with existing property type constraints

Example 458. Create a node that complies with existing node property type constraint

Create an `Movie` node with a `STRING` `title` property

```
CREATE (movie:Movie {title:'Iron Man'})
```

Result

```
Added 1 label, created 1 node, set 1 properties
```

Example 459. Create a relationship that complies with existing relationship property type constraint

Create a `PART_OF` relationship with an `INTEGER` `order` property

```
MATCH (movie:Movie {title:'Iron Man'})
CREATE (movie)-[part:PART_OF {order: 3}]->(franchise:Franchise {name:'MCU'})
```

Result

```
Added 1 label, added 1 node, created 1 relationship, set 2 properties
```

Create key constraints

Enterprise Edition

Key constraints ensure that the property exist and the property value is unique for all nodes with a specific label or all relationships with a specific type. For composite key constraints on multiple properties, all properties must exists and the combination of property values must be unique.

Queries that try to create new nodes of the specified label, or relationships of the specified type, without the constrained property will fail. The same is true for queries that try to remove the mandatory property or add duplicated property values.

Create a single property key constraint

Single property key constraints are created with the following commands:

- Node key constraints: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE n.property IS NODE KEY`.
- Relationship key constraints: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE r.property IS RELATIONSHIP KEY`.

For the full command syntax to create a key constraint, see Syntax → Create key constraints.

Example 460. Create a node key constraint on a single property

Create a constraint requiring `Director` nodes to have a unique `imdbId` property as a node key.

```
CREATE CONSTRAINT director_imdbId
FOR (director:Director) REQUIRE (director.imdbId) IS NODE KEY
```

Result

```
Added 1 constraint.
```

Example 461. Create a relationship key constraint on a single property

Create a constraint requiring `OWNS` relationships to have a unique `ownershipId` property as a relationship key

```
CREATE CONSTRAINT ownershipId
FOR ()-[owns:OWNS]-() REQUIRE owns.ownershipId IS RELATIONSHIP KEY
```

Result

```
Added 1 constraint.
```

Create a composite key constraint

Constraints created for multiple properties are called composite constraints. Note that the constrained properties must be parenthesized when creating composite key constraints.

Composite key constraints are created with the following commands:

- Node key constraints: `CREATE CONSTRAINT constraint_name FOR (n:Label) REQUIRE (n.propertyName_1, …, n.propertyName_n) IS NODE KEY`.
- Relationship key constraints: `CREATE CONSTRAINT constraint_name FOR ()-[r:REL_TYPE]-() REQUIRE (r.propertyName_1, …, r.propertyName_n) IS RELATIONSHIP KEY`.

For the full command syntax to create a key constraint, see Syntax → Create key constraints.

Example 462. Create a composite node key constraint on multiple properties

Create a constraint requiring `Actor` nodes to have a unique combination of `firstname` and `surname` properties as a node key

```
CREATE CONSTRAINT actor_fullname
FOR (actor:Actor) REQUIRE (actor.firstname, actor.surname) IS NODE KEY
```

Result

```
Added 1 constraint.
```

Example 463. Create a composite relationship key constraint label on multiple properties

Create a constraint requiring `KNOWS` relationships to have a unique combination of `since` and `how` properties as a relationship key

```
CREATE CONSTRAINT knows_since_how
FOR ()-[knows:KNOWS]-() REQUIRE (knows.since, knows.how) IS RELATIONSHIP KEY
```

Result

```
Added 1 constraint.
```

Create data that complies with existing key constraints

Example 464. Create a node that complies with existing node key constraints

Create an `Actor` node with unique `firstname` and `surname` properties

```
CREATE (actor:Actor {firstname: 'Keanu', surname: 'Reeves'})
```

Result

```
Added 1 label, created 1 node, set 2 properties.
```

Example 465. Create a relationship that complies with existing relationship key constraints

Create a `KNOWS` relationship with unique `since` and `how` properties

```
CREATE (:Actor {firstname: 'Jensen', surname: 'Ackles'})-[:KNOWS {since: 2008, how: 'coworkers',
friend: true}]->(:Actor {firstname: 'Misha', surname: 'Collins'})
```

Result

```
Added 2 labels, created 2 nodes, set 7 properties, created 1 relationship.
```

Create a constraint with a parameter

All constraint types can be created with a parameterized name.

Example 466. Create a node property uniqueness constraint using a parameter

Parameters

```
{
  "name": "node_uniqueness_param"
}
```

Create a node property uniqueness constraint with a parameterized name

```
CREATE CONSTRAINT $name
FOR (book:Book) REQUIRE book.prop1 IS UNIQUE
```

Result

```
Added 1 constraint.
```

Example 467. Create a relationship property existence constraint using a parameter

Parameters

```
{
  "name": "rel_exist_param"
```

```
}
```

Create a relationship property existence constraint with a parameterized name

```
CREATE CONSTRAINT $name
FOR ()-[wrote:WROTE]-() REQUIRE wrote.published IS NOT NULL
```

Result

```
Added 1 constraint.
```

Handling multiple constraints

Creating an already existing constraint will fail. This includes the following scenarios:

- Creating a constraint identical to an already existing constraint.
- Creating a constraint with a different name but on the same constraint type and same label/relationship type and property combination as an already existing constraint. For property type constraints the property type also needs to be the same.
- Creating a constraint with the same name as an already existing constraint, regardless of what that constraint is.

Additionally, some constraints cannot coexist and attempting to create them together will therefore fail as well. This includes:

- Property type constraints on the same label/relationship type and property but with different property types. This includes `VECTOR` types with different dimensions or coordinate types.
- Property uniqueness and key constraints on the same label/relationship type and property combination.

However, some constraint types are allowed on the same label/relationship type and property combination. For example, it is possible to have a property uniqueness and a property existence constraint on the same label/relationship type and property combination, though this would be the equivalent of having a node or relationship key constraint. A more useful example would be to combine a property type and a property existence constraint to ensure that the property exists and has the given type.

Handling existing constraints when creating a constraint

To avoid failing on existing constraints, `IF NOT EXISTS` can be added to the `CREATE` command. This will ensure that no error is thrown and that no constraint is created if any other constraint with the given name, or another constraint on the same constraint type and schema, or both, already exists. For property type constraints the property type also needs to be the same. Instead, an informational notification is returned showing the existing constraint which blocks the creation.

Example 468. Create a constraint identical to an existing constraint

Create a constraint requiring all `SEQUEL_OF` *relationships to have unique* `order` *properties*

```
CREATE CONSTRAINT sequels IF NOT EXISTS
FOR ()-[sequel:SEQUEL_OF]-() REQUIRE sequel.order IS UNIQUE
```

Because the same constraint already exists, nothing will happen:

Result

```
(no changes, no records)
```

Notification

```
`CREATE CONSTRAINT sequels IF NOT EXISTS FOR ()-[e:SEQUEL_OF]-() REQUIRE (e.order) IS UNIQUE` has no
effect.
`CONSTRAINT sequels FOR ()-[e:SEQUEL_OF]-() REQUIRE (e.order) IS UNIQUE` already exists.
```

Example 469. Create a relationship property uniqueness constraint when the same constraint with a different name already exists

Create a constraint requiring all `SEQUEL_OF` *relationships to have unique* `order` *properties*

```
CREATE CONSTRAINT new_sequels IF NOT EXISTS
FOR ()-[sequel:SEQUEL_OF]-() REQUIRE sequel.order IS UNIQUE
```

Because a constraint with a different name (`sequels`) on the same schema exists, nothing will happen:

Result

```
(no changes, no records)
```

Notification

```
`CREATE CONSTRAINT new_sequels IF NOT EXISTS FOR ()-[e:SEQUEL_OF]-() REQUIRE (e.order) IS UNIQUE` has
no effect.
`CONSTRAINT sequels FOR ()-[e:SEQUEL_OF]-() REQUIRE (e.order) IS UNIQUE` already exists.
```

Example 470. Create a relationship property uniqueness constraint with the same name as an existing constraint of a different type

Create a constraint requiring all `AUTHORED` *relationships to have unique* `name` *properties*

```
CREATE CONSTRAINT author_name IF NOT EXISTS
FOR ()-[a:AUTHORED]-() REQUIRE a.name IS UNIQUE
```

Because a node property existence constraint named `author_name` already exists, nothing will happen:

Result

```
(no changes, no records)
```

Notification

```
`CREATE CONSTRAINT author_name IF NOT EXISTS FOR ()-[e:AUTHORED]-() REQUIRE (e.name) IS UNIQUE` has
no effect.
`CONSTRAINT author_name FOR (e:Author) REQUIRE (e.name) IS NOT NULL` already exists.
```

Creating an already existing constraint will fail

Creating a constraint with the same name or on the same node label or relationship type and properties that are already constrained by a constraint of the same type will fail. Property uniqueness and key constraints are also not allowed on the same schema.

Example 471. Create a constraint identical to an existing constraint

Create a constraint requiring all `SEQUEL_OF` relationships to have unique `order` properties, given an identical constraint already exists

```
CREATE CONSTRAINT sequels
FOR ()-[sequel:SEQUEL_OF]-() REQUIRE sequel.order IS UNIQUE
```

GQLSTATUS error chain

22N65: error: data exception - equivalent constraint already exists. An equivalent constraint already exists: 'sequels'

Example 472. Create a constraint with a different name but on the same schema as an existing constraint

Create a constraint requiring all `Book` nodes to have unique `isbn` properties, given that a constraint on that schema already exists

```
CREATE CONSTRAINT new_book_isbn
FOR (book:Book) REQUIRE book.isbn IS UNIQUE
```

GQLSTATUS error chain

22N65: error: data exception - equivalent constraint already exists. An equivalent constraint already exists: 'Constraint(id=3, name='book_isbn', type='NODE PROPERTY UNIQUENESS', schema=(:Book {isbn}), ownedIndex=2, graphTypeDependence='UNDESIGNATED')'

Example 473. Creating a constraint with the same name but on a different schema as an existing constraint

Create a constraint requiring all `AUTHORED` relationships to have unique `name` properties, given that a constraint on a different schema with the same name already exists

```
CREATE CONSTRAINT author_name
```

```
FOR ()-[a:AUTHORED]-() REQUIRE a.name IS UNIQUE
```

GQLSTATUS error chain

22N67: error: data exception - duplicated constraint name. A constraint with the same name already exists: `author_name`

Example 474. Creating a property type constraint on a property when a property type constraint constraining the property to a different type already exist

Create a constraint requiring `order` properties on `PART_OF` relationships to be of type `FLOAT`, given a constraint requiring the same properties to be of type `INTEGER` already exists

```
CREATE CONSTRAINT new_part_of
FOR ()-[part:PART_OF]-() REQUIRE part.order IS :: FLOAT
```

GQLSTATUS error chain

22N66: error: data exception - conflicting constraint already exists. A conflicting constraint already exists: 'Constraint(id=13, name='part_of', type='RELATIONSHIP PROPERTY TYPE', schema=()-[:PART_OF {order}]-(), graphTypeDependence='INDEPENDENT', propertyType=INTEGER)'

Example 475. Creating a node key constraint on the same schema as an existing property uniqueness constraint

Create a node key constraint on the properties `title` and `publicationYear` on nodes with the `Book` label, when a property uniqueness constraint already exists on the same label and property combination

```
CREATE CONSTRAINT book_titles FOR (book:Book) REQUIRE (book.title, book.publicationYear) IS NODE KEY
```

GQLSTATUS error chain

22N66: error: data exception - conflicting constraint already exists. A conflicting constraint already exists: 'Constraint(id=7, name='book_title_year', type='NODE PROPERTY UNIQUENESS', schema=(:Book {title, publicationYear}), ownedIndex=6, graphTypeDependence='UNDESIGNATED')'

Constraints and indexes

Constraints and backing indexes

Property uniqueness constraints and key constraints are backed by range indexes. This means that creating a property uniqueness or key constraint will create a range index with the same name, node label/relationship type and property combination as its owning constraint. Single property constraints will create single property indexes and multiple property composite constraints will create composite indexes.

Indexes of the same index type, label/relationship type, and property combination cannot be

added separately. However, dropping a property uniqueness or key constraint will also drop its backing index. If the backing index is still required, the index needs to be explicitly re-created.

Property uniqueness and key constraints require an index because it allows the system to quickly check if a node with the same label and property value or a relationship with the same type and property value already exists. Without an index, the system would need to scan all nodes with the same label, which would be slow and inefficient, especially as the graph grows. The index makes these checks much faster by enabling direct lookups instead of scanning the entire graph. Cypher will use the indexes with an owning constraint in the same way that it utilizes other performance indexes. For more information about how indexes impact query performance, see The impact of indexes on query performance.

These indexes are listed in the `owningConstraint` column returned by the `SHOW INDEX` command, and the `ownedIndex` column returned by the `SHOW CONSTRAINT` command.

Example 476. Show constraints with backing indexes

Query

```
SHOW CONSTRAINTS WHERE ownedIndex IS NOT NULL
```

Result

```
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------+
| id | name                    | type                                 | entityType     | labelsOrTypes | properties                    | ownedIndex              | propertyType |
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------+
| 21 | "actor_fullname"        | "NODE_KEY"                           | "NODE"         | ["Actor"]     | ["firstname", "surname"]      | "actor_fullname"        | NULL         |
| 3  | "book_isbn"             | "NODE_PROPERTY_UNIQUENESS"           | "NODE"         | ["Book"]      | ["isbn"]                      | "book_isbn"             | NULL         |
| 7  | "book_title_year"       | "NODE_PROPERTY_UNIQUENESS"           | "NODE"         | ["Book"]      | ["title", "publicationYear"]  | "book_title_year"       | NULL         |
| 17 | "director_imdbId"       | "NODE_KEY"                           | "NODE"         | ["Director"]  | ["imdbId"]                    | "director_imdbId"       | NULL         |
| 23 | "knows_since_how"       | "RELATIONSHIP_KEY"                   | "RELATIONSHIP" | ["KNOWS"]     | ["since", "how"]              | "knows_since_how"       | NULL         |
| 25 | "node_uniqueness_param" | "NODE_PROPERTY_UNIQUENESS"           | "NODE"         | ["Book"]      | ["prop1"]                     | "node_uniqueness_param" | NULL         |
| 19 | "ownershipId"           | "RELATIONSHIP_KEY"                   | "RELATIONSHIP" | ["OWNS"]      | ["ownershipId"]               | "ownershipId"           | NULL         |
| 9  | "prequels"              | "RELATIONSHIP_PROPERTY_UNIQUENESS"   | "RELATIONSHIP" | ["PREQUEL_OF"]| ["order", "author"]           | "prequels"              | NULL         |
| 5  | "sequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS"   | "RELATIONSHIP" | ["SEQUEL_OF"] | ["order"]                     | "sequels"               | NULL         |
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------+
```

Example 477. Show indexes with owning constraints

Query

```
SHOW INDEXES WHERE owningConstraint IS NOT NULL
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
---------------+
| id | name                    | state    | populationPercent | type    | entityType     |
labelsOrTypes  | properties                  | indexProvider | owningConstraint        | lastRead
| readCount |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
---------------+
| 20 | "actor_fullname"        | "ONLINE" | 100.0             | "RANGE" | "NODE"         | ["Actor"]
| ["firstname", "surname"]     | "range-1.0"   | "actor_fullname"        | 2024-10-07T12:12:51.893Z |
3         |
| 2  | "book_isbn"             | "ONLINE" | 100.0             | "RANGE" | "NODE"         | ["Book"]
| ["isbn"]                     | "range-1.0"   | "book_isbn"             | 2024-10-07T11:58:09.252Z |
2         |
| 6  | "book_title_year"       | "ONLINE" | 100.0             | "RANGE" | "NODE"         | ["Book"]
| ["title", "publicationYear"] | "range-1.0"   | "book_title_year"       | NULL                     |
0         |
| 16 | "director_imdbId"       | "ONLINE" | 100.0             | "RANGE" | "NODE"         |
["Director"]   | ["imdbId"]                  | "range-1.0"   | "director_imdbId"       | NULL
| 0         |
| 22 | "knows_since_how"       | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" | ["KNOWS"]
| ["since", "how"]             | "range-1.0"   | "knows_since_how"       | 2024-10-07T12:12:51.894Z |
1         |
| 24 | "node_uniqueness_param" | "ONLINE" | 100.0             | "RANGE" | "NODE"         | ["Book"]
| ["prop1"]                    | "range-1.0"   | "node_uniqueness_param" | NULL                     |
0         |
| 18 | "ownershipId"           | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" | ["OWNS"]
| ["ownershipId"]              | "range-1.0"   | "ownershipId"           | NULL                     |
0         |
| 8  | "prequels"              | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" |
["PREQUEL_OF"] | ["order", "author"]          | "range-1.0"   | "prequels"              | NULL
| 0         |
| 4  | "sequels"               | "ONLINE" | 100.0             | "RANGE" | "RELATIONSHIP" |
["SEQUEL_OF"]  | ["order"]                   | "range-1.0"   | "sequels"               | 2024-10-
07T11:57:12.999Z | 1         |
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------
---------------+
```

Property existence and property type constraints are not backed by indexes.

Constraint failures and indexes

Attempting to create any type of constraint with the same name as an existing index will fail.

Example 478. Creating a node property type constraint with the same name as an existing index

Create an index with the name `directors`

```
CREATE INDEX directors FOR (director:Director) ON (director.name)
```

Create a node property type constraint with the name `directors`

```
CREATE CONSTRAINT directors FOR (movie:Movie) REQUIRE movie.director IS :: STRING
```

GQLSTATUS error chain

22N71: error: data exception - duplicated index name. An index with the same name already exists: `directors`

Creating key or property uniqueness constraints on the same schema as an existing index will fail.

Example 479. Creating a node property uniqueness constraint on the same schema as an existing index

Create an index for `wordCount` properties on `Book` nodes

```
CREATE INDEX book_word_count FOR (book:Book) ON (book.wordCount)
```

Create a constraint requiring all `Book` nodes to have unique `wordCount` properties

```
CREATE CONSTRAINT word_count FOR (book:Book) REQUIRE book.wordCount IS UNIQUE
```

GQLSTATUS error chain

```
22N73: error: data exception - constraint conflicts with existing index. Constraint conflicts with already existing index '(:Book {wordCount})'.
```

Constraints and data violation scenarios

Creating data that violates existing constraints will fail

Existing constraints preventing data creation

Constraint type	Create nodes and relationships without an existence constrained property	Create nodes and relationships with non-unique properties/property combinations	Create nodes and relationships with the wrong property type
Property uniqueness constraint		✓	
Property existence constraint	✓		
Property type constraint			✓
Key constraint	✓	✓	

Example 480. Create a node that violates a node property uniqueness constraint

Create a `Book` node with an `isbn` property that already exists

```
CREATE (book:Book {isbn: '1449356265', title: 'Graph Databases'})
```

GQLSTATUS error chain

22N80: error: data exception - index entry conflict. Index entry conflict: Node(0) already exists with label `Book` and property `isbn` = '1449356265'.

22N79: error: data exception - property uniqueness constraint violated. Property uniqueness constraint violated: Node(0) already exists with label `Book` and property `isbn` = '1449356265'.

Example 481. Create a node that violates an existing node property existence constraint

Create an `Author` node without a `name` property, given a property existence constraint on `:Author(name)`

```
CREATE (author:Author {surname: 'Austen'})
```

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. NODE (11) with label 'Author' must have the following properties: `name`.

Example 482. Create a relationship that violates an existing relationship property type constraint

Create a `PART_OF` relationship with a `STRING` `order` property, given a property type constraint on the relationship type `PART_OF` restricting the `order` property to `INTEGER` values

```
MATCH (movie:Movie {title:'Iron Man'}), (franchise:Franchise {name:'MCU'})
CREATE (movie)-[part:PART_OF {order: '1'}]->(franchise)
```

GQLSTATUS error chain

22N78: error: data exception - property type verification failed. RELATIONSHIP (1152924803141730310) with type 'PART_OF' must have the property `order` with value type INTEGER.

Example 483. Create a node that violates an existing node key constraint

Create an `Actor` node without a `firstname` property, given a node key constraint on `:Actor(firstname, surname)`

```
CREATE (actor:Actor {surname: 'Wood'})
```

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. NODE (11) with label 'Actor' must have the following properties: `firstname`, `surname`.

Removing existence and key constrained properties will fail

Example 484. Remove a node property existence constrained property

Remove the `name` property from an existing `Author` node, given a property existence constraint on `:Author(name)`

```
MATCH (author:Author {name: 'Virginia Woolf'})
REMOVE author.name
```

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. NODE (3) with label 'Author' must have the following properties: `name`.

Example 485. Remove a node key constrained property

Remove the `firstname` property from an existing node `Actor`, given a node key constraint on `:Actor(firstname, surname)`

```
MATCH (actor:Actor {firstname: 'Keanu', surname: 'Reeves'})
REMOVE actor.firstname
```

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. NODE (8) with label 'Actor' must have the following properties: `firstname`, `surname`.

Modifying type constrained properties will fail

Example 486. Modify a type constrained property

Modify the `title` for the `Movie` 'Iron Man' to an `INTEGER` value, given a constraint requiring `title` properties to be of type `STRING`

```
MATCH (m:Movie {title: 'Iron Man'})
SET m.title = 13
```

GQLSTATUS error chain

22N78: error: data exception - property type verification failed. NODE (6) with label 'Movie' must have the property `title` with value type STRING.

Creating constraints when there exists conflicting data will fail

Existing data preventing constraint creation

Constraint type	Non-existing property	Non-unique property/property combination	Property of wrong type
Property uniqueness constraint		✓	
Property existence constraint	✓		
Property type constraint			✓
Key constraint	✓	✓	

Example 487. Create a node property uniqueness constraint when conflicting nodes exist

Create two `Book` nodes with the same `name` property value

```
CREATE (:Book {isbn: '9780393972832', title: 'Moby Dick'}),
       (:Book {isbn: '9780763630188', title: 'Moby Dick'})
```

Create a constraint requiring `Book` nodes to have unique `title` properties, when there already exists two `Book` nodes with the same `title`

```
CREATE CONSTRAINT book_title FOR (book:Book) REQUIRE book.title IS UNIQUE
```

In this case, the constraint cannot be created because it is in conflict with the existing graph. Either use indexes instead, or remove/correct the offending nodes and then re-apply the constraint.

GQLSTATUS error chain

22N80: error: data exception - index entry conflict. Index entry conflict: Both Node(11) and Node(12) have the label `Book` and property `title` = 'Moby Dick'.

22N79: error: data exception - property uniqueness constraint violated. Property uniqueness constraint violated: Both Node(11) and Node(12) have the label `Book` and property `title` = 'Moby Dick'.

50N11: error: general processing exception - constraint creation failed. Unable to create 'book_title'.

The constraint creation fails on the first offending nodes that are found. This does not guarantee that there are no other offending nodes in the graph. Therefore, all the data should be checked and cleaned up before re-attempting the constraint creation.

Find all offending nodes with the non-unique property values for the constraint above

```
MATCH (book1:Book), (book2:Book)
WHERE book1.title = book2.title AND NOT book1 = book2
RETURN book1, book2
```

Example 488. Create a relationship property existence constraint when conflicting relationships exist

Create a constraint requiring all `WROTE` relationships to have a `language` property, when there already exists a `WROTE` relationship without a `language` property

```
CREATE CONSTRAINT wrote_language FOR ()-[wrote:WROTE]-() REQUIRE wrote.language IS NOT NULL
```

In this case, the constraint cannot be created because it is in conflict with the existing graph. Remove or correct the offending relationships and then re-apply the constraint.

GQLSTATUS error chain

22N77: error: data exception - property presence verification failed. RELATIONSHIP (1152923703630102532) with type 'WROTE' must have the following properties: `language`.

50N11: error: general processing exception - constraint creation failed. Unable to create 'Constraint(type='RELATIONSHIP PROPERTY EXISTENCE', schema=()-[:WROTE {language}]-(), graphTypeDependence='INDEPENDENT')'.

The constraint creation fails on the first offending relationship that is found. This does not guarantee that there are no other offending relationships in the graph. Therefore, all the data should be checked and cleaned up before re-attempting the constraint creation.

Find all offending relationships missing the property for the constraint above

```
MATCH ()-[wrote:WROTE]-()
WHERE wrote.language IS NULL
RETURN wrote
```

Generic `MATCH` queries to find the properties preventing the creation of particular constraints:

Constraint	Query

Node property uniqueness constraint	`MATCH (n1:Label), (n2:Label)`
	`WHERE n1.prop = n2.prop AND NOT n1 = n2`
	`RETURN n1, n2`

Relationship property uniqueness constraint	`MATCH ()-[r1:REL_TYPE]->(), ()-[r2:REL_TYPE]->()`
	`WHERE r1.prop = r2.prop AND NOT r1 = r2`
	`RETURN r1, r2`

Node property existence constraint	`MATCH (n:Label)`
	`WHERE n.prop IS NULL`
	`RETURN n`

Relationship property existence constraint	`MATCH ()-[r:REL_TYPE]->()`
	`WHERE r.prop IS NULL`
	`RETURN r`

Node property type constraint	`MATCH (n:Label)`
	`WHERE n.prop IS NOT :: <TYPE>`
	`RETURN n`

Constraint	Query

Relationship property type constraint	`MATCH ()-[r:REL_TYPE]->()`
	`WHERE r.prop IS NOT :: <TYPE>`
	`RETURN r`

Node key constraint	`MATCH (n1:Label), (n2:Label)`
	`WHERE n1.prop = n2.prop AND NOT n1 = n2`
	`UNWIND [n1, n2] AS node`
	`RETURN node, 'non-unique' AS reason`
	`UNION`
	`MATCH (n:Label)`
	`WHERE n.prop IS NULL`
	`RETURN n AS node, 'non-existing' AS reason`

Relationship key constraint	`MATCH ()-[r1:REL_TYPE]->(), ()-[r2:REL_TYPE]->()`
	`WHERE r1.prop = r2.prop AND NOT r1 = r2`
	`UNWIND [r1, r2] AS relationship`
	`RETURN relationship, 'non-unique' AS reason`
	`UNION`
	`MATCH ()-[r:REL_TYPE]->()`
	`WHERE r.prop IS NULL`
	`RETURN r AS relationship, 'non-existing' AS reason`

Show constraints

To show all constraints with the default output columns, use `SHOW CONSTRAINTS`. If all columns are required, use `SHOW CONSTRAINTS YIELD *`. For the full command syntax to list constraints, see Syntax → Show constraints. For general information about the `SHOW` command, see the `SHOW` page. The constraints shown on this page are created on the page Create constraints.

> **Note:**
>
> All constraints created when setting a graph type will be returned by the `SHOW CONSTRAINTS` command. Similarly, all constraints created using the `CREATE CONSTRAINT` syntax will be returned by the `SHOW CURRENT GRAPH TYPE` command. For more information, see the following sections of the Show graph types page:
>
> - Show graph types → Constraints in `SHOW CURRENT GRAPH TYPE`
> - Show graph types → Graph type elements in `SHOW CONSTRAINTS`

One of the output columns from `SHOW CONSTRAINTS` is the name of the constraint. This can be used to drop the constraint with the `DROP CONSTRAINT` command.

> Showing constraints requires the `SHOW CONSTRAINTS` privilege.

Examples

- Show all constraints
- Return specific columns
- Show constraints with filtering
- Combine `SHOW CONSTRAINTS` with other Cypher clauses

Show all constraints

To show all constraints with the default output columns, use the `SHOW CONSTRAINTS`command. If all columns are required, use `SHOW CONSTRAINTS YIELD *`.

Show all constraints and return default columns

```
SHOW CONSTRAINTS
```

Result

```
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------+
| id | name                     | type                               | entityType     | labelsOrTypes  |
properties                  | enforcedLabel | ownedIndex             | propertyType
|
+-----------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------+
| 28 | "actor_fullname"         | "NODE_KEY"                         | "NODE"         | ["Actor"]      |
["firstname", "surname"]    | NULL          | "actor_fullname"       | NULL
|
| 27 | "author_name"            | "NODE_PROPERTY_EXISTENCE"          | "NODE"         | ["Author"]     |
["name"]                    | NULL          | NULL                   | NULL
|
| 34 | "book_isbn"              | "NODE_PROPERTY_UNIQUENESS"         | "NODE"         | ["Book"]       |
["isbn"]                    | NULL          | "book_isbn"            | NULL
|
| 23 | "book_title_year"        | "NODE_PROPERTY_UNIQUENESS"         | "NODE"         | ["Book"]       |
["title", "publicationYear"] | NULL          | "book_title_year"      | NULL
|
| 36 | "director_imdbId"        | "NODE_KEY"                         | "NODE"         | ["Director"]   |
["imdbId"]                  | NULL          | "director_imdbId"      | NULL
|
| 26 | "knows_since_how"        | "RELATIONSHIP_KEY"                 | "RELATIONSHIP" | ["KNOWS"]      |
["since", "how"]            | NULL          | "knows_since_how"      | NULL
|
| 12 | "movie_tagline"          | "NODE_PROPERTY_TYPE"               | "NODE"         | ["Movie"]      |
["tagline"]                 | NULL          | NULL                   | "STRING | LIST<STRING NOT NULL>"
|
| 33 | "movie_title"            | "NODE_PROPERTY_TYPE"               | "NODE"         | ["Movie"]      |
["title"]                   | NULL          | NULL                   | "STRING"
|
| 17 | "node_uniqueness_param"  | "NODE_PROPERTY_UNIQUENESS"         | "NODE"         | ["Book"]       |
["prop1"]                   | NULL          | "node_uniqueness_param" | NULL
|
| 2  | "node_vector_constraint" | "NODE_PROPERTY_TYPE"               | "NODE"         | ["Movie"]      |
["embedding"]               | NULL          | NULL                   | "VECTOR<INTEGER32 NOT NULL>(42)"
|
| 38 | "ownershipId"            | "RELATIONSHIP_KEY"                 | "RELATIONSHIP" | ["OWNS"]       |
["ownershipId"]             | NULL          | "ownershipId"          | NULL
|
| 41 | "part_of"                | "RELATIONSHIP_PROPERTY_TYPE"       | "RELATIONSHIP" | ["PART_OF"]    |
["order"]                   | NULL          | NULL                   | "INTEGER"
|
| 39 | "part_of_tags"           | "RELATIONSHIP_PROPERTY_TYPE"       | "RELATIONSHIP" | ["PART_OF"]    |
["tags"]                    | NULL          | NULL                   | "STRING | LIST<STRING NOT NULL>"
|
| 24 | "prequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["PREQUEL_OF"] |
["order", "author"]         | NULL          | "prequels"             | NULL
|
| 30 | "rel_exist_param"        | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
["published"]               | NULL          | NULL                   | NULL
|
| 3  | "rel_vector_constraint"  | "RELATIONSHIP_PROPERTY_TYPE"       | "RELATIONSHIP" | ["CONTAINS"]   |
["embedding"]               | NULL          | NULL                   | "VECTOR<FLOAT32 NOT NULL>(1536)"
|
| 19 | "sequels"                | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["SEQUEL_OF"]  |
["order"]                   | NULL          | "sequels"              | NULL
|
| 14 | "wrote_year"             | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
["year"]                    | NULL          | NULL                   | NULL
|
+-----------------------------------------------------------------------------------------------------------
```

```
-----------------------------------------------------------------------------------------------------+
```

Show all constraints and return all columns

```
SHOW CONSTRAINTS YIELD *
```

Result

```
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-------------------------------------------------------------+
| id | name                     | type                              | entityType     | labelsOrTypes  |
properties                  | enforcedLabel | classification | ownedIndex              | propertyType
| options           | createStatement
|
+-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
-------------------------------------------------------------+
| 28 | "actor_fullname"         | "NODE_KEY"                        | "NODE"         | ["Actor"]      |
["firstname", "surname"]     | NULL          | "undesignated" | "actor_fullname"        | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `actor_fullname` FOR (n:`Actor`) REQUIRE (n.`firstname`,
n.`surname`) IS KEY"                                  |
| 27 | "author_name"            | "NODE_PROPERTY_EXISTENCE"         | "NODE"         | ["Author"]     |
["name"]                     | NULL          | "independent"  | NULL                    | NULL
| NULL              | "CREATE CONSTRAINT `author_name` FOR (n:`Author`) REQUIRE (n.`name`) IS NOT NULL"
|
| 34 | "book_isbn"              | "NODE_PROPERTY_UNIQUENESS"        | "NODE"         | ["Book"]       |
["isbn"]                     | NULL          | "undesignated" | "book_isbn"             | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `book_isbn` FOR (n:`Book`) REQUIRE (n.`isbn`) IS UNIQUE"
|
| 23 | "book_title_year"        | "NODE_PROPERTY_UNIQUENESS"        | "NODE"         | ["Book"]       |
["title", "publicationYear"] | NULL          | "undesignated" | "book_title_year"       | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `book_title_year` FOR (n:`Book`) REQUIRE (n.`title`,
n.`publicationYear`) IS UNIQUE"                               |
| 36 | "director_imdbId"        | "NODE_KEY"                        | "NODE"         | ["Director"]   |
["imdbId"]                   | NULL          | "undesignated" | "director_imdbId"       | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `director_imdbId` FOR (n:`Director`) REQUIRE (n.`imdbId`) IS KEY"
|
| 26 | "knows_since_how"        | "RELATIONSHIP_KEY"                | "RELATIONSHIP" | ["KNOWS"]      |
["since", "how"]             | NULL          | "undesignated" | "knows_since_how"       | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `knows_since_how` FOR ()-[r:`KNOWS`]-() REQUIRE (r.`since`,
r.`how`) IS KEY"                                      |
| 12 | "movie_tagline"          | "NODE_PROPERTY_TYPE"              | "NODE"         | ["Movie"]      |
["tagline"]                  | NULL          | "independent"  | NULL                    | "STRING |
LIST<STRING NOT NULL>" | NULL              | "CREATE CONSTRAINT `movie_tagline` FOR (n:`Movie`) REQUIRE
(n.`tagline`) IS :: STRING | LIST<STRING NOT NULL>"                    |
| 33 | "movie_title"            | "NODE_PROPERTY_TYPE"              | "NODE"         | ["Movie"]      |
["title"]                    | NULL          | "independent"  | NULL                    | "STRING"
| NULL              | "CREATE CONSTRAINT `movie_title` FOR (n:`Movie`) REQUIRE (n.`title`) IS :: STRING"
|
| 17 | "node_uniqueness_param"  | "NODE_PROPERTY_UNIQUENESS"        | "NODE"         | ["Book"]       |
["prop1"]                    | NULL          | "undesignated" | "node_uniqueness_param" | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `node_uniqueness_param` FOR (n:`Book`) REQUIRE (n.`prop1`) IS
UNIQUE"                                               |
| 2  | "node_vector_constraint" | "NODE_PROPERTY_TYPE"              | "NODE"         | ["Movie"]      |
["embedding"]                | NULL          | "independent"  | NULL                    |
"VECTOR<INTEGER32 NOT NULL>(42)" | NULL              | "CREATE CONSTRAINT `node_vector_constraint` FOR
(n:`Movie`) REQUIRE (n.`embedding`) IS :: VECTOR<INTEGER32 NOT NULL>(42)"         |
| 38 | "ownershipId"            | "RELATIONSHIP_KEY"                | "RELATIONSHIP" | ["OWNS"]       |
["ownershipId"]              | NULL          | "undesignated" | "ownershipId"           | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `ownershipId` FOR ()-[r:`OWNS`]-() REQUIRE (r.`ownershipId`) IS
KEY"                                                  |
| 41 | "part_of"                | "RELATIONSHIP_PROPERTY_TYPE"      | "RELATIONSHIP" | ["PART_OF"]    |
["order"]                    | NULL          | "independent"  | NULL                    | "INTEGER"
| NULL              | "CREATE CONSTRAINT `part_of` FOR ()-[r:`PART_OF`]-() REQUIRE (r.`order`) IS ::
INTEGER"                                              |
| 39 | "part_of_tags"           | "RELATIONSHIP_PROPERTY_TYPE"      | "RELATIONSHIP" | ["PART_OF"]    |
["tags"]                     | NULL          | "independent"  | NULL                    | "STRING |
LIST<STRING NOT NULL>" | NULL              | "CREATE CONSTRAINT `part_of_tags` FOR ()-[r:`PART_OF`]-()
REQUIRE (r.`tags`) IS :: STRING | LIST<STRING NOT NULL>"                |
| 24 | "prequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["PREQUEL_OF"] |
["order", "author"]          | NULL          | "undesignated" | "prequels"              | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `prequels` FOR ()-[r:`PREQUEL_OF`]-() REQUIRE (r.`order`,
```

```
r.`author`) IS UNIQUE"                                    |
| 30 | "rel_exist_param"        | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
["published"]                | NULL          | "independent"  | NULL                       | NULL
| NULL              | "CREATE CONSTRAINT `rel_exist_param` FOR ()-[r:`WROTE`]-() REQUIRE (r.`published`)
IS NOT NULL"                                      |
| 3  | "rel_vector_constraint"  | "RELATIONSHIP_PROPERTY_TYPE"       | "RELATIONSHIP" | ["CONTAINS"]   |
["embedding"]                | NULL          | "independent"  | NULL                       | "VECTOR<FLOAT32
NOT NULL>(1536)" | NULL              | "CREATE CONSTRAINT `rel_vector_constraint` FOR ()-[r:`CONTAINS`]-()
REQUIRE (r.`embedding`) IS :: VECTOR<FLOAT32 NOT NULL>(1536)" |
| 19 | "sequels"                | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["SEQUEL_OF"]  |
["order"]                    | NULL          | "undesignated" | "sequels"                  | NULL
| {indexConfig: {}} | "CREATE CONSTRAINT `sequels` FOR ()-[r:`SEQUEL_OF`]-() REQUIRE (r.`order`) IS
UNIQUE"                                           |
| 14 | "wrote_year"             | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
["year"]                     | NULL          | "independent"  | NULL                       | NULL
| NULL              | "CREATE CONSTRAINT `wrote_year` FOR ()-[r:`WROTE`]-() REQUIRE (r.`year`) IS NOT
NULL"                                             |
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------+
```

Return specific columns

It is possible to return only specific columns of the available constraints using the `YIELD` clause:

Show only the `name`, `type`, and `createStatement` columns

```
SHOW CONSTRAINTS
YIELD name, type, createStatement
```

Result

```
+-------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------+
| name                     | type                              | createStatement
|
+-------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------+
| "actor_fullname"         | "NODE_KEY"                        | "CREATE CONSTRAINT `actor_fullname` FOR
(n:`Actor`) REQUIRE (n.`firstname`, n.`surname`) IS KEY"                                    |
| "author_name"            | "NODE_PROPERTY_EXISTENCE"         | "CREATE CONSTRAINT `author_name` FOR
(n:`Author`) REQUIRE (n.`name`) IS NOT NULL"                                                  |
| "book_isbn"              | "NODE_PROPERTY_UNIQUENESS"        | "CREATE CONSTRAINT `book_isbn` FOR
(n:`Book`) REQUIRE (n.`isbn`) IS UNIQUE"                                                    |
| "book_title_year"        | "NODE_PROPERTY_UNIQUENESS"        | "CREATE CONSTRAINT `book_title_year` FOR
(n:`Book`) REQUIRE (n.`title`, n.`publicationYear`) IS UNIQUE"                              |
| "director_imdbId"        | "NODE_KEY"                        | "CREATE CONSTRAINT `director_imdbId` FOR
(n:`Director`) REQUIRE (n.`imdbId`) IS KEY"                                                 |
| "knows_since_how"        | "RELATIONSHIP_KEY"                | "CREATE CONSTRAINT `knows_since_how` FOR
()-[r:`KNOWS`]-() REQUIRE (r.`since`, r.`how`) IS KEY"                                      |
| "movie_tagline"          | "NODE_PROPERTY_TYPE"              | "CREATE CONSTRAINT `movie_tagline` FOR
(n:`Movie`) REQUIRE (n.`tagline`) IS :: STRING | LIST<STRING NOT NULL>"                      |
| "movie_title"            | "NODE_PROPERTY_TYPE"              | "CREATE CONSTRAINT `movie_title` FOR
(n:`Movie`) REQUIRE (n.`title`) IS :: STRING"                                               |
| "node_uniqueness_param"  | "NODE_PROPERTY_UNIQUENESS"        | "CREATE CONSTRAINT
`node_uniqueness_param` FOR (n:`Book`) REQUIRE (n.`prop1`) IS UNIQUE"
|
| "node_vector_constraint" | "NODE_PROPERTY_TYPE"              | "CREATE CONSTRAINT
`node_vector_constraint` FOR (n:`Movie`) REQUIRE (n.`embedding`) IS :: VECTOR<INTEGER32 NOT NULL>(42)"
|
| "ownershipId"            | "RELATIONSHIP_KEY"                | "CREATE CONSTRAINT `ownershipId` FOR ()-
[r:`OWNS`]-() REQUIRE (r.`ownershipId`) IS KEY"                                              |
| "part_of"                | "RELATIONSHIP_PROPERTY_TYPE"      | "CREATE CONSTRAINT `part_of` FOR ()-
[r:`PART_OF`]-() REQUIRE (r.`order`) IS :: INTEGER"                                           |
| "part_of_tags"           | "RELATIONSHIP_PROPERTY_TYPE"      | "CREATE CONSTRAINT `part_of_tags` FOR
()-[r:`PART_OF`]-() REQUIRE (r.`tags`) IS :: STRING | LIST<STRING NOT NULL>"                  |
| "prequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "CREATE CONSTRAINT `prequels` FOR ()-
[r:`PREQUEL_OF`]-() REQUIRE (r.`order`, r.`author`) IS UNIQUE"                               |
| "rel_exist_param"        | "RELATIONSHIP_PROPERTY_EXISTENCE" | "CREATE CONSTRAINT `rel_exist_param` FOR
()-[r:`WROTE`]-() REQUIRE (r.`published`) IS NOT NULL"                                      |
| "rel_vector_constraint"  | "RELATIONSHIP_PROPERTY_TYPE"      | "CREATE CONSTRAINT
```

```
`rel_vector_constraint` FOR ()-[r:`CONTAINS`]-() REQUIRE (r.`embedding`) IS :: VECTOR<FLOAT32 NOT
NULL>(1536)" |
| "sequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "CREATE CONSTRAINT `sequels` FOR ()-
[r:`SEQUEL_OF`]-() REQUIRE (r.`order`) IS UNIQUE"                                          |
| "wrote_year"            | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "CREATE CONSTRAINT `wrote_year` FOR ()-
[r:`WROTE`]-() REQUIRE (r.`year`) IS NOT NULL"                                             |
+-------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------+
```

Show constraints with filtering

The `SHOW CONSTRAINTS` command can be filtered in various ways. The filtering of rows can be done using constraint type keywords or a `WHERE` clause, while filtering of columns is achieved by specifying the desired columns in a `YIELD` clause.

Show only key constraints

```
SHOW KEY CONSTRAINTS
```

Result

```
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------+
| id | name              | type               | entityType     | labelsOrTypes | properties
| enforcedLabel | ownedIndex        | propertyType |
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------+
| 28 | "actor_fullname"  | "NODE_KEY"         | "NODE"         | ["Actor"]     | ["firstname", "surname"]
| NULL          | "actor_fullname"  | NULL         |
| 36 | "director_imdbId" | "NODE_KEY"         | "NODE"         | ["Director"]  | ["imdbId"]
| NULL          | "director_imdbId" | NULL         |
| 26 | "knows_since_how" | "RELATIONSHIP_KEY" | "RELATIONSHIP" | ["KNOWS"]     | ["since", "how"]
| NULL          | "knows_since_how" | NULL         |
| 38 | "ownershipId"     | "RELATIONSHIP_KEY" | "RELATIONSHIP" | ["OWNS"]      | ["ownershipId"]
| NULL          | "ownershipId"     | NULL         |
+-------------------------------------------------------------------------------------------------------------
-------------------------------------------------+
```

For a full list of all the constraint types (and synonyms) available in this command see Syntax → SHOW CONSTRAINTS.

Filter `SHOW CONSTRAINTS` with `WHERE` to show only constraints with a `RELATIONSHIP` `entityType`

```
SHOW CONSTRAINTS
WHERE entityType = 'RELATIONSHIP'
```

Result

```
+-------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------+
| id | name                   | type                              | entityType     | labelsOrTypes  |
properties          | enforcedLabel | ownedIndex        | propertyType                      |
+-------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------+
| 26 | "knows_since_how"      | "RELATIONSHIP_KEY"                | "RELATIONSHIP" | ["KNOWS"]      |
["since", "how"]    | NULL          | "knows_since_how" | NULL                              |
| 38 | "ownershipId"          | "RELATIONSHIP_KEY"                | "RELATIONSHIP" | ["OWNS"]       |
["ownershipId"]     | NULL          | "ownershipId"     | NULL                              |
| 41 | "part_of"              | "RELATIONSHIP_PROPERTY_TYPE"      | "RELATIONSHIP" | ["PART_OF"]    |
["order"]           | NULL          | NULL              | "INTEGER"                         |
| 39 | "part_of_tags"         | "RELATIONSHIP_PROPERTY_TYPE"      | "RELATIONSHIP" | ["PART_OF"]    |
["tags"]            | NULL          | NULL              | "STRING | LIST<STRING NOT NULL>" |
| 24 | "prequels"             | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["PREQUEL_OF"] |
["order", "author"] | NULL          | "prequels"        | NULL                              |
| 30 | "rel_exist_param"      | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
```

```
["published"]       | NULL          | NULL              | NULL                                |
| 3  | "rel_vector_constraint" | "RELATIONSHIP_PROPERTY_TYPE"       | "RELATIONSHIP" | ["CONTAINS"]   |
["embedding"]       | NULL          | NULL              | "VECTOR<FLOAT32 NOT NULL>(1536)" |
| 19 | "sequels"               | "RELATIONSHIP_PROPERTY_UNIQUENESS" | "RELATIONSHIP" | ["SEQUEL_OF"]  |
["order"]           | NULL          | "sequels"         | NULL                                |
| 14 | "wrote_year"            | "RELATIONSHIP_PROPERTY_EXISTENCE"  | "RELATIONSHIP" | ["WROTE"]      |
["year"]            | NULL          | NULL              | NULL                                |
+-------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------+
```

Combine `SHOW CONSTRAINTS` with other Cypher clauses

Cypher 25 only | Introduced in Neo4j 2026.05

The `SHOW CONSTRAINTS` command can be combined with general Cypher clauses in a single query.

For example, returning the labels and properties of nodes in the graph that are constrained by a node key constraint.

Return labels and properties of node key constrained nodes

```
SHOW CONSTRAINTS
YIELD name, type, labelsOrTypes, properties
WHERE type = 'NODE_KEY'
WITH name, labelsOrTypes[0] AS label, properties
MATCH (n:$(label))
WHERE all(p IN properties WHERE p IN keys(n))
RETURN name AS constraint, labels(n) AS labels, properties(n) AS properties
```

Result

```
+-------------------------------------------------------------------------------------------------------------------
-------+
| constraint        | labels               | properties
|
+-------------------------------------------------------------------------------------------------------------------
-------+
| "actor_fullname"  | ["Director","Actor"] | {firstname -> "Jensen", imdbId -> "nm0010075", surname ->
"Ackles"} |
| "actor_fullname"  | ["Actor"]            | {firstname -> "Misha", surname -> "Collins"}
|
| "director_imdbId" | ["Director","Actor"] | {firstname -> "Jensen", imdbId -> "nm0010075", surname ->
"Ackles"} |
+-------------------------------------------------------------------------------------------------------------------
-------+
3 rows
```

> **Note:**
>
> When combining `SHOW CONSTRAINTS` with other clauses, the `YIELD` clause is mandatory and must not be omitted. `YIELD` must explicitly list the yielded columns. `YIELD *` is not permitted. The query must also end with a valid last clause (like a `RETURN` or an updating clause).

Return columns

show constraints output

Column	Description	Type
`id`	The id of the constraint. Default Output	`INTEGER`
`name`	Name of the constraint (explicitly set by the user or automatically assigned). Default Output	`STRING`
`type`	The ConstraintType of this constraint (`NODE_PROPERTY_UNIQUENESS`, `RELATIONSHIP_PROPERTY_UNIQUENESS`, `NODE_PROPERTY_EXISTENCE`, `RELATIONSHIP_PROPERTY_EXISTENCE`, `NODE_PROPERTY_TYPE`, `RELATIONSHIP_PROPERTY_TYPE`, `NODE_LABEL_EXISTENCE`, `RELATIONSHIP_SOURCE_LABEL`, `RELATIONSHIP_TARGET_LABEL`, `NODE_KEY`, or `RELATIONSHIP_KEY`). Default Output	`STRING`
`entityType`	Type of entities this constraint represents (`NODE` or `RELATIONSHIP`). Default Output	`STRING`
`labelsOrTypes`	The labels or relationship types of this constraint. The list returned will only include a single value (the name of the constrained node label or relationship type). Default Output	`LIST<STRING>`
`properties`	The properties of this constraint, or `null` for node label existence, relationship source label, and relationship target constraints which do not constrain properties. Default Output	`LIST<STRING>`
`enforcedLabel`	The implied labels required by a node label existence constraint or the source node and target node labels required by a relationship source or target label constraint, or `null` for other constraints which do not constrain labels. Default Output Cypher 25 only Introduced in Neo4j 2026.02	`STRING`
`ownedIndex`	The name of the index associated with the constraint or `null`, in case no index is associated with it. Default Output	`STRING`
`propertyType`	The property type the property is restricted to for property type constraints, or `null` for the other constraints. Default Output	`STRING`
`classification`	The status of a constraint in a graph type (`dependent`, `undesignated`, or `independent`). Cypher 25 only Introduced in Neo4j 2026.02	`STRING`
`options`	The options passed to `CREATE` command, for the index associated to the constraint, or `null` if no index is associated with the constraint.	`MAP`
`createStatement`	Statement used to create the constraint, or `null` if the constraint cannot be created through `CREATE CONSTRAINT` (for example in case of version incompatibilities or for constraints with `dependent` classification).	`STRING`

Drop constraints

Constraints are dropped using the `DROP CONSTRAINT` command. It is possible to drop constraints created in a different Cypher version than the one in use. For example, although `VECTOR` property type constraints

were added in Cypher 25, such constraints can still be dropped using Cypher 5.

For the full command syntax to drop constraints, see Syntax → Drop constraints. The constraints dropped on this page are created on the page Create constraints.

Dropping a constraint requires the `DROP CONSTRAINT` privilege.

Drop a constraint by name

A constraint can be dropped using the name with the `DROP CONSTRAINT constraint_name` command. It is the same command for all constraint types. The name of the constraint can be found using the `SHOW CONSTRAINTS` command, given in the output column `name`.

Example 489. Drop a constraint by name

Drop the constraint `book_isbn`

```
DROP CONSTRAINT book_isbn
```

Result

```
Removed 1 constraint.
```

Note:

Constraints that are created as part of node or relationship element types in a graph type cannot be dropped with the `DROP CONSTRAINT constraint_name` command. This includes the following constraints:

- Property existence and property type constraints on identifying node labels or relationship types
- Node label existence constraints
- Relationship source and target label constraints

These constraints can only be dropped by dropping the full node or relationship element type. For more information, see Drop node and relationship element types.

Drop a constraint with a parameter

Constraints can be dropped with a parameterized name.

Example 490. Drop a constraint using a parameter

Parameters

```
{
  "name": "actor_fullname"
```

```
}
```

Drop a constraint with a parameterized name

```
DROP CONSTRAINT $name
```

Result

```
Removed 1 constraint.
```

Drop a non-existing constraint

If it is uncertain if any constraint with a given name exists and you want to drop it if it does but not get an error should it not, use `IF EXISTS`. This will ensure that no error is thrown. Instead, an informational notification is returned stating that the constraint does not exist.

Example 491. Drop a non-existing constraint

Drop the non-existing constraint `missing_constraint_name`

```
DROP CONSTRAINT missing_constraint_name IF EXISTS
```

Result

```
(no changes, no records)
```

Notification

```
`DROP CONSTRAINT missing_constraint_name IF EXISTS` has no effect. `missing_constraint_name` does not
exist.
```

Syntax

This page contains the syntax for the graph types and constraints commands. More details about the syntax can be found in the Operations Manual → Cypher syntax for administration commands.

Graph types

Cypher 25 only | Enterprise Edition | Introduced in Neo4j 2026.02

Node and relationship element types

Syntax for node element types, <node element type>

```
([alias] {IS | :} IdentifyingLabelName {=> | IMPLIES} [{IS | :} ImpliedLabelName[&...]] [“{“ {propertyName
[[IS] :: | IS TYPED] <TYPE> [IS [NODE] {KEY | UNIQUE}]}[, ...] “}”])
[REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [NODE] {KEY | UNIQUE}][...]
```

A node element type requires either one or more implied labels or a non-empty property map.

For more information, see Set graph types → Node element types.

Syntax for relationship element types, <relationship element type>

```
([nodeAlias | [nodeAlias] {IS | :} LabelName [=> | IMPLIES]])-"[" [alias] {IS | :}
IDENTIFYING_RELATIONSHIP_TYPE {=> | IMPLIES} ["{" {propertyName [[IS] :: | IS TYPED] <TYPE> [IS
[REL[ATIONSHIP]] {KEY | UNIQUE}]}[, ...] "}"] "]"->([nodeAlias | [nodeAlias] {IS | :} LabelName [=> |
IMPLIES]])
[REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [REL[ATIONSHIP]] {KEY | UNIQUE}][...]
```

Note:

A relationship element type requires either:

- A non-empty source or a target node.
- A non-empty property map.

For more information, see Set graph types → Relationship element types.

`<TYPE>` can be one of the property types listed in constraint definitions in graph types, the `NOT NULL` versions of those types, or `ANY NOT NULL`. Allowed syntax variations of these types are listed in Values and types → Types and their synonyms.

The three variations of the property type expression, `IS ::`, `::`, and `IS TYPED` are syntactic synonyms for the same expression. The preferred syntax when defining node and relationship element types in a graph type is the `::` variant.

Constraint definitions in graph types

Syntax for property uniqueness and key constraints on identifying node labels and relationship types, <constraint on identifying label/type>

Property uniqueness and key constraints on labels identifying node element types	`CONSTRAINT [constraint_name]`
	`FOR (alias [{IS | :} IdentifyingLabelName [=> | IMPLIES]])`
	`REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [NODE] {KEY | UNIQUE}`

Property uniqueness and key constraints on relationship types identifying relationship element types	`CONSTRAINT [constraint_name]`
	`FOR ()-"[“alias [{IS | :} IDENTIFYING_RELATIONSHIP_TYPE [=> | IMPLIES]] "]"->()`
	`REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [REL[ATIONSHIP]] {KEY | UNIQUE}`

For more information, see Set graph types → Property uniqueness and key constraints on identifying node labels and relationship types.

Syntax for property uniqueness, property existence, property type, and key constraints on non-identifying node labels and relationship types, <constraint on non-identifying label/type>

Node key and property uniqueness constraint	`CONSTRAINT [constraint_name]`
	`FOR (alias {IS | :} NonIdentifyingLabelName)`
	`REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [NODE] {KEY | UNIQUE}`

Node property existence and property type constraint	`CONSTRAINT [constraint_name]`
	`FOR (alias {IS | :} NonIdentifyingLabelName)`
	`REQUIRE {alias.propertyName | (alias.propertyName)} { IS NOT NULL | {[IS] :: | IS TYPED}`
	`<TYPE> }`

Relationship key and property uniqueness constraint	`CONSTRAINT [constraint_name]`
	`FOR ()-"["alias {IS | :} NON_IDENTIFYING_RELATIONSHIP_TYPE "]"->()`
	`REQUIRE {alias.propertyName | (alias.propertyName[, ...])} IS [REL[ATIONSHIP]] {KEY |`
	`UNIQUE}`

Relationship property existence and property type constraint	`CONSTRAINT [constraint_name]`
	`FOR ()-"["alias {IS | :} NON_IDENTIFYING_RELATIONSHIP_TYPE "]"->()`
	`REQUIRE {alias.propertyName | (alias.propertyName)} { IS NOT NULL | {[IS] :: | IS TYPED}`
	`<TYPE> }`

For more information, see see Set graph types → Constraints on non-identifying labels and types.

The three variations of the property type expression, `IS ::`, `::`, and `IS TYPED` are syntactic synonyms for the same expression. The preferred syntax when defining constraints in a graph type is the `IS ::` variant.

`<TYPE>` can be one of the following property types:

- `BOOLEAN`
- `STRING`
- `INTEGER`
- `FLOAT`
- `DATE`
- `LOCAL TIME`
- `ZONED TIME`
- `LOCAL DATETIME`
- `ZONED DATETIME`
- `DURATION`
- `POINT`
- `UUID` Cypher 25 only Introduced in Neo4j 2026.08
- `VECTOR<TYPE>(DIMENSION)` Cypher 25 only Introduced in Neo4j 2025.10
- `LIST<BOOLEAN NOT NULL>`
- `LIST<STRING NOT NULL>`
- `LIST<INTEGER NOT NULL>`
- `LIST<FLOAT NOT NULL>`
- `LIST<DATE NOT NULL>`
- `LIST<LOCAL TIME NOT NULL>`

- `LIST<ZONED TIME NOT NULL>`
- `LIST<LOCAL DATETIME NOT NULL>`
- `LIST<ZONED DATETIME NOT NULL>`
- `LIST<DURATION NOT NULL>`
- `LIST<POINT NOT NULL>`
- Any closed dynamic union of the above types, e.g. `INTEGER | FLOAT | STRING`.

> Because storing lists of `VECTOR` values is not supported, property type constraints cannot be created for `LIST<VECTOR<TYPE>(DIMENSION) NOT NULL>`. Additionally, `VECTOR` property type constraints must be created with a specific dimension and coordinate value, where the dimension must be greater than `0` and less than or equal to `4096`. For more information, see Values and types → Vectors.

Allowed syntax variations of these types are listed in Values and types → Types and their synonyms.

Set graph types

> **Note:**
>
> Setting a graph type requires the following privileges:
>
> - `CREATE CONSTRAINT`
> - `DROP CONSTRAINT`
> - `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

Syntax for `ALTER CURRENT GRAPH TYPE SET`

```
ALTER CURRENT GRAPH TYPE SET “{“
  [{<node element type> | <relationship element type> | <constraint on identifying label/type> |
<constraint on non-identifying label/type>}[, ...]]
“}”
```

For more information, see Set graph types.

Extend graph types

> **Note:**
>
> Extending a graph type requires the following privileges:
>
> - `CREATE CONSTRAINT`
> - `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

Syntax for `ALTER CURRENT GRAPH TYPE ADD`

```
ALTER CURRENT GRAPH TYPE ADD “{“
  [{<node element type> | <relationship element type> | <constraint on identifying label/type> |
<constraint on non-identifying label/type>}[, ...]]
“}”
```

Any added element types and constraints must, together with the existing ones, form a valid graph type. This means, for example, that `ADD` cannot modify existing element types but only add new ones. Independent constraints cannot be added to node labels/relationship types that serve as the identifying labels/types of element types. Nor can element types be added if their identifying labels/types already have independent constraints defined on them.

For more information, see Extend graph types.

Alter element types

Note:

Altering element types in a graph type requires the following privileges:

- `CREATE CONSTRAINT`
- `DROP CONSTRAINT`
- `NAME MANAGEMENT` (if graph type includes node labels, relationship types, or properties not already in the graph)

Syntax for `ALTER CURRENT GRAPH TYPE ALTER`

```
ALTER CURRENT GRAPH TYPE ALTER “{“
  [{<node element type> | <relationship element type>}[, ...]]
“}”
```

Note:

Note the following:

- Key or property uniqueness constraints cannot be included when altering an element type.
- `ALTER` cannot introduce new element types to the graph type, only modify existing ones. And it replaces the previous definition of that element type with the new definition.

For more information, see Alter element types.

Show graph types

Showing a graph type requires the `SHOW CONSTRAINTS` privilege.

Syntax to list the current graph type with default return columns

```
SHOW CURRENT GRAPH TYPE [AS GRAPH]
  [WHERE expression]
```

Syntax to list the current graph type with full or given return columns

```
SHOW CURRENT GRAPH TYPE [AS GRAPH]
YIELD { * | field[, ...] } [ORDER BY field[, ...]] [SKIP n] [LIMIT n]
  [WHERE expression]
  [RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]
```

For more information, see Show graph types. For full details of the columns returned by the `SHOW CURRENT GRAPH TYPE` command, see Show graph types → Return columns. From 2026.05, it is possible to combine `SHOW CURRENT GRAPH TYPE` with general Cypher clauses in a single query. `AS GRAPH` was introduced in Neo4j 2026.07.

Drop graph type elements

Dropping element types and constraints from a graph type requires the `DROP CONSTRAINT` privilege.

Syntax for dropping node element type, <drop node element type>

Full definition	`([alias] {IS | :} IdentifyingLabelName {=> | IMPLIES} [{IS | :} ImpliedLabelName[&...]] ["{" {propertyName [[IS] :: | IS TYPED] <TYPE>}[, ...] "}"])`
Identifying label only	`([alias] {IS | :} IdentifyingLabelName {=> | IMPLIES})`

Syntax for dropping relationship element type, <drop node relationship type>

Full definition	`([nodeAlias | [nodeAlias] {IS | :} LabelName [=> | IMPLIES]])-"[" [alias] {IS | :} IDENTIFYING_RELATIONSHIP_TYPE {=> | IMPLIES} ["{" {propertyName [[IS] :: | IS TYPED] <TYPE>}[, ...] "}"] "]"->([nodeAlias | [nodeAlias] {IS | :} LabelName [=> | IMPLIES]])`
Identifying type only	`()-"[" [alias] {IS | :} IDENTIFYING_RELATIONSHIP_TYPE {=> | IMPLIES} "]"->()`

`<TYPE>` can be the same property types as for node and relationship element types.

If the full definition is provided, then it needs to match exactly the node or relationship element type being dropped.

Syntax for dropping constraints on identifying or non-identifying labels/types, <drop constraint>

```
CONSTRAINT constraint_name
```

Syntax for `ALTER CURRENT GRAPH TYPE DROP`

```
ALTER CURRENT GRAPH TYPE DROP "{"
  [{<drop node element type> | <drop relationship element type> | <drop constraint>}[, ...]]
```

```
"}"
```

For more information, see Drop graph type elements.

Constraints

Create constraints

Constraints are created with the `CREATE CONSTRAINT` command. When creating a constraint, it is recommended to provide a constraint name. This name must be unique among both indexes and constraints. If a name is not explicitly given, a unique name will be auto-generated.

Creating a constraint requires the `CREATE CONSTRAINT` privilege.

The `CREATE CONSTRAINT` command is optionally idempotent. This means its default behavior is to throw an error if an attempt is made to create the same constraint twice. With the `IF NOT EXISTS` flag, no error is thrown and nothing happens should a constraint with the same name or same schema and constraint type already exist. It may still throw an error if conflicting data, indexes, or constraints exist. Examples of this are nodes with missing properties, indexes with the same name, or constraints with same schema but a different conflicting constraint type. An informational notification is returned in case nothing happens showing the existing constraint which blocks the creation.

Create property uniqueness constraints

Syntax for creating a node property uniqueness constraint on a single property

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE n.propertyName IS [NODE] UNIQUE
```

Syntax for creating a composite node property uniqueness constraint on multiple properties

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE (n.propertyName_1, ..., n.propertyName_n) IS [NODE] UNIQUE
```

Syntax for creating a relationship property uniqueness constraint on a single property

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE r.propertyName IS [REL[ATIONSHIP]] UNIQUE
```

Syntax for creating a composite relationship property uniqueness constraint on multiple properties

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE (r.propertyName_1, ..., r.propertyName_n) IS [REL[ATIONSHIP]] UNIQUE
```

For examples on how to create property uniqueness constraints, see Create constraints → Create property uniqueness constraint. Property uniqueness constraints are index-backed.

Create property existence constraints

Enterprise Edition

Syntax for creating a node property existence constraint

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE n.propertyName IS NOT NULL
```

Syntax for creating a relationship property existence constraint

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE r.propertyName IS NOT NULL
```

For examples on how to create property existence constraints, see Create constraints → Create property existence constraints.

Create property type constraints

Enterprise Edition

Syntax for creating a node property type constraint

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE n.propertyName {[IS] :: | IS TYPED} <TYPE>
```

Syntax for creating a relationship property type constraint

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE r.propertyName {[IS] :: | IS TYPED} <TYPE>
```

The three variations of the expression, `IS ::`, `::`, and `IS TYPED` are syntactic synonyms for the same expression. The preferred syntax is the `IS ::` variant.

Where `<TYPE>` is one of the following property types:

- `BOOLEAN`
- `STRING`
- `INTEGER`
- `FLOAT`
- `DATE`
- `LOCAL TIME`
- `ZONED TIME`
- `LOCAL DATETIME`
- `ZONED DATETIME`

- `DURATION`
- `POINT`
- `UUID` Cypher 25 only Introduced in Neo4j 2026.08
- `VECTOR<TYPE>(DIMENSION)` Cypher 25 only Introduced in Neo4j 2025.10
- `LIST<BOOLEAN NOT NULL>`
- `LIST<STRING NOT NULL>`
- `LIST<INTEGER NOT NULL>`
- `LIST<FLOAT NOT NULL>`
- `LIST<DATE NOT NULL>`
- `LIST<LOCAL TIME NOT NULL>`
- `LIST<ZONED TIME NOT NULL>`
- `LIST<LOCAL DATETIME NOT NULL>`
- `LIST<ZONED DATETIME NOT NULL>`
- `LIST<DURATION NOT NULL>`
- `LIST<POINT NOT NULL>`
- Any closed dynamic union of the above types, e.g. `INTEGER | FLOAT | STRING`.

> Because storing lists of `VECTOR` values is not supported, property type constraints cannot be created for `LIST<VECTOR<TYPE>(DIMENSION) NOT NULL>`. Additionally, `VECTOR` property type constraints must be created with a specific dimension and coordinate value, where the dimension must be greater than `0` and less than or equal to `4096`. For more information, see Values and types → Vectors.

Allowed syntax variations of these types are listed in Types and their synonyms.

For examples on how to create property type constraints, see Create constraints → Create property type constraints.

Create key constraints

Enterprise Edition

Syntax for creating a node key constraint on a single property

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE n.propertyName IS [NODE] KEY
```

Syntax for creating a composite node key constraint on multiple properties

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR (n:LabelName)
REQUIRE (n.propertyName_1, ..., n.propertyName_n) IS [NODE] KEY
```

Syntax for creating a relationship key constraint on a single property

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE r.propertyName IS [REL[ATIONSHIP]] KEY
```

Syntax for creating a composite relationship key constraint on multiple properties

```
CREATE CONSTRAINT [constraint_name] [IF NOT EXISTS]
FOR ()-"["r:RELATIONSHIP_TYPE"]"-()
REQUIRE (r.propertyName_1, ..., r.propertyName_n) IS [REL[ATIONSHIP]] KEY
```

For examples on how to create key constraints, see Create constraints → Create key constraints. Key constraints are index-backed.

Show constraints

To list all constraints with the default output columns, use `SHOW CONSTRAINTS`. If all columns are required, use `SHOW CONSTRAINTS YIELD *`. If only specific columns are required, use `SHOW CONSTRAINTS YIELD field[, …]`. The `SHOW CONSTRAINTS` clause can also be filtered using the `WHERE` clause.

Showing constraints requires the `SHOW CONSTRAINTS` privilege.

Syntax to list constraints with default return columns

```
SHOW [
      ALL
     |NODE [PROPERTY] UNIQUE[NESS]
     |REL[ATIONSHIP] [PROPERTY] UNIQUE[NESS]
     |[PROPERTY] UNIQUE[NESS]
     |NODE [PROPERTY] EXIST[ENCE]
     |REL[ATIONSHIP] [PROPERTY] EXIST[ENCE]
     |[PROPERTY] EXIST[ENCE]
     |NODE PROPERTY TYPE
     |REL[ATIONSHIP] PROPERTY TYPE
     |PROPERTY TYPE
     |NODE KEY
     |REL[ATIONSHIP] KEY
     |KEY
] CONSTRAINT[S]
  [WHERE expression]
```

Syntax for listing constraints with full or given return columns

```
SHOW [
      ALL
     |NODE [PROPERTY] UNIQUE[NESS]
     |REL[ATIONSHIP] [PROPERTY] UNIQUE[NESS]
     |[PROPERTY] UNIQUE[NESS]
     |NODE [PROPERTY] EXIST[ENCE]
     |REL[ATIONSHIP] [PROPERTY] EXIST[ENCE]
     |[PROPERTY] EXIST[ENCE]
     |NODE PROPERTY TYPE
     |REL[ATIONSHIP] PROPERTY TYPE
     |PROPERTY TYPE
     |NODE KEY
     |REL[ATIONSHIP] KEY
     |KEY
] CONSTRAINT[S]
YIELD { * | field[, ...] } [ORDER BY field[, ...]] [SKIP n] [LIMIT n]
  [WHERE expression]
  [RETURN field[, ...] [ORDER BY field[, ...]] [SKIP n] [LIMIT n]]
```

The type filtering keywords filters the returned constraints on constraint type:

Type filters

Filter	Description
`ALL`	Returns all constraints, no filtering on constraint type. This is the default if none is given.
`NODE [PROPERTY] UNIQUE[NESS]`	Returns node property uniqueness constraints.
`REL[ATIONSHIP] [PROPERTY] UNIQUE[NESS]`	Returns relationship property uniqueness constraints.
`[PROPERTY] UNIQUE[NESS]`	Returns property uniqueness constraints, for both nodes and relationships.
`NODE PROPERTY EXIST[ENCE]`	Returns node property existence constraints.
`REL[ATIONSHIP] PROPERTY EXIST[ENCE]`	Returns relationship property existence constraints.
`PROPERTY EXIST[ENCE]`	Returns property existence constraints, for both nodes and relationships.
`NODE EXIST[ENCE]`	Returns node existence constraints.
`REL[ATIONSHIP] EXIST[ENCE]`	Returns relationship existence constraints.
`EXIST[ENCE]`	Returns existence constraints, for both nodes and relationships.
`NODE PROPERTY TYPE`	Returns node property type constraints.
`REL[ATIONSHIP] PROPERTY TYPE`	Returns relationship property type constraints.
`PROPERTY TYPE`	Returns property type constraints, for both nodes and relationships.
`NODE KEY`	Returns node key constraints.
`REL[ATIONSHIP] KEY`	Returns relationship key constraints.
`KEY`	Returns key constraints, for both nodes and relationships.

For examples on how to list constraints, see Show constraints. For full details of the returned columns for the `SHOW CONSTRAINTS` command, see Show constraints → Return columns. In Cypher 25 from 2026.05, it is possible to combine `SHOW CONSTRAINTS` with general Cypher clauses in a single query.

Drop constraints

Constraints are dropped using the `DROP` CONSTRAINT` command. Dropping a constraint is done by specifying the name of the constraint.

Dropping a constraint requires the `DROP CONSTRAINT` privilege.

Syntax for dropping a constraint by name

```
DROP CONSTRAINT constraint_name [IF EXISTS]
```

This command is optionally idempotent. This means its default behavior is to throw an error if an attempt is made to drop the same constraint twice. With the `IF EXISTS` flag, no error is thrown and nothing

happens should the constraint not exist. Instead, an informational notification is returned detailing that the constraint does not exist.

For examples on how to drop constraints, see Drop constraints.

Query plans, tuning, and caching

Cypher queries are executed according to a particular plan. The query plan consists of a binary tree of operators, with information about the step-by-step execution of a query, and it may differ depending on which runtime the query uses. Apart from selecting a different runtime, there are numerous other ways in which a query can be tuned.

More information about each of these topics can be found in the following sections:

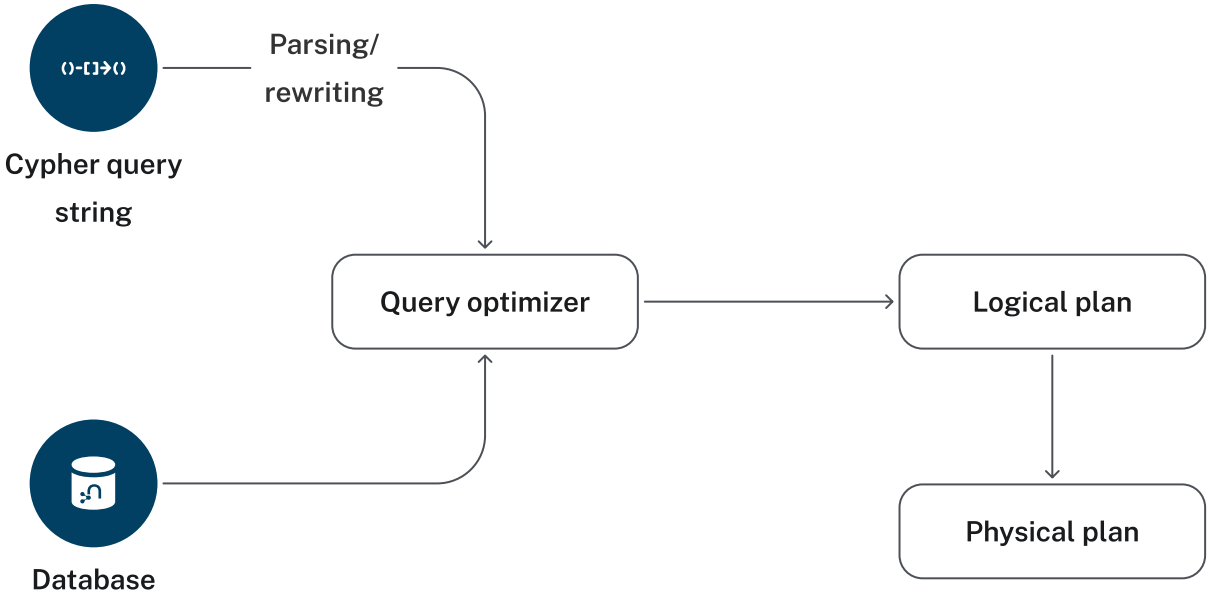
- Understanding query plans
- Operators
- Cypher runtimes
- Query tuning
- Query caches

Understanding query plans

This page explains how Cypher queries are planned, and how to understand the query plans shown when using the `EXPLAIN` or `PROFILE` keywords.

The lifecycle of a Cypher query

A Cypher query begins as a declarative query represented as a string, describing the graph pattern to match in a database. After parsing, the query string goes through the query optimizer (also known as the planner), which produces a logical plan, determining the most efficient way to execute the query given the current state of the database.[1] In the final phase, this logical plan is turned into an executable physical plan, which actually runs the query against the database. Executing this physical plan is the task of the Cypher runtime.



`EXPLAIN` and `PROFILE` query plans

Query plans can be shown by prepending a query with either `EXPLAIN` or `PROFILE`.

- `EXPLAIN`: plans the query but does not run it. Only returns estimated rows.
- `PROFILE`: runs the query, and the plan shows real measurements (for example, the number of DB hits the plan involved). Unlike `EXPLAIN`, queries prepended with `PROFILE` can write data to the database. Note that *profiling your query uses more resources*, so you should not profile unless you are actively working on a query.

Example query plan

Find all `Person` *nodes not* `FRIENDS_WITH` `me`

```
PROFILE
MATCH (me:Person {name: 'me'}), (other:Person)
WHERE NOT (me)-[:FRIENDS_WITH]->(other)
RETURN other.name
```

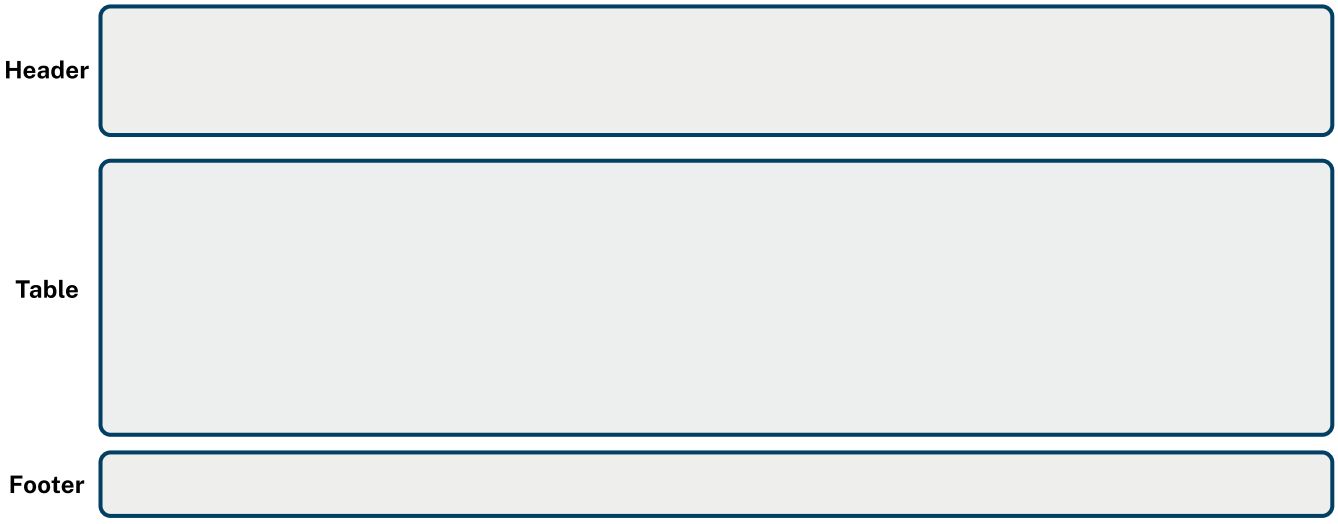
Query plan

```
Cypher 25

Planner COST

Runtime PIPELINED

Runtime version 2026.05.0

Batch size 128

+--------------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
| Operator           | Id | Details                                                   | Estimated Rows | Rows
| DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            | Indexes Used
|
+--------------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
| +ProduceResults    |  0 | `other.name`                                              |             13 |   12
|       0 |              0 |                   0/0 |     0,086 |                     |
|
| |                  +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+
+---------------------+
| +Projection        |  1 | other.name AS `other.name`                                |             13 |   12
|      24 |                |                   0/0 |     0,216 | In Pipeline 3       |
|
| |                  +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
| +Apply             |  2 |                                                           |             13 |   12
|       0 |                |                   0/0 |     0,008 |                     |
|
| |\                 +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
| | +Apply           |  3 |                                                           |             13 |   12
|       0 |                |                   0/0 |           |                     |
|
| | |\               +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
| | | +Anti          |  9 |                                                           |             13 |   12
|       0 |           1256 |                   0/0 |     0,224 | In Pipeline 3       |
|
| | | |              +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------
+---------------------+
```

```
| | | +Limit         |  8 | 1                                                                |             1 |    2
|       0 |            752 |                          |           |                    |
|
| | | |             +----+------------------------------------------------------------+---------------
+------+---------+----------------+                        |           |
+---------------------+
| | | +Expand(Into)  |  4 | (me)-[:FRIENDS_WITH]->(other)                            |             1 |    2
|      16 |            632 |                          |           |                    |
|
| | | |             +----+------------------------------------------------------------+---------------
+------+---------+----------------+                        |           |
+---------------------+
| | | +Argument      |  5 | me, other                                                |            14 |   14
|       0 |           4472 |                   1/0 |     6,150 | Fused in Pipeline 2 |
|
| | |               +----+------------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+---------------------+
| | +NodeByLabelScan |  6 | other:Person                                             |            14 |   14
|      15 |           2424 |                   1/0 |     0,310 | In Pipeline 1       |
|
| |                 +----+------------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+---------------------+
| +NodeIndexSeek     |  7 | RANGE INDEX me:Person(name) WHERE name = $autostring_0 |             1 |    1
|       2 |            376 |                   0/1 |     3,368 | In Pipeline 0       | range_index_name:
1 |
+--------------------+----+------------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+---------------------+

Total database accesses: 57, total allocated memory: 7024

12 rows
ready to start consuming query after 75 ms, results consumed after another 17 ms
```

Query plan parts

A query plan contains three parts:



- **Header**: Cypher version, planner, runtime, and batch size (not included when using slotted runtime).
- **Table**: binary tree of operators showing how the query is planned. The operators are listed in the leftmost column, and each operator is assigned one row in the plan.
- **Footer**: total database accesses (sum of DB hits across all operators), plus optional totals for memory and parallel-runtime workers.
    - If `EXPLAIN` is used, `?` will be shown instead of the number of total database accesses. Does not show the total allocated memory.

- If `PROFILE` is used, the number of actual total database accesses will be shown. If some operators cannot be measured, the output appears as `<n> + ?`, where `<n>` indicates a lower bound. The total allocated memory represents the query-wide peak memory (not equivalent to the sum of each operator's peak memory).

Query plan table columns

Note:

- Columns only appear if at least one operator has a value for them.
- Empty cells indicate that no data is available for the corresponding operator.
- When a query uses either pipelined or parallel runtime, some operator profile data gets merged due to fusing and so can only be displayed per-pipeline rather than per-operator.

Column	When	Description	Considerations
`Operator`	Always	Name of the operator (e.g. `NodeByLabelScan`, `Expand(All)`, and `Filter`).	
`Id`	Always	A stable id for the operator within this plan. Only useful for cross-referencing.	
`Details`	Always	Operator-specific arguments. For example: • `AllNodesScan`: a variable (e.g. `n`) • `NodeByLabelScan`: variable + label (e.g. `n:Person`). • `Expand(All)` / `Expand(Into)`: a pattern (e.g. `(n)-[r:KNOWS]->(m)`) • `Filter`: the predicate, (e.g. `n.age > 30`)	
`Estimated Rows`	Always	The planner's estimate of how many rows this operator will produce. Used for plan choice.	Derived from index/label statistics and a selectivity model, with no runtime feedback and can, therefore, be incorrect. Treat it as a hint, not a measurement, to compare with the `Rows` column.
`Rows`	`PROFILE` only	The number of rows the operator actually produced when the query executed.	

Column	When	Description	Considerations
`DB Hits`	`PROFILE` only	A count of low-level database access operations, including entity reads, property reads, and index entry reads. This metric reflects the amount of work performed by the storage layer. Lower values generally indicate more efficient query execution and improved performance.	`DB Hits` is not the same as `Rows`. A single matched row can produce many DB hits. For more information, see Database hits below.
`Memory (Bytes)`	`PROFILE` only	Per-operator peak heap memory used. Blank for operators that do not allocate memory.	`Memory (Bytes)` is per-operator peak, not cumulative. Two operators each reporting `100 MB` does not mean 200 MB was used in total; they may have been active at different times. The query-wide peak is the total allocated memory line in the query plan footer.
`Page Cache Hits/Misses`	`PROFILE` only, enterprise edition	Page cache stats.	Measures how often data is found in memory versus requiring a read from disk. A warm cache means most data is already stored in memory, resulting in mostly hits. A cold cache means data must be loaded from disk, resulting in more misses, especially on the first execution. Results should not be compared across runs with different cache states.
`Time (ms)`	`PROFILE` only, pipelined / parallel runtime	Wall-clock time spent in this operator. For parallel runtime this gives the cumulative time spent by all workers.	`Time (ms)` in pipelined / parallel runtimes is per pipeline, not per operator. Operators that are fused together share a single time figure. A large `Time (ms)` on one row represents the whole fused pipeline.
`Ordered by`	When results are ordered using `ORDER BY`.	The provided order this operator guarantees, e.g. `n.name ASC`.	
`Pipeline`	Pipelined / Parallel runtime	Which runtime pipeline the operator belongs to, e.g. `Fused in Pipeline 3`. Operators in the same fused pipeline are compiled together.	
`Indexes Used`	`PROFILE` only	Which indexes the operator used and how many times each was accessed (not the number of rows that were read from the index). Useful in particular when debugging dynamic lookups.	

Column	When	Description	Considerations
`Concurrency`	`PROFILE` only, `CALL { … } IN CONCURRENT TRANSACTIONS`	The peak and the time-weighted average number of batches that ran at the same time in a `TransactionApply` or `TransactionForeach` operator, for example `peak 4 / avg 3.64`. Introduced in Neo4j 2026.10	Compare the values with the concurrency value of `IN [n] CONCURRENT TRANSACTIONS`. An average well below the limit means that fewer batches ran at the same time than allowed, for example because input rows arrived too slowly or because batch scheduling held batches back. For the full set of statistics and how to interpret them, see Measuring concurrency with `PROFILE`.

Reading query plan tables

The Cypher planner produces logical plans that describe how a particular query is going to be executed. This query plan is a binary tree of operators. An operator is a specialized execution module that is responsible for some type of transformation to the data before passing it on to the next operator until the desired graph pattern has been matched. The plans produced by the planner thus decide which operators will be used and in what order they are applied to achieve the aim declared in the original query.

The most important thing to remember when reading execution plans is that they are read from the bottom up. The **root operator** (the top row of the query plan table) is `ProduceResults`, the operator responsible for producing results, and data flows upward from the **leaf operator** (the bottom row of the query plan table). The relationships between operators are described in terms of parent and child: an operator's inputs are its **child operators**, and the operator receiving those inputs is its **parent operator**.

```
+ProduceResults <- Root operator
|
+Projection     <- Parent operator of Apply
|
+Apply          <- Child operator of Projection
|
| ...           (intermediate operators omitted)
|
+NodeIndexSeek  <- Leaf operator
```

Each operator can have 0, 1, or 2 child operators:

- 0 children = leaf operator
- 1 children = unary operator
- 2 children = binary operator

For binary operators the convention is:

The right-hand input (RHS) is shown first, indented one level deeper. The left-hand input (LHS) is shown second, at the same indentation level as the operator itself.

```
+Apply         <- the binary operator
|\
| +RHS-subtree <- right-hand input (evaluated first / indented)
|
+LHS-subtree   <- left-hand input (same level as the binary operator)
```

For an `Apply`-style (i.e. performing nested loops) binary operator, the LHS is run once, then the RHS is run once per LHS row, with the LHS bindings available. For a `Join` or `CartesianProduct`, both sides are full inputs combined by the operator.

Full example

```
+--------------------+----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| Operator           | Id | Details                                                   | Estimated Rows | Rows
| DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            | Indexes Used
|
+--------------------+----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| +ProduceResults    |  0 | `other.name`                                              |             13 |   12
|       0 |              0 |                    0/0 |     0,086 |                     |
|
| |                  +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+
+----------------------+
| +Projection        |  1 | other.name AS `other.name`                                |             13 |   12
|      24 |                |                    0/0 |     0,216 | In Pipeline 3       |
|
| |                  +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| +Apply             |  2 |                                                           |             13 |   12
|       0 |                |                    0/0 |     0,008 |                     |
|
| |\                 +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| | +Apply           |  3 |                                                           |             13 |   12
|       0 |                |                    0/0 |           |                     |
|
| | |\               +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| | | +Anti          |  9 |                                                           |             13 |   12
|       0 |           1256 |                    0/0 |     0,224 | In Pipeline 3       |
|
| | | |              +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| | | +Limit         |  8 | 1                                                         |              1 |    2
|       0 |            752 |                       |           |                     |
|
| | | |              +----+-----------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| | | +Expand(Into)  |  4 | (me)-[:FRIENDS_WITH]->(other)                             |              1 |    2
|      16 |            632 |                       |           |                     |
|
| | | |              +----+-----------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| | | +Argument      |  5 | me, other                                                 |             14 |   14
|       0 |           4472 |                    1/0 |     6,150 | Fused in Pipeline 2 |
|
| | |                +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| | +NodeByLabelScan |  6 | other:Person                                              |             14 |   14
|      15 |           2424 |                    1/0 |     0,310 | In Pipeline 1       |
|
| |                  +----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| +NodeIndexSeek     |  7 | RANGE INDEX me:Person(name) WHERE name = $autostring_0 |              1 |    1
|       2 |            376 |                    0/1 |     3,368 | In Pipeline 0       | range_index_name:
1 |
+--------------------+----+-----------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------------
```

```
+--------------------+
```

1. `NodeIndexSeek` (id `7`) — leaf operator at the bottom. Uses the range index on `Person(name)` to find the single node where `name = 'me'`. Produces `1` row.
2. `NodeByLabelScan` (id `6`) — leaf on the RHS branch. Scans all `Person` nodes, producing `14` rows (one per `Person`). This forms the LHS input to `Apply` (id `3`), providing the other rows that are then tested by the `Anti` (id `9`) operator.
3. `Argument` (id `5`) — leaf inside the RHS of the `Anti` subtree. Carries the already-bound variables `me` and `other` through. Produces `14` rows (no computation, just variable propagation).
4. `Expand(Into)` (id `4`) — for each `(me, other)` pair, checks whether a `(me)-[:FRIENDS_WITH]→(other)` pattern exists. This is executed once per input row (`14` invocations), producing rows when matches are found.
5. `Limit` (id `8`) — restricts the expansion input to 1 row per evaluation context, controlling how many expansions are attempted per bound row in the anti pattern evaluation.
6. `Anti` (id `9`) — binary operator implementing `NOT EXISTS`. For each `(me, other)` pair from its LHS, it runs the RHS (`Limit` -> `Expand`) and only emits the LHS row if the RHS produces zero rows. In this case, 12 of 14 `(me, other)` pairs survive because no friendship edge exists.
7. `Apply` (id `3`) — binary operator. Its LHS is the `NodeByLabelScan` (providing `other` rows), and its RHS is the `Anti` subtree. For each LHS row, the RHS is executed with bindings available, so the anti-check is evaluated per `other`.
8. `Apply` (id `2`) — outer binary operator combining LHS: NodeIndexSeek (binding `me`) and RHS: `Apply` (id `3`). The RHS runs once per me row, with `me` bound, producing the filtered `other` rows.
9. `Projection` (id `1`) — computes `other.name` for each remaining row. Produces `12` rows.
10. `ProduceResults` (id `0`) — root operator. Emits the final 12 rows to the client.

Lazy and eager query evaluation

In general, query evaluation is *lazy*. This means that most operators pipe their output rows to their parent operators as soon as they are produced. In other words, a child operator may not be fully exhausted before the parent operator starts consuming the input rows produced by the child.

However, some operators, such as those used for aggregation and sorting, need to aggregate all their rows before they can produce output. These operators are called eager operators. Such operators need to complete execution in its entirety before any rows are sent to their parents as input in order for the planner to preserve correctness (e.g. when reads and writes touch the same properties). Eager operators have a memory cost — the entire upstream output is buffered. The reason for an eager operator is shown in its `Details` column.

Database hits

Each operator will send a request to the storage engine to do work such as retrieving or updating data. A *database hit* (DBHits) is an abstract unit of this storage engine work.

These are all the actions that trigger one or more database hits:

- **Create actions**
  - Create a node.
  - Create a relationship.
  - Create a new node label.
  - Create a new relationship type.
  - Create a new ID for property keys with the same name.
- **Delete actions**
  - Delete a node.
  - Delete a relationship.
- **Update actions**
  - Set one or more labels on a node.
  - Remove one or more labels from a node.
- **Node-specific actions**
  - Get a node by its ID.
  - Get the degree of a node.
  - Determine whether a node is dense.
  - Determine whether a label is set on a node.
  - Get the labels of a node.
  - Get a property of a node.
  - Get an existing node label.
  - Get the name of a label by its ID, or its ID by its name.
- **Relationship-specific actions**
  - Get a relationship by its ID.
  - Get a property of a relationship.
  - Get an existing relationship type.
  - Get a relationship type name by its ID, or its ID by its name.
- **General actions**
  - Get the name of a property key by its ID, or its ID by the key name.
  - Find a node or relationship through an index seek or index scan.
  - Find a path in a variable-length expand.
  - Find a shortest path.
  - Ask the count store for a value.
- **Schema actions**
  - Add an index.
  - Drop an index.

    - Get the reference of an index.
    - Create a constraint.
    - Drop a constraint.
- Call a procedure.
- Call a user-defined function.

Operators

This page provides an overview of the available operators in Cypher. An operator is a specialized execution module that is responsible for some type of transformation to the data in a query execution plan before passing it on to the next operator, until the desired graph pattern has been matched.

For information about how to understand execution plans (and the role operators play in them), see Understanding query plans.

For more information about each operator, see Operators in detail.

Summary of operators

This table comprises all the execution plan operators ordered lexicographically.

- *Leaf* operators, in most cases, locate the starting nodes and relationships required in order to execute the query.
- *Updating* operators are used in queries that update the graph.
- *Eager* operators accumulate all their rows before piping them to the next operator.

Name	Description	Leaf?	Updating?	Considerations
AllNodesScan	Reads all nodes from the node store.	Yes		
AlterCurrentGraphTypeAdd	Adds elements to the current graph type.	Yes		Cypher 25 only Introduced in Neo4j 2026.02
AlterCurrentGraphTypeAlter	Alters elements of the current graph type.	Yes		Cypher 25 only Introduced in Neo4j 2026.02
AlterCurrentGraphTypeDrop	Drops elements from the current graph type.	Yes		Cypher 25 only Introduced in Neo4j 2026.02
AlterCurrentGraphTypeSet	Replaces the current graph type.	Yes		Cypher 25 only Introduced in Neo4j 2026.02
Anti	Tests for the absence of a pattern.			

Name	Description	Leaf?	Updating?	Considerations
AntiSemiApply	Performs a nested loop. Tests for the absence of a pattern predicate.			
Apply	Performs a nested loop. Yields rows from both the left-hand and right-hand side operators.			
Argument	Indicates the variable to be used as an argument to the right-hand side of an `Apply` operator.	Yes		
ArgumentTracker	Used to ensure row-by-row semantics. Restricts the Cypher runtime to not batch operations in larger chunks.	Yes		
AssertSameNode	Ensures that no node property uniqueness constraints are violated.			
AssertSameRelationship	Ensures that no relationship property uniqueness constraints are violated.			
AssertingMultiNodeIndexSeek	Used to ensure that no property uniqueness constraints are violated when creating nodes.	Yes		
AssertingMultiRelationshipIndexSeek	Used to ensure that no property uniqueness constraints are violated when creating relationships.	Yes		
CacheProperties	Reads node or relationship properties and caches them.			
CartesianProduct	Produces a cartesian product of the inputs from the left-hand and right-hand operators.			
Create	Creates nodes and relationships.		Yes	
CreateIndex	Creates an index for either nodes or relationships.		Yes	
CreateConstraint	Creates a constraint for either nodes or relationships.		Yes	
Delete	Deletes a node or relationship.		Yes	
DetachDelete	Deletes a node and its relationships.		Yes	
DirectedAllRelationshipsScan	Fetches all relationships and their start and end nodes in the database.	Yes		
DirectedRelationshipByElementIdSeek	Reads one or more relationships by element id (specified via the function elementId()) from the relationship store and produces the relationship as well as the source and target node of the relationship.	Yes		

Name	Description	Leaf?	Updating?	Considerations
DirectedRelationshipByIdSeek	Reads one or more relationships by id (specified via the function Id()) from the relationship store, and produces the relationship as well as the source and target node of the relationship.	Yes		
DirectedRelationshipIndexContainsScan	Examines all values stored in an index, searching for entries containing a specific `STRING`; for example, in queries including `CONTAINS`.	Yes		
DirectedRelationshipIndexEndsWithScan	Examines all values stored in an index, searching for entries ending in a specific `STRING`; for example, in queries containing `ENDS WITH`.	Yes		
DirectedRelationshipIndexScan	Examines all values stored in an index, returning all relationships and their start and end nodes with a particular relationship type and a specified property.	Yes		
DirectedRelationshipIndexSeek	Finds relationships and their start and end nodes using an index seek.	Yes		
DirectedRelationshipIndexSeekByRange	Finds relationships and their start and end nodes using an index seek where the value of the property matches a given prefix `STRING`.	Yes		
DirectedRelationshipTypeScan	Fetches all relationships and their start and end nodes with a specific type from the relationship type index.	Yes		
DirectedRelationshipVectorIndexSearch	Finds relationships from an index by doing a similarity search.	Yes		
DirectedUnionRelationshipTypesScan	Fetches all relationships and their start and end nodes with at least one of the provided types from the relationship type index.			
Distinct	Drops duplicate rows from the incoming stream of rows.			Eager
DoNothingIfExists(CONSTRAINT)	Checks if a constraint already exists, if it does then it stops the execution, if not it continues.	Yes		
DoNothingIfExists(INDEX)	Checks if an index already exists, if it does then it stops the execution, if not it continues.	Yes		
DropConstraint	Drops a constraint using its name.	Yes	Yes	
DropIndex	Drops an index using its name.	Yes	Yes	

Name	Description	Leaf?	Updating?	Considerations
DynamicLabelNodeLookup	Allows Cypher to use token lookup indexes when planning queries using dynamic node labels.	Yes		Introduced in Neo4j 2025.08
DynamicDirectedRelationshipTypeLookup	Allows Cypher to use token lookup indexes when planning queries using dynamic relationship types in directed relationship patterns.	Yes		Introduced in Neo4j 2025.08
DynamicUndirectedRelationshipTypeLookup	Allows Cypher to use token lookup indexes when planning queries using dynamic relationship types in undirected relationship patterns.	Yes		Introduced in Neo4j 2025.08
Eager	For isolation purposes, `Eager` ensures that operations affecting subsequent operations are executed fully for the whole dataset before continuing execution.			Eager
EagerAggregation	Evaluates a grouping expression.			Eager
EmptyResult	Eagerly loads all incoming data and discards it.			Eager
EmptyRow	Returns a single row with no columns.	Yes		
ExhaustiveLimit	The `ExhaustiveLimit` operator is similar to the `Limit` operator, but always exhausts the input. Used when combining `LIMIT` and updates.			
Expand(All)	Traverses incoming or outgoing relationships from a given node.			
Expand(Into)	Finds all relationships between two nodes.			
Filter	Filters each row coming from the child operator, only passing through rows that evaluate the predicates to `true`.			
Foreach	Performs a nested loop. Yields rows from the left-hand operator and discards rows from the right-hand operator.			
IntersectionNodeByLabelsScan	Fetches all nodes that have all of the provided labels from the node label index.	Yes		
LetAntiSemiApply	Performs a nested loop. Tests for the absence of a pattern predicate in queries containing multiple pattern predicates.			

Name	Description	Leaf?	Updating?	Considerations
LetSelectOrAntiSemiApply	Performs a nested loop. Tests for the absence of a pattern predicate that is combined with other predicates.			
LetSelectOrSemiApply	Performs a nested loop. Tests for the presence of a pattern predicate that is combined with other predicates.			
LetSemiApply	Performs a nested loop. Tests for the presence of a pattern predicate in queries containing multiple pattern predicates.			
Limit	Returns the first `n` rows from the incoming input.			
LoadCSV	Loads data from a CSV source into the query.	Yes		
LockingMerge	Similar to the `Merge` operator but will lock the start and end node when creating a relationship if necessary.			
LockNodes	Sometimes used in conjunction with the `LockingMerge` operator to lock nodes.			Introduced in Neo4j 2025.09
Merge	The `Merge` operator will either read or create nodes and/or relationships.			
MergeInto	Similar to the `Merge` operator, but is used when the start and end node of the pattern is matched outside the `MERGE` pattern.			Introduced in Neo4j 2025.11
MergeUniqueNode	Similar to the `Merge` operator, but is used when there is a property uniqueness constraint on the property used in the `MERGE` statement.			Introduced in Neo4j 2026.02
MultiNodeIndexSeek	Finds nodes using multiple index seeks.	Yes		
NodeByElementIdSeek	Reads one or more nodes by id from the node store, specified via the function elementId().	Yes		
NodeByIdSeek	Reads one or more nodes by id from the node store, specified via the function id().	Yes		
NodeByLabelScan	Fetches all nodes with a specific label from the node label index.	Yes		
NodeCountFromCountStore	Uses the count store to answer questions about node counts.	Yes		
NodeHashJoin	Executes a hash join on node ID.			Eager

Name	Description	Leaf?	Updating?	Considerations
NodeIndexContainsScan	Examines all values stored in an index, searching for entries containing a specific `STRING`.	Yes		
NodeIndexEndsWithScan	Examines all values stored in an index, searching for entries ending in a specific `STRING`.	Yes		
NodeIndexScan	Examines all values stored in an index, returning all nodes with a particular label with a specified property.	Yes		
NodeIndexSeek	Finds nodes using an index seek.	Yes		
NodeIndexSeekByRange	Finds nodes using an index seek where the value of the property matches the given prefix `STRING`.	Yes		
NodeLeftOuterHashJoin	Executes a left outer hash join.			Eager
NodeRightOuterHashJoin	Executes a right outer hash join.			Eager
NodeUniqueIndexSeek	Finds nodes using an index seek within a unique index.	Yes		
NodeUniqueIndexSeekByRange	Finds nodes using an index seek within a unique index where the value of the property matches the given prefix `STRING`.	Yes		
NodeVectorIndexSearch	Finds nodes from an index by doing a similarity search.	Yes		
NullifyMetadata	responsible for cleaning up the state produced by `Repeat`. It is only planned directly after `Repeat`.			
Optional	Yields a single row with all columns set to `null` if no data is returned by its source.			
OptionalExpand(All)	Traverses relationships from a given node, producing a single row with the relationship and end node set to `null` if the predicates are not fulfilled.			
OptionalExpand(Into)	Traverses all relationships between two nodes, producing a single row with the relationship and end node set to `null` if no matching relationships are found (the start node is the node with the smallest degree).			
OrderedAggregation	Similar to the `EagerAggregation` operator but relies on the ordering of incoming rows. It is not eager.			

Name	Description	Leaf?	Updating?	Considerations
OrderedDistinct	Similar to the `DISTINCT` operator but relies on the ordering of incoming rows.			
PartialSort	Sorts a row by multiple columns if there is already an ordering.			
PartialTop	Returns the first `n` rows sorted by multiple columns if there is already an ordering.			
PartitionedAllNodesScan	Used by the parallel runtime to read all nodes from the node store.	Yes		
PartitionedDirectedAllRelationshipsScan	Used by the parallel runtime to fetch all relationships and their start and end nodes from the database.	Yes		
PartitionedDirectedRelationshipIndexScan	Used by the parallel runtime to examine all values stored in an index. It returns all relationships with a particular type and a specified property, along with their start and end nodes.	Yes		
PartitionedDirectedRelationshipIndexSeek	Finds relationships and their start and end nodes using a parallel index seek.	Yes		
PartitionedDirectedRelationshipIndexSeekByRange	Finds relationships using a parallel index seek where the value of the of the specified relationship type property is within a given range. It also finds the start and end nodes of those relationships.	Yes		
PartitionedDirectedRelationshipTypesScan	Fetches all relationships with a specific type from the relationship type index using a parallel scan. It also fetches the start and end nodes of those relationships.	Yes		
PartitionedDirectedUnionRelationshipTypesScan	Fetches all relationships with at least one of the provided types from the relationship type index using a parallel scan. It also fetches the start and end nodes of those relationships.			
PartitionedNodeByLabelScan	Used by the parallel runtime to fetch all nodes with a specific label from the node label index.	Yes		
PartitionedNodeIndexScan	Used by the parallel runtime to examine all values stored in an index, returning all nodes with a particular label and a specified property.	Yes		

Name	Description	Leaf?	Updating?	Considerations
PartitionedNodeIndexSeek	Used by the parallel runtime to find nodes using an index seek.	Yes		
PartitionedNodeIndexSeekByRange	Finds nodes using a parallel index seek where the value of the specified property is within a given range.	Yes		
PartitionedSubtractionNodeByLabelsScan	Used by the parallel runtime to fetch all nodes that have all of the first set of provided labels and none of the second provided set of labels from the node label index.	Yes		
PartitionedUndirectedAllRelationshipsScan	Used by the parallel runtime to fetch all relationships and their start and end nodes from the database.	Yes		
PartitionedUndirectedRelationshipIndexScan	Used by the parallel runtime to examine all values stored in an index, returning all relationships with a particular relationship type and a specified property. It also returns the start and end nodes of those relationships.	Yes		
PartitionedUndirectedRelationshipIndexSeek	Finds relationships and their start and end nodes using a parallel index seek.	Yes		
PartitionedUndirectedRelationshipIndexSeekByRange	Finds relationships using a parallel index seek where the value of the specified relationship property type is within a given range. It also finds the start and end nodes of those relationships.	Yes		
PartitionedUndirectedRelationshipTypeScan	Used by the parallel runtime to fetch all relationships with a specific type from the relationship type index. It also fetches the start and end nodes of those relationships.	Yes		
PartitionedUndirectedUnionRelationshipTypesScan	Used by the parallel runtime to fetch all relationships with at least one of the provided types from the relationship type index. It also fetches the start and end nodes of those relationships.	Yes		
PartitionedUnionNodeByLabelsScan	Used by the parallel runtime to fetch all nodes that have at least one of the provided labels from the node label index.	Yes		
PartitionedUnwind	Used by the parallel runtime to return one row per item in a list.			
ProcedureCall	Calls a procedure.			

Name	Description	Leaf?	Updating?	Considerations
ProduceResults	Prepares the result so that it is consumable by the user.			
ProjectEndpoints	Projects the start and end node of a relationship.			
Projection	Evaluates a set of expressions, producing a row with the results thereof.			
RelationshipCountFromCountStore	Uses the count store to answer questions about relationship counts.	Yes		
Repeat	Solves quantified path patterns.			
RemoveLabels	Deletes labels from a node.		Yes	
RemoteBatchProperties	Fetches properties from shards for a batch of rows to use in subsequent operators.			Introduced in Neo4j 2026.01 Infinigraph only
RemoteBatchPropertiesWithFilter	Fetches properties from shards for a batch if they satisfy a filter criterion.			Introduced in Neo4j 2026.01 Infinigraph only
RemoteNodeIndexSeek	Batches nested index seeks on the right hand side of an apply.	Yes		Introduced in Neo4j 2026.07 Infinigraph only
RemoteUniqueNodeIndexSeek	Batches nested unique index seeks on the right hand side of an apply.	Yes		Introduced in Neo4j 2026.07 Infinigraph only
RollUpApply	Performs a nested loop. Executes a pattern expression or pattern comprehension.			
SelectOrAntiSemiApply	Performs a nested loop. Tests for the absence of a pattern predicate if an expression predicate evaluates to `false`.			
SelectOrSemiApply	Performs a nested loop. Tests for the presence of a pattern predicate if an expression predicate evaluates to `false`.			
SemiApply	Performs a nested loop. Tests for the presence of a pattern predicate.			
SetLabels	Sets labels on a node.		Yes	
SetNodePropertiesFromMap	Sets properties from a map on a node.		Yes	

Name	Description	Leaf?	Updating?	Considerations
SetProperty	Sets a property on a node or relationship.		Yes	
SetProperties	Used when setting multiple properties on a node or relationship.		Yes	
SetRelationshipPropertiesFromMap	Sets properties from a map on a relationship.		Yes	
ShortestPath	Finds one or all shortest paths between two previously matches node variables.			
ShowConstraints	Shows the available constraints.	Yes		
ShowCurrentGraphType	Shows the current graph type.	Yes		Cypher 25 only Introduced in Neo4j 2026.02
ShowDatabases	Shows the available databases.	Yes		Cypher 25 only Introduced in Neo4j 2026.05
ShowFunctions	Shows the available functions.	Yes		
ShowIndexes	Shows the available indexes.	Yes		
ShowProcedures	Shows the available procedures.	Yes		
ShowSettings	Shows the available configuration settings.	Yes		
ShowTransactions	Shows the available transactions on the current server.	Yes		
Skip	Skips `n` rows from the incoming rows.			
Sort	Sorts rows by a provided key.			Eager
StatefulShortestPath(All)	Finds shortest paths from a previously matched node variable to an endpoint that was not previously matched.			
StatefulShortestPath(Into)	Finds shortest paths between two previously matched node variables. It uses a bidirectional breadth-first search (BFS) algorithm, which performs two BFS invocations at the same time, one from the left boundary node and one from the right boundary node.			
SubqueryForeach	Works like the `Foreach` operator but it is only used for executing subqueries.			

Name	Description	Leaf?	Updating?	Considerations
SubtractionNodeByLabelsScan	Fetches all nodes that have all of the first set of provided labels and none of the second provided set of labels from the node label index.	Yes		
TerminateTransactions	Terminate transactions with the given IDs.	Yes		
Top	Returns the first 'n' rows sorted by a provided key.			Eager
TransactionApply	Works like the `Apply` operator but will commit the current transaction after a specified number of rows.			
TransactionForeach	Works like the `Foreach` operator but will commit the current transaction after a specified number of rows.			
TriadicBuild	Used in conjunction with `TriadicFilter` to solve triangular queries.			
TriadicFilter	Used in conjunction with `TriadicBuild` to solve triangular queries.			
TriadicSelection	Solves triangular queries, such as the very common 'find my friends-of-friends that are not already my friends'.			
UndirectedAllRelationshipsScan	Fetches all relationships and their start and end nodes in the database.	Yes		
UndirectedRelationshipByElementIdSeek	Reads one or more relationships by element id (specified via the function ElementId()) from the relationship store. As the direction is unspecified, two rows are produced for each relationship as a result of alternating the combination of the start and end node.	Yes		
UndirectedRelationshipByIdSeek	Reads one or more relationships by id (specified via the function Id()) from the relationship store. As the direction is unspecified, two rows are produced for each relationship as a result of alternating the combination of the start and end node.	Yes		
UndirectedRelationshipIndexContainsScan	Examines all values stored in an index, searching for entries containing a specific `STRING`; for example, in queries including `CONTAINS`.	Yes		

Name	Description	Leaf?	Updating?	Considerations
UndirectedRelationshipIndexEndsWithScan	Examines all values stored in an index, searching for entries ending in a specific `STRING`; for example, in queries containing `ENDS WITH`.	Yes		
UndirectedRelationshipIndexScan	Examines all values stored in an index, returning all relationships and their start and end nodes with a particular relationship type and a specified property.	Yes		
UndirectedRelationshipIndexSeek	Finds relationships and their start and end nodes using an index seek.	Yes		
UndirectedRelationshipIndexSeekByRange	Finds relationships and their start and end nodes using an index seek where the value of the property matches a given prefix `STRING`.	Yes		
UndirectedRelationshipTypeScan	Fetches all relationships and their start and end nodes with a specific type from the relationship type index.	Yes		
UndirectedRelationshipVectorIndexSearch	Finds relationships from an index by doing a similarity search.	Yes		
UndirectedUnionRelationshipTypesScan	Fetches all relationships and their start and end nodes with at least one of the provided types from the relationship type index.	Yes		
Union	Concatenates the results from the right-hand operator with the results from the left-hand operator.			
UnionNodeByLabelsScan	Fetches all nodes that have at least one of the provided labels from the node label index.	Yes		
Unwind	Returns one row per item in a list.			
ValueHashJoin	Executes a hash join on arbitrary values.			Eager
VarLengthExpand(All)	Traverses variable-length relationships from a given node.			
VarLengthExpand(Into)	Finds all variable-length relationships between two nodes.			
VarLengthExpand(Pruning)	Traverses variable-length relationships from a given node and only returns unique end nodes.			
VarLengthExpand(Pruning,BFS)	Traverses variable-length relationships from a given node and only returns unique end nodes.			

Operators in detail

Understanding the roles of different operators in an execution plan can be an important step in making queries more efficient. This page contains details and examples for each of the operators used by the Cypher planner. For an overview of how the Cypher planner uses operators, see Understanding query plans.

The operators are grouped into categories based on the role they fulfill in executing a Cypher query:

- Leaf operators (scans and seeks)
- Nested loops and join operators
- Traversal operators
- Union operators
- Aggregation operators
- Filter, order, and projection operators
- Sort and limit operators
- Data modification operators
- Schema and system operators

Certain operators are only used by particular runtime. If so, the example queries will be prefixed with a runtime selection.

Leaf operators (scans and seeks)

Leaf operators are the initial operations in a query execution plan that directly access data. They are called "leaf" because they are at the outermost level of the query execution tree, with no operations below them. Once these operators retrieve the necessary data, the results are passed to subsequent operators that perform more complex operations.

Seek operators target specific data items (indexed or otherwise) while scan operators iterate through the data (indexed or otherwise) to find matches.

All Nodes Scan

The `AllNodesScan` operator reads all nodes from the node store. The variable that will contain the nodes is seen in the arguments. Any query using this operator is likely to encounter performance problems on a non-trivial database.

Example 492. AllNodesScan

Query

```
PROFILE
MATCH (n)
RETURN n
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| Operator        | Details | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| +ProduceResults | n       |             35 |   35 |       0 |                |
|           |                     |
| |               +---------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   | n       |             35 |   35 |      36 |            120 |
3/0 |     0.354 | Fused in Pipeline 0 |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+

Total database accesses: 36, total allocated memory: 184
```

Partitioned All Nodes Scan

The `PartitionedAllNodesScan` is a variant of the `AllNodesScan` operator used by the parallel runtime. It allows the store to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 493. PartitionedAllNodesScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (n)
RETURN n
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+--------------------------+----+---------+----------------+------+---------+------------------------
+-----------+---------------+
| Operator                 | Id | Details | Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses
| Time (ms) | Pipeline      |
+--------------------------+----+---------+----------------+------+---------+------------------------
+-----------+---------------+
| +ProduceResults          |  0 | n       |             35 |   35 |      71 |                    2/0
|     6.470 | In Pipeline 1 |
| |                        +----+---------+----------------+------+---------+------------------------
+-----------+---------------+
| +PartitionedAllNodesScan |  1 | n       |             35 |   35 |      35 |                    3/0
|     1.491 | In Pipeline 0 |
+--------------------------+----+---------+----------------+------+---------+------------------------
+-----------+---------------+
```

```
Total database accesses: 106
```

Node By Label Scan

The `NodeByLabelScan` operator fetches all nodes with a specific label from the node label index.

Example 494. NodeByLabelScan

Query

```
PROFILE
MATCH (person:Person)
RETURN person
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator         | Details       | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults  | person        |             14 |   14 |       0 |                |
|           |                     |
| |                +---------------+----------------+------+---------+----------------+
|           |                     |
| +NodeByLabelScan | person:Person |             14 |   14 |      15 |            120 |
2/1 |     0.522 | Fused in Pipeline 0 |
+------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 15, total allocated memory: 184
```

Partitioned Node By Label Scan

The `PartitionedNodeByLabelScan` is a variant of the `NodeByLabelScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 495. PartitionedNodeByLabelScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (person:Person)
RETURN person
```

Query Plan

```
Planner COST
```

```
Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------+----+--------------+----------------+------+---------
+------------------------+-----------+---------------+
| Operator                    | Id | Details       | Estimated Rows | Rows | DB Hits | Page Cache
Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------+----+--------------+----------------+------+---------
+------------------------+-----------+---------------+
| +ProduceResults             |  0 | person        |             14 |   14 |      28 |
2/0 |     0.623 | In Pipeline 1 |
| |                           +----+---------------+----------------+------+---------
+------------------------+-----------+---------------+
| +PartitionedNodeByLabelScan |  1 | person:Person |             14 |   14 |      15 |
1/0 |     0.094 | In Pipeline 0 |
+-----------------------------+----+--------------+----------------+------+---------
+------------------------+-----------+---------------+

Total database accesses: 43
```

Intersection Node By Labels Scan

The `IntersectionNodeByLabelsScan` operator fetches all nodes that have all of the provided labels from the node label index.

Query

```
PROFILE
MATCH (countryAndLocation:Country&Location)
RETURN countryAndLocation
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128


+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+
| Operator                    | Id | Details                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+
| +ProduceResults             |  0 | countryAndLocation                  |             10 |    0 |
0 |                |                        |           |                     |
| |                           +----+-------------------------------------+----------------+------
+---------+----------------+                        |           |                     |
| +IntersectionNodeByLabelsScan |  1 | countryAndLocation:Country&Location |             10 |    0 |
0 |            120 |                    0/0 |     1.011 | Fused in Pipeline 0 |
+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+

Total database accesses: 13, total allocated memory: 184
```

Partitioned Intersection Node By Labels Scan

The `PartitionedIntersectionNodeByLabelsScan` is a variant of the `IntersectionNodeByLabelsScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (countryAndLocation:Country&Location)
RETURN countryAndLocation
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128


-----------------------------------------+----+-------------------------------------+
---------------+------+---------+----------------------+-----------+---------------+
| Operator                                | Id | Details                             | Estimated
Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+----------------------------------------+----+-------------------------------------
+---------------+------+---------+----------------------+-----------+---------------+
| +ProduceResults                         |  0 | countryAndLocation                  |
3 |    0 |       0 |                  0/0 |     0.018 | In Pipeline 1 |
| |                                       +----+-------------------------------------
+---------------+------+---------+----------------------+-----------+---------------+
| +PartitionedIntersectionNodeByLabelsScan |  1 | countryAndLocation:Country&Location |
3 |    0 |      13 |                  2/0 |     0.770 | In Pipeline 0 |
+-----------------------------------------+----+-------------------------------------
+---------------+------+---------+----------------------+-----------+---------------+

Total database accesses: 13
```

Subtraction Node By Labels Scan

The `SubtractionNodeByLabelsScan` operator fetches all nodes that have all of the first set of provided labels and none of the second provided set of labels from the node label index. In the example below, `SubtractionNodeByLabelsScan` retrieves all nodes that have the `Location` label but do not have the `Country` label.

Query

```
PROFILE
MATCH (locationNotCountry:Location&!Country)
RETURN locationNotCountry
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+-----------------------+----------+--------------------+
| Operator                    | Id | Details                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+-----------------------+----------+--------------------+
| +ProduceResults             |  0 | locationNotCountry                  |              7 |   10 |
0 |              0 |                       |          |                    |
| |                           +----+-------------------------------------+----------------+------
+---------+----------------+                       |          |                    |
| +SubtractionNodeByLabelsScan |  1 | locationNotCountry:Location&!Country |              7 |   10 |
13 |            248 |                   2/0 |     3.081 | Fused in Pipeline 0 |
+-----------------------------+----+-------------------------------------+----------------+------
+---------+----------------+-----------------------+----------+--------------------+

Total database accesses: 13, total allocated memory: 312
```

Partitioned Subtraction Node By Labels Scan

The `PartitionedSubtractionNodeByLabelsScan` is a variant of the `SubtractionNodeByLabelsScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (locationNotCountry:Location&!Country)
RETURN locationNotCountry
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+----------------------------------------+----+-------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------+
| Operator                               | Id | Details                             | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+----------------------------------------+----+-------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------+
| +ProduceResults                        |  0 | locationNotCountry                  |
7 |   10 |       0 |            136 |                   0/0 |     0.614 | In Pipeline 1 |
| |                                      +----+-------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------+
| +PartitionedSubtractionNodeByLabelsScan |  1 | locationNotCountry:Location&!Country |
7 |   10 |      13 |            120 |                   2/0 |     5.173 | In Pipeline 0 |
+----------------------------------------+----+-------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------+

Total database accesses: 13, total allocated memory: 262144
```

Union Node By Labels Scan

The `UnionNodeByLabelsScan` operator fetches all nodes that have at least one of the provided labels from the node label index.

Query

```
PROFILE
MATCH (countryOrLocation:Country|Location)
RETURN countryOrLocation
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-------------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+----------------------+---------------------+
| Operator              | Details                             | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by           | Pipeline            |
+-----------------------+-------------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+----------------------+---------------------+
| +ProduceResults       | countryOrLocation                   |             17 |   11 |       0 |
|                       |           |                       |                |                     |
| |                     +-----------------------------------+----------------+------+---------
+----------------+                      |           |                      |                     |
| +UnionNodeByLabelsScan | countryOrLocation:Country|Location |             17 |   11 |      13 |
120 |                     3/1 |     0.660 | countryOrLocation ASC | Fused in Pipeline 0 |
+-----------------------+-------------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+----------------------+---------------------+

Total database accesses: 13, total allocated memory: 184
```

Partitioned Union Node By Labels Scan

The `PartitionedUnionNodeByLabelsScan` is a variant of the `UnionNodeByLabelsScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (countryOrLocation:Country|Location)
RETURN countryOrLocation
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+----------------------------------+----+-------------------------------------+---------------
+------+---------+-----------------------+-----------+---------------+
| Operator                         | Id | Details                             | Estimated Rows | Rows
```

```
| DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------+----+-----------------------------------+---------------
+------+---------+----------------------+-----------+---------------+
| +ProduceResults                   |  0 | countryOrLocation                 |             17 |   11
|      22 |                   2/0 |     1.548 | In Pipeline 1 |
| |                                 +----+-----------------------------------+---------------
+------+---------+----------------------+-----------+---------------+
| +PartitionedUnionNodeByLabelsScan |  1 | countryOrLocation:Country|Location |             17 |   11
|      13 |                   2/0 |     1.976 | In Pipeline 0 |
+-----------------------------------+----+-----------------------------------+---------------
+------+---------+----------------------+-----------+---------------+

Total database accesses: 35
```

Node Index Contains Scan

The `NodeIndexContainsScan` operator examines all values stored in an index, searching for entries containing a specific `STRING`; for example, in queries including `CONTAINS`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a label scan using `NodeByLabelScan`, and a property store filter.

Example 496. NodeIndexContainsScan

Query

```
PROFILE
MATCH (l:Location)
WHERE l.name CONTAINS 'al'
RETURN l
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------------+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator               | Details                                                          | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------------+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults        | l                                                                |
0 |    2 |       0 |                |                        |           |                     |
| |                      +------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexContainsScan | TEXT INDEX l:Location(name) WHERE name CONTAINS $autostring_0 |
0 |    2 |       3 |            120 |                    2/0 |     1.305 | Fused in Pipeline 0 |
+------------------------+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 3, total allocated memory: 184
```

Node Index Ends With Scan

The `NodeIndexEndsWithScan` operator examines all values stored in an index, searching for entries ending in a specific `STRING`; for example, in queries containing `ENDS WITH`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a label scan using `NodeByLabelScan`, and a property store filter.

Example 497. NodeIndexEndsWithScan

Query

```
PROFILE
MATCH (l:Location)
WHERE l.name ENDS WITH 'al'
RETURN l
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator               | Details                                                                 | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults        | l                                                                       |
0 |    0 |       0 |                |                          |           |                     |
| |                       +-------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexEndsWithScan | TEXT INDEX l:Location(name) WHERE name ENDS WITH $autostring_0 |
0 |    0 |       1 |            120 |                     0/0 |     4.409 | Fused in Pipeline 0 |
+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 1, total allocated memory: 184
```

Node Index Scan

The `NodeIndexScan` operator examines all values stored in an index, returning all nodes with a particular label and a specified property.

Example 498. NodeIndexScan

Query

```
PROFILE
MATCH (l:Location)
WHERE l.name IS NOT NULL
RETURN l
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| Operator        | Details                                            | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults | l                                                  |             10 |   10 |
0 |                |                         |           |                    |
| |               +----------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +NodeIndexScan  | RANGE INDEX l:Location(name) WHERE name IS NOT NULL |             10 |   10 |
11 |            120 |                    2/1 |     0.557 | Fused in Pipeline 0 |
+-----------------+----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 11, total allocated memory: 184
```

Partitioned Node Index Scan

The `PartitionedNodeIndexScan` is a variant of the `NodeIndexScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 499. PartitionedNodeIndexScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (l:Location)
WHERE l.name IS NOT NULL
RETURN l
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+--------------------------+----+-----------------------------------------------------
+----------------+------+---------+-----------------------+-----------+----------------+
| Operator                 | Id | Details                                              | Estimated
Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+--------------------------+----+-----------------------------------------------------
+----------------+------+---------+-----------------------+-----------+----------------+
| +ProduceResults          |  0 | l                                                    |
1 |   10 |      20 |                    2/0 |     0.472 | In Pipeline 1 |
| |                        +----+-----------------------------------------------------
+----------------+------+---------+-----------------------+-----------+----------------+
| +PartitionedNodeIndexScan |  1 | RANGE INDEX l:Location(name) WHERE name IS NOT NULL |
1 |   10 |      11 |                    1/0 |     0.187 | In Pipeline 0 |
+--------------------------+----+-----------------------------------------------------
+----------------+------+---------+-----------------------+-----------+----------------+
```

```
Total database accesses: 31
```

Node By ElementId Seek

The `NodeByElementIdSeek` operator reads one or more nodes by ID from the node store, specified via the function elementId().

Example 500. NodeByElementIdSeek

Query

```
PROFILE
MATCH (n)
WHERE elementId(n) = 0
RETURN n
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-----------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| Operator              | Details                           | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------------+-----------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| +ProduceResults       | n                                 |              1 |    1 |       0 |
|                       |           |                       |
| |                     +-----------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------------
|           |                     |
| +NodeByElementIdSeek  | n WHERE elementId(n) = $autoint_0 |              1 |    1 |       1 |
120 |                     3/0 |     2.108 | Fused in Pipeline 0 |
+-----------------------+-----------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------------+

Total database accesses: 1, total allocated memory: 184
```

Node By Id Seek

The `NodeByIdSeek` operator reads one or more nodes by id from the node store, specified via the function id().

Example 501. NodeByIdSeek

Query

```
PROFILE
MATCH (n)
WHERE id(n) = 0
RETURN n
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+---------------------------+---------------+------+---------
+---------------+-----------------------+-----------+--------------------+
| Operator        | Id | Details                   | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+----+---------------------------+---------------+------+---------
+---------------+-----------------------+-----------+--------------------+
| +ProduceResults |  0 | n                         |             1 |    1 |       2 |
0 |                       |           |                    |
| |               +----+---------------------------+---------------+------+---------
+---------------+                        |           |                    |
| +NodeByIdSeek   |  1 | n WHERE id(n) = $autoint_0 |             1 |    1 |       1 |
248 |                    2/0 |     1.109 | Fused in Pipeline 0 |
+-----------------+----+---------------------------+---------------+------+---------
+---------------+-----------------------+-----------+--------------------+

Total database accesses: 3, total allocated memory: 312
```

Node Index Seek

The `NodeIndexSeek` operator finds nodes using an index seek. The node variable and the index used are shown in the arguments of the operator. If the index is backs up a property uniqueness constraint, the operator is instead called NodeUniqueIndexSeek.

Example 502. NodeIndexSeek

Query

```
PROFILE
MATCH (location:Location {name: 'Malmo'})
RETURN location
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| Operator        | Details                                                               | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+-----------------------------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults | location                                                              |              1 |
1 |       0 |               |                       |           |                    |
| |               +-----------------------------------------------------------------------+---------------
+------+---------+---------------+                       |           |                    |
| +NodeIndexSeek  | RANGE INDEX location:Location(name) WHERE name = $autostring_0 |              1 |
1 |       2 |           120 |                   2/1 |     0.401 | Fused in Pipeline 0 |
+-----------------+-----------------------------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------+
```

```
Total database accesses: 2, total allocated memory: 184
```

Partitioned Node Index Seek

The `PartitionedNodeIndexSeek` is a variant of the `NodeIndexSeek` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 503. PartitionedNodeIndexSeek

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (location:Location {name: 'Malmo'})
RETURN location
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+---------------------------+----+-------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------+
| Operator                  | Id | Details                                                                |
Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+---------------------------+----+-------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------+
| +ProduceResults           |  0 | location                                                               |
0 |    1 |       2 |                     2/0 |     0.179 | In Pipeline 1 |
| |                         +----+-------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------+
| +PartitionedNodeIndexSeek |  1 | RANGE INDEX location:Location(name) WHERE name = $autostring_0 |
0 |    1 |       2 |                     1/0 |     0.167 | In Pipeline 0 |
+---------------------------+----+-------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------+

Total database accesses: 4
```

Node Unique Index Seek

The `NodeUniqueIndexSeek` operator finds nodes using an index seek within an index backing up a property uniqueness constraint. The node variable and the index used are shown in the arguments of the operator. If the index does not back up a property uniqueness constraint, the operator is instead called NodeIndexSeek. If the index seek is used to solve a MERGE clause, it will also be marked with `(Locking)`. This makes it clear that any nodes returned from the index will be locked in order to prevent concurrent conflicting updates.

Example 504. NodeUniqueIndexSeek

Query

```
PROFILE
```

```
MATCH (t:Team {name: 'Malmo'})
RETURN t
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+-----------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+
| Operator             | Details                                       | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+----------------------+-----------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+
| +ProduceResults      | t                                             |              1 |    0 |
0 |                |                        |           |                    |
| |                    +-----------------------------------------------+----------------+------
+---------+---------------+                                |           |                    |
| +NodeUniqueIndexSeek | UNIQUE t:Team(name) WHERE name = $autostring_0 |              1 |    0 |
1 |            120 |                    0/1 |     0.280 | Fused in Pipeline 0 |
+----------------------+-----------------------------------------------+----------------+------
+---------+---------------+------------------------+-----------+--------------------+

Total database accesses: 1, total allocated memory: 184
```

Multi Node Index Seek

The `MultiNodeIndexSeek` operator finds nodes using multiple index seeks. It supports using multiple distinct indexes for different nodes in the query. The node variables and the indexes used are shown in the arguments of the operator.

The operator yields a cartesian product of all index seeks. For example, if the operator does two seeks and the first seek finds the nodes `a1, a2` and the second `b1, b2, b3`, the `MultiNodeIndexSeek` will yield the rows `(a1, b1), (a1, b2), (a1, b3), (a2, b1), (a2, b2), (a2, b3)`.

Example 505. MultiNodeIndexSeek

Query

```
PROFILE
MATCH
  (location:Location {name: 'Malmo'}),
  (person:Person {name: 'Bob'})
RETURN location, person
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------+------------------------------------------------------------------
+---------------+------+---------+---------------+----------------------+-----------
+--------------------+
```

```
| Operator           | Details                                                                  | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+--------------------+--------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults     | location, person                                                         |
1 |    1 |       0 |               |                         |           |                  |
| |                   +--------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +MultiNodeIndexSeek | RANGE INDEX location:Location(name) WHERE name = $autostring_0, |
1 |    0 |       0 |           120 |                     2/2 |     1.910 | Fused in Pipeline 0 |
|                     | RANGE INDEX person:Person(name) WHERE name = $autostring_1      |
|      |         |               |                           |           |                    |
+--------------------+--------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 0, total allocated memory: 184
```

Asserting Multi Node Index Seek

The `AssertingMultiNodeIndexSeek` operator is used to ensure that no property uniqueness constraints are violated when creating nodes. The example looks for the presence of a team with the supplied name and id, and if one does not exist, it will be created. Owing to the existence of two property uniqueness constraints on `:Team(name)` and `:Team(id)`, any node that would be found by the `UniqueIndexSeek` operator must be the very same node or the constraints would be violated.

Example 506. AssertingMultiNodeIndexSeek

Query

```
PROFILE
MERGE (t:Team {name: 'Engineering', id: 42})
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                    | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults              |
|              1 |    0 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +EmptyResult                 |
|              1 |    0 |       0 |                |                        |           |
|
| |
```

```
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Merge                       | CREATE (t:Team {name: $autostring_0, id: $autoint_1})
|              1 |    1 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +AssertingMultiNodeIndexSeek | UNIQUE t:Team(name) WHERE name = $autostring_0, UNIQUE t:Team(id)
WHERE id = $autoint_1 |              0 |    2 |       4 |            120 |                    0/2 |
1.584 | Fused in Pipeline 0 |
+------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 4, total allocated memory: 184
```

Asserting Multi Relationship Index Seek

The `AssertingMultiRelationshipIndexSeek` operator is used to ensure that no property uniqueness constraints are violated when creating relationships. The example looks for the presence of a relationship with specified `name` and `id` properties.

Example 507. AssertingMultiRelationshipIndexSeek

Query

```
PROFILE
MERGE (person)-[work:WORKS_IN {id: 0, badgeNumber: 4332}]->(location)
```

Query Plan

```
+--------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+-----------------------------------------------+
| Operator                             | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used                                 |
+--------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+-----------------------------------------------+
| +ProduceResults                      |  0 |
|              1 |    0 |       0 |              0 |                        |           |
|                                      |
| |                                    +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
+-------------------------------------------+
| +EmptyResult                         |  1 |
|              1 |    0 |       0 |                |                        |           |
|                                      |
| |                                    +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
+-------------------------------------------+
| +Merge                               |  2 | CREATE (person), (location), (person)-[work:WORKS_IN
{id: $autoint_0, badgeNumber: $autoint_1}]->(lo |              1 |    1 |       0 |               |
|           |                      |                                             |
| |                                    |    | cation)
|                |      |         |                |                        |           |
|                                      |
| |                                    +----
+-----------------------------------------------------------------------------------------------------
```

```
--+---------------+------+---------+---------------+                        |           |
+-----------------------------------------+
| +AssertingMultiRelationshipIndexSeek |  3 | RANGE INDEX (person)-[work:WORKS_IN(id)]->(location)
WHERE id = $autoint_0,                          |             0 |    2 |       4 |            248 |
3/0 |     0,393 | Fused in Pipeline 0 | constraint_WORKS_IN_badgeNumber_unique: 1 |
|                                     |    | RANGE INDEX (person)-[work:WORKS_IN(badgeNumber)]-
>(location) WHERE badgeNumber = $autoint_1         |               |      |         |
|                         |           |                     | constraint_WORKS_IN_id_unique: 1
|
+-------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+---------------+------+---------+---------------+------------------------+-----------+
---------------------+-----------------------------------------+
```

Node Index Seek By Range

The `NodeIndexSeekByRange` operator finds nodes using an index seek where the value of the property matches a given prefix `STRING`. `NodeIndexSeekByRange` can be used for `STARTS WITH` and comparison operators such as `<`, `>`, `<=` and `>=`. If the index is a unique index, the operator is instead called `NodeUniqueIndexSeekByRange`.

Example 508. NodeIndexSeekByRange

Query

```
PROFILE
MATCH (l:Location)
WHERE l.name STARTS WITH 'Lon'
RETURN l
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+---------------+------------------------+-----------
+---------------------+
| Operator              | Details                                                                 |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+---------------+------------------------+-----------
+---------------------+
| +ProduceResults       | l                                                                       |
2 |    1 |       0 |               |                        |           |                    |
| |                     +-------------------------------------------------------------------
+----------------+------+---------+---------------+                        |           |
|
| +NodeIndexSeekByRange | RANGE INDEX l:Location(name) WHERE name STARTS WITH $autostring_0 |
2 |    1 |       2 |           120 |                    3/0 |     0.825 | Fused in Pipeline 0 |
+-----------------------+-----------------------------------------------------------------------
+----------------+------+---------+---------------+------------------------+-----------
+---------------------+

Total database accesses: 2, total allocated memory: 184
```

Partitioned Node Index Seek By Range

The `PartitionedNodeIndexSeekByRange` is a variant of the `NodeIndexSeekByRange` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 509. PartitionedNodeIndexSeekByRange

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (l:Location)
WHERE l.name STARTS WITH 'Lon'
RETURN l
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+----------------------------------+----
+-------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| Operator                         | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+----------------------------------+----
+-------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +ProduceResults                  |  0 | l
|              0 |    1 |       2 |                    2/0 |     0.191 | In Pipeline 1 |
| |                                |----
+-------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +PartitionedNodeIndexSeekByRange |  1 | RANGE INDEX l:Location(name) WHERE name STARTS WITH
$autostring_0 |              0 |    1 |       2 |                    1/0 |     0.087 | In Pipeline 0
|
+----------------------------------+----
+-------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+

Total database accesses: 4
```

Node Unique Index Seek By Range

The `NodeUniqueIndexSeekByRange` operator finds nodes using an index seek within a unique index, where the value of the property matches a given prefix `STRING`. `NodeUniqueIndexSeekByRange` is used by `STARTS WITH` and comparison operators such as `<`, `>`, `<=`, and `>=`. If the index is not unique, the operator is instead called `NodeIndexSeekByRange`.

Example 510. NodeUniqueIndexSeekByRange

Query

```
PROFILE
MATCH (t:Team)
WHERE t.name STARTS WITH 'Ma'
RETURN t
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------+-------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator                   | Details                                                    | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+----------------------------+-------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults            | t                                                          |
2 |    0 |       0 |               |                       |           |                   |
| |                          +------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +NodeUniqueIndexSeekByRange | UNIQUE t:Team(name) WHERE name STARTS WITH $autostring_0 |
2 |    0 |       1 |           120 |                   1/0 |     0.623 | Fused in Pipeline 0 |
+----------------------------+-------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 1, total allocated memory: 184
```

Node Vector Index Search

Introduced in 2026.01

The `NodeVectorIndexSearch` operator searches a vector index for similar vector embeddings.

Example 511. NodeVectorIndexSearch

Query

```
PROFILE
MATCH (movie:Movie)
  SEARCH movie IN (
    VECTOR INDEX moviePlots
    FOR vector([1, 2, 3], 3, INTEGER)
    LIMIT 4
  )
RETURN movie.title AS title
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+---------------+-----------------------+-----------+
---------------------+---------------+
| Operator               | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
```

```
| Indexes Used  |
+-----------------------+----
+-------------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+---------------+
| +ProduceResults        |  0 | title
|             10 |    0 |       0 |              0 |                        |           |
|               |
| |                      +----
+-------------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
+---------------+
| +Projection            |  1 | movie.title AS title
|             10 |    0 |       0 |                |                        |           |
|               |
| |                      +----
+-------------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
+---------------+
| +NodeVectorIndexSearch |  2 | SEARCH movie IN (VECTOR INDEX moviePlots FOR
RuntimeConstant(vector($autolist_0, $autoint_1, INTEGER |             10 |    0 |       1 |
248 |                    0/0 |     1.364 | Fused in Pipeline 0 | moviePlots: 1 |
|                      |    |  NOT NULL)) LIMIT 4)
|                |      |         |                |                        |           |
|               |
+-----------------------+----
+-------------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+---------------+

Total database accesses: 1, total allocated memory: 312
```

Directed All Relationships Scan

The `DirectedAllRelationshipsScan` operator fetches all relationships and their start and end nodes in the database.

Query

```
PROFILE
MATCH ()-[r]->() RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------+-----------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| Operator                      | Details               | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------------------+-----------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +ProduceResults               | r                     |             28 |   28 |       0 |
|                         |           |                     |
| |                             +-----------------------+----------------+------+---------
+----------------+                         |           |                    |
| +DirectedAllRelationshipsScan | (anon_0)-[r]->(anon_1) |             28 |   28 |      28 |
120 |                    3/0 |     0.502 | Fused in Pipeline 0 |
+-------------------------------+-----------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+

Total database accesses: 28, total allocated memory: 184
```

Partitioned Directed All Relationships Scan

The `PartitionedDirectedAllRelationshipsScan` is a variant of the `DirectedAllRelationshipsScan` operator used by the parallel runtime. It allows the store to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r]->() RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

| Operator                                | Id | Details                | Estimated Rows | Rows | DB
Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------+----+------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +ProduceResults                         |  0 | r                      |             28 |   28 |
83 |                   2/0 |     3.872 | In Pipeline 1 |
| |                                       +----+------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +PartitionedDirectedAllRelationshipsScan |  1 | (anon_0)-[r]->(anon_1) |             28 |   28 |
28 |                   3/0 |     1.954 | In Pipeline 0 |
+-----------------------------------------+----+------------------------+----------------+------
+---------+-----------------------+-----------+---------------+

Total database accesses: 111
```

Undirected All Relationships Scan

The `UndirectedAllRelationshipsScan` operator fetches all relationships and their start and end nodes in the database.

Query

```
PROFILE
MATCH ()-[r]-() RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------------------+----------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator                        | Details              | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
```

```
+---------------------------------+----------------------+----------------+------+---------
+----------------+----------------------+-----------+--------------------+
| +ProduceResults                 | r                    |             56 |   56 |       0 |
|                          |           |                     |
| |                               +----------------------+----------------+------+---------
+----------------+                        |          |                    |
| +UndirectedAllRelationshipsScan | (anon_0)-[r]-(anon_1) |             56 |   56 |      28 |
120 |                    3/0 |     1.110 | Fused in Pipeline 0 |
+---------------------------------+----------------------+----------------+------+---------
+----------------+----------------------+-----------+--------------------+

Total database accesses: 28, total allocated memory: 184
```

Partitioned Undirected All Relationships Scan

The `PartitionedUndirectedAllRelationshipsScan` is a variant of the `UndirectedAllRelationshipsScan` operator used by the parallel runtime. It allows the store to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r]-() RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----+----------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| Operator                                | Id | Details              | Estimated Rows | Rows |
DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------+----+----------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +ProduceResults                         |  0 | r                    |             56 |   56 |
166 |                   2/0 |     4.905 | In Pipeline 1 |
| |                                       +----+----------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +PartitionedUndirectedAllRelationshipsScan |  1 | (anon_0)-[r]-(anon_1) |             56 |   56 |
28 |                   9/0 |     1.573 | In Pipeline 0 |
+-----------------------------------------+----+----------------------+----------------+------
+---------+-----------------------+-----------+---------------+

Total database accesses: 194
```

Directed Relationship Type Scan

The `DirectedRelationshipTypeScan` operator fetches all relationships and their start and end nodes with a specific type from the relationship type index.

Example 512. DirectedRelationshipTypeScan

Query

```
PROFILE
MATCH ()-[r: FRIENDS_WITH]->()
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------+-----------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| Operator                      | Details                           | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------------------+-----------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| +ProduceResults               | r                                 |             12 |   12 |
0 |                |                       |           |                    |
| |                             +-----------------------------------+----------------+------
+---------+----------------+                       |           |                    |
| +DirectedRelationshipTypeScan | (anon_0)-[r:FRIENDS_WITH]->(anon_1) |             12 |   12 |
13 |            120 |                     2/1 |     0.557 | Fused in Pipeline 0 |
+-------------------------------+-----------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+

Total database accesses: 13, total allocated memory: 184
```

Partitioned Directed Relationship Type Scan

The `PartitionedDirectedRelationshipTypeScan` is a variant of the `DirectedRelationshipTypeScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 513. PartitionedDirectedRelationshipTypeScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r: FRIENDS_WITH]->()
RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

++-----------------------------------------+----+-----------------------------------
+---------------+------+---------+-----------------------+-----------+---------------+
 | Operator                                | Id | Details                           | Estimated
```

```
Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
 +----------------------------------------+----+-----------------------------------
+---------------+------+---------+----------------------+-----------+---------------+
 | +ProduceResults                          |  0 | r                                 |
12 |   12 |      12 |                   0/0 |     0.560 | In Pipeline 1 |
 | |                                        +----+-----------------------------------
+---------------+------+---------+----------------------+-----------+---------------+
 | +PartitionedDirectedRelationshipTypeScan |  1 | (anon_0)-[r:FRIENDS_WITH]->(anon_1) |
12 |   12 |      13 |                   2/0 |     0.167 | In Pipeline 0 |
 +----------------------------------------+----+-----------------------------------
+---------------+------+---------+----------------------+-----------+---------------+

 Total database accesses: 25
```

Undirected Relationship Type Scan

The `UndirectedRelationshipTypeScan` operator fetches all relationships and their start and end nodes with a specific type from the relationship type index.

Query

```
PROFILE
MATCH ()-[r: FRIENDS_WITH]-()
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------------------+-----------------------------------+----------------+------
+---------+---------------+----------------------+-----------+---------------------+
| Operator                        | Details                           | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+---------------------------------+-----------------------------------+----------------+------
+---------+---------------+----------------------+-----------+---------------------+
| +ProduceResults                 | r                                 |             24 |   24 |
0 |               |                      |           |                     |
| |                               +-----------------------------------+----------------+------
+---------+---------------+                      |           |                      |
| +UndirectedRelationshipTypeScan | (anon_0)-[r:FRIENDS_WITH]-(anon_1) |             24 |   24 |
13 |           120 |                   2/1 |     0.749 | Fused in Pipeline 0 |
+---------------------------------+-----------------------------------+----------------+------
+---------+---------------+----------------------+-----------+---------------------+

Total database accesses: 13, total allocated memory: 184
```

Partitioned Undirected Relationship Type Scan

The `PartitionedUndirectedRelationshipTypeScan` is a variant of the `UndirectedRelationshipTypeScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r: FRIENDS_WITH]-()
RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----+-----------------------------------
+---------------+------+---------+----------------------+----------+---------------+
| Operator                                | Id | Details                           | Estimated
Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------+----+-----------------------------------
+---------------+------+---------+----------------------+----------+---------------+
| +ProduceResults                         |  0 | r                                 |
24 |   24 |      24 |                  1/0 |    1.466 | In Pipeline 1 |
| |                                       +----+-----------------------------------
+---------------+------+---------+----------------------+----------+---------------+
| +PartitionedUndirectedRelationshipTypeScan |  1 | (anon_0)-[r:FRIENDS_WITH]-(anon_1) |
24 |   24 |      13 |                  2/0 |    0.171 | In Pipeline 0 |
+-----------------------------------------+----+-----------------------------------
+---------------+------+---------+----------------------+----------+---------------+

Total database accesses: 37
```

Directed Relationship Index Search

Introduced in 2026.01

The `DirectedRelationshipVectorIndexSearch` operator searches a vector index for similar vector embeddings.

Example 514. DirectedRelationshipVectorIndexSearch

Query

```
PROFILE
MATCH ()-[r:REVIEWED]->()
  SEARCH r IN (
    VECTOR INDEX reviews
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE r.rating >= 3
    LIMIT 4
  )
RETURN r.content AS content, r.rating AS rating
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128


+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+
| +ProduceResults                         |  0 | content, rating
|              1 |    0 |       0 |              0 |                        |           |
|              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +Projection                             |  1 | cache[r.content] AS content, cache[r.rating] AS
rating                                            |              1 |    0 |       0 |
|                        |           |                      |              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +CacheProperties                        |  2 | cache[r.content], cache[r.rating]
|              1 |    0 |       0 |                |                        |           |
|              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +DirectedRelationshipVectorIndexSearch |  3 | SEARCH ()-[r:REVIEWED]->() IN (VECTOR INDEX reviews
FOR RuntimeConstant(vector($autolist_0, $autoint |              1 |    0 |       1 |            248 |
0/0 |     2.126 | Fused in Pipeline 0 | reviews: 1   |
|                                         |    | _1, INTEGER NOT NULL)) WHERE rating >= $autoint_2
LIMIT 4)                                          |                |      |         |
|                        |           |                      |              |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+

Total database accesses: 1, total allocated memory: 312
```

Undirected Relationship Index Search

Introduced in 2026.01

The `UndirectedRelationshipVectorIndexSearch` operator searches a vector index for similar vector embeddings.

Example 515. UndirectedRelationshipVectorIndexSearch

Query

```
PROFILE
MATCH ()-[r:REVIEWED]-()
  SEARCH r IN (
    VECTOR INDEX reviews
    FOR vector([1, 2, 3], 3, INTEGER)
    WHERE r.rating >= 3
    LIMIT 4
  )
```

```
RETURN r.content AS content, r.rating AS rating
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+
| +ProduceResults                         |  0 | content, rating
|              2 |    0 |       0 |              0 |                        |           |
|              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +Projection                             |  1 | cache[r.content] AS content, cache[r.rating] AS
rating                                             |              2 |    0 |       0 |
|                         |           |                    |              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +CacheProperties                        |  2 | cache[r.content], cache[r.rating]
|              2 |    0 |       0 |                |                        |           |
|              |
| |                                       +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
+--------------+
| +UndirectedRelationshipVectorIndexSearch |  3 | SEARCH ()-[r:REVIEWED]-() IN (VECTOR INDEX reviews
FOR RuntimeConstant(vector($autolist_0, $autoint_ |              2 |    0 |       1 |           248
|                    0/0 |     1.298 | Fused in Pipeline 0 | reviews: 1   |
|                                         |    | 1, INTEGER NOT NULL)) WHERE rating >= $autoint_2
LIMIT 4)                                           |                |      |         |
|                         |           |                    |              |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+--------------+

Total database accesses: 1, total allocated memory: 312
```

Directed Union Relationship Types Scan

The `DirectedUnionRelationshipTypesScan` operator fetches all relationships and their start and end nodes with at least one of the provided types from the relationship type index.

As the block storage format becomes the default, this operator will cease to be used in generating plans. Please refer to Operations Manual → Store formats for futher details on the various store formats available.

Example 516. DirectedUnionRelationshipTypesScan

Query

```
PROFILE
MATCH ()-[friendOrFoe: FRIENDS_WITH|FOE]->()
RETURN friendOrFoe
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------------+-------------------------------------------------
+---------------+------+--------+---------------+-----------------------+-----------
+-----------------+--------------------+
| Operator                          | Details                                                 | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by      |
Pipeline            |
+-----------------------------------+-------------------------------------------------
+---------------+------+--------+---------------+-----------------------+-----------
+-----------------+--------------------+
| +ProduceResults                   | friendOrFoe                                             |
15 |   12 |       0 |               |                       |           |                 |
|
| |                                  +-------------------------------------------------
+---------------+------+--------+---------------+                       |           |
|                    |
| +DirectedUnionRelationshipTypesScan | (anon_0)-[friendOrFoe:FRIENDS_WITH|FOE]->(anon_1) |
15 |   12 |      14 |            120 |                     3/1 |     2.027 | friendOrFoe ASC | Fused
in Pipeline 0 |
+-----------------------------------+-------------------------------------------------
+---------------+------+--------+---------------+-----------------------+-----------
+-----------------+--------------------+

Total database accesses: 14, total allocated memory: 184
```

Partitioned Directed Union Relationship Types Scan

The `PartitionedDirectedUnionRelationshipTypeScan` is a variant of the `DirectedUnionRelationshipTypesScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

As the block storage format becomes the default, this operator will cease to be used in generating plans. Please refer to Operations Manual → Store formats for futher details on the various store formats available.

Example 517. PartitionedDirectedUnionRelationshipTypesScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[friendOrFoe: FRIENDS_WITH|FOE]->()
RETURN friendOrFoe
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| Operator                                        | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| +ProduceResults                                 |  0 | friendOrFoe
|             15 |   12 |      12 |                    0/0 |     0.570 | In Pipeline 1 |
| |                                               +----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| +PartitionedDirectedUnionRelationshipTypesScan |  1 | (anon_0)-[friendOrFoe:FRIENDS_WITH|FOE]-
>(anon_1) |             15 |   12 |      13 |                    2/0 |     0.170 | In Pipeline 0 |
+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+

Total database accesses: 25
```

Undirected Union Relationship Types Scan

The `UndirectedUnionRelationshipTypesScan` operator fetches all relationships and their start and end nodes with at least one of the provided types from the relationship type index.

As the block storage format becomes the default, this operator will cease to be used in generating plans. Please refer to Operations Manual → Store formats for futher details on the various store formats available.

Example 518. UndirectedUnionRelationshipTypeScan

Query

```
PROFILE
MATCH ()-[friendOrFoe: FRIENDS_WITH|FOE]-()
RETURN friendOrFoe
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------------+-----------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+------------------+---------------------+
| Operator                            | Details                                       |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline           |
```

```
+-----------------------------------+-----------------------------------------------------
+---------------+------+--------+---------------+----------------------+----------
+----------------+--------------------+
| +ProduceResults                   | friendOrFoe                                       |
30 |   24 |       0 |               |                      |          |                |
|
| |                                 +---------------------------------------------------
+----------------+------+--------+---------------+                      |          |
|                    |
| +UndirectedUnionRelationshipTypesScan | (anon_0)-[friendOrFoe:FRIENDS_WITH|FOE]-(anon_1) |
30 |   24 |      14 |            120 |                   3/1 |     0.887 | friendOrFoe ASC | Fused
in Pipeline 0 |
+-----------------------------------+-----------------------------------------------------
+---------------+------+--------+---------------+----------------------+----------
+----------------+--------------------+

Total database accesses: 14, total allocated memory: 184
```

Partitioned Undirected Union Relationship Types Scan

The `PartitionedUndirectedUnionRelationshipTypeScan` is a variant of the `UndirectedUnionRelationshipTypesScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

> As the block storage format becomes the default, this operator will cease to be used in generating plans. Please refer to Operations Manual → Store formats for futher details on the various store formats available.

Example 519. PartitionedUndirectedUnionRelationshipTypesScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[friendOrFoe: FRIENDS_WITH|FOE]-()
RETURN friendOrFoe
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| Operator                                        | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| +ProduceResults                                 |  0 | friendOrFoe
|             30 |   24 |      24 |                   0/0 |     0.896 | In Pipeline 1 |
| |                                               +----
+-------------------------------------------------+----------------+------+---------
+------------------------+-----------+---------------+
| +PartitionedUndirectedUnionRelationshipTypesScan |  1 | (anon_0)-[friendOrFoe:FRIENDS_WITH|FOE]-
(anon_1) |             30 |   24 |      13 |                   2/0 |     0.818 | In Pipeline 0 |
+-------------------------------------------------+----
+-------------------------------------------------+----------------+------+---------
```

```
+-----------------------+----------+--------------+

Total database accesses: 37
```

Directed Relationship Index Scan

The `DirectedRelationshipIndexScan` operator examines all values stored in an index, returning all relationships and their start and end nodes with a particular relationship type and a specified property.

Example 520. DirectedRelationshipIndexScan

Query

```
PROFILE
MATCH ()-[r: WORKS_IN]->()
WHERE r.title IS NOT NULL
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------
+-------------------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+
| Operator                       | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------------
+-------------------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+
| +ProduceResults                | r
|             15 |   15 |       0 |                |                         |           |
|
| |
+-------------------------------------------------------------------------------+----------------+------
+---------+----------------+                        |           |                     |
| +DirectedRelationshipIndexScan | RANGE INDEX (anon_0)-[r:WORKS_IN(title)]->(anon_1) WHERE title IS
NOT NULL |             15 |   15 |      16 |            120 |                     3/1 |     2.464 |
Fused in Pipeline 0 |
+--------------------------------
+-------------------------------------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+---------------------+

Total database accesses: 16, total allocated memory: 184
```

Partitioned Directed Relationship Index Scan

The `PartitionedDirectedRelationshipIndexScan` is a variant of the `DirectedRelationshipIndexScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 521. PartitionedDirectedRelationshipIndexScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r: WORKS_IN]->()
WHERE r.title IS NOT NULL
RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----
+-----------------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +ProduceResults                         |  0 | r
|              15 |   15 |      70 |                    1/0 |     2.865 | In Pipeline 1 |
| |                                       +----
+-----------------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +PartitionedDirectedRelationshipIndexScan |  1 | RANGE INDEX (anon_0)-[r:WORKS_IN(title)]->(anon_1)
WHERE title IS NOT NULL |             15 |   15 |      16 |                    2/0 |     0.527 | In
Pipeline 0 |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+

Total database accesses: 86
```

Undirected Relationship Index Scan

The `UndirectedRelationshipIndexScan` operator examines all values stored in an index, returning all relationships and their start and end nodes with a particular relationship type and a specified property.

Example 522. UndirectedRelationshipIndexScan

Query

```
PROFILE
MATCH ()-[r: WORKS_IN]-()
WHERE r.title IS NOT NULL
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+----------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator                         | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults                  | r
|             30 |   30 |       0 |               |                       |           |
|
| |
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +UndirectedRelationshipIndexScan | RANGE INDEX (anon_0)-[r:WORKS_IN(title)]-(anon_1) WHERE title IS
NOT NULL |             30 |   30 |      16 |            120 |                     3/1 |     1.266 |
Fused in Pipeline 0 |
+----------------------------------
+-----------------------------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+

Total database accesses: 16, total allocated memory: 184
```

Partitioned Undirected Relationship Index Scan

The `PartitionedUndirectedRelationshipIndexScan` is a variant of the `UndirectedRelationshipIndexScan` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 523. PartitionedUndirectedRelationshipIndexScan

Query

```
CYPHER runtime=parallel
PROFILE
MATCH ()-[r: WORKS_IN]-()
WHERE r.title IS NOT NULL
RETURN r
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------------+----
+-----------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------------+----
+-----------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +ProduceResults                         |  0 | r
|             30 |   30 |     140 |                    1/0 |     3.088 | In Pipeline 1 |
| |                                       +----
+-----------------------------------------------------------------------------+----------------+------
+---------+-----------------------+-----------+---------------+
| +PartitionedUndirectedRelationshipIndexScan |  1 | RANGE INDEX (anon_0)-[r:WORKS_IN(title)]-
(anon_1) WHERE title IS NOT NULL |             30 |   30 |      16 |                    2/0 |
0.572 | In Pipeline 0 |
```

```
+-----------------------------------------+----
+-------------------------------------------------------------------------------+---------------+------
+---------+----------------------+----------+--------------+

Total database accesses: 156
```

Directed Relationship Index Contains Scan

The `DirectedRelationshipIndexContainsScan` operator examines all values stored in an index, searching for entries containing a specific `STRING`; for example, in queries including `CONTAINS`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a type scan using `DirectedRelationshipTypeScan`, and a property store filter.

Example 524. DirectedRelationshipIndexContainsScan

Query

```
PROFILE
MATCH ()-[r: WORKS_IN]->()
WHERE r.title CONTAINS 'senior'
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                                | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults                         | r
|              0 |    4 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +DirectedRelationshipIndexContainsScan | TEXT INDEX (anon_0)-[r:WORKS_IN(title)]->(anon_1) WHERE
title CONTAINS $autostring_0 |              0 |    4 |       5 |            120 |
3/0 |     1.051 | Fused in Pipeline 0 |
+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 5, total allocated memory: 184
```

Undirected Relationship Index Contains Scan

The `UndirectedRelationshipIndexContainsScan` operator examines all values stored in an index, searching

for entries containing a specific `STRING`; for example, in queries including `CONTAINS`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a type scan using `DirectedRelationshipTypeScan`, and a property store filter.

Example 525. UndirectedRelationshipIndexContainsScan

Query

```
PROFILE
MATCH ()-[r: WORKS_IN]-()
WHERE r.title CONTAINS 'senior'
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                                | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults                         | r
|              0 |    8 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +UndirectedRelationshipIndexContainsScan | TEXT INDEX (anon_0)-[r:WORKS_IN(title)]-(anon_1) WHERE
title CONTAINS $autostring_0 |              0 |    8 |       5 |            120 |
3/0 |     2.684 | Fused in Pipeline 0 |
+----------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 5, total allocated memory: 184
```

Directed Relationship Index Ends With Scan

The `DirectedRelationshipIndexEndsWithScan` operator examines all values stored in an index, searching for entries ending in a specific `STRING`; for example, in queries containing `ENDS WITH`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a label scan using `NodeByLabelScan`, and a property store filter.

Example 526. DirectedRelationshipIndexEndsWithScan

Query

```
PROFILE
```

```
MATCH ()-[r: WORKS_IN]->()
WHERE r.title ENDS WITH 'developer'
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                                | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults                         | r
|              0 |    8 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +DirectedRelationshipIndexEndsWithScan | TEXT INDEX (anon_0)-[r:WORKS_IN(title)]->(anon_1) WHERE
title ENDS WITH $autostring_0 |              0 |    8 |       9 |            120 |
3/0 |     1.887 | Fused in Pipeline 0 |
+--------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 9, total allocated memory: 184
```

Undirected Relationship Index Ends With Scan

The `UndirectedRelationshipIndexEndsWithScan` operator examines all values stored in an index, searching for entries ending in a specific `STRING`; for example, in queries containing `ENDS WITH`. Although this is slower than an index seek (since all entries need to be examined), it is still faster than the indirection resulting from a label scan using `NodeByLabelScan`, and a property store filter.

Example 527. UndirectedRelationshipIndexEndsWithScan

Query

```
PROFILE
MATCH ()-[r: WORKS_IN]-()
WHERE r.title ENDS WITH 'developer'
RETURN r
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+----------------------------------------
+----------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator                                | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------------
+----------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults                         | r
|              0 |   16 |       0 |               |                       |           |
|
| |
+----------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +UndirectedRelationshipIndexEndsWithScan | TEXT INDEX (anon_0)-[r:WORKS_IN(title)]-(anon_1) WHERE
title ENDS WITH $autostring_0 |              0 |   16 |       9 |            120 |
3/0 |     1.465 | Fused in Pipeline 0 |
+----------------------------------------
+----------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 9, total allocated memory: 184
```

Directed Relationship Index Seek

The `DirectedRelationshipIndexSeek` operator finds relationships and their start and end nodes using an index seek. The relationship variable and the index used are shown in the arguments of the operator.

Example 528. DirectedRelationshipIndexSeek

Query

```
PROFILE
MATCH (candidate)-[r:WORKS_IN]->()
WHERE r.title = 'chief architect'
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------
+----------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator                       | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------------
+----------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults                | candidate
|              2 |    1 |       0 |               |                       |           |
|
| |
+----------------------------------------------------------------------------------+----------------
```

```
+------+---------+---------------+                          |          |                    |
| +DirectedRelationshipIndexSeek | RANGE INDEX (candidate)-[r:WORKS_IN(title)]->(anon_0) WHERE title
= $autostring_0 |              2 |    1 |       2 |            120 |                    3/1 |
0.591 | Fused in Pipeline 0 |
+-------------------------------
+-----------------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 2, total allocated memory: 184
```

Partitioned Directed Relationship Index Seek

The `PartitionedDirectedRelationshipIndexSeek` is a variant of the the `DirectedRelationshipIndexSeek` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 529. PartitionedDirectedRelationshipIndexSeek

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (candidate)-[r:WORKS_IN]->()
WHERE r.title = 'chief architect'
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----
+-----------------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| Operator                                | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| +ProduceResults                         |  0 | candidate
|              2 |    1 |       2 |                   2/0 |     0.284 | In Pipeline 1 |
| |                                       +----
+-----------------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| +PartitionedDirectedRelationshipIndexSeek |  1 | RANGE INDEX (candidate)-[r:WORKS_IN(title)]-
>(anon_0) WHERE title = $autostring_0 |              2 |    1 |       2 |                   2/0 |
0.148 | In Pipeline 0 |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+

Total database accesses: 4
```

Undirected Relationship Index Seek

The `UndirectedRelationshipIndexSeek` operator finds relationships and their start and end nodes using an index seek. The relationship variable and the index used are shown in the arguments of the operator.

Example 530. UndirectedRelationshipIndexSeek

Query

```
PROFILE
MATCH (candidate)-[r:WORKS_IN]-()
WHERE r.title = 'chief architect'
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------------
+-----------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+
| Operator                        | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------
+-----------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults                 | candidate
|              4 |    2 |       0 |               |                       |           |
|
| |
+-----------------------------------------------------------------------------------------+----------------
+------+---------+----------------+                       |           |                     |
| +UndirectedRelationshipIndexSeek | RANGE INDEX (candidate)-[r:WORKS_IN(title)]-(anon_0) WHERE title
= $autostring_0 |              4 |    2 |       2 |            120 |                     3/1 |
0.791 | Fused in Pipeline 0 |
+----------------------------------
+-----------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------+

Total database accesses: 2, total allocated memory: 184
```

Partitioned Undirected Relationship Index Seek

The `PartitionedUndirectedRelationshipIndexSeek` operator finds relationships and their start and end nodes using an index seek. The relationship variable and the index used are shown in the arguments of the operator.

Example 531. PartitionedUndirectedRelationshipIndexSeek

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (candidate)-[r:WORKS_IN]-()
WHERE r.title = 'chief architect'
RETURN candidate
```

Query Plan

```
Planner COST
```

```
Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-------------------------------------------+----
+-------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| Operator                                 | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-------------------------------------------+----
+-------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| +ProduceResults                          |  0 | candidate
|              4 |    2 |       4 |                   2/0 |     0.333 | In Pipeline 1 |
| |                                        +----
+-------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+
| +PartitionedUndirectedRelationshipIndexSeek |  1 | RANGE INDEX (candidate)-[r:WORKS_IN(title)]-
(anon_0) WHERE title = $autostring_0 |              4 |    2 |       2 |                   2/0 |
0.151 | In Pipeline 0 |
+-------------------------------------------+----
+-------------------------------------------------------------------------------+----------------
+------+---------+-----------------------+-----------+---------------+

Total database accesses: 6
```

Directed Relationship By Element Id Seek

The `DirectedRelationshipByElementIdSeek` operator reads one or more relationships by element id from the relationship store (specified via the function elementId()) and produces the relationship as well as the source and target node of the relationship.

Example 532. DirectedRelationshipByElementIdSeek

Query

```
PROFILE
MATCH (n1)-[r]->()
WHERE elementId(r) = 0
RETURN r, n1
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------------+----+-----------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| Operator                             | Id | Details                                                |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------------------+----+-----------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +ProduceResults                      |  0 | r, n1                                                  |
1 |    0 |       0 |              0 |                    0/0 |     0.314 |               |
| |                                    +----+-----------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +DirectedRelationshipByElementIdSeek |  1 | (n1)-[r]->(anon_0) WHERE elementId(r) = $autoint_0 |
```

```
1 |    0 |       0 |           248 |                  0/0 |     2.337 | In Pipeline 0 |
+-----------------------------------+----+-----------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------+

Total database accesses: 0, total allocated memory: 312
```

Directed Relationship By Id Seek

The `DirectedRelationshipByIdSeek` operator reads one or more relationships by id from the relationship store, and produces the relationship as well as the source and target node of the relationship.

Example 533. DirectedRelationshipByIdSeek

Query

```
PROFILE
MATCH (n1)-[r]->()
WHERE id(r) = 0
RETURN r, n1
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------+----+-------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator                      | Id | Details                                   | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------------------+----+-------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults               |  0 | r, n1                                     |              1 |
1 |       7 |              0 |                       |           |                     |
| |                             +----+-------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| +DirectedRelationshipByIdSeek |  1 | (n1)-[r]->(anon_0) WHERE id(r) = $autoint_0 |              1 |
1 |       1 |            248 |                   3/0 |     0.483 | Fused in Pipeline 0 |
+-------------------------------+----+-------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+

Total database accesses: 8, total allocated memory: 312
```

Undirected Relationship By Element Id Seek

The `UndirectedRelationshipByElementIdSeek` operator reads one or more relationships by element id from the relationship store (specified via the function elementId()). As the direction is unspecified, two rows are produced for each relationship as a result of alternating the combination of the start and end node.

Example 534. UndirectedRelationshipByElementIdSeek

Query

```
PROFILE
MATCH (n1)-[r]-()
```

```
WHERE elementId(r) = 1
RETURN r, n1
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------------+-----------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator                             | Details                                 | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+--------------------------------------+-----------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults                      | r, n1                                   |              2
|    2 |       0 |                |                       |           |                     |
| |                                    +-----------------------------------------
+---------------+------+---------+---------------+                       |           |
|
| +UndirectedRelationshipByElementIdSeek| (n1)-[r]-(anon_0) WHERE elementId(r) = $autoint_0 |       2
|    2 |       1 |            120 |                   4/0 |     0.332 | Fused in Pipeline 0 |
+---------------------------------+-------------------------------------------------+-----+---------------
+------+---------+---------------+-----------------------+-----------+---------------------+

Total database accesses: 1, total allocated memory: 184
```

Undirected Relationship By Id Seek

The `UndirectedRelationshipByIdSeek` operator reads one or more relationships by id from the relationship store (specified via the function id()). As the direction is unspecified, two rows are produced for each relationship as a result of alternating the combination of the start and end node.

Example 535. UndirectedRelationshipByIdSeek

Query

```
PROFILE
MATCH (n1)-[r]-()
WHERE id(r) = 1
RETURN r, n1
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------------------+----+-----------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator                        | Id | Details                                 | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+---------------------------------+----+-----------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults                 |  0 | r, n1                                   |              2
```

```
|    2 |      14 |             0 |                      |          |                  |
| |                              +----+-----------------------------------------+---------------
+------+---------+---------------+                      |          |                  |
| +UndirectedRelationshipByIdSeek |  1 | (n1)-[r]-(anon_0) WHERE id(r) = $autoint_0 |              2
|    2 |       1 |           248 |                  3/0 |    1.005 | Fused in Pipeline 0 |
+---------------------------------+----+-----------------------------------------+---------------
+------+---------+---------------+----------------------+----------+--------------------+

Total database accesses: 15, total allocated memory: 312
```

Directed Relationship Index Seek By Range

The `DirectedRelationshipIndexSeekByRange` operator finds relationships and their start and end nodes using an index seek where the value of the property matches a given prefix `STRING`.
`DirectedRelationshipIndexSeekByRange` can be used for `STARTS WITH` and comparison operators such as `<`, `>`, `<=` and `>=`.

Example 536. DirectedRelationshipIndexSeekByRange

Query

```
PROFILE
MATCH (candidate: Person)-[r:WORKS_IN]->(location)
WHERE r.duration > 100
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                             | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+--------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults                      | candidate
|              4 |   15 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                              | candidate:Person
|              4 |   15 |      30 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +DirectedRelationshipIndexSeekByRange | RANGE INDEX (candidate)-[r:WORKS_IN(duration)]->(location)
WHERE duration > $autoint_0 |              4 |   15 |      16 |            120 |
4/1 |     0.703 | Fused in Pipeline 0 |
+--------------------------------------
+-----------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
```

```
+--------------------+

Total database accesses: 46, total allocated memory: 184
```

Partitioned Directed Relationship Index Seek By Range

The `PartitionedDirectedRelationshipIndexSeekByRange` is a variant of the `DirectedRelationshipIndexSeekByRange` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 537. PartitionedDirectedRelationshipIndexSeekByRange

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (candidate: Person)-[r:WORKS_IN]->(location)
WHERE r.duration > 100
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-------------------------------------------------+----
+-------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| Operator                                         | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------------------------------------+----
+-------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| +ProduceResults                                  |  0 | candidate
|              4 |   15 |      30 |                   1/0 |     1.031 | In Pipeline 1       |
| |                                                +----
+-------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| +Filter                                          |  1 | candidate:Person
|              4 |   15 |      30 |                       |           |                     |
| |                                                +----
+-------------------------------------------------------------------------------------
+----------------+------+---------+                       |           |                     |
| +PartitionedDirectedRelationshipIndexSeekByRange |  2 | RANGE INDEX (candidate)-
[r:WORKS_IN(duration)]->(location) WHERE duration > $autoint_0 |              4 |   15 |      16 |
3/0 |     0.203 | Fused in Pipeline 0 |
+-------------------------------------------------+----
+-------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+

Total database accesses: 76
```

Undirected Relationship Index Seek By Range

The `UndirectedRelationshipIndexSeekByRange` operator finds relationships and their start and end nodes using an index seek where the value of the property matches a given prefix `STRING`. `UndirectedRelationshipIndexSeekByRange` can be used for `STARTS WITH` and comparison operators such as

`<`, `>`, `<=` and `>=`.

Example 538. UndirectedRelationshipIndexSeekByRange

Query

```
PROFILE
MATCH (candidate: Person)-[r:WORKS_IN]-(location)
WHERE r.duration > 100
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator                                | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults                         | candidate
|              5 |   15 |       0 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Filter                                 | candidate:Person
|              5 |   15 |      60 |                |                        |           |
|
| |
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +UndirectedRelationshipIndexSeekByRange | RANGE INDEX (candidate)-[r:WORKS_IN(duration)]-(location)
WHERE duration > $autoint_0 |              8 |   30 |      16 |            120 |
4/1 |     1.214 | Fused in Pipeline 0 |
+----------------------------------------
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 76, total allocated memory: 184
```

Partitioned Undirected Relationship Index Seek By Range

The `PartitionedUndirectedRelationshipIndexSeekByRange` is a variant of the `UndirectedRelationshipIndexSeekByRange` operator used by the parallel runtime. It allows the store to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 539. PartitionedUndirectedRelationshipIndexSeekByRange

Query

```
CYPHER runtime=parallel
PROFILE
MATCH (candidate: Person)-[r:WORKS_IN]-(location)
WHERE r.duration > 100
RETURN candidate
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------------------------------+----
+-----------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| Operator                                             | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------------------------------------------+----
+-----------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| +ProduceResults                                      |  0 | candidate
|              5 |   15 |      30 |                   1/0 |     0.918 | In Pipeline 1       |
| |                                                    +----
+-----------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| +Filter                                              |  1 | candidate:Person
|              5 |   15 |      60 |                       |           |                     |
| |                                                    +----
+-----------------------------------------------------------------------------------
+----------------+------+---------+                             |           |                     |
| +PartitionedUndirectedRelationshipIndexSeekByRange |  2 | RANGE INDEX (candidate)-
[r:WORKS_IN(duration)]-(location) WHERE duration > $autoint_0 |              8 |   30 |      16 |
3/0 |     0.413 | Fused in Pipeline 0 |
+-----------------------------------------------------+----
+-----------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+

Total database accesses: 106
```

Dynamic Label Node Lookup

Introduced in 2025.08

Allows Cypher to use token lookup indexes when planning queries using dynamic node labels.

Example 540. DynamicLabelNodeLookup

Query

```
PROFILE
WITH "Person" AS label
MATCH (people:$(label))
RETURN people.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128


+---------------------------+----+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator                  | Id | Details                     | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+---------------------------+----+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +ProduceResults           |  0 | `people.name`               |             26 |   14 |       0 |
0 |                         |           |                     |
| |                         +----+-----------------------------+----------------+------+---------
+----------------+                        |           |                     |
| +Projection               |  1 | people.name AS `people.name` |             26 |   14 |      28 |
|                         |           |                     |
| |                         +----+-----------------------------+----------------+------+---------
+----------------+                        |           |                     |
| +Apply                    |  2 |                             |             26 |   14 |       0 |
|                         |           |                     |
| |\                        +----+-----------------------------+----------------+------+---------
+----------------+                        |           |                     |
| | +DynamicLabelNodeLookup |  3 | people:$all(label)          |             26 |   14 |      15 |
1832 |                    2/0 |     0.366 | Fused in Pipeline 1 |
| |                         +----+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +Projection               |  4 | $autostring_0 AS label      |              1 |    1 |       0 |
|                         |           | Fused in Pipeline 0 |
+---------------------------+----+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
```

Dynamic Directed Relationship Type Lookup

Introduced in 2025.08

Allows Cypher to use token lookup indexes when planning queries using dynamic relationship types in directed relationship patterns.

Example 541. DynamicDirectedRelationshipTypeLookup

Query

```
PROFILE
WITH "FRIENDS_WITH" AS relType
MATCH ()-[r:$(relType)]->()
RETURN count(r) as relCount
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------------------+----+-------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
```

```
| Operator                                  | Id | Details                | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------------------------------+----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +ProduceResults                           |  0 | relCount               |              1 |    1 |
0 |              0 |                    0/0 |     0.022 | In Pipeline 2      |
| |                                         +----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +EagerAggregation                         |  1 | count(r) AS relCount   |              1 |    1 |
0 |             40 |                         |           |                    |
| |                                         +----+------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +Apply                                    |  2 |                        |             21 |   12 |
0 |                |                         |           |                    |
| |\                                        +----+------------------------+----------------+------
+---------+----------------+                         |           |                    |
| | +DynamicDirectedRelationshipTypeLookup |  3 | ()-[r:$all(relType)]->() |             21 |   12 |
13 |           1968 |                    2/0 |     0.359 | Fused in Pipeline 1 |
| |                                         +----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +Projection                               |  4 | $autostring_0 AS relType |              1 |    1 |
0 |                |                         |           | Fused in Pipeline 0 |
+-------------------------------------------+----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
```

Dynamic Undirected Relationship Type Lookup

Introduced in 2025.08

Allows Cypher to use token lookup indexes when planning queries using dynamic relationship types in undirected relationship patterns.

Example 542. DynamicUndirectedRelationshipTypeLookup

Query

```
PROFILE
WITH "FRIENDS_WITH" AS relType
MATCH ()-[r:$(relType)]-()
RETURN count(r) as relCount
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------------------------+----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| Operator                                  | Id | Details                | Estimated Rows | Rows
| DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------------------------------+----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +ProduceResults                           |  0 | relCount               |              1 |    1
|       0 |              0 |                    0/0 |     0.011 | In Pipeline 2      |
| |                                         +----+------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +EagerAggregation                         |  1 | count(r) AS relCount   |              1 |    1
|       0 |             40 |                         |           |                    |
| |                                         +----+------------------------+----------------+------
+---------+----------------+                         |           |                    |
| +Apply                                    |  2 |                        |             42 |   24
|       0 |                |                         |           |                    |
```

```
| |\                                      +----+-------------------------+----------------+------
+---------+----------------+                      |          |                    |
| | +DynamicUndirectedRelationshipTypeLookup |  3 | ()-[r:$all(relType)]-()  |             42 |   24
|      13 |           1968 |                   2/0 |     0.121 | Fused in Pipeline 1 |
| |                                       +----+-------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +Projection                             |  4 | $autostring_0 AS relType |              1 |    1
|       0 |                |                       |           | Fused in Pipeline 0 |
+-----------------------------------------+----+-------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
```

Nested loops and join operators

Nested loop operators process data by iterating over the right-hand side (RHS) for each row from the left-hand side (LHS). Each row from the LHS triggers the execution of the RHS, effectively creating a loop over the RHS for each LHS element.

Apply

All the different `Apply` operators (listed below) share the same basic functionality: they perform a nested loop by taking a single row from the left-hand side, and using the `Argument` operator on the right-hand side, execute the operator tree on the right-hand side. The versions of the `Apply` operators differ in how the results are managed. The `Apply` operator (i.e. the standard version) takes the row produced by the right-hand side — which at this point contains data from both the left-hand and right-hand sides — and yields it.

Example 543. Apply

Query

```
PROFILE
MATCH (p:Person {name: 'me'})
MATCH (q:Person {name: p.secondName})
RETURN p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator        | Details                                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+-----------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults | p, q                                                |              1 |    0 |
0 |                |                       |           |                     |
| |               +-----------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| +Apply          |                                                     |              1 |    0 |
0 |                |                       |           |                     |
| |\              +-----------------------------------------------------+----------------+------
+---------+----------------+                       |           |                     |
| | +NodeIndexSeek | RANGE INDEX q:Person(name) WHERE name = p.secondName  |              1 |    0 |
0 |           2152 |                   0/0 |     0.219 | Fused in Pipeline 1 |
| |               +-----------------------------------------------------+----------------+------
```

```
+---------+---------------+----------------------+----------+--------------------+
| +NodeIndexSeek   | RANGE INDEX p:Person(name) WHERE name = $autostring_0 |              1 |    1 |
2 |            120 |                    0/1 |     0.236 | In Pipeline 0       |
+-----------------+-------------------------------------------------------+----------------+------
+---------+---------------+----------------------+----------+--------------------+

Total database accesses: 2, total allocated memory: 2216
```

Semi Apply

The `SemiApply` operator tests for the presence of a pattern predicate, and is a variation of the `Apply` operator. If the right-hand side operator yields at least one row, the row from the left-hand side operator is yielded by the `SemiApply` operator. This makes `SemiApply` a filtering operator, used mostly for pattern predicates in queries.

`SemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `SemiApply` is instead performed by the `Apply` and `Limit` operators.

Example 544. SemiApply

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (p:Person)
WHERE (p)-[:FRIENDS_WITH]->(:Person)
RETURN p.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+-----------------------------------+----------------+------+---------
+------------------------+
| Operator        | Details                           | Estimated Rows | Rows | DB Hits | Page
Cache Hits/Misses |
+-----------------+-----------------------------------+----------------+------+---------
+------------------------+
| +ProduceResults | `p.name`                          |             11 |   10 |       0 |
0/0 |
| |               +-----------------------------------+----------------+------+---------
+------------------------+
| +Projection     | p.name AS `p.name`                |             11 |   10 |      10 |
1/0 |
| |               +-----------------------------------+----------------+------+---------
+------------------------+
| +SemiApply      |                                   |             11 |   10 |       0 |
0/0 |
| |\              +-----------------------------------+----------------+------+---------
+------------------------+
| | +Filter       | anon_3:Person                     |             12 |    0 |      10 |
0/0 |
| | |             +-----------------------------------+----------------+------+---------
+------------------------+
| | +Expand(All)  | (p)-[anon_2:FRIENDS_WITH]->(anon_3) |             12 |   10 |      51 |
28/0 |
| | |             +-----------------------------------+----------------+------+---------
+------------------------+
| | +Argument     | p                                 |             14 |   14 |       0 |
0/0 |
| |               +-----------------------------------+----------------+------+---------
+------------------------+
```

```
| +NodeByLabelScan| p:Person                          |            14 |   14 |      35 |
1/0 |
+-----------------+-----------------------------------+---------------+------+---------
+------------------------+

Total database accesses: 142, total allocated memory: 64
```

Anti Semi Apply

The `AntiSemiApply` operator tests for the absence of a pattern, and is a variation of the `Apply` operator. If the right-hand side operator yields no rows, the row from the left-hand side operator is yielded by the `AntiSemiApply` operator. This makes `AntiSemiApply` a filtering operator, used for pattern predicates in queries.

`AntiSemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `AntiSemiApply` is instead performed by the `Anti`, `Apply`, and `Limit` operators.

Example 545. AntiSemiApply

Query

```
PROFILE
CYPHER runtime=slotted
MATCH
  (me:Person {name: 'me'})-[:WORKS_IN]->(location),
  (other:Person)-[:WORKS_IN]->(location)
WHERE NOT (me)-[:FRIENDS_WITH]->(other)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| Operator        | Id | Details                                                   | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Indexes Used        |
+-----------------+----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| +ProduceResults |  0 | `other.name`                                              |              0 |
5 |       0 |                |                    0/0 |                     |
| |               +----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| +Projection     |  1 | other.name AS `other.name`                                |              0 |
5 |       5 |                |                    0/0 |                     |
| |               +----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| +AntiSemiApply  |  2 |                                                           |              0 |
5 |       0 |                |                    0/0 |                     |
| |\              +----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| | +Expand(Into) |  3 | (me)-[:FRIENDS_WITH]->(other)                             |              0 |
0 |       7 |            592 |                    0/0 |                     |
| | |             +----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| | +Argument     |  4 | me, other                                                 |              0 |
6 |       0 |                |                    0/0 |                     |
| |               +----+-----------------------------------------------------------+----------------
+------+---------+----------------+------------------------+---------------------+
| +Filter         |  5 | NOT anon_0 = anon_1 AND other:Person                      |              0 |
6 |       6 |                |                    0/0 |                     |
| |               +----+-----------------------------------------------------------+----------------
```

```
+------+---------+---------------+-----------------------+---------------------+
| +Expand(All)    |  6 | (location)<-[anon_1:WORKS_IN]-(other)                  |             0 |
7 |       8 |               |                     1/0 |                     |
| |               +----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+---------------------+
| +Expand(All)    |  7 | (me)-[anon_0:WORKS_IN]->(location)                     |             1 |
1 |       2 |               |                     1/0 |                     |
| |               +----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+---------------------+
| +NodeIndexSeek  |  8 | RANGE INDEX me:Person(name) WHERE name = $autostring_0 |             1 |
1 |       2 |               |                     0/1 | range_person_name: 1 |
+-----------------+----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+---------------------+

Total database accesses: 166, total allocated memory: 976
```

Let Semi Apply

The `LetSemiApply` operator tests for the presence of a pattern predicate, and is a variation of the `Apply` operator. When a query contains multiple pattern predicates separated with `OR`, `LetSemiApply` will be used to evaluate the first of these. It will record the result of evaluating the predicate but will leave any filtering to another operator.

`LetSemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `LetSemiApply` is instead performed by the `Apply`, `EagerAggregation`, and `Limit` operators.

Example 546. LetSemiApply

In the example, `LetSemiApply` will be used to check for the presence of the `FRIENDS_WITH` relationship from each person.

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (other:Person)
WHERE (other)-[:FRIENDS_WITH]->(:Person) OR (other)-[:WORKS_IN]->(:Location)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+--------------------+----------------------------------------+----------------+------+---------
+-------------------------+
| Operator           | Details                                | Estimated Rows | Rows | DB Hits |
Page Cache Hits/Misses |
+--------------------+----------------------------------------+----------------+------+---------
+-------------------------+
| +ProduceResults    | `other.name`                           |             13 |   14 |       0 |
0/0 |
| |                  +----------------------------------------+----------------+------+---------
+-------------------------+
| +Projection        | other.name AS `other.name`             |             13 |   14 |      14 |
1/0 |
| |                  +----------------------------------------+----------------+------+---------
+-------------------------+
| +SelectOrSemiApply | anon_9                                 |             14 |   14 |       0 |
0/0 |
| |\                 +----------------------------------------+----------------+------+---------
```

```
+-----------------------+
| | +Filter          | anon_7:Location                          |             14 |    0 |       4 |
0/0 |
| | |                +------------------------------------------+----------------+------+---------
+-----------------------+
| | +Expand(All)     | (other)-[anon_6:WORKS_IN]->(anon_7)      |             14 |    4 |      15 |
8/0 |
| | |                +------------------------------------------+----------------+------+---------
+-----------------------+
| | +Argument        | other                                    |             14 |    4 |       0 |
0/0 |
| |                  +------------------------------------------+----------------+------+---------
+-----------------------+
| +LetSemiApply      |                                          |             14 |   14 |       0 |
0/0 |
| |\                 +------------------------------------------+----------------+------+---------
+-----------------------+
| | +Filter          | anon_5:Person                            |             12 |    0 |      10 |
0/0 |
| | |                +------------------------------------------+----------------+------+---------
+-----------------------+
| | +Expand(All)     | (other)-[anon_4:FRIENDS_WITH]->(anon_5) |             12 |   10 |      51 |
28/0 |
| | |                +------------------------------------------+----------------+------+---------
+-----------------------+
| | +Argument        | other                                    |             14 |   14 |       0 |
0/0 |
| |                  +------------------------------------------+----------------+------+---------
+-----------------------+
| +NodeByLabelScan   | other:Person                             |             14 |   14 |      35 |
1/0 |
+--------------------+------------------------------------------+----------------+------+---------
+-----------------------+

Total database accesses: 165, total allocated memory: 64
```

Let Anti Semi Apply

The `LetAntiSemiApply` operator tests for the absence of a pattern, and is a variation of the `Apply` operator. When a query contains multiple negated pattern predicates — i.e. predicates separated with `OR`, where at least one predicate contains `NOT` — `LetAntiSemiApply` will be used to evaluate the first of these. It will record the result of evaluating the predicate but will leave any filtering to another operator.

`LetAntiSemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `LetAntiSemiApply` is instead performed by the `Apply`, `EagerAggregation`, and `Limit` operators.

Example 547. LetAntiSemiApply

In the example, `LetAntiSemiApply` will be used to check for the absence of the `FRIENDS_WITH` relationship from each person.

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (other:Person)
WHERE NOT ((other)-[:FRIENDS_WITH]->(:Person)) OR (other)-[:WORKS_IN]->(:Location)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED
```

```
Runtime version 2026.10
+--------------------+----------------------------------------+----------------+------+---------
+------------------------+
| Operator           | Details                                | Estimated Rows | Rows | DB Hits |
Page Cache Hits/Misses |
+--------------------+----------------------------------------+----------------+------+---------
+------------------------+
| +ProduceResults    | `other.name`                           |             11 |   14 |       0 |
0/0 |
| |                  +----------------------------------------+----------------+------+---------
+------------------------+
| +Projection        | other.name AS `other.name`             |             11 |   14 |      14 |
1/0 |
| |                  +----------------------------------------+----------------+------+---------
+------------------------+
| +SelectOrSemiApply | anon_9                                 |             14 |   14 |       0 |
0/0 |
| |\                 +----------------------------------------+----------------+------+---------
+------------------------+
| | +Filter          | anon_7:Location                        |             14 |    0 |      10 |
0/0 |
| | |                +----------------------------------------+----------------+------+---------
+------------------------+
| | +Expand(All)     | (other)-[anon_6:WORKS_IN]->(anon_7)    |             14 |   10 |      38 |
20/0 |
| | |                +----------------------------------------+----------------+------+---------
+------------------------+
| | +Argument        | other                                  |             14 |   10 |       0 |
0/0 |
| |                  +----------------------------------------+----------------+------+---------
+------------------------+
| +LetAntiSemiApply  |                                        |             14 |   14 |       0 |
0/0 |
| |\                 +----------------------------------------+----------------+------+---------
+------------------------+
| | +Filter          | anon_5:Person                          |             12 |    0 |      10 |
0/0 |
| | |                +----------------------------------------+----------------+------+---------
+------------------------+
| | +Expand(All)     | (other)-[anon_4:FRIENDS_WITH]->(anon_5) |             12 |   10 |      51 |
28/0 |
| | |                +----------------------------------------+----------------+------+---------
+------------------------+
| | +Argument        | other                                  |             14 |   14 |       0 |
0/0 |
| |                  +----------------------------------------+----------------+------+---------
+------------------------+
| +NodeByLabelScan   | p:Person                               |             14 |   14 |      35 |
1/0 |
+--------------------+----------------------------------------+----------------+------+---------
+------------------------+

Total database accesses: 142, total allocated memory: 64
```

Select Or Semi Apply

The `SelectOrSemiApply` operator tests for the presence of a pattern predicate and evaluates a predicate, and is a variation of the `Apply` operator. This operator allows for the mixing of normal predicates and pattern predicates that check for the presence of a pattern. First, the normal expression predicate is evaluated, and, only if it returns `false`, is the costly pattern predicate evaluated.

Example 548. SelectOrSemiApply

Query

```
PROFILE
MATCH (other:Person)
WHERE other.age > 25 OR (other)-[:FRIENDS_WITH]->(:Person)
```

```
RETURN other.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator           | Details                                | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+--------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +ProduceResults    | `other.name`                           |             11 |   10 |       0 |
|                        |           |                     |
| |                  +----------------------------------------+----------------+------+---------
+----------------+                           |           |                     |
| +Projection        | other.name AS `other.name`             |             11 |   10 |      20 |
|                        |           |                     |
| |                  +----------------------------------------+----------------+------+---------
+----------------+                           |           |                     |
| +SelectOrSemiApply | other.age > $autoint_0                 |             14 |   10 |       0 |
392 |                    0/0 |     0.190 | Fused in Pipeline 2 |
| |\                 +----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| | +Limit           | 1                                      |             14 |   10 |       0 |
752 |                          |           |                     |
| | |                +----------------------------------------+----------------+------+---------
+----------------+                           |           |                     |
| | +Filter          | anon_3:Person                          |             12 |   10 |      20 |
|                        |           |                     |
| | |                +----------------------------------------+----------------+------+---------
+----------------+                           |           |                     |
| | +Expand(All)     | (other)-[anon_2:FRIENDS_WITH]->(anon_3) |             12 |   10 |      37 |
|                        |           |                     |
| | |                +----------------------------------------+----------------+------+---------
+----------------+                           |           |                     |
| | +Argument        | other                                  |             14 |   14 |       0 |
2168 |                   2/0 |     0.435 | Fused in Pipeline 1 |
| |                  +----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +NodeByLabelScan   | other:Person                           |             14 |   14 |      35 |
|                        |           | Fused in Pipeline 0 |
+--------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+

Total database accesses: 148, total allocated memory: 2952
```

Select Or Anti Semi Apply

The `SelectOrAntiSemiApply` operator is used to evaluate `OR` between a predicate and a negative pattern predicate (i.e. a pattern predicate preceded with `NOT`), and is a variation of the `Apply` operator. If the predicate returns `true`, the pattern predicate is not tested. If the predicate returns `false` or `null`, `SelectOrAntiSemiApply` will instead test the pattern predicate.

Example 549. SelectOrAntiSemiApply

Query

```
PROFILE
MATCH (other:Person)
WHERE other.age > 25 OR NOT (other)-[:FRIENDS_WITH]->(:Person)
```

```
RETURN other.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| Operator              | Details                                | Estimated Rows | Rows | DB Hits
| Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +ProduceResults       | `other.name`                           |              4 |    4 |       0
|                |                       |           |                    |
| |                     +----------------------------------------+----------------+------+---------
+----------------+                       |           |                    |
| +Projection           | other.name AS `other.name`             |              4 |    4 |       8
|                |                       |           |                    |
| |                     +----------------------------------------+----------------+------+---------
+----------------+                       |           |                    |
| +SelectOrAntiSemiApply | other.age > $autoint_0                |             14 |    4 |       0
|            200 |                   0/0 |     0.155 | Fused in Pipeline 3 |
| |\                    +----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| | +Anti               |                                        |             14 |    4 |       0
|           1256 |                   0/0 |     0.170 | In Pipeline 2      |
| | |                   +----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| | +Limit              | 1                                      |              0 |   10 |       0
|            752 |                       |           |                    |
| | |                   +----------------------------------------+----------------+------+---------
+----------------+                       |           |                    |
| | +Filter             | anon_3:Person                          |             12 |   10 |      20
|                |                       |           |                    |
| | |                   +----------------------------------------+----------------+------+---------
+----------------+                       |           |                    |
| | +Expand(All)        | (other)-[anon_2:FRIENDS_WITH]->(anon_3) |             12 |   10 |      37
|                |                       |           |                    |
| | |                   +----------------------------------------+----------------+------+---------
+----------------+                       |           |                    |
| | +Argument           | other                                  |             14 |   14 |       0
|           2168 |                   2/0 |     0.449 | Fused in Pipeline 1 |
| |                     +----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +NodeByLabelScan      | other:Person                           |             14 |   14 |      35
|                |                       |           | Fused in Pipeline 0 |
+-----------------------+----------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+

Total database accesses: 136, total allocated memory: 4208
```

Let Select Or Semi Apply

The `LetSelectOrSemiApply` operator is planned for pattern predicates that are combined with other predicates using `OR`. This is a variation of the `Apply` operator.

`LetSelectOrSemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `LetSelectOrSemiApply` is instead performed by the `SelectOrSemiApply`, `EagerAggregation`, `Limit`, and `Projection` operators.

Example 550. LetSelectOrSemiApply

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (other:Person)
WHERE (other)-[:FRIENDS_WITH]->(:Person) OR (other)-[:WORKS_IN]->(:Location) OR other.age = 5
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------------+----------------------------------------+----------------+------+---------
+-------------------------+
| Operator              | Details                                | Estimated Rows | Rows | DB Hits |
Page Cache Hits/Misses |
+-----------------------+----------------------------------------+----------------+------+---------
+-------------------------+
| +ProduceResults       | `other.name`                           |             13 |   14 |       0 |
0/0 |
| |                     +----------------------------------------+----------------+------+---------
+-------------------------+
| +Projection           | other.name AS `other.name`             |             13 |   14 |      14 |
1/0 |
| |                     +----------------------------------------+----------------+------+---------
+-------------------------+
| +SelectOrSemiApply    | anon_9                                 |             14 |   14 |       0 |
0/0 |
| |\                    +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Filter             | anon_7:Location                        |             14 |    0 |       4 |
0/0 |
| | |                   +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Expand(All)        | (other)-[anon_6:WORKS_IN]->(anon_7)    |             14 |    4 |      15 |
8/0 |
| | |                   +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Argument           | other                                  |             14 |    4 |       0 |
0/0 |
| |                     +----------------------------------------+----------------+------+---------
+-------------------------+
| +LetSelectOrSemiApply | other.age = $autoint_0                 |             14 |   14 |      14 |
0/0 |
| |\                    +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Filter             | anon_5:Person                          |             12 |    0 |      10 |
0/0 |
| | |                   +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Expand(All)        | (other)-[anon_4:FRIENDS_WITH]->(anon_5) |             12 |   10 |      51 |
28/0 |
| | |                   +----------------------------------------+----------------+------+---------
+-------------------------+
| | +Argument           | other                                  |             14 |   14 |       0 |
0/0 |
| |                     +----------------------------------------+----------------+------+---------
+-------------------------+
| +NodeByLabelScan      | other:Person                           |             14 |   14 |      35 |
1/0 |
+-----------------------+----------------------------------------+----------------+------+---------
+-------------------------+

Total database accesses: 179, total allocated memory: 64
```

Let Select Or Anti Semi Apply

The `LetSelectOrAntiSemiApply` operator is planned for negated pattern predicates — i.e. pattern predicates preceded with `NOT` — that are combined with other predicates using `OR`. This operator is a variation of the `Apply` operator.

`LetSelectOrAntiSemiApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `LetSelectOrAntiSemiApply` is instead performed by the `SelectOrSemiApply`, `EagerAggregation`, `Limit`, and `Projection` operators.

Example 551. LetSelectOrAntiSemiApply

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (other:Person)
WHERE NOT (other)-[:FRIENDS_WITH]->(:Person) OR (other)-[:WORKS_IN]->(:Location) OR other.age = 5
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+---------------------------+----------------------------------------+----------------+------
+---------+-----------------------+
| Operator                  | Details                                | Estimated Rows | Rows | DB
Hits | Page Cache Hits/Misses |
+---------------------------+----------------------------------------+----------------+------
+---------+-----------------------+
| +ProduceResults           | `other.name`                           |             12 |   14 |
0 |                    0/0 |
| |                         +----------------------------------------+----------------+------
+---------+-----------------------+
| +Projection               | other.name AS `other.name`             |             12 |   14 |
14 |                    1/0 |
| |                         +----------------------------------------+----------------+------
+---------+-----------------------+
| +SelectOrSemiApply        | anon_9                                 |             14 |   14 |
0 |                    0/0 |
| |\                        +----------------------------------------+----------------+------
+---------+-----------------------+
| | +Filter                 | anon_7:Location                        |             14 |    0 |
10 |                    0/0 |
| | |                       +----------------------------------------+----------------+------
+---------+-----------------------+
| | +Expand(All)            | (other)-[anon_6:WORKS_IN]->(anon_7)    |             14 |   10 |
38 |                   20/0 |
| | |                       +----------------------------------------+----------------+------
+---------+-----------------------+
| | +Argument               | other                                  |             14 |   10 |
0 |                    0/0 |
| |                         +----------------------------------------+----------------+------
+---------+-----------------------+
| +LetSelectOrAntiSemiApply | other.age = $autoint_0                 |             14 |   14 |
14 |                    0/0 |
| |\                        +----------------------------------------+----------------+------
+---------+-----------------------+
| | +Filter                 | anon_5:Person                          |             12 |    0 |
10 |                    0/0 |
| | |                       +----------------------------------------+----------------+------
+---------+-----------------------+
| | +Expand(All)            | (other)-[anon_4:FRIENDS_WITH]->(anon_5) |             12 |   10 |
51 |                   28/0 |
```

```
| | |                      +----------------------------------------+---------------+------
+---------+-----------------------+
| | +Argument               | other                                  |             14 |   14 |
0 |                    0/0 |
| |                         +----------------------------------------+---------------+------
+---------+-----------------------+
| +NodeByLabelScan          | other:Person                           |             14 |   14 |
35 |                     1/0 |
+---------------------------+----------------------------------------+---------------+------
+---------+-----------------------+

Total database accesses: 208, total allocated memory: 64
```

Roll Up Apply

The `RollUpApply` operator is used to execute an expression which takes as input a pattern, and returns a list with content from the matched pattern; for example, when using a pattern expression or pattern comprehension in a query. This operator is a variation of the `Apply` operator.

`RollUpApply` is only applied by the slotted runtime. In the pipelined runtime, the task of `RollUpApply` is instead performed by the `Apply` and `EagerAggregation` operators.

Example 552. RollUpApply

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (p:Person)
RETURN p.name, [(p)-[:WORKS_IN]->(location) | location.name] AS cities
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+------------------------------------+----------------+------+---------
+-------------------------+
| Operator        | Details                            | Estimated Rows | Rows | DB Hits | Page Cache
Hits/Misses |
+-----------------+------------------------------------+----------------+------+---------
+-------------------------+
| +ProduceResults | `p.name`, cities                   |             14 |   14 |       0 |
0/0 |
| |               +------------------------------------+----------------+------+---------
+-------------------------+
| +Projection     | p.name AS `p.name`                 |             14 |   14 |      14 |
0/0 |
| |               +------------------------------------+----------------+------+---------
+-------------------------+
| +RollUpApply    | cities, anon_0                     |             14 |   14 |       0 |
0/0 |
| |\              +------------------------------------+----------------+------+---------
+-------------------------+
| | +Projection   | location.name AS anon_0            |             15 |   15 |      15 |
1/0 |
| | |             +------------------------------------+----------------+------+---------
+-------------------------+
| | +Expand(All)  | (p)-[anon_2:WORKS_IN]->(location) |             15 |   15 |      53 |
28/0 |
| | |             +------------------------------------+----------------+------+---------
+-------------------------+
| | +Argument     | p                                  |             14 |   14 |       0 |
```

```
0/0 |
| |              +----------------------------------+---------------+------+---------
+------------------------+
| +NodeByLabelScan| p:Person                         |            14 |   14 |      35 |
1/0 |
+-----------------+----------------------------------+---------------+------+---------
+------------------------+

Total database accesses: 153, total allocated memory: 64
```

TransactionApply

`TransactionApply` works like the `Apply` operator but will commit the current transaction after a specified number of rows. With `IN CONCURRENT TRANSACTIONS`, `PROFILE` also reports the concurrency the operator achieved. For more information, see Measuring concurrency with `PROFILE`.

Example 553. TransactionApply

> The below query uses a variable scope clause to import variables into the `CALL` subquery.

Query

```
PROFILE
LOAD CSV FROM 'https://neo4j.com/docs/cypher-refcard/3.3/csv/artists.csv' AS line
CALL (line) {
  CREATE (a: Artist {name: line[0]})
  RETURN a
} IN TRANSACTIONS OF 100 ROWS
RETURN a;
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+-------------------------------------------------+---------------+------
+---------+---------------+------------------------+-----------+--------------------+
| Operator         | Id | Details                                         | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+------------------+----+-------------------------------------------------+---------------+------
+---------+---------------+------------------------+-----------+--------------------+
| +ProduceResults  |  0 | a                                               |            10 |    4 |
8 |              0 |                        |           |                    |
| |                +----+-------------------------------------------------+---------------+------
+---------+---------------+                        |           |                    |
| +TransactionApply |  1 | IN TRANSACTIONS OF $autoint_1 ROWS ON ERROR FAIL |            10 |    4 |
0 |           2152 |                    0/0 |     2.036 | Fused in Pipeline 3 |
| |\               +----+-------------------------------------------------+---------------+------
+---------+---------------+------------------------+-----------+--------------------+
| | +Create        |  2 | (a:Artist {name: line[$autoint_0]})             |            10 |    4 |
16 |                |                        |           |                    |
| | |              +----+-------------------------------------------------+---------------+------
+---------+---------------+                        |           |                    |
| | +Argument      |  3 | line                                            |            10 |    4 |
0 |           3472 |                    0/0 |    32.746 | Fused in Pipeline 2 |
| |                +----+-------------------------------------------------+---------------+------
+---------+---------------+------------------------+-----------+--------------------+
| +LoadCSV         |  4 | line                                            |            10 |    4 |
0 |            328 |                        |           | In Pipeline 1      |
```

```
+------------------+----+-----------------------------------------------------+---------------+------
+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 24, total allocated memory: 5472
```

Argument

The `Argument` operator indicates the variable to be used as an argument to the right-hand side of an `Apply` operator.

Example 554. Argument

Query

```
PROFILE
MATCH (s:Person {name: 'me'})
MERGE (s)-[:FRIENDS_WITH]->(s)
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------
+----------------------+
| Operator        | Id | Details                                             | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            | Indexes
Used         |
+-----------------+----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------
+----------------------+
| +ProduceResults |  0 |                                                     |             1 |
0 |       0 |              0 |                       |           |                     |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+---------------+                       |           |
+----------------------+
| +EmptyResult    |  1 |                                                     |             1 |
0 |       0 |                |                       |           |                     |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+---------------+                       |           |
+----------------------+
| +Apply          |  2 |                                                     |             1 |
1 |       0 |                |                       |           |                     |
|
| |\              +----+-----------------------------------------------------+---------------
+------+---------+---------------+                       |           |
+----------------------+
| | +MergeInto    |  3 | MERGE (s)-[anon_0:FRIENDS_WITH]->(s)                |             1 |
1 |       1 |                |                       |           |                     |
|
| | |             +----+-----------------------------------------------------+---------------
+------+---------+---------------+                       |           |
+----------------------+
| | +Argument     |  4 | s                                                   |             1 |
1 |       0 |           2288 |                   1/0 |     0.673 | Fused in Pipeline 1 |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+---------------+-----------------------+-----------+--------------------
+----------------------+
| +NodeIndexSeek  |  5 | RANGE INDEX s:Person(name) WHERE name = $autostring_0 |             1 |
1 |       2 |            376 |                   1/0 |     0.256 | In Pipeline 0       |
```

```
range_person_name: 1 |
+-----------------+----+-----------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+

Total database accesses: 15, total allocated memory: 2232
```

Argument Tracker

The `ArgumentTracker` operator is used to ensure row-by-row semantics. This restricts the Cypher runtime to not batch operations in larger chunks.

Example 555. ArgumentTracker

> The below query uses a variable scope clause to import variables into the `CALL` subquery.

Query

```
PROFILE
MATCH (s:Person {name: 'me'})
CALL (s) {
   SET s.seen = coalesce(s.seen + 1,1)
   RETURN s.seen AS result
}
RETURN result;
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

+--------------------+----+-----------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| Operator           | Id | Details                                                   | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+--------------------+----+-----------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| +ProduceResults    |  0 | result                                                    |             1
|    1 |       0 |              0 |                       |           |                    |
| |                  +----+-----------------------------------------------------------+---------------
+------+---------+----------------+                       |           |                    |
| +Apply             |  1 |                               |           |                    |             1
|    0 |       0 |                |                       |           |                    |
| |\                 +----+-----------------------------------------------------------+---------------
+------+---------+----------------+                       |           |                    |
| | +ArgumentTracker |  7 |                                                           |             1
|    0 |       0 |            736 |                       |           |                    |
| | |                +----+-----------------------------------------------------------+---------------
+------+---------+----------------+                       |           |                    |
| | +Projection      |  2 | s.seen AS result                                          |             1
|    1 |       2 |                |                       |           |                    |
| | |                +----+-----------------------------------------------------------+---------------
+------+---------+----------------+                       |           |                    |
| | +Eager           |  3 | read/set conflict for property: seen (Operator: 4 vs 2) |             1
|    1 |       0 |            976 |                   0/0 |     0.298 | Fused in Pipeline 2 |
| | |                +----+-----------------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------+
| | +SetProperty     |  4 | s.seen = coalesce(s.seen + $autoint_1, $autoint_2)        |             1
|    1 |       2 |                |                       |           |                    |
| | |                +----+-----------------------------------------------------------+---------------
+------+---------+----------------+                       |           |                    |
```

```
| | +Argument        |  5 | s                                                                  |                 1
|    1 |       0 |           2408 |                     2/0 |     1.734 | Fused in Pipeline 1 |
| |                  +----+--------------------------------------------------------------------+----------------
+------+---------+----------------+-------------------------+-----------+---------------------+
| +NodeIndexSeek     |  6 | RANGE INDEX s:Person(name) WHERE name = $autostring_0   |                 1
|    1 |       2 |            368 |                     1/0 |     0.183 | In Pipeline 0       |
+--------------------+----+--------------------------------------------------------------------+----------------
+------+---------+----------------+-------------------------+-----------+---------------------+

Total database accesses: 6, total allocated memory: 4136
```

Cartesian Product

The `CartesianProduct` operator produces a cartesian product of the two inputs — each row coming from the left child operator will be combined with all the rows from the right child operator. `CartesianProduct` generally exhibits bad performance and ought to be avoided if possible.

Example 556. CartesianProduct

Query

```
PROFILE
MATCH
  (p:Person),
  (t:Team)
RETURN p, t
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator           | Details  | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+--------------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults    | p, t     |            140 |  140 |       0 |                |
2/0 |     1.917 |                     |
| |                  +----------+----------------+------+---------+----------------
+------------------------+-----------+                     |
| +CartesianProduct  |          |            140 |  140 |       0 |           1736 |
|     1.209 | In Pipeline 2       |
| |\                 +----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| | +NodeByLabelScan | t:Team   |             10 |   10 |      11 |            136 |
1/0 |     1,145 | In Pipeline 1       |
| |                  +----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +NodeByLabelScan   | p:Person |             15 |   15 |      16 |            120 |
1/0 |     0,409 | In Pipeline 0       |
+--------------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 142, total allocated memory: 1816
```

Hash joins in general

Hash joins have two inputs: the build input and probe input. The query planner assigns these roles so that the smaller of the two inputs is the build input. The build input is pulled in eagerly, and is used to build a probe table. Once this is complete, the probe table is checked for each row coming from the probe input side.

In query plans, the build input is always the left operator, and the probe input the right operator.

Node Hash Join

The `NodeHashJoin` operator is a variation of the hash join. `NodeHashJoin` executes the hash join on node ids. As primitive types and arrays can be used, it can be done very efficiently.

Example 557. NodeHashJoin

Query

```
PROFILE
MATCH (bob:Person {name: 'Bob'})-[:WORKS_IN]->(loc)<-[:WORKS_IN]-(matt:Person {name: 'Mattias'})
USING JOIN ON loc
RETURN loc.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator         | Details                                                     | Estimated Rows | Rows
| DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults  | `loc.name`                                                  |             10 |    0
|       0 |                |                   0/0 |     0.000 |                     |
| |                +-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                      |
| +Projection      | loc.name AS `loc.name`                                      |             10 |    0
|       0 |                |                   0/0 |     0.000 |                     |
| |                +-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                      |
| +Filter          | not anon_0 = anon_1                                         |             10 |    0
|       0 |                |                   0/0 |     0.000 |                     |
| |                +-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                      |
| +NodeHashJoin    | loc                                                         |             10 |    0
|       0 |           3688 |                       |     0.053 | In Pipeline 2       |
| |\               +-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| | +Expand(All)   | (matt)-[anon_1:WORKS_IN]->(loc)                             |             19 |    0
|       0 |                |                       |           |                     |
| | |              +-------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                      |
| | +NodeIndexSeek | RANGE INDEX matt:Person(name) WHERE name = $autostring_1 |              1 |    0
|       1 |            120 |                   1/0 |     0.288 | Fused in Pipeline 1 |
| |                +-------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +Expand(All)     | (bob)-[anon_0:WORKS_IN]->(loc)                              |             19 |    1
|       4 |                |                       |           |                     |
| |                +-------------------------------------------------------------+----------------
+------+---------+---------------+                       |           |                      |
| +NodeIndexSeek   | RANGE INDEX bob:Person(name) WHERE name = $autostring_0  |              1 |    1
|       2 |            120 |                   3/0 |     0.556 | Fused in Pipeline 0 |
```

```
+-----------------+------------------------------------------------------------+---------------
+------+---------+---------------+----------------------+-----------+--------------------+

Total database accesses: 7, total allocated memory: 3888
```

Value Hash Join

The `ValueHashJoin` operator is a variation of the hash join. This operator allows for arbitrary values to be used as the join key. It is most frequently used to solve predicates of the form: `n.prop1 = m.prop2` (i.e. equality predicates between two property columns).

Example 558. ValueHashJoin

Query

```
PROFILE
MATCH
  (p:Person),
  (q:Person)
WHERE p.age = q.age
RETURN p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator          | Details       | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults    | p, q         |             10 |    0 |       0 |                |
0/0 |     0.000 |                     |
| |                 +---------------+----------------+------+---------+----------------
+------------------------+-----------+                     |
| +ValueHashJoin     | p.age = q.age|             10 |    0 |       0 |            344 |
|           | In Pipeline 2       |
| |\                +---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| | +NodeByLabelScan| q:Person      |             15 |    0 |       0 |            120 |
0/0 |     0,000 | In Pipeline 1       |
| |                 +---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +NodeByLabelScan  | p:Person      |             15 |   15 |      16 |            120 |
1/0 |     0,211 | In Pipeline 0       |
+-------------------+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 71, total allocated memory: 664
```

Node Left/Right Outer Hash Join

The `NodeLeftOuterHashJoin` and `NodeRightOuterHashJoin` operators are variations of the hash join. The query below can be planned with either a left or a right outer join. The decision depends on the cardinalities of the left-hand and right-hand sides; i.e. how many rows would be returned, respectively, for

`(a:Person)` and `(a)-->(b:Person)`. If `(a:Person)` returns fewer results than `(a)-->(b:Person)`, a left outer join — indicated by `NodeLeftOuterHashJoin` — is planned. On the other hand, if `(a:Person)` returns more results than `(a)-->(b:Person)`, a right outer join — indicated by `NodeRightOuterHashJoin` — is planned instead.

Example 559. NodeRightOuterHashJoin

Query

```
PROFILE
MATCH (a:Person)
OPTIONAL MATCH (a)-->(b:Person)
USING JOIN ON a
RETURN a.name, b.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------+-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| Operator                | Details                                             | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------------+-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +ProduceResults         | `a.name`, `b.name`                                  |             14 |
16 |       0 |               |                   0/0 |     0.102 |                     |
| |                       +-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                     |
| +Projection             | cache[a.name] AS `a.name`, cache[b.name] AS `b.name` |             14 |
16 |       8 |               |                   0/0 |     0.055 |                     |
| |                       +-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                     |
| +NodeRightOuterHashJoin | a                                                   |             14 |
16 |       0 |          4232 |                       |     0.269 | In Pipeline 2       |
| |\                      +-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| | +NodeByLabelScan      | a:Person                                            |             15 |
15 |      16 |           120 |                   1/0 |     0,049 | In Pipeline 1       |
| |                       +-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+
| +CacheProperties        | cache[b.name], cache[a.name]                        |             13 |
13 |      39 |               |                       |           |                     |
| |                       +-----------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| +Expand(All)            | (b)<-[anon_0]-(a)                                   |             13 |
13 |      55 |               |                       |           |                     |
| |                       +-----------------------------------------------------+----------------
+------+---------+---------------+                       |           |                     |
| +NodeByLabelScan        | b:Person                                            |             15 |
15 |      16 |           120 |                   5/0 |     1,150 | Fused in Pipeline 0 |
+-------------------------+-----------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------+

Total database accesses: 211, total allocated memory: 4312
```

Traversal operators

Traversal operators enable complex graph traversals by defining how nodes and relationships are connected in a given pattern. They allow Cypher to express and match various pattern types, such as

fixed-length patterns, variable-length patterns, shortest paths, and non-linear patterns.

Anti

The `Anti` operator tests for the absence of a pattern. If there are incoming rows, the `Anti` operator will yield no rows. If there are no incoming rows, the `Anti` operator will yield a single row.

Example 560. Anti

Query

```
PROFILE
MATCH
  (me:Person {name: 'me'})-[:WORKS_IN]->(location),
  (other:Person)-[:WORKS_IN]->(location)
WHERE NOT (me)-[:FRIENDS_WITH]->(other)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------
+----------------------+
| Operator        | Id | Details                                                   | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            | Indexes
Used         |
+-----------------+----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------
+----------------------+
| +ProduceResults |  0 | `other.name`                                              |              0 |
5 |       0 |              0 |                   0/0 |     0.080 |                     |
|
| |               +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+
+----------------------+
| +Projection     |  1 | other.name AS `other.name`                                |              0 |
5 |      10 |                |                   0/0 |     0.114 | In Pipeline 2       |
|
| |               +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------
+----------------------+
| +Apply          |  2 |                                                           |              0 |
5 |       0 |                |                   0/0 |           |                     |
|
| |\              +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------
+----------------------+
| | +Anti         | 10 |                                                           |              0 |
5 |       0 |            936 |                   0/0 |     0.096 | In Pipeline 2       |
|
| | |             +----+-----------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+---------------------
+----------------------+
| | +Limit        |  9 | 1                                                         |              0 |
1 |       0 |            752 |                     |           |                     |
|
| | |             +----+-----------------------------------------------------------+----------------
+------+---------+---------------+                       |           |
+----------------------+
| | +Expand(Into) |  3 | (me)-[:FRIENDS_WITH]->(other)                             |              0 |
1 |       7 |            592 |                     |           |                     |
|
```

```
| | |            +----+-------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| | +Argument     |  4 | me, other                                             |              0 |
6 |       0 |           6520 |                   0/0 |     0.533 | Fused in Pipeline 1 |
|
| |              +----+-------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------
+----------------------+
| +Filter         |  5 | NOT anon_0 = anon_1 AND other:Person                  |              0 |
6 |      12 |               |                       |           |                     |
|
| |              +----+-------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| +Expand(All)    |  6 | (location)<-[anon_1:WORKS_IN]-(other)                 |              0 |
7 |       8 |               |                       |           |                     |
|
| |              +----+-------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| +Expand(All)    |  7 | (me)-[anon_0:WORKS_IN]->(location)                    |              1 |
1 |       2 |               |                       |           |                     |
|
| |              +----+-------------------------------------------------------+----------------
+------+---------+----------------+                       |           |
+----------------------+
| +NodeIndexSeek  |  8 | RANGE INDEX me:Person(name) WHERE name = $autostring_0 |              1 |
1 |       2 |            376 |                   3/0 |     0.866 | Fused in Pipeline 0 |
range_person_name: 1 |
+-----------------+----+-------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+---------------------
+----------------------+
09:13:20   E

Total database accesses: 178, total allocated memory: 6744
```

Optional

The `Optional` operator is used to solve some OPTIONAL MATCH queries. It will pull data from its source, simply passing it through if any data exists. However, if no data is returned by its source, `Optional` will yield a single row with all columns set to `null`.

Example 561. Optional

Query

```
PROFILE
MATCH (p:Person {name: 'me'})
OPTIONAL MATCH (q:Person {name: 'Lulu'})
RETURN p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-------------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+---------------+
| Operator        | Details                                               | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+-----------------+-------------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+---------------+
```

```
| +ProduceResults  | p, q                                                                 |              1 |    1 |
0 |                  |                    2/0 |     0.079 | In Pipeline 2 |
| |                  +--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------+
| +Apply           |                                                                      |              1 |    1 |
0 |                  |                    0/0 |     0.096 |               |
| |\                 +--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------+
| | +Optional      | p                                                                    |              1 |    1 |
0 |              768 |                    0/0 |     0.043 | In Pipeline 2 |
| | |              +--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------+
| | +NodeIndexSeek | RANGE INDEX q:Person(name) WHERE name = $autostring_1 |              1 |    0 |
1 |             2152 |                    1/0 |     0.098 | In Pipeline 1 |
| |                  +--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------+
| +NodeIndexSeek   | RANGE INDEX p:Person(name) WHERE name = $autostring_0 |              1 |    1 |
2 |             120 |                    0/1 |     0.364 | In Pipeline 0 |
+-----------------+--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------+

Total database accesses: 3, total allocated memory: 3000
```

Expand All

Given a start node, and depending on the pattern relationship, the `Expand(All)` operator will traverse incoming or outgoing relationships.

Example 562. Expand(All)

Query

```
PROFILE
MATCH (p:Person {name: 'me'})-[:FRIENDS_WITH]->(fof)
RETURN fof
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| Operator        | Details                                                | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+
| +ProduceResults | fof                                                    |              1 |    2 |
0 |                 |                         |           |                     |
| |               +--------------------------------------------------------+----------------+------
+---------+-----------------+                       |           |                     |
| +Expand(All)    | (p)-[anon_0:FRIENDS_WITH]->(fof)                     |              1 |    2 |
5 |                 |                         |           |                     |
| |               +--------------------------------------------------------+----------------+------
+---------+-----------------+                       |           |                     |
| +NodeIndexSeek  | RANGE INDEX p:Person(name) WHERE name = $autostring_0 |              1 |    1 |
2 |             120 |                    4/1 |     1.137 | Fused in Pipeline 0 |
+-----------------+--------------------------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+---------------------+

Total database accesses: 7, total allocated memory: 184
```

Expand Into

When both the start and end node have already been found, the `Expand(Into)` operator is used to find all relationships connecting the two nodes. As both the start and end node of the relationship are already in scope, the node with the smallest degree will be used. This can make a noticeable difference when dense nodes appear as end points.

Example 563. Expand(Into)

Query

```
PROFILE
MATCH (p:Person {name: 'me'})-[:FRIENDS_WITH]->(fof)-->(p)
RETURN fof
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-------------------------+-----------+---------------------+
| Operator        | Details                                                   | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-------------------------+-----------+---------------------+
| +ProduceResults | fof                                                       |              0 |    0 |
0 |                 |                         |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                     |
| +Filter         | not anon_1 = anon_0                                       |              0 |    0 |
0 |                 |                         |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                     |
| +Expand(Into)   | (p)-[anon_0:FRIENDS_WITH]->(fof)                          |              0 |    0 |
6 |             896 |                         |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                     |
| +Expand(All)    | (p)<-[anon_1]-(fof)                                       |              1 |    1 |
5 |                 |                         |           |                     |
| |               +-----------------------------------------------------------+----------------+------
+---------+----------------+                         |           |                     |
| +NodeIndexSeek  | RANGE INDEX p:Person(name) WHERE name = $autostring_0 |              1 |    1 |
2 |             120 |                     4/1 |     0.546 | Fused in Pipeline 0 |
+-----------------+-----------------------------------------------------------+----------------+------
+---------+----------------+-------------------------+-----------+---------------------+

Total database accesses: 13, total allocated memory: 976
```

Optional Expand All

The `OptionalExpand(All)` operator is analogous to Expand(All), apart from when no relationships match the direction, type and property predicates. In this situation, `OptionalExpand(all)` will return a single row with the relationship and end node set to `null`.

Example 564. OptionalExpand(All)

Query

```
PROFILE
MATCH (p:Person)
OPTIONAL MATCH (p)-[works_in:WORKS_IN]->(l)
  WHERE works_in.duration > 180
RETURN p, l
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+-------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator             | Details                                                                 |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------+-------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults      | p, l                                                                    |
14 |   15 |       1 |                |                          |           |                     |
| |                    +-------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +OptionalExpand(All) | (p)-[works_in:WORKS_IN]->(l) WHERE works_in.duration > $autoint_0 |
14 |   15 |      53 |                |                          |           |                     |
| |                    +-------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeByLabelScan     | p:Person                                                                |
14 |   14 |      15 |            120 |                    5/0 |     1,233 | Fused in Pipeline 0 |
+----------------------+-------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 125, total allocated memory: 184
```

Optional Expand Into

The `OptionalExpand(Into)` operator is analogous to Expand(Into), apart from when no matching relationships are found. In this situation, `OptionalExpand(Into)` will return a single row with the relationship and end node set to `null`. As both the start and end node of the relationship are already in scope, the node with the smallest degree will be used. This can make a noticeable difference when dense nodes appear as end points.

Example 565. OptionalExpand(Into)

Query

```
PROFILE
MATCH (p:Person)-[works_in:WORKS_IN]->(l)
OPTIONAL MATCH (l)-->(p)
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator             | Details                     | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+----------------------+-----------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +ProduceResults      | p                           |             15 |   15 |       0 |
|                      |          |                    |
| |                    +-----------------------------+----------------+------+---------
+----------------+                          |          |                    |
| +OptionalExpand(Into) | (l)-[anon_0]->(p)           |             15 |   15 |     105 |
3360 |                       |          |                    |
| |                    +-----------------------------+----------------+------+---------
+----------------+                          |          |                    |
| +Expand(All)         | (p)-[works_in:WORKS_IN]->(l) |             15 |   15 |      39 |
|                      |          |                    |
| |                    +-----------------------------+----------------+------+---------
+----------------+                          |          |                    |
| +NodeByLabelScan     | p:Person                    |             14 |   14 |      15 |
120 |                     7/0 |     3,925 | Fused in Pipeline 0 |
+----------------------+--- -------------------------+----------------+------+---------+
----------------+-----------------------+-----------+---------------------+

Total database accesses: 215, total allocated memory: 3440
```

VarLength Expand All

Given a start node, the `VarLengthExpand(All)` operator will traverse variable-length and quantified relationships.

Example 566. VarLengthExpand(All)

Query

```
PROFILE
MATCH (p:Person)-[:FRIENDS_WITH *1..2]-(q:Person)
RETURN p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+-----------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator             | Details                           | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+----------------------+-----------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +ProduceResults      | p, q                              |             40 |   48 |       0 |
|                      |          |                    |
| |                    +-----------------------------------+----------------+------+---------
```

```
+----------------+                          |           |                    |
| +Filter               | q:Person                          |            40 |   48 |      96 |
|                        |           |                     |
| |                     +-----------------------------------+---------------+------+---------
+----------------+                          |           |                    |
| +VarLengthExpand(All) | (p)-[anon_0:FRIENDS_WITH*..2]-(q) |            40 |   48 |     151 |
128 |                         |           |                      |
| |                     +-----------------------------------+---------------+------+---------
+----------------+                          |           |                    |
| +NodeByLabelScan      | p:Person                          |            14 |   14 |      15 |
120 |                      6/0 |    10,457 | Fused in Pipeline 0 |
+----------------------+-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+

Total database accesses: 318, total allocated memory: 208
```

VarLength Expand Into

When both the start and end node have already been found, the `VarLengthExpand(Into)` operator is used to find all variable-length and quantified relationships connecting the two nodes.

Example 567. VarLengthExpand(Into)

Query

```
PROFILE
MATCH (p:Person)-[:FRIENDS_WITH *1..2]-(p:Person)
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| Operator              | Details                           | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------+-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +ProduceResults       | p                                 |              3 |    4 |       0 |
|                        |           |                     |
| |                     +-----------------------------------+---------------+------+---------
+----------------+                          |           |                    |
| +VarLengthExpand(Into) | (p)-[anon_0:FRIENDS_WITH*..2]-(p) |              3 |    4 |     151 |
128 |                         |           |                      |
| |                     +-----------------------------------+---------------+------+---------
+----------------+                          |           |                    |
| +NodeByLabelScan      | p:Person                          |             14 |   14 |      15 |
120 |                      6/0 |     0,797 | Fused in Pipeline 0 |
+-----------------------+-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+


Total database accesses: 222, total allocated memory: 192
```

VarLength Expand Pruning

Given a start node, the `VarLengthExpand(Pruning)` operator will traverse variable-length and quantified

relationships much like the `VarLengthExpand(All)` operator. However, as an optimization, some paths will not be explored if they are guaranteed to produce an end node that has already been found (by means of a previous path traversal).

This kind of expand is only planned when:

- The individual paths are not of interest.
- The relationships have an upper bound.

The `VarLengthExpand(Pruning)` operator guarantees that all the end nodes produced will be unique.

Example 568. VarLengthExpand(Pruning)

Query

```
PROFILE
MATCH (p:Person)-[:FRIENDS_WITH *3..4]-(q:Person)
RETURN DISTINCT p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------------+----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------+
| Operator                  | Id | Details                       | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by | Pipeline      |
+---------------------------+----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------+
| +ProduceResults           |  0 | p, q                          |              0 |    0 |       0 |
|                     0/0 |    0.005 |            |               |
| |                         +----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+            |               |
| +OrderedDistinct          |  1 | p, q                          |              0 |    0 |       0 |
40 |                    0/0 |    0.014 |            |               |
| |                         +----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+            |               |
| +Filter                   |  2 | q:Person                      |              0 |    0 |       0 |
|                     0/0 |    0.014 |            |               |
| |                         +----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+            |               |
| +VarLengthExpand(Pruning) |  3 | (p)-[:FRIENDS_WITH*3..4]-(q) |              1 |    0 |      15 |
400 |                         |           |            | In Pipeline 1 |
| |                         +----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+            +---------------+
| +NodeByLabelScan          |  4 | p:Person                      |             14 |   14 |      15 |
120 |                    1/0 |    0.020 | p ASC      | In Pipeline 0 |
+---------------------------+----+-------------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------+

Total database accesses: 30, total allocated memory: 480
```

Breadth First VarLength Expand Pruning

Given a start node, the `VarLengthExpand(Pruning,BFS,All)` operator traverses variable-length and quantified relationships much like the `VarLengthExpand(All)` operator. However, as an optimization, it

instead performs a breadth-first search (BFS) and while expanding, some paths are not explored if they are guaranteed to produce an end node that has already been found (by means of a previous path traversal). This is only used in cases where the individual paths are not of interest.

This kind of expand is only planned when:

- The individual paths are not of interest.
- The lower bound is either `0` or `1` (default).

This operator guarantees that all the end nodes produced are unique.

Query

```
PROFILE
MATCH (p:Person)-[:FRIENDS_WITH *..4]-(q:Person)
RETURN DISTINCT p, q
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------------+----+-----------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------------+
| Operator                          | Id | Details                     | Estimated Rows | Rows | DB Hits
| Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by | Pipeline            |
+-----------------------------------+----+-----------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------------+
| +ProduceResults                   |  0 | p, q                        |             12 |    0 |       0
|               0 |                       |           |            |                     |
| |                                 +----+-----------------------------+----------------+------+---------
+----------------+                        |           |            |                     |
| +OrderedDistinct                  |  1 | p, q                        |             12 |    0 |       0
|              40 |                       |           |            |                     |
| |                                 +----+-----------------------------+----------------+------+---------
+----------------+                        |           |            |                     |
| +Filter                           |  2 | q:Person                    |             13 |    0 |       0
|                 |                       |           |            |                     |
| |                                 +----+-----------------------------+----------------+------+---------
+----------------+                        |           |            |                     |
| +VarLengthExpand(Pruning,BFS,All) |  3 | (p)-[:FRIENDS_WITH*..4]-(q) |             13 |    0 |      38
|             952 |                       |           |            |                     |
| |                                 +----+-----------------------------+----------------+------+---------
+----------------+                        |           |            |                     |
| +NodeByLabelScan                  |  4 | p:Person                    |             10 |   10 |      11
|             248 |                     3/0 |     4.662 | p ASC      | Fused in Pipeline 0 |
+-----------------------------------+----+-----------------------------+----------------+------+---------
+---------------+-----------------------+-----------+------------+---------------------+

Total database accesses: 49, total allocated memory: 1200
```

Repeat

Given a start node, the `Repeat` operator will traverse quantified path patterns that cannot be solved (or solved efficiently) with the `VarLengthExpand(All)` operator. Similar to an `Apply` operator, it takes a single row from the left-hand side and applies the operators on the right-hand side. In contrast to `Apply`, however, it repeatedly applies these operators in accordance with the quantifiers on the quantified path pattern. In the following example, the operator will repeat twice and produce rows for both repetitions.

Example 569. Repeat

Query

```
PROFILE
MATCH (me:Person) ((a)-[:FRIENDS_WITH]-(b)-[:FRIENDS_WITH]-(c) WHERE a.name <> b.name AND a.name <>
c.name AND b.name <> c.name){1,2} (friend:Person)
RETURN me, friend
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| Operator         | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| +ProduceResults  |  0 | me, friend
|              2 |   34 |     136 |              0 |                        |           |
|
| |                +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +Filter          |  1 | friend:Person
|              2 |   34 |      68 |                |                        |           |
|
| |                +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +NullifyMetadata |  9 |
|              2 |   34 |       0 |                |                        |           |
|
| |                +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +Repeat(Trail)   |  2 | (me) (...){1, 2} (friend)
|              2 |   34 |       0 |          30112 |                    0/0 |     4.943 | Fused in
Pipeline 2 |
| |\               +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| | +Filter        |  3 | NOT anon_5 = anon_3 AND (NOT cache[a.name] = cache[c.name] AND NOT
cache[b.name] = cache[c.name]) AN |              1 |   34 |      92 |                |
|           |                      |
| | |              |    | D isRepeatTrailUnique(anon_5)
|                |      |         |                |                        |           |
|
| | |              +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| | +Expand(All)   |  4 | (b)-[anon_5:FRIENDS_WITH]-(c)
|              3 |   92 |     138 |                |                        |           |
|
| | |              +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
```

```
|
| | +Filter        |  5 | NOT cache[a.name] = cache[b.name] AND isRepeatTrailUnique(anon_3)
|              5 |   46 |     198 |                |                           |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                           |           |
|
| | +Expand(All)   |  6 | (a)-[anon_3:FRIENDS_WITH]-(b)
|             10 |   66 |     100 |                |                           |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                           |           |
|
| | +Argument      |  7 | a
|             15 |   34 |       0 |          15992 |                       2/0 |     5.253 | Fused in
Pipeline 1 |
| |                +----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-------------------------+-----------+
---------------------+
| +NodeByLabelScan |  8 | me:Person
|             14 |   14 |      15 |            376 |                       1/0 |     1.130 | In Pipeline
0       |
+------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-------------------------+-----------+
---------------------+

Total database accesses: 747, total allocated memory: 45832
```

Nullify Metadata

`NullifyMetadata` is responsible for cleaning up the state produced by `Repeat`. It is only planned directly after `Repeat`.

Example 570. NullifyMetadata

Query

```
PROFILE
MATCH (me:Person) ((a)-[:FRIENDS_WITH]-(b)-[:FRIENDS_WITH]-(c) WHERE a.name <> b.name AND a.name <>
c.name AND b.name <> c.name){1,2} (friend:Person)
RETURN me, friend
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-------------------------+-----------+
---------------------+
| Operator         | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+------------------+----
+-------------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-------------------------+-----------+
---------------------+
| +ProduceResults  |  0 | me, friend
|              2 |   34 |     136 |              0 |                         |           |
```

```
|
| |               +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +Filter          |  1 | friend:Person
|              2 |   34 |      68 |                |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +NullifyMetadata |  9 |
|              2 |   34 |       0 |                |                        |           |
|
| |               +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| +Repeat(Trail)   |  2 | (me) (...){1, 2} (friend)
|              2 |   34 |       0 |          30112 |                    0/0 |     4.824 | Fused in
Pipeline 2 |
| |\              +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+
| | +Filter        |  3 | NOT anon_5 = anon_3 AND (NOT cache[a.name] = cache[c.name] AND NOT
cache[b.name] = cache[c.name]) AN |              1 |   34 |      92 |                |
|           |                     |
| | |              |    | D isRepeatTrailUnique(anon_5)
|                |      |         |                |                        |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| | +Expand(All)   |  4 | (b)-[anon_5:FRIENDS_WITH]-(c)
|              3 |   92 |     138 |                |                        |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| | +Filter        |  5 | NOT cache[a.name] = cache[b.name] AND isRepeatTrailUnique(anon_3)
|              5 |   46 |     198 |                |                        |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| | +Expand(All)   |  6 | (a)-[anon_3:FRIENDS_WITH]-(b)
|             10 |   66 |     100 |                |                        |           |
|
| | |              +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                        |           |
|
| | +Argument      |  7 | a
|             15 |   34 |       0 |          15992 |                    2/0 |     5.307 | Fused in
Pipeline 1 |
| |               +----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+
| +NodeByLabelScan |  8 | me:Person
|             14 |   14 |      15 |            376 |                    1/0 |     0.183 | In Pipeline
0       |
+------------------+----
+-------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
---------------------+

Total database accesses: 747, total allocated memory: 45832
```

Shortest path

The `ShortestPath` operator finds one or all shortest paths between two previously matched node variables. This operator is used for the `shortestPath()` and `allShortestPaths` functions.

Example 571. ShortestPath

Query

```
PROFILE
MATCH
  (andy:Person {name: 'Andy'}),
  (mattias:Person {name: 'Mattias'}),
  p = shortestPath((andy)-[*]-(mattias))
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+--------------------+-------------------------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| Operator           | Details                                                           | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+--------------------+-------------------------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| +ProduceResults    | p                                                                 |              1
|    1 |       0 |                |                    1/0 |     0.241 |               |
| |                  +-------------------------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+               |
| +ShortestPath      | p = (andy)-[anon_0*]-(mattias)                                    |              1
|    1 |       1 |           1424 |                        |           | In Pipeline 1 |
| |                  +-------------------------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| +MultiNodeIndexSeek | RANGE INDEX andy:Person(name) WHERE name = $autostring_0,    |              1
|    1 |       4 |            120 |                    1/1 |     0.308 | In Pipeline 0 |
|                    | RANGE INDEX mattias:Person(name) WHERE name = $autostring_1 |
|      |         |                |                        |           |               |
+--------------------+-------------------------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+

Total database accesses: 5, total allocated memory: 1488
```

StatefulShortestPath(Into)

The `StatefulShortestPath(Into)` operator finds shortest paths between a start node and a single target node. It uses a bidirectional breadth-first search (BFS) algorithm, which performs two BFS invocations at the same time, one from the left boundary node and one from the right boundary node. Once a node is found by both BFS invocations, which indicates that it can be reached from both boundary nodes, the algorithm successfully terminates. If one of the BFS invocations exhausts its search before intersecting, either because no further nodes can be reached or because the maximum number of hops has been reached, then there is no valid path between the boundary nodes and the algorithm terminates.

Example 572. StatefulShortestPath(Into)

Query

```
PROFILE
MATCH
  p = ALL SHORTEST (chris:Person {name: 'Chris'})(()-[]-()-[]-()){1,}(stefan:Person {name: 'Stefan'})
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| Operator                    | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +ProduceResults             |  0 | p
|              2 |    2 |       0 |              0 |                    0/0 |     0.039 |
|
| |                           +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +Projection                 |  1 | (chris) ((anon_12)-[anon_14]-(anon_13)-[anon_11]-())* (stefan)
AS p                            |              2 |    2 |       0 |                |
0/0 |     1.365 |               |
| |                           +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +StatefulShortestPath(Into) |  2 | SHORTEST 1 GROUPS (chris) ((`anon_5`)-[`anon_6`]-(`anon_7`)-
[`anon_8`]-(`anon_9`)){1, } (stefan) |              2 |    2 |      39 |          22237 |
1/0 |    37.376 | In Pipeline 1 |
| |                           +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +MultiNodeIndexSeek         |  3 | UNIQUE chris:Person(name) WHERE name = $autostring_0,
|              1 |    1 |       4 |            376 |                    1/1 |    10.245 | In Pipeline
0 |
|                             |    | UNIQUE stefan:Person(name) WHERE name = $autostring_1
|                |      |         |                |                        |           |
|
+-----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+

Total database accesses: 43, total allocated memory: 22557
```

StatefulShortestPath(All)

The `StatefulShortestPath(All)` operator finds shortest paths from a single node to multiple target nodes. It uses a breadth-first search algorithm.

Example 573. StatefulShortestPath(All)

Query

```
PROFILE
MATCH
  p = ALL SHORTEST (chris:Person {name:'Chris'})(()-[]-()-[]-()){1,}(location:Location)
RETURN length(p) AS pathLength, location.name AS locationName
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| Operator                   | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +ProduceResults            |  0 | pathLength, locationName
|             14 |   20 |       0 |              0 |                    0/0 |     0.074 |
|
| |                          +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +Projection                |  1 | length((chris) ((anon_12)-[anon_14]-(anon_13)-[anon_11]-())*
(location)) AS pathLength,            |             14 |   20 |      40 |                |
1/0 |     6.828 |               |
| |                          |    | location.name AS locationName
|                |      |         |                |                         |           |
|
| |                          +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------+
|
| +StatefulShortestPath(All) |  2 | SHORTEST 1 GROUPS (chris) ((`anon_5`)-[`anon_6`]-(`anon_7`)-
[`anon_8`]-(`anon_9`)){1, } (location) |             14 |   20 |     179 |          37663 |
1/0 |    52.849 | In Pipeline 1 |
| |                          |    |         expanding from: chris
|                |      |         |                |                         |           |
|
| |                          |    |     inlined predicates: location:Location
|                |      |         |                |                         |           |
|
| |                          +----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+
| +NodeUniqueIndexSeek       |  3 | UNIQUE chris:Person(name) WHERE name = $autostring_0
|              1 |    1 |       2 |            376 |                    0/1 |     9.078 | In Pipeline
0 |
+----------------------------+----
+-----------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+

Total database accesses: 221, total allocated memory: 37983
```

Triadic Selection

The `TriadicSelection` operator is used to solve triangular queries, such as the very common 'find my friends-of-friends that are not already my friend'. It does so by putting all the friends into a set, and uses the set to check if the friends-of-friends are already connected to me.

`TriadicSelection` is only applied by the slotted runtime. In the pipelined runtime, the task of `TriadicSelection` is instead performed by the `TriadicBuild`, `Apply`, and `TriadicFilter` operators.

Example 574. TriadicSelection

The example finds the names of all friends of my friends that are not already my friends.

Query

```
PROFILE
CYPHER runtime=slotted
MATCH (me:Person)-[:FRIENDS_WITH]-(:Person)-[:FRIENDS_WITH]-(other:Person)
WHERE NOT (me)-[:FRIENDS_WITH]-(other)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-------------------+----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| Operator          | Id | Details                                | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses |
+-------------------+----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +ProduceResults   |  0 | `other.name`                           |             35 |   24 |       0 |
0 |                   0/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +Projection       |  1 | other.name AS `other.name`             |             35 |   24 |      24 |
|                    0/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +Filter           |  2 | NOT anon_2 = anon_0 AND other:Person   |             35 |   24 |      24 |
|                    0/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +TriadicSelection |  3 | WHERE NOT (me)--(other)                |             38 |   48 |       0 |
|                    0/0 |
| |\                +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| | +Expand(All)    |  4 | (anon_1)-[anon_2:FRIENDS_WITH]-(other) |             41 |   48 |      72 |
|                    0/0 |
| | |               +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| | +Argument       |  5 | anon_1, anon_0                         |             24 |   24 |       0 |
|                    0/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +Filter           |  6 | anon_1:Person                          |             24 |   24 |      24 |
|                    0/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +Expand(All)      |  7 | (me)-[anon_0:FRIENDS_WITH]-(anon_1)    |             24 |   24 |      38 |
|                    2/0 |
| |                 +----+----------------------------------------+----------------+------+---------
+----------------+------------------------+
| +NodeByLabelScan  |  8 | me:Person                              |             14 |   14 |      15 |
```

```
|                  1/0 |
+------------------+----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+

Total database accesses: 246, total allocated memory: 64
```

Triadic Build

The `TriadicBuild` operator is used in conjunction with `TriadicFilter` to solve triangular queries, such as the very common 'find my friend-of-friends that are not already my friend'. These two operators are specific to Pipelined runtime and together perform the same logic as `TriadicSelection` does for other runtimes. `TriadicBuild` builds a set of all friends, which is later used by `TriadicFilter`. The example finds the names of all friends of my friends that are not already my friends.

Example 575. TriadicBuild

Query

```
PROFILE
MATCH (me:Person)-[:FRIENDS_WITH]-(:Person)-[:FRIENDS_WITH]-(other:Person)
WHERE NOT (me)-[:FRIENDS_WITH]-(other)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| Operator         | Id | Details                                | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| +ProduceResults  |  0 | `other.name`                           |            40 |   24 |       0 |
0 |                   0/0 |     0.102 |                     |
| |                +----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+                     |
| +Projection      |  1 | other.name AS `other.name`             |            40 |   24 |      48 |
|                    0/0 |     0.077 |                     |
| |                +----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+                     |
| +Filter          |  2 | NOT anon_2 = anon_0 AND other:Person   |            40 |   24 |      48 |
|                    0/0 |     0.259 |                     |
| |                +----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+                     |
| +TriadicFilter   | 11 | WHERE NOT (me)--(other)                |            48 |   45 |       0 |
7552 |                    0/0 |     0.387 | In Pipeline 3       |
| |                +----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| +Apply           | 10 |                                        |            48 |   55 |       0 |
|                    0/0 |           |                     |
| |\               +----+----------------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------------+
| | +Expand(All)   |  4 | (anon_1)-[anon_2:FRIENDS_WITH]-(other) |            48 |   55 |      80 |
|                      |           |                     |
| | |              +----+----------------------------------------+---------------+------+---------
+---------------+                       |           |                     |
| | +Argument      |  5 | anon_1, anon_0                         |            26 |   25 |       0 |
4464 |                    0/0 |     0.503 | Fused in Pipeline 2 |
| |                +----+----------------------------------------+---------------+------+---------
```

```
+----------------+-----------------------+-----------+--------------------+
| +TriadicBuild  |  9 | (me)--(anon_1)                        |            26 |   25 |       0 |
1104 |                    0/0 |     2.199 | In Pipeline 1       |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +Filter        |  6 | anon_1:Person                         |            26 |   25 |      50 |
|                      |           |                       |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+                       |           |                    |
| +Expand(All)   |  7 | (me)-[anon_0:FRIENDS_WITH]-(anon_1)   |            26 |   25 |      25 |
|                      |           |                       |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+                       |           |                    |
| +NodeByLabelScan |  8 | me:Person                           |            14 |   14 |      15 |
376 |                    3/0 |     0.760 | Fused in Pipeline 0 |
+------------------+----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+

Total database accesses: 256, total allocated memory: 7376
```

Triadic Filter

The `TriadicFilter` operator is used in conjunction with `TriadicBuild` to solve triangular queries, such as the very common 'find my friend-of-friends that are not already my friend'. These two operators are specific to Pipelined runtime and together perform the same logic as `TriadicSelection` does for other runtimes. `TriadicFilter` uses a set of friends previously built by `TriadicBuild` to check if the friend-of-friends are already connected to me. The example finds the names of all friends of my friends that are not already my friends.

Example 576. TriadicFilter

Query

```
PROFILE
MATCH (me:Person)-[:FRIENDS_WITH]-(:Person)-[:FRIENDS_WITH]-(other:Person)
WHERE NOT (me)-[:FRIENDS_WITH]-(other)
RETURN other.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| Operator         | Id | Details                                 | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+------------------+----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +ProduceResults  |  0 | `other.name`                            |            40 |   24 |       0 |
0 |                    0/0 |     0.102 |                     |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+                    |
| +Projection      |  1 | other.name AS `other.name`              |            40 |   24 |      48 |
|                    0/0 |     0.077 |                     |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+                    |
| +Filter          |  2 | NOT anon_2 = anon_0 AND other:Person    |            40 |   24 |      48 |
|                    0/0 |     0.259 |                     |
| |                +----+-----------------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+                    |
```

```
| +TriadicFilter   | 11 | WHERE NOT (me)--(other)               |            48 |   45 |       0 |
7552 |                    0/0 |     0.387 | In Pipeline 3       |
| |                +----+---------------------------------------+---------------+------+---------
+----------------+----------------------+-----------+--------------------+
| +Apply           | 10 |                                       |            48 |   55 |       0 |
|                    0/0 |           |                     |
| |\               +----+---------------------------------------+---------------+------+---------
+----------------+----------------------+-----------+--------------------+
| | +Expand(All)   |  4 | (anon_1)-[anon_2:FRIENDS_WITH]-(other) |            48 |   55 |      80 |
|                      |           |                     |
| | |              +----+---------------------------------------+---------------+------+---------
+----------------+                      |           |                     |
| | +Argument      |  5 | anon_1, anon_0                        |            26 |   25 |       0 |
4464 |                    0/0 |     0.503 | Fused in Pipeline 2 |
| |                +----+---------------------------------------+---------------+------+---------
+----------------+----------------------+-----------+--------------------+
| +TriadicBuild    |  9 | (me)--(anon_1)                        |            26 |   25 |       0 |
1104 |                    0/0 |     2.199 | In Pipeline 1       |
| |                +----+---------------------------------------+---------------+------+---------
+----------------+----------------------+-----------+--------------------+
| +Filter          |  6 | anon_1:Person                         |            26 |   25 |      50 |
|                      |           |                     |
| |                +----+---------------------------------------+---------------+------+---------
+----------------+                      |           |                     |
| +Expand(All)     |  7 | (me)-[anon_0:FRIENDS_WITH]-(anon_1)   |            26 |   25 |      25 |
|                     |           |                     |
| |                +----+---------------------------------------+---------------+------+---------
+----------------+                      |           |                     |
| +NodeByLabelScan |  8 | me:Person                             |            14 |   14 |      15 |
376 |                   3/0 |     0.760 | Fused in Pipeline 0 |
+------------------+----+---------------------------------------+---------------+------+---------
+----------------+----------------------+-----------+--------------------+

Total database accesses: 256, total allocated memory: 7376
```

Union operators

Union operators in Cypher combine the results from multiple query parts by merging their rows. For more information, see the page about the `UNION` clause.

Union

The `Union` operator concatenates the results from the right child operator with the results from the left child operator.

Example 577. Union

Query

```
PROFILE
MATCH (p:Location)
  RETURN p.name
UNION ALL
MATCH (p:Country)
  RETURN p.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128
```

```
+--------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| Operator           | Id | Details            | Estimated Rows | Rows | DB Hits | Memory (Bytes) |
Page Cache Hits/Misses | Time (ms) | Pipeline            |
+--------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +ProduceResults    |  0 | `p.name`           |             20 |    0 |       0 |                |
|            |                     |
| |                  +----+--------------------+----------------+------+---------+----------------+
|            |                     |
| +Union             |  1 |                    |             20 |    0 |       0 |              0 |
0/0 |     0.000 | Fused in Pipeline 2 |
| |\                 +----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| | +Projection      |  2 | `p.name`           |             10 |    0 |       0 |                |
|            |                     |
| | |                +----+--------------------+----------------+------+---------+----------------+
|            |                     |
| | +Projection      |  3 | p.name AS `p.name` |             10 |    0 |       0 |                |
|            |                     |
| | |                +----+--------------------+----------------+------+---------+----------------+
|            |                     |
| | +NodeByLabelScan |  4 | p:Country          |             10 |    0 |       0 |            120 |
0/0 |     0.049 | Fused in Pipeline 1 |
| |                  +----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +Projection        |  5 | `p.name`           |             10 |    0 |       0 |                |
|            |                     |
| |                  +----+--------------------+----------------+------+---------+----------------+
|            |                     |
| +Projection        |  6 | p.name AS `p.name` |             10 |    0 |       0 |                |
|            |                     |
| |                  +----+--------------------+----------------+------+---------+----------------+
|            |                     |
| +NodeByLabelScan   |  7 | p:Location         |             10 |    0 |       0 |            120 |
0/0 |     0.077 | Fused in Pipeline 0 |
+--------------------+----+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+

Total database accesses: 0, total allocated memory: 320
```

Aggregation operators

Aggregation operators are used to compute summary statistics over groups of graph data, enabling operations such as counting nodes, relationships or properties.

Eager Aggregation

The `EagerAggregation` operator evaluates a grouping expression and uses the result to group rows into different groupings. For each of these groupings, `EagerAggregation` will then evaluate all aggregation functions and return the result. To do this, `EagerAggregation`, as the name implies, must pull in all data eagerly from its source and build up state, which leads to increased memory pressure in the system.

Note:

The cost of memory pressure of the `EagerAggregation` operator is still much lower than that of the `Eager` operator. `EagerAggregation` consumes all input rows and outputs as many rows as there are groups, which generally reduces cardinality (row count). If there are no grouping keys, all input rows are in a single group, producing a single output row. In contrast, the `Eager` operator buffers all input rows in memory, then passes them on, with cardinality unchanged.

`EagerAggregation` can be replaced with a less expensive `OrderedAggregation` operator if the aggregation has a grouping key and the input is ordered by that key.

Example 578. EagerAggregation

Query

```
PROFILE
MATCH (l:Location)<-[:WORKS_IN]-(p:Person)
RETURN
  l.name AS location,
  collect(p.name) AS people
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------+----+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+
| Operator          | Id | Details                                             | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+
| +ProduceResults   |  0 | location, people                                    |              4 |
6 |       0 |                |                    0/0 |     0.022 | In Pipeline 1       |
| |                 +----+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+
| +EagerAggregation |  1 | cache[l.name] AS location, collect(p.name) AS people |              4 |
6 |      30 |           2584 |                        |           |                     |
| |                 +----+-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +Filter           |  2 | p:Person                                            |             15 |
15 |      30 |                |                        |           |                     |
| |                 +----+-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +Expand(All)      |  3 | (l)<-[anon_0:WORKS_IN]-(p)                          |             15 |
15 |      26 |                |                        |           |                     |
| |                 +----+-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +CacheProperties  |  4 | cache[l.name]                                       |             10 |
10 |      20 |                |                        |           |                     |
| |                 +----+-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +NodeByLabelScan  |  5 | l:Location                                          |             10 |
10 |      11 |            120 |                    4/0 |     0.813 | Fused in Pipeline 0 |
+-------------------+----+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+

Total database accesses: 117, total allocated memory: 2664
```

Ordered Aggregation

The `OrderedAggregation` operator is an optimization of the `EagerAggregation` operator that takes advantage of the ordering of the incoming rows. This operator uses lazy evaluation and has a lower memory pressure in the system than the `EagerAggregation` operator.

Example 579. OrderedAggregation

Query

```
PROFILE
MATCH (p:Person)
WHERE p.name STARTS WITH 'P'
RETURN p.name, count(*) AS count
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------
+---------------+
| Operator              | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline      |
+-----------------------
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------
+---------------+
| +ProduceResults       | `p.name`, count
|              0 |    2 |       0 |                |                    0/0 |     0.045 |
|               |
| |
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+              |
|
| +OrderedAggregation   | cache[p.name] AS `p.name`, count(*) AS count
|              0 |    2 |       0 |            288 |                    0/0 |     0.175 | `p.name`
ASC | In Pipeline 1 |
| |
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------
+---------------+
| +NodeIndexSeekByRange | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    2 |       3 |            120 |                    0/1 |     0.529
| p.name ASC   | In Pipeline 0 |
+-----------------------
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+--------------
+---------------+

Total database accesses: 3, total allocated memory: 352
```

Node Count From Count Store

The `NodeCountFromCountStore` operator uses the count store to answer questions about node counts. This is much faster than the `EagerAggregation` operator which achieves the same result by actually counting. However, as the count store only stores a limited range of combinations, `EagerAggregation` will still be used for more complex queries. For example, we can get counts for all nodes, and nodes with a label, but not nodes with more than one label.

Example 580. NodeCountFromCountStore

Query

```
PROFILE
MATCH (p:Person)
RETURN count(p) AS people
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------+-----------------------------+----------------+------+---------
+---------------+----------------------+-----------+--------------------+
| Operator                | Details                     | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-------------------------+-----------------------------+----------------+------+---------
+---------------+----------------------+-----------+--------------------+
| +ProduceResults         | people                      |              1 |    1 |       0 |
|                         |          |                      |
| |                       +-----------------------------+----------------+------+---------
+----------------+                         |          |                    |
| +NodeCountFromCountStore | count( (:Person) ) AS people |              1 |    1 |       1 |
120 |                    0/0 |     0.169 | Fused in Pipeline 0 |
+-------------------------+-----------------------------+----------------+------+---------
+---------------+----------------------+-----------+--------------------+

Total database accesses: 1, total allocated memory: 184
```

Relationship Count From Count Store

The `RelationshipCountFromCountStore` operator uses the count store to answer questions about relationship counts. This is much faster than the `EagerAggregation` operator which achieves the same result by actually counting. However, as the count store only stores a limited range of combinations, `EagerAggregation` will still be used for more complex queries. For example, we can get counts for all relationships, relationships with a type, relationships with a label on one end, but not relationships with labels on both end nodes.

Example 581. RelationshipCountFromCountStore

Query

```
PROFILE
MATCH (p:Person)-[r:WORKS_IN]->()
RETURN count(r) AS jobs
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128
```

```
+-----------------------------------+-----------------------------------------+----------------
+------+---------+---------------+----------------------+----------+--------------------+
| Operator                          | Details                                 | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------------------------+-----------------------------------------+----------------
+------+---------+---------------+----------------------+----------+--------------------+
| +ProduceResults                   | jobs                                    |              1 |
1 |       0 |               |                      |          |                    |
| |                                 +-----------------------------------------+----------------
+------+---------+---------------+                      |          |                    |
| +RelationshipCountFromCountStore | count( (:Person)-[:WORKS_IN]->() ) AS jobs |              1 |
1 |       1 |           120 |                  0/0 |    0.625 | Fused in Pipeline 0 |
+-----------------------------------+-----------------------------------------+----------------
+------+---------+---------------+----------------------+----------+--------------------+

Total database accesses: 1, total allocated memory: 184
```

Filter, order, and projection operators

The operators in this group handle how rows of data are transformed, filtered, and finalized for query results. They are responsible for crafting the structure of the returned data,

Empty Result

The `EmptyResult` operator eagerly loads all incoming data and discards it.

Example 582. EmptyResult

Query

```
PROFILE
CREATE (:Person)
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-----------------+---------------+------+---------+-----------------------
+-----------+--------------------+
| Operator        | Details         | Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time
(ms) | Pipeline            |
+-----------------+-----------------+---------------+------+---------+-----------------------
+-----------+--------------------+
| +ProduceResults |                 |              1 |    0 |       0 |                        |
|                     |
| |               +-----------------+---------------+------+---------+                        |
|                     |
| +EmptyResult    |                 |              1 |    0 |       0 |                        |
|                     |
| |               +-----------------+---------------+------+---------+                        |
|                     |
| +Create         | (anon_0:Person) |              1 |    1 |       1 |                    0/0 |
0.000 | Fused in Pipeline 0 |
+-----------------+-----------------+---------------+------+---------+-----------------------
+-----------+--------------------+

Total database accesses: 1, total allocated memory: 184
```

Produce Results

The `ProduceResults` operator prepares the result so that it is consumable by the user, such as transforming internal values to user values. It is present in every single query that returns data to the user, and has little bearing on performance optimisation.

Example 583. ProduceResults

Query

```
PROFILE
MATCH (n)
RETURN n
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator        | Details | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults | n       |             35 |   35 |       0 |                |
|           |                     |
| |               +---------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   | n       |             35 |   35 |      36 |            120 |
3/0 |     0.508 | Fused in Pipeline 0 |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 36, total allocated memory: 184
```

Filter

The `Filter` operator filters each row coming from the child operator, only passing through rows that evaluate the predicates to `true`.

Example 584. Filter

Query

```
PROFILE
MATCH (p:Person)
WHERE p.name =~ '^a.*'
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED
```

```
Runtime version 2026.10

Batch size 128

+-----------------+-----------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator        | Details                                                                  | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+-----------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults | p                                                                        |             14
|    0 |       0 |                |                           |           |                     |
| |               +-------------------------------------------------------------
+-----------------+------+---------+----------------+                        |           |
|
| +Filter         | cache[p.name] =~ $autostring_0                                           |             14
|    0 |       0 |                |                           |           |                     |
| |               +-------------------------------------------------------------
+-----------------+------+---------+----------------+                        |           |
|
| +NodeIndexScan  | RANGE INDEX p:Person(name) WHERE name IS NOT NULL, cache[p.name] |             14
|   14 |      15 |            120 |                       0/1 |     0.763 | Fused in Pipeline 0 |
+-----------------+-----------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 15, total allocated memory: 184
```

Empty Row

The `EmptyRow` operator returns a single row with no columns.

`EmptyRow` is only applied by the slotted runtime. In the pipelined runtime, the task of `EmptyRow` is instead performed by the `Projection` operator.

Example 585. EmptyRow

Query

```
PROFILE
CYPHER runtime=slotted
FOREACH (value IN [1,2,3] | MERGE (:Person {age: value}))
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+---------------------------------------+---------------+------+---------
+------------------------+
| Operator        | Details                               | Estimated Rows | Rows | DB Hits | Page
Cache Hits/Misses |
+-----------------+---------------------------------------+---------------+------+---------
+------------------------+
| +ProduceResults |                                       |             1 |    0 |       0 |
0/0 |
| |               +---------------------------------------+---------------+------+---------
+------------------------+
| +EmptyResult    |                                       |             1 |    0 |       0 |
0/0 |
| |               +---------------------------------------+---------------+------+---------
+------------------------+
| +Foreach        | value IN [1, 2, 3]                    |             1 |    1 |       0 |
```

```
0/0 |
| |\                 +--------------------------------------+----------------+------+---------
+-----------------------+
| | +Merge           | CREATE (anon_0:Person {age: value})  |              1 |    3 |       9 |
0/0 |
| | |                +--------------------------------------+----------------+------+---------
+-----------------------+
| | +Filter          | anon_0.age = value                   |              1 |    0 |     184 |
2/0 |
| | |                +--------------------------------------+----------------+------+---------
+-----------------------+
| | +NodeByLabelScan | anon_0:Person                        |             35 |  108 |     111 |
3/0 |
| |                  +--------------------------------------+----------------+------+---------
+-----------------------+
| +EmptyRow          |                                      |              1 |    1 |       0 |
0/0 |
+--------------------+--------------------------------------+----------------+------+---------
+-----------------------+

Total database accesses: 304, total allocated memory: 64
```

Cache Properties

The `CacheProperties` operator reads nodes and relationship properties and caches them in the current row. Future accesses to these properties can avoid reading from the store which will speed up the query. In the plan below we will cache `l.name` before `Expand(All)` where there are fewer rows.

Example 586. CacheProperties

Query

```
PROFILE
MATCH (l:Location)<-[:WORKS_IN]-(p:Person)
RETURN
  l.name AS location,
  p.name AS name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+-----------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| Operator         | Id | Details                                 | Estimated Rows | Rows | DB Hits
| Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+----+-----------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults  |  0 | location, name                          |             13 |   13 |       0
|                  |                       |           |                    |
| |                +----+-----------------------------------------+----------------+------
+---------+---------------+                       |           |                    |
| +Projection      |  1 | cache[l.name] AS location, p.name AS name |             13 |   13 |      26
|                  |                       |           |                    |
| |                +----+-----------------------------------------+----------------+------
+---------+---------------+                       |           |                    |
| +Filter          |  2 | p:Person                                |             13 |   13 |      26
|                  |                       |           |                    |
| |                +----+-----------------------------------------+----------------+------
+---------+---------------+                       |           |                    |
| +Expand(All)     |  3 | (l)<-[anon_0:WORKS_IN]-(p)              |             13 |   13 |      24
|                  |                       |           |                    |
```

```
| |              +----+-----------------------------------------+---------------+------
+---------+---------------+                          |           |                    |
| +CacheProperties |  4 | cache[l.name]                            |            10 |   10 |      20
|                |                         |           |                    |
| |              +----+-----------------------------------------+---------------+------
+---------+---------------+                          |           |                    |
| +NodeByLabelScan |  5 | l:Location                               |            10 |   10 |      11
|            120 |                    4/0 |     0.344 | Fused in Pipeline 0 |
+-----------------+----+-----------------------------------------+---------------+------
+---------+---------------+------------------------+-----------+--------------------+

Total database accesses: 107, total allocated memory: 200
```

Projection

For each incoming row, the `Projection` operator evaluates a set of expressions and produces a row with the results of the expressions.

Example 587. Projection

Query

```
PROFILE
RETURN 'hello' AS greeting
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+---------------------------+---------------+------+---------
+------------------------+-----------+---------------------+
| Operator        | Details                   | Estimated Rows | Rows | DB Hits | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+---------------------------+---------------+------+---------
+------------------------+-----------+---------------------+
| +ProduceResults | greeting                  |              1 |    1 |       0 |
|           |                     |
| |               +---------------------------+---------------+------+---------+
|           |                     |
| +Projection     | $autostring_0 AS greeting |              1 |    1 |       0 |
0/0 |     0.000 | Fused in Pipeline 0 |
+-----------------+---------------------------+---------------+------+---------
+------------------------+-----------+---------------------+

Total database accesses: 0, total allocated memory: 184
```

Project Endpoints

The `ProjectEndpoints` operator projects the start and end node of a relationship.

Example 588. ProjectEndpoints

Query

```
PROFILE
```

```
CREATE (n)-[p:KNOWS]->(m)
WITH p AS r
MATCH (u)-[r]->(v)
RETURN u, v
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------+----+---------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| Operator            | Id | Details                               | Estimated Rows | Rows | DB
Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+---------------------+----+---------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| +ProduceResults     |  0 | u, v                                  |              1 |    1 |
2 |              0 |                       |           |                    |
| |                   +----+---------------------------------------+----------------+------
+---------+----------------+                       |           |                    |
| +Apply              |  1 |                                       |              1 |    1 |
0 |                |                       |           |                    |
| |\                  +----+---------------------------------------+----------------+------
+---------+----------------+                       |           |                    |
| | +ProjectEndpoints |  2 | (u)-[r]->(v)                          |              1 |    1 |
0 |                |                       |           |                    |
| | |                 +----+---------------------------------------+----------------+------
+---------+----------------+                       |           |                    |
| | +Argument         |  3 | r                                     |              1 |    1 |
0 |           4328 |                     0/0 |     0.194 | Fused in Pipeline 2 |
| |                   +----+---------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| +Eager              |  4 | read/create conflict (Operator: 6 vs 2) |              1 |    1 |
0 |            368 |                     0/0 |     0.025 | In Pipeline 1       |
| |                   +----+---------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+
| +Projection         |  5 | p AS r                                |              1 |    1 |
0 |                |                       |           |                    |
| |                   +----+---------------------------------------+----------------+------
+---------+----------------+                       |           |                    |
| +Create             |  6 | (n), (m), (n)-[p:KNOWS]->(m)          |              1 |    1 |
3 |                |                     0/0 |     0.000 | Fused in Pipeline 0 |
+---------------------+----+---------------------------------------+----------------+------
+---------+----------------+-----------------------+-----------+--------------------+

Total database accesses: 5, total allocated memory: 4920
```

Distinct

The `Distinct` operator removes duplicate rows from the incoming stream of rows. To ensure only distinct elements are returned, `Distinct` will pull in data lazily from its source and build up state. This may lead to increased memory pressure in the system.

Example 589. Distinct

Query

```
PROFILE
MATCH (l:Location)<-[:WORKS_IN]-(p:Person)
RETURN DISTINCT p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+---------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| Operator         | Id | Details                   | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+------------------+----+---------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+
| +ProduceResults  |  0 | p                         |             14 |   14 |      29 |
0 |                        |           |                    |
| |                +----+---------------------------+----------------+------+---------
+----------------+                      |           |                     |
| +Distinct        |  1 | p                         |             14 |   14 |       0 |
352 |                          |           |                    |
| |                +----+---------------------------+----------------+------+---------
+----------------+                      |           |                     |
| +Filter          |  2 | p:Person                  |             15 |   15 |      30 |
|                       |           |                    |
| |                +----+---------------------------+----------------+------+---------
+----------------+                      |           |                     |
| +Expand(All)     |  3 | (l)<-[anon_0:WORKS_IN]-(p) |             15 |   15 |      25 |
|                       |           |                    |
| |                +----+---------------------------+----------------+------+---------
+----------------+                      |           |                     |
| +NodeByLabelScan |  4 | l:Location                |             10 |   10 |      11 |
248 |                    8/0 |     0.758 | Fused in Pipeline 0 |
+------------------+----+---------------------------+----------------+------+---------
+----------------+-----------------------+-----------+--------------------+

Total database accesses: 95, total allocated memory: 432
```

Ordered Distinct

The `OrderedDistinct` operator is an optimization of the `Distinct` operator that takes advantage of the ordering of the incoming rows. This operator has a lower memory pressure in the system than the `Distinct` operator.

Example 590. OrderedDistinct

Query

```
PROFILE
MATCH (p:Person)
WHERE p.name STARTS WITH 'P'
RETURN DISTINCT p.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+-------------
```

```
+---------------+
| Operator              | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline      |
+----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-------------
+---------------+
| +ProduceResults       | `p.name`
|              0 |    2 |       0 |                |                    0/0 |     0.046 |
|               |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+              |
|
| +OrderedDistinct      | cache[p.name] AS `p.name`
|              0 |    2 |       0 |             32 |                    0/0 |     0.090 | `p.name`
ASC |               |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+--------------+
|
| +NodeIndexSeekByRange | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    2 |       3 |            120 |                    0/1 |     0.493
| p.name ASC   | In Pipeline 0 |
+----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-------------
+---------------+

Total database accesses: 3, total allocated memory: 184
```

Procedure Call

The `ProcedureCall` operator indicates an invocation to a procedure.

Example 591. ProcedureCall

Query

```
PROFILE
CALL db.labels() YIELD label
RETURN *
ORDER BY label
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------+-----------------------------------+---------------+------+---------
+---------------+-----------------------+-----------+------------+---------------------+
| Operator        | Details                           | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by | Pipeline            |
+----------------+-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+------------+---------------------+
| +ProduceResults | label                             |             10 |    4 |       0 |
|                    0/0 |     0.091 |            |                    |
| |               +-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+            |                     |
| +Sort           | label ASC                         |             10 |    4 |       0 |
536 |                  0/0 |     0.178 | label ASC  | In Pipeline 1       |
| |               +-----------------------------------+---------------+------+---------
+----------------+-----------------------+-----------+------------+---------------------+
```

```
| +ProcedureCall  | db.labels() :: (label :: STRING)  |            10 |    4 |         |
|                       |          |            | Fused in Pipeline 0 |
+-----------------+-----------------------------------+---------------+------+---------
+---------------+----------------------+-----------+-----------+--------------------+

Total database accesses: ?, total allocated memory: 600
```

Unwind

The `Unwind` operator returns one row per item in a list.

Example 592. Unwind

Query

```
PROFILE
UNWIND range(1, 5) AS value
RETURN value
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----------------------------------------+---------------+------+---------
+------------------------+-----------+--------------------+
| Operator        | Details                                | Estimated Rows | Rows | DB Hits | Page
Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+----------------------------------------+---------------+------+---------
+------------------------+-----------+--------------------+
| +ProduceResults | value                                  |             10 |    5 |       0 |
|           |                    |
| |               +----------------------------------------+---------------+------+---------+
|           |                    |
| +Unwind         | range($autoint_0, $autoint_1) AS value |             10 |    5 |       0 |
0/0 |     0.000 | Fused in Pipeline 0 |
+-----------------+----------------------------------------+---------------+------+---------
+------------------------+-----------+--------------------+

Total database accesses: 0, total allocated memory: 184
```

Partitioned Unwind

The `PartitionedUnwind` is a variant of the `Unwind` operator used by the parallel runtime. It allows the index to be partitioned into different segments where each segment can be scanned independently in parallel.

Example 593. PartitionedUnwind

Query

```
CYPHER runtime=parallel
PROFILE
UNWIND range(1, 5) AS value
RETURN value
```

Query Plan

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+--------------------+----+---------------------------------------+----------------+------+---------
+-----------------------+-----------+--------------------+
| Operator           | Id | Details                               | Estimated Rows | Rows | DB Hits
| Page Cache Hits/Misses | Time (ms) | Pipeline            |
+--------------------+----+---------------------------------------+----------------+------+---------
+-----------------------+-----------+--------------------+
| +ProduceResults    |  0 | value                                 |             10 |    5 |       0
|                    0/0 |     0.119 | In Pipeline 1       |
| |                  +----+---------------------------------------+----------------+------+---------
+-----------------------+-----------+--------------------+
| +PartitionedUnwind |  1 | range($autoint_0, $autoint_1) AS value |             10 |    5 |       0
|                       |           | Fused in Pipeline 0 |
+--------------------+----+---------------------------------------+----------------+------+---------
+-----------------------+-----------+--------------------+

Total database accesses: 0
```

Sort and limit operators

The operators in this group manage result set ordering and control the flow of data by limiting or skipping rows. They optimize query performance and ensure users receive results in the desired order and quantity.

Sort

The `Sort` operator sorts rows by a provided key. In order to sort the data, all data from the source operator needs to be pulled in eagerly and kept in the query state, which will lead to increased memory pressure in the system.

Example 594. Sort

Query

```
PROFILE
MATCH (p:Person)
RETURN p
ORDER BY p.name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+--------------------+
| Operator         | Details            | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Ordered by | Pipeline            |
+------------------+--------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+--------------------+
```

```
| +ProduceResults  | p                 |            14 |   14 |       0 |                |
2/0 |     0.178 |           |                    |
| |                +-------------------+---------------+------+---------+----------------
+-----------------------+-----------+            |                    |
| +Sort            | `p.name` ASC      |            14 |   14 |       0 |           1192 |
0/0 |     0.107 | p.name ASC | In Pipeline 1       |
| |                +-------------------+---------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
| +Projection      | p.name AS `p.name` |            14 |   14 |      14 |                |
|           |           |                    |
| |                +-------------------+---------------+------+---------+----------------+
|           +------------+                    |
| +NodeByLabelScan |p:Person           |            14 |   14 |      35 |            120 |
3/0 |     0,221 |           | Fused in Pipeline 0 |
+-----------------+-------------------+---------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+

Total database accesses: 85, total allocated memory: 1272
```

Partial Sort

The `PartialSort` operator is an optimization of the `Sort` operator that takes advantage of the ordering of the incoming rows. This operator uses lazy evaluation and has a lower memory pressure in the system than the `Sort` operator. Partial sort is only applicable when sorting on multiple columns.

Example 595. PartialSort

Query

```
PROFILE
MATCH (p:Person)
WHERE p.name STARTS WITH 'P'
RETURN p
ORDER BY p.name, p.age
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
| Operator              | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline            |
+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
| +ProduceResults       | p
|              0 |    2 |       0 |               |                   2/0 |     0.087 |
|                     |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                       |
|
| +PartialSort          | `p.name` ASC, `p.age` ASC
|              0 |    2 |       0 |           544 |                   0/0 |     0.184 | p.name ASC,
p.age ASC | In Pipeline 1       |
| |
+-------------------------------------------------------------------------------+----------------
```

```
+------+---------+----------------+-----------------------+-----------+----------------------
+---------------------+
| +Projection           | cache[p.name] AS `p.name`, p.age AS `p.age`
|              0 |    2 |       0 |                |                         |           | `p.name`
ASC          |                     |
| |
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+                         |           +-----------------------+
|
| +NodeIndexSeekByRange | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    2 |       3 |            120 |                    0/1 |     0.362
| p.name ASC            | Fused in Pipeline 0 |
+-----------------------
+-------------------------------------------------------------------------------------+----------------
+------+---------+----------------+-----------------------+-----------+----------------------
+---------------------+

Total database accesses: 3, total allocated memory: 608
```

Top

The `Top` operator returns the first `n` rows sorted by a provided key. Instead of sorting the entire input, only the top `n` rows are retained.

Example 596. Top

Query

```
PROFILE
MATCH (p:Person)
RETURN p
ORDER BY p.name
LIMIT 2
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+---------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
| Operator         | Details             | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Ordered by | Pipeline            |
+-----------------+---------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
| +ProduceResults  | p                   |              2 |    2 |       0 |                |
2/0 |     0.093 |           |                     |
| |                +---------------------+----------------+------+---------+----------------
+-----------------------+-----------+            |                     |
| +Top             | `p.name` ASC LIMIT 2 |              2 |    2 |       0 |           1184 |
0/0 |     0.295 | p.name ASC | In Pipeline 1       |
| |                +---------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
| +Projection      | p.name AS `p.name`   |             14 |   14 |      14 |                |
|           |           |                     |
| |                +---------------------+----------------+------+---------+----------------+
|           +------------+                     |
| +NodeByLabelScan | p:Person             |             14 |   14 |      35 |            120 |
3/0 |     0,166 |           | Fused in Pipeline 0 |
+-----------------+---------------------+----------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
```

```
Total database accesses: 85, total allocated memory: 1264
```

Partial Top

The `PartialTop` operator is an optimization of the `Top` operator that takes advantage of the ordering of the incoming rows. This operator uses lazy evaluation and has a lower memory pressure in the system than the `Top` operator. Partial top is only applicable when sorting on multiple columns.

Example 597. PartialTop

Query

```
PROFILE
MATCH (p:Person)
WHERE p.name STARTS WITH 'P'
RETURN p
ORDER BY p.name, p.age
LIMIT 2
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
| Operator              | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by
| Pipeline            |
+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
| +ProduceResults       | p
|              0 |    2 |       0 |               |                   2/0 |     0.093 |
|                     |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+                       |
|
| +PartialTop           | `p.name` ASC, `p.age` ASC LIMIT 2
|              0 |    2 |       0 |           640 |                   0/0 |     0.870 | p.name ASC,
p.age ASC | In Pipeline 1       |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
| +Projection           | cache[p.name] AS `p.name`, p.age AS `p.age`
|              0 |    2 |       0 |               |                       |           | `p.name`
ASC          |                     |
| |
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+                       |           +-----------------------+
|
| +NodeIndexSeekByRange | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    2 |       3 |           120 |                   0/1 |     0.556
| p.name ASC            | Fused in Pipeline 0 |
+-----------------------
+-------------------------------------------------------------------------------+----------------
+------+---------+---------------+-----------------------+-----------+-----------------------
+---------------------+
```

```
Total database accesses: 3, total allocated memory: 704
```

Limit

The `Limit` operator returns the first `n` rows from the incoming input.

Example 598. Limit

Query

```
PROFILE
MATCH (p:Person)
RETURN p
LIMIT 3
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator        | Details  | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults | p        |              3 |    3 |       0 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +Limit          | 3        |              3 |    3 |       0 |             32 |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +NodeByLabelScan| p:Person |              3 |    4 |       5 |            120 |
3/0 |     0,540 | Fused in Pipeline 0 |
+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 8, total allocated memory: 184
```

Exhaustive Limit

The `ExhaustiveLimit` operator is similar to the `Limit` operator but will always exhaust the input. Used when combining `LIMIT` and updates

Example 599. ExhaustiveLimit

Query

```
PROFILE
MATCH (p:Person)
SET p.seen = true
RETURN p
```

```
LIMIT 3
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator         | Id | Details       | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+----+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults  |  0 | p             |              3 |    3 |      10 |              0 |
|           |                     |
| |                +----+---------------+----------------+------+---------+----------------+
|           |                     |
| +ExhaustiveLimit |  1 | 3             |              3 |    3 |       0 |             32 |
|           |                     |
| |                +----+---------------+----------------+------+---------+----------------+
|           |                     |
| +SetProperty     |  2 | p.seen = true |             17 |   17 |      34 |                |
|           |                     |
| |                +----+---------------+----------------+------+---------+----------------+
|           |                     |
| +NodeByLabelScan |  3 | p:Person      |             17 |   17 |      18 |            240 |
3/0 |     1.966 | Fused in Pipeline 0 |
+------------------+----+---------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+

Total database accesses: 62, total allocated memory: 304
```

Skip

The `Skip` operator skips `n` rows from the incoming rows.

Example 600. Skip

Query

```
PROFILE
MATCH (p:Person)
RETURN p
ORDER BY p.id
SKIP 1
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----------------+----------------+------+---------+----------------
+------------------------+-----------+------------+---------------------+
| Operator         | Details        | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Ordered by | Pipeline            |
+------------------+----------------+----------------+------+---------+----------------
+------------------------+-----------+------------+---------------------+
```

```
| +ProduceResults  | p              |            13 |   13 |       0 |                |
2/0 |     0.165 |            |                |
| |                +----------------+---------------+------+---------+----------------
+-----------------------+-----------+            |                      |
| +Skip            | $autoint_0     |            13 |   13 |       0 |             32 |
0/0 |     0.043 |            |                |
| |                +----------------+---------------+------+---------+----------------
+-----------------------+-----------+            |                      |
| +Sort            | `p.id` ASC     |            14 |   14 |       0 |            400 |
0/0 |     0.155 | p.id ASC   | In Pipeline 1  |
| |                +----------------+---------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+
| +Projection      | p.id AS `p.id` |            14 |   14 |       0 |                |
|           |            |                     |
| |                +----------------+---------------+------+---------+----------------+
|           +------------+                     |
| +NodeByLabelScan | p:Person       |            18 |   18 |      19 |            120 |
3/0 |     0,157 |            | Fused in Pipeline 0 |
+------------------+----------------+---------------+------+---------+----------------
+-----------------------+-----------+------------+---------------------+

Total database accesses: 71, total allocated memory: 512
```

Data modification operators

The operators in this group modify graph data by creating, deleting, and altering nodes, relationships, and properties.

Create

The `Create` operator is used to create nodes and relationships.

Example 601. Create

Query

```
PROFILE
CREATE
  (max:Person {name: 'Max'}),
  (chris:Person {name: 'Chris'})
CREATE (max)-[:FRIENDS_WITH]->(chris)
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| Operator        | Details                                                                       |
Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+-------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+-----------+---------------------+
| +ProduceResults |                                                                               |
1 |    0 |       0 |                       |           |                     |
| |               +-------------------------------------------------------------------------------
+----------------+------+---------+                       |           |                     |
| +EmptyResult    |                                                                               |
1 |    0 |       0 |                       |           |                     |
| |               +-------------------------------------------------------------------------------
+----------------+------+---------+                       |           |                     |
```

```
| +Create         | (max:Person {name: $autostring_0}), (chris:Person {name: $autostring_1}), |
1 |    1 |       7 |                     0/0 |     0.000 | Fused in Pipeline 0 |
|                  | (max)-[anon_0:FRIENDS_WITH]->(chris)                                  |
|      |          |                         |           |                     |
+-----------------+-----------------------------------------------------------------------
+----------------+------+---------+----------------------+-----------+---------------------+

Total database accesses: 7, total allocated memory: 184
```

Delete

The `Delete` operator is used to delete a node or a relationship.

Example 602. Delete

Query

```
PROFILE
MATCH (you:Person {name: 'you'})
DELETE you
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-------------------------------------------------------------+----------------
+------+---------+----------------+----------------------+-----------+---------------------+
| Operator        | Id | Details                                                     | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----+-------------------------------------------------------------+----------------
+------+---------+----------------+----------------------+-----------+---------------------+
| +ProduceResults |  0 |                                                             |              0 |
0 |       0 |                |                      |           |                     |
| |               +----+-------------------------------------------------------------+----------------
+------+---------+----------------+                      |           |                     |
| +EmptyResult    |  1 |                                                             |              0 |
0 |       0 |                |                      |           |                     |
| |               +----+-------------------------------------------------------------+----------------
+------+---------+----------------+                      |           |                     |
| +Delete         |  2 | you                                                          |              0 |
0 |       0 |                |                      |           |                     |
| |               +----+-------------------------------------------------------------+----------------
+------+---------+----------------+                      |           |                     |
| +NodeIndexSeek  |  3 | RANGE INDEX you:Person(name) WHERE name = $autostring_0 |              0 |
0 |       1 |            120 |                  1/0 |     0.330 | Fused in Pipeline 0 |
+-----------------+----+-------------------------------------------------------------+----------------
+------+---------+----------------+----------------------+-----------+---------------------+

Total database accesses: 13, total allocated memory: 216
```

Detach Delete

The `DetachDelete` operator is used in all queries containing the DETACH DELETE clause, when deleting nodes and their relationships.

Example 603. DetachDelete

Query

```
PROFILE
MATCH (p:Person)
DETACH DELETE p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----------+---------------+------+---------+---------------
+-----------------------+-----------+--------------------+
| Operator        | Details  | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----------+---------------+------+---------+---------------
+-----------------------+-----------+--------------------+
| +ProduceResults  |          |             14 |    0 |       0 |                |
|           |                     |
| |               +----------+---------------+------+---------+----------------+
|           |                     |
| +EmptyResult     |          |             14 |    0 |       0 |                |
|           |                     |
| |               +----------+---------------+------+---------+----------------+
|           |                     |
| +DetachDelete    | p        |             14 |   14 |      41 |                |
|           |                     |
| |               +----------+---------------+------+---------+----------------+
|           |                     |
| +NodeByLabelScan | p:Person |             14 |   14 |      35 |            120 |
21/0 |    12,439 | Fused in Pipeline 0 |
+-----------------+----------+---------------+------+---------+---------------
+-----------------------+-----------+--------------------+

Total database accesses: 112, total allocated memory: 200
```

Merge

The `Merge` operator will either read or create nodes and/or relationships.

If matches are found it will execute the provided `ON MATCH` operations foreach incoming row. If no matches are found instead nodes and relationships are created and all `ON CREATE` operations are run.

Example 604. Merge

Query

```
PROFILE
MERGE (p:Person {name: 'Andy'})
ON MATCH SET p.existed = true
ON CREATE SET p.existed = false
```

Query Plan

```
Planner COST
```

```
Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+-------------------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| Operator        | Details                                                                      |
Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+-----------------+-------------------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+
| +ProduceResults |                                                                              |
1 |    0 |       0 |                |                         |           |                    |
| |               +------------------------------------------------------------------------------
+----------------+------+---------+---------------+                         |           |
|
| +EmptyResult    |                                                                              |
1 |    0 |       0 |                |                         |           |                    |
| |               +------------------------------------------------------------------------------
+----------------+------+---------+---------------+                         |           |
|
| +Merge          | CREATE (p:Person {name: $autostring_0}), ON MATCH SET p.existed = true, |
1 |    1 |       2 |                |                         |           |                    |
| |               | ON CREATE SET p.existed = false                                            |
|      |         |                |                         |           |                     |
| |               +------------------------------------------------------------------------------
+----------------+------+---------+---------------+                         |           |
|
| +NodeIndexSeek  | RANGE INDEX p:Person(name) WHERE name = $autostring_0                      |
1 |    1 |       2 |            120 |                    2/1 |     0.749 | Fused in Pipeline 0 |
+-----------------+-------------------------------------------------------------------------------
+----------------+------+---------+---------------+-----------------------+-----------
+---------------------+

Total database accesses: 4, total allocated memory: 184
```

Merge Into

Introduced in 2025.11

The `MergeInto` operator is similar to the `Merge` operator, but is used when the start and end node of the pattern is matched outside the `MERGE` pattern.

If necessary, the operator locks the pattern nodes before creating the relationship, to avoid creating multiple relationships between the nodes.

Example 605. MergeInto

Query

```
PROFILE
MATCH (s:Person {name: 'me'})
MERGE (s)-[:FRIENDS_WITH]->(s)
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+-----------------+----+-----------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| Operator        | Id | Details                                                 | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            | Indexes
Used         |
+-----------------+----+-----------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| +ProduceResults |  0 |                                                     |             1 |
0 |       0 |              0 |                       |           |                     |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+----------------+                       |           |
+----------------------+
| +EmptyResult    |  1 |                                                     |             1 |
0 |       0 |                |                       |           |                     |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+----------------+                       |           |
+----------------------+
| +Apply          |  2 |                                                     |             1 |
1 |       0 |                |                       |           |                     |
|
| |\              +----+-----------------------------------------------------+---------------
+------+---------+----------------+                       |           |
+----------------------+
| | +MergeInto    |  3 | MERGE (s)-[anon_0:FRIENDS_WITH]->(s)                |             1 |
1 |       1 |                |                       |           |                     |
|
| | |             +----+-----------------------------------------------------+---------------
+------+---------+----------------+                       |           |
+----------------------+
| | +Argument     |  4 | s                                                   |             1 |
1 |       0 |           2288 |                   1/0 |     0.673 | Fused in Pipeline 1 |
|
| |               +----+-----------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+
| +NodeIndexSeek  |  5 | RANGE INDEX s:Person(name) WHERE name = $autostring_0 |             1 |
1 |       2 |            376 |                   1/0 |     0.256 | In Pipeline 0       |
range_person_name: 1 |
+-----------------+----+-----------------------------------------------------+---------------
+------+---------+----------------+-----------------------+-----------+--------------------
+----------------------+

Total database accesses: 15, total allocated memory: 2232
```

Merge Unique Node

Introduced in 2026.02

> **Note:**
>
> `MergeUniqueNode` was added in Neo4j 2025.11 but it did not work as intended in that version. For this reason, the operator was disabled in Neo4j 2025.12. Neo4j 2026.02 fixes the bug and is thus the recommended version to use.

The `MergeUniqueNode` operator is similar to the `Merge` operator, but is used when there is a property uniqueness constraint on the property used in the `MERGE` statement.

Example 606. MergeUniqueNode

Query

```
PROFILE
MERGE (t:Team {name: 'Engineering'})
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+-----------------------------------------------------+---------------+------
+---------+---------------+-----------------------+-----------+--------------------+-------------
+
| Operator         | Id | Details                                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           | Indexes Used |
+------------------+----+-----------------------------------------------------+---------------+------
+---------+---------------+-----------------------+-----------+--------------------+-------------
+
| +ProduceResults  |  0 |                                                     |             1 |    0 |
0 |             0 |                       |           |                    |              |
| |                +----+-----------------------------------------------------+---------------+------
+---------+----------------+                       |           |                    +-------------
+
| +EmptyResult     |  1 |                                                     |             1 |    0 |
0 |                |                       |           |                    |              |
| |                +----+-----------------------------------------------------+---------------+------
+---------+----------------+                       |           |                    +-------------
+
| +MergeUniqueNode |  2 | UNIQUE t:Team(name) WHERE name = $autostring_0,     |             1 |    1 |
2 |           248 |                    1/0 |     2.922 | Fused in Pipeline 0 | team_name: 1 |
+------------------+----+-----------------------------------------------------+---------------+------
+---------+----------------+-----------------------+-----------+--------------------+-------------
+

Total database accesses: 2, total allocated memory: 312
```

Locking Merge

The `LockingMerge` operator is similar to the `Merge` operator but will lock the start and end node when creating a relationship if necessary.

Example 607. LockingMerge

Query

```
PROFILE
MATCH (s:Person {name: 'me'})
MERGE (s)-[:FRIENDS_WITH]->(:Person {name: 'you'})
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+------------------+----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+----------------------+
| Operator         | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used         |
+------------------+----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+----------------------+
| +ProduceResults  |  0 |
|              0 |    0 |       0 |               |                       |           |
|                     |
| |               +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
+----------------------+
| +EmptyResult     |  1 |
|              0 |    0 |       0 |               |                       |           |
|                     |
| |               +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
+----------------------+
| +Apply           |  2 |
|              0 |    0 |       0 |               |                       |           |
|                     |
| |\              +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
+----------------------+
| | +LockingMerge  |  3 | CREATE (anon_1:Person {name: $autostring_1}), (s)-[anon_0:FRIENDS_WITH]-
>(anon_1), LOCK(s) |              0 |    0 |       0 |                |                          |
|                     |                      |
| | |             +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
+----------------------+
| | +Expand(Into)  |  4 | (s)-[anon_0:FRIENDS_WITH]->(anon_1)
|              0 |    0 |       0 |               |                       |           |
|                     |
| | |             +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+                       |           |
+----------------------+
| | +NodeIndexSeek |  5 | RANGE INDEX anon_1:Person(name) WHERE name = $autostring_1
|              0 |    0 |       0 |           128 |                   0/0 |     0.000 | Fused in
Pipeline 1 |                      |
| |               +----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+----------------------+
| +NodeIndexSeek   |  6 | RANGE INDEX s:Person(name) WHERE name = $autostring_0
|              0 |    0 |       1 |           376 |                   1/0 |     0.186 | In Pipeline
0       | range_person_name: 1 |
+------------------+----
+-----------------------------------------------------------------------------------------------
+---------------+------+---------+---------------+-----------------------+-----------
+---------------------+----------------------+

Total database accesses: 1, total allocated memory: 584
```

Lock Nodes

Introduced in 2025.09

The `LockNodes` operator is sometimes used in conjunction with the `LockingMerge` operator to lock nodes.

Example 608. LockNodes

Query

```
PROFILE
MATCH (s:Person {name: 'me'})
MERGE (s)-[:FRIENDS_WITH]->(t:Person {size: size([()-->()|1])})
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 4

+----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| Operator                               | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| +ProduceResults                        |  0 |
|              1 |    0 |       0 |              0 |                    0/0 |     0.018 |
|
| |                                      +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
|
| +EmptyResult                           |  1 |
|              1 |    0 |       0 |                |                    0/0 |     0.016 | In Pipeline
7       |
| |                                      +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| +Apply                                 |  2 |
|              1 |    0 |       0 |                |                    0/0 |           |
|
| |\                                     +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| | +ArgumentTracker                     | 19 |
|              1 |    0 |       0 |            240 |                    0/0 |     0.000 |
|
| | |                                    +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
|
| | +LockingMerge                        |  3 | CREATE (t:Person {size: COUNT {   MATCH (`anon_2`)-
[`anon_3`]->(`anon_4`)   RETURN $`autoint_1` AS ` |              1 |    1 |       3 |            400
|                    0/0 |    15.340 | In Pipeline 7       |
| | |                                    |    | anon_0` }}), (s)-[anon_1:FRIENDS_WITH]->(t), LOCK(s)
|               |      |         |                |                        |           |
|
| | |                                    +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| | +AntiConditionalApply                | 18 |
|               |    0 |       0 |            384 |                        |     0.010 | In Pipeline
6       |
| | |\                                   +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
```

```
| | | +Filter                            | 17 | t.size = anon_5 AND t:Person
|              0 |    0 |       4 |                |                          |           |
|
| | | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | | +Apply                           | 16 |
|              1 |    2 |       0 |                |                          |           |
|
| | | |\                                 +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | | | +RelationshipCountFromCountStore | 15 | count( ()-[]->() ) AS anon_5
|              1 |    2 |       2 |            608 |                      0/0 |     0.113 | Fused in
Pipeline 5 |
| | | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-----------------------+-----------+
---------------------+
| | | +Expand(All)                     | 14 | (s)-[anon_1:FRIENDS_WITH]->(t)
|              1 |    2 |       3 |                |                          |           |
|
| | | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | | +LockNodes                       | 13 | s
|              1 |    1 |       0 |                |                          |           |
|
| | | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | | +Argument                        | 12 | s
|              1 |    1 |       0 |            584 |                      0/0 |     1.494 | Fused in
Pipeline 4 |
| | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-----------------------+-----------+
---------------------+
| | +Optional                          | 11 | s
|              0 |    1 |       0 |            528 |                      0/0 |     2.057 | In Pipeline
3       |
| | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-----------------------+-----------+
---------------------+
| | +Filter                            |  5 | t.size = anon_5 AND t:Person
|              0 |    0 |       4 |                |                          |           |
|
| | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | +Apply                             |  6 |
|              1 |    2 |       0 |                |                          |           |
|
| | |\                                 +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | | +RelationshipCountFromCountStore   |  7 | count( ()-[]->() ) AS anon_5
|              1 |    2 |       2 |            528 |                      0/0 |     4.220 | Fused in
Pipeline 2 |
| | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+-----------------------+-----------+
---------------------+
| | +Expand(All)                       |  8 | (s)-[anon_1:FRIENDS_WITH]->(t)
|              1 |    2 |       3 |                |                          |           |
|
| | |                                  +----
+-----------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+                          |           |
|
| | +Argument                          |  9 | s
|              1 |    1 |       0 |            432 |                      1/0 |     0.184 | Fused in
```

```
Pipeline 1 |
| |                                    +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+
| +NodeIndexSeek                         | 10 | RANGE INDEX s:Person(name) WHERE name = $autostring_0
|              1 |    1 |       2 |            376 |                    0/1 |     9.263 | In Pipeline
0       |
+-----------------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------------+

Total database accesses: 23, total allocated memory: 3248
```

Foreach

The `Foreach` operator executes a nested loop between the left child operator and the right child operator. In an analogous manner to the Apply operator, it takes a row from the left-hand side and, using the Argument operator, provides it to the operator tree on the right-hand side. `Foreach` will yield all the rows coming in from the left-hand side; all results from the right-hand side are pulled in and discarded.

Example 609. Foreach

Query

```
PROFILE
FOREACH (value IN [1,2,3] | CREATE (:Person {age: value}))
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

+-----------------+-------------------------------------------------------------+----------------+------
+---------+------------------------+
| Operator        | Details                                                     | Estimated Rows | Rows |
DB Hits | Page Cache Hits/Misses |
+-----------------+-------------------------------------------------------------+----------------+------
+---------+------------------------+
| +ProduceResults |                                                             |              1 |    0 |
0 |                    0/0 |
| |               +-------------------------------------------------------------+----------------+------
+---------+------------------------+
| +EmptyResult    |                                                             |              1 |    0 |
0 |                    0/0 |
| |               +-------------------------------------------------------------+----------------+------
+---------+------------------------+
| +Foreach        | value IN [1, 2, 3], CREATE (anon_0:Person {age: value}) |              1 |    1 |
9 |                    0/0 |
+-----------------+-------------------------------------------------------------+----------------+------
+---------+------------------------+

Total database accesses: 9, total allocated memory: 64
```

SubqueryForeach

`SubqueryForeach` works like the `Foreach` operator but it is only used for executing subqueries.

Example 610. SubqueryForeach

The below query uses a variable scope clause to import variables into the `CALL` subquery.

Query

```
PROFILE
LOAD CSV FROM 'https://neo4j.com/docs/cypher-refcard/3.3/csv/artists.csv' AS line
CALL (line) {
  CREATE (a: Artist {name: line[0]})
}
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+--------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| Operator         | Id | Details                              | Estimated Rows | Rows | DB Hits |
Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+------------------+----+--------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +ProduceResults  |  0 |                                      |             10 |    0 |       0 |
0 |                        |           |                     |
| |                +----+--------------------------------------+----------------+------+---------
+----------------+                      |           |                     |
| +EmptyResult     |  1 |                                      |             10 |    0 |       0 |
|                    0/0 |     0.000 | Fused in Pipeline 3 |
| |                +----+--------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +SubqueryForeach |  2 |                                      |             10 |    4 |       0 |
4080 |                         |           |                       |
| |\               +----+--------------------------------------+----------------+------+---------
+----------------+                      |           |                     |
| | +Create        |  3 | (a:Artist {name: line[$autoint_0]}) |             10 |    4 |      12 |
|                      |           |                     |
| | |              +----+--------------------------------------+----------------+------+---------
+----------------+                      |           |                     |
| | +Argument      |  4 | line                                 |             10 |    4 |       0 |
3472 |                     0/0 |     0.852 | Fused in Pipeline 2 |
| |                +----+--------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+
| +LoadCSV         |  5 | line                                 |             10 |    4 |       0 |
328 |                       |           | In Pipeline 1       |
+------------------+----+--------------------------------------+----------------+------+---------
+----------------+-----------------------+-----------+---------------------+

Total database accesses: 12, total allocated memory: 4928
```

TransactionForeach

`TransactionForeach` works like the `Foreach` operator but will commit the current transaction after a specified number of rows. With `IN CONCURRENT TRANSACTIONS`, `PROFILE` also reports the concurrency the operator achieved. For more information, see Measuring concurrency with `PROFILE`.

Example 611. TransactionForeach

The below query uses a variable scope clause to import variables into the `CALL` subquery.

Query

```
PROFILE
LOAD CSV FROM 'https://neo4j.com/docs/cypher-refcard/3.3/csv/artists.csv' AS line
CALL (line) {
  CREATE (a: Artist {name: line[0]})
} IN TRANSACTIONS OF 100 ROWS
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

++--------------------+----+-------------------------------------------------+---------------
+------+---------+----------------+------------------------+-----------+--------------------+
 | Operator           | Id | Details                                         | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
 +--------------------+----+-------------------------------------------------+---------------
+------+---------+----------------+------------------------+-----------+--------------------+
 | +ProduceResults    |  0 |                                                 |            10 |
0 |       0 |              0 |                        |           |                    |
 | |                  +----+-------------------------------------------------+---------------
+------+---------+----------------+                        |           |                    |
 | +EmptyResult       |  1 |                                                 |            10 |
0 |       0 |                |                    0/0 |     0.000 | Fused in Pipeline 3 |
 | |                  +----+-------------------------------------------------+---------------
+------+---------+----------------+------------------------+-----------+--------------------+
 | +TransactionForeach |  2 | IN TRANSACTIONS OF $autoint_1 ROWS ON ERROR FAIL |            10 |
4 |       0 |           4856 |                        |           |                    |
 | |\                 +----+-------------------------------------------------+---------------
+------+---------+----------------+                        |           |                    |
 | | +Create          |  3 | (a:Artist {name: line[$autoint_0]})             |            10 |
4 |      12 |                |                        |           |                    |
 | | |                +----+-------------------------------------------------+---------------
+------+---------+----------------+                        |           |                    |
 | | +Argument        |  4 | line                                            |            10 |
4 |       0 |           3472 |                    0/0 |     0.712 | Fused in Pipeline 2 |
 | |                  +----+-------------------------------------------------+---------------
+------+---------+----------------+------------------------+-----------+--------------------+
 | +LoadCSV           |  5 | line                                            |            10 |
4 |       0 |            328 |                        |           | In Pipeline 1      |
 +--------------------+----+-------------------------------------------------+---------------
+------+---------+----------------+------------------------+-----------+--------------------+

 Total database accesses: 12, total allocated memory: 5704
```

Set Labels

The `SetLabels` operator is used when setting labels on a node.

Example 612. SetLabels

Query

```
PROFILE
MATCH (n)
SET n:Person
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| Operator        | Details  | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| +ProduceResults |          |             35 |    0 |       0 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +EmptyResult    |          |             35 |    0 |       0 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +SetLabels      | n:Person |             35 |   35 |      22 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   | n        |             35 |   35 |      36 |            120 |
3/0 |     0.873 | Fused in Pipeline 0 |
+-----------------+----------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+

Total database accesses: 58, total allocated memory: 184
```

Remove Labels

The `RemoveLabels` operator is used when deleting labels from a node.

Example 613. RemoveLabels

Query

```
PROFILE
MATCH (n)
REMOVE n:Person
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128
```

```
+----------------+----------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| Operator        | Details  | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+----------------+----------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+
| +ProduceResults |          |             35 |    0 |       0 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +EmptyResult    |          |             35 |    0 |       0 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +RemoveLabels   | n:Person |             35 |   35 |      15 |                |
|           |                     |
| |               +----------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   | n        |             35 |   35 |      36 |            120 |
3/0 |     0.765 | Fused in Pipeline 0 |
+----------------+----------+----------------+------+---------+----------------
+-----------------------+-----------+--------------------+

Total database accesses: 51, total allocated memory: 184
```

Remote Batch Properties

The `RemoteBatchProperties` operator is used to fetch and cache properties from shards for batches of rows, so that they can be used by subsequent operators.

Infinigraph | Introduced in 2026.01

Example 614. RemoteBatchProperties

Query

```
PROFILE
MATCH (m:Movie)
RETURN m.title AS title
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 4

+------------------------+----+-------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------+
| Operator               | Id | Details                 | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+------------------------+----+-------------------------+----------------+------+---------
+---------------+-----------------------+-----------+---------------+
| +ProduceResults        |  0 | title                   |             10 |   10 |       0 |
0 |                    0/0 |     0.074 |               |
| |                      +----+-------------------------+----------------+------+---------
+---------------+-----------------------+-----------+               |
| +Projection            |  1 | cache[m.title] AS title |             10 |   10 |       0 |
|                    0/0 |     0.077 |               |
| |                      +----+-------------------------+----------------+------+---------
+---------------+-----------------------+-----------+               |
| +RemoteBatchProperties |  2 | cache[m.title]          |             10 |   10 |      10 |
376 |                    0/0 |   477.772 | In Pipeline 1 |
```

```
| |                       +----+-------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------+
| +NodeByLabelScan       |  3 | m:Movie                 |            10 |   10 |      11 |
376 |                       1/0 |     3.830 | In Pipeline 0 |
+-----------------------+----+-------------------------+---------------+------+---------
+---------------+-----------------------+-----------+---------------+

Total database accesses: 21, total allocated memory: 696
```

Remote Batch Properties With Filter

The `RemoteBatchPropertiesWithFilter` operator is used to fetch and cache properties from shards for batches of rows, if they satisfy a predicate. The filtering of rows happens on the shards thereby reducing the number of rows processed on the graph shard. This is only planned if the predicate is applicable to the entity whose properties are being fetched and the expression doesn't depend on any other entities.

Infinigraph | Introduced in 2026.01

Example 615. RemoteBatchPropertiesWithFilter

Query

```
PROFILE
MATCH (m:Movie)
WHERE m.released > 2000
RETURN m.title AS title
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 4

+----------------------------------+----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| Operator                         | Id | Details                                 | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline      |
+----------------------------------+----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| +ProduceResults                  |  0 | title                                   |              3 |
3 |       0 |              0 |                    0/0 |     0.063 |               |
| |                                +----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+               |
| +Projection                      |  1 | cache[m.title] AS title                 |              3 |
3 |       0 |                |                    0/0 |     0.052 |               |
| |                                +----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+               |
| +RemoteBatchPropertiesWithFilter |  2 | cache[m.title], m.released > $autoint_0 |              3 |
3 |       3 |            376 |                    0/0 |   515.105 | In Pipeline 1 |
| |                                +----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+
| +NodeByLabelScan                 |  3 | m:Movie                                 |             10 |
10 |      11 |            376 |                    1/0 |     3.656 | In Pipeline 0 |
+----------------------------------+----+-----------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------+

Total database accesses: 14, total allocated memory: 696
```

Remote Node Index Seek

Infinigraph | Introduced in 2026.07

The `RemoteNodeIndexSeek` operator batches index seeks when applied within a loop.

Example 616. RemoteNodeIndexSeek

Query

```
PROFILE
MATCH (acc: Account {id: "foo"}), (purchase:Purchase)
WHERE acc.createdAt = purchase.transactedAt
RETURN *
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| Operator               | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used           |
+-------------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| +ProduceResults        |  0 | acc, purchase
|              0 |    0 |       0 |                |                    0/0 |     0.000 | In Pipeline
2 |                        |
| |                      +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| +Apply                 |  1 |
|              0 |    0 |       0 |                |                    0/0 |     0.014 |
|                        |
| |\                     +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| | +RemoteNodeIndexSeek |  2 | RANGE INDEX purchase:Purchase(transactedAt) WHERE transactedAt =
cache[acc.createdAt] |              0 |    0 |       0 |            128 |                    0/0 |
0.000 | In Pipeline 2 |                        |
| |                      +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| +RemoteBatchProperties |  3 | cache[acc.createdAt]
|              0 |    0 |       0 |            328 |                    0/0 |     1.110 | In Pipeline
1 |                        |
| |                      +----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------+------------------------+
| +NodeUniqueIndexSeek   |  4 | UNIQUE acc:Account(id) WHERE id = $autostring_0
|              0 |    0 |       1 |            376 |                    0/0 |    25.825 | In Pipeline
0 | constraint_93192f92: 1 |
+-------------------------+----
+-----------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
```

```
+--------------+-----------------------+

Total database accesses: 1, total allocated memory: 912
```

Remote Unique Node Index Seek

Infinigraph Introduced in 2026.07

The `RemoteUniqueNodeIndexSeek` operator batches unique index seeks within loops.

Example 617. RemoteUniqueNodeIndexSeek

Query

```
PROFILE
MATCH (purchase:Purchase)
WHERE  purchase.transactedAt > datetime('2019-01-01T00:00:00.000Z')
MATCH (active: Account)
WHERE active.id = purchase.accountId
RETURN DISTINCT active
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| Operator                     | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
| Indexes Used      |
+------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| +ProduceResults              |  0 | active
|              0 |    0 |       0 |                |                    0/0 |     0.000 |
|                   |
| |                            +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
+-------------------+
| +Distinct                    |  1 | active
|              0 |    0 |       0 |              0 |                    0/0 |     0.000 | In Pipeline
2 |                   |
| |                            +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| +Apply                       |  2 |
|              0 |    0 |       0 |                |                    0/0 |     0.009 |
|                   |
| |\                           +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| | +RemoteNodeUniqueIndexSeek |  3 | UNIQUE active:Account(id) WHERE id = cache[purchase.accountId]
|              0 |    0 |       0 |            128 |                    0/0 |     0.000 | In Pipeline
2 |                   |
| |                            +----
```

```
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| +RemoteBatchProperties       |  4 | cache[purchase.accountId]
|              0 |    0 |       0 |            328 |                    0/0 |     1.019 | In Pipeline
1 |                   |
| |                            +----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+
| +NodeIndexSeekByRange        |  5 | RANGE INDEX purchase:Purchase(transactedAt) WHERE transactedAt
> RuntimeConstant(datetime($autostrin |              0 |    0 |       1 |            376 |
0/0 |    50.717 | In Pipeline 0 | index_64559e44: 1 |
|                              |    | g_0))
|                |      |         |                |                        |           |
|                   |
+------------------------------+----
+-----------------------------------------------------------------------------------------------------
--+----------------+------+---------+----------------+------------------------+-----------+
--------------+-------------------+

Total database accesses: 1, total allocated memory: 912
```

Set Node Properties From Map

The `SetNodePropertiesFromMap` operator is used when setting properties from a map on a node.

Example 618. SetNodePropertiesFromMap

Query

```
PROFILE
MATCH (n)
SET n = {weekday: 'Monday', meal: 'Lunch'}
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+---------------------------+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+----------------------+
| Operator                  | Details                                             | Estimated Rows |
Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline             |
+---------------------------+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+----------------------+
| +ProduceResults           |                                                     |             35 |
0 |       0 |                |                        |           |                      |
| |                         +-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                      |
| +EmptyResult              |                                                     |             35 |
0 |       0 |                |                        |           |                      |
| |                         +-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                      |
| +SetNodePropertiesFromMap | n = {weekday: $autostring_0, meal: $autostring_1} |             35 |
35 |     105 |                |                        |           |                      |
| |                         +-----------------------------------------------------+----------------
+------+---------+----------------+                        |           |                      |
| +AllNodesScan             | n                                                   |             35 |
35 |      36 |            120 |                    5/0 |     3.954 | Fused in Pipeline 0 |
+---------------------------+-----------------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+----------------------+
```

```
Total database accesses: 141, total allocated memory: 184
```

Set Relationship Properties From Map

The `SetRelationshipPropertiesFromMap` operator is used when setting properties from a map on a relationship.

Example 619. SetRelationshipPropertiesFromMap

Query

```
PROFILE
MATCH (n)-[r]->(m)
SET r = {weight: 5, unit: 'kg'}
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------------------+-----------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+
| Operator                          | Details                                       | Estimated Rows
| Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------------------------+-----------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+
| +ProduceResults                   |                                               |             28
|    0 |       0 |                |                        |           |                     |
| |                                 +-----------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +EmptyResult                      |                                               |             28
|    0 |       0 |                |                        |           |                     |
| |                                 +-----------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +SetRelationshipPropertiesFromMap | r = {weight: $autoint_0, unit: $autostring_1} |             28
|   28 |      84 |                |                        |           |                     |
| |                                 +-----------------------------------------------+----------------
+------+---------+----------------+                        |           |                     |
| +DirectedAllRelationshipsScan     | (n)-[r]->(m)                                  |             28
|   28 |      28 |            120 |                    5/0 |    15.278 | Fused in Pipeline 0 |
+-----------------------------------+-----------------------------------------------+----------------
+------+---------+----------------+------------------------+-----------+---------------------+

Total database accesses: 112, total allocated memory: 184
```

Set Property

The `SetProperty` operator is used when setting a property on a node or relationship.

Example 620. SetProperty

Query

```
PROFILE
MATCH (n)
SET n.checked = true
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+------------------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| Operator        | Details          | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline            |
+-----------------+------------------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+
| +ProduceResults |                  |             35 |    0 |       0 |                |
|           |                     |
| |               +------------------+----------------+------+---------+----------------+
|           |                     |
| +EmptyResult    |                  |             35 |    0 |       0 |                |
|           |                     |
| |               +------------------+----------------+------+---------+----------------+
|           |                     |
| +SetProperty    | n.checked = true |             35 |   35 |      70 |                |
|           |                     |
| |               +------------------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   | n                |             35 |   35 |      36 |            120 |
3/0 |     0.753 | Fused in Pipeline 0 |
+-----------------+------------------+----------------+------+---------+----------------
+------------------------+-----------+--------------------+

Total database accesses: 106, total allocated memory: 184
```

Set Properties

The `SetProperties` operator is used when setting multiple properties on a node or relationship.

Example 621. SetProperties

Query

```
PROFILE
MATCH (n)
SET n.weekDay = 'Monday', n.meal = 'Lunch'
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| Operator        | Id | Details                                         | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----+-------------------------------------------------+----------------+------
+---------+----------------+------------------------+-----------+--------------------+
| +ProduceResults |  0 |                                                 |             35 |    0 |
0 |              0 |                        |           |                    |
| |               +----+-------------------------------------------------+----------------+------
+---------+----------------+                        |           |                    |
| +EmptyResult    |  1 |                                                 |             35 |    0 |
```

```
0 |               |                       |          |                    |
| |               +----+-----------------------------------------------------+----------------+------
+---------+---------------+                                  |          |                    |
| +SetProperties  |  2 | n.weekDay = $autostring_0, n.meal = $autostring_1 |             35 |   35 |
105 |               |                         |          |                    |
| |               +----+-----------------------------------------------------+----------------+------
+---------+---------------+                       |          |                    |
| +AllNodesScan   |  3 | n                                                    |             35 |   35 |
36 |            248 |                   3/0 |   152.289 | Fused in Pipeline 0 |
+-----------------+----+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+----------+--------------------+

Total database accesses: 141, total allocated memory: 312
```

Load CSV

The `LoadCSV` operator loads data from a CSV source into the query. It is used whenever the LOAD CSV clause is used in a query.

Example 622. LoadCSV

Query

```
PROFILE
LOAD CSV FROM 'https://neo4j.com/docs/cypher-refcard/3.3/csv/artists.csv' AS line
RETURN line
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------+
| Operator        | Details | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache
Hits/Misses | Time (ms) | Pipeline      |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------+
| +ProduceResults | line    |             10 |    4 |       0 |                |
0/0 |     0.210 |               |
| |               +---------+----------------+------+---------+----------------
+-------------------------+-----------+                |
| +LoadCSV        | line    |             10 |    4 |       0 |             72 |
|           | In Pipeline 1 |
+-----------------+---------+----------------+------+---------+----------------
+------------------------+-----------+---------------+

Total database accesses: 0, total allocated memory: 184
```

Eager

The `Eager` operator causes all preceding operators to execute fully, for the whole dataset, before continuing execution. This is done to ensure isolation between parts of the query plan that might otherwise affect each other.

Values from the graph are fetched in a lazy manner; i.e. a pattern matching might not be fully exhausted

before updates are applied. To maintain correct semantics, the query planner will insert `Eager` operators into the query plan to prevent updates from influencing pattern matching, or other read operations. This scenario is exemplified by the query below, where the `DELETE` clause would otherwise influence both the `MATCH` clause and the `MERGE` clause. For more information on how the `Eager` operator can ensure correct semantics, see the section on Clause composition.

The `Eager` operator can cause high memory usage when importing data or migrating graph structures. In such cases, the operations should be split into simpler steps; e.g. importing nodes and relationships separately. Alternatively, the records to be updated can be returned, followed by an update statement.

Example 623. Eager

Query

```
PROFILE
MATCH (a:Person {name: 'me'}), (b:Person {name: 'Bob'})
DETACH DELETE a, b
MERGE (:Person {name: 'me'})
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+----------------------+----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+----------------------+
| Operator             | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline
|
+----------------------+----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+----------------------+
| +ProduceResults      |  0 |
|              0 |    0 |       0 |              0 |                        |           |
|
| |                    +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +EmptyResult         |  1 |
|              0 |    0 |       0 |                |                        |           |
|
| |                    +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Apply               |  2 |
|              0 |    1 |       0 |                |                        |           |
|
| |\                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| | +Merge             |  3 | CREATE (anon_0:Person {name: $autostring_2})
|              0 |    1 |       3 |                |                        |           |
|
| | |                  +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
```

```
| | +NodeIndexSeek    |  4 | RANGE INDEX anon_0:Person(name) WHERE name = $autostring_2
|              0 |    0 |       1 |           3304 |                    1/0 |     0.663 | Fused in
Pipeline 3 |
| |                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +Eager              |  5 | read/delete conflict for variable: anon_0 (Operator: 6 vs 4, and 1 more
conflicting operators) |              0 |    1 |       0 |            360 |                    0/0 |
0.008 | In Pipeline 2       |
| |                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +DetachDelete       |  6 | b
|              0 |    1 |       4 |                |                        |           |
|
| |                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +DetachDelete       |  7 | a
|              0 |    1 |       5 |                |                        |           |
|
| |                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +Eager              |  8 | read/delete conflict for variable: b (Operator: 6 vs 9, and 1 more
conflicting operators),     |              0 |    1 |       0 |            360 |
1/0 |     0.226 | Fused in Pipeline 1 |
| |                   |    | read/set conflict for label: Person (Operator: 3 vs 9),
|                |      |         |                |                        |           |
|
| |                   |    | read/set conflict for property: name (Operator: 3 vs 9)
|                |      |         |                |                        |           |
|
| |                   +----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +MultiNodeIndexSeek |  9 | RANGE INDEX a:Person(name) WHERE name = $autostring_0,
|              0 |    1 |       4 |            376 |                    2/0 |     0.218 | In Pipeline
0       |
|                   |    | RANGE INDEX b:Person(name) WHERE name = $autostring_1
|                |      |         |                |                        |           |
|
+---------------------+----
+-------------------------------------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 17, total allocated memory: 4184
```

Assert Same Node

The `AssertSameNode` operator is used to ensure that no node property uniqueness constraints are violated in the slotted and interpreted runtime.

Example 624. AssertSameNode

The example looks for the presence of a team node with the supplied name and id, and if one does not exist, it will be created. Owing to the existence of two node property uniqueness constraints on `:Team(name)` and `:Team(id)`, any node that would be found by the `UniqueIndexSeek` operator must be the very same node or the constraints would be violated. Had the query been run with the pipelined runtime, the task of `AssertSameNode` would have been performed by the `AssertingMultiNodeIndexSeek` operator.

Query

```
PROFILE
CYPHER runtime=slotted
MERGE (t:Team {name: 'Engineering', id: 42})
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------------------------+-----------------------------------------------------
+----------------+------+---------+-----------------------+
| Operator                          | Details                                                | Estimated
Rows | Rows | DB Hits | Page Cache Hits/Misses |
+-----------------------------------+-----------------------------------------------------
+----------------+------+---------+-----------------------+
| +ProduceResults                   |                                                        |
1 |    0 |       0 |                  0/0 |
| |                                 +--------------------------------------------------------
+----------------+------+---------+-----------------------+
| +EmptyResult                      |                                                        |
1 |    0 |       0 |                  0/0 |
| |                                 +--------------------------------------------------------
+----------------+------+---------+-----------------------+
| +Merge                            | CREATE (t:Team {name: $autostring_0, id: $autoint_1}) |
1 |    1 |       0 |                  0/0 |
| |                                 +--------------------------------------------------------
+----------------+------+---------+-----------------------+
| +AssertSameNode                   | t                                                      |
0 |    1 |       0 |                  0/0 |
| |\                                +--------------------------------------------------------
+----------------+------+---------+-----------------------+
| | +NodeUniqueIndexSeek(Locking) | UNIQUE t:Team(id) WHERE id = $autoint_1                |
1 |    1 |       1 |                  0/1 |
| |                                 +--------------------------------------------------------
+----------------+------+---------+-----------------------+
| +NodeUniqueIndexSeek(Locking)     | UNIQUE t:Team(name) WHERE name = $autostring_0         |
1 |    1 |       1 |                  0/1 |
+-----------------------------------+-----------------------------------------------------
+----------------+------+---------+-----------------------+

Total database accesses: 2, total allocated memory: 64
```

Assert Same Relationship

The `AssertSameRelationship` operator is used to ensure that no relationship property uniqueness constraints are violated in the slotted and interpreted runtime.

Example 625. AssertSameRelationship

The example looks for the presence of a `WORKS_IN` relationship with the supplied `id` and `badgeNumber`. If it can't be found, then it will be created. Owing to the existence of two property uniqueness constraints on `:WORKS_IN(id)` and `:WORKS_IN(badgeNumber)`, any relationship that would be found by the `DirectedRelationshipUniqueIndexSeek` operator must be the very same relationship or the constraints would be violated.

Had the query been run with the pipelined runtime, the task of `AssertRelationship` would have been performed by the `AssertingMultiRelationshipIndexSeek` operator.

Query

```
PROFILE
CYPHER runtime=slotted
MERGE (person)-[work:WORKS_IN {id: 0, badgeNumber: 4332}]->(location)
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------------------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| Operator                                      | Id | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses |
+-----------------------------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| +ProduceResults                               |  0 |
|              1 |    0 |       0 |                   0/0 |
| |                                             +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| +EmptyResult                                  |  1 |
|              1 |    0 |       0 |                   0/0 |
| |                                             +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| +Merge                                        |  2 | CREATE (person), (location), (person)-
[work:WORKS_IN {id: $autoint_0, badgeNumber: $autoint_1}]->(lo |              1 |    1 |       0 |
0/0 |
| |                                             |    | cation)
|                |      |         |                       |
| |                                             +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| +AssertSameRelationship                       |  3 | work
|              0 |    1 |       0 |                   0/0 |
| |\                                            +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| | +DirectedRelationshipUniqueIndexSeek(Locking) |  4 | RANGE INDEX (person)-
[work:WORKS_IN(badgeNumber)]->(location) WHERE badgeNumber = $autoint_1         |              1 |
1 |       1 |                    0/1 |
| |                                             +----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+
| +DirectedRelationshipUniqueIndexSeek(Locking)   |  5 | RANGE INDEX (person)-[work:WORKS_IN(id)]-
>(location) WHERE id = $autoint_0                          |              1 |    1 |       1 |
1/1 |
+-----------------------------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+----------------+------+---------+-----------------------+

Total database accesses: 2, total allocated memory: 64
```

Schema and system operators

The operators in this group manage the schema and system-level operations within the database. They handle tasks such as creating or dropping constraints and indexes, showing or terminating transactions, and listing the available procedures, functions, and settings.

Create Constraint

The `CreateConstraint` operator creates a constraint.

This constraint can have any of the available constraint types:

- Property uniqueness constraints
- Property existence constraints Enterprise Edition
- Property type constraints Enterprise Edition
- Node or relationship key constraints Enterprise Edition

The following query will create a property uniqueness constraint with the name `uniqueness` on the `name` property of nodes with the `Country` label.

Example 626. CreateConstraint

Query

```
PROFILE
CREATE CONSTRAINT uniqueness
FOR (c:Country) REQUIRE c.name is UNIQUE
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+-------------------+-------------------------------------------------------------------+
| Operator          | Details                                                           |
+-------------------+-------------------------------------------------------------------+
| +CreateConstraint | CONSTRAINT uniqueness FOR (c:Country) REQUIRE (c.name) IS UNIQUE |
+-------------------+-------------------------------------------------------------------+

Total database accesses: ?
```

Do Nothing If Exists (constraint)

To not get an error creating the same constraint twice, we use the `DoNothingIfExists` operator for constraints. This will make sure no other constraint with the given name or another constraint of the same type and schema already exists before the specific `CreateConstraint` operator creates the constraint. If it finds a constraint with the given name or with the same type and schema it will stop the execution and no new constraint is created. The following query will create a property uniqueness constraint with the name `uniqueness` on the `name` property of nodes with the `Country` label only if no constraint named `uniqueness` or property uniqueness constraint on `(:Country {name})` already exists.

Example 627. DoNothingIfExists(CONSTRAINT)

Query

```
PROFILE
CREATE CONSTRAINT uniqueness IF NOT EXISTS
FOR (c:Country) REQUIRE c.name is UNIQUE
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+--------------------------------+-------------------------------------------------------------------------+
| Operator                       | Details                                                                 |
+--------------------------------+-------------------------------------------------------------------------+
| +CreateConstraint              | CONSTRAINT uniqueness FOR (c:Country) REQUIRE (c.name) IS UNIQUE |
| |                              +-------------------------------------------------------------------------+
| +DoNothingIfExists(CONSTRAINT) | CONSTRAINT uniqueness FOR (c:Country) REQUIRE (c.name) IS UNIQUE |
+--------------------------------+-------------------------------------------------------------------------+

Total database accesses: ?
```

Drop Constraint

The `DropConstraint` operator removes a constraint using the name of the constraint, no matter the type.

Example 628. DropConstraint

Query

```
PROFILE
DROP CONSTRAINT uniqueness IF EXISTS
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+-----------------+----------------------------------+
| Operator        | Details                          |
+-----------------+----------------------------------+
| +DropConstraint | CONSTRAINT uniqueness IF EXISTS |
+-----------------+----------------------------------+

Total database accesses: ?
```

Show Constraints

The `ShowConstraints` operator lists constraints. It may include filtering on constraint type and can have either default or full output.

Example 629. ShowConstraints

Query

```
PROFILE
SHOW CONSTRAINTS
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+-------------------------------------------------------------------------
+---------------+------+---------+-----------------------+
| Operator        | Details                                                                 | Estimated
Rows | Rows | DB Hits | Page Cache Hits/Misses |
+-----------------+-------------------------------------------------------------------------
+---------------+------+---------+-----------------------+
| +ProduceResults  | id, name, type, entityType, labelsOrTypes, properties, ownedIndex |
10 |    3 |       0 |                   0/0 |
| |                +------------------------------------------------------------------
+---------------+------+---------+-----------------------+
| +ShowConstraints | allConstraints, defaultColumns                                       |
10 |    3 |       2 |                   0/0 |
+-----------------+-------------------------------------------------------------------------
+---------------+------+---------+-----------------------+

Total database accesses: 2, total allocated memory: 64
```

Alter Current Graph Type Add

Cypher 25 only | Introduced in Neo4j 2026.02

The `AlterCurrentGraphTypeAdd` operator adds the specified graph type elements to the current graph type.

Example 630. AlterCurrentGraphTypeAdd

Query

```
PROFILE
ALTER CURRENT GRAPH TYPE ADD {
 (:Label1 => :Label2 {prop :: STRING NOT NULL}),
 (:Label3)-[:REL_TYPE => {prop :: INT NOT NULL}]->()
}
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+---------------------------+----
+-------------------------------------------------------------------------------------------------------
--+
| Operator                  | Id | Details
|
+---------------------------+----
+-------------------------------------------------------------------------------------------------------
--+
| +AlterCurrentGraphTypeAdd |  0 | { (:Label1 => :Label2 {prop :: STRING NOT NULL}), (:Label3)-
[:REL_TYPE => {prop :: INTEGER NOT NULL} |
|                           |    | ]->() }
|
+---------------------------+----
+-------------------------------------------------------------------------------------------------------
--+
```

```
Total database accesses: ?
```

Alter Current Graph Type Alter

Cypher 25 only | Introduced in Neo4j 2026.02

The `AlterCurrentGraphTypeAlter` operator updates the specified graph type elements. This is done by replacing the current definition of those elements with the specified definition.

Example 631. AlterCurrentGraphTypeAlter

Query

```
PROFILE
ALTER CURRENT GRAPH TYPE ALTER {
 (:Label1 => {prop :: STRING NOT NULL}),
 (:Label3)-[:REL_TYPE => {prop :: INT}]->()
}
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+------------------------------+----
+-----------------------------------------------------------------------------------------------+
| Operator                     | Id | Details
|
+------------------------------+----
+-----------------------------------------------------------------------------------------------+
| +AlterCurrentGraphTypeAlter |  0 | { (:Label1 => {prop :: STRING NOT NULL}), (:Label3)-[:REL_TYPE
=> {prop :: INTEGER}]->() } |
+------------------------------+----
+-----------------------------------------------------------------------------------------------+

Total database accesses: ?
```

Alter Current Graph Type Drop

Cypher 25 only | Introduced in Neo4j 2026.02

The `AlterCurrentGraphTypeDrop` operator drops the specified graph type elements.

Example 632. AlterCurrentGraphTypeDrop

Query

```
PROFILE
ALTER CURRENT GRAPH TYPE DROP {
 (:Label1 =>),
 ()-[:REL_TYPE =>]->()
}
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+----------------------------+----+----------------------------------------+
| Operator                   | Id | Details                                |
+----------------------------+----+----------------------------------------+
| +AlterCurrentGraphTypeDrop |  0 | { (:Label1 =>), ()-[:REL_TYPE =>]->() } |
+----------------------------+----+----------------------------------------+

Total database accesses: ?
```

Alter Current Graph Type Set

Cypher 25 only | Introduced in Neo4j 2026.02

The `AlterCurrentGraphTypeSet` operator replaces the current graph type with the specified graph type.

Example 633. AlterCurrentGraphTypeSet

Query

```
PROFILE
ALTER CURRENT GRAPH TYPE SET {
 (:Label1 => :Label2 {prop :: STRING NOT NULL}),
 (:Label3)-[:REL_TYPE => {prop :: INT NOT NULL}]->()
}
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+----------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+
| Operator                   | Id | Details
|
+----------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+
| +AlterCurrentGraphTypeSet |  0 | { (:Label1 => :Label2 {prop :: STRING NOT NULL}), (:Label3)-
[:REL_TYPE => {prop :: INTEGER NOT NULL} |
|                            |    | ]->() }
|
+----------------------------+----
+-----------------------------------------------------------------------------------------------------------
--+

Total database accesses: ?
```

Show Current Graph Type

Cypher 25 only | Introduced in Neo4j 2026.02

The `ShowCurrentGraphType` operator lists the current graph type.

Example 634. ShowCurrentGraphType

Query

```
PROFILE
SHOW CURRENT GRAPH TYPE
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------------+----+-----------------------------------+----------------+------+---------
+-----------------------+
| Operator              | Id | Details                           | Estimated Rows | Rows | DB Hits |
Page Cache Hits/Misses |
+-----------------------+----+-----------------------------------+----------------+------+---------
+-----------------------+
| +ProduceResults       |  0 | specification                     |             10 |    1 |       0 |
0/0 |
| |                     +----+-----------------------------------+----------------+------+---------
+-----------------------+
| +ShowCurrentGraphType |  1 | graphTypeAsString, defaultColumns |             10 |    1 |       1 |
0/0 |
+-----------------------+----+-----------------------------------+----------------+------+---------
+-----------------------+

Total database accesses: 1, total allocated memory: 64
```

The `graphTypeAsString` was added in Neo4j 2026.07, before that the `ShowCurrentGraphType` `Details` were `defaultColumns`.

Create Index

The `CreateIndex` operator creates an index.

This index can either be a fulltext, point, range, text, vector, or lookup index.

Example 635. CreateIndex

The following query will create an index with the name `my_index` on the `name` property of nodes with the `Country` label.

Query

```
PROFILE
CREATE INDEX my_index
FOR (c:Country) ON (c.name)
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA
```

```
Runtime version 2026.10

+--------------+-----------------------------------------------+
| Operator     | Details                                       |
+--------------+-----------------------------------------------+
| +CreateIndex | RANGE INDEX my_index FOR (:Country) ON (name) |
+--------------+-----------------------------------------------+

Total database accesses: ?
```

Do Nothing If Exists (index)

To not get an error creating the same index twice, we use the `DoNothingIfExists` operator for indexes. This will make sure no other index with the given name or schema already exists before the `CreateIndex` operator creates an index. If it finds an index with the given name or schema it will stop the execution and no new index is created. The following query will create an index with the name `my_index` on the `since` property of relationships with the `KNOWS` relationship type only if no such index already exists.

Example 636. DoNothingIfExists(INDEX)

Query

```
PROFILE
CREATE INDEX my_index IF NOT EXISTS
FOR ()-[k:KNOWS]-() ON (k.since)
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+---------------------------+-----------------------------------------------------+
| Operator                  | Details                                             |
+---------------------------+-----------------------------------------------------+
| +CreateIndex              | RANGE INDEX my_index FOR ()-[:KNOWS]-() ON (since) |
| |                         +-----------------------------------------------------+
| +DoNothingIfExists(INDEX) | RANGE INDEX my_index FOR ()-[:KNOWS]-() ON (since) |
+---------------------------+-----------------------------------------------------+

Total database accesses: ?
```

Drop Index

The `DropIndex` operator removes an index using the name of the index.

Example 637. DropIndex

Query

```
PROFILE
DROP INDEX my_index
```

Query Plan

```
Planner ADMINISTRATION

Runtime SCHEMA

Runtime version 2026.10

+------------+---------------+
| Operator   | Details       |
+------------+---------------+
| +DropIndex | INDEX my_index|
+------------+---------------+

Total database accesses: ?
```

Show Indexes

The `ShowIndexes` operator lists indexes. It may include filtering on index type and can have either default or full output.

Example 638. ShowIndexes

Query

```
PROFILE
SHOW INDEXES
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| Operator        | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses |
+-----------------
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| +ProduceResults | id, name, state, populationPercent, type, entityType, labelsOrTypes, properties,
indexProvider, |             10 |    9 |       0 |                   0/0 |
| |               | owningConstraint
|                |      |         |                       |
| |
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| +ShowIndexes    | allIndexes, defaultColumns
|             10 |    9 |       2 |                   0/0 |
+-----------------
+-------------------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+

Total database accesses: 2, total allocated memory: 64
```

Show Functions

The `ShowFunctions` operator lists functions. It may include filtering on built-in vs user-defined functions as

well as if a given user can execute the function. The output can either be default or full output.

Example 639. ShowFunctions

Query

```
PROFILE
SHOW FUNCTIONS
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+-------------------------------------------------------+----------------+------
+---------+-----------------------+
| Operator        | Details                                               | Estimated Rows | Rows | DB
Hits | Page Cache Hits/Misses |
+-----------------+-------------------------------------------------------+----------------+------
+---------+-----------------------+
| +ProduceResults | name, category, description                           |             10 |  147 |
0 |                   0/0 |
| |               +-------------------------------------------------------+----------------+------
+---------+-----------------------+
| +ShowFunctions  | allFunctions, functionsForUser(all), defaultColumns |             10 |  147 |
0 |                   0/0 |
+-----------------+-------------------------------------------------------+----------------+------
+---------+-----------------------+

Total database accesses: 0, total allocated memory: 64
```

Show Procedures

The `ShowProcedures` operator lists procedures. It may include filtering on whether a given user can execute the procedure and can have either default or full output.

Example 640. ShowProcedures

Query

```
PROFILE
SHOW PROCEDURES
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+-----------------------------------------+----------------+------+---------
+------------------------+
| Operator        | Details                                 | Estimated Rows | Rows | DB Hits | Page
Cache Hits/Misses |
+-----------------+-----------------------------------------+----------------+------+---------
+------------------------+
| +ProduceResults | name, description, mode, worksOnSystem |             10 |   55 |       0 |
0/0 |
| |               +-----------------------------------------+----------------+------+---------
```

```
+------------------------+
| +ShowProcedures | proceduresForUser(all), defaultColumns |             10 |   55 |       0 |
0/0 |
+-----------------+-----------------------------------------+----------------+------+---------
+------------------------+

Total database accesses: 0, total allocated memory: 64
```

Show Settings

The `ShowSettings` operator lists configuration settings.

Example 641. ShowSettings

Query

```
PROFILE
SHOW SETTINGS
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------+-----------------------------------------------------+----------------+------
+---------+------------------------+
| Operator        | Details                                             | Estimated Rows | Rows | DB
Hits | Page Cache Hits/Misses |
+-----------------+-----------------------------------------------------+----------------+------
+---------+------------------------+
| +ProduceResults | name, value, isDynamic, defaultValue, description |             10 |  264 |
0 |                    0/0 |
| |               +-----------------------------------------------------+----------------+------
+---------+------------------------+
| +ShowSettings   | allSettings, defaultColumns                         |             10 |  264 |
0 |                    0/0 |
+-----------------+-----------------------------------------------------+----------------+------
+---------+------------------------+

Total database accesses: 0, total allocated memory: 64
```

Show Transactions

The `ShowTransactions` operator lists transactions. It may include filtering on given ids and can have either default or full output.

Example 642. ShowTransactions

Query

```
PROFILE
SHOW TRANSACTIONS
```

Query Plan

```
Planner COST
```

```
Runtime SLOTTED

Runtime version 2026.10

+------------------
+-----------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| Operator          | Details
| Estimated Rows | Rows | DB Hits | Page Cache Hits/Misses |
+------------------
+-----------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| +ProduceResults   | database, transactionId, currentQueryId, connectionId, clientAddress, username,
currentQuery, |             10 |    1 |       0 |                   0/0 |
| |                 | startTime, status, elapsedTime
|                |      |         |                       |
| |
+-----------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+
| +ShowTransactions | defaultColumns, allTransactions
|             10 |    1 |       0 |                   0/0 |
+------------------
+-----------------------------------------------------------------------------------------------
+----------------+------+---------+-----------------------+

Total database accesses: 0, total allocated memory: 64
```

Terminate Transactions

The `TerminateTransactions` operator terminates transactions by ID.

Example 643. TerminateTransactions

Query

```
PROFILE
TERMINATE TRANSACTIONS 'database-transaction-123'
```

Query Plan

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-----------------------+-------------------------------------------------------------+----------------
+------+---------+-----------------------+
| Operator              | Details                                                     | Estimated Rows |
Rows | DB Hits | Page Cache Hits/Misses |
+-----------------------+-------------------------------------------------------------+----------------
+------+---------+-----------------------+
| +ProduceResults       | transactionId, username, message                            |             10 |
1 |       0 |                   0/0 |
| |                     +-------------------------------------------------------------+----------------
+------+---------+-----------------------+
| +TerminateTransactions | defaultColumns, transactions(database-transaction-123) |             10 |
1 |       0 |                   0/0 |
+-----------------------+-------------------------------------------------------------+----------------
+------+---------+-----------------------+

Total database accesses: 0, total allocated memory: 64
```

Show Databases

Cypher 25 only | Introduced in Neo4j 2026.05

The `ShowDatabases` operator lists databases. It may include filtering on all databases, a single database, the DBMS default database or the users home database as well as if the user can access the database. The output can either be default or full output.

Example 644. ShowDatabases

Query

```
PROFILE
SHOW DATABASES
```

Query Plan

```
Planner ADMINISTRATION

Runtime SYSTEM

Runtime version 2026.10

+-----------------+----
+-------------------------------------------------------------------------------------------------------
+------+---------+
| Operator        | Id | Details
| Rows | DB Hits |
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+------+---------+
| +ProduceResults |  0 | name, type, aliases, access, address, role, writer, requestedStatus,
currentStatus, statusMessage, |    0 |       1 |
| |               |    | default, home, constituents
|      |         |
| |               +----
+-------------------------------------------------------------------------------------------------------
+------+---------+
| +ShowDatabases  |  1 | allDatabases, defaultColumns
|    0 |       1 |
+-----------------+----
+-------------------------------------------------------------------------------------------------------
+------+---------+

Total database accesses: 2
```

Cypher runtimes

The runtime is the final step of a Cypher query, where query plans received from the planner are executed as quickly and efficiently as possible.

Cypher has three available runtimes: slotted, pipelined, and parallel. Though the default runtime generally provides the best performance, there are situations when deciding which runtime to use is an important part of maximizing the efficiency of queries. It is, therefore, important for advanced users to understand the different runtimes offered by Neo4j.

This chapter contains the following sections:

- Runtime concepts - a deep-dive into the concepts behind Cypher runtimes.
- Parallel runtime: reference - information about queries, configuration settings, and using the parallel

runtime on Aura.

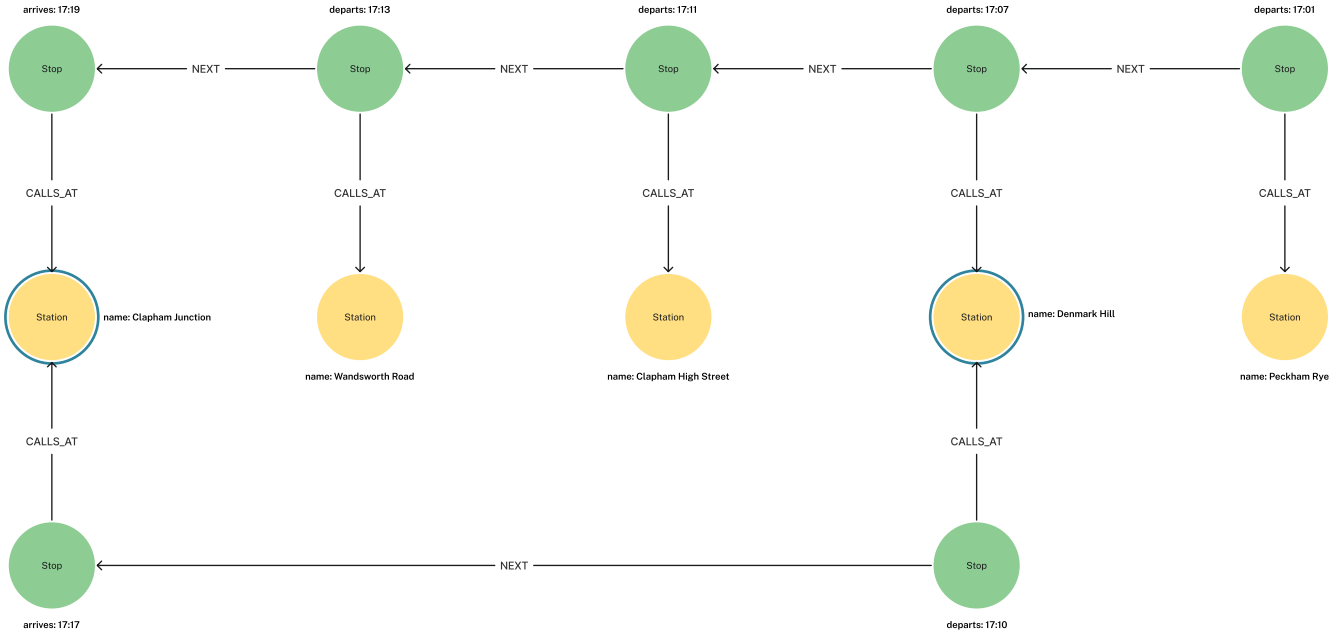
Runtime concepts

In Cypher, there are three types of runtimes: slotted, pipelined, and parallel. In general, the default runtimes (the pipelined runtime in Enterprise Edition) provide the best query performance. However, each runtime offers advantages and disadvantages, and there are scenarios when deciding which runtime to use is an important step in maximizing the efficiency of queries.

This is a step-by-step guide to the concepts behind each of the three available Cypher runtimes. For readers not familiar with reading the execution plans produced by Cypher queries, it is recommended to first read the section on Understanding query plans.

Example graph

The following graph is used for the queries on this page:



The graph contains two types of nodes: `Stop` and `Station`. Each `Stop` on a train service `CALLS_AT` one `Station`, and has the properties `arrives` and `departs` that give the times the train is at the `Station`. Following the `NEXT` relationship of a `Stop` will give the next `Stop` of a service.

To recreate the graph, run the following query against an empty Neo4j database:

Query

```
CREATE (pmr:Station {name: 'Peckham Rye'}),
  (dmk:Station {name: 'Denmark Hill'}),
  (clp:Station {name: 'Clapham High Street'}),
  (wwr:Station {name: 'Wandsworth Road'}),
  (clj:Station {name: 'Clapham Junction'}),
  (s1:Stop {arrives: time('17:19'), departs: time('17:20')}),
  (s2:Stop {arrives: time('17:12'), departs: time('17:13')}),
  (s3:Stop {arrives: time('17:10'), departs: time('17:11')}),
  (s4:Stop {arrives: time('17:06'), departs: time('17:07')}),
  (s5:Stop {arrives: time('16:58'), departs: time('17:01')}),
  (s6:Stop {arrives: time('17:17'), departs: time('17:20')}),
  (s7:Stop {arrives: time('17:08'), departs: time('17:10')}),
  (clj)<-[:CALLS_AT]-(s1), (wwr)<-[:CALLS_AT]-(s2),
```

```
  (clp)<-[:CALLS_AT]-(s3), (dmk)<-[:CALLS_AT]-(s4),
  (pmr)<-[:CALLS_AT]-(s5), (clj)<-[:CALLS_AT]-(s6),
  (dmk)<-[:CALLS_AT]-(s7),
  (s5)-[:NEXT {distance: 1.2}]->(s4),(s4)-[:NEXT {distance: 0.34}]->(s3),
  (s3)-[:NEXT {distance: 0.76}]->(s2), (s2)-[:NEXT {distance: 0.3}]->(s1),
  (s7)-[:NEXT {distance: 1.4}]->(s6)
```

Slotted runtime

The slotted runtime is the default runtime for Neo4j Community Edition. Users of Neo4j Enterprise Edition must prepend their query with `CYPHER runtime = slotted` in order for a query to run with slotted runtime. For example:

Query

```
EXPLAIN
CYPHER runtime = slotted
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
      ((:Stop)-[:NEXT]->(:Stop))+
      (a:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
RETURN count(*)
```

This query will generate the following execution plan:

```
Planner COST

Runtime SLOTTED

Runtime version 2026.10

+-------------------+----+-------------------------------------------------------------------------
+----------------+
| Operator          | Id | Details                                                                 |
Estimated Rows |
+-------------------+----+-------------------------------------------------------------------------
+----------------+
| +ProduceResults   |  0 | `count(*)`                                                              |
1 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +EagerAggregation |  1 | count(*) AS `count(*)`                                                  |
1 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Filter           |  2 | not anon_1 = anon_5 AND anon_0.name = $autostring_0 AND anon_0:Station |
0 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Expand(All)      |  3 | (d)-[anon_1:CALLS_AT]->(anon_0)                                         |
0 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Filter           |  4 | d:Stop                                                                  |
0 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Repeat(Trail)    |  5 | (a) (...){1, *} (d)                                                     |
0 |
| |\                +----+-------------------------------------------------------------------------
+----------------+
| | +Filter         |  6 | isRepeatTrailUnique(anon_7) AND anon_2:Stop                             |
6 |
| | |               +----+-------------------------------------------------------------------------
+----------------+
| | +Expand(All)    |  7 | (anon_4)<-[anon_7:NEXT]-(anon_2)                                        |
6 |
| | |               +----+-------------------------------------------------------------------------
+----------------+
| | +Filter         |  8 | anon_4:Stop                                                             |
```

```
11 |
| | |               +----+-------------------------------------------------------------------------
+----------------+
| | +Argument       |  9 | anon_4                                                                  |
13 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Filter           | 10 | a:Stop                                                                  |
0 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Expand(All)      | 11 | (anon_6)<-[anon_5:CALLS_AT]-(a)                                         |
0 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +Filter           | 12 | anon_6.name = $autostring_1                                             |
1 |
| |                 +----+-------------------------------------------------------------------------
+----------------+
| +NodeByLabelScan  | 13 | anon_6:Station                                                          |
10 |
+-------------------+----+-------------------------------------------------------------------------
+----------------+
```

The physical plan produced by slotted runtimes is a one-to-one mapping from the logical plan, where each logical operator maps to a corresponding physical operator, and where the operators are processed row-by-row. When using slotted runtime, each variable in the query gets a dedicated “slot”, which the runtime uses for accessing the data mapped to the given variable, hence the name “slotted”.

The slotted runtime uses the traditional execution model of most databases known as the iterator or “Volcano” model. This is a pull-based process where each operator in the tree “pulls” rows of data from its child operator by using a virtual call function. In this way, data is pulled up from the bottom of the execution plan to the top, generating an eruption-like flow of data.

Considerations

The slotted runtime is the first high-performance runtime introduced in Neo4j, replacing the original (and slower) interpreted runtime, which is now retired.

The slotted runtime is an interpreted runtime, meaning that it interprets the logical plan sent by the planner operator-by-operator. In general, this is a convenient and flexible approach capable of handling all operators and queries. The slotted runtime is conceptually similar to interpreted programming languages, in that it has a shorter planning phase because it does not need to generate all the code for the query before execution (unlike compiled runtimes - discussed in more detail below).[2]

In general, users of Neo4j Enterprise Edition should not have to use slotted runtime. However, there are scenarios where the fast planning phase of the slotted runtime may be useful. For example, if you are using an application that generates short queries that are not cached (i.e. never, or very rarely, repeated), then the slotted runtime may be preferable because of its faster planning time.

There are, however, limitations to the slotted runtime. The continuous calling of virtual functions between each operator uses CPU cycles which results in slower query execution. Furthermore, the iterator model can lead to poor data locality, which can cause a slower query execution. This is because the process of individual rows being pulled from different operators makes it difficult to make efficient use of CPU caches.

Pipelined runtime

Enterprise Edition

The pipelined runtime is the default runtime for Neo4j Enterprise Edition and all tiers of Aura. This means that unless users of Neo4j Aura or Enterprise Edition specify a different runtime, queries will be run using the pipelined runtime.

Query

```
EXPLAIN
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
      ((:Stop)-[:NEXT]->(:Stop))+
      (a:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
RETURN count(*)
```

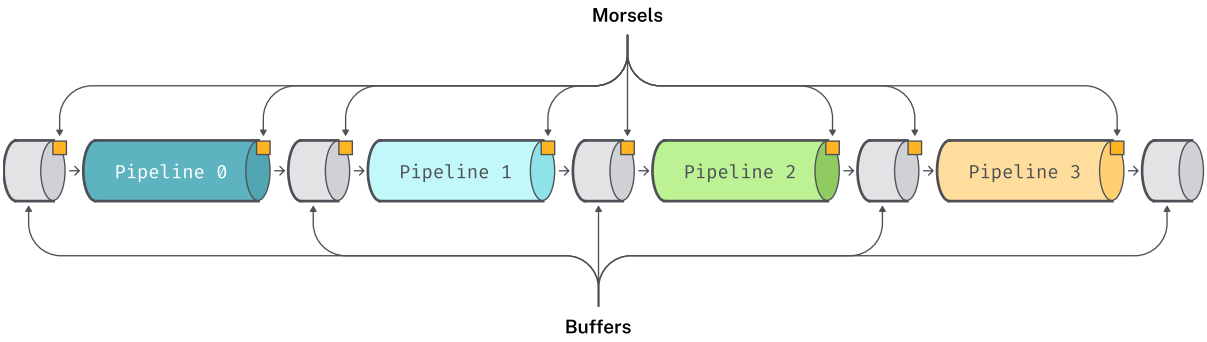
The resulting execution plan contains notable differences from the one produced by slotted runtime:

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------+----+-------------------------------------------------------------------------------
+----------------+---------------------+
| Operator          | Id | Details                                                                       |
Estimated Rows | Pipeline            |
+-------------------+----+-------------------------------------------------------------------------------
+----------------+---------------------+
| +ProduceResults   |  0 | `count(*)`                                                                    |
1 | In Pipeline 3       |
| |                 +----+-------------------------------------------------------------------------------
+----------------+---------------------+
| +EagerAggregation |  1 | count(*) AS `count(*)`                                                        |
1 |                     |
| |                 +----+-------------------------------------------------------------------------------
+----------------+                          |
| +Filter           |  2 | not anon_1 = anon_5 AND anon_0.name = $autostring_0 AND anon_0:Station |
0 |                     |
| |                 +----+-------------------------------------------------------------------------------
+----------------+                          |
| +Expand(All)      |  3 | (d)-[anon_1:CALLS_AT]->(anon_0)                                               |
0 |                     |
| |                 +----+-------------------------------------------------------------------------------
+----------------+                          |
| +Filter           |  4 | d:Stop                                                                        |
0 |                     |
| |                 +----+-------------------------------------------------------------------------------
+----------------+                          |
| +NullifyMetadata  | 14 |                                                                               |
0 |                     |
| |                 +----+-------------------------------------------------------------------------------
+----------------+                          |
| +Repeat(Trail)    |  5 | (a) (...){1, *} (d)                                                           |
0 | Fused in Pipeline 2 |
| |\                +----+-------------------------------------------------------------------------------
+----------------+---------------------+
| | +Filter         |  6 | isRepeatTrailUnique(anon_7) AND anon_2:Stop                                   |
6 |                     |
| | |               +----+-------------------------------------------------------------------------------
+----------------+                          |
| | +Expand(All)    |  7 | (anon_4)<-[anon_7:NEXT]-(anon_2)                                              |
6 |                     |
| | |               +----+-------------------------------------------------------------------------------
+----------------+                          |
| | +Filter         |  8 | anon_4:Stop                                                                   |
11 |                     |
| | |               +----+-------------------------------------------------------------------------------
+----------------+                          |
```

```
| | +Argument       |  9 | anon_4                                                                    |
13 | Fused in Pipeline 1 |
| |                 +----+---------------------------------------------------------------------------
+----------------+--------------------+
| +Filter           | 10 | a:Stop                                                                    |
0 |                   |
| |                 +----+---------------------------------------------------------------------------
+----------------+                    |
| +Expand(All)      | 11 | (anon_6)<-[anon_5:CALLS_AT]-(a)                                           |
0 |                   |
| |                 +----+---------------------------------------------------------------------------
+----------------+                    |
| +Filter           | 12 | anon_6.name = $autostring_1                                               |
1 |                   |
| |                 +----+---------------------------------------------------------------------------
+----------------+                    |
| +NodeByLabelScan  | 13 | anon_6:Station                                                            |
10 | Fused in Pipeline 0 |
+------------------+----+---------------------------------------------------------------------------
+----------------+--------------------+
```

The rightmost column of the plan shows that it has been divided into four different **pipelines**. In order to understand what pipelines are, it is first necessary to understand that queries using pipelined runtime are, unlike those run in slotted runtime, not executed one row at a time. Rather, the pipelined runtime allows the physical operators to consume and produce **batches** of between roughly 100 and 1000 rows each (referred to as **morsels**), which are written into **buffers** containing data and tasks for a pipeline. A pipeline can, in turn, be defined as a sequence of operators which have been fused into one another so that they may be executed together in the same task by the runtime.

The logical operators are thus not mapped to a corresponding physical operator when using the pipelined runtime. Instead, the logical operator tree is transformed into an execution graph containing pipelines and buffers:



In this execution graph, query execution starts at `pipeline 0` which will eventually produce a morsel to be written into the buffer of `pipeline 1`. Once there is data for `pipeline 1` to process, it can begin executing and in turn write data for the next pipeline to process, and so on. In this way, data is being pushed along the execution graph.

Considerations

The pipelined runtime is a push-based execution model, where data is pushed from the leaf operator to its parent operators. Unlike pull-based models (which the slotted runtime uses), data can be kept in local variables when using push-based execution models, and this has several benefits; it enables direct use of CPU registers, improves the use of CPU caches, and avoids the costly virtual function calls used in pull-based models.

The pipelined runtime is ideal for transactional use cases, with a large number of queries running in parallel

on the system. This covers most usage scenarios, and for this reason, it is the default Neo4j runtime.

The pipelined runtime is a combined model, that can either use an interpreted or compiled runtime. However, because it predominantly uses the latter, it is considered a compiled runtime. Unlike interpreted runtimes, compiled runtimes have a code generation phase followed by an execution phase, and this typically causes a longer query planning time, but a shorter execution time.

As stated above, there are rare scenarios in which users of Neo4j Enterprise Edition may benefit from not using the pipelined runtime for their queries. However, for most queries, the pipelined runtime is a more efficient runtime capable of handling all operators and queries.

Parallel runtime

Enterprise Edition

Both the slotted and pipelined runtimes execute queries in a single thread assigned to one CPU core. It is still possible to achieve parallelism (broadly defined as when two or more sets of operations can be processed concurrently within a single database environment) when using these two runtimes by running multiple queries in separate CPU threads concurrently (this is the typical scenario in OLTP (Online Transaction Processing) use cases). Another alternative is to run multiple transactions concurrently within the same query using `CALL {…} IN CONCURRENT TRANSACTIONS`.

However, there are scenarios, principally when performing graph analytics, where it is beneficial for a single query to use several cores to boost its performance. This can be achieved by using parallel runtime, which is multi-threaded and allows queries to potentially utilize all available cores on the server running Neo4j.

To specify that a query should use the parallel runtime, prepend it with `CYPHER runtime = parallel`. For example:

Query

```
EXPLAIN
CYPHER runtime = parallel
MATCH (:Station { name: 'Denmark Hill' })<-[:CALLS_AT]-(d:Stop)
      ((:Stop)-[:NEXT]->(:Stop))+
      (a:Stop)-[:CALLS_AT]->(:Station { name: 'Clapham Junction' })
RETURN count(*)
```

This is the resulting execution plan:

```
Planner COST

Runtime PARALLEL

Runtime version 2026.10

Batch size 128

+-----------------------------+----
+-------------------------------------------------------------------------+---------------
+---------------------+
| Operator                    | Id | Details
| Estimated Rows | Pipeline           |
+-----------------------------+----
+-------------------------------------------------------------------------+---------------
+---------------------+
```

```
| +ProduceResults            |  0 | `count(*)`
|              1 | In Pipeline 6       |
| |                           +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| +EagerAggregation          |  1 | count(*) AS `count(*)`
|              1 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +Filter                    |  2 | NOT anon_1 = anon_5 AND anon_0.name = $autostring_0 AND
anon_0:Station |              0 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +Expand(All)               |  3 | (d)-[anon_1:CALLS_AT]->(anon_0)
|              0 | Fused in Pipeline 5 |
| |                           +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| +Filter                    |  4 | d:Stop
|              0 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +NullifyMetadata           | 14 |
|              0 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +Repeat(Trail)             |  5 | (a) (...){1, *} (d)
|              0 | Fused in Pipeline 4 |
| |\                          +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| | +Filter                  |  6 | isRepeatTrailUnique(anon_8) AND anon_7:Stop
|              6 |                     |
| | |                         +----
+-------------------------------------------------------------------------+----------------+
|
| | +Expand(All)             |  7 | (anon_9)<-[anon_8:NEXT]-(anon_7)
|              6 | Fused in Pipeline 3 |
| | |                         +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| | +Filter                  |  8 | anon_9:Stop
|             11 |                     |
| | |                         +----
+-------------------------------------------------------------------------+----------------+
|
| | +Argument                |  9 | anon_9
|             13 | Fused in Pipeline 2 |
| |                           +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| +Filter                    | 10 | a:Stop
|              0 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +Expand(All)               | 11 | (anon_6)<-[anon_5:CALLS_AT]-(a)
|              0 | Fused in Pipeline 1 |
| |                           +----
+-------------------------------------------------------------------------+----------------
+--------------------+
| +Filter                    | 12 | anon_6.name = $autostring_1
|              1 |                     |
| |                           +----
+-------------------------------------------------------------------------+----------------+
|
| +PartitionedNodeByLabelScan | 13 | anon_6:Station
|             10 | Fused in Pipeline 0 |
+-----------------------------+----
+-------------------------------------------------------------------------+----------------
+--------------------+
```

A key difference between the physical plans produced by the parallel runtime compared to those

generated by pipelined runtime is that, in general, more pipelines are produced when using the parallel runtime (in this case, seven instead of the four produced by the same query being run on pipelined runtime). This is because, when executing a query in the parallel runtime, it is more efficient to have more tasks that can be run in parallel, whereas when running a single-threaded execution in the pipelined runtime it is more efficient to fuse several pipelines together.

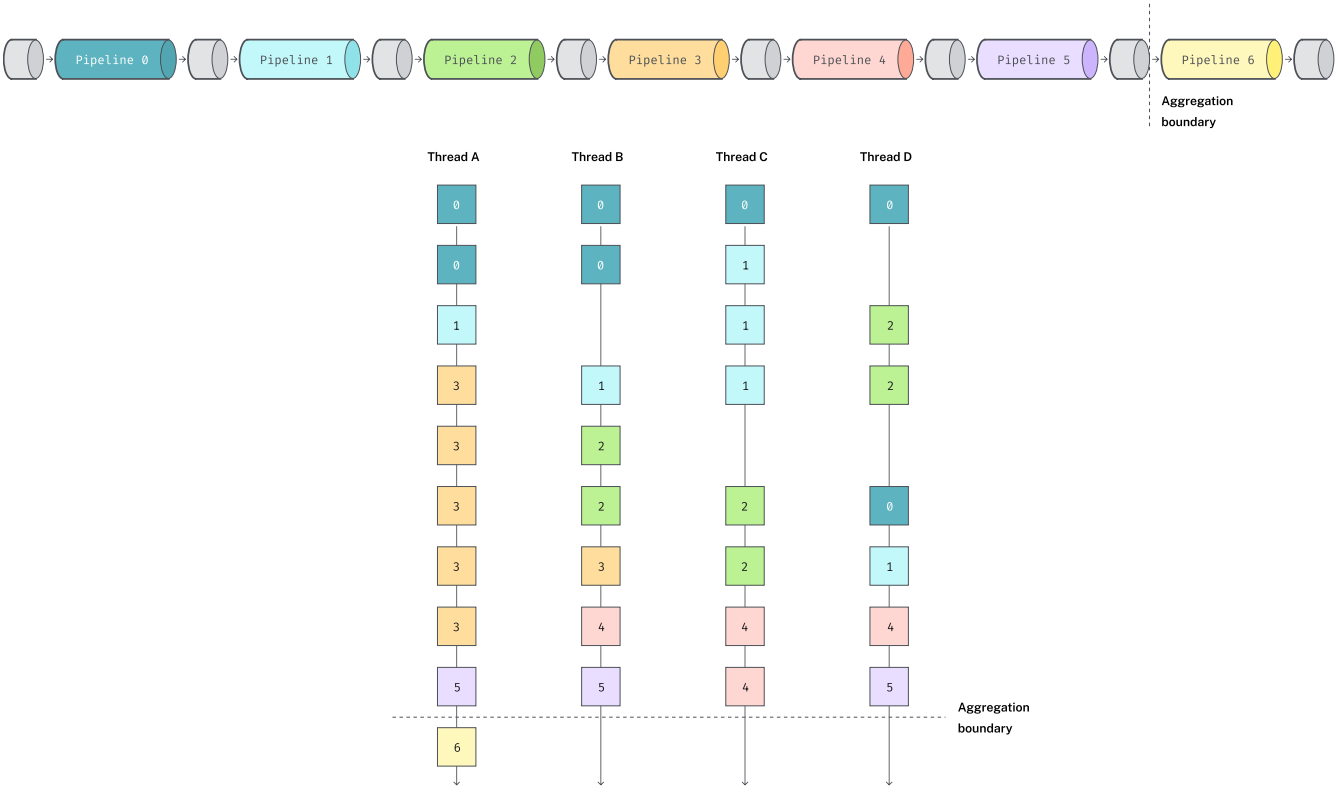
Another important difference is that the parallel runtime uses partitioned operators (`PartitionedNodeByLabelScan` in this case). These operators first segment the retrieved data and then operate on each segment in parallel.

The parallel runtime shares the same architecture as the pipelined runtime, meaning that it will transform the logical plan into the same type of execution graph as described above. However, when using parallel runtime, each pipeline task can be executed in a separate thread. Another similarity with pipelined runtime is that queries run on the parallel runtime will begin by generating the first pipeline which eventually will produce a morsel in the input buffer of the subsequent pipeline. But, whereas only one pipeline can progress at a time when using the pipelined runtime, parallel runtime allows pipelines to concurrently produce morsels. Therefore, as each task finishes, more and more input morsels will be made available for the tasks which means that more and more workers can be utilized to execute the query.

To further explain how parallel runtime works, a set of new terms need to be defined:

- **Worker**: a thread that executes work units to evaluate incoming queries.
- **Task**: a unit of work. A task executes one pipeline on one input morsel and produces one output morsel. If any condition prevents a task from completing, it can be rescheduled as a Continuation to resume at a later time.
- **Continuation**: a task that did not finish execution and must be scheduled again.
- **Scheduler**: responsible for deciding which unit of work to process next. Scheduling is decentralized, and each worker has its own scheduler instance.

Consider the execution graph below, based on the same example query:



The execution graph shows that execution starts at `pipeline 0`, which consists of the operator `PartitionedNodeByLabelScan` and can be executed simultaneously on all available threads working on different morsels of data. Once pipeline `0` has produced at least one full morsel of data, any thread can then start executing `pipeline 1`, while other threads may continue to execute `pipeline 0`. More specifically, once there is data from a pipeline, the scheduler can proceed to the next pipeline while concurrently executing earlier pipelines. In this case, `pipeline 5` ends with an aggregation (performed by the `EagerAggregation` operator), which means that the last pipeline (`6`) cannot start until all preceding pipelines are completely finished for all the preceding morsels of data.

Considerations

When to use the parallel runtime

In most situations where multiple CPU cores are available, long-running queries can be expected to run significantly faster on the parallel runtime. While it is not possible to define the exact duration at which a query would benefit from being run on the parallel runtime (as this depends on the data model, the query structure, the load of the system, and the number of cores available), it can be assumed as a general rule that any query that takes longer than approximately 500 milliseconds would be a good candidate.

This means that the parallel runtime is suitable for **analytical**, **graph-global** queries. These queries are often not anchored to a particular start node and therefore process a large section of the graph in order to gain valuable insights from it.

However, queries that start with anchoring a specific node may benefit from being run on the parallel runtime, if either of the following is true:

- The anchored starting node is a densely connected node or super node.
- The query proceeds to expand from the anchored node to a large section of the graph.

There is, therefore, no fixed rule as to when a query should be run with the parallel runtime, but these guidelines provide some useful information about the scenarios when users would very likely benefit from trying to use it.

When not to use the parallel runtime

Unlike the pipelined runtime, which was designed as the most efficient method for most queries to be planned, the use cases for the parallel runtime are more specific, and there are situations where it is not possible or beneficial to use it. Most notably, the parallel runtime **only supports read queries**. It also does not support procedures functions that are not considered thread-safe (i.e. not safe to run from multiple threads).

Moreover, not all queries will run faster by using the parallel runtime. For example, a **graph-local** query that starts with anchoring a node and proceeds to only match a small portion of the graph will probably not run any faster on the parallel runtime (it may even run slower when executed with the parallel runtime, because of its scheduling and the additional book-keeping required for executing a query on multiple threads). As a general rule of thumb, the parallel runtime is probably not beneficial for queries which take less than half a second to complete.

The parallel runtime may also perform worse than the pipelined runtime for queries that contain subclauses where ORDER BY is used to order a property that is indexed. This is because the parallel runtime is unable to take advantage of property indexes for ordering, and therefore must re-sort the aggregated results on the selected properties before returning any results.

Finally, though individual queries may run faster when running the parallel runtime, the overall throughput of the database may decrease as a result of running many concurrent queries.

The parallel runtime is accordingly not suitable for transactional processing queries with high throughput workloads. It is, however, ideal for analytical use cases where the database runs relatively few, but demanding read queries.

Overview

In general, the parallel runtime should be considered if the following conditions are met:

- Graph-global read-queries are constructed to target a large section of a graph.
- The speed of queries is important.
- The server has many CPUs and enough memory.
- There is a low concurrency workload on the database.

For more information about the parallel runtime, including more details about queries, procedures, functions, configuration settings, and using the parallel runtime on Aura, see the Parallel runtime: reference page.

Summary

The below table summarizes the most important distinctions between the three different runtimes available in Cypher:

	Slotted	Pipelined	Parallel
Execution model	Pull	Push	Push
Physical operator consumption	Row-by-row	Batched	Batched
Processor threads	Single-threaded	Single-threaded	Multi-threaded
Runtime type	Interpreted	Compiled or interpreted	Compiled or interpreted
Supported query type	Read and write	Read and write	Read only

Further reading

- Neo4j Developer Blog: Speed Speed Up Your Queries With Neo4j's New Parallel Runtime

Parallel runtime: reference

Enterprise Edition

The parallel runtime behaves differently compared to the slotted and pipelined runtimes in several regards. This page explains the relevant configuration settings for the parallel runtime and the scenarios in which it is either not supported or considered thread-safe. It also includes relevant information for Neo4j Aura users.

Readers not familiar with the parallel runtime are encouraged to read about the parallel runtime concepts before reading this page.

Updating queries

The parallel runtime will throw an error if a query attempts to update the graph. For example, any query run on the parallel runtime which uses the `CREATE` clause, will throw the following error:

```
1 CYPHER runtime=parallel
2 CREATE (:Person)
```

GQLSTATUS error chain

22N46: error: data exception - unsupported use of parallel runtime. Parallel runtime does not support updating queries or a change in the state of transactions. Use another runtime.

22000: error: data exception

For a full list of all available Cypher write clauses, see the Clauses overview page.

Transactions

It is not possible to use the parallel runtime if a change has been made to the state of a transaction.

For example, the following transaction (initiated on Cypher Shell) will be rolled back, because executing a Cypher query will make changes to the state of a transaction.

Step 1: Initiate a new transaction and change its state by creating a node

```
:begin
CREATE (n:Person)
```

Step 2: Attempt to execute a Cypher query with the parallel runtime on the existing transaction

```
1 CYPHER runtime = parallel
2 RETURN 42
```

GQLSTATUS error chain

22N46: error: data exception - unsupported use of parallel runtime. Parallel runtime does not support updating queries or a change in the state of transactions. Use another runtime.

22000: error: data exception

For more information about transactions in Neo4j, see the Operations Manual → Transaction management.

Configuration settings

The following setting can be configured to modify the behavior of the parallel runtime:

server.cypher.parallel.worker_limit

Description	Number of threads to allocate to Cypher worker threads for the parallel runtime. If set to a positive number, that number of workers will be started. If set to `0`, one worker will be started for every logical processor available to the Java Virtual Machine. If set to a negative number we will subtract the value from the number of logical processors available; for example, say Neo4j is running on a server with 16 available processors, using `server.cypher.parallel.worker_limit = -1` would then mean that 15 threads are available for the parallel runtime.
Valid values	Integer
Default value	`0`

Setting `server.cypher.parallel.worker_limit` to a negative number `-n` where `n` is greater than the total number of cores will disable the parallel runtime.

For more information about configuration settings in Neo4j, see the Operations Manual → Configuration.

Aura

The parallel runtime is available on all non-free AuraDB instances, regardless of their size or CPU count. Additionally, when a query is run with parallel runtime on an Aura instance, it can utilize up to the total number of available CPUs.

The parallel runtime is disabled on AuraDB Free instances. Attempting to run a query with parallel runtime

on AuraDB Free will throw the following error message:

Error message

```
Parallel runtime has been disabled, please enable it or upgrade to a bigger Aura instance.
```

> **Note:**
>
> Users of AuraDB Professional, AuraDB Business Critical, and AuraDB Virtual Dedicated Cloud select the the size and the number of available CPUs when creating an instance. More information about the various tiers of AuraDB can be found on the Neo4j Pricing page.

Procedures and functions

Procedures and functions that read the database are supported by the parallel runtime. Apart from this, there are two categories of procedures and functions to keep in mind when using the parallel runtime.

The first can be categorized as *updating procedures*. These are procedures that update the graph with write queries, such as the Neo4j procedures `db.createProperty()`. If such procedures are called in a query run on the parallel runtime, the query will fail.

The second can be categorized as *non-thread-safe* procedures and functions. These are procedures and functions which perform tasks that are not protected against multiple worker threads concurrently interacting with the targeted data. This includes procedures and functions that perform either of the following tasks:

- Executes a Cypher query (because that will start a new transaction, which is not supported by the parallel runtime).
- Starts a new transaction (because this is not supported by the parallel runtime).

Calling procedures which perform any of these tasks in a query run on the parallel runtime will not fail the query. Instead the query will automatically run on the pipelined runtime.

Neo4j procedures

The following Neo4j procedures are not considered thread-safe and cannot be run on the parallel runtime. Trying to call them in a query run on the parallel runtime will not fail the query. Instead the query will automatically run on the pipelined runtime.

Non-thread-safe Neo4j procedures

Procedure
`db.awaitIndex()`
`db.awaitIndexes()`
`db.checkpoint()`
`db.info()`

Procedure
`db.labels()`
`db.listLocks()`
`db.ping()`
`db.propertyKeys()`
`db.prepareForReplanning()`
`db.relationshipTypes()`
`db.resampleIndex()`
`db.resampleOutdatedIndexes()`
`db.schema.nodeTypeProperties()`
`db.schema.relTypeProperties()`
`db.schema.visualization()`
`dbms.checkConfigValue()`
`dbms.listActiveLocks()`
`dbms.listPools()`
`dbms.scheduler.failedJobs()`
`dbms.scheduler.groups()`
`dbms.scheduler.jobs()`

APOC

The APOC library contains procedures and functions which extend the use of Cypher. There are a number of APOC procedures and functions that are not considered thread-safe, and **cannot** be run on the parallel runtime. For information about these, refer to the pages of the individual procedures and functions in the APOC Manual.

User-defined functions

User-defined functions are simpler forms of procedures that return a single value and are read-only. To learn more about user-defined functions in Neo4j, see the Java Reference Manual → User-defined functions.

Similar to Neo4j and APOC procedures, any user-defined function that starts a new transaction by executing a Cypher query is not considered thread-safe and will not be supported by the parallel runtime (this includes all user-defined aggregating functions).

For example, consider the two following user-defined functions:

```
class MyFunctions {
  @Context
  public Transaction transaction;
```

```
4
5   @UserFunction("examples.return42")
6   public long return42() {
7     return 42L;
8   }
9
10  @UserFunction("examples.return42ViaCypher")
11  public long return42ViaCypher() {
12    return (long) transaction.execute("RETURN 42 AS res").next().get("n");
13  }
14 }
```

Running `examples.return42()` will succeed with the parallel runtime, whereas `examples.return42ViaCypher()` will fail because executing a new Cypher query will start a new transaction.

However, if `@NotThreadSafe` is added to the method, then the query will automatically not run on the parallel runtime. The query will instead default to the single-threaded pipelined runtime and generate a notification.

The user-defined function below does not fail with the parallel runtime. Instead, the Cypher query gets automatically run on the pipelined runtime.

```
1 class MyFunctions {
2   @Context
3   public Transaction transaction;
4
5   @UserFunction("examples.return42ViaCypher")
6   @NotThreadSafe
7   public long return42ViaCypher() {
8     return (long) transaction.execute("RETURN 42 AS res").next().get("n");
9   }
10 }
```

Query tuning

Neo4j aims to execute queries as fast as possible. However, when optimizing for maximum query execution performance, it may be helpful to rephrase queries using knowledge about the domain and the application.

This page contains information about how to tune queries using different strategies.

For information about changing the runtime of a query, see the page about Cypher runtime concepts.

General recommendations

The overall goal of manual query performance optimization is to ensure that only necessary data is retrieved from the graph.

Queries should aim to filter data as early as possible in order to reduce the amount of work that has to be done in the later stages of query execution. This also applies to what gets returned: returning whole nodes and relationships ought to be avoided in favour of selecting and returning only the data that is needed. You should also make sure to set an upper limit on variable-length patterns, so they don't cover larger portions of the dataset than needed.

Each Cypher query gets optimized and transformed into an execution plan by the Cypher query planner. To minimize the resources used for this, try to use parameters instead of literals when possible. This allows Cypher to re-use queries instead of having to parse and build new execution plans.

To read more about the execution plan operators mentioned in this section, see Operators.

Query options

Query execution can be fine-tuned through the use of query options.

In order to use one or more of these options, the query must be prepended with `CYPHER`, followed by the query option(s), as exemplified thus:

```
CYPHER query-option [further-query-options] query
```

For information about the various runtimes available in Cypher, see Cypher runtimes.

Cypher planner

The Cypher planner takes a Cypher query and computes an execution plan that solves it. For any given query there is likely a number of execution plan candidates that each solve the query in a different way. The planner uses a search algorithm to find the execution plan with the lowest estimated execution cost.

This table describes the available planner options:

Query option	Description	Default
`planner=cost`	Use cost based planning with default limits on plan search space and time.	✔
`planner=idp`	Synonym for `planner=cost`.	
`planner=dp`	Use cost based planning without limits on plan search space and time to perform an exhaustive search for the best execution plan. **Note:** Using this option can significantly *increase* the planning time of the query.	

Cypher connect-components planner

Deprecated

One part of the Cypher planner is responsible for combining sub-plans for separate patterns into larger plans - a task referred to as *connecting components*.

This table describes the available query options for the connect-components planner:

Query option	Description	Default
`connectComponentsPlanner=greedy`	Use a greedy approach when combining sub-plans. **Note:** Using this option can significantly *reduce* the planning time of the query.	
`connectComponentsPlanner=idp`	Use the cost based IDP search algorithm when combining sub-plans. **Note:** Using this option can significantly *increase* the planning time of the query but usually finds better plans.	✔

Note:

The Cypher query option `connectComponentsPlanner` is deprecated and will be removed without a replacement. The product's default behavior of using a cost-based IDP search algorithm when combining sub-plans will be kept.

Cypher update strategy

This option affects the eagerness of updating queries.

The possible values are:

Query option	Description	Default
`updateStrategy=default`	Update queries are executed eagerly when needed.	✔
`updateStrategy=eager`	Update queries are always executed eagerly.	

Cypher expression engine

This option affects how the runtime evaluates expressions.

The possible values are:

Query option	Description	Default
`expressionEngine=default`	Compile expressions and use the compiled expression engine when needed.	✔
`expressionEngine=interpreted`	Always use the *interpreted* expression engine.	
`expressionEngine=compiled`	Always compile expressions and use the *compiled* expression engine.	

Cypher operator engine

This query option affects whether the pipelined runtime attempts to generate compiled code for groups of operators.

The possible values are:

Query option	Description	Default
`operatorEngine=default`	Attempt to generate compiled operators when applicable.	✔
`operatorEngine=interpreted`	Never attempt to generate compiled operators.	
`operatorEngine=compiled`	Always attempt to generate *compiled* operators. Cannot be used together with `runtime=slotted`.	

Cypher interpreted pipes fallback

This query option affects how the pipelined runtime behaves for operators it does not directly support.

The available options are:

Query option	Description	Default
`interpretedPipesFallback=default`	Equivalent to `interpretedPipesFallback=whitelisted_plans_only`.	✔
`interpretedPipesFallback=disabled`	If the plan contains any operators not supported by the pipelined runtime then another runtime is chosen to execute the entire plan. Cannot be used together with `runtime=slotted`.	
`interpretedPipesFallback=whitelisted_plans_only`	Parts of the execution plan can be executed on another runtime. Only certain operators are allowed to execute on another runtime. Cannot be used together with `runtime=slotted`.	
`interpretedPipesFallback=all`	Parts of the execution plan may be executed on another runtime. Any operator is allowed to execute on another runtime. Queries with this option set might produce incorrect results, or fail. Cannot be used together with or `runtime=slotted`. **Warning:** This setting is experimental, and using it in a production environment is discouraged.	

Cypher replanning

Translating a query string into an efficient execution plan can be an expensive operation. Once an execution plan is obtained for a query, it is placed in a cache. If the exact same query is to be executed again, the planning step is skipped. Instead, the execution plan is obtained from the cache.

"Replanning" refers to cases where a query must be planned again, even though it has been planned before. Cypher replanning occurs in the following circumstances:

- When the query is not in the cache. This can either be when the server is first started or restarted, if the cache has recently been cleared, or if `server.memory.query_cache.per_db_cache_num_entries` was exceeded.
- When the time has passed the `dbms.cypher.statistics_divergence_threshold` value.
- When the cached query plan has notifications that have become invalid. Consider that a query plan had the notification `01N50: Label does not exist`. After adding a node with the label that did not exist before and running the same query again, replanning the query is required.

There may be situations where Cypher query planning can occur at a non-ideal time. For example, when a query must be as fast as possible and a valid plan is already in place.

> **Note:**
>
> Replanning is not performed for all queries at once; it is performed in the same thread as running the query, and can block the query. However, replanning one query does not replan any other queries.

There are three different replan options available:

Option	Description	Default
`replan=default`	This is the planning and replanning option as described above.	✔
`replan=force`	This will force a replan, even if the plan is valid according to the planning rules. Once the new plan is complete, it replaces the existing one in the query cache.	
`replan=skip`	If a valid plan already exists, it will be used even if the planning rules would normally dictate that it should be replanned.	

The replan option is prepended to queries.

For example:

```
CYPHER replan=force MATCH ...
```

In a mixed workload, you can force replanning by using the Cypher `EXPLAIN` commands. This can be useful to schedule replanning of queries which are expensive to plan, at known times of low load. Using `EXPLAIN` will make sure the query is only planned, but not executed.

For example:

```
CYPHER replan=force EXPLAIN MATCH ...
```

During times of known high load, `replan=skip` can be useful to not introduce unwanted latency spikes.

When a schema change is committed while a query is being planned, replanning occurs for the query that is being planned. For example, dropping an index is a schema change. The schema change can make the obtained execution plan invalid or inefficient. Instead of continuing with the obtained execution plan, the query will be planned again. The Cypher option `replan` does not have any effect on replanning due to schema changes.

Cypher infer schema parts

For some queries, the planner can infer predicates such as labels or types from the graph structure, thereby enhancing its ability to estimate the number of rows each operator will produce. (See Understanding query plans for more information about the role of operators and estimated row counts in query execution plans.) The option `inferSchemaParts` controls the extent to which the planner should infer predicates.

Option	Description
`inferSchemaParts=off`	No predicates are inferred.
`inferSchemaParts=most_selective_label`	Relationship types are used to infer labels on connected nodes. The label corresponding to the smallest number of nodes is used to estimate rows. Avoiding the inference of multiple labels improves accuracy for nodes with several dependent labels, such as every `:Actor` being a `:Person`.

If this query option is not provided, then the value set in Operations Manual → Configuration settings → dbms.cypher.infer_schema_parts will be used.

Query caches

Translating a Cypher query string into an executable query plan can be an expensive operation. To avoid repeating this work, Neo4j caches query plans when possible so that when the same query is submitted again, the planning step can be skipped and the cached plan reused directly.

By default, the set of query caches is **per database**. That means that a new set of caches is initialized for each new database.

How the query cache works

Neo4j caches a query at several stages:

- Raw query string → pre-parsed query string + query options, such as `CYPHER key=value`, `EXPLAIN`, and `PROFILE` (pre-parsing)
- Query string → abstract syntax tree (AST) (parsing)
- AST → logical plan (logical planning, reported in the JSON query log as `logicalPlanCacheHit`)
- Logical plan → physical execution plan (execution planning)

- Raw query string → physical execution plan (executable query cache, reported in JSON query log as `executableQueryCacheHit`)

> Logging `logicalPlanCacheHit` and `executableQueryCacheHit` in the JSON query log requires setting `internal.dbms.logs.query.query_cache_usage=true`. This is enabled by default on Neo4j Aura, but must be set explicitly for on-premises deployments.

The fastest path is the last one: if `executableQueryCacheHit` is true, the exact same query string (and parameter types) has been seen before, and the previously computed plan is reused directly, with no reparsing or replanning.

If the query string is new, `executableQueryCacheHit` is `false`, and Neo4j falls back through the earlier stages (parsing, then logical planning) each of which can still be reused if that particular stage's input has been seen before, even if the overall query text is new. This is why two differently-worded but structurally equivalent queries can still hit the logical plan cache and avoid some replanning work, without getting the full benefit of an exact executable-query-cache hit.

Logical planning is typically the most expensive of these stages, so its outcome tends to have the largest impact on planning time. Parsing can also take meaningful time, but since it depends on the exact query text, it sometimes cannot be avoided when the query string itself is new.

The same query run twice

```
+-------------------------------------------+-----------+-----------+----------+
| Query                                     | Hash      | Cache Hit | Planning |
+-------------------------------------------+-----------+-----------+----------+
| CREATE (:Person {id: 789, name: 'John'});  | 18270B26  | false     | 25 ms    |
| MATCH (n:Person) WHERE n.id = 789 RETURN n; | D4DE3B34  | false     | 22 ms    |
| MATCH (n:Person) WHERE n.id = 789 RETURN n; | D4DE3B34  | true      | 0 ms     |
+-------------------------------------------+-----------+-----------+----------+
```

The two identical `MATCH` queries have identical query strings, so the second hits the executable query cache and skips planning entirely (0ms). Both happen to share the same `executionPlanCacheKeyHash`, confirming they resolved to the same plan, but it is the identical query string, not the hash, that determines the hit.

Two textually different, structurally equivalent queries

```
+-------------------------------------------+-----------+-----------+----------+
| Query                                     | Hash      | Cache Hit | Planning |
+-------------------------------------------+-----------+-----------+----------+
| MATCH (n:Person) WHERE n.id = 789 RETURN n; | BDAF71B3  | true      | 1 ms     |
| MATCH (n:Person {id: 789}) RETURN n;      | BDAF71B3  | false     | 63 ms    |
+-------------------------------------------+-----------+-----------+----------+
```

These two queries have different query strings, so the second misses the executable query cache regardless of the first query's result, and pays real planning time. Both end up with the same `executionPlanCacheKeyHash`, showing that despite the miss, they resolved to the same execution plan (this is only confirmed after planning, not something used to skip it).

> For full information about the metrics on query cache usage, refer to the Operations Manual → Cypher metrics.

Parameter types and the executable query cache

The executable query cache's key includes the type of each parameter. Two executions of the same query text with a differently-typed parameter (for example, an `INTEGER` in one call and a `STRING` in another) are treated as separate cache entries.

Internally, each parameter is represented as a `ParameterTypeInfo`, capturing both its type and a size category. For example, a `STRING` is represented with its type and a size bucket, e.g. `ParameterTypeInfo(String, ExactSize(1))` for a single-character string.

Same query text, different parameter types

```
+-------------------------------------------+-----------------+---------+-----------+-----------
+----------+
| Query                                     | Parameter       | Type    | Hash      | Cache Hit |
Planning|
+-------------------------------------------+-----------------+---------+-----------+-----------
+----------+
| MATCH (n:Person) WHERE n.id = $id RETURN n;   | id: 789         | INTEGER | 855E6DCD  | false     | 56
ms   |
| MATCH (n:Person) WHERE n.id = $id RETURN n;   | id: '789'       | STRING  | 855E6DCD  | false     | 30
ms   |
| MATCH (n:Person) WHERE n.id = $id RETURN n;   | id: null        | ANY     | 855E6DCD  | false     | 23
ms   |
| MATCH (n:Person) WHERE n.id = $id RETURN n;   | id: 999         | INTEGER | 855E6DCD  | true      | 0
ms    |
+-------------------------------------------+-----------------+---------+-----------+-----------
+----------+
```

Note:

- `null` is treated as `ANY`, so it does not reuse the cache entry created by the first query with an integer parameter.
- All four rows share the same `executionPlanCacheKeyHash`, since the resulting plan is identical regardless of the parameter's type or value. This confirms the four executions resolved to the same plan, but the hash is not what determines the hit, the executable query cache key (query text + parameter types) is.

Each distinct `ParameterTypeInfo` gets its own executable query cache entry. The fourth query hits the entry created by the first, since both use an `INTEGER` parameter; the `STRING` and `null` parameters in between each created their own separate entries and did not interfere with this reuse.

Therefore, if an application occasionally sends `null` for a parameter and a concrete value at other times, this results in at least two cache entries for what may otherwise look like a single, unchanging query.

Caching and schema changes

Creating an index or a constraint clears the executable query cache, the logical plan cache, and the execution plan cache for the database, forcing every query to go through logical planning and execution planning again on its next execution.

This means `executionPlanCacheKeyHash` is recomputed after a schema change, even for queries unrelated to it. The hash is tied to the logical plan produced for the query: if logical planning yields the same plan as before, the recomputed hash will be identical, so an unchanged hash does not necessarily mean the cache

entry survived untouched (it may simply mean planning produced the same result again).

Comparing two unrelated queries before and after an index is created on one of them

```
+------+-------------------------------------------------+-----------+-----------+----------+
| Step | Query / Action                                  | Hash      | Cache Hit | Planning |
+------+-------------------------------------------------+-----------+-----------+----------+
| 1    | MATCH (n:A) WHERE n.val = 1 RETURN n            | A746F892  | false     | 31 ms    |
| 2    | MATCH (n:B) WHERE n.val = 1 RETURN n            | CC8AAF69  | false     | 20 ms    |
| 3    | CREATE INDEX idx_a_val FOR (n:A) ON (n.val);    |           |           | 10 ms    |
| 4    | CALL db.awaitIndexes();                         |           |           | 12 ms    |
| 5    | MATCH (n:A) WHERE n.val = 1 RETURN n            | 4505044A  | false     | 20 ms    |
| 6    | MATCH (n:B) WHERE n.val = 1 RETURN n            | CC8AAF69  | false     | 17 ms    |
+------+-------------------------------------------------+-----------+-----------+----------+
```

Both queries show `executableQueryCacheHit: false` after the index creation (step 5 and step 6), even though `:B` is completely unaffected by the new index. This is because the index update clears the executable query cache broadly, so neither query can take the fast, top-level shortcut on its next run.

However, the two queries diverge after that point. For `:A`, the new index gives the planner a better access path, so logical planning produces a **different** plan, and its `executionPlanCacheKeyHash` changes (step 1 to step 5). For `:B`, logical planning is redone but produces the **same** plan as before, since nothing about the new index is relevant to it. As a result, its recomputed hash is identical to before (step 2 equals step 6); not because the execution plan cache entry survived the schema change, but because planning it again produced the same result.

In the case of `CREATE INDEX`, the cache is cleared when the index comes online, but with `CREATE CONSTRAINT` the cache is cleared immediately.

Notifications when a cached plan is not used

Introduced in Neo4j 2026.04

Neo4j writes a message to `debug.log` whenever a cached plan is not used for a query, along with a brief explanation of why. This requires no configuration and provides a way to notice unexpected replanning without inspecting query log internals.

Example message, captured after repeated executions of the same query:

```
Cached query plan is not used because recompilation with expression code generation is triggered.
```

This particular trigger occurs when a query has been executed enough times that Neo4j recompiles it with more aggressive expression code generation, replacing the previously cached plan.

Configuring query caches

By default, the set of query caches is **per database**, and their memory is allocated on the JVM heap. For more information, see Operations Manual → Memory configuration.

Query cache memory use is usually small compared to the total heap, but it can become noticeable under specific conditions, such as multi-tenant environments where many active databases run workloads of

programmatically generated queries without proper parameterization. In such scenarios, the cache stores many unique query plans. Whether this cache usage causes problems depends a lot on the total heap size, with small systems with limited heap being more affected than larger deployments. Note that running queries usually uses much more memory because heap usage can grow with the amount of data. To improve memory predictability in multi-tenant cases where many databases share an identical schema and run the same queries, you can configure the DBMS to use one shared set of query caches for all databases. For more information, see Unifying query caches.

The following is a summary of the query cache configurations:

Query cache configurations

Setting	Description	Default
`server.memory.query_cache.shared_cache_num_entries`	The number of cached queries for all databases. This setting is only deciding cache size when `server.memory.query_cache.sharing_enabled` is set to `true`. Enterprise Edition	`1000`
`server.memory.query_cache.per_db_cache_num_entries`	The number of cached queries per database. This setting is only deciding cache size when `server.memory.query_cache.sharing_enabled` is set to `false`.	`1000`

Unifying query caches

To enable the unified query caches, set the option `server.memory.query_cache.sharing_enabled=true`.

Unified query cache configurations

Setting	Description	Default
`server.memory.query_cache.sharing_enabled`	Enable sharing cache space between different databases. With this option turned on, databases will share cache space, but not cache entries. Enterprise Edition Schema-change cache flushes are always scoped to the database where the change occurred, even with sharing enabled; sharing affects available cache **space**, not cache **entries**.	`false`
`server.memory.query_cache.shared_cache_num_entries`	The number of cached queries for all databases. This setting is only deciding cache size when `server.memory.query_cache.sharing_enabled` is set to `true`. Enterprise Edition	`1000`

When this feature is enabled, all databases use only one set of query caches. A database may store and retrieve entries from the shared cache, but it may not retrieve entries produced by another database.

> While databases use the same set of caches, a database may not observe entries

originating from other databases.

The database may, however, evict entries from other databases as necessary, according to the constrained cache size and cache eviction policy. In essence, databases may compete for cache space, but may not observe each other's entries.

When this option is turned on, the cache space available to all databases is configured using the setting `server.memory.query_cache.shared_cache_num_entries`.

Query size limit

Introduced in 2026.01

The default query size limit for a query to be considered for query caching is 128 KiB of query text.

This limit prevents large generated query text strings from occupying too much memory in the query cache. Such query strings often contain inlined data, and are unlikely to be reusable before auto-parameterization of literals is applied.

To circumvent the default of 128 KiB, you can prefix a query with either `CYPHER cache=force` to always cache it, or with `CYPHER cache=skip` to never cache it and skip the query cache lookup.

As a best practice, avoid passing data in the query text string and use parameters instead.

[1] The relevant information about the current state of the database includes which indexes and constraints are available, as well as various statistics maintained by the database. The Cypher planner uses this information to determine which access patterns will produce the best execution plan.
[2] The classification of a runtime as interpreted or compiled is not entirely accurate. Most runtime implementations are not fully interpreted or fully compiled but are rather a blend of the two styles. For example, when the slotted runtime is run in Neo4j Enterprise Edition, code is generated for the expressions included in the query. Nevertheless, the slotted runtime is considered interpreted, since that is the predominant method of implementation.

Administration

The pages previously in this chapter have been moved to the Operations Manual.

More specific information about the content relocation is listed in the table:

Content	New location in Operations Manual
Database management	Database administration
Alias management	Managing aliases
Server management	Managing servers in a cluster
Access control	Authentication and authorization

Syntax

Cypher follows a several syntactical rules and recommendations that are important to know when constructing queries. Further information can be found in the following sections:

- Parsing
- Naming rules and recommendations
- Variables
- Keywords
- Parameters
- Comments

Parsing

This page provides a general overview of how Cypher parses an input `STRING`.

The Cypher parser takes an arbitrary input `STRING`. This page details the general rules on which characters are considered valid input.

Using unicodes in Cypher

Unicodes can generally be escaped as `\uxxx`. For example, the below query uses the Unicode `u00B0` to search for any recipe descriptions containing the degree symbol, `º`:

Using Unicodes in `STRING` *matching*

```
MATCH (r:Recipe)
WHERE r.description CONTAINS "\u00B0"
RETURN r
```

Additional documentation on escaping rules for `STRING` literals, names and regular expressions can be found here:

- String literal escape sequences
- Using special characters in names
- Regular expressions

The Unicode version used by Cypher depends on the running JVM version.

Neo4j version	JVM compliancy	Unicode version
3.x	Java SE 8 Platform Specification	Unicode 6.2
4.x	Java SE 11 Platform Specification	Unicode 10.0
5.x	Java SE 17 Platform Specification	Unicode 13.0

Neo4j version	JVM compliancy	Unicode version
5.14	Java SE 17 and Java SE 21 Platform Specification	Unicode 13.0 and Unicode 15.0

Supported whitespace

Whitespace can be used as a separator between keywords and has no semantic meaning. The following unicode characters are considered as whitespace:

Description	List of included Unicode characters
Unicode general category Zp	`\u2029`
Unicode general category Zs	`\u0020` (space), `\u1680`, `\u2000-200A`, `\u202F`, `\u205F`, `\u3000`
Unicode general category class Zl	`\u2028`
Horizontal tabulation	`\t`, `\u0009`
Line feed	`\n`, `\u000A`
Vertical tabulation	`\u000B`
Form feed	`\f`, `\u000C`
Carriage return	`\r`, `\u000D`
File separator	`\u001C`
Group separator	`\u001D`
Record separator	`\u001E`
Unit separator	`\u001F`
Next Line	`\u0085`

It is possible to have multiple whitespace characters in a row, and will have the same effect as using a single whitespace.

The following example query uses vertical tabulation (`\u000B`) as whitespace between the `RETURN` keyword and the variable `m`:

```
MATCH (m) RETURN\u000Bm;
```

Supported newline characters

A newline character identifies a new line in the query and is also considered whitespace. The supported newline characters in Cypher are:

Description	List of included Unicode characters
Line feed	`\n`, `\u000A`
Carriage return	`\r`, `\u000D`
Carriage return + line feed	`\r\n`, `\u000D\u000A`

Naming rules and recommendations

This page describes rules and recommendations for the naming of node labels, relationship types, property names, variables, indexes, and constraints.

Naming rules

- Alphabetic characters:
  - Names should begin with an alphabetic character.
  - This includes "non-English" characters, such as `å`, `ä`, `ö`, `ü` etc.
- Numbers:
  - Names should not begin with a number.
  - To illustrate, `1first` is not allowed, whereas `first1` is allowed.
- Symbols:
  - Names should not contain symbols, except for underscore, as in `my_variable`, or `$` as the first character to denote a parameter, as given by `$myParam`.
- Length:
  - Can be very long, up to `65535` (`2^16 - 1`) or `65534` characters, depending on the version of Neo4j.
- Case-sensitive:
  - Names are case-sensitive and thus, `:PERSON`, `:Person` and `:person` are three different labels, and `n` and `N` are two different variables.
- Whitespace characters:
  - Leading and trailing whitespace characters will be removed automatically. For example, `MATCH ( a ) RETURN a` is equivalent to `MATCH (a) RETURN a`.

Using special characters in names

Non-alphabetic characters, including numbers, symbols and whitespace characters, **can** be used in names, but **must** be quoted using backticks. For example: `` `^n` ``, `` `1first` ``, `` `$$n` ``, and `` `my variable has spaces` ``. Database names are an exception and may include dots without the need for quoting using backticks, although this behavior is deprecated as it may introduce ambiguity when addressing composite databases. For example: naming a database `foo.bar.baz` is valid, but deprecated. `` `foo.bar.baz` `` is valid.

Within a name quoted by backticks, the following character representations are allowed:

Character representation	Description
``	Backtick
`\uxxxx`	Unicode UTF-16 code point (4 hex digits must follow the `\u`)

> **Note:**
>
> Using names quoted in backticks with unsanitized user input makes you vulnerable to Cypher injection. Some techniques to mitigate this are:
>
> - sanitizing (and validating) the user input.
> - remodeling your data model to avoid this data access pattern.

Scoping and namespace rules

- Node labels, relationship types and property names may re-use names.
  - The following query — with `a` for the label, type and property name — is valid: `CREATE (a:a {a: 'a'})-[r:a]->(b:a {a: 'a'})`.
- Variables for nodes and relationships must not re-use names within the same query scope.
  - The following query is not valid as the node and relationship both have the name `a`: `CREATE (a)-[a]->(b)`.

Recommendations

Node labels, relationship types, and properties are case sensitive. For example, the property `name` is different from the property `Name`.

The following naming conventions are recommended:

Naming conventions

Graph entity	Recommended style	Examples
Node label	PascalCase (capitalized words, no separators)	`(:Person)`, `(:VehicleOwner)`
Relationship type	SCREAMING_SNAKE_CASE (upper case, underscores as separators)	`[:PURCHASED]`, `[:OWNS_VEHICLE]`
Property	camelCase (lower-case first word, capitalized thereafter)	`name`, `firstName`

Length limit of identifiers

Neo4j's GQL's limit on the maximum length of identifiers.

The maximum limit is set to 16,383 characters in an identifier. This means that node labels, relationship types, and property keys cannot include more than 16,383 characters.

Variables

This page provides an overview of variables in Cypher.

When you reference parts of a pattern or a query, you do so by naming them. The names you give the different parts are called variables.

In this example:

```
MATCH (n)-->(b)
RETURN b
```

The variables are `n` and `b`.

Note:

Variables are only visible in the same query part

Variables are not carried over to subsequent queries. If multiple query parts are chained together using `WITH`, variables defined in one part have to be listed in the `WITH` clause to be carried over to the next part. For more information see WITH.

Variables imported into a `CALL` subquery are visible in the whole subquery

Even if the subquery consists of multiple query parts chained together using `WITH`, variables imported from the outer query do not have to be listed in a `WITH` clause to be visible in subsequent parts of the subquery. For more information see The variable scope clause.

Keywords

Keywords are words with a predefined meaning in Cypher.

Keywords are not recommended to be used as identifiers in the following contexts:

- Variables
- Labels
- Relationship types
- Function names
- Procedure names

If a snippet of Cypher can both be a keyword and an unquoted identifier, it is interpreted as a keyword.

For instance in the following, `true` could be the variable `true` or the Boolean literal value `true`. Since `true` is a keyword, the query returns a single Boolean `true` value.

Query

```
WITH 123 AS true
RETURN true AS x
```

Result

x
true
Rows: 1

If any keyword is quoted in backticks (`` ` ``), such as `` `true` ``, it is always interpreted as an identifier in the given context.

Query

```
WITH 123 AS true
RETURN `true` AS x
```

Result

x
123
Rows: 1

List of keywords

The following lists all keywords in Cypher.

Note that with future functionality, Cypher may be extended with additional keywords.

A

- `ACCESS`
- `ACTIVE`
- `ADMIN`
- `ADMINISTRATOR`
- `ALIAS`
- `ALIASES`
- `ALL`
- `ALL_SHORTEST_PATHS`
- `ALTER`
- `AND`
- `ANY`

- `ARRAY`
- `AS`
- `ASC`
- `ASCENDING`
- `ASSIGN`
- `AT`
- `AUTH`
- `AUTO`

B

- `BINDINGS`
- `BOOL`
- `BOOLEAN`
- `BOOSTED`
- `BOTH`
- `BREAK`
- `BUILT`
- `BY`

C

- `CALL`
- `CASCADE`
- `CASE`
- `CHANGE`
- `CIDR`
- `COLLECT`
- `COMMAND`
- `COMMANDS`
- `COMPOSITE`
- `CONCURRENT`
- `CONSTRAINT`
- `CONSTRAINTS`
- `CONTAINS`
- `CONTINUE`
- `COPY`

- `COUNT`
- `CREATE`
- `CSV`
- `CURRENT`

D

- `DATA`
- `DATABASE`
- `DATABASES`
- `DATE`
- `DATETIME`
- `DBMS`
- `DEALLOCATE`
- `DEFAULT`
- `DEFINED`
- `DELETE`
- `DENY`
- `DESC`
- `DESCENDING`
- `DESTROY`
- `DETACH`
- `DIFFERENT`
- `DISJOINT`
- `DISTINCT`
- `DRIVER`
- `DROP`
- `DRYRUN`
- `DUMP`
- `DURATION`

E

- `EACH`
- `EDGE`
- `ELEMENT`
- `ELEMENTS`

- `ELSE`
- `ENABLE`
- `ENCRYPTED`
- `END`
- `ENDS`
- `ERROR`
- `EXECUTABLE`
- `EXECUTE`
- `EXIST`
- `EXISTENCE`
- `EXISTS`

F

- `FAIL`
- `false`
- `FIELDTERMINATOR`
- `FINISH`
- `FILTER`
- `FLOAT`
- `FOR`
- `FOREACH`
- `FROM`
- `FULLTEXT`
- `FUNCTION`
- `FUNCTIONS`

G, H

- `GRANT`
- `GRAPH`
- `GRAPHS`
- `GROUP`
- `GROUPS`
- `HEADERS`
- `HOME`

I

- `ID`
- `IF`
- `IMMUTABLE`
- `IMPERSONATE`
- `IN`
- `INDEX`
- `INDEXES`
- `INF`
- `INFINITY`
- `INSERT`
- `INT`
- `INTEGER`
- `IS`

J, K, L, M

- `JOIN`
- `KEY`
- `LABEL`
- `LABELS`
- `LEADING`
- `LET`
- `LIMITROWS`
- `LIST`
- `LOAD`
- `LOCAL`
- `LOOKUP`
- `MANAGEMENT`
- `MAP`
- `MATCH`
- `MERGE`

N

- `NAME`

- `NAMES`
- `NAN`
- `NEW`
- `NEXT`
- `NFC`
- `NFD`
- `NFKC`
- `NFKD`
- `NODE`
- `NODES`
- `NODETACH`
- `NONE`
- `NORMALIZE`
- `NORMALIZED`
- `NOT`
- `NOTHING`
- `NOWAIT`
- `null`

O

- `OF`
- `OFFSET`
- `ON`
- `ONLY`
- `OPTION`
- `OPTIONAL`
- `OPTIONS`
- `OR`
- `ORDER`

P

- `PASSWORD`
- `PASSWORDS`
- `PATH`
- `PATHS`

- `PLAINTEXT`
- `POINT`
- `POPULATED`
- `PRIMARIES`
- `PRIMARY`
- `PRIVILEGE`
- `PRIVILEGES`
- `PROCEDURE`
- `PROCEDURES`
- `PROPERTIES`
- `PROPERTY`
- `PROVIDER`
- `PROVIDERS`

R

- `RANGE`
- `READ`
- `REALLOCATE`
- `REDUCE`
- `REL`
- `RELATIONSHIP`
- `RELATIONSHIPS`
- `REMOVE`
- `RENAME`
- `REPEATABLE`
- `REPLACE`
- `REPORT`
- `REQUIRE`
- `REQUIRED`
- `RESTRICT`
- `RETURN`
- `REVOKE`
- `ROLE`
- `ROLES`
- `ROW`

- `ROWS`

S

- `SCAN`
- `SCORE`
- `SEARCH`
- `SEC`
- `SECOND`
- `SECONDARIES`
- `SECONDARY`
- `SECONDS`
- `SEEK`
- `SERVER`
- `SERVERS`
- `SET`
- `SETTING`
- `SETTINGS`
- `SHORTEST`
- `SHORTEST_PATH`
- `SHOW`
- `SIGNED`
- `SINGLE`
- `SKIPROWS`
- `START`
- `STARTS`
- `STATUS`
- `STOP`
- `STRING`
- `SUPPORTED`
- `SUSPENDED`

T

- `TARGET`
- `TERMINATE`
- `TEXT`

- `THEN`
- `TIME`
- `TIMESTAMP`
- `TIMEZONE`
- `TO`
- `TOPOLOGY`
- `TRAILING`
- `TRANSACTION`
- `TRANSACTIONS`
- `TRAVERSE`
- `TRIM`
- `true`
- `TYPE`
- `TYPED`
- `TYPES`

U

- `UNION`
- `UNIQUE`
- `UNIQUENESS`
- `UNWIND`
- `URL`
- `USE`
- `USER`
- `USERS`
- `USING`

V, W, X, Y, Z

- `VALUE`
- `VARCHAR`
- `VECTOR`
- `VERTEX`
- `WAIT`
- `WHEN`
- `WHERE`

- `WITH`
- `WITHOUT`
- `WRITE`
- `XOR`
- `YIELD`
- `ZONE`
- `ZONED`

Parameters

This page describes parameterized querying.

Introduction

Cypher supports querying with parameters. A parameterized query is a query in which placeholders are used for parameters and the parameter values are supplied at execution time. This means developers do not have to resort to string building to create a query. Additionally, parameters make caching of execution plans much easier for Cypher, thus leading to faster query execution times.

Parameters can be used for:

- Literals and expressions.
- Node and relationship ids.
- Properties, when referenced dynamically (for more information, see `WHERE` → Filter on dynamic properties).
- Node labels and relationship types, when referenced dynamically (for more information, see `MATCH` using dynamic node labels and relationship types).

Parameters cannot be used for the following constructs, as these form part of the query structure that is compiled into a query plan:

- Property keys; `MATCH (n) WHERE n.$param = 'something'` is invalid.
- Relationship types; `MATCH (n)-[:$param]→(m)` is invalid.
- Node labels; `MATCH (n:$param)` is invalid.

Parameter names can start with and contain underscores, letters, numbers, and combining marks such as accents (e.g., `é`). Currency symbols (e.g., `$` and `€`), non-printable Unicode control characters that only affect formatting (e.g., `\t` for Tab), punctuation (e.g., `!`), and whitespaces are not allowed in any position of a parameter name. For example, `$héllo`, `$0param`, and `$_param` are valid, while `$€value`, `$hello!`, and `$bad param` are not.

Setting parameters when running a query is dependent on the client environment. For example:

- To set a parameter in Cypher Shell use `:param name => 'Joe'`. For more information refer to Operations Manual → Cypher Shell - Query Parameters.
- For Neo4j Browser use the same syntax as Cypher Shell, `:param name => 'Joe'`.
- When using drivers, the syntax is dependent on the language choice. See the examples in *Transactions* in the Neo4j Driver manuals.
- For usage via the Neo4j HTTP API, see the HTTP API documentation.

We provide below a comprehensive list of examples of parameter usage. In these examples, parameters are given in JSON; the exact manner in which they are to be submitted depends upon the driver being used.

Auto-parameterization

From Neo4j 5 onwards, even when a query is partially parameterized, Cypher will try to infer parameters anyway. Each literal in the query is replaced with a parameter. This increases the re-usability of the computed plan for queries that are identical except for the literals. It is not recommended to rely on this behavior - users should rather use parameters where they think it is appropriate.

String literal

Parameters

```
{
  "name": "Johan"
}
```

Query

```
MATCH (n:Person)
WHERE n.name = $name
RETURN n
```

You can use parameters in this syntax as well:

Parameters

```
{
  "name": "Johan"
}
```

Query

```
MATCH (n:Person {name: $name})
RETURN n
```

Regular expression

Parameters

```
{
  "regex": ".*h.*"
```

```
}
```

Query

```
MATCH (n:Person)
WHERE n.name =~ $regex
RETURN n.name
```

Case-sensitive `STRING` pattern matching

Parameters

```
{
  "name": "Michael"
}
```

Query

```
MATCH (n:Person)
WHERE n.name STARTS WITH $name
RETURN n.name
```

Create node with properties

Parameters

```
{
  "props": {
    "name": "Andy",
    "position": "Developer"
  }
}
```

Query

```
CREATE ($props)
```

Create multiple nodes with properties

Parameters

```
{
  "props": [ {
    "awesome": true,
    "name": "Andy",
    "position": "Developer"
  }, {
    "children": 3,
    "name": "Michael",
    "position": "Developer"
  } ]
}
```

Query

```
UNWIND $props AS properties
CREATE (n:Person)
```

```
SET n = properties
RETURN n
```

Setting all properties on a node

Note that this will replace all the current properties.

Parameters

```
{
  "props": {
    "name": "Andy",
    "position": "Developer"
  }
}
```

Query

```
MATCH (n:Person)
WHERE n.name = 'Michaela'
SET n = $props
```

`SKIP` and `LIMIT`

Parameters

```
{
  "s": 1,
  "l": 1
}
```

Query

```
MATCH (n:Person)
RETURN n.name
SKIP $s
LIMIT $l
```

Node id

Parameters

```
{
  "id" : "4:1fd57deb-355d-47bb-a80a-d39ac2d2bcdb:0"
}
```

Query

```
MATCH (n)
WHERE elementId(n) = $id
RETURN n.name
```

Multiple node ids

Parameters

```
{
  "ids" : [ "4:1fd57deb-355d-47bb-a80a-d39ac2d2bcdb:0", "4:1fd57deb-355d-47bb-a80a-d39ac2d2bcdb:1" ]
}
```

Query

```
MATCH (n)
WHERE elementId(n) IN $ids
RETURN n.name
```

Calling procedures

Parameters

```
{
  "indexname" : "My_index"
}
```

Query

```
CALL db.resampleIndex($indexname)
```

Comments

This page describes how to use comments in Cypher.

A single line comment begins with double slash (`//`) and continues to the end of the line. A multi-line comment begins with a slash and asterisk (`/*`) and continues until it ends with an asterisk and a slash ('*/'). Comments do not execute, they are for humans to read.

Examples:

```
MATCH (n) RETURN n //This is an end of line comment
```

```
MATCH (n)
//This is a whole line comment
RETURN n
```

```
MATCH (n) /* This is a multi line comment,
the comment continues on this line
but it ends on this line. */
RETURN n
```

```
MATCH (n) WHERE n.property = '//This is NOT a comment' RETURN n
```

Additions, deprecations, removals, and compatibility

Cypher is a language that is constantly evolving. New features are added to the language continuously, and occasionally, some features become deprecated and are subsequently removed. All changes to Cypher are introduced in Neo4j versions.

This section lists all of the features that have been removed, deprecated, added, or extended in different versions of Neo4j. Replacement syntax for deprecated and removed features are also indicated.

> Neo4j 2026.02+ databases explicitly set Cypher 25 as their query language. Cypher 25 was introduced in Neo4j 2025.06 and can only be used on Neo4j 2025.06+ databases. Any new Cypher features introduced in Neo4j 2025.06 or later will be exclusively added to Cypher 25. Features removed in Cypher 25 are still available on Neo4j 2025.06+ databases either by prepending a query with `CYPHER 5` or by having Cypher 5 as the default language for the database. For more information, see Select Cypher version.

Neo4j 2026.10

Deprecations

Feature	Details

Deprecated
`CALL db.index.fulltext.queryNodes(...)`
`CALL db.index.fulltext.queryRelationships(...)`
	The procedures `db.index.fulltext.queryNodes` and `db.index.fulltext.queryRelationships` have been deprecated in favor of the new `FULLTEXT` search syntax, see Cypher's `SEARCH` clause for details.

Updates

Feature	Details

Updated
`SHOW CURRENT USER`
`YIELD user, home`
`SHOW DATABASES`
`YIELD name, aliases`
`WHERE name = home`
`RETURN user, name AS database, aliases`
	`SHOW CURRENT USER` can now be combined with the other composable show commands and general Cypher clauses in a single query. However, as for `SHOW DATABASES`, all parts of the query must be allowed on the system database. For more information about composable `SHOW` commands, see Composable `SHOW` commands. For more information about show current user, see the Operations Manual → Show current user.

Feature	Details
Updated `GRANT READ {*} ON GRAPH * FOR (n:Document) WHERE n.classification IN abac.native.user_tags() TO regularUsers`	The `abac.native.user_tags()` function can now also be used in an `IN` predicate of the `WHERE` clause of a property-based privilege `GRANT` command. This allows a single privilege to grant different users different data access based on their tags, as long as the native provider is configured for ABAC. For more information, see the Operations Manual → Grant a property-based privilege using native user tags.
Updated `PROFILE UNWIND range(1, 1000) AS i CALL (i) { CREATE (:Person {id: i}) } IN 4 CONCURRENT TRANSACTIONS OF 100 ROWS`	`PROFILE` now reports the concurrency achieved by `CALL { … } IN CONCURRENT TRANSACTIONS`. A new `Concurrency` column on the `TransactionApply` and `TransactionForeach` operators shows the peak and average number of batches that ran at the same time, and the profiled plan includes further statistics such as queued batches and retries. For more information, see Measuring concurrency with `PROFILE`.

New

Feature	Details
Introduced `WITH {a: 1, b: 2} AS map1, {c: 3, d:4} AS map2 RETURN map1 + map2`	Introduction of map addition, allowing the merging of two `MAP` values.

Neo4j 2026.09

Updates

Feature	Details
Updated `RETURN toString(), toStringList(), toStringOrNull()`	The functions toString(), toStringList(), toStringOrNull() have been updated to accept `LIST`, `MAP`, `NODE`, `RELATIONSHIP`, and `PATH` types.
Updated `SHOW USERS AS COMMANDS; SHOW USERS WITH AUTH AS COMMANDS;`	Show users can now return the command needed to recreate each user in the DBMS, including their authentication configuration and credentials. For more information, see the Operations Manual → Manage users.
Updated `SHOW ROLES AS COMMANDS; SHOW ROLES WITH USERS AS COMMANDS; SHOW ROLES WITH AUTH RULES AS COMMANDS;`	Show roles can now return the commands needed to recreate each role in the DBMS, optionally including its assigned users and associated auth rules. For more information, see the Operations Manual → Manage roles.

Feature	Details
Updated `SHOW AUTH RULE ruleName PRIVILEGES [AS [REVOKE] COMMANDS]`	The `SHOW PRIVILEGES` and `SHOW PRIVILEGES AS [REVOKE] COMMANDS` commands now allow filtering the privileges based on auth rules, similar to the existing role and user filters. For more information, see the Operations Manual → `SHOW PRIVILEGES`.

Updated
`SHOW [ROLE roleName] PRIVILEGES AS [REVOKE] COMMANDS YIELD access, action, roles;`
`SHOW USER [username] PRIVILEGES AS [REVOKE] COMMANDS YIELD access, action, roles, users;`
`SHOW AUTH RULE ruleName PRIVILEGES AS [REVOKE] COMMANDS YIELD access, action, roles, authRules;`
	Several new columns for filtering have been added to the `SHOW PRIVILEGES AS [REVOKE] COMMANDS` commands output. Most are non-default but the `users` and `authRules` columns are returned by default when adding the `USER` or `AUTH RULE` filtering. For more information, see the Operations Manual → `SHOW PRIVILEGES`.

New

Feature	Details
Introduced `WITH {a: 1, b: 2, c: 3} AS map RETURN {k: v IN map | k: v * 10} AS result`	Introduced map comprehension, which allows the creation of a `MAP` value by iterating over an existing `MAP` or `LIST`. For more information, see Map expressions → Map comprehension.
Introduced `SHOW USER CREDENTIALS`	New privilege that, together with the `SHOW USER` privilege, allows a user to show users as commands. This is a sub-privilege of the `USER MANAGEMENT` privilege. Like all `GRANT` / `DENY` commands this is only available in Enterprise Edition. For more information, see the Operations Manual → Grant privilege to show users as commands.
Introduced `MATCH (node) SEARCH node IN ( FULLTEXT INDEX namesAndTeams FOR "nils" LIMIT 5 ) SCORE AS score RETURN node.name, score`	Introduced the new `FULLTEXT` keyword in Cypher's `SEARCH` clause to enable full-text search.

Neo4j 2026.08

New

Feature	Details

Introduced
`WITH 'Alice' AS name`
`RETURN s"Hello, {name}!" AS greeting`
	Introduction of string interpolation, a `STRING` literal syntax (`s"…"` or `S"…"`) that embeds expressions enclosed in `{}` directly within a string. Each embedded expression is converted to a `STRING` using `toString()`, so expressions that evaluate to `MAP`, `LIST`, `NODE`, `PATH`, or `RELATIONSHIP` values are not supported.

Introduced `uuid("550e8400-e29b-41d4-a716-446655440000")`	Introduction of a `UUID` value type that can be stored on properties of nodes and relationships. For more information, see Values and types → UUID.

Introduced
`RETURN`
`  uuid() AS randomUUID,`
`  uuid(42, 42) AS uuidFromInts,`
`  uuid("") AS uuidFromString`
	New `uuid()` functions for the construction of `UUID` values.

Introduced
`WITH UUID() AS UUID`
`RETURN`
`  uuid.mostSignificantBits(UUID) AS mostSigBits,`
`  uuid.leastSignificantBits(UUID) AS leastSigBits`
	New `uuid.mostSignificantBits()` function and `uuid.leastSignificantBits()` function, which return the most/least significant bits of a of a `UUID` value.

Neo4j 2026.07

Updates

Feature	Details

Updated
`CREATE VECTOR INDEX moviePlots IF NOT EXISTS`
`FOR (m:Movie) ON m.embedding`
`WITH [m.releaseDate, m.rating]`
`OPTIONS {`
`  indexConfig: {`
`  `vector.similarity_function`: 'cosine',`
`  `vector.dimensions`: 3,`
`  `vector.quantization.type`: 'binary',`
`  `vector.default_search_expansion_factor`: 3.0`
`  }`
`}`
	Binary Quantization and High-Fidelity Quantization are now out of preview.
	
	For more details, see Vector index configuration settings → `config-vector.default_search_expansion_factor` and `config-vector.quantization.type`.

Updated `SHOW CURRENT GRAPH TYPE AS GRAPH`	In addition to returning the graph type as a string, show current graph type can now return the graph type as a virtual graph. For more information, see Show graph types → `SHOW CURRENT GRAPH TYPE AS GRAPH` columns.

New

Feature	Details

Introduced
`RETURN cardinality([1, 2, 3]);`
`RETURN cardinality({a: 1, b: 2});`
`MATCH p = (a)-[r]->(b) RETURN cardinality(p);`
	Introduction of the `cardinality()` function. It returns the number of keys in a `MAP`, the number of elements in a `LIST<ANY>`, or the number of nodes and relationships in a `PATH`.

Introduced
`MATCH (p:Person)-[:ACTED_IN]->(m:Movie)`
`RETURN p.name AS actor_name, m.genre AS genre, avg (m.rating) AS averageRating`
`GROUP BY actor_name, genre`
	A new `GROUP BY` subclause used in conjunction with the `WITH` or `RETURN` clauses has been added. It allows a user to explicitly state the grouping keys for the query. This is an alternative to the implicit grouping available in Cypher.

Introduced Added two `RemoteUniqueNodeIndexSeek`.	`RemoteNodeIndexSeek` batches index seeks when applied within a loop, while `RemoteUniqueNodeIndexSeek` does the same for unique index seeks.

Neo4j 2026.06

Deprecations

Feature	Details

Deprecated
`CREATE VECTOR INDEX moviePlots IF NOT EXISTS`
`FOR (m:Movie) ON m.embedding`
`WITH [m.releaseDate, m.rating]`
`OPTIONS {`
`  indexConfig: {`
`  `vector.similarity_function`: 'cosine',`
`  `vector.dimensions`: 3,`
`  `vector.quantization.enabled`: false`
`  }`
`}`
	The vector index configuration setting `vector.quantization.enabled` has been deprecated in favor of `vector.quantization.type`. The latter allows you to specify quantization types (`'none'`, `'scalar'`, and `'binary'`).

Updates

Feature	Details

Updated
 	
`LOAD CSV WITH HEADERS FROM $csv AS line`
`CALL (line) {`
`  MERGE (a:Person {id: line.actorId})`
`  MERGE (m:Movie  {id: line.movieId})`
`  CREATE (a)-[:ACTS_IN {name: line.roleName}]->(m)`
`} IN CONCURRENT TRANSACTIONS OF 500 ROWS`
`  DISJOINT BY AUTO`
	`CALL {…} IN CONCURRENT TRANSACTIONS` now accepts an optional `DISJOINT BY` clause. This adds optional deadlock prevention scheduling to concurrent transactions. The runtime schedules batches that share resources serially and batches that do not concurrently, preventing deadlocks rather than recovering from them. This is particularly useful when importing relationships.
	
	`DISJOINT BY` has three forms:
	
	• `DISJOINT BY (<expr>, …)`: declares lock-prone resources explicitly per row.
	• `DISJOINT BY AUTO`: the runtime infers lock-prone resources automatically.
	• `DISJOINT BY NONE`: disables scheduling explicitly, overriding any database-level default.
	
	For more information, see `CALL` subqueries in transactions → Batch scheduling with `DISJOINT BY`.

Functionality Updated
 	
`MATCH (movie:Movie)`
`  SEARCH movie IN (`
`  VECTOR INDEX moviePlots`
`  FOR [1, 2, 3]`
`  WHERE movie.rating IN [7.6, 7.7, 8.0]`
`  LIMIT 10`
`  )`
`RETURN movie`
	`WHERE` predicates inside a vector `SEARCH` now support the `IN` operator. For more information, see Predicates for vector search with filters.

Updated
 	
`ALTER CURRENT GRAPH TYPE SET {`
`  (:Person => :Resident {name :: STRING NOT NULL}),`
`  (:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}),`
`  (:City => {name :: STRING NOT NULL, population :: INTEGER}),`
`  (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]->(:City)`
`}`
	Graph types are now generally available. This includes the ability to:
	
	• Set a graph type
	• Alter elements in a graph type
	• Extend a graph type
	• Drop elements in a graph type
	• Show a graph type
	
	Prior to this release, the feature was in preview.

Updated Preview
 	
`CREATE VECTOR INDEX moviePlots IF NOT EXISTS`
`FOR (m:Movie) ON m.embedding`
`WITH [m.releaseDate, m.rating]`
`OPTIONS {`
`  indexConfig: {`
`  `vector.similarity_function`: 'cosine',`
`  `vector.dimensions`: 3,`
`  `vector.quantization.type`: 'binary',`
`  `vector.default_search_expansion_factor`: 2.0`
`  }`
`}`
	Vector indexes now support binary quantization (`vector.quantization.type: 'binary'`) and a search expansion factor (`vector.default_search_expansion_factor`), both currently in preview, for tuning the trade-off between accuracy and query time.
	
	For more details, see Vector index configuration settings → `config-vector.default_search_expansion_factor` and `config-vector.quantization.type`.

New

Feature	Details

Introduced
 	
`CREATE AUTH RULE nativeAdminRule`
`SET CONDITION 'admin' IN abac.native.user_tags()`
	Introduction of the security function `abac.native.user_tags()`, which returns the list of tags for the native user.
	
	This function cannot be called directly in queries. It can only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command. For more information about defining auth rules and more extensive examples, see Operations Manual → Attribute-based access control.

Neo4j 2026.05

Updates

Feature	Details

Updated
 	
`MATCH p = ANY SHORTEST ACYCLIC (a:A)-->+(b:!A)`
`RETURN p`
	Explicit path modes such as `ACYCLIC` can now be combined with restrictive path selectors, such as `SHORTEST`. For more information, see Patterns → Mixing restrictive path selectors and explicit path modes.

Updated
 	
`SHOW FUNCTIONS`
`YIELD name, signature`
`WITH name, collect(signature) AS signatures`
`FILTER size(signatures) > 1`
`RETURN name, signatures`
	Show indexes, constraints, current graph type, functions, procedures, settings, databases, and transactions, and terminate transactions can be combined with each other and general Cypher clauses in a single query. For more information, see SHOW → Composable `SHOW` commands. For `SHOW DATABASES`, all parts of the query must be allowed on the system database. For more information, see the Operations Manual → Show databases.

Updated
 	
`SHOW SETTINGS null`
 	
`SHOW TRANSACTIONS null`
 	
`TERMINATE TRANSACTIONS null`
 	
`SHOW DATABASE $nullParam`
	Show settings, databases, and transactions, and terminate transactions treat `null` input the same as non-existent input. The show commands filter out the `null` input. Terminate transactions return a row about the transaction not being found.
	
	For full information about these commands, see the following pages in the Operations Manual:
	
	• Show databases
	• Show settings
	• Show and terminate transactions

Feature	Details

Updated
`RETURN stDev(null) AS stDevNull,`
`stdev_samp(null) AS stdev_sampNull,`
`stDevP(null) AS stDevPNull,`
`stdev_pop(null) AS stdev_popNull`
	`stDev()`, `stdev_samp()`, `stDevP()`, and `stdev_pop()` now return `null` instead of `0` when given a `null` argument.

New

Feature	Details

Introduced
`RETURN string.indexOf('hello', 'l');`
`RETURN string.join(['one', 'two'], ', ');`
`RETURN string.regexReplace("hello", "l.", "w");`
	Introduction of three new string-based Cypher functions: `string.indexOf()`, `string.join()`, and `string.regexReplace()`.

Neo4j 2026.04

Deprecations

Feature	Details

Deprecated
`db.index.vector.queryNodes()`
`db.index.vector.queryRelationships()`
	Deprecated the two vector index procedures `db.index.vector.queryNodes()` and `db.index.vector.queryRelationships()` in favor of Cypher's `SEARCH` clause.

New

Feature	Details

Introduced
`FOR x IN [1, 2, 3]`
`RETURN x`
	The `FOR` clause (optional GQL feature GQ10) expands a list into rows using GQL conformant syntax: `FOR variable IN expression`.

Introduced
`MATCH (n:Person)`
`RETURN n IS LABELED Manager AS isManager`
	`IS [NOT\` LABELED predicate]: tests a node or relationship against a label expression, equivalent to the existing colon form.

Neo4j 2026.03

New

Feature	Details

Introduced
 	
`MATCH p = ACYCLIC (start)--{,2}(end)`
`RETURN p`
	Introduction of the `ACYCLIC` keyword, which adds a further restriction to a pattern match which ensures that nodes cannot be repeated within a path (though they can still be repeated across paths to form equijoins.)

Introduced `PROPERTY_EXISTS` predicate	A new `PROPERTY_EXISTS(element, propertyName)` predicate had been added for testing whether a property exists on a node or relationship and is not `NULL`.
Introduced `currentQueryProgress` column for `SHOW TRANSACTIONS`	A new column `currentQueryProgress` has been added to the output table of `SHOW TRANSACTIONS`. See the Operations Manual → Show and terminate transactions for details.

Introduced
 	
`CREATE AUTH RULE nativeAdminRule`
`SET CONDITION 'admin' IN abac.native.user_tags()`
	Introduction of the security function `abac.oidc.user_attribute()`, which returns the value of the specified user attribute from the OIDC claims.
	
	This function cannot be called directly in queries. It can only be used in the condition of a `CREATE AUTH RULE` or `ALTER AUTH RULE` command. For more information about defining auth rules and more extensive examples, see Operations Manual → Attribute-based access control.

Neo4j 2026.02

Updates

Feature	Details
Updated New databases now explicitly set Cyper 25 as their query language.	New self-managed installations and new Aura instances now explicitly set Cyper 25 as their query language. New self-managed deployments have the setting `db.query.default_language=CYPHER_25` in their *neo4j.conf* file. Existing deployments retain their *neo4j.conf* file. Both can be adjusted via the setting `db.query.default_language`. + In Aura, Cypher 25 is the default for newly created organisations. The default query language attached to created databases is determined by the organisation setting "Default Cypher Version" and can be manually changed.

Feature	Details
Updated `SHOW CONSTRAINTS YIELD name, enforcedLabel, classification`	Introduced two new columns returned by `SHOW CONSTRAINTS`: • `enforcedLabel`: shows the implied labels belonging to a node element type, and the source and target node labels belonging to a relationship element type. • `classification`: shows the status of a constraint in a graph type. For more information, see Constraints → Return columns and Graph types → Graph type elements in `SHOW CONSTRAINTS`.
Updated `SHOW [NODE] EXIST[ENCE] CONSTRAINTS`	The `NODE EXIST[ENCE]` and `EXIST[ENCE]` type filtering options allowed by the `SHOW CONSTRAINTS` command now also return node label existence constraints (created as part of node element types in a graph type). For more information, see Syntax → Type filters.
Updated `SHOW CONSTRAINTS`	The `type` column returned by `SHOW CONSTRAINTS` can now return three additional constraint types: `NODE_LABEL_EXISTENCE`, `RELATIONSHIP_SOURCE_LABEL` and `RELATIONSHIP_TARGET_LABEL`. These new constraint types are created as part of node and relationship element types when setting a graph type. For more information, see Set graph types.

New

Feature	Details
Introduced Added the performance-improving operator `MergeUniqueNode`.	`MergeUniqueNode` is similar to the `Merge` operator, but is used when there is a property uniqueness constraint on the property used in the `MERGE` statement.
Introduced Preview `ALTER CURRENT GRAPH TYPE SET { (:Person => :Resident {name :: STRING NOT NULL}), (:Pet => :Resident&Animal {healthCertificate :: STRING, name :: STRING}), (:City => {name :: STRING NOT NULL, population :: INTEGER}), (:Resident)-[:LIVES_IN => {since :: DATE NOT NULL}]-> (:City) }`	Introduced the ability to set a graph type for a database using the `ALTER CURRENT GRAPH TYPE SET` command. A graph type enforces holistically an open schema, providing a defined structure for an entire graph, and includes constraint types (node label existence, relationship source node, and relationship target node constraints) not possible to create using the older `CREATE CONSTRAINT` syntax. For more information, see Set graph types.

Feature	Details
Introduced Preview `ALTER CURRENT GRAPH TYPE ADD { (p:Person => :Resident {name :: STRING, ssn :: INTEGER, nationality :: STRING}) REQUIRE (p.name, p.ssn) IS KEY, CONSTRAINT pet_address FOR (pet:Pet) REQUIRE pet.address IS NOT NULL, (:Animal => {type :: STRING NOT NULL}) }`	Introduced the ability to extend graph types using the `ALTER CURRENT GRAPH TYPE ADD` command. For more information, see Extend graph types.
Introduced Preview `ALTER CURRENT GRAPH TYPE ALTER { (:Robot => :Resident&Machine {application :: STRING NOT NULL, id :: INTEGER NOT NULL}), (:Resident)-[:LIVES_IN => {since :: ANY NOT NULL}]-> (:City) }`	Introduced the ability to alter element types in a graph type using the `ALTER CURRENT GRAPH TYPE ALTER` command. For more information, see Alter element types.
Introduced Preview `SHOW CURRENT GRAPH TYPE`	Introduced the ability to show graph types using the `SHOW CURRENT GRAPH TYPE` command. For more information, see Show graph types.
Introduced Preview `ALTER CURRENT GRAPH DROP { (:Pet =>), CONSTRAINT pet_address }`	Introduced the ability to drop element types and constraints from graph types using the `ALTER CURRENT GRAPH TYPE DROP` command. For more information, see Drop graph type elements.

Introduced GQL naming aliases for functions
`RETURN ceiling(1.2) AS c, ln(2.0) AS natural_log, char_length("Hello") AS len, lower("Neo4j") AS lower, upper("Neo4j") AS upper;`
`RETURN local_time() AS t, local_datetime() AS ldt, zoned_time() AS zt, zoned_datetime() AS zdt;`
`RETURN duration_between(date("2020-01-01"), date("2020-02-01")) AS d;`
`MATCH p = (a)-[r]->(b) RETURN path_length(p) AS path_len;`
`UNWIND [1, 2, 3] AS n RETURN collect_list(n) AS list, percentile_cont(n, 0.5) AS p50, percentile_disc(n, 0.5) AS p50_disc, stdev_samp(n) AS sd_samp, stdev_pop(n) AS sd_pop;`
	Cypher now provides GQL naming aliases so that queries written for other GQL implementations can run with minimal changes. Aliases added: `ceiling()`, `local_time()`, `local_datetime()`, `zoned_time()`, `zoned_datetime()`, `duration_between()`, `path_length()`, `ln()`, `collect_list()`, `percentile_cont()`, `percentile_disc()`, `stdev_samp()` and `stdev_pop()`.

Neo4j 2026.01

Updates

Feature	Details

Introduced
 	
`CREATE VECTOR INDEX multiLabel FOR (n:Label1|Label2) ON n.embedding OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}}`
 	
`CREATE VECTOR INDEX multiRelType FOR ()-[r:REL_TYPE1|REL_TYPE2]-() ON r.embedding OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}}`
 	
`CREATE VECTOR INDEX additionalProperties FOR (n:Label) ON n.embedding WITH [n.property1, n.property2] OPTIONS {indexConfig: {`vector.dimensions`: 1536, `vector.similarity_function`: 'cosine'}}`
	Vector indexes can now have multiple labels, relationship types or include additional properties for filtering.

New

Feature	Details
Introduced Added two `RemoteBatchPropertiesWithFilter`.	`RemoteBatchProperties` fetches and caches properties from shards for batches of rows, so that they can be used by subsequent operators, while `RemoteBatchPropertiesWithFilter` does the same for rows which satisfy a predicate.

Introduced
 	
`MATCH (n) SEARCH n IN ( VECTOR INDEX nodeVectorIndex FOR $vectorEmbedding WHERE n.prop > 10 LIMIT 5 ) RETURN n`
 	
`MATCH ()-[r]->() SEARCH r IN ( VECTOR INDEX relVectorIndex FOR $vectorEmbedding LIMIT 5 ) RETURN r`
	A new SEARCH subclause for `MATCH` and `OPTIONAL MATCH` has been added which is used for filtering the results based on vector similarity.

Introduced Introduced a query size limit.	Introduced a query size limit for a query to be considered for query caching. This limit is used to prevent large generated query text strings from occupying memory in the query cache.

Neo4j 2025.12

No documentation updates for this release.

Neo4j 2025.11

Updates

Feature	Details

Updated
 	
`label,name`
`Actor,Henry Cavill`
 	
``CREATE RANGE INDEX actor_has_name FOR (a:Actor) ON (a.name);
 	
LOAD CSV WITH HEADERS FROM 'people.csv' AS row
MERGE (:$(row.label) { name: row.name })```
	Cypher can now leverage indexes on property values, improving the performance of `MATCH` and `MERGE` when doing so, however:
	
	- It only supports exact seeks on range indexes (no full text or spatial).
	- The index order cannot be leveraged, so the planner must insert separate ordering if required later on in the query.
	- Parallel runtime seeks and scans are single-threaded.
	- The planner doesn't combine multiple property index seeks when generating the results for the dynamic part of the query.

New

Feature	Details
Introduced ```RETURN coll.distinct([true, false, false, true, true, false]) RETURN coll.flatten([1, [2]]), coll.flatten([false, ['a']], 2); RETURN coll.indexOf(['A', 'new', 'function'], 'new'); RETURN coll.insert([1, 2, 4], 2, 3); RETURN coll.max([1.5, 2, 5.4, 0, 4]); RETURN coll.min([1.5, 2, 5.4, 0, 4]); RETURN coll.remove(['a', 'a', 'b', 'c', 'd'], 2); RETURN coll.sort([3, 1, 4, 2, 'a', 'c', 'b']);```	Introduction of eight new collection-based Cypher functions. For more information, see Functions → List functions.
Introduced Added the performance-improving operators `MergeInto` .	`MergeInto` is similar to the `Merge` operator, but it is used when the start and end node of the pattern are matched outside the `MERGE` pattern.

Neo4j 2025.10

Updates

Feature	Details
Updated `RETURN toFloatList(Vector([1, 2, 3], 3, INTEGER64)), toIntegerList(Vector([1, 2, 3], 3, INTEGER8)), vector.similarity.cosine(vector([3, 5, 7], 3, INT), vector([-1, -2, -3], 3, INT)), vector.similarity.euclidean(vector([3, 5, 7], 3, INT), vector([-1, -2, -3], 3, INT))`	The `toFloatList()`, `toIntegerList()`, `vector.similarity.cosine()`, and `vector.similarity.euclidean()` functions now accept `VECTOR` values as input arguments.
Updated `RETURN datetime('11/18/1986', "MM/dd/yyyy") AS dt`	The following constructors of temporal types have been extended with the optional argument `pattern`: date(), datetime(), localdatetime(), localtime(), time() and duration(). These patterns are the same as those used to create temporal strings with the `format()` function introduced in 2025.09 and thus allow for the parsing of temporal strings into temporal values.

New

Feature	Details
Introduced `VECTOR([1.05, 0.123, 5], 3, FLOAT32 NOT NULL)`	Introduced a `VECTOR` value type that can be stored as embedding properties on nodes and relationships and utilized for efficient semantic retrieval using Neo4j's vector indexes and GenAI plugin. For more information, see Values and types → Vectors.
Introduced `WITH vector([1, 2, 3], 3, INTEGER) AS vector RETURN vector, valueType(vector) AS vectorType`	New `vector()` function for the construction of `VECTOR` values.
Introduced `RETURN vector_dimension_count(vector([1, 2, 3], 3, INTEGER)) AS size`	New `vector_dimension_count()` function, which returns the dimension of a `VECTOR` value.
Introduced `RETURN vector_distance(vector([1, 2, 3], 3, INT), vector([1, 2, 4], 3, INT), COSINE) AS distance`	New `vector_distance()` function, which returns the distance between two `VECTOR` values based on the selected `vectorDistanceMetric` algorithm.
Introduced `RETURN vector_norm(vector([1.0, 5.0, 3.0, 6.7], 4, FLOAT), EUCLIDEAN) AS norm`	New `vector_norm()` function, which returns the distance between the given vector and an origin vector, which is a vector with the same dimension with all coordinates set to zero, calculated using the specified `vectorDistanceMetric`.

Feature	Details

Introduced
 	
`CREATE CONSTRAINT node_vector_constraint FOR (n:Movie) REQUIRE n.embedding IS :: VECTOR<INT32>(42)`
 	
`CREATE CONSTRAINT rel_vector_constraint FOR ()-[r:CONTAINS]->() REQUIRE r.embedding IS :: VECTOR <FLOAT32>(1536)`
	Introduced `VECTOR` property type constraints. For more information, see Create property type constraints.

Neo4j 2025.09

New

Feature	Details
Introduced `WITH datetime('1986-11-18T6:04:45.123456789+01:00[Europe/Berlin]') AS dt RETURN format(dt, "MM/dd/yyyy") AS US, format(dt, "dd/MM/yyyy") AS EU`	Cypher's new `format()` function can create dynamically formatted string representations of temporal instance and duration types.
Introduced New operator: `LockNodes`	Introduced `LockNodes` operator, sometimes used in conjunction with the `LockingMerge` operator to lock nodes.

Neo4j 2025.08

Updates

Feature	Details
Updated `MATCH (p:Product) WHERE p.name <> "Coffee" CALL (p) { MATCH (p)<-[:BUYS]-(c:Customer)-[:BUYS]->(otherProduct) RETURN c, otherProduct NEXT RETURN count(DISTINCT c) AS customers, 0 AS customersAlsoBuyingCoffee UNION FILTER otherProduct.name = "Coffee" RETURN 0 as customers, count(DISTINCT c) AS customersAlsoBuyingCoffee NEXT RETURN max(customers) AS customers, max(customersAlsoBuyingCoffee) AS customersAlsoBuyingCoffee } RETURN p.name AS product, round(toFloat(customersAlsoBuyingCoffee) * 100 / customers, 1) AS percentageOfCustomersAlsoBuyingCoffee ORDER BY product`	`NEXT` now correctly supports aggregations in the context of `UNION` and `CALL`. For more information, see Sequential queries (`NEXT`) > Known issues and fixes.

Feature	Details

Updated
`PROFILE`
`WITH "Person" AS label`
`MATCH (people:$(label))`
`RETURN people.name`
	Cypher can now leverage token lookup indexes when planning queries with dynamic labels and relationship types. This is enabled by the introduction of three new query plan operators: `DynamicLabelNodeLookup`, `DynamicDirectedRelationshipTypeLookup`, and `DynamicUndirectedRelationshipTypeLookup`.

New

Feature	Details

Introduced
`MATCH (()-->(n))+`
`WHERE allReduce(acc = 0, node IN n | acc + node.x, 6 < acc < 30)`
`RETURN [i IN n | i.x] AS sequence`
`ORDER BY head(n).x, size(n)`
	New `allReduce()` function. It enables the stepwise evaluation of a value accumulated over a path, allowing for early pruning of paths that do not satisfy a given predicate, and is optimized for path expansions.

Neo4j 2025.07

Updates

Feature	Details

Updated
`MATCH (n)-[r]->()`
`WHERE n:$(<expr1>)`
`WITH n, r:$(<expr2>) AS hasType`
`RETURN n:$(<expr3>)`
	Added the ability to dynamically reference node labels and relationship types in places where label expressions are allowed.

Neo4j 2025.06

Removals

Feature	Details
Removed `CREATE DATABASE db OPTIONS { seedCredentials: ... }`	The option `seedCredentials` is removed from the `CREATE DATABASE OPTIONS` map. For more information, see Operations Manual → Database administration → Create databases.

Feature	Details
Removed `CREATE DATABASE db OPTIONS { existingDataSeedInstance: ... }`	The option `existingDataSeedInstance` is removed from the `CREATE DATABASE` `OPTIONS` map and replaced by `existingDataSeedServer`. For more information, see Operations Manual → Database administration → Create databases.

Removed
`CREATE ALIAS composite.`1` FOR DATABASE neo4j`
`USE composite.`1``
	Graph references with separately backticked name parts have been removed. Use parameters or backtick the entire name, e.g. `USE `composite.1``

Removed `RETURN 1 as my\u0085identifier`	The Unicode character `\u0085` (`Next Line`) has been removed for unescaped identifiers and is now considered a whitespace character. To continue using it, escape the identifier by adding backticks (``) around it. This applies to all unescaped identifiers in Cypher, such as label expressions, properties, variable names, or parameters. In the given example, the quoted identifier would be `my\u0085identifier`.
Removed `RETURN 1 as my$Identifier`	The character with the Unicode representation `\u0024` (`$`) has been removed for unescaped identifiers. To continue using it, escape the identifier by adding backticks (``) around the identifier. This applies to all unescaped identifiers in Cypher, such as label expressions, properties, variable names, or parameters. In the given example, the quoted identifier would be `my\u0024identifier`. The following Unicode Characters are removed in identifiers: `\u0000`, `\u0001`, `\u0002`, `\u0003`, `\u0004`, `\u0005`, `\u0006`, `\u0007`, `\u0008`, `\u000E`, `\u000F`, `\u0010`, `\u0011`, `\u0012`, `\u0013`, `\u0014`, `\u0015`, `\u0016`, `\u0017`, `\u0018`, `\u0019`, `\u001A`, `\u001B`, `\u007F`, `\u0080`, `\u0081`, `\u0082`, `\u0083`, `\u0084`, `\u0086`, `\u0087`, `\u0088`, `\u0089`, `\u008A`, `\u008B`, `\u008C`, `\u008D`, `\u008E`, `\u008F`, `\u0090`, `\u0091`, `\u0092`, `\u0093`, `\u0094`, `\u0095`, `\u0096`, `\u0097`, `\u0098`, `\u0099`, `\u009A`, `\u009B`, `\u009C`, `\u009D`, `\u009E`, `\u009F`, `\u0024`, `\u00A2`, `\u00A3`, `\u00A4`, `\u00A5`, `\u00AD`, `\u0600`, `\u0601`, `\u0602`, `\u0603`, `\u0604`, `\u0605`, `\u061C`, `\u06DD`, `\u070F`, `\u08E2`, `\u180E`, `\u200B`, `\u200C`, `\u200D`, `\u200E`, `\u200F`, `\u202A`, `\u202B`, `\u202C`, `\u202D`, `\u202E`, `\u2060`, `\u2061`, `\u2062`, `\u2063`, `\u2064`, `\u2066`, `\u2067`, `\u2068`, `\u2069`, `\u206A`, `\u206B`, `\u206C`, `\u206D`, `\u206E`, `\u206F`, `\u2E2F`, `\uFEFF`, `\uFFF9`, `\uFFFA`, `\uFFFB`

Feature	Details

Removed
`MATCH (n)-[r:REL]->(m) SET n = r;`
`MATCH (n)-[r:REL]->(m) SET n += r;`
`MATCH (n)-[r:REL]->(m) SET r = n;`
`MATCH (n)-[r:REL]->(m) SET r += n;`
	Using a `NODE` or `RELATIONSHIP` instead of a `MAP` on the RHS of a `SET` properties clause is no longer supported. Instead, use the `properties()` function to get the map of properties from nodes or relationships, which can then be used in the `SET` clause.
	`MATCH (n)-[r:REL]->(m) SET n = properties(r)`

Removed
`REVOKE READ {*} ON GRAPH * NODES A FROM missingUser`
`REVOKE ROLE missingRole FROM bob`
	Errors have replaced notifications for impossible `REVOKE` commands where a user, role, or database does not exist.

Removed `MERGE (a {foo:1})-[:T]->(b {foo:a.foo})`	It is no longer possible to specify a property of one entity (node or relationship) by referring to another entity's property within the same `MERGE` clause.

Removed
`CREATE ... INDEX ... OPTIONS { indexProvider: ... }`
`CREATE ... CONSTRAINTS ... OPTIONS { indexProvider: ... }`
	Specifying an index provider in the `OPTIONS` map when creating an index or constraint is no longer supported.

Removed
`db.create.setVectorProperty()`
`db.index.vector.createNodeIndex()`
`dbms.cluster.readReplicaToggle()`
`dbms.cluster.uncordonServer()`
`dbms.quarantineDatabase()`
`dbms.upgrade()`
`dbms.upgradeStatus()`
	These procedures have been removed from Cypher 25. For more information, see the Operations Manual → Procedures.

Deprecations

Feature	Details
Deprecated `CREATE DATABASE db OPTIONS { existingData: ... }`	The option `existingData` is deprecated. For more information, see Operations Manual → Database administration → Create databases.

Updates

Feature	Details

Updated
 	
``
WITH 1 AS g
RETURN COLLECT {
UNWIND [1,2,3] AS x
WITH * WHERE x < 0
WITH count(*) AS agg
RETURN agg + g
} AS x
``
	Imported variables are now correctly handled as constants inside `COLLECT`, `COUNT`, and `EXISTS` subquery expressions. The example query previously returned no results (`[]`) because the imported variable was used incorrectly as an implicit grouping key. It now returns `[1]`, since the variable is not used as a grouping key, allowing `count(*)` to return `0` and `0 + g` to evaluate to `1`.

Updated ``` RETURN $0hello ```	Parameters can start with extended identifier characters (such as numbers), in line with the GQL standard. For more information, see Parameters.
Updated ``` MATCH SHORTEST $param (:A)-[:R]->{0,10}(:B) ``` ``` MATCH p = ANY $param (:A)-[:R]->{0,10}(:B) ``` ``` MATCH SHORTEST $param GROUPS (:A)-[:R]->{0,10}(:B) ```	Parameters can now be used in `SHORTEST` and `ANY` path patterns.
Updated ``` CALL db.schema.nodeTypeProperties() YIELD propertyTypes RETURN propertyTypes; CALL db.schema.relTypeProperties() YIELD propertyTypes RETURN propertyTypes; ```	The column `propertyTypes` returned by the procedures `db.schema.relTypeProperties()` previously returned a list of strings representing the potential Java types for a given property. It now returns a list of strings representing the possible Cypher Types the given property has. For all available Cypher types, see the section on types and their synonyms.
Syntax Updated ``` SHOW NODE PROPERTY UNIQUENESS CONSTRAINTS SHOW RELATIONSHIP PROPERTY UNIQUENESS CONSTRAINTS SHOW PROPERTY UNIQUENESS CONSTRAINTS ```	The constraint type keyword filtering for `SHOW CONSTRAINTS` now allows the optional keyword `PROPERTY` when filtering on property uniqueness constraints. The constraint type column returned is also updated to return `NODE_PROPERTY_UNIQUENESS` and `RELATIONSHIP_PROPERTY_UNIQUENESS`.
Updated ``` SHOW TRANSACTIONS YIELD startTime, clientAddress, outerTransactionId, currentQuery, currentQueryId, parameters, planner, runtime, indexes, currentQueryStartTime, currentQueryElapsedTime, currentQueryCpuTime, currentQueryIdleTime, currentQueryStatus ```	Several `SHOW TRANSACTIONS` columns have been updated: - `startTime` and `currentQueryStartTime` now return a `ZONED DATETIME` instead of a `STRING` representation of a temporal value. - `clientAddress` and `outerTransactionId` now return `null` instead of an empty `STRING` when unavailable. - The current query-related columns — `currentQuery`, `currentQueryId`, `parameters`, `planner`, `runtime`, `indexes`, `currentQueryStartTime`, `currentQueryElapsedTime`, `currentQueryCpuTime`, `currentQueryIdleTime`, and `currentQueryStatus` — now return `null` when no query is executing. For more information, see the Operations Manual → Show and terminate transactions.

Feature	Details

Updated
 	
`USE graph.byName('composite.with.dot.constituent')`
`USE graph.propertiesByName`
`('composite.with.dot.constituent')`
	Graph references in the arguments of the functions `graph.byName` and `graph.propertiesByName` now require no syntactic quotes.
	
	In Cypher 5, if a composite database or constituent name contains dots, those name parts have to be wrapped in quotes to resolve the name correctly, e.g., `USE graph.byName(``composite.with.dot`.constituent')`.

Updated
 	
`CREATE (:Person)`
`MATCH (p:Person)`
`RETURN count(p) AS count`
	Queries no longer require `WITH` to transition between reading and writing operations. For more information, see Clause composition → Read-write queries.

Updated `RETURN replace("hello world", "l", "", 1)`	The `replace()` function now accepts an optional `limit` argument, defining the number of times the search string is replaced.

New

Feature	Details

Introduced
 	
`CREATE [COMPOSITE] DATABASE actors SET DEFAULT LANGUAGE`
`CYPHER 25`
	Set the default Cypher version for a standard or composite database when creating it. The available versions are `CYPHER 25` and `CYPHER 5`. If not specified, the default language for the database is set to the default language of the DBMS. For more information, see the following sections in the Operations Manual:
	
	• Set a default Cypher version for a standard database.
	• Set the default Cypher version for a composite database.

Introduced `CREATE ALIAS `remote-with-default-language``  `FOR DATABASE `northwind-graph-2020`` `AT "neo4j+s://location:7687"` `USER alice` `PASSWORD 'example_secret'` `DEFAULT LANGUAGE CYPHER 25`	Set the default Cypher version for a remote database alias when creating it. The available versions are `CYPHER 5` and `CYPHER 25`. Local database aliases and database aliases in composite databases cannot be assigned a default Cypher version. Local database aliases always have the Cypher version of their target database and database aliases in composite databases always have the Cypher version of the composite database they belong to. For more information, see the Operations Manual → Set a default Cypher version for remote database aliases.
Introduced `ALTER DATABASE movies SET DEFAULT LANGUAGE CYPHER 25`	Alter the default Cypher version of an existing standard or composite database. The available versions are `CYPHER 25` and `CYPHER 5`. For more information, see the Operations Manual → Alter the default Cypher version of an existing database.

Feature	Details
Introduced `ALTER ALIAS `remote-with-default-language` SET DATABASE DEFAULT LANGUAGE CYPHER 25`	Alter the default Cypher version of a remote database alias. The available versions are `CYPHER 25` and `CYPHER 5`. It is not possible to alter the default Cypher version of a local database alias or an alias belonging to a composite database. Local database aliases always have the Cypher version of their target database and aliases belonging to composite databases always have the Cypher version of the composite database. For more information, see the Operations Manual → Alter the default Cypher version of a remote database alias.
Introduced `SHOW DATABASES YIELD name, defaultLanguage`	The new return column `defaultLanguage` for the `SHOW DATABASE` command returns the default language of a database. This column is not returned by default; it can only be returned using `YIELD`. For more information, see the Operations Manual → Show the default Cypher version of a database.
Introduced `SHOW ALIAS `remote-with-default-language` FOR DATABASE YIELD name, defaultLanguage`	The new return column `defaultLanguage` for the `SHOW ALIAS` command returns the default language of a database alias. This column is not returned by default; it can only be returned using `YIELD`. For more information, see the Operations Manual → List database aliases.

Introduced
 	
`CYPHER 25`
`MATCH (n:Person)`
`FILTER n.age < 35`
`RETURN n.name AS name`
	New query option: `CYPHER 25`. Prepending a query with `CYPHER 25` ensures that a query is run with Cypher 25, regardless of the default language of a database. (The ability to select `CYPHER 5` was introduced in Neo4j 5.21).

Introduced
 	
`RETURN 1 AS a`
 	
`NEXT`
 	
`RETURN 1 AS b`
	New `NEXT` keyword used for linear composition of queries. For more information, see Sequential queries (`NEXT`).

Introduced
 	
`MATCH (s:Supplier)-[:SUPPLIES]->(p:Product)`
`LET supplier =  s.name`
`RETURN supplier, p.name AS product`
	New `LET` clause used to bind values to variables.

Introduced
 	
`UNWIND [1, 2, 3, 4, 5, 6] AS x`
`FILTER x > 2`
`RETURN x`
 	
`LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row`
`FILTER row.Id IS NOT NULL`
`MERGE (c:Company {id: row.Id})`
	New `FILTER` clause used to filter queries, similar to `WHERE`.

Feature	Details

Introduced
 	
`WHEN false THEN RETURN 1 AS x`
`WHEN true THEN RETURN 2 AS x`
`ELSE RETURN 3 AS x`
 	
`MATCH (n:Person)`
`OPTIONAL MATCH (n)-[:KNOWS]->(m)`
`CALL (*) {`
`  WHEN m IS NULL THEN {`
`  CREATE (f: Person {name: 'Peter', age: n.age}),`
`  (n)-[:KNOWS]->(f)`
`  RETURN f, n.name AS newConnection`
`  }`
`}`
`RETURN f.name AS newNode,`
`  collect(newConnection) AS newConnections`
	Introduction of `WHEN` / `ELSE` branches which enable the composition of conditional queries, similar to the `IF` statement in other programming languages. For more information, see Conditional queries (`WHEN`).

Introduced
 	
`{`
`  MATCH (n:Actor)`
`  RETURN n.name AS name`
`  UNION`
`  MATCH (n:Director)`
`  RETURN n.name AS name`
`}`
`UNION ALL`
`MATCH (n:Movie)`
`RETURN n.title AS name`
	`UNION [DISTINCT]` and `UNION ALL` can now be combined in the same query by using curly braces. For more information, see Combining `UNION` and `UNION ALL`.

Introduced `CREATE DATABASE db OPTIONS { seedRestoreUntil: ... }`	The option `seedRestoreUntil` can now be specified in the `CREATE DATABASE` `OPTIONS` map. This allows a database to be seeded up to a specific date or transaction ID. For more information, see Operations Manual → Database administration → Create a database from a URI.
Introduced `CREATE DATABASE db OPTIONS { seedSourceDatabase: ... }`	You can specify the name of a source database if the `seedURI` points to a folder containing backups for multiple databases. For more information, see Operations Manual → Database administration → Create databases.
Introduced `RETURN cosh(0.5), coth(0.5), sinh(0.5), tanh(0.5)`	Introduction of four new hyperbolic trigonometric Cypher functions. For more information, see Mathematical functions - trigonometric.

Introduced
 	
`MATCH (n)`
`RETURN ALL n.prop AS prop`
	The keyword `ALL` can now be added after a RETURN as the explicit form of a `RETURN` without duplicate removal.

Introduced
 	
`MATCH (n)`
`WITH ALL n.prop AS prop`
`RETURN prop`
	The keyword `ALL` can now be added after a WITH as the explicit form of a `WITH` without duplicate removal.

Feature	Details

Introduced
 	
`MATCH REPEATABLE ELEMENTS p = (:B)-->{,5}()`
`RETURN [n IN nodes(p) | n.q] AS nodes`
	Introduction of `REPEATABLE ELEMENTS`, which ensures that relationships as well as nodes can be repeatedly traversed in a graph pattern.

Introduced
 	
`MATCH DIFFERENT RELATIONSHIPS p = (:B)-->{,5}()`
`RETURN [n IN nodes(p) | n.q] AS nodes`
	New keyword, `DIFFERENT RELATIONSHIPS`, which enables explicitly specifying Cypher's default behavior of not allowing relationships to be re-traversed in a graph pattern.

Appendix

Appendix A: Cypher styleguide

The purpose of the Cypher styleguide is to make queries as easy to read as possible.

For rules and recommendations for naming of labels, relationship types and properties, please see the Naming rules and recommendations.

The best way to make sure your queries follow the styling rules is to use `cypherfmt`, the official Cypher formatter. It is available as part of the Neo4j VS Code Extension, and as a command line tool through the Neo4j Cypher Language Support npm package.

General recommendations

- When using Cypher language constructs in prose, use a `monospaced` font and follow the styling rules.
- When referring to labels and relationship types, the colon should be included as follows: `:Label`, `:REL_TYPE`.
- When referring to functions, use lower camel case and parentheses. For example: `toString()`.
- If you are storing Cypher statements in a separate file, use the file extension `.cypher`.

Indentation and line breaks

- Start a new clause on a new line.

Bad

```
MATCH (n) WHERE n.name CONTAINS 's' RETURN n.name
```

Good

```
MATCH (n)
WHERE n.name CONTAINS 's'
RETURN n.name
```

- Indent `ON CREATE` and `ON MATCH` with two spaces. Put `ON CREATE` before `ON MATCH` if both are present.

Bad

```
MERGE (n) ON CREATE SET n.prop = 0
MERGE (a:A)-[:T]->(b:B)
ON MATCH SET b.name = 'you'
ON CREATE SET a.name = 'me'
RETURN a.prop
```

Good

```
MERGE (n)
  ON CREATE SET n.prop = 0
```

```
MERGE (a:A)-[:T]->(b:B)
  ON CREATE SET a.name = 'me'
  ON MATCH SET b.name = 'you'
RETURN a.prop
```

- Start a subquery on a new line after the opening brace, indented with two (additional) spaces. Leave the closing brace on its own line.

Bad

```
MATCH (a:A)
WHERE
  EXISTS { MATCH (a)-->(b:B) WHERE b.prop = 'yellow' }
RETURN a.foo
```

Also bad

```
MATCH (a:A)
WHERE EXISTS
{MATCH (a)-->(b:B)
WHERE b.prop = 'yellow'}
RETURN a.foo
```

Good

```
MATCH (a:A)
WHERE EXISTS {
  MATCH (a)-->(b:B)
  WHERE b.prop = 'yellow'
}
RETURN a.foo
```

- Do not break the line if the simplified subquery form is used.

Bad

```
MATCH (a:A)
WHERE EXISTS {
  (a)-->(b:B)
}
RETURN a.prop
```

Good

```
MATCH (a:A)
WHERE EXISTS { (a)-->(b:B) }
RETURN a.prop
```

- Limit line length to 80 characters if possible. Line breaks should be applied starting from the outermost group, with each nested group breaking until either the line fits or no more line breaks can be inserted. Add two spaces of indentation for each nested broken group.

Bad

```
MATCH (n)
WHERE n.prop <> 'a' AND n.prop <> 'b' AND n.prop <> 'c' AND n.prop <> 'd' AND n.prop <> 'e'
RETURN n
```

Good

```
MATCH (n)
WHERE
  n.prop <> 'a' AND
  n.prop <> 'b' AND
  n.prop <> 'c' AND
  n.prop <> 'd' AND
  n.prop <> 'e'
RETURN n
```

- Start `ORDER BY` and `LIMIT` clauses on new lines.

Bad

```
MATCH (n)
RETURN 5 ORDER BY n.prop LIMIT 10
```

Good

```
MATCH (n)
RETURN 5
ORDER BY n.prop
LIMIT 10
```

- A group containing a `CASE` expression should always break, and each `WHEN`, `ELSE`, and `END` should be put on a new line. Additionally, `WHEN` and `ELSE` should add two spaces of indentation.

Bad

```
MATCH (n:Person {name: 'Alice'})
RETURN CASE WHEN n.age >= 18 THEN 'Adult' ELSE 'Minor' END AS ageGroup
```

Good

```
MATCH (n:Person {name: 'Alice'})
RETURN
  CASE
    WHEN n.age >= 18 THEN 'Adult'
    ELSE 'Minor'
  END AS ageGroup
```

- When a line ends with a list or map literal that does not fit within the maximum line width, place the opening bracket or brace on the same line as the clause that introduces it. Start each element of the literal on a new line, indented two spaces further than the line containing the opening bracket or brace. Close the literal at the original indentation level.

Bad

```
RETURN
  [
    "Alice",
    "Bob",
    "Charlie",
    "David",
    "Eve",
    "Frank",
    "Grace",
    "Heidi",
    "Ivan",
    "Judy"
```

```
  ]
```

Good

```
RETURN [
  "Alice",
  "Bob",
  "Charlie",
  "David",
  "Eve",
  "Frank",
  "Grace",
  "Heidi",
  "Ivan",
  "Judy"
]
```

- A single blank line may be used to separate clauses, queries or comments.

Good

```
MATCH (n)-[r]->(m)
RETURN n, r, m
```

Also good

```
MATCH (n)-[r]->(m)

RETURN n, r, m
```

Casing

- Write keywords in upper case.

Bad

```
match (p:Person)
where p.name starts with 'Ma'
return p.name
```

Good

```
MATCH (p:Person)
WHERE p.name STARTS WITH 'Ma'
RETURN p.name
```

- Write the value `null` in lower case.

Bad

```
WITH NULL AS n1, Null AS n2
RETURN n1 IS NULL AND n2 IS NOT NULL
```

Good

```
WITH null AS n1, null AS n2
RETURN n1 IS NULL AND n2 IS NOT NULL
```

- Write `BOOLEAN` literals (`true` and `false`) in lower case.

Bad

```
WITH TRUE AS b1, False AS b2
RETURN b1 AND b2
```

Good

```
WITH true AS b1, false AS b2
RETURN b1 AND b2
```

- Use camel case, starting with a lower-case character, for:
  - functions
  - properties
  - variables
  - parameters

Bad

```
CREATE (N:Label {Prop: 0})
WITH N, RAND() AS Rand, $pArAm AS MAP
RETURN Rand, MAP.property_key, count(N)
```

Good

```
CREATE (n:Label {prop: 0})
WITH n, rand() AS rand, $param AS map
RETURN rand, map.propertyKey, count(n)
```

Spacing

- For literal maps:
  - No space between the opening brace and the first key
  - No space between key and colon
  - One space between colon and value
  - No space between value and comma
  - One space between comma and next key
  - No space between the last value and the closing brace

Bad

```
WITH { key1 :'value' ,key2  :  42 } AS map
RETURN map
```

Good

```
WITH {key1: 'value', key2: 42} AS map
```

```
RETURN map
```

- One space between label/type predicates and property predicates in patterns.

Bad

```
MATCH (p:Person{property: -1})-[:KNOWS   {since: 2016}]->()
RETURN p.name
```

Good

```
MATCH (p:Person {property: -1})-[:KNOWS {since: 2016}]->()
RETURN p.name
```

- No space in patterns.

Bad

```
MATCH (:Person) --> (:Vehicle)
RETURN count(*)
```

Good

```
MATCH (:Person)-->(:Vehicle)
RETURN count(*)
```

- Use a wrapping space around operators.

Bad

```
MATCH p=(s)-->(e)
WHERE s.name<>e.name
RETURN length(p)
```

Good

```
MATCH p = (s)-->(e)
WHERE s.name <> e.name
RETURN length(p)
```

- No space in label predicates.

Bad

```
MATCH (person    : Person  :  Owner   )
RETURN person.name
```

Good

```
MATCH (person:Person:Owner)
RETURN person.name
```

- Use a space after each comma in lists and enumerations.

Bad

```
MATCH (),()
WITH ['a','b',3.14] AS list
RETURN list,2,3,4
```

Good

```
MATCH (), ()
WITH ['a', 'b', 3.14] AS list
RETURN list, 2, 3, 4
```

- No padding space within function call parentheses.

Bad

```
RETURN split( 'original', 'i' )
```

Good

```
RETURN split('original', 'i')
```

- Use padding space within simple subquery expressions.

Bad

```
MATCH (a:A)
WHERE EXISTS {(a)-->(b:B)}
RETURN a.prop
```

Good

```
MATCH (a:A)
WHERE EXISTS { (a)-->(b:B) }
RETURN a.prop
```

Patterns

- When patterns wrap lines, break after arrows, not before.

Bad

```
MATCH (:Person)-->(vehicle:Car)-->(:Company)
      <--(:Country)
RETURN count(vehicle)
```

Good

```
MATCH (:Person)-->(vehicle:Car)-->(:Company)<--
      (:Country)
RETURN count(vehicle)
```

- Use anonymous nodes and relationships when the variable would not be used.

Bad

```
MATCH (kate:Person {name: 'Kate'})-[r:LIKES]-(c:Car)
RETURN c.type
```

Good

```
MATCH (:Person {name: 'Kate'})-[:LIKES]-(c:Car)
RETURN c.type
```

- Chain patterns together to avoid repeating variables.

Bad

```
MATCH (:Person)-->(vehicle:Car), (vehicle:Car)-->(:Company)
RETURN count(vehicle)
```

Good

```
MATCH (:Person)-->(vehicle:Car)-->(:Company)
RETURN count(vehicle)
```

- Put named nodes before anonymous nodes.

Bad

```
MATCH ()-->(vehicle:Car)-->(manufacturer:Company)
WHERE manufacturer.foundedYear < 2000
RETURN vehicle.mileage
```

Good

```
MATCH (manufacturer:Company)<--(vehicle:Car)<--()
WHERE manufacturer.foundedYear < 2000
RETURN vehicle.mileage
```

- Keep anchor nodes at the beginning of the `MATCH` clause.

Bad

```
MATCH (:Person)-->(vehicle:Car)-->(manufacturer:Company)
WHERE manufacturer.foundedYear < 2000
RETURN vehicle.mileage
```

Good

```
MATCH (manufacturer:Company)<--(vehicle:Car)<--(:Person)
WHERE manufacturer.foundedYear < 2000
RETURN vehicle.mileage
```

- Prefer outgoing (left to right) pattern relationships to incoming pattern relationships.

Bad

```
MATCH (:Country)-->(:Company)<--(vehicle:Car)<--(:Person)
RETURN vehicle.mileage
```

Good

```
MATCH (:Person)-->(vehicle:Car)-->(:Company)<--(:Country)
RETURN vehicle.mileage
```

Meta-characters

- Use single quotes, `'`, for literal `STRING` values.

Bad

```
RETURN "Cypher"
```

Good

```
RETURN 'Cypher'
```

- Disregard this rule for literal `STRING` values that contain a single quote character. If the `STRING` has both, use the form that creates the fewest escapes. In the case of a tie, prefer single quotes.

Bad

```
RETURN 'Cypher\'s a nice language', "Mats' quote: \"statement\""
```

Good

```
RETURN "Cypher's a nice language", 'Mats\' quote: "statement"'
```

- Avoid using characters and keywords that require the input to be quoted with backticks.

Bad

```
MATCH (`odd-ch@racter$`:`Spaced Label` {`&property`: 42})
RETURN labels(`odd-ch@racter$`)
```

Good

```
MATCH (node:NonSpacedLabel {property: 42})
RETURN labels(node)
```

- Do not use a semicolon at the end of the statement.

Bad

```
RETURN 1;
```

Good

```
RETURN 1
```

Full Cypher grammar

```
<program> ::=
    <standalone procedure call>
  | <procedure specification>
  | <standalone operation statement>

<procedure specification> ::=
  <statement block>

<nested procedure specification> ::=
  <left brace> <procedure specification> <right brace>

<statement block> ::=
  <statement> [ <next statement>... ]

<next statement> ::=
  "NEXT" <statement>

<standalone operation statement> ::=
    <standalone catalog statement>
  | <standalone operation control statement>
  | <standalone access control statement>

<statement> ::=
    <composite statement>
  | <conditional statement>

<composite statement> ::=
  <linear statement> [ { <composite conjunction> <linear statement> }... ]

<composite conjunction> ::=
  "UNION" [ <set quantifier> ]

<conditional statement> ::=
  <conditional statement when clause>... [ <conditional statement else clause> ]

<conditional statement when clause> ::=
  "WHEN" <search condition> "THEN" <conditional statement result>

<conditional statement else clause> ::=
  "ELSE" <conditional statement result>

<conditional statement result> ::=
    <nested procedure specification>
  | <linear statement>

<linear statement> ::=
  [ <use graph clause> ] { <nested procedure specification> | <statement chain> }

<statement chain> ::=
    <primitive result statement>
  | <ambient primitive statement chain> [ <focused primitive statement chain>... ] [ <primitive result
statement> ]

<focused primitive statement chain> ::=
  <use graph clause> <ambient primitive statement chain>

<ambient primitive statement chain> ::=
  <primitive statement>...

<primitive statement> ::=
    <primitive query statement>
  | <primitive data load statement>
  | <primitive data update statement>
  | <call procedure statement>
  | <primitive operation statement>

<primitive query statement> ::=
    <match statement>
  | <unwind statement>
  | <for statement>
  | <let statement>
  | <filter statement>
  | <order by and page statement>
  | <with statement>

<match statement> ::=
```

```
    <optional match statement>
  | <simple match statement>

<optional match statement> ::=
  "OPTIONAL" <optional match operand>

<optional match operand> ::=
  <simple match statement>

<simple match statement> ::=
  "MATCH" <graph pattern binding table>

<unwind statement> ::=
  "UNWIND" <value expression> "AS" <binding variable>

<for statement> ::=
  "FOR" <binding variable> "IN" <value expression>

<let statement> ::=
  "LET" <let variable definition list>

<let variable definition list> ::=
  <let variable definition> [ { <comma> <let variable definition> }... ]

<let variable definition> ::=
  <binding variable> <equals operator> <value expression>

<filter statement> ::=
  "FILTER" [ "WHERE" ] <search condition>

<order by and page statement> ::=
  <order by and page clause>

<with statement> ::=
  "WITH" <return statement body> [ <order by and page clause> ] [ <where clause> ]

<primitive data load statement> ::=
  <load csv statement>

<load csv statement> ::=
  "LOAD CSV" [ <csv headers option> ] <csv source> [ <csv field terminator> ]

<csv headers option> ::=
  "WITH HEADERS"

<csv source> ::=
  "FROM" <value expression> "AS" <binding variable>

<csv field terminator> ::=
  "FIELDTERMINATOR" <character string literal>

<primitive data update statement> ::=
    <create or insert statement>
  | <merge statement>
  | <set statement>
  | <remove statement>
  | <delete statement>
  | <foreach statement>

<create or insert statement> ::=
    <create statement>
  | <insert statement>

<create statement> ::=
  "CREATE" <create graph pattern>

<insert statement> ::=
  "INSERT" <insert graph pattern>

<merge statement> ::=
  "MERGE" <merge graph pattern> [ <merge action> ]

<merge action> ::=
  "ON" { "MATCH" | "CREATE" } <set statement>

<set statement> ::=
  "SET" <set item list>
```

```
<set item list> ::=
  <set item> [ { <comma> <set item> }... ]

<set item> ::=
    <set all properties item>
  | <add all properties item>
  | <set labels item>
  | <set property item>

<set all properties item> ::=
  <binding variable reference> <equals operator> <value expression>

<add all properties item> ::=
  <binding variable reference> <plus equals> <value expression>

<set labels item> ::=
  <binding variable reference> <node label set specification for set and remove>

<set property item> ::=
  <postfix expression> <equals operator> <value expression>

<remove statement> ::=
  "REMOVE" <remove item list>

<remove item list> ::=
  <remove item> [ { <comma> <remove item> }... ]

<remove item> ::=
    <remove labels item>
  | <remove property item>

<remove labels item> ::=
  <binding variable reference> <node label set specification for set and remove>

<remove property item> ::=
  <postfix expression>

<delete statement> ::=
  [ "DETACH" | "NODETACH" ] "DELETE" <delete item list>

<delete item list> ::=
  <delete item> [ { <comma> <delete item> }... ]

<delete item> ::=
  <value expression>

<foreach statement> ::=
  "FOREACH" <left paren> <foreach element source> <foreach action> <right paren>

<foreach element source> ::=
  <binding variable reference> "IN" <value expression>

<foreach action> ::=
  <vertical bar> <foreach action statement>...

<foreach action statement> ::=
    <create or insert statement>
  | <merge statement>
  | <set statement>
  | <remove statement>
  | <delete statement>

<call procedure statement> ::=
  [ "OPTIONAL" ] "CALL" <procedure call>

<standalone procedure call> ::=
  "CALL" <procedure reference> [ <explicit procedure arguments> ] [ <standalone procedure call yield
clause> ]

<procedure call> ::=
    <inline procedure call>
  | <named procedure call>

<inline procedure call> ::=
  [ <variable scope clause> ] <nested procedure specification> [ <in transactions> ]
```

```
<variable scope clause> ::=
  <left paren> [ <binding variable reference list> ] <right paren>

<in transactions> ::=
  "IN" [ [ <value expression> ] "CONCURRENT" ] "TRANSACTIONS" [ <in transactions parameters>... ]

<in transactions parameters> ::=
    <in transactions batch parameters>
  | <in transactions disjoint by parameters>
  | <in transactions error parameters>
  | <in transactions error retry parameters>
  | <in transactions report parameters>

<in transactions batch parameters> ::=
  "OF" <value expression> { "ROW" | "ROWS" }

<in transactions disjoint by parameters> ::=
    "DISJOINT BY NONE"
  | "DISJOINT BY AUTO"
  | "DISJOINT BY" <disjoint by expression group>

<disjoint by expression group> ::=
  <left paren> <disjoint by expression list> <right paren>

<disjoint by expression list> ::=
  <value expression> [ { <comma> <value expression> }... ]

<in transactions error parameters> ::=
  "ON ERROR" <in transactions error action>

<in transactions error retry parameters> ::=
  "ON ERROR RETRY" [ <in transactions error retry time> ] [ "THEN" <in transactions error action> ]

<in transactions error action> ::=
    "CONTINUE"
  | "BREAK"
  | "FAIL"

<in transactions error retry time> ::=
  "FOR" <value expression> <seconds keywords>

<in transactions report parameters> ::=
  "REPORT STATUS AS" <binding variable>

<named procedure call> ::=
  <procedure reference> <explicit procedure arguments> [ <yield clause> ]

<explicit procedure arguments> ::=
  <left paren> [ <procedure argument list> ] <right paren>

<procedure argument list> ::=
  <procedure argument> [ { <comma> <procedure argument> }... ]

<procedure argument> ::=
  <value expression>

<yield clause> ::=
  "YIELD" <yield item list> [ <where clause> ]

<standalone procedure call yield clause> ::=
  "YIELD" { <yield item list> | <asterisk> }

<yield item list> ::=
  <yield item> [ { <comma> <yield item> }... ]

<yield item> ::=
  <yield item name> [ <yield item alias> ]

<yield item name> ::=
  <field name>

<yield item alias> ::=
  "AS" <binding variable>

<primitive result statement> ::=
    <return statement>
  | "FINISH"
```

```
<return statement> ::=
  "RETURN" <return statement body> [ <order by and page clause> ]

<return statement body> ::=
  [ <set quantifier> ] <return item list> [ <group by clause> ]

<return item list> ::=
  { <asterisk> | <return item> } [ { <comma> <return item> }... ]

<return item> ::=
  <value expression> [ <return item alias> ]

<return item alias> ::=
  "AS" <identifier>

<order by and page clause> ::=
    <order by clause> [ <offset clause> ] [ <limit clause> ]
  | <offset clause> [ <limit clause> ]
  | <limit clause>

<order by clause> ::=
  "ORDER BY" <sort specification list>

<sort specification list> ::=
  <sort specification> [ { <comma> <sort specification> }... ]

<sort specification> ::=
  <sort key> [ <ordering specification> ]

<sort key> ::=
  <value expression>

<ordering specification> ::=
    <ascending order>
  | <descending order>

<ascending order> ::=
    "ASC"
  | "ASCENDING"

<descending order> ::=
    "DESC"
  | "DESCENDING"

<offset clause> ::=
  <offset synonym> <value expression>

<offset synonym> ::=
    "SKIP"
  | "OFFSET"

<limit clause> ::=
  "LIMIT" <value expression>

<group by clause> ::=
  "GROUP BY" <grouping element list>

<grouping element list> ::=
    <grouping element> [ { <comma> <grouping element> }... ]
  | <empty grouping set>
  | "ALL"

<grouping element> ::=
  <value expression>

<empty grouping set> ::=
  <left paren> <right paren>

<standalone catalog statement> ::=
    <catalog create statement>
  | <catalog drop statement>
  | <catalog alter statement>
  | <standalone catalog show statement>

<command options> ::=
  "OPTIONS" <map value or parameter specification>
```

```
<set option> ::=
  "OPTION" <identifier> <value expression>

<remove option> ::=
  "OPTION" <identifier>

<wait clause> ::=
    "WAIT" <unsigned decimal integer> <seconds keywords>
  | "NOWAIT"

<if not exists> ::=
  "IF NOT EXISTS"

<if exists> ::=
  "IF EXISTS"

<property names specification> ::=
    <property name inner specification>
  | <parenthesized property names specification>

<parenthesized property names specification> ::=
  <left paren> <property name specification list> <right paren>

<bracketed property names specification> ::=
  <left bracket> <property name specification list> <right bracket>

<property name specification list> ::=
  <property name inner specification> [ { <comma> <property name inner specification> }... ]

<property name specification> ::=
    <property name inner specification>
  | <left paren> <property name inner specification> <right paren>

<property name inner specification> ::=
  <binding variable reference> <period> <property name>

<catalog create statement> ::=
    <create alias statement>
  | <create auth rule statement>
  | <create constraint statement>
  | <create database statement>
  | <create index statement>
  | <create role statement>
  | <create user statement>

<create alias statement> ::=
  "CREATE" [ "OR REPLACE" ] "ALIAS" <alias name specification> [ <if not exists> ] "FOR DATABASE" <alias
target>

<alias target> ::=
    <alias remote target>
  | <alias local target>

<alias local target> ::=
  <alias target name specification> [ <alias target properties> ]

<alias remote target> ::=
  <alias target name specification> <remote target connection> [ <alias target properties> ]

<remote target connection> ::=
  <remote target connection host> <remote target connection credentials> [ <remote target connection
driver settings> ] [ <default language specification> ]

<remote target connection host> ::=
  "AT" <alias target name specification>

<remote target connection credentials> ::=
    <remote target connection user> <remote target connection password>
  | <remote target credential forwarding>

<remote target connection user> ::=
  "USER" <user name specification>

<remote target connection password> ::=
  "PASSWORD" <password value specification>
```

```
<remote target credential forwarding> ::=
  "OIDC CREDENTIAL FORWARDING"

<remote target connection driver settings> ::=
  "DRIVER" <map value or parameter specification>

<alias target properties> ::=
  "PROPERTIES" <map value or parameter specification>

<create auth rule statement> ::=
  "CREATE" [ "OR REPLACE" ] "AUTH RULE" <auth rule name specification> [ <if not exists> ] <auth rule set
condition> [ <auth rule set enabled> ]

<auth rule set condition> ::=
  "SET CONDITION" <value expression>

<auth rule set enabled> ::=
  "SET ENABLED" <boolean literal>

<create constraint statement> ::=
  "CREATE CONSTRAINT" [ <constraint name specification> ] [ <if not exists> ] <constraint specification> [
<command options> ]

<constraint specification> ::=
    <node constraint specification>
  | <relationship constraint specification>

<node constraint specification> ::=
    <node key constraint>
  | <node uniqueness constraint>
  | <node property existence constraint>
  | <node property type constraint>

<relationship constraint specification> ::=
    <relationship key constraint>
  | <relationship uniqueness constraint>
  | <relationship property existence constraint>
  | <relationship property type constraint>

<node key constraint> ::=
  <node constraint for> <node key constraint require>

<node uniqueness constraint> ::=
  <node constraint for> <node uniqueness constraint require>

<node property existence constraint> ::=
  <node constraint for> <property existence constraint require>

<node property type constraint> ::=
  <node constraint for> <property type constraint require>

<relationship key constraint> ::=
  <relationship constraint for> <relationship key constraint require>

<relationship uniqueness constraint> ::=
  <relationship constraint for> <relationship uniqueness constraint require>

<relationship property existence constraint> ::=
  <relationship constraint for> <property existence constraint require>

<relationship property type constraint> ::=
  <relationship constraint for> <property type constraint require>

<node constraint for> ::=
  "FOR" <command node pattern>

<relationship constraint for> ::=
  "FOR" <command relationship pattern>

<node key constraint require> ::=
  "REQUIRE" <property names specification> "IS" [ "NODE" ] "KEY"

<relationship key constraint require> ::=
  "REQUIRE" <property names specification> "IS" [ <relationship singular keywords> ] "KEY"

<node uniqueness constraint require> ::=
  "REQUIRE" <property names specification> "IS" [ "NODE" ] "UNIQUE"
```

```
<relationship uniqueness constraint require> ::=
  "REQUIRE" <property names specification> "IS" [ <relationship singular keywords> ] "UNIQUE"

<property existence constraint require> ::=
  "REQUIRE" <property name specification> "IS NOT NULL"

<property type constraint require> ::=
  "REQUIRE" <property name specification> <is typed> <value type>

<is typed> ::=
    [ "IS" ] <double colon>
  | "IS TYPED"

<create database statement> ::=
    <create standard database statement>
  | <create composite database statement>
  | <create replica database statement>

<create standard database statement> ::=
  "CREATE" [ "OR REPLACE" ] "DATABASE" <database name specification> [ <if not exists> ] [ [ "SET" ]
<default language specification> ] [ <topology specification> ] [ <command options> ] [ <wait clause> ]

<create composite database statement> ::=
  "CREATE" [ "OR REPLACE" ] "COMPOSITE DATABASE" <database name specification> [ <if not exists> ] [ [
"SET" ] <default language specification> ] [ <command options> ] [ <wait clause> ]

<create replica database statement> ::=
  "CREATE" [ "OR REPLACE" ] "REPLICA DATABASE" <database name specification> [ <if not exists> ] [ [ "SET"
] <default language specification> ] [ <topology specification> ] [ <command options> ] [ <wait clause> ]

<default language specification> ::=
  "DEFAULT LANGUAGE CYPHER" <unsigned decimal integer>

<topology specification> ::=
    [ "SET" ] <simple topology specification>
  | <shard topology specification>

<simple topology specification> ::=
  "TOPOLOGY" { <primary topology> [ <secondary topology> ] | <secondary topology> [ <primary topology> ] }

<primary topology> ::=
  <unsigned decimal integer specification> <primary keywords>

<secondary topology> ::=
  <unsigned decimal integer specification> <secondary keywords>

<shard topology specification> ::=
  [ [ "SET" ] <graph shard> ] [ "SET" ] <property shard>

<graph shard> ::=
  "GRAPH SHARD" <left brace> [ [ "SET" ] <simple topology specification> ] <right brace> <property shard>

<property shard> ::=
  "PROPERTY" <shard keywords> <left brace> "COUNT" <unsigned decimal integer specification> [ [ "SET" ]
<replica topology> ] <right brace>

<replica topology> ::=
  "TOPOLOGY" <unsigned decimal integer specification> <replica keywords>

<create index statement> ::=
  "CREATE" <index specification> [ <command options> ]

<index specification> ::=
    <fulltext index>
  | <lookup index>
  | <point index>
  | <range index>
  | <text index>
  | <vector index>

<fulltext index> ::=
  "FULLTEXT" <index prefix> <multi-entity-token index for> <fulltext index on>

<lookup index> ::=
  "LOOKUP" <index prefix> { <node lookup index for and on> | <relationship lookup index for and on> }
```

```
<point index> ::=
  "POINT" <index prefix> <index for> <single property index on>

<range index> ::=
  "RANGE" <index prefix> <index for> <composite index on>

<text index> ::=
  "TEXT" <index prefix> <index for> <single property index on>

<vector index> ::=
  "VECTOR" <index prefix> <multi-entity-token index for> <single property index on> [ <with properties> ]

<index prefix> ::=
  "INDEX" [ <index name specification> ] [ <if not exists> ]

<index for> ::=
  "FOR" <command element pattern>

<composite index on> ::=
  "ON" <property names specification>

<single property index on> ::=
  "ON" <property name specification>

<node lookup index for and on> ::=
  <node lookup index for> <node lookup index on>

<node lookup index for> ::=
  "FOR" <lookup index node pattern>

<node lookup index on> ::=
  "ON EACH" <function name> <left paren> <binding variable> <right paren>

<relationship lookup index for and on> ::=
  <relationship lookup index for> <relationship lookup index on>

<relationship lookup index for> ::=
  "FOR" <lookup index relationship pattern>

<relationship lookup index on> ::=
  "ON" [ "EACH" ] <function name> <left paren> <binding variable> <right paren>

<multi-entity-token index for> ::=
  "FOR" <multi-entity-token index element pattern>

<fulltext index on> ::=
  "ON EACH" <bracketed property names specification>

<with properties> ::=
  "WITH" <bracketed property names specification>

<create role statement> ::=
  "CREATE" [ "OR REPLACE" ] [ "IMMUTABLE" ] "ROLE" <role name specification> [ <if not exists> ] [ <as
copy of role> ]

<as copy of role> ::=
  "AS COPY OF" <role name specification>

<create user statement> ::=
  "CREATE" [ "OR REPLACE" ] "USER" <user name specification> [ <if not exists> ] <user create set
clause>...

<user create set clause> ::=
  "SET" <user create set item>

<user create set item> ::=
    <user set password>
  | <user set password change required>
  | <user set status>
  | <user set home database>
  | <user set authentication provider>
  | <user set tags clause>

<user set password> ::=
  <password specification> [ <password change required specification> ]

<password specification> ::=
```

```
  [ "PLAINTEXT" | "ENCRYPTED" ] "PASSWORD" <password value specification>

<user set password change required> ::=
  <password change required specification>

<password change required specification> ::=
  "PASSWORD" <change required specification>

<change required specification> ::=
  "CHANGE" [ "NOT" ] "REQUIRED"

<user set status> ::=
  "STATUS" { "SUSPENDED" | "ACTIVE" }

<user set home database> ::=
  "HOME DATABASE" <alias or database name specification>

<user set authentication provider> ::=
  "AUTH" [ "PROVIDER" ] <character string literal> <user set authentication provider attributes>

<user set authentication provider attributes> ::=
  <left brace> <user set authentication provider attribute>... <right brace>

<user set authentication provider attribute> ::=
  "SET" <user authentication provider attribute>

<user authentication provider attribute> ::=
    <user authentication provider id>
  | <user authentication provider password>
  | <user authentication provider password change required>

<user authentication provider id> ::=
  "ID" <character string value specification>

<user authentication provider password> ::=
  <password specification>

<user authentication provider password change required> ::=
  <password change required specification>

<user set tags clause> ::=
  <explicit user tags>

<explicit user tags> ::=
  <tag keywords> <character string or string list value specification>

<catalog drop statement> ::=
    <drop alias statement>
  | <drop auth rule statement>
  | <drop constraint statement>
  | <drop database statement>
  | <drop index statement>
  | <drop role statement>
  | <drop server statement>
  | <drop user statement>

<drop alias statement> ::=
  "DROP ALIAS" <alias name specification> [ <if exists> ] "FOR DATABASE"

<drop auth rule statement> ::=
  "DROP AUTH RULE" <auth rule name specification> [ <if exists> ]

<drop constraint statement> ::=
  "DROP CONSTRAINT" <constraint name specification> [ <if exists> ]

<drop database statement> ::=
  "DROP" [ "COMPOSITE" ] "DATABASE" <alias or database name specification> [ <if exists> ] [ <drop
database alias action> ] [ <drop database data action> ] [ <wait clause> ]

<drop database alias action> ::=
    "RESTRICT"
  | "CASCADE" <alias keywords>

<drop database data action> ::=
    <drop database dump data action>
  | <drop database destroy data action>
```

```
<drop database dump data action> ::=
  "DUMP DATA"

<drop database destroy data action> ::=
  "DESTROY DATA"

<drop index statement> ::=
  "DROP INDEX" <index name specification> [ <if exists> ]

<drop role statement> ::=
  "DROP ROLE" <role name specification> [ <if exists> ]

<drop server statement> ::=
  "DROP SERVER" <server name specification>

<drop user statement> ::=
  "DROP USER" <user name specification> [ <if exists> ]

<catalog alter statement> ::=
    <alter alias statement>
  | <alter auth rule statements>
  | <alter current graph type statement>
  | <alter database statement>
  | <alter role statements>
  | <alter server statements>
  | <alter user statements>

<alter role statements> ::=
  <rename role statement>

<alter server statements> ::=
    <alter server statement>
  | <rename server statement>

<alter user statements> ::=
    <alter current user statement>
  | <alter user statement>
  | <alter users statement>
  | <rename user statement>

<alter auth rule statements> ::=
    <alter auth rule statement>
  | <rename auth rule statement>

<alter alias statement> ::=
  "ALTER ALIAS" <alias name specification> [ <if exists> ] "SET DATABASE" <alias alter target>

<alias alter target> ::=
    <alias alter local target item>...
  | <alias alter remote target item>...

<alias alter local target item> ::=
    <alias target database name>
  | <alias target properties>

<alias alter remote target item> ::=
    <alias target database name> <remote target connection host>
  | <remote target connection user>
  | <remote target connection password>
  | <remote target connection driver settings>
  | <default language specification>
  | <alias target properties>

<alias target database name> ::=
  "TARGET" <alias target name specification>

<rename auth rule statement> ::=
  "RENAME AUTH RULE" <auth rule name specification> [ <if exists> ] "TO" <auth rule name specification>

<alter auth rule statement> ::=
  "ALTER AUTH RULE" <auth rule name specification> [ <if exists> ] <auth rule set clause>...

<auth rule set clause> ::=
    <auth rule set condition>
  | <auth rule set enabled>

<alter current graph type statement> ::=
```

```
  "ALTER CURRENT GRAPH TYPE" <alter current graph type action>

<alter current graph type action> ::=
    <alter current graph type add>
  | <alter current graph type alter>
  | <alter current graph type drop>
  | <alter current graph type set>

<alter current graph type add> ::=
  "ADD" <graph type specification>

<alter current graph type alter> ::=
  "ALTER" <graph type specification>

<alter current graph type drop> ::=
  "DROP" <graph type drop specification>

<alter current graph type set> ::=
  "SET" <graph type specification>

<alter database statement> ::=
  "ALTER DATABASE" <alias or database name specification> [ <if exists> ] { <database set clause>... |
<database remove clause>... } [ <wait clause> ]

<database set clause> ::=
  "SET" <database set item>

<database set item> ::=
    <access specification>
  | <alter topology>
  | <set option>
  | <default language specification>

<access specification> ::=
  "ACCESS" { <read-only access> | <read write access> }

<read-only access> ::=
  "READ ONLY"

<read write access> ::=
  "READ WRITE"

<alter topology> ::=
    <simple topology specification>
  | <replica topology>
  | <graph shard>
  | <alter property shard>

<alter property shard> ::=
  "PROPERTY" <shard keywords> <left brace> "SET" <replica topology> <right brace>

<database remove clause> ::=
  "REMOVE" <database remove item>

<database remove item> ::=
  <remove option>

<rename role statement> ::=
  "RENAME ROLE" <role name specification> [ <if exists> ] "TO" <role name specification>

<rename server statement> ::=
  "RENAME SERVER" <server name specification> "TO" <server name specification>

<alter server statement> ::=
  "ALTER SERVER" <server name specification> "SET" <command options>

<rename user statement> ::=
  "RENAME USER" <user name specification> [ <if exists> ] "TO" <user name specification>

<alter current user statement> ::=
  "CURRENT USER SET PASSWORD FROM" <password value specification> "TO" <password value specification>

<alter user statement> ::=
  "ALTER USER" <user name specification> [ <if exists> ] [ <user remove clause>... ] [ <user add
clause>... ] [ <user alter set clause>... ]

<user alter set clause> ::=
```

```
  "SET" <user alter set item>

<user alter set item> ::=
    <user set password>
  | <user set password change required>
  | <user set status>
  | <user set home database>
  | <user set authentication provider>
  | <user set tags clause>

<user add clause> ::=
  "ADD" <user add item>

<user add item> ::=
  <user add tags clause>

<user add tags clause> ::=
  <explicit user tags>

<user remove clause> ::=
  "REMOVE" <user remove item>

<user remove item> ::=
    <user remove home database>
  | <user remove all authentication provider>
  | <user remove named authentication provider>
  | <user remove tags clause>

<user remove home database> ::=
  "HOME DATABASE"

<user remove all authentication provider> ::=
  "ALL AUTH" [ <provider keywords> ]

<user remove named authentication provider> ::=
  "AUTH" [ <provider keywords> ] <character string or string list value specification>

<user remove tags clause> ::=
    <explicit user tags>
  | "ALL" <tag keywords>

<alter users statement> ::=
  "ALTER USERS" <user name specification list> [ <if exists> ] [ <user remove bulk clause>... ] [ <user
add bulk clause>... ] [ <user set bulk clause>... ]

<user add bulk clause> ::=
  "ADD" <user add bulk item>

<user add bulk item> ::=
  <user add tags clause>

<user remove bulk clause> ::=
  "REMOVE" <user remove bulk item>

<user remove bulk item> ::=
  <user remove tags clause>

<user set bulk clause> ::=
  "SET" <user set bulk item>

<user set bulk item> ::=
  <user set tags clause>

<standalone catalog show statement> ::=
    <show aliases statement>
  | <show current user statement>
  | <show privileges statement>
  | <show auth rules statement>
  | <show roles statement>
  | <show servers statement>
  | <show users statement>

<primitive catalog show statement> ::=
    <show constraints statement>
  | <show current graph type statement>
  | <show indexes statement>
  | <show functions statement>
```

```
  | <show procedures statement>
  | <show settings statement>
  | <show transactions statement>
  | <show databases statement>

<show aliases statement> ::=
  "SHOW" <alias keywords> [ <alias name specification> ] "FOR" <database keywords> [ <show result clause>
]

<show auth rules statement> ::=
  "SHOW" <auth rule keywords> [ "AS" <command keywords> ] [ <show result clause> ]

<show constraints statement> ::=
  "SHOW" [ <show constraint type> ] <constraint keywords> [ <show yield where clause> ]

<show constraint type> ::=
    <show all constraints>
  | <show key constraints>
  | <show uniqueness constraints>
  | <show property existence constraints>
  | <show existence constraints>
  | <show property type constraints>

<show all constraints> ::=
  "ALL"

<show key constraints> ::=
  [ <show constraints for element> ] "KEY"

<show uniqueness constraints> ::=
  [ <show constraints for element> ] [ "PROPERTY" ] { "UNIQUENESS" | "UNIQUE" }

<show property existence constraints> ::=
  [ <show constraints for element> ] "PROPERTY" { "EXISTENCE" | "EXIST" }

<show existence constraints> ::=
  [ <show constraints for element> ] { "EXISTENCE" | "EXIST" }

<show property type constraints> ::=
  [ <show constraints for element> ] "PROPERTY TYPE"

<show constraints for element> ::=
    "NODE"
  | <relationship singular keywords>

<show current graph type statement> ::=
  "SHOW CURRENT GRAPH TYPE" [ "AS GRAPH" ] [ <show yield where clause> ]

<show current user statement> ::=
  "SHOW CURRENT USER" [ <show result clause> ]

<show users statement> ::=
  "SHOW" <user keywords> [ "WITH AUTH" ] [ "AS" <command keywords> ] [ <show result clause> ]

<show databases statement> ::=
    <show all databases statement>
  | <show default database statement>
  | <show home database statement>
  | <show named database statement>

<show all databases statement> ::=
  "SHOW" <database keywords> [ <show yield where clause> ]

<show default database statement> ::=
  "SHOW DEFAULT DATABASE" [ <show yield where clause> ]

<show home database statement> ::=
  "SHOW HOME DATABASE" [ <show yield where clause> ]

<show named database statement> ::=
  "SHOW" <database keywords> <alias or database name specification> [ <show yield where clause> ]

<show functions statement> ::=
  "SHOW" [ <show function type> ] <function keywords> [ <executable by> ] [ <show yield where clause> ]

<show procedures statement> ::=
  "SHOW" <procedure keywords> [ <executable by> ] [ <show yield where clause> ]
```

```
<show function type> ::=
    <show all functions>
  | <show built-in functions>
  | <show user-defined functions>

<show all functions> ::=
  "ALL"

<show built-in functions> ::=
  "BUILT IN"

<show user-defined functions> ::=
  "USER DEFINED"

<executable by> ::=
  "EXECUTABLE" [ <by current user> | <by named user> ]

<by current user> ::=
  "BY CURRENT USER"

<by named user> ::=
  "BY" <user name>

<show indexes statement> ::=
  "SHOW" [ <show index type> ] <index keywords> [ <show yield where clause> ]

<show index type> ::=
    <show all indexes>
  | <show fulltext indexes>
  | <show lookup indexes>
  | <show point indexes>
  | <show range indexes>
  | <show text indexes>
  | <show vector indexes>

<show all indexes> ::=
  "ALL"

<show fulltext indexes> ::=
  "FULLTEXT"

<show lookup indexes> ::=
  "LOOKUP"

<show point indexes> ::=
  "POINT"

<show range indexes> ::=
  "RANGE"

<show text indexes> ::=
  "TEXT"

<show vector indexes> ::=
  "VECTOR"

<show privileges statement> ::=
    <show all privileges statement>
  | <show supported privileges statement>
  | <show role privileges statement>
  | <show user privileges statement>
  | <show auth rule privileges statement>

<show all privileges statement> ::=
  "SHOW" [ "ALL" ] <privilege keywords> [ <show privilege as> ] [ <show result clause> ]

<show supported privileges statement> ::=
  "SHOW SUPPORTED" <privilege keywords> [ <show result clause> ]

<show role privileges statement> ::=
  "SHOW" <role keywords> <role name specification list> <privilege keywords> [ <show privilege as> ] [
<show result clause> ]

<show user privileges statement> ::=
  "SHOW" <user keywords> [ <user name specification list> ] <privilege keywords> [ <show privilege as> ] [
<show result clause> ]
```

```
<show auth rule privileges statement> ::=
  "SHOW" <auth rule keywords> <auth rule name specification list> <privilege keywords> [ <show privilege
as> ] [ <show result clause> ]

<show privilege as> ::=
    <show privilege as grant-deny command>
  | <show privilege as revoke command>

<show privilege as grant-deny command> ::=
  "AS" <command keywords>

<show privilege as revoke command> ::=
  "AS REVOKE" <command keywords>

<show roles statement> ::=
    <show all roles statement>
  | <show populated roles statement>

<show all roles statement> ::=
  "SHOW" [ "ALL" ] <role keywords> [ <with users> | <with auth rules> ] [ "AS" <command keywords> ] [
<show result clause> ]

<show populated roles statement> ::=
  "SHOW POPULATED" <role keywords> [ <with users> | <with auth rules> ] [ "AS" <command keywords> ] [
<show result clause> ]

<with users> ::=
  "WITH" <user keywords>

<with auth rules> ::=
  "WITH" <auth rule keywords>

<show servers statement> ::=
  "SHOW" <server keywords> [ <show result clause> ]

<show settings statement> ::=
  "SHOW" <setting keywords> [ <character string list element list or expression> ] [ <show yield where
clause> ]

<show transactions statement> ::=
  "SHOW" <transaction keywords> [ <character string list element list or expression> ] [ <show yield where
clause> ]

<show result clause> ::=
    <show yield result clause>
  | <show where result clause>

<show yield where clause> ::=
    <show yield only clause>
  | <show where result clause>

<show yield result clause> ::=
  "YIELD" { <yield item list> | <asterisk> } [ <order by clause> ] [ <offset clause> ] [ <limit clause> ]
[ <where clause> ] [ <return statement> ]

<show yield only clause> ::=
  "YIELD" { <yield item list> | <asterisk> } [ <order by clause> ] [ <offset clause> ] [ <limit clause> ]
[ <where clause> ]

<show where result clause> ::=
  <where clause>

<graph type specification> ::=
  <left brace> [ <graph type specification body> ] <right brace>

<graph type specification body> ::=
  <graph type element> [ { <comma> <graph type element> }... ]

<graph type element> ::=
    <relationship element type with constraints specification>
  | <node element type with constraints specification>
  | <graph type constraint specification>

<node element type with constraints specification> ::=
  <node element type specification> [ <node element type inline constraint list> ]
```

```
<relationship element type with constraints specification> ::=
  <relationship element type specification> [ <relationship element type inline constraint list> ]

<node element type inline constraint list> ::=
  <node element type inline constraint require>...

<relationship element type inline constraint list> ::=
  <relationship element type inline constraint require>...

<node element type inline constraint require> ::=
    <graph type node key constraint require>
  | <graph type node property uniqueness constraint require>

<relationship element type inline constraint require> ::=
    <graph type relationship key constraint require>
  | <graph type relationship property uniqueness constraint require>

<node element type specification> ::=
  <left paren> [ <element type alias> ] <node element type filler> <right paren>

<node element type filler> ::=
  <identifying label> [ <implied label set> ] [ <property types specification> ]

<implied label set> ::=
  { <colon> | "IS" } <label name> [ { <ampersand> <label name> }... ]

<node element type reference> ::=
    <node element type alias reference>
  | <node element type in-situ reference>

<node element type alias reference> ::=
  <left paren> <element type alias> <right paren>

<node element type in-situ reference> ::=
  <left paren> [ [ <element type alias> ] <single label> [ <implies> ] ] <right paren>

<empty node element type reference> ::=
  <left paren> <right paren>

<relationship element type specification> ::=
  <source endpoint> <arc pointing right> <destination endpoint>

<arc pointing right> ::=
  <arc pointing right start> [ <element type alias> ] <relationship element type filler> <arc pointing
right end>

<relationship element type filler> ::=
  <identifying label> [ <property types specification> ]

<source endpoint> ::=
  <node element type reference>

<destination endpoint> ::=
  <node element type reference>

<arc pointing right start> ::=
  <arrow line> <left bracket>

<arc pointing right end> ::=
  <right bracket> <arrow line> <right arrowhead>

<relationship element type reference> ::=
    <relationship element type alias reference>
  | <relationship element type in-situ reference>

<relationship element type alias reference> ::=
  <relationship element type reference start> <element type alias> <relationship element type reference
end>

<relationship element type in-situ reference> ::=
  <relationship element type reference start> <element type alias> <single label> [ <implies> ]
<relationship element type reference end>

<relationship element type reference start> ::=
  <empty node element type reference> <arc pointing right start>

<relationship element type reference end> ::=
```

```
  <arc pointing right end> <empty node element type reference>

<property types specification> ::=
  <left brace> [ <property type list> ] <right brace>

<property type list> ::=
  <property type with constraint> [ { <comma> <property type with constraint> }... ]

<property type with constraint> ::=
  <property type> [ <property type inline constraint> ]

<property type> ::=
  <property name> [ <typed> ] <value type>

<property type inline constraint> ::=
    <property type inline key constraint>
  | <property type inline uniqueness constraint>

<property type inline key constraint> ::=
  "IS" [ "NODE" | <relationship singular keywords> ] "KEY"

<property type inline uniqueness constraint> ::=
  "IS" [ "NODE" | <relationship singular keywords> ] "UNIQUE"

<graph type constraint specification> ::=
    <graph type node constraint>
  | <graph type relationship constraint>

<graph type node constraint> ::=
    <graph type node key constraint>
  | <graph type node uniqueness constraint>
  | <graph type node property existence constraint>
  | <graph type node property type constraint>

<graph type relationship constraint> ::=
    <graph type relationship key constraint>
  | <graph type relationship uniqueness constraint>
  | <graph type relationship property existence constraint>
  | <graph type relationship property type constraint>

<graph type node key constraint> ::=
  <graph type node constraint prefix> <graph type node key constraint require>

<graph type node uniqueness constraint> ::=
  <graph type node constraint prefix> <graph type node property uniqueness constraint require>

<graph type node property existence constraint> ::=
  <graph type node constraint prefix> <graph type property existence constraint require>

<graph type node property type constraint> ::=
  <graph type node constraint prefix> <graph type property type constraint require>

<graph type relationship key constraint> ::=
  <graph type relationship constraint prefix> <graph type relationship key constraint require>

<graph type relationship uniqueness constraint> ::=
  <graph type relationship constraint prefix> <graph type relationship property uniqueness constraint
require>

<graph type relationship property existence constraint> ::=
  <graph type relationship constraint prefix> <graph type property existence constraint require>

<graph type relationship property type constraint> ::=
  <graph type relationship constraint prefix> <graph type property type constraint require>

<graph type node constraint prefix> ::=
  <graph type constraint prefix> <node element type reference>

<graph type relationship constraint prefix> ::=
  <graph type constraint prefix> <relationship element type reference>

<graph type constraint prefix> ::=
  "CONSTRAINT" [ <constraint name> ] "FOR"

<graph type node key constraint require> ::=
  "REQUIRE" <graph type constraint property names specification> "IS" [ "NODE" ] "KEY" [ <command options>
]
```

```
<graph type relationship key constraint require> ::=
  "REQUIRE" <graph type constraint property names specification> "IS" [ <relationship singular keywords> ]
"KEY" [ <command options> ]

<graph type node property uniqueness constraint require> ::=
  "REQUIRE" <graph type constraint property names specification> "IS" [ "NODE" ] "UNIQUE" [ <command
options> ]

<graph type relationship property uniqueness constraint require> ::=
  "REQUIRE" <graph type constraint property names specification> "IS" [ <relationship singular keywords> ]
"UNIQUE" [ <command options> ]

<graph type property existence constraint require> ::=
  "REQUIRE" <graph type constraint property name specification> "IS NOT NULL" [ <command options> ]

<graph type property type constraint require> ::=
  "REQUIRE" <graph type constraint property name specification> <is typed> <value type> [ <command
options> ]

<graph type constraint property names specification> ::=
    <graph type constraint property name specification>
  | <left paren> <graph type constraint property name specification list> <right paren>

<graph type constraint property name specification list> ::=
  <graph type constraint property name specification> [ { <comma> <graph type constraint property name
specification> }... ]

<graph type constraint property name specification> ::=
  <element type alias> <period> <property name>

<graph type drop specification> ::=
  <left brace> [ <graph type drop specification body> ] <right brace>

<graph type drop specification body> ::=
  <graph type drop element> [ { <comma> <graph type drop element> }... ]

<graph type drop element> ::=
    <graph type element drop specification>
  | <graph type constraint drop specification>

<graph type element drop specification> ::=
    <relationship element type specification>
  | <node element type specification>

<graph type constraint drop specification> ::=
  "CONSTRAINT" <constraint name>

<identifying label> ::=
  <single label> <implies>

<single label> ::=
  { <colon> | "IS" } <label name>

<implies> ::=
    <right double arrow>
  | "IMPLIES"

<element type alias> ::=
  <identifier>

<standalone operation control statement> ::=
    <deallocate database statement>
  | <enable server statement>
  | <reallocate database statement>
  | <start database statement>
  | <stop database statement>

<primitive operation statement> ::=
    <primitive catalog show statement>
  | <terminate transaction statement>

<deallocate database statement> ::=
  [ "DRYRUN" ] "DEALLOCATE" <database keywords> "FROM" <server keywords> <character string list element
list specification>

<reallocate database statement> ::=
```

```
  [ "DRYRUN" ] "REALLOCATE" <database keywords>

<start database statement> ::=
  "START DATABASE" <alias or database name specification> [ <wait clause> ]

<stop database statement> ::=
  "STOP DATABASE" <alias or database name specification> [ <wait clause> ]

<terminate transaction statement> ::=
  "TERMINATE" <transaction keywords> <character string list element list or expression> [ <show yield only
clause> ]

<enable server statement> ::=
  "ENABLE SERVER" <server name specification> <command options>

<standalone access control statement> ::=
    <grant statement>
  | <deny statement>
  | <revoke statement>

<grant statement> ::=
  "GRANT" { <grant privilege> | <grant role> }

<grant privilege> ::=
  [ "IMMUTABLE" ] <privilege> "TO" <role name specification list>

<grant role> ::=
  <role keywords> <role name specification list> "TO" <user or auth rule>

<user or auth rule> ::=
    [ <user keywords> ] <user name specification list>
  | <auth rule keywords> <auth rule name specification list>

<deny statement> ::=
  "DENY" [ "IMMUTABLE" ] <privilege> "TO" <role name specification list>

<revoke statement> ::=
  "REVOKE" { <revoke privilege> | <revoke role> }

<revoke privilege> ::=
  [ "GRANT" | "DENY" ] [ "IMMUTABLE" ] <privilege> "FROM" <role name specification list>

<revoke role> ::=
  <role keywords> <role name specification list> "FROM" <user or auth rule>

<privilege> ::=
    <graph privilege>
  | <database privilege>
  | <dbms privilege>
  | <load privilege>

<graph privilege> ::=
    <all graph privilege>
  | <merge privilege>
  | <read graph privilege group>
  | <write graph privilege group>

<all graph privilege> ::=
  "ALL" [ [ "GRAPH" ] "PRIVILEGES" ] <graph privilege target>

<merge privilege> ::=
  "MERGE" <property privilege target> <qualified write graph privilege target>

<read graph privilege group> ::=
    <traverse element privilege>
  | <read and match element privilege>

<traverse element privilege> ::=
  "TRAVERSE" <qualified read graph privilege target>

<read and match element privilege> ::=
  { "READ" | "MATCH" } <property privilege target> <qualified read graph privilege target>

<write graph privilege group> ::=
    <create element privilege>
  | <delete element privilege>
  | <remove label privilege>
```

```
  | <set label privilege>
  | <set property privilege>
  | <write privilege>

<create element privilege> ::=
  "CREATE" <qualified write graph privilege target>

<delete element privilege> ::=
  "DELETE" <qualified write graph privilege target>

<remove label privilege> ::=
  "REMOVE" <label privilege target> <graph privilege target>

<set label privilege> ::=
  "SET" <label privilege target> <graph privilege target>

<set property privilege> ::=
  "SET PROPERTY" <named property privilege target> <qualified write graph privilege target>

<write privilege> ::=
  "WRITE" <graph privilege target>

<database privilege> ::=
    <all database privilege>
  | <access privilege>
  | <start privilege>
  | <stop privilege>
  | <alter database on database privilege group>
  | <index management privilege group>
  | <constraint management privilege group>
  | <name management privilege group>
  | <transaction management privilege group>

<all database privilege> ::=
  "ALL" [ [ "DATABASE" ] "PRIVILEGES" ] <database privilege target>

<access privilege> ::=
  "ACCESS" <database privilege target>

<start privilege> ::=
  "START" <database privilege target>

<stop privilege> ::=
  "STOP" <database privilege target>

<alter database on database privilege group> ::=
    <alter database on database privilege>
  | <set database access on database privilege>
  | <set database default language on database privilege>

<alter database on database privilege> ::=
  "ALTER" [ "COMPOSITE" ] "DATABASE" <database privilege target>

<set database access on database privilege> ::=
  "SET DATABASE ACCESS" <database privilege target>

<set database default language on database privilege> ::=
  "SET DATABASE DEFAULT LANGUAGE" <database privilege target>

<index management privilege group> ::=
    <create index privilege>
  | <drop index privilege>
  | <show index privilege>
  | <index management privilege>

<create index privilege> ::=
  "CREATE" <index keywords> <database privilege target>

<drop index privilege> ::=
  "DROP" <index keywords> <database privilege target>

<show index privilege> ::=
  "SHOW" <index keywords> <database privilege target>

<index management privilege> ::=
  <index keywords> [ "MANAGEMENT" ] <database privilege target>
```

```
<constraint management privilege group> ::=
    <create constraint privilege>
  | <drop constraint privilege>
  | <show constraint privilege>
  | <constraint management privilege>

<create constraint privilege> ::=
  "CREATE" <constraint keywords> <database privilege target>

<drop constraint privilege> ::=
  "DROP" <constraint keywords> <database privilege target>

<show constraint privilege> ::=
  "SHOW" <constraint keywords> <database privilege target>

<constraint management privilege> ::=
  <constraint keywords> [ "MANAGEMENT" ] <database privilege target>

<name management privilege group> ::=
    <create node label privilege>
  | <create relationship type privilege>
  | <create property name privilege>
  | <name management privilege>

<create node label privilege> ::=
  "CREATE NEW" [ "NODE" ] <label keywords> <database privilege target>

<create relationship type privilege> ::=
  "CREATE NEW" [ "RELATIONSHIP" ] <type keywords> <database privilege target>

<create property name privilege> ::=
  "CREATE NEW" [ "PROPERTY" ] <name keywords> <database privilege target>

<name management privilege> ::=
  "NAME" [ "MANAGEMENT" ] <database privilege target>

<transaction management privilege group> ::=
    <show transaction privilege>
  | <transaction termination privilege>
  | <transaction management privilege>

<show transaction privilege> ::=
  "SHOW" <transaction keywords> [ <user privilege target> ] <database privilege target>

<transaction termination privilege> ::=
  "TERMINATE" <transaction keywords> [ <user privilege target> ] <database privilege target>

<transaction management privilege> ::=
  "TRANSACTION" [ "MANAGEMENT" ] [ <user privilege target> ] <database privilege target>

<dbms privilege> ::=
    <all dbms privilege>
  | <impersonate user privilege>
  | <show setting privilege>
  | <execute privilege group>
  | <user management privilege group>
  | <auth rule management privilege group>
  | <role management privilege group>
  | <user metadata management privilege group>
  | <privilege management privilege group>
  | <alias management privilege group>
  | <database management privilege group>
  | <server management privilege group>

<all dbms privilege> ::=
  "ALL" [ [ "DBMS" ] "PRIVILEGES" ] <dbms privilege target>

<impersonate user privilege> ::=
  "IMPERSONATE" <user privilege target> <dbms privilege target>

<show setting privilege> ::=
  "SHOW" <setting keywords> <setting privilege target> <dbms privilege target>

<execute privilege group> ::=
    <execute admin procedures privilege>
  | <execute procedures privilege>
  | <execute functions privilege>
```

```
<execute admin procedures privilege> ::=
  "EXECUTE" <admin keywords> "PROCEDURES" <dbms privilege target>

<execute procedures privilege> ::=
  "EXECUTE" [ "BOOSTED" ] <procedure keywords> <procedure privilege target> <dbms privilege target>

<execute functions privilege> ::=
  "EXECUTE" [ "BOOSTED" ] [ "USER" [ "DEFINED" ] ] <function keywords> <function privilege target> <dbms
privilege target>

<user management privilege group> ::=
    <create user privilege>
  | <rename user privilege>
  | <alter user privilege>
  | <set password privilege>
  | <set user status privilege>
  | <set user home database privilege>
  | <set auth privilege>
  | <drop user privilege>
  | <show user privilege>
  | <show user credentials privilege>
  | <user management privilege>

<create user privilege> ::=
  "CREATE USER" <dbms privilege target>

<rename user privilege> ::=
  "RENAME USER" <dbms privilege target>

<alter user privilege> ::=
  "ALTER USER" <dbms privilege target>

<set password privilege> ::=
  "SET" <password keywords> <dbms privilege target>

<set user status privilege> ::=
  "SET USER STATUS" <dbms privilege target>

<set user home database privilege> ::=
  "SET USER" <home database keyword> <dbms privilege target>

<set auth privilege> ::=
  "SET AUTH" <dbms privilege target>

<drop user privilege> ::=
  "DROP USER" <dbms privilege target>

<show user privilege> ::=
  "SHOW USER" <dbms privilege target>

<show user credentials privilege> ::=
  "SHOW USER CREDENTIALS" <dbms privilege target>

<user management privilege> ::=
  "USER MANAGEMENT" <dbms privilege target>

<auth rule management privilege group> ::=
    <create auth rule privilege>
  | <rename auth rule privilege>
  | <alter auth rule privilege>
  | <drop auth rule privilege>
  | <show auth rule privilege>
  | <auth rule management privilege>

<create auth rule privilege> ::=
  "CREATE AUTH RULE" <dbms privilege target>

<rename auth rule privilege> ::=
  "RENAME AUTH RULE" <dbms privilege target>

<alter auth rule privilege> ::=
  "ALTER AUTH RULE" <dbms privilege target>

<drop auth rule privilege> ::=
  "DROP AUTH RULE" <dbms privilege target>
```

```
<show auth rule privilege> ::=
  "SHOW AUTH RULE" <dbms privilege target>

<auth rule management privilege> ::=
  "AUTH RULE MANAGEMENT" <dbms privilege target>

<user metadata management privilege group> ::=
    <show user metadata privilege>
  | <set user metadata privilege>
  | <user metadata management privilege>

<show user metadata privilege> ::=
  "SHOW USER METADATA" <dbms privilege target>

<set user metadata privilege> ::=
  "SET USER METADATA" <dbms privilege target>

<user metadata management privilege> ::=
  "USER METADATA MANAGEMENT" <dbms privilege target>

<role management privilege group> ::=
    <create role privilege>
  | <rename role privilege>
  | <drop role privilege>
  | <assign role privilege>
  | <remove role privilege>
  | <show role privilege>
  | <role management privilege>

<create role privilege> ::=
  "CREATE ROLE" <dbms privilege target>

<rename role privilege> ::=
  "RENAME ROLE" <dbms privilege target>

<drop role privilege> ::=
  "DROP ROLE" <dbms privilege target>

<assign role privilege> ::=
  "ASSIGN ROLE" <dbms privilege target>

<remove role privilege> ::=
  "REMOVE ROLE" <dbms privilege target>

<show role privilege> ::=
  "SHOW ROLE" <dbms privilege target>

<role management privilege> ::=
  "ROLE MANAGEMENT" <dbms privilege target>

<privilege management privilege group> ::=
    <assign privilege privilege>
  | <remove privilege privilege>
  | <show privilege privilege>
  | <privilege management privilege>

<assign privilege privilege> ::=
  "ASSIGN PRIVILEGE" <dbms privilege target>

<remove privilege privilege> ::=
  "REMOVE PRIVILEGE" <dbms privilege target>

<show privilege privilege> ::=
  "SHOW PRIVILEGE" <dbms privilege target>

<privilege management privilege> ::=
  "PRIVILEGE MANAGEMENT" <dbms privilege target>

<alias management privilege group> ::=
    <create alias privilege>
  | <alter alias privilege>
  | <drop alias privilege>
  | <show alias privilege>
  | <alias management privilege>

<create alias privilege> ::=
  "CREATE ALIAS" <dbms privilege target>
```

```
<alter alias privilege> ::=
  "ALTER ALIAS" <dbms privilege target>

<drop alias privilege> ::=
  "DROP ALIAS" <dbms privilege target>

<show alias privilege> ::=
  "SHOW ALIAS" <dbms privilege target>

<alias management privilege> ::=
  "ALIAS MANAGEMENT" <dbms privilege target>

<database management privilege group> ::=
    <create database privilege>
  | <alter database on dbms privilege>
  | <set database access on dbms privilege>
  | <set database default language on dbms privilege>
  | <drop database privilege>
  | <database management privilege>

<create database privilege> ::=
  "CREATE" [ "COMPOSITE" ] "DATABASE" <dbms privilege target>

<alter database on dbms privilege> ::=
  "ALTER" [ "COMPOSITE" ] "DATABASE" <dbms privilege target>

<set database access on dbms privilege> ::=
  "SET DATABASE ACCESS" <dbms privilege target>

<set database default language on dbms privilege> ::=
  "SET DATABASE DEFAULT LANGUAGE" <dbms privilege target>

<drop database privilege> ::=
  "DROP" [ "COMPOSITE" ] "DATABASE" <dbms privilege target>

<database management privilege> ::=
  [ "COMPOSITE" ] "DATABASE MANAGEMENT" <dbms privilege target>

<server management privilege group> ::=
    <show server privilege>
  | <server management privilege>

<show server privilege> ::=
  "SHOW" <server keywords> <dbms privilege target>

<server management privilege> ::=
  "SERVER MANAGEMENT" <dbms privilege target>

<load privilege> ::=
    <load cidr privilege>
  | <load all data privilege>

<load cidr privilege> ::=
  "LOAD ON CIDR" <character string value specification>

<load all data privilege> ::=
  "LOAD ON ALL DATA"

<qualified write graph privilege target> ::=
  <graph privilege target> [ <write graph element privilege target> ]

<qualified read graph privilege target> ::=
  <graph privilege target> [ <read graph element privilege target> ]

<graph privilege target> ::=
    <home graph target>
  | <named graph privilege target>

<database privilege target> ::=
    <home database target>
  | <named database privilege target>

<dbms privilege target> ::=
  "ON DBMS"

<home graph target> ::=
```

```
  "ON HOME GRAPH"

<home database target> ::=
  "ON" <home database keyword>

<named graph privilege target> ::=
  "ON" { "GRAPH" | "GRAPHS" } { <asterisk> | <graph specification list> }

<named database privilege target> ::=
  "ON" { "DATABASE" | "DATABASES" } { <asterisk> | <alias or database specification list> }

<write graph element privilege target> ::=
    <targeted elements>
  | <targeted nodes>
  | <targeted relationships>

<read graph element privilege target> ::=
    <targeted elements>
  | <targeted nodes>
  | <targeted relationships>
  | "FOR" <targeted node pattern>

<targeted elements> ::=
  <element keywords> <target labels>

<targeted nodes> ::=
  <node keywords> <target labels>

<targeted relationships> ::=
  <relationship keywords> <target labels>

<label privilege target> ::=
  "LABEL" <target labels>

<target labels> ::=
    <asterisk>
  | <label name list>

<user privilege target> ::=
  <left paren> { <asterisk> | <user name specification list> } <right paren>

<procedure privilege target> ::=
  <identifier glob list>

<function privilege target> ::=
  <identifier glob list>

<setting privilege target> ::=
  <identifier glob list>

<property privilege target> ::=
    <all property privilege target>
  | <named property privilege target>

<all property privilege target> ::=
  <left brace> <asterisk> <right brace>

<named property privilege target> ::=
  <targeted property names specification>

<identifier glob list> ::=
  <identifier glob> [ { <comma> <identifier glob> }... ]

<identifier glob> ::=
    <delimited identifier> [ <identifier glob recursion> ]
  | <identifier glob recursion>

<identifier glob recursion> ::=
  <identifier glob part> [ <identifier glob recursion> ]

<identifier glob part> ::=
    <period> [ <delimited identifier> ]
  | <question mark>
  | <asterisk>
  | <regular identifier>

<use graph clause> ::=
```

```
  "USE" <graph expression>

<graph expression> ::=
    <left paren> <graph expression> <right paren>
  | <graph reference>
  | <function invocation>

<graph pattern binding table> ::=
  <graph pattern>

<where clause> ::=
  "WHERE" <search condition>

<set quantifier> ::=
    "ALL"
  | "DISTINCT"

<graph pattern> ::=
  [ <match mode> ] <path pattern list> [ <hints> ] [ <graph pattern where and search clause> ]

<hints> ::=
  <hint> [ <hints> ]

<hint> ::=
  "USING" <hint specification>

<hint specification> ::=
    <index hint specification>
  | "JOIN ON" <binding variable reference list>
  | "SCAN" <binding variable> <label or rel type>

<index hint specification> ::=
  [ <index type> ] "INDEX" [ "SEEK" ] <binding variable> <label or rel type> <left paren> <property name
list> <right paren>

<index type> ::=
    "TEXT"
  | "RANGE"
  | "POINT"

<label or rel type> ::=
  <colon> <label name>

<graph pattern where and search clause> ::=
    <graph pattern where clause> [ <search clause> ]
  | <search clause> [ <graph pattern where clause> ]

<match mode> ::=
    <repeatable elements match mode>
  | <different edges match mode>

<repeatable elements match mode> ::=
  "REPEATABLE" <element bindings or elements>

<different edges match mode> ::=
  "DIFFERENT" <edge bindings or edges>

<element bindings or elements> ::=
    "ELEMENT" [ "BINDINGS" ]
  | "ELEMENTS"

<edge bindings or edges> ::=
    "RELATIONSHIP" [ "BINDINGS" ]
  | "RELATIONSHIPS"

<path pattern list> ::=
  <path pattern> [ { <comma> <path pattern> }... ]

<graph pattern where clause> ::=
  <where clause>

<search clause> ::=
  "SEARCH" <binding variable reference> "IN" <left paren> <index specification clause> <for clause> [
<where clause> ] [ <offset clause> ] <limit clause> <right paren> [ <score clause> ]

<index specification clause> ::=
  { "VECTOR" | "FULLTEXT" } "INDEX" <index name specification>
```

```
<for clause> ::=
  "FOR" <value expression> [ "WITH ANALYZER" <value expression> ]

<score clause> ::=
  "SCORE AS" <binding variable>

<path pattern> ::=
  [ <path variable declaration> ] [ <path pattern prefix> ] <path pattern expression>

<path variable declaration> ::=
  <binding variable> <equals operator>

<path pattern prefix> ::=
    <path mode prefix>
  | <path search prefix>

<path mode prefix> ::=
  <path mode> [ <path keywords> ]

<path mode> ::=
    "WALK"
  | "TRAIL"
  | "ACYCLIC"

<path search prefix> ::=
    <all path search>
  | <any path search>
  | <shortest path search>

<all path search> ::=
  "ALL" [ <path mode> ] [ <path keywords> ]

<any path search> ::=
  "ANY" [ <number of paths> ] [ <path mode> ] [ <path keywords> ]

<shortest path search> ::=
    <all shortest path search>
  | <any shortest path search>
  | <counted shortest path search>
  | <counted shortest group search>

<all shortest path search> ::=
  "ALL SHORTEST" [ <path mode> ] [ <path keywords> ]

<any shortest path search> ::=
  "ANY SHORTEST" [ <path mode> ] [ <path keywords> ]

<counted shortest path search> ::=
  "SHORTEST" <number of paths> [ <path mode> ] [ <path keywords> ]

<counted shortest group search> ::=
  "SHORTEST" [ <number of groups> ] [ <path mode> ] [ <path keywords> ] { "GROUP" | "GROUPS" }

<number of paths> ::=
  <unsigned decimal integer specification>

<number of groups> ::=
  <unsigned decimal integer specification>

<simple path pattern> ::=
  <node pattern> { <relationship pattern> <node pattern> }...

<path pattern expression> ::=
    <path term>
  | <legacy shortest path pattern>

<legacy shortest path pattern> ::=
  { "SHORTESTPATH" | "ALLSHORTESTPATHS" } <left paren> <node pattern> <relationship pattern> <node
pattern> <right paren>

<path term> ::=
    <path factor>
  | <path term> <path factor>

<path factor> ::=
    <path primary>
```

```
  | <quantified path primary>

<quantified path primary> ::=
  <path primary> <graph pattern quantifier>

<path primary> ::=
    <element pattern>
  | <parenthesized path pattern expression>

<parenthesized path pattern expression> ::=
  <left paren> [ <subpath variable declaration> ] <path pattern expression> [ <parenthesized path pattern
where clause> ] <right paren>

<subpath variable declaration> ::=
  <binding variable> <equals operator>

<parenthesized path pattern where clause> ::=
  "WHERE" <value expression>

<element pattern> ::=
    <node pattern>
  | <relationship pattern>

<node pattern> ::=
  <left paren> [ <node pattern filler> ] <right paren>

<node pattern filler> ::=
    <binding variable> [ <is node label expression> ] [ <element pattern predicate> ]
  | <is node label expression> [ <element pattern predicate> ]
  | <element pattern predicate>

<element pattern predicate> ::=
    <element pattern where clause>
  | <element property specification>
  | <general parameter reference>

<element pattern where clause> ::=
  "WHERE" <value expression>

<element property specification> ::=
  <left brace> <property key value pair list> <right brace>

<property key value pair list> ::=
  <property key value pair> [ { <comma> <property key value pair> }... ]

<property key value pair> ::=
  <property name> <colon> <value expression>

<relationship pattern> ::=
  <full relationship pattern>

<full relationship pattern> ::=
    <full relationship pointing left>
  | <full relationship pointing right>
  | <full relationship left or right>
  | <full relationship any direction>

<full relationship pointing left> ::=
  <left arrowhead> <arrow line> [ <left bracket> [ <relationship pattern filler> ] <right bracket> ]
<arrow line>

<full relationship pointing right> ::=
  <arrow line> [ <left bracket> [ <relationship pattern filler> ] <right bracket> ] <arrow line> <right
arrowhead>

<full relationship left or right> ::=
  <left arrowhead> <arrow line> [ <left bracket> [ <relationship pattern filler> ] <right bracket> ]
<arrow line> <right arrowhead>

<full relationship any direction> ::=
  <arrow line> [ <left bracket> [ <relationship pattern filler> ] <right bracket> ] <arrow line>

<left arrowhead> ::=
  <left angle bracket>

<arrow line> ::=
  <minus sign>
```

```
<right arrowhead> ::=
  <right angle bracket>

<relationship pattern filler> ::=
    <binding variable> [ <is relationship label expression> ] [ <path length> ] [ <element pattern
predicate> ]
  | <is relationship label expression> [ <path length> ] [ <element pattern predicate> ]
  | <path length> [ <element pattern predicate> ]
  | <element pattern predicate>

<path length> ::=
  <asterisk> [ <lower and upper bound path length> | <fixed path length> ]

<lower and upper bound path length> ::=
  [ <lower bound path length> ] <double period> [ <upper bound path length> ]

<graph pattern quantifier> ::=
    <asterisk>
  | <plus sign>
  | <fixed quantifier>
  | <general quantifier>

<fixed quantifier> ::=
  <left brace> <unsigned decimal integer> <right brace>

<general quantifier> ::=
  <left brace> [ <lower bound path length> ] <comma> [ <upper bound path length> ] <right brace>

<lower bound path length> ::=
  <unsigned decimal integer>

<upper bound path length> ::=
  <unsigned decimal integer>

<fixed path length> ::=
  <unsigned decimal integer>

<create graph pattern> ::=
  <create path pattern list>

<create path pattern list> ::=
  <create path pattern> [ { <comma> <create path pattern> }... ]

<create path pattern> ::=
  [ <path variable declaration> ] <create node pattern> [ { <create relationship pattern> <create node
pattern> }... ]

<create node pattern> ::=
  <left paren> [ <create node pattern filler> ] <right paren>

<create relationship pattern> ::=
    <create relationship pointing left>
  | <create relationship pointing right>

<create relationship pointing left> ::=
  <left arrowhead> <arrow line> <left bracket> <create relationship pattern filler> <right bracket> <arrow
line>

<create relationship pointing right> ::=
  <arrow line> <left bracket> <create relationship pattern filler> <right bracket> <arrow line> <right
arrowhead>

<create node pattern filler> ::=
    <binding variable> [ <create node label and property set specification> ]
  | [ <binding variable> ] <create node label and property set specification>

<create relationship pattern filler> ::=
  [ <binding variable> ] <create relationship label and property set specification>

<create node label and property set specification> ::=
    <create node label set specification> [ <create element property specification> ]
  | [ <create node label set specification> ] <create element property specification>

<create relationship label and property set specification> ::=
  <create relationship label specification> [ <create element property specification> ]
```

```
<create node label set specification> ::=
  <node label set specification for create and merge>

<create relationship label specification> ::=
  <relationship label specification for create and merge>

<create element property specification> ::=
    <element property specification>
  | <general parameter reference>

<insert graph pattern> ::=
  <insert path pattern list>

<insert path pattern list> ::=
  <insert path pattern> [ { <comma> <insert path pattern> }... ]

<insert path pattern> ::=
  <insert node pattern> [ { <insert relationship pattern> <insert node pattern> }... ]

<insert node pattern> ::=
  <left paren> [ <insert node pattern filler> ] <right paren>

<insert relationship pattern> ::=
    <insert relationship pointing left>
  | <insert relationship pointing right>

<insert relationship pointing left> ::=
  <left arrowhead> <arrow line> <left bracket> <insert relationship pattern filler> <right bracket> <arrow
line>

<insert relationship pointing right> ::=
  <arrow line> <left bracket> <insert relationship pattern filler> <right bracket> <arrow line> <right
arrowhead>

<insert node pattern filler> ::=
    <binding variable> [ <insert node label and property set specification> ]
  | [ <binding variable> ] <insert node label and property set specification>

<insert relationship pattern filler> ::=
  [ <binding variable> ] <insert relationship label and property set specification>

<insert node label and property set specification> ::=
    <insert node label set specification> [ <insert element property specification> ]
  | [ <insert node label set specification> ] <insert element property specification>

<insert relationship label and property set specification> ::=
  <insert relationship label specification> [ <insert element property specification> ]

<insert node label set specification> ::=
  <node label set specification for insert>

<insert relationship label specification> ::=
  <relationship label specification for insert>

<insert element property specification> ::=
  <element property specification>

<merge graph pattern> ::=
  <merge path pattern>

<merge path pattern> ::=
  [ <path variable declaration> ] <merge node pattern> [ { <merge relationship pattern> <merge node
pattern> }... ]

<merge node pattern> ::=
  <left paren> [ <merge node pattern filler> ] <right paren>

<merge relationship pattern> ::=
    <merge relationship pointing left>
  | <merge relationship pointing right>

<merge relationship pointing left> ::=
  <left arrowhead> <arrow line> <left bracket> <merge relationship pattern filler> <right bracket> <arrow
line>

<merge relationship pointing right> ::=
  <arrow line> <left bracket> <merge relationship pattern filler> <right bracket> <arrow line> <right
```

```
arrowhead>

<merge node pattern filler> ::=
    <binding variable> [ <merge node label and property set specification> ]
  | [ <binding variable> ] <merge node label and property set specification>

<merge relationship pattern filler> ::=
  [ <binding variable> ] <merge relationship label and property set specification>

<merge node label and property set specification> ::=
    <merge node label set specification> [ <merge element property specification> ]
  | [ <merge node label set specification> ] <merge element property specification>

<merge relationship label and property set specification> ::=
  <merge relationship label specification> [ <merge element property specification> ]

<merge node label set specification> ::=
  <node label set specification for create and merge>

<merge relationship label specification> ::=
  <relationship label specification for create and merge>

<merge element property specification> ::=
  <element property specification>

<command element pattern> ::=
    <command node pattern>
  | <command relationship pattern>

<command node pattern> ::=
  <left paren> <binding variable> <colon> <label name> <right paren>

<command relationship pattern> ::=
  <left paren> <right paren> { <command relationship body pointing left> | <command relationship body
pointing right> | <command relationship body pointing both> | <command relationship body> } <left paren>
<right paren>

<command relationship body pointing left> ::=
  <left arrowhead> <command relationship body>

<command relationship body pointing right> ::=
  <command relationship body> <right arrowhead>

<command relationship body pointing both> ::=
  <left arrowhead> <command relationship body> <right arrowhead>

<command relationship body> ::=
  <arrow line> <left bracket> <binding variable> <colon> <label name> <right bracket> <arrow line>

<lookup index node pattern> ::=
  <left paren> <binding variable> <right paren>

<lookup index relationship pattern> ::=
  <left paren> <right paren> { <lookup index relationship body pointing left> | <lookup index relationship
body pointing right> | <lookup index relationship body pointing both> | <lookup index relationship body> }
<left paren> <right paren>

<lookup index relationship body pointing left> ::=
  <left arrowhead> <lookup index relationship body>

<lookup index relationship body pointing right> ::=
  <lookup index relationship body> <right arrowhead>

<lookup index relationship body pointing both> ::=
  <left arrowhead> <lookup index relationship body> <right arrowhead>

<lookup index relationship body> ::=
  <arrow line> <left bracket> <binding variable> <right bracket> <arrow line>

<multi-entity-token index element pattern> ::=
    <multi-label index node pattern>
  | <multi-reltype index relationship pattern>

<multi-label index node pattern> ::=
  <left paren> <binding variable> <colon> <multi-entity-token index label expression> <right paren>

<multi-reltype index relationship pattern> ::=
```

```
  <left paren> <right paren> { <multi-reltype index relationship body pointing left> | <multi-reltype
index relationship body pointing right> | <multi-reltype index relationship body pointing both> | <multi-
reltype index relationship body> } <left paren> <right paren>

<multi-reltype index relationship body pointing left> ::=
  <left arrowhead> <multi-reltype index relationship body>

<multi-reltype index relationship body pointing right> ::=
  <multi-reltype index relationship body> <right arrowhead>

<multi-reltype index relationship body pointing both> ::=
  <left arrowhead> <multi-reltype index relationship body> <right arrowhead>

<multi-reltype index relationship body> ::=
  <arrow line> <left bracket> <binding variable> <colon> <multi-entity-token index label expression>
<right bracket> <arrow line>

<multi-entity-token index label expression> ::=
  <disjunctive label specification>

<targeted node pattern> ::=
  <left paren> [ <targeted node pattern filler> ] <right paren>

<targeted node pattern filler> ::=
    <binding variable> [ <targeted node label expression> ] [ <element pattern predicate> ]
  | <targeted node label expression> [ <element pattern predicate> ]
  | <element pattern predicate>

<targeted node label expression> ::=
  <colon> <disjunctive label specification>

<targeted property names specification> ::=
  <left brace> <property name list> <right brace>

<is node label expression> ::=
    <colon> <node label expression legacy>
  | <is label expression>

<is relationship label expression> ::=
    <colon> <relationship label expression legacy>
  | <is label expression>

<is label expression> ::=
  { <colon> | "IS" } <label expression>

<node label expression legacy> ::=
  <label name> { <colon> <label name> }...

<relationship label expression legacy> ::=
  <label name> { <vertical bar> <colon> <label name> }...

<wildcard label> ::=
  <percent>

<label expression> ::=
    <label term>
  | <label expression> <vertical bar> <label term>

<label term> ::=
    <label factor>
  | <label term> <ampersand> <label factor>

<label factor> ::=
    <label primary>
  | <label negation>

<label negation> ::=
  <exclamation mark> <label primary>

<label primary> ::=
    <label name>
  | <parenthesized label expression>
  | <wildcard label>
  | <dynamic label expression>

<parenthesized label expression> ::=
  <left paren> <label expression> <right paren>
```

```
<dynamic label expression> ::=
  <dollar sign> [ <dynamic label expression quantifier> ] <left paren> <value expression> <right paren>

<dynamic label expression quantifier> ::=
    "ALL"
  | "ANY"

<node label set specification for create and merge> ::=
    <dynamic node label set specification with ampersand>
  | <dynamic node label set specification with colon>
  | <dynamic node label set single specification>

<node label set specification for insert> ::=
  <static node label set specification with ampersand>

<node label set specification for set and remove> ::=
    <dynamic node label set specification with colon>
  | <dynamic node label set single specification>

<static node label set specification with ampersand> ::=
  { "IS" | <colon> } <label name> [ { <ampersand> <label name> }... ]

<dynamic node label set specification with ampersand> ::=
  { "IS" | <colon> } <dynamic label specification> { <ampersand> <dynamic label specification> }...

<dynamic node label set specification with colon> ::=
  <colon> <dynamic label specification> { <colon> <dynamic label specification> }...

<dynamic node label set single specification> ::=
  { "IS" | <colon> } <dynamic label specification>

<relationship label specification for create and merge> ::=
  <dynamic relationship label specification>

<relationship label specification for insert> ::=
  <static relationship label specification>

<static relationship label specification> ::=
  { <colon> | "IS" } <label name>

<dynamic relationship label specification> ::=
  { <colon> | "IS" } <dynamic label specification>

<dynamic label specification> ::=
    <label name>
  | <dynamic label specification expression>

<dynamic label specification expression> ::=
  <dollar sign> <left paren> <value expression> <right paren>

<disjunctive label specification> ::=
  <label name> [ { <vertical bar> <label name> }... ]

<alias reference> ::=
    [ <regular catalog object parent reference> ] <regular alias name>
  | <alias name>

<database reference> ::=
    [ <regular catalog object parent reference> ] <regular database name>
  | <database name>

<alias or database reference> ::=
    <alias reference>
  | <database reference>

<graph reference> ::=
    [ <regular catalog object parent reference> ] <regular graph name>
  | <graph name>

<procedure reference> ::=
  [ <catalog object parent reference> ] <procedure name>

<function reference> ::=
  [ <catalog object parent reference> ] <function name>

<catalog object parent reference> ::=
```

```
  { <object name> <period> }...

<regular catalog object parent reference> ::=
  { <regular object name> <period> }...

<value expression> ::=
  <boolean value expression>

<search condition> ::=
  <boolean value expression>

<boolean value expression> ::=
    <boolean term xor>
  | <boolean value expression> "OR" <boolean term xor>

<boolean term xor> ::=
    <boolean term>
  | <boolean term xor> "XOR" <boolean term>

<boolean term> ::=
    <boolean factor>
  | <boolean term> "AND" <boolean factor>

<boolean factor> ::=
  [ { "NOT" }... ] <boolean primary>

<boolean primary> ::=
    <pattern expression>
  | <predicate>

<predicate> ::=
    <property exists predicate>
  | <comparison predicate>

<comparison predicate> ::=
  <simple comparison predicand> [ <simple comparison predicate part 2>... ]

<simple comparison predicand> ::=
  <advanced comparison predicand> [ <advanced comparison predicate part 2> ]

<simple comparison predicate part 2> ::=
  <simple comp op> <advanced comparison predicand>

<simple comp op> ::=
    <equals operator>
  | <not equals operator>
  | <less than operator>
  | <greater than operator>
  | <less than or equals operator>
  | <greater than or equals operator>

<advanced comparison predicand> ::=
  <arithmetic value expression>

<advanced comparison predicate part 2> ::=
    <advanced comp op> <advanced comparison predicand>
  | <value type predicate part 2>
  | <null predicate part 2>
  | <normalized predicate part 2>
  | <is labeled predicate part 2>

<advanced comp op> ::=
    <contains operator>
  | <in operator>
  | <regex equals operator>
  | <start with operator>
  | <end with operator>

<null predicate part 2> ::=
  "IS" [ "NOT" ] "NULL"

<normalized predicate part 2> ::=
  "IS" [ "NOT" ] [ <normal form> ] "NORMALIZED"

<normal form> ::=
    "NFC"
  | "NFD"
```

```
  | "NFKC"
  | "NFKD"

<value type predicate part 2> ::=
  { "IS" [ "NOT" ] { <double colon> | "TYPED" } | <double colon> } <value type>

<start with operator> ::=
  "STARTS WITH"

<end with operator> ::=
  "ENDS WITH"

<contains operator> ::=
  "CONTAINS"

<in operator> ::=
  "IN"

<is labeled predicate part 2> ::=
    <is label expression>
  | "IS" [ "NOT" ] "LABELED" <label expression>
  | "IS NOT" <label expression>

<property exists predicate> ::=
  "PROPERTY_EXISTS" <left paren> <binding variable reference> <comma> <property name> <right paren>

<arithmetic value expression> ::=
    <arithmetic term>
  | <arithmetic value expression> <plus sign> <arithmetic term>
  | <arithmetic value expression> <minus sign> <arithmetic term>
  | <arithmetic value expression> <concatenation operator> <arithmetic term>

<arithmetic term> ::=
    <arithmetic factor>
  | <arithmetic term> <asterisk> <arithmetic factor>
  | <arithmetic term> <solidus> <arithmetic factor>
  | <arithmetic term> <percent> <arithmetic factor>

<arithmetic factor> ::=
    <arithmetic unary>
  | <arithmetic factor> <circumflex> <arithmetic unary>

<arithmetic unary> ::=
  [ <sign> ] <postfix expression>

<postfix expression> ::=
    <value expression primary>
  | <postfix expression> <postfix operator>

<postfix operator> ::=
    <static property reference>
  | <dynamic element reference>
  | <slicing>

<static property reference> ::=
  <period> <property name>

<dynamic element reference> ::=
  <left bracket> <value expression> <right bracket>

<slicing> ::=
  <left bracket> [ <slicing from> ] <double period> [ <slicing to> ] <right bracket>

<slicing from> ::=
  <value expression>

<slicing to> ::=
  <value expression>

<value expression primary> ::=
    <parenthesized value expression>
  | <non-parenthesized value expression primary>

<parenthesized value expression> ::=
  <left paren> <value expression> <right paren>

<non-parenthesized value expression primary> ::=
```

```
    <general parameter reference>
  | <case expression>
  | <count star>
  | <exists expression>
  | <count expression>
  | <collect expression>
  | <map projection>
  | <list comprehension>
  | <pattern comprehension>
  | <reduce expression>
  | <all reduce expression>
  | <quantifier expression>
  | <trim function>
  | <vector value function>
  | <shortest path expression>
  | <function invocation>
  | <value specification>
  | <binding variable reference>

<case expression> ::=
    <simple case>
  | <search case>

<simple case> ::=
  "CASE" <case operand> <simple when clause>... [ <else clause> ] "END"

<search case> ::=
  "CASE" <searched when clause>... [ <else clause> ] "END"

<simple when clause> ::=
  "WHEN" <when operand list> "THEN" <value expression>

<searched when clause> ::=
  "WHEN" <search condition> "THEN" <value expression>

<case operand> ::=
  <value expression>

<when operand list> ::=
  <when operand> [ { <comma> <when operand> }... ]

<when operand> ::=
    <simple comparison predicate part 2>
  | <advanced comparison predicate part 2>
  | <value expression>

<else clause> ::=
  "ELSE" <value expression>

<exists expression> ::=
  "EXISTS" <left brace> <subquery expression argument> <right brace>

<count expression> ::=
  "COUNT" <left brace> <subquery expression argument> <right brace>

<collect expression> ::=
  "COLLECT" <nested procedure specification>

<subquery expression argument> ::=
    <procedure specification>
  | <graph pattern>

<map projection> ::=
  <binding variable> <left brace> [ <map projection element list> ] <right brace>

<map projection element list> ::=
  <map projection element> [ { <comma> <map projection element> }... ]

<map projection element> ::=
    <literal map field>
  | <field selector>
  | <variable selector>
  | <all fields selector>

<list comprehension> ::=
  <left bracket> <list element source> [ <list element filter and projection> ] <right bracket>
```

```
<list element filter and projection> ::=
    [ <list element filter> ] <list element projection>
  | <list element filter>

<list element source> ::=
  <binding variable> "IN" <value expression>

<list element filter> ::=
  "WHERE" <value expression>

<list element projection> ::=
  <vertical bar> <value expression>

<pattern comprehension> ::=
  <left bracket> <pattern source> <pattern filter and projection> <right bracket>

<pattern source> ::=
  [ <binding variable> <equals operator> ] <simple path pattern>

<pattern filter and projection> ::=
  [ <pattern filter> ] <pattern projection>

<pattern filter> ::=
  "WHERE" <value expression>

<pattern projection> ::=
  <vertical bar> <value expression>

<reduce expression> ::=
  "REDUCE" <left paren> <reduce accumulator initialization> <comma> <reduce element source> <reduce step>
<right paren>

<reduce accumulator initialization> ::=
  <binding variable> <equals operator> <value expression>

<reduce element source> ::=
  <binding variable> "IN" <value expression>

<reduce step> ::=
  <vertical bar> <value expression>

<all reduce expression> ::=
  "ALLREDUCE" <left paren> <all reduce accumulator initialization> <comma> <all reduce element source>
<all reduce step> <comma> <all reduce predicate> <right paren>

<all reduce accumulator initialization> ::=
  <binding variable> <equals operator> <value expression>

<all reduce element source> ::=
  <binding variable> "IN" <value expression>

<all reduce step> ::=
  <vertical bar> <value expression>

<all reduce predicate> ::=
  <boolean value expression>

<literal map field> ::=
  <property name> <colon> <value expression>

<field selector> ::=
  <static property reference>

<variable selector> ::=
  <binding variable>

<all fields selector> ::=
  <period> <asterisk>

<quantifier expression> ::=
  <quantifier> <left paren> <quantifier element source> <quantifier element predicate> <right paren>

<quantifier> ::=
    "ALL"
  | "ANY"
  | "SINGLE"
  | "NONE"
```

```
<quantifier element source> ::=
  <binding variable> "IN" <value expression>

<quantifier element predicate> ::=
  "WHERE" <value expression>

<trim function> ::=
  <single-character trim function>

<single-character trim function> ::=
  "TRIM" <left paren> <trim operands> <right paren>

<trim operands> ::=
  [ [ <trim specification> ] [ <trim character string> ] "FROM" ] <trim source>

<trim source> ::=
  <value expression>

<trim specification> ::=
    "LEADING"
  | "TRAILING"
  | "BOTH"

<trim character string> ::=
  <value expression>

<vector value function> ::=
  "VECTOR" <left paren> <vector coordinate values> <comma> <dimension> <comma> <coordinate type> <right
paren>

<vector coordinate values> ::=
  <value expression>

<function invocation> ::=
  <function reference> <left paren> [ <set quantifier> ] [ <function argument list> ] <right paren>

<function argument list> ::=
  <function argument> [ { <comma> <function argument> }... ]

<function argument> ::=
  <value expression>

<count star> ::=
  "COUNT" <left paren> <asterisk> <right paren>

<pattern expression> ::=
  <simple path pattern>

<shortest path expression> ::=
  <legacy shortest path pattern>

<value specification> ::=
    <literal>
  | <general parameter reference>
  | <list value constructor>
  | <map value constructor>

<list value constructor> ::=
  <left bracket> [ <list element list> ] <right bracket>

<list element list> ::=
  <value expression> [ { <comma> <value expression> }... ]

<character string list literal> ::=
  <left bracket> [ <character string list element list> ] <right bracket>

<character string list element list> ::=
  <character string literal> [ { <comma> <character string literal> }... ]

<map value constructor> ::=
  <record value constructor>

<record value constructor> ::=
  <fields specification>

<fields specification> ::=
```

```
  <left brace> [ <field list> ] <right brace>

<field list> ::=
  <field> [ { <comma> <field> }... ]

<field> ::=
  <field name> <colon> <value expression>

<map value or parameter specification> ::=
    <map value constructor>
  | <general parameter reference>

<unsigned decimal integer specification> ::=
    <unsigned decimal integer>
  | <general parameter reference>

<password value specification> ::=
  <character string value specification>

<character string value specification> ::=
    <character string literal>
  | <general parameter reference>

<character string or string list value specification> ::=
    <character string literal>
  | <character string list literal>
  | <general parameter reference>

<character string list element list specification> ::=
    <character string list element list>
  | <general parameter reference>

<character string list element list or expression> ::=
    <character string list element list>
  | <value expression>

<value type> ::=
    <value type primary>
  | <undelimited closed dynamic union type>

<value type primary> ::=
    <predefined type>
  | <constructed value type>
  | <delimited dynamic union type>

<typed> ::=
    <double colon>
  | "TYPED"

<predefined type> ::=
    <boolean type>
  | <character string type>
  | <numeric type>
  | <temporal type>
  | <point type>
  | <vector type>
  | <uuid type>
  | <reference value type>
  | <immaterial value type>

<boolean type> ::=
  { "BOOL" | "BOOLEAN" } [ <not null> ]

<character string type> ::=
    "STRING" [ <not null> ]
  | "VARCHAR" [ <not null> ]

<numeric type> ::=
    <exact numeric type>
  | <approximate numeric type>

<exact numeric type> ::=
  <binary exact numeric type>

<binary exact numeric type> ::=
  <signed binary exact numeric type>
```

```
<signed binary exact numeric type> ::=
    "INT" [ <not null> ]
  | [ "SIGNED" ] "INTEGER" [ <not null> ]

<approximate numeric type> ::=
  "FLOAT" [ <not null> ]

<temporal type> ::=
    <temporal instant type>
  | <temporal duration type>

<temporal instant type> ::=
    <datetime type>
  | <localdatetime type>
  | <date type>
  | <time type>
  | <localtime type>

<datetime type> ::=
  "ZONED DATETIME" [ <not null> ]

<localdatetime type> ::=
  "LOCAL DATETIME" [ <not null> ]

<date type> ::=
  "DATE" [ <not null> ]

<time type> ::=
  "ZONED TIME" [ <not null> ]

<localtime type> ::=
  "LOCAL TIME" [ <not null> ]

<temporal duration type> ::=
  "DURATION" [ <not null> ]

<point type> ::=
  "POINT" [ <not null> ]

<vector type> ::=
    "VECTOR" <left paren> <dimension> <comma> <coordinate type> <right paren> [ <not null> ]
  | "VECTOR" [ <left angle bracket> <coordinate type> <right angle bracket> ] [ <left paren> <dimension>
<right paren> ] [ <not null> ]

<dimension> ::=
  <unsigned decimal integer>

<coordinate type> ::=
    <numeric type>
  | <vector-only numeric coordinate type>

<vector-only numeric coordinate type> ::=
  { "FLOAT64" | "FLOAT32" | "INTEGER64" | "INT64" | "INTEGER32" | "INT32" | "INTEGER16" | "INT16" |
"INTEGER8" | "INT8" } [ <not null> ]

<uuid type> ::=
  "UUID" [ <not null> ]

<reference value type> ::=
    <node reference value type>
  | <edge reference value type>

<node reference value type> ::=
  <open node reference value type>

<open node reference value type> ::=
  [ "ANY" ] <node synonym> [ <not null> ]

<edge reference value type> ::=
  <open edge reference value type>

<open edge reference value type> ::=
  [ "ANY" ] <edge synonym> [ <not null> ]

<immaterial value type> ::=
    <null type>
  | <empty type>
```

```
<null type> ::=
  "NULL"

<constructed value type> ::=
    <path value type>
  | <list value type>
  | <map type>

<path value type> ::=
  "PATH" [ <not null> ]

<list value type> ::=
  { <list value type cypher-notation> | <list value type sql-notation> } [ <not null> ]

<list value type cypher-notation> ::=
  <list value type name> <left angle bracket> <value type> <right angle bracket>

<list value type sql-notation> ::=
    <basic list element type> <list value type name>
  | <list value type sql-notation> <list value type name>

<list value type name> ::=
    "LIST"
  | "ARRAY"

<basic list element type> ::=
    <predefined type>
  | <path value type>
  | <map type>
  | <delimited dynamic union type>

<map type> ::=
  [ "ANY" ] "MAP" [ <not null> ]

<empty type> ::=
    "NULL" <not null>
  | "NOTHING"

<delimited dynamic union type> ::=
    <open dynamic union type>
  | <dynamic property value type>
  | <delimited closed dynamic union type>

<open dynamic union type> ::=
  "ANY" [ "VALUE" ] [ <not null> ]

<dynamic property value type> ::=
  [ "ANY" ] "PROPERTY VALUE" [ <not null> ]

<delimited closed dynamic union type> ::=
  "ANY" [ "VALUE" ] <left angle bracket> <delimited component type list> <right angle bracket>

<delimited component type list> ::=
  <delimited component type> [ { <vertical bar> <delimited component type> }... ]

<undelimited closed dynamic union type> ::=
  <undelimited component type> { <vertical bar> <undelimited component type> }...

<delimited component type> ::=
  <value type>

<undelimited component type> ::=
  <value type primary>

<not null> ::=
    "NOT NULL"
  | <exclamation mark>

<parameter name> ::=
  <separated identifier>

<label name list> ::=
  <label name> [ { <comma> <label name> }... ]

<label name> ::=
  <identifier>
```

```
<property name list> ::=
  <property name> [ { <comma> <property name> }... ]

<property name> ::=
  <identifier>

<field name> ::=
  <identifier>

<binding variable reference> ::=
  <binding variable>

<binding variable> ::=
  <identifier>

<binding variable reference list> ::=
  <binding variable reference> [ { <comma> <binding variable reference> }... ]

<graph specification list> ::=
  <graph name specification> [ { <comma> <graph name specification> }... ]

<graph name specification> ::=
    <graph reference>
  | <general parameter reference>

<graph name> ::=
  <identifier>

<regular graph name> ::=
    <regular identifier>
  | <non-reserved word>

<procedure name> ::=
  <identifier>

<function name> ::=
  <identifier>

<alias name specification> ::=
    <alias reference>
  | <general parameter reference>

<alias name> ::=
  <identifier>

<regular alias name> ::=
    <regular identifier>
  | <non-reserved word>

<alias target name specification> ::=
    <alias or database reference>
  | <general parameter reference>

<constraint name specification> ::=
    <constraint name>
  | <general parameter reference>

<constraint name> ::=
  <identifier>

<database name specification> ::=
    <database reference>
  | <general parameter reference>

<database name> ::=
  <identifier>

<regular database name> ::=
    <regular identifier>
  | <non-reserved word>

<alias or database specification list> ::=
  <alias or database name specification> [ { <comma> <alias or database name specification> }... ]

<alias or database name specification> ::=
    <alias or database reference>
```

```
  | <general parameter reference>

<index name specification> ::=
    <index name>
  | <general parameter reference>

<index name> ::=
  <identifier>

<role name specification list> ::=
  <role name specification> [ { <comma> <role name specification> }... ]

<role name specification> ::=
    <role name>
  | <general parameter reference>

<role name> ::=
  <identifier>

<auth rule name specification list> ::=
  <auth rule name specification> [ { <comma> <auth rule name specification> }... ]

<auth rule name specification> ::=
    <auth rule name>
  | <general parameter reference>

<auth rule name> ::=
  <identifier>

<server name specification> ::=
    <server name>
  | <general parameter reference>

<server name> ::=
  <identifier>

<user name specification list> ::=
  <user name specification> [ { <comma> <user name specification> }... ]

<user name specification> ::=
    <user name>
  | <general parameter reference>

<user name> ::=
  <identifier>

<object name> ::=
  <identifier>

<regular object name> ::=
    <regular identifier>
  | <non-reserved word>

<literal> ::=
    <signed numeric literal>
  | <general literal>

<general literal> ::=
    <boolean literal>
  | <character string literal>
  | <interpolated character string literal>
  | <null literal>
  | <list literal>
  | <map literal>

<boolean literal> ::=
    "TRUE"
  | "FALSE"

<null literal> ::=
  "NULL"

<character string literal> ::=
    <single quoted character sequence>
  | <double quoted character sequence>

<single quoted character sequence> ::=
```

```
  <unbroken single quoted character sequence>

<double quoted character sequence> ::=
  <unbroken double quoted character sequence>

<accent quoted character sequence> ::=
  <unbroken accent quoted character sequence>

<unbroken single quoted character sequence> ::=
  <quote> [ <single quoted character representation>... ] <quote>

<unbroken double quoted character sequence> ::=
  <double quote> [ <double quoted character representation>... ] <double quote>

<unbroken accent quoted character sequence> ::=
  <grave accent> [ <accent quoted character representation>... ] <grave accent>

<single quoted character representation> ::=
    any character but: \, '
  | <double single quote>
  | <escaped character>

<double quoted character representation> ::=
    any character but: \, "
  | <double double quote>
  | <escaped character>

<accent quoted character representation> ::=
    any character but: \, `
  | <double grave accent>
  | <escaped character>

<double single quote> ::=
  <quote> <quote>

<double double quote> ::=
  <double quote> <double quote>

<double grave accent> ::=
  <grave accent> <grave accent>

<escaped character> ::=
    <escaped reverse solidus>
  | <escaped quote>
  | <escaped double quote>
  | <escaped grave accent>
  | <escaped tab>
  | <escaped backspace>
  | <escaped newline>
  | <escaped carriage return>
  | <escaped form feed>
  | <unicode escape value>

<escaped reverse solidus> ::=
  <reverse solidus> <reverse solidus>

<escaped quote> ::=
  <reverse solidus> <quote>

<escaped double quote> ::=
  <reverse solidus> <double quote>

<escaped grave accent> ::=
  <reverse solidus> <grave accent>

<escaped tab> ::=
  <reverse solidus> "t"

<escaped backspace> ::=
  <reverse solidus> "b"

<escaped newline> ::=
  <reverse solidus> "n"

<escaped carriage return> ::=
  <reverse solidus> "r"
```

```
<escaped form feed> ::=
  <reverse solidus> "f"

<unicode escape value> ::=
    <unicode 4 digit escape value>
  | <unicode 6 digit escape value>

<unicode 4 digit escape value> ::=
  <reverse solidus> "u" <hex digit> <hex digit> <hex digit> <hex digit>

<unicode 6 digit escape value> ::=
  <reverse solidus> "U" <hex digit> <hex digit> <hex digit> <hex digit> <hex digit> <hex digit>

<interpolated character string literal> ::=
    <single quoted non-interpolated character string literal>
  | <double quoted non-interpolated character string literal>
  | <single quoted interpolated character string literal>
  | <double quoted interpolated character string literal>

<single quoted non-interpolated character string literal> ::=
  <unbroken single quoted non-interpolated character sequence>

<double quoted non-interpolated character string literal> ::=
  <unbroken double quoted non-interpolated character sequence>

<single quoted interpolated character string literal> ::=
  <unbroken single quoted interpolated character sequence left> <value expression> [ { <unbroken single
quoted interpolated character sequence middle> <value expression> }... ] <unbroken single quoted
interpolated character sequence right>

<double quoted interpolated character string literal> ::=
  <unbroken double quoted interpolated character sequence left> <value expression> [ { <unbroken double
quoted interpolated character sequence middle> <value expression> }... ] <unbroken double quoted
interpolated character sequence right>

<unbroken single quoted interpolated character sequence left> ::=
  { "s" | "S" } <quote> push(') [ <single quoted interpolated character representation>... ] <left brace>

<unbroken double quoted interpolated character sequence left> ::=
  { "s" | "S" } <double quote> push(") [ <double quoted interpolated character representation>... ] <left
brace>

<unbroken single quoted interpolated character sequence right> ::=
  check(') <right brace> [ <single quoted interpolated character representation>... ] <quote> pop()

<unbroken double quoted interpolated character sequence right> ::=
  check(") <right brace> [ <double quoted interpolated character representation>... ] <double quote> pop()

<unbroken single quoted interpolated character sequence middle> ::=
  check(') <right brace> [ <single quoted interpolated character representation>... ] <left brace>

<unbroken double quoted interpolated character sequence middle> ::=
  check(") <right brace> [ <double quoted interpolated character representation>... ] <left brace>

<unbroken single quoted non-interpolated character sequence> ::=
  { "s" | "S" } <quote> [ <single quoted interpolated character representation>... ] <quote>

<unbroken double quoted non-interpolated character sequence> ::=
  { "s" | "S" } <double quote> [ <double quoted interpolated character representation>... ] <double quote>

<single quoted interpolated character representation> ::=
    any character but: \, ', {, }
  | <double single quote>
  | <escaped character>
  | <escaped left brace>
  | <escaped right brace>

<double quoted interpolated character representation> ::=
    any character but: \, ", {, }
  | <double double quote>
  | <escaped character>
  | <escaped left brace>
  | <escaped right brace>

<escaped left brace> ::=
  <reverse solidus> <left brace>
```

```
<escaped right brace> ::=
  <reverse solidus> <right brace>

<signed numeric literal> ::=
    [ <minus sign> ] { <unsigned numeric literal> | "INF" | "INFINITY" }
  | "NAN"

<sign> ::=
    <plus sign>
  | <minus sign>

<unsigned numeric literal> ::=
    <exact numeric literal>
  | <approximate numeric literal>

<exact numeric literal> ::=
  <unsigned integer>

<unsigned decimal in scientific notation> ::=
  <mantissa> <exponent indicator> <exponent>

<exponent indicator> ::=
    "E"
  | "e"

<mantissa> ::=
    <unsigned decimal in common notation>
  | <unsigned decimal integer>

<exponent> ::=
  <signed decimal integer>

<unsigned decimal in common notation> ::=
    <unsigned decimal integer> <period> <unsigned decimal integer>
  | <period> <unsigned decimal integer>

<unsigned integer> ::=
    <unsigned decimal integer>
  | <unsigned hexadecimal integer>
  | <unsigned octal integer>

<signed decimal integer> ::=
  [ <sign> ] <unsigned decimal integer>

<unsigned decimal integer> ::=
  <digit> [ { [ <underscore> ] <digit> }... ]

<unsigned hexadecimal integer> ::=
  { "0x" | "0X" } { [ <underscore> ] <hex digit> }...

<unsigned octal integer> ::=
  "0o" { [ <underscore> ] <octal digit> }...

<approximate numeric literal> ::=
    <unsigned decimal in scientific notation> [ <approximate number suffix> ]
  | <unsigned decimal in common notation> [ <approximate number suffix> ]

<approximate number suffix> ::=
    "F"
  | "D"
  | "f"
  | "d"

<list literal> ::=
  <left bracket> [ <list element list literal> ] <right bracket>

<list element list literal> ::=
  <literal> [ { <comma> <literal> }... ]

<map literal> ::=
  <record literal>

<record literal> ::=
  <fields literal>

<fields literal> ::=
  <left brace> [ <field list literal> ] <right brace>
```

```
<field list literal> ::=
  <field literal> [ { <comma> <field literal> }... ]

<field literal> ::=
  <field name> <colon> <literal>

<edge synonym> ::=
    "EDGE"
  | "RELATIONSHIP"

<node synonym> ::=
    "NODE"
  | "VERTEX"

<home database keyword> ::=
  "HOME DATABASE"

<alias keywords> ::=
    "ALIAS"
  | "ALIASES"

<admin keywords> ::=
    "ADMIN"
  | "ADMINISTRATOR"

<command keywords> ::=
    "COMMAND"
  | "COMMANDS"

<constraint keywords> ::=
    "CONSTRAINT"
  | "CONSTRAINTS"

<database keywords> ::=
    "DATABASE"
  | "DATABASES"

<element keywords> ::=
    "ELEMENT"
  | "ELEMENTS"

<function keywords> ::=
    "FUNCTION"
  | "FUNCTIONS"

<index keywords> ::=
    "INDEX"
  | "INDEXES"

<name keywords> ::=
    "NAME"
  | "NAMES"

<node keywords> ::=
    "NODE"
  | "NODES"

<label keywords> ::=
    "LABEL"
  | "LABELS"

<password keywords> ::=
    "PASSWORD"
  | "PASSWORDS"

<procedure keywords> ::=
    "PROCEDURE"
  | "PROCEDURES"

<path keywords> ::=
    "PATH"
  | "PATHS"

<primary keywords> ::=
    "PRIMARY"
  | "PRIMARIES"
```

```
<relationship keywords> ::=
    "RELATIONSHIP"
  | "RELATIONSHIPS"

<relationship singular keywords> ::=
    "RELATIONSHIP"
  | "REL"

<replica keywords> ::=
    "REPLICA"
  | "REPLICAS"

<role keywords> ::=
    "ROLE"
  | "ROLES"

<privilege keywords> ::=
    "PRIVILEGE"
  | "PRIVILEGES"

<secondary keywords> ::=
    "SECONDARY"
  | "SECONDARIES"

<seconds keywords> ::=
    "SEC"
  | "SECOND"
  | "SECONDS"

<server keywords> ::=
    "SERVER"
  | "SERVERS"

<setting keywords> ::=
    "SETTING"
  | "SETTINGS"

<shard keywords> ::=
    "SHARD"
  | "SHARDS"

<type keywords> ::=
    "TYPE"
  | "TYPES"

<transaction keywords> ::=
    "TRANSACTION"
  | "TRANSACTIONS"

<user keywords> ::=
    "USER"
  | "USERS"

<auth rule keywords> ::=
    "AUTH RULE"
  | "AUTH RULES"

<provider keywords> ::=
    "PROVIDER"
  | "PROVIDERS"

<tag keywords> ::=
    "TAG"
  | "TAGS"

<identifier> ::=
    <regular identifier>
  | <delimited identifier>
  | <non-reserved word>

<separated identifier> ::=
    <extended identifier>
  | <delimited identifier>

<regular identifier> ::=
  <identifier start> [ <identifier extend>... ]
```

```
<extended identifier> ::=
  <identifier extend>...

<general parameter reference> ::=
  <dollar sign> <parameter name>

<delimited identifier> ::=
  <accent quoted character sequence>

<identifier start> ::=
  unicode: XID_START

<identifier extend> ::=
  unicode: XID_CONTINUE

<non-reserved word> ::=
    "ACCESS"
  | "ACTIVE"
  | "ACYCLIC"
  | "ADD"
  | "ADMIN"
  | "ADMINISTRATOR"
  | "ALIAS"
  | "ALIASES"
  | "ALLREDUCE"
  | "ALLSHORTESTPATHS"
  | "ALL"
  | "ALTER"
  | "ANALYZER"
  | "AND"
  | "ANY"
  | "ARRAY"
  | "AS"
  | "ASC"
  | "ASCENDING"
  | "ASSIGN"
  | "AT"
  | "AUTH"
  | "AUTO"
  | "BINDINGS"
  | "BOOL"
  | "BOOLEAN"
  | "BOOSTED"
  | "BOTH"
  | "BREAK"
  | "BUILT"
  | "BY"
  | "CALL"
  | "CASCADE"
  | "CASE"
  | "CHANGE"
  | "CIDR"
  | "COLLECT"
  | "COMMAND"
  | "COMMANDS"
  | "COMPOSITE"
  | "CONCURRENT"
  | "CONDITION"
  | "CONSTRAINT"
  | "CONSTRAINTS"
  | "CONTAINS"
  | "CONTINUE"
  | "COPY"
  | "COUNT"
  | "CREATE"
  | "CREDENTIAL"
  | "CREDENTIALS"
  | "CSV"
  | "CYPHER"
  | "CURRENT"
  | "DATA"
  | "DATABASE"
  | "DATABASES"
  | "DATE"
  | "DATETIME"
  | "DBMS"
```

```
  | "DEALLOCATE"
  | "DEFAULT"
  | "DEFINED"
  | "DELETE"
  | "DENY"
  | "DESC"
  | "DESCENDING"
  | "DESTROY"
  | "DETACH"
  | "DIFFERENT"
  | "DISJOINT"
  | "DISTINCT"
  | "DRIVER"
  | "DROP"
  | "DRYRUN"
  | "DUMP"
  | "DURATION"
  | "EACH"
  | "EDGE"
  | "ELEMENT"
  | "ELEMENTS"
  | "ELSE"
  | "ENABLE"
  | "ENABLED"
  | "ENCRYPTED"
  | "END"
  | "ENDS"
  | "ERROR"
  | "EXECUTABLE"
  | "EXECUTE"
  | "EXIST"
  | "EXISTENCE"
  | "EXISTS"
  | "FAIL"
  | "FALSE"
  | "FIELDTERMINATOR"
  | "FILTER"
  | "FINISH"
  | "FLOAT"
  | "FLOAT64"
  | "FLOAT32"
  | "FOREACH"
  | "FOR"
  | "FORWARDING"
  | "FROM"
  | "FULLTEXT"
  | "FUNCTION"
  | "FUNCTIONS"
  | "GRANT"
  | "GRAPH"
  | "GRAPHS"
  | "GROUP"
  | "GROUPS"
  | "HEADERS"
  | "HOME"
  | "ID"
  | "IF"
  | "IMMUTABLE"
  | "IMPERSONATE"
  | "IMPLIES"
  | "IN"
  | "INDEX"
  | "INDEXES"
  | "INF"
  | "INFINITY"
  | "INSERT"
  | "INT"
  | "INT64"
  | "INT32"
  | "INT16"
  | "INT8"
  | "INTEGER"
  | "INTEGER64"
  | "INTEGER32"
  | "INTEGER16"
  | "INTEGER8"
  | "IS"
```

```
  | "JOIN"
  | "KEY"
  | "LANGUAGE"
  | "LABEL"
  | "LABELED"
  | "LABELS"
  | "LEADING"
  | "LET"
  | "LIMIT"
  | "LIST"
  | "LOAD"
  | "LOCAL"
  | "LOOKUP"
  | "MATCH"
  | "MANAGEMENT"
  | "MAP"
  | "MERGE"
  | "METADATA"
  | "NAME"
  | "NAMES"
  | "NAN"
  | "NEW"
  | "NEXT"
  | "NFC"
  | "NFD"
  | "NFKC"
  | "NFKD"
  | "NODE"
  | "NODETACH"
  | "NODES"
  | "NONE"
  | "NORMALIZED"
  | "NOT"
  | "NOTHING"
  | "NOWAIT"
  | "NULL"
  | "OF"
  | "OFFSET"
  | "OIDC"
  | "ON"
  | "ONLY"
  | "OPTIONAL"
  | "OPTIONS"
  | "OPTION"
  | "OR"
  | "ORDER"
  | "PASSWORD"
  | "PASSWORDS"
  | "PATH"
  | "PATHS"
  | "PLAINTEXT"
  | "POINT"
  | "POPULATED"
  | "PRIMARY"
  | "PRIMARIES"
  | "PRIVILEGE"
  | "PRIVILEGES"
  | "PROCEDURE"
  | "PROCEDURES"
  | "PROPERTIES"
  | "PROPERTY"
  | "PROPERTY_EXISTS"
  | "PROVIDER"
  | "PROVIDERS"
  | "RANGE"
  | "READ"
  | "REALLOCATE"
  | "REDUCE"
  | "REL"
  | "RELATIONSHIP"
  | "RELATIONSHIPS"
  | "REMOVE"
  | "RENAME"
  | "REPEATABLE"
  | "REPLACE"
  | "REPLICA"
  | "REPLICAS"
```

```
  | "REPORT"
  | "RETRY"
  | "REQUIRE"
  | "REQUIRED"
  | "RESTRICT"
  | "RETURN"
  | "REVOKE"
  | "ROLE"
  | "ROLES"
  | "ROW"
  | "ROWS"
  | "RULE"
  | "RULES"
  | "SCAN"
  | "SCORE"
  | "SEARCH"
  | "SEC"
  | "SECOND"
  | "SECONDS"
  | "SECONDARIES"
  | "SECONDARY"
  | "SEEK"
  | "SERVER"
  | "SERVERS"
  | "SET"
  | "SETTING"
  | "SETTINGS"
  | "SHARD"
  | "SHARDS"
  | "SHORTEST"
  | "SHORTESTPATH"
  | "SHOW"
  | "SIGNED"
  | "SINGLE"
  | "SKIP"
  | "START"
  | "STARTS"
  | "STATUS"
  | "STOP"
  | "STRING"
  | "SUPPORTED"
  | "SUSPENDED"
  | "TAG"
  | "TAGS"
  | "TARGET"
  | "TERMINATE"
  | "TEXT"
  | "THEN"
  | "TIME"
  | "TO"
  | "TOPOLOGY"
  | "TRAIL"
  | "TRAILING"
  | "TRANSACTION"
  | "TRANSACTIONS"
  | "TRAVERSE"
  | "TRIM"
  | "TRUE"
  | "TYPE"
  | "TYPED"
  | "TYPES"
  | "UNION"
  | "UNIQUE"
  | "UNIQUENESS"
  | "UNWIND"
  | "USE"
  | "USER"
  | "USERS"
  | "USING"
  | "UUID"
  | "VALUE"
  | "VARCHAR"
  | "VECTOR"
  | "VERTEX"
  | "WAIT"
  | "WALK"
  | "WHEN"
```

```
  | "WHERE"
  | "WITH"
  | "WRITE"
  | "XOR"
  | "YIELD"
  | "ZONED"

<concatenation operator> ::=
  "||"

<double colon> ::=
  "::"

<double period> ::=
  ".."

<regex equals operator> ::=
  "=~"

<greater than operator> ::=
  <right angle bracket>

<greater than or equals operator> ::=
  ">="

<less than operator> ::=
  <left angle bracket>

<less than or equals operator> ::=
  "<="

<not equals operator> ::=
  "<>"

<plus equals> ::=
  "+="

<right double arrow> ::=
  "=>"

<ampersand> ::=
  "&"

<asterisk> ::=
  "*"

<colon> ::=
  ":"

<comma> ::=
  ","

<dollar sign> ::=
  "$"

<double quote> ::=
  """

<equals operator> ::=
  "="

<exclamation mark> ::=
  "!"

<right angle bracket> ::=
  ">"

<grave accent> ::=
  "`"

<left brace> ::=
  "{"

<left bracket> ::=
  "["

<left paren> ::=
```

```
  "("

<left angle bracket> ::=
  "<"

<minus sign> ::=
  "-"

<percent> ::=
  "%"

<period> ::=
  "."

<plus sign> ::=
  "+"

<question mark> ::=
  "?"

<quote> ::=
  "'"

<reverse solidus> ::=
  "\"

<right brace> ::=
  "}"

<right bracket> ::=
  "]"

<right paren> ::=
  ")"

<solidus> ::=
  "/"

<underscore> ::=
  "_"

<vertical bar> ::=
  "|"

<circumflex> ::=
  "^"

<hex digit> ::=
    <standard digit>
  | "A"
  | "B"
  | "C"
  | "D"
  | "E"
  | "F"
  | "a"
  | "b"
  | "c"
  | "d"
  | "e"
  | "f"

<digit> ::=
  <standard digit>

<standard digit> ::=
    <octal digit>
  | "8"
  | "9"

<octal digit> ::=
    <binary digit>
  | "2"
  | "3"
  | "4"
  | "5"
  | "6"
```

```
  | "7"

<binary digit> ::=
    "0"
  | "1"
```

GQL conformance

Last updated: 1 June 2026
Neo4j version: 2026.06

GQL is the new ISO International Standard query language for graph databases.

GQL has adopted much of Cypher's query construction semantics, such as adhering to the `MATCH/RETURN` format. Consequently, Cypher now accommodates most mandatory GQL features and a substantial portion of its optional ones (defined by the ISO/IEC 39075:2024(en) - Information technology - Database languages - GQL Standard). Users should, therefore, only expect minimal differences between crafting queries in Cypher and GQL. For example, the following query is valid in both languages:

Cypher and GQL

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE a.name = 'Tom Hanks'
RETURN m.title
```

Cypher supports the majority of mandatory GQL features. For a full list see Supported mandatory GQL features. There are, however, currently a few mandatory GQL features not yet in Cypher that Neo4j is actively working towards implementing. These are listed in the page Currently unsupported mandatory GQL features.

Neo4j is also working towards increasing its support of optional GQL features. These are listed in the page Supported optional GQL features.

Some optional GQL features not yet implemented in Cypher already have analogous Cypher equivalents. These features are listed in the page Optional GQL features and analogous Cypher .

Additionally, Cypher contains additional features that are not part of GQL and no GQL alternatives currently exist for them. These features are listed in the page Additional Cypher features.

Note on minimum GQL conformance

Following the GQL Standard subclause 24.2, *Minimum conformance*, Cypher's support of the following mandatory GQL features is explicitly declared:

- Graph with an open graph type (Feature GG01).
- The Unicode Standard version used by Cypher depends on the running JVM version. Neo4j 5 added support for JavaSE 17 and version 13 of The Unicode Standard. Neo4j 5.14 added support for JavaSE 21 and version 15 of the Unicode Standard. For more information, see Parsing → Using Unicode in Cypher.
- Cypher supports the following mandatory GQL property types: `BOOLEAN` (`BOOL`), `FLOAT` [1], `INTEGER`

(`SIGNED INTEGER`, or `INT`)[2], and `STRING` (`VARCHAR`).

Cypher also supports the following optional GQL property types: `DATE`, `DURATION`, `LIST<INNER_TYPE NOT NULL>` (`ARRAY<INNER_TYPE NOT NULL>`, `INNER_TYPE LIST`, or `INNER_TYPE ARRAY`)[3], `LOCAL DATETIME` (`TIMESTAMP WITHOUT TIME ZONE`), `LOCAL TIME` (`TIME WITHOUT TIME ZONE`), `POINT`, `ZONED DATETIME` (`TIME WITH TIME ZONE`), and `ZONED TIME` (`TIMESTAMP WITH TIME ZONE`). For more information, see Values and types → property types.

Supported mandatory GQL features

Unlike optional GQL features, mandatory GQL features are not assigned a GQL feature ID code. The below table is instead listed in order of their appearance in the ISO/IEC 39075:2024(en) GQL Standard.

GQL Standard subclause	Description	Documentation	Comment
4.9.2	GQL-status objects	Status Codes for Errors & Notifications	Neo4j exposes successful execution results, errors, exceptions, and warnings as GQL-status objects.
4.11	Graph pattern matching	Patterns	
4.13	GQL object types	Structural types, Types and their synonyms.	Includes: `NODE` (`ANY NODE`, `VERTEX`, `ANY VERTEX`) and `RELATIONSHIP` (`ANY RELATIONSHIP`, `EDGE`, `ANY EDGE`).
4.16	Predefined value types	Property types, Types and their synonyms.	Includes: `BOOLEAN` (`BOOL`), `FLOAT`, `INTEGER` (`SIGNED INTEGER`, `INT`), and `STRING` (`VARCHAR`). Cypher supports the boolean type predicate for `true`, `false`, and `null` but does not support the GQL keyword `UNKNOWN`.
9.1	<nested procedure specification>	Combining `UNION` and `UNION ALL`	
13.2	<insert statement>	`INSERT`	
13.3	<set statement>	`SET`	GQL's `SET` has no order dependencies because all right-hand side operations are completed before any assignments occur. In Cypher's `SET`, the order of rows can affect the outcome because changes made during execution may depend on the sequence of assignments. The only way to guarantee row order in Neo4j is to use `ORDER BY`.
13.4	<remove statement>	`REMOVE`	
13.5	<delete statement>	`DELETE`	
14.4	<match statement>	`MATCH`, `OPTIONAL MATCH`	
14.9	<order by and page statement>	`ORDER BY`, `SKIP`, `OFFSET`, `LIMIT`	
14.10	<primitive result statement>	`FINISH`	

GQL Standard subclause	Description	Documentation	Comment
14.11	<return statement>	`RETURN`	
15.4	<conditional statement>	Conditional queries (`WHEN`)	
16.2	<limit clause>	`LIMIT`	
16.4	<graph pattern>	Graph patterns	
16.5	<insert graph pattern>	<<, `CREATE` >>	
16.6	<order by clause>	`ORDER-BY`	
16.7	<path pattern expression>	Path patterns	
16.8	<label expression>	Label expressions	
16.9	<path variable reference>	Patterns	
16.11	<graph pattern quantifier>	Quantifiers	
16.17	<sort specification list>	Ascending and descending order	
16.19	<offset clause>	`SKIP`, `OFFSET`	
19.3	<comparison predicate>	Comparison operators	
19.4	<exists predicate>	`exists()`	
19.5	<null predicate>	Type predicate expressions for `null`	
19.6	<value type predicate>	<<>>	
19.7	<normalized predicate>	`IS NORMALIZED`, `IS NOT NORMALIZED`	
20.2	<value expression primary>	Expressions	
20.3	<value specification>		GQL defines the `SESSION_USER` value expression, which enables accessing a user's username within a query. In Cypher, current user details can be seen using the `SHOW CURRENT USER` command.
20.7	<case expression>	`CASE`, `nullIf()`, `coalesce()`	
20.9	<aggregate function>	`avg()`, `count()`, `max`, `min()`, `sum()`	Cypher and GQL handle `null` values differently for the `sum()` function when queries return 0 rows. For example, `RETURN sum(<expr>)` on an empty table returns `null` in GQL, but in Cypher it returns `0`.
20.11	<property reference>	Core concepts	
20.21	<numeric value expression>	Mathematical operators	

GQL Standard subclause	Description	Documentation	Comment
20.22	<numeric value function>	`char_length()`, `character_length()`	
20.23	<string value expression>	`STRING` concatenation operator (`||`)	
20.24	<character string function>	`left()`, `lower()`, `normalize()`, `right()`, `trim()`, `upper()`	In GQL, `TRIM()` removes only space characters. In Cypher, `trim()` removes any whitespace character.
21.1	Names and variables	Syntax	Cypher supports GQL's lexical elements, with the following caveat: • GQL allows identifiers that are not variables to be delimited with both backticks and quotes. Cypher only allows backticks. For example: `MATCH (n) RETURN n."a prop"` is allowed in GQL but not Cypher.
22.15	Grouping operations	Counting with and without duplicates	

Currently unsupported mandatory GQL features

Cypher supports most mandatory GQL features. There are, however, currently a few mandatory GQL features not yet in Cypher that Neo4j is actively working towards implementing. The table below provides an overview of these GQL features and, where applicable, their functional equivalents in Neo4j.

Unlike optional GQL features, mandatory GQL features are not assigned a GQL feature ID code. The below table is instead listed in order of their appearance in the ISO/IEC 39075:2024(en) GQL Standard.

GQL Standard subclause	Description	Comment and similar Neo4j functionality
7.1-7.3	Session management	GQL defines the following session commands: `SESSION SET`, `SESSION RESET`, and `SESSION CLOSE`. Neo4j offers session management through the driver session API.
8.1-8.4	Transaction management	GQL defines the following transaction commands: `START TRANSACTION`, `COMMIT`, and `ROLLBACK`. Neo4j offers transaction management through the driver transaction API. Cypher Shell also offers specific commands to manage transactions.
11.1	Graph expressions	GQL defines the following graph reference values commands: `CURRENT_GRAPH` and `CURRENT_PROPERTY_GRAPH`.

GQL Standard subclause	Description	Comment and similar Neo4j functionality
17.1	Schema reference	GQL defines an `AT` clause for selecting the current schema and the following schema selection options: `HOME_SCHEMA` and `CURRENT_SCHEMA`.
21.3	<token>, <separator>, and <identifier>	GQL specifies a list of reserved words that cannot be used for unquoted variable names, labels, and property names. Cypher also specifies a list of reserved keywords, but it differs from GQL's.

Supported optional GQL features

This page lists the optional GQL features Cypher is either fully or partially conformant with.

Optional GQL features are assigned a feature ID code. These codes order the features in the table below.

GQL Feature ID	Description	Documentation	Comment
G002	Different-edges match mode	`DIFFERENT RELATIONSHIPS`	The semantic for this feature is the default Cypher semantic.
G003	Explicit `REPEATABLE ELEMENTS` keyword	`REPEATABLE ELEMENTS`	
G004	Path variables	Path patterns	
G005	Path search prefix in a path pattern	For example, `SHORTEST`.	
G010	Explicit `WALK` keyword	Path modes	
G011	Advanced path modes: `TRAIL`	Path modes	
G013	Advanced path modes: `ACYCLIC`	Acyclic patterns	
G016	Any path search	`ANY`	
G017	All shortest path search	`ALL SHORTEST`	
G018	Any shortest path search	`ANY`	
G019	Counted shortest path search	`SHORTEST`	
G020	Counted shortest group search	`SHORTEST GROUPS`	
G035	Quantified paths	Quantified path patterns	
G036	Quantified edges	Quantified relationships	

GQL Feature ID	Description	Documentation	Comment
G043	Complete full edge patterns	Relationship patterns	Note the following: • Cypher supports GQL's directed forms (e.g. `()-[r]->()`) but not its tilde forms (e.g. `()~[r]~>()`). • Cypher does not support GQL's `()<-->()` (matches any direct edge regardless of its direction). In Cypher, this is written as `()--()`. • In GQL `()--()` additionally matches undirected edges. Neo4j does not implement undirected relationships.
G050	Parenthesized path pattern: `WHERE` clause	Path patterns	
G051	Parenthesized path pattern: non-local predicate	Graph patterns → Rules	
G060	Bounded graph pattern quantifier	Quantifiers	
G061	Unbounded graph pattern quantifier	Quantifiers	
G074	Label expressions: wildcard label	Label expressions	
G111	Labeled predicate	`IS [NOT]` LABELED	Tests a node or relationship against a label expression. Equivalent to the colon form (`n:expr`), it is only valid in the context of an expression. Note that `IS NOT LABELED` negates the entire label expression, unlike the `!` operator used within an expression.
G115	Property exists predicate	`PROPERTY_EXISTS`	GQL allows testing whether a value exists and is not null. In Cypher, the same semantics are expressed with the `IS NOT NULL` predicate: for nodes and relationships, a missing property evaluates to `null`, so `n.prop IS NOT NULL` is true when the property is present and not null. As of Neo4j 2026.03, the same semantics can also be expressed with the `PROPERTY_EXISTS(variable, propertyName)` predicate (e.g. `WHERE PROPERTY_EXISTS(n, name)`).
GA06	Value type predicates	Type predicate expressions	
GA07	Ordering by discarded binding variables	Graph patterns → Rules	
GB01	Long identifiers	Naming rules and recommendations → Identifier length limit	

GQL Feature ID	Description	Documentation	Comment
GB03	Double solidus comments	Comments	
GD01	Updatable graphs	Cypher clauses → Writing clauses	• Feature GD01 implies compliance with GQL feature GT01, "Explicit transaction commands", which states that conforming languages shall contain the following transaction control commands: `START TRANSACTION`, `ROLLBACK`, and `COMMIT`. These are not present in Cypher. However, Neo4j offers transaction management through the driver transaction API. Cypher Shell also offers specific commands to manage transactions. • Feature GD01 implies conformance to GQL's `<set-statement>` (subclause 13.3). GQL's `SET` has no order dependencies because all right-hand side operations are completed before any assignments occur. However, In Cypher's `SET`, the order of rows can affect the outcome because changes made during execution may depend on the sequence of assignments. The only way to guarantee row order in Neo4j is to use `ORDER BY`.
GD02	Graph label set changes	`SET`	
GD04	`DELETE` with simple expression	`DELETE`	
GE07	Boolean `XOR`	Boolean operators	Cypher supports `XOR`, but with different operator precedence than GQL. Equivalent behavior is achieved when expressions are explicitly parenthesized.
GF01	Enhanced numeric functions	`abs()`, `floor()`, `sqrt()`. GQL uses `MOD()` for modulus. Cypher uses the `%` operator instead.	GQL supports `CEILING()` as a synonym for the `CEIL()` function. Cypher supports `ceil()` and provides the GQL naming alias `ceiling()`.
GF02	Trigonometric functions	`acos()`, `asin()`, `atan()`, `cos()`, `cosh()`, `cot()`, `degrees()`, `radians()`, `sin()`, `sinh()`, `tan()`, `tanh()`	
GF03	Logarithmic functions	`exp()`, `log10()`, `ln()`.	• Cypher uses the `log()` function instead of GQL's `LN()` function. In GQL, the natural logarithm function must raise an exception for input zero or negative. In Cypher, `ln(0)` returns `-Infinity` and `ln(negative)` returns `NaN`. • Cypher uses the exponentiation operator (`^`) instead of GQL's `POWER()` function.

GQL Feature ID	Description	Documentation	Comment
GF05	Multi-character trim functions	`btrim()`, `ltrim()`, `rtrim()`	
GF06	Explicit `TRIM` function	`trim()`	In GQL, `TRIM()` removes only space characters. In Cypher, `trim()` removes any whitespace character.
GF12	`CARDINALITY` function	`cardinality()`	Cypher supports `CARDINALITY` for `LIST<ANY>`, `MAP`, and `PATH` values. For a list, the element count is returned (equivalent to `size()`). For a map, the key count is returned (equivalent to `size(keys(map))`). For a path, the total number of nodes and relationships is returned; for a non-empty path, `cardinality(p) = 2 * length(p) + 1`. Binding table reference values are not supported in Cypher.
GG01	Graph with open graph type		
GL01	Hexadecimal literals	Boolean, numeric, and string literals: Numeric literals	
GL02	Octal literals	Boolean, numeric, and string literals: Numeric literals	
GP01	Inline procedure	`CALL` subqueries	
GP03	Inline procedure with explicit nested variable scope	`CALL` subqueries → Variable scope clause	
GP04	Named procedure calls	`CALL` procedure	
GQ01	`USE` graph clause	`USE`	Cypher's `USE` clause supports static graph references (e.g. `USE myComposite.myGraph`)and dynamic graph references (e.g. `USE graph.byName(<expression>)`). However, Cypher does not support GQL's full graph reference syntax. For example, GQL's graph reference values `CURRENT_GRAPH` and `CURRENT_PROPERTY_GRAPH` cannot be used in Cypher.
GQ03	Composite query: `UNION`	`UNION`	
GQ08	`FILTER` statement	`FILTER`	
GQ09	`LET` statement	`LET`	
GQ10	`FOR` statement: list value support	`FOR`	As of Neo4j 2026.04, Cypher supports `FOR variable IN expression`, equivalent to `UNWIND expression AS variable`.
GQ12	`OFFSET` clause	`SKIP`: `OFFSET` as a synonym to `SKIP`	
GQ13	`ORDER BY` and page statement: `LIMIT`	`LIMIT`, `ORDER BY`	Cypher requires using the `WITH` clause, which GQL does not.

GQL Feature ID	Description	Documentation	Comment
GQ14	Complex expressions in sort keys	`ORDER BY`	
GQ15	`GROUP BY` clause	`GROUP BY`	
GQ16	Pre-projection aliases in sort keys	`ORDER BY`	
GQ20	Advanced linear composition with `NEXT`	Sequential queries (`NEXT`)	The GQL standard includes a `YIELD` clause for its `NEXT` statement which Cypher does not implement.
GQ22	`EXISTS` predicate: multiple `MATCH`	`EXISTS` subqueries	
GV39	Temporal types: date, local datetime, and local time support	Temporal types, `date()`, `local_datetime()`, `local_time()`	Note the following exceptions: • GQL defines a parameterless version of the `date()` function not in Cypher: `CURRENT_DATE`. • GQL defines the literal `DATE`. Cypher instead uses the `date()` function. • GQL supports `local_time()` as a synonym for the `localtime()` function. Cypher supports `localtime()` and provides the GQL naming alias `local_time()`. • GQL supports `local_datetime()` as a synonym for the `localdatetime()` function. Cypher supports `localdatetime()` and provides the GQL naming alias `local_datetime()`. In GQL, invalid sets of time field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument. The GQL keyword `LOCAL_TIMESTAMP` is not introduced as a reserved keyword in Cypher.

GQL Feature ID	Description	Documentation	Comment
GV40	Temporal types: zoned datetime and zoned time support	Temporal types, `zoned_time()`, `zoned_datetime()`.	Note the following exceptions: • GQL's `ZONED_TIME()` function is equivalent to Cypher's `time()` function. In GQL, invalid sets of time (or timezone) field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument. The GQL keyword `CURRENT_TIME` is not introduced as a reserved keyword in Cypher. • GQL's `ZONED_DATETIME()` function is equivalent to Cypher's `datetime()` function. In GQL, invalid sets of datetime (or timezone) field names raise a data exception (22G05); Cypher uses an error of equivalent severity (22007/22N12). Cypher also allows an optional `pattern` argument. The GQL keyword `CURRENT_TIMESTAMP` is not introduced as a reserved keyword in Cypher.
GV50	List value types	Lists	
GV55	Path value types	Structural types → `PATH`	
GV66	Open dynamic unions	Type predicate expressions → `ANY` and `NOTHING`	
GV67	Closed dynamic unions	Closed dynamic unions	
GV70	Immaterial value types: null type support (`null`)	Working with `null`	
GV71	Immaterial value types: empty type support (`NOTHING`)	Type predicate expressions → `ANY` and `NOTHING`	

Cypher and GQL sometimes name functions differently and, as a result, several Cypher functions offer the same (or very similar) functionality to their GQL counterpart. For more information, see the page Optional GQL features and analogous Cypher .

Optional GQL features and analogous Cypher

This page lists optional GQL features that have analogous but not identical Cypher features.

Optional GQL features are assigned a feature ID code. These codes order the features in the table below.

GQL Feature ID	Description	Comment and similar Cypher functionality
GA05	Cast specification	Cypher has the following alternatives to GQL's `CAST()`: • `toBoolean()` • `toFloat()` • `toInteger()`
G100	`ELEMENT_ID` function	GQL's `ELEMENT_ID()` function is equivalent to Cypher's `elementId()` function.
GF04	Enhanced path functions	Cypher provides the path_length() function, which is an alias to Cypher's length() function
GF07	Temporal duration functions	Cypher provides the GQL naming alias `duration_between()` function, which is an alias to Cypher's `duration.between()`.. In GQL, if either argument is not a temporal instant type, a data exception is raised (22G03). In Cypher, a different error (e.g. invalid call signature) may be reported, but with the same severity. Cypher returns the date difference in years, months, and days; GQL returns the date difference in days only. For the same interval the results can therefore differ (e.g. P1M1D in Cypher vs P29D in GQL for 2026-02-01 to 2026-03-02).
GF10	Advanced aggregate functions: general set functions	• GQL's `STDDEV_SAMP()` function is equivalent to Cypher's `stDev()` function. • GQL's `STDDEV_POP()` function is equivalent to Cypher's `stDevP()` function. Cypher provides the GQL naming aliases `collect_list()`, `stdev_samp()`, and `stdev_pop()`.
GF11	Advanced aggregate functions: binary set functions	• GQL's `PERCENTILE_CONT()` function is equivalent to Cypher's `percentileCont()` function. • GQL's `PERCENTILE_DISC()` function is equivalent to Cypher's `percentileDisc()` function. Cypher provides the GQL naming aliases `percentile_cont()` and `percentile_disc()`.
GQ11, GQ23, GQ24	`FOR` statement: `WITH ORDINALITY`, `WITH OFFSET`, binding table support	GQL extends the `FOR` statement with ordinality, offset, and binding table sources. Cypher does not implement these optional GQL forms. For list unnesting, as of Neo4j 2026.04, Cypher provides `FOR variable IN expression` (feature GQ10) and `UNWIND expression AS variable`, which are equivalent for plain lists but do not support `WITH ORDINALITY`, `WITH OFFSET`, or `FOR` over a binding table reference.
GV12	64-bit signed integer numbers	GQL's `SIGNED INTEGER64` (alternatively: `INTEGER64`, `INT64`) type is equivalent to Cypher's `INTEGER` type.
GV23	Floating point type name synonyms	GQL's `DOUBLE` type is equivalent to Cypher's `FLOAT` type, which is implemented as 64-bit IEEE 754.
GV24	64-bit floating number	GQL's `FLOAT64` type is equivalent to Cypher's `FLOAT` type, which is implemented as 64-bit IEEE 754.
GV41	Duration types	GQL's `DURATION()` is equivalent to Cypher's `duration()` function.

GQL Feature ID	Description	Comment and similar Cypher functionality
GV45	Record types	GQL's open `RECORD` type is equivalent to the `MAP` type in Cypher.

Additional Cypher features

While the GQL Standard incorporates a lot of capabilities in Cypher, Cypher contains additional features that are not part of GQL and no GQL alternatives currently exist for them. This page covers those Cypher features.

Clauses

Cypher feature	Description
`LOAD CSV`	Import data from CSV files.
`MERGE`	Ensures that a pattern exists in the graph. Either the pattern already exists, or it needs to be created.

Subqueries

Cypher feature	Description
`CALL { … } IN TRANSACTIONS`	`CALL` subqueries executed in separate, inner transactions, producing intermediate commits.
`COLLECT`	Used to create a list with the rows returned by a subquery.
`COUNT`	Used to count the number of rows returned by a subquery.
`EXISTS`	Used to discover if a specified pattern exists at least once in the graph

Values and types

Cypher feature	Description
`MAP` values.	Map values - the GQL equivalent is Records.
`POINT` values	Spatial values.
`VECTOR<TYPE>(DIMENSION)` values	Vector values, defined with a coordinate type and dimension.

Comprehensions and projections

Cypher feature	Description
List comprehension	Syntactic construct for creating a `LIST` based on existing lists.
Map projection	Constructs `MAP` projections from nodes, relationships, and other `MAP` values.
Pattern comprehension	Syntactic construct for creating a `LIST` based on matchings of a pattern.

Dynamic queries

Node labels, relationship types, properties, and CSV columns can be referenced dynamically using Cypher. This allows for more flexible queries and mitigates the risk of Cypher injection. (For more information about Cypher injection, see Neo4j Knowledge Base → Protecting against Cypher injection).

Cypher feature	Description
`MATCH (n:$($label)), ()-[r:$($type)]->()`	`MATCH` nodes and relationships using dynamic node labels and relationship types
`CREATE (n:$($label)), ()-[r:$($type)]->()`	`CREATE` nodes and relationships using dynamic node labels and relationship types
`MERGE (n:$($label)), ()-[r:$($type)]->()`	`MERGE` nodes and relationships using dynamic node labels and relationship types
`LOAD CSV WITH HEADERS FROM 'file:///artists-with-headers.csv' AS line CREATE (n:$(line.label) {name: line.Name})`	Import CSV files using dynamic columns
`MATCH (n) SET n[$key] = value`	Dynamically `SET` or update a property
`MATCH (n:Label) SET n:$(n.property)`	Dynamically `SET` a node label
`MATCH (n {name: 'Peter'}) REMOVE n:$($label)`	Dynamically `REMOVE` a property
`MATCH (n {name: 'Peter'}) REMOVE n:$($label)`	Dynamically `REMOVE` a node label

Functions

Database functions

Cypher feature	Description
`db.nameFromElementId()`	Resolves the database name for the given element id.

Graph functions

Cypher feature	Description
`graph.byElementId()`	Returns the graph reference with the given element id. It is only supported in the `USE` clause.
`graph.byName()`	Returns the graph reference of the given name. It is only supported in the `USE` clause, on composite databases.

Cypher feature	Description
`graph.names()`	Lists the names of graphs in the current database.
`graph.propertiesByName()`	Returns the `MAP` of properties associated with a graph.

List functions

Cypher feature	Description
`keys()`	Returns a `LIST<STRING>` containing the `STRING` representations for all the property names of a `NODE`, `RELATIONSHIP`, or `MAP`.
`labels()`	Returns a `LIST<STRING>` containing the `STRING` representations for all the labels of a `NODE`.
`nodes()`	Returns a `LIST<NODE>` containing all the `NODE` values in a `PATH`.
`range()`	Returns a `LIST<INTEGER>` comprising all `INTEGER` values within a specified range.
`reduce()`	Runs an expression against individual elements of a `LIST<ANY>`, storing the result of the expression in an accumulator.
`relationships()`	Returns a `LIST<RELATIONSHIP>` containing all the `RELATIONSHIP` values in a `PATH`.
`reverse()`	Returns a `STRING` or `LIST<ANY>` in which the order of all characters or elements in the given `STRING` or `LIST<ANY>` have been reversed.
`tail()`	Returns all but the first element in a `LIST<ANY>`.
`toBooleanList()`	Converts a `LIST<ANY>` of values to a `LIST<BOOLEAN>` values. If any values are not convertible to `BOOLEAN` they will be null in the `LIST<BOOLEAN>` returned.
`toFloatList()`	Converts a `LIST<ANY>` to a `LIST<FLOAT>` values. If any values are not convertible to `FLOAT` they will be null in the `LIST<FLOAT>` returned.
`toIntegerList()`	Converts a `LIST<ANY>` to a `LIST<INTEGER>` values. If any values are not convertible to `INTEGER` they will be null in the `LIST<INTEGER>` returned.
`toStringList()`	Converts an `INTEGER`, `FLOAT`, `BOOLEAN`, `POINT` or temporal type (i.e. `DATE`, `ZONED TIME`, `LOCAL TIME`, `ZONED DATETIME`, `LOCAL DATETIME` or `DURATION`) value to a `STRING`, or null if the value cannot be converted.

LOAD CSV functions

Cypher feature	Description
`file()`	Returns the absolute path of the file that `LOAD CSV` is using.
`lineNumber`	Returns the line number that `LOAD CSV` is currently using.

Logarithmic functions

Cypher feature	Description
`e()`	Returns the base of the natural logarithm, e.

Numeric functions

Cypher feature	Description
`isNaN()`	Returns whether the given `INTEGER` or `FLOAT` is NaN.
`rand()`	Returns a random `FLOAT` in the range from 0 (inclusive) to 1 (exclusive).
`round()`	Returns the value of a number rounded to the nearest `INTEGER`.
`sign()`	Returns the signum of an `INTEGER` or `FLOAT`: 0 if the number is 0, -1 for any negative number, and 1 for any positive number.

Predicate functions

Cypher feature	Description
`all()`	Returns true if the predicate holds for all elements in the given `LIST<ANY>`.
`allReduce()`	Returns true if, during the stepwise evaluation of a value across the elements in a given `LIST<ANY>`, the accumulated result satisfies a specified predicate at every step. Where that list is a group variable defined in a quantified path pattern, it allows for the early pruning of paths that do not satisfy the predicate.
`any()`	Returns true if the predicate holds for at least one element in the given `LIST<ANY>`.
`isEmpty()`	Checks whether a `STRING`, `MAP`, or `LIST<ANY>` is empty.
`none()`	Returns true if the predicate holds for no element in the given `LIST<ANY>`.
`single()`	Returns true if the predicate holds for exactly one of the elements in the given `LIST<ANY>`.

Scalar functions

Cypher feature	Description
`endNode()`	Returns the end `NODE` of a `RELATIONSHIP`.
`head()`	Returns the first element in a `LIST<ANY>`.
`last()`	Returns the last element in a `LIST<ANY>`.
`properties()`	Returns a `MAP` containing all the properties of a `NODE`, `RELATIONSHIP`, or `MAP`.
`randomUUID()`	Generates a random UUID.
`startNode()`	Returns the start `NODE` of a `RELATIONSHIP`.
`type()`	Returns a `STRING` representation of the `RELATIONSHIP` type.
`valueType()`	Returns a `STRING` representation of the most precise value type that the given expression evaluates to.

Spatial functions

Cypher feature	Description
`point()`	Returns a 2D or 3D point object, given two or respectively three coordinate values in the Cartesian coordinate system or WGS 84 geographic coordinate system.
`point.distance()`	Returns a `FLOAT` representing the distance between any two points in the same CRS. If the points are in the WGS 84 CRS, the function returns the geodesic distance (i.e., the shortest path along the curved surface of the Earth). If the points are in a Cartesian CRS, the function returns the Euclidean distance (i.e., the shortest straight-line distance in a flat, planar space).
`point.withinBBox()`	Returns true if the provided point is within the bounding box defined by the two provided points.

String functions

Cypher feature	Description
`replace()`	Returns a `STRING` in which all occurrences of a specified search `STRING` in the given `STRING` have been replaced by another (specified) replacement `STRING`.
`reverse()`	Returns a `STRING` or `LIST<ANY>` in which the order of all characters or elements in the given `STRING` or `LIST<ANY>` have been reversed.
`split()`	Returns a `LIST<STRING>` resulting from the splitting of the given `STRING` around matches of the given delimiter(s).
`substring()`	Returns a substring of the given `STRING`, beginning with a 0-based index start.

Trigonometric functions

Cypher feature	Description
`atan2()`	Returns the arctangent2 of a set of coordinates in radians.
`haversin()`	Returns half the versine of a number.
`pi()`	Returns the mathematical constant pi.

Temporal duration functions

Cypher feature	Description
`duration.inDays()`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in days.
`duration.inMonths()`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in months.
`duration.inSeconds()`	Computes the `DURATION` between the `from` instant (inclusive) and the `to` instant (exclusive) in seconds.

Temporal instant functions

Cypher feature	Description
`date.realtime()`	Returns the current `DATE` instant using the realtime clock.
`date.statement()`	Returns the current `DATE` instant using the statement clock.
`date.transaction()`	Returns the current `DATE` instant using the transaction clock.
`date.truncate()`	Returns the current `DATE` instant using the transaction clock.
`datetime.fromEpoch()`	Creates a `ZONED DATETIME` given the seconds and nanoseconds since the start of the epoch.
`datetime.fromEpochMillis()`	Creates a `ZONED DATETIME` given the milliseconds since the start of the epoch.
`datetime.realtime()`	Returns the current `ZONED DATETIME` instant using the realtime clock.
`datetime.statement()`	Returns the current `ZONED DATETIME` instant using the statement clock.
`datetime.transaction()`	Returns the current `ZONED DATETIME` instant using the transaction clock.
`datetime.truncate()`	Truncates the given temporal value to a `ZONED DATETIME` instant using the specified unit.
`localdatetime.realtime()`	Returns the current `LOCAL DATETIME` instant using the realtime clock.
`localdatetime.statement()`	Returns the current `LOCAL DATETIME` instant using the statement clock.
`localdatetime.transaction()`	Returns the current `LOCAL DATETIME` instant using the transaction clock.
`localdatetime.truncate()`	Truncates the given temporal value to a `LOCAL DATETIME` instant using the specified unit.
`localtime.realtime()`	Returns the current `LOCAL TIME` instant using the realtime clock.
`localtime.statement()`	Returns the current `LOCAL TIME` instant using the statement clock.
`localtime.transaction()`	Returns the current `LOCAL TIME` instant using the transaction clock.
`localtime.truncate()`	Truncates the given temporal value to a `LOCAL TIME` instant using the specified unit.
`time.realtime()`	Returns the current `ZONED TIME` instant using the realtime clock.
`time.statement()`	Returns the current `ZONED TIME` instant using the statement clock.
`time.transaction()`	Returns the current `ZONED TIME` instant using the transaction clock.
`time.truncate()`	Truncates the given temporal value to a `ZONED TIME` instant using the specified unit.

Temporal format functions

Cypher feature	Description
`format()`	Returns the temporal value as an ISO-formatted `STRING` or as a `STRING` formatted by the provided pattern.

Vector functions

Cypher feature	Description
`vector.similarity.cosine()`	Returns a `FLOAT` representing the similarity between the argument vectors based on their cosine.

Cypher feature	Description
`vector.similarity.euclidean()`	Returns a `FLOAT` representing the similarity between the argument vectors based on their Euclidean distance.

Indexes

Cypher feature	Description
Range indexes	Neo4j's default index. Supports most types of predicates.
Text indexes	Solves predicates operating on `STRING` values. Optimized for queries filtering with the `STRING` operators `CONTAINS` and `ENDS WITH`.
Point indexes	Solves predicates on spatial `POINT` values. Optimized for queries filtering on distance or within bounding boxes.
Token lookup indexes	Only solves node label and relationship type predicates (i.e. they cannot solve any predicates filtering on properties).
Full text indexes	Enables searching within the content of `STRING` properties and for similarity comparisons between query strings and `STRING` values stored in the database.
Vector indexes	Enables similarity searches and complex analytical queries by representing nodes or properties as vectors in a multidimensional space.
Index hints	Cypher allows for index hints to influence the planner when creating execution plans. Index hints are specified with the `USING` keyword.

Constraints

GQL supports `GRAPH TYPES` as a way of constraining a graph schema, but does not support individual constraints.

Cypher feature	Description
Property uniqueness constraints	Ensures that the combined property values are unique for all nodes with a specific label or all relationships with a specific type.
Property existence constraints	Ensures that a property exists either for all nodes with a specific label or for all relationships with a specific type.
Property type constraints	Ensures that a property has the required property type for all nodes with a specific label or for all relationships with a specific type.
Key constraints	Ensures that all properties exist and that the combined property values are unique for all nodes with a specific label or all relationships with a specific type.

Operators

Cypher feature	Description
`STARTS WITH`, `CONTAINS`, `ENDS WITH` and regular expressions.	`STRING` comparison operators.

Cypher feature	Description
`IN`	`IN` predicate for `LIST` values.

Query optimization

Cypher feature	Description
`EXPLAIN` / `PROFILE`	Optionally prepended to queries to produce execution plans. `EXPLAIN` will only generate an execution plan but not run the query; `PROFILE` will do both.
`CYPHER runtime=parallel`	Cypher allows for setting the runtime of queries, determining how the query will be executed. The available Cypher runtimes are: slotted, pipelined, parallel.
`CYPHER inferSchemaParts=off`	Cypher allows for setting numerous query options. For more information, see Query options.

Administration

> The documentation for Cypher's administration commands is located in Neo4j's Operation Manual.

Cypher feature	Description
Database management	Commands to `CREATE`, `SHOW`, `ALTER`, and `DROP` standard and composite databases.
Alias management	Commands to `CREATE`, `SHOW`, `ALTER`, and `DROP` database aliases.
Server management	Commands to administer servers in a cluster and the databases allocated to them.
Authentication and authorization	Commands to manage users, roles, and privileges.

Tutorials and extended examples

- Basic query tuning example
- Advanced query tuning example

Basic query tuning example

This page describes how to profile a query by using optimizations based on native index capabilities.

The data set

In this section, examples demonstrates the impact native indexes can have on query performance under certain conditions. You will use a movies dataset to illustrate this more advanced query tuning.

In this tutorial, you import data from the following CSV files:

- *movies.csv*

- actors.csv
- directors.csv

Movies

The *movies.csv* file contains two columns `title`, `released`, and `tagline`.

The content of the *movies.csv* file:

movies.csv

```
title,released,tagline
Something's Gotta Give,1975,null
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Devil's Advocate,1997,Evil has its winning ways
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
V for Vendetta,2006,Freedom! Forever!
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Speed Racer,2008,Speed has no limits
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix,1999,Welcome to the Real World
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
The Devil's Advocate,1997,Evil has its winning ways
The Devil's Advocate,1997,Evil has its winning ways
The Devil's Advocate,1997,Evil has its winning ways
Jerry Maguire,2000,The rest of his life begins now.
Top Gun,1986,"I feel the need, the need for speed."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Something's Gotta Give,1975,null
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Hoffa,1992,He didn't want law. He wanted justice.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
```

```
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Apollo 13,1995,"Houston, we have a problem."
Frost/Nixon,2008,400 million people were waiting for the truth.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
What Dreams May Come,1998,After life there is more. The end is just the beginning.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
Jerry Maguire,2000,The rest of his life begins now.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Hoffa,1992,He didn't want law. He wanted justice.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
When Harry Met Sally,1998,At odds in life... in love on-line.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
When Harry Met Sally,1998,At odds in life... in love on-line.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
When Harry Met Sally,1998,At odds in life... in love on-line.
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
The Green Mile,1999,Walk a mile you'll never forget.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Cast Away,2000,"At the edge of the world, his journey begins."
Twister,1996,Don't Breathe. Don't Look Back.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
You've Got Mail,1998,At odds in life... in love on-line.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
```

```
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
Snow Falling on Cedars,1999,First loves last. Forever.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
The Birdcage,1996,Come as you are
What Dreams May Come,1998,After life there is more. The end is just the beginning.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
Snow Falling on Cedars,1999,First loves last. Forever.
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Snow Falling on Cedars,1999,First loves last. Forever.
The Green Mile,1999,Walk a mile you'll never forget.
Snow Falling on Cedars,1999,First loves last. Forever.
Snow Falling on Cedars,1999,First loves last. Forever.
You've Got Mail,1998,At odds in life... in love on-line.
You've Got Mail,1998,At odds in life... in love on-line.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
You've Got Mail,1998,At odds in life... in love on-line.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
The Polar Express,2004,This Holiday Season… Believe
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
Cast Away,2000,"At the edge of the world, his journey begins."
Apollo 13,1995,"Houston, we have a problem."
The Green Mile,1999,Walk a mile you'll never forget.
The Da Vinci Code,2006,Break The Codes
Cloud Atlas,2012,Everything is connected
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
The Birdcage,1996,Come as you are
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
Unforgiven,1992,"It's a hell of a thing, killing a man"
The Birdcage,1996,Come as you are
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Twister,1996,Don't Breathe. Don't Look Back.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
The Birdcage,1996,Come as you are
Unforgiven,1992,"It's a hell of a thing, killing a man"
Unforgiven,1992,"It's a hell of a thing, killing a man"
Unforgiven,1992,"It's a hell of a thing, killing a man"
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
```

```
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Cloud Atlas,2012,Everything is connected
Cloud Atlas,2012,Everything is connected
Cloud Atlas,2012,Everything is connected
The Da Vinci Code,2006,Break The Codes
The Da Vinci Code,2006,Break The Codes
The Da Vinci Code,2006,Break The Codes
Apollo 13,1995,"Houston, we have a problem."
Frost/Nixon,2008,400 million people were waiting for the truth.
The Da Vinci Code,2006,Break The Codes
V for Vendetta,2006,Freedom! Forever!
V for Vendetta,2006,Freedom! Forever!
V for Vendetta,2006,Freedom! Forever!
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
Ninja Assassin,2009,Prepare to enter a secret world of assassins
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
Frost/Nixon,2008,400 million people were waiting for the truth.
The Green Mile,1999,Walk a mile you'll never forget.
Apollo 13,1995,"Houston, we have a problem."
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
Frost/Nixon,2008,400 million people were waiting for the truth.
Frost/Nixon,2008,400 million people were waiting for the truth.
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
Frost/Nixon,2008,400 million people were waiting for the truth.
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Hoffa,1992,He didn't want law. He wanted justice.
Hoffa,1992,He didn't want law. He wanted justice.
Hoffa,1992,He didn't want law. He wanted justice.
Apollo 13,1995,"Houston, we have a problem."
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
Twister,1996,Don't Breathe. Don't Look Back.
Apollo 13,1995,"Houston, we have a problem."
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
Twister,1996,Don't Breathe. Don't Look Back.
Twister,1996,Don't Breathe. Don't Look Back.
The Polar Express,2004,This Holiday Season… Believe
Cast Away,2000,"At the edge of the world, his journey begins."
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Da Vinci Code,2006,Break The Codes
The Birdcage,1996,Come as you are
Unforgiven,1992,"It's a hell of a thing, killing a man"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
Cloud Atlas,2012,Everything is connected
The Da Vinci Code,2006,Break The Codes
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
```

Actors

The actors.csv file contains two columns `title`, `roles`, `name`, and `born`.

The content of the actors.csv file:

actors.csv

```
title,roles,name,born
Something's Gotta Give,Julian Mercer,Keanu Reeves,1964
Johnny Mnemonic,Johnny Mnemonic,Keanu Reeves,1964
The Replacements,Shane Falco,Keanu Reeves,1964
The Devil's Advocate,Kevin Lomax,Keanu Reeves,1964
The Matrix Revolutions,Neo,Keanu Reeves,1964
The Matrix Reloaded,Neo,Keanu Reeves,1964
The Matrix,Neo,Keanu Reeves,1964
The Matrix Revolutions,Trinity,Carrie-Anne Moss,1967
The Matrix Reloaded,Trinity,Carrie-Anne Moss,1967
The Matrix,Trinity,Carrie-Anne Moss,1967
The Matrix Revolutions,Morpheus,Laurence Fishburne,1961
The Matrix Reloaded,Morpheus,Laurence Fishburne,1961
The Matrix,Morpheus,Laurence Fishburne,1961
V for Vendetta,V,Hugo Weaving,1960
Cloud Atlas,Bill Smoke;Haskell Moore;Tadeusz Kesselring;Nurse Noakes;Boardman Mephi;Old Georgie,Hugo
Weaving,1960
The Matrix Revolutions,Agent Smith,Hugo Weaving,1960
The Matrix Reloaded,Agent Smith,Hugo Weaving,1960
The Matrix,Agent Smith,Hugo Weaving,1960
The Matrix,Emil,Emil Eifrem,1978
That Thing You Do,Tina,Charlize Theron,1975
The Devil's Advocate,Mary Ann Lomax,Charlize Theron,1975
The Devil's Advocate,John Milton,Al Pacino,1940
Jerry Maguire,Jerry Maguire,Tom Cruise,1962
Top Gun,Maverick,Tom Cruise,1962
A Few Good Men,Lt. Daniel Kaffee,Tom Cruise,1962
Something's Gotta Give,Harry Sanborn,Jack Nicholson,1937
One Flew Over the Cuckoo's Nest,Randle McMurphy,Jack Nicholson,1937
Hoffa,Hoffa,Jack Nicholson,1937
As Good as It Gets,Melvin Udall,Jack Nicholson,1937
A Few Good Men,Col. Nathan R. Jessup,Jack Nicholson,1937
A Few Good Men,Lt. Cdr. JoAnne Galloway,Demi Moore,1962
Apollo 13,Jack Swigert,Kevin Bacon,1958
Frost/Nixon,Jack Brennan,Kevin Bacon,1958
A Few Good Men,Capt. Jack Ross,Kevin Bacon,1958
Stand By Me,Ace Merrill,Kiefer Sutherland,1966
A Few Good Men,Lt. Jonathan Kendrick,Kiefer Sutherland,1966
A Few Good Men,Cpl. Jeffrey Barnes,Noah Wyle,1971
What Dreams May Come,Albert Lewis,Cuba Gooding Jr.,1968
As Good as It Gets,Frank Sachs,Cuba Gooding Jr.,1968
Jerry Maguire,Rod Tidwell,Cuba Gooding Jr.,1968
A Few Good Men,Cpl. Carl Hammaker,Cuba Gooding Jr.,1968
A Few Good Men,Lt. Sam Weinberg,Kevin Pollak,1957
Hoffa,Frank Fitzsimmons,J.T. Walsh,1943
A Few Good Men,Lt. Col. Matthew Andrew Markinson,J.T. Walsh,1943
A Few Good Men,Pfc. Louden Downey,James Marshall,1967
A Few Good Men,Dr. Stone,Christopher Guest,1948
A Few Good Men,Man in Bar,Aaron Sorkin,1961
Top Gun,Charlie,Kelly McGillis,1957
Top Gun,Iceman,Val Kilmer,1959
Top Gun,Goose,Anthony Edwards,1962
Top Gun,Viper,Tom Skerritt,1933
When Harry Met Sally,Sally Albright,Meg Ryan,1961
Joe Versus the Volcano,DeDe;Angelica Graynamore;Patricia Graynamore,Meg Ryan,1961
Sleepless in Seattle,Annie Reed,Meg Ryan,1961
You've Got Mail,Kathleen Kelly,Meg Ryan,1961
Top Gun,Carole,Meg Ryan,1961
Jerry Maguire,Dorothy Boyd,Renee Zellweger,1969
Jerry Maguire,Avery Bishop,Kelly Preston,1962
Stand By Me,Vern Tessio,Jerry O'Connell,1974
Jerry Maguire,Frank Cushman,Jerry O'Connell,1974
Jerry Maguire,Bob Sugar,Jay Mohr,1970
The Green Mile,Jan Edgecomb,Bonnie Hunt,1961
Jerry Maguire,Laurel Boyd,Bonnie Hunt,1961
Jerry Maguire,Marcee Tidwell,Regina King,1971
Jerry Maguire,Ray Boyd,Jonathan Lipnicki,1990
Stand By Me,Chris Chambers,River Phoenix,1970
Stand By Me,Teddy Duchamp,Corey Feldman,1971
Stand By Me,Gordie Lachance,Wil Wheaton,1972
Stand By Me,Denny Lachance,John Cusack,1966
```

```
RescueDawn,Admiral,Marshall Bell,1942
Stand By Me,Mr. Lachance,Marshall Bell,1942
Cast Away,Kelly Frears,Helen Hunt,1963
Twister,Dr. Jo Harding,Helen Hunt,1963
As Good as It Gets,Carol Connelly,Helen Hunt,1963
You've Got Mail,Frank Navasky,Greg Kinnear,1963
As Good as It Gets,Simon Bishop,Greg Kinnear,1963
What Dreams May Come,Simon Bishop,Annabella Sciorra,1960
Snow Falling on Cedars,Nels Gudmundsson,Max von Sydow,1929
What Dreams May Come,The Tracker,Max von Sydow,1929
What Dreams May Come,The Face,Werner Herzog,1942
Bicentennial Man,Andrew Marin,Robin Williams,1951
The Birdcage,Armand Goldman,Robin Williams,1951
What Dreams May Come,Chris Nielsen,Robin Williams,1951
Snow Falling on Cedars,Ishmael Chambers,Ethan Hawke,1970
Ninja Assassin,Takeshi,Rick Yune,1971
Snow Falling on Cedars,Kazuo Miyamoto,Rick Yune,1971
The Green Mile,Warden Hal Moores,James Cromwell,1940
Snow Falling on Cedars,Judge Fielding,James Cromwell,1940
You've Got Mail,Patricia Eden,Parker Posey,1968
You've Got Mail,Kevin Jackson,Dave Chappelle,1973
RescueDawn,Duane,Steve Zahn,1967
You've Got Mail,George Pappas,Steve Zahn,1967
A League of Their Own,Jimmy Dugan,Tom Hanks,1956
The Polar Express,Hero Boy;Father;Conductor;Hobo;Scrooge;Santa Claus,Tom Hanks,1956
Charlie Wilson's War,Rep. Charlie Wilson,Tom Hanks,1956
Cast Away,Chuck Noland,Tom Hanks,1956
Apollo 13,Jim Lovell,Tom Hanks,1956
The Green Mile,Paul Edgecomb,Tom Hanks,1956
The Da Vinci Code,Dr. Robert Langdon,Tom Hanks,1956
Cloud Atlas,Zachry;Dr. Henry Goose;Isaac Sachs;Dermot Hoggins,Tom Hanks,1956
That Thing You Do,Mr. White,Tom Hanks,1956
Joe Versus the Volcano,Joe Banks,Tom Hanks,1956
Sleepless in Seattle,Sam Baldwin,Tom Hanks,1956
You've Got Mail,Joe Fox,Tom Hanks,1956
Sleepless in Seattle,Suzy,Rita Wilson,1956
Sleepless in Seattle,Walter,Bill Pullman,1953
Sleepless in Seattle,Greg,Victor Garber,1949
A League of Their Own,Doris Murphy,Rosie O'Donnell,1962
Sleepless in Seattle,Becky,Rosie O'Donnell,1962
The Birdcage,Albert Goldman,Nathan Lane,1956
Joe Versus the Volcano,Baw,Nathan Lane,1956
When Harry Met Sally,Harry Burns,Billy Crystal,1948
When Harry Met Sally,Marie,Carrie Fisher,1956
When Harry Met Sally,Jess,Bruno Kirby,1949
That Thing You Do,Faye Dolan,Liv Tyler,1977
The Replacements,Annabelle Farrell,Brooke Langton,1970
Unforgiven,Little Bill Daggett,Gene Hackman,1930
The Birdcage,Sen. Kevin Keeley,Gene Hackman,1930
The Replacements,Jimmy McGinty,Gene Hackman,1930
The Replacements,Clifford Franklin,Orlando Jones,1968
RescueDawn,Dieter Dengler,Christian Bale,1974
Twister,Eddie,Zach Grenier,1954
RescueDawn,Squad Leader,Zach Grenier,1954
Unforgiven,English Bob,Richard Harris,1930
Unforgiven,Bill Munny,Clint Eastwood,1930
Johnny Mnemonic,Takahashi,Takeshi Kitano,1947
Johnny Mnemonic,Jane,Dina Meyer,1968
Johnny Mnemonic,J-Bone,Ice-T,1958
Cloud Atlas,Luisa Rey;Jocasta Ayrs;Ovid;Meronym,Halle Berry,1966
Cloud Atlas,Vyvyan Ayrs;Captain Molyneux;Timothy Cavendish,Jim Broadbent,1949
The Da Vinci Code,Sir Leight Teabing,Ian McKellen,1939
The Da Vinci Code,Sophie Neveu,Audrey Tautou,1976
The Da Vinci Code,Silas,Paul Bettany,1971
V for Vendetta,Evey Hammond,Natalie Portman,1981
V for Vendetta,Eric Finch,Stephen Rea,1946
V for Vendetta,High Chancellor Adam Sutler,John Hurt,1940
Ninja Assassin,Ryan Maslow,Ben Miles,1967
Speed Racer,Cass Jones,Ben Miles,1967
V for Vendetta,Dascomb,Ben Miles,1967
Speed Racer,Speed Racer,Emile Hirsch,1985
Speed Racer,Pops,John Goodman,1960
Speed Racer,Mom,Susan Sarandon,1946
Speed Racer,Racer X,Matthew Fox,1966
Speed Racer,Trixie,Christina Ricci,1980
Ninja Assassin,Raizo,Rain,1982
Speed Racer,Taejo Togokahn,Rain,1982
```

```
Ninja Assassin,Mika Coretti,Naomie Harris,null
The Green Mile,John Coffey,Michael Clarke Duncan,1957
The Green Mile,Brutus 'Brutal' Howell,David Morse,1953
Frost/Nixon,"James Reston, Jr.",Sam Rockwell,1968
The Green Mile,'Wild Bill' Wharton,Sam Rockwell,1968
Apollo 13,Ken Mattingly,Gary Sinise,1955
The Green Mile,Burt Hammersmith,Gary Sinise,1955
The Green Mile,Melinda Moores,Patricia Clarkson,1959
Frost/Nixon,Richard Nixon,Frank Langella,1938
Frost/Nixon,David Frost,Michael Sheen,1969
Bicentennial Man,Rupert Burns,Oliver Platt,1960
Frost/Nixon,Bob Zelnick,Oliver Platt,1960
One Flew Over the Cuckoo's Nest,Martini,Danny DeVito,1944
Hoffa,Robert 'Bobby' Ciaro,Danny DeVito,1944
Hoffa,Peter 'Pete' Connelly,John C. Reilly,1965
Apollo 13,Gene Kranz,Ed Harris,1950
A League of Their Own,Bob Hinson,Bill Paxton,1955
Twister,Bill Harding,Bill Paxton,1955
Apollo 13,Fred Haise,Bill Paxton,1955
Charlie Wilson's War,Gust Avrakotos,Philip Seymour Hoffman,1967
Twister,Dustin 'Dusty' Davis,Philip Seymour Hoffman,1967
Something's Gotta Give,Erica Barry,Diane Keaton,1946
Charlie Wilson's War,Joanne Herring,Julia Roberts,1967
A League of Their Own,'All the Way' Mae Mordabito,Madonna,1954
A League of Their Own,Dottie Hinson,Geena Davis,1956
A League of Their Own,Kit Keller,Lori Petty,1963
```

Directors

The *directors.csv* file contains two columns `title`, `name`, and `born`.

The content of the *directors.csv* file:

directors.csv

```
title,name,born
Speed Racer,Andy Wachowski,1967
Cloud Atlas,Andy Wachowski,1967
The Matrix Revolutions,Andy Wachowski,1967
The Matrix Reloaded,Andy Wachowski,1967
The Matrix,Andy Wachowski,1967
Speed Racer,Lana Wachowski,1965
Cloud Atlas,Lana Wachowski,1965
The Matrix Revolutions,Lana Wachowski,1965
The Matrix Reloaded,Lana Wachowski,1965
The Matrix,Lana Wachowski,1965
The Devil's Advocate,Taylor Hackford,1944
Ninja Assassin,James Marshall,1967
V for Vendetta,James Marshall,1967
When Harry Met Sally,Rob Reiner,1947
Stand By Me,Rob Reiner,1947
A Few Good Men,Rob Reiner,1947
Top Gun,Tony Scott,1944
Jerry Maguire,Cameron Crowe,1957
As Good as It Gets,James L. Brooks,1940
RescueDawn,Werner Herzog,1942
What Dreams May Come,Vincent Ward,1956
Snow Falling on Cedars,Scott Hicks,1953
That Thing You Do,Tom Hanks,1956
Sleepless in Seattle,Nora Ephron,1941
You've Got Mail,Nora Ephron,1941
Joe Versus the Volcano,John Patrick Stanley,1950
The Replacements,Howard Deutch,1950
Charlie Wilson's War,Mike Nichols,1931
The Birdcage,Mike Nichols,1931
Unforgiven,Clint Eastwood,1930
Johnny Mnemonic,Robert Longo,1953
Cloud Atlas,Tom Tykwer,1965
Apollo 13,Ron Howard,1954
Frost/Nixon,Ron Howard,1954
The Da Vinci Code,Ron Howard,1954
The Green Mile,Frank Darabont,1959
```

```
Hoffa,Danny DeVito,1944
Twister,Jan de Bont,1943
The Polar Express,Robert Zemeckis,1951
Cast Away,Robert Zemeckis,1951
One Flew Over the Cuckoo's Nest,Milos Forman,1932
Something's Gotta Give,Nancy Meyers,1949
Bicentennial Man,Chris Columbus,1958
A League of Their Own,Penny Marshall,1943
```

Prerequisites

The example uses the Linux or macOS tarball installation. It assumes that your current work directory is the *<neo4j-home>* directory of the tarball installation, and the CSV files are placed in the default *import* directory.

Note:

- For the default directory of other installations see, Operations Manual → File locations.
- The import location can be configured with Operations Manual → `server.directories.import`.

Importing the data

Import the movies.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///movies.csv' AS line
MERGE (m:Movie {title: line.title})
ON CREATE SET
  m.released = toInteger(line.released),
  m.tagline = line.tagline
```

Result

```
Added 38 nodes, Set 114 properties, Added 38 labels
```

Import the actors.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///actors.csv' AS line
MATCH (m:Movie {title: line.title})
MERGE (p:Person {name: line.name})
ON CREATE SET p.born = toInteger(line.born)
MERGE (p)-[:ACTED_IN {roles:split(line.roles, ';')}]->(m)
```

Result

```
Added 102 nodes, Created 172 relationships, Set 375 properties, Added 102 labels
```

Import the directors.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///directors.csv' AS line
MATCH (m:Movie {title: line.title})
MERGE (p:Person {name: line.name})
ON CREATE SET p.born = toInteger(line.born)
```

```
MERGE (p)-[:DIRECTED]->(m)
```

Result

```
Added 23 nodes, Created 44 relationships, Set 46 properties, Added 23 labels
```

Profile query

Let's say you want to write a query to find **'Tom Hanks'**.

The naive way of doing this would be to write the following:

```
MATCH (p {name: 'Tom Hanks'})
RETURN p
```

Result

p
(:Person {name: "Tom Hanks", born: 1956})
Rows: 1

This query will find the **'Tom Hanks'** node but as the number of nodes in the database increase it will become slower and slower. We can profile the query to find out why that is.

You can learn more about the options for profiling queries in Query tuning but in this case you are going to prefix our query with `PROFILE`:

```
PROFILE
MATCH (p {name: 'Tom Hanks'})
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-----------------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| Operator        | Id | Details               | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Pipeline            |
+-----------------+----+-----------------------+----------------+------+---------+----------------
+------------------------+-----------+---------------------+
| +ProduceResults |  0 | p                     |              8 |    1 |       3 |                |
|           |                     |
| |               +----+-----------------------+----------------+------+---------+----------------+
|           |                     |
| +Filter         |  1 | p.name = $autostring_0 |              8 |    1 |     125 |                |
|           |                     |
| |               +----+-----------------------+----------------+------+---------+----------------+
|           |                     |
| +AllNodesScan   |  2 | p                     |            163 |  163 |     164 |            120 |
4/0 |     0.860 | Fused in Pipeline 0 |
+-----------------+----+-----------------------+----------------+------+---------+----------------
```

```
+----------------------+----------+--------------------+

Total database accesses: 292, total allocated memory: 184

1 row
ready to start consuming query after 17 ms, results consumed after another 10 ms
```

The first thing to keep in mind when reading execution plans is that you need to read from the bottom up.

In that vein, starting from the last row, the first thing you notice is that the value in the `Rows` column seems high given there is only one node with the name property **'Tom Hanks'** in the database. If you look across to the `Operator` column, you will see that AllNodesScan has been used which means that the query planner scanned through all the nodes in the database.

The Filter operator which will check the `name` property on each of the nodes passed through by `AllNodesScan`.

This seems like an inefficient way of finding **'Tom Hanks'** given that you are looking at many nodes that are not even people and therefore are not what you are looking for.

The solution to this problem is that whenever you are looking for a node you should specify a label to help the query planner narrow down the search space.

For this query you need to add a `Person` label.

```
MATCH (p:Person {name: 'Tom Hanks'})
RETURN p
```

This query will be faster than the first one, but as the number of people in your database increase you may notice that the query slows down.

Again you can profile the query to work out why:

```
PROFILE
MATCH (p:Person {name: 'Tom Hanks'})
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-----------------------+----------------+------+---------+----------------
+----------------------+----------+--------------------+
| Operator        | Id | Details               | Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page
Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+----+-----------------------+----------------+------+---------+----------------
+----------------------+----------+--------------------+
| +ProduceResults |  0 | p                     |              6 |    1 |       3 |                |
|           |                    |
| |               +----+-----------------------+----------------+------+---------+----------------+
|           |                    |
| +Filter         |  1 | p.name = $autostring_0 |              6 |    1 |     250 |                |
|           |                    |
| |               +----+-----------------------+----------------+------+---------+----------------+
```

```
|          |                    |
| +NodeByLabelScan |  2 | p:Person              |           125 |  125 |     126 |           120 |
3/0 |     0.772 | Fused in Pipeline 0 |
+------------------+----+-----------------------+---------------+------+---------+---------------
+----------------------+-----------+--------------------+

Total database accesses: 379, total allocated memory: 184

1 row
```

This time the `Rows` value on the last row has reduced so you are not scanning some nodes that you were before which is a good start. The NodeByLabelScan operator indicates that you achieved this by first doing a linear scan of all the `Person` nodes in the database.

Once you have done that, you can again scan through all those nodes using the `Filter` operator, comparing the name property of each one.

This might be acceptable in some cases but if you are going to be looking up people by name frequently then you will see better performance if you create an index on the `name` property for the `Person` label:

```
CREATE INDEX FOR (p:Person)
ON (p.name)
```

Result

```
Added 1 indexes
```

```
CALL db.awaitIndexes
```

Now if you run the query again it will run more quickly:

```
MATCH (p:Person {name: 'Tom Hanks'})
RETURN p
```

A profile for the query to see why that is:

```
PROFILE
MATCH (p:Person {name: 'Tom Hanks'})
RETURN p
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------+----+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| Operator        | Id | Details                                             | Estimated Rows | Rows |
DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+-----------------+----+-----------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+--------------------+
| +ProduceResults |  0 | p                                                   |              1 |    1 |
3 |               |                       |           |                    |
```

```
| |              +----+-------------------------------------------------------+---------------+------
+---------+---------------+                       |           |                       |
| +NodeIndexSeek  |  1 | RANGE INDEX p:Person(name) WHERE name = $autostring_0 |             1 |    1 |
2 |           120 |                    2/1 |     0.688 | Fused in Pipeline 0 |
+-----------------+----+-------------------------------------------------------+---------------+------
+---------+---------------+-----------------------+-----------+--------------------+

Total database accesses: 5, total allocated memory: 184

1 row
```

Our execution plan is down to a single row and uses the Node Index Seek operator which does an index seek (see Performance indexes) to find the appropriate node.

Advanced query tuning example

This page describes advanced query optimizations based on native index capabilities.

One of the most important and useful ways of optimizing Cypher queries involves creating appropriate indexes. This is described in more detail in Create indexes, and demonstrated in Basic query tuning example. In summary, an index will be based on the combination of a `Label` and a `property`. Any Cypher query that searches for nodes with a specific label and some predicate on the property (equality, range or existence) will be planned to use the index if the cost planner deems that to be the most efficient solution.

In order to benefit from enhancements provided by native indexes, it is useful to understand when *index-backed property lookup* and *index-backed ORDER BY* will come into play. Let's explain how to use these features with a more advanced query tuning example.

> **Note:**
>
> If you are upgrading an existing store, it may be necessary to drop and re-create existing indexes. For information on native index support and upgrade considerations regarding indexes, see Operations Manual → Performance → Index configuration.

The data set

In this section, examples demonstrates the impact native indexes can have on query performance under certain conditions. You will use a movies dataset to illustrate this more advanced query tuning.

In this tutorial, you import data from the following CSV files:

- *movies.csv*
- *actors.csv*
- *directors.csv*

Movies

The *movies.csv* file contains two columns `title`, `released` and `tagline`.

The content of the *movies.csv* file:

movies.csv

```
title,released,tagline
Something's Gotta Give,1975,null
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Devil's Advocate,1997,Evil has its winning ways
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
V for Vendetta,2006,Freedom! Forever!
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Speed Racer,2008,Speed has no limits
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
Cloud Atlas,2012,Everything is connected
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
The Matrix Revolutions,2003,Everything that has a beginning has an end
The Matrix Reloaded,2003,Free your mind
The Matrix,1999,Welcome to the Real World
The Matrix,1999,Welcome to the Real World
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
The Devil's Advocate,1997,Evil has its winning ways
The Devil's Advocate,1997,Evil has its winning ways
The Devil's Advocate,1997,Evil has its winning ways
Jerry Maguire,2000,The rest of his life begins now.
Top Gun,1986,"I feel the need, the need for speed."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Something's Gotta Give,1975,null
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Hoffa,1992,He didn't want law. He wanted justice.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Apollo 13,1995,"Houston, we have a problem."
Frost/Nixon,2008,400 million people were waiting for the truth.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
What Dreams May Come,1998,After life there is more. The end is just the beginning.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
Jerry Maguire,2000,The rest of his life begins now.
```

```
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Hoffa,1992,He didn't want law. He wanted justice.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Ninja Assassin,2009,Prepare to enter a secret world of assassins
V for Vendetta,2006,Freedom! Forever!
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
When Harry Met Sally,1998,At odds in life... in love on-line.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
When Harry Met Sally,1998,At odds in life... in love on-line.
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
A Few Good Men,1992,"In the heart of the nation's capital, in a courthouse of the U.S. government, one man
will stop at nothing to keep his honor, and one will stop at nothing to find the truth."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
When Harry Met Sally,1998,At odds in life... in love on-line.
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Top Gun,1986,"I feel the need, the need for speed."
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
The Green Mile,1999,Walk a mile you'll never forget.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Jerry Maguire,2000,The rest of his life begins now.
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Stand By Me,1995,"For some, it's the last real taste of innocence, and the first real taste of life. But
for everyone, it's the time that memories are made of."
Cast Away,2000,"At the edge of the world, his journey begins."
Twister,1996,Don't Breathe. Don't Look Back.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
You've Got Mail,1998,At odds in life... in love on-line.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
As Good as It Gets,1997,A comedy from the heart that goes for the throat.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
Snow Falling on Cedars,1999,First loves last. Forever.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
The Birdcage,1996,Come as you are
What Dreams May Come,1998,After life there is more. The end is just the beginning.
What Dreams May Come,1998,After life there is more. The end is just the beginning.
Snow Falling on Cedars,1999,First loves last. Forever.
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Snow Falling on Cedars,1999,First loves last. Forever.
```

```
The Green Mile,1999,Walk a mile you'll never forget.
Snow Falling on Cedars,1999,First loves last. Forever.
Snow Falling on Cedars,1999,First loves last. Forever.
You've Got Mail,1998,At odds in life... in love on-line.
You've Got Mail,1998,At odds in life... in love on-line.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
You've Got Mail,1998,At odds in life... in love on-line.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
The Polar Express,2004,This Holiday Season… Believe
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
Cast Away,2000,"At the edge of the world, his journey begins."
Apollo 13,1995,"Houston, we have a problem."
The Green Mile,1999,Walk a mile you'll never forget.
The Da Vinci Code,2006,Break The Codes
Cloud Atlas,2012,Everything is connected
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
You've Got Mail,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
Sleepless in Seattle,1993,"What if someone you never met, someone you never saw, someone you never knew
was the only someone for you?"
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
The Birdcage,1996,Come as you are
Joe Versus the Volcano,1990,"A story of love, lava and burning desire."
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
When Harry Met Sally,1998,At odds in life... in love on-line.
That Thing You Do,1996,In every life there comes a time when that thing you dream becomes that thing you
do
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
Unforgiven,1992,"It's a hell of a thing, killing a man"
The Birdcage,1996,Come as you are
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Twister,1996,Don't Breathe. Don't Look Back.
RescueDawn,2006,Based on the extraordinary true story of one man's fight for freedom
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
The Birdcage,1996,Come as you are
Unforgiven,1992,"It's a hell of a thing, killing a man"
Unforgiven,1992,"It's a hell of a thing, killing a man"
Unforgiven,1992,"It's a hell of a thing, killing a man"
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Johnny Mnemonic,1995,The hottest data on earth. In the coolest head in town
Cloud Atlas,2012,Everything is connected
Cloud Atlas,2012,Everything is connected
Cloud Atlas,2012,Everything is connected
The Da Vinci Code,2006,Break The Codes
The Da Vinci Code,2006,Break The Codes
The Da Vinci Code,2006,Break The Codes
Apollo 13,1995,"Houston, we have a problem."
Frost/Nixon,2008,400 million people were waiting for the truth.
The Da Vinci Code,2006,Break The Codes
V for Vendetta,2006,Freedom! Forever!
V for Vendetta,2006,Freedom! Forever!
V for Vendetta,2006,Freedom! Forever!
```

```
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
V for Vendetta,2006,Freedom! Forever!
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Speed Racer,2008,Speed has no limits
Ninja Assassin,2009,Prepare to enter a secret world of assassins
Speed Racer,2008,Speed has no limits
Ninja Assassin,2009,Prepare to enter a secret world of assassins
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
Frost/Nixon,2008,400 million people were waiting for the truth.
The Green Mile,1999,Walk a mile you'll never forget.
Apollo 13,1995,"Houston, we have a problem."
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
The Green Mile,1999,Walk a mile you'll never forget.
Frost/Nixon,2008,400 million people were waiting for the truth.
Frost/Nixon,2008,400 million people were waiting for the truth.
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
Frost/Nixon,2008,400 million people were waiting for the truth.
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Hoffa,1992,He didn't want law. He wanted justice.
Hoffa,1992,He didn't want law. He wanted justice.
Hoffa,1992,He didn't want law. He wanted justice.
Apollo 13,1995,"Houston, we have a problem."
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
Twister,1996,Don't Breathe. Don't Look Back.
Apollo 13,1995,"Houston, we have a problem."
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
Twister,1996,Don't Breathe. Don't Look Back.
Twister,1996,Don't Breathe. Don't Look Back.
The Polar Express,2004,This Holiday Season… Believe
Cast Away,2000,"At the edge of the world, his journey begins."
One Flew Over the Cuckoo's Nest,1975,"If he's crazy, what does that make you?"
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Something's Gotta Give,1975,null
Bicentennial Man,1999,One robot's 200 year journey to become an ordinary man.
Charlie Wilson's War,2007,A stiff drink. A little mascara. A lot of nerve. Who said they couldn't bring
down the Soviet empire.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
A League of Their Own,1992,Once in a lifetime you get a chance to do something different.
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
The Da Vinci Code,2006,Break The Codes
The Birdcage,1996,Come as you are
Unforgiven,1992,"It's a hell of a thing, killing a man"
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
Cloud Atlas,2012,Everything is connected
The Da Vinci Code,2006,Break The Codes
The Replacements,2000,"Pain heals, Chicks dig scars... Glory lasts forever"
```

Actors

The actors.csv file contains two columns `title`, `roles`, `name`, and `born`.

The content of the actors.csv file:

actors.csv

```
title,roles,name,born
Something's Gotta Give,Julian Mercer,Keanu Reeves,1964
Johnny Mnemonic,Johnny Mnemonic,Keanu Reeves,1964
The Replacements,Shane Falco,Keanu Reeves,1964
The Devil's Advocate,Kevin Lomax,Keanu Reeves,1964
The Matrix Revolutions,Neo,Keanu Reeves,1964
```

```
The Matrix Reloaded,Neo,Keanu Reeves,1964
The Matrix,Neo,Keanu Reeves,1964
The Matrix Revolutions,Trinity,Carrie-Anne Moss,1967
The Matrix Reloaded,Trinity,Carrie-Anne Moss,1967
The Matrix,Trinity,Carrie-Anne Moss,1967
The Matrix Revolutions,Morpheus,Laurence Fishburne,1961
The Matrix Reloaded,Morpheus,Laurence Fishburne,1961
The Matrix,Morpheus,Laurence Fishburne,1961
V for Vendetta,V,Hugo Weaving,1960
Cloud Atlas,Bill Smoke;Haskell Moore;Tadeusz Kesselring;Nurse Noakes;Boardman Mephi;Old Georgie,Hugo
Weaving,1960
The Matrix Revolutions,Agent Smith,Hugo Weaving,1960
The Matrix Reloaded,Agent Smith,Hugo Weaving,1960
The Matrix,Agent Smith,Hugo Weaving,1960
The Matrix,Emil,Emil Eifrem,1978
That Thing You Do,Tina,Charlize Theron,1975
The Devil's Advocate,Mary Ann Lomax,Charlize Theron,1975
The Devil's Advocate,John Milton,Al Pacino,1940
Jerry Maguire,Jerry Maguire,Tom Cruise,1962
Top Gun,Maverick,Tom Cruise,1962
A Few Good Men,Lt. Daniel Kaffee,Tom Cruise,1962
Something's Gotta Give,Harry Sanborn,Jack Nicholson,1937
One Flew Over the Cuckoo's Nest,Randle McMurphy,Jack Nicholson,1937
Hoffa,Hoffa,Jack Nicholson,1937
As Good as It Gets,Melvin Udall,Jack Nicholson,1937
A Few Good Men,Col. Nathan R. Jessup,Jack Nicholson,1937
A Few Good Men,Lt. Cdr. JoAnne Galloway,Demi Moore,1962
Apollo 13,Jack Swigert,Kevin Bacon,1958
Frost/Nixon,Jack Brennan,Kevin Bacon,1958
A Few Good Men,Capt. Jack Ross,Kevin Bacon,1958
Stand By Me,Ace Merrill,Kiefer Sutherland,1966
A Few Good Men,Lt. Jonathan Kendrick,Kiefer Sutherland,1966
A Few Good Men,Cpl. Jeffrey Barnes,Noah Wyle,1971
What Dreams May Come,Albert Lewis,Cuba Gooding Jr.,1968
As Good as It Gets,Frank Sachs,Cuba Gooding Jr.,1968
Jerry Maguire,Rod Tidwell,Cuba Gooding Jr.,1968
A Few Good Men,Cpl. Carl Hammaker,Cuba Gooding Jr.,1968
A Few Good Men,Lt. Sam Weinberg,Kevin Pollak,1957
Hoffa,Frank Fitzsimmons,J.T. Walsh,1943
A Few Good Men,Lt. Col. Matthew Andrew Markinson,J.T. Walsh,1943
A Few Good Men,Pfc. Louden Downey,James Marshall,1967
A Few Good Men,Dr. Stone,Christopher Guest,1948
A Few Good Men,Man in Bar,Aaron Sorkin,1961
Top Gun,Charlie,Kelly McGillis,1957
Top Gun,Iceman,Val Kilmer,1959
Top Gun,Goose,Anthony Edwards,1962
Top Gun,Viper,Tom Skerritt,1933
When Harry Met Sally,Sally Albright,Meg Ryan,1961
Joe Versus the Volcano,DeDe;Angelica Graynamore;Patricia Graynamore,Meg Ryan,1961
Sleepless in Seattle,Annie Reed,Meg Ryan,1961
You've Got Mail,Kathleen Kelly,Meg Ryan,1961
Top Gun,Carole,Meg Ryan,1961
Jerry Maguire,Dorothy Boyd,Renee Zellweger,1969
Jerry Maguire,Avery Bishop,Kelly Preston,1962
Stand By Me,Vern Tessio,Jerry O'Connell,1974
Jerry Maguire,Frank Cushman,Jerry O'Connell,1974
Jerry Maguire,Bob Sugar,Jay Mohr,1970
The Green Mile,Jan Edgecomb,Bonnie Hunt,1961
Jerry Maguire,Laurel Boyd,Bonnie Hunt,1961
Jerry Maguire,Marcee Tidwell,Regina King,1971
Jerry Maguire,Ray Boyd,Jonathan Lipnicki,1990
Stand By Me,Chris Chambers,River Phoenix,1970
Stand By Me,Teddy Duchamp,Corey Feldman,1971
Stand By Me,Gordie Lachance,Wil Wheaton,1972
Stand By Me,Denny Lachance,John Cusack,1966
RescueDawn,Admiral,Marshall Bell,1942
Stand By Me,Mr. Lachance,Marshall Bell,1942
Cast Away,Kelly Frears,Helen Hunt,1963
Twister,Dr. Jo Harding,Helen Hunt,1963
As Good as It Gets,Carol Connelly,Helen Hunt,1963
You've Got Mail,Frank Navasky,Greg Kinnear,1963
As Good as It Gets,Simon Bishop,Greg Kinnear,1963
What Dreams May Come,Simon Bishop,Annabella Sciorra,1960
Snow Falling on Cedars,Nels Gudmundsson,Max von Sydow,1929
What Dreams May Come,The Tracker,Max von Sydow,1929
What Dreams May Come,The Face,Werner Herzog,1942
Bicentennial Man,Andrew Marin,Robin Williams,1951
```

```
The Birdcage,Armand Goldman,Robin Williams,1951
What Dreams May Come,Chris Nielsen,Robin Williams,1951
Snow Falling on Cedars,Ishmael Chambers,Ethan Hawke,1970
Ninja Assassin,Takeshi,Rick Yune,1971
Snow Falling on Cedars,Kazuo Miyamoto,Rick Yune,1971
The Green Mile,Warden Hal Moores,James Cromwell,1940
Snow Falling on Cedars,Judge Fielding,James Cromwell,1940
You've Got Mail,Patricia Eden,Parker Posey,1968
You've Got Mail,Kevin Jackson,Dave Chappelle,1973
RescueDawn,Duane,Steve Zahn,1967
You've Got Mail,George Pappas,Steve Zahn,1967
A League of Their Own,Jimmy Dugan,Tom Hanks,1956
The Polar Express,Hero Boy;Father;Conductor;Hobo;Scrooge;Santa Claus,Tom Hanks,1956
Charlie Wilson's War,Rep. Charlie Wilson,Tom Hanks,1956
Cast Away,Chuck Noland,Tom Hanks,1956
Apollo 13,Jim Lovell,Tom Hanks,1956
The Green Mile,Paul Edgecomb,Tom Hanks,1956
The Da Vinci Code,Dr. Robert Langdon,Tom Hanks,1956
Cloud Atlas,Zachry;Dr. Henry Goose;Isaac Sachs;Dermot Hoggins,Tom Hanks,1956
That Thing You Do,Mr. White,Tom Hanks,1956
Joe Versus the Volcano,Joe Banks,Tom Hanks,1956
Sleepless in Seattle,Sam Baldwin,Tom Hanks,1956
You've Got Mail,Joe Fox,Tom Hanks,1956
Sleepless in Seattle,Suzy,Rita Wilson,1956
Sleepless in Seattle,Walter,Bill Pullman,1953
Sleepless in Seattle,Greg,Victor Garber,1949
A League of Their Own,Doris Murphy,Rosie O'Donnell,1962
Sleepless in Seattle,Becky,Rosie O'Donnell,1962
The Birdcage,Albert Goldman,Nathan Lane,1956
Joe Versus the Volcano,Baw,Nathan Lane,1956
When Harry Met Sally,Harry Burns,Billy Crystal,1948
When Harry Met Sally,Marie,Carrie Fisher,1956
When Harry Met Sally,Jess,Bruno Kirby,1949
That Thing You Do,Faye Dolan,Liv Tyler,1977
The Replacements,Annabelle Farrell,Brooke Langton,1970
Unforgiven,Little Bill Daggett,Gene Hackman,1930
The Birdcage,Sen. Kevin Keeley,Gene Hackman,1930
The Replacements,Jimmy McGinty,Gene Hackman,1930
The Replacements,Clifford Franklin,Orlando Jones,1968
RescueDawn,Dieter Dengler,Christian Bale,1974
Twister,Eddie,Zach Grenier,1954
RescueDawn,Squad Leader,Zach Grenier,1954
Unforgiven,English Bob,Richard Harris,1930
Unforgiven,Bill Munny,Clint Eastwood,1930
Johnny Mnemonic,Takahashi,Takeshi Kitano,1947
Johnny Mnemonic,Jane,Dina Meyer,1968
Johnny Mnemonic,J-Bone,Ice-T,1958
Cloud Atlas,Luisa Rey;Jocasta Ayrs;Ovid;Meronym,Halle Berry,1966
Cloud Atlas,Vyvyan Ayrs;Captain Molyneux;Timothy Cavendish,Jim Broadbent,1949
The Da Vinci Code,Sir Leight Teabing,Ian McKellen,1939
The Da Vinci Code,Sophie Neveu,Audrey Tautou,1976
The Da Vinci Code,Silas,Paul Bettany,1971
V for Vendetta,Evey Hammond,Natalie Portman,1981
V for Vendetta,Eric Finch,Stephen Rea,1946
V for Vendetta,High Chancellor Adam Sutler,John Hurt,1940
Ninja Assassin,Ryan Maslow,Ben Miles,1967
Speed Racer,Cass Jones,Ben Miles,1967
V for Vendetta,Dascomb,Ben Miles,1967
Speed Racer,Speed Racer,Emile Hirsch,1985
Speed Racer,Pops,John Goodman,1960
Speed Racer,Mom,Susan Sarandon,1946
Speed Racer,Racer X,Matthew Fox,1966
Speed Racer,Trixie,Christina Ricci,1980
Ninja Assassin,Raizo,Rain,1982
Speed Racer,Taejo Togokahn,Rain,1982
Ninja Assassin,Mika Coretti,Naomie Harris,null
The Green Mile,John Coffey,Michael Clarke Duncan,1957
The Green Mile,Brutus 'Brutal' Howell,David Morse,1953
Frost/Nixon,"James Reston, Jr.",Sam Rockwell,1968
The Green Mile,'Wild Bill' Wharton,Sam Rockwell,1968
Apollo 13,Ken Mattingly,Gary Sinise,1955
The Green Mile,Burt Hammersmith,Gary Sinise,1955
The Green Mile,Melinda Moores,Patricia Clarkson,1959
Frost/Nixon,Richard Nixon,Frank Langella,1938
Frost/Nixon,David Frost,Michael Sheen,1969
Bicentennial Man,Rupert Burns,Oliver Platt,1960
Frost/Nixon,Bob Zelnick,Oliver Platt,1960
```

```
One Flew Over the Cuckoo's Nest,Martini,Danny DeVito,1944
Hoffa,Robert 'Bobby' Ciaro,Danny DeVito,1944
Hoffa,Peter 'Pete' Connelly,John C. Reilly,1965
Apollo 13,Gene Kranz,Ed Harris,1950
A League of Their Own,Bob Hinson,Bill Paxton,1955
Twister,Bill Harding,Bill Paxton,1955
Apollo 13,Fred Haise,Bill Paxton,1955
Charlie Wilson's War,Gust Avrakotos,Philip Seymour Hoffman,1967
Twister,Dustin 'Dusty' Davis,Philip Seymour Hoffman,1967
Something's Gotta Give,Erica Barry,Diane Keaton,1946
Charlie Wilson's War,Joanne Herring,Julia Roberts,1967
A League of Their Own,'All the Way' Mae Mordabito,Madonna,1954
A League of Their Own,Dottie Hinson,Geena Davis,1956
A League of Their Own,Kit Keller,Lori Petty,1963
```

Directors

The *directors.csv* file contains two columns `title`, `name`, and `born`.

The content of the *directors.csv* file:

directors.csv

```
title,name,born
Speed Racer,Andy Wachowski,1967
Cloud Atlas,Andy Wachowski,1967
The Matrix Revolutions,Andy Wachowski,1967
The Matrix Reloaded,Andy Wachowski,1967
The Matrix,Andy Wachowski,1967
Speed Racer,Lana Wachowski,1965
Cloud Atlas,Lana Wachowski,1965
The Matrix Revolutions,Lana Wachowski,1965
The Matrix Reloaded,Lana Wachowski,1965
The Matrix,Lana Wachowski,1965
The Devil's Advocate,Taylor Hackford,1944
Ninja Assassin,James Marshall,1967
V for Vendetta,James Marshall,1967
When Harry Met Sally,Rob Reiner,1947
Stand By Me,Rob Reiner,1947
A Few Good Men,Rob Reiner,1947
Top Gun,Tony Scott,1944
Jerry Maguire,Cameron Crowe,1957
As Good as It Gets,James L. Brooks,1940
RescueDawn,Werner Herzog,1942
What Dreams May Come,Vincent Ward,1956
Snow Falling on Cedars,Scott Hicks,1953
That Thing You Do,Tom Hanks,1956
Sleepless in Seattle,Nora Ephron,1941
You've Got Mail,Nora Ephron,1941
Joe Versus the Volcano,John Patrick Stanley,1950
The Replacements,Howard Deutch,1950
Charlie Wilson's War,Mike Nichols,1931
The Birdcage,Mike Nichols,1931
Unforgiven,Clint Eastwood,1930
Johnny Mnemonic,Robert Longo,1953
Cloud Atlas,Tom Tykwer,1965
Apollo 13,Ron Howard,1954
Frost/Nixon,Ron Howard,1954
The Da Vinci Code,Ron Howard,1954
The Green Mile,Frank Darabont,1959
Hoffa,Danny DeVito,1944
Twister,Jan de Bont,1943
The Polar Express,Robert Zemeckis,1951
Cast Away,Robert Zemeckis,1951
One Flew Over the Cuckoo's Nest,Milos Forman,1932
Something's Gotta Give,Nancy Meyers,1949
Bicentennial Man,Chris Columbus,1958
A League of Their Own,Penny Marshall,1943
```

Prerequisites

The example uses the Linux or macOS tarball installation. It assumes that your current work directory is the *<neo4j-home>* directory of the tarball installation, and the CSV files are placed in the default *import* directory.

Note:

- For the default directory of other installations see, Operations Manual → File locations.
- The import location can be configured with Operations Manual → `server.directories.import`.

Importing the data

Import the movies.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///movies.csv' AS line
MERGE (m:Movie {title: line.title})
ON CREATE SET
  m.released = toInteger(line.released),
  m.tagline = line.tagline
```

Result

```
Added 38 nodes, Set 114 properties, Added 38 labels
```

Import the actors.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///actors.csv' AS line
MATCH (m:Movie {title: line.title})
MERGE (p:Person {name: line.name})
ON CREATE SET p.born = toInteger(line.born)
MERGE (p)-[:ACTED_IN {roles:split(line.roles, ';')}]->(m)
```

Result

```
Added 102 nodes, Created 172 relationships, Set 375 properties, Added 102 labels
```

Import the directors.csv file

```
LOAD CSV WITH HEADERS FROM 'file:///directors.csv' AS line
MATCH (m:Movie {title: line.title})
MERGE (p:Person {name: line.name})
ON CREATE SET p.born = toInteger(line.born)
MERGE (p)-[:DIRECTED]->(m)
```

Result

```
Added 23 nodes, Created 44 relationships, Set 46 properties, Added 23 labels
```

Create an index for nodes with the `Person` label

```
CREATE INDEX FOR (p:Person)
ON (p.name)
```

Result

```
Added 1 indexes
```

```
CALL db.awaitIndexes
```

Index-backed property-lookup

In this example you want to write a query to find persons with the name 'Tom' that acted in a movie.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE p.name STARTS WITH 'Tom'
RETURN
  p.name AS name,
  count(m) AS count
```

Result

name	count
"Tom Cruise"	3
"Tom Hanks"	12
"Tom Skerritt"	1

Rows:3

The query request the database to return all the actors with the first name 'Tom'. There are three of them: *'Tom Cruise'*, *'Tom Skerritt'* and *'Tom Hanks'*. With native indexes, however, you can leverage the fact that indexes store the property values. In this case, it means that the names can be looked up directly from the index. This allows Cypher to avoid the second call to the database to find the property, which can save time on very large queries.

If we profile the above query, we see that the `NodeIndexSeekByRange` in the `Details` column contains `cache[p.name]`, which means that `p.name` is retrieved from the index. We can also see that the `OrderedAggregation` has no `DB Hits`, which means it does not have to access the database again.

```
PROFILE
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE p.name STARTS WITH 'Tom'
RETURN
  p.name AS name,
  count(m) AS count
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10
```

```
Batch size 128

+----------------------+----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+------------+--------------------+
| Operator             | Id | Details
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by |
Pipeline            |
+----------------------+----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+------------+--------------------+
| +ProduceResults      |  0 | name, count
|              1 |    3 |       0 |               |                   0/0 |     0.060 |            |
|
| |                    +----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+            |                    |
| +OrderedAggregation  |  1 | cache[p.name] AS name, count(m) AS count
|              1 |    3 |       0 |          1520 |                   0/0 |     3.119 | name ASC   | In
Pipeline 1       |
| |                    +----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+------------+--------------------+
| +Filter              |  2 | m:Movie
|              1 |   16 |      32 |               |                       |           |            |
|
| |                    +----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+                      |           |            |                    |
| +Expand(All)         |  3 | (p)-[anon_0:ACTED_IN]->(m)
|              1 |   16 |      22 |               |                       |           |            |
|
| |                    +----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+                      |           |            |                    |
| +NodeIndexSeekByRange |  4 | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    4 |       5 |            120 |                   7/0 |     0.611 |
p.name ASC | Fused in Pipeline 0 |
+----------------------+----
+-------------------------------------------------------------------------------+---------------+------
+---------+---------------+----------------------+-----------+------------+--------------------+

Total database accesses: 59, total allocated memory: 1600

3 rows
```

If we change the query, such that it can no longer use an index, we will see that there will be no `cache[p.name]` in the `Details` column, and that the `EagerAggregation` now has `DB Hits`, since it accesses the database again to retrieve the name.

```
PROFILE
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN
  p.name AS name,
  count(m) AS count
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+------------------+----+----------------------------------+---------------+------+---------
+---------------+----------------------+-----------+--------------------+
| Operator         | Id | Details                          | Estimated Rows | Rows | DB Hits | Memory
(Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline           |
+------------------+----+----------------------------------+---------------+------+---------
```

```
+---------------+----------------------+----------+--------------------+
| +ProduceResults   |  0 | name, count                      |            13 |  102 |       0 |
|                    0/0 |     0.155 | In Pipeline 1       |
| |                +----+----------------------------------+---------------+------+---------
+---------------+----------------------+----------+--------------------+
| +EagerAggregation |  1 | p.name AS name, count(m) AS count |            13 |  102 |     344 |
17296 |                          |          |                    |
| |                +----+----------------------------------+---------------+------+---------
+----------------+                       |          |                    |
| +Filter           |  2 | p:Person                         |           172 |  172 |     344 |
|                        |          |                     |
| |                +----+----------------------------------+---------------+------+---------
+----------------+                       |          |                    |
| +Expand(All)      |  3 | (m)<-[anon_0:ACTED_IN]-(p)       |           172 |  172 |     254 |
|                        |          |                     |
| |                +----+----------------------------------+---------------+------+---------
+----------------+                       |          |                    |
| +NodeByLabelScan  |  4 | m:Movie                          |            38 |   38 |      39 |
120 |                   29/0 |     1.444 | Fused in Pipeline 0 |
+-------------------+----+----------------------------------+---------------+------+---------
+---------------+----------------------+----------+--------------------+

Total database accesses: 981, total allocated memory: 17376

102 rows
```

For non-native indexes there will still be a second database access to retrieve those values.

Predicates that can be used to enable this optimization are:

- Existence (e.g. `WHERE n.name IS NOT NULL`)
- Equality (e.g. `WHERE n.name = 'Tom Hanks'`)
- Range (e.g. `WHERE n.uid > 1000 AND n.uid < 2000`)
- Prefix (e.g. `WHERE n.name STARTS WITH 'Tom'`)
- Suffix (e.g. `WHERE n.name ENDS WITH 'Hanks'`)
- Substring (e.g. `WHERE n.name CONTAINS 'a'`)
- Several predicates of the above types combined using `OR`, given that all of them are on the same property (e.g. `WHERE n.prop < 10 OR n.prop = 'infinity'`)

Note:

If there is an existence constraint on the property, no predicate is required to trigger the optimization. For example, `CREATE CONSTRAINT constraint_name FOR (p:Person) REQUIRE p.name IS NOT NULL`.

Aggregating functions

For all built-in aggregating functions in Cypher, the *index-backed property-lookup* optimization can be used even without a predicate.

Consider this query which returns the number of distinct names of people in the movies dataset:

```
PROFILE
MATCH (p:Person)
RETURN count(DISTINCT p.name) AS numberOfNames
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------+----+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| Operator          | Id | Details                                                          | Estimated
Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +ProduceResults   |  0 | numberOfNames                                                    |
1 |    1 |       0 |                |                    0/0 |     0.026 | In Pipeline 1       |
| |                 +----+------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+
| +EagerAggregation |  1 | count(DISTINCT cache[p.name]) AS numberOfNames                   |
1 |    1 |       0 |           9888 |                         |           |                     |
| |                 +----+------------------------------------------------------------------
+----------------+------+---------+----------------+                        |           |
|
| +NodeIndexScan    |  2 | RANGE INDEX p:Person(name) WHERE name IS NOT NULL, cache[p.name] |
125 |  125 |     126 |            120 |                     1/0 |     1.400 | Fused in Pipeline 0 |
+-------------------+----+-------------------------------------------------------------------
+----------------+------+---------+----------------+------------------------+-----------
+---------------------+

Total database accesses: 126, total allocated memory: 9952

1 row
```

Note that the `NodeIndexScan` in the `Details` column contains `cache[p.name]` and that the `EagerAggregation` has no `DB Hits`. In this case, the semantics of aggregating functions works like an implicit existence predicate because `Person` nodes without the property `name` will not affect the result of an aggregation.

Index-backed ORDER BY

Now consider the following refinement to the query:

```
PROFILE
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE p.name STARTS WITH 'Tom'
RETURN
  p.name AS name,
  count(m) AS count
ORDER BY name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-----------------------+----
+-------------------------------------------------------------------------------+----------------+------
+---------+---------------+-----------------------+-----------+------------+---------------------+
| Operator              | Id | Details
```

```
| Estimated Rows | Rows | DB Hits | Memory (Bytes) | Page Cache Hits/Misses | Time (ms) | Ordered by |
Pipeline           |
+----------------------+----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+-----------------------+-----------+------------+--------------------+
| +ProduceResults       |  0 | name, count
|              1 |    3 |       0 |               |                   0/0 |     0.025 |            |
|
| |                     +----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+-----------------------+-----------+            |                    |
| +OrderedAggregation   |  1 | cache[p.name] AS name, count(m) AS count
|              1 |    3 |       0 |          1520 |                   0/0 |     0.097 | name ASC   | In
Pipeline 1       |
| |                     +----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+-----------------------+-----------+------------+--------------------+
| +Filter               |  2 | m:Movie
|              1 |   16 |      32 |               |                       |           |            |
|
| |                     +----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+                       |           |            |                    |
| +Expand(All)          |  3 | (p)-[anon_0:ACTED_IN]->(m)
|              1 |   16 |      22 |               |                       |           |            |
|
| |                     +----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+                       |           |            |                    |
| +NodeIndexSeekByRange |  4 | RANGE INDEX p:Person(name) WHERE name STARTS WITH $autostring_0,
cache[p.name] |              0 |    4 |       5 |            120 |                   7/0 |     0.406 |
p.name ASC | Fused in Pipeline 0 |
+----------------------+----
+-------------------------------------------------------------------------------------+---------------+------
+---------+----------------+-----------------------+-----------+------------+--------------------+
```

We are asking for the results in ascending alphabetical order. The native index happens to store String properties in ascending alphabetical order, and Cypher knows this. In Neo4j 3.5 and later, the Cypher planner will recognize that the index already returns data in the correct order, and skip the `Sort` operation.

The `Order by` column describes the order of rows after each operator. We see that the `Order by` column contains `p.name ASC` from the index seek operation, meaning that the rows are ordered by `p.name` in ascending order.

Index-backed ORDER BY can also be used for queries that expect their results is descending order, but with slightly lower performance.

> **Note:**
>
> In cases where the Cypher planner is unable to remove the `Sort` operator, the planner can utilize knowledge of the `ORDER BY` clause to plan the `Sort` operator at a point in the plan with optimal cardinality.

`min()` and `max()`

For the `min` and `max` functions, the *index-backed ORDER BY* optimization can be used to avoid aggregation and instead utilize the fact that the minimum/maximum value is the first/last one in a sorted index. Consider the following query which returns the fist actor in alphabetical order:

```
PROFILE
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
```

```
RETURN min(p.name) AS name
```

Query Plan

```
Planner COST

Runtime PIPELINED

Runtime version 2026.10

Batch size 128

+-------------------+----+---------------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| Operator          | Id | Details                   | Estimated Rows | Rows | DB Hits | Memory (Bytes) |
Page Cache Hits/Misses | Time (ms) | Pipeline            |
+-------------------+----+---------------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +ProduceResults   |  0 | name                      |              1 |    1 |       0 |                |
0/0 |     0.027 | In Pipeline 1       |
| |                 +----+---------------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+
| +EagerAggregation |  1 | min(p.name) AS name       |              1 |    1 |     344 |             32 |
|           |                      |
| |                 +----+---------------------------+----------------+------+---------+----------------+
|           |                      |
| +Filter           |  2 | p:Person                  |            172 |  172 |     344 |                |
|           |                      |
| |                 +----+---------------------------+----------------+------+---------+----------------+
|           |                      |
| +Expand(All)      |  3 | (m)<-[anon_0:ACTED_IN]-(p) |            172 |  172 |     254 |                |
|           |                      |
| |                 +----+---------------------------+----------------+------+---------+----------------+
|           |                      |
| +NodeByLabelScan  |  4 | m:Movie                   |             38 |   38 |      39 |            120 |
29/0 |     0.990 | Fused in Pipeline 0 |
+-------------------+----+---------------------------+----------------+------+---------+----------------
+-----------------------+-----------+---------------------+

Total database accesses: 981, total allocated memory: 232

1 row
```

Aggregations are usually using the `EagerAggregation` operation. This would mean scanning all nodes in the index to find the name that is first in alphabetic order. Instead, the query is planned with `Projection`, followed by `Limit`, followed by `Optional`. This will simply pick the first value from the index.

For large datasets, this can improve performance dramatically.

Index-backed ORDER BY can also be used for corresponding queries with the `max` function, but with slightly lower performance.

Restrictions

The optimization can only work on native indexes. It does not work for predicates only querying for the spatial type `POINT`.

Predicates that can be used to enable this optimization are:

- Existence (e.g. `WHERE n.name IS NOT NULL`)
- Equality (e.g. `WHERE n.name = 'Tom Hanks'`)
- Range (e.g. `WHERE n.uid > 1000 AND n.uid < 2000`)

- Prefix (e.g. `WHERE n.name STARTS WITH 'Tom'`)
- Suffix (e.g. `WHERE n.name ENDS WITH 'Hanks'`)
- Substring (e.g. `WHERE n.name CONTAINS 'a'`)

Predicates that will not work:

- Several predicates combined using `OR`
- Equality or range predicates querying for points (e.g. `WHERE n.place > point({ x: 1, y: 2 })`)
- Spatial distance predicates (e.g. `WHERE point.distance(n.place, point({ x: 1, y: 2 })) < 2`)

Note:

If there is a property existence constraint on the property, no predicate is required to trigger the optimization. For example, `CREATE CONSTRAINT constraint_name FOR (p:Person) REQUIRE p.name IS NOT NULL`

Predicates with parameters, such as `WHERE n.prop > $param`, can trigger *index-backed ORDER BY*. The only exception are queries with parameters of type `POINT`.

Cypher Cheat Sheet

The Cypher Cheat Sheet is a reference guide that contains syntax examples and usage notes for Cypher queries.

Cypher 25 Cheat Sheet

License

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)

You are free to

Share

copy and redistribute the material in any medium or format

Adapt

remix, transform, and build upon the material

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms

Attribution

You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

NonCommercial

You may not use the material for commercial purposes.

ShareAlike

If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions

You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Notices

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation.

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

See https://creativecommons.org/licenses/by-nc-sa/4.0/ for further details. The full license text is available at https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.

[1] The `FLOAT` type in Cypher always represents a 64-bit double-precision floating point number.
[2] The `INTEGER` type in Cypher always represents a 64-bit `INTEGER`.
[3] The `INNER_TYPE` cannot be a `LIST` type.